The Fortress Language Specification

Working Draft

July 19, 2012

with the changes of the 2026 revival, listed in Appendix I

Eric Allen
David Chase
Joe Hallett
Victor Luchangco
Jan-Willem Maessen
Sukyoung Ryu
Guy L. Steele Jr.
Sam Tobin-Hochstadt

Additional contributors:
Joao Dias
Carl Eastlund
Christine Flood
Yossi Lev
Cheryl McCosh
Janus Dam Nielsen
Dan Smith

© Sun Microsystems, Inc.

The Fortress Language Specification, Version 1.0[2] was the first to be released in tandem with a compliant interpreter (as of March 31, 2008), available as open source and online at:

`http://projectfortress.sun.com`

Our tandem release of a specification and matching interpreter was a major milestone for the project. Our reference implementation has been evolving gradually, in parallel with the evolution of the language specification and the development of the core libraries. In order to synchronize the specification with the implementation, it was necessary both to add features to the implementation and to drop features from the specification. All Fortress source code appearing in the specification has been tested by executing it with the open source Fortress implementation. Moreover, all code has been rendered automatically with the tool *Fortify*, also included with the standard Fortress distribution. Fortify is an open source tool for converting Fortress source code to LaTeX.

Since the release, we have incrementally added back features taken out of the specification as they have been implemented. In particular, we have been building a Fortress compiler with a static type checker and various static constraints checker. We also made considerable amount of modification and evolution of the language design.

This specification is a working draft; it is meant to include all language features included in the Fortress Language Specification, Version 1.0β (perhaps with additional) and any language changes since then. This version of the specification is not yet for a release; it may include wild ideas that require additional research before they can be implemented reliably. This specification includes descriptions of unimplemented features in boxes at the beginning of sections, and informal comments and notes in boxes in the margin.

In 2026 the Fortress revival revised this working draft where a decision taken for the implementation it is completing would otherwise leave the text and the implementation in disagreement. The first such revision states the type group's rule that no type is a subtype of two different instantiations of one parameterized trait, which the compiled type checker enforces for type arguments and numerals, with the exceptions Appendix I names, and this draft had not stated, and rewrites the examples that the rule refuses. Each revised passage carries a box headed "Revised by the 2026 revival"; Appendix I lists every change with its reason, the original text, and the alternative not taken.

The not-yet-supported features include:

- widening coercion
- multifix operators
- type coverage check
- generic overloading check
- definite assignment check
- keyword and varargs parameters
- varargs expressions
- some modifiers (`override`, `io`, `atomic`, `value`, ...)
- qualified names
- identifiers including connecting punctuations
- ASCII conversion
- Unicode canonicalization
- array comprehensions
- reduction variables
- object copy
- unboxed types
- mutual recursion in value objects
- thread fairness guarantee
- distributions
- matrix unpasting
- Non-type static parameters
- dimensions and units
- tests and properties
- type aliases
- where clauses and conditional extension
- abstract function declarations
- checked exceptions, deferred exceptions, and chained exceptions
- contract checking (overloaded functionals and method overriding)
- static expressions, static range types, and some range operations
- some types (HeapSequence, LinearSequence, $\mathbb{Q}$, Numeral, $\mathbb{Z}$, $\mathbb{N}$, $\mathbb{Q}$, ...)
- some operators on aggregate expressions
- some functions in Section 13.33
- tail-call optimization
- compound APIs check
- import API names
- components linking

Contents

I Preliminaries **17**

1 Introduction **18**

1.1 Fortress in a Nutshell 18

1.2 Acknowledgments 19

1.3 Organization 20

2 Overview **21**

2.1 The Fortress Programming Environment 21

2.2 Exports and Imports 23

2.3 Automatic Generation of APIs 26

2.4 Rendering 27

2.5 Some Common Types in Fortress 28

2.6 Functions in Fortress 28

2.7 Some Common Expressions in Fortress 30

2.8 For Loops Are Parallel by Default 31

2.9 Atomic Expressions 31

2.10 Dimensions and Units 32

2.11 Aggregate Expressions 33

2.12 Comprehensions 34

2.13 Summations and Products 35

2.14 Tests and Properties 35

2.15 Objects and Traits 36

2.16 Features for Library Development 39

II Fortress for Application Programmers **41**

3 Programs **42**

4 Lexical Structure **43**

4.1 Characters . . . 44
4.2 Words and Chunks . . . 48
4.3 Lines, Pages and Position . . . 48
4.4 ASCII Conversion . . . 49
4.5 Input Elements and Scanning . . . 49
4.6 Comments . . . 50
4.7 Whitespace Elements . . . 50
4.8 Reserved Words . . . 51
4.9 Character Literals . . . 51
4.10 String Literals . . . 53
4.11 Boolean Literals . . . 54
4.12 The Void Literal . . . 54
4.13 Numerals . . . 54
4.14 Operator Tokens . . . 55
4.15 Identifiers . . . 57
4.16 Special Tokens . . . 57
4.17 Rendering of Fortress Programs . . . 57

5 Evaluation **59**

5.1 Values . . . 59
5.2 Normal and Abrupt Completion of Evaluation . . . 60
5.3 Memory and Memory Operations . . . 61
5.4 Threads and Parallelism . . . 61
5.5 Environments . . . 65
5.6 Input and Output Actions . . . 65

6 Types **67**

6.1 Kinds of Types . . . 68
6.2 Relationships between Types . . . 68
6.3 Trait Types . . . 69
6.4 Object Expression Types . . . 70
6.5 Tuple Types . . . 71
6.6 Arrow Types . . . 71

6.7 Function Types . . . 73
6.8 Special Types . . . 73
6.9 Types in the Fortress Standard Libraries . . . 74
6.10 Intersection and Union Types . . . 74
6.11 Type Aliases . . . 75

7 Names and Declarations **77**
7.1 Kinds of Declarations . . . 78
7.2 Namespaces . . . 80
7.3 Reach and Scope of Declarations . . . 80
7.4 Imported Declarations and Qualified Names . . . 82

8 Variables **83**
8.1 Top-Level Variable Declarations . . . 83
8.2 Local Variable Declarations . . . 84
8.3 Matrix Unpasting . . . 85

9 Functions **89**
9.1 Function Declarations . . . 89
9.2 Function Applications . . . 91
9.3 Abstract Function Declarations . . . 93
9.4 Function Contracts . . . 94
9.5 Local Function Declarations . . . 96

10 Traits **97**
10.1 Trait Declarations . . . 97
10.2 Method Declarations . . . 102
10.3 Abstract Field Declarations . . . 106
10.4 Method Contracts . . . 107
10.5 Value Traits . . . 108

11 Objects **109**
11.1 Object Declarations . . . 109
11.2 Field Declarations . . . 111
11.3 Value Objects . . . 113
11.4 Object Equivalence . . . 114

12 Static Parameters **115**

12.1 Type Parameters . . . 115
12.2 Nat and Int Parameters . . . 116
12.3 Bool Parameters . . . 116
12.4 Dimension and Unit Parameters . . . 117
12.5 Operator Parameters . . . 117
12.6 Where Clauses . . . 119

13 Expressions **122**

13.1 Literals . . . 122
13.2 Identifier References . . . 125
13.3 Dotted Field Accesses . . . 126
13.4 Dotted Method Invocations . . . 126
13.5 Naked Method Invocations . . . 127
13.6 Function Calls . . . 127
13.7 Operator Applications . . . 128
13.8 Function Expressions . . . 129
13.9 Object Expressions . . . 130
13.10 Assignments . . . 131
13.11 Do Expressions . . . 132
13.12 Label and Exit . . . 135
13.13 While Loops . . . 135
13.14 Generators . . . 136
13.15 For Loops . . . 137
13.16 Ranges . . . 138
13.17 Summations and Other Reduction Expressions . . . 139
13.18 Parallel Prefix and Parallel Suffix . . . 140
13.19 If Expressions . . . 141
13.20 Case Expressions . . . 141
13.21 Extremum Expressions . . . 142
13.22 Typecase Expressions . . . 143
13.23 Atomic Expressions . . . 145
13.24 Spawn Expressions . . . 146
13.25 Throw Expressions . . . 147

13.26Try Expressions . . . 147
13.27Static Expressions . . . 149
13.28Tuple Expressions . . . 150
13.29Aggregate Expressions . . . 150
13.30Comprehensions . . . 153
13.31Type Ascription . . . 154
13.32Type Assumption . . . 154
13.33Expression-like Functions . . . 155

14 Exceptions **157**

14.1 Causes of Exceptions . . . 157
14.2 Types of Exceptions . . . 157
14.3 Information of Exceptions . . . 158

15 Overloading and Multiple Dispatch **160**

15.1 Principles of Overloading . . . 160
15.2 Applicability to Named Functional Calls . . . 161
15.3 Applicability to Dotted Method Calls . . . 162
15.4 Applicability for Functionals with Varargs and Keyword Parameters . . . 163
15.5 Overloading Resolution . . . 163

16 Operators **165**

16.1 Operator Names . . . 166
16.2 Operator Precedence and Associativity . . . 166
16.3 Operator Fixity . . . 168
16.4 Chained and Multifix Operators . . . 169
16.5 Enclosing Operators . . . 170
16.6 Conditional Operators . . . 171
16.7 Big Operators . . . 171
16.8 Juxtaposition . . . 171
16.9 Overview of Operators in the Fortress Standard Libraries . . . 173

17 Conversions and Coercions **179**

17.1 Principles of Coercion . . . 179
17.2 Coercion Declarations . . . 181
17.3 Coercion Invocations . . . 182
17.4 Applicability with Coercion . . . 183
17.5 Coercion Resolution . . . 184
17.6 Restrictions on Coercion Declarations . . . 186
17.7 Coercions for Tuple and Arrow Types . . . 187
17.8 Automatic Widening . . . 188

18 Dimensions and Units **191**

19 Tests and Properties **195**

19.1 The Purpose of Tests and Properties . . . 196
19.2 Test Declarations . . . 196
19.3 Other Test Constructs . . . 197
19.4 Running Tests . . . 197
19.5 Test Suites . . . 197
19.6 Property Declarations . . . 198

20 Type Inference **199**

20.1 The Static Arguments of a Call . . . 199
20.2 A Numeral Whose Conversions Tie . . . 202

21 Memory Model **205**

21.1 Principles . . . 205
21.2 Programming Discipline . . . 206
21.3 Read and Write Atomicity . . . 209
21.4 Ordering Dependencies among Operations . . . 209

22 Components and APIs **212**

22.1 Overview . . . 214
22.2 Components . . . 215
22.3 APIs . . . 220
22.4 Component and API Identity . . . 223
22.5 Tests in Components and APIs . . . 224

22.6 Type Inference for Components . . . 224
22.7 Initialization Order for Components . . . 224
22.8 Basic Fortress Operations . . . 225
22.9 Advanced Features of Fortress Operations . . . 232

III Fortress for Library Writers 236

23 Parallelism and Locality 237
23.1 Regions . . . 237
23.2 Placing Threads . . . 238
23.3 Shared and Local Data . . . 239
23.4 Distributed Arrays . . . 241
23.5 Abortable Atomicity . . . 242
23.6 Distributions . . . 242
23.7 Early Termination of Threads . . . 244
23.8 Use and Definition of Generators . . . 245

24 Overloaded Functional Declarations 252
24.1 Principles of Overloading . . . 252
24.2 Subtype Rule . . . 254
24.3 Incompatibility Rule . . . 254
24.4 Meet Rule . . . 255
24.5 Declarations with Static Parameters . . . 258
24.6 Coercion and Overloading Resolution . . . 260

25 Operator Declarations 261
25.1 Multifix Operator Declarations . . . 262
25.2 Infix Operator Declarations . . . 262
25.3 Prefix Operator Declarations . . . 262
25.4 Postfix Operator Declarations . . . 263
25.5 Nofix Operator Declarations . . . 263
25.6 Bracketing Operator Declarations . . . 263
25.7 Subscripting Operator Method Declarations . . . 264
25.8 Subscripted Assignment Operator Method Declarations . . . 264
25.9 Conditional Operator Declarations . . . 264
25.10 Big Operator Declarations . . . 265

26 Dimension and Unit Declarations **266**

26.1 Dimension Declarations . . . 266

26.2 Unit Declarations . . . 267

26.3 Abbreviating Dimension and Unit Declarations . . . 268

26.4 Absorbing Units . . . 270

27 Support for Domain-Specific Languages **271**

IV Fortress Interpreter Library APIs and Documentation **272**

28 Structure of the Fortress Libraries **274**

29 Default Libraries **275**

29.1 FortressLibrary . . . 275

29.2 Builtins . . . 341

30 Additional Libraries Available to Fortress Programmers **350**

30.1 Constants . . . 350

30.2 List . . . 350

30.3 PureList . . . 353

30.4 File . . . 355

30.5 Set . . . 358

30.6 Map . . . 359

30.7 Sparse . . . 362

30.8 Heap . . . 363

30.9 SkipList . . . 365

30.10QuickSort . . . 366

V Fortress APIs and Documentation for Application Programmers **367**

31 Objects **369**

31.1 The Trait Fortress.Core.Any . . . 369

31.2 The Trait Fortress.Core.Object . . . 371

31.3 The Trait Fortress.Core.Tuple . . . 371

32 Booleans and Boolean Intervals **373**

32.1 The Trait Fortress.Core.Boolean . . . 373

32.2 The Trait Fortress.Standard.BooleanInterval . . . 377

32.3 Top-level BooleanInterval Values . . . 383

33 Numbers **384**

33.1 Rational Numbers . . . 384

33.2 Integers . . . 391

34 Negated Relational Operators **403**

34.1 Negated Equivalence Operators . . . 403

34.2 Negated Plain Comparison Operators . . . 404

34.3 Negated Set Comparison Operators . . . 404

34.4 Negated Square Comparison Operators . . . 405

34.5 Negated Curly Comparison Operators . . . 405

34.6 Negated Miscellaneous Relational Operators . . . 406

34.7 Other Negated Operators . . . 406

35 Exceptions **407**

35.1 The Trait Fortress.Standard.Exception . . . 407

35.2 The Trait Fortress.Standard.CheckedException . . . 408

35.3 The Trait Fortress.Standard.UncheckedException . . . 408

36 Threads **409**

36.1 The Trait Fortress.Standard.Thread . . . 409

37 Dimensions and Units **410**

37.1 Fortress.SIUnits . . . 410

37.2 Fortress.EnglishUnits . . . 412

37.3 Fortress.InformationUnits . . . 413

38 Tests **414**

38.1 The Object Fortress.Standard.TestSuite . . . 414

38.2 Test Functions . . . 414

39 Convenience Functions and Types **415**

39.1 Convenience Functions . . . 415

39.2 Convenience Types . . . 416

VI Fortress APIs and Documentation for Library Writers **417**

40 Algebraic Constraints **419**

40.1 Predicates and Equivalence Relations . . . 419

40.2 Partial and Total Orders . . . 421

40.3 Operators and Their Properties . . . 424

40.4 Lattices . . . 430

40.5 Monoids, Groups, Rings, and Fields . . . 431

40.6 Boolean Algebras . . . 436

41 Numbers **438**

41.1 The Trait Fortress.Standard.RationalQuantity . . . 439

41.2 The Trait Fortress.Standard.TotalComparison . . . 443

41.3 Top-level Total Comparison Values . . . 444

41.4 The Trait Fortress.Standard.Comparison . . . 445

41.5 Top-level Comparison Value . . . 446

42 Components and APIs **447**

43 Memory Sequences and Binary Words **449**

43.1 The Trait Fortress.Core.LinearSequence . . . 450

43.2 Constructing Linear Sequences . . . 454

43.3 The Trait Fortress.Core.HeapSequence . . . 454

43.4 Constructing Heap Sequences . . . 456

43.5 The Trait Fortress.Core.BinaryWord . . . 457

43.6 The Trait Fortress.Core.BinaryEndianWord . . . 461

43.7 The Trait Fortress.Core.BasicBinaryOperations . . . 466

43.8 The Trait Fortress.Core.BasicBinaryWordOperations . . . 470

43.9 The Trait Fortress.Core.BinaryLinearEndianSequence . . . 472

43.10 The Trait Fortress.Core.BinaryEndianLinearEndianSequence . . . 476

43.11 The Trait Fortress.Core.BinaryHeapEndianSequence . . . 481

43.12The Trait Fortress.Core.BinaryEndianHeapEndianSequence 481
43.13The Trait Fortress.Core.BasicBinaryHeapSubsequenceOperations 482

VII Appendices **490**

A Fortress Calculi **491**
A.1 Basic Core Fortress 491
A.2 Core Fortress with Where Clauses 496
A.3 Core Fortress with Overloading 504
A.4 Acyclic Core Fortress with Field Definitions 509

B Overloaded Functional Declarations **515**
B.1 Proof of Coercion Resolution for Functions 515
B.2 Proof of Overloading Resolution for Functions 516

C Components and APIs **519**

D Rendering of Fortress Code **521**
D.1 Rendering of Fortress Identifiers 521
D.2 Rendering of Other Fortress Constructs 524

E Support for Unicode Input in ASCII **527**
E.1 Word Pasting across Line Terminators 527
E.2 Converting Names of Unicode Characters and ASCII Shorthand 528
E.3 Apostrophe Replacement within Numerals 532
E.4 Equivalence under ASCII Conversion 532

F Operator Precedence, Associativity, Chaining, and Enclosure **533**
F.1 Bracket Pairs for Static Type Information 533
F.2 Bracket Pairs for Enclosing Operators 533
F.3 Vertical-Line Operators 535
F.4 Arithmetic Operators 535
F.5 Relational Operators 539
F.6 Boolean Operators 545
F.7 Other Operators 546

G Simplified Grammar for Application Programmers and Library Writers **558**

G.1 Components and APIs . . . 560
G.2 Top-level Declarations . . . 561
G.3 Trait and Object Declarations . . . 561
G.4 Variable Declarations . . . 563
G.5 Function Declarations . . . 563
G.6 Dimension, Unit, Type Alias, Test, Property, and External Syntax Declarations . . . 564
G.7 Headers . . . 564
G.8 Parameters . . . 568
G.9 Method Declarations . . . 569
G.10 Method Parameters . . . 570
G.11 Field Declarations . . . 570
G.12 Abstract Field Declarations . . . 570
G.13 Expressions . . . 571
G.14 Expressions Enclosed by Keywords or Symbols . . . 573
G.15 Local Declarations . . . 574
G.16 Literals . . . 575
G.17 Types . . . 575
G.18 Symbols and Operators . . . 576
G.19 Identifiers . . . 576
G.20 Spaces and Comments . . . 576

H Full Grammar for Fortress Implementors **577**

H.1 Components and APIs . . . 577
H.2 Top-level Declarations . . . 578
H.3 Trait and Object Declarations . . . 579
H.4 Variable Declarations . . . 581
H.5 Function Declarations . . . 582
H.6 Dimension, Unit, Type Alias, Test, Property, and External Syntax Declarations . . . 582
H.7 Headers without Newlines . . . 583
H.8 Headers Maybe with Newlines . . . 584
H.9 Parameters . . . 587
H.10 Method Declarations . . . 587
H.11 Method Parameters . . . 588

H.12 Expressions . . . 589
H.13 Expressions Enclosed by Keywords or Symbols . . . 592
H.14 Local Declarations . . . 594
H.15 Literals . . . 595
H.16 Literals within Array Expressions . . . 596
H.17 Expressions without Newlines . . . 597
H.18 Expressions within Array Expressions . . . 598
H.19 Types . . . 598
H.20 Types without Newlines . . . 600
H.21 Symbols and Operators . . . 601
H.22 Identifiers . . . 604
H.23 Spaces and Comments . . . 604

I Changes Since Fortress 1.0 Specifications **607**

I.1 Changes Made by the 2026 Revival to the Working Draft . . . 607

J Internal Document **662**

J.1 Frequently Asked Questions . . . 662
J.2 Proposed Features . . . 665
J.3 Examples . . . 676
J.4 Things to Check for Consistency . . . 683

Part I

Preliminaries

Chapter 1

Introduction

The Fortress programming language is a general-purpose, statically typed, component-based programming language designed for producing robust high-performance software with high programmability.

In many ways, Fortress is intended to be a "growable language", i.e., a language that can be gracefully extended and applied in new and unanticipated contexts. Fortress supports state-of-the-art compiler optimization techniques, scaling to unprecedented levels of parallelism and of addressable memory. Fortress has an extensible component system, allowing separate program components to be independently developed, deployed, and linked in a modular and robust fashion. Fortress also supports modular and extensible parsing, allowing new notations and static analyses to be added to the language.

The name "Fortress" is derived from the intent to produce a "secure Fortran", i.e., a language for high-performance computation that provides abstraction and type safety on par with modern programming language principles. Despite this etymology, the language is a new language with little relation to Fortran other than its intended domain of application. No attempt has been made to support backward compatibility with existing versions of Fortran; indeed, many new language features were invented during the design of Fortress. Many aspects of Fortress were inspired by other object-oriented and functional programming languages, including the Java™ Programming Language [7], NextGen [8], Scala [24], Eiffel [19], Self [1], Standard ML [21], Objective Caml [17], Haskell [27], and Scheme [16]. The result is a language that employs cutting-edge features from the programming-language research community to achieve an unprecedented combination of performance and programmability.

Fortress is an open source project. An initial interpreter, implementing a core of the language features presented in this specification, is available at the Fortress project website:

`http://projectfortress.sun.com`

There you will find source code, supporting documents, and access to discussion groups related to the Fortress project.

1.1 Fortress in a Nutshell

> DRAFT NOTE
> Keyword and varargs parameters, components linking, distributions are not yet supported.

Two basic concepts in Fortress are that of *object* and of *trait*. An object consists of *fields* and *methods*. The fields of an object are specified in its definition. An object definition may also include method definitions.

Traits are named program constructs that declare sets of methods. They were introduced in the Self programming language, and their semantic properties (and advantages over conventional class inheritance) were analyzed by Ducasse, Nierstrasz, Schärli, Wuyts and Black [9]. In Fortress, a method declared by a trait may be either *abstract* or *concrete*: abstract methods have only *headers*; concrete methods also have *definitions*. A trait may *extend* other traits: it *inherits* the methods provided by the traits it extends.

> DRAFT NOTE
>
> The terminology I'm trying to adopt here is as follows: traits *declare* methods. Method declarations are syntactic entities contained in trait (and object) definitions (which are also syntactic entities), so a trait declares those methods for which its definition contains method declarations, as well as those methods it inherits. – Victor

A trait provides the methods that it inherits as well as those explicitly declared in its declaration.

Every object extends a set of traits (its "supertraits"). An object inherits the concrete methods of its supertraits and must include a definition for every method declared but not defined by its supertraits.

Note that the identifiers used in this example are not restricted to ASCII character sequences. Fortress allows the use of Unicode characters [29] in program identifiers, as well as subscripts and superscripts (See Appendix E for a discussion of Unicode and support for entering programs in ASCII.) Fortress also includes a set of standard formatting rules that follow the conventions of mathematical notation. For example, most variable references in Fortress programs are italicized. Moreover, multiplication can be expressed by simple juxtaposition. There is also support for operator overloading, as well as a facility for extending the syntax with domain-specific languages.

Although Fortress is statically and nominally typed, types are not specified for all fields, nor for all method parameters and return values. Instead, wherever possible, *type inference* is used to reconstruct types. In the examples throughout this specification, we often omit the types when they are clear from context. Additionally, types can be parametric with respect to other types and values (most notably natural numbers).

These design decisions are motivated in part by our goal of making the scientist/programmer's life as easy as possible without compromising good software engineering. In particular, they allow us to write Fortress programs that preserve the look of standard mathematical notation.

In addition to objects and traits, Fortress allows the programmer to define top-level functions. Functions are first-class values: They can be passed to and returned from functions, and assigned as values to fields and variables. Functions and methods can be overloaded, with calls to overloading methods resolved by multiple dynamic dispatch similarly to the manner described in [20]. Keyword parameters and variable size argument lists are also supported.

Fortress programs are organized into *components*, which export and import APIs and can be linked together. APIs describe the "shape" of a component, specifying the types in traits, objects and functions provided by a component. All external references within a component (i.e., references to traits, objects and functions implemented by other components) are to APIs imported by the component. We discuss components and APIs in detail in Chapter 22.

To address the needs of modern high-performance computation, Fortress also supports a rich set of operations for defining parallel execution and distribution of large data structures. This support is built into the core of the language. For example, `for` loops in Fortress are parallel by default.

1.2 Acknowledgments

With the help of many contributors and supporters, we are in the midst of building an open source interpreter for Fortress. Many people have contributed to the Fortress project with useful comments and suggestions for both the specification and implementation, code contributions, and supporting tools. Special thanks go to Bob Apthorpe, Mike Atkinson, Alex Battisti, David J. Biesack, Steve Brandt, Martin Buchholz, Zoran Budimlic, Bill Callahan, Corky Cartwright, Darren Dale, Joe Darcy, David Detlefs, Dave Dice, Kento Emoto, Matthias Felleisen, Robby Findler,

Kathi Fisler, Richard Gabriel, Markus Gaisbauer, Alex Garthwaite, Jesse Glick, Brian Goetz, Robert Grimm, Yuto Hayamizu, Steve Heller, Maurice Herlihy, James Hughes, Zhenjiang Hu, Andy Johnson, Mackale Joyner, Miriam Kadansky, Kazuhiko Kakehi, Ken Kennedy, Scott Kilpatrick, Alex Kravets, Shriram Krishnamurthi, Eric Lavigne, Doug Lea, Vass Litvinov, Eugene Loh, Kiminori Matsuzaki, Daniel MD, John Mellor-Crummey, Daniel Mendes, Jim Mitchell, Mark Moir, Akhilesh Mritunjai, Dan Nussbaum, Andrew Pitonyak, Bill Pugh, Doron Rajwan, John Reppy, John Rose, Ulrich Schreiner, Nir Shavit, Michael Spiegel, Bob Sproull, Walid Taha, Masato Takeichi, Peter Vanderbilt, Michael Van de Vanter, Chris Vick, Larry Votta, Jim Waldo, Russel Winder, Kathy Yelick, and the many other participants in the Fortress community.

We would also like to thank the anonymous referees of peer-reviewed papers related to the Fortress project, who have influenced the language design in many ways, as well as the DARPA reviewers from the HPCS project for their thoughtful feedback throughout the design of the language.

1.3 Organization

This language specification is organized as follows. In Part II, the Fortress language features for application programmers are explained, including objects, types, and functions. Relevant parts of the concrete syntax are provided with many examples. The full concrete syntax of Fortress is described in Appendix G. Part III describes advanced Fortress language features for library writers. In Part IV, APIs and documentation of the Fortress interpreter Library are presented. In Part V, APIs and documentation of some of the Fortress standard libraries for application programmers are presented and Part VI presents APIs and documentation for some of the Fortress standard libraries for library writers. The APIs presented in Part V and Part VI are not yet tested. Finally, in Part VII, the Fortress calculi, support for Unicode characters, and the Fortress grammars are described.

A note on the presented libraries in Parts V and VI: The Fortress standard libraries presented in this draft specification should not be construed as exhaustive or complete. Presentation of additional libraries is planned for future drafts, as are modifications to the libraries included here.

Chapter 2

Overview

> DRAFT NOTE
> Keyword and varargs parameters, array comprehensions, dimensions and units, tests and properties, distributions, some ASCII conversion, and some common types are not yet supported.
>
> From Section 2.1 to Section 2.3 are out of date.
>
> Some examples in this chapter are not tested nor run by the interpreter.

> DRAFT NOTE
> Eric: Note that APIs are difficult to remove from fortresses: we have to first remove every component referring to them. Also note that an autogenerated API source file isn't automatically compiled into the fortress. I think the error should be signaled when the programmer tries to compile the API.

In this chapter, we provide a high-level overview of the entire Fortress language. We present most features in this chapter through the use of examples, which should be accessible to programmers of other languages. In this chapter, unlike the rest of the specification, no attempt is made to provide complete descriptions of the various language features presented. Instead, we intend this overview to provide useful context for reading other sections of this specification, which provide rigorous definitions for what is merely introduced here.

2.1 The Fortress Programming Environment

Although Fortress is independent of the properties of a particular platform on which it is implemented, it is helpful for concreteness to present the programming model used in the Fortress reference implementation. In this programming model, Fortress source code is stored in files and organized in directories, and there is a text-based shell from which we can store environment variables and issue commands to execute and compile programs.

A Fortress program can be processed in one of two ways:

- It can be *executed*. The Fortress program is stored in a file with the suffix "`.fss`" and executed directly from an underlying operating system shell by calling the command "`fortress run`" on it. For example, suppose we write the following "`Hello, world!`" program to a file "`HelloWorld.fss`":

  `export` Executable

  run() = *print* "`Hello, world!`"

The first line is an *export statement*; we ignore it for the moment. The second line defines a function run, which takes a parameter named $args$ and prints the string "`Hello, world!`". Note that the parameter $args$ does not include a declaration of its type. In many cases, types can be elided in Fortress and inferred from context. (In this case, the type of $args$ is inferred based on the program's export statement, explained in Section 2.2.)

We can execute this program by issuing the following command to the shell:

```
fortress run HelloWorld.fss
```

The instruction `fortress run` can be abbreviated as `fortress`:

```
fortress HelloWorld.fss
```

- It can be *compiled*. In this case, the Fortress program is compiled into a *component*, which is stored in a hidden persistent cache maintained by the implementation, called a *fortress*. Typically, a single fortress holds all the components of a user, or group of users sharing programs and libraries. In our examples, we often refer to the fortress we are storing components in as *the resident fortress*.

  For example, we could have written our "`Hello, world!`" program as follows:

  `component` HelloWorld

  `export` Executable

  $run() = print$ "`Hello, world!`"

  `end`

  We can compile this program, by issuing the command "`fortress compile`" on it:

  ```
  fortress compile HelloWorld.fss
  ```

  As a result of this command, a component named "HelloWorld" is stored in the resident fortress. The name of this component is provided by the enclosing component declaration surrounding the code. If there is no enclosing component declaration, then the contents of the file are understood to belong to a single component whose name is that of the file it is stored in, minus its suffix. For example, suppose we write the following program in a source file named "`HelloWorld2.fss`":

  `export` Executable

  $run() = print$ "`Hi, it's me again!`"

  When we compile this file:

  ```
  fortress compile HelloWorld2.fss
  ```

  the result is that a new component with the name HelloWorld2 is stored in the resident fortress. Once this component is compiled, we can execute it by issuing the following command:

  ```
  fortress run HelloWorld2.fss
  ```

  Compiling a source file allows us to catch static errors before running a program. It also allows the system to perform static optimizations on a program and use those optimizations when executing.

  Once a program is compiled, the cached code will be used during execution unless modifications have been made to the source file since the most recent compilation. If the source file is newer, the program is recompiled before execution.

A source file must contain exactly one component declaration, and its name must match the file name.

In a compiled file, multiple component declarations may be included. For example, we could write the following file `HelloWorld3.fss`:

```
component HelloWorld
    export Executable
    run(args) = print "Hello, world!"
end

component HelloWorld2
    export Executable
    run(args) = print "Hi, it's me again!"
end
```

When we compile this file, the result is that both the components HelloWorld and HelloWorld2 are stored in the resident fortress.

If a fortress already contains a component with the same name as a newly installed component, the new component shadows the old one. For example, if we first compile the source file `HelloWorld3.fss` above and then compile the following file `HelloWorld4.fss`:

```
component HelloWorld
    export Executable
    run(args) = print "I didn't expect that!"
end
```

then executing the component HelloWorld on our fortress will result in printing of the following text:

```
I didn't expect that!
```

We can also "remove" a component from a fortress. For example:

```
fortress remove HelloWorld
```

After issuing this command, we can no longer refer to `HelloWorld` component when issuing commands to the fortress. (However, a removed component might still exist as a constituent of other, *linked*, components; see Section 2.2.)

2.2 Exports and Imports

When a component is defined, it can include *export statements*. For example, all of the components we have defined thus far have included the export statement "`export` Executable". Export statements list various *APIs* that a component implements. Unlike in other languages, APIs in Fortress are themselves program constructs; programmers can rely on standard APIs, and declare new ones. API declarations are sequences of declarations of variables, functions, and other program constructs, along with their types and other supporting declarations. For example, here is the definition of API Executable:

```
api Executable

    run(): ()

end
```

This API contains the declaration of a single function run, whose type is $(\text{String}\ldots) \to ()$. This type is an *arrow type*; it declares the type of a function's parameter, and its return type. The function run includes a single parameter; the notion $(\text{String}\ldots)$ indicates that it is a *varargs* parameter; the function run can be called with an arbitrary number of string arguments. For example, here are valid calls to this function:

run(“a simple”, “ example”)
run(“run(...)”)
run(“Nobody”, “expects”, “that”)

The return type of *run* is (), pronounced “void”. Type () may be used in Fortress as a return type for functions that have no meaningful return value. There is a single value with type (): the value (), also pronounced “void”. References to value () as opposed to type () are resolved by context.

As with components, APIs can be defined in files and compiled. APIs must be defined in files with the suffix `.fsi`. An `.fsi` file contains source code for exactly one API, and its name must match the file name, minus the suffix. If there are no explicit “`api`” headers, the file is understood to define a single API whose name is the name of the containing file (minus its suffix).

An API is compiled with the shell command “`fortress compile`”. When an API is compiled, it is installed in the resident fortress.

For example, if we store the following API in a file named “`Zeepf.fsi`”:

`api` Zeepf

foo: String → ()

baz: String → String

`end`

then we can compile this API with the following shell command:

```
fortress compile Zeepf.fsi
```

This command compiles the API Zeepf and installs it in the resident fortress.

Unlike component compilation, API compilation does not shadow existing elements of a fortress. If we attempt to compile an API with the same name as an API already defined in the resident fortress, an error is signaled and the fortress is left unchanged. To remove an API, we must first remove all components referring to the API, and then issue the shell command “`fortress removeApi`”.

A component that exports an API must provide a definition for every program construct declared in the API. For example, because our component HelloWorld:

`component` HelloWorld

`export` Executable

run() = *print* “`Hello, world!`”

`end`

exports the API Executable, it must include a definition for the function *run*. The definition of *run* in HelloWorld need not include declarations of the parameter type or return type of *run*, as these can be inferred from the definition of API Executable.

Components are also allowed to *import* APIs. A component that imports an API is allowed to use any of the program constructs declared in that API. For example, the following component imports the API Zeepf and calls the function *foo* declared in Zeepf:

`component` Blargh

`import` Zeepf.{...}

`export` Executable

$run() = foo($"whatever")

`end`

The component Blargh imports declaration *foo* from the API Zeepf and exports the API Executable. Its *run* function is defined by calling function *foo*, defined in Zeepf. In an import statement of the form:

`import` $A.\{S\}$

all names in the list of names S are imported from API A, and can be referred to as unqualified names within the importing component. In the example above, the names we have imported consist of a single name: *foo*. If we had instead written:

`import` Zeepf.$\{foo, baz\}$

then we would have been able to refer to both *foo* and *baz* as unqualified names in Blargh.

Note that no component refers directly to another component, or to constructs defined in another component. Instead, *all external references go through APIs*. This level of indirection provides us with significant power. It provides a way to hide implementation details in a component. Also, our intention is that in future implementations, it will be possible to make use of APIs to provide sophisticated link and upgrade operations for the purposes of building large programs.

Components that contain no import statements and export the API Executable are referred to as *executable components*. They can be compiled and executed directly as stand-alone components. All of our HelloWorld components are executable components. However, if a component imports one or more APIs, it cannot be executed as a stand-alone program. Instead, its imports are resolved to other components that export all of the APIs it imports, to form a new *compound* component. For example, we define the following component in a file named `Ralph.fss`:

> DRAFT NOTE
> The example is commented out because it is not supported yet nor run by the interpreter.

We can now issue the following shell commands:

```
fortress compile Ralph.fss
fortress compile Blargh.fss
fortress run Blargh.fss
```

The first two commands compile files `Ralph.fss` and `Blargh.fss`, respectively, and install them in the resident fortress. The third command tells the resident fortress to run component Blargh. References to API Zeepf in Blargh are resolved to their implementation in component Ralph.

All references to API Zeepf in Gary are resolved to the declarations provided in Ralph.

Note that forming the compound component Gary has no effect on the components Ralph and Blargh. These components remain in the resident fortress, and they can be linked together with other components to form yet more compound components. Conversely, if Blargh or Ralph is recompiled, deleted, or otherwise updated, there is no effect on Gary. Conceptually, Gary contains its own copies of the components Blargh and Ralph, and these copies are not corrupted by actions on other components. For this reason, we say that components in Fortress are *encapsulated*. (Of course, there are optimization tricks that a fortress can use to maintain the illusion of encapsulation without actually copying. But these tricks are beyond the scope of this specification.)

Compound components are *upgradable*: They can be upgraded with new components that export some of the APIs used by their constituents. For example, if a new version of Ralph is compiled and installed in the resident fortress, we can manually *upgrade* Gary with the new version by performing the following shell command:

```
fortress upgrade NewGary from Gary with Ralph
```

This command produces a new component, named `NewGary`, resulting from an upgrade of `Gary` with `Ralph`. The components referred to as `Gary` and `Ralph` are unaffected. An important property of fortress components is that they are *stateless*; once constructed, they are never modified. Even if a component is "removed" from a fortress, the components it is a constituent of are unaffected.

However, we can rebind *the name* `Gary` in the resident fortress to our new component with the following command:

```
fortress upgrade Gary from Gary with Ralph
```

or simply:

```
fortress upgrade Gary with Ralph
```

Now, the original component referred to by `Gary` is shadowed; it cannot be referred to directly from the shell. However, it might still exist in the fortress as a constituent of other components, which are unmodified by the upgrade. To upgrade all components in a fortress at once with a new version of `Ralph` (rebinding all names to the resulting upgrades), we can issue the command:

```
fortress upgradeAll Ralph
```

2.3 Automatic Generation of APIs

Note that the component named Zeepf exports the API Zeepf. Components and APIs exist in separate namespaces, and therefore it is allowed for a component to have the same name as an API. In fact, if we hadn't already defined and compiled API Zeepf, we could generate it automatically from the definition of component Zeepf with the following shell command:

```
fortress api Zeepf.fss
```

This command generates a new source file, `Zeepf.fsi`, in the same directory as `Zeepf.fss`, which includes declarations for all program constructs defined in `Zeepf.fss`.

In general, if a component C exports an API A with the same name as C, it is possible to automatically generate a source file for the API A from C by issuing the shell command `fortress api` on the file containing the definition of C. This API contains declarations for all definitions in C that are not declared in other APIs exported by C, and that do not include the modifier `private`. If a source file with the name of A already exists in the same directory as the source file of C, an error is signaled, and no file is created.

If there is more than one component defined in a file, APIs are generated for all components defined in the file that export APIs with the same name as the respective component. Each generated API is placed in a separate source file whose name corresponds to the name of the API it defines.

Note that the API corresponding to an automatically generated source file is not automatically added to the resident fortress; the source file must still be compiled via a separate action. Often, programmers may want to edit the auto-generated file before compiling it to the fortress.

It is not always desirable for a component to export an API of the same name. Many components will export only publicly defined standard APIs. However, automatic generation of APIs from components may be useful, particularly for components defined internally by a development team. Large projects may contain many internally defined components that are never released externally, except as constituents of compound components.

2.4 Rendering

One aspect of Fortress that is quite different from many other languages is that various program constructs are rendered in particular fonts, so as to emulate mathematical notation. For example, variable names are rendered in italic fonts. Many other program constructs have their own rendering rules. For example, the operator ^ indicates superscripting in Fortress. A function definition consisting of the following ASCII characters:

```
f(x) = x^2 + sin x - cos 2 x
```

is rendered as follows:

$f(x) = x^2 + \sin\ x - \cos\ 2\,x$

Array indexing, written with brackets:

```
a[i]
```

is rendered as follows:

a_i

There are many other examples of special rendering conventions.

Fortress also supports the use of Unicode characters [29] in identifiers. In order to make it easy to enter such characters with today's input devices, Fortress defines a convention for keyboard entry: ASCII names (and abbreviations) of Unicode characters can be written in a program in all caps (with spaces replaced by underscores). Such identifiers are converted to Unicode characters. For example, the identifier:

```
GREEK_CAPITAL_LETTER_LAMBDA
```

is automatically converted into the identifier:

Λ

There are also ASCII abbreviations for writing down commonly used Fortress characters. For example, ASCII identifiers for all Greek letters are converted to Greek characters (e.g., "`lambda`" becomes λ and "`LAMBDA`" becomes Λ). Here are some other common ASCII shorthands:

BY	*becomes*	$\times$	TIMES	*becomes*	$\times$
DOT	*becomes*	$\cdot$	CROSS	*becomes*	$\times$
CUP	*becomes*	$\cup$	CAP	*becomes*	$\cap$
BOTTOM	*becomes*	$\bot$	TOP	*becomes*	$\top$
SUM	*becomes*	$\sum$	PROD	*becomes*	$\prod$
INTEGRAL	*becomes*	$\int$	EMPTYSET	*becomes*	$\emptyset$
SUBSET	*becomes*	$\subset$	NOTSUBSET	*becomes*	$\not\subset$
SUBSETEQ	*becomes*	$\subseteq$	NOTSUBSETEQ	*becomes*	$\not\subseteq$
EQUIV	*becomes*	$\equiv$	NOTEQUIV	*becomes*	$\not\equiv$
IN	*becomes*	$\in$	NOTIN	*becomes*	$\notin$
LT	*becomes*	$<$	LE	*becomes*	$\leq$
GT	*becomes*	$>$	GE	*becomes*	$\geq$
EQ	*becomes*	$=$	NE	*becomes*	$\neq$
AND	*becomes*	$\wedge$	OR	*becomes*	$\vee$
NOT	*becomes*	$\neg$	XOR	*becomes*	$\oplus$
INF	*becomes*	∞	SQRT	*becomes*	$\surd$

A comprehensive description of ASCII conversion is provided in Appendix E.

2.5 Some Common Types in Fortress

Fortress provides a wide variety of standard types, including String, $\mathrm{Boolean}$, and various numeric types. The floating-point type $\mathbb{R}64$ (written in ASCII as `RR64`) is the type of 64-bit precision floating-point numbers. For example, the following function takes a 64-bit float and returns a 64-bit float:

$$halve(x\colon \mathbb{R}64)\colon \mathbb{R}64 = \frac{x}{2}$$

Predictably, 32-bit precision floats are written $\mathbb{R}32$.

64-bit integers are denoted by the type $\mathbb{Z}64$ and 32-bit integers by the type $\mathbb{Z}32$ and infinite precision integers by the type $\mathbb{Z}$.

2.6 Functions in Fortress

Fortress allows recursive, and mutually recursive function definitions. Here is a simple definition of a *factorial* function in Fortress:

$factorial(n) =$

`if` $n = 0$ `then` 1

`else` $n\, factorial(n-1)$ `end`

2.6.1 Juxtaposition and function application

In the definition of *factorial*, note the juxtaposition of parameter n with the recursive call $factorial(n-1)$. In Fortress, as in mathematics, multiplication is represented through juxtaposition. By default, two expressions of numeric type that are juxtaposed represent a multiplication. On the other hand, juxtaposition of an expression of function type with another expression to its right represents function application, as in the following example:

$\sin x$

Moreover, juxtaposition of expressions of string type represents concatenation, as in the following example:

"`Hi,`" "` it's`" "` me`" "` again.`"

In fact, juxtaposition is an operator in Fortress, just like $+$ and $-$, that is overloaded based on the runtime types of its arguments.

There is a special case concerning juxtaposition that is important to mention: When writing a numeric literal, the *digit-group separator*, `NARROW NO-BREAK SPACE` (U+202F), may be included between the digits in order to improve readability. In ASCII, this character can be abbreviated in the context of a numeric literal (*and only in this context*) with the character `'`. For example, the number 299,792,458 can be denoted in ASCII by the following numeric literal:

```
299'792'458
```

This literal is converted into Unicode by replacing all occurrences of `'` with `NARROW NO-BREAK SPACE`:

299 792 458

This is not an application of the juxtaposition operator; rather, it is a single token. Note that commas cannot be used to separate digits because ambiguities would arise with the use of commas in other expressions, such as tuples.

2.6.2 Keyword Parameters

Functions in Fortress can be defined to take keyword arguments by providing default values for parameters. In the following example:

$$
\begin{array}{l}
\mathit{makeColor}(\mathit{red}:\mathbb{Z}64 = 0, \mathit{green}:\mathbb{Z}64 = 0, \mathit{blue}:\mathbb{Z}64 = 0) = \\
\quad \texttt{if } 0 \le \mathit{red} \le 255 \wedge 0 \le \mathit{green} \le 255 \wedge 0 \le \mathit{blue} \le 255 \\
\quad \texttt{then } \mathrm{Color}(\mathit{red}, \mathit{green}, \mathit{blue}) \\
\quad \texttt{else throw } \mathrm{Error} \texttt{ end}
\end{array}
$$

the function $\mathit{makeColor}$ takes three keyword arguments, all of which default to 0. If we call it as follows:

$$\mathit{makeColor}(\mathit{green} = 255)$$

the arguments red and blue are both given the value 0.

There are some other aspects of this example worth mentioning. For example, the body of this function consists of an `if` expression. The test of the `if` expression checks that all three parameters have values between 0 and 255. The boolean operator $\le$ is *chained*: An expression of the form $x \le y \le z$ is equivalent to the expression $x \le y \wedge y \le z$, just as in mathematical notation. The `then` clause provides an example of a constructor call in Fortress, and the `else` clause shows us an example of a `throw` expression in Fortress.

> DRAFT NOTE
> How can we tell this from the example? Indeed, in general, we can't distinguish constructors from other functions looking only at the call sites.

2.6.3 Varargs Parameters

It is also possible to define functions that take a variable number of arguments. For example:

$$
\begin{array}{l}
\mathit{printFirst}(\mathit{xs}\colon \mathbb{Z}32\ldots) = \\
\quad \texttt{if } \mathit{xs}.\mathit{reduce}\big(\mathrm{SizeReduction}[\![\mathbb{Z}32]\!]()\big) > 0 \texttt{ then } \mathit{println}\ \mathit{xs}_0 \\
\quad \texttt{else throw } \mathrm{Error} \texttt{ end}
\end{array}
$$

This function takes an arbitrary number of integers and prints the first one (unless it is given zero arguments; then it throws an exception).

2.6.4 Function Overloading

Functions can be overloaded in Fortress by the types of their parameters. Calls to overloaded functions are resolved based on the runtime types of the arguments. For example, the following function is overloaded based on parameter type:

$$\mathit{size}(x\colon \mathrm{Nil}) = 0$$

$$\mathit{size}(x\colon \mathrm{Cons}) = 1 + \mathit{size}\big(\mathit{rest}(x)\big)$$

If we call size on an object with runtime type Cons, the second definition of size will be invoked *regardless of the static type of the argument*. Of course, function applications are statically checked to ensure that *some* definition will be applicable at run time, and that the definition to apply will be unambiguous.

2.6.5 Function Contracts

Fortress allows *contracts* to be included in function declarations. Among other things, contracts allow us to *require* that the argument to a function satisfies a given set of constraints, and to *ensure* that the resulting value satisfies some constraints. They provide essential documentation for the clients of a function, enabling us to express semantic properties that cannot be expressed through the static type system.

Contracts are placed at the end of a function header, before the function body. For example, we can place a contract on our $factorial$ function requiring that its argument be nonnegative as follows:

$factorial(n)$ `requires` $\{\, n \geq 0 \,\}$

$=$ `if` $n = 0$ `then` 1

`else` $n\, factorial(n-1)$ `end`

We can also ensure that the result of $factorial$ is itself nonnegative:

$factorial(n)$

`requires` $\{\, n \geq 0 \,\}$

`ensures` $\{\, outcome \geq 1$ `provided` $true \,\}$

$=$ `if` $n = 0$ `then` 1

`else` $n\, factorial(n-1)$ `end`

The variable `outcome` is bound in the `ensures` clause to the return value of the function.

2.7 Some Common Expressions in Fortress

We have already seen an `if` expression in Fortress. Here's an example of a `while` expression:

```
while x < 10 do
  println x
  x += 1
end
```

A sequence of expressions in Fortress may be delimited by `do` and `end`. Here is an example of a function that prints three words:

```
printThreeWords() = do
  print "print"
  print " three"
  print " words"
end
```

A *tuple* expression contains a sequence of elements delimited by parentheses and separated by commas:

(“this”, “is”, “a”, “tuple”, “of”, “mostly”, “strings”, 0)

When a tuple expression is evaluated, the various subexpressions are evaluated *in parallel*. For example, the following tuple expression denotes a parallel computation:

$\big(factorial(10), factorial(5), factorial(2)\big)$

The elements in this expression may be evaluated in parallel. This same computation can be expressed as follows:

```
do
  factorial(10)
also do
  factorial(5)
also do
  factorial(2)
end
```

2.8 For Loops Are Parallel by Default

Here is an example of a simple `for` loop in Fortress:

```
for i ← 1:10 do
  print(i " ")
end
```

This `for` loop iterates over all elements i between 1 and 10 and prints the value of i. Expressions such as $1:10$ are referred to as *range expressions*. They can be used in any context where we wish to denote all the integers between a given pair of integers.

A significant difference between Fortress and most other programming languages is that *for loops are parallel by default*. Thus, printing in the various iterations of this loop can occur in an arbitrary order, such as:

5 4 6 3 7 2 9 10 1 8

2.9 Atomic Expressions

In order to control interactions of parallel executions, Fortress includes the notion of *atomic expressions*, as in the following example:

```
atomic do
  x += 1
  y += 1
end
```

An atomic expression is executed in such a manner that all other threads observe either that the computation has completed, or that it has not yet begun; no other thread observes an atomic expression to have only partially completed. Consider the following parallel computation:

```
do
  x: ℤ32 := 0
  y: ℤ32 := 0
  z: ℤ32 := 0
  atomic do
    x += 1
    y += 1
  also atomic do
    z := x + y
  end
  z
end
```

Both parallel blocks are atomic; thus the second parallel block either observes that both x and y have been updated, or that neither has. Thus, possible values of the outermost `do` expression are 0 and 2, but not 1.

2.10 Dimensions and Units

Numeric types in Fortress can be annotated with physical units and dimensions. For example, the following function declares that its parameter is a tuple represented in the units kg and $\mathrm{m/s}$, respectively:

> DRAFT NOTE
> I MANUALLY EDITED THIS CODE -Victor

$$kineticEnergy(m : \mathbb{R}64\,\mathrm{kg}, v : \mathbb{R}64\,\mathrm{m/s}) : \mathbb{R}64\,\mathrm{kg\,m^2/s^2} = (m\ v^2)/2$$

A value of type $\mathbb{R}64\,\mathrm{kg}$ is a 64-bit float representing a measurement in kilograms. When the function $kineticEnergy$ is called, two values in its tuple argument are statically checked to ensure that they are in the right units.

All commonly used dimensions and units are provided in the Fortress standard libraries. Unit symbols are encoded with trailing underscores; such identifiers are rendered in roman font. For example the unit m is represented as $\mathrm{m_}$. For each unit, both longhand and shorthand names are provided (e.g., m, meter, and meters). The various names of a given unit can be used interchangeably. Also, some units (and dimensions) are defined to be synonymous with algebraic combinations of other units (and dimensions). For example, the unit N is defined to be synonymous with the unit $\mathrm{kg\,m/s^2}$ and the dimension Force is defined to be synonymous with $\mathrm{Mass\,Acceleration}$. Likewise, the dimension $\mathrm{Acceleration}$ is defined to be synonymous with $\mathrm{Velocity/Time}$.

Measurements in the same unit can be compared, added, subtracted, multiplied and divided. Measurements in different units can be multiplied and divided. For example, we can write the following variable declaration:

$v : \mathbb{R}\,\mathrm{m/s} = (3\,\mathrm{meters} + 4\,\mathrm{meters})/5\,\mathrm{seconds}$

However, the following variable declaration is a static error:

$v : \mathbb{R}\,\mathrm{m/s} = (3\,\mathrm{meters} + 4\,\mathrm{seconds})/5\,\mathrm{seconds}$

In addition, the following variable declaration is a static error because the unit of the left hand side of this declaration is $\mathrm{m/s}$ whereas the unit of the right hand side is simply m (or meters):

$v : \mathbb{R}\,\mathrm{m/s} = (3\,\mathrm{meters} + 4\,\mathrm{meters})/5$

It is also possible to convert a measurement in one unit to a measurement of another unit of the same dimension, as in the following example:

$kineticEnergy(3.14\,\mathrm{kg}, 32\,\mathrm{f/s}\ \texttt{in}\ \mathrm{m/s})$

The second argument to $kineticEnergy$ is a measurement in feet per second, converted to meters per second.

2.11 Aggregate Expressions

As with mathematical notation, Fortress includes special syntactic support for writing down many common kinds of collections, such as arrays, matrices, vectors, maps, sets, and lists simply by enumerating all of the collection's elements. We refer to an expression formed by enumerating the elements of a collection as an *aggregate expression*. The elements of an aggregate expression are computed in parallel.

For example, we can define an array a in Fortress by explicitly writing down its elements, enclosed in brackets and separated by whitespace, as follows:

$a\colon \mathbb{Z}32_5 = [0\ 1\ 2\ 3\ 4]$

Two-dimensional arrays can be written down by separating rows by newlines (or by semicolons). For example, we can bind b to a two-dimensional array as follows:

$$b\colon \mathbb{Z}32_{2,2} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$$

There is also support for writing down arrays of dimension three and higher. We bind c to a three-dimensional array as follows:

$$c\colon \mathbb{Z}32_{2,2,3} = \left[\begin{array}{cc} 1 & 2 \\ 3 & 4 \\ \\ 5 & 6 \\ 7 & 8 \\ \\ 9 & 10 \\ 11 & 12 \end{array}\right]$$

Various slices of the array along the third dimension are separated by pairs of semicolons. (Higher dimensional arrays are also supported. When writing a four-dimensional array, slices along the fourth dimension are separating by triples of semicolons, and so on.)

Vectors are written down just like one-dimensional arrays. Similarly, matrices are written down just like two-dimensional arrays. Whether an array aggregate expression evaluates to an array, a vector, or a matrix is inferred from context (e.g., the static type of a variable that such an expression is bound to). Of course, all elements of vectors and matrices must be numbers.

Note that there is a syntactic conflict between the use of juxtaposition to represent elements along a row in an array and the use of juxtaposition to represent applications of the juxtaposition operator, and the use of spaces to seperate digits in a numeric literal. In order to include an application of the juxtaposition operator or a numeric literal with spaces as an outermost subexpression of an array aggregate, it is necessary to include extra parentheses. For example, the following aggregate expression represents a counterclockwise rotation matrix by an angle θ in $\mathbb{R}64^2$:

$$d\colon \mathbb{R}64_{2,2} = \begin{bmatrix} (\cos\,\theta) & (-\sin\,\theta) \\ (\sin\,\theta) & (\cos\,\theta) \end{bmatrix}$$

A set can be written down by enclosing its elements in braces and separating its elements by commas. Here we bind s to the set of integers 0 through 4:

$s = \{0, 1, 2, 3, 4\}$

The elements of a list are enclosed in angle brackets (written in ASCII as `<|` and `|>`):

$l = \langle 0, 1, 2, 3, 4 \rangle$

The elements of a map are enclosed in curly braces, with key/value pairs joined by the arrow $\mapsto$ (written in ASCII as `|->`):

$m = \{\, \text{“}\texttt{a}\text{”} \mapsto 0, \text{“}\texttt{b}\text{”} \mapsto 1, \text{“}\texttt{c}\text{”} \mapsto 2 \,\}$

2.12 Comprehensions

Another way in which Fortress mimics mathematical notation is in its support for *comprehensions*. Comprehensions describe the elements of a collection by providing a rule that holds for all of the collection's elements. The elements of the collection are computed in parallel by default. For example, we define a set s that consists of all elements of another set t divided by 2, as follows:

$s = \{\, x \div 2 \mid x \leftarrow t \}$

The expression to the left of the vertical bar explains that elements of s consist of every value $x/2$ for every valid value of x (determined by the right hand side). The expression to the right of the vertical bar explains how the elements x are to be *generated* (in this case, from the set t). The right hand side of a comprehension can consist of multiple generators. For example, the following set consists of every element resulting from the sum of an element of s with an element of t:

$u = \{x + y \mid x \leftarrow s, y \leftarrow t\}$

The right hand side of a comprehension can also contain *filtering expressions* to constrain the elements provided by other clauses. For example, we can stipulate that v consists of all nonnegative elements of t as follows:

$v = \{x \mid x \leftarrow t, x \geq 0\}$

There is a comprehension expression for every form of aggregate expression. For example, here is a list comprehension in Fortress:

$l = \langle\, 2\,x \mid x \leftarrow v \,\rangle$

The elements of this list consist of all elements of the set v multiplied by 2.

In the case of an array comprehension, the expression to the left of the bar includes a tuple indexing the elements of the array. For example, the following comprehension describes a 3×3 array, all of whose elements are zero.

$$[(x, y) = 0 \mid x \leftarrow \{0, 1, 2\}, y \leftarrow \{0, 1, 2\}]$$

The collection of elements 0 through 2 can also be expressed via a range expression, as follows:

$$[(x, y) = 0 \mid x \leftarrow 0:2, y \leftarrow 0:2]$$

An array comprehension can consist of multiple clauses. The clauses are run in the order provided, with declarations from later clauses shadowing declarations from earlier clauses. For example, the following comprehension describes a 3×3 identity matrix:

$$\begin{array}{l}[(x, y) = 0 \mid x \leftarrow 0:2, y \leftarrow 0:2 \\ \ (x, x) = 1 \mid x \leftarrow 0:2]\end{array}$$

2.13 Summations and Products

As with mathematical notation, Fortress provides syntactic support for summations and productions (and other big operations) over the elements of a collection. For example, an alternative definition of *factorial* is as follows:

$$factorial(n) = \prod_{i \leftarrow 1:n} i$$

This function definition can be written in ASCII as follows:

```
factorial(n) = PROD[i <- 1:n] i
```

The character $\prod$ is written `PROD`. Likewise, $\sum$ is written `SUM`. As with comprehensions, the values in the iteration are generated from specified collections.

2.14 Tests and Properties

Fortress includes support for automated program testing. A new test in a component can be defined using the `test` modifier on a top-level function definition with type "$() \to ()$". For example, here is a test function that calls the *factorial* function on some representative values and checks that the calls result in values greater than or equal to what was provided:

`test` $factorialResultLarger() =$ `do`

$assert\big(0 \le factorial(0)\big)$

$assert\big(1 \le factorial(1)\big)$

$assert\big(10 \le factorial(10)\big)$

println "`Test factorialResultLarger succeeded.`"

`end`

`test` $factorialResultLarger[x \leftarrow 0:100] = x \le factorial(x)$

The values for x are generated from the provided range $0:100$.

Programs are also allowed to include *property* declarations, documenting boolean conditions that a program is expected to obey. Property declarations are similar syntactically to test declarations. Unlike test declarations, property declarations do not specify explicit finite collections over which the property is expected to hold. Instead, the parameters in a property declaration are declared to have types, and the property is expected to hold over all values of those types. For example, here is a property declaring that $factorial$ is greater than or equal to every argument of type $\mathbb{Z}$:

```
property factorialResultAlwaysLarger = ∀(x : ℤ) (x ≤ factorial(x))
```

When a property declaration includes a name, the property identifier is bound to a function whose parameter and body are that of the property, and whose return type is $\mathrm{Boolean}$. A function bound in this manner is referred to as a *property function*. A property function can be called from within tests to ensure that a property holds at least for all values in some finite set of values.

2.15 Objects and Traits

A great deal of programming can be done simply through the use of functions, top-level variables, and standard types. However, Fortress also includes a trait and object system for defining new types, as well as objects that belong to them. Traits in Fortress exist in a multiple inheritance hierarchy rooted at the traits Object and Index.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.42
>
> The Working Draft of February 2011 rooted the hierarchy at Object; Index, which the revival adds to the library, extends only Any (Section 10.1).

A trait declaration includes a set of method declarations, some of which may be abstract. For example, here is a declaration of a simple trait named Moving:

```
trait Moving extends { Tangible, Object }
  position(): ℝ^3
  velocity(): ℝ^3
end
```

The set of traits extended by trait Moving are listed in braces after `extends`. Trait Moving inherits all methods declared by each trait it extends. The two methods $position$ and $velocity$ declared in trait Moving are *abstract*; they contain no body. Their return types are vectors of length 3, whose elements are of type $\mathbb{R}$. As in mathematical notation, a vector of length n with element type T is written T^n.

Traits can also declare concrete methods, as in the following example:

```
trait Fast extends Moving
  velocity() = [0 0 299792458]
end
```

Trait Fast extends a single trait, Moving. (Because it extends only one trait, we can elide the braces in its `extends` clause.) It inherits both abstract methods defined in Moving, and it provides a concrete body for method $velocity$.

Trait declarations can be extended by other trait declarations, as well as by *object declarations*. There are two kinds of object declarations: *singleton declarations* and *constructor declarations*.

A singleton declaration declares a sole, stand-alone, *singleton object*. For example:

```
object Sol extends { Moving, Stellar }
  spectralClass = G2
  position() = [0 0 0]
  velocity() = [0 0 0]
end
```

The object Sol extends two traits: Moving and Stellar, and provides definitions for the abstract methods it inherits. Objects must provide concrete definitions for all abstract methods they inherit. Sol also defines a field *spectralClass*. For every field included in an object definition, an implicit getter is defined for that field. Calls to the getter of the field of an object consist of an expression denoting the object followed by a dot and the name of the field. For example, here is a call to the getter *spectralClass* of object Sol:

```
Sol.spectralClass
```

If a field includes modifier `settable`, an implicit setter is defined for the field. Calls to setters look like assignments. Here is an assignment to field *spectralClass* of object Sol:

```
Sol.spectralClass := G3
```

In fact, all accesses to a field from outside the object to which the field belongs must go through the getter of the field, and all assignments to it must go through the setter. (There is simply no other way syntactically to refer to the field.)

If a field includes modifier `hidden`, no implicit getter is defined for the object.

The self parameter of a method can be given a position other than the default position. For example, here is a definition of a type where the self parameters appear in nonstandard positions:

```
trait List
  cons(x: Object, self): List
  append(xs: List, self): List
end
```

In both methods *cons* and *append*, the self parameter occurs as the second parameter. Calls to these methods look more like function calls than method calls. For example, in the following call to *append*, the receiver of the call is l_2:

```
append(l1, l2)
```

A constructor declaration declares an *object constructor*. In contrast to singleton declarations, a constructor declaration includes value parameters in its header, as in the following example:

```
object Particle(position: ℝ³, velocity: ℝ³)
  extends Moving
end
```

Every call to the constructor of Particle yields a new object. For example:

$pos\colon \mathbb{R}^3 = [3\ 2\ 5]$

$vel\colon \mathbb{R}^3 = [1\ 0\ 0]$

$p_1 = \text{Particle}(pos, vel)$

The parameters to an object constructor implicitly define fields of the object, along with their getters (by default). These parameters can include all of the modifiers that an ordinary field can. For example, a parameter can include the modifier `hidden`, in which case a getter is not implicitly defined for the field.

In the definition of Particle, it is the implicitly defined getters of fields *position* and *velocity* that provide concrete definitions for the inherited abstract methods from trait Moving.

In both singleton and constructor declarations, implicitly defined getters and setters can be overridden by defining explicit getters and setters, using the modifiers `getter` and `setter`. For example, we alter the definition of Particle to include an explicit getter for field *velocity* as follows:

```
object Particle(position : (ℝ Length)^3,
                velocity : (ℝ Velocity)^3)
  extends Moving
  getter velocity() = do
    print "velocity getter accessed"
    velocity
  end
end
```

The explicitly defined getter first prints a message and then returns the value of the field *velocity*. *Note that the final variable reference in this getter refers directly to the field velocity, not to the getter.* Only within an object definition can fields be accessed directly. In fact, even in an object definition, fields are only accessed directly when they are referred to as simple variables. In the following definition of Particle, the getter *velocity* is recursively accessed through the special variable `self` (bound to the receiver of the method call):

```
object Particle(position : (ℝ Length)^3,
                velocity : (ℝ Velocity)^3)
  extends Moving
  getter velocity() = do
    print "velocity getter accessed"
    self.velocity
  end
end
```

Now, the result of a call to getter *velocity* is an infinite loop (along with a lot of output).

2.15.1 Traits, Getters, and Setters

Although traits do not include field declarations, they can include getter and setter declarations, as in the following alternative definition of trait Moving:

```
trait Moving extends { Tangible, Object }
  getter position(): (ℝ Length)^3
  getter velocity(): (ℝ Velocity)^3
end
```

Now, an object extending trait Moving must provide the definitions of *getters* (as opposed to ordinary methods) for $position$ and $velocity$. The advantage of this definition is that we can use getter notation on a variable of static type Moving:

$v : \mathrm{Moving} = \ldots$
$v.position$

In fact, getters can be declared in a trait definition using field declaration syntax, as in the following example (which is equivalent to the definition of Moving above):

```
trait Moving extends { Tangible, Object }
  position: (ℝ Length)^3
  velocity: (ℝ Velocity)^3
end
```

Getter declarations in trait definitions must not include bodies. A getter declaration can include the various modifiers allowed on a field declaration, with similar results. For example, if the modifier `settable` is used, then a setter is implicitly declared, in addition to a getter.

2.16 Features for Library Development

The language features introduced above are sufficient for the vast majority of applications programming. However, Fortress has also designed to be a good language for *library* programming. In fact, much of the Fortress language as viewed by applications programmers actually consists of code defined in libraries, written in a small core language. By defining as much of the language as possible in libraries, our aim is to allow the language to evolve gracefully as new demands are placed on it. In this section, we briefly mention some of the features that make Fortress a good language for library development.

2.16.1 Generic Types and Static Parameters

As in other languages, Fortress allows types to be parametric with respect to other types, as well as other "static" parameters, including integers, booleans, dimensions, units, and operators. Fortress provides some standard parametric types, such as Array and Vector. Programmers can also define new traits, objects, and functions that include static parameters.

2.16.2 Specification of Locality and Data Distribution

Fortress includes a facility for expressing programmer intent concerning the distribution of large data structures at run time. This intent is expressed through special data structures called *distributions*. In fact, all arrays and matrices include distributions. These distributions are defined in the Fortress standard libraries. In most cases, the default distributions will provide programmers with good performance over a variety of machines. However, in some circumstances, performance can be improved by overriding the default selection of distributions. Moreover, some programmers might wish to define new distributions, tailored for particular platforms and programs.

2.16.3 Operator Overloading

Operators in Fortress can be overloaded with new definitions. Here is an alternative definition of the $factorial$ function, defined as a postfix operator:

$$\texttt{opr}\ (n\colon \mathbb{Z}32)!\ = \prod_{i \leftarrow 1:n} i$$

As with mathematical notation, Fortress allows operators to be defined as prefix, postfix, and infix. More exotic operators can even be defined as subscripting (i.e., applications of the operators look like subscripts), and as bracketing (i.e., applications of the operators look like the operands have been simply enclosed in brackets).

2.16.4 Definition of New Syntax

Fortress provides a facility for the definition of new syntax in libraries. This facility is useful for defining libraries for domain-specific languages, such as SQL, XML, etc. It is also used to encode some of the language constructs seen by applications programmers.

Part II

Fortress for Application Programmers

Chapter 3

Programs

> DRAFT NOTE
> Unicode synonyms and most of the ASCII encodings of Unicode characters are not yet supported.

A *program* consists of a finite sequence of Unicode 5.0 abstract characters. (Fortress does not distinguish between different code-point encodings of the same character.) In order to more closely approximate mathematical notation, certain sequences of characters are rendered as subscripts or superscripts, italicized or boldface text, or text in special fonts, according to the rules in Appendix D. For the purposes of entering program text, ASCII encodings of Unicode characters are also provided in Appendix E. Although much of the program text in this specification is rendered as formatted Unicode, some text is presented unformatted, or in its ASCII encoding, to aid in exposition.

A program is *valid* if it satisfies all *static constraints* stipulated in this specification. Failure to satisfy a static constraint is a *static error*. Only valid programs can be *executed*; the validity of a program must be checked before it is executed.

Executing a valid Fortress program consists of *evaluating expressions*. Evaluation of an expression may modify the *program state* yielding a *result*. A result is either a *value*, or an *abrupt completion*.

The characters of a valid program determine a sequence of *input elements*. In turn, the input elements of a program determine the *program constructs*. Some program constructs may contain other program constructs. The two most common kinds of constructs in Fortress are *declarations* and *expressions*. We explain the structure of input elements, and of each program construct, in turn, along with accompanying static constraints. We also explain how the outcome of a program execution is determined from the sequence of constructs in the program.

Programs are developed, compiled and deployed as *encapsulated upgradable components* as described in Chapter 22.

Fortress is block-structured: A Fortress program consists of nested *blocks* of code. The entire program is a single block. Each component is a block. Any top-level declaration is a block, as is any function declaration. Several expressions are also blocks, or have blocks as parts of the expression, or both (e.g., a `while` expression is a block, and its body is a different block). See Chapter 13 for a discussion of expressions in Fortress. In addition, a local declaration begins a block that continues to the end of the smallest enclosing block unless it is a local function declaration and is immediately preceded by another local function declaration. (This exception allows overloaded and mutually recursive local function declarations.) Because Fortress is block-structured, and because the entire program is a block, the smallest block that syntactically contains a program construct is always well defined.

Fortress is expression-oriented: What are often called "statements" in other languages are just expressions with type $()$ in Fortress. They do not evaluate to an interesting value, and are typically evaluated solely for their effects.

Fortress is whitespace-sensitive: Fortress has different contexts influencing the whitespace-sensitivity of expressions as described in Appendix G.

Chapter 4

Lexical Structure

DRAFT NOTE

Unicode synonyms, `SUBSTITUTE`, some forbidden and restricted characters, connecting punctuation, `LINE SEPARATOR`, `PARAGRAPH SEPARATOR`, most of the ASCII Conversion, and dimensions and units are not yet supported.

DRAFT NOTE

- Towards eliminating transformation expressions in the syntax abstraction mechanism (07/14/08)
  - Jan's desugaring generated expressions requires a non-Fortress syntax: invoking a big operator declaration with a usual functional call syntax (`BIG OPLUS()`) instead of a usual big-operator application syntax (reductions or comprehensions).
  - It is ambiguous because `BIG Op Expr` is a syntactic sugar for `BIG Op [x <- Expr] x`.
  - In order to eliminate transformation expressions from the syntax abstraction mechanism, we need to be able to write the desugaring in Fortress.
  - So, we are proposing to eliminate that syntactic sugar and allow a usual functional call syntax for invoking a big operator declaration.
  - **Fortress syntax for using big operators**:
    - `BIG Op [ GeneratorClauseList ] Expr`: reduction; desugared into the call to the `BIG Op` declaration
    - `BIG Op Expr`: call to the `BIG Op` declaration; not desugared
    - `BIG? LeftEncloser Expr | GeneratorClauseList RightEncloser`: comprehension; desugared into the call to the `BIG LeftEncloser RightEncloser` declaration
    - `BIG LeftEncloser RightEncloser Expr`: call to the `BIG LeftEncloser RightEncloser` declaration; not desugared
  - **Syntactic sugar**: `BIG Op Expr` (for `BIG Op [x <- Expr] x`) Supports it as a call to yet another `BIG Op` declaration. The body of the declaration would be a call to the usual `BIG Op` declaration.
- Incorporate Stix fonts

DRAFT NOTE

- Identifier rendering (12/10/07) `a''`, `a'_prime`, `a_prime'`, `a_prime_prime` are rendered identically and they are meant to denote the same identifier. Victor proposed to treat them as we treat synonym operators: a single definition for one is for all the other names.
- No special rendering after the keyword `opr`. (06/18/07)

DRAFT NOTE

Victor:

- NEXT LINE (0085)? (Is this a line terminator?)
- Is : an operator? (Does this work with parsing?)
- New "preprocessing" story–not preprocessing any more
- How should $a^{+}b$ be interpreted? Should be a static error!
- In operators section, note that some words that would be operators are not actually possible because of preprocessing, e.g., ALPHA. (Also true for character literals, and there they may show up as escape sequences in string literals.)
- Are true and false not reserved?

A Fortress program consists of a finite sequence of Unicode 5.0 abstract characters [29]. Every character in a program is part of an input element. The partitioning of the character sequence into input elements is uniquely determined by the characters themselves. In this chapter, we explain how the sequence of input elements of a program is determined from a program's character sequence.

This chapter also describes standard ways to *render* (that is, *display*) individual input elements in order to approximate conventional mathematical notation more closely. Other rules, presented in later chapters, govern the rendering of certain *sequences* of input elements; for example, the sequence of three input elements a, `^`, and b may be rendered a^b. The rules of rendering are "merely" a convenience intended to make programs more readable. Alternatively, the reader may prefer to think of the rendered presentation of a program as its "true form" and to think of the underlying sequence of Unicode characters as "merely" a convenient way of encoding mathematical notation for keyboarding purposes.

Most of the program text in this specification is shown in rendered presentation form. However, sometimes, particularly in this chapter, unformatted code is presented to aid in exposition. In many cases, the unformatted form is shown alongside the rendered form in a table, or following the rendered form in parentheses.

DRAFT NOTE

Victor: removed the following example; it is a case of ASCII conversion, not rendering:

as an example, consider the operator $\oplus$ (`OPLUS`).

4.1 Characters

A Unicode 5.0 abstract character is the smallest element of a Fortress program. [1] Many characters have standard *glyphs*, which are how these characters are most commonly depicted. However, more than one character may be represented by the same glyph. Thus, Unicode 5.0 specifies a representation for each character as a sequence of *code points*. Fortress programs are not permitted to contain characters that map to multiple code points, or to sequences of code points of length greater than 1. Thus, every character in a Fortress program is associated with a single code point,

[1] Note that a single Unicode abstract character may have multiple encodings. Fortress does not distinguish between different encodings of the same character that may be used in representing source code, and it treats canonically equivalent characters as identical. See the Unicode Standard [29] for a full discussion of encoding and canonical equivalence.

designated by a hexadecimal numeral preceded by "U+". Unicode also specifies a name for each character;[2] when introducing a character, we specify its code point and name, and sometimes the glyph we use to represent it in this specification. In some cases, we use such glyphs without explicitly introducing the characters (as, for example, with the simple upper- and lowercase letters of the Latin and Greek alphabets). When the character represented by a glyph is unclear and the distinction is important, we specify the code point or the name (or both). The Unicode Standard [29] specifies a *general category* for each character, which we use to describe sets of characters below.

We partition the Unicode 5.0 character set into the following (disjoint) classes:

> DRAFT NOTE
> The Unicode general categories are:
>
> Lu, Ll, Lt, Lm, and Lo (uppercase, lowercase, titlecase, modifier, other)
>
> Mn, Mc, Me ("marks": nonspacing, spacing combining, enclosing)
>
> Nd, Nl, No (numbers: decimal, letter, or other)
>
> Zs, Zl, Zp (separators: space, line, and paragraph)
>
> Cc, Cf, Cs, Co, Cn ("control" characters: control, format, surrogate, private use, unassigned)
>
> Pc, Pd, Ps, Pe, Pi, Pf and Po (punctuation: connector, dash, open, close, initial quote, final quote, other);
>
> Sm, Sc, Sk, So (symbols: math, currency, modifier, other).
>
> Note however, that these are designations for Unicode *codepoints*, not abstract characters. I'm going to talk as if these are the same right now, because I don't know how to characterize abstract characters. However, I'm not sure that "letter modifiers" and "format characters" for example, should be considered abstract characters.

- *special non-operator characters*, which are:

U+0026	AMPERSAND	&	U+0027	APOSTROPHE	′
U+0028	LEFT PARENTHESIS	(	U+0029	RIGHT PARENTHESIS	)
U+002C	COMMA	,	U+002E	FULL STOP	.
U+0038	SEMICOLON	;	U+005C	REVERSE SOLIDUS	\
U+2026	HORIZONTAL ELLIPSIS	...			
U+2200	FOR ALL	∀	U+2203	THERE EXISTS	∃
U+2254	COLON EQUALS	:=			
U+27E6	MATHEMATICAL LEFT WHITE SQUARE BRACKET	⟦			
U+27E7	MATHEMATICAL RIGHT WHITE SQUARE BRACKET	⟧			

- *special operator characters*, which are

U+002A	ASTERISK	*	U+002F	SOLIDUS	/
U+003A	COLON	:	U+003C	LESS-THAN SIGN	<
U+003D	EQUALS SIGN	=	U+003E	GREATER-THAN SIGN	>
U+005B	LEFT SQUARE BRACKET	[	U+005D	RIGHT SQUARE BRACKET	]
U+005E	CIRCUMFLEX ACCENT	ˆ	U+007B	LEFT CURLY BRACKET	{
U+007C	VERTICAL LINE	|	U+007D	RIGHT CURLY BRACKET	}
U+2192	RIGHTWARDS ARROW	→	U+21A6	RIGHTWARDS ARROW FROM BAR	↦
U+21D2	RIGHTWARDS DOUBLE ARROW	⇒			

[2] There are sixty-five "control characters", which do not have proper names. However, many of them have *Unicode 1.0 names*, or other standard names specified in the Unicode Character Database, which we use instead.

> DRAFT NOTE
>
> 1) SOLIDUS (U+002F) was an ordinary operator character in 1.0b but it is a special operator character in 1.0. Why is it so?
>
> When I compiled the list of special characters for F1.0beta, I was only thinking about those characters that had special meaning by themselves. However, when rethinking about this for F1.0, I thought that didn't make sense, since this information used by the "scanner". So I included as special, any character that I could think really had some special use. This included all characters that combine to form multicharacter enclosing operators. Hence the inclusion of *, ¡, ¿, and of course, /, on that list. That said, / should probably have been listed as special anyway, as it is handled specially in the parsing of expressions (i.e., we need to distinguish "tight fractions" from "loose" ones).
>
> This classification of "special" is somewhat arbitrary. Basically, the way I thought about it is that a character is special if it shows up directly in the grammar, and not in some group that we've otherwise distinguished (e.g., string literal delimiters), or if it has different uses in different contexts (e.g., : and -¿). I thought about creating a class of "combining characters for multicharacter enclosers", but all those characters have other purposes too, so I just listed them all as special.

- *letters*, which are characters with Unicode general category Lu, Ll, Lt, Lm or Lo—those with Unicode general category Lu are *uppercase letters*—and the following *honorary letters*:

  U+221E INFINITY ∞ U+22A4 DOWN TACK ⊤ U+22A5 UP TACK ⊥

- *connecting punctuation* (Unicode general category Pc);
- *digits* (Unicode general category Nd);
- *prime characters*, which are:

  U+2032 PRIME U+2033 DOUBLE PRIME
  U+2034 TRIPLE PRIME U+2035 REVERSED PRIME
  U+2036 REVERSED DOUBLE PRIME U+2037 REVERSED TRIPLE PRIME

- *whitespace characters*, which are *spaces* (Unicode general category Zs) and the following characters:

  U+0009 CHARACTER TABULATION U+000A LINE FEED
  U+000B LINE TABULATION U+000C FORM FEED
  U+000D CARRIAGE RETURN
  U+001C INFORMATION SEPARATOR FOUR U+001D INFORMATION SEPARATOR THREE
  U+001E INFORMATION SEPARATOR TWO U+001F INFORMATION SEPARATOR ONE
  U+0020 SPACE
  U+2028 LINE SEPARATOR U+2029 PARAGRAPH SEPARATOR

> DRAFT NOTE
>
> Victor: All the characters listed above are control characters (Cc) except U+2028 and U+2029, which have general category Zl and Zp respectively and indeed are the only characters in those categories (in Unicode 5.0).

- *character literal delimiters*, which are:

  U+0060 GRAVE ACCENT `
  U+2018 LEFT SINGLE QUOTATION MARK ‘
  U+2019 RIGHT SINGLE QUOTATION MARK ’

- *string literal delimiters*, which are:

U+0022	QUOTATION MARK	"
U+201C	LEFT DOUBLE QUOTATION MARK	“
U+201D	RIGHT DOUBLE QUOTATION MARK	”

- *ordinary operator characters*, enumerated (along with the special operator characters) in Appendix F, which include the following characters with code points less than U+007F:

U+0021	EXCLAMATION MARK	!	U+0023	NUMBER SIGN	#
U+0024	DOLLAR SIGN	$	U+0025	PERCENT SIGN	%
U+002B	PLUS SIGN	+	U+002D	HYPHEN-MINUS	−
U+003F	QUESTION MARK	?	U+0040	COMMERCIAL AT	@
U+007E	TILDE	~			

  and most Unicode characters specified to be *mathematical operators* (i.e., characters with code points in the range 2200-22FF) are operators in Fortress, but there are some exceptions (e.g., $\forall$, $\exists$ and $:=$).

- *other characters*

Some other classes of characters, which overlap with the ones above, are useful to distinguish:

- *control characters* are those with Unicode general category Cc (i.e., characters with code points from U+0000 to U+001F, or from U+007F to U+009F);
- *ASCII characters* are those with code points U+007F and below;
- *protected characters* are the ASCII characters, control characters, and string literal delimiters;
- *word characters* are letters, digits, connecting punctuation, prime characters, and apostrophe;
- *restricted-word characters* are ASCII letters, ASCII digits, and the underscore character (i.e., ASCII word characters other than apostrophe);
- *hexadecimal digits* are the digits and the ASCII letters `A`, `B`, `C`, `D`, `E`, `F`, `a`, `b`, `c`, `d`, `e` and `f`;
- the *digit-group separator* is `NARROW NO-BREAK SPACE` (U+202F);
- *operator characters* are special operator characters and ordinary operator characters;
- *special characters* are special non-operator characters and special operator characters;
- *enclosing characters* are the enclosing operator characters enumerated in Section F.2, left and right parenthesis characters, and mathematical left and right white square brackets;

Forbidden and Restricted Characters

It is a static error for a Fortress program to contain any control character other than the above-listed whitespace characters, except that the character `SUBSTITUTE` (U+001A; also known as “control-Z”) is allowed and ignored if it is the last character of a program.

It is a static error for the following characters to occur outside a comment:

U+0009	CHARACTER TABULATION	U+000B	LINE TABULATION
U+001C	INFORMATION SEPARATOR FOUR	U+001D	INFORMATION SEPARATOR THREE
U+001E	INFORMATION SEPARATOR TWO	U+001F	INFORMATION SEPARATOR ONE

Thus, `LINE FEED`, `FORM FEED` and `CARRIAGE RETURN` are the only control characters—and the only whitespace characters other than spaces, `LINE SEPARATOR`, and `PARAGRAPH SEPARATOR`—outside of comments in a valid Fortress program.

It is a static error for any of the following characters to appear outside of comments and string and character literals:

- REVERSE SOLIDUS ('\';U+005C),
- any connecting punctuation other than LOW LINE ('_'; U+005F, also known as SPACING UNDERSCORE),
- any character in the "other characters" class above, or
- any of the following spaces:

U+2000	EN QUAD	U+2003	EM SPACE	U+2006	SIX-PER-EM SPACE
U+2001	EM QUAD	U+2004	THREE-PER-EM SPACE	U+2007	FIGURE SPACE
U+2002	EN SPACE	U+2005	FOUR-PER-EM SPACE	U+2008	PUNCTUATION SPACE

4.2 Words and Chunks

In a sequence of characters, a *chunk* is a nonempty contiguous subsequence. A *word* is a maximal chunk consisting of only word characters (letters, digits, underscore, connecting punctuation, prime characters, and apostrophe); that is, a word is one or more consecutive word characters delimited by characters other than word characters (or the beginning or end of the entire sequence). A *restricted word* is a maximal chunk that has only restricted-word characters (ASCII letters and digits and underscore characters). Note that a restricted word is *not* a word if it is delimited by word characters.

For example, in the sequence

$A'bc_123a.def\ \phi_{456}$

there are three words (A'bc_123, def and ϕ456), and four restricted words (A, bc_123, def and 456).

4.3 Lines, Pages and Position

The characters in a Fortress program are partitioned into *lines* and into *pages*, delimited by *line terminators* and *page terminators* respectively. A *page terminator* is an occurrence of the character FORM_FEED. A *line terminator* is an occurrence of any of the following:

- LINE FEED,
- CARRIAGE RETURN not immediately followed by LINE FEED,
- LINE SEPARATOR, or
- PARAGRAPH SEPARATOR.

A character is on page n (respectively line n) of a program if exactly $n-1$ page terminators (respectively line terminators) precede that character in the program. Thus, for example, the nth line terminator of a program is the last character on line n. A character is on line k of page n if it is on page n and is preceded by exactly $k-1$ line terminators on page n. A character is at line position k on line n if it is preceded by exactly $k-1$ characters on line n other than page terminators. Note that a page terminator does *not* terminate a line; thus, the character immediately following a page terminator need not be at line position 1.

Before any other processing, a Fortress program undergoes *ASCII conversion* (see Section 4.4), which may replace chunks of ASCII characters with single (typically non-ASCII) characters and remove some other characters. We expect that IDEs will typically display a program by rendering the converted sequence of characters rather than the actual input sequence. Thus, a program may appear to have fewer (and different) characters than it actually does. Nonetheless, the page, line and position of a character is based on the program before conversion.[3]

[3]Of course, an IDE may actually do the conversion and manipulate the converted program, in which case, the page, line and position of characters will reflect the conversion.

If a character (or any other syntactic entity) x precedes another character y in the program, we say that x is *to the left of* y and that y is *to the right of* x, regardless of how they may appear in a typical rendered display of the program. Thus, it is always meaningful to speak, for example, of the left-hand and right-hand operands of a binary operator, or the left-hand side of an assignment expression.

4.4 ASCII Conversion

To aid in program entry and facilitate interaction with legacy tools, Fortress provides an ASCII encoding for programs. To support this encoding, a Fortress program undergoes *ASCII conversion*, which takes a program and yields an equivalent program.[4] Unless otherwise specified, all constraints and properties of Fortress programs stipulated in this specification apply to the programs after ASCII conversion.

ASCII conversion consists of three steps. The first step consists of "pasting" words across line terminators, so that long identifiers and numerals can be split across lines. Identifiers may be very long because non-ASCII Unicode characters may be encoded by long chunks of ASCII characters (the actual conversion of such chunks is done in the next step). Roughly speaking, two consecutive lines are pasted together if the first ends with an ampersand that is immediately preceded by a word character, and the second begins with an ampersand that is immediately followed by a word character.

The second step replaces certain restricted words, and sequences of operator and special characters, with single Unicode characters. Roughly speaking, if a restricted word is either the official Unicode 5.0 name with underscores in place of spaces and hyphens, or a specified alternative name, of some character other than a protected character (i.e., an ASCII or control character, or a string literal delimiter), then the restricted word is replaced by that character. In some cases, even a fragment of a restricted word may be replaced by a single character (most commonly a Greek letter). Some multicharacter sequences of ASCII operator and special characters are also replaced by non-ASCII operator or special characters; we call such sequences *ASCII shorthand*. However, within string literals, neither restricted words nor ASCII shorthand is converted. Instead, we provide *escape sequences* to get non-ASCII characters within strings (see Section 4.10).

The third step replaces apostrophes that appear within what would otherwise be numerals with digit-group separators.

Precise descriptions of these steps are given in Appendix E, including the rules for replacing fragments of restricted words and the specification of alternative names and ASCII shorthand for non-operator characters. Alternative names and ASCII shorthand for operator characters are given in Appendix F.

> DRAFT NOTE
> Is the following true?

ASCII conversion (all three steps taken together) is idempotent: converting a program resulting from ASCII conversion yields the same program. The ampersand is particularly important in this process, and its use is severely restricted in the converted program to preserve the idempotence of ASCII conversion (see Section 4.7).

4.5 Input Elements and Scanning

After ASCII conversion, a Fortress program is partitioned into *input elements* by a process called *scanning*. [5] Scanning transforms a Fortress program from a sequence of Unicode characters to a sequence of input elements. Input elements

[4] See Appendix E for the precise notion of equivalence guaranteed by ASCII conversion.

[5] Fortress has a facility for defining new syntax, discussed in Chapter 27. However, except for that chapter, this specification describes the Fortress language only for programs that use the standard Fortress syntax without using this facility.

are always chunks: the characters that comprise an input element always appear contiguously in the program. Input elements are either *whitespace elements* (including *comments*) or *tokens*. A token is a *reserved word*, a *literal*, an *identifier*, an *operator token*, or a *special token*. There are five kinds of literals: boolean literals, character literals, string literals, the void literal, and numerals (i.e., numeric literals).

Conceptually, we can think of scanning as follows: First, the comments, character literals and string literals are identified. Then the remaining characters are partitioned into words (i.e., maximal chunks of letters, digits, connecting punctuation, underscore, primes, and apostrophes), whitespace characters, and other characters. In some cases, words separated by a single '.' or digit-group separator (and no other characters) are joined to form a single numeral (see Section 4.13). Words that are not so joined are classified as reserved words, boolean literals, numerals, identifiers, or operator tokens, as described in later sections in this chapter. It is a static error if any word in a Fortress program is not part of one of the input elements described above. All remaining whitespace characters, together with the comments, form whitespace elements, which may be *line-breaking*. Finally, chunks of symbols (and a few other special cases) are checked to see whether they form void literals (see Section 4.12) or multicharacter operator tokens (see Section 4.14). Every other character is a token by itself, either a special token (if it is a special character) or an operator token.

4.6 Comments

> DRAFT NOTE
> Describe to-the-end-of-line comments. (06/23/08)

When not within string literals, occurrences of "(∗" and "∗)" are *opening comment delimiters* and *closing comment delimiters*, respectively. In a valid program, every opening comment delimiter is balanced by a closing comment delimiter; it is a static error if comment delimiters are not properly balanced. All the characters between a balanced pair of comment delimiters, including the comment delimiters themselves, comprise a *comment*. Comments may be nested. For example, the following illustrates three comments, one of which is nested:

```
(* This is a comment. *) (* As is this (* nested *)
comment *)
```

4.7 Whitespace Elements

A whitespace element is a maximal chunk consisting of comments, whitespace characters that are not within string or character literals or numerals, and ampersands (U+0026) that are not within string or character literals.

We distinguish *line-breaking whitespace* from *non-line-breaking whitespace* using the following terminology:

- A *line-terminating comment* is a comment that encloses one or more line terminators. All other comments are called *spacing comments*.
- *Spacing* refers to any chunk of spaces, `FORM FEED` characters, whitespace characters other than line terminators, and spacing comments.
- A *line break* is a line terminator or nonnested line-terminating comment that is not immediately preceded by an ampersand, possibly with intervening spacing.
- *Whitespace* refers to any nonempty sequence of spacing, ampersands, line terminators, and line-terminating comments.
- *Line-breaking whitespace* is whitespace that contains at least one line break.

It is a static error if an ampersand occurs in a program (after ASCII conversion) unless it is within a character or string literal or a comment, or it is immediately followed by a line terminator or line-terminating comment (possibly with intervening spacing).

4.8 Reserved Words

The following tokens are *reserved words*:

`BIG`	`FORALL`	`SI_unit`	`absorbs`	`abstract`	`also`	`api`
`asif`	`at`	`atomic`	`bool`	`case`	`catch`	`coerce`
`coerces`	`component`	`comprises`	`default`	`dim`	`do`	`elif`
`else`	`end`	`ensures`	`except`	`excludes`	`exit`	`export`
`extends`	`finally`	`fn`	`for`	`forbid`	`from`	`getter`
`hidden`	`if`	`import`	`int`	`invariant`	`io`	`juxtaposition`
`label`	`most`	`nat`	`native`	`object`	`of`	`opr`
`or`	`outcome`	`override`	`private`	`property`	`provided`	`requires`
`self`	`settable`	`setter`	`spawn`	`syntax`	`test`	`then`
`throw`	`throws`	`trait`	`try`	`tryatomic`	`type`	`typecase`
`typed`	`unit`	`value`	`var`	`where`	`while`	`widens`
`with`	`wrapped`					

> DRAFT NOTE
>
> Victor: I don't think "or" should be reserved. It is only so for its occurrence in "widens or coerces" but we can recognize it specially in that context, which is never ambiguous because "widens" and "coerces" are reserved.

The following operators on units are also reserved words:

cubed cubic in inverse per square squared

To avoid confusion, Fortress reserves the following tokens:

`goto` `idiom` `public` `pure` `reciprocal` `static`

They do not have any special meanings but they cannot be used as identifiers.

> DRAFT NOTE
>
> Victor: Some other words we might want to reserve: subtype, subtypes, is, coercion, function, exception, match

4.9 Character Literals

A character literal consists of one or more characters enclosed in single quotation marks. The character that begins a character literal is called the literal's *opening mark*; the character that ends it is its *closing mark*. For convenience, the marks may be true typographical "curly" single quotation marks (U+2018 and U+2019), a pair of apostrophe characters (U+0027), or a "backquote" character (U+0060) and an apostrophe character. It is a static error if the opening and closing marks do not match: a character literal beginning with a left single quotation mark must end with a right single quotation mark; one beginning with either an apostrophe or a backquote must end with an apostrophe. As discussed in Section 13.1, a character literal evaluates to a value of type Char, which represents an abstract Unicode character.

A left single quotation mark (U+2018) or a backquote begins a character literal unless it is within a comment, another character literal, or a string literal.

> DRAFT NOTE
> What if it is preceded by a backslash? (should be illegal?)

An apostrophe (U+2018) begins a character literal unless it is within a comment, another character literal, or a string literal, or it is immediately preceded by a word character (i.e., a letter, digit, connecting punctuation, underline, prime, or apostrophe).

> DRAFT NOTE
> What if it is preceded by a backslash? (should be illegal)

In either case, the character literal ends with the nearest apostrophe or right single quotation mark *after* the first character following the opening mark. In particular, an apostrophe or right single quotation mark immediately following the opening mark of a character literal is *not* a closing mark. Thus, for example, the sequence `'''` is a single character literal with one enclosed character `'`.

It is a static error if any of the enclosed characters of a character literal is a line feed, form feed, carriage return, line separator, paragraph separator, or any character forbidden outside comments in a Fortress program (see Section 4.1), or if there is exactly one enclosed character and it is a backslash (U+005C). It is also a static error if any of the enclosed characters is a string literal delimiter that is not immediately preceded by a backslash (i.e., an *unescaped* string literal delimiter). This last restriction is necessary to prevent ASCII conversion from changing the boundaries of string literals (see Appendix E).

The sequence of enclosed characters may be a single character (e.g., 'a', `'$'`, 'α', '$\oplus$'), a sequence of four or more hexadecimal digits identifying the code point of a Unicode character (e.g., '`001C`', '`FBAB`', '`1D11E`'), one of `TAB`, `NEWLINE` or `RETURN`, a sequence of ASCII characters that would be converted by the second step of ASCII conversion to a single Unicode character (see Section E.2), the official Unicode 5.0 name or an alternative name of a Unicode character with spaces and hyphens intact (e.g., '`PLUS-MINUS SIGN`'), or a *character-literal escape sequence*. The character-literal escape sequences, and the characters such character literals evaluate to, are:

`\b`	U+0008	`BACKSPACE`
`\t`	U+0009	`CHARACTER TABULATION`
`\n`	U+000A	`LINE FEED`
`\f`	U+000C	`FORM FEED`
`\r`	U+000D	`CARRIAGE RETURN`
`\"`	U+0022	`QUOTATION MARK`
`\\`	U+005C	`REVERSE SOLIDUS`
`\“`	U+201C	`LEFT DOUBLE QUOTATION MARK`
`\”`	U+201D	`RIGHT DOUBLE QUOTATION MARK`

> DRAFT NOTE
> Victor: What about 'BACKQUOTE'?

It is a static error if the sequence of enclosed characters is not one of the kinds listed above. In particular, it is a static error if the hexadecimal digits enclosed do not correspond to the code point of a Unicode 5.0 abstract character.

Note that ASCII conversion is performed within character literals. Thus a character literal written as

```
'GREEK_CAPITAL_LETTER_LAMBDA'
```

is equivalent to a character literal written as

```
'Λ'
```

Although names for control and other protected characters are *not* converted during ASCII conversion, they *are* permitted within character literals. Both the standard form of such names (i.e., with spaces and hyphens) and the form with spaces and hyphens replaced by underscore characters are permitted. Thus, for example, the literals `'BACKSPACE'`, `'\b'` and `'0008'` all evaluate to the `BACKSPACE` character.

4.10 String Literals

A string literal is a sequence of characters enclosed in double quotation marks (for example, `"Hello, world!"` or "$\pi\, r^2$"). The character that begins a string literal is the literal's *opening mark*; the character that ends it is its *closing mark*. For convenience, the opening and closing marks of a string literal may be either true typographical "curly" double quotation marks (U+201C and U+201D) or a pair of "neutral" double-quote characters. It is a static error if the opening and closing marks of a string literal do not "match", that is, if one is "curly" and the other "neutral". As discussed in Section 13.1, a string literal evaluates to a value of type String, which represents a finite sequence of abstract Unicode characters.

A left double quotation mark (U+201C) or "neutral" quotation mark (U+0022) begins a string literal unless it is within a comment, a character literal, or another string literal.

> DRAFT NOTE
> What if it is preceded by a backslash? (should be illegal)

It ends with the nearest following unescaped (i.e., not immediately preceded by an unescaped backslash) right double quotation mark or "neutral" quotation mark. Therefore, it is not possible for a string literal to include an unescaped right double quotation mark or "neutral" quotation mark as an enclosed character. In addition, it is a static error for an unescaped left double quotation mark to be an enclosed character of a string literal.

Within a string literal, a backslash introduces an *escape sequence*, unless it is immediately preceded by an odd number of backslashes, in which case the backslash is itself escaped. There are three kinds of escape sequences recognized within a string literal: the character-literal escape sequences (see Section 4.9), *restricted-word escape sequences*, and *quoted-character escape sequences*.

A restricted-word escape sequence consists of an unescaped backslash immediately followed by a restricted word not beginning with a lowercase letter. It is a static error if enclosing the restricted word in single quotes does not result in a valid character literal.

A quoted-character escape sequence consists of an unescaped backslash immediately followed by an apostrophe and the sequence of characters immediately following the apostrophe up to and including the next apostrophe.

> DRAFT NOTE
> Victor: Should we allow any character literal (e.g., one delimited by "curly" quotes)?

Note that this kind of escape sequence looks just like a backslash followed by a character literal. It is a static error if such a sequence with the initial backslash removed would not be a valid character literal (see Section 4.9). Quoted-character escape sequences are useful for ASCII "shorthands" that begin with a lowercase letter or are not restricted words, and when the escape sequence is immediately followed in the string by a letter, digit, or underscore. For example, `"\beta"` evaluates to a string containing a backspace character (indicated by `\b`) followed by the letters `e`, `t`, and `a`, but `"\'beta'"` evaluates to a string containing the single letter β. As another example, `"x\AND\'NOT'y"` becomes `"x∧¬y"`, but `"x\AND\NOTy"` is a static error, because the name "`NOTy`" does not correspond to a Unicode character. Also, the string `"Foo\'[\'T\']'"` becomes `"Foo⟦T⟧"`.

It is a static error if any of the enclosed characters of a string literal is a line feed, form feed, carriage return, line separator, paragraph separator, or any character forbidden outside comments in a Fortress program (see Section 4.1).

It is also a static error if an unescaped backslash is immediately followed by a character other than another backslash or, an apostrophe, a string literal delimiter, or a restricted-word character. In addition, it is also a static error if an unescaped backslash within a string literal is followed by any lowercase letter other than ‘b’, ‘t’, ‘n’, ‘f’ or ‘r’. This rule preserves a certain level of compatibility with the C and Java programming languages.

Unlike elsewhere in a program, the enclosed characters of a string literal are *not* subject to the second or third step of ASCII conversion (word pasting across line terminators still occurs within string literals). Also, the formatting of identifiers and numerals described in Section 4.17 is not performed within string literals.

4.11 Boolean Literals

The boolean literals are $false$ and $true$.

4.12 The Void Literal

The void literal is $()$ (pronounced “void”).

4.13 Numerals

A numeric literal, or *numeral*, in Fortress is a maximal chunk consisting one or more words (and the intervening characters) satisfying the following properties:

- each word consists of only digits and letters, except that the last word may have one underscore as part of a *radix specifier*, defined below;
- consecutive words are separated by exactly one character, either a digit-group separator or a ‘. ’ character; and
- either the first word begins with a digit. or the last word has a radix specifier.

A *radix specifier* is a word suffix consisting of an underscore and then either by a sequence of one or more digits or by the English name in all uppercase ASCII letters of an integer from 2 to 16. The *radix* of a numeral with a radix specifier is the number denoted by the sequence of digits (interpreted in base 10) or named by the ASCII letters of the radix specifier. It is a static error if the radix of a numeral is not an integer from 2 to 16. It is also a static error if any of the following conditions hold:

- the numeral contains letters and does not have a radix specifier;
- the numeral contains letters other than `A`, `B`, `C`, `D`, `E`, `F`, `a`, `b`, `c`, `d`, `e` or `f`, and has a radix other than 12;
- the numeral has radix 12 and contains letters other than `A`, `B`, `X`, `E` `a`, `b`, `x` or `e`, or contains at least one `A`, `B`, `a` or `b` and at least one `X`, `E`, `x` or `e`;
- the numeral has a radix specifier and contains a digit or letter that denotes a value greater than or equal to the numeral’s radix;[6] or
- the numeral contains both uppercase and lowercase letters.

[6]The value denoted by a digit is specified by the Unicode Standard [29]. The value denoted by the letters are as follows: `A` or `a`: 10; `B` or `b`: 11; `C` or `c`: 12; `D` or `d`: 13; `E` or `e`: 11 if the radix is 12, 14 otherwise; `F` or `f`: 15; `X` or `x`: 10.

A numeral is *simple* if it does not contain a '.' character; otherwise, the number is *compound*. It is a static error if a numeral contains more than one '.' character.

Here are some examples of valid numerals, shown in unformatted form:

```
17   7fff_16   0fff_SIXTEEN   10101101_2   XE_12   3.14159265   DEAD.BEEF_16
```

On the other hand, the following would be classified as numerals but then rejected as static errors:

```
0.a  0x6A35    FF_EIGHT       12.52.23     57_50   PI_FIFTEEN   dead.BEEF_16
```

> DRAFT NOTE
> Victor: Are FF_EIGHT and PI_FIFTEEN numerals or operator tokens?

4.14 Operator Tokens

In this section, we describe how to determine the operator tokens of a program. Because operator tokens do not occur within comments, string literals and character literals, we henceforth in this section consider only characters that are outside these constructs.

An *operator word* of a program is a word that is not reserved, consists only of uppercase letters and underscores (no digits or non-uppercase letters), does not begin or end with an underscore, and has at least two different letters.

A *base operator* is an ordinary operator character (described Section 4.1), an operator word, a (contiguous) sequence of two or more vertical-line characters (U+007C), or a multicharacter enclosing operator, as defined in Section 4.14.1. A base operator is *maximal* in a program if it is not contained within any other base operator of that program. It is a static error if two maximal base operators overlap (which is only possible if both are multicharacter enclosing operators). A *simple operator* is a maximal base operator that is an operator character or an operator word.

A maximal base operator is an operator token unless it is a simple operator other than an enclosing or vertical-line operator character, and it is immediately preceded by '^' or immediately followed by '='. Such an operator is an *enclosing operator* if it is an enclosing operator character (see Section F.2) or a multicharacter enclosing operator; it is a *vertical-line operator* if it has only vertical-line operator characters (see Section F.3); otherwise, it is an *ordinary operator*.

If a simple operator is immediately preceded by '^' then the '^' and the simple operator together comprise a single operator token; such an operator token is called a *superscripted postfix operator*. In addition, "$\hat{}T$" is also a superscripted postfix operator provided that it is not immediately followed by a word character. It is a static error for a superscripted postfix operator to be immediately followed by a word character other than apostrophe.

Finally, if a simple operator is not immediately preceded by '^' and is immediately followed by '=' then the simple operator and the '=' together comprise an operator token; such an operator token is called a *compound assignment operator*.

> DRAFT NOTE
> Can't be simple assignment because COLON is special.

4.14.1 Multicharacter Enclosing Operators

The following multicharacter sequences (in which there must be no other characters, and particularly no whitespace) can be used as paired enclosing tokens as described below:

1. Any chunk of vertical-line characters is a vertical-line operator. Such an operator can be used in an enclosing pair matching itself.

2. A '(' may be immediately followed by a '.' and then one or more '/' characters or one or more '\' characters. Such a token is a left encloser, and it is matched by the multicharacter token consisting of the same number and kind of '/' or '\' characters followed immediately by a '.' and then a ')'; Thus, for example, "(.////" and "////.)" are matching left and right enclosers, respectively. (In the future, we may allow tokens with mixtures of '/' and '\', in which case the character sequences must match from the outside in. But for now, such tokens are simply illegal.)

3. Either '[' or '{' may be immediately followed by one or more '/' characters or by one or more '\' characters. Such a token is a left encloser, and it is matched by the multicharacter token consisting of the same number and kind of '/' or '\' characters followed immediately by a matching ']' or '}', as appropriate. Thus, for example, "[////" and "////]" are matching left and right enclosers respectively. (In the future, we may allow tokens with mixtures of '/' and '\', in which case the character sequences must match from outside in. But for now, such tokens are simply illegal.)

   Exceptions to this rule are the multicharacter sequences [\ and \], which are ASCII shorthand for the special non-operator characters ⟦ and ⟧; these are used to enclose static type parameters and arguments but are not considered *operators*.

4. One or more '<' characters may be followed immediately by one or more '|' characters. Such a token is a left encloser, and it is matched by the multicharacter token consisting of the same number of '|' characters followed by as many '>' characters as there are '<' characters in the left encloser.

5. One or more '<' characters, or one or more '|' characters, may be followed immediately by one or more '/' characters or by one or more '\' characters. Such a token is a left encloser, and it is matched by the multicharacter token consisting of the same number and *opposite* kind of '/' or '\' characters followed immediately by as many matching '>' or '|' characters as appropriate. Thus, for example, "<<//" matches "\\>>', and "|\\\" matches "///|". (In the future, we may allow tokens with mixtures of '/' and '\', in which case the character sequences must "opposite-match" from outside in. But for now, such tokens are simply illegal.)

6. Finally, any number of '⋆' (U+002A) or '.' (U+002E) characters may be placed within any of the above multicharacter sequences, except those that contain '(' or ')', as long as no '⋆' or '.' is the first or last character in the sequence, and no '⋆' or '.' character is adjacent to another '⋆' or '.' character. (Thus [⋆⋆/ and [../ and [.⋆/ are all improper.) The rule for matching is as above, except that in addition, the positions of the '⋆' and the '.' characters must match from the outside in. (For example, [⋆/./ matches /./⋆], and <⋆/./ matches \.\⋆>.)

   Note that some of these multicharacter sequences are considered to be ASCII shorthand for certain single Unicode characters. Note also that four other multicharacter sequences not covered by the rules given here (the matched pair (.< and >.), and the matched pair ((.> and <.))) are also considered to be ASCII shorthand for certain single Unicode characters. See Section F.2.

Note that some of the character sequences described above cannot occur in programs after ASCII conversion. For example, "||" is converted to '‖' (U+2016). Note also that [\ \] (which are converted to ⟦ ⟧) are not operators; they play a special role in the syntax of Fortress, and their behavior cannot be redefined by a library.

4.14.2 Special Operators

Note that in the preceding discussion, a single operator character can be an operator token only if it is an ordinary operator character. In some cases, some of the special operator characters (and even some special non-operator characters) form part of an operator token. However, most of the special operator characters cannot be determined to be operators before parsing the program because they are also used for various parts of Fortress syntax. The one exception is '^':

if an occurrence of this character is not part of a superscripted postfix operator, then it is an operator token by itself: the special *superscripting operator*. This operator is always an infix operator, and it is a static error if it appears in a context in which a prefix or postfix operator is expected.

Every other special operator character, when not part of an operator token, is a special token that may be used as an operator. There is also a special `juxtaposition` operator (described in Section 16.8), but this operator is a reserved word rather than an operator token. Occurrences of this operator are determined by the Fortress grammar. The reserved words `in`, cubed, cubic, in, inverse, per, square, and squared, are used as operators on units.

4.15 Identifiers

A word is an identifier if it begins with a letter or an underscore and is not a reserved word, an operator, or all or part of a numeral.

4.16 Special Tokens

Every special character (operator or non-operator) that is not part of a token (or within a comment) as described above is a *special token* by itself. The special operator characters may be operators in the appropriate context.

4.17 Rendering of Fortress Programs

In order to more closely approximate mathematical notation, Fortress mandates standard rendering for various input elements, particularly for numerals and identifiers, as specified in this section. In the remainder of this specification, programs are presented formatted unless stated otherwise.

4.17.1 Fonts

Throughout this section, we refer to different fonts or styles in which certain characters are rendered, with names suggestive of their appearance.

- roman
- italic
- math (often identical to italic)
- script
- fraktur
- sans-serif
- italic sans-serif
- monospace
- double-struck

Additionally, the following fonts may be specified to be bold: roman, italic, script, fraktur, sans-serif, italic sans-serif.

However, a particular environment may substitute different fonts either because of local practice or because the desired fonts are not available.

4.17.2 Numerals

A numeral is rendered in roman type, with the radix, if present, as a subscript.

`27`	*is rendered as*	27
`10101101_TWO`	*is rendered as*	10101101_{TWO}
`3.143159265`	*is rendered as*	3.143159265
`3.11037552_8`	*is rendered as*	3.11037552_{8}
`11.0010010000111111011010_2`	*is rendered as*	$11.0010010000111111011010_{2}$

Note: the elegant way to write Avogadro's number is `6.02 TIMES 10^23`, which is not a single token but is a constant expression; its rendered form is 6.02×10^{23}.

4.17.3 Identifiers

Fortress has rather complicated rules for rendering an identifier; as in other parts of Fortress, the rules are complicated so that the simple cases will be very simple, but also so that difficult cases of interest will be possible.

It is conventional in mathematical notation to make use of variables, particularly single-letter variables, in a number of different fonts or styles, with italic being the most common, then boldface, and roman: a, $\mathbf{b}$, c. Frequently such variables are also decorated with accents and subscripts: $\bar{p}$, q', $\hat{r}$, $T_{\max}$, $\vec{\mathbf{u}}$, $\mathbf{v}_{\mathrm{x}}$, w_{17}, $\bar{z}'_{17}$. Fortress provides conventions for typing such variables using plain ASCII characters: for example, the unformatted presentations of these same variables are `a`, `_b`, `c_`, `p_bar`, `q'` or `q_prime`, `r_hat`, `T_max`, `_u_vec`, `_v_x`, `w17`, and `z17_bar'`. The rules are also intended to accommodate the typical use of multicharacter variable names for computer programming, such as $count$, $isUpperCase$, and $\mathrm{Boolean}$.

The most important rules of thumb are that simple variables are usually italic z (`z`), a leading underscore usually means boldface font $\mathbf{z}$ (`_z`), a trailing underscore usually means roman font z (`z_`), and a doubled capital letter means double-struck (or "blackboard bold") font $\mathbb{Z}$ (`ZZ`). However, mixed-case variable names that begin with a capital letter, which are usually used as names of types, are rendered in roman font even if there is no trailing underscore.

The detailed rules are described in Section D.1.

4.17.4 Other Rendering Rules

Reserved words are rendered in monospace, except that the reserved words that are used as operators on units, namely cubed, cubic, in, $\mathrm{inverse}$, per, square, and $\mathrm{squared}$, are rendered in roman type. Operator words are rendered in monospace. Comments are rendered in roman font. Any character within a character literal or string literal is rendered in monospace if possible.

The detailed rules for rendering other constructions are described in Section D.2.

Chapter 5

Evaluation

The state of an executing Fortress program consists of a set of *threads*, and a *memory*. Communication with the outside world is accomplished through *input and output actions*. In this chapter, we give a general overview of the execution process and introduce terminology used throughout this specification to describe the behavior of Fortress constructs.

Executing a Fortress program consists of *evaluating* the body expression of the run function and the initial-value expressions of all top-level variables and singleton object fields in parallel. All initializations must complete normally before any reference to the objects or variables being initialized.

Threads evaluate expressions by taking *steps*. We say evaluation of an expression *begins* when the first step is taken. A step may *complete* the evaluation, in which case no more steps are possible on that expression, or it may result in an *intermediate expression*, which requires further evaluation. *Dynamic program order* is a partial order among the expressions evaluated in a particular execution of a program. When two expressions are ordered by dynamic program order, the first must complete execution before any step can be taken in the second. Chapter 13 gives the dynamic program order constraints for each expression in the language.

Intermediate expressions are generalizations of ordinary Fortress expressions: some intermediate expressions cannot be written in programs. We say that one expression is *dynamically contained* within a second expression if all steps taken in evaluating the first expression must be taken between the beginning and completion of the second. A step may also have effects on the program state beyond the thread taking the step, for example, by modifying one or more locations in memory, creating new threads to evaluate other expressions, or performing an input or output action. Threads are discussed further in Section 5.4. The memory consists of a set of *locations*, which can be *read* and *written*. New locations may also be *allocated*. The memory is discussed further in Section 5.3. Finally, *input actions* and *output actions* are described in Section 5.6.

5.1 Values

> DRAFT NOTE
>
> LinearSequence and HeapSequence are not yet supported.

> DRAFT NOTE
>
> Comments: 1. Operators are bound in an environment just like functions. Methods are not values. They are bound in a type.
>
> 2. I'm distinguishing fields from captured variables, even though there is no semantic distinction, except that

captured variables could be shared (but they might not be, and if they are immutable, we can't tell anyway).

DRAFT NOTE

Jan: perform promised fixes to definition and usage of environment. In particular, environments permit sharing of mutable fields.

A *value* is the result of normal completion of the evaluation of an expression. (See Section 5.2 for a discussion of completion of evaluation.) A value is an object, a tuple or the void value $()$. Every value has a *type*, and every object has an environment (see Section 5.5). See Chapter 6 for a description of the types corresponding to these different values. An object may be a *value object*, a *reference object*, or a *function*. A nonfunction object has a finite set of *fields*; the names and types of these fields are specified by the type of the object. The type of a nonfunction object also specifies its methods.

A field consists of a name, a type, and either a value or a location: in value objects, fields have values; in reference objects, locations. The name of a field is either an identifier or an index. Only values of type LinearSequence (defined in Section 43.1) or HeapSequence (defined in Section 43.3) have fields named by indices. Every field in a value object is *immutable*. Reference objects may have both *mutable* and *immutable* fields. No two distinct values share any mutable field.

Values are constructed:

1. by top-level function declarations (see Section 9.1) and singleton declarations (see Section 11.1), and
2. by evaluating an object expression (see Section 13.9), a function expression (see Section 13.8), a local function declaration (see Section 9.5), a call to an object constructor (declared by a constructor declaration; see Chapter 11), a literal (see Section 13.1), a `spawn` expression (see Section 13.24), a tuple expression (see Section 13.28), an aggregate expression (see Section 13.29), or a comprehension (see Section 13.30).

In the latter case, the constructed value is the result of the normal completion of such an evaluation.

5.2 Normal and Abrupt Completion of Evaluation

DRAFT NOTE

Victor:

What happens when a thread completes abruptly? What happens if the program completes abruptly?

Conceptually, an expression is evaluated until it *completes*. Evaluation of an expression may *complete normally*, resulting in a value, or it may *complete abruptly*. Each abrupt completion has an associated value, either an exception value that is thrown and uncaught or the `with` clause value of an `exit` expression (described in Section 13.12). In addition to programmer-defined exceptions thrown explicitly by a `throw` expression (described in Section 13.25), there are predefined exceptions thrown by the Fortress standard libraries. For example, dividing an integer by zero (using the $/$ operator) causes a DivisionByZero to be thrown.

When an expression completes abruptly, control passes to the dynamically immediately enclosing expression. This continues until the abrupt completion is handled either by a `try` expression (described in Section 13.26) if an exception is being thrown or by an appropriately tagged `label` expression (described in Section 13.12) if an `exit` expression was evaluated. If abrupt completion is not handled within a thread and its outermost expression completes abruptly, the thread itself completes abruptly. If the main thread of a program completes abruptly, the program as a whole also completes abruptly.

> DRAFT NOTE
>
> Jan: abrupt completion only causes the thread to complete abruptly. A program completes abruptly if its initial thread completes abruptly. Pointer to discussion of abrupt completion of threads, or pull that discussion here.

5.3 Memory and Memory Operations

> DRAFT NOTE
>
> Check for *initializing write* is not yet supported.

In this specification, the term *memory* refers to a set of abstract *locations*; the memory is used to model sharing and mutation. A location has an associated type, and *contains* a value of that type (i.e., the type of the value is a subtype of the type of the location). Locations can have non-object trait types; by contrast, a value always has an object type.

There are three operations that can be performed on memory: *allocation*, *read*, and *write*. Reads and writes are collectively called *memory accesses*. Intuitively, a read operation takes a location and returns the value contained in that location, and a write operation takes a location and a value of the location's type and changes the contents of the location so the location contains the given value. Accesses need not take place in the order in which they occur in the program text; a detailed account of memory behavior appears in Chapter 21.

Allocation creates a new location of a given type. Allocation occurs when a mutable variable is declared, or when a reference object is constructed. In the latter case, a new location is allocated for each field of the object. Abstractly, locations are never reclaimed; in practice, memory reclamation is handled by garbage collection.

A freshly allocated location is *uninitialized*. The type system and memory model in Fortress guarantee that an *initializing write* is performed to every location if it is ever read, and that this write occurs before any read of the location. Any location whose value can be written after initialization is *mutable*. Mutable locations consist of mutable variables and mutable fields of reference objects. Any location whose value cannot be written after initialization is *immutable*. Immutable locations are the non-`settable` immutable fields of a reference object.

5.4 Threads and Parallelism

> DRAFT NOTE
>
> Reduction variables and deferred exceptions are not yet supported.

> DRAFT NOTE
>
> Thread fairness guarantee (Jan's email titled "Re: Clarifying fairness guarantees" sent on 14 Mar 2007)

> DRAFT NOTE
>
> Fix citations:
>
> Victor's email titled "Re: Fortress 1.0 spec (revision 4793)" sent on 31 Mar 2008

There are two kinds of threads in Fortress: *implicit* threads and *spawned* (or *explicit*) threads. Spawned threads are objects created by the `spawn` construct, described in Section 13.24.

A thread may be in one of five states: *not started*, *executing*, *suspended*, *normally completed* or *abruptly completed*. We say a thread is *not started* after it is created but before it has taken a step. This is only important for purposes of

determining whether two threads are executing simultaneously; see Section 23.3. A thread is *executing* or *suspended* if it has taken a step, but has not completed; see below for the distinction between executing and suspended threads. A thread is *complete* if its expression has completed evaluation. If the expression completes normally, its value is the result of the thread.

Every thread has a *body* and an *execution environment*. The body is an intermediate expression, which the thread evaluates in the context of the execution environment; both the body and the execution environment may change when the thread takes a step. This environment is used to look up names in scope but bound in a block that encloses the construct that created the thread. The execution environment of a newly created thread is the environment of the thread that created the new thread.

In Fortress, a number of constructs are *implicitly parallel*. An implicitly parallel construct creates a *group* of one or more implicit threads. The implicitly parallel constructs are:

- **Tuple expressions:** Each element expression of the tuple expression is evaluated in a separate implicit thread (see Section 13.28).
- `also do` **blocks:** each sub-block separated by an `also` clause is evaluated in a separate implicit thread (see Section 13.11.2).
- **Method invocations and function calls:** The receiver or the function and each of the arguments is evaluated in a separate implicit thread (see Section 13.4, Section 13.5, and Section 13.6).
- `for` **loops, comprehensions, sums, generated expressions, and big operators:** Parallelism in looping constructs is specified by the generators used (see Section 13.14). Programmers must assume that generators that do not extend $\mathrm{SequentialGenerator}$ can execute each iteration of the body expression in a separate implicit thread. The functional method seq and the equivalent function $sequential$ can be used to turn any $\mathrm{Generator}$ into a $\mathrm{SequentialGenerator}$.
- **Extremum expressions:** Each guarding expression of the extremum expression is evaluated in a separate implicit thread (see Section 13.21).
- **Tests:** Each test is evaluated in a separate implicit thread (see Chapter 19).

Implicit threads run fork-join style: all threads in a group are created together, and they all must complete before the group as a whole completes. There is no way for a programmer to single out an implicit thread and operate upon it in any way; they are managed purely by the compiler, runtime, and libraries of the Fortress implementation. Implicit threads need not be scheduled fairly; indeed, a Fortress implementation may choose to serialize portions of any group of implicit threads, interleaving their operations in any way it likes. For example, the following code fragment may loop forever:

> DRAFT NOTE
> The TYPThread example is commented out.

If any implicit thread in a group completes abruptly, the group as a whole will complete abruptly as well. Every thread in the group will either not run at all, complete (normally or abruptly), or may be terminated early as described in Section 23.7. The result of the abrupt completion of the group is the result of one of the constituent threads that completes abruptly. After abrupt completion of a group of implicit threads, each reduction variable (see Section 5.4.1) may reflect an arbitrary subset of updates performed by the threads in the group. This means in general that reduction variables should not be accessed after abrupt completion. The exact behavior of reduction variables depends on the supporting generator structure and is described in Section 23.8.

Spawned thread objects are reference objects of type $\mathrm{Thread}[\![T]\!]$, where T is the static type of the expression spawned; this trait has the following methods, each taking no arguments:

- The val method returns the value computed by the subexpression of the `spawn` expression. If the thread has not yet completed execution, the invocation of val blocks until it has done so. A spawned thread is not permitted

to exit (discussed in Section 13.12) to a surrounding label (the label is considered to be out of scope), but may fail to catch an exception and thus complete abruptly. When this happens, the exception is *deferred*. Any invocation of the val method on the spawned thread throws the deferred exception. If the spawned thread object is discarded, the exception will be silently ignored. In particular, if the result of a `spawn` expression is dropped, it is impossible to detect completion of the spawned thread or to recover from any deferred exceptions.

- The $wait$ method waits for a thread to complete, but does not return a value and will not throw a deferred exception.
- The $ready$ method returns $true$ if a thread has completed, and returns $false$ otherwise.
- The $stop$ method attempts to terminate a thread as described in Section 23.7.

We say a spawned thread has been *observed to complete* after invoking the val or $wait$ methods, or when an invocation of the $ready$ method returns $true$.

In the absence of sufficient parallel resources, an attempt is made to run the subexpression of the `spawn` expression before continuing succeeding evaluation. We can imagine that it is actually the *rest* of the evaluation *after* the parallel block which is spawned off in parallel. This is a subtle technical point, but makes the sequential execution of parallel code simpler to understand, and avoids subtle problems with the asymptotic stack usage of parallel code [22, 11].

There are three ways in which a thread can be suspended. First, a thread that begins evaluating an implicitly parallel construct, creating a thread group, is suspended until that thread group has completed. Second, a thread that invokes val or $wait$ is suspended until the spawned thread completes. Finally, invoking the $abort$ function from within an `atomic` expression may cause a thread to suspend; see Section 23.5.

Threads in a Fortress program can perform operations simultaneously on shared objects. In order to synchronize data accesses, Fortress provides `atomic` expressions (see Section 13.23). Chapter 21 describes the memory model which is obeyed by Fortress programs.

5.4.1 Reduction Variables

> DRAFT NOTE
> Reduction variables are not yet supported.

To perform computations as locally as possible, and avoid the need to synchronize in the middle of relatively simple `for` loops, Fortress gives special treatment to *reductions*. We say that an operator $\odot$ is a *reduction operator* for type T if T is a subtype of $\mathrm{Monoid}[\![T, \odot]\!]$, which implies that $\odot$ is an associative binary infix operator on T (see Section 40.5 for details about the Monoid trait).

We say that a variable l is a *reduction variable* reduced using the reduction operator $\odot$ for a particular thread group if it satisfies the following conditions:

- Every assignment to l within the thread group is of the form $l \oplus\!= e$, where exactly one operator $\oplus$ or its group inverse $\ominus$ (see below) is used in these assignments.
- The value of l is not otherwise read within the thread group.
- The variable l is not a free variable of a functional. This includes the fields of the receiver object in a method definition.

Other threads which simultaneously reference a reduction variable while a loop is running may see an arbitrary value for that variable. Any updates performed by those threads may be lost. The association of terms in the reduction is arbitrary and guided by the loop generators (Section 23.8). The order of terms in the reduction is the natural order of elements in the generator. When the operator is commutative, the order of terms is also arbitrary.

Several common mathematical operators are declared to be monoids in the Fortress standard libraries. These include $+$, $\cdot$, $\wedge$, $\vee$, and $\veebar$. If a type T extends $\mathrm{Group}[\![T, \oplus, \ominus]\!]$ (see Section 40.5 for details about the Group trait) then reduction expressions of the following form are also permitted:

$$x \ominus= y$$

Such expressions may be transformed into an algebraic equivalent such as the following:

$$x \oplus= \mathrm{Identity}[\![\oplus]\!] \ominus y$$

The semantics of reductions enables implementation strategies such as the one used in OpenMP [26]: A reduction variable l is assigned $\mathrm{Identity}[\![\oplus]\!]$ at the beginning of each iteration. The original variable value may be read ahead of time, resulting in the loss of parallel updates to the variable which occur in other threads while the loop is running. When all iterations are complete, the initial value of the reduction variable and values of the variable at the end of each implicit thread are reduced and the result is assigned to the reduction variable as the group completes.

In the following example, sum is a reduction variable:

> DRAFT NOTE
> I manually rendered the array type.

```
arraySum[[nat x]](a : ℤ64[x]) : ℤ64 = do
    sum : ℤ64 := 0
    for i ← a.indices do
        sum += a[i]
    end
    sum
end
```

The following demonstrates the use of a reduction variable $prod$ in a parallel `do`:

> DRAFT NOTE
> The · is manually edited.

```
prod : ℤ64 := 1
do
    prod ·= g(y)
also do
    prod ·= f(x)
    prod ·= g(x)
also do
    h(x, y)
end
```

The operator associated with a reduction variable can be invoked in several places during evaluation of a thread group. Most obviously, it can be invoked anywhere the reduction operation occurs textually in the code for that group. The operator can also be invoked multiple times implicitly and in parallel after the completion of some or all of the threads in the group. The group as a whole does not complete normally until all these operator invocations have completed normally and the reduced result has been assigned to the reduction variable for use in subsequent computation. If any operator invocation completes abruptly, it is treated just as if an ordinary implicit thread in the group completed abruptly.

> DRAFT NOTE
> This sticks out like a sore thumb in this chapter. Can we find a happier home for it? It seems like it really belongs in the introductory material.

Different implicit threads (even different iterations of the same loop body) may execute in very different amounts of time. A naively parallelized loop will cause processors to idle until every iteration finishes. The simplest way to mitigate this delay is to expose substantially more parallel work than the number of underlying processors available to run them. Load balancing can move the resulting (smaller) units of work onto idle processors to balance load.

The ratio between available work and number of threads is called *parallel slack* [6, 5]. With support for very lightweight threading and load balancing, slack in hundreds or thousands (or more) proves beneficial; very slack computations easily adapt to differences in the number of available processors. Slack is a desirable property, and the Fortress programmer should endeavor to expose parallelism where possible.

5.5 Environments

An *environment* maps *names* to values or locations. Environments are immutable, and two environments that map exactly the same names to the same values or locations are identical.

A program starts executing with an empty environment. Environments are extended with new mappings by variable, function, and object declarations, and functional calls (including calls to object constructors). After initializing all top-level variables and singleton objects, as described in Section 22.7, the *top-level environment* for each component is constructed.

The environment of a value is determined by how it is *constructed*. For all but object expressions (described in Section 13.9), function expressions (described in Section 13.8) and local function declarations (described in Section 9.5), the environment of the constructed value is the top-level environment of the component in which the expression or declaration occurs. For object and function expressions and local function declarations, the environment of the constructed value is the lexical environment in which the expression or declaration was evaluated.

We carefully distinguish a spawned thread from its associated spawned thread object. In particular, note that the execution environment of a spawned thread, in which the body expression is evaluated, is distinct from the environment of the associated thread object, in which calls to the thread methods are evaluated.

5.6 Input and Output Actions

> DRAFT NOTE
> Check for I/O actions is not yet supported.

Certain functionals in Fortress perform primitive input/output (I/O) actions. These actions have an externally visible effect. Any functional which may perform an I/O action—either because it is a primitive action or because it invokes other functionals which perform I/O actions—must be declared with the `io` modifier.

Any primitive I/O action may take many internal steps; each step may read or write any memory locations referred to either directly or transitively by object references passed as arguments to the action. Each I/O action is free to complete either normally or abruptly. I/O actions may block and be prevented from taking a step until any necessary external conditions are fulfilled (input is available, data has been written to disk, and so forth).

Each I/O action taken by an expression is considered part of that expression's effects. The steps taken by an I/O action are considered part of the context in which an expression evaluates, in much the same way as effects of simultaneously-executing threads must be considered when describing the behavior of an expression. For example, we cannot consider two functionals to be equivalent unless the possible I/O actions they take are the same, given identical internal steps by each I/O action.

Chapter 6

Types

> DRAFT NOTE
> This chapter should be revised according to Victor's tuple story (his email titled "Tuple story" on 12/07/07).

> DRAFT NOTE
> Dimensions and units, type aliases, and some types in Section 6.9 are not yet supported.

The Fortress type system supports a mixture of nominal and structural types. Specifically, Fortress provides nominal trait types, and dimension types (i.e., dimension types are also nominal) which are defined in programs (often in libraries), and a few constructs that combine types structurally.

> DRAFT NOTE
> Victor: This doesn't cover the special types, dimension types, or object expression types. But I think it's okay for this intro, as long as we don't assert that these are all the types.

Every expression has a *static type*, and every value has a *runtime type* (also known as a *dynamic type*). Fortress programs are checked before they are executed to ensure that if an expression e evaluates to a value v, the runtime type of v is a subtype of the static type of e. Sometimes we abuse terminology by saying that an expression has the runtime type of the value it evaluates to (see Section 5.2 for a discussion about evaluation of expressions).

Some types may be parameterized by *static parameters*; we call these types *generic types*. See Chapter 12 for a discussion of static parameters. A static parameter may be instantiated with a type or a value depending on whether it is a type parameter.

> DRAFT NOTE
> Victor: I don't know if the follow really belongs here. It isn't complete in any case, because of union/intersection types.

Two types are identical if and only if they are the same kind and their names and static arguments (if any) are identical. Two `nat` or `int` static arguments (Section 12.2, Section 13.27) are identical if and only if they are written alike once each operation whose operands are numerals is replaced by its value: $2\ 3$ and 6 are identical, and $s_0\ s_1$ is identical to itself but not to $s_1\ s_0$. Two such arguments are known to differ when they are two different numerals; a static parameter, or an expression in which one occurs, is not known to differ from a numeral, since the parameter may be instantiated with a value at which they are equal.

REVISED BY THE 2026 REVIVAL SEE Section I.1.40

The Working Draft of February 2011 said only that the static arguments of identical types are identical, and did not say when two static arguments written as expressions are. The two sentences above say it, in the reading of the team's own questions and answers on overloaded operators in the appendix "Internal Document": a numeral differs from another numeral, and an expression with a static parameter in it is not known to differ from a numeral.

Types are related by several relationships as described in Section 6.2.

Syntactically, the positions in which a type may legally appear (i.e., *type contexts*) is determined by the nonterminal *Type* in the Fortress grammar, defined in Appendix G.

6.1 Kinds of Types

Fortress supports the following kinds of types:

- trait types,
- object expression types,
- tuple types,
- arrow types,
- function types, and
- three special types: Any, $\mathrm{BottomType}$ and $()$.

Object expression types, function types and $\mathrm{BottomType}$ are not first-class types: they cannot be written in a program. However, some values have object expression types and function types and `throw` and `exit` expressions have the type $\mathrm{BottomType}$.

Collectively, trait types and object expression types are *defined* types; the trait and object declarations and object expressions are the types' *definitions*.

DRAFT NOTE

Victor:

I don't know if you like this terminology for trait types and object expression types. Putting this all together allowed some repetition to be eliminated, and emphasizes the similarity, which I think you wanted to retain. I still want to separate object expression types from object trait types. I think the difference here is not as evident as it might be if we handled generics properly: it is the type environment of object expression types that make them a pain, and I'm not dealing with that for the most part.

6.2 Relationships between Types

We define two relations on types: *subtype* and *exclusion*.

DRAFT NOTE

add coercion in full language

The subtype relation is a partial order on types that defines the type hierarchy; that is, it is reflexive, transitive and antisymmetric. Any is the top of the type hierarchy: all types are subtypes of Any. BottomType is the bottom of the type hierarchy: all types are supertypes of BottomType. For types T and U, we write $T \preceq U$ when T is a subtype of U, and $T \prec U$ when $T \preceq U$ and $T \neq U$; in the latter case, we say T is a *strict* subtype of U. We also say that T is a *supertype* of U if U is a subtype of T. Thus, in the execution of a valid Fortress program, an expression's runtime type is always a subtype of its static type. We say that a value is *an instance of* its runtime type and of every supertype of its runtime type; immediate subtypes of Any comprises of tuple types, arrow types, (), Object, and Index, the trait of the types that a subscript takes, of which a tuple type whose element types are all subtypes of Index is a subtype (Section 6.5).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.42
>
> The Working Draft of February 2011 listed tuple types, arrow types, () and Object only; Index, which extends Any and not Object, is new.

The exclusion relation is a symmetric relation between two types whose intersection is BottomType. Because BottomType is uninhabited (i.e., no value has type BottomType), types that exclude each other are disjoint: no value can have a type that is a subtype of two types that exclude each other.

> DRAFT NOTE
>
> The converse is technically not true: we can define two traits with method declarations such that no trait or object can extend them both, so no value can have a type that is a subtype of both trait types, but the trait types do not exclude each other. For example: trait A f(self, y: Object) = 1; end trait B f(x: Object, self) = 2; end trait C f() = 3; end
>
> No pair of the three trait types defined above can be extended but none of them exclude any of the others.

Note that BottomType excludes every type including itself (because the intersection of BottomType and any type is BottomType), and also that no type other than BottomType excludes Any (because the intersection of any type T and Any is T). BottomType is the only type that excludes itself. Also note that if two types exclude each other then any subtypes of these types also exclude each other.

Fortress also allows *coercion* between types (see Chapter 17). A coercion from T to U is defined in the declaration of U. We write $T \to U$ if U defines a coercion from T. We say that T *can be coerced to* U, and write $T \rightsquigarrow U$, if U defines a coercion from T or any supertype of T: $T \rightsquigarrow U \iff \exists T' : T \preceq T' \wedge T' \to U$.

The Fortress type hierarchy is acyclic with respect to both subtyping and coercion relations except for the following:

- The trait Any is a single root of the type hierarchy and it forms a cycle as described in Chapter 31.
- There exists a bidirectional coercion between two tuple types if and only if they have the same sorted form.

These relations are defined more precisely in the following sections describing each kind of type in more detail. Specifically, the relations are the smallest ones that satisfy all the properties given in those sections (and this one).

6.3 Trait Types

Syntax:

Type ::= *TraitType*

A *trait type* is a named type defined by a trait or object declaration. It is an *object trait type* if it is defined by an object declaration.

DRAFT NOTE

Victor: The following text really belongs to traits, not to types. Traits are declared by trait and object declarations (described in Chapter 10 and Chapter 11). A trait has a *trait type* of the same name.

A trait type is a subtype of every type that appears in the `extends` clause of its definition. In addition, every trait type is a subtype of Any.

A trait declaration may also include an `excludes` clause, in which case the defined trait type excludes every type that appears in that clause. Every trait type also excludes every arrow type and the special types () and BottomType, and every trait type other than Index excludes every tuple type; Index excludes a tuple type only where one of its element types excludes Index (Section 6.5).

REVISED BY THE 2026 REVIVAL SEE Section I.1.42

The Working Draft of February 2011 said that every trait type excludes every tuple type. A tuple type whose element types are all subtypes of Index is now a subtype of Index, so Index does not exclude it.

Two instantiations of the same parameterized trait exclude each other unless each static argument of one, other than an operator argument, is the same as the corresponding static argument of the other: the same type for a type parameter, and the same value for a `nat`, `int`, `bool`, dimension, or unit parameter. This property is called *instantiation exclusion*. Operator arguments do not count: an operator argument names the operator that a subtrait inherits (Section 12.5), so instantiations that differ only in their operator arguments, such as the instantiations of BooleanAlgebra that Boolean extends (Section 32.1), do not exclude each other on that account. Because types that exclude each other are disjoint, no type is a subtype of two instantiations of one parameterized trait that differ in a static argument other than an operator argument; the rule that this imposes on trait and object declarations is stated in Section 10.1.

REVISED BY THE 2026 REVIVAL SEE Section I.1.1

The Working Draft of February 2011 stated no exclusion between instantiations, so the relations defined in this chapter, the smallest ones that satisfy the properties given, let a type be a subtype of two different instantiations of one parameterized trait. The paragraph above adds the property. It is the rule the type group named *multiple instantiation exclusion* (Allen, Hilburn, Kilpatrick, Luchangco, Ryu, Chase, and Steele, "Type Checking Modular Multiple Dispatch with Parametric Polymorphism and Multiple Inheritance", OOPSLA 2011), which the compiled type checker enforces for type arguments and numerals, not yet in every case (Section I.1.1 states its coverage), stated here as the team's later Types chapter (Luchangco, 2012) states it, under its shorter name.

6.3.1 Object Trait Types

An object trait type is a trait type defined by an object declaration. Object declarations do not include `excludes` clauses, but an object trait type cannot be extended. Thus, an object trait type excludes any type that is not its supertype.

6.4 Object Expression Types

An object expression type is defined by an object expression (described in Section 13.9) and the program location of the object expression. Every evaluation of a given object expression has the same object expression type. Two object expressions at different program locations have different object expression types.

Like trait types, an object expression type is a subtype of all trait types that appear in the `extends` clause of its definition, as well as the type Any.

Like object trait types, an object expression type excludes any type that is not its supertype.

6.5 Tuple Types

Syntax:

TupleType ::= (*Type*, *TypeList*)
TypeList ::= *Type*(, *Type*)*

> DRAFT NOTE
> Victor: The following two sentences don't belong in this section: A tuple expression is an ordered sequence of expressions separated by commas and enclosed in parentheses. See Section 13.28 for a discussion of tuple expressions.

A tuple type consists of a parenthesized, comma-separated list of two or more types.

Every tuple type is a subtype of Any. No other type encompasses all tuple types. Tuple types cannot be extended by trait types. Tuple types are covariant: a tuple type X is a subtype of tuple type Y if and only if they have the same number of element types and each element type of X is a subtype of the corresponding element type of Y. A tuple type whose element types are all subtypes of the trait Index is a subtype of Index, the trait of the types that a subscript takes (Section 25.7). Index extends Any and not Object (Section 10.1), so such a tuple type is no subtype of Object.

A tuple type excludes any non-tuple type other than Any and Index, and it excludes Index if and only if one of its element types excludes Index.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.42
> The Working Draft of February 2011 made no tuple type a subtype of a trait type, and said that a tuple type excludes any non-tuple type other than Any. The rule that a tuple of indices is an Index, and the exclusion that follows from it, are the revival's: they let an index parameter be bounded by Index while a subscript such as $a[i, j]$ passes the tuple (i, j) as its one index.

A tuple type excludes every tuple type that does not have the same number of element types. Also, tuple types that have the same number of element types exclude each other if any pair of corresponding element types exclude each other.

Intersection of nonexclusive tuple types are defined elementwise; the intersection of nonexclusive tuple type X and Y is a tuple type with exactly corresponding elements, where the type in each element type is the intersection of the types in the corresponding element types of X and Y. Note that intersection of any exclusive types is $\mathrm{BottomType}$ as described in Section 6.8.

6.6 Arrow Types

> DRAFT NOTE
> Arrow types should include `io`-ness.

> DRAFT NOTE
> Arrow types with contracts

Function contracts consist of three parts: a `requires` part, an `ensures` part, and an `invariant` part. All three parts are evaluated in the scope of the function body extended with a special variable *arg*, bound to an immutable array of all function arguments. (This array is useful for describing contracts in the presence of higher-order functions; see Section 6.6).

Is the special variable *arg* reserved? What if one of the function's parameters is named arg?

At runtime, when a function is bound to a variable or parameter f whose type includes a contract, the contract is evaluated when the function is called through a reference to f. (Note that this contract is distinct from the contract attached to the function bound to f, which is also evaluated upon a call to f). This contract is evaluated in the enclosing scope of the arrow type, extended with any keyword argument names provided in the arrow type, and the special variable arg bound to an immutable array containing the arguments provided at the call site (in the order provided at the call site). If the `requires` clauses stipulated in the type of f are not satisfied, a $\mathrm{CallerViolation}$ exception is thrown. Otherwise, if the `requires` clauses attached to the function bound to f is not satisfied, a $\mathrm{ContractBinding}$ exception is thrown. If any other part of the contract of the function bound to f is not satisfied, a $\mathrm{CalleeViolation}$ exception is thrown. Otherwise, if any part of the contract stipulated in the type of f is not satisfied, a $\mathrm{ContractBinding}$ exception is thrown.

Syntax:

Type	::=	*ArgType* $\rightarrow$ *Type Throws*?
ArgType	::=	((*Type*,)* *Type*...)
	|	*TupleType*

The static type of an expression that evaluates to a function value is an *arrow type* (or possibly an intersection of arrow types). An arrow type has three constituent parts: a *parameter type*, a *return type*, and a set of *exception types*. (Multiple parameters or return values are handled by having tuple types for the parameter type or return type respectively.)

DRAFT NOTE

Victor: I'd rather say "an exception type", where that type can be a union type, but I'm avoiding mentioning union types for now.

Syntactically, an arrow type consists of the parameter type followed by the token $\rightarrow$, followed by the return type, and optionally a `throws` clause that specifies the set of exception types.

DRAFT NOTE

Victor: What about io? Ignoring that for now.

If there is no `throws` clause, the set of exception types is empty.

The parameter type of an arrow type may end with a "varargs entry", $T\ldots$. The resulting parameter type is not a first-class type; rather, it is the union of all the types that result from replacing the varargs entry with zero or more entries of the type T. For example, $(T\ldots)$ is the union of $()$, T, (T,T), (T,T,T), and so on.

We say that an arrow type is *applicable* to a type A if A is a subtype of the parameter type of the arrow type. (If the parameter type has a varargs entry, then A must be a subtype of one of the types in the union defined by the parameter type.)

Every arrow type is a subtype of Any. No other type encompasses all arrow types. A type parameter can be instantiated with an arrow type. Arrow types cannot be extended by trait types. Arrow types are covariant in their return type and exception types and contravariant in their parameter type; that is, arrow type "$A \rightarrow B$ `throws` C" is a subtype of arrow type "$D \rightarrow E$ `throws` F" if and only if:

- D is a subtype of A,

- B is a subtype of E, and
- for all X in C, there exists Y in F such that X is a subtype of Y.

For arrow types S and T, we say that S is *more specific than* T if the parameter type of S is a subtype of the parameter type of T. We also say that S is *more restricted than* T if the return type of S is a subtype of the return type of T and for every X in the set of exception types of S, there exists Y in the set of exception types of T such that X is a subtype of Y. Thus, S is a subtype of T if and only if T is more specific than S and S is more restricted than T.

An arrow type excludes any non-arrow type other than Any and the function types that are its subtypes (see Section 6.7). However, arrow types do not exclude other arrow types because of overloading as described in Chapter 15.

Here are some examples:

$f\colon (\mathbb{R}64, \mathbb{R}64) \to \mathbb{R}64$

$g\colon \mathbb{Z}32 \to (\mathbb{Z}32, \mathbb{Z}32)$ `throws` IOFailure

6.7 Function Types

Function types are the runtime types of function values. We distinguish them from arrow types to handle overloaded functions. A function type consists of a finite set of arrow types. However, not every set of arrow types is a well-formed function type. Rather, a function type F is *well-formed* if, for every pair (S_1, S_2) of distinct arrow types in F, the following properties hold:

- the parameter types of S_1 and S_2 are not the same,
- if S_1 is more specific than S_2 then S_1 is more restricted than S_2, and
- if the intersection of the parameter types of S_1 and S_2 is not $\mathrm{BottomType}$ (i.e., the parameter types of S_1 and S_2 do not exclude each other) then F has some constituent arrow type that is more specific than both S_1 and S_2 (recall that the more specific relation is reflexive, so the required constituent type may be S_1 or S_2).

Henceforth, we consider only well-formed function types. The overloading rules ensure that all function values have well-formed function types.

We say that a function type F is *applicable* to a type A if any of its constituent arrow types is applicable to A. We extend this terminology to values of the corresponding types. That is, for example, we say that a function of type F is applicable to a value of type A if F is applicable to A. Note that if a well-formed function type F is applicable to a type A, then among all constituent arrow types of F that are applicable to A (and there must be at least one), one is more specific than all the others. We say that this constituent type is the *most specific* type of F applicable to A.

A function type is a subtype of each of its constituent arrow types, and also of Any. Like object trait types and object expression types, a function type excludes every type that is not its supertype.

6.8 Special Types

Fortress provides three special types: Any, $\mathrm{BottomType}$ and $()$. The type Any is the top of the type hierarchy: it is a supertype of every type. The only type it excludes is $\mathrm{BottomType}$.

The type $()$ is the type of the value $()$. Its only supertype (other than itself) is Any, and it excludes every other type.

Fortress provides a special *bottom type*, $\mathrm{BottomType}$, which is an uninhabited type. No value in Fortress has $\mathrm{BottomType}$; `throw` and `exit` expressions have $\mathrm{BottomType}$. $\mathrm{BottomType}$ is a subtype of every type and

it excludes every type (including itself). As mentioned above, $\mathrm{BottomType}$ is not a first-class type: programmers must not write $\mathrm{BottomType}$.

> DRAFT NOTE
> Victor's comments The complex numbers have precision like the integers, but are they intended to be only integral complex numbers? That seems odd to me. Also, why say "imaginary and complex", since we don't provide a separate type for just the imaginary numbers? Guy's going to provide APIs for complex numbers.

6.9 Types in the Fortress Standard Libraries

The Fortress standard libraries define simple standard types for literals such as $\mathrm{BooleanLiteral}[\![b]\!]$, $()$ (pronounced "void"), $\mathrm{Character}$, String, and $\mathrm{Numeral}[\![n, m, r, v]\!]$ for appropriate values of b, n, m, r, and v (See Section 13.1 for a discussion of Fortress literals). Moreover, there are several simple standard numeric types. These types are mutually exclusive; no value has more than one of them. Values of these types are immutable.

The numeric types share the common supertype Number. Fortress includes types for arbitrary-precision integers (of type $\mathbb{Z}$), their unsigned equivalents (of type $\mathbb{N}$), rational numbers (of type $\mathbb{Q}$), real numbers (of type $\mathbb{R}$), complex numbers (of type $\mathbb{C}$), fixed-size representations for integers including the types $\mathbb{Z}8$, $\mathbb{Z}16$, $\mathbb{Z}32$, $\mathbb{Z}64$, $\mathbb{Z}128$, their unsigned equivalents $\mathbb{N}8$, $\mathbb{N}16$, $\mathbb{N}32$, $\mathbb{N}64$, $\mathbb{N}128$, floating-point numbers (described below), intervals (of type $\mathrm{Interval}[\![X]\!]$, abbreviated as $\langle\!|X|\!\rangle$, where X can be instantiated with any number type), and imaginary and complex numbers of fixed size (in rectangular form with types $\mathbb{C}n$ for $n = 16, 32, 64, 128, 256$ and polar form with type $\mathrm{Polar}[\![X]\!]$ where X can be instantiated with any real number type).

The Fortress standard libraries also define other simple standard types such as Any, Object, $\mathrm{Exception}$, $\mathrm{Boolean}$, and $\mathrm{BooleanInterval}$ as well as low-level binary data types such as $\mathrm{LinearSequence}$, $\mathrm{HeapSequence}$, and $\mathrm{BinaryWord}$. See Parts V and VI for discussions of the Fortress standard libraries.

6.10 Intersection and Union Types

For every finite set of types, there is a type denoting a unique *intersection* of those types. The intersection of a set of types S is a subtype of every type $T \in S$ and of the intersection of every subset of S. There is also a type denoting a unique *union* of those types. The union of a set of types S is a supertype of every type $T \in S$ and of the union of every subset of S. Neither intersection types nor union types are first-class types; they are used solely for type inference (as described in Chapter 20) and they cannot be expressed directly in programs.

The intersection of a set of types S is equal to a named type U when any subtype of every type $T \in S$ and of the intersection of every subset of S is a subtype of U. Similarly, the union of a set of types S is equal to a named type U when any supertype of every type $T \in S$ and of the union of every subset of S is a supertype of U.

A type U *covers* a type T when every value of T is a value of U. A type covers each of its subtypes, and a trait with a `comprises` clause is covered by the union of the types listed in the clause (Section 10.1). Covering is a relation between the values of types and not the subtype relation: a type may cover another type that is not its subtype.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.33
> The Working Draft of February 2011 did not define covering. The team's later Types chapter (Luchangco, 2012) names covering beside subtyping and exclusion as a relation between types, and says that an instantiation of a trait with a `comprises` clause is covered by the union of the listed types.

For example:

```
trait S comprises {U, V} end
trait T comprises {V, W} end
trait U extends S excludes W end
trait V extends {S, T} end
trait W extends T end
```

because of the `comprises` clauses of S and T and the `excludes` clause of U, any value of both S and T is a value of V. Thus, V covers $S \cap T$. A trait may stand between S or T and their listed types (Section 10.1), so $S \cap T$ is not in general equal to a named type.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.33
>
> The Working Draft of February 2011 concluded that any subtype of both S and T must be a subtype of V, so that $V = S \cap T$. Since the revision of Section 10.1, a trait with static parameters may extend both S and T unlisted when every trait or object that extends it is a subtype of V; an instantiation of it is a subtype of S and of T and not of V, though every value of it is a value of V.

Intersection types (denoted by $\cap$) possess the following properties:

- Commutativity: $T \cap U = U \cap T$.
- Associativity: $S \cap (T \cap U) = (S \cap T) \cap U$.
- Subsumption: If $S \preceq T$ then $S \cap T = S$.
- Preservation of shared subtypes: If $T \preceq S$ and $T \preceq U$ then $T \preceq S \cap U$.
- Preservation of supertype: If $S \preceq T$ then $\forall U.\ S \cap U \preceq T$.
- Distribution over union types: $S \cap (T \cup U) = (S \cap T) \cup (S \cap U)$.

Union types (denoted by $\cup$) possess the following properties:

- Commutativity: $T \cup U = U \cup T$.
- Associativity: $S \cup (T \cup U) = (S \cup T) \cup U$.
- Subsumption: If $S \preceq T$ then $S \cup T = T$.
- Preservation of shared supertypes: If $S \preceq T$ and $U \preceq T$ then $S \cup U \preceq T$.
- Preservation of subtype: If $T \preceq S$ then $\forall U.\ T \preceq S \cup U$.
- Distribution over intersection types: $S \cup (T \cap U) = (S \cup T) \cap (S \cup U)$.

6.11 Type Aliases

Syntax:

TypeAlias ::= `type` *Id StaticParams*? $=$ *Type*

Fortress allows names to serve as aliases for more complex type instantiations. A *type alias* begins with `type` followed by the name of the alias type, followed by optional static parameters, followed by $=$, followed by the type it stands for. Parameterized type aliases are allowed but recursively defined type aliases are not. Here are some examples:

```
type IntList = List⟦ℤ64⟧
type BinOp = Float × Float → Float
type SimpleFloat⟦nat e, nat s⟧ = DetailedFloat⟦Unity, e, s, false, false, false, false, true⟧
```

All uses of type aliases are expanded before type checking. Type aliases do not define new types nor nominal equivalence relations among types.

Chapter 7

Names and Declarations

> DRAFT NOTE
> This chapter should be revised according to Victor's new description.
>
> Qualified names, dimensions and units, tests and properties, where clauses, keyword and varargs parameters, and type aliases are not yet supported.

> DRAFT NOTE
> Issues – Victor
>
> - opr parameter scope, etc.
> - delayed initialization of variables
>   - Synonyms? (especially for operators, but maybe for identifers)
>   - T.`coerce` is a new kind of "name", so we need to be careful how we phrase this. (If we can't call T.`coerce` explicitly, then should coercion declarations even be considered declarations?)
> - Should distinguish simple names, API names, and qualified names.
>   - Need to discuss (perhaps in components/APIs chapter) restriction on API names so that "first part" does not conflict with any name declared in (or imported unqualified by) the component.
>   - Should also discuss declaration by import statements. Is that a declaration? Even if we don't consider normal import statements to be declarations–the declarations are in the APIs–when import statements provide aliases, that must be considered a declaration.

> DRAFT NOTE
> Expressions that declare names: typecase, catch clauses, generators, labeled blocks, bindings? the declaration in a typecase is just a binding so it's not actually that special, except for the static type
>
> I think we can consider "self" and "coerce" to be "special identifiers". Or else the "names" declared by declarations are more than just identifiers and operators.
>
> Search for comments for some recommendations for local changes.

Names are used to refer to certain kinds of entities in a Fortress program. Names may be simple or qualified. A simple name is either an identifier or an operator. An operator may be an operator token, a special token corresponding to a special operator character, or a matching pair of enclosing operator tokens. A qualified name consists of an API name

followed by ".", followed by an identifier, where an API name consists of a sequence of identifiers separated by "." tokens. Note that operators cannot be qualified. Except in Section 7.4, we consider only simple names in this chapter.

Simple names are typically introduced by declarations, which bind the name to an entity. In some cases, the declaration is implicit. Every declaration has a scope, in which the declared name can be used to refer to the declared entity.

Declarations introduce *named entities*; we say that a declaration *declares* an entity and a name by which that entity can be referred to, and that the declared name *refers to* the declared entity. As discussed later, there is not a one-one correspondence between declarations and named entities: some declarations declare multiple named entities, some declare multiple names, and some named entities are declared by multiple declarations.

Some declarations contain other declarations. For example, a trait declaration may contain method declarations, and a function declaration may contain parameter declarations.

The positions in which a declaration may legally appear in a component are determined by the nonterminal *Decl* in the simplified Fortress grammar in Appendix G.

7.1 Kinds of Declarations

> DRAFT NOTE
> The grammar the follows is only for top-level declarations.

Syntax:

Decl ::= *TraitDecl*
| *ObjectDecl*
| *VarDecl*
| *FnDecl*
| *DimUnitDecl*
| *TypeAlias*
| *TestDecl*
| *PropertyDecl*
| *ExternalSyntax* [1]

There are two kinds of declarations: *top-level declarations* and *local declarations*.

Top-level declarations occur at the top level of a component, not within any other declaration or expression. A top-level declaration is one of the following:[2]

- trait declarations (see Chapter 10);
- object declarations (see Chapter 11), which may be singleton declarations or constructor declarations;
- top-level variable declarations (see Section 8.1);
- top-level function declarations (see Chapter 9), including top-level operator declarations (see Chapter 16);
- dimension declarations (see Chapter 18);
- unit declarations (see Chapter 18);
- top-level type aliases (see Section 6.11);
- test declarations (see Chapter 19);

[2]The Fortress component system, defined in Chapter 22, includes declarations of *components* and *APIs*. Because component names are not used in a Fortress program and API names are used only in qualified names and `import` and `export` statements, we do not discuss them in this chapter. However, these declarations are not proper constituents of a valid program. Rather, valid programs are *assembled* from them.

- top-level property declarations; (see Chapter 19)

Local declarations occur in another declaration or in some expression (or both). A local declaration is one of the following:

- field declarations (see Section 11.2), which occur in object declarations and object expressions, and include field declarations in the parameter list of a constructor declaration;
- method declarations (see Section 10.2), which occur in trait and object declarations and object expressions;
- coercion declarations (see Chapter 17), which occur in trait and object declarations;
- local variable declarations (see Section 8.2), which occur in expression blocks;
- local function declarations (see Section 9.5), which occur in expression blocks;
- local property declarations (see Chapter 19), which occur in trait and object declarations and object expressions;
- labeled blocks (see Section 13.12), which are expressions;
- static-parameter declarations, which may declare type parameters, `nat` parameters, `int` parameters, `bool` parameters, `dim` parameters, `unit` parameters, or `opr` parameters (see Chapter 12), and occur in static-parameter lists of trait and object declarations, top-level type aliases, top-level function declarations, method declarations, and local function declarations.
- hidden-type-variable declarations, which occur in `where` clauses (see Section 12.6) of trait and object declarations, top-level function declarations, and method declarations;
- type aliases in `where` clauses, of trait and object declarations, top-level function declarations, and method declarations;
- (value) parameter declarations, which may be keyword-parameter declarations which occur in parameter lists of constructor declarations, top-level function declarations, method declarations, local function declarations, and function expressions

Some declarations are syntactic sugar for other declarations. Throughout this chapter, we consider declarations after they have been desugared. Thus, apparent field declarations in trait declarations are actually method declarations (as described in Section 10.2), and a dimension and unit declaration may desugar into several separate declarations (as described in Section 26.3). After desugaring, the kinds of declarations listed above are disjoint.

In addition to these explicit declarations, there are two cases in which names are declared implicitly:

- the special name `self` is implicitly declared as a parameter of dotted methods (see Section 10.2 for details); and
- the name *result* is implicitly declared as a variable for the `ensures` clause of a contract (see Section 9.4 for details).

> DRAFT NOTE
> This is *not* the proper reference, but I don't know what is.

As discussed in Chapter 8, the initialization of a local variable may be separated from its declaration. In this case we do not consider the initialization of an immutable variable to be a declaration, even though it is syntatically indistinguishable from a declaration of an immutable variable.

Trait declarations, object declarations, top-level type aliases, type-parameter declarations, and hidden-type-variable declarations are collectively called *type declarations*; they declare names that refer to *types* (see Chapter 6). Dimension declarations and `dim`-parameter declarations are *dimension declarations*, and unit declarations and `unit`-parameter declarations are *unit declarations*. Constructor declarations, top-level function declarations, method declarations,

and local function declarations are collectively called *functional declarations*. Singleton declarations, top-level variable declarations, field declarations, local variable declarations (including implicit declarations of $result$) and (value) parameter declarations (including implicit declarations of `self`) are collectively called *variable declarations*. Static-parameter declarations and hidden-type-variable declarations are also collectively called *static-variable declarations*. Note that static-variable declarations are disjoint from variable declarations.

> DRAFT NOTE
>
> Victor: May want to add "module-wide" declarations, which are the top-level declarations plus functional method declarations. These are the declarations whose reach is the entire component and that can be imported from an API.

The groups of declarations defined in the previous paragraph are neither disjoint nor exhaustive. For example, labeled blocks are not included in any of these groups, and an object declaration is both a type declaration and either a function or variable declaration, depending on whether it is singleton.

Most declarations declare a single name given explicitly in the declaration (though, as discussed in Section 7.2, they may declare this name in multiple namespaces). There is one exception: wrapped field declarations (described in Section 10.3) in object declarations and object expressions declare both the field name and names for methods provided by the declared type of the field.

Method declarations in a trait may be either *abstract* or *concrete*. Abstract declarations do not have bodies; concrete declarations, sometimes called *definitions*, do.

7.2 Namespaces

Fortress supports three namespaces, one for types, one for values, and one for labels. (If we consider the Fortress component system, there is another namespace for APIs.) These namespaces are logically disjoint: names in one namespace do not conflict with names in another.

Type declarations, of course, declare names in the type namespace. Function and variable declarations declare names in the value namespace. (This implies that object names are declared in both the type and value namespaces.) Labeled blocks declare names in the label namespace. Although they are not all type declarations, all the static-variable declarations declare names in the type namespace, as do dimension declarations. In addition, `nat` parameters, `int` parameters, `bool` parameters, `unit` parameters, and `opr` parameters are also declared in the value namespace. Section 12.5 describes `opr` parameters in more detail.

Every occurrence of a name in a program is either a declaration or a reference. [3] A reference to a name is resolved to the entity that the name refers to the namespace appropriate to the context in which the reference occurs. For example, a name refers to a label if and only if it occurs immediately following the reserved word `exit`. It refers to a type if and only if it appears in a type context (described in Chapter 6). Otherwise, it refers to a value.

7.3 Reach and Scope of Declarations

In this section, we define the *reach* and *scope* of a declaration, which determine where a declared name may be used to refer to the entity declared by the declaration. It is a static error for a reference to a name to occur at any point in a program at which the name is not *in scope* in the appropriate namespace (as defined below), except immediately after the '.' of a dotted field access or dotted method invocation when the name is the name of the field or dotted method

[3] We are abusing terminology here by using "declaration" to denote the occurrence of a name, whereas we have generally been using "declaration" to denote the syntactic construct that declares the name. Many declarations in the latter sense allow the declared name to occur several times: only one such occurrence is a declaration in the former sense; the rest are references.

provided by the static type of the receiver expression (see Sections 13.3 and 13.4). It is also a static error for multiple declarations with overlapping reaches to declare the same name in the same namespace unless the declarations are *overloaded* or one declaration *shadows* the other, as defined below.

We first define a declaration's *reach*. The reach of a labeled block is the block itself. The reach of a functional method declaration is the component containing that declaration. A dotted method declaration not in an object expression (described in Section 13.9) or declaration must be in the declaration of some trait T, and its reach is the declaration of T and any trait or object declarations or object expressions that extend T; that is, if the declaration of trait T contains a method declaration, and trait S extends trait T, then the reach of that method declaration includes the declaration of trait S. The reach of any other (explicit) declaration is the smallest block strictly containing that declaration (i.e., not just the declaration itself). For example, the reach of any top-level declaration (including any imported declaration) is the component containing (or importing) that declaration; the reach of a field declaration is the enclosing object declaration or expression; the reach of a parameter declaration is the constructor declaration, functional declaration, or function expression in whose parameter list it occurs; and the reach of a local variable declaration is the smallest block in which that declaration occurs (recall from Chapter 3 that a local variable declaration always starts a new block). The reach of an implicit declaration of `self` by a dotted method declaration is the method declaration (the same as the explicit parameters of the method), and the reach of an implicit declaration of $result$ by an `ensures` clause of a contract is the `ensures` clause. We say that a declaration *reaches* any point within its reach.

One declaration *shadows* another for a name in a namespace if they both declare the name in the namespace, and any of the following conditions hold:

- The first declaration is a field or dotted method declaration in a trait declaration, object declaration or object expression contained strictly within the reach of the second declaration, unless the first declaration is in an object expression that extends a trait that provides the second declaration (in which case the declarations are overloaded).

  > DRAFT NOTE
  > Alternatively, split above condition into two:
  >
  > – The first declaration is a field or dotted method declaration in a trait or object declaration and the second declaration is a top-level or functional method declaration.
  >
  > – The first declaration is a field or dotted method declaration in an object expression contained strictly entirely within the reach of the second declaration, unless the object expression extends a trait that provides the second declaration (in which case the declarations are overloaded).

- The first declaration is a keyword-parameter declaration of a function or method declaration contained strictly in the reach of the second declaration.

- the name is `self`, and the reach of the first declaration is contained in the reach of the second declaration.

  > DRAFT NOTE
  > I'm not sure this is necessary because self should not be in scope in an enclosed object expression in any case. However, we still have two declarations of self with overlapping reaches, but which are not overloaded, so it may be safer to say there is shadowing in any case.

- the name is $result$ and the first declaration is an implicit declaration by an `ensures` clause contained in the reach of the second declaration.

We omit the name and the namespace when they are clear from context. We say that the second declaration *is shadowed by* the first at any point in the program that their reaches overlap. No other shadowing is permitted in a Fortress program.

The *scope* of a declaration for a name in a namespace (assuming the declaration declares the name in the namespace) consists of all points in the reach of the declaration except the following:

- any point where the declaration is shadowed by another for that name and namespace;
- the declaration is a type alias (top-level or not), a dimension declaration or a unit declaration, and the program point is in the declaration;
- if the declaration is a field, local variable or parameter declaration, any point in the declaration or that textually precedes the declaration;
- if the name is `self`, any point in an object expression contained in the reach of the declaration;
- if the declaration is a labeled block, any point in a `spawn` expression in the labeled block.

We say that a name is *in scope* in a namespace at any point in the program that is in the scope of some declaration for that name and namespace. Again, we omit the name and/or namespace when they are clear from context.

As mentioned above, it is a static error for a reference to a name to occur where it is not in scope in the appropriate namespace. In addition, local variables must be properly initialized before they are used, as discussed in Chapter 8.

If the scope of multiple declarations for the same name and namespace overlap, then we say that the name is *overloaded* in that namespace in the overlap, and we say that the declarations are *overloaded* for that name.

Declarations may be overloaded only if one of the following conditions hold:

- They are all dotted method declarations.
- They are all top-level function declarations and functional method declarations.
- One is a field declaration, and the rest are getter and setter declarations.

In the first two cases, the declarations define an *overloaded functional* (i.e., function or method), and they must satisfy certain rules that ensure that the functional does not admit ambiguous calls. See Chapter 24 for further discussion on this topic. In the last case, the type of the field must be a subtype of the return type of any getter and a supertype of the parameter type of any setter.

> DRAFT NOTE
>
> Can we put the rules for fields overloading into the overloading chapter and over here simply say, ”in either case, see Chapter XX”? We might also join the first two cases, and leave the discussion of which kinds of functional declarations can be overloaded to the overloading chapter.

7.4 Imported Declarations and Qualified Names

Syntax:

QualifiedName ::= *Id*(. *Id*)*

Fortress provides a component system in which the entities declared in a component are described by an API. A component may *import* APIs, allowing it to refer to these entities declared by the imported APIs. In some cases, references to these entities must be *qualified* by the API name. These qualified names can be used in any place that a simple name would be used had the entity been declared directly in the component rather than being imported. Note that qualified names are distinguished from simple names by the inclusion of a “.” token, so they never shadow, nor are they shadowed by, simple names. For further discussion on APIs and the component system, see Chapter 22.

Chapter 8

Variables

> DRAFT NOTE
> Functionals with the modifier `io` and matrix unpasting are not yet supported.

8.1 Top-Level Variable Declarations

Syntax:

VarDecl	::=	*VarMods*? *VarWTypes InitVal*
	|	*VarImmutableMods*? *BindIdOrBindIdTuple* = *Expr*
	|	*VarMods*? *BindIdOrBindIdTuple* : *Type*... *InitVal*
	|	*VarMods*? *BindIdOrBindIdTuple* : *TupleType InitVal*
VarMods	::=	*VarMod*$^+$
VarImmutableMods	::=	*VarImmutableMod*$^+$
VarMod	::=	*AbsVarMod* | `private`
VarImmutableMod	::=	*AbsVarImmutableMod* | `private`
AbsVarMod	::=	`var` | `test`
AbsVarImmutableMod	::=	`test`
VarWTypes	::=	*VarWType*
	|	(*VarWType*(, *VarWType*)$^+$)
VarWType	::=	*BindId IsType*
InitVal	::=	(= |:=) *Expr*
TupleType	::=	(*Type*, *TypeList*)
TypeList	::=	*Type*(, *Type*)*
IsType	::=	: *Type*

A *variable*'s name can be any valid Fortress identifier. There are three forms of variable declarations. The first form:

$id : \mathrm{Type} = expr$

declares id to be an immutable variable with static type Type whose value is computed to be the value of the *initializer* expression $expr$. The static type of $expr$ must be a subtype of Type.

The second (and most convenient) form:

$id = expr$

declares id to be an immutable variable whose value is computed to be the value of the expression $expr$; the static type of the variable is the static type of $expr$.

The third form:

$\texttt{var}\ id : \mathrm{Type} = expr$

declares id to be a mutable variable of type Type whose initial value is computed to be the value of the expression $expr$. As before, the static type of $expr$ must be a subtype of Type. The modifier `var` is optional when '$:=$' is used instead of '$=$' as follows:

$\texttt{var}?\ id : \mathrm{Type} := expr$

In short, immutable variables are declared and initialized by '$=$' and mutable variables are declared and initialized either by '$:=$' or by declaring them according to the third form above with the modifier `var`.

All forms can be used with *tuple notation* to declare multiple variables together. Variables to declare are enclosed in parentheses and separated by commas, as are the types declared for them:

$(id(, id)^+) : (\mathrm{Type}(, \mathrm{Type})^+)$

Alternatively, the types can be included alongside the respective variables, optionally eliding types that can be inferred from context:

$(id(: \mathrm{Type})?(, id(: \mathrm{Type})?)^+)$

Alternatively, a single type followed by '$\ldots$' can be declared for all of the variables:

$(id(, id)^+)\colon \mathrm{Type}\ldots$

This notation is especially helpful when a function application returns a tuple of values.

The initializer expressions of top-level variable declarations can refer to variables declared later in textual order. Evaluation of top-level initializer expressions cannot call functionals with the modifier `io`.

Here are some simple examples of variable declarations:

$\pi = 3.14159265358979323846264338327950288419716939937510820974944592307 8$

declares the variable π to be an approximate representation of the mathematical object π. It is also legal to write:

$\pi\colon \mathbb{R}64 = 3.14159265358979323846264338327950288419716939937510820974944592307 8$

This definition enforces that π has static type $\mathbb{R}64$.

The following example declares multiple variables using tuple notation:

$\texttt{var}\ (x, y)\colon \mathbb{Z}64\ldots = (5, 6)$

The following three declarations are equivalent:

$(x, y, z)\colon (\mathbb{Z}64, \mathbb{Z}64, \mathbb{Z}64) = (0, 1, 2)$
$(x\colon \mathbb{Z}64, y\colon \mathbb{Z}64, z\colon \mathbb{Z}64) = (0, 1, 2)$
$(x, y, z)\colon \mathbb{Z}64\ldots = (0, 1, 2)$

8.2 Local Variable Declarations

> DRAFT NOTE
> Delayed initialization of local variable declarations:
>
> We want to disallow type annotation for delayed initialization of local variable declarations:

```
do
  i: ℤ32
  _ = if b
      then i: ℤ32 = 1
      else i: ℤ32 = 0
      end
  println i
end
```

Syntax:

LocalVarDecl	::=	`var` ? *LocalVarWTypes InitVal*
	|	`var` ? *LocalVarWTypes*
	|	*LocalVarWoTypes* = *Expr*
	|	`var` ? *LocalVarWoTypes* : *Type*... *InitVal*?
	|	`var` ? *LocalVarWoTypes* : *TupleType InitVal*?
LocalVarWTypes	::=	*LocalVarWType*
	|	(*LocalVarWType*(, *LocalVarWType*)$^+$)
LocalVarWType	::=	*BindId IsType*
LocalVarWoTypes	::=	*LocalVarWoType*
	|	(*LocalVarWoType*(, *LocalVarWoType*)$^+$)
LocalVarWoType	::=	*BindId*
	|	*Unpasting*

Variables can be declared within expression blocks (described in Section 13.11) via the same syntax as is used for top-level variable declarations (described in Section 8.1) except that local variables must not include modifiers besides `var` and additional syntax is allowed as follows:

- The form:

  `var`? id : Type

  declares a variable without giving it an initial value (where mutability is determined by the presence of the `var` modifier). It is a static error if such a variable is referred to before it has been given a value; an immutable variable is initialized by another variable declaration and a mutable variable is initialized by assignment. It is also a static error if an immutable variable is initialized more than once. Whenever a variable bound in this manner is assigned a value, the type of that value must be a subtype of its declared type. This form allows declaration of the types of variables to be separated from definitions, and it allows programmers to delay assigning to a variable before a sensible value is known. In the following example, the declaration of the type of a variable and its definition are separated:

  π: ℝ64

  $\pi = 3.14159265358979323846264338327950288419716939937510820974944592307 8$

- Special syntax for declaring local variables as parts of a matrix is provided as described in Section 8.3.

8.3 Matrix Unpasting

> DRAFT NOTE
> Matrix unpasting is not yet supported.

> **DRAFT NOTE**
> We should add a static check for an unpasting to avoid runtime unpasting exceptions.

Syntax:

Unpasting	::=	[*UnpastingElems*]
UnpastingElems	::=	*UnpastingElem RectSeparator UnpastingElems*
	|	*UnpastingElem*
UnpastingElem	::=	*BindId* ([*UnpastingDim*])?
	|	*Unpasting*
UnpastingDim	::=	*ExtentRange* ($\times$ *ExtentRange*)$^+$
ExtentRange	::=	*StaticArg*?# *StaticArg*?
	|	*StaticArg*?: *StaticArg*?
	|	*StaticArg*
RectSeparator	::=	;+
	|	*Whitespace*

Matrix unpasting is an extension of local variable declaration syntax as a shorthand for breaking a matrix into parts. On the left-hand side of a declaration, what looks like a matrix pasting of unbound variables is actually a declaration of several new variables. This syntax serves to break the right-hand side into pieces and bind the pieces to the variables. Matrix unpastings are concise, eliminate several opportunities for fencepost errors, guarantee unaliased parts, and avoid overspecification of how the matrix should be taken apart.

The motivating example for matrix unpasting is cache-oblivious matrix multiplication. The general plan in a cache oblivious algorithm is to break the input apart on its largest dimension, and recursively attack the resulting smaller and more compact problems.

> **DRAFT NOTE**
> Several bugs in the emacs tool for the code in this section. - superscripts were too complicated for the tool - left-hand bracket for the array was wrong

$$
\begin{array}{l}
mm[\![\texttt{nat}\ m, \texttt{nat}\ n, \texttt{nat}\ p]\!](left : \mathbb{R}^{m\times n}, right : \mathbb{R}^{n\times p}, result : \mathbb{R}^{m\times p}) : () = \\
\quad \texttt{case largest of} \\
\quad\quad 1 \Rightarrow result_{0,0} \mathrel{+}= (left_{0,0} right_{0,0}) \\
\quad\quad m \Rightarrow [\, lefttop \\
\quad\quad\quad\quad\ leftbottom\,] = left \\
\quad\quad\quad\quad [\, resulttop \\
\quad\quad\quad\quad\ resultbottom\,] = result \\
\quad\quad\quad\quad t_1 = \texttt{spawn}\ mm(lefttop, right, resulttop) \\
\quad\quad\quad\quad mm(leftbottom, right, resultbottom) \\
\quad\quad\quad\quad t_1.wait() \\
\quad\quad p \Rightarrow [\, rightleft\ rightright\,] = right \\
\quad\quad\quad\quad [\, resultleft\ resultright\,] = result \\
\quad\quad\quad\quad t_1 = \texttt{spawn}\ mm(left, rightleft, resultleft) \\
\quad\quad\quad\quad mm(left, rightright, resultright) \\
\quad\quad\quad\quad t_1.wait() \\
\quad\quad n \Rightarrow [\, leftleft\ leftright\,] = left \\
\quad\quad\quad\quad [\, righttop \\
\quad\quad\quad\quad\ rightbottom\,] = right \\
\quad\quad\quad\quad mm(leftleft, righttop, result) \\
\quad\quad\quad\quad mm(leftright, rightbottom, result) \\
\quad \texttt{end}
\end{array}
$$

In unpasting, the element syntax is slightly enhanced both to permit some specification of the split location and to receive information about the split that was performed. For example, perhaps only the upper left square of a matrix is interesting. The programmer can annotate bounds to the square unpasted element:

> DRAFT NOTE
> I manually edited superscripts, subscripts, and the left square bracket.

$$
\begin{array}{l}
foo[\![\texttt{nat } m, \texttt{nat } n]\!](A : \mathbb{R}^{m \times n}) : () = \\
\quad \texttt{if } m < n \texttt{ then} \\
\quad\quad [\, squareShape_{m \times m} rest \,] = A \\
\quad\quad \ldots \\
\quad \texttt{elif } m > n \texttt{ then} \\
\quad\quad\quad [\, squareShape_{n \times n} \\
\quad\quad\quad\ rest \,] = A \\
\quad\quad\quad \ldots \\
\quad \texttt{else } (* \text{ A already square } *) \\
\quad\quad\quad squareShape = A \\
\quad\quad\quad \ldots \\
\quad \texttt{end}
\end{array}
$$

The types of the elements of the newly declared matrix variables on the left-hand side of an unpasting are inferred (trivially) to be the type of the elements on the right-hand side.

If an unpasting into explicitly sized pieces does not exactly cover the right-hand-side matrix, an UnpastingError is thrown.

Each element of the left-hand-side of unpasting includes an optional *extent specification*. An extent specification $low \,\#\, num$ describes the indexing and the size of the given part of the matrix. The lower extent must be bound, either before the unpasting, or earlier (left-or-above) in the unpasting. For example, suppose that an algorithm chooses to break a matrix into 4 pieces, but retain the original indices for each piece:

$$
\begin{array}{l}
bar[\![\texttt{nat } p, \texttt{nat } q]\!](X : \mathbb{R}^{r_0 \# p \times c_0 \# q}) : () = \texttt{do} \\
\quad [\, \mathrm{A}_{r_0 \# m \quad \times c_0 \# n} \ \mathrm{B}_{r_0 \# m \quad \times c_0 + n \# q - n} \\
\quad\ \mathrm{C}_{r_0 + m \# p - m \times c_0 \# n} \ \mathrm{D}_{r_0 + m \# p - m \times c_0 + n \# q - n} \,] = X \\
\quad \ldots \\
\texttt{end}
\end{array}
$$

Unpasting does not directly support non-uniform decomposition, and does not provide any sort of constraint satisfaction between the extents of the parts. For example, the following decomposition is not legal because it constrains the split sizes to be equal with respect to unbound `nat` parameters:

$$
\begin{array}{l}
(* \text{ Not allowed! } *) \\
fubar[\![\texttt{nat } m, \texttt{nat } n]\!](X : \mathbb{R}^{m \times n}) : () = \texttt{do} \\
\quad (* \text{ p and q unbound } *) \\
\quad [\, \mathrm{A}_{p \times q} \ \mathrm{B}_{p \times q} \\
\quad\ \mathrm{C}_{p \times q} \ \mathrm{D}_{p \times q} \,] = X \\
\quad \ldots \\
\texttt{end}
\end{array}
$$

To get this effect, the programmer should compute the constrained values:

$$
\begin{array}{l}
fubar[\![\texttt{nat } m, \texttt{nat } n]\!](X : \mathbb{R}^{2m \times 2n}) : () = \texttt{do} \\
\quad [\, \mathrm{A}_{m \times n} \ \mathrm{B}_{m \times n} \\
\quad\ \mathrm{C}_{m \times n} \ \mathrm{D}_{m \times n} \,] = X
\end{array}
$$

```
  ...
end
```

Some non-uniform unpastings can be obtained with composition, which can be expressed either by repeated unpasting:

$$
\begin{array}{l}
unequalRows[\![\texttt{nat}\ m, \texttt{nat}\ n]\!](X : R^{4m \times 2n}) = \texttt{do} \\
\quad [\, c1_{4m \times n}\ c2_{4m \times n}\,] = X \\
\quad [\, \mathrm{A}_{m \times n} \\
\quad\ \mathrm{C}_{3m \times n}\,] = c1 \\
\quad [\, \mathrm{B}_{3m \times n} \\
\quad\ \mathrm{D}_{m \times n}\,] = c2 \\
\quad \ldots \\
\texttt{end}
\end{array}
$$

or simply by nesting matrices in the unpasting:

$$
\begin{array}{l}
unequalRows[\![\texttt{nat}\ m, \texttt{nat}\ n]\!](X : R^{2m \times 4n}) = \texttt{do} \\
\quad [\,[\, \mathrm{A}_{m \times n} \mathrm{B}_{m \times 3n}] \\
\quad\ [\, \mathrm{C}_{m \times 3n} \mathrm{D}_{m \times n}\,]\,] = X \\
\quad \ldots \\
\texttt{end}
\end{array}
$$

Chapter 9

Functions

> DRAFT NOTE
> Abstract function declarations and their checking, modifiers on functions such as `atomic` and `io`, keyword and varargs parameters, where clauses, contract checking, and tail-call optimization are not supported yet.
>
> Varargs and keyword examples and abstract function declarations examples are not run by the interpreter.

> DRAFT NOTE
> Guy's email titled "Re: Function equality issues" on 02/13/08
>
> Yes with returning false for overloaded functions (with the implementation-dependent features). (03/03/08)

A *function* is a value that has a function type (described in Section 6.7). Each function takes exactly one argument, which may be a tuple, and returns exactly one result, which may be a tuple. A function may be declared as top level or local as described in Section 7.1. Fortress allows functions to be *overloaded* (as described in Chapter 15); there may be multiple function declarations with the same function name in a single lexical scope. Functions can be passed as arguments and returned as values. Single variables may be bound to functions including overloaded functions.

9.1 Function Declarations

Syntax:

FnDecl	::=	*FnMods*? *FnHeaderFront FnHeaderClause* = *Expr*
	|	*FnSig*
FnSig	::=	*SimpleName* : *Type*
FnMods	::=	$FnMod^+$
FnMod	::=	*AbsFnMod* | `private`
AbsFnMod	::=	*LocalFnMod* | `test`
LocalFnMod	::=	`atomic` | `io`
FnHeaderFront	::=	*NamedFnHeaderFront*
	|	*OpHeaderFront*
NamedFnHeaderFront	::=	*Id StaticParams*? *ValParam*
ValParam	::=	*BindId*
	|	(*Params*?)
Params	::=	(*Param*,)* (*Varargs*,)? *Keyword*(, *Keyword*)*
	|	(*Param*,)* *Varargs*
	|	*Param*(, *Param*)*
VarargsParam	::=	*BindId*: *Type*...
Varargs	::=	*VarargsParam*
Keyword	::=	*Param*= *Expr*
PlainParam	::=	*BindId IsType*?
	|	*Type*
Param	::=	*PlainParam*
FnHeaderClause	::=	*IsType*? *FnClauses*
FnClauses	::=	*Throws*? *Where*? *Contract*
Throws	::=	`throws` *MayTraitTypes*

Syntactically, a function declaration consists of an optional sequence of modifiers followed by the name of the function, optional static parameters (described in Chapter 12), the value parameter with its (optionally) declared type, an optional type of a return value, an optional declaration of thrown checked exceptions (discussed in Chapter 14), an optional `where` clause (discussed in Section 12.6), a contract for the function (discussed in Section 9.4), and finally an optional body expression preceded by the token $=$. A `throws` clause does not include naked type variables. Every element in a `throws` clause is a subtype of $\mathrm{CheckedException}$. When a function declaration includes a body expression, it is called a *function definition*. Function declarations can be mutually recursive.

Function declarations can include the following special modifiers:

`atomic:` A function with the modifier `atomic` acts as if its entire body were surrounded in an `atomic` expression discussed in Section 13.23.

`io:` Functions that perform externally visible input/output actions are said to be `io` functions. An `io` function must not be invoked from a non-`io` function.

A function takes exactly one argument, which may be a tuple. When a function takes a tuple argument, we abuse terminology by saying that the function takes multiple arguments. Value parameters cannot be mutated inside the function body.

A function's value parameter consists of a parenthesized, comma-separated list of bindings where each binding is one of:

- A plain binding "$identifier$" or "$identifier : T$"
- A varargs binding "$identifier : T \ldots$"
- A keyword binding "$identifier = e$" or "$identifier : T = e$"

When the parameter is a single plain binding without a declared type, enclosing parentheses may be elided. The following restrictions apply: No two bindings may have the same identifier. No keyword binding may precede a plain binding. No varargs binding may follow a keyword binding or precede a plain binding. Note that it is permitted to have a single plain binding, or to have no bindings. The latter case, "()", is considered equivalent to a single plain binding of the ignored identifier "_" of type (), that is, "$(_ : ())$". Also, there can be at most one varargs binding.

A parameter declared by keyword binding is called a *keyword parameter*; a keyword parameter must be declared with a *default* expression, which is used when no argument is bound to the parameter explicitly. Syntactically, the default expression is specified after an $=$ sign. The default expression of a parameter x of function f is evaluated each time the function is called without a value provided for x at the call site. All parameters occurring to the left of x are in scope of its default expression. All parameters following x must include default expressions as well; x is in scope of their default expressions and the body of the function. When an argument is passed explicitly for a keyword parameter, that argument must be passed as a *keyword argument*. (See Section 9.2.) If no type is declared for a keyword parameter, the type is inferred from the static type of its default expression.

A parameter declared by varargs binding is called a *varargs parameter*; it is used to pass a variable number of arguments to a function as a single heap sequence. The type of a varargs parameter is $\mathrm{HeapSequence}[\![T]\!]$ where T is the type mentioned in (or inferred for) that binding.

> DRAFT NOTE
>
> See the library section for a discussion of $\mathrm{HeapSequence}$.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.37
>
> No library declares $\mathrm{HeapSequence}$, nor the immutable sequence that the team's draft note at the start of Section 9.2 agreed on in its place. Both implementations give a varargs parameter the array type of the Fortress standard libraries that the compiled type checker's authors named for it, $\mathrm{ImmutableArray}[\![T, \mathbb{Z}32]\!]$: the interpreter binds the parameter to an immutable one-dimensional array of the arguments, and the compiled type checker types the parameter in the body at that type. The compiled path's own library declares no such type, so there the compiled type checker refuses every use of a varargs parameter, in a body, a contract or a function expression, naming the type; and the code generator does not yet compile a function with a varargs parameter.

Note that the type of a varargs parameter cannot be omitted. If a function does not have a varargs parameter then the number of arguments is fixed by the function's type. Note that a varargs parameter is not allowed to have a default expression.

For example, here is a simple polymorphic function for creating lists:

$\mathrm{List}[\![T\ \texttt{extends}\ \mathrm{Object}, \texttt{nat}\ \mathit{length}]\!](\mathit{rest}: T_{\mathit{length}}) =$

$\texttt{if}\ \mathit{length} = 0\ \texttt{then}\ \mathrm{Empty}$

$\texttt{else}\ \mathrm{Cons}\big(\mathit{rest}_0, \mathrm{List}(\mathit{rest}_{1:(\mathit{length}-1)})\big)$

$\texttt{end}$

9.2 Function Applications

> DRAFT NOTE
>
> varargs arguments (10/30/07)
>
> Is there any reason to use $\mathrm{HeapSequence}$ which is mutable to implement varargs tuples? How about implementing

them in terms of some covariant data type?

We agreed to implement varargs tuples using $\mathrm{ReadOnlyHeapSequence}$ which is immutable and invariant. We'll add $\mathrm{MakeReadOnlyHeapSequence}$ into the core library.

Fortress allows functions to be overloaded; there may be multiple function declarations with the same function name in a single lexical scope. Thus, we need to determine which functional declarations are applicable to a function application.

An overloaded function has multiple arrow types in its function type, and it associates a declaration with each constituent arrow type. When an overloaded function of type T is called with an argument of type A, the call is "dispatched" to the declaration associated with the most specific type of T applicable to A (if no such type exists, then the function is not applicable to A). This is well defined because the overloading rules guarantee that the type of any function value is a well-formed function type.

If a function's argument type is $()$, then function declarations with the following forms of parameter lists are considered to be applicable:

- $()$ which means the same thing as $(_ : ())$
- $(x : ())$ which is something programmers don't ordinarily write
- $(x : T \ldots)$

In the last case, x is bound to an empty $\mathrm{HeapSequence}[\![T]\!]$.

If a function's argument type A is neither $()$ nor a tuple type, then function declarations with the following forms of parameter lists are considered to be applicable:

- $(x : T)$ where A is a subtype of T
- $(x : T \ldots)$ where A is a subtype of T

In the last case, x is bound to a $\mathrm{HeapSequence}[\![T]\!]$ of length 1, containing the actual argument value.

If a function's argument type A is a tuple type, then function declarations with the following forms of parameter lists are considered to be applicable:

- $(x : T)$ where A is a subtype of T
- $(x : T \ldots)$ where A is a subtype of T
- a parameter list with no varargs binding, provided that
  - type A has exactly as many plain types as the parameter list has plain bindings, and
  - for every keyword-type pair (described in Section 6.5) in A, the parameter list has a binding with the same keyword, and
  - for every element type in A, the type in the element type is a subtype of the type of the corresponding binding in the parameter list.
- a parameter list with a varargs binding, provided that
  - type A has at least as many plain types as the parameter list has plain bindings, and
  - for every keyword-type pair in A, the parameter list has a binding with the same keyword, and
  - for every element type in A, the type in the element type is a subtype of the type of the corresponding binding in the parameter list—but if there is no corresponding binding, then the type in the element type must be a subtype of the type in the varargs binding.

In the latter case, the parameter named by the identifier in the varargs binding is bound to a $\mathrm{HeapSequence}[\![T]\!]$ that contains, in order, all the values of the tuple that did not correspond to plain bindings, followed by all the values in the varargs $\mathrm{HeapSequence}$ of the tuple, if any.

When an argument is passed explicitly for a keyword parameter, that argument must be passed as a *keyword argument*. Syntactically, a keyword argument is a keyword-value pair "$identifier = e$". Keyword parameters not explicitly bound are bound to their default values. If a parameter that has no default value is not explicitly bound to an argument, it is a static error.

When a function is called (See Section 13.6 for a discussion of function call expressions), function arguments are evaluated in parallel, keyword parameters not explicitly bound are bound to their default values sequentially, and the body of the function is evaluated in a new environment, extending the environment in which it is defined with all parameters bound to their arguments.

If the application of a function f ends by calling another function g, tail-call optimization must be applied. Storage used by the new environments constructed for the application of f must be reclaimed.

Here are some examples:

$\sin(\pi)$

$\arctan(y, x)$

If the function's argument is not a tuple, then the argument need not be parenthesized:

$\sin\ x$

$\log\ \log\ n$

Here are a few varargs and keyword examples:

$f(x : \mathbb{Z}, y : \mathbb{Z} \ldots, z : \mathbb{Z} = 0) = \langle x, \langle q \mid q \leftarrow y \rangle, z \rangle$

$f(1)$	*returns*	$\langle 1, \langle\rangle, 0 \rangle$
$f(1, 2, 3)$	*returns*	$\langle 1, \langle 2, 3 \rangle, 0 \rangle$
$f(1, [2\ 3] \ldots)$	*returns*	$\langle 1, \langle 2, 3 \rangle, 0 \rangle$
$f(1, 2, 3, [4\ 5] \ldots)$	*returns*	$\langle 1, \langle 2, 3, 4, 5 \rangle, 0 \rangle$
$f(1, 2, 3, 17 \# 3 \ldots)$	*returns*	$\langle 1, \langle 2, 3, 17, 18, 19 \rangle, 0 \rangle$
$f(1, 2, 3, z = 8)$	*returns*	$\langle 1, \langle 2, 3 \rangle, 8 \rangle$
$f([2\ 3] \ldots)$	*declaration not applicable*	

9.3 Abstract Function Declarations

Syntax:

AbsFnDecl	::=	*AbsFnMods*? *FnHeaderFront FnHeaderClause*
	|	*FnSig*
AbsFnMods	::=	*AbsFnMod*$^+$
AbsFnMod	::=	*LocalFnMod* | `test`
LocalFnMod	::=	`atomic` | `io`
FnSig	::=	*Name* : *ArrowType*

In a component, overloaded functions may be declared separately from their definitions using *abstract function declarations*. Abstract function declarations themselves may be overloaded. For an abstract function declaration with name f, argument type T, and return type U (T and U may be tuple types), any concrete declaration for f must be for a function whose argument type is a subtype of T and whose return type is a subtype of U. Furthermore, the argument

types of the concrete declarations for f must together cover T (Section 6.10). Concrete function declarations must satisfy the overloading rules. Unless T has a `comprises` clause (or is a tuple type, at least one of whose entries has a `comprises` clause), this implies that some concrete declaration for f must have argument type T.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.33
>
> The Working Draft of February 2011 said that the union of the argument types of the concrete declarations must be equal to T; under a `comprises` clause read at the level of values (Section 10.1) the union of the listed types covers T without being equal to it.

Syntactically, an abstract function declaration is a function declaration without a body. Parameter names may be elided but parameter types cannot be omitted. Additionally, when a function's type is not parameterized, Fortress provides an alternative mathematical notation for an abstract function declaration: function name followed by the token ':', followed by an arrow type.

For example, after the following abstract function declaration:

$$printMolecule(\text{Molecule}) : ()$$

where trait Molecule is defined as follows:

`trait` Molecule `comprises` {OrganicMolecule, InorganicMolecule} `end`

the programmer could write:

$$printMolecule(molecule : \text{Molecule}) = \ldots$$

or could write:

$$printMolecule(molecule : \text{OrganicMolecule}) = \ldots$$
$$printMolecule(molecule : \text{InorganicMolecule}) = \ldots$$

For the latter, the programmer must provide a definition for every type listed in the `comprises` clause of Molecule, or definitions that together cover those types, or it is a static error.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.33
>
> The Working Draft of February 2011 asked here for a definition for every immediate subtype of Molecule. Where a trait with static parameters stands unlisted between a trait with a `comprises` clause and its listed types, the immediate subtypes and the listed types differ; the type group's calculus Welterweight Fortress (2012) asks for definitions for all the types comprised. The compiled type checker checks, at each object declaration, that the object's concrete methods cover the abstract methods it inherits, and reads `comprises` clauses to do so: concrete methods over the types that a trait lists together cover an abstract method over that trait; top-level abstract function declarations run on neither implementation of the revival.

9.4 Function Contracts

> DRAFT NOTE
>
> For nested contracts, the inner `outcome` shadows the outer `outcome`.

Syntax:

Contract	::=	*Requires*? *Ensures*? *Invariant*?
Requires	::=	`requires` { *ExprList*? }
Ensures	::=	`ensures` { *EnsuresClauseList*? }
EnsuresClauseList	::=	*EnsuresClause*(, *EnsuresClause*)*
EnsuresClause	::=	*Expr* (`provided` *Expr*)?
Invariant	::=	`invariant` { *ExprList*? }

Function contracts consist of three optional clauses: a `requires` clause, an `ensures` clause, and an `invariant` clause. All three clauses are evaluated in the scope of the function body.

> DRAFT NOTE
> extended with a special variable *arg*, bound to an immutable array of all function arguments. (This array is useful for describing contracts in the presence of higher-order functions; see Section 6.6).

> DRAFT NOTE
> Jan: Which of the following must be pure? Presumably all of them. We should say so.
>
> Eric: Requiring declared purity on all called functions in a contract is too restrictive.

The `requires` clause consists of a sequence of expressions of type $\mathrm{Boolean}$ separated by commas and enclosed in curly braces. The `requires` clause is evaluated during a function call before the body of the function. If any expression in a `requires` clause does not evaluate to true, a $\mathrm{CallerViolation}$ exception is thrown.

The `ensures` clause consists of a sequence of `ensures` subclauses. Each such subclause consists of an expression of type $\mathrm{Boolean}$, optionally followed by a `provided` subclause. A `provided` subclause begins with `provided` followed by an expression of type $\mathrm{Boolean}$. For each subclause in the `ensures` clause of a contract, the `provided` subclause is evaluated immediately after the `requires` clause during a function call (before the function body is evaluated). If a `provided` subclause evaluates to true, then the expression preceding this `provided` subclause is evaluated after the function body is evaluated. If the expression evaluated after function evaluation does not evaluate to true, a $\mathrm{CalleeViolation}$ exception is thrown. The expression preceding the `provided` subclause can refer to the return value of the function. A `outcome` variable is implicitly bound to a return value of the function and is in scope of the expression preceding the `provided` subclause. The implicitly declared `outcome` shadows any other declaration with the same name in scope.

The `invariant` clause consists of a sequence of expressions of *any type* enclosed by curly braces. These expressions are evaluated before and after a function call. For each expression e in this sequence, if the value of e when evaluated before the function call is not equal to the value of e after the function call, a $\mathrm{CalleeViolation}$ exception is thrown.

Here are some examples:

$\mathit{factorial}(n\colon \mathbb{Z}64)$ `requires` $\{\, n \geq 0 \,\} =$

  `if` $n = 0$ `then` 1

  `else` $n\, \mathit{factorial}(n - 1)$

  `end`

$\mathit{mangle}(\mathit{input}\colon \mathrm{List})$ `ensures` $\{\, \mathit{sorted}(\mathit{outcome})$ `provided` $\mathit{sorted}(\mathit{input}) \,\} =$

  `if` $\mathit{input} \neq \mathrm{Empty}$

  `then` $\mathit{mangle}\big(\mathit{first}(\mathit{input})\big)$

  $\quad\quad \mathit{mangle}\big(\mathit{rest}(\mathit{input})\big)$

  `end`

Overloaded function contracts are handled similarly with method contracts described in Section 10.4. In particular, substitutability is preserved: the statically most applicable function to a call should be substitutable with the dynamically most applicable function to the call. For a call of function f, we use the term *static contract* of f to refer to a contract declared in the statically most applicable function declaration and the term *dynamic contract* of f to refer to

a contract declared in the dynamically most applicable function declaration. Three exceptions may be thrown due to an overloaded function contract violation: CallerViolation is thrown when the `requires` clause of the static contract fails, CalleeViolation is thrown when the `ensures` or `invariant` clause of the dynamic contract fails, and ContractOverloadingViolation is thrown when the `requires` clause of the dynamic contract or the `ensures` or `invariant` clause of the static contract fails.

Evaluation of a call of function f proceeds as follows. Let C and C' be the static and dynamic contracts of f, respectively. If the `requires` clause of C fails, a CallerViolation exception is thrown. Otherwise, if the `requires` clause of C' fails, a ContractOverloadingViolation exception is thrown. Otherwise, the `provided` subclauses of C and C' are evaluated. For every `provided` subclause that evaluates to $true$, the corresponding `ensures` subclause is recorded in a table E for later comparison. Similarly, the `invariant` clauses of C and C' are evaluated and the results are stored in E for later comparison. Then the body of the dynamically most applicable function declaration of f is evaluated. After evaluation of the body, all `ensures` subclauses of the dynamic contract recorded in E are checked to ensure that they evaluate to $true$, and all `invariant` clauses of the dynamic contract recorded in E are checked to ensure that they evaluate to values equal to the values they evaluated to before evaluation of the body. If any such check fails, a CalleeViolation exception is thrown. Otherwise, all `ensures` subclauses and `invariant` clauses of the static contract in E are checked. If any of these checks fails, a ContractOverloadingViolation exception is thrown.

9.5 Local Function Declarations

Syntax:

LocalFnDecl	::=	*LocalFnMods*? *NamedFnHeaderFront FnHeaderClause* = *Expr*
LocalFnMods	::=	*LocalFnMod*$^+$
LocalFnMod	::=	`atomic` | `io`

Functions can be declared within expression blocks (described in Section 13.11) via the same syntax as is used for top-level function declarations (described in Chapter 9) except that locally declared functions must not be operators and must not include the modifiers `private` and `test`. As with top-level function declarations, locally declared functions in a single scope are allowed to be overloaded and mutually recursive.

Chapter 10

Traits

> DRAFT NOTE
> Where clauses, the `override` modifier, value traits, method contracts, and abstract functional declarations are not yet supported.

> DRAFT NOTE
> Covariant list with `comprises` clause (Victor's email titled "Problem with covariant List?" on 10/27/07)

Traits are declared by trait declarations. Traits define new named types. A trait specifies a collection of *methods* (described in Section 10.2). One trait can extend others, which means that it inherits the methods from those traits, and that the type defined by that trait is a subtype of the types of traits it extends.

10.1 Trait Declarations

Syntax:

TraitDecl ::= *TraitMods*? *TraitHeaderFront TraitClauses GoInATrait*? `end` (`trait` ? *Id*)?
TraitHeaderFront ::= `trait` *Id StaticParams*? *ExtendsWhere*?
TraitClauses ::= *TraitClause**
TraitClause ::= *Excludes*
| *Comprises*
| *Where*
GoInATrait ::= *Coercions*? *GoFrontInATrait GoBackInATrait*?
| *Coercions*? *GoBackInATrait*
| *Coercions*
Coercions ::= *Coercion*$^+$
GoFrontInATrait ::= *GoesFrontInATrait*$^+$
GoesFrontInATrait ::= *AbsFldDecl*
| *GetterSetterDecl*
| *PropertyDecl*
GoBackInATrait ::= *GoesBackInATrait*$^+$
GoesBackInATrait ::= *MdDecl*
| *PropertyDecl*
TraitMods ::= *TraitMod*$^+$
TraitMod ::= *AbsTraitMod* | `private`
AbsTraitMod ::= `value` | `test`
ExtendsWhere ::= `extends` *TraitTypeWheres*
TraitTypeWheres ::= *TraitTypeWhere*
| { *TraitTypeWhereList* }
TraitTypeWhereList ::= *TraitTypeWhere*(, *TraitTypeWhere*)*
TraitTypeWhere ::= *TraitType Where*?

Excludes ::= `excludes` *TraitTypes*
Comprises ::= `comprises` *TraitTypes*
TraitTypes ::= *TraitType*
| { *TraitTypeList* }
TraitTypeList ::= *TraitType*(, *TraitType*)*

TraitType ::= *TypeRef*
| *Type* `[` *ArraySize*? `]`
| *Type* `^` *IntExpr*
| *Type* `^` `(` *ExtentRange* (× *ExtentRange*)* `)`
ArraySize ::= *ExtentRange*(, *ExtentRange*)*
ExtentRange ::= *StaticArg*?`#` *StaticArg*?
| *StaticArg*?: *StaticArg*?
| *StaticArg*
StaticArgs ::= ⟦*StaticArgList* ⟧
StaticArgList ::= *StaticArg*(, *StaticArg*)*
StaticArg ::= *Op*
| Unity
| dimensionless
| 1 / *Type*
| *DimPrefixOp Type*
| *IntExpr*
| *BoolExpr*
| *Type*
| *UnitExpr*

Syntactically, a trait declaration starts with an optional sequence of modifiers followed by `trait`, followed by the name of the trait, an optional sequence of static parameters (described in Chapter 12), an optional set of *extended* traits, an optional set of *excluded* traits, an optional set of *comprises* on the trait, an optional `where` clause (described in Section 12.6), zero or more *coercion* declarations (discussed in Chapter 17), zero or more declarations of abstract fields, getter and setter methods, and zero or more declarations of methods separated by newlines or semicolons, that are not getters or setters, and finally `end`. A trait declaration may end with an optional `trait` and its name again. It is a static error if the identifier after `end` is not the name of the trait being declared. Property declarations (described in Section 19.6) can be freely commingled with the declarations of methods and abstract fields. An `extends` clause comes first, if any, and `excludes`, `comprises`, and `where` clauses come in any order.

Each of `extends`, `excludes`, and `comprises` clauses consists of `extends`, `excludes`, and `comprises` respectively followed by a set of trait references separated by commas and enclosed in braces '{' and '}'. If such a clause contains only one trait, the enclosing braces may be elided. In an API (but not a component), a `comprises` clause may include "...". A trait reference listed in the `comprises` clause is a declared trait identifier (within the same component or an API imported by the component).

> DRAFT NOTE
>
> Victor: Why is this property stated only for a comprises clause? Doesn't it apply to extends and excludes clauses as well? Also, do we really mean "trait identifier" above? Where is that term defined? Does it include instantiations of parametric traits?

A trait with an `extends` clause *explicitly extends* those traits listed in its `extends` clause.

> DRAFT NOTE
>
> Victor: It is possible for a trait to explicitly extend itself using hidden type variables. In particular, the straightforward interpretation of covariant/contravariant declarations has this property. (But we could just rule this a special case, since it is special in other ways already.)

> REVISED BY THE 2026 REVIVAL SEE Section I.1.2
>
> The revival rules on this note and on the one below: a `where`-clause variable may not appear as a static argument in an `extends` clause (Section 12.6), so no trait extends itself through a hidden type variable and explicit extension has no self cycles.

In addition, every trait except Any, Object and Index implicitly extends the trait Object if it does not do so explicitly. The trait Index of the library, the trait of the types that a subscript takes, extends only Any, as Object does: a tuple type whose element types are all subtypes of Index is a subtype of Index (Section 6.5), and no tuple is an Object. A trait that extends Index and no other trait but Any implicitly extends Object too. We define the extension relation to be the transitive closure of implicit and explicit extension. That is, trait T *extends* trait U if and only if T explicitly or implicitly extends U or if there is some trait S that T explicitly extends and that extends U. The extension relation induced by a program is the smallest relation satisfying these conditions. This relation must form an acyclic hierarchy rooted at the traits Object and Index.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.42
>
> The Working Draft of February 2011 named only Any and Object as the traits that do not implicitly extend Object, and rooted the hierarchy at Object; Index, which the revival adds to the library, is the third.

> DRAFT NOTE
>
> Victor: Technically, there may be self cycles in explicit extension due to hidden type variables, as described above.

If a trait T extends trait U, or if T and U are the same trait, we say that T is a subtrait of U and that U is a supertrait of T.

We say that trait T *strictly extends* trait U if and only if (*i*) T extends U and (*ii*) T is not U. We say that trait T *immediately extends* trait U

> DRAFT NOTE
> Victor: I think we don't use "immediately extends" anywhere in the spec, only "immediate supertype/subtype".

if and only if (*i*) T strictly extends U and (*ii*) there is no trait V such that T strictly extends V and V strictly extends U. We call U an *immediate supertrait* of T and T an *immediate subtrait* of U.

> DRAFT NOTE
> Victor: this belongs in the type chapter.

A trait with an `excludes` clause excludes every trait listed in its `excludes` clause. If a trait T excludes a trait U, the two traits are mutually exclusive: neither can extend the other, and no trait can extend them both. As discussed in Section 6.2, the exclusion relation is symmetric. Thus, if T excludes U then U excludes T whether or not T is listed in an `excludes` clause in the declaration of U.

If a trait declaration of T includes a `comprises` clause then every type listed in its `comprises` clause extends T, and every value of type T is a value of one of the listed types. A listed type may be an instantiation of a parameterized trait, provided that every static variable that occurs in its static arguments is a static parameter of T or a `where`-clause variable of the declaration of T (Section 12.6), which the listed type then holds at every value of: a trait declared `trait` $\mathrm{NatParam}$ `comprises` $\{N[\![n]\!]\}$ `where` $[\![$`nat` $n]\!]$ lists $N[\![n]\!]$ at every `nat` value of n. A trait or object that explicitly extends T must be a subtype of a listed type, unless it is a trait with a `comprises` clause of its own each of whose listed types meets this requirement, or a trait with static parameters such that at least one trait or object extends it and every trait or object that extends it is a subtype of a listed type. If T has static parameters, these statements hold of each instantiation of T, with the static parameters of T in the listed types replaced by the corresponding static arguments.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.21
> The Working Draft of February 2011 said here, in a draft note, that the traits listed in a `comprises` clause are exactly the traits that immediately extend T and that each of them must list T in its `extends` clause. The revival states the reading of the team's later Types chapter (Luchangco, 2012) and of the type group's calculus Welterweight Fortress (2012): the clause says to which types the values of T belong. A parameterized trait with a static parameter that T does not have can be listed only at particular static arguments; it may stand unlisted between T and the listed types, adding no value outside them, when at least one trait or object extends it and every one that does is a subtype of a listed type. This also answers the third question of the draft note on trait identifiers above, whether an instantiation of a parameterized trait may be listed: it may. A `where`-clause variable of the declaration of T may also occur in a listed type, as in the clause of the trait $\mathrm{NatParam}$ of the Fortress standard libraries, which the team wrote as a comment and the revival restores.

> DRAFT NOTE
> If the `comprises` clause of a trait T includes "..." (and must therefore be in an API), then in a component exporting the enclosing API, other traits may explicitly extend T, but these traits may not be declared or imported by the API (see Section 22.3).

A trait declaration of T that has no `comprises` clause, or one that lists no type but "...", may have a *self-type parameter*: a static parameter X of T whose bounds include T at its own static parameters, as in `trait` $T[\![X$ `extends` $T[\![X]\!]]\!]$, or include an instantiation of a trait with a self-type parameter that T also lists in its `extends` clause, with X as the

static argument for that self-type parameter. In the bodies of the methods of T, `self` then has the type $T \cap X$ (Section 13.2). A trait or object declaration, or an object expression, that extends an instantiation of T must be a subtype of the static argument of that instantiation for the self-type parameter of T; for a trait declaration the requirement is on the type of `self` in the bodies of its methods (Section 13.2), the intersection of its type with the types its `comprises` clause lists, or with its own self-type parameter Y. It is a static error otherwise. An object declaration has no self-type parameter. For example, given the declarations:

```
trait Shape⟦S extends Shape⟦S⟧⟧ end
object Point extends Shape⟦Point⟧ end
```

the following object declaration is not allowed:

```
(* Not allowed! *)
object Mark extends Shape⟦Point⟧ end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.41
>
> The self-type parameter and the rule on the declarations that extend a trait with one are new. The Working Draft of February 2011 typed `self` as the type of the enclosing trait, or, with a `comprises` clause, as its intersection with the listed types (Section 13.2). The revival also reads the type of `self` from the bound of a self-type parameter, and refuses, as Scala refuses a class whose self type does not conform to the self types of the classes it inherits, a declaration that does not conform to it, so that inside T every value of `self` is an X.

For example, the following trait declaration:

```
trait Catalyst extends Object

  catalyze(reaction: Reaction): ()

end
```

declares a trait Catalyst with no modifiers, no static parameters, and no `excludes` clauses, no `comprises` clauses, and no `where` clauses. Trait Catalyst extends a trait named Object. A single method (named *catalyze*) is declared, which has a parameter of type Reaction and the return type ().

The following example trait:

```
trait Molecule comprises { OrganicMolecule, InorganicMolecule }
  mass(): Mass
end
```

comprises of two traits: OrganicMolecule and InorganicMolecule. Therefore, the following trait declaration is not allowed:

```
(* Not allowed! *)
trait ExclusiveMolecule extends Molecule end
```

Traits OrganicMolecule and InorganicMolecule may be exclusive:

```
trait OrganicMolecule extends Molecule excludes InorganicMolecule end

trait InorganicMolecule extends Molecule end
```

OrganicMolecule and InorganicMolecule exclude each other, even though only OrganicMolecule has an `excludes` clause. For example, the following trait declaration is not allowed:

```
(* Not allowed! *)
trait InclusiveMolecule extends { InorganicMolecule, OrganicMolecule } end
```

For the same reason, a trait or object may not extend two instantiations of one parameterized trait that exclude each other by instantiation exclusion (Section 6.3), directly or through its supertraits: any two instantiations of one parameterized trait that it extends must have the same static arguments, operator arguments excepted. For example, the following object declaration is not allowed:

```
trait Parent⟦T⟧ end
(* Not allowed! *)
object Child extends { Parent⟦ℤ32⟧, Parent⟦String⟧ } end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.1
>
> The rule on declarations is new. The Working Draft of February 2011 allowed a trait to extend two different instantiations of one parameterized trait; the compiled type checker refuses such a declaration when the arguments that differ are types or numerals (Section I.1.1 states the exceptions), reporting that the two instantiations exclude each other. The team's later Types chapter (Luchangco, 2012) states the rule as: a trait that extends two instantiations of a generic type, directly or indirectly, must extend an expressible instantiation that is a subtype of both. Where every static parameter is invariant, as in this specification, that is the rule stated above.

A trait is allowed to have multiple immediate supertraits. The following trait has two immediate supertraits:

```
trait Enzyme extends { OrganicMolecule, Catalyst } end
```

10.2 Method Declarations

> DRAFT NOTE
>
> hidden without settable is a static error only within traits not within objects !!! – Sukyoung

> DRAFT NOTE
>
> Accessing a field by a "naked" identifier reference within trait declarations are not allowed. We require a dotted field access by a "self." prefix. (08/11/08) – Sukyoung

> DRAFT NOTE
>
> Getters must always be called with the special syntax: $receiver.getterName$
>
> A getter is required to have a type consistent with a corresponding setter.

Syntax:

MdDecl	::=	*MdDef*
	|	`abstract` ? *MdMods*? `getter` ? *MdHeaderFront FnHeaderClause*
MdDef	::=	*MdMods*? `getter` ? *MdHeaderFront FnHeaderClause* $=$ *Expr*
AbsMdDecl	::=	`abstract` ? *AbsMdMods*? `getter` ? *MdHeaderFront FnHeaderClause*
MdMods	::=	$MdMod^+$
MdMod	::=	*FnMod* | `override`
FnMod	::=	*AbsFnMod* | `private`
AbsFnMod	::=	*LocalFnMod* | `test`
LocalFnMod	::=	`atomic` | `io`
AbsMdMods	::=	$AbsMdMod^+$
AbsMdMod	::=	*AbsFnMod* | `override`
GetterSetterDecl	::=	*GetterSetterDef*
	|	`abstract` ? *FnMods*? *GetterSetterMod MdHeaderFront FnHeaderClause*
GetterSetterDef	::=	*FnMods*? *GetterSetterMod MdHeaderFront FnHeaderClause* $=$ *Expr*
GetterSetterMod	::=	`getter` | `setter`
AbsGetterSetterDecl	::=	`abstract` ? *AbsFnMods*? *GetterSetterMod MdHeaderFront FnHeaderClause*
MdHeaderFront	::=	*NamedMdHeaderFront*
	|	*OpMdHeaderFront*
NamedMdHeaderFront	::=	*Id StaticParams*? *MdValParam*
MdValParam	::=	(*MdParams*?)
MdParams	::=	$(MdParam,)^*$ $(Varargs,)$? $MdKeyword(,MdKeyword)^*$
	|	$(MdParam,)^*$ *Varargs*
	|	$MdParam(,\ MdParam)^*$
MdKeyword	::=	*MdParam*$=$ *Expr*
MdParam	::=	*Param*
	|	`self`
Param	::=	*PlainParam*
PlainParam	::=	*BindId IsType*?
	|	*Type*
FnHeaderClause	::=	*IsType*? *FnClauses*
IsType	::=	: *Type*
FnClauses	::=	*Throws*? *Where*? *Contract*
Throws	::=	`throws` *MayTraitTypes*
MayTraitTypes	::=	$\{\}$
	|	*TraitTypes*

A trait declaration contains a set of method declarations. Syntactically, a method declaration begins with an optional sequence of modifiers followed by the method's name, optional static parameters (described in Chapter 12), the value parameter with its (optionally) declared type, an optional type of a return value, an optional declaration of thrown checked exceptions (discussed in Chapter 14), an optional `where` clause (discussed in Section 12.6), a contract for the method (discussed in Section 10.4), and finally an optional body expression preceded by the token $=$. If a method declaration has the `getter` modifier, it does not affect the behavior of a program; rather, it documents that the method is intended to serve as a getter. A `throws` clause does not include naked type variables. Every element in a `throws` clause is a subtype of $\mathrm{CheckedException}$.

Method declarations can include the following special modifiers:

`getter:` A method declaration with the modifier `getter` explicitly declares a getter method for a field, even in the absence of an actual field declaration.

> DRAFT NOTE
> The following description is too confusing. Reword it!!!

If such a field exists, there is no implicit getter for the field. An explicitly declared getter method must take no arguments and return a result of the field's type. A getter method must not throw any checked exception. Getter names may not overlap ordinary method names. A getter method must be invoked with the field access syntax:

$expr.id$

where id is the name of the getter method.

`setter:` A method declaration with the modifier `setter` explicitly declares a setter method for a field, even in the absence of an actual field.

> DRAFT NOTE
> The following description is too confusing. Reword it!!!

If such a field exists, there is no implicit setter for the field. An explicitly declared setter method must take a single argument—the value being set—and return $()$. A setter method must not throw any checked exception. Setter names may not overlap ordinary method names. A setter method must be invoked with the *assignment syntax*:

$expr_1.id := expr_2$

We say that a method declaration *occurs* in a trait declaration. A trait declaration *declares* a method declaration that occurs in that trait declaration. A trait declaration *inherits* method declarations from the declarations of its immediate supertraits except those method declarations that are *overridden* (as described later in this section) or whose parameter type (not including the type of the self parameter) is equal to the parameter type of a method declaration that occurs in the trait declaration. A trait declaration *provides* the method declarations that it declares or inherits.

There are two sorts of method declarations: *dotted method* declarations and *functional method* declarations. Syntactically, a dotted method declaration is identical to a function declaration. When a method is invoked, a special self parameter is bound to the object on which it is invoked.

A functional method declaration has a parameter named `self` at an arbitrary position in its parameter list. This parameter is not given a type and implicitly has the type of the enclosing trait or object.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.41
> In a trait T with a self-type parameter X (Section 10.1), this remains the type of the `self` parameter in the declaration's signature, which overloading and the choice of a declaration at a call read, save the test of whether two overloaded declarations' parameter types exclude each other, which reads it as $T \cap X$; in the method's body `self` has the type $T \cap X$ (Section 13.2).

Semantically, functional method declarations can be viewed as top-level function declarations. For example, the following overloaded functional method f declared within a trait declaration A:

`trait` A

$f(\texttt{self}, t : T) = e_1$

$f(s : S, \texttt{self}) = e_2$

`end`

$f(a, t)$

may be rewritten as top-level functions as follows:

`trait` A

$internalF(t:T) = e_1$

$internalF(s:S) = e_2$

`end`

$f_1(a:A, t:T) = a.internalF(t)$

$f_2(s:S, a:A) = a.internalF(s)$

$f_1(a, t)$

where $internalF$ is a freshly generated name. Functional method declarations may be overloaded with top-level function declarations. An abstract function declaration (described in Section 9.3) can be provided also for overloaded functional method declarations. See Chapter 15 for a discussion of overloaded functionals in Fortress.

When a method declaration includes a body expression, it is called a *method definition*. A method declaration that does not have its body expression is referred to as an *abstract method declaration*. An abstract method declaration declares an *abstract method*; any object inheriting an abstract method must define a body expression for the method: a method definition with the same name and the self parameter (if any) at the same position whose parameter type, not counting the type of the self parameter, is that of the abstract method declaration, or definitions whose parameter types are subtypes of it and together cover it (Section 6.10), or it is a static error.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.33
>
> The Working Draft of February 2011 said only that any object inheriting an abstract method must define a body expression for the method. The object defines the method by a body at its parameter type, or by bodies at narrower parameter types that together cover it, `comprises` clauses read at the level of values (Section 10.1), as Section 9.3 asks of the concrete declarations for an abstract function declaration; one body at a narrower parameter type that does not cover the abstract method's does not define it. This is the rule of the type group's compiled type checker (2012) and of its calculus Welterweight Fortress (2012). The requirement of Section 11.1 that an object define all abstract methods inherited from its supertraits is unchanged: it defines them in this sense.

An abstract method declaration may include the modifier `abstract`. It may elide parameter names but parameter types cannot be omitted except for the self parameter.

> DRAFT NOTE
>
> Method declarations that are overridden by some declaration are not accessible. They are not considered for overloading checks.
>
> Partial overriding of an overloaded functional is allowed but partial overriding of a functional declaration is not allowed.

A method declaration may have the modifier `override`. Such a declaration *overrides* any inherited declaration with the same name and the self parameter (if any) at the same position, whose parameter type, not counting the type of the self parameter, is a strict subtype of the overriding declaration's parameter type. The return type of the overriding declaration must be subtype of that of the overridden declaration to preserve type safety. A declaration with the modifier `override` must override some inherited declaration. It is a static error if a declaration with the modifier `override` does not override any inherited declaration.

Here is an example trait Enzyme which provides methods $reactionSpeed$ and $catalyze$:

```
trait Enzyme extends { OrganicMolecule, Catalyst }
  reactionSpeed(): Speed
  catalyze(reaction) = reaction.accelerate(reactionSpeed())
end
```

Enzyme inherits the abstract method *mass* from OrganicMolecule, declares the abstract method *reactionSpeed*, and declares the concrete method *catalyze* which is inherited as an abstract method from its supertrait Catalyst.

10.3 Abstract Field Declarations

Syntax:

AbsFldDecl	::=	*AbsFldMods*? *AbsFldWTypes*
	|	*AbsFldMods*? *BindIdOrBindIdTuple* : *Type*...
	|	*AbsFldMods*? *BindIdOrBindIdTuple* : *TupleType*
AbsFldWTypes	::=	*AbsFldWType*
	|	(*AbsFldWType*(, *AbsFldWType*)$^+$)
AbsFldWType	::=	*BindId IsType*
AbsFldMods	::=	*AbsFldMod*$^+$
AbsFldMod	::=	*ApiFldMod* | `wrapped` | `private`
ApiFldMod	::=	`hidden` | `settable` | `test`

Traits may also include abstract field declarations that are implicit declarations of abstract getter methods. Syntactically, an abstract field declaration consists of an optional sequence of modifiers followed by the field name, followed by the token ': ', and the type of the field.

By default, a field declaration implicitly declares a getter method for the field unless there is an explicit getter declared in the enclosing trait. An implicit getter method takes no arguments, has the same name as the field, and has a return type equal to the field type. When called, the implicit getter returns the value of the field when called.

Abstract field declarations can include the following special modifiers:

`hidden:` A field declaration with the modifier `hidden` has no implicit getter method.

`settable:` A field declaration with the modifier `settable` has an implicit setter method unless there is an explicit setter declared in the enclosing trait. An implicit setter method takes a parameter (with no default expression) whose type is the type of the field, and returns `()`. When called, the implicit setter rebinds the corresponding field to its argument. If a field declaration includes the modifier `settable` and `hidden`, only an abstract setter is declared. If a field declaration includes the modifier `hidden` without `settable`, it is a static error. If an abstract field declaration includes the modifier `settable`, it is mutable. Because the `settable` modifier implies `var`, and the `var` modifier without `settable` only affects implementations, abstract field declarations do not include the `var` modifier.

`wrapped:` If a field declaration of f has the modifier `wrapped` and the type of f is trait type T, and T is not a naked type variable, then the enclosing trait S implicitly includes "forwarding methods" for all methods in T that are also inherited from any supertrait of S. Each of these methods simply calls the corresponding method on the trait referred to by field f. If the trait declaration enclosing f explicitly declares a method m that conflicts with an implicitly declared forwarding method m', then the enclosing trait contains only method m, not m'. If the trait declaration enclosing f inherits a concrete method m that conflicts with an implicitly declared forwarding method m', then the enclosing trait

contains only method m, not m'. Because wrapped fields do not change declarations of methods but change definitions of methods, they only affect implementations; APIs do not include wrapped fields.

For example, in the following declarations:

```
trait Dictionary⟦T⟧
    put(T): ()
    get(): T
end
trait WrappedDictionary⟦T⟧ extends Dictionary⟦T⟧
    wrapped val: Dictionary⟦T⟧
    get(): T
end
```

the parametric trait WrappedDictionary implicitly includes the following forwarding method:

$$put(x) = val.put(x)$$

If get were not explicitly declared in WrappedDictionary, then WrappedDictionary would also include the forwarding method:

$$get() = val.get()$$

> DRAFT NOTE
> Wrapped fields imply subtyping.
>
> what methods are implicitly defined?
>
> ```
> trait T
>     f(x) = 17
>     abstract h(x)
> end
> object A extends T
>     f(x) = 23
>     g(x) = 5
>     h(x) = 3
> end
> object B extends T
>     wrapped a: A
> end
> f(x) = (self asif T).f(x)
> h(x) = a.h(x)
> trait U
>     abstract g(x)
> end
> object B extends {T, U}
>     wrapped a: AERROR!
> end
> ```

10.4 Method Contracts

Syntax:

Contract ::= *Requires*? *Ensures*? *Invariant*?
Requires ::= `requires` { *ExprList*? }
Ensures ::= `ensures` { *EnsuresClauseList*? }
EnsuresClauseList ::= *EnsuresClause*(, *EnsuresClause*)*
EnsuresClause ::= *Expr* (`provided` *Expr*)?
Invariant ::= `invariant` { *ExprList*? }

Method contracts consist of three optional clauses: a `requires` clause, an `ensures` clause, and an `invariant` clause. All three clauses are evaluated in the scope of the method body. See Section 9.4 for a discussion of each clause.

Method contracts are handled similarly to the manner described in [10]. In particular, substitutability under subtyping is preserved. For a call to a method m with receiver e, we use the term *static contract* of m to refer to a contract declared in the statically most applicable method declaration provided by the static type of e and the term *dynamic contract* of m to refer to a contract declared in the dynamically most applicable method declaration provided by the runtime type of e. Three exceptions may be thrown due to a method contract violation: $\mathrm{CallerViolation}$ is thrown when the `requires` clause of the static contract fails, $\mathrm{CalleeViolation}$ is thrown when the `ensures` or `invariant` clause of the dynamic contract fails, and $\mathrm{ContractHierarchyViolation}$ is thrown when the `requires` clause of the dynamic contract or the `ensures` or `invariant` clause of the static contract fails.

Evaluation of a call to a method m with receiver e proceeds as follows. First, e is evaluated to a value v with runtime type U. Let C and C' be the static and dynamic contracts of m, respectively. If the `requires` clause of C fails, a $\mathrm{CallerViolation}$ exception is thrown. Otherwise, if the `requires` clause of C' fails, a $\mathrm{ContractHierarchyViolation}$ exception is thrown. Otherwise, the `provided` subclauses of C and C' are evaluated. For every `provided` subclause that evaluates to $true$, the corresponding `ensures` subclause is recorded in a table E for later comparison. Similarly, the `invariant` clauses of C and C' are evaluated and the results are stored in E for later comparison. Then the body of m provided by U is evaluated. After evaluation of the body, all `ensures` subclauses of the dynamic contract recorded in E are checked to ensure that they evaluate to $true$, and all `invariant` clauses of the dynamic contract recorded in E are checked to ensure that they evaluate to values equal to the values they evaluated to before evaluation of the body. If any such check fails, a $\mathrm{CalleeViolation}$ exception is thrown. Otherwise, all `ensures` subclauses and `invariant` clauses of the static contract in E are checked. If any of these checks fails, a $\mathrm{ContractHierarchyViolation}$ exception is thrown.

10.5 Value Traits

Syntax:

TraitMod ::= `value`

If a trait declaration has the modifier `value`, all subtraits of that trait must also have the modifier `value`, and all objects extending that trait are required to be value objects (described in Section 11.3). If a field declaration of a value trait has the modifier `settable`, the return type of its implicit setter method is the value trait type. If a value trait has an explicit setter method, the setter must be an abstract method and its return type must be the value trait type. See Section 11.3 for a discussion of updating fields of value objects.

Chapter 11

Objects

> DRAFT NOTE
> Value objects and properties are not yet supported.

Objects are values that have object trait types described in Section 6.3.1. Objects contain *methods* (described in Section 10.2) and *fields* (described in Section 11.2).

An object is a *value object*, a *reference object*, or a *function object*: It is a function object if it has an arrow type, a reference object if it has an object trait type that is not declared with the `value` modifier (see Section 11.3), and a value object otherwise (i.e., if it has a tuple type, the type (), or an object trait type declared with the `value` modifier).

A value object cannot have mutable fields, and it is completely determined by its type, environment and its fields: Value objects with the same type, environment and fields are indistinguishable. Thus, an implementation may freely copy value objects. Most objects with simple standard types, such as booleans, numeric literals, IEEE floating-point numbers, and integers are value objects. In contrast, reference objects are thought to "reside in memory", and are identified by an *object reference*. A new object reference is created whenever a reference object is constructed, so that reference objects constructed separately are always distinct. What about reference objects that have no mutable fields? Reference objects include arbitrary-precision numbers and aggregates such as arrays, lists and sets. Function objects are immutable and have no fields. Identity is not well-defined for function objects, and attempting to check whether two functions are equivalent returns an approximate result. Section 11.4 describes object equivalence in further detail.

11.1 Object Declarations

> DRAFT NOTE
> The `transient` modifier and varargs parameters are eliminated.

> DRAFT NOTE
> What does it mean to have an invariant on an object?
>
> Christine: The easy answer is that on entrance and exit of every method the invariant is true. Unfortunately I could be in the middle of a method with the invariant violated and want to call another method.
>
> Guy: One might argue that invariants should hold on entry and exit of methods that appear in the API. This can be approximated as: invariants must hold before and after every method that is not marked private.

> Need a more concrete proposal.
>
> Does the modifier `atomic` on a functional check its contract also?

Syntax:

ObjectDecl	::=	*ObjectMods*? *ObjectHeader GoInAnObject*? `end` (`object` ? *Id*)?
ObjectHeader	::=	`object` *Id StaticParams*? *ObjectValParam*? *ExtendsWhere*? *FnClauses*
ObjectMods	::=	*TraitMods*
ObjectValParam	::=	`(` *ObjectParams*? `)`
ObjectParams	::=	(*ObjectParam* `,`)* (*ObjectVarargs* `,`)? *ObjectKeyword*(`,` *ObjectKeyword*)*
	|	(*ObjectParam* `,`)* *ObjectVarargs*
	|	*ObjectParam* (`,` *ObjectParam*)*
ObjectVarargs	::=	`transient` *Varargs*
ObjectKeyword	::=	*ObjectParam*`=` *Expr*
ObjectParam	::=	*ParamFldMods*? *Param*
	|	`var transient` *ParamWType*
	|	`transient` ? `var` *ParamWType*
	|	`transient` *Param*
ParamWType	::=	*BindId IsType*?
ParamFldMods	::=	*ParamFldMod*+
ParamFldMod	::=	| `hidden` | `settable` | `wrapped`
GoInAnObject	::=	*Coercions*? *GoFrontInAnObject GoBackInAnObject*?
	|	*Coercions*? *GoBackInAnObject*
	|	*Coercions*
Coercions	::=	*Coercion*+
GoFrontInAnObject	::=	*GoesFrontInAnObject*+
GoesFrontInAnObject	::=	*FldDecl*
	|	*GetterSetterDef*
	|	*PropertyDecl*
GoBackInAnObject	::=	*GoesBackInAnObject*+
GoesBackInAnObject	::=	*MdDef*
	|	*PropertyDecl*
FnClauses	::=	*Throws*? *Where*? *Contract*
Throws	::=	`throws` *MayTraitTypes*

Object declarations declare both object values and object trait types. Object declarations extend a set of traits from which they inherit methods. An object declaration inherits the concrete methods of its supertraits and must include a definition for every method declared but not defined by its supertraits. Especially, an object declaration must not include abstract methods (discussed in Section 10.2); it must define all abstract methods inherited from its supertraits. It is also allowed to define overloaded declarations of concrete methods inherited from its supertraits.

Syntactically, an object declaration begins with an optional sequence of modifiers followed by `object`, followed by the identifier of the object, optional static parameters (described in Chapter 12), optional value parameters, optional traits the object extends, an optional declaration of thrown checked exceptions (discussed in Chapter 14), an optional `where` clause (discussed in Section 12.6), a contract for the object (discussed in Section 9.4), zero or more declarations of fields, getter methods, and setter methods, and zero or more declarations of methods, that are not getter or setter, separated by newlines or semicolons, and finally `end`. An object declaration may end with an optional `object` and its name again. It is a static error if the identifier after `end` is not the name of the object being declared. Property declarations (described in Section 19.6) can be freely commingled with the field and method declarations. Method declarations in object declarations are syntactically identical to method declarations in trait declarations. If an object declaration has no `extends` clause, the object implicitly extends only trait Object. A `throws` clause does not include naked type variables. Every element in a `throws` clause is a subtype of CheckedException. A contract of an object declaration is evaluated when the object is created, as with a function contract (see Section 9.4).

There are two kinds of object declarations: singleton declarations and constructor declarations. A singleton declaration declares a sole, stand-alone, singleton object. It may have static parameters but it does not have a list of value parameters; every instantiation of such an object with the same static arguments yields the same singleton object. A constructor declaration declares an object constructor function. It may have static parameters and it includes a list of value parameters; every call to the constructor of such an object with the same argument yields a new object. Initialization of singleton objects is nondeterministic as described in Chapter 5. The type of an object constructor function is an arrow type consisting of the object's value parameter type followed by the token $\rightarrow$, followed by the object trait type.

A constructor declaration has a (possibly empty) parenthesized list of value parameters. Each value parameter of a constructor declaration may be preceded by the special modifier `transient` or by a sequence of field modifiers. A value parameter preceded by the modifier `transient` does not correspond to a field in an instantiation of the object: `transient` parameters are not in scope in the object's method declarations, but are in scope when the initial-value expressions of the fields are evaluated during object creation.

For example, the following leaf object extending trait Tree:

`object` Leaf `extends` Tree

  $printTree(): () = println$ "`leaf`"

  `opr` $|$`self`$| : \mathbb{Z}32 = 0$

`end`

has no fields and two methods.

Here is an example of a Cons object constructor extending trait $\text{List}[\![T]\!]$:

`object` $\text{Cons}[\![T]\!]\big(first : T, rest : \text{List}[\![T]\!]\big)$

  `extends` $\text{List}[\![T]\!]$

  $cons(x) = \text{Cons}(x, \texttt{self})$

  $append(xs) = \text{Cons}\big(first, rest.append(xs)\big)$

`end`

Note that this declaration introduces the "factory" function $\text{Cons}[\![T]\!](first : T, rest : \text{List}[\![T]\!]) : \text{Cons}[\![T]\!]$ which is used in the body of the object declaration to define the $cons$ and $append$ methods. Multiple factory functions can be defined by overloading an object constructor with functions. For example:

$\text{Cons}[\![T]\!](firstitem : T) = \text{Cons}(firstitem, \text{Empty})$

11.2 Field Declarations

Syntax:

FldDecl ::= *FldMods*? *FldWTypes InitVal*
| *BindIdOrBindIdTuple* = *Expr*
| *FldMods*? *BindIdOrBindIdTuple* : *Type*... *InitVal*
| *FldMods*? *BindIdOrBindIdTuple* : *TupleType InitVal*
FldMods ::= *FldMod*$^+$
FldMod ::= `var` | *AbsFldMod*
AbsFldMod ::= *ApiFldMod* | `wrapped` | `private`
ApiFldMod ::= `hidden` | `settable` | `test`
FldWTypes ::= *FldWType*
| (*FldWType*(, *FldWType*)$^+$)
FldWType ::= *BindId IsType*

> DRAFT NOTE
> I think this should be phrased differently. When I originally read that, I took it to mean that fields were always "private." – Scott

Fields are variables local to an object. They must not be referred to outside their enclosing object declarations. Each field is either a non-transient value parameter of a constructor declaration, or it is explicitly defined by a field declaration within the body of a constructor declaration, a singleton declaration, or an object expression (described in Section 13.9). A field declaration in an object declaration is syntactically identical to a top-level variable declaration (described in Section 8.1), with the same meanings attached to the form of variable declarations. Additional modifiers apply to fields, as described in Section 10.3. Unlike with an abstract field in a trait declaration, an object declaration may declare a `hidden` field which is not `settable`.

Each field declaration includes an *initial-value* expression, which specifies the initial value of that field. Field initialization occurs in textual program order: evaluation of each initial-value expression must complete before evaluation of the next initial-value expression, and all previous field names (and the parameters of the constructor, in a constructor declaration) are in scope when evaluating an initial-value expression (see Section 7.3). All fields of an object are initialized before that object is made available for subsequent computation; thus, it is illegal to invoke methods on an object being initialized: `self` is *not* implicitly declared by a field declaration.

> DRAFT NOTE
> Victor: Is the following code legal:
>
> ```
> trait T
>   x:ℤ
>   f() = object extends T
>           x = self.g()(* self here is bound by f *)
>         end
>   g() = 4
> end
> ```
>
> What about:
>
> ```
> trait T
>   x:ℤ
>   f() = object extends T
>           x = g()
>         end
>   g() = 4
> end
> ```

> DRAFT NOTE
> Victor: This may not belong here, but not sure where!

Within an object declaration or object expression, a field can be accessed by a "naked" identifier reference (see Section 13.2). Unlike within trait declarations, such a reference does *not* invoke the getter or setter method of that name. To invoke a getter or setter of that name, it is necessary to use dotted field access (see Section 13.3).

11.3 Value Objects

An object declaration with the modifier `value` declares a value object that is called in many languages a *primitive* value. The object trait type declared by a value object implicitly has the modifier `value`.

The fields of a `value` object are immutable; they cannot be changed directly or it is a static error. However, Fortress allows value objects to have settable fields as an abbreviation for constructing a new value object with a different value for one field. If a value object has a setter method or a subscripted assignment operator method (described in Section 25.8), then the return type of the method must be the value object trait type instead of `()`. When such a method is invoked, the receiver must itself be assignable, and the value returned by the method is assigned to the receiver.

For example, here is a value object Complex number:

```
value object Complex(settable real : Double, settable imaginary : Double = 0)
  opr +(self, other : Complex) = Complex(real + other.real, imaginary + other.imaginary)
end
```

When a mutable variable z:

```
var z : Complex = Complex(0)
```

updates its $imaginary$ field, the following syntax:

$$z.imaginary := v$$

can be used as an abbreviation for:

$$z := \text{Complex}(z.real, v)$$

So the setter for the field $imaginary$ in Complex would do the work of constructing and returning $\text{Complex}(z.real, v)$, and the assignment:

$$z.imaginary := v$$

would be construed to mean:

$$z := z.imaginary(v)$$

Note that modifying a settable field directly within the value object is not allowed. For example, the following:

$$imaginary := 3$$

within the Complex object means:

$$\texttt{self}.imaginary := 3$$

and because `self` is not mutable, the assignment is disallowed.

11.4 Object Equivalence

The trait Any declares the object equivalence operator $\equiv$. This operator is automatically defined for all objects; it is a static error for the programmer to override it. The $\equiv$ operator is used to decide whether its two arguments refer to "the same object" in the strictest sense possible. If the arguments have different dynamic types—including the instantiations of all static parameters—the result is always $false$. If both arguments are value objects with the same type, then the result is $true$ if and only if corresponding fields of the objects are themselves equivalent as defined by this operator; in particular, two binary words are strictly equivalent if and only if they contain the same bit pattern. If both arguments are object references, then the result is $true$ if and only if the two object references refer to the identical reference object (in implementation terms, occupying the same memory locations in the heap). If both arguments are functions, the result is $true$ only if the functions behave identically for any choice of type-correct arguments. Even if two functions behave identically, the fortress implementation is free to return $false$ when they are compared for object equivalence.

Chapter 12

Static Parameters

> DRAFT NOTE
> Of the non-type static parameters, `nat` and `int` parameters are supported by the interpreter and the compiled path, and so are static expressions in a size, as far as Section 13.27 notes; the compiled type checker does not infer a `bool`, `dim` or `unit` static argument.
>
> The examples in this chapter are not tested nor run by the interpreter.

Trait, object, and functional declarations may be parameterized with *static parameters*. Static parameters are *static variables* listed in white square brackets ⟦ and ⟧ immediately after the name of a trait, object, or functional and they are in scope of the entire body of the declaration. Static parameters may be instantiated with static expressions discussed in Section 13.27. In this chapter, we describe the forms that these static parameters can take.

Syntax:

StaticParams ::= ⟦*StaticParamList*⟧
StaticParamList ::= *StaticParam*(, *StaticParam*)*

12.1 Type Parameters

Syntax:

StaticParam ::= *Id Extends*? (`absorbs unit`)?

Static parameters may include one or more type parameters. Syntactically, a type parameter consists of an identifier followed by an optional `extends` clause, followed by an optional "`absorbs unit`" clause (described in Section 26.4). If a type parameter does not have an `extends` clause, it has an implicit "`extends` Any" clause.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.30
> The Working Draft of February 2011 gave such a type parameter the implicit clause "`extends` Object". Any holds every value, tuples and () among them, which Object does not (the team's later Types chapter, Luchangco, 2012); the Fortress standard libraries are written under Any, instantiating such parameters at tuple types, and the interpreter reads the bound so. The type group's paper of 2019 says the same: "By default, each type parameter has an upper bound Any" (Park, Hong, Steele and Ryu, POPL 2019). The compiled path still gives such a type parameter of a program's own declarations the bound Object (row 412 of the revival's gap ledger), a divergence that lasts until the compiled path is switched over to the interpreter's library.

Type parameters are instantiated with types such as trait types, tuple types, and arrow types (See Chapter 6 for a discussion of Fortress types). We use the term *naked type variable* to refer to an occurrence of a type variable as

a stand-alone type (rather than as a parameter to another type). Type parameters can appear in any context that an ordinary type can appear, except that a naked type variable must not appear in the `extends` clause of a trait or object declaration, nor in the `throws` clause of a functional or object declaration, nor as the type of a `wrapped` field (discussed in Section 11.2).

Here is a parameterized trait List:

```
trait List⟦T⟧
  first(): T
  rest(): List⟦T⟧
  cons(T): List⟦T⟧
  append(List⟦T⟧): List⟦T⟧
end
```

12.2 Nat and Int Parameters

Syntax:

StaticParam	::=	`nat` *Id*
	|	`int` *Id*

Static parameters may include one or more `nat` and `int` parameters. Syntactically, a `nat` parameter consists of `nat` followed by an identifier. An `int` parameter consists of `int` followed by an identifier. These parameters are instantiated at runtime with numeric values. A `nat` parameter may be used to instantiate other `nat` parameters, or to appear in any context that a variable of type $\mathbb{N}32$ can appear, except that it cannot be assigned to. An `int` parameter may be used to instantiate other `int` parameters, or to appear in any context that a variable of type $\mathbb{Z}32$ can appear, except that it cannot be assigned to. An argument whose type is a trait whose `comprises` clause lists a type at a `nat` or `int` `where`-clause variable (Section 10.1), passed where that listed type is expected with a `nat` or `int` parameter in the variable's place, instantiates that parameter, for that call, with the size of the value itself, a size known to equal only itself: where `trait` NatParam `comprises` $\{N⟦n⟧\}$ `where` ⟦`nat` n⟧ is declared, a NatParam passed to $f⟦$`nat` $m⟧(x: N⟦m⟧)$ instantiates m with the size of the N that it is.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.21
>
> The Working Draft of February 2011 did not say how a size known only at run time reaches a `nat` parameter. The sentence above states the rule of the team's comment on the trait NatParam of the Fortress standard libraries: "within that function n becomes a static nat parameter".

For example, the following function *makeVector*:

makeVector⟦T `extends` Number, `nat` s_0⟧(): Vector⟦T, s_0⟧ = *vector*⟦T, s_0⟧

declares a `nat` parameter s_0, which appears in both the parameter type and return type of *makeVector*.

12.3 Bool Parameters

Syntax:

StaticParam	::=	`bool` *Id*

Static parameters may include one or more `bool` parameters. Syntactically, a `bool` parameter consists of `bool` followed by an identifier. These parameters are instantiated at runtime with boolean values. They may be used to instantiate other `bool` parameters, or to appear in any context that a variable of type Boolean can appear, except that they cannot be assigned to.

For example, the following `coerce` declared in the trait Boolean:

```
trait Boolean
  coerce ⟦bool b⟧(x: BooleanLiteral⟦b⟧) = f(x)
end
```

declares a `bool` parameter b, which appears in the parameter type.

12.4 Dimension and Unit Parameters

Syntax:

StaticParam ::= `dim` *Id*
| `unit` *Id* (: *Type*)? (`absorbs unit`)?

Static parameters may include one or more `dim` and `unit` parameters. Syntactically, a `dim` parameter begins with `dim` followed by an identifier. A `unit` parameter begins with `unit` followed by an identifier, optionally followed by the token ': ' and a dimension, and the unit is thereby restricted to be a unit of the specified dimension. A `unit` parameter may include the clause "`absorbs unit`"; the meaning of this is described in Section 26.4. A `dim` parameter is allowed to appear in any context that a dimension can appear. A `unit` parameter is allowed to appear in any context that a unit can appear.

For example, here is a function (in this case actually a prefix operator definition) that is parameterized with a unit:

> DRAFT NOTE
> I manually added a space before U – Sukyoung

```
opr √⟦unit U⟧(x: ℝ64 U²): ℝ64 U = numericalsqrt(x/U²) U
```

12.5 Operator Parameters

Syntax:

StaticParam ::= `opr` *Op*

Static parameters may include one or more operator names. Syntactically, an operator parameter begins with `opr` followed by an operator name.

Operator parameters may be freely intermixed with other static parameters. For example, the following trait IdentityOp:

```
trait IdentityOp⟦T extends IdentityOp⟦T, ⊙⟧, opr ⊙⟧
  opr ⊙(self): T = self
end
```

is parameterized with a type parameter T and an operator parameter $\odot$. Unlike other static parameters, operator parameters may be used in both type context and value context. The operator parameter $\odot$ is declared as the second static parameter of $\mathrm{IdentityOp}$, instantiated as a static argument in $\mathrm{IdentityOp}[\![T, \odot]\!]$ which is the bound of T, and declared as an operator method in $\mathrm{IdentityOp}$. If a trait or object has an operator parameter `OP` and uses `OP` in an expression, then the trait or object must declare or inherit at least one operator method for `OP` of matching fixity. (That is because it would be impossible to type-check T with any global definition for `OP`, because the actual operator name is not known yet, and a definition has to come from somewhere.)

Operator parameters are instantiated with operators. They may be used to instantiate other operator parameters and the names of operator declarations. For example, the following trait $\mathrm{MyIdentity}$:

```
(* BROKEN... object MyIdentity extends IdentityOp[\MyIdentity, IDENTITY\] end
*)
```

instantiates the operator parameter of its supertrait $\mathrm{IdentityOp}$ with the operator name `IDENTITY`. It inherits the `IDENTITY` operator from $\mathrm{IdentityOp}$. Two restrictions apply to instantiations of operator parameters: 1) If a trait or object T has operator parameters $\texttt{OP}_1$ and $\texttt{OP}_2$, and T declares or inherits operator methods of the same fixity for both $\texttt{OP}_1$ and $\texttt{OP}_2$, then the actual operators passed in any instantiation of T must be different. (That's because it would be impossible to type check any code in trait T that uses either $\texttt{OP}_1$ or $\texttt{OP}_2$ in an expression if such aliasing were not forbidden.) Note that the clause "of the same fixity" allows both $\texttt{OP}_1$ and $\texttt{OP}_2$ to be given the same actual operator, say "$-$", if $\texttt{OP}_1$ has only a prefix definition and $\texttt{OP}_2$ has only an infix definition; this is desirable. 2) If a trait or object T has an operator parameter `OP` and declares or inherits an operator method for `OP`, and also declares or inherits an operator method of the same fixity for any actual operator $@$ (that is, where $@$ is not an operator parameter), then the actual operator passed for the `OP` parameter must not be $@$. (That's because it would be impossible to type check any code in trait T that uses the operator $@$ in an expression if such aliasing were not forbidden.)

Note that any declarations that may be associated with an actual operator name that is passed for an operator parameter are *irrelevant* to the behavior of the operator parameter within the parameterized trait, object, or functional. When a trait or object extends a trait parameterized by operator names, the subtrait inherits methods whose names are the actual operator names instead of the operator parameter names.

An operator method declaration whose name is one of the operator parameters of its enclosing trait or object may be overloaded with other operator declarations in the same component; the operator parameter may be instantiated with any operator in the same component. Therefore, such an operator method declaration must satisfy the overloading rules (described in Chapter 15) with every operator declaration in the same component. For example, the above `IDENTITY` operator declaration is overloaded with the following `IDENTITY` operator declaration:

$$\texttt{opr IDENTITY}(x\colon \mathbb{Z}) = x$$
$$\texttt{opr} +(x\colon \mathbb{Z}) = x$$

where both `IDENTITY` and $+$ are top-level operator declarations in the same component. Therefore, the declaration of the operator $\odot$ in $\mathrm{IdentityOp}$ must satisfy the overloading rules with `IDENTITY` and with $+$.

Here are the things programmers can do with an operator parameter:

(1) Use it as the defined name in an operator declaration. This is what trait $\mathrm{BinaryOperator}[\![T, \odot]\!]$ (described in Section 40.3) does.

(2) Pass it as an actual operator to another trait that takes an operator parameter. This is what $\mathrm{Associative}[\![T, \odot]\!]$ (described in Section 40.3) does when it extends $\mathrm{BinaryOperator}[\![T, \odot]\!]$.

(3) Use it to distinguish various objects. This is what $\mathrm{Identity}[\![\odot]\!]$ (described in Section 40.3) does.

(4) Use it as an operator in an expression. To do this, an operator declaration for that operator parameter must be accessible at the ponit of use, and for that to happen, the parameterized trait must either declare an operator method with that operator parameter as the name, as in case (1), or inherit such a declaration, and the only way to do the latter is to use case (2).

Many interesting examples are described in Section 40.3.

12.6 Where Clauses

> DRAFT NOTE
> The `where` clause syntax in this section is out of date.

Syntax:

Where	::=	`where` { *WhereClauseList* }
WhereClauseList	::=	*WhereClause*(, *WhereClause*)*
WhereClause	::=	*Id Extends*
	|	*TypeAlias*
	|	*NatConstraint*
	|	*IntConstraint*
	|	*BoolConstraint*
	|	*UnitConstraint*
	|	*TypeRef* `coerces` *TypeRef*
	|	*TypeRef* `widens` *TypeRef*

Static parameters may have constraints placed on them in a `where` clause. A `where` clause begins with `where`, followed by a sequence of static parameter constraints enclosed in braces { and }, and separated by commas.

A `where` clause may introduce new static variables, i.e., identifiers for types and other static entities that may not be static parameters. We use the term `where`*-clause variables* to refer to static variables that are not also static parameters. The `where`-clause variables must be bound in a `where` clause.

A static parameter constraint is one of the following forms:

- a trait constraint consisting of the identifier of a naked type variable, followed by `extends` followed by a set of trait references which may include naked type variables,
- a type alias (described in Section 6.11),
- an arithmetic constraint,
- a boolean constraint,
- a unit equality constraint,
- a `coerces` constraint (described in Section 17.2), or
- a `widens` constraint (described in Section 17.2).

A `where` clause may include mutually recursive constraints. All static variables in a trait, object, or functional declaration must occur either as a static parameter or as a `where`-clause variable. Appendix A.2 describes a Fortress core calculus with `where` clauses.

An entry of an `extends` clause may carry a `where` clause of its own (Section 10.1). If that clause introduces no `where`-clause variable and all its constraints are trait constraints, the declared trait or object extends the entry's type exactly where the constraints hold: an instance whose static arguments satisfy them is of that type, and an instance whose static arguments do not satisfy them is not. The constraints hold while the entry itself, the declaration's value parameters and its members are checked, as if they were in the declaration's own `where` clause. The rest of the declaration holds at every instance and is checked without them: its static parameters, its own `where` clause, its `throws` clause and contract, its `excludes` and `comprises` clauses, and the other entries of its `extends` clause. So a list is totally ordered exactly where its elements are:

```
trait List⟦E⟧ extends {ZeroIndexed⟦E⟧,
      LexicographicOrder⟦List⟦E⟧, E⟧ where {E extends StandardTotalOrder⟦E⟧}}
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.3
>
> This paragraph and its example are new. The Working Draft of February 2011 gave each entry of an `extends` clause an optional `where` clause in its grammar and stated no meaning for it, and its front matter lists conditional extension among the features not yet supported.

A `where`-clause variable may not appear as a static argument in an `extends` clause. Such an occurrence would make the declared trait or object a subtype of one instantiation for each type that the variable may stand for, which instantiation exclusion forbids (Section 6.3 and Section 10.1); the second of the three declarations below shows why.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.2
>
> This restriction is new. The three declarations that follow were offered as legal by the Working Draft of February 2011; they are kept as examples of what is not allowed, each with the way to write it.

In particular, a trait declaration may not extend other instantiations of itself through a `where`-clause variable. The following declaration would express that the static parameter S of C is covariant, making $C⟦S⟧$ a subtype of $C⟦T⟧$ for every supertype T of S; it is not allowed:

```
(* Not allowed! *)
trait C⟦S⟧ extends C⟦T⟧
  where {S extends T, T extends Object}
end
```

Static parameters are invariant: $C⟦S⟧$ is a subtype of $C⟦T⟧$ only if S and T are the same type. A value of one instantiation is widened to another by a function whose bounded static parameters state that one argument extends the other:

```
trait C⟦S⟧
  getter item(): S
end
object CItem⟦S⟧(item: S) extends C⟦S⟧ end
widen⟦T, S extends T⟧(c: C⟦S⟧): C⟦T⟧ = CItem⟦T⟧(c.item)
```

so that, where a trait Dog extends a trait Animal and d is a $C⟦\text{Dog}⟧$, $widen⟦\text{Animal}, \text{Dog}⟧(d)$ is a $C⟦\text{Animal}⟧$ with the same item. The library's covariant collections are written this way.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.4
>
> The Working Draft of February 2011 said that trait declarations are allowed to extend other instantiations of themselves, and gave the declaration above as the way to express covariance. The type group's later design declares variance instead with a `covariant` modifier on a static parameter (the team's later Types chapter, Luchangco, 2012), which is implemented on neither the interpreter nor the compiled path; the revival's decision record and row 404 of its gap ledger say what is known of it.

Trait declarations need not have any static parameters in order to have a `where` clause. But the following trait declaration is not allowed:

```
(* Not allowed! *)
trait C extends D⟦T⟧
  where {T extends Object}
```

```
end
```

In this declaration, trait C would be a subtrait of *every* instantiation of parametric trait D. Thus, trait C would have all of the methods of every instantiation of D. By thinking of the declaration this way, we can see what restrictions we need to impose on the trait C in order for it to be sensible. If trait C inherits a method declaration that refers to T, it really contains infinitely many methods (one for each instantiation of T). However, instantiations of the `where`-clause variables are not explicit from the program text as static parameters are. It must be possible to infer which method is referred to at the call site. If there is not enough information to infer which method is called, type checking rejects the program and requires more type information from the programmer. Programmers always can provide more type information by using type ascription as described in Section 13.31. But trait C would also be a subtype of every pair of different instantiations of D, which exclude each other by instantiation exclusion, so no such restriction makes the declaration sensible: that is why it is not allowed.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.5
>
> The Working Draft of February 2011 said that this declaration is legal. The paragraph above, the team's own account of why such a trait is dangerous, is kept as the reason for the restriction stated at the start of this section.

Object or functional declarations may include `where` clauses, under the same restriction. The following declaration of an Empty list is not allowed:

```
(* Not allowed! *)
object Empty extends List⟦T⟧ where {T extends Object}
  first() = throw Error
  rest() = throw Error
  cons(x) = Cons(x, self)
  append(xs) = xs
end
```

where Cons is declared in Section 11.1. Instead, the empty list is one object for each element type, and each use writes the element type, as in Empty⟦ℤ32⟧:

```
object Empty⟦T⟧ extends List⟦T⟧
  first(): T = throw NotFound
  rest(): List⟦T⟧ = throw NotFound
  cons(x: T): List⟦T⟧ = Cons⟦T⟧(x, self)
  append(xs: List⟦T⟧): List⟦T⟧ = xs
end
```

The library declares its empty collections and its empty optional value (Section 39.2) the same way.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.6
>
> The Working Draft of February 2011 gave the declaration marked "Not allowed" as its example of a `where` clause on an object. It makes Empty a List⟦T⟧ for every T at once, which instantiation exclusion forbids. The declaration after it is the library's own shape; it needs the element type written at each use until the static arguments of a generic object can be inferred.

Chapter 13

Expressions

Fortress is an expression-oriented language. The positions in which an expression may legally appear (*value context*) are determined by the nonterminal *Expr* in the Fortress grammar defined in Appendix G. We say that an expression is a *subexpression* of any expression (or any other program construct) that (syntactically) contains it. When evaluation of one subexpression must complete before another subexpression is evaluated, those subexpressions are ordered by dynamic program order (see Chapter 5). This constrains the memory behavior of program constructs, as described in Chapter 21. Unless otherwise specified, abrupt completion of the evaluation of a subexpression causes the evaluation of the expression as a whole to complete abruptly in the same way. Also, if one expression precedes another by dynamic program order, and the evaluation of the first expression completes abruptly, the second is not evaluated at all.

13.1 Literals

> DRAFT NOTE
> We need to check whether the various types and values defined in this section are still correct.

Syntax:

LiteralExpr ::= ( )
| *NumericLiteralExpr*
| *CharLiteralExpr*
| *StringLiteralExpr*

Fortress provides boolean literals, () literal, character literals, string literals, and numeric literals. Literals are values; they do not require evaluation.

> DRAFT NOTE
> From here, copied from F1.0β.

The literal $false$ has type $\mathrm{BooleanLiteral}[\![false]\!]$. The literal $true$ has type $\mathrm{BooleanLiteral}[\![true]\!]$.

> DRAFT NOTE
> And these are the only values of those types.

The literal `()` is the only value with type `()`. Whether any given occurrence of `()` refers to the value `()` or to the type `()` is determined by context.

A character literal has type `Character`. Each character literal consists of an abstract character in Unicode 5.0 [29], enclosed in single quotation marks (for example, 'a', 'A', '$', 'α', '⊕'). The single quotes may be either true typographical "curly" single quotation marks or a pair of ordinary apostrophe characters (for example, `'a'`, `'A'`, `'$'`, `'α'`, `'⊕'`). See Section 4.9 for a description of how names of characters may be used rather than actual characters within character literals, for example '`APOSTROPHE`' and '`GREEK_CAPITAL_LETTER_GAMMA`'.

This is not true: Character literals include "" and "LATIN CAPITAL A", which do not "consist of" an abstract character enclosed in single quotation marks. I think it's better to remove most of this discussion and direct the reader to the relevant section of Lexical Structure. This section should be about **semantics** of literals: their values and types.

A string literal has type `String`. Each string literal is a sequence of Unicode 5.0 characters enclosed in double quotation marks (for example, "`Hello, world!`" or "πr^2"). The double quotes may be either true typographical "curly" double quotation marks or a pair of "neutral" double-quote characters (for example, `"Hello, world!"` or "πr^2"). Section 4.10 also describes how names of characters may be used rather than actual characters within string literals. One may also use the escape sequences `\b` and `\t` and `\n` and `\f` and `\r` as described in [7].

> DRAFT NOTE
>
> We may want to eliminate the distinction between simple and compound numerals, and just distinguish them by whether they have a radix point or not, when necessary (i.e., "Numerals may have a radix point."). Again, that distinction is primarily a syntactic one, and can be discussed there if appropriate.

Again, I think this paragraph should mostly be eliminated and replaced with a pointer to the relevant section of Lexical Structure. Here's a sentence I wrote as a replacement: "A string literal (i.e., a sequence of characters enclosed in double quotation marks—see Section 5.10 for details) has type String."

Numeric literals in Fortress are referred to as *numerals*, corresponding to various expressible numbers. Numerals may be either *simple* or *compound* (as described in Section 4.13).

A numeral containing only digits (let n be the number of digits) has type $\mathrm{NaturalNumeral}[\![n, 10, v]\!]$ where v is the value of the numeral interpreted in radix ten. If the numeral has no leading zeros, or is the literal `0`, then it also has type $\mathrm{Literal}[\![v]\!]$.

> DRAFT NOTE
>
> Can the type $\mathrm{NaturalNumeral}[\![n, 10, v]\!]$ have any value other than what you get from evaluating the corresponding literal?
>
> What is the relationship between $\mathrm{Literal}[\![v]\!]$ and $\mathrm{NaturalNumeral}[\![n, 10, v]\!]$?
>
> We need to describe the `Numeral` type hierarchy. All the numeric/numeral types should be defined and their relations to each other given. And a reference to where that is done should be appear in this section (probably at the beginning of the section).

> DRAFT NOTE
>
> Ought we discuss digit-group separators here? Or is that just part of lexical structure? (I think we intend that a numeral consisting of digits and digit-group separators has type $\mathrm{NaturalNumeral}[\![n, 10, v]\!]$ for appropriate values of n and v.)

A numeral containing only digits (let n be the number of digits) then an underscore, then a radix indicator (let r be the radix) has type $\mathrm{NaturalNumeralWithExplicitRadix}[\![n, r, v]\!]$ where v is the value of the n-digit numeral interpreted in radix r.

A numeral containing only digits (let n be the total number of digits) and a radix point (let m be the number of digits after the radix point) has type $\mathrm{RadixPointNumeral}[\![n, m, 10, v]\!]$ where v is the value of the numeral, with the radix point deleted, interpreted in radix ten.

A numeral containing only digits (let n be the total number of digits) and a radix point (let m be the number of digits after the radix point), then an underscore, then a radix indicator (let r be the radix) has the following type:

$\mathrm{RadixPointNumeralWithExplicitRadix}[\![n, m, r, v]\!]$

where v is the value of the n-digit numeral, with the radix point deleted, interpreted in radix r.

Every numeral also has type $\mathrm{Numeral}[\![n, m, r, v]\!]$ for appropriate values of n, m, r, and v.

Numerals are not directly converted to any of the number types because, as in common mathematical usage, we expect them to be polymorphic. For example, consider the numeral 3.1415926535897932384; converting it immediately to a floating-point number may lose precision. If that numeral is used in an expression involving floating-point intervals, it would be better to convert it directly to an interval. Therefore, numerals have their own types as described above. This approach allows library designers to decide how numerals should interact with other types of objects by defining coercion operations (see Section 17.1 for an explanation of coercion in Fortress). The Fortress standard libraries define coercions from numerals to integers (for simple numerals) and rational numbers (for compound numerals). In the Fortress standard libraries, every numeral without a radix point has the type $\mathrm{IntLiteral}$, which extends Number and none of the number types (Section 33.1); each of the five integer types (Section 33.2), $\mathbb{Q}$ and $\mathbb{R}64$ declares a coercion from it.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.36
>
> The Working Draft of February 2011 described the numeral types above, which no library declares, and did not name the type that the Fortress standard libraries give a numeral. The revival decided on 29 September 2026 that a numeral has a type of its own, $\mathrm{IntLiteral}$, from which each number type converts, the type the compiled path's own library gives it, and the Fortress standard libraries now declare it so. The compiled type checker gives every numeral without a radix point that type. The interpreter does not yet: it gives such a numeral the type $\mathbb{Z}32$, or $\mathbb{Z}64$ or $\mathbb{Z}$ when its value does not fit in the narrower type, and it is to give it the type $\mathrm{IntLiteral}$ once it runs with the static types of the compiled type checker.

What is an "integer" in this context?

Not if the second integer is 0.

In Fortress, dividing two integers using the $/$ operator produces a *rational number*; this is true regardless of whether the integers are of type $\mathbb{Z}$ (or `ZZ`), $\mathbb{Z}64$ (or `ZZ64`), $\mathbb{N}32$ (or `NN32`), or whatever. Addition, subtraction, multiplication, and division of rationals are always exact; thus values such as $1/3$ are represented exactly in Fortress.

> DRAFT NOTE
>
> We should give an example with a radix specifier.

Numerals containing a radix point are actually rational literals; thus 3.125 has the rational value $3125/1000$. The quotient of two integer literals is a static expression (described in Section 13.27) whose value is rational. Similarly, a sequence of digits with a radix point followed by the symbol $\times$ and an integer literal raised to an integer literal, such as 6.0221415×10^{23} is a static expression whose value is rational. If such static expressions are mentioned as part of a floating-point computation, the compiler performs the rational arithmetic exactly and then converts the result to a floating-point value, thus incurring at most one floating-point rounding error. But in general rational computations may also be performed at run time, not just at compile time.

> DRAFT NOTE
>
> This specifies too much about the implementation: I don't think we should mention a compiler at all, except perhaps as informal discussion. In particular, wouldn't it be correct if this arithmetic were actually deferred and performed exactly at run time?

A rational number can be thought of as a pair of integers p and q that have been reduced to "standard form in lowest terms"; that is, $q > 0$ and there is no nonzero integer k such that $\frac{p}{k}$ and $\frac{q}{k}$ are integers and $\left|\frac{p}{k}\right| + \left|\frac{q}{k}\right| < |p| + |q|$. The type $\mathbb{Q}$ includes all such rational numbers. Relaxing the requirement $q > 0$ to $q \geq 0$ adds three more values, which $\mathbb{Q}$ also includes: $1/0$ and $-1/0$, the infinite rationals, and $0/0$, the indefinite rational. The advantage of these values is that $\mathbb{Q}$ is closed under the rational operations $+$, $-$, $\times$, and $/$. In ASCII, $\mathbb{Q}$ is written as `QQ`. See Section 33.1 for its operations.

Is there any subtyping or coercion relationship between $\mathbb{Q}$ and $\mathbb{Q}*$? If $\mathbb{Q}$ is a subtype of $\mathbb{Q}*$ then static checking would normally disallow the assignment, so is this a relaxation to that rule? If so, the static checker needs to know about it.

REVISED BY THE 2026 REVIVAL SEE Section I.1.15

The Working Draft of February 2011 described $\mathbb{Q}$, $\mathbb{Q}^*$ and $\mathbb{Q}^\#$ as three types, said that a value of type $\mathbb{Q}^*$ assigned to a variable of type $\mathbb{Q}$ throws a DivisionByZero at run time if it is $1/0$ or $-1/0$, gave the ASCII names `QQ_star` and `QQ_splat`, and pointed to Section 41.1 for the definitions. The three are one type now, as in the Fortress standard libraries (Section 33.1), so no such assignment converts or throws; that is also the answer to the margin note's question whether $\mathbb{Q}$ and $\mathbb{Q}^*$ are related by subtyping or by coercion.

13.1.1 Pi

The object named π (or `pi`) may be used to represent the ratio of the circumference of a circle to its diameter rather than a specific floating-point value or interval value. In Fortress, π has type $\mathrm{RationalValueTimesPi}[\![\mathit{false}, 1, 1]\!]$. When used in a floating-point computation, it becomes a floating-point value of the appropriate precision; when used in an interval computation, it becomes an interval of the appropriate precision.

13.1.2 Infinity and Zero

The object named ∞ has type $\mathrm{ExtendedIntegerValue}[\![\mathit{true}, 0, \mathit{true}]\!]$. One can negate ∞ to get a negative infinity.

DRAFT NOTE

What about spaces? That is, is "- 0" treated the same as "-0", for example?

Negating the literal 0 produces a special negative-zero object, which refuses to participate in static arithmetic (discussed in Section 13.27). It has type $\mathrm{NegativeZero}$. The main thing it is good for is coercion to a floating-point number (discussed in Chapter 17). (Negating any other zero-valued expression simply produces zero.)

13.2 Identifier References

Syntax:

Primary	::=	*VarOrFnRef*
	|	`self`
VarOrFnRef	::=	*Id StaticArgs*?

A name that is not an operator name appearing in an expression context is called an *identifier reference*. It evaluates to the value of the name in the enclosing scope in the value namespace. The type of an identifier reference is the declared type of the name. See Chapter 7 for a discussion of names. An identifier reference performs a memory read operation. If a name is not in scope, it is a static error (as described in Section 7.3).

Is this true even for immutable variables?

An identifier reference which denotes a polymorphic function may include explicit static arguments (described in Chapter 9) but most identifier references do not include them.; the static arguments are statically inferred from the context of the function call (as described in Chapter 20). For example, $\mathit{double}[\![\mathrm{String}]\!]$ is an identifier reference with an explicit static argument where the function double is defined as follows:

$$\mathit{double}[\![T]\!](x:T):(T,T) = (x,x)$$

The special name `self` is declared as a parameter of a method. When a dotted method is invoked, its receiver is bound to the `self` parameter; the value of `self` is the receiver. When a functional method is invoked, the corresponding argument is bound to the `self` parameter; the value of `self` is the argument passed to it. The type of `self` is the type of the trait or object being declared by the innermost enclosing trait or object declaration or object expression. If

I thought that for object expressions, the type of `self` was the intersection of the traits it lists in its `extends` clause. In particular, if it includes a "method declaration" with a new name m (not one inherited from one of its supertypes), we cannot invoke `self.`m, because m isn't really a new method–

the innermost enclosing such construct is a trait declaration (for a trait called, say, 'T'), and that trait declaration has a `comprises` clause, and 'U_1', 'U_2', ..., 'U_n' are the traits mentioned in the `comprises` clause, then the type of `self` is instead "$T \cap (U_1 \cup U_2 \cup \ldots \cup U_n)$". If that trait declaration has no `comprises` clause, or one that lists no type but "...", and has a self-type parameter 'X' (Section 10.1), a static parameter bounded by the trait itself at its own static parameters, as in `trait` $T[\![X$ `extends` $T[\![X]\!]]\!]$, or by a trait with a self-type parameter that 'T' extends at 'X', then the type of `self` in the bodies of its methods is instead "$T \cap X$"; the `self` parameter keeps the type 'T' in the signature of each method declaration, which overloading and the choice of a declaration at a call read, save the test of whether two overloaded declarations' parameter types exclude each other, which reads it as "$T \cap X$".

> REVISED BY THE 2026 REVIVAL SEE Section I.1.41
>
> The sentence on a self-type parameter is new. The Working Draft of February 2011 gave `self` the type of the trait in such a declaration, so that a method body that returns `self` where the self-type parameter is declared, as the default bodies of the orders and of Integral in the Fortress standard libraries do, was not well typed. The revival reads the self type from the bound, as the self-type idiom above reads it from a `comprises` clause, and checks the declarations that extend the trait against it (Section 10.1). The signatures keep the type of the trait, so that no declared type holds an intersection.

See Section 10.2 for details about `self` parameters.

13.3 Dotted Field Accesses

Syntax:

Primary ::= *Primary*. *Id*

An expression consisting of a single subexpression (called the *receiver expression*), followed by '`.`', followed by an identifier, not immediately followed by a parenthesis or a white square bracket, is a *field access*. The receiver expression must denote an object (called the *receiver*) and the field access is evaluated to a call to a getter mapped from that identifier by the receiver. The type of the field access is the return type of its getter. The static type of the receiver indicates whether a getter mapped from that identifier is provided by the denoted object. If a getter is not provided, it is a static error. See Section 10.2 for a discussion of getters. The evaluation of the receiver must complete normally before the body of the getter is evaluated.

Not exactly: An object (i.e., the actual value at run time) may provide a getter but if the static type of the receiver expression doesn't provide the getter, then the getter can't be called in this context. So just say that it is a static error if the static type of the receiver doesn't provide the getter, without making any claim about whether the denoted object provides the getter.

13.4 Dotted Method Invocations

> DRAFT NOTE
>
> Should be a static check that there is an applicable declaration.

Syntax:

Primary	::=	*Primary*. *Id StaticArgs*? *ParenthesisDelimited*
ParenthesisDelimited	::=	*Parenthesized*
	|	*ArgExpr*
	|	( )
Parenthesized	::=	(*Expr*)
ArgExpr	::=	*TupleExpr*
	|	($(Expr,)^*$ *Expr*...)
TupleExpr	::=	($(Expr,)^+$ *Expr*)

A *dotted method invocation* consists of a subexpression (called the receiver expression), followed by '`.`', followed by an identifier, an optional list of static arguments (described in Chapter 9) and a subexpression (called the *argument*

expression). Unlike in function calls (described in Section 13.6), the argument expression must be parenthesized, even if it is not a tuple. There must be no whitespace on the left-hand side of the '`.`' and the left-hand side of the left parenthesis of the argument expression. The receiver expression evaluates to the receiver of the invocation (bound to the `self` parameter (discussed in Section 10.2) of the method). A method invocation may include explicit instantiations of static parameters but most method invocations do not include them; the static arguments are statically inferred from the context of the method invocation (as described in Chapter 20).

The receiver and arguments of a method invocation are each evaluated in parallel in a separate implicit thread (see Section 5.4). After this thread group completes normally, the body of the method is evaluated with the parameter of the method bound to the value of the argument expression (thus evaluation of the body occurs after evaluation of the receiver and arguments in dynamic program order). The value and the type of a dotted method invocation are the value and the type of the method body.

We say that methods or functions (collectively called as *functionals*) may be *applied to* (also "*invoked on*" or "*called with*") an argument. We use "call", "invocation", and "application" interchangeably.

Here are some examples:

$myString.toUppercase()$

$myString.replace(\text{“}\texttt{foo}\text{”}, \text{“}\texttt{few}\text{”})$

$myNum.add(otherNum)$ `(*` NOT myNum.add otherNum `*)`

13.5 Naked Method Invocations

> DRAFT NOTE
> Should be a static check that there is an applicable declaration.

Syntax:

Primary ::= *Id Primary*

Method invocations that are not prefixed by receivers are *naked method invocations*. A naked method invocation is either a functional method call (see Section 10.2 for a discussion of functional methods) or a method invocation within a trait or object that provides the method declaration. Syntactically, a naked method invocation is same as a function call except that the method name is used instead of an arbitrary expression denoting the applied method. Like function calls, an argument expression need not be parenthesized unless it is a tuple. After the evaluation of the argument expression completes normally, the body of the method is evaluated with the parameter of the method bound to the value of the argument expression. The value and the type of a naked method invocation are the value and the type of the method body.

13.6 Function Calls

> DRAFT NOTE
> Should be a static check that there is an applicable declaration.

Syntax:

Primary ::= *Primary Primary*

A *function call* consists of two subexpressions: an expression denoting the applied function and an argument expression. The argument expression and the expression denoting the applied function are evaluated *in parallel* in separate

implicit threads (see Section 5.4). As with languages such as Scheme and the Java Programming Language, function calls in Fortress are call-by-value. After the evaluation of the function and its arguments completes normally, the body of the function is evaluated with the parameter of the function bound to the value of the argument expression. The value and the type of a function call are the value and the type of the function body. See Section 9.2 for a detailed description of function calls.

Here are some examples:

$\cos(x)$

$\arctan(y, x)$

If the function's argument is not a tuple, then the argument need not be parenthesized:

$\sin\ x$

$\log\ \log\ n$

13.7 Operator Applications

> DRAFT NOTE
>
> Examples/discussion of juxtaposition as an operator.
>
> Should be a static check that an applicable declaration exists.

Syntax:

OpExpr	::=	*EncloserOp OpExpr*? *EncloserOp*?
	|	*OpExpr EncloserOp OpExpr*?
	|	*Primary*
EncloserOp	::=	*Encloser*
	|	*Op*
Primary	::=	*LeftEncloser StaticArgs*? *ExprList*? *RightEncloser*
	|	*Primary* `^` *Exponent*
	|	*Primary ExponentOp*
Exponent	::=	*Id*
	|	*ParenthesisDelimited*
	|	*LiteralExpr*
	|	`self`
ExponentOp	::=	`^T` | `^(`*Encloser* | *Op*`)`

To support a rich mathematical notation, Fortress allows most Unicode characters that are specified to be mathematical operators to be used as operators in Fortress expressions, as well as various tokens described in Chapter 16. Most of the operators can be used as prefix, infix, postfix, or nofix operators as described in Section 16.3; the fixity of an operator is determined syntactically, and the same operator may have definitions for multiple fixities.

Syntactically, an operator application consists of an operator and its argument expressions. If the operator is a prefix operator, it is followed by its argument expression. If the operator is an infix operator, its two argument expressions come both sides of the operator. If the operator is a postfix operator, it comes right after its argument expression. Like function calls, argument expressions are evaluated *in parallel* in separate implicit threads. After evaluation of arguments completes normally, the body of the operator definition is evaluated with the parameters of the operator bound to the values of the argument expressions. The value and the type of an operator application are the value and the type of the operator body. See Chapter 16 for a detailed description of operators.

Here are some examples:

$$\left(-b + \sqrt{b^2 - 4\,a\,c}\right)/2\,a$$

$$n^n\,e^n + \sqrt{2\,\pi\,n}$$

$$x_1\,y_2 - x_2\,y_1$$

$$1/2\,g\,t^2$$

$$n\,(n+1)/2$$

$$\big((j+k)!\big)/(j!\,k!)$$

$$\frac{1}{3}\frac{3}{5}\frac{5}{7}\frac{7}{9}\frac{9}{11}$$

$$println\big(\text{“}\texttt{The answers are }\text{”}\,(p+q)\,\text{“}\texttt{ and }\text{”}\,(p-q)\big)$$

> DRAFT NOTE
> The following examples are not yet working:

$$a_k\,b_{n-k}$$
$$17.3\,\mathrm{meter/second}$$
$$17.3m/s$$
$$u \cdot (v \times w)$$
$$(A \cup B)\ \texttt{INTERSECT}\ C$$
$$(A \cup B) \cap C$$
$$i < j \leq k \wedge p \prec q$$
$$\langle 2, 3, 4, 5 \rangle$$

13.8 Function Expressions

> DRAFT NOTE
> Syntax is not up to date.
>
> Allow `io` function expressions?
>
> Types in `throws` clause can't be naked type parameter?
>
> Check that *Expr* type is subtype of the declared return type.

> DRAFT NOTE
> We want to allow function expressions to carry contracts; we need to work out how these contracts are evaluated, and what the implications are on variable scoping. – Eric

Syntax:

Expr ::= `fn` *ValParam IsType*? *Throws*? ⇒ *Expr*

Function expressions denote function values; they do not require evaluation. Syntactically, they start with `fn` followed by a parameter, optional return type, optional `throws` clause, ⇒, and finally an expression. The type of a function expression is an arrow type consisting of the function's parameter type followed by the token →, followed by the function's return type, and the function's optional `throws` clause. Unlike declared functions (described in Chapter 9), function expressions are not allowed to include static parameters nor `where` clauses (described in Chapter 12).

Here is a simple example:

`fn` $(x{:}\,\mathbb{R}64) \Rightarrow$ `if` $x < 0$ `then` $-x$ `else` x `end`

13.9 Object Expressions

> DRAFT NOTE
> Property declarations and `where` clauses are not yet supported.
>
> No naked type variables in `extends` clause.
>
> The type (tag?) of the **object** resulting from evaluating an object expression is this anonymous object trait type, but is the static type of the object expression that?
>
> No new functional methods within object expressions.

Syntax:

DelimitedExpr ::= `object` *ExtendsWhere*? *GoInAnObject*? `end`

Object expressions denote object values. Syntactically, they start with `object`, followed by an optional `extends` clause, a series of field declarations, method declarations, or property declarations, and finally `end`. The type of an object expression is an anonymous object expression type that extends the traits listed in the `extends` clause of the object expression. The object expression type does not include the methods introduced by the object expression (i.e., those methods not provided by any supertraits of the object expression). Each object expression type is associated with a program location; every evaluation of a given object expression has the same object expression type. Two object expressions at different program locations have different object expression types.

Unlike object declarations (described in Chapter 11), object expressions are not allowed to include modifiers, value parameters, static parameters, or `where` clauses (described in Chapter 12). Object expressions may include free static variables unlike object declarations, which must not include any free static variables (i.e., each static variable in an object declaration must occur either as a static parameter or as a `where`-clause variable). An object expression is not allowed to declare "new" functional methods; it can provide a functional method only with a name that is already declared by one of its supertraits. Field initializers and methods may refer to any variables that are in scope in the context in which the object expression occurs. As with object declarations, initializers are evaluated in textual program order and may refer to previous fields; the object being constructed might not be referred to in any way.

In an object expression, any declaration of `self` (including implicit declarations) and any declaration which declares a name of a field or dotted method provided (declared or inherited) by the object expression in a block enclosing the expression is shadowed unless the declaration is a method declaration of a trait extended by the object expression. In order for an object to be referred to within a nested object expression, the outer object must be renamed before the object expression because `self` declared in the outer object is shadowed in the object expression:

```
object O
  m() = do outer = self
           object
             getOuterSelf() = outer      (* outer "self" *)
             getInnerSelf() = self       (* inner "self" *)
           end
         end
```

```
end
```

For example, the following object expression:

$f[\![T]\!](x\colon T) =$ `object`

$f'\colon T = x$

`end`

has a static variable T that is not its static parameter nor its `where`-clause variable.

The following example expression evaluates to a new object extending trait Tree:

`object extends` Tree

printTree(): () = *println* "`leaf`"

`opr` |`self`| : $\mathbb{Z}32 = 0$

`end`

13.10 Assignments

> DRAFT NOTE
> Qualified names are not yet supported.

Syntax:

AssignExpr	::=	*AssignLefts AssignOp Expr*
AssignLefts	::=	(*AssignLeft*(, *AssignLeft*)*)
	|	*AssignLeft*
AssignLeft	::=	*SubscriptExpr*
	|	*FieldSelection*
	|	*QualifiedName*
SubscriptExpr	::=	*Primary LeftEncloser StaticArgs*? *ExprList*? *RightEncloser*
FieldSelection	::=	*Primary*. *Id*
AssignOp	::=	:= | (*Encloser* | *Op*)=

An assignment expression consists of a left-hand side (*AssignLefts*) indicating one or more variables, subscripted expressions (as described in Section 25.8), or field accesses to be updated, an assignment token, and a right-hand-side expression. Multiple left-hand sides must be grouped using tuple notation (comma-separated and parenthesized). Variables updated in an assignment expression must already be declared.

The assignment token ':=' indicates ordinary assignment. Ordinary assignment proceeds in two phases. In the first, the evaluation phase, the right-hand-side expression is evaluated in parallel with each of the left-hand-side subexpressions, forming an implicit thread group. Evaluating a left-hand variable does nothing. Evaluating a left-hand field reference evaluates the receiving object. Evaluating a left-hand subscripting operation evaluates the receiving object and the index in parallel in a nested thread group. After the outer implicit thread group completes normally, the assignment phase begins. Each component of the right-hand-side value is assigned to the corresponding component of the left-hand side in parallel, forming an implicit thread group. Assigning a left-hand variable simply changes the binding for the variable's location to contain the new value. Assigning a left-hand field reference calls the corresponding setter method on the reiver object, passing the new value. Assigning a left-hand subscript simply calls the subscripted assignment operation on the corresponding object with the new value.

Any operator (other than ‘:’ or ‘=’ or ‘<’ or ‘>’) followed by ‘=’ with no intervening whitespace indicates compound (updating) assignment. This adds an additional phase to assignment between the two phases of ordinary assignment. In the first phase, a left-hand field reference invokes the getter for the corresponding field after the receiving object has been evaluated. A left-hand subscripting operation invokes the subscripting operator on the receiving object once receiver and index have been evaluated. A left-hand variable simply returns the current value of the location associated with that variable. In the new second phase, the operator indicated in the assignment is invoked. The left-hand argument is the value of the left-hand expression evaluated in the first phase; the right-hand argument is the value evaluated for the right-hand side of the assignment expression. When the operator evaluation completes normally, the assignment phase is run as in ordinary assignment.

The important point to understand is that compound assignment evaluates its subexpressions exactly once, and that the parts of assignment proceed implicitly in parallel where possible. The value and type of an assignment expression is $()$. The type of the right-hand side must be a subtype of the type of the left-hand side.

Consider the following assignment:

$(a_i, b.x, c) \mathrel{+}= f(t, u, v)$

Here in the first phase we evaluate a and i in parallel, then when this completes we invoke the indexing method of a to evaluate a_i. In parallel, we evaluate b and then call the getter method $b.x$. In parallel, we look up the value of variable c. Finally, we evaluate $f(t, u, v)$ in parallel with all of these left-hand sides. In the second phase, we combine the results using the $+$ operator, to evaluate $(a_i, b.x, c) + f(t, u, v)$. This must return a tuple of three values (p, q, r). In the final phase, in parallel we call the indexed assignment operator $a_i := p$, call the setter $b.x := q$, and perform the local assignment $c := r$.

Here are some simpler and more commonplace examples of assignment:

$x := f(0)$

$\big(n[i\,j] = n[i\,j] + l[i\,k]\,m[k\,j]\big)$

$(a, b, c) := (b, c, a)$ ($\star$ Permuta a, b, and c $\star$)

$x \mathrel{+}= 1$

$(x, y) \mathrel{+}= (\delta_{\mathrm{x}}, \delta_{\mathrm{y}})$

$myBag \cup= newItems$

13.10.1 Definite Assignment

> DRAFT NOTE
> Definite assignment check is not supported yet.

References to uninitialized variables are statically forbidden. As with the Java Programming Language, this static constraint is ensured with a specific conservative flow analysis. In essence, an initialization of a variable must occur on every possible execution path to each reference to a variable.

13.11 Do Expressions

Syntax:

Do	::=	(*DoFront* `also`)* *DoFront* `end`
DoFront	::=	(`at` *Expr*)? `atomic`? `do` *BlockElems*?
BlockElems	::=	*BlockElem*$^+$
BlockElem	::=	*LocalVarFnDecl*
	|	*Expr*(, *GeneratorClauseList*)?
LocalVarFnDecl	::=	*LocalFnDecl*$^+$
	|	*LocalVarDecl*

A `do` expression consists of a series of *expression blocks* separated by `also` and terminated by `end`. Each expression block is preceded by an optional `at` expression (described in Section 23.2), an optional `atomic`, and `do`. When prefixed by `at` or `atomic`, it is as though that expression block were evaluated as the body expression of an `at` or `atomic` expression (described in Section 13.23), respectively. An expression block consists of a (possibly empty) series of *elements*–expressions, generated expressions (described in Section 13.11.1), local variable declarations, or local function declarations–separated by newlines or semicolons.

A single expression block evaluates its elements in order: each element must complete before evaluation of the next can begin, and the expression block as a whole does not complete until the final element completes. Each expression except the last element of the expression block must have type $()$. There are two ways to make an expression e of type non-$()$ have type $()$ in an expression block: 1) "$ignore(e)$" and 2) "$_ = e$". If the last element of the expression block is an expression, the value and type of this expression are the value and type of the expression block as a whole. Otherwise, the value and type of the expression block is $()$. Each expression block introduces a new scope. Some compound expressions have clauses that are implicitly expression blocks.

Because a local declaration shares a syntax with an *equality testing expression*, we require that any equality testing expression within an expression block be parenthesized.

Here are examples of function declarations whose bodies are `do` expressions:

```
f(x: ℝ64) = do
  (sin(x) + 1)^2
end

foo(x: ℝ64) = do
  y = x
  z = 2 x
  y + z
end

mySum(i: ℤ32): ℤ64 = do
  acc: ℤ64 := 0
  for j ← 0 : i do
    acc := acc + j
  end
  acc
end
```

REVISED BY THE 2026 REVIVAL SEE Section I.1.20

The Working Draft of February 2011 declared the parameter of $mySum$ of type $\mathbb{Z}64$, so that its loop ran over a range of $\mathbb{Z}64$ values, which Section 13.16 no longer admits; the parameter is of type $\mathbb{Z}32$, and the sum keeps its type $\mathbb{Z}64$.

13.11.1 Generated Expressions

If a subexpression of a `do` expression has type $()$, the expression may be followed by a ‘,’ and a generator clause list (described in Section 13.14). Writing “$expr$, $gens$” is equivalent to writing “`for` $gens$ `do` $expr$ `end`”. See Section 13.15 for the semantics of the `for` expression. Note in particular that $expr$ can be a reducing assignment of the form “$variable$ `OP=` $expr$”.

13.11.2 Parallel Do Expressions

DRAFT NOTE

Reduction variables are not yet supported.

A series of blocks may be run in parallel using the `also` construct. Any number of contiguous blocks may be joined together by `also`. Each block is run in a separate implicit thread; these threads together form a group. The expression as a whole completes when the group is complete. A thread can be placed in a particular region by using an `at` expression as described in Section 23.2. When multiple expression blocks are separated by `also`, each expression block must have type $()$; the result and type of the parallel `do` expression is also $()$.

For example:

$treeSum(t\colon \text{TreeLeaf}) = 0$

$treeSum(t\colon \text{TreeNode}) =$ `do`

  `var` $accum\colon \mathbb{Z}32 := 0$

  `do`

    $accum \mathrel{+}= treeSum(t.left)$

  `also do`

    $accum \mathrel{+}= treeSum(t.right)$

  `also do`

    $accum \mathrel{+}= t.datum$

  `end`

  $accum$

`end`

Note the use of the reduction variable $accum$ (Section 5.4.1) within the threads in the group.

13.12 Label and Exit

Syntax:

DelimitedExpr ::= `label` *Id BlockElems* `end` *Id*
FlowExpr ::= `exit` *Id*? (`with` *Expr*)?

An expression block may be labeled using a `label` expression, which consists of `label`, an identifier (the block's *name*), an expression block (its *body*), `end`, and finally its name again (it is a static error if the identifier after `end` is not the name). The name of a `label` expression is in scope in the label namespace at any point in its body that is not within a `spawn` subexpression of the body. A `label` expression is evaluated by evaluating its body.

An `exit` expression consists of `exit`, an optional identifier (the *target*) and an optional `with` clause, which consists of `with` followed by an expression. If a target is specified, it is a static error if the target is not in scope in the label namespace at the `exit` expression. That is, the target must be the name of a statically enclosing `label` expression, and the `exit` expression must not be within a `spawn` expression that is contained in the `label` expression. If no target is specified, the target is implicitly the name of the smallest statically enclosing `label` expression; it is a static error if there is no such expression.

An `exit` expression with a `with` clause evaluates its `with` clause expression to yield a `with` clause value. The `exit` expression completes abruptly with the `with` clause value (see Section 5.2). An `exit` expression with no `with` clause has an implicit `with` clause whose expression is $()$. The type of an `exit` expression is $\mathrm{BottomType}$.

If the evaluation of the body of a `label` expression completes normally, its value is the value of the body. If the evaluation of the body completes abruptly with an `exit` expression whose target is the name of the `label` expression, then the evaluation of the `label` expression completes normally and its value is the `with` clause value of the `exit` expression. The type of a `label` expression is the union of the type of the last expression of its expression block and the types of the `with` clause values of any `exit` expressions within the `label` expression whose target is the `label` expression's name.

If one or more `try` expressions are nested between an `exit` expression and the targeted `label` block, the `finally` clauses of these expressions are run in order, from innermost to outermost, as described in Section 13.26. If any `finally` clause completes abruptly by throwing an exception, the `exit` expression fails to exit, the evaluation of the `label` expression completes abruptly, and the exception is propagated.

Here is a simple example:

```
label I95
  if goingTo(Sun)
  then exit I95 with x32B
  else x32A
  end
end I95
```

The expression "`exit` I_{95} `with` $x32B$" completes abruptly and attempts to transfer control to the end of the targeted labeled block "`label` I_{95}". The targeted labeled block completes normally with value $x32B$.

13.13 While Loops

Syntax:

DelimitedExpr ::= `while` *GeneratorClause Do*

A `while` loop consists of a generator clause (discussed in Section 13.14) followed by a simple `do` expression (see Section 13.11). An iteration of a `while` loop evaluates the expression of its generator clause (*condition expression*). If the generator clause is a generator binding, the type of the condition expression must be a subtype of $\mathrm{Condition}[\![E]\!]$ for some E and the type of the identifiers bound in the clause must be a subtype of E. A $\mathrm{Condition}$ is a generator that generates 0 or 1 element. Otherwise, the generator clause must be an expression of type $\mathrm{Boolean}$. Note in particular that true is a $\mathrm{Boolean}$ value yielding $()$ exactly once, while false is a $\mathrm{Boolean}$ value that yields no elements. If the evaluation of the condition expression completes normally and generates exactly one element, the body expression is evaluated. When one iteration completes a new one is run until either an iteration completes abruptly (in which case the evaluation of the `while` expression completes abruptly), or the condition expression generates no elements (in which case the `while` loop completes normally with value $()$).

13.14 Generators

> DRAFT NOTE
> Distributions are not yet supported.

Syntax:

GeneratorClauseList	::=	*GeneratorBinding*(, *GeneratorClause*)*
GeneratorBinding	::=	*BindIdOrBindIdTuple*← *Expr*
GeneratorClause	::=	*GeneratorBinding*
	|	*Expr*
BindIdOrBindIdTuple	::=	*BindId*
	|	(*BindId*, *BindIdList*)
BindIdList	::=	*BindId*(, *BindId*)*
BindId	::=	*Id*
	|	_

Fortress makes extensive use of comma-separated *generator clause lists* to express parallel iteration. Generator clause lists occur in generated expressions (described in Section 13.11.1) and `for` loops (described in Section 13.15), sums and big operators (described in Section 13.17), and comprehensions (described in Section 13.30). We refer to these collectively as *expressions with generator clauses*. Every expression with generator clauses contains a *body expression* which is evaluated for each combination of values bound in the generator clause list (each such combination yields an *iteration* of the body).

A generator clause is either a *generator binding* or an expression of type $\mathrm{Generator}[\![()]\!]$ (this includes the type $\mathrm{Boolean}$). A generator clause list must begin with a generator binding. A generator binding consists of one or more comma-separated identifiers followed by the token $\leftarrow$, followed by a subexpression (called the *generator expression*). A generator expression evaluates to an object whose type is $\mathrm{Generator}$. A generator encapsulates zero or more *generator iterations*. By default, the programmer must assume that generator iterations are run in parallel in separate implicit threads unless the generators are instances of $\mathrm{SequentialGenerator}$; the actual behavior of generators is dictated by library code, as described in Section 23.8. No generator iterations are run until the generator expression completes. For each generator iteration, a generator object produces a value or a tuple of values. These values are bound to the identifiers to the left of the arrow, which are in scope of the subsequent generator clause list and of the body of the construct containing the generator clause list.

An expression of type $\mathrm{Generator}[\![()]\!]$ in a generator clause list is interpreted as a *filter*. A generator iteration is performed only if the filter yields $()$. If the filter yields no value, subsequent expressions in the generator clause list will not be evaluated. Note in particular that true is a $\mathrm{Boolean}$ value yielding $()$ exactly once, while false is a $\mathrm{Boolean}$ value that yields no elements.

The order of nesting of generators need not imply anything about the relative order of nesting of iterations. In most cases, multiple generators can be considered equivalent to multiple nested loops. However, the compiler will make an effort to choose the best possible iteration order it can for a multiple-generator loop, and may even combine generators together; there may be no such guarantee for nested loops. Thus loops with multiple generators are preferable to distinct nested loops in general. Note that the early termination behavior of nested looping is subtly different from a single multi-generator loop, since nested loops give rise to nested thread groups; see Section 23.7.

Each generator iteration of the innermost generator clause corresponds to a *body iteration*, or simply an *iteration* of the generator clause list. Each iteration is run in its own implicit thread. Each expression in the generator clause list can be considered to evaluate in a separate implicit thread. Together these implicit threads form a thread group. Evaluation of an expression with generators completes only when this thread group has completed.

Some common $\mathrm{Generator}\mathrm{s}$ include:

$l : u$	Any range expression
a	Array a generates its elements
$a.indices$	The index set of array a
$\{0, 1, 2, 3\}$	The elements of an aggregate expression
$sequential(g)$	A sequential version of generator g

The generator $sequential(g)$ forces the iterations using values from g to be performed in order. Every generator has an associated *natural order* which is the order obtained by $sequential$. For example, a sequential `for` loop starting at 1 and going to n can be written as follows:

```
for i ← sequential(1 : n) do
  ...
end
```

The $sequential$ generator respects generator clause list ordering; it will always nest strictly inside preceding generator clauses and outside succeeding ones.

Given a multidimensional array, the $indices$ generator returns a tuple of values, which can be bound by a tuple of variables to the left of the arrow:

$(i, j) \leftarrow my2DArray.indices$

The parallelism of a loop on this generator follows the spatial distribution (discussed in Section 23.6) of $my2DArray$ as closely as possible.

13.15 For Loops

> DRAFT NOTE
> Reduction variables are not yet supported.

Syntax:

DelimitedExpr ::= `for` *GeneratorClauseList DoFront* `end`
DoFront ::= (`at` *Expr*)? `atomic`? `do` *BlockElems*?

A `for` loop consists of `for` followed by a generator clause list (discussed in Section 13.14), followed by a non-parallel `do` expression (the loop *body*; see Section 13.11). Parallelism in `for` loops is specified by the generators used (see Section 13.14); in general the programmer must assume that each loop iteration will occur independently in parallel unless every generator is explicitly $sequential$. For each iteration, the body expression is evaluated in the scope of the values bound by the generators. The body of a `for` expression can make use of reduction variables as dscribed in Section 5.4.1. The value and type of a `for` loop is $()$.

13.16 Ranges

> DRAFT NOTE
> Static expressions, static range types, and some range operations are not yet supported.

Syntax:

Range ::= *Expr*?: *Expr*?(: *Expr*?)?
| *Expr*# *Expr*

A *range expression* is used to create a special kind of Generator for a set of integers, called a Range, useful for indexing an array or controlling a `for` loop. Generators in general are discussed further in Section 13.14.

An *explicit range* is self-contained and completely describes a set of integers. Its components are values of type $\mathbb{Z}32$: assume that a, b, and c are expressions that produce such values. An integer numeral converts to $\mathbb{Z}32$ (Section 13.1), and a `nat` or `int` parameter (Section 12.2) used as a component converts to $\mathbb{Z}32$ as an integer numeral does; it is a static error if any other component of an explicit range has an integer type other than $\mathbb{Z}32$.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.26
> The revival's first revision of this section (Section I.1.19) left open whether a `nat` parameter, which may appear wherever a variable of type $\mathbb{N}32$ can (Section 12.2), may be a component of a range. The sentence above settles it, as both implementations of the revival already run it: the compiled type checker gives a `nat` or `int` parameter used as a value the type of a numeral, and the interpreter makes it a $\mathbb{Z}32$.

- The range $a:b$ is the set of $n = \max(0, b - a + 1)$ integers $\{a, a+1, a+2, \ldots, b-2, b-1, b\}$. This is a *nonstrided* range. If a and b are both static expressions (described in Section 13.27), then it is a *static range* of type $\mathrm{StaticRange}[\![a, n, 1]\!]$ and therefore also a *range of static size* of type $\mathrm{RangeOfStaticSize}[\![n]\!]$.
- The range $a:b:c$ is the set of $n = \max\left(0, \left\lfloor \frac{b-a+c}{c} \right\rfloor\right)$ integers $\{a, a+c, a+2c, \ldots, a+\left\lfloor \frac{b-a}{c} \right\rfloor c\}$, unless c is zero, in which case it throws an exception. (If c is a static expression, then it is a static error if c is zero.) This is a *strided* range. If a, b, and c are all static expressions, then it is a *static range* of type $\mathrm{StaticRange}[\![a, n, c]\!]$ and therefore also a *range of static size* of type $\mathrm{RangeOfStaticSize}[\![n]\!]$.
- The range $a \# n$ is the set of $\max(0, n)$ integers $\{a, a+1, a+2, \ldots, a+n-3, a+n-2, a+n-1\}$. This is a *nonstrided* range. If a and n are both static expressions, then it is a *static range* of type $\mathrm{StaticRange}[\![a, n, 1]\!]$ If n is a static expression, then it is a *range of static size* of type $\mathrm{RangeOfStaticSize}[\![n]\!]$, even if a is not a static expression.

A loop whose counter must have a wider integer type runs over a range of $\mathbb{Z}32$ values and widens each element, as in the following loop, where the binding converts i to $\mathbb{Z}64$ (Section 33.2):

```
for i ← 0 # n do
    j: ℤ64 = i
    ...
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.19
> The Working Draft of February 2011 assumed only that the components of an explicit range produce integer values, and named no integer type; two of its own examples built ranges over $\mathbb{Z}64$, and a property of $\mathbb{Z}$ a range over $\mathbb{Z}$. The Fortress standard libraries declare the ranges over $\mathbb{Z}32$ alone, as the smaller library of the compiled path already did: a range over another integer type is a static error, and a loop that needs a wider counter widens each element of a range of $\mathbb{Z}32$ values.

Non-static components of a range expression are computed in separate implicit threads. The range is constructed when all components have completed normally.

An *implicit range* may be used only in certain contexts, such as array subscripts, that can supply implicit information. Suppose an implicit range is used as a subscript for an axis of an array for which the lower bound is l and the upper bound is u.

- The implicit range `:` is treated as $l : u$.
- The implicit range `::` c is treated as $l : u : c$.
- The implicit range `:` b is treated as $l : b$.
- The implicit range `:` b `:` c is treated as $l : b : c$.
- The implicit range a `:` and a `#` are treated as $a : u$.
- The implicit range a `::` c is treated as $a : u : c$.
- The implicit range `#` s is treated as l `#` s.

One may test whether an integer is in a range by using the operator $\in$:

`if` $j \in (a : b)$ `then` *println* "`win`" `end`

Ranges may be compared as if they were sets of integers by using $\subset$ (`SUBSET`) and $\subseteq$ (`SUBSETEQ`) and $=$ and $\supseteq$ (`SUPSETEQ`) and $\supset$ (`SUPSET`).

Ranges may be intersected using the operator $\cap$ (`INTERSECTION`).

The size of a range (the number of integers in the set) may be found by using the set-cardinality operator $|\ldots|$. For example, the value of $|3 : 7|$ is 5 and the value of $|1 : 100 : 2|$ is 50.

Note that a range is very different from an interval with integer endpoints. The range $3 : 5$ contains only the values 3, 4, and 5, whereas the interval $[3, 5]$ contains all real numbers x such that $3 \leq x \leq 5$.

13.17 Summations and Other Reduction Expressions

Syntax:

FlowExpr ::= *Accumulator StaticArgs*? (`[` *GeneratorClauseList* `]`)? *Expr*
Accumulator ::= $\sum$ | $\prod$ | `BIG` (*Encloser* | *Op*)

A *reduction expression* begins with a big operator such as $\sum$ or $\prod$ followed by an optional static arguments and an optional generator clause list (described in Section 13.14), followed by a body expression.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.17
>
> The desugaring of a reduction is directed by type in this specification: a sum is desugared with the reduction for the type N of the expression (Section 23.8.1). Neither implementation directs it by the type N of the expression yet. When no argument of the big operator fixes the element type, the interpreter, which has no static types, directs it by run-time types (Section 20.1): it instantiates the big operator by the declared type of the local variable, or the declared return type of the top-level or local function, whose value the reduction is, where that type fixes the operator's result type, and otherwise by the join of the run-time types of the reduction's elements, or, where there is no element, at the bound of the big operator's static parameter, or open where that bound mentions the static parameter itself, as the bounds of $\sum$, $\prod$ and `BIG MIN` do; so an empty reduction that no declared type directs takes the identity of $\mathbb{Z}32$ whatever the type of the expression (row 716 of the revival's gap ledger); the compiled type checker, when it checks the libraries this specification describes, takes the bound of the big operator's static parameter, which is not the element type, or, where the reduction's type at that bound does not fit the reduction's expected type, that expected type (Section 20.1): a bound such as Number gives the reduction that type where it has no expected type, and a bound that mentions the static parameter itself gives no instance that applies, so that the reduction is refused (row 425). Until the compiled type checker directs the desugaring by type, a reduction with a generator clause list whose element type

no argument fixes writes its static argument in a program that both implementations are to run, as the team's own tests do, as in $\sum[\![\mathbb{Z}32]\!][j \leftarrow 0\#i]\ f(j)$; a reduction without a generator clause list, such as $\sum a$, needs none. Until the compiled path is built on those libraries, it compiles against a smaller library of its own, whose big operators $\sum$ and `BIG MAX` take no static argument and range over $\mathbb{Z}32$ only; there the clause form runs without its static argument, and a written static argument is refused. The normative text is unchanged.

There is no explicit relationship between `BIG` *Op* and *Op*. Instead, a reduction expression corresponds to a call to the `BIG` *Op* operator, which has the following header:

$$\texttt{opr BIG}\ \mathit{Op}[\![T]\!](g : (\mathrm{Reduction}[\![R_0]\!], T \to R_0) \to R_0) : R$$

Here R is the result type of the operator, and g corresponds to the method $\mathit{generate}[\![R_0]\!]$ of the trait $\mathrm{Generator}[\![T]\!]$: it is a function that takes a reduction (of type $\mathrm{Reduction}[\![R_0]\!]$) and a body (of type $T \to R_0$) and returns a value of type R_0 by running body on each generated element and combining them using the reduction. When a generator clause list is provided, generator clauses produce values and bind the values to the identifiers that are used in the subexpression. Each iteration of the body expression is assumed to be evaluated in a separate implicit thread as described in Section 13.14. The resulting values are combined together using *Op* as with reduction variables (see Section 5.4.1). The value of a reduction expression is this combined value. When no generator clause list is provided, the body is taken to be an object of type $\mathrm{Generator}$ and the elements it generates are combined. Thus, the reduction expression $\sum a$ is equivalent to $\sum_{x \leftarrow a} x$.

A reduction expression with a generator list:

$$\sum[v_1 \leftarrow g_1, v_2 \leftarrow g_2, \ldots]e$$

is equivalent to the following code:

```
do
   var result: ℤ32 = 0
   for v1 ← g1, v2 ← g2, ... do
      result += e
   end
   result
end
```

where result is a fresh variable. A reduction expression without a generator clause list:

$$\sum g$$

is equivalent to the following:

$$\sum[x \leftarrow g]x$$

Note that reduction expressions without generator clause lists can be used to conveniently sum any aggregate expression (described in Section 13.29), since every aggregate expression is a generator.

13.18 Parallel Prefix and Parallel Suffix

DRAFT NOTE

This section was not in F1.0β nor in F1.0. We don't quite have a story on this yet.

13.19 If Expressions

Syntax:

DelimitedExpr	::=	`if` *GeneratorClause* `then` *BlockElems Elifs*? *Else*? `end`
	|	`(if` *GeneratorClause* `then` *BlockElems Elifs*? *Else* `end ?)`
Elifs	::=	*Elif*$^+$
Elif	::=	`elif` *GeneratorClause* `then` *BlockElems*
Else	::=	`else` *BlockElems*
GeneratorClause	::=	*GeneratorBinding*
	|	*Expr*
GeneratorBinding	::=	*BindIdOrBindIdTuple*$\leftarrow$ *Expr*

An `if` expression consists of `if` followed by a generator clause (discussed in Section 13.14), followed by `then`, an expression block, an optional sequence of `elif` clauses (each consisting of `elif` followed by a generator clause, `then`, and an expression block), an optional `else` clause (consisting of `else` followed by an expression block), and finally `end`. Each clause forms an expression block and has the various properties of expression blocks (described in Section 13.11). An `if` expression first evaluates the expression of its generator clause (*condition expression*). If the generator clause is a generator binding, the type of the condition expression must be a subtype of $\mathrm{Condition}[\![E]\!]$ for some E and the type of the identifiers bound in the clause must be a subtype of E. A $\mathrm{Condition}$ is a generator that generates 0 or 1 element. Otherwise, the generator clause must be an expression of type $\mathrm{Boolean}$. Note in particular that true is a $\mathrm{Boolean}$ value yielding $()$ exactly once, while false is a $\mathrm{Boolean}$ value that yields no elements. If the evaluation of the condition expression completes normally and generates exactly one element, the `then` clause is evaluated. If the condition expression generates no elements, the next clause (either `elif` or `else`), if any, is evaluated. An `elif` clause works just as the original `if`, evaluating its condition expression and continuing with its `then` clause if the condition generates exactly one element. An `else` clause simply evaluates its expression block. The type of an `if` expression is the union of the types of the expression blocks of each clause. If there is no `else` clause in an `if` expression, then every clause must have type $()$. The result of the `if` expression is the result of the expression block which is evaluated; if no expression block is evaluated it is $()$. The reserved word `end` may be elided if the `if` expression is immediately enclosed by parentheses. In such a case, an `else` clause is required.

For example,

`if` $x \in \{0, 1, 2\}$ `then` 0

`elif` $x \in \{3, 4, 5\}$ `then` 3

`else` 6 `end`

13.20 Case Expressions

Syntax:

DelimitedExpr	::=	`case` *Expr* (*Encloser* | *Op*)? `of` *CaseClauses CaseElse*? `end`
CaseClauses	::=	*CaseClause*$^+$
CaseClause	::=	*Expr* $\Rightarrow$ *BlockElems*
CaseElse	::=	`else` $\Rightarrow$ *BlockElems*

A `case` expression begins with `case` followed by a condition expression, followed by an optional operator, `of`, a sequence of case clauses (each consisting of a *guarding expression* followed by the token $\Rightarrow$, followed by an expression block), an optional `else` clause (consisting of `else` followed by the token $\Rightarrow$, followed by an expression block), and finally `end`.

A `case` expression evaluates its condition expression and checks each case clause to determine which case clause matches. To find a matched case clause, the guarding expression of each case clause is evaluated in order and compared

to the value of the condition expression. For the first clause, the condition expression and guarding expression are evaluated in separate implicit threads; for subsequent clauses the value of the condition expression is retained and only the guarding expression is evaluated. Once both guard and condition expressions have completed normally, the two values are compared according to an optional operator specified. If the operator is omitted, it defaults to $=$ or $\in$. If the type of the guarding expression is a subtype of type Contains and the condition expression does not, the default operator is $\in$; otherwise, it is $=$. It is a static error if the specified operator is not defined for these types or if the operator's return type is not Boolean.

If the operator application completes normally and returns *true*, the corresponding expression block is evaluated (see Section 13.11) and its value is returned. If the operator application returns *false*, matching continues with the next clause. If no matched clause is found, a MatchFailure exception is thrown. The optional `else` clause always matches without requiring a comparison. The value of a `case` expression is the value of the right-hand side of the matched clause. The type of a `case` expression is the union of the types of all right-hand sides of the case clauses.

For example, the following `case` expression specifies the operator $\in$:

```
case planet ∈ of
  {"Mercury", "Venus", "Earth", "Mars"} ⇒ "inner"
  {"Jupiter", "Saturn", "Uranus", "Neptune"} ⇒ "outer"
  else ⇒ "remote"
end
```

but the following does not:

```
case 2 + 2 of
  4 ⇒ "it really is 4"
  5:7 ⇒ "we were wrong again"
end
```

13.21 Extremum Expressions

> DRAFT NOTE
> Reduction variables are not yet supported.

Syntax:

DelimitedExpr	::=	`case most` (*Encloser* | *Op*) `of` *CaseClauses* `end`
CaseClauses	::=	*CaseClause*$^+$
CaseClause	::=	*Expr* ⇒ *BlockElems*

An extremum expression uses the same syntax as a `case` expression (described in Section 13.20) except that `most` is used where a `case` expression would have a condition expression, the specified operator is not optional, and an extremum expression does not have an optional `else` clause.

All guarding expressions are evaluated in parallel in separate implicit threads as part of the same group in which the guarding expressions themselves are evaluated, in a manner analogous to reduction (see Section 5.4.1). The values of the guarding expressions are compared in parallel according to the operator specified. The specified operator must be a total order operator. Which pairs of guarding expressions are compared is unspecified, except that the pairwise comparisons performed will be sufficient to determine that the chosen clause is indeed the extremum (largest

or smallest depending on the specified operator) assuming a total order. Any or all pairwise comparisons may be considered.

The expression block of the clause with the extremum guarding expression (and only that clause) is evaluated. If more than one guarding expressions are tied for the extremum, the first clause in textual order is evaluated to yield the result of the extremum expression. The type of an extremum expression is the union of the types of all right-hand sides of the clauses.

For example, the following code:

`case most` $>$ `of`

$1\ mile \Rightarrow$ "`miles are larger`"

$1\ kilometer \Rightarrow$ "`we were wrong again`"

`end`

evaluates to "`miles are larger`".

13.22 Typecase Expressions

> DRAFT NOTE
> This section should be revised according to the changes in the pattern matching proposal.

> DRAFT NOTE
> Do we want to support `typecase self of` ...?
>
> How about `typecase` $(x, \texttt{self}, y)$ `of` ...?
>
> [07/23/2007] `self` is not allowed in a shorthand form of `typecase` expresiosn.

```
foo(x) = 8
bar(x) = 3
trait Parent
    foo(x) = 10
    baz(x) =
      typecase self of
        Child ⇒ bar(x)
        Parent ⇒ 23
      end
end
trait Child extends Parent
    bar(x) = foo(x)
end
```

Syntax:

DelimitedExpr	::=	`typecase` *TypecaseBindings* `of` *TypecaseClauses CaseElse*? `end`
TypecaseBindings	::=	*TypecaseVars* (= *Expr*)?
TypecaseVars	::=	*BindId*
	|	(*BindId*(, *BindId*)$^+$)
TypecaseClauses	::=	*TypecaseClause*$^+$
TypecaseClause	::=	*TypecaseTypes* ⇒ *BlockElems*
TypecaseTypes	::=	(*TypeList*)
	|	*Type*
CaseElse	::=	`else` ⇒ *BlockElems*

A `typecase` expression begins with `typecase` followed by an identifier or a sequence of parenthesized identifiers, optionally followed by the token = and a *binding* expression, followed by `of`, a sequence of typecase clauses (each consisting of a sequence of *guarding types* followed by the token ⇒, followed by an expression block), an optional `else` clause (consisting of `else` followed by the token ⇒, followed by an expression block), and finally `end`.

A `typecase` expression with a binding expression evaluates the expression first; if it completes normally, the value of the expression is bound to the identifiers and the first matching typecase clause is chosen. If there are multiple identifiers, the binding expression must be evaluated to a tuple value of the same number of elements. A typecase clause matches if the type of the value bound to each identifier is a subtype of the corresponding type in the clause. The expression block of the first matched clause (and only that clause) is evaluated (see Section 13.11) to yield the value of the `typecase` expression. If no matched clause is found, a MatchFailure exception is thrown (MatchFailure is an unchecked exception). Unlike bindings in other contexts, the static type of such an identifier is *not* determined by the static type of the expression it is bound to. If the static type of a subexpression in the bindings for a `typecase` expression has type T, when typechecking each typecase clause, the static type of the corresponding identifier is the intersection of T and the corresponding guarding type for that clause. The type of a `typecase` expression is the union of types of all right-hand sides of the typecase clauses.

> DRAFT NOTE
>
> Q. [Victor] In a typecase expression, suppose a more specific clause appears after a less specific one? Do we want that to be a static error (as it never makes sense to write that later clause since it will never be evaluated)?
>
> A. [Sukyoung & Eric] No. We may want to use the feature for reasoning and debugging in a devlopment environment. Some IDE may give a warning.

For example:

`typecase` $myLoser.myField$ `of`

x: String ⇒ x "`foo`"

x: Number ⇒ $x + 3$

Object ⇒ $yogiBerraAutograph$

`end`

Note that x has a different type in each clause.

For a `typecase` expression without a binding expression, the identifiers must be immutable variables in scope at that point. In that case, the `typecase` expression is equivalent to one that rebinds the variables to their values, with the new bindings shadowing the old ones: the first matching typecase clause is evaluated, and within that clause, the static type of the variable is the intersection of its original type and the guarding type. Otherwise, it is a static error.

DRAFT NOTE

- `typecase` shorthand form with a mutable variable:

```
typecase x of (* suppose x has static type Object *)
      String ⇒ x
      else⇒ x.toString()
  end
```

  Forbid this form if x is a mutable variable. We need to give an explanation with a concrete example why we disallow this. In general, we have a rule against rebinding an expression in a local scope.

- `self` is not allowed in a shorthand form of `typecase` expresiosn:

```
foo(x) = 8
bar(x) = 3
trait Parent
    foo(x) = 10
    baz(x) =
      typecase self of
        Child ⇒ bar(x)
        Parent ⇒ 23
      end
end
trait Child extends Parent
    bar(x) = foo(x)
end
```

13.23 Atomic Expressions

DRAFT NOTE

Modifiers on functionals such as `atomic` and `io` are not yet supported.

Syntax:

FlowExpr	::=	`atomic` *AtomicBack*
	|	`tryatomic` *AtomicBack*
AtomicBack	::=	*AssignExpr*
	|	*OpExpr*
	|	*DelimitedExpr*

As Fortress is a parallel language, an executing Fortress program consists of a set of threads (See Section 5.4 for a discussion of parallelism in Fortress). In multithreaded programs, it is often convenient for a thread to evaluate some expressions *atomically*. For this purpose, Fortress provides `atomic` expressions.

An `atomic` expression consists of `atomic` followed by a *body expression*. Evaluating an `atomic` expression is simply evaluating the body expression. All reads and all writes which occur as part of this evaluation will appear to occur simultaneously in a single atomic step with respect to *any* action performed by any thread which is dynamically outside. This is specified in detail in Chapter 21. The value and type of an `atomic` expression are the value and type of its body expression.

A `tryatomic` expression consists of `tryatomic` followed by an expression. It acts exactly like `atomic` except that in certain circumstances (see Section 23.5) it throws TryAtomicFailure and discards the effects of its body.

A function or method with the modifier `atomic` acts as if its entire body were surrounded in an `atomic` expression.

However, it is a static error if an API declares a functional f with the modifier `atomic` but a component implementing the API defines f whose body is an `atomic` expression without the modifier. Functionals with the modifier `io` cannot be called within an `atomic` expression. Thus, a functional must not have both `atomic` and `io` modifiers.

When the body of an `atomic` expression completes abruptly, the `atomic` expression completes abruptly in the same way. If it completes abruptly by exiting to an enclosing `label` expression, writes within the block are retained and become visible to other threads. If it completes abruptly by throwing an uncaught exception, all writes to objects allocated before the `atomic` expression began evaluation are discarded. Writes to newly allocated objects are retained. Any variable reverts to the value it held before evaluation of the `atomic` expression began. Thus, the only values retained from the abruptly completed `atomic` expression will be reachable from the exception object through a chain of newly allocated objects.

Atomic expressions may be nested arbitrarily; the above semantics imply that an inner `atomic` expression is atomic with respect to evaluations which occur dynamically outside the inner `atomic` expression but dynamically inside an enclosing `atomic`.

Implicit threads may be created dynamically within an `atomic` expression. These implicit threads will complete before the `atomic` expression itself does so. The implicit threads may run in parallel, and will see one another's writes; they may synchronize with one another using nested `atomic` expressions.

Note that `atomic` expressions may be evaluated in parallel with other expressions. An `atomic` expression experiences *conflict* when another thread attempts to read or write a memory location which is accessed by the `atomic` expression. The evaluation of such an expression must be partially serialized with the conflicting memory operation (which might be another `atomic` expression). The exact mechanism by which this occurs will vary; the necessary serialization is provided by the implementation. In general, the evaluation of a conflicting `atomic` expression may be abandoned, forcing the effects of execution to be discarded and execution to be retried. The longer an `atomic` expression evaluates and the more memory it touches the greater the chance of conflict and the larger the bottleneck a conflict may impose.

For example, the following code uses a shared counter atomically:

$sum\colon \mathbb{Z}32 := 0$

$accumArray[\![N \texttt{ extends Number}, \texttt{nat } x]\!](a\colon \mathrm{Array1}[\![N, 0, x]\!])\colon () =$

```
  for i ← a.indices do

    atomic sum += a_i

  end
```

The loop body reads a_i and sum, then adds them and writes the result back to sum; this will appear to occur atomically with respect to all other threads—including both other iterations of the loop body and other simultaneous calls to $accumArray$. Note in particular that the `atomic` expression will appear atomic with respect to reads and writes of a_i and sum that do not occur in `atomic` expressions.

13.24 Spawn Expressions

> DRAFT NOTE
> Static checking for `io` actions are not yet supported.

Syntax:

FlowExpr ::= `spawn` *Expr*

A spawned thread is created using a `spawn` expression. A `spawn` expression consists of `spawn` followed by an expression. A `spawn` expression spawns a thread which evaluates its subexpression in parallel with any succeeding evaluation. The value of a `spawn` expression is the spawned thread and the type of the expression is $\mathrm{Thread}[\![T]\!]$, where T is the static type of the expression spawned. A `spawn` expression constitutes an `io` action, and thus cannot be run within the body of an `atomic` expression. A `spawn` expression cannot be run within the body of an `atomic` expression. The semantics of spawned threads are discussed in Section 5.4.

13.25 Throw Expressions

Syntax:

FlowExpr ::= `throw` *Expr*

A `throw` expression consists of `throw` followed by a subexpression. The type of the subexpression must be a subtype of the type Exception (see Chapter 14). A `throw` expression evaluates its subexpression to an exception value and throws the exception value; the expression completes abruptly and has BottomType.

The type Exception has exactly two direct mutually exclusive subtypes, CheckedException and UncheckedException. Every CheckedException that is thrown must be caught or forbidden by an enclosing `try` expression (see Section 13.26), or it must be declared in the `throws` clause of an enclosing functional declaration (see Section 9.1). Similarly, every CheckedException declared to be thrown in the static type of a functional called must be either caught or forbidden by an enclosing `try` expression, or declared in the `throws` clause of an enclosing functional declaration.

13.26 Try Expressions

> DRAFT NOTE
>
> Chained exceptions are not yet supported.

Syntax:

DelimitedExpr ::= `try` *BlockElems Catch*? (`forbid` *TraitTypes*)? (`finally` *BlockElems*)? `end`
Catch ::= `catch` *BindId CatchClauses*
CatchClauses ::= *CatchClause*$^+$
CatchClause ::= *TraitType* $\Rightarrow$ *BlockElems*

A `try` expression starts with `try` followed by an expression block (the `try` *block*), followed by an optional `catch` clause, an optional `forbid` clause, an optional `finally` clause, and finally `end`. A `catch` clause consists of `catch` followed by an identifier, followed by a sequence of subclauses (each consisting of an exception type followed by the token $\Rightarrow$ followed by an expression block). A `forbid` clause consists of `forbid` followed by a set of exception types. A `finally` clause consists of `finally` followed by an expression block. Note that the `try` block and the clauses form expression blocks and have the various properties of expression blocks (described in Section 13.11).

The expressions in the `try` block are first evaluated in order until they have all completed normally, or until one of them completes abruptly. If the `try` block completes normally, the *provisional* value of the `try` expression is the value of the last expression in the `try` block. In this case, and in case of exiting to an enclosing `label` expression, the `catch` and `forbid` clauses are ignored.

If an expression in the `try` block completes abruptly by throwing an exception, the exception value is bound to the identifier specified in the `catch` clause, and the type of the exception is matched against the subclauses of the `catch` clause in turn, exactly as in a `typecase` expression (Section 13.22). The right-hand-side expression block of the first matching subclause is evaluated. If it completes normally, its value is the provisional value of the `try` expression. If

the `catch` clause completes abruptly, the `try` expression completes abruptly. If a thrown exception is not matched by the `catch` clause (or this clause is omitted), but it is a subtype of the exception type listed in a `forbid` clause, a new ForbiddenException is created with the thrown exception as its argument and thrown. The exception thrown by the `try` block is *chained* to the ForbiddenException as described in Section 14.3.

If an exception thrown from a `try` block is matched by both `catch` and `forbid` clauses, the exception is caught by the `catch` clause. If an exception thrown from a `try` block is not matched by any `catch` or `forbid` clause, the `try` expression completes abruptly.

The expression block of the `finally` clause is evaluated after completion of the `try` block and any `catch` or `forbid` clause. The type of this expression block must be (). The expressions in the `finally` clause are evaluated in order until they have all completed normally, or until one of them completes abruptly. In the latter case, the `try` expression completes abruptly exactly as the subexpression in the `finally` clause does. If the `finally` clause completes abruptly by throwing an exception, any thrown exception earlier in the `try` expression is chained to the exception thrown by the `finally` clause as described in Section 14.3.

If the `finally` clause completes normally, and the `try` block or the `catch` clause completes normally, then the `try` expression completes normally with the provisional value of the `try` expression. Otherwise, the `try` expression completes abruptly as specified above. The type of a `try` expression is the union of the type of the `try` block and the types of all the right-hand sides of the `catch` clauses.

For example, the following `try` expression:

`try`

$inp = read(file)$

$write(inp, newFile)$

`forbid` IOFailure

`end`

is equivalent to:

`try`

$inp = read(file)$

$write(inp, newFile)$

`catch` e

IOFailure $\Rightarrow$ `throw` ForbiddenException(e)

`end`

The following example ensures that $file$ is closed properly even if an IO error occurs:

`try`

$open(file)$

$inp = read(file)$

$write(inp, newFile)$

`catch` e

IOFailure $\Rightarrow$ `throw` ForbiddenException(e)

```
finally
  close(file)
end
```

13.27 Static Expressions

> DRAFT NOTE
>
> Of the static expressions in a size, the interpreter computes a sum, a difference, a product and a power exactly, and refuses a power at a negative exponent; the compiled type checker computes each operation on numerals, refusing as out of range a power it does not compute, and compares other size expressions by their written form (Chapter 6); the compiled path computes a size when it names a class. The compiled type checker does not infer a `bool`, `dim` or `unit` static argument.

Static expressions denote *static values*. Conceptually, a static expression is not evaluated while a program is running, and thus its evaluation cannot complete abruptly. Given instantiations of all static parameters (described in Chapter 12) in scope of a static expression, the value of the static expression can be determined statically. Static expressions which use a restricted subset of operations can occur as *static arguments* which instantiate static parameters and `where`-clause constraints (described in Section 12.6). We define the set of static expressions by first defining the types of static expressions, and distinguishing static values from the closely related literal values. We then describe the expressions that evaluate to the various kinds of static values.

13.27.1 Types of Static Expressions

There are three groups of traits that describe literals and static expressions:

1. The *literal* traits, which describe boolean, character, string, and numerals. For example, the literal $true$ has trait $\mathrm{BooleanLiteral}[\![true]\!]$ and a character literal has trait $\mathrm{Character}$. See Section 13.1 for a discussion of Fortress literals.
2. The *constant* traits, which describe values denoted by expressions composed from literals and operators (described in Section 13.27.2); the type of a constant expression encodes the value of the expression. For example, the type of a constant expression $3+5$, $\mathrm{IntegerConstant}[\![false, 3+5]\!]$, encodes the value of the expression; the operator $+$ is used in the static arguments. When an operator is not supported in static arguments, the Fortress standard libraries can still encode the value of the expression by providing a specific constant trait such as $\mathrm{RationalValueTimesPi}[\![false, 1, 1]\!]$ (described in Section 13.1.1).
3. The *static* traits, which describe values denoted by expressions composed from literals, operators (described in Section 13.27.2), and `bool`, `nat`, and `int` parameters; here the type does not encode the value of the expression, but the value of the expression can nevertheless be known statically if specific values are specified for the static parameters. Also, in situations where the type of an expression composed solely from literals and operators nevertheless cannot be described by a constant trait, then a static trait may be used to describe it instead. For example, a static expression $2(3+m)$ where m is a `nat` parameter has trait $\mathrm{NaturalStatic}$.

The only operation on literals that produces a new literal (as opposed to a constant) is concatenation by using the operator $\|$. One may concatenate mixed string and character literals, producing a string literal. For example, "`foo`" $\|$ "`bar`" yields "`foobar`". One may also concatenate two natural numerals of the same radix, producing a new natural numeral of that radix. For example, $\texttt{deadc}_{16} \| \texttt{0de}_{16}$ yields $\texttt{deadc0de}_{16}$.

Every literal trait extends an appropriate constant trait, and every constant trait extends an appropriate static trait. So every literal is also a constant expression, and every constant expression is a static expression.

13.27.2 Static Expressions and Values

Static parameters are static expressions. A `nat` parameter denotes a value that has type NaturalStatic (which extends IntegerStatic). An `int` parameter denotes a value that has type IntegerStatic. A `bool` parameter denotes a value that has type BooleanStatic.

Boolean static expressions may be combined using the operators $\wedge$, $\vee$, $\oplus$, $\equiv$, $\leftrightarrow$, `NAND`, `NOR`, $=$, $\neq$, and $\rightarrow$ (See Appendix F for a discussion of Fortress operators.) to produce other static expressions denoting boolean static expressions. If both operands are boolean constant expressions, then the result is also a boolean constant expression.

Character static expressions may be compared using the operators $<$, $\leq$, $\geq$, $>$, $=$, and $\neq$ to produce boolean static expressions. If both operands are character constant expressions, then the result is a boolean constant expression.

Numeric static expressions may be compared using the operators $<$, $\leq$, $\geq$, $>$, $=$, and $\neq$ to produce boolean static expressions. If both operands are numeric constant expressions, then the result is a boolean constant expression.

Numeric static expressions may be combined using the operators and functions $+$, $-$, $\times$, $\cdot$, $/$, `!`, `MIN`, `MAX`, $\sqrt{}$, *floor*, *ceiling*, *hyperfloor*, *hyperceiling*, gcd, lcm, sin, cos, tan, arcsin, arccos, and arctan to produce new numeric static expressions. If the result is indeed a numeric static expression, and both operands are numeric constant expressions, then the result is also a numeric constant expression as long as the constant traits provided by the Fortress standard libraries can encode the value of the expression.

13.28 Tuple Expressions

Syntax:

TupleExpr ::= ((*Expr*,)$^+$ *Expr*)

A tuple expression is an ordered sequence of expressions separated by commas and enclosed in parentheses. There must be at least two element expressions. The type of a tuple expression is a tuple type (as discussed in Section 6.5). Each element of a tuple is evaluated in parallel in a separate implicit thread (see Section 5.4).

13.29 Aggregate Expressions

> DRAFT NOTE
> Some operators and matrix unpasting are not yet supported.

> DRAFT NOTE
> $\mapsto$ is a special syntax only in two contexts: map aggregates and map comprehensions. (02/12/09)

Syntax:

Primary	::=	*LeftEncloser StaticArgs*? *ExprList*? *RightEncloser*
ExprList	::=	*Expr*(, *Expr*)*
ArrayExpr	::=	[*StaticArgs*? *RectElements*]
RectElements	::=	*Expr MultiDimCons**
MultiDimCons	::=	*RectSeparator Expr*
RectSeparator	::=	;+
	|	*Whitespace*
MapExpr	::=	{ *StaticArgs*? *EntryList*? }

Aggregate expressions evaluate to values that are themselves homogeneous collections of values. Each subexpression of an aggregate expression is evaluated in parallel in a separate implicit thread (see Section 5.4). With the exception of array expressions, the resulting values are passed to the appropriate bracketing operator, which is a function with a varargs parameter constructing the aggregate. The array aggregate operations construct the aggregate directly, after all subexpressions have completed normally. Any aggregate expression may reserve storage for its result before its elements complete evaluation.

Functions defining aggregate expressions are provided in the Fortress standard libraries for sets, maps, lists, matrices, vectors, and arrays.

13.29.1 Set Expressions

Set element expressions are enclosed in braces and separated by commas. The type of a set expression is $\mathrm{Set}[\![T]\!]$, where T is the union type of the types of all element expressions of the set expression.

Set containment is checked with the operator $\in$ and the subset relationship is checked with the operator $\subseteq$. For example:

$3 \in \{0, 1, 2, 3, 4, 5\}$

evaluates to $true$. and

$$\{0, 1, 2\} \subseteq \{0, 3, 2\}$$

This example should be added to the SpecData directory when it works.

evaluates to $false$.

13.29.2 Map Expressions

Map entries are enclosed in curly braces, separated by commas, and matching pairs are separated by $\mapsto$. The type of a map expression is $\mathrm{Map}[\![K, V]\!]$ where K is the union of the types of all left-hand-side expressions of the map entries, and V is the union of the types of all right-hand-side expressions of the map entries.

A map m is indexed by placing an element in the domain of m enclosed in brackets immediately after an expression evaluating to m. Thus, the index is rendered as a subscript. For example, if:

The new fortify fixes this.

$m = \{\, \text{“a”} \mapsto 0, \text{“b”} \mapsto 1, \text{“c”} \mapsto 2 \,\}$

then $m_{\text{“b”}}$ evaluates to 1. In contrast, $m_{\text{“x”}}$ throws a $\mathrm{NotFound}$ exception, as “x” is not an index of m.

13.29.3 List Expressions

List element expressions are enclosed in angle brackets $[\![$ and $]\!]$ and are separated by commas. The type of a list expression is $\mathrm{List}[\![T]\!]$ where T is the union type of the types of all element expressions.

A list l is indexed by placing an index enclosed in square brackets immediately after an expression evaluating to l. Thus, the index is rendered as a subscript. Lists are always indexed from 0. For example:

The new fortify fixes this.

$\langle 3, 2, 1, 0 \rangle [2]$

evaluates to 1.

13.29.4 Array Expressions

Array element expressions are enclosed in brackets. Element expressions along a row are separated only by whitespace. Two dimensional array expressions are written by separating rows with newlines or semicolons. If a semicolon appears, whitespace before and after the semicolon is ignored. The parts of higher-dimensional array expressions are separated by repeated-semicolons, where the dimensionality of the result is equal to one plus the number of repeated semicolons. The type of a k-dimensional array expression is $\mathrm{Array}k[\![T, 0, n_0, \cdots, 0, n_{k-1}]\!]$, where T is the union type of the types of the element expressions and $n_0, ..., n_{k-1}$ are the sizes of the array in each dimension[1]. This type can be abbreviated as $T[n_0, \cdots, n_{k-1}]$.

A k-dimensional array A is indexed by placing a sequence of k indices enclosed in brackets, and separated by commas, after an expression evaluating to A. Thus, the index is rendered as a subscript. By default, arrays are indexed from 0. The horizontal dimension of an array is the last dimension mentioned in the array index. For example:

$$A{:}\,\mathbb{Z}32_{3,3} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

then $A_{1,0}$ evaluates to 4.

two or more – Jan: I continue to disbelieve.

An array of two dimensions whose elements are a subtype of Number is a matrix. Matrices are indexed in the same manner as arrays.

can be coerced to a matrix. Matrix types are written $\mathrm{Matrix}[\![T]\!][n_0 \times \ldots \times n_{k-1}]$, where $k > 1$. A type of this form can be abbreviated as $T^{n_0 \times \cdots \times n_{k-1}}$

A one-dimensional array whose elements are a subtype of Number is a vector.

can be coerced to a vector. Vector types are written $\mathrm{Vector}[\![T]\!][n]$. A type of this form can be abbreviated as T^n, unless T is declared in the enclosing scope to be a physical dimension or unit. Matrices occurring inside a matrix literal on the same row are pasted along dimension 0. Each row is pasted along dimension 1 and the resulting matrix is "flattened". This works because matrices never contain other matrices as elements.

The element expressions in an array expression may be either scalars or array expressions themselves. If an element is an array expression, it is "flattened" (pasted) into the enclosing expression. This pasting works because array expressions never contain other arrays as elements. The elements along a row (or column) must have the same number of columns (or rows), though two elements in different rows (columns) need not have the same number of columns (rows). See Section 8.3 for a discussion of matrix unpasting.

The following four examples are all equivalent:

$$A{:}\,\mathbb{Z}32_{2,2} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$$

$$A{:}\,\mathbb{Z}32_{2,2} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$$

$$A{:}\,\mathbb{Z}32_{2,2} = \begin{bmatrix} 3 & 4 \\ \\ 5 & 6 \end{bmatrix}$$

$$A{:}\,\mathbb{Z}32_{2,2} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Here is a $3 \times 3 \times 3$ matrix example:

[1 0 0

0 1 0

0 0 1;;0 1 0

[1] As of March 2008, the Fortress library provides $\mathrm{Array}k$ types for $1 \leq k \leq 3$.

1 0 1

0 1 0; ; 1 0 1

0 1 0

1 0 1]

13.30 Comprehensions

> DRAFT NOTE
> Array comprehensions are not yet supported.
>
> Array comprehensions do not have to be comprehensive – may not define a value for every cell. When an unitialized array cell is accessed, an uncaught exception is raised.

Syntax:

Comprehension	::=	`BIG` ? `[` *StaticArgs*? *ArrayComprehensionClause*$^+$ `]`
	|	`BIG` ? `{` *StaticArgs*? *Entry* `|` *GeneratorClauseList* `}`
	|	`BIG` ? *LeftEncloser StaticArgs*? *Expr* `|` *GeneratorClauseList RightEncloser*
Entry	::=	*Expr* $\mapsto$ *Expr*
ArrayComprehensionClause	::=	*ArrayComprehensionLeft* `|` *GeneratorClauseList*
ArrayComprehensionLeft	::=	*IdOrInt* $\mapsto$ *Expr*
	|	`(` *IdOrInt*, *IdOrIntList* `)` $\mapsto$ *Expr*
IdOrInt	::=	*Id*
	|	*IntLiteralExpr*
IdOrIntList	::=	*IdOrInt*(, *IdOrInt*)*

Fortress provides *comprehension* syntax, in which a generator clause list binds values used in the body expression on the left-hand side of the token `|`. As described in Section 13.14, each iteration of the body expression must be assumed to execute in its own implicit thread. Comprehensions evaluate to aggregate values and have corresponding aggregate types. The rules for evaluation of a comprehension are similar to those for a reduction expression (see Section 13.17).

The relationship between a comprehension and an aggregate expression (Section 13.29) is similar to the relationship between a reduction expression (Section 13.17) and the corresponding infix operator application (Section 13.7). The language does not enforce an explicit connection between comprehension syntax and the corresponding aggregate syntax, but in practice libraries that provide definitions for aggregate expressions are expected to define a corresponding comprehension and *vice versa*. As with reduction expressions, a comprehension using a particular set of enclosers corresponds to a call to a *big bracketing operator*. Thus the definition of set comprehensions is given by a function with the following signature:

$$\texttt{opr BIG}\{[\![T]\!]g : (\mathrm{Reduction}[\![R_0]\!], T \to R_0) \to R_0\} : \mathrm{Set}[\![T]\!]$$

This is almost identical to the signature required to define a reduction expression `BIG` *Op*, shown in Section 13.17. Further information on defining comprehensions are described in Section 23.8.

A set comprehension is enclosed in braces, with a left-hand body separated by the token `|` from a generator clause list. For example, the comprehension:

$$\left\{ x^2 \;\middle|\; x \leftarrow \{0,1,2,3,4,5\}, x \;\texttt{MOD}\; 2 = 0 \right\}$$

evaluates to the set

$$\{0, 4, 16\}$$

Map comprehensions are like set comprehensions, except that the left-hand body must be of the form $e_1 \mapsto e_2$. If e_1 produces the same value but e_2 a different value on more than one iteration of the generator list, a KeyOverlap exception is thrown. For example:

$\{ x^2 \mapsto x^3 \mid x \leftarrow \{0,1,2,3,4,5\}, x \texttt{ MOD } 2 = 0 \}$

evaluates to the map

$\{0 \mapsto 0, 4 \mapsto 8, 16 \mapsto 64\}$

List comprehensions are like set comprehensions, except that they are syntactically enclosed in angle brackets. For example:

$\langle x^2 \mid x \leftarrow \{0,1,2,3,4,5\}, x \texttt{ MOD } 2 = 0 \rangle$

evaluates to the list

$\langle 0, 4, 16 \rangle$

Note that the order of elements in the resulting list corresponds to the *natural order* of the generators in the generator clause list (see Section 23.8).

Array comprehensions are like set comprehensions, except that they are syntactically enclosed in brackets, and the left-hand body must be of the form $\texttt{(}index_1\texttt{, } index_2\texttt{, ..., } index_n\texttt{)} \mapsto e$. Moreover an array comprehension may have multiple clauses separated by semicolons or line breaks. Each clause conceptually corresponds to an independent loop. Clauses are run in order. The result is an n-dimensional array. For example:

$$\begin{aligned} a = [&(x,y,1) \mapsto 0.0 \mid x \leftarrow 1 : xSize, y \leftarrow 1 : ySize \\ &(1,y,z) \mapsto 0.0 \mid y \leftarrow 1 : ySize, z \leftarrow 2 : zSize \\ &(x,1,z) \mapsto 0.0 \mid x \leftarrow 2 : xSize, z \leftarrow 2 : zSize \\ &(x,y,z) \mapsto x + y \cdot z \mid x \leftarrow 2 : xSize, y \leftarrow 2 : ySize, z \leftarrow 2 : zSize\,] \end{aligned}$$

13.31 Type Ascription

Syntax:

Expr ::= *Expr* `typed` *Type*

An expression consisting of a single subexpression, followed by `typed`, followed by a type, is a *type ascription*. The value of the expression is the value of the subexpression. The static type of the expression is the ascribed type. The type of the subexpression must be a subtype of the ascribed type. A type ascription does not affect the dynamic type of the value the expression evaluates to (unlike a type assumption described in Section 13.32). Type ascription can be used to provide type information when type inference (see Chapter 20) cannot infer a type for an expression.

13.32 Type Assumption

Syntax:

Expr ::= *Expr* `asif` *Type*

An expression consisting of a single subexpression, followed by `asif`, followed by a type, is a *type assumption*. The value of the expression is the value of the subexpression. The static type of the expression is the given type. The type of the subexpression must be a subtype of the given type. A type assumption considers both the static and the dynamic type of the value of the expression to be the specified type for the purposes of the immediately enclosing function, method, or operator invocation or field access. This is in contrast to type ascription, which only gives a *static* type to

an expression. Type assumption is used to access a method provided by a supertrait when multiple supertraits provide different methods with the same name. Fortress thus provides a richer version of type assumption operations such as `super` in the Java Programming Language.

13.33 Expression-like Functions

> DRAFT NOTE
> Some functions are not yet supported. Even for the supported functions, they are not run by the interpreter yet.

For convenience, the Fortress standard libraries provide functions such as *cast* and *instanceOf* that are often provided by other programming languages.

13.33.1 Casting

Although there is no "casting" operator (equivalent to casts in the Java Programming Language) built into Fortress, the effect of a cast can be provided by the following function:

I manually added space around the second ⇒ – Sukyoung

```
cast⟦T extends Any⟧(x: Any) : T =
  typecase x of
    T ⇒ x
    else⇒ throw CastError
  end
```

The function converts the type of its argument to a given type. If the static type of the argument is not a subtype of the given type, a CastError is thrown. For convenience, the function *cast* is included in the Fortress standard libraries.

13.33.2 Instanceof Testing

Although there is no "instanceof" operator (equivalent to instanceof testing in the Java Programming Language) built into Fortress, the effect of an instanceof testing can be provided by the following function:

I manually added space around the second ⇒ – Sukyoung

```
instanceOf⟦T extends Any⟧(x: Any) : Boolean =
  typecase x of
    T ⇒ true
    else⇒ false
  end
```

The function tests whether its argument has a given type and returns a boolean value. For convenience, the function *instanceOf* is included in the Fortress standard libraries.

13.33.3 Ignoring Values

For convenience, the function *ignore* (equivalent to the `ignore` function in the Objective Caml programming language) is included in the Fortress standard libraries:

```
ignore(x: Any) = ()
```

The function discards the value of its argument and returns `()`. For example, the following:

I manually added space between f and x – Sukyoung

```
ignore(f x)
```

I manually added space between f and x – Sukyoung

is equivalent to:

$$\texttt{do}\ f\ x; ()\ \texttt{end}$$

13.33.4 Identity

An identity function $identity$ is defined in the Fortress standard libraries:

$$identity[\![T\ \texttt{extends}\ \mathrm{Any}]\!](x{:}\,T) = x$$

The function returns its argument as it is.

13.33.5 Coercion

An identity function coerce is defined in the Fortress standard libraries to convert the type of its argument to its type argument:

$$\mathrm{coerce}[\![T]\!](x{:}\,T) = x$$

The function returns its argument as the given type. Unlike coercions described in Chapter 17, the coerce function can apply to an argument whose type is a subtype of the type being coerced to.

Chapter 14

Exceptions

> DRAFT NOTE
> Methods and fields of exceptions, chained exceptions, and static checking of checked exceptions are not yet supported.
>
> Fortress examples in this chapter are not run by the interpreter.

Exceptions are values that can be thrown and caught, via `throw` expressions (described in Section 13.25) and `catch` clauses of `try` expressions (described in Section 13.26). When a `throw` expression "`throw` e" is evaluated, the subexpression e is evaluated to an exception. The static type of e must be a subtype of $\mathrm{Exception}$. Then the `throw` expression tries to transfer control to its *dynamically containing block* (described in Chapter 5), from the innermost outward, until either (*i*) an enclosing `try` expression is reached, with a `catch` clause matching a type of the thrown exception, or (*ii*) the outermost dynamically containing block is reached.

If a matching `catch` clause is reached, the right-hand side of the first matching subclause is evaluated. If no matching `catch` clause is found before the outermost dynamically containing block is reached, the outermost dynamically containing block completes abruptly whose cause is the thrown exception.

If an enclosing `try` expression of a `throw` expression includes a `finally` clause, and the `try` expression completes abruptly, the `finally` clause is evaluated before control is transferred to the dynamically containing block.

14.1 Causes of Exceptions

Every exception is thrown for one of the following reasons:

1. A `throw` expression is evaluated.
2. An implementation resource is exceeded (e.g., an attempt is made to allocate beyond the set of available locations).

14.2 Types of Exceptions

All exceptions are subtypes of the type $\mathrm{Exception}$ declared as follows:

```
trait Exception comprises { CheckedException, UncheckedException }
  settable message: Maybe⟦String⟧
  settable chain: Maybe⟦Exception⟧
  printStackTrace(): ()
end
```

Every exception is a subtype of either type CheckedException or UncheckedException:

```
trait CheckedException extends Exception excludes UncheckedException

end

trait UncheckedException extends Exception excludes CheckedException

end
```

A functional declaration (described in Section 10.2 and Section 9.1) includes an optional `throws` clause in its header listing the CheckedExceptions (also written *checked exceptions*) that can be thrown by invocation of the functional. If a `throws` clause is not explicitly provided, the `throws` clause of the functional declaration is empty. The body of a functional is statically checked to ensure that no checked exceptions are thrown by any subexpression of the functional body other than those listed in the `throws` clause. This static check is performed by examining each `throw` expression and functional invocation I, determining the static type of the functional f invoked in I, and determining the `throws` clause of f. (If f is polymorphic, or occurs in a polymorphic context, instantiations of type variables free in the `throws` clause of f are substituted for formal type variables). For each checked exception thrown in I, the enclosing expressions of I are checked for a matching `catch` clause. The set A of all checked exceptions thrown by all invocations without a matching `catch` clause in the functional body is accumulated and compared against the `throws` clause of the enclosing functional declaration. If an exception that is not a subtype of an exception listed in the `throws` clause occurs in A, it is a static error.

A similar analysis is performed on top-level variable declarations. If it is determined that their initialization expressions can throw a checked exception, it is a static error.

14.3 Information of Exceptions

Every exception has optional fields: a message and a chained exception. These fields are default to Nothing⟦String⟧ and Nothing⟦Exception⟧ as follows:

```
trait Exception comprises { CheckedException, UncheckedException }
  getter message(): Maybe⟦String⟧ = Nothing⟦String⟧
  setter message(Maybe⟦String⟧) : ()
  getter chain(): Maybe⟦Exception⟧ = Nothing⟦Exception⟧
  setter chain(Maybe⟦Exception⟧) : ()
  printStackTrace(): ()
end
```

where an optional value v of type Maybe⟦T⟧ is either Nothing⟦T⟧ or Just(v).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.7
>
> The Working Draft of February 2011 wrote both defaults as a bare Nothing, one object for every instantiation of Maybe, which instantiation exclusion forbids (Section 39.2).

The *chain* field can be set at most once. If it is set more than once, an InvalidChain exception is thrown. It is generally set when the exception is created.

When an exception is created, the execution stack of its thread at the time of the exception creation is captured in the exception. The invocation of $printStackTrace$ prints the captured stack trace. There is no way to update the captured stack trace. If a programmer wants to catch a thrown exception and rethrow it, and capture the stack trace at the time of the second throwing of the exception, the programmer has to create a new exception (perhaps with the original exception as its $chain$ field).

When an exception is thrown, its $message$ and $chain$ fields may be set. For example, if a checked exception is caught in a `catch` clause, and the `catch` clause in turn throws an unchecked exception, the unchecked exception can be chained so that an examination of the unchecked exception reveals information about the original exception. For example:

$read(fileName) =$
  `try`
    $readFile(fileName)$
  `catch` e
    IOFailure $\Rightarrow$ `throw` Error("`This code can't handle IOFailures`", e)
  `end`

where the $message$ and $chain$ fields of Error are set to "`This code can't handle IOFailures`" and IOFailure respectively.

By default, a `forbid` clause in a `try` expression throws a new ForbiddenException by *chaining* the exception thrown by the `try` block in the `try` expression that is a subtype of the exception type listed in the `forbid` clause. For example, the following $read$ function:

$read(fileName) =$
  `try`
    $readFile(fileName)$
  `forbid` IOFailure
  `end`

is equivalent to:

$read(fileName) =$
  `try`
    $readFile(fileName)$
  `catch` e
    IOFailure $\Rightarrow$`throw` ForbiddenException(e)
  `end`

where the $chain$ of ForbiddenException is set to IOFailure.

Chapter 15

Overloading and Multiple Dispatch

> DRAFT NOTE
> Keyword and varargs parameters are not yet supported.

Fortress allows functions and methods (collectively called *functionals*) to be *overloaded*. That is, there may be multiple declarations for the same functional name visible in a single scope (which may include inherited method declarations), and several of them may be applicable to any particular functional call.

However, with these benefits comes the potential for ambiguous calls at run time. For an overloaded functional call, the most specific applicable declaration is chosen, if any. Otherwise, any of the applicable declarations such that no other applicable declaration is more specific than them is chosen.

In this chapter, we describe how to determine which declarations are *applicable* to a particular functional call, and when several are applicable, how to select among them. We also provide overloading rules for the declarations of functionals to eliminate *some possibility* of ambiguous calls at run time, whether or not these calls actually appear in the program. Section 15.1 introduces some terminology and notation. In Section 15.2, we show how to determine which declarations are applicable to a *named functional call* (a function call described in Section 13.6 or a naked method invocation described in Section 13.5) when all declarations have only ordinary parameters (without varargs or keyword parameters). We discuss how to handle dotted method calls (described in Section 13.4) in Section 15.3, and declarations with varargs and keyword parameters in Section 15.4. Determining which declaration is applied, if several are applicable, is discussed in Section 15.5.

15.1 Principles of Overloading

Fortress allows multiple functional declarations of the same name to be declared in a single scope. However, recall from Chapter 7 the following shadowing rules:

- dotted method declarations shadow top-level function declarations with the same name, and
- dotted method declarations provided by a trait or object declaration or object expression shadow functional method declarations with the same name that are provided by a different trait or object declaration or object expression.

Also, note that a trait or object declaration or object expression must not have a functional method declaration and a dotted method declaration with the same name, either directly or by inheritance. Therefore, top-level functions can overload with other top-level functions and functional methods, dotted methods with other dotted methods, and

functional methods with other functional methods and top-level functions. It is a static error if any top-level function declaration is more specific than any functional method declaration. If a top-level function declaration is overloaded with a functional method declaration, the top-level function declaration must not be more specific than the functional method declaration.

Operator declarations with the same name but different fixity are not a valid overloading; they are unambiguous declarations. An operator method declaration whose name is one of the operator parameters (described in Section 12.5) of its enclosing trait or object may be overloaded with other operator declarations in the same component. Therefore, such an operator method declaration must satisfy the overloading rules (described in Chapter 24) with every operator declaration in the same component.

> DRAFT NOTE
> This restriction will be relaxed.

Declarations with the same functional name may have different static parameters, or some of them may have static parameters and others none. A declaration with static parameters is read as one declaration whose parameter type ranges over every instance of it whose static arguments satisfy their bounds: it is applicable to a call when some such instance is (Section 15.2), and it is compared with other declarations on the parameter types they declare, taken over all their instances (Section 15.5). A declaration without static parameters is the case with none. The rules that a set of such declarations must satisfy are given in Chapter 24.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
> The Working Draft of February 2011 said here that declarations of one name may not differ in their static parameters, nor may one have static parameters and another not, so that static parameters do not enter into applicability. Neither implementation enforced that: the interpreter has overloaded declarations with different static parameters since 2007, and the compiled type checker applies to them the rules of the type group's paper of 2011, which reads the parameter type of a declaration with static parameters as quantified over them; the Fortress standard libraries overload across different static parameters throughout. This chapter and Chapter 24 now state that model.

Recall from Chapter 6 that we write $T \preceq U$ when T is a subtype of U, and $T \prec U$ when $T \preceq U$ and $T \neq U$.

15.2 Applicability to Named Functional Calls

In this section, we show how to determine which declarations are applicable to a named functional call when all declarations have only ordinary parameters (i.e., neither varargs nor keyword parameters).

For the purpose of defining applicability, a named functional call can be characterized by the name of the functional and its argument type. Recall that a functional has a single parameter, which may be a tuple (a dotted method has a receiver as well). We abuse notation by using *static call* $f(A)$ to refer to a named functional call with name f and whose argument has static type A, and *dynamic call* $f(X)$ to refer to a named functional call with We use *call* $f(C)$ to refer to a named functional call with name f and whose argument, when evaluated, has dynamic type C. (Note that if the type system is sound—and we certainly hope that it is!—then $X \preceq A$ for all well-typed calls to f.) We use the term *call* $f(C)$ to refer to static and dynamic calls collectively. We assume throughout this chapter that the static arguments that a call writes (Section 13.2, Section 13.4) have been substituted for the static parameters of the declarations that they instantiate. The static arguments that a call does not write are inferred for the declaration that the call selects, once it is selected (Chapter 20); until then a declaration with static parameters is taken over all its instances.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 assumed here that all static variables in functional calls have been instantiated or inferred, which with the sentences below had the static arguments a call does not write inferred before the declarations are compared.

We also use *function declaration* $f(P) : U$ to refer to a function declaration with function name f, parameter type P, and return type U.

For method declarations, we must take into account the self parameter, as follows:

A *dotted method declaration* $P_0.f(P) : U$ is a dotted method declaration with name f, where P_0 is the trait or object type in which the declaration appears, P is the parameter type, and U is the return type. (Note that despite the suggestive notation, a dotted method declaration does not explicitly list its self parameter.)

A *functional method declaration* $f(P) : U$ *with self parameter at* i is a functional method declaration with name f, with the parameter `self` in the ith position of the parameter type P, and return type U. Note that the static type of the self parameter is the trait or object trait type in which the declaration $f(P) : U$ occurs. In the following, we will use P_i to refer to the ith element of P.

We elide the return type of a declaration, writing $f(P)$ and $P_0.f(P)$, when the return type is not relevant to the discussion. Note that static parameters may appear in the types P_0, P, and U.

A declaration $f(P)$ is *applicable* to a call $f(C)$ if the call is in the scope of the declaration and $C \preceq P$. (See Chapter 7 for the definition of scope.) If the declaration has static parameters, it is applicable to the call if some instance of it whose static arguments satisfy their bounds is, that is, if $C \preceq P'$ for the parameter type P' of such an instance. Its static arguments are not inferred to decide this: they are inferred only for the declaration that the call selects, after it is selected (Section 20.1).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 said that static parameters in the parameter type are inferred as described in Chapter 20 before the applicability of the declaration to the call is checked. The type group's paper of 2011 defines a declaration with static parameters to be applicable when at least one of its instances is.

Note that a named functional call $f(C)$ may invoke a dotted method declaration if the declaration is provided by the trait or object enclosing the call. To account for this, let C_0 be the trait or object declaration immediately enclosing the call. Then we consider a named functional call $f(C)$ as $C_0.f(C)$ if C_0 provides dotted method declarations applicable to $f(C)$, and use the rule for applicability to dotted method calls (described in Section 15.3) to determine which declarations are applicable to $C_0.f(C)$.

15.3 Applicability to Dotted Method Calls

Dotted method applications can be characterized similarly to named functional applications, except that, analogously to dotted method declarations, we use C_0 to denote the dynamic type of the receiver object, and, as for named functional calls, C to denote the dynamic type of the argument of a dotted method call. We write $C_0.f(C)$ to refer to the call.

A dotted method declaration $P_0.f(P)$ is *applicable* to a dotted method call $C_0.f(C)$ if $C_0 \preceq P_0$ and $C \preceq P$. If the declaration has static parameters, it is applicable to the call if some instance of it whose static arguments satisfy their bounds is, that is, if $C_0 \preceq P_0'$ and $C \preceq P'$ for the types P_0' and P' of such an instance; its static arguments are inferred only for the declaration that the call selects (Section 20.1).

REVISED BY THE 2026 REVIVAL SEE Section I.1.29

The Working Draft of February 2011 said, as for named functional calls, that static parameters in P_0 and P are inferred before the applicability of the declaration is checked.

15.4 Applicability for Functionals with Varargs and Keyword Parameters

The basic idea for handling varargs and keyword parameters is that we can think of a functional declaration that has such parameters as though it were (possibly infinitely) many declarations, one for each set of arguments it may be called with. In other words, we expand these declarations so that there exists a declaration for each number of arguments that can be passed to it.

A declaration with a varargs parameter corresponds to an infinite number of declarations, one for every number of arguments that may be passed to the varargs parameter. In practice, we can bound that number by the maximum number of arguments that the functional is called with anywhere in the program (in other words, a given program will contain only a finite number of calls with different numbers of arguments). The expansion described here is a conceptual one to simplify the description of the semantics; we do not expect a real implementation to actually expand these declarations at compile time. For example, the following declaration:

$$f(x : \mathbb{Z}, y : \mathbb{Z}, z : \mathbb{Z} \ldots) : \mathbb{Z}$$

would be expanded into:

$$f(x : \mathbb{Z}, y : \mathbb{Z}) : \mathbb{Z}$$
$$f(x : \mathbb{Z}, y : \mathbb{Z}, z_1 : \mathbb{Z}) : \mathbb{Z}$$
$$f(x : \mathbb{Z}, y : \mathbb{Z}, z_1 : \mathbb{Z}, z_2 : \mathbb{Z}) : \mathbb{Z}$$
$$f(x : \mathbb{Z}, y : \mathbb{Z}, z_1 : \mathbb{Z}, z_2 : \mathbb{Z}, z_3 : \mathbb{Z}) : \mathbb{Z}$$
$$\ldots$$

A declaration with a varargs parameter is applicable to a call if any one of the expanded declarations is applicable.

15.5 Overloading Resolution

DRAFT NOTE

Victor: Does this paragraph, other than the last sentence, really belong in this section?

To evaluate a given functional call, it is necessary to determine which functional declaration to dispatch to. To do so, we consider the declarations that are applicable to that call at run time. If there is exactly one such declaration, then the call dispatches to that declaration. If there is no such declaration, then the call is *undefined*, which is a static error. (However, see Section 17.4 for how coercion may add to the set of applicable declarations.) If multiple declarations are applicable to the call at run time, then we choose an arbitrary declaration among the declarations such that no other applicable declaration is more specific than them. A declaration with static parameters is applicable to a call at run time, as to a static call, when some instance of it whose static arguments satisfy their bounds is applicable to the call, so the declaration to which a call dispatches may be one that the static call did not select. A declaration that the static call selected runs at the instance inferred for the call (Section 20.1); any other declaration runs at the instance in which each type parameter is the intersection of the upper bounds placed on it: its declared bound, Any if it has none (Section 12.1), and the bounds that the dynamic types of the arguments place on it through the parameter types and that the static type of the call places on it through the return type. Where the type parameter is a static argument of a parameterized trait in a parameter type, as X is in $t\colon \mathrm{Tag}[\![X]\!]$, the value of the argument fixes it, since no value belongs to two instantiations of one parameterized trait (Section 6.3); where it is the whole type of a parameter, the dynamic type of the argument is only a lower bound, and the type parameter is its bound under the static type of the

call. The type parameters of a declaration that only dispatch reaches are never instantiated with $\mathrm{BottomType}$, nor with the union of the types of the arguments.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.31
>
> The Working Draft of February 2011 did not say at which instance a declaration with static parameters runs when dispatch reaches it and the static call selected another. The rule stated is the one by which the dispatcher of the type group's paper of 2019 instantiates a declaration (Park, Hong, Steele and Ryu, "Polymorphic Symmetric Multiple Dispatch with Variance", POPL 2019); the team had assumed the union of the lower bounds and could not prove it correct. Both implementations of the revival take the dynamic type of the argument where the value does not fix the type parameter: the interpreter because it has no static types, and so no static type of the call to bound the instance, as with its choice of coercions (Section 17.5); the compiled path because its dispatcher reads the static arguments of a dispatched declaration from the value and is not given the static type of the call. Where the value fixes the type parameter, the two rules give the same instance. The compiled path's departure is gated as an expected failure (row 538 of the revival's gap ledger), and the compiled path does not yet run the second example below (row 496).

We use the subtype relation to compare parameter types to determine a more specific declaration. Formally, a declaration $f(P)$ is *more specific* than a declaration $f(Q)$ if $P \prec Q$. Similarly, a declaration $P_0.f(P)$ is more specific than a declaration $Q_0.f(Q)$ if $P_0 \prec Q_0$ and $P \prec Q$. (See Section 17.5 for how coercion changes the definition of "more specific".) Restrictions on the definition of overloaded functionals (see Chapter 24) guarantee that among all applicable declarations, one is more specific than all the others. Where declarations have static parameters, they are compared on the parameter types they declare, each taken over all its instances: a declaration is more specific than another when the second is applicable to every call to which the first is applicable, and not the other way round. Where neither has static parameters, this is the subtype relation above. No static arguments are inferred for the comparison.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 said that static parameters are inferred as described in Chapter 20 before the parameter types are compared. The definition stated is that of the type group's paper of 2011, which checks it as subtyping of the parameter types read as existential types.

For example, where Shape comprises the two object types Circle and Square, of the declarations

$$\mathit{describe}[\![T \texttt{ extends } \mathrm{Shape}]\!](x\colon T)\colon \mathrm{String} = \text{“}\texttt{generic}\text{”}$$
$$\mathit{describe}(x\colon \mathrm{Circle})\colon \mathrm{String} = \text{“}\texttt{circle}\text{”}$$

the second is more specific than the first, although the first has an instance whose parameter type is Circle. The calls $\mathit{describe}(\mathrm{Circle})$ and $\mathit{describe}(s)$, where s has static type Shape and holds Circle, run the second declaration, and $\mathit{describe}(\mathrm{Square})$ runs the first. And of the declarations

$$\mathit{grab}[\![X]\!](t\colon \mathrm{Tag}[\![X]\!])\colon \mathrm{Marker}[\![X]\!] = \mathrm{Marker}[\![X]\!]$$
$$\mathit{grab}(a\colon \mathrm{Any})\colon \mathrm{Any} = \mathrm{Red}$$

where Marker is an object with a type parameter and Red an object type, the call $\mathit{grab}(a)$, where a has static type Any, selects the second declaration statically; if a holds a value of an object type that extends $\mathrm{Tag}[\![\mathrm{Red}]\!]$, the call dispatches to the first, which is applicable to that value and more specific, at $X = \mathrm{Red}$, which the value fixes, and returns $\mathrm{Marker}[\![\mathrm{Red}]\!]$.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.31
>
> The Working Draft of February 2011 forbade both pairs of declarations, which differ in their static parameters.

Chapter 16

Operators

> DRAFT NOTE
> Multifix operators and dimensions and units are not yet supported.

> DRAFT NOTE
> Synonym Operators: Victor's email titled "Synonyms [Fwd: Re: What the heck are 0-width spaces for?]" on 09/12/07

Operators are like functions or methods; operator declarations are described in Chapter 25 and operator applications are described in Section 13.7, Section 13.17, and Section 13.30. Operators are not values; when an operator is passed as an argument to a function, it should be eta expanded. Just as functions or methods may be overloaded (see Chapter 15 for a discussion of overloading), so operators may have overloaded declarations of the same fixity. Operator declarations with the same operator name but with different fixities are valid declarations because it is always unambiguous which declaration shall be applied to an application of the operator. Calls to overloaded operators are resolved first via the fixity of the operators based on the context of the calls. Then, among the applicable declarations with that fixity, the most specific declaration is chosen.

Most operators can be used as prefix, infix, postfix, or nofix operators as described in Section 16.3 (nofix operators take no arguments); the fixity of an operator is determined syntactically, and the same operator may have declarations for multiple fixities. A simple example is that '$-$' may be either infix or prefix, as is conventional. As another example, '!' may be a postfix operator that computes factorial when applied to integers. These operators might not be used as enclosing operators.

Several pairs of operators can be used as enclosing operators. Any number of '$|$' (vertical line) can be used as both infix operators and enclosing operators.

Some operators are always postfix: a '$\hat{}$' followed by any ordinary operator (with no intervening whitespace) is considered to be a superscripted postfix operator. For example, '$\hat{}*$' and '$\hat{}+$' and '$\hat{}?$' are available for use as part of the syntax of extended regular expressions. As a very special case, '$\hat{}T$' is also considered to be a superscripted postfix operator, typically used to signify matrix transposition.

Finally, there are special operators such as juxtaposition and operators on dimensions and units. Juxtaposition may be a function application or an infix operator in Fortress. When the left-hand-side expression is a function, juxtaposition performs function application; when the left-hand-side expression is a number, juxtaposition conventionally performs multiplication; when the left-hand-side expression is a string, juxtaposition conventionally performs string concatenation. Fortress provides several operators on dimensions and units as described in Chapter 18.

16.1 Operator Names

> DRAFT NOTE
> Victor: This should refer to the appropriate sections of Lexical Structure Operator names are either operator tokens or special operator characters, or a few reserved words.

To support a rich mathematical notation, Fortress allows most Unicode characters that are specified to be mathematical operators to be used as operators in Fortress expressions, as well as these characters:

```
!    @    #    $    %    *    +    -    =    |    :    <    >    /    ?    ^    ~
->    -->    =>    ==>    <=    >=    =/=    !!    ||    |||
<<    <<<    >>    >>>    <->    <-/-    -/->    <=>    ===
```

In addition, a token that is made up of a mixture of uppercase letters and underscores (but no digits), does not begin or end with an underscore, and contains at least two different letters is also considered to be an operator:

```
MAX    MIN
```

The above operators are rendered as: `MAX MIN`. (See Section 4.14 and Appendix F for detailed descriptions of operator names in Fortress.)

16.2 Operator Precedence and Associativity

Fortress specifies that certain operators have higher precedence than certain other operators and certain operators are associative, so that one need not use parentheses in all cases where operators are mixed in an expression. (See Appendix F for a detailed description of operator precedence and associativity in Fortress.) However, Fortress does not follow the practice of other programming languages in simply assigning an integer to each operator and then saying that the precedence of any two operators can be compared by comparing their assigned integers. Instead, Fortress relies on defining traditional groups of operators based on their meaning and shape, and specifies specific precedence relationships between some of these groups. If there is no specific precedence relationship between two operators, then parentheses must be used. For example, Fortress does not accept the expression $a + b \cup c$; one must write either $(a + b) \cup c$ or $a + (b \cup c)$. (Whether or not the result then makes any sense depends on what definitions have been made for the $+$ and $\cup$ operators—see Chapter 25.)

Here are the basic principles of operator precedence and associativity in Fortress:

- Member selection (`.`) and method invocation ($.name(\ldots)$) are not operators. They have higher precedence than any operator listed below.
- Subscripting (`[ ]` and any kind of subscripting operators, which can be any kind of enclosing operators), superscripting (`^`), and postfix operators have higher precedence than any operator listed below; within this group, these operations are left-associative (performed left-to-right).
- *Tight juxtaposition*, that is, juxtaposition without intervening whitespace, has higher precedence than any operator listed below. The associativity of tight juxtaposition is type-dependent; see Section 16.8.
- Next, *tight fractions*, that is, the use of the operator '`/`' with no whitespace on either side, have higher precedence than any operator listed below. The tight-fraction operator has no precedence compared with itself, so it is not permitted to be used more than once in a tight fraction without use of parentheses.
- *Loose juxtaposition*, that is, juxtaposition with intervening whitespace, has higher precedence than any operator listed below. The associativity of loose juxtaposition is type-dependent and is different from that for tight juxtaposition; see Section 16.8. Note that *lopsided juxtaposition* (having whitespace on one side but not the other) is a static error as described in Section 16.3.

- Prefix operators have higher precedence than any operator listed below. However, it is a static error for an operand of a loose prefix operator to be an operand of a tight infix operator.
- The infix operators are partitioned into certain traditional groups, as explained below. They have higher precedence than any operator listed below.
- The equal symbol '$=$' in binding context, the assignment operator '$:=$', and compound assignment operators ($+=$, $-=$, $\wedge=$, $\vee=$, $\cap=$, $\cup=$, and so on as described in Section 13.7) have lower precedence than any operator listed above. Note that compound assignment operators themselves are not operator names.

The infix binary operators are divided into four general categories: arithmetic, relational, boolean, and other. The arithmetic operators are further categorized as multiplication/division/intersection, addition/subtraction/union, and other. The relational operators are further categorized as equivalence, inequivalence, ordering, and other. The boolean operators are further categorized as conjunctive, disjunctive, and other.

The arithmetic and relational operators are further divided into groups based on shape:

- "ordinary" operators: $+ - \cdot \times / \pm \mp \oplus \ominus \odot \otimes \oslash \boxplus \boxminus \boxdot \boxtimes$ $< \leq \geq > \ll \lll \ggg \gg \not< \not\leq \not\geq \not>$ etc.

  The arithmetic operations in this group are further subdivided into "plain" ($+ - \cdot \times / \pm \mp$ etc.), "circled" ($\oplus \ominus \odot \otimes \oslash$ etc.), "boxed" ($\boxplus \boxminus \boxdot \boxtimes$ etc.), and so on; any of these groups may be used with the plain relational operators ($< \leq \geq > \ll \lll \ggg \gg \not< \not\leq \not\geq \not>$ etc.), but the groups might not be mixed.
- "rounded horseshoe" or "set" operators: $\cap \Cap \cup \Cup \uplus \subset \subseteq \supseteq \supset \Subset \Supset \not\subset \not\subseteq \not\supseteq \not\supset$ etc.
- "square horseshoe" operators: $\sqcap \sqcup \sqsubset \sqsubseteq \sqsupseteq \sqsupset \not\sqsubseteq \not\sqsupseteq$ etc.
- "curly" operators: $\curlywedge \curlyvee \prec \preceq \succeq \succ \not\prec \not\preceq \not\succeq \not\succ$ etc.
- "triangular" relations: $\lhd \trianglelefteq \trianglerighteq \rhd \not\lhd \not\trianglelefteq \not\trianglerighteq \not\rhd$ etc.
- "chickenfoot" relations: $<\!\!->$ etc.

The principles of precedence for binary operators are then as follows:

- A multiplication or division or intersection operator has higher precedence than any addition or subtraction or union operator that is in the same shape group.
- Certain addition and subtraction operators come in pairs, such as $+$ and $-$, or $\oplus$ and $\ominus$, which are considered to have the same precedence and so may be mixed within an expression and are grouped left-associatively. These addition-subtraction pairs are the *only* cases where two different operators are considered to have the same precedence.
- An arithmetic operator has higher precedence than any equivalence or inequivalence operator.
- An arithmetic operator has higher precedence than any relational operator that is in the same shape group.
- A relational operator has higher precedence than any boolean operator.
- A conjunctive boolean operator has higher precedence than any disjunctive boolean operator.

While the rules of precedence are complicated, they are intended to be both unsurprising and conservative. Note that operator precedence in Fortress is not always transitive; for example, while $+$ has higher precedence than $<$ (so you can write $a + b < c$ without parentheses), and $<$ has higher precedence than $\vee$ (so you can write $a < b \vee c < d$ without parentheses), it is *not* true that $+$ has higher precedence than $\vee$—the expression $a \vee b + c$ is not permitted, and one must instead write $(a \vee b) + c$ or $a \vee (b + c)$.

Another point is that the various multiplication and division operators do *not* have "the same precedence"; they may not be mixed freely with each other. For example, one cannot write $u \cdot v \times w$; one must write $(u \cdot v) \times w$ or (more likely) $u \cdot (v \times w)$. Similarly, one cannot write $a \odot b / c \odot d$; but juxtaposition does bind more tightly

than a loose (whitespace-surrounded) division slash, so one is allowed to write $a\,b\,/\,c\,d$, and this means the same as $(a\,b)/(c\,d)$. On the other hand, loose juxtaposition binds less tightly than a tight division slash, so that $a\,b/c\,d$ means the same as $a\,(b/c)\,d$. On the other other hand, tight juxtaposition binds more tightly than tight division, so that $(n+1)/(n+2)(n+3)$ means the same as $(n+1)/((n+2)(n+3))$.

There are two additional rules intended to catch misleading code: it is a static error for an operand of a tight infix or tight prefix operator to be a loose juxtaposition, and it is a static error if the rules of precedence determine that a use of infix operator a has higher or equal precedence than a use of infix operator b, but that particular use of a is loose and that particular use of b is tight. Thus, for example, the expression $\sin\ x\ +\ y$ is permitted, but $\sin\ x+y$ is not permitted. Similarly, the expression $a\ \cdot\ b\ +\ c$ is permitted, as are $a\cdot b\ +\ c$ and $a\cdot b+c$, but $a\ \cdot\ b+c$ is not permitted. (The rule detects only the presence or absence of whitespace, not the amount of whitespace, so $a\ \ \cdot\ b\ +\ c$ is permitted. You have to draw the line somewhere.)

When in doubt, just use parentheses. If there's a problem, the compiler will (probably) let you know.

16.3 Operator Fixity

> DRAFT NOTE
> Multifix operators are not yet supported.

Most operators in Fortress can be used variously as prefix, postfix, infix, nofix, or multifix operators. (See Section 16.4 for a discussion of how infix operators may be chained or treated as multifix operators.)

> DRAFT NOTE
> Victor: I think that we shouldn't list "multifix" separately here. (Note that multifix does not appear in any of the tables.) Rather multifix is a way to interpret infix operators.

Some operators can be used in pairs as enclosing (bracketing) operators—see Section 16.5. The Fortress language dictates only the rules of syntax; whether an operator has a meaning when used in a particular way depends only on whether there is a definition in the program for that operator when used in that particular way (see Chapter 25).

The fixity of a non-enclosing operator is determined by context. To the left of such an operator we may find (1) a *primary tail* (described below), (2) another operator, or (3) a comma, semicolon, or left encloser. To the right we may find (1) a *primary front* (described below), (2) another operator, (3) a comma, semicolon, or right encloser, or (4) a line break. A primary tail is an identifier, a literal, a right encloser, or a superscripted postfix operator (exponent operator). A primary front is an identifier, a literal, or a left encloser.

> DRAFT NOTE
> A primary expression is an identifier, a literal, an expression enclosed by matching enclosers, a field selection, or an expression followed by a postfix operator.

Considered in all combinations, this makes twelve possibilities. In some cases one must also consider whether or not whitespace separates the operator from what lies on either side. The rules of operator fixity are specified by Figure 16.1, where the center column indicates the fixity that results from the left and right context specified by the other columns.

A case described in the center column of the table as an **error** is a static error; for such cases, the fixity mentioned in parentheses is the recommended treatment of the operator for the purpose of attempting to continuing the parse in search of other errors.

The table may seem complicated, but it all boils down to a couple of practical rules of thumb:

left context	whitespace	operator fixity	whitespace	right context
primary tail	yes yes no no	**infix** **error** (infix) **postfix** **infix**	yes no yes no	primary front
primary tail	yes yes no no	**infix** **error** (infix) **postfix** **infix**	yes no yes no	operator
primary tail	yes no	**error** (postfix) **postfix**		, ; right encloser
primary tail	yes no	**infix** **postfix**		line break
operator		**prefix**		primary front
operator		**prefix**		operator
operator		**error** (nofix)		, ; right encloser
operator		**error** (nofix)		line break
, ; left encloser		**prefix**		primary front
, ; left encloser		**prefix**		operator
, ; left encloser		**nofix**		, ; right encloser
, ; left encloser		**error** (prefix)		line break

Figure 16.1: Operator Fixity (I)

1. *Any* operator can be prefix, postfix, infix, or nofix.
2. An infix operator can be *loose* (having whitespace on both sides) or *tight* (having whitespace on neither side), but it mustn't be *lopsided* (having whitespace on one side but not the other).
3. A postfix operator must have no whitespace before it and must be followed (possibly after some whitespace) by a comma, semicolon, right encloser, or line break.

16.4 Chained and Multifix Operators

Certain infix mathematical operators that are traditionally regarded as *relational* operators, delivering boolean results, may be *chained*. For example, an expression such as $A \subseteq B \subset C \subseteq D$ is treated as being equivalent to $(A \subseteq B) \wedge (B \subset C) \wedge (C \subseteq D)$ except that the expressions B and C are evaluated only once (which matters only if they have side effects such as writes or input/output actions). Similarly, the expression $A \subseteq B = C \subset D$ is treated as being equivalent to $(A \subseteq B) \wedge (B = C) \wedge (C \subset D)$, except that B and C are evaluated only once. Fortress restricts such chaining to a mixture of equivalence operators and ordering operators; if a chain contains two or more ordering operators, then they must be of the same kind and have the same sense of monotonicity; for example, neither $A \subseteq B \leq C$ nor $A \subseteq B \supset C$ is permitted. This transformation is done before type checking. In particular, it is done even if these operators do not return boolean values, and the resulting expression is checked for type correctness. (See Section F.5 for a detailed description of which operators may be chained.)

Any infix operator that does not chain may be treated as *multifix*. If $n - 1$ occurrences of the same operator separate n operands where $n \geq 3$, then the compiler first checks to see whether there is a definition for that operator that will accept n arguments. If so, that definition is used; if not, then the operator is treated as left-associative and the compiler looks for a two-argument definition for the operator to use for each occurrence. As an example, the cartesian product $S_1 \times S_2 \times \cdots \times S_n$ of n sets may usefully be defined as a multifix operator, but ordinary addition $p + q + r + s$ is normally treated as $((p + q) + r) + s$.

left context	whitespace	operator fixity	whitespace	right context
primary tail	yes yes no no	**infix** **left encloser** **right encloser** **infix**	yes no yes no	primary front
primary tail	yes yes no no	**infix** **left encloser** **right encloser** **infix**	yes no yes no	operator
primary tail	yes no	**error** (right encloser) **right encloser**		`,` `;` right encloser
primary tail	yes no	**infix** **right encloser**		line break
operator		**error** (left encloser) **left encloser**	yes no	primary front
operator		**error** (left encloser) **left encloser**	yes no	operator
operator		**error** (nofix)		`,` `;` right encloser
operator		**error** (nofix)		line break
`,` `;` left encloser		**left encloser**		primary front
`,` `;` left encloser		**left encloser**		operator
`,` `;` left encloser		**nofix**		`,` `;` right encloser
`,` `;` left encloser		**error** (left encloser)		line break

Figure 16.2: Operator Fixity (II)

16.5 Enclosing Operators

These operators are always used in pairs as enclosing operators:

```
                 (/   /)        (\   \)
[   ]            [/   /]                       [*   *]
{   }            {/   /}        {\   \}        {*   *}
                 </   />        <\   \>
                <<</  />>      <<\   \>>
```

(ASCII encodings are shown here; they all correspond to particular single Unicode characters.) There are other pairs as well, such as ⌊ ⌋ and ⌈ ⌉ and multicharacter enclosing operators described in Section 4.14.1. Note that the pairs `(` `)` and `[\` `\]` (also known as ⟦ ⟧) are not operators; they play special roles in the syntax of Fortress, and their behavior cannot be redefined by a library. The bracket pairs that may be used as enclosing operators are described in Section F.2.

Any number of '`|`' (vertical line) may also be used in pairs as enclosing operators but there is a trick to it, because on the face of it you can't tell whether any given occurrence is a left encloser or a right encloser. Again, context is used to decide, this time according to Figure 16.2.

This is very similar to Figure 16.1 in Section 16.3; a rough rule of thumb is that if an ordinary operator would be considered a prefix operator, then one of these will be considered a left encloser; and if an ordinary operator would be considered a postfix operator, then one of these will be considered a right encloser.

In this manner, one may use `|...|` for absolute values and `||...||` for matrix norms.

16.6 Conditional Operators

If a binary operator other than ‘`:`’ and ‘`::`’ is immediately followed by a ‘`:`’ then it is *conditional*: evaluation of the right-hand operand cannot begin until evaluation of the left-hand operand has completed, and whether or not the right-hand operand is evaluated may depend on the value of the left-hand operand. If the left-hand operand throws an exception, then the right-hand operand is not evaluated.

The Fortress standard libraries define several conditional operators on boolean values including $\wedge:$ and $\vee:$.

See Section 25.9 for a discussion of how conditional operators are declared.

16.7 Big Operators

A big operator application is either a *reduction expression* described in Section 13.17 or a *comprehension* described in Section 13.30.

The Fortress standard libraries define several big operators including $\sum$, $\prod$, and `BIG` $\wedge$.

See Section 25.10 for a discussion of how big operators are declared.

16.8 Juxtaposition

> DRAFT NOTE
> Qualified names are not yet supported.

Juxtaposition in Fortress may be a function call or a special infix operator. The Fortress standard libraries include several declarations of a `juxtaposition` operator.

When two expressions are juxtaposed, the juxtaposition is interpreted as follows: if the left-hand-side expression is a function, juxtaposition performs function application; otherwise, juxtaposition performs the `juxtaposition` operator application.

The manner in which a juxtaposition of three or more items must be associated requires type information and awareness of whitespace. (This is an inherent property of customary mathematical notation, which Fortress is designed to emulate where feasible.) Therefore a Fortress compiler must produce a provisional parse in which such multi-element juxtapositions are held in abeyance, then perform a type analysis on each element and use that information to rewrite the n-ary juxtaposition into a tree of binary juxtapositions.

All we need to know is whether each element of a juxtaposition has an arrow type. There are actually three legitimate possibilities for each element of a juxtaposition: (a) it has an arrow type, in which case it is considered to be a function element; (b) it has a type that is not an arrow type, in which case it is considered to be a non-function element. (c) it is an identifier that has no visible declaration, in which case it is considered to be a function element (and everything will work out okay if it turns out to be the name of an appropriate functional method).

The rules below are designed to forbid certain forms of notational ambiguity that can arise if the name of a functional method happens to be used also as the name of a variable. For example, suppose that trait T has a functional method of one parameter named n; then in the code

```
do
  a:T = t
  n:ℤ = 14
```

```
    z = n a
end
```

it might not be clear whether the intended meaning was to invoke the functional method n on a or to multiply a by 14. The rules specify that such a situation is a static error.

The rules for reassociating a loose juxtaposition are as follows:

- First the loose juxtaposition is broken into nonempty chunks; wherever there is a non-function element followed by a function element, the latter begins a new chunk. Thus a chunk consists of some number (possibly zero) of functions followed by some number (possibly zero) of non-functions.
- It is a static error if any non-function element in a chunk is an unparenthesized identifier f and is followed by another non-function element whose type is such that f can be applied to that latter element as a functional method.
- The non-functions in each chunk, if any, are replaced by a single element consisting of the non-functions grouped left-associatively into binary juxtapositions.
- What remains in each chunk is then grouped right-associatively.

  (Notice that, up to this point, no analysis of the types of newly constructed chunks is needed during this process.)
- It is a static error if an element of the original juxtaposition was the last element in its chunk before reassociation, the chunk was not the last chunk (and therefore the element in question is a non-function element), the element was an unparenthesized identifier f, and the type of the following chunk after reassociation is such that f can be applied to that following chunk as a functional method.
- If any element that remains has type String, then it is a static error if there is any pair of adjacent elements within the juxtaposition such that neither element is of type String.
- What remains is considered to be a binary or multifix application of the juxtaposition operator; if more than two elements remain, the application is handled in the same way as for any other multifix operator (see Section 16.4).

Here is an example: $n\ (n+1)\ \sin\ 3\ n\ x\ \log\ \log\ x$. Assuming that $\sin$ and $\log$ name functions in the usual manner and that n, $(n+1)$, and x are not functions, this loose juxtaposition splits into three chunks: $n\ (n+1)$, $\sin\ 3\ n\ x$, and $\log\ \log\ x$. The first chunk has only two elements and needs no further reassociation. In the second chunk, the non-functions $3\ n\ x$ are replaced by $((3\ n)\ x)$. In the third chunk, there is only one non-function, so that remains unchanged; the chunk is the right-associated to form $(\log\ (\log\ x))$. Assuming that no multifix definition of juxtaposition has been provided, the three chunks are left-associated, to produce the final interpretation $((n\ (n+1))\ (\sin\ ((3\ n)\ x)))\ (\log\ (\log\ x))$. Now the original juxtaposition has been reduced to binary juxtaposition expressions.

A tight juxtaposition is always left-associated if it contains any dot (i.e., ".") not within parentheses or some pair of enclosing operators. If such a tight juxtaposition begins with an identifier immediately followed by a dot then the maximal prefix of identifiers separated by dots (whitespace may follow but not precede the dots) is collected into a "dotted id chain", which is subsequently partitioned into the longest prefix, if any, that is a qualified name, and the remaining the dots and identifiers, which are interpreted as selectors. If the last identifier in the dotted id chain is part of a selector (i.e., the entire dotted id chain is not a qualified name), and it is immediately followed by a left parenthesis, then the last selector together with the subsequent parenthesis-delimited expression is a method invocation.

A tight juxtaposition without any dots might not be entirely left-associated. Rather, it is considered as a nonempty sequence of elements: the front expression, any "math items", and any postfix operator, and subject to reassociation as described below. A math item may be a subscripting, an exponentiation, or one of a few kinds of expressions. It is a static error if an exponentiation is immediately followed by a subcripting or an exponentiation.

The procedure for reassociation is as follows:

- For each expression element (i.e., not a subscripting, exponentiation or postfix operator), determine whether it is a function.
- If some function element is immediately followed by an expression element then, find the first such function element, and call the next element the argument. It is a static error if either the argument is not parenthesized, or the argument is immediately followed by a non-expression element. Otherwise, replace the function and argument with a single element that is the application of the function to the argument. This new element is an expression. Reassociate the resulting sequence (which is one element shorter).
- If there is any non-expression element (it cannot be the first element) then replace the first such element and the element immediately preceding it (which must be an expression) with a single element that does the appropriate operator application. This new element is an expression. Reassociate the resulting sequence (which is one element shorter).
- Otherwise, the sequence necessarily has only expression elements, only the last of which may be a function. If any element that remains has type String, then it is a static error if there is any pair of adjacent elements within the juxtaposition such that neither element is of type String.
- What remains is considered to be a binary or multifix application of the juxtaposition operator; if more than two elements remain, the application is handled in the same way as for any other multifix operator (see Section 16.4).

(Note that this process requires type analysis of newly created chunks all along the way.)

Here is an (admittedly contrived) example: $reduce(f)(a)(x+1)atan(x+2)$. Suppose that $reduce$ is a curried function that accepts a function f and returns a function that can be applied to an array a (the idea is to use the function f, which ought to take two arguments, to combine the elements of the array to produce an accumulated result).

The leftmost function is $reduce$, and the following element (f) is parenthesized, so the two elements are replaced with one: $(reduce(f))(a)(x+1)atan(x+2)$. Now type analysis determines that the element $(reduce(f))$ is a function.

The leftmost function is $(reduce(f))$, and the following element (a) is parenthesized, so the two elements are replaced with one: $((reduce(f))(a))(x+1)atan(x+2)$. Now type analysis determines that the element $((reduce(f))(a))$ is not a function.

The leftmost function is $atan$, and the following element $(x+2)$ is parenthesized, so the two elements are replaced with one: $((reduce(f))(a))(x+1)(atan(x+2))$. Now type analysis determines that the element $(atan(x+2))$ is not a function.

There are no functions remaining in the juxtaposition. Assuming that no multifix definition of juxtaposition has been provided, the remaining elements are left-associated:

$$(((reduce(f))(a))(x+1))(atan(x+2))$$

Now the original juxtaposition has been reduced to binary juxtaposition expressions.

16.9 Overview of Operators in the Fortress Standard Libraries

> DRAFT NOTE
> The operators in this section are not tested nor run by the interpreter yet.

This section provides a high-level overview of the operators in the Fortress standard libraries. See Appendix F for the detailed rules for the operators provided by the Fortress standard libraries.

16.9.1 Prefix Operators

For all standard numeric types, the prefix operator $+$ simply returns its argument and the prefix operator $-$ returns the negative of its argument.

The operator $\neg$ is the logical NOT operator on boolean values and boolean intervals.

The operator $\dashv$ computes the bitwise NOT of an integer.

Big operators such as $\sum$ begin a *reduction expression* (Section 13.17). The big operators include $\sum$ (summation) and $\prod$ (product), along with $\bigcap$, $\bigcup$, $\bigwedge$, $\bigvee$, $\underline{\bigvee}$, $\bigoplus$, $\bigotimes$, $\biguplus$, $\boxplus$, $\boxtimes$, `MAX`, `MIN`, and so on.

16.9.2 Postfix Operators

The operator $!$ computes factorial; the operator $!!$ computes double factorials. They may be applied to a value of any integral type and produces a result of the same type.

When applied to a floating-point value x, $x!$ computes $\Gamma(1 + x)$, where Γ is the Euler gamma function.

16.9.3 Enclosing Operators

When used as left and right enclosing operators, $|\ \ |$ computes the absolute value or magnitude of any number, and is also used to compute the number of elements in an aggregate, for example the cardinality of a set or the length of a list. Similarly, $\|\ \ \|$ is used to compute the norm of a vector or matrix.

The floor operator $\lfloor\ \ \rfloor$ and ceiling operator $\lceil\ \ \rceil$ may be applied to any standard integer, rational, or real value (their behavior is trivial when applied to integers, of course). The operators hyperfloor $\lfloor\!\lfloor x \rfloor\!\rfloor = 2^{\lfloor \log_2 x \rfloor}$, hyperceiling $\lceil\!\lceil x \rceil\!\rceil = 2^{\lceil \log_2 x \rceil}$, hyperhyperfloor $\lfloor\!\lfloor\!\lfloor x \rfloor\!\rfloor\!\rfloor = 2^{\lfloor\!\lfloor \log_2 x \rfloor\!\rfloor}$, and hyperhyperceiling $\lceil\!\lceil\!\lceil x \rceil\!\rceil\!\rceil = 2^{\lceil\!\lceil \log_2 x \rceil\!\rceil}$ are also available.

16.9.4 Exponentiation

Given two expressions e and e' denoting numeric quantities v and v' that are not vectors or matrices, the expression $e^{e'}$ denotes the quantity obtained by raising v to the power v'. This operation is defined in the usual way on numerals.

Given an expression e denoting a vector and an expression e' denoting a value of type $\mathbb{Z}$, the expression $e^{e'}$ denotes repeated vector multiplication of e by itself e' times.

Given an expression e denoting a square matrix and an expression e' denoting a value of type $\mathbb{Z}$, the expression $e^{e'}$ denotes repeated matrix multiplication of e by itself e' times.

> DRAFT NOTE
>
> Eric: Do we also want to support the matrix exponential via this operator? Doing so requires having a special constant value for e.

16.9.5 Superscript Operators

The superscript operator $\hat{}\,T$ transposes a matrix. It also converts a column vector to a row vector or a row vector to a column vector.

16.9.6 Subscript Operators

Subscripting of arrays and other aggregates is written using square brackets:

`a[i]`	*is displayed as*	a_i	ith element of one-dimensional array a
`m[i,j]`	*is displayed as*	m_{ij}	i,jth element of two-dimensional matrix m
`space[i,j,k]`	*is displayed as*	$space_{ijk}$	i,j,kth element of three-dimensional array $space$
`a[3] := 4`	*is displayed as*	$a_3 := 4$	assign 4 to the third element of mutable array a
`m["foo"]`	*is displayed as*	m“`foo`”	fetch the entry associated with string “`foo`” from map m

16.9.7 Multiplication, Division, Modulo, and Remainder Operators

For most integer, rational, floating-point, complex, and interval expressions, multiplication can be expressed using any of ‘$\cdot$’ or ‘$\times$’ or simply juxtaposition. However, the $\times$ operator is used to express the shape of matrices, so if expressions using multiplication are used in expressing the shape of a matrix, it may be necessary to avoid the use of ’$\times$’ to express multiplication, or to use parentheses.

For integer, rational, floating-point, complex, and interval expressions, division is expressed by $/$. When the operator $/$ is used to divide one integer by another, the result is rational. The operator $\div$ performs truncating integer division: $m \div n = signum\left(\frac{m}{n}\right)\left\lfloor\left|\frac{m}{n}\right|\right\rfloor$. The operator `REM` gives the remainder from such a truncating division: $m \,\texttt{REM}\, n = m - n(m \div n)$. The operator `MOD` gives the remainder from a floor division: $m \,\texttt{MOD}\, n = m - n\left\lfloor\frac{m}{n}\right\rfloor$; when $n > 0$ this is the usual modulus computation that evaluates integer m to an integer k such that $0 \le k < n$ and n evenly divides $m - k$.

The special operators `DIVREM` and `DIVMOD` each return a pair of values, the quotient and the remainder; $m \,\texttt{DIVREM}\, n$ returns $(m \div n, m \,\texttt{REM}\, n)$ while $m \,\texttt{DIVMOD}\, n$ returns $(\left\lfloor\frac{m}{n}\right\rfloor, m \,\texttt{MOD}\, n)$.

Multiplication of a vector or matrix by a scalar is done with juxtaposition, as is multiplication of a vector by a matrix (on either side). Vector dot product is expressed by ‘$\cdot$’ and vector cross product by ‘$\times$’. Division of a matrix or vector by a scalar may be expressed using ‘$/$’.

The syntactic interaction of juxtaposition, $\cdot$, $\times$, and $/$ is subtle. See Section 16.2 for a discussion of the relative precedence of these operations and how precedence may depend on the use of whitespace.

The handling of overflow depends on the type of the number produced. For integer results, overflow throws an $\mathrm{IntegerOverflow}$. Rational computations do not overflow. For floating-point results, overflow produces $+\infty$ or $-\infty$ according to the rules of IEEE 754. For intervals, overflow produces an appropriate containing interval.

Underflow is relevant only to floating-point computations and is handled according to the rules of IEEE 754.

The handling of division by zero depends on the type of the number produced. For integer results, division by zero throws a $\mathrm{DivisionByZero}$. For rational results, division by zero produces $1/0$. For floating-point results, division by zero produces a NaN value according to the rules of IEEE 754. For intervals, division by zero produces an appropriate containing interval (which under many circumstances will be the interval of all possible real values and infinities).

Wraparound multiplication on fixed-size integers is expressed by $\dot\times$. Saturating multiplication on fixed-size integers is expressed by $\boxdot$ or $\boxtimes$. These operations do not overflow.

Ordinary multiplication and division of floating-point numbers always use the IEEE 754 “round to nearest” rounding mode. This rounding mode may be emphasized by using the operators $\otimes$ (or $\odot$) and $\oslash$. Multiplication and division in “round toward zero” mode may be expressed with $\boxtimes$ (or $\boxdot$) and $\boxslash$. Multiplication and division in “round toward positive infinity” mode may be expressed with $\overset{\scriptscriptstyle\triangle}{\boxtimes}$ (or $\overset{\scriptscriptstyle\triangle}{\boxdot}$) and $\overset{\scriptscriptstyle\triangle}{\boxslash}$. Multiplication and division in “round toward negative infinity” mode may be expressed with $\overset{\scriptscriptstyle\triangledown}{\boxtimes}$ (or $\overset{\scriptscriptstyle\triangledown}{\boxdot}$) and $\overset{\scriptscriptstyle\triangledown}{\boxslash}$.

16.9.8 Addition and Subtraction Operators

Addition and subtraction are expressed with $+$ and $-$ on all numeric quantities, including intervals, as well as vectors and matrices.

The handling of overflow depends on the type of the number produced. For integer results, overflow throws an $\mathrm{IntegerOverflow}$. Rational computations do not overflow. For floating-point results, overflow produces $+\infty$ or $-\infty$ according to the rules of IEEE 754. For intervals, overflow produces an appropriate containing interval.

Underflow is relevant only to floating-point computations and is handled according to the rules of IEEE 754.

Wraparound addition and subtraction on fixed-size integers are expressed by $\dotplus$ and $\dot{-}$. Saturating addition and subtraction on fixed-size integers are expressed by $\boxplus$ and $\boxminus$. These operations do not overflow.

Ordinary addition and subtraction of floating-point numbers always use the IEEE 754 "round to nearest" rounding mode. This rounding mode may be emphasized by using the operators $\oplus$ and $\ominus$. Addition and subtraction in "round toward zero" mode may be expressed with $\boxplus$ and $\boxminus$. Addition and subtraction in "round toward positive infinity" mode may be expressed with $\overset{\Delta}{+}$ and $\overset{\Delta}{-}$. Addition and subtraction in "round toward negative infinity" mode may be expressed with $\overset{\nabla}{+}$ and $\overset{\nabla}{-}$.

The construction $x \pm y$ produces the interval $\langle\!|x - y, x + y|\!\rangle$.

16.9.9 Intersection, Union, and Set Difference Operators

Sets support the operations of intersection $\cap$, union $\cup$, disjoint union $\uplus$, set difference $\setminus$, and symmetric set difference $\ominus$. Disjoint union throws $\mathrm{DisjointUnionError}$ if the arguments are in fact not disjoint.

Intervals support the operations of intersection $\cap$, union $\cup$, and interior hull $\Cup$. The operation $\Cap$ returns a pair of intervals; if the intersection of the arguments is a single contiguous span of real numbers, then the first result is an interval representing that span and the second result is an empty interval, but if the intersection is two spans, then two (disjoint) intervals are returned. The operation $\Cup$ returns a pair of intervals; if the arguments overlap, then the first result is the union of the two intervals and the second result is an empty interval, but if the arguments are disjoint, they are simply returned as is.

16.9.10 Minimum and Maximum Operators

The operator `MAX` returns the larger of its two operands, and `MIN` returns the smaller of its two operands.

For floating-point numbers, if either argument is a NaN then NaN is returned. The floating-point operations `MAXNUM` and `MINNUM` behave similarly except that if one argument is NaN and the other is a number, the number is returned. For all four of these operators, when applied to floating-point values, -0 is considered to be smaller than $+0$.

16.9.11 GCD, LCM, and CHOOSE Operators

The infix operator `GCD` computes the greatest common divisor of its integer operands, and `LCM` computes the least common multiple. The operator `CHOOSE` computes binomial coefficients: $n \texttt{ CHOOSE } k = \binom{n}{k} = \frac{n!}{k!(n-k)!}$.

The results of `GCD` and `LCM` are nonnegative (Section 33.2). On a fixed-size integer type, by the rule for integer results above, a result that the type cannot hold throws an $\mathrm{IntegerOverflow}$: on $\mathbb{Z}32$, for example, the greatest common divisor of -2^{31} and 0 is 2^{31}, and the least common multiple of $2^{31} - 1$ and 2 is $2^{32} - 2$, neither of which $\mathbb{Z}32$ holds.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.23
>
> The Working Draft of February 2011 said neither that these results are nonnegative nor what happens when one does not fit its type.

The expression $e \neq e'$ is semantically equivalent to the expression $\neg(e = e')$.

16.9.12 Comparisons Operators

Unless otherwise noted, the operators described in this section produce boolean ($true$/$false$) results.

The operators $<$, $\leq$, $\geq$, and $>$ are used for numerical comparisons and are supported by integer, rational, and floating-point types. Comparison of rational values throws $\mathrm{RationalComparisonError}$ if either argument is the rational infinity $1/0$ or the rational indefinite $0/0$. Comparison of floating-point values throws $\mathrm{FloatingComparisonError}$ if either argument is a NaN.

The operators $<$, $\leq$, $\geq$, and $>$ may also be used to compare characters (according to the numerical value of their Unicode codepoint values) and strings (lexicographic order based on the character ordering). They also use lexicographic order when used to compare lists whose elements support these same comparison operators.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.3
>
> The Fortress standard libraries order a list only where its element type is totally ordered (Section 12.6): a list of floating-point numbers or of rational numbers, whose elements support these comparison operators but are only partially ordered, has no order on the compiled type checker, while its $=$ still compares the two lists element by element. The interpreter still orders every list (row 774 of the revival's gap ledger).

When $<$, $\leq$, $\geq$, and $>$ are used to compare numerical intervals, the result is a boolean interval. The functions $possibly$ and $certainly$ are useful for converting boolean intervals to boolean values for testing. Thus $possibly(x > y)$ is true if and only if there is some value in the interval x that is greater than some value in the interval y, while $certainly(x > y)$ is true if and only if x and y are nonempty and every value in x is greater than every value in y.

The operators $\subset$, $\subseteq$, $\supseteq$, and $\supset$ may be used to compare sets or intervals regarded as sets.

The operator $\in$ may be used to test whether a value is a member of a set, list, array, interval, or range.

16.9.13 Logical Operators

The following binary operators may be used on boolean values:

$\wedge$	AND
$\vee$	inclusive OR
$\veebar$ or $\oplus$ or $\neq$	exclusive OR
$\equiv$ or $\leftrightarrow$ or $=$	equivalence (if and only if)
$\rightarrow$	IMPLIES
$\overline{\wedge}$	NAND
$\overline{\vee}$	NOR

These same operators may also be applied to boolean intervals to produce boolean interval results.

The following operators may be used on integers to perform "bitwise" operations:

$\between$	bitwise AND
$\Vbar$	bitwise inclusive OR
$\underline{\Vbar}$	bitwise exclusive OR

The prefix operator ⫬ computes the bitwise NOT of an integer. The operators `LSHIFT` and `RSHIFT` shift the bits of an integer to the left and to the right (Section 33.2).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.22
>
> The Working Draft of February 2011 did not name `LSHIFT` and `RSHIFT`.

16.9.14 Conditional Operators

If $p(x)$ and $q(x)$ are expressions that produce boolean results, the expression $p(x)\wedge\!:q(x)$ computes the logical AND of those two results by first evaluating $p(x)$. If the result of $p(x)$ is $true$, then $q(x)$ is also evaluated, and its result becomes the result of the entire expression; but if the result of $p(x)$ is $false$, then $q(x)$ is not evaluated, and the result of the entire expression is $false$ without further ado. (Similarly, evaluating the expression $p(x)\vee\!:q(x)$ does not evaluate $q(x)$ if the result of $p(x)$ is $true$.) Contrast this with the expression $p(x)\wedge q(x)$ (with no colon), which evaluates both $p(x)$ and $q(x)$, in no specified order and possibly in parallel.

Chapter 17

Conversions and Coercions

DRAFT NOTE

Widening and where clauses are not yet supported.

The examples in this chapter are not tested nor run by the interpreter.

DRAFT NOTE

- "has a type" vs "is a subtype of the type or could be coerced to the type"
- Coercion between tuple types (Victor's email titled "Coercion between tuple types" on 07/16/07) and arrow types
- Vectors and matrices: Section 13.29
- Do we allow coercion declarations in object expressions?
- If tests and coercion

  Suppose a test in an expression has a type that is not $\mathrm{Boolean}$ (or a subtype of $\mathrm{Boolean}$) but is coercible to $\mathrm{Boolean}$. Presumably, that is not a static error? But note that F1.0 beta makes no mention of this case; all tests in an if expression must be of type $\mathrm{Boolean}$. There are similar problems with other expression types; we need to revisit them all in light of the coercion facility...

Fortress provides a mechanism, called *coercion*, to allow a value of one type to be automatically converted to a value of another type. Programmers can define coercions (described in Section 17.2) and coercions may change the set of applicable declarations for a call (as described in Section 17.4). If multiple coercions can be applied to a particular functional call then the most appropriate coercion is chosen statically. Restrictions on coercion declarations (described in Section 17.6) guarantee that the static resolution of coercion (described in Section 17.5) is well-defined. Fortress provides implicit coercions for tuple types and arrow types (described in Section 17.7). Fortress also provides an additional feature of coercion that allows "widest-need" evaluation of numerical expressions (described in Section 17.8).

17.1 Principles of Coercion

In certain situations it is convenient to be able to use a value of one type T as if it were a value of another type U even though T is not a subtype of U. For example, it is convenient to be able to use an integer-valued expression in a floating-point expression even though its type is not a subtype of any floating-point type. Fortress supports the

automatic conversion of values of type $\mathbb{Z}32$, and of integer numerals, to the floating-point type $\mathbb{R}64$ (the technical term for this kind of automatic conversion is coercion); a value of any other integer type is converted to $\mathbb{R}64$ explicitly (Section 33.1). In this way one can write $(x+1)/2$ rather than $(x+1.0)/2.0$, for example. Such coercion applies generally to function and method (collectively called as functional) calls as well as to operators; one can write $\ln 2$ rather than $\ln 2.0$, or $\arctan(1,1)$ rather than $\arctan(1.0,1.0)$, for example.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.16
>
> The Working Draft of February 2011 said that Fortress supports the automatic conversion of integer values to floating-point values. The revival decided on 26 September 2026 that a conversion between number types is a coercion where it is exact and stays explicit where it can lose precision, and the Fortress standard libraries declare the coercion into $\mathbb{R}64$ from $\mathbb{Z}32$, and the decision names the integer numerals too. The conversion from $\mathbb{N}32$, though exact, stays explicit, since that decision names only $\mathbb{Z}32$ and the numerals. The examples of this paragraph convert numerals and are unchanged. The Fortress standard libraries give an integer numeral the type $\mathrm{IntLiteral}$, from which $\mathbb{R}64$ coerces, and the compiled type checker types a numeral so (Section 13.1, Section I.1.36). The interpreter does not yet: it gives an integer numeral outside the range of $\mathbb{Z}32$ a wider integer type, and so does not yet convert it. The compiled path's own library declares no coercion into $\mathbb{R}64$ from an integer type or an integer numeral; there $\mathbb{Z}32$, $\mathbb{N}32$ and integer numerals convert to $\mathbb{R}64$ only explicitly, and the other integer types not at all.

One way to think about coercion is that if type T can be coerced to type U, then a value of type T can be used to "stand in" for a value of type U in a functional call, for example. This is different from actually *being* of type U; it means only that, given any value of type T, an appropriate value of type U can be computed to substitute for it. Also, coercion occurs only when the declared type of the corresponding parameter in the functional declaration is exactly the type U being coerced to not if it is a supertype of U. Note, however, that if type T can be coerced to type U then any subtype of T can also be coerced to type U.

Coercion from T to U may occur in an expression context where the context expects the expression to have type U and the type of the expression is a subtype of T except for the following cases:

- type ascriptions (see Section 13.31)
- type assumptions (see Section 13.32)

For example, a coercion may occur in a variable definition when the declared type of the variable is U and the type of the expression on the right-hand side is a subtype of T.

The expression contexts where a coercion can occur are:

- the right-hand sides of variable and field declarations where the left-hand sides have declared types
- arguments to functionals and constructors where the corresponding parameters have declared types
- body expressions of functionals and constructors where the return types are declared
- the right-hand sides of test and property declarations
- expressions in contracts
- expressions whose enclosing expressions constrain the types of their subexpressions

Chapter 13 describes the type constraints for each expression in Fortress.

> DRAFT NOTE
>
> - assignment expressions with type on the left-hand sides
> - filter in a generator

- spawn
- throw
- while Expr
- if Expr
- if . then Expr end
- at Expr

Coercion is *not* automatically chained in Fortress, unlike some other programming languages: even if type T can be coerced to type U and type U can be coerced to type V, it is not necessarily the case that type T can be coerced to type V. Type T can be coerced to type V only if this coercion is explicitly defined.

The basic principles behind coercion are (1) a decision by the designer of a trait U that, in all circumstances where a functional is to be called, an actual argument of some other type T may be used to satisfy a parameter of type T, and (2) a language mechanism for putting this decision into effect automatically and concisely.

Example 1: For any floating-point parameter, a decimal integer literal argument may be used.

Example 2: For any floating-point parameter, a floating-point argument of a shorter format may be used.

Example 3: For any floating-point interval parameter, a non-interval floating-point argument (of the same or shorter floating-point format) may be used.

17.2 Coercion Declarations

Syntax:

Coercion	::=	`coerce` *StaticParams*? (*BindId IsType*) *CoercionClauses* `widens` ? = *Expr*
CoercionClauses	::=	*CoercionWhere*? *Ensures*? *Invariant*?
CoercionWhere	::=	`where` ⟦ *WhereBindingList* ⟧ ({ *CoercionWhereConstraintList* })?
	|	`where` { *CoercionWhereConstraintList* }
CoercionWhereConstraintList	::=	*CoercionWhereConstraint*(, *CoercionWhereConstraint*)*
CoercionWhereConstraint	::=	*WhereConstraint*
	|	*Type* `widens or coerces` *Type*

To declare that trait U allows a coercion from type T, the declaration of trait U must provide a coercion declaration whose parameter type is T. Coercion declarations are like functional declarations, except that `coerce` is actually a reserved word, a coercion declaration may have special `where`-clause constraints (described below), it is not allowed to have a `throws` clause or a `requires` clause, and it is not permitted to specify a return type, because the return type must be the very trait in whose declaration the coercion declaration appears. The coercion body is required; there is no such thing as an abstract coercion declaration. Within a trait or object declaration, coercion declarations appear before any field declarations.

Coercions may have static parameters (described in Chapter 12) and a `where` clause (described in Section 12.6), just like functionals. The `where` clause may contain one of the following special constraints:

Type_1 `coerces` Type_2
Type_1 `widens` Type_2
Type_1 `widens or coerces` Type_2

The first is true if trait Type_1 has a coercion from Type_2. The second is true if trait Type_1 has a widening coercion (described in Section 17.8) from Type_2. The third may be used only in a coercion with the "`widens`" reserved word; it is true if Type_1 has a coercion from Type_2, and furthermore the coercion declaration to which the `where` clause belongs is widening only if Type_1 has a widening coercion from Type_2. For example:

```
trait Vector⟦T extends Number⟧
  coerce ⟦S extends Number⟧(x: Vector⟦S⟧)
    where {T widens or coerces S} = ...
end
```

Coercions, unlike methods, are not inherited. If trait V extends trait U, and trait U has a coercion from type T, then V does *not* thereby have a coercion from type T. (It may, however, have its own coercion from type T, separately defined within the body of V.) For example, given the following declarations,

```
trait A
  coerce (b: B) = ...
end
object B end
trait C extends A end
f(c: C) = 5
```

a call to $f(B)$ is considered a static error because trait C does not inherit a coercion from type B.

17.3 Coercion Invocations

One way to think about coercion is that explicit calls to the coerce function are automatically inserted into the expression in a context where a coercion can occur. For example, if trait U has a coercion from type T, then whenever a functional, f, is declared with a parameter of type U, a call $f(t)$ where t has type T can be rewritten to $f(\text{coerce}⟦U⟧(t))$ making the declaration of f applicable to the call. However, this rewriting does not occur if there exists a declaration that is applicable to the call before the rewriting (see Section 17.5 for further discussion).

If coercion is possible for more than one element of a tuple argument, a cross-product effect is obtained. For example, in this code:

```
object X
  coerce (kiki: Y) = ...
  coerce (tutu: Q) = ...
  hack(other: X) = 1
end
object Y end
object Q end
foo(dodo: X) = 2
bar(hyar: X, yon: X) = 3
bar(hyar: Q, yon: Q) = 4
```

the following method invocations are valid:

```
X.hack(Y)
X.hack(Q)
```

Because there is no declaration of $hack$ applicable to Y and Q, these invocations are rewritten to:

```
X.hack(coerce⟦X⟧(Y))
X.hack(coerce⟦X⟧(Q))
```

Similarly, the function calls in the left-hand side of the following are valid and rewritten to the right-hand side:

$foo(Y)$	*is rewritten to*	$foo(\text{coerce}[\![X]\!](Y))$
$foo(Q)$	*is rewritten to*	$foo(\text{coerce}[\![X]\!](Q))$
$bar(Y, X)$	*is rewritten to*	$bar(\text{coerce}[\![X]\!](Y), X)$
$bar(Q, X)$	*is rewritten to*	$bar(\text{coerce}[\![X]\!](Q), X)$
$bar(X, Y)$	*is rewritten to*	$bar(X, \text{coerce}[\![X]\!](Y))$
$bar(X, Q)$	*is rewritten to*	$bar(X, \text{coerce}[\![X]\!](Q))$
$bar(Y, Y)$	*is rewritten to*	$bar(\text{coerce}[\![X]\!](Y), \text{coerce}[\![X]\!](Y))$
$bar(Q, Q)$	*is rewritten to*	$bar(Q, Q)$
$bar(Q, Y)$	*is rewritten to*	$bar(\text{coerce}[\![X]\!](Q), \text{coerce}[\![X]\!](Y))$
$bar(Y, Q)$	*is rewritten to*	$bar(\text{coerce}[\![X]\!](Y), \text{coerce}[\![X]\!](Q))$

Note that the call $bar(Q, Q)$ remains unchanged because there exists a declaration for bar that is applicable without coercion.

To continue the example, let us illustrate a point about type parameters. If we add the following definitions:

```
trait Frobboz
  frob(item : X) = 5
  coerce (m : Matrix) = ... (* irrelevant! *)
end
object Bozo⟦T extends Frobboz⟧(t : T) ... end
```

then this says nothing about whether the type parameter T of Bozo has coercions, because coercions are not inherited. (In fact, we can make a stronger statement: types named by type parameters *do not have coercions*!) But it is guaranteed that the following method invocations are valid if they occur within the declaration of Bozo:

> DRAFT NOTE
> (if t has type Bozo:[???]) – Sukyoung

$t.frob(X)$
$t.frob(Y)$
$t.frob(Q)$

because T inherits the method declaration $frob(item : X)$ from Frobboz and X contains coercions from Y and Q.

17.4 Applicability with Coercion

As discussed in the previous section, coercion in Fortress alters the set of applicable declarations for a particular functional call. In this section we formally define the applicability of declarations with coercion. (This level of formality is necessary to discuss the interaction of overloading and coercion.) We build on the terminology and notation defined in Chapter 15.

A *coercion* from T to U is defined in the declaration of U. We write $T \to U$ if U defines a coercion from T. We say that T *can be coerced to* U, and we write $T \rightsquigarrow U$, if U defines a coercion from T or any supertype of T; that is, $T \rightsquigarrow U \iff \exists T' : T \preceq T' \wedge T' \to U$.

We define coercion between tuple types and arrow types in Section 17.7.

Because of automatic coercion, a declaration may be applicable even if its parameter type is not a supertype of the argument type of the call. We define a new relation, *substitutability*, that takes coercion into account.

We say that a type T is *substitutable* for type U, and we write $T \sqsubseteq U$, if T is a subtype of U or T can be coerced to U; that is,

$$T \sqsubseteq U \iff T \preceq U \vee T \rightsquigarrow U.$$

Note that $\sqsubseteq$ need not be transitive. This is a result of the fact that coercion does not chain. However, if T is substitutable for U then so are T's subtypes; that is, $A \preceq T \wedge T \sqsubseteq U \implies A \sqsubseteq U$.

A declaration $f(P)$ is *applicable with coercion* to a call $f(C)$ if the call is in the scope of the declaration and $C \sqsubseteq P$. (See Chapter 7 for the definition of scope.)

Recall a rewriting of named functional calls into dotted method calls when no declarations are applicable to the named functional call. This rewriting is also used to determine applicability with coercion. If there is no declaration applicable with coercion to a named functional call then the call is rewritten into a dotted method call. Applicable declarations are then determined by the rules for applicability with coercion for dotted method calls.

Similarly, a dotted method declaration $P_0.f(P)$ is *applicable with coercion* to a dotted method call $C_0.f(C)$ if it $C_0 \preceq P_0$ and $C \sqsubseteq P$. Notice that the receiver is compared using the subtype relation instead of the substitutable relation. This reflects the restriction that receivers of dotted methods cannot be coerced. However, `self` parameters of functional methods can be coerced. This distinction is consistent with the intuition that functional methods are rewritten into top-level functions.

Note that the only difference between applicability and applicability with coercion is that substitutability is used instead of subtyping to compare the parameter types of the declarations.

Also note that the definition of applicability with coercion does not take keyword parameters or varargs parameters into account. Such declarations are conceptually rewritten into numerous declarations which cannot be called with elided arguments nor with variable number of arguments (as described in Section 15.4). If one of the expanded declarations is applicable with coercion to a call, then the original declaration (with keyword or varargs parameters) is applicable with coercion to the call.

17.5 Coercion Resolution

For a given functional call, we first determine whether there exists a declaration that is applicable without coercion. If so, the most specific declaration is selected; if not, then coercions are explicitly added to the functional call. If more than one coercion is possible then the coercion that yields the most specific type is chosen. Section 17.6 and Chapter 24 describe overloading rules for the declarations of coercions and overloaded functionals that guarantee there exists always a most specific coercion when two or more are possible. A declaration with static parameters is applicable, without or with coercion, when some instance of it whose static arguments satisfy their bounds is, and declarations are compared on the parameter types they declare (Section 15.2, Section 15.5); the declaration selected is then instantiated as Section 20.1 describes, with the rule for a numeral whose conversions tie (Section 20.2).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 did not say here when a declaration with static parameters is applicable. The revival decided on 28 September 2026 that a coercion makes a call possible and never changes which declaration runs when one is applicable without it: declarations are chosen in the order this section gives, on the parameter types they declare, and the declaration chosen is then instantiated.

We first define a notion of more specific types in the presence of coercion. Recall from Section 6.2 that Fortress defines an *exclusion* relation between types which is denoted by $\Diamond$.

For types T and U, we say that T *rejects* U, and write $T \not\rightarrow U$, if for all coercions to T, the type coerced from excludes U:

$$T \not\rightarrow U \iff \forall A : A \rightarrow T \implies A \Diamond U.$$

Note that $T \not\rightarrow U$ implies $U \not\rightarrow T$.

We say that T is *no less specific* than U, and write $T \trianglelefteq U$, if T is a subtype of U or T excludes, can be coerced to, and rejects U:

$$T \trianglelefteq U \iff T \preceq U \vee (T \lozenge U \wedge T \rightsquigarrow U \wedge T \not\rightarrow U).$$

The $\trianglelefteq$ relation is reflexive and antisymmetric but not necessarily transitive. It is possible, for example, for three types, T, U and V, each excluding the other two, to have coercions from T to U and from U to V. In this case, $T \trianglelefteq U$ and $U \trianglelefteq V$ but $T \not\trianglelefteq V$.

Notice that if two tuple types have a bijective correspondence between their element types then the $\trianglelefteq$ relationship between the tuple types is equivalent to applying the $\trianglelefteq$ relation elementwise.

We say that T is *more specific* than U, and write $T \triangleleft U$, if $T \trianglelefteq U \wedge T \neq U$.

We extend the definitions of no less specific and more specific to declarations. We say that a declaration $f(P)$ is *no less specific* than a declaration $f(Q)$ if $P \trianglelefteq Q$. Similarly, we say that a declaration $P_0.f(P)$ is *no less specific* than a declaration $Q_0.f(Q)$ if $(P_0, P) \trianglelefteq (Q_0, Q)$. A declaration $f(P)$ is *more specific* than a declaration $f(Q)$ if $P \triangleleft Q$. Similarly, a declaration $P_0.f(P)$ is *more specific* than a declaration $Q_0.f(Q)$ if $(P_0, P) \triangleleft (Q_0, Q)$.

Now we describe how to determine which coercion is applied to a given call in terms of rewriting functional calls. The rewriting is for pedagogical purposes; implementation techniques may vary.

Consider a static call $f(A)$ or $A_0.f(A)$. Let Σ be the set of parameter types of functional declarations of f that are applicable to the call. Let Σ' be the set of parameter types of functional declarations of f that are applicable with coercion to the call. If Σ is not empty then we use the overloading resolution described in Section 15.5 to determine which element of Σ is called. The parameter type of a declaration with static parameters is in Σ, or in Σ', when some instance of the declaration whose static arguments satisfy their bounds is applicable to the call, without or with coercion; the static arguments of the declaration selected are inferred afterwards (Section 20.1).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 did not say when the parameter type of a declaration with static parameters is in these sets.

If Σ is empty but Σ' is not, then we rewrite the call as follows. Define:

$$\mathcal{C} = \{T \in \Sigma' \mid S \rightarrow T \wedge A \preceq S\}.$$

Let $T \in \mathcal{C}$ be the most specific element of $\mathcal{C}$. In other words, there does not exist $T' \in \mathcal{C}$ such that $T' \triangleleft T$. The restrictions given in Section 17.6 and Chapter 24 guarantee that such a T exists and that it is unique. The call $f(A)$ or $A_0.f(A)$ is rewritten to $f(\text{coercion}[\![T]\!](\text{A}))$ or $A_0.f(\text{coerce}[\![T]\!](A))$ and the declaration with parameter type T is applied to the call.

As an example, consider the following definitions:

```
trait 𝕫32 excludes { 𝕫64, 𝕫128 } end
trait 𝕫64 excludes { 𝕫128 }
  coerce (z : 𝕫32) = ...
end
trait 𝕫128
  coerce (z : 𝕫32) = ...
  coerce (z : 𝕫64) = ...
end
f(z : 𝕫64) = 1
f(z : 𝕫128) = 2
```

Then the call $f(\mathbb{Z}32)$ resolves to the declaration $f(\mathbb{Z}64)$ because $\mathbb{Z}64 \lhd \mathbb{Z}128$, which follows from the facts that $\mathbb{Z}64$ excludes $\mathbb{Z}128$, coerces to it, and rejects it: the one type that coerces to $\mathbb{Z}64$, $\mathbb{Z}32$, excludes $\mathbb{Z}128$. The same facts make the two declarations of f a valid overloading, and the exclusion of $\mathbb{Z}64$ by $\mathbb{Z}32$ does the same for the two coercions to $\mathbb{Z}128$ (Section 24.4).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.24
>
> The Working Draft of February 2011 declared the three traits with no `excludes` clause and said that the resolution follows from the fact that $\mathbb{Z}64$ coerces to $\mathbb{Z}128$. By the definition above, $\mathbb{Z}64$ is then not more specific than $\mathbb{Z}128$, and neither the two declarations of f nor the two coercions to $\mathbb{Z}128$ are a valid overloading; both implementations of the revival refuse the example as the Working Draft wrote it. The clauses added are spelled as the Fortress standard libraries spell those of their integer types, each trait excluding the wider ones.

Notice that coercion is resolved statically. Once a coercion is statically chosen for a call, this coercion is conceptually inserted into the call. This means that the statically chosen coercion is applied at run time. The coercions that the instance of a declaration with static parameters needs are chosen and inserted into the call in the same way (Section 20.1).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 did not say how the coercions of an instance of a declaration with static parameters are chosen.

For example, in the following program:

```
trait A
  coerce (c:C) = ...
end
trait B excludes A end
trait C end
object D extends {B, C} end

f(a:A) = 3
f(b:B) = 4

c:C = D
f(c)
```

the call to $f(C)$ resolves to the declaration $f(A)$ despite the fact that the declaration $f(B)$ is applicable to the dynamic call $f(D)$ and does not require coercion.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.32
>
> The compiled path of the revival resolves the example as this section says and runs $f(A)$. The interpreter has no static types, so it chooses a coercion on the value: for $f(c)$ it finds $f(B)$ applicable to the value without coercion and runs it, and it answers the same way wherever an argument's static type needs a coercion and the argument's value is of a type that another declaration already fits. The two cases are gated as expected failures of the interpreter (`ProjectFortress/tests/XXXCoercionStaticRungC.fss` and `XXXCoercionStaticNarrowRungC.fss`, row 19 of the revival's gap ledger). The interpreter is to follow this section once the compiled type checker checks the programs it runs and it takes the coercions that the checker inserts.

17.6 Restrictions on Coercion Declarations

We place two restrictions on coercion declarations to ensure that it is always possible to resolve coercions.

It is a static error for a type to define a coercion from any of its subtypes. For example, the following program is statically rejected:

```
trait Number
  coerce (z : ℤ) = ...
end
trait ℤ extends Number end
```

Instead, z of type $\mathbb{Z}$ can be coerced to type Number by "$\mathrm{coerce}[\![\mathrm{Number}]\!](z)$" where coerce is defined in the Fortress standard libraries (as described in Section 13.33.5).

It is also a static error for cycles to exist in the type hierarchy produced by extension and coercion declarations. Such cycles may be composed of solely subtype relationships or solely coercion or a mixture of the two. In all cases the cycles are statically rejected. An example showing a cycle that is a mixture of subtype and coercion relationships follows:

```
trait A end
trait B extends A end
trait C extends B
  coerce (a : A) = ...
end
```

17.7 Coercions for Tuple and Arrow Types

Unlike other types, tuple types (described in Section 6.5) and arrow types (described in Section 6.6) are not defined explicitly. Therefore coercions to these types are also not defined explicitly. Instead, the following rules describe when these coercions are implicitly defined.

There is a coercion from a tuple type X to a tuple type Y if all the following conditions hold:

1. X is not a subtype of Y;
2. for every plain type T in Y, there is a corresponding plain type in X whose type is substitutable for T;
3. if neither X nor Y has a varargs type, then they have the same number of plain types;
4. if X has a varargs type, then Y has a varargs type, the type S of the varargs type $S...$ in X is substitutable for the type T of the varargs type $T...$ in Y, and X and Y have the same number of plain types;
5. if X has no varargs type and Y has a varargs type $T...$, then every plain type in X that has no corresponding plain type in Y is substitutable for T; and
6. the correspondence between keyword-type pairs in X and Y is bijective, and the type of each such pair in X is substitutable for the type in the corresponding pair in Y.

Tuple type coercions are invoked by distributing the coercion elementwise. However, if an element type of the tuple type being coerced from is a subtype of the corresponding element type of the tuple type being coerced to, the coercion is ignored. For example, the following coercions:

$\mathrm{coerce}[\![(A, B)]\!](x, y)$
$\mathrm{coerce}[\![(kwd = A)]\!](kwd = x)$

are rewritten to:

$(\text{coerce}[\![A]\!](x), \text{coerce}[\![B]\!](y))$
$(\text{coerce}[\![(kwd = A)]\!](x))$

However, if the type of y were a subtype of B then the first coercion would instead be rewritten to:

$(\text{coerce}[\![A]\!](x), y)$

Coercions to varargs types such as the following:

$\text{coerce}[\![(A \ldots)]\!](x, y)$

are invoked by applying the coercion to the type in the varargs type, A in this example, to each of the elements of the tuple. Additionally, there is a coercion from any type T to a tuple type solely with varargs type $(T \ldots)$.

There is a coercion from arrow type "$A \to B$ `throws` C" to arrow type "$D \to E$ `throws` F" if all of the following conditions hold:

1. "$A \to B$ `throws` C" is not a subtype of "$D \to E$ `throws` F";
2. D is substitutable for A;
3. B is substitutable for E; and
4. for all X in C, there exists Y in F such that X is substitutable for Y.

Arrow type coercions are invoked by wrapping the argument function of the coercion in a function expression that applies the appropriate coercions to the argument and result of the function. For example, assuming that f is a function from type A to type B the following coercion:

$\text{coerce}[\![C \to D]\!](f)$

is rewritten to:

`fn` $x \Rightarrow \text{coerce}[\![D]\!](f(\text{coerce}[\![A]\!](x)))$

17.8 Automatic Widening

Syntax:

Coercion ::= `coerce` *StaticParams*? (*BindId IsType*) *CoercionClauses* `widens` = *Expr*

Fortress supports what is sometimes called "widest-need" evaluation of numerical expressions. To see the problem, suppose that a and b are 32-bit floating-point numbers and c is a 64-bit floating-point number. It is easy, and tempting, to write an expression such as

$c = c + a \cdot b$

but if you stop to think about it, if interpreted naively it will compute the product $a \cdot b$ as a 32-bit floating-point number, and the impression that the sum may be accurate to the precision of a 64-bit floating-point number is only an illusion. Widest-need processing takes context into account and "widens" (or "upgrades") the operands a and b to 64-bit floating-point numbers before the product is computed, so that the product will be computed as a 64-bit result.

Before widening is considered, a Fortress compiler, in its normal course of operation, analyzes an expression bottom-up. For every functional invocation, it needs to statically identify a specific declaration to be invoked. Once this is done, then for every functional invocation, one of two conditions holds: (a) The type of the argument expression is a subtype of the type of the parameter in the identified declaration. (b) The type of the argument expression can be coerced to the type of the parameter in the identified declaration.

This process has to be done bottom-up because in order to select a specific declaration for a functional invocation, it is necessary to know the types of the argument expressions, and if an argument expression is itself a functional invocation, it is necessary to select the specific declaration for that invocation in order to find out the return type of the invocation.

Once an expression has been fully analyzed in this way, then widening is performed as a top-down process. For each functional invocation that appears in a context where a coercion can occur, ask whether the argument expression (which, remember, is a functional invocation) falls under case (a) or case (b) above. If case (b), coercion of the functional result is required; if the relevant coercion declaration is a "widening" coercion, then this functional invocation is a candidate for widening. Call the type of the functional invocation T, and call the expected type of the coercion context, U.

When the functional call was considered during the bottom-up analysis, in general many overloaded declarations may have been considered; of all those that were applicable with coercion to the static call, the most specific one was chosen. When considering widening, we consider the same set of declarations, but first discard all declarations whose return types are not subtypes of U. Then if any remain, and there is a unique most specific one among them, then that declaration is attached to the functional call instead. In this way a coercion step is eliminated (coercion is no longer needed to convert the result of the functional invocation from type T to type U), possibly at the expense of requiring a new coercion in a subexpression—but this is exactly the desired effect. Once this is done, then the subexpressions of the functional call are processed recursively.

This process is general enough not only to provide for widening floating-point precision, but to handle two other cases of interest as well: widening point computations to interval computations, and widening computations involving aggregate data structures whose components are widenable. For example, in $d(v \times w)$ suppose that d is of type $\mathbb{R}64$ and v and w are 3-vectors with components of type $\mathbb{R}32$; this strategy is capable of promoting v and w to 3-vectors with components of type $\mathbb{R}64$ and then performing all computations with $\mathbb{R}64$ precision.

Let's go through the example of $c + a \cdot b$ in detail. The relevant declarations might look something like this:

```
trait ℝ32
  ...
end
trait ℝ64
  coerce (x : ℝ32) widens = ...
  ...
end
opr ·(l: ℝ32, r: ℝ32): ℝ32 = ... (* 1 *)
opr ·(l: ℝ64, r: ℝ64): ℝ64 = ... (* 2 *)
opr +(l: ℝ32, r: ℝ32): ℝ32 = ... (* 3 *)
opr +(l: ℝ64, r: ℝ64): ℝ64 = ... (* 4 *)
```

The bottom-up analysis observes that a and b are both of type $\mathbb{R}32$, and discovers that declarations 1 and 2 are both applicable to the product, but declaration 1 would be chosen because it requires no coercion and declaration 2 would require coercion of both arguments to type $\mathbb{R}64$. The analysis then observes that c has type $\mathbb{R}64$ and the product expression has type $\mathbb{R}32$, so declaration 3 is not applicable but declaration 4 is applicable (though requiring a coercion from $\mathbb{R}32$ to $\mathbb{R}64$ for its second argument).

Now the top-down widening analysis is performed. The product is a functional call that is an argument to another functional call, and its result requires coercion, and that coercion is a widening coercion. Therefore the compiler reconsiders the applicable declarations, which were declarations 1 and 2. The result type of declaration 1 is not a subtype of $\mathbb{R}64$, but the result type of declaration 2 is a subtype of $\mathbb{R}64$ (in fact, it *is* $\mathbb{R}64$). Declaration 2 is the most specific among the applicable declarations with that property (in fact, in this example it's the only one), so declaration 2 replaces declaration 1 for the product invocation. This in turn requires a and b to be coerced from type $\mathbb{R}32$ to $\mathbb{R}64$ before the product is performed.

Widening is a tricky business, best left to expert library designers. When used judiciously, it can greatly improve the precision of numerical computations without requiring a great deal of thought on the part of the application programmer and without cluttering up code with explicit conversions.

Future versions of this specification will include tables of coercions and widening coercions supported by the Fortress standard libraries.

Chapter 18

Dimensions and Units

> DRAFT NOTE
> Dimensions and units are not yet supported.
>
> The examples in this chapter are not tested nor run by the interpreter.

Syntax:

TypeRef :	:=	*DimType*
DimType	::=	*DimRef*
	|	*TypeRef DimRef*
	|	*TypeRef* · *DimRef*
	|	*TypeRef* / *DimRef*
	|	*TypeRef* per *DimRef*
	|	*TypeRef UnitRef*
	|	*TypeRef* · *UnitRef*
	|	*TypeRef* / *UnitRef*
	|	*TypeRef* per *UnitRef*
	|	*TypeRef* `in` *DimRef*
DimRef	::=	*StaticArg*
StaticArg	::=	Unity
	|	dimensionless
	|	*StaticArg* · *StaticArg*
	|	*StaticArg StaticArg*
	|	*StaticArg* / *StaticArg*
	|	1 / *StaticArg*
	|	*StaticArg* ^ *StaticArg*
	|	*StaticArg* per *StaticArg*
	|	*DUPreOp StaticArg*
	|	*StaticArg DUPostOp*
	|	*TypeRef*
	|	(*StaticArg*)
DUPreOp	::=	square
	|	cubic
	|	inverse
DUPostOp	::=	squared
	|	cubed

Expr	::=	*UnitExpr*
UnitExpr	::=	*UnitRef*
	|	*Expr UnitRef*
	|	*Expr* · *UnitRef*
	|	*Expr* / *UnitRef*
	|	*Expr* per *UnitRef*
	|	*Expr* `in` *UnitRef*
UnitRef	::=	*StaticArg*

There are special type-like constructs called *dimensions* that are separate from other types (described in Chapter 6). There are also special constructs that modify types and values called *units* that are instances of dimensions. These are used to describe physical quantities.

The Fortress standard libraries define dimensions and units for the standard SI system of measurement based on meters, kilograms, seconds, amperes, and so on (as described in Section 37.1). The Fortress standard libraries also provide supplemental units of measurement, such as feet and miles (as described in Section 37.2). For example, the Fortress standard libraries provide a dimension named Length whose default unit is named meter and abbreviated $\mathrm{m_}$. By rendering convention, this abbreviation is rendered in roman type without the underscore: m. In contrast, the variable m is rendered as in standard mathematical notation: m. See Section 4.17 for a discussion of formatting conventions for tokens.

For readability, plural forms of the unit names are defined as equivalent to the corresponding singular forms; thus one can write $\mathrm{meters\ per\ second}$, for example. Standard SI prefixes may be used on both the name and the symbol, so that $\mathrm{nanometer}$ and nm are also units of the dimension named Length, related to meter and m by a conversion factor of 10^{-9}.

Every dimension may have a default unit that is used for representing values of quantities of that dimension if no unit is specified explicitly. The Fortress standard libraries define these default dimensions and units:

Length	meter	MagneticFlux	weber
Mass	kilogram	MagneticFluxDensity	tesla
Time	second	Inductance	henry
Frequency	hertz	Velocity	meters per second
Force	newton	Acceleration	meters per second squared
Pressure	pascal	Angle	radian
Energy	joule	SolidAngle	steradian
Power	watt	LuminousIntensity	candela
Temperature	kelvin	LuminousFlux	lumen
Area	square meter	Illuminance	lux
Volume	cubic meter	RadionuclideActivity	becquerel
ElectricCurrent	ampere	AbsorbedDose	gray
ElectricCharge	coulomb	DoseEquivalent	*sievert*
ElectricPotential	volt	AmountOfSubstance	mole
Capacitance	farad	CatalyticActivity	katal
Resistance	ohm	MassDensity	kilograms per cubic meter
Conductance	siemens		

In addition, the Fortress standard libraries define the dimension $\mathrm{Information}$ with units bit and byte (and the plurals bits and bytes), a byte being equal to 8 bits. To avoid confusion, SI prefixes are *not* provided for these units; instead, programmers must use appropriate powers of 2 or 10, for example $10^6\mathrm{bits}$ or $2^{20}\mathrm{bits}$.

Here are some examples of the use of dimensions and units:

$x\colon \mathbb{R}64\,\mathrm{Length} = 1.3\,\mathrm{m}$
$t\colon \mathbb{R}64\,\mathrm{Time} = 5\,\mathrm{s}$

v: $\mathbb{R}64$ Velocity $= x/t$
w: $\mathbb{R}64$ Velocity `in` $\mathrm{nm/s} = 17\,\mathrm{nm/s}$
x: $\mathbb{R}64$ Velocity `in` furlongs per fortnight $= v$ `in` furlongs per fortnight

Dimensions and units can be multiplied, divided, and raised to rational powers to produce new dimensions and units. Both the numerator and the denominator of a rational power of a dimension or a unit must be a valid `nat` *parameter* instantiation (as described in Section 12.2). Multiplication can be expressed using juxtaposition or $\cdot$; division can be expressed using / or per. The syntactic operators square and cubic may be applied to a dimension or unit in order to raise it to the second power, third power, respectively; the special postfix syntactic operators squared and cubed may be used for the same purpose. The syntactic operator inverse may be applied to a dimension or unit to divide it into 1. All of these syntactic operators are merely syntactic sugar, expanded before type checking.

> DRAFT NOTE
> I manually added a space between inverse and ohms below.

grams per cubic centimeter
meter per second squared
inverse ohms

One can also write $1/X$ as a synonym for X^{-1} if X is either a dimension or a unit.

Most numerical values in Fortress are dimensionless quantity values. Multiplying or dividing a dimensionless value by a unit produces a dimensioned value; thus $5\,\mathrm{s}$ is the dimensioned value equal to five seconds, which has numerical value 5 and second for its unit. A dimensioned value may also be multiplied or divided by a unit, and the result is to combine the units; the expression $(17\,\mathrm{nm})/\mathrm{s}$ first multiplies 17 by nm to produce the dimensioned value seventeen nanometers, which is then divided by the unit s to produce seventeen nanometers per second.

The unit of a dimensioned value may be changed to another unit of the same dimension by using the `in` operator, which takes a quantity to its left and a unit to its right. The `in` operator changes the unit and multiplies or divides the numerical value by an appropriate conversion constant so as to preserve the overall dimensioned value. Thus $1.3\,\mathrm{m}$ `in` nm produces $1300000000\,\mathrm{nm}$.

Multiplying or dividing a dimensionless numerical type by a unit produces an equivalent dimensioned numerical type with that unit associated; thus $\mathbb{R}64\,\mathrm{meter}$ is a type that is just like $\mathbb{R}64$ but whose values are values of dimension Length measured in meters. A dimensioned numerical type may be further multiplied or divided by a unit. As a convenience, if a dimension has a default unit, a numerical type may also be multiplied or divided by a dimension, in which case the result is as if the default unit for that dimension had been used. The `in` operator may also be used to change the unit associated with a dimensioned type; in this situation the effect is merely to alter the unit associated with the type; no numerical operation is performed at run time.

Certain aggregate types, such as Vector, may also have associated units.

There are two reasons for using dimensions and units. One is that the `in` operator can supply conversion factors automatically. The other is that certain programming errors may be detected at compile time. When dimensioned values are added, subtracted, or compared, it is a static error if the units do not match. When dimensioned values are multiplied or divided, their units are multiplied or divided. When taking the square root of a dimensioned value, the unit of the result is the square root of the argument's unit. Other numerical functions, such as sin and log, require dimensionless arguments.

A variable whose declared type includes a dimension without an accompanying unit is understood to have the default unit for that dimension. Thus, in most cases, the runtime unit of an expression can be statically inferred. However, there are exceptions. For example, consider the following declaration:

> DRAFT NOTE
> I manually edited the array type.

$$a : \mathrm{Object}[3] = [5\,\mathrm{mg}, 3\,\mathrm{m}, 4\,\mathrm{s}]$$

Now suppose we declare a function that takes an array of objects and returns one of its elements:

> DRAFT NOTE
> I manually edited the array type.

$$f(xs : \mathrm{Object}[3]) = xs_1$$

The value of the call $f(a)$ is $3\,\mathrm{m}$. However, the static type of $f(a)$ is simply Object. When the value $3\,\mathrm{m}$ is placed in an array of objects, the value is boxed, and the unit associated with the value must be included as part of the boxed value. However, unboxed values need not include unit information at runtime, as this information is statically evident in such cases.

> DRAFT NOTE
> Guy's explanation — The thing to understand is that a measurement is a value object, but storing it in an $\mathrm{Array}[\![\mathrm{Object}]\!]$ will force the value object to be boxed. Boxing causes type information to be associated with the value. Unit information is not really "erased" at compile time; rather, unit information is "static", that is, part of the type, and so the unit information can be omitted from a run-time representation exactly when type information can be omitted from a run-time representation. The situation is exactly the same as for dimensionless numerical types such as ints and floats.

There are also special static parameters for units and dimensions; see Section 12.4.

Chapter 19

Tests and Properties

> DRAFT NOTE
> Tests and properties are not yet supported.
>
> Examples in this chapter are not tested nor run by the interpreter.

> DRAFT NOTE
> Ticket #248
>
> By default, tests actually in a given component are run. Do not run the run function while running tests. Overloading the same name with test and non-test functionals is not allowed. To test multiple components:
>
> `fortress test blah1.fss blah2.fss ...`

> DRAFT NOTE
> Introduce another keyword `testhelper` for auxiliary functions in tests. The `testhelper` codes need to be in APIs.

> DRAFT NOTE
> A limited form of tests are supported by the interpreter as follows:
>
> To help make programs more robust, the `test` modifier can appear on a top-level function definition with type "$() \to ()$". Functions with modifier `test` must not be overloaded with functions that do not have modifier `test`. The collection of all functions in a program that include modifier `test` are referred to collectively as the program's *tests*. Tests can refer to non-tests but it is a static error for a non-test to refer to any test. Tests can refer to non-tests but non-tests must not refer to any test.
>
> For example, we can write the following test function:
>
> `test` $testFactorial_1() =$ `do`
>
> $assert\big(fact(0) = 1\big)$
>
> $assert\big(fact(5) = 120\big)$
>
> $println$("`testFactorial1 passed`");
>
> `end`

When a program's tests are *run*, each top-level function definition with modifier `test` is run in textual program order.

Fortress supports automated program testing. Components and APIs can declare *tests*. Test declarations specify explicit finite collections over which the test is run. Components, APIs, traits, and objects can declare *properties* which describe boolean conditions that the enclosing construct is expected to obey. Tests and properties may modify the program state.

19.1 The Purpose of Tests and Properties

To help make programs more robust, Fortress programs are allowed to include special constructs called *tests* and *properties*. Tests consist of test data along with code that can be run on that data. Properties are documentation used to describe the behavior of the traits and functions of a program; they can be thought of as comments written in a formal language. For each property, there is a special function that can be called by a program's tests to ensure that the property holds for specific test data.

A fortress includes hooks to allow programmers to run a specific test on an executable component, and to run all tests on such a component. A particularly useful time to run the tests of an executable component is at component link time; errors in the behavior of constituent components can be caught before the linked program is run.

19.2 Test Declarations

Syntax:

TestDecl	::=	`test` *Id* `[` *GeneratorClauseList* `]` = *Expr*
GeneratorClauseList	::=	*GeneratorBinding*(, *GeneratorClause*)*
GeneratorBinding	::=	*BindIdOrBindIdTuple* ← *Expr*
GeneratorClause	::=	*GeneratorBinding*
	|	*Expr*
BindIdOrBindIdTuple	::=	*BindId*
	|	(*BindId*, *BindIdList*)

A test declaration begins with `test` followed by an identifier, a list of zero or more generators (described in Section 13.14) enclosed in square brackets, the token =, and a subexpression. When a test is *run* (See Section 22.8), the subexpression is evaluated in each extension of the enclosing environment corresponding to each point in the cross product of bindings determined by the generators in the generator list.

For example, the following test:

DRAFT NOTE

The fortify has some problems here...

`test` $fxLessThanFy[x \leftarrow E, y \leftarrow F] = assert(f(x) < f(y))$

checks that, for each value x supplied by generator E, and for each value y supplied by generator F, $f(x)$ is less than $f(y)$.

19.3 Other Test Constructs

The `test` modifier can also appear on a function definition, trait definition, object definition, or top-level variable definition. In these contexts, the modifier indicates that the program construct it modifies can be referred to by a test. Functions with modifier `test` must not be overloaded with functions that do not have modifier `test`, and traits with modifier `test` must not be extended by traits or objects that do not have modifier `test`. The collection of all constructs in a program that include modifier `test` are referred to collectively as the program's *tests*. Tests can refer to non-tests but it is a static error for a non-test to refer to any test.

For example, we can write the following helper function:

```
test ensureApplicationFails(g, x) = do
  applicationSucceeded := false
  try
    g(x)
    applicationSucceeded := true
  catch e
    Exception ⇒ ()
  end
  if applicationSucceeded then
    fail "Application succeeded!"
  end
end
```

The library function *fail* displays the error message provided and terminates execution of the enclosing test.

19.4 Running Tests

When a program's tests are *run*, the following actions are taken:

1. All top-level definitions, including constructs beginning with modifier `test`, are initialized. A test declaration with name t declares a function named t that takes a tuple of parameters corresponding to the variables bound in the generator list of the test declaration. For each variable v in the generator list of t, if the type of generator supplied for v is $\mathrm{Generator}[\![\alpha]\!]$ then the parameter in the function corresponding to v has type α. The return type of the function is $()$.

   Each such function bound in this manner is referred to as a *test function*. Test functions can be called from the rest of the program's tests.

2. Each test t in a program is run in each extension of t's enclosing environment with a point in the cross product of bindings determined by the generators in the test's generator list.

19.5 Test Suites

In order to allow programmers to run strict subsets of all tests defined in a program, Fortress allows tests to be assembled into *test suites*.

The convenience object $\mathrm{TestSuite}$ is defined in the Fortress standard libraries. An instance of this object contains a set of test functions that can all be called by invoking the method run:

```
test object TestSuite(testFunctions = {})
  add(f: () → ()) = testFunctions.insert(f)
  run() =
    for t ← testFunctions do
      t()
    end
end
```

Note that all tests in a TestSuite are run in parallel.

19.6 Property Declarations

Syntax:

PropertyDecl	::=	`property` (*Id* =)? (∀ *ValParam*)? *Expr*
ValParam	::=	*BindId*
	|	(*Params*?)
Params	::=	(*Param*,)* (*Varargs*,)? *Keyword*(, *Keyword*)*
	|	(*Param*,)* *Varargs*
	|	*Param*(, *Param*)*
VarargsParam	::=	*BindId*: *Type*...
Varargs	::=	*VarargsParam*
Keyword	::=	*Param*= *Expr*
PlainParam	::=	*BindId IsType*?
	|	*Type*
Param	::=	*PlainParam*

Components and APIs may include *property* declarations, documenting boolean conditions that a program is expected to obey. Syntactically, a property declaration begins with `property` followed by an optional identifier followed by the token $=$, an optional value parameter, which may be a tuple, preceded by the token $\forall$, and a boolean subexpression. In any execution of the program, the boolean subexpression is expected to evaluate to true at any time for any binding of the property declaration's parameter to any value of its declared type. When a property declaration includes an identifier, the property identifier is bound to a function whose parameter and body are that of the property, and whose return type is Boolean. A function bound in this manner is referred to as a *property function*.

Properties can also be declared in trait or object declarations. Such properties are expected to hold for all instances of the trait or object and for all bindings of the property's parameter. If a property in a trait or object includes a name, the name is bound to a method whose parameter and body are that of the property, and whose return type is Boolean. A method bound in this manner is referred to as a *property method*. A property method of a trait T can be called, via dotted method notation, on an instance of T.

Property functions and methods can be referred to in a program's tests. If the result of a call to a property function or method is not *true*, a TestFailure exception is thrown. For example, we can write:

```
property fIsMonotonic = ∀(x: ℤ, y: ℤ) (x < y) → (f(x) < f(y))

test s : Set[[ℤ]] = {−2000, 0, 1, 7, 42, 59, 100, 1000, 5697}
test fIsMonotonicOverS[x ← s, y ← s] = fIsMonotonic(x, y)
test fIsMonotonicHairy[x ← s, y ← s] = fIsMonotonic(x, x² + y)
```

The test *fIsMonotonicOverS* tests that function f is monotonic over all values in set s. The test *fIsMonotonicHairy* tests that f is monotonic with respect to the values in s compared to the set of values corresponding to all the ways in which we can choose an element of s, square it, and add it to another element of s.

Chapter 20

Type Inference

A call of a function, a method or an operator whose declaration has static parameters, or of the constructor of an object that has static parameters, need not write its static arguments, and most calls do not (Section 13.2, Section 13.4). The static arguments of a call that does not write them are inferred statically. This chapter describes that inference for type parameters (Section 12.1) and for `nat` and `int` parameters (Section 12.2); the inference of the static arguments of other kinds of static parameter, and the inference over a whole program of the types that declarations elide (Section 22.6), are not yet described.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.26
>
> The Working Draft of February 2011 left this chapter a heading and two draft notes, the second of which is kept at its end, although Chapter 15, Section 13.2 and Section 13.4 cite it. The rule this chapter states is the one the revival decided on 26, 27 and 28 September 2026, and on 29 September 2026 for a lone type parameter with no narrowest candidate; the compiled type checker was built with this text, and the cases in which it departs from the text are listed in Section I.1.26. The Working Draft had no counterpart to the rule for a numeral whose conversions tie (Section 20.2). A type parameter declared without an `extends` clause is bounded by Any (Section 12.1); the compiled path still takes Object for such a parameter of a program's own declarations (row 412 of the revival's gap ledger). Chapter 15 and Chapter 24 were revised with this chapter's order, a declaration chosen on the parameter types it declares and then instantiated (Section I.1.29). The team's later Types chapter gives the coercion relation and the valid instantiations of a quantified type, and says nothing of inference.

20.1 The Static Arguments of a Call

The static arguments of a call that does not write them are found in two steps: the declaration that the call invokes is chosen first, and it is instantiated second.

Choice. The declaration is chosen as Section 17.5 describes. A declaration with static parameters is applicable to a call, without or with coercion, if some instance of it whose static arguments satisfy their bounds is applicable to the call. Declarations are compared on the parameter types they declare, a declaration with static parameters on its parameter types taken over every instance that its bounds allow, and not on an instance inferred for the call. So a coercion is applied only to make a call possible: it never changes which declaration is chosen when one is applicable to the call without it.

Instantiation. The static arguments of the chosen declaration are then inferred, and with them the instance of the declaration that the call invokes:

- A type parameter, or a `nat` or `int` parameter, that the declared type of a parameter mentions other than as the whole type, as $b\colon \mathrm{List}[\![T]\!]$ mentions T, is fixed by the argument at that position, whose type must be a subtype

of the parameter type of the instance.

- Where the call has an expected type (below), a type parameter that the declared return type mentions other than as the whole type is fixed by the expected type as well.
- A type parameter that is the whole declared type of one or more parameters, as T is of $x\colon T$, and that is not fixed as above, takes, among the types of the arguments at those positions and the types to which they can be coerced, the narrowest under $\trianglelefteq$ (Section 17.5) that every one of those arguments is substitutable for (Section 17.4) and that the bounds of the type parameter permit. For number arguments this is the narrowest type into which every one of them converts: $\mathbb{Z}64$ for a $\mathbb{Z}32$ and a $\mathbb{Z}64$, and also for a $\mathbb{Z}32$ and an $\mathbb{N}32$, though neither of them is a $\mathbb{Z}64$ (Section 33.2). Where there is no such narrowest type, and the rule for numerals (Section 20.2) does not settle the tie, the type parameter takes the intersection of its upper bounds: its declared bound, Any if it has none (Section 12.1), and what the expected type (below), where the call has one, requires of the declared return type, provided the types of those arguments are subtypes of it; none of those arguments is converted.

  > REVISED BY THE 2026 REVIVAL SEE Section I.1.26
  >
  > The revival's text of 28 September 2026 gave such a type parameter the union of the types of those arguments; on 29 September 2026 the revival took the rule by which the dispatcher of the type group's paper of 2019 instantiates a type parameter, which gives the bound.

- A type parameter that neither an argument nor the expected type fixes as above, such as a type parameter that only the return type mentions, as its whole type, likewise takes the intersection of its upper bounds: its declared bound, Any if it has none (Section 12.1), and what the expected type (below), where the call has one, requires of the declared return type. Inference never instantiates a static parameter at $\mathrm{BottomType}$, which programmers must not write (Section 6.8). So, for the declaration $stop[\![T]\!](s\colon \mathrm{String})\colon T$, the variable declaration $x\colon \mathbb{Z}32 = stop(\text{“}\texttt{no}\text{”})$ instantiates $stop$ at $\mathbb{Z}32$, and the same call with no expected type instantiates it at Any.

  > REVISED BY THE 2026 REVIVAL SEE Section I.1.26
  >
  > The revival's text of 28 September 2026 did not describe such a type parameter and left open whether inference may give it $\mathrm{BottomType}$, which the team's draft note at the end of this chapter asks. The revival decided on 29 September 2026 that the rule by which the dispatcher of the type group's paper of 2019 instantiates a type parameter that the arguments do not fix, the intersection of its upper bounds, which never instantiates a type parameter at $\mathrm{BottomType}$, is the rule of static inference as well.

- Every argument is then admitted against the corresponding parameter type of the instance by substitutability: an argument whose type is a subtype of it is passed as it is, and any other argument is coerced to it (Section 17.1), as an argument is coerced to the declared type of a parameter of a declaration without static parameters.

The coercions that the instance needs are inserted into the call statically, as is any coercion that Section 17.5 chooses.

The expected type. A call has an expected type in these contexts: as the right-hand side of a variable or field declaration that declares its type, that type; as the right-hand side of an assignment to a variable whose type is declared, that type; as the body of a functional that declares its return type, that type; as the last expression of a block that has an expected type, whether or not local declarations precede it in the block, the block's expected type; as a branch of an `if` expression with an `else` clause that has an expected type, the expected type of the `if` expression; as a clause of an `if` expression without an `else` clause, $()$, the type that every such clause must have (Section 13.19); as a clause of a `typecase` expression that has an expected type, its `else` clause among them, the expected type of the `typecase` expression, of which the union of the types of its clauses, the type of the `typecase` expression (Section 13.22), is then a subtype where the type of each clause is; as the body of a `label` expression that has an expected type, or as the `with` clause expression of an `exit` expression whose target is that `label` expression, the expected type of the `label` expression, of which the union of the type of its body and the types of those `with` clause values, the type of the `label` expression (Section 13.12), is then a subtype where each of those types is; and as an argument of a call

that this section does not accept with its arguments typed without an expected type, where the argument is itself a call with inferred static arguments, the parameter type at the argument's position of each declaration of the enclosing call in turn, instantiated, where that declaration has static parameters, by the arguments that are not so typed and by the expected type of the enclosing call alone, as described below. The expected type is used whether the call is written by tight juxtaposition, as $f(x)$ (Section 16.8), or by loose juxtaposition, as $f\ x$, or as a method invocation or an operator application.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.27
>
> The revival's text of 29 September 2026 gave the last expression of a block an expected type only where no local declaration precedes it, and gave none to a clause of an `if` expression without an `else` clause, to a clause of a `typecase` expression or to a call written by loose juxtaposition; the revival decided on 8 October 2026 that each has the type that the text already requires of it. Its text of 8 October 2026 gave none to the body of a `label` expression or to the `with` clause expression of an `exit` expression; the revival decided on 9 October 2026 that they have the expected type of the `label` expression, as a clause of a `typecase` expression has that of the `typecase` expression. Its text of 9 October 2026 gave none to an argument of another call; the revival decided on 9 October 2026 that an argument that is itself a call with inferred static arguments has the parameter type of each declaration of the enclosing call in turn where the enclosing call is not accepted without it, since the static arguments of a call "are statically inferred from the context of the function call" (Section 13.2).

The return type of the instance that the steps above find must be a subtype of the expected type. If they find no such instance, the static arguments are inferred again without the expected type, and the call is accepted only if the return type of the instance then found is a subtype of the expected type or can be coerced to it; in the second case the result of the call is coerced to the expected type (Section 17.1). So, for the declarations $wrap[\![T]\!](x\colon T)\colon \mathrm{List}[\![T]\!]$ and $id[\![T]\!](x\colon T)\colon T$, the variable declaration $a\colon \mathrm{List}[\![\mathbb{Z}64]\!] = wrap(3)$ instantiates $wrap$ at $\mathbb{Z}64$, which the expected type fixes, and coerces the numeral, while $b\colon \mathbb{Z}64 = id(3)$ instantiates id at the numeral's own type (Section 13.1) and coerces the result.

The arguments of a call are typed first without an expected type. Where this section then accepts no instance of the call, each argument that is itself a call with inferred static arguments is typed again, once for each declaration of the enclosing call, with the parameter type at its position as its expected type, as the list of contexts above says. Of the declarations that are applicable to the enclosing call with its arguments so typed, each declaration with its own, and whose instances have return types that are subtypes of the expected type of the enclosing call, where it has one, or can be coerced to it, the enclosing call invokes the one that is more specific than every other on the parameter types they declare; where there is no such declaration, the call is typed with its arguments typed without an expected type, and is accepted or refused as above. So where this section accepts an instance of a call with its arguments typed without an expected type, no argument is typed again, and the declaration it invokes is never passed over for one that only an argument typed again makes applicable; a declaration that is applicable to those arguments but whose return type neither is a subtype of the expected type nor can be coerced to it is not accepted, and another may then be chosen. For example, for the declarations $mk[\![T]\!]()\colon \mathrm{Box}[\![T]\!]$, $takes(b\colon \mathrm{Box}[\![\mathbb{Z}64]\!])$, $over(b\colon \mathrm{Box}[\![\mathbb{Z}64]\!])$ and $over(x\colon \mathrm{Any})$, the call $takes(mk())$ instantiates mk at $\mathbb{Z}64$, which the parameter type of $takes$ fixes, while the call $over(mk())$ invokes the second declaration of $over$, which is applicable to $mk()$ typed without an expected type, the instance of mk at the bound of its type parameter.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.27
>
> The revival's text of 9 October 2026 had no such paragraph; the revival added it with the argument of a call to the list of contexts above, following its decision of 9 October 2026.

Dispatch. The call, with the coercions of its instance inserted, dispatches at run time as Section 15.5 describes, by subtyping and without further coercion, to the most specific declaration applicable to the converted values.

For example, of the declarations $op[\![T\ \texttt{extends}\ \mathrm{Number}]\!](a\colon T, b\colon T)$ and $op(a\colon \mathrm{Any}, b\colon \mathrm{Any})$, the call $op(z, w)$, where z has type $\mathbb{Z}32$ and w has type $\mathbb{Z}64$, chooses the first, which is applicable without coercion, since its instance at Number is, and is more specific than the second; the first is instantiated at $\mathbb{Z}64$, z is coerced, and the converted

call runs that instance.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.26
>
> The interpreter has no static types and applies the same rule at dispatch, from the run-time types of the arguments: it infers the static arguments of a declaration with static parameters with coercion, and instantiates the declaration that its dispatch chooses by the rule for a type parameter that is a parameter's whole type. So where the declared type of a variable is wider than the type of its value, or an argument's static type is Any, it may run a declaration at a narrower instance than the compiled path, which instantiates it at the static types. Where neither an argument's type nor a type into which the arguments convert is the narrowest, it gives a type parameter that is the whole type of parameters its declared bound, Any if it has none, as this chapter does: for a $\mathbb{Z}64$ and an $\mathbb{R}64$, Number where that is the bound and Any where the type parameter is declared without an `extends` clause. It does the same where the arguments that fix a type parameter inside other types, as the results of two function arguments do, have several minimal common supertypes. Neither holds of a type parameter whose bound mentions a static parameter, itself or another, or that is declared with several bounds that the interpreter cannot meet, as it cannot meet two bounds neither of which is a subtype of the other: such a type parameter it binds, as before, to the narrowest named common supertype of the arguments' run-time types that the bounds permit, where there is one, and where there is none a call that fixes a type parameter of the second kind fails (row 591 of the revival's gap ledger). When its dispatch chooses among declarations, it passes over a declaration with static parameters whose arguments have several minimal common supertypes in favour of any other declaration applicable to the call without coercion (row 589 of the revival's gap ledger). It fixes a type parameter by the expected type only at a reduction or a comprehension that writes no static argument and whose value is that of a local variable declared with a type or of a top-level or local function declared with a return type, as the box after the list in Section 20.2 says; elsewhere it does not yet fix a type parameter by the expected type. A type parameter that an argument bounds only from above, as a function argument bounds the type parameter of its parameter type, it instantiates at the intersection of the upper bounds that its declared bound and the arguments place on it: at $\mathbb{Z}32$ for a function of a $\mathbb{Z}32$ where the type parameter is declared without an `extends` clause, and at Number for a function of an Any where it is bounded by Number; such a type parameter whose bound mentions a static parameter, or that is declared with several bounds that it cannot meet, it still instantiates at $\mathrm{BottomType}$; and where the arguments place on such a type parameter two upper bounds that it cannot meet, as a function over one trait and a function over another do when neither trait extends the other, the call fails with an error of unification (row 616 of the revival's gap ledger). It infers the static arguments of a generic dotted method that a call does not write as it infers a generic function's, whether the method is alone under its name or among the overloads of its receiver. It compares declarations with static parameters on the parameter types they declare, but for two kinds: a functional method of a generic trait or object it compares on the instance that a call makes, and a generic dotted method on the parameter types it declares only where exactly one of the two declarations' declared parameter types lies inside the other's, and otherwise on that instance. It stops at load on an object or trait that declares a generic dotted method beside a plain one of the same name (row 760 of the revival's gap ledger), and it stops on an object that extends a generic trait whose dotted method has a static parameter of its own bounded by the trait's, at load or at a generic object's first instance (row 762). It reads the bounds of a generic method that a trait declares, and the types into which the arguments convert, where the trait declares the method, as it reads those of a method that an object declares. It dispatches a converted call again, except a call of an overloaded method.

20.2 A Numeral Whose Conversions Tie

A numeral has a type of its own, from which the Fortress standard libraries define coercions to the number types (Section 13.1). Where the types to which a numeral can be converted are incomparable under $\trianglelefteq$, as $\mathbb{Z}32$ and $\mathbb{N}32$ are, each taking a numeral and neither converting into the other, the numeral is read as a $\mathbb{Z}32$, or as a $\mathbb{Z}64$ or a $\mathbb{Z}$ when its value does not fit in the narrower type. This holds in either step: in the choice among declarations that are applicable to the call only with coercion and that the numeral would otherwise leave tied, and in the instantiation of a type parameter that only numerals fix. So of the declarations $g(x{:}\,\mathbb{N}32)$ and $g(x{:}\,\mathbb{Z}32)$, the call $g(0)$ invokes the second; and a type parameter that only numerals fix, whose bounds $\mathbb{Z}32$ and $\mathbb{N}32$ satisfy and the numeral's own type does not, is instantiated at $\mathbb{Z}32$ when $\mathbb{Z}32$ holds the numerals' values.

The rule settles only a tie that a numeral makes. Where an argument that is not a numeral leaves the declarations applicable to a call only with coercion tied, no declaration is the most specific, and the call is a static error.

This chapter does not yet describe:

- the static arguments of a `bool`, `dim`, `unit` or operator parameter (Section 12.3, Section 12.4, Section 12.5);
- a `nat` or `int` parameter that neither an argument nor the expected type fixes;
- the element type of a reduction that only its generator clauses determine (Section 13.17): the rule above gives a static parameter of the big operator that nothing at the call fixes its bounds, not the type of the elements;
- whether an argument whose declared parameter type mentions a static parameter inside another type may be converted where only a conversion of its elements would satisfy the bounds of the parameter; under the rule above such an argument fixes the parameter by subtyping and is not converted;
- an expected type in any context other than those listed above, such as the body of a `for` loop;
- the static arguments of an object with static parameters that is named without a call, which a program writes (Section 12.6);
- the inference over a whole program of the types that declarations elide (Section 22.6).

REVISED BY THE 2026 REVIVAL SEE Section I.1.27

The revival's text of 29 September 2026 named here also an expression that a local declaration precedes in a block, an `if` expression without an `else` clause, a `typecase` expression and a call written by loose juxtaposition, its text of 8 October 2026 also the body of a `label` expression, and its text of 9 October 2026 also an argument of another call, to which the list of contexts in Section 20.1 now gives an expected type where the argument is itself a call with inferred static arguments and the enclosing call is not accepted without it.

REVISED BY THE 2026 REVIVAL SEE Section I.1.26

The compiled type checker instantiates a type parameter that neither an argument nor the expected type fixes at the intersection of its upper bounds, as Section 20.1 says; on the compiled path a type parameter of a program's own declaration that is declared without an `extends` clause is bounded by Object (row 412 of the revival's gap ledger), so that a call that fixes such a parameter by nothing else, where the expected type is one that Object excludes, such as $()$, is a static error there. A reduction's big operator is an argument of the call that the reduction's desugaring makes, so it has an expected type only as such an argument has one: the compiled type checker instantiates a generic big operator whose static argument the reduction does not write at its bound, and at the reduction's expected type only where the reduction's type at that bound does not fit it, and it refuses the reduction where the big operator has no instance at its bound, as one whose bound mentions its own static parameter has none (row 425 of the revival's gap ledger). The interpreter instantiates a type parameter of a function that nothing at a call fixes at its bound, Any if it has none, and one whose bound mentions other static parameters at that bound with their instances, with two exceptions: a type parameter that only the bound of a type parameter that the arguments fix mentions, as A `extends` T mentions T, it instantiates at the arguments' type, not at its bound, since it has no expected type to fix it (row 592 of the revival's gap ledger); and a type parameter declared with several bounds that it cannot meet, as two bounds neither of which is a subtype of the other, it erases to $\mathrm{BottomType}$ (row 591). A type parameter whose bound mentions the type parameter itself, as the big operators $\sum$ and $\prod$ declare theirs, and that nothing at a call fixes, it leaves open: it does not instantiate it, and it admits every value where it checks a value against that type, as an argument, a variable declared with that type, a `typecase` clause or a type assertion. The big operator that a reduction or a comprehension invokes without its static arguments it instantiates as the desugaring by the type N of the expression would (Section 13.17), with run-time types in place of the static type that it does not have: by the declared type of the local variable, or the declared return type of the top-level or local function, whose value the reduction or the comprehension is, also as the last expression of a block, where that type fixes the big operator's result type; otherwise by the join of the run-time types of the elements that its generator clauses give, which it collects before it reduces them; and where there is no element, at the bound of each static parameter, or open where the bound mentions the parameter itself. So an empty sum that is the value of a function declared $\mathbb{R}64$ is the $\mathbb{R}64$ zero, and the library's set reductions $\bigcup$ and $\bigcap$ run on sets of their elements' type; but an empty reduction that no declared type directs takes the identity of $\mathbb{Z}32$ whatever the type of the expression, and an empty comprehension that no declared type

directs is built at the bound, a list comprehension as a $\mathrm{List}[\![\mathrm{Any}]\!]$ (row 716 of the revival's gap ledger); a comprehension that no declared type reaches, such as the value of a top-level variable, a field, a method or a function expression, the right-hand side of an assignment, the body of another reduction or an argument of a call, is built at the run-time types of its elements, which the declared type of an enclosing expression may not admit (rows 20, 510 and 717). A big operator whose declared result is none of the library's $\mathrm{BigReduction}$, $\mathrm{Comprehension}$ and $\mathrm{BigOperator}$ it instantiates through the one of those three that its declared result extends, with that trait's static arguments. It gives a function expression that declares no return type the return type $\mathrm{BottomType}$, so that a type parameter that only its result fixes is instantiated there (row 587); and it instantiates a static parameter of a generic method that only the expected type would fix, as one of a generic function, at its bound (row 510), a method that a trait declares among them. The third item of the team's draft note below asks whether inference may give a static parameter $\mathrm{BottomType}$; the rule of Section 20.1, taken from the type group's paper of 2019, answers that it may not. Neither implementation infers a `bool` parameter yet: the compiled type checker refuses by name to infer a `bool`, `dim` or `unit` parameter, and the interpreter does not unify a `bool` one (row 307 of the revival's gap ledger) and stops at load on an object or trait whose method has a parameter type with a `bool` static argument (row 761). The example of Section 12.3, a coercion whose `bool` parameter only its parameter type mentions, can be applied only if its argument fixes that parameter.

DRAFT NOTE

- There seems to be a circular dependency between inference and juxtaposition disambiguation – you have to know if the subexpressions have arrow types to disambiguate, but you can't do that without using inference, which requires knowing how the variable of unknown type is used.
- Type Inference (Dan's email titled "Comments on a first reading of the spec" on 06/05/07)
- Do we want to forbid such cases where type inference infers $\mathrm{BottomType}$s for static parameters?

Chapter 21

Memory Model

Fortress programs are highly multithreaded by design; the language makes it easy to expose parallelism. However, many Fortress objects are mutable; without judicious use of synchronization constructs—reductions and `atomic` expressions—*data races* will occur and programs will behave in an unpredictable way. The memory model has two important functions:

1. Define a programming discipline for the use of data and synchronization, and describe the behavior of programs that obey this discipline. This is the purpose of Section 21.2.

2. Define the behavior of programs that do not obey the programming discipline. This constrains the optimizations that can be performed on Fortress programs. The remaining sections of this chapter specify the detailed memory model that Fortress programs must obey.

21.1 Principles

The Fortress memory model has been written with several important guiding principles in mind. Violations of these principles must be taken as a flaw in the memory model specification rather than an opportunity to be exploited by the programmer or implementor. The most important principle is this: violations of the Fortress memory model must still respect the underlying data abstractions of the Fortress programming language. All data structures must be properly initialized before they can be read by another thread, and a program must not read values that were never written. When a program fails, it must fail gracefully by throwing an exception.

The second goal is nearly as important, and much more difficult: present a memory model which can be understood thoroughly by programmers and implementors. It must never be difficult to judge whether a particular program behavior is permitted by the model. Where possible, it must be possible to check that a program obeys the programming discipline.

The final goal of the Fortress memory model is to permit aggressive optimization of Fortress programs. A multiprocessor memory model can rule out optimizations that might be performed by a compiler for a uniprocessor. The Fortress memory model attempts to rule out as few behaviors as possible, but more importantly attempts to make it easy to judge whether a particular optimization is permitted or not. The semantics of Fortress already allows permissive operation reordering in many programs, simply by virtue of the implicitly parallel semantics of tuple evaluation and looping.

21.2 Programming Discipline

If Fortress programmers obey the following discipline, they can expect sequentially consistent behavior from their Fortress programs:

- Updates to shared mutable locations must always be performed using an `atomic` expression. A location is considered to be shared if and only if that location can be accessed by more than one thread at a time. For example, statically partitioning an array among many threads need not make the array elements shared; only elements actually accessed by more than one thread are considered to be shared.
- Within a thread or group of implicit threads objects must not be accessed through aliased references; this can yield unexpected results. Section 21.2.3 defines the notion of *apparently disjoint* references. An object must not be written through one reference when it is accessed through another apparently disjoint reference.

The following stylistic guidelines reduce the possibility of pathological behavior when a program strays from the above discipline:

- Where feasible, reduction must be used in favor of updating a single shared object.
- Immutable fields and variables must be used wherever practical. We discuss this further in Section 21.2.1.
- A getter or a setter should behave as though it is performing a simple field access, even if it internally accesses many locations. The simplest (but not necessarily most efficient) way to obtain the appropriate behavior is to make hand-coded accessors `atomic`. Section 21.2.2 expands on this.

21.2.1 Immutability

Recall from Section 5.3 that we can distinguish mutable and immutable memory locations. Any thread that reads an immutable field will obtain the initial value written when the object was constructed. In this regard it is worth re-emphasizing the distinction between an object reference and the object it refers to. A location that does not contain a value object contains an object reference. If the field is immutable, that means the reference is not subject to change; however, fields of the object referred to may still be modified in accordance with the memory model.

By contrast, recall that all the fields of a value object are immutable; however, a mutable location may have a value type. Such a location can be written, completely replacing the value object it contains. Similarly, reading the value contained in such a location conceptually causes the entire value object to be read; if this isn't followed by an immediate field reference, the read must be performed atomically. This may potentially be expensive (see Section 21.3).

21.2.2 Providing the Appearance of a Single Object

In practice, most accesses to fields occur through a mediating getter or setter method. It should not be possible for the programmer to tell whether a given getter or setter directly accesses a field or if it performs additional computations. Similarly, any subscripting operation must be indistinguishable from accessing a single field. Thus, accessor methods and subscripting methods should provide the appearance of atomicity, as described in Section 21.3. The Fortress standard libraries are written to preserve this abstraction. For example, the Array type in Fortress makes use of one or more underlying $\mathrm{HeapSequences}$, and array subscripting preserves the atomicity guarantees of this underlying object.

21.2.3 Modifying Aliased Objects

In common with Fortran, and unlike most other popular programming languages, Fortress gives special treatment to accesses to a location through different aliases. For the purposes of variable aliasing, it is important to define the notion

of *apparently disjoint* (or simply disjoint) object and field references. If two references are not disjoint, we say they are *certainly the same*, or just *the same*. By contrast, we say object references are *identical* if they refer to the same object, and *distinct* otherwise. Accesses to fields reached via apparently disjoint object references may be reordered (except an initializing write is never reordered with respect to other accesses to the identical location).

> DRAFT NOTE
> Problem: Consider a function call on a tree and its subtree. The common subtree won't be considered disjoint, right? We bottom out to comparing an argument to a field reference, and have recourse to the calling context. Is that what we want, or do we want to consider them a priori distinct (I think so).

> DRAFT NOTE
> Questions – Sukyoung
>
> 1. What and how do we guarantee with these?
>
> 2. coverage? if or if and only if?

Distinct references are always disjoint. Two identical references are apparently disjoint if they are obtained in any of the following ways:

- distinct parameters of a single function call
- distinct fields
- a parameter and a field
- identically named fields read from apparently disjoint object references
- distinct reads of a single location for which there may be an interposing write

When comparing variables defined in different scopes, these rules will eventually lead to reads of fields or to reads of parameters in some common containing scope.

We extend this to field references as follows: two field references are apparently disjoint if they refer to distinct fields, or they refer to identically named fields read from apparently disjoint object references.

Consider the following example:

$f(x\colon \mathbb{Z}64_2, y\colon \mathbb{Z}64_2)\colon \mathbb{Z}64 =$ `do`

$x_0 := 17$

$y_0 := 32$

`end`

Here x and y in f are apparently disjoint; the writes may be reordered, so the call $f(a, a)$ may assign either 17 or 32 to a_0.

A similar phenomenon occurs in the following example:

$g(x\colon \mathbb{Z}64_2, y\colon \mathbb{Z}64_2)\colon \mathbb{Z}64 =$ `do`

$x_0 := 17$

y_0

`end`

Again x and y are apparently distinct in g, so the write to x_0 and the read of y_0 may be reordered. The call $g(a, a)$ will assign 17 to a_0 but may return either the initial value of a_0 or 17.

It is safe to *read* an object through apparently disjoint references:

$h(x\colon \mathbb{Z}64_2, y\colon \mathbb{Z}64_2)\colon \mathbb{Z}64 =$ `do`

  $u\colon \mathbb{Z}64 = x_0$

  $v\colon \mathbb{Z}64 = y_0$

  $u + v$

`end`

A call to $h(a, a)$ will read a_0 twice without ambiguity. Note, however, that the reads may still be reordered, and if a_0 is written in parallel by another thread this reordering can be observed.

If necessary, `atomic` expressions can be used to order disjoint field references:

$f'(x\colon \mathbb{Z}64_2, y\colon \mathbb{Z}64_2)\colon () =$ `do`

  `atomic` $x_0 := 17$

  `atomic` $y_0 := 32$

`end`

Here the call $f'(a, a)$ ends up setting a_0 to 32. Note that simply using a single `atomic` expression containing one or both writes is not sufficient; the two writes must be in distinct `atomic` expressions to be required to occur in order.

When references occur in distinct calling contexts, they are disambiguated at the point of call:

$j(x\colon \mathbb{Z}64_2, y\colon \mathbb{Z}64)\colon () = x_0 := y$

$k(x\colon \mathbb{Z}64_2)\colon () =$ `do`

  $j(x, 17)$

  $j(x, 32)$

`end`

$l(x\colon \mathbb{Z}64_2, y\colon \mathbb{Z}64_2)\colon () =$ `do`

  $j(x, 17)$

  $j(y, 32)$

`end`

Here if we call $k(a)$ the order of the writes performed by the two calls to j is unambiguous, and a_0 is 32 in the end. By contrast, $l(a, a)$ calls j with two apparently disjoint references, and the writes in these two calls may thus be reordered.

> DRAFT NOTE
> Some stuff here about overwriting.
>
> Some stuff here about nested functions and accesses to stuff in the enclosing scope.

21.3 Read and Write Atomicity

This section makes potentially very controversial choices. In particular, the unit of atomic read/write can be considerably larger than the natural read/write granularity of the underlying machine. So we have to use hugely expensive mechanisms (the easiest of which is boxing and copying) if we read or write non-trivial objects non-transactionally.

> DRAFT NOTE
> I take comfort in the fact that we can obtain the required semantics by just putting these large operations into a read-only or write-only transaction and detecting/resolving conflict using the same set of underlying mechanisms.

Any read or write to a location is *indivisible*. In practical terms, this means that each read operation will see exactly the data written by a single write operation. Note in particular that indivisibility holds for a mutable location containing a large value object. It is convenient to imagine that every access to a mutable location is surrounded by an `atomic` expression. However, there are a number of ordering guarantees provided by `atomic` accesses that are not respected by non-`atomic` accesses.

21.4 Ordering Dependencies among Operations

The Fortress memory model is specified in terms of two orderings: dynamic program order (see Chapter 5 and Chapter 13) and memory order. The actual order of memory operations in a given program execution is *memory order*, a total order on all memory operations. Dynamic program order constrains memory order. However, memory operations need not be ordered according to dynamic program order; many memory operations, even reads and writes to a single field or array element, can be reordered. Programmers who adhere to the model in Section 21.2 can expect sequentially consistent behavior: the global ordering of memory operations will respect dynamic program order.

Here is a summary of the salient aspects of memory order:

- There is a single memory order which is respected in all threads.
- Every read obtains the value of the immediately preceding write to the identical location in memory order.
- Memory order on `atomic` expressions respects dynamic program order.
- Memory order respects dynamic program order for operations that certainly access the same location.
- Initializing writes are ordered before any other memory access to the same location.

21.4.1 Dynamic Program Order

Much of the definition of *dynamic program order* is given in the descriptions of individual expressions in Chapter 13. It is important to understand that dynamic program order represents a conceptual, naive view of the order of operations in an execution; this naive view is used to define the more permissive memory order permitted by the memory model. Dynamic program order is a partial order, rather than a total order; in most cases operations in different threads will not be ordered with respect to one another. There is an important exception: there is an ordering dependency among threads when a thread starts or must be complete.

> DRAFT NOTE
> TODO: re-factor into expressions chapter.

An expression is ordered in dynamic program order after any expression it dynamically contains, with one exception: a `spawn` expression is dynamically ordered before any subexpression of its body. The body of the `spawn` is dynamically ordered before any point at which the spawned thread object is observed to have completed.

Only expressions whose evaluation completes normally occur in dynamic program order, unless the expression is "directly responsible" for generating abrupt termination. Examples of the latter case are `throw` and `exit` expressions and division by zero. In particular, when the evaluation of a subexpression of an expression completes abruptly, causing the expression itself to complete abruptly, the containing expression does not occur in dynamic program order. A `label` block is ordered after an `exit` that targets it. The expressions in a `catch` clause whose `try` block throws a matching exception are ordered after the `throw` and before any expression in the `finally` clause. If the `catch` completes normally, the `try` block as a whole is ordered after the expressions in the `finally` clause. For this reason, when we refer to the place of non-`spawn` expression in dynamic program order, we mean the expression or any expression it dynamically contains.

For any construct giving rise to implicit threads—tuple evaluation, function or method call, or the body of an expression with generators such as `for`—there is no ordering in dynamic program order between the expression executed in each thread in the group.

> DRAFT NOTE
> Ugh, this is bogus:

These subexpressions are ordered with respect to expressions which precede or succeed the group.

When a function or method is called, the body of the function or method occurs dynamically after the arguments and function or receiver; the call expression is ordered after the body of the called function or method.

For conditional expressions such as `if`, `case`, and `typecase`, the expression being tested is ordered dynamically before any chosen branch. This branch is in turn ordered dynamically before the conditional expression itself.

Iterations of the body of a `while` loop are ordered by dynamic program order. Each evaluation of the guarding predicate is ordered after any previous iteration and before any succeeding iteration. The `while` loop as a whole is ordered after the final evaluation of the guarding predicate, which yields $false$.

An iteration of the body of a `for` loop, and each evaluation of the body expression in a comprehension or big operator, is ordered after the generator expressions.

21.4.2 Memory Order

> DRAFT NOTE
> Question: Should variables (esp parameters and mutable locals) be considered locations which can be read from / written to? This makes things easy to explain, but may not give the behavior we expect.

Memory order gives a total order on all memory accesses in a program execution. A read obtains the value of the most recent prior write to the identical location in memory order. In this section we describe the constraints on memory order, guided by dynamic program order. We can think of these constraints as specifying a partial order which must be respected by memory order. The simplest constraint is that accesses certainly to the same location must respect dynamic program order. Apparently disjoint accesses need not respect dynamic program order, but an initializing write must be ordered before all other accesses to the identical location in program order.

> DRAFT NOTE
> The following doesn't fully address Bill Pugh's concerns: it still allows cheesy outs where we write a bogus location which is never read. Also, we order atomics in memory order even if they don't share common data; can we tell,

and do we care

Accesses in distinct (non-nested) `atomic` expressions respect dynamic program order. Given an `atomic` expression, we divide accesses into four classes:

1. Constituents, dynamically contained within the `atomic` expression.
2. Ancestors, dynamically ordered before the `atomic` expression.
3. Descendants, dynamically ordered after the `atomic` expression.
4. Peers, dynamically unordered with respect to operations dynamically contained within the `atomic` expression.

We say an `atomic` expression is *effective* if it contains an access to a location, there is a peer access to the identical location, and at least one of these accesses is a write. For an effective `atomic` expression, every peer access must either be a *predecessor* or a *successor*. A predecessor must occur before every constituent and every descendant in memory order. A successor must occur after every constituent and every ancestor in memory order. Every ancestor must occur before every descendant in memory order.

The above conditions guarantee that there is a single, global ordering for the effective `atomic` expressions in a Fortress program. This means that for any executions of `atomic` expressions A and B one of the following conditions holds:

- A is dynamically contained inside B.
- B is dynamically contained inside A.
- Every expression dynamically contained in A precedes every expression dynamically contained in B in memory order. This will always hold when A is dynamically ordered before B.
- Every expression dynamically contained in B precedes every expression dynamically contained in A in memory order. This will always hold when B is dynamically ordered before A.

The above rules are also sufficient to guarantee that `atomic` expressions nested inside an enclosing `atomic` behave with respect to one another just as if they had occurred at the top level in an un-nested context.

Any access preceding a `spawn` in dynamic program order will precede accesses in the spawned expression in memory order. Any access occurring after a spawned thread has been observed to complete in dynamic program order will occur after accesses in the spawned expression in memory order.

A reduction variable in a `for` loop does not have a single associated location; instead, there is a distinct location for each loop iteration, initialized by writing the identity of the reduction. These locations are distinct from the location associated with the reduction variable in the surrounding scope. In memory order there is a read of each of these locations each of which succeeds the last access to that variable in the loop iteration, along with a read of the location in the enclosing scope which succeeds all accesses to that location preceding the loop in dynamic program order. These reads are followed by a write of the location in the enclosing scope which in turn precedes all accesses to that location that succeed the loop in dynamic program order.

DRAFT NOTE

The following is controversial. I think this is effectively the condition the X10 guys are trying to capture formally in their memory model work.

Finally, reads and writes in Fortress programs must respect dynamic program order for operations that are *semantically related*. If the read A precedes the write B in dynamic program order, and the value of B can be determined in some fashion without recourse to A, then these operations are not semantically related. A simple example is if A is a reference to variable x and B is the assignment $y := x \cdot 0$. Here it can be determined that $y := 0$ without recourse to x and these variables are not semantically related. By contrast, the write $y := x$ is always semantically related to the read of x. Note that two operations can only be semantically related if a transitive data or control dependency exists between them.

Chapter 22

Components and APIs

DRAFT NOTE

This chapter including various syntax is out of date.

Imported functionals may be overloaded with declared top-level functions.

Reject identical import statements.

At most one on-demand import statement from a single API.

The component/API names should be the same as their enclosing file names.

Only a single component/API declaration should occur in a single file.

No order between `import` and `export` statements.

Import aliasing syntax should use $\Rightarrow$ instead of `as`.

Eric's email titled "Proposal: Compound APIs" on 06/08/09: `api` Foo `comprises` $\{\mathrm{Foo1}, \mathrm{Foo2}, \mathrm{Foo3}, \mathrm{Foo4}\}$ `end`

Declaration exported by multiple APIs: A definition in a component must not match more than one declaration in exported APIs.

Jan's email titled "Proposed changes: scripting interface, args in a library" on 02/05/09

DRAFT NOTE

- It is a static error if a name specified by an import statement is also declared by a top-level or functional method declaration in the importing component or API, or if it is also specified by another import statement, unless all top-level or functional declarations for that name in the importing component or any of APIs imported by an import statement that specifies that name form a valid set of overloaded declarations. It is also a static error if any specified name is not declared by a declaration imported from the API.
- In addition, it is a static error if in a set of overloaded declarations, any of the following are true:
  1. Any declaration imported on demand is more specific than any explicit declaration.
  2. Any top-level function declaration is more specific than any functional method declaration. (This rule actually applies to declarations in a single component as well.)
  3. Any top-level function declaration imported on demand is more specific than any other declaration not in the same API as the top-level function declaration.

It is also a static error to explicitly import a name from an API that does not declare it, or to have multiple import-on-demand statements with the same API.

DRAFT NOTE

API integration

- We agreed to extend Fortress to allow APIs to extend other APIs. If an API A extends another API B, then every declaration of B is also a declaration of A. It is a static error if a name declared in B is also declared in A unless both declarations are of top-level functions or functional methods, in which case, the declarations are overloaded.

  The extension relation among APIs must be acyclic.
- Selective extension of specific types

```
api FortressLibrary
  extends Whatever.{Boolean}
  extends Whatever2.{Thread}
  ...
end
```

DRAFT NOTE

Resolve ambiguity in favor of longest API name. (07/23/09)

If there are an API $A.B$ and an API A which declares a trait B declaring a method f, $A.B.f$ in a component importing both $A.B$ and A is ambiguous. We agreed with the following:

- Importing an API: `import api` $A.B$
- Importing an entity: `import` $A.B$, `import` $A.\{B$ *as*`B_A`$\}$
- Resolve ambiguity in favor of longest API name.

DRAFT NOTE

Component and Value Namespace

```
component Surprise
import api Foo
export Executable

object Foo
  foo() : String = "foo"
  bar() : String = "foo"
  baz() : String = "foo"
end

run(args : String ...) = do
  Foo.foo()
  Foo.bar()
  Foo.baz()
end

end
```

Static error for the declaration of the object Foo in the above example.

DRAFT NOTE

Importing operators:

Operators with different fixities are not overloaded but different. When an ordinary operator (i.e., not a "vertical-line operator") is imported, all the operators with different fixities are imported. When a vertical-line operator is imported, only one fixity is imported.

DRAFT NOTE

"`import` $A.f$" does not imply "`import api` A".

"`api` A; `import api` B; `end`" does not export B.

Fortress programs are developed, compiled, and deployed as *encapsulated upgradable components* that exist not only as programming language features, but also as self-contained run-time entities that are managed throughout the life of the software. The imported and exported references of a component are described with explicit *APIs*. With components and APIs, Fortress provides the stability benefits of static linking with the sharing and upgrading benefits of dynamic linking. [1] In addition to an informal description of the component system in this chapter, we also formally specify key functionality of the system, and illustrate how we can reason about the correctness of the system in Appendix C.

22.1 Overview

Syntax:

File	::=	*CompilationUnit*
	|	*Imports*? *Exports Decls*?
	|	*Imports*? *AbsDecls*
	|	*Imports AbsDecls*?
CompilationUnit	::=	*Component*
	|	*Api*

Components are the fundamental structure of Fortress programs. They export and import APIs, which serve as "interfaces" of the components. A key design choice we make is to require that components never refer to other components directly; all external references are to APIs. This requirement allows programmers to extend and test existing components more easily, swapping new implementations of libraries in and out of programs at will. External references are resolved by linking components together: the references of a component to an imported API are resolved to a component that exports that API. Linking components produces new components, whose *constituents* are the components that were linked together.

Components are similar to modules in other programming languages, such as those of ML and Scheme [21, 17, 16]. But, unlike modules in those languages, components are designed for use during both development and deployment of software. In addition to compilation and linking, components can be produced by upgrading one component using another component that exports some of the APIs exported by the first component.

A key aspect of Fortress components is that they are encapsulated, so that upgrading one component does not affect any other component, even those produced by linking with the component that was upgraded. Abstractly, each component has its own copy of its constituents. However, implementations are expected to share common constituents when possible.

Users do not manipulate components directly. Instead, every component is installed in a persistent database on the system. We think of this database, which we call a *fortress*, as the agent that actually performs operations such as compilation, linking, upgrading, and execution of components: a virtual machine, a compiler, and a library registry

[1] The system described in this chapter is based on that described in [4].

all rolled into one. A fortress also maintains a list of APIs that are installed on it. A fortress also provides a shell by which the user can issue commands to it. Components and APIs are immutable objects. A fortress maps names to components installed on the system. The fortress operations are modeled as methods of the fortress that change the mapping.

The ways in which fortresses are actually realized on particular platforms are beyond the scope of this specification. An implementor might choose to instantiate a fortress as a process, or as a persistent object database stored in a file system, with fortress operations being implemented as scripts that manipulate this database.

> DRAFT NOTE
> Victor:
>
> Define (in lexical structure?) when program constructs are "the same" (i.e., identical except for "syntax" issues). Or else, we may want o to include a generic statement in the lexical structure chapter saying that after that chapter, except for the appendices, unless otherwise stated, two fragments of program text are considered the same if they are scanned into the same sequence of tokens (we need to be careful about the whitespace-sensitivity though).

We call the source code for a single software component a *project*. Typically, when a project written in other programming languages is compiled, each file in the project is separately compiled. To ship an application, these files are linked together to form an application or library. Fortress uses a different model: a project is compiled directly into a single component, which is installed in the compiling fortress.

From the point of view of the compiler, all the source code for a project is contained in a single file. This approach simplifies the design but it is unworkable for components of substantial size. Therefore, the compiler can be instructed to concatenate several source files together before compiling, while maintaining the original source location information.

After these components are compiled from source files, they can then be linked together to form larger components.

22.2 Components

> DRAFT NOTE
> "`import api` *AliasedAPINames*" syntax and qualified names are not yet supported.

Syntax:

Component	::=	`native`? `component` *APIName Imports*? *Exports Decls*? `end` (`component`? *APIName*)?
APIName	::=	*Id*(. *Id*)*
Imports	::=	*Import*+
Import	::=	`import` *ImportedNames*
	|	`import api` *AliasedAPINames*
ImportedNames	::=	*APIName* . { ... } (`except` *SimpleNames*)?
	|	*APIName* . { *SimpleNameList* (, ...)? }
{ *AliasedSimpleNameList* (, ...)? }		
	|	*QualifiedName* (`as` *Id*)?
SimpleNames	::=	*SimpleName*
	|	{ *SimpleNameList* }
SimpleNameList	::=	*SimpleName*(, *SimpleName*)*

SimpleName	::=	*Id*
	|	`opr BIG?` (*Encloser* | *Op*)
	|	`opr BIG?` *EncloserPair*
AliasedSimpleNameList	::=	*AliasedSimpleName*(, *AliasedSimpleName*)*
AliasedSimpleName	::=	*Id* (`as` *Id*)?
	|	`opr` *Op* (`as` *Op*)?
	|	`opr` *EncloserPair* (`as` *EncloserPair*)?
EncloserPair	::=	(*LeftEncloser* | *Encloser*) $\cdot$? (*RightEncloser* | *Encloser*)
AliasedAPINames	::=	*AliasedAPIName*
	|	{ *AliasedAPINameList* }
AliasedAPIName	::=	*APIName* (`as` *Id*)?
AliasedAPINameList	::=	*AliasedAPIName*(, *AliasedAPIName*)*
Exports	::=	*Export*$^+$
Export	::=	`export` *APINames*
APINames	::=	*APIName*
	|	{ *APINameList* }
APINameList	::=	*APIName*(, *APIName*)*
Decls	::=	*Decl*$^+$

In this specification, we will refer to components created by compiling a file as *simple components*, while components created by linking components together will be known as *compound components*.

The source code of a simple component definition begins with an optional modifier `native` followed by `component` followed by a *possibly qualified name* (an identifier or a sequence of identifiers separated by periods with no intervening whitespace), followed by a sequence of *import* statements, and a sequence of *export* statements, and finally a sequence of declarations, where all sequences are separated by newlines or semicolons. A component declaration may end with an optional `component` and its name again. It is a static error if the identifier after `end` is not the name of the component being declared.

22.2.1 Import Statements

An import statement imports declarations from one or more APIs. There are two forms of import statements. The first form imports all top-level and functional method declarations (i.e., declarations whose reach is the entire API) from the listed APIs, and allows the names declared by these declarations to be used *qualified* by the name of the API from which it is imported.

`import` $apiName, anotherApiName$

This form may also provide an *alias* for one or more of the APIs.

`import` $apiName$ `as` $alias, anotherApiName$

A name imported from an API with an alias must be qualified by the alias rather than the name of the API. It is a static error for any alias to be the name of any imported API. Multiple APIs may be given the same alias, but it is a static error if declarations imported from two APIs with the same alias declare the same name unless the set of imported declarations with that name from APIs with that alias form a valid set of overloaded declarations.

There are two forms of import statements: *explicit import statements* and *on-demand import statements*.

An explicit import statement specifies a single API and a set of names; it imports the top-level and functional method declarations in the specified API. It allows the specified names to be used *unqualified* in the importing component or API:

`import` APIName.{ $name(, name)^+$ }

We call an import statement of this form an *import-unqualified* statement. We say an import statement of this form is an "explicit import" statement. It is a static error if a name specified by an import-unqualified statement is also declared by a top-level or functional method declaration in the importing component or API, or if it is also specified by another import-unqualified statement, unless all top-level or functional declarations for that name in the importing component or any of APIs imported by an import-unqualified statement that specifies that name form a valid set of overloaded declarations. It is also a static error if any specified name is not declared by a declaration imported from the API.

An "ordinary" operator (i.e., operators other than enclosing or vertical-line operators) is a single token, and it is imported simply by putting `opr` before it in the import statement. This imports all of its prefix, postfix, infix, multifix and nofix declarations. A matched pair of enclosing operators is written `opr` the left encloser, the right encloser, with $\cdot$ optionally between the two enclosers. A vertical-line operator may be imported either as an infix operator or a bracketing operator, according to the rules above for ordinary and enclosing operators respectively.

An import-unqualified statement may include an alias for each name it specifies.

`import` APIName.{ $name$ `as` $name'(, name)^+$ }

In this case, the alias (unqualified) must be used instead of the name declared by the actual imported declaration, and the restriction about declarations in the importing component or API and names specified in other import-unqualified statements applies to the alias rather than the specified name.

For convenience, an on-demand import statement allows all names declared by imported declarations of the specified API to be referred to with unqualified names:

`import` APIName.{...}

This form permits an `except` clause, which lists names that are *not* permitted to be used unqualified by that import-unqualified statement (it may be permitted by a declaration in the importing component or API, or by some other import-unqualified statement).

`import` APIName.{...} `except` { $name(, name)^+$ }

A name $name$ is imported on demand in a component C from an API A only if all the following conditions hold:

1. C contains an on-demand import statement.
2. $name$ is neither listed in the `except` clause nor explicitly imported by C.
3. Either 1) a top-level or functional method declaration of $name$ appears in C or 2) a reference to $name$ appears in component C and C does not provide an explicit declaration of $name$.

The set of all declarations that are declared or imported (either explicitly or on demand) by a component must satisfy the overloading rules (and in particular, any nonfunctional declaration must not be overloaded). In addition, it is a static error if in a set of overloaded declarations, any of the following are true:

1. Any declaration imported on demand is more specific than any explicit declaration.
2. Any top-level function declaration is more specific than any functional method declaration. (This rule actually applies to declarations in a single component as well.)
3. Any top-level function declaration imported on demand is more specific than any other declaration not in the same API as the top-level function declaration.

It is also a static error to explicitly import a name from an API that does not declare it, or to have multiple import-on-demand statements with the same API.

If there is no imported declaration matching a reference, it is a static error. If there is more than one imported declaration that a reference may refer to, it is a static error. For example, it is a static error to have a reference List in the context of the following import statements:

```
import List.{...}
import PureList.{...}
```

A reference List may refer to the type List declared in the API List or the type List declared in the API PureList.

22.2.2 Export Statements

> DRAFT NOTE
> Victor's email on March 7, 2009 1:28:48 AM EST titled "Re: Exposing a partial type hierarchy in an API"
>
> Because the extends clause of a trait declaration in a component must be the same as in an API exported by the component, the type hierarchy above any trait can be determined simply by looking at the API declaring that trait, and those APIs it imports. Fortress does not allow components to contain private supertypes for traits in an exported API.

Export statements specify the APIs that a component exports. One important restriction on components is that no API may be both imported and exported by the same component. This restriction helps to avoid some (but not all) accidental cyclic dependencies.

Every component implicitly imports the Fortress core APIs; every fortress has at least one component implementing all of these APIs. A *preferred* component exporting these APIs (configurable by the user) is implicitly linked to every component installed in the fortress.

A component must provide a declaration, or a set of declarations, that *satisfies* every top-level declaration in any API that it exports, as described below. A component may include declarations that do not participate in satisfying any exported declaration (i.e., a declaration of any exported API), and in particular, no declaration modified by `private` participates in satisfying any exported declaration. Such declarations are not visible from outside the component.

Type aliases and dimension, unit, test and property declarations are satisfied only by declarations that are the same.[2]

> DRAFT NOTE
> Victor: what about trait/object/etc declarations with test modifier?

A top-level variable declaration declaring a single variable is satisfied by any top-level variable declaration that declares the name with the same type (in the component, the type may be inferred). A top-level variable declaration declaring multiple variables is satisfied by a set of declarations (possibly just one) that declare all the names with their respective types (which again, may be inferred). In either case, the modifiers including the mutability of a variable must be the same in the exported and satisfying declarations. A top-level variable declaration declaring a single immutable variable with arrow type $P \to R$ is also satisfied by a set of top-level function and functional method declarations if the union of their parameter types is P and each of their return types is a subtype of R. Thus, an abstract function declaration in the form of variable declarations in an API need not be replicated in a component exporting that API.

A top-level function declaration is satisfied by a set of top-level function and functional method declarations the union of whose parameter types is the parameter type of the exported declaration, and each of whose return types is a subtype of the return type of the exported declaration.

> DRAFT NOTE
> Victor: also need type of thrown checked exceptions

[2]The declarations are compared after ASCII conversion and scanning, and differences in the characters within whitespace elements is permitted provided they do not change whether the whitespace element is line-breaking.

Victor: it may also be satisfied by a declaration or set of declarations whose parameter type is a supertype of the exported declaration's parameter type provided that there is an overloaded exported declaration whose parameter type is a supertype of the satisfying declarations. (This comment also applied to the method declarations below. We need a term for this concept.)

A trait or object declaration is satisfied by a declaration that has the same header,[3] and contains, for each field declaration and non-abstract method declaration in the exported declaration, a satisfying declaration (or a set of declarations). When a trait has an abstract method declared, a satisfying trait declaration is allowed to provide a concrete declaration. If a trait declaration (in an API) has a `comprises` clause containing "...", then the `comprises` clause of its satisfying declaration must contain all the types listed in the declaration in the API but it cannot contain "..."; instead, it may contain other types declared in the component provided that those types are not declared by the API. A field declaration is satisfied by a (possibly implicit) getter and/or setter declaration, depending on whether the field is declared to be `hidden` and/or `settable` by the exported declaration.

A method declaration is satisfied by a set of method declarations the union of whose parameter types is the parameter type of the exported declaration, and each of whose return types is a subtype of the return type of the exported declaration.

DRAFT NOTE

Victor: need to be dotted iff exported decl is dotted.

Victor: also need type of thrown checked exceptions

A satisfying trait or object declaration may contain method and field declarations not exported by the API but these might not be overloaded with method or field declarations provided by (contained in or inherited by) any declarations exported by the API.

DRAFT NOTE

Victor: is this true even for getters and setters? That is, can we not define both a getter and a setter with the same name and export only one of them? I think we should be allowed to do this.

For functional declarations, recall that several functional declarations may define the same entity (i.e., they may be overloaded). Given a set of overloaded declarations, it is not permitted to export some of them and not others.

DRAFT NOTE

2. A trait with a comprises clause is considered to have a concrete method declaration with a particular signature even if its declaration is actually abstract provided that all its listed types are considered to have concrete method declaration with that signature. Thus, such a trait can have a method declaration that is abstract in the component, but not so declared in the API. (And we might also allow the analogous thing for top-level functions.)

3. A component may have a functional declaration that is not in the API provided that it is "covered" by some overloaded declaration.

4. A component exporting an API with an "abstract function declaration" need not have that declaration (or any matching one). It does, however, need matching declarations for the concrete functional declarations in that API.

22.2.3 Cross-Component Overloading

When a component is compiled, the APIs it refers to must be present in the fortress. The import statements in a component are not a way to abbreviate unqualified names of objects or functions. In our system, an import statement

[3] The order of the modifiers, the clauses, and the types in the `extends`, `excludes` and `comprises` clauses may differ.

merely allows references to the imported API to appear in the component definition. References to elements of an imported API must be fully qualified unless they are imported by an import statement with a `from` clause. When a component imports a functional f (either a top-level function or a functional method) by an import statement, with a `from` clause, the imported f may be overloaded with a functional f declared by the component. When a component imports a top-level declaration f from an API A, all the relevant types to type check the uses of f are implicitly imported from A. However, these implicitly imported types for type checking are not expressible by programmers; programmers must import the types by import statements to use them. For example, the two functional calls in the following component C are valid:

> DRAFT NOTE
> This example is not tested nor run by the interpreter.

```
api A
    trait T
      m() : ()
    end
end
api B
    import A.{...}
    f(): T
    g(T): ()
end
component C
    import B.{f, g}
    export Executable
    run(args) = do f().m(); g(f()) end
end
```

because T is implicitly imported from B to type check the functional calls. However, the programmers cannot write T in C because T is not imported by an import statement.

22.2.4 Native Components

A component may have the modifier `native` to declare an "unsafe component". Within an unsafe component, the syntax and semantics are implementation dependent. However, an unsafe component can export an API, which can be imported by safe components and APIs as usual.

22.3 APIs

> DRAFT NOTE
> All APIs should provide all types explicitly in all declarations.

Syntax:

Api ::= `api` *APIName Imports*? *AbsDecls*? `end` (`api` ? *APIName*)?
AbsDecls ::= *AbsDecl*$^+$
AbsDecl ::= *AbsTraitDecl*
| *AbsObjectDecl*
| *AbsVarDecl*
| *AbsFnDecl*
| *DimUnitDecl*
| *TypeAlias*
| *TestDecl*
| *PropertyDecl*
| *AbsExternalSyntax*

AbsTraitDecl ::= *AbsTraitMods*? *TraitHeaderFront AbsTraitClauses AbsGoInATrait*? `end` (`trait` ? *Id*)?
AbsTraitMods ::= *AbsTraitMod*$^+$
AbsTraitMod ::= `value` | `test`
AbsTraitClauses ::= *AbsTraitClause**
AbsTraitClause ::= *Excludes*
| *AbsComprises*
| *Where*
AbsComprises ::= `comprises` *ComprisingTypes*
ComprisingTypes ::= *TraitType*
| { *ComprisingTypeList* }
ComprisingTypeList ::= ...
| *TraitType*(, *TraitType*)* (, ...)?

AbsGoInATrait	::=	*AbsCoercions*? *AbsGoFrontInATrait AbsGoBackInATrait*?
	|	*AbsCoercions*? *AbsGoBackInATrait*
	|	*AbsCoercions*
AbsCoercions	::=	*AbsCoercion*$^+$
AbsGoFrontInATrait	::=	*AbsGoesFrontInATrait*$^+$
AbsGoesFrontInATrait	::=	*ApiFldDecl*
	|	*AbsGetterSetterDecl*
	|	*PropertyDecl*
AbsGoBackInATrait	::=	*AbsGoesBackInATrait*$^+$
AbsGoesBackInATrait	::=	*AbsMdDecl*
	|	*PropertyDecl*
AbsObjectDecl	::=	*AbsObjectMods*? *ObjectHeader AbsGoInAnObject*? `end (object` ? *Id*)?
AbsObjectMods	::=	*AbsTraitMods*
AbsGoInAnObject	::=	*AbsCoercions*? *AbsGoFrontInAnObject AbsGoBackInAnObject*?
	|	*AbsCoercions*? *AbsGoBackInAnObject*
	|	*AbsCoercions*
AbsGoFrontInAnObject	::=	*AbsGoesFrontInAnObject*$^+$
AbsGoesFrontInAnObject	::=	*ApiFldDecl*
	|	*AbsGetterSetterDecl*
	|	*PropertyDecl*
AbsGoBackInAnObject	::=	*AbsGoesBackInAnObject*$^+$
AbsGoesBackInAnObject	::=	*AbsMdDecl*
	|	*PropertyDecl*
AbsCoercion	::=	`coerce` *StaticParams*? (*BindId IsType*) *CoercionClauses* `widens` ?
ApiFldDecl	::=	*ApiFldMods*? *BindId IsType*
ApiFldMods	::=	*ApiFldMod*$^+$
ApiFldMod	::=	`hidden` | `settable` | `test`
AbsVarDecl	::=	*AbsVarMods*? *VarWTypes*
	|	*AbsVarMods*? *BindIdOrBindIdTuple* : *Type*...
	|	*AbsVarMods*? *BindIdOrBindIdTuple* : *TupleType*
AbsVarMods	::=	*AbsVarMod*$^+$
AbsVarMod	::=	`var` | `test`

APIs are compiled from special API definitions. These are source files which declare the entities declared by the API, the names of all APIs referred to by those declarations, and prose documentation. In short, the source code of an API must specify all the information that is traditionally provided for the published APIs of libraries in other languages.

The syntax of an API definition is identical to the syntax of a component definition, except that:

1. An API definition begins with `api` rather than `component`. As with components, the identifiers associated with APIs that are not included in the Fortress standard libraries are prefixed with the reverse of the URL of the development team.

2. An API does not include `export` statements. (However, it does include `import` statements, which name the other APIs used in the API definition.)

3. Only declarations (not definitions!) are included in an API definition except test and property declarations. A method or field declaration may include the modifier `abstract`. (Whether a declaration includes the modifier `abstract` has a significant effect on its meaning, as discussed below).

Import statements in APIs permit names declared in imported APIs to be used in the importing API either qualified or unqualified, just as in components. Those names are *not*, however, part of the importing API, and thus cannot be imported from that API by a component or another API.

For the sake of simplicity, every identifier reference in an API definition must refer either to a declaration in a used

API (i.e., an API named in an import statement, or the Fortress core APIs, which are implicitly imported), or to a declaration in the API itself. In this way, APIs differ from signatures in most module systems: they are not parametric in their external dependencies.

22.4 Component and API Identity

Formally, we introduce two functions on components, *imp* and *exp*, that return the imported and exported APIs of the component, respectively. For any component c, $\mathit{imp}(c) \cap \mathit{exp}(c) = \emptyset$. This restriction is required throughout to ground the semantics of operations on components, as discussed in Section 22.8.

Every component has a unique name, used for the purposes of component linking. This name includes a user-provided identifier. In the case of a simple component, the identifier is determined by a component name given at the top of the source file from which it is compiled. A build script may keep a tally on version numbers and append them to the first line of a component, incrementing its tally on each compilation. The name of a compound component is specified as an argument to the `link` operation (described in Section 22.8) that defines it. Component equivalence is determined nominally to allow mutually recursive linking of components. By programmer convention, identifiers associated with components that are not included in the Fortress standard libraries begin with the reverse of the URL of the development team. A fortress does not allow the installation of distinct components with the same name. Component names are used during `link` and `upgrade` operations to ensure that the restrictions on upgrades to a component are respected, as explained in Section 22.8.

Every component also includes a vendor name, the name of the fortress it is compiled on, and a timestamp, denoting the time of compilation. The time of compilation is measured by the compiling fortress, and the name of the fortress is provided by the fortress automatically. Every timestamp issued by a fortress must be unique. The vendor name typically remains the same throughout a significant portion of the life of a user account, and is best provided as a user environment variable.

Every API has a unique name that consists of a user-provided identifier. As with components, API equivalence is determined nominally. Every API also includes a vendor name, the name of the fortress it is compiled on, and a timestamp.

> DRAFT NOTE
> Victor: Is it the API name that includes the vendor, etc.? It used to just say API, and I changed it to API name.
>
> Jan: I changed it back, because we don't claim to distinguish APIs this way in the remainder of the text, and don't give a mechanism for specifying these details in an import.

Component names must not conflict with API names. For convenience, a compiler can also produce an API directly from a project with the same name as the component it is derived from. Such an API includes *matching* declarations of the component. All declarations in the component appear in the API.

A component must include, for every API A it exports, matching definitions for all the declarations in A. A matching definition of a declaration d is a definition d' with the same name as d that includes definitions for all declarations other than the methods or fields declared `abstract` in d. The header and type of d' must be the same as the header and type of d. d' may include additional definitions not declared in d.

Other than its identity, the only relevant characteristic of an API a is the set of APIs that it uses, denoted by $\mathit{uses}(a)$. Because an API a might expose types defined in $\mathit{uses}(a)$, we require that a component that exports a also exports all APIs in $\mathit{uses}(a)$ that it does not import. Formally, the following condition holds on the exported APIs of a component c:

$$a \in \mathit{exp}(c) \wedge a' \in \mathit{uses}(a) \implies a' \in \mathit{imp}(c) \cup \mathit{exp}(c)$$

22.5 Tests in Components and APIs

A component may include definitions of tests, as described in Chapter 19. These definitions are allowed to refer to both test and non-test code defined in the same component or declared in APIs imported by the component.

An API may also include definitions of tests. These definitions may refer to all declarations in the API as well as in any APIs it imports. Tests defined in APIs should be thought of as "executable documentation" that partially specifies the required behavior of the declared entities.

See Section 22.8 for an explanation of how tests defined in components and APIs are executed.

22.6 Type Inference for Components

Type inference for Fortress has been described as a procedure performed over a whole Fortress program in Chapter 20. In this section, we explain how this procedure can be adapted to perform type inference over a simple program component. For a compound component, concatenate all declarations of all constituents in the order specified by the constructing `link` operation. Constituent compound component declarations are recursively concatenated.

Type inference over a simple component C is performed by first expanding C into a self-contained Fortress program, as follows:

1. All program constructs corresponding to declarations in APIs exported by C are expanded so that they include all types and static parameters included in the exported APIs.
2. All types provided by all declarations in the APIs imported by C are prepended to C. Note that these declarations must include types for all variables, functions, fields, and methods; otherwise the APIs that declare them are not well-formed.
3. In order for the resulting expanded program to be well-formed, we assume that all declarations in these APIs are expanded into special definitions that include *empty bodies*. The empty body of such a definition is a conceptual body which cannot be expressed directly in Fortress programs. Because declarations in APIs do not have any elided type, type inference ignores empty bodies.

Once C is expanded, type inference is performed over all program constructs that still include elided types. Empty bodies are ignored.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.38
>
> The compiled type checker does not yet infer the type of a parameter that a function declaration leaves out, at top level, in a trait or object, or local to an expression block: it refuses the component with the message "Missing parameter type for" followed by the parameter's name. The inference this paragraph asks for is not yet described (Chapter 20).

22.7 Initialization Order for Components

To ensure that all objects and all variables are initialized before their use, execution of program components proceeds according to the procedure defined in this section. This procedure assumes that the program's type hierarchy is already checked to be acyclic.

If a component is a compound component, all constituent components are initialized nondeterministically, but before first use. If a simple component has imports, take the transitive closure of all imported APIs. Collect all declarations in this transitive closure, in any order, and prepend them to the component definition. Finally, for a simple component without imports, initialize all top-level variables and singleton object fields.

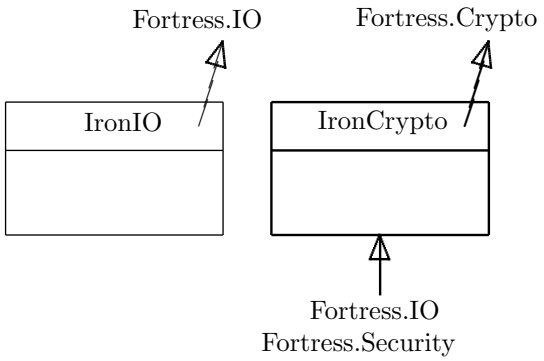


Figure 22.1: Simple components in box notation: A component is represented by a box, with the name of the component at the top of the box. The arrow protruding from the upper right corner of a box is labeled with the APIs exported by the component. The arrow pointing into the bottom of a box is labeled with APIs imported by the component. If no APIs are imported, we elide the arrow.

22.8 Basic Fortress Operations

We now describe the operations that can be performed on a fortress by developers and end-users for developing, installing, testing, and maintaining components. We can think of these operations as commands to an interactive shell provided by the fortress.

In this section, we discuss operations on a fortress in their most basic form, postponing the discussion of more advanced options, including additional optional parameters, to Section 22.9. Although these more advanced options are critical to performing some real-world tasks with components, it is easier to describe their behavior after the basic forms of operations have been discussed.

Compile This operation takes the source code for a simple component (or API) definition and produces a new component object (or API object) that is installed on the fortress. Its type is as follows:

```
compile(file:String):()
```

For example, suppose `IronCrypto.fss` contains the source code for the aforementioned IronCrypto application, which imports Fortress.IO and Fortress.Security, and exports Fortress.Crypto. Suppose we also have source code, `IronIO.fss`, for another application, IronIO, which imports nothing and exports Fortress.IO. We generate these components by compiling the source files:

```
compile("IronIO.fss")
compile("IronCrypto.fss")
```

The results are depicted diagrammatically in Figure 22.1.

Link A collection of one or more components exporting different APIs may be combined to form a new, compound, component by calling the `link` operation, passing the names of the components to link along with the name of the resulting compound component. Syntactically, a `link` operation is written as follows:[4]

```
link(result:String, constituents:String...):()
```

The components being linked are called *constituents* of the resulting component, which exports all the APIs exported by any of its constituents, and imports the APIs imported by at least one of its constituents but not exported by any of them.

[4]We present only the basic form of `link` here. `link` has additional optional arguments that we discuss in the Section 22.9.

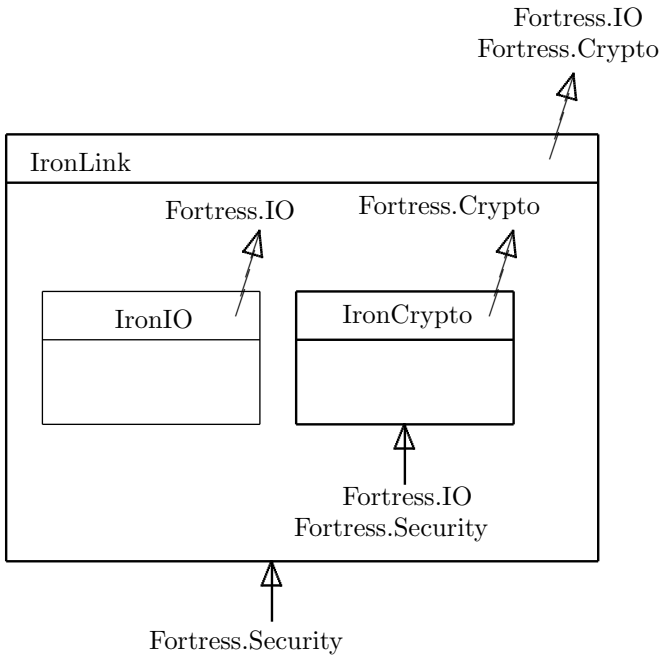


Figure 22.2: A compound component: A component inside another component is a constituent of the component that immediately encloses it.

For example, we can link the IronIO and IronCrypto libraries compiled above:

```
link(IronLink, IronIO, IronCrypto)
```

The resulting component, illustrated in Figure 22.2, imports Fortress.Security and exports Fortress.IO and Fortress.Crypto.

`link` does not distinguish between simple and compound components, so we can get arbitrarily nested components. For example, we can construct an application CoolCryptoApp by compiling another source code, `IronSecurity.fss`, for the library IronSecurity that imports Fortress.IO and exports Fortress.Security, and then linking the result with IronLink.

```
compile(IronSecurity.fss)
link(CoolCryptoApp, IronSecurity, IronLink)
```

The resulting components are illustrated in Figure 22.3.

Two components cannot be linked if they export the same API.[5] This restriction is made for the sake of simplicity; it allows programmers to link a set of components without having to specify explicitly which constituent exporting an API A provides the implementation exported by the linked component, and which constituent connects to the constituents that import A: only one component exports A, so there is only one choice. Although we lose expressiveness with this design, the user interface to link is vastly simplified, and it is rare that including multiple components that export a given API in a set of linked components is even desirable. We discuss how even such rare cases can be supported in Section 22.9.

> DRAFT NOTE
> THERE IS ANOTHER RESTRICTION ON LINKING, but it is trivially met if you can't hide APIs.

[5] There is one exception to this rule: the special API Upgradable, which is used during upgrades discussed below.

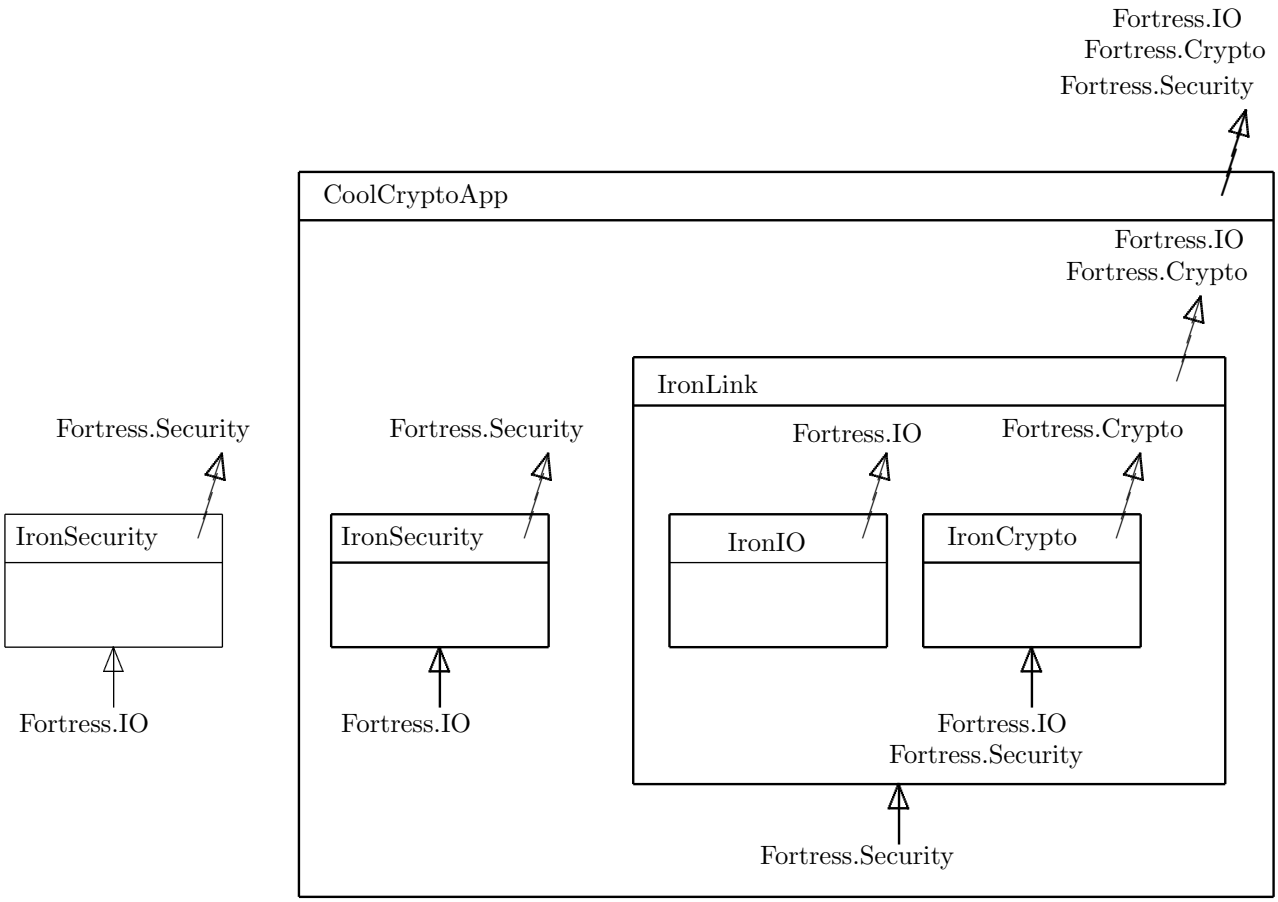


Figure 22.3: Repeated linking

For a compound component, in addition to the exported and imported APIs, we want to know what its constituents are. It is an invariant of the system that for any compound component C, any API imported by any of its constituents is either imported by C or exported by one of its constituents. This property is crucial for executing components, as we discuss below. A simple component (i.e., one produced directly by compilation) has no constituents.

Execute Components provide implementations of the APIs they export. A component is *executable* if it imports no APIs and it exports the special API Executable, defined as follows:

```
api Executable
run(args : String ...) : ()
end
```

An executable component may be *executed* by calling the `execute` operation, resulting in a call to the component's implementation of the *run* function in a new process. Arguments to the *run* function are passed to the shell:

```
execute(componentName:String, args:String...):()
```

We say that a component is being executed when `execute` has been called on that component and has not yet returned, or if it is the constituent component of a component being executed. During an execution, references may be made to APIs exported by a component being executed, which may in turn make references to APIs that it imports.

For references to an API A exported by the component, if the component is simple, then it contains the code necessary to evaluate any reference to an API it exports, possibly making references to APIs that it imports to do so. If the component is compound, then it contains a unique constituent that exports A; the reference is resolved to that constituent component.

For external references within a constituent component, recall that all such references in a component must be to APIs that the component imports. A component being executed either does not import any API (and thus there are no external references to resolve), or else is a constituent of another component that is being executed. In the latter case, the constituent defers the reference to its enclosing component.

For example, suppose CoolCryptoApp above is the constituent of some executable component, and when that component is executed, it generates a reference to SecretKey in Fortress.Crypto, which it resolves to CoolCryptoApp. CoolCryptoApp resolves this reference to IronLink, which resolves it to IronCrypto, which is a simple component. Suppose that in evaluating this reference, IronCrypto generates a reference to PublicKey in Fortress.Security. Because IronCrypto imports Fortress.Security, it resolves this reference to its enclosing component, IronLink, which in turn resolves it to CoolCryptoApp, which resolves it to IronSecurity, which is a simple component.

Not all projects are compiled to components that export Executable. For example, a library component does not usually export Executable.

Upgrade Compound components may be upgraded with new constituent components by calling an `upgrade` operation, passing the name of the component to upgrade (the *target*), the name of a component to upgrade with (the *replacement*), and a name for the resulting component (which we call the *result*). The type of the `upgrade` operation is as follows:

```
upgrade(target:String, replacement:String, result = target):()
```

If no result name is provided, the result is bound to the name of the target, and the target is uninstalled (see below).

For example, we can upgrade CoolCryptoApp with a component CoolSecurity, which exports Fortress.Security and imports nothing to CoolCryptoApp.2.0.

```
upgrade(CoolCryptoApp, CoolSecurity, CoolCryptoApp.2.0)
```

The resulting component is illustrated in Figure 22.4. Notice that the constituent, IronSecurity, exporting Fortress.Security has been replaced.

A component can be upgraded only if it exports the special API Upgradable, defined as follows:

```
api Upgradable
import Component from Components
isValidUpgrade(that : Component) : Boolean
upgrade(that : Component) : Component requires { isValidUpgrade(that) }
end
```

The Upgradable API imports a special API Components that provides handles on Component and Api objects. The Components API is described in Chapter 42.

An `upgrade` operation on a component invokes the *isValidUpgrade* function, as declared in the API Upgradable. This function must take a component and return *true* if and only if it is legal to upgrade with respect to that component. Developers can define their own versions of this component to restrict how their components can be upgraded. For example, they can prevent upgrades with older versions of a component, or with a matching component from an untrusted vendor.

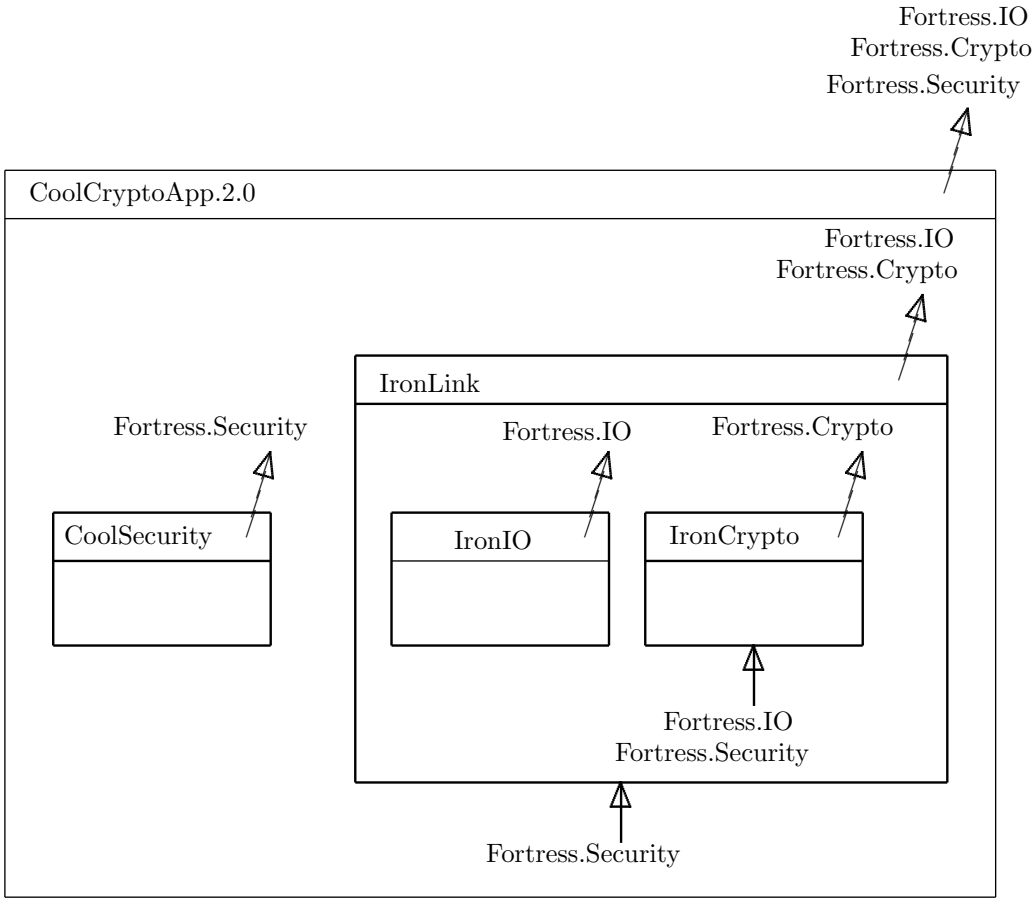


Figure 22.4: An upgraded component

The Upgradable API presents a problem for our model. Its implementation by the various constituent components in a compound component must be accessed during an `upgrade` operation. However, because the exported APIs of the constituent components must be disjoint, they cannot all export Upgradable after linking.

We solve this problem by introducing an additional step during linking. In a `link` operation, a special component, called a *restriction component*, is constructed automatically, based on the provided constituents. This component exports the Upgradable API; its implementation is a function of all the constituents provided to the `link` operation. The provided constituents are then used to construct a new set of constituents that are identical to the provided constituents except that they do not export Upgradable. These new constituents are then combined, along with the restriction component, to form the constituents of a new compound component.

In addition to the constraints imposed by a component's $isValidUpgrade$ function, there are several other conditions that must be met in order for an upgrade to be valid. These conditions are necessary to ensure that the resulting component is well-formed and imports and exports the same APIs as the target: [6]

1. Every API imported by the replacement must be either imported or exported by the target.
2. The APIs exported by the replacement must be a subset of those exported by the target.
3. If the replacement does not export all the APIs that a constituent exports then either the replacement and constituent do not export any APIs in common or the constituent can be upgraded with the replacement.

The rationale for the first two conditions is straightforward: If an API is imported by the replacement but not imported or exported by the target, then references to that API cannot be resolved in the result (unless we also import that API in the result). If an API is exported by the replacement but not the target, then the result will export an API not exported by the target.

The third condition says that the constituents of the target can be partitioned into three sets: those that are subsumed by the replacement, those that are unaffected by the upgrade, and all the rest, which can be upgraded with the replacement. This condition enables recursive propagation of upgrades. That is, an upgrade not only replaces constituents at the top level of the component, but is also propagated into any constituents with which it exports some APIs in common. Thus, in the example above, we could have upgraded CoolCryptoApp with a component that exports Fortress.IO. However, we could not have upgraded CoolCryptoApp with a component that exports both Fortress.Security and Fortress.IO because IronLink exports Fortress.IO but not Fortress.Security. In Section 22.9, we show how hiding and constraining APIs can help us get around many of the limitations that this condition imposes.

Recall that in our system, unlike with dynamic linking, components are encapsulated so that an upgrade to one component does not affect any other component on the system. We can imagine that all operations on components copy the components that they operate on rather than share them. Because components are immutable, these two interpretations are semantically indistinguishable. Convenience operations that support mass upgrades are provided on fortresses (e.g., an `upgradeAll` operation that takes a component and upgrades all components in the fortress that can be upgraded with its argument).

Extract and Install A component installed on a fortress may be *extracted* by calling an `extract` operation on the fortress, passing the name of the component as an argument, along with an argument `prereqs`, denoting the names of all APIs that must be installed on any fortress before this component can be installed.

```
extract(componentName:String, prereqs:Set[\String\] = {}):()
```

Furthermore, the destination fortress must have a component that exports these APIs and is a valid upgrade of the extracted component. Intuitively, a `prereqs` argument allows a component to be serialized without having to include all of its libraries; new libraries can be provided when the component is installed at a destination fortress.

The `prereqs` argument is optional; if omitted, the extracted component can be installed on any fortress. Any component can be extracted; however only compound components can be extracted with a `prereqs` argument: because

[6] These conditions are sufficient provided there are no hidden or constrained APIs, which are discussed in Section 22.9.

extracted components must be upgradable with respect to a component exporting their `prereqs`, no `prereqs` argument makes sense for a simple component.

The APIs included in a `prereqs` argument must be the APIs exported by some subset of the extracted component's constituents (or a subset of the constituents of one of its constituents, and so on, due to recursive updating).

The extracted component is serialized to a file, including all the APIs it refers to (and, transitively, all APIs they refer to) and all constituent components, except those that export the `prereqs`. This operation does not remove the extracted component from the fortress; there is a separate `uninstall` operation for that.

When the component is extracted, if no `prereqs` were passed to the `extract` operation, then the contents of the file can be deserialized by any fortress into the extracted component, which can be installed on the fortress. However, if `prereqs` were passed to `extract`, then the file must be deserialized into a component that exports only the Installable API:

```
api Installable
import Component from Components
reconstitute(candidate : Component) : Component
end
```

The deserialized component is immediately linked with preferred implementations of all of its imported APIs. (Preferred implementations of APIs are maintained in a table by a fortress, which maps each API to a list of components that implements it, in order of preference). Because the deserialized and linked component exports the Installable API, it has a *reconstitute* function that takes a *candidate* component, which exports the `prereqs` APIs, and checks whether the given component satisfies the *isValidUpgrade* condition of the extracted component. If so, it returns the extracted component upgraded with the given component. The *reconstitute* function is called by the fortress with a new component, formed by linking the preferred components for each API in the extracted components' `prereqs` argument.

Note that an extracted component with `prereqs` APIs is *not* the same as an extracted component that imports the same APIs but has no `prereqs` APIs. The latter can always be installed on a fortress, and then can be subsequently linked with any component that exports the imported APIs. In contrast, the fortress has no access to an extracted component with `prereqs` APIs unless it has a component that exports these APIs and satisfies the *isValidUpgrade* function of the extracted component. This difference provides a means for controlling access to the extracted component, for security, legal, or other reasons.

Syntactically, an `install` operation takes the name of a file constraining an extracted component. The `install` operation is overloaded with another operation that takes the name of a component to match `prereqs`. If this optional argument is provided, and the deserialized component exports the Installable API, then the *reconstitute* function is called with the component denoted by the optional argument of `install`, rather than the fortress' preferred implementation of the `prereqs` APIs. Install operations are written as follows:

```
install(file:String):()
install(file:String, prereqs:Set[\String\]):()
```

By default, a fortress adds a newly installed component to the head of the "preferred" list for every API it exports. However, this default may be overridden by the end-user; an end-user may modify the table or even map some APIs differently during a particular installation. If one or more of the APIs required by an extracted component is not mapped to an API on the destination fortress, an exception is thrown.

There is a corresponding operation for APIs, `installAPI`, that takes a serialization of a set of APIs and installs them into a fortress.

```
installAPI(file:String):()
```

This set of APIs must be closed under imports. If an API that is installed in this way is already installed on the fortress, the definitions must match exactly, or an exception is thrown.

Uninstall An `uninstall` operation takes the name of a component as an argument and removes the top-level binding of that component from a fortress. Note that the uninstalled component may have been linked to other components, or used as a replacement in an upgrade, and the result may still be installed; an `uninstall` operation will not affect these other components.

```
uninstall(file:String):()
```

There is a corresponding operation for APIs, `uninstallAPI`, that removes an API from a fortress.

```
uninstallAPI(file:String):()
```

Typically, this operation is used only to remove APIs that have been corrupted in some fashion.

Testing A component can be tested by calling the method $runTests$ on it:

$runTests(inclusive = true)\colon ()$

This method runs all test functions defined in the component. All test functions are run in parallel; each test function is run for each combination of test cases (bound in its generator list as described in Section 19.2) in parallel. In the case of a compound component, the set of defined test functions consists of all test functions defined by all constituent components and by all exported APIs. The set of test functions run can be limited by first hiding the tested component in a more restrictive API. The set of test functions can also be expanded by linking with a component defining additional test functions.

The $runTests$ method includes a keyword parameter $inclusive$ that defaults to $true$. If this parameter is set to $false$, only test functions defined in the APIs exported by the component are run.

22.9 Advanced Features of Fortress Operations

The system we have described thus far provides much of the desired functionality of a component system. However it has a few significant weaknesses:

1. It exposes to everyone all the APIs used in the development of a project.
2. By allowing access to these APIs, it inhibits significant cross-component optimization.
3. It prevents components that use two different implementations of the same API from being linked, even if they never actually pass references to that API between each other.
4. It restricts the upgradability of compound components, as described earlier.

We can mitigate all these shortcomings by providing two simple operations, `hide` and `constrain`. Informally, `hide` makes APIs no longer visible from outside the component so that they cannot be upgraded, and `constrain` merely prevents them from being exported. An API that is constrained but not hidden can still be upgraded. There are other subtle consequences of this distinction, which we discuss as they arise.

Some of the properties about the APIs exported by a component discussed in Section 22.8 are actually properties of APIs that are visible or provided by a component. For example, APIs visible in a component cannot be imported by that component, even if they are not exported. Other properties are really properties only of the exported APIs. Most importantly, components that do not export any common APIs can be linked, as can components that share only visible APIs.

Constrain A `constrain` operation takes a component name of an installed component, a new component name, and a set of APIs, and produces a new component that does not export any of the APIs specified. Syntactically, we write:

```
constrain(source:String, destination = source, apis:Set[\String\]):()
```

If no `destination` name is provided, the name of the `source` is used.

The set of APIs provided must be a subset of the APIs exported by the component. Also, recall that every API used by an API exported by a component must be imported or exported by that component. Thus, if we constrain an API that is used by any other API exported by the component, then we must also constrain that other API.

If the component is a simple component, we first link it by itself, and then apply `constrain` to the result.

Hide A `hide` operation is like a `constrain` operation, except that the given set of APIs is subtracted from the visible and provided APIs, along with the exported APIs, in the resulting component.

```
hide(source:String, destination = source, apis:Set[\String\]):()
```

The requirement of APIs being imported or exported whenever an API using them is exported also applies to visible APIs. Thus, if we hide an API used by another exported API, we must hide that other API as well.

Link With constrained APIs, there is a new restriction on link: Any API visible in one constituent and imported by another must be exported by some constituent. This restriction is necessary because an API visible in a component cannot be imported by that component. Thus, if one of the component's constituents imports that API, then the API must be provided by some other constituent. Other than that, the `link` operation is largely unchanged: the visible APIs are just all the APIs visible in any constituent, and the provided APIs are just those exported by any constituent. There is a subtle additional restriction on how linked components can be upgraded, which we discuss below.

Rather than requiring users and developers to call `constrain` and `hide` directly, we provide optional parameters to the `link` operation to do these operations immediately. The `link` operation has the following type:

```
link(result:String, constituents:String..., exports = {}, hide = {}):()
```

If the `exports` clause is present, only those APIs listed in the set following `exports` are exported; the others are constrained. If the `hide` clause is present, those APIs listed in the set following `hide` are hidden. An exception is thrown if the `exports` clause contains any API not exported by any constituent, or if the `hide` clause contains any API not visible in any constituent.

Hiding enables us to handle the rare case in which programmers want to link multiple components that implement the same API without upgrading them to use the same implementation. Before linking, the programmer simply hides (or constrains) the API in every component that exports it except the one that should provide the implementation for the new compound component.

For example, suppose we wish to link the following two components:

- A component NetApp that imports Fortress.IO and exports the Fortress.Net API.
- A component EditApp that imports Fortress.IO and exports the Fortress.Swing.Textrf API.

We want to link these two components to use in building an application for editing messages and sending them over a network. But we want to use different implementations of Fortress.IO (e.g., IOApp1 and IOApp2 for the two components). We simply perform the following operations:

```
link(temp1, NetApp, IOApp1, exports = {Fortress.Net}, hide = {Fortress.IO})
link(temp2, EditApp,IOApp2, exports = {Fortress.Swing.Textrf}, hide = {Fortress.IO})
link(NetEdit, temp1, temp2)
```

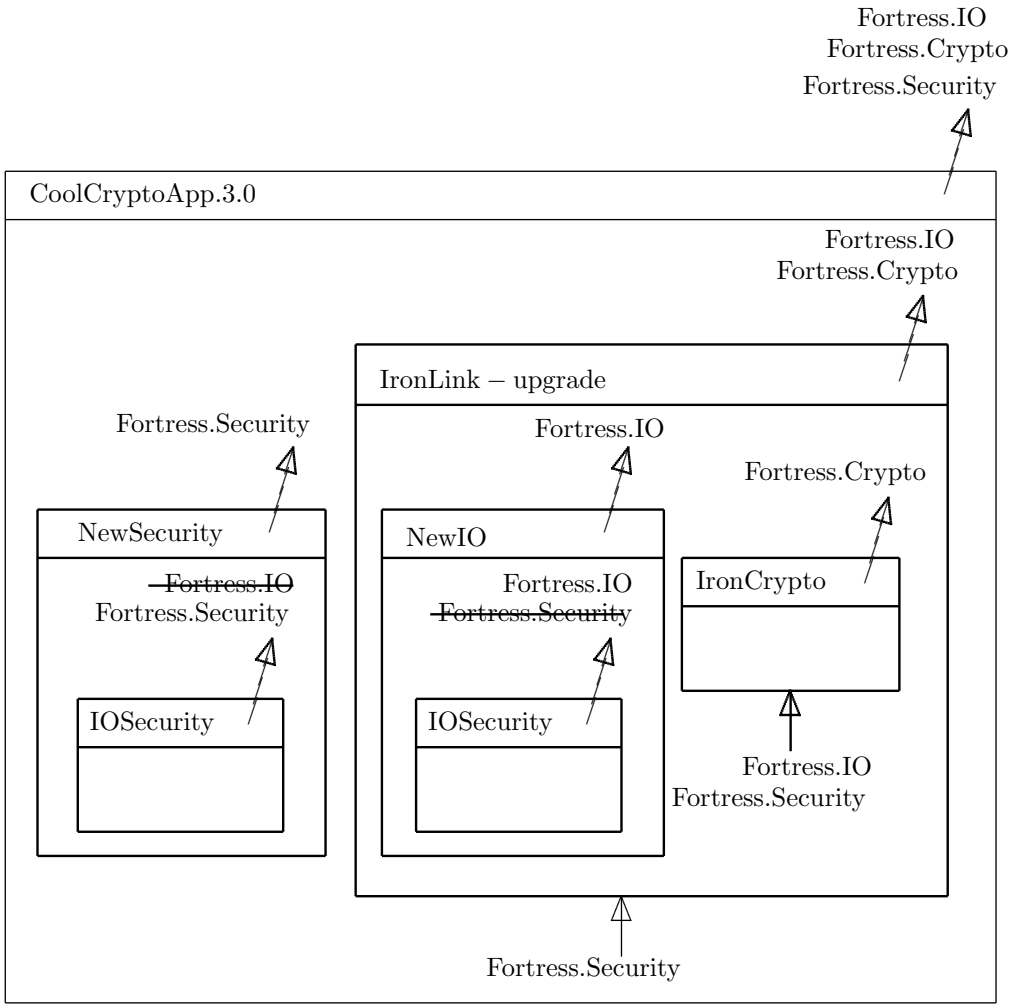


Figure 22.5: Upgrading with hidden APIs: Crossed out APIs are hidden.

In this case, the NetEdit component does not export, or even make visible, Fortress.IO at all.

Upgrade For the `upgrade` operation, there is no change at all in the semantics. However, because hiding and constraining APIs allow us to change the APIs exported by a component, it is possible to do some upgrades that are not possible without these operations.

For example, suppose we have a component IOSecurity that exports Fortress.IO and Fortress.Security, and we want to upgrade CoolCryptoApp with IOSecurity. As discussed above, we cannot use IOSecurity directly because IronLink exports Fortress.IO but not Fortress.Security. We can get around this restriction by doing two upgrades, one with Fortress.Security hidden and the other with Fortress.IO hidden.

```
hide(IOSecurity, NewIO, Fortress.Security)
hide(IOSecurity, NewSecurity, Fortress.IO)
upgrade(CoolCryptoApp, NewSecurity, temp1)
upgrade(CoolCryptoApp.3.0, temp1, NewIO)
```

The resulting component is shown in Figure 22.5.

The interplay between imported, exported, visible and provided APIs introduces subtleties that not present in our

discussion above. In particular, the last of the three conditions imposed for well-formedness of upgrades is modified to state that for any constituent that is not subsumed by a replacement component, either it can be upgraded with the replacement, or its *visible* APIs are disjoint from the APIs exported by the replacement (i.e., it is unaffected by the upgrade). To maintain the invariant that no two constituents export the same API, we need another condition, which was implied by the previous condition when no APIs were constrained or hidden: if the replacement subsumes any constituents of the target, then its exported APIs must exactly match the exported APIs of some subset of the constituents of the target. In practice, this restriction is rarely a problem; in most cases, a user wishes to upgrade a target with a new version of a single constituent component, where the APIs exported by the old and new versions are either an exact match, or there are new APIs introduced by the new component that have no implementation in the target.

Part III

Fortress for Library Writers

Chapter 23

Parallelism and Locality

> DRAFT NOTE
> Distributions are not yet supported.

Fortress is designed to make parallel programming as simple and as painless as possible. This chapter describes the internals of Fortress parallelism designed for use by authors of library code (such as *distributions*, generators, and arrays). We adopt a multi-tiered approach to parallelism:

- At the highest level, we provide libraries that allocate locality-aware distributed shared arrays (Section 23.4) and implicitly parallel constructs such as tuples and loops. Synchronization is accomplished through the use of atomic sections (Section 13.23). More complex synchronization makes use of abortable atomicity, described in Section 23.5.
- There is an extensive library of distributions, which permits the programmer to specify locality and data distribution explicitly (Section 23.6).
- Immediately below that, the `at` expression requests that a computation take place in a particular region of the machine (Section 23.2). We also provide a mechanism to terminate a spawned thread early (Section 23.7).
- Finally, there are mechanisms for constructing new generators via recursive subdivision into tree structures with individual elements at the leaves. Section 23.8 explains how iterative constructs such as `for` loops and comprehensions are desugared into calls to methods of trait Generator, and how new instances of this trait may be defined.

We begin by describing the abstraction of *regions*, which Fortress uses to describe the machine on which a program is run.

23.1 Regions

Every thread (either explicit or implicit) and every object in Fortress, and every element of a Fortress array (the physical storage for that array element), has an associated *region*. The Fortress libraries provide a function *region* which returns the region in which a given object resides. Regions abstractly describe the structure of the machine on which a Fortress program is running. They are organized hierarchically to form a tree, the *region hierarchy*, reflecting in an abstract way the degree of locality which those regions share. The distinguished region Global represents the root of the region hierarchy.[1] The different levels of this tree reflect underlying machine structure, such as execution

[1] Note: the initial implementation of the Fortress language assumes a single machine with shared memory and exposes only the Global region.

engines within a CPU, memory shared by a group of processors, or resources distributed across the entire machine. The function $here()$ returns the region in which execution is currently occurring. Objects which reside in regions near the leaves of the tree are local entities; those which reside at higher levels of the region tree are logically spread out. The method call $r.isLocalTo(s)$ returns $true$ if r is contained within the region tree rooted at s.

It is important to understand that regions and the structures (such as distributions, Section 23.6) built on top of them exist purely for performance purposes. The placement of a thread or an object does not have any semantic effect on the meaning of a program; it is simply an aid to enable the implementation to make informed decisions about data placement.

It might not be possible for an object or a thread to reside in a given region. Threads of execution reside at the *execution level* of the region hierarchy, generally the bottommost level in the region tree. Each thread is generally associated with some region at the execution level, indicating where it will preferentially be run. The programmer can affect the choice of region by using an `at` expression (Section 23.2) when the thread is created. A thread may have an associated region which is not at the execution level of the region hierarchy, either because a higher region was requested with an `at` expression or because scheduling decisions permit the thread to run in several possible execution regions. The region to which a thread is assigned may also change over time due to scheduling decisions. For the object associated with a spawned thread, the $region$ function provided by the Fortress libraries returns the region of the associated thread.

The *memory level* of the region hierarchy is where individual reference objects reside; on a machine with nodes composed of multiple processor cores sharing a single memory, this generally will not be a leaf of the region hierarchy. Imagine a constructor for a reference object is called by a thread residing in region r, yielding an object o. Except in very rare circumstances (for example when a local node is out of memory) either $r.isLocalTo(region(o))$ or $region(o).isLocalTo(r)$ ought to hold: data is allocated locally to the thread which runs the constructor. For a value object v being manipulated by a thread residing in region r either $region(v).isLocalTo(r)$ or $r.isLocalTo(region(v))$ (value objects always appear to be local).

Note that $region$ is a top-level function provided by the Fortress libraries and can be overridden by any functional method. The chief example of this is arrays, which are generally composed from many reference objects; the $region$ function can be overridden to return the location of the array as a whole—the region which contains all of its constituent reference objects.

23.2 Placing Threads

A thread can be placed in a particular region by using an `at` expression:

$(v, w) := (a_i,$

`at` $a.region(j)$ `do`

a_j

`end`$)$

In this example, two implicit threads are created; the first computes a_i locally, the second computes a_j in the region where the jth element of a resides, specified by $a.region(j)$. The expression after `at` must return a value of type Region, and the block immediately following `do` is run in that region; the result of the block is the result of the `at` expression as a whole. Often it is more graceful to use the `also do` construct (described inSection 13.11.2) in these cases:

`do`

$v := a_i$

`also at` $a.region(j)$ `do`

$w := a_j$

`end`

We can also use `at` with a `spawn` expression:

$s =$ `spawn` `at` $a.region(i)$ `do`

a_i

`end`

$t =$ `spawn` `at` $region(s)$ `do`

a_j

`end`

Finally, note that it is possible to use an `at` expression within a block:

`do`

$v := a_i$

`at` $a.region(j)$ `do`

$w := a_j$

`end`

$x = v + w$

`end`

We can think of this as the degenerate case of `also do`: a thread group is created with a single implicit thread running the contents of the `at` expression in the given region; when this thread completes control returns to the original location.

Note that the regions given in an `at` expression are non-binding: the Fortress implementation may choose to run the computations elsewhere—for example, thread migration might not be possible within an `atomic` expression, or load balancing might cause code to be executed in a different region. In general, however, implementations should attempt to respect thread placement annotations when they are given.

23.3 Shared and Local Data

> DRAFT NOTE
> The method *copy* is not yet supported.

Every object in a Fortress program is considered to be either *shared* or *local* (collectively referred to as the *sharedness* of the object). A local object must be transitively reachable (through zero or more object references) from the variables of at most one running thread. A local object may be accessed more cheaply than a shared object, particularly in the case of atomic reads and writes. Sharedness is ordinarily managed implicitly by the Fortress implementation. Control over sharedness is intended to be a performance optimization; however, functions such as *isShared* and *localize* provided by the Fortress libraries can affect program semantics, and must be used with care.

The sharedness of an object must be contrasted with its region. The region of an object describes where that object is located on the machine. The sharedness of an object describes whether the object is visible to one thread or to many. A local object need not actually reside in a region near the thread to which it is visible (though ordinarily it will).

The following rules govern sharedness:

- Reference objects are initially local when they are constructed.
- The sharedness of an object may change as the program executes.[2]
- If an object is currently transitively reachable from more than one running thread, it must be shared.
- When a reference to a local object is stored into a field of a shared object, the local object must be *published*: Its sharedness is changed to shared, and all of the data to which it refers is also published.
- The value of a local variable referenced by a thread must be published before that thread may be run in parallel with the thread which created it. Values assigned to the variable while the threads run in parallel must also be published.
- A field with value type is assigned by copying, and thus has the sharedness of the containing object or closure.

Publishing can be expensive, particularly if the structure being broadcast is large and heavily nested; this can cause an apparently short `atomic` expression (a single write, say) to run arbitrarily long. To avoid this, the library programmer can request that an object be published by calling the semantically transparent function $shared$ provided by the Fortress libraries:

$y := shared\,\mathrm{Cons}(x, xs)$

$shared(y)$

A local copy of an object can be obtained by calling $copy$, a method on trait Any:

> DRAFT NOTE
> Example is commented out because it is not yet supported nor run by the interpreter.

Two additional functions are provided which permit different choices of program behavior based on the sharedness of objects:

- The function $isShared(o)$ returns $true$ when o is shared, and $false$ when it is local. This permits the program to take different actions based on sharedness.
- The function $localize(o)$ is provided that attempts to make a local version of object o, by copying if necessary equivalent to the following expression:

  > DRAFT NOTE
  > Example is commented out because it is not yet supported nor run by the interpreter.

These functions must be used with extreme caution. For example, $localize$ should be used only when there is a unique reference to the object being localized. The $localize$ function can have unexpected behavior if there is a reference to o from another local object p. Updates to o will be visible through p; subsequent publication of p will publish o. By contrast, if o was already shared, and referred to by another shared object, the newly-localized copy will be entirely distinct; changes to the copy will not be visible through p, and publishing p will not affect the locality of the copy.

[2]Note, for example, that the present Fortress implementation immediately makes every object shared after construction, so that $isShared()$ will always return $true$.

$array[\![E]\!](size : I) : \text{Array}[\![E, I]\!]$	Creates an uninitialized 0-indexed array of the specified runtime-determined size. The size is an integer for a 1-dimensional array, a 2-tuple for a two-dimensional array, and so forth.
$array_1[\![E, n]\!]()$	Creates an uninitialized 0-indexed 1-dimensional array of statically determined size n.
$array_2[\![E, n, m]\!]()$	Creates an uninitialized 0-indexed 2-dimensional array of statically determined size n by m.
$array_3[\![E, n, m, p]\!]()$	Creates an uninitialized 0-indexed 3-dimensional array of statically determined size n by m by p.
$a.fill(v : E)$	Initializes all elements with value v.
$a.tabulate(f : I \to E)$	Calls f at each index and initializes the corresponding element with the result of the call.
$a.init(i : I, v : E)$	Initializes element at index i with value v.

Figure 23.1: Factories for creating arrays and methods for initializing their elements

23.4 Distributed Arrays

> DRAFT NOTE
> Distributions, array comprehensions, and HeapSequence are not yet supported.

Arrays, vectors, and matrices in Fortress are assumed to be spread out across the machine. As in Fortran, Fortress arrays are complex data structures; simple linear storage is encapsulated by the HeapSequence type, which is used in the implementation of arrays (see Section 23.2). The default distribution of an array is determined by the Fortress libraries; in general it depends on the size of the array, and on the size and locality characteristics of the machine running the program. For advanced users, the distribution library (introduced in Section 23.6) provides a way of combining and pivoting distributions, or of redistributing two arrays so that their distributions match. Programmers must create arrays by using an array comprehension (Section 13.30) or an aggregate expression (Section 13.29), or by using one of the factory functions at the top of Figure 23.1. After calling any of these factories, the elements of the resulting array must be initialized using either indexed assignment or using one of the initialization methods described at the bottom of the figure.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.18
> The Working Draft of February 2011 gave the method of Figure 23.1 that calls f at each index the name *fill*, which is also the name of the method above it that initializes every element with one value. When the element type E is Any or a function type, a function is a value of type E as well as a function from I to E, and the two declarations are ambiguous; the method that calls f is named *tabulate*, and *fill* keeps the value.

Each array element may be initialized at most once using the methods of Figure 23.1; the programmer must assure that this initialization completes before any other access to the corresponding array element. The programmer must also assure that an element is initialized before it is first read.[3] Note that these factories and methods are intended to be used to library programmers to build higher-level functionality (for example, the *fill* methods themselves are defined in terms of calls to *init*, and the *array* factory is defined in terms of $array_1$, $array_2$, and so forth).

Because the elements of a fortress array may reside in multiple regions of the machine, there is an additional method $a.region(i)$ which returns the region in which the array element a_i resides. An element of an array is always local to the region in which the array as a whole is contained, so $(a.region(i)).isLocalTo(region(a))$ must always return *true*. When an array contains reference objects, the programmer must be careful to distinguish the region in which the array element a_i resides, $a.region(i)$, from the region in which the object referred to by the array element resides,

[3] The present implementation signals a fatal error in case of duplicate initialization, or if an uninitialized element is read.

$region(a_i)$. The former describes the region of the array itself; the latter describes the region of the data referred to by the array. These may differ.

23.5 Abortable Atomicity

Fortress provides a user-level $abort()$ function which abandons execution of the innermost `atomic` expression and rolls back its changes, requiring the `atomic` expression to execute again from the beginning. This permits an atomic section to perform consistency checks as it runs. Invoking the $abort$ function not within an `atomic` expression has no effect. The functionality provided by $abort()$ can be abused; it is possible to induce deadlock or livelock by creating an atomic section that always fails. Here is a simple example of a program using $abort()$ which is incorrect because Fortress does not guarantee that the two implicit threads (created by evaluating the two elements of the tuple) will always run in parallel; it is possible for the first element of the tuple to continually abort without ever running the second element of the tuple:

> DRAFT NOTE
> The example is commented out because it is not yet supported nor run by the interpreter.

Fortress also includes a `tryatomic` expression, which attempts to run its body expression atomically. If it succeeds, the result is returned; if it aborts due to a call to $abort$ or due to conflict (as described in Section 13.23), the checked exception TryAtomicFailure is thrown. In addition, `tryatomic` is permitted to throw TryAtomicFailure in the absence of conflict; however, it is not permitted to fail unless some other thread performs an access to shared state while the body is being run. Conceptually `atomic` can be defined in terms of `tryatomic` as follows:

```
label AtomicBlock
  while true do
    try
      result = tryatomic body
      exit AtomicBlock with result
    catch e
      TryAtomicFailure ⇒ () (* continue execution *)
    end
  end
  throw UnreachableCode      (* inserted for type correctness *)
end AtomicBlock
```

Unlike the above definition, an implementation may choose to suspend a thread running an `atomic` expression which invokes $abort$, re-starting it at a later time when it may be possible to make further progress. The above definition restarts the body of the `atomic` expression immediately without suspending.

23.6 Distributions

> DRAFT NOTE
> Distributions are not yet supported. Examples in this section are not tested.

Most of the heavy lifting in mapping threads and arrays to regions is performed by *distributions*. An instance of the trait Distribution describes the parallel structure of ranges and other numeric generators (such as the generators for the index space of an array), and provides for the allocation and distribution of arrays on the machine:

```
trait Distribution
  distribute⟦E, B extends ArrayIndex⟧(Range⟦B⟧) : Range⟦B⟧
  distribute⟦E, B extends ArrayIndex⟧(a : Array⟦E, B⟧) : Array⟦E, B⟧ =
    distributeFromTo⟦E, B⟧(a, a.distribution, self)
end
```

Abstractly, a Distribution acts as a transducer for generators and arrays. The *distribute* method applied to a multidimensional Range organizes its indices into the leaves of a tree whose inner nodes correspond to potential levels of parallelism and locality in the underlying computation, producing a fresh Range whose behavior as a Generator may differ from that of the passed-in Range. The *distribute* method applied to an array creates a copy of that array distributed according to the given distribution. This is specified in terms of a call to the overloaded function *distributeFromTo*. This permits the definition of specialized versions of this function for particular pairs of distributions.

The intention of distributions is to separate the task of data distribution and program correctness. That is, it should be possible to write and debug a perfectly acceptable parallel program using only the default data distribution provided by the system. Imposing a distribution on particular computations, or designing and implementing distributions from scratch, is a task best left for performance tuning, and one which should not affect the correctness of a working program.

There is a DefaultDistribution which is defined by the underlying system. This distribution is designed to be reasonably adaptable to different system scales and architectures, at the cost of some runtime efficiency. Arrays and generators that are not explicitly allocated through a distribution are given the DefaultDistribution.

There is a generator, *indices*, associated with every array. This generator is distributed in the same way as the array itself. When we re-distribute an array, we also re-distribute the generator; thus $d.distribute(a.indices)$ is equivalent to $(d.distribute(a)).indices$.

There are a number of built-in distributions:

DefaultDistribution	Name for distribution chosen by system.
Sequential	Sequential distribution. Arrays are allocated in one contiguous piece of memory.
Local	Equivalent to Sequential.
Par	Blocked into chunks of size 1.
Blocked	Blocked into roughly equal chunks.
Blocked(n)	Blocked into n roughly equal chunks.
Subdivided	Chopped into 2^k-sized chunks, recursively.
Interleaved($d_1, d_2, \ldots d_n$)	The first n dimensions are distributed according to $d_1 \ldots d_n$, with subdivision alternating among dimensions.
Joined($d_1, d_2, \ldots d_n$)	The first n dimensions are distributed according to $d_1 \ldots d_n$, subdividing completely in each dimension before proceeding to the next.

From these, a number of composed distributions are provided:

Morton	Bit-interleaved Morton order [23], recursive subdivision in all dimensions.
Blocked($x_1, x_2, \ldots x_n$)	Blocked in n dimensions into chunks of size x_i in dimension i; remaining dimensions (if any) are local.

To allocate an array which is local to a single thread (and most likely allocated in contiguous storage), the Local distribution can be used:

$a = \text{Local}.distribute[\,1\ 0\ 0; 0\ 1\ 0; 0\ 0\ 1\,]$

Other distributions can be requested in a similar way.

Distributions can be constructed and given names:

$spatialDist = \text{Blocked}(n, n, 1)(*$ Pencils along the z axis $*)$

The system will lay out arrays with the same distribution in the same way in memory (as much as this is feasible), and will run loops with the same distribution in the same way (as much as this is feasible). By contrast, if we replace every occurrence of $spatialDist$ by $\text{Blocked}(n, n, 1)$, this code will likely divide up arrays and ranges into the same-sized pieces as above, but these pieces need not be collocated.

23.7 Early Termination of Threads

As noted in Section 5.4, an implicit thread can be terminated if its group is going to throw an exception. Similarly, a spawned thread t may be terminated by calling $t.stop()$. A successful attempt to terminate a thread causes the thread to complete asynchronously. There is no guarantee that termination attempts will be prompt, or that they will occur at all; the implementation will make its best effort. If a thread completes normally or exceptionally before an attempt to terminate it succeeds, the result is retained and the termination attempt is simply dropped.

At present stopping a thread immediately causes it to cease execution; no outstanding `finally` blocks are run and the thread is not considered to return a result.

> DRAFT NOTE
> From here till the end of this section, copied from F1.0β.

A termination attempt acts as if a special hidden *stop exception* is thrown in that thread. This exception cannot be thrown by `throw` or caught by `catch`; however, `finally` clauses are run as with any other exception. If the stopped thread was in the middle of an `atomic` expression, the effects of that expression are rolled back just as with an ordinary `throw`. A special wrapper around every spawned thread is provided by the Fortress implementation; it catches the stop exception and transforms it into a deferred Stopped exception. This is visible to the programmer and should be caught by invoking the val method on the thread object. Implicit threads are terminated only if another thread in the group completes abruptly, and the threads that are terminated are ignored for the purposes of the completion of the group.

Typical code for stopping a thread looks something like the following example:

```
x : ℤ64 := 0
t = spawn do
          try
            atomic if x = 0 then abort() else () end
          finally
            x := 1
          end
    end
t.stop()
try
  t.val()
catch s
  Stopped ⇒ x += 2; x
end
```

Here the spawned thread t blocks until it is killed by the call to $t.stop()$; it sets x to 1 in the `finally` clause before exiting. In this case, the call to $t.val()$ will throw $\mathrm{Stopped}$, which is caught, causing 2 to be added to x and returning 3.

Note that there is a race in the above code, so the `try` block in t may not have been entered when $t.stop()$ is called, causing x to be 2 at the end of execution. Note also that the call to $t.stop()$ occurs asynchronously; in the absence of the call to $t.val()$, the spawning thread would not have waited for t to complete.

23.8 Use and Definition of Generators

Several expressions in Fortress make use of *generator clause lists* to express parallel iteration (see Section 13.14). A generator clause list binds a series of variables to the values produced by a series of objects that extend the trait $\mathrm{Generator}$. A generator clause list is simply syntactic sugar for a nested series of invocations of methods on these objects.

A type that extends $\mathrm{Generator}[\![E]\!]$ acts as a generator of elements of type E. An instance of $\mathrm{Generator}[\![E]\!]$ only needs to define the $generate$ method:

$$generate[\![R]\!](r : \mathrm{Reduction}[\![R]\!], body : E \to R) : R$$

The mechanics of object generation are embedded entirely in the $generate$ method. This method takes two arguments. The $generate$ method invokes $body$ once for each object which is to be generated, passing the generated object as an argument. Each call to $body$ returns a result of type R; these results are combined using the reduction r, which encapsulates a monoidal operator on objects of type R. *All the parallelism provided by a particular generator is specified by definition of the* $generate$ *method.*

In practice, calls to $generate$ are produced by desugaring expressions with generator clause lists, as described below (Section 23.8.1). However it is possible to call the $generate$ method directly, as in the following example:

`object` SumZZ32 `extends` CommutativeMonoidReduction$[\![\mathbb{Z}32]\!]$

$asString() : \mathrm{String} =$ "`SumZZ32`"

$empty() : \mathbb{Z}32 = 0$

$join(a : \mathbb{Z}32, b : \mathbb{Z}32) : \mathbb{Z}32 = a + b$

`end`

$z = (1 \# 100).generate[\![\mathbb{Z}32]\!]\big(\mathrm{SumZZ32}, \texttt{fn}\ (x) \Rightarrow 3\,x + 2\big)$

Any reduction must define two methods: an associative binary operation $join$, and $empty$, a method that returns the identity of this operation. Here we define reduction $\mathrm{SumZZ32}$ representing integer addition. We use this to compute the sum of $3x + 2$ for x drawn from the range $1 \# 100$, yielding the expected answer of 15350.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.20
>
> The Working Draft of February 2011 defined the generator of Figure 23.2 over $\mathbb{Z}64$, and its method seq built a range over $\mathbb{Z}64$, which Section 13.16 no longer admits; the generator is defined over $\mathbb{Z}32$. Its midpoint was $\lfloor (lo + hi)/2 \rfloor$, a value of type $\mathbb{Z}$, which converts to neither fixed-size type; it is $lo + (hi - lo) \div 2$.

For non-commutative reductions such as the Concat reduction used for list comprehensions in Figure 23.4, it is important to note that results must be combined in the natural order of the generator. If $join$ is not associative, or $empty$ is not the identity of $join$, passing the reduction to $generate$ will produce unpredictable results. A generator

```
object BlockedRange(lo: ℤ32, hi: ℤ32, b: ℤ32) extends Generator⟦ℤ32⟧
  size : ℤ32 = hi − lo + 1
  seq(self): SequentialGenerator⟦ℤ32⟧ = seq(lo : hi)
  generate⟦R⟧(reduction: Reduction⟦R⟧, body: ℤ32 → R): R =
    if size ≤ (b MAX 1) then
      (⋆ Blocks smaller than b run sequentially ⋆)
      r : R := reduction.empty()
      i : ℤ32 := lo
      while i ≤ hi do
        v : R = body(i)
        r := reduction.join(r, v)
        i += 1
      end
      r
    else
      (⋆ Blocks larger than b are split in half and generated in parallel. ⋆)
      mid = lo + (hi − lo) ÷ 2
      reduction.join(BlockedRange(lo, mid, b).generate⟦R⟧(reduction, body),
                     BlockedRange(mid + 1, hi, b).generate⟦R⟧(reduction, body))
    end
end
f() = ⟨ 2 x | x ← BlockedRange(1, 10, 3) ⟩
```

Figure 23.2: Sample Generator definition: blocked integer ranges.

$$
\begin{array}{lll}
\mathcal{C}[\ ] & \mathit{body} & = u(\mathit{body}) \\
\mathcal{C}[\, x \leftarrow g, gs\,] & \mathit{body} & = g.\mathit{generate}(r, \texttt{fn}(x) \Rightarrow \mathcal{C}[gs]\ \mathit{body}) \\
\mathcal{C}[\, p, gs\,] & \mathit{body} & = p.\mathit{generate}(r, \texttt{fn}() \Rightarrow \mathcal{C}[gs]\ \mathit{body})
\end{array}
$$

Figure 23.3: Simple syntax-directed desugaring of a generator clause list. Here the reduction r and unit u are variables chosen by the desugarer to be distinct from the variables in gs and *body*.

is permitted to group reduction operations in any way it likes consistent with its natural order, and insert an arbitrary number of empty elements.

Figure 23.2 defines a generator that generates the integers between lo and hi in sequential blocks of size at most b. In this example, we divide the range in half if it is larger than the block size b; these two halves are computed in parallel (recall that the arguments to the method call $\mathit{reduction.join}$ are evaluated in parallel). If the range is smaller than b, then it is enumerated serially using a `while` loop, accumulating the result r as it goes. Observe that the parallelism obtained from a generator is *dictated by the code of the* $\mathit{generate}$ *method*. While programmers using a generator should assume that calls to body may occur in parallel, the library programmer is free to choose the amount of parallelism that is actually exposed.

This example uses *recursive subdivision* to divide a blocked range into approximately equal-sized chunks of work. Recursive subdivision is the recommended technique for exposing large amounts of parallelism in a Fortress program because it adjusts easily to varying machine sizes and can be dynamically load balanced.

Generator defines the functional method $\mathit{seq}(\texttt{self})$ that returns an equivalent generator that runs iterations of the body sequentially in natural order. In most cases (such as in this example), it is prudent to override the default definition of this method; the default implementation of seq effectively collects the generated elements together in parallel and traverses the result sequentially.

> DRAFT NOTE
>
> Note that at the moment there is no way to tell the compiler for performance reasons that we really mean it when we ask for sequentiality, as opposed to saying that we should preserve sequential semantics. Future versions of this specification may use the Local distribution for this purpose, or provide additional functions on generators which guarantee serial execution (rather than simply providing sequential semantics).

The remainder of this section describes in detail the desugaring of expressions with generator clause lists into invocations of the $\mathit{generate}$ methods of the generators in the generator clause list.

> DRAFT NOTE
>
> It then outlines how method overloading may be used to specialize the behavior of particular combinations of generators and reductions.

23.8.1 Simple Desugaring of Expressions with Generators

> DRAFT NOTE
>
> NOTE: tweaked by hand by Jan, as above.

An expression with a generator clause list gs and body expression *body* is desugared into the following general form:

> DRAFT NOTE
>
> NOTE: tweaked by hand by Jan. Replaced BIG OP by $\mathcal{C}$, emph'd gs and body.

expr	type	*wrapper*	$u(body)$	r
$\sum_{gs} e$	N	`SUM`$[\![N]\!]$	$identity[\![N]\!](e)$	SumReduction$[\![N]\!]$
$\langle\, e \mid gs \,\rangle$	List$[\![E]\!]$	`BIG`$\langle[\![E]\!]\rangle$	$singleton[\![E]\!](e)$	Concat$[\![E]\!]$
$lv := e, gs$	$()$	built in	$ignore(lv := e)$	NoReduction

Figure 23.4: Examples of wrappers for expressions with generator clause lists. Top to bottom: big operators (here $\sum$ is used as an example; the appropriate library function is called on the right-hand side), comprehensions (here list comprehensions are shown; other comprehensions are similar to list comprehensions) and generated assignment.

$$wrapper(\texttt{fn}\ (r, u) \Rightarrow \mathcal{C}[gs]\ body)$$

The generator clause list and body can be desugared using the syntax-directed desugaring $\mathcal{C}$ defined in Figure 23.3. This yields a function that is in turn passed as an argument to *wrapper*. The particular choice of the function *wrapper* depends upon the construct that is being desugared. For a reduction or a comprehension, the wrapper function is the corresponding big operator definition; see Section 13.17 and Section 13.30. For a `for` loop (Section 13.15) or a generated expression (Section 13.11.1), a special built-in wrapper is used. Examples are shown in Figure 23.4. A wrapper function always has the following type:

$$wrapper(g : (\text{Reduction}[\![R_0]\!], T \to R_0) \to R_0) : R$$

Here the type T is the type of values returned by the body expression, and R_0 and R are arbitrary types chosen by the wrapper function.

Note that the function g passed to the wrapper has essentially the same type signature as the *generate* method itself. It is instructive to think of *wrapper* as having the following similar type signature:[4]

$$wrapper'(g\text{: Generator}[\![T]\!])\text{: } R(*\ \text{NOT THE ACTUAL TYPE}\ *)$$

> DRAFT NOTE
> From here till the end of this chapter, copied from F1.0β.

An array comprehension simply desugars into a factory function call and a series of assignments:

$$\begin{array}{l}[\ i_1 = e_1 \mid gs_1 \\ \quad i_2 = e_2 \mid gs_2 \\ \quad \ldots \\ \quad i_n = e_n \mid gs_n\ ]\end{array} \longrightarrow \begin{array}{l}\texttt{do}\ a = array() \\ \quad a[i_1] := e_1, gs_1 \\ \quad a[i_2] := e_2, gs_2 \\ \quad \ldots \\ \quad a[i_n] := e_n, gs_n \\ \quad a \\ \texttt{end}\end{array}$$

The desugaring of a `for` loop depends upon the set of reduction variables. We conceptually desugar the `for` loop with reduction variables, $r_1, r_2, \ldots, r_n$, reduced using the reduction operator $\oplus$ for type $(T_1, T_2, \ldots T_n)$ as follows:

$$\texttt{for}\ gs\ \texttt{do}\ block\ \texttt{end} \longrightarrow$$

[4] In future, it is likely that Fortress will use a desugaring that in fact yields a Generator rather than a higher-order function. This permits type-directed nesting and composition of generators.

$$(r_1, r_2, \ldots r_n) \oplus= \mathcal{T}[\![(T_1, T_2, \ldots T_n), \oplus]\!][gs]\ (\texttt{do}$$
$$(r_1, r_2, \ldots r_n) : (T_1, T_2, \ldots T_n) := \text{Identity}[\![\oplus]\!]$$
$$\textit{block}$$
$$(r_1, r_2, \ldots r_n)$$
$$\texttt{end})$$

In practice, a tuple type is not a monoid. If there is only one reduction variable, this is not a problem. If there are no reduction variables, we simply use the type NoReduction used in desugaring assignments. When there are multiple reduction variables, we use nested applications of types extending the trait ReductionPair. These types encode the common properties of the variables being reduced. Recall that every reduction variable must at least have type Monoid, so it is not difficult to guarantee that ReductionPair itself also extends Monoid.

23.8.2 Accounting for Dependencies among Generators

The naive desugaring for generator lists in Figure 23.3 assumes there are always data dependencies among generators. The actual desugaring makes use of the *join* method in the Generator trait to group together generators that have no data dependencies. The goal is to permit library code to define more efficient merged generators for generator pairs. For example, it is possible for the *join* method to take the generator list $i \leftarrow 1 \# 100, j \leftarrow 2 \# 200$ and generate a blocked two-dimensional traversal. This could then be joined with $k \leftarrow 3 \# 300$ to obtain a three-dimensional blocked traversal.

However, most generators will simply make use of the default definition of *join* which calls SimplePairGenerator:

$$\texttt{object}\ \text{SimplePairGenerator}[\![\, A\ \texttt{extends}\ \text{Any}, B\ \texttt{extends}\ \text{Any}\,]\!]$$
$$(\textit{outer} : \text{Generator}[\![\, A\,]\!], \textit{inner} : \text{Generator}[\![\, B\,]\!])\ \texttt{extends}\ \text{Generator}[\![\,(A, B)\,]\!]$$
$$\textit{size} : \mathbb{Z}64 = \textit{outer.size} \cdot \textit{inner.size}$$
$$\textit{generate}[\![\, R\ \texttt{extends}\ \text{Monoid}[\![\, R, \oplus\,]\!], \texttt{opr}\ \oplus\,]\!](\textit{body} : (A, B) \to R) : R =$$
$$\textit{outer.generate}(\texttt{fn}\ (a :\ A) \Rightarrow \textit{inner.generate}(\texttt{fn}\ (b :\ B) \Rightarrow \textit{body}(a, b)))$$
$$\textit{join}[\![\, N\ \texttt{extends}\ \text{Any}\,]\!](\textit{other} :\ \text{Generator}[\![\, N\,]\!]) :\ \text{Generator}[\![\,((A, B), N)\,]\!] =$$
$$\text{SimpleMapGenerator}(\textit{outer.join}(\textit{inner.join}(\textit{other})),$$
$$(\texttt{fn}\ (a, (b, n)) \Rightarrow ((a, b), n)))$$
$$\texttt{end}$$

Note how SimplePairGenerator itself overrides the *join* method. When we attempt to join an existing pair of joined generators, we first attempt to *join* the *inner* generator of the pair with the *other* generator (the new innermost generator) passed in. This means that every generator will have the opportunity to combine with both its left and right neighbors if neither has a dependency which prevents it. Note that we use a SimpleMapGenerator, which simply applies a function to the result of another generator, to re-nest the tuples produced by the nested *join* operation.

Which pairs of adjacent traversals are combined using *join*? This question is complicated by examples such as $i \leftarrow 1 : 100, j \leftarrow 1 : 100, k \leftarrow i : 100, l \leftarrow j : 100$. We can either combine i and j traversals, or we can combine j and k traversals. In the former case we can also combine k and l traversals. The Fortress compiler is free to choose any grouping subject to the following constraints:

- Two generators may not be combined using *join* if the second is data dependent upon the first.
- Generator order must be preserved when invoking *join*.
- When a chain of three or more generators is joined, the traversals must be combined left-associatively.

We can obtain a simple greedy desugaring which joins together traversals in accordance with the above rules by simply adding the following desugaring rule which takes precedence over those given in Figure 23.3 when each variable bound in v_1 does not occur free in g_2.

$$\mathcal{T}[\![R, \boxplus]\!][\, v_1 \leftarrow g_1, v_2 \leftarrow g_2, gs\,]\ body \quad = \quad \mathcal{T}[\![R, \boxplus]\!][\, (v_1, v_2) \leftarrow g_1.join(g_2), gs\,]\ body$$

23.8.3 Using Overloading to Adapt Generators and Traversals

Overloaded instances of the *generate* method can be used to adapt a generator to the particular properties of the reduction being performed. For example, a commutative monoid need only maintain a single variable *result* containing the reduced value so far:

```
value object BlockedRange(lo: ℤ32, hi: ℤ32, b: ℤ32) extends Generator⟦ℤ32⟧
  ...
  generate⟦R extends CommutativeMonoid⟦R, ⊕⟧, opr ⊕⟧(body : ℤ32 → R) : R = do
    result : R = Identity⟦⊕⟧
    traverse(l, u) =
      if u − l + 1 ≤ max(b, 1) then
        i : ℤ32 = l
        while i ≤ u do
          t = body(i)
          atomic result := result ⊕ t
          i += 1
        end
        ()
      else
        mid = ⌈l/2⌉ + ⌊u/2⌋
        (traverse(l, mid), traverse(mid + 1, u))
        ()
      end
    traverse(lo, hi)
    result
  end
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.20
>
> The Working Draft of February 2011 declared this generator over ℤ64, as it declared the generator of Figure 23.2, whose members the ellipsis stands for; both are declared over ℤ32.

The choice of whether to apply this transformation is left up to the author of the generator; when many iterations run in parallel the *result* variable becomes a scalability bottleneck and this technique should not be used.

Various other properties of the reduction operator can be exploited:

- Idempotent reductions permit redundant computation. For example, when computing the maximum element of a set it might be simpler to enumerate set elements more than once.
- On the other hand, sometimes a more efficient non-idempotent operator can be used for a reduction if the generator promises never to produce duplicates—this fact can be used to advantage in set, multiset, or map comprehensions.
- If the reduction operator has a zero, this can be used to exit early from a partial computation. This requires that the body expression have no visible side effects such as writes or `io` actions.

At the moment, the author of a Generator is responsible for taking advantage of opportunities such as these. In future, we expect some standardized support for efficient versions of various traversals based on experience with the definitions provided here.

23.8.4 Making a Serial Version of a Generator or Distribution

DRAFT NOTE

TODO: the mechanism for generator serialization is once again magic. Formerly we constrained generator structure so the serialization was straightforward (but this probably made things slow).

A generator g can be made sequential simply by calling the builtin function $sequential$ as follows:

$v \leftarrow sequential(g)$

The resulting generator yields its elements in the natural order specified by the original generator. Several builtin generators (such as those for array indices) have an associated distribution. For these generators, $sequential$ function simply re-distributes the underlying object as follows:

$sequential(r) = \text{Sequential}.distribute(r)$

As a convenient shorthand, the $sequential$ function is also defined to work for distributions themselves. The complete declarations of the overloaded $sequential$ function are as follows:

$sequential[\![E \texttt{ extends } \text{Any}]\!](g : \text{Generator}[\![E]\!]) : \text{Generator}[\![E]\!]$
$sequential[\![E \texttt{ extends } \text{Any}]\!](d : \text{Distribution}) : \text{Distribution}$

The $sequential$ function has special meaning to the Fortress implementation; there is no need to distinguish reduction variables in loops for which generator is surrounded by a direct call to $sequential$.

Chapter 24

Overloaded Functional Declarations

> DRAFT NOTE
> Keyword and varargs parameters are not yet supported.

> DRAFT NOTE
> Return types are required for overloaded functional declarations.

Fortress allows multiple functional declarations to be in scope of a particular program point. We call this overloading. Chapter 15 describes how to determine which overloaded declarations are applicable to a particular functional call, and when several are applicable, how to select the most specific one among them. In this chapter, we give a set of rules on overloaded declarations that guarantee there exists a most specific declaration for any given functional call. These rules are complicated by the presence of coercion, which may enlarge the set of declarations that are applicable to a functional call, as discussed in Chapter 17.

24.1 Principles of Overloading

Fortress allows multiple functional declarations of the same name to be declared in a single scope. However, recall from Chapter 7 the following shadowing rules:

- dotted method declarations shadow top-level function declarations with the same name, and
- dotted method declarations provided by a trait or object declaration or object expression shadow functional method declarations with the same name that are provided by a different trait or object declaration or object expression.

Also, note that a trait or object declaration or object expression must not have a functional method declaration and a dotted method declaration with the same name, either directly or by inheritance. Therefore, top-level functions can overload with other top-level functions and functional methods, dotted methods with other dotted methods, and functional methods with other functional methods and top-level functions.

Overloading functional declarations allows the benefits of polymorphic declarations. However, with these benefits comes the potential for ambiguous calls at run time. Fortress provides overloading rules for the *declarations* of functionals to eliminate the *possibility* of ambiguous calls at run time, whether or not these calls actually appear in the program.

For each functional application expression, it should be a static error if there does not exist a statically most applicable declaration for that expression. Furthermore, these rules are checked statically. In fact, the overloading rules in Fortress allow the compiler to identify the statically most specific declaration for a particular call. Therefore an implementation strategy may be used in which the statically most specific declaration is identified statically, and the runtime dispatch mechanism need only consider dispatching among that declaration plus declarations that are more specific than that declaration (proof of this is given in Section B.2). One case is an exception. Where the static type of a call's argument stands between a trait with a `comprises` clause and its listed types, several declarations applicable to the static call may each have no other applicable declaration more specific than them (Section 24.4). Such a call is not a static error: it dispatches to a declaration no less specific than each of them, and where they, and every declaration more specific than them that the call may run, have no static parameters, its type is the intersection of their return types (Section 24.6).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.34
>
> The Working Draft of February 2011 knew no such call: it read a `comprises` clause as saying which types are below a trait, where Section 10.1 now says which values belong to it, so that a trait with static parameters may stand unlisted between a trait with a `comprises` clause and its listed types.

This chapter outlines several criteria for valid functional overloading. At any given program point, there is a set of overloaded declarations that are in scope. Fortress determines whether there is a possibility for ambiguous calls from this set by comparing declarations pairwise. The following three sections each describes a rule to accept a pair of overloaded functional declarations. If a pair of overloaded declarations satisfies any one of the three rules, it is considered a valid overloading.

The overloading rules given in this chapter are disjoint. That is, at most one rule applies to each pair of overloaded declarations. It is possible for new rules to be added which allow additional overloadings.

Overloaded declarations may have different static parameters, or some of them may have static parameters and others none (Section 15.1); Section 24.5 states how the rules of this chapter apply to declarations with static parameters. Valid overloadings for declarations that contain keyword or varargs parameters are determined by analyzing the expansion of these declarations as described in Section 15.4. Therefore, keyword parameters and varargs parameters are ignored in the remainder of this chapter, and static parameters are ignored until Section 24.5.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The Working Draft of February 2011 said here that overloaded declarations must have static parameters that are identical up to α-equivalence, and ignored static parameters in the remainder of this chapter.

An operator method declaration whose name is one of the operator parameters (described in Section 12.5) of its enclosing trait or object may be overloaded with other operator declarations in the same component. Therefore, such an operator method declaration must satisfy the overloading rules with every operator declaration in the same component.

In addition, a functional which takes a single parameter whose type is a naked type parameter declared with the bound Any written, as T `extends` Any, cannot be overloaded. This restriction prevents ambiguous overloadings such as the following:

$$makeSet[\![T \texttt{ extends } \mathrm{Any}]\!](x\colon T)\colon \mathrm{Set}[\![T]\!]$$
$$makeSet[\![T \texttt{ extends } \mathrm{Any}]\!](x\colon T, y\colon T)\colon \mathrm{Set}[\![T]\!]$$

The type parameter of $makeSet$ may be instantiated with $(\mathbb{Z}, \mathbb{Z})$ or $\mathbb{Z}$. If this is the case then the call $makeSet(3, 4)$, where 3 and 4 have type $\mathbb{Z}$, is ambiguous. A type parameter declared without an `extends` clause, whose bound is Any as well (Section 12.1), does not fall under this restriction: the Fortress standard libraries declare the factory $array_1[\![T, \texttt{nat } s_0]\!](v\colon T)$ beside $array_1[\![T, \texttt{nat } s_0]\!]()$.

REVISED BY THE 2026 REVIVAL SEE Section I.1.30

The Working Draft of February 2011 stated this restriction for a naked type parameter of bound Any while a type parameter declared without an `extends` clause was bounded by Object (Section 12.1). Under the implicit bound Any it would forbid overloading every functional whose single parameter's type is such a type parameter, the factory above among them. The restriction is stated as the compiled type checker applies it, to a bound written Any; the checker draws no such refusal on the Fortress standard libraries.

Section 24.2 states the *Subtype Rule*, which stipulates that the parameter type of one declaration be a subtype of the parameter type of the other. In this case, there is no possibility of ambiguous calls, because one declaration is more specific than the other. This section also places a restriction on the return types of the overloaded declarations to ensure that type safety is not violated. Section 24.3 defines the *Incompatibility Rule* that, if satisfied by a pair of declarations, guarantees that neither declaration is applicable to the same functional call. In Section 24.4, the *Meet Rule* requires the existence of a declaration that is more specific than both overloaded declarations in the situation that both are applicable to a given call.

In the remainder of this chapter, we build on the terminology and notation defined in Chapter 15 and Chapter 17.

24.2 Subtype Rule

If the parameter type of one declaration is a subtype of the parameter type of another (and they are not the same) then there is no ambiguity between these two declarations: for every call to which both are applicable, the first is more specific. This is the basis of the Subtype Rule.

The Subtype Rule also requires a relationship between the return types of the two declarations. Without such a requirement, a program may violate type safety.

The Subtype Rule for Functions and Functional Methods: Suppose that $f(P) : U$ and $f(Q) : V$ are two distinct function or functional method declarations in a single scope. If $P \prec Q$ and $U \preceq V$ then $f(P)$ and $f(Q)$ are a valid overloading.

The Subtype Rule for Dotted Methods: Suppose that $P_0.f(P) : U$ and $Q_0.f(Q) : V$ are two distinct dotted method declarations provided by a trait or object C. If $P_0 \prec Q_0$, $P \prec Q$, and $U \preceq V$ then $P_0.f(P)$ and $Q_0.f(Q)$ are a valid overloading.

24.3 Incompatibility Rule

The basic idea behind the Incompatibility Rule is that if there is no call to which two overloaded declarations are both applicable then there is no potential for ambiguous calls. In such a case, we say that the declarations are incompatible. Without coercion, incompatibility is equivalent to exclusion. However, the presence of coercion complicates the definition of incompatibility. To formally define incompatibility we first define the following notation. For types T and U, we say that T and U *do not share coercions*, and write $T \between U$, if any type that coerces to T excludes any type that coerces to U:

$$T \between U \iff \forall A, B : A \to T \wedge B \to U \implies A \lozenge B.$$

We say that T *is incompatible with* U, and write $T \blacklozenge U$, if T and U exclude, reject each other, and do not share coercions:

$$\begin{aligned} T \blacklozenge U &\iff T \lozenge U \wedge T \not\rightarrow U \wedge U \not\rightarrow T \wedge T \not\between U \\ &\iff \quad T \lozenge U \\ &\qquad \wedge \ (\forall A : A \to T \implies A \lozenge U) \\ &\qquad \wedge \ (\forall B : B \to U \implies B \lozenge T) \\ &\qquad \wedge \ (\forall A, B : A \to T \wedge B \to U \implies A \lozenge B) \end{aligned}$$

Note that if $T \blacklozenge U$ then no type is substitutable for both T and U.

The Incompatibility Rule for Functions and Functional Methods: Suppose that $f(P)$ and $f(Q)$ are two distinct function or functional method declarations in a single scope. If $P \blacklozenge Q$ then $f(P)$ and $f(Q)$ are a valid overloading.

The Incompatibility Rule for Dotted Methods: Suppose that $P_0.f(P)$ and $Q_0.f(Q)$ are two distinct dotted method declarations provided by a trait or object C. If $P \blacklozenge Q$ then $P_0.f(P)$ and $Q_0.f(Q)$ are a valid overloading.

24.4 Meet Rule

If neither the Subtype Rule nor the Incompatibility Rule holds for a pair of overloaded declarations then the declarations introduce the possibility of ambiguity. To avoid this ambiguity, we require a *disambiguating declaration*: for any possible arguments to which both declarations are applicable, there must be another more specific declaration that is also applicable. Thus, if the functional is called with such arguments, neither of the pair of declarations is executed because the disambiguating declaration is also applicable, and it is more specific than both.

> DRAFT NOTE
> What is "an intersection of two declarations"? – Sukyoung
>
> For function declarations we require that the disambiguating declaration be the intersection of the pair of overloaded declarations.

The Meet Rule for Functions: Suppose that $f(P)$ and $f(Q)$ are two function declarations in a single scope such that neither P nor Q is a subtype of the other and P and Q are not incompatible with one another. Let $\mathcal{S}$ be the set of types that P defines coercions from and $\mathcal{T}$ be the set of types that Q defines coercions from. $f(P)$ and $f(Q)$ are a valid overloading if there is a declaration $f(P \cap Q)$ in the scope.

all of the following hold:

- either $P \lozenge Q$ or there is a declaration $f(P \cap Q)$ in the scope, and
- either $P \lhd Q$ or $Q \lhd P$ or for all $P' \in \mathcal{S}$ and $Q' \in \mathcal{T}$ one of the two conditions holds:
  - $P' \lozenge Q'$, or
  - there is a declaration $f(P' \cap Q')$ in the scope.

Where the Meet Rule for functions asks for a declaration $f(P \cap Q)$ in the scope, it is also satisfied by declarations in the scope whose parameter types are each a subtype of both P and Q and together cover $P \cap Q$ (Section 6.10): every value of both P and Q is a value of the parameter type of one of them, for every value a program can make, not only for the values a program does make. Each such declaration is more specific than both $f(P)$ and $f(Q)$, and at run time

it is applicable to every argument whose value its parameter type holds. Every pair of the covering declarations must satisfy the overloading rules in turn, as every pair of the set must. This case does not apply to the condition on the types from which P and Q define coercions: covering does not choose between two coercions.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.34
>
> The Working Draft of February 2011 asked for a declaration on the intersection itself, and the draft note above asked what the intersection of two declarations is. The answer is the one the team's list of proposed features records for two traits with `comprises` clauses, "We agreed to revise the Meet rule to address this with the coverage check". It is an addition to the Meet Rule of the type group's paper of 2011, which asks for a declaration on the meet; the paper of 2019 has no traits with `comprises` clauses. Where a trait with static parameters stands between two traits with `comprises` clauses and their listed types, the intersection of the two is not equal to a named type (Section 6.10), and only declarations that together cover it can disambiguate them. The coverage is a judgement of the overloading rules alone: it makes no type a subtype of another (row 492 of the revival's gap ledger).

Recall that $P \cap Q$ is the intersection of types P and Q as defined in Section 6.10. We write $P \cap Q$ to denote the intersection of types P and Q. If for some type S we have $S \preceq P$ and $S \preceq Q$ then $S \preceq (P \cap Q)$, but it is not necessarily the case that $S = (P \cap Q)$ since another type may be more specific than both P and Q. For example, suppose the following:

```
trait S comprises {U, V} end
trait T comprises {V, W} end
trait U extends S excludes W end
trait V extends {S, T} end
trait W extends T end
```

$$f(s : S) = 1$$
$$f(t : T) = 1$$
$$f(v : V) = 1$$

Because of the `comprises` clauses of S and T and the `excludes` clause of U, any value of both S and T is a value of V. Thus, V covers $S \cap T$, and the declaration $f(V)$ "disambiguates" $f(S)$ and $f(T)$, i.e., it is applicable to and more specific for any call at run time to which both $f(S)$ and $f(T)$ are applicable.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.34
>
> The Working Draft of February 2011 concluded that any subtype of both S and T must be a subtype of V, so that $V = S \cap T$. That holds of types only while no type stands between S or T and their listed types; a trait with static parameters may (Section 10.1), and a static call whose argument has such a type has $f(S)$ and $f(T)$ applicable and $f(V)$ not. It runs $f(V)$, and its type is given in Section 24.6.

The Meet Rule shall not be difficult to obey, especially because the compiler can give useful feedback. For example:

$$foo(x : \text{Number}, y : \mathbb{Z}64) = \ldots$$
$$foo(x : \mathbb{Z}64, y : \text{Number}) = \ldots$$

Assuming that $\mathbb{Z}64 \prec \text{Number}$, the compiler reports that these two declarations are a problem because of ambiguity and suggests that a new declaration for $foo(\mathbb{Z}64, \mathbb{Z}64)$ would resolve the ambiguity. As another example, consider:

$$bar(x : \text{Printable}) = \ldots$$
$$bar(x : \text{Throwable}) = \ldots$$

Assuming that Printable and Throwable are neither comparable by the subtyping relation nor disjoint, the compiler reports that these two declarations are a problem because Printable and Throwable are incomparable but possibly overlapping types. As a result, these two declarations are statically rejected.

Unlike for functions, the Meet Rule for dotted methods only applies to dotted methods that are provided by the same trait or object. This is possible because two dotted methods are applicable to a given call $A_0.f(A)$ only if they are both provided by the trait or object A_0.

The Meet Rule for Dotted Methods: Suppose that $P_0.f(P)$ and $Q_0.f(Q)$ are two dotted method declarations provided by a trait or object C such that neither (P_0, P) nor (Q_0, Q) is a subtype of the other and P and Q are not incompatible with one another. Let $\mathcal{S}$ be the set of types that P defines coercions from and $\mathcal{T}$ be the set of types that Q defines coercions from. $P_0.f(P)$ and $Q_0.f(Q)$ are a valid overloading if there is a declaration $R_0.f(P \cap Q)$ provided by C with $R_0 \preceq (P_0 \cap Q_0)$.

all of the following hold:

- either $P \Diamond Q$ or there is a declaration $R_0.f(P \cap Q)$ provided by C with $R_0 \preceq (P_0 \cap Q_0)$, and
- either $P \triangleleft Q$ or $Q \triangleleft P$ or for all $P' \in \mathcal{S}$ and $Q' \in \mathcal{T}$ one of the two conditions holds:
  - $P' \Diamond Q'$, or
  - there is a declaration $R_0.f(P' \cap Q')$ provided by C with $R_0 \preceq (P_0 \cap Q_0)$.

Recall that functional methods can be viewed semantically as top-level functions, as described in Section 10.2. However, treating functional methods as top-level functions for determining valid overloading is too restrictive. In the following example:

```
trait ℤ

  opr @(self): ℤ

end

trait ℝ

  opr @(self): ℝ

end
```

if the functional methods were interpreted as top-level functions then this program would define two top-level functions with parameter types $\mathbb{Z}$ and $\mathbb{R}$. These declarations would be statically rejected as an invalid overloading because there is no relation between $\mathbb{Z}$ and $\mathbb{R}$; another trait may extend them both without declaring its own version of the functional method which may lead to an ambiguous call at run time. However, notice that declaraions are ambiguous only for calls on arguments that extend both $\mathbb{Z}$ and $\mathbb{R}$, and any type that extends both can include a new declaration that disambiguates them. We use this intuition to allow such overloadings.

The Meet Rule for Functional Methods: Suppose that $f(P)$ and $f(Q)$ are two functional method declarations occurring in trait or object declarations or object expressions such that neither P nor Q is a subtype of the other and P and Q are not incompatible with one another. Let $f(P)$ and $f(Q)$ have self parameters at i and j respectively. $f(P)$ and $f(Q)$ are a valid overloading if all of the following hold:

- $i = j$
- if there exists a trait or object C that provides both $f(P)$ and $f(Q)$ then $P \neq Q$ and there is a declaration $f(P \cap Q)$ provide by C having self parameter at i.

Also, let $\mathcal{S}$ be the set of types that P defines coercions from and $\mathcal{T}$ be the set of types that Q defines coercions from. $f(P)$ and $f(Q)$ are a valid overloading if all of the following hold:

- $i = j$

- either $P \lozenge Q$ or if there exists a trait or object C that provides both $f(P)$ and $f(Q)$ then $P \neq Q$ and there is a declaration $f(P \cap Q)$ provide by C, and
- either $P \triangleleft Q$ or $Q \triangleleft P$ or for all $P' \in \mathcal{S}$ and $Q' \in \mathcal{T}$ one of the two conditions holds:
  - $P' \lozenge Q'$, or
  - there is a declaration $f(P' \cap Q')$ provided by C.

As for functions, where this rule asks for a declaration $f(P \cap Q)$ provided by C, it is also satisfied by declarations provided by C, having self parameter at i, whose parameter types are each a subtype of both P and Q and together cover $P \cap Q$; the case does not apply to the condition on the types from which P and Q define coercions.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.34
>
> The Working Draft of February 2011 asked here, as for functions, for a declaration on the intersection itself.

Notice that the Meet Rule for functional methods requires the self parameters of two overloaded declarations to be in the same position. This requirement guarantees that no ambiguity is caused by the position of the self parameter. Two declarations which differ in the position of the self parameter must satisfy either the Subtype Rule or the Incompatibility Rule to be a valid overloading.

Functional method declarations can overload with function declaraions. A valid overloading between a functional method declaration and a function declaration is determined by applying the (more restrictive) Meet Rule for functions to both declarations.

24.5 Declarations with Static Parameters

The rules above compare the parameter types and the return types of declarations without static parameters. A declaration with static parameters is read as one declaration whose parameter type ranges over every instance of it whose static arguments satisfy their bounds (Section 15.1). A declaration d_1 is more specific than a declaration d_2 when d_2 is applicable to every call to which d_1 is applicable, and not the other way round (Section 15.5); where neither has static parameters, this is the relation $P \prec Q$ of the rules above. A set of overloaded declarations, some or all of which have static parameters, is a valid overloading when every pair d_1, d_2 of them satisfies the following three rules:

- The *No Duplicates Rule*: d_1 and d_2 are not each applicable to every call to which the other is applicable.
- The *Meet Rule*: d_1 and d_2 exclude each other, or one of them is more specific than the other, or some declaration of the set is more specific than both and applicable to every call to which both are applicable.
- The *Return Type Rule*: if d_1 is more specific than d_2, then for every argument type other than $\mathrm{BottomType}$ to which d_1 is applicable, and for every instance of d_2 that is applicable to it, some instance of d_1 is applicable to it whose return type is a subtype of the return type of that instance of d_2.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The revival's text of 30 September 2026 stated the No Duplicates Rule as "d_1 and d_2 are not each more specific than the other", which the strict sense of more specific above made refuse no pair. It is stated on the relation without "and not the other way round", as the type group's papers of 2011 and 2019 state it, and so it refuses $dup[\![T]\!](x\colon \mathrm{Box}[\![T]\!])$ beside $dup[\![U]\!](x\colon \mathrm{Box}[\![U]\!])$, each applicable to every call to which the other is.

The Subtype Rule, the Incompatibility Rule and the Meet Rule above are these rules for declarations without static parameters, with the conditions that coercion adds; the Return Type Rule is the Subtype Rule's condition on return types, checked over every instance of the less specific declaration and not over one instance chosen for the pair.

As in Section 24.4, declarations of the set that are each more specific than both d_1 and d_2 and that together cover every argument to which both are applicable also satisfy the Meet Rule.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.34
>
> The type group's paper of 2011 states the Meet Rule without this case.

Two parameter types with static parameters exclude each other, for the Meet Rule and the Incompatibility Rule, as the type group's paper of 2011 decides it by *existential reduction*: each static parameter stands for any type within its bound, and where the two parameter types, or their elements at one position, are or extend instances of one parameterized trait, instantiation exclusion (Section 6.3) lets them meet only where the static arguments of those instances are the same, every static argument but an operator argument counting. So where Box and Pair are distinct object types with static parameters, the parameter types $\mathrm{Box}[\![T]\!]$ and $\mathrm{Pair}[\![A, B]\!]$ of two declarations exclude each other whatever T, A and B stand for.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> The revival's text of 30 September 2026 read instantiation exclusion here only where the two parameter types mention one parameterized trait. The type group's paper of 2011 reads it at every pair of instances of one parameterized trait among the two types' supertypes, so that $\mathit{copyOf}[\![T]\!](a\colon \mathrm{Frozen}[\![T]\!])$ and $\mathit{copyOf}[\![T]\!](a\colon \mathrm{Thawed}[\![T]\!])$ exclude each other where $\mathrm{Frozen}[\![E]\!]$ and $\mathrm{Thawed}[\![E]\!]$ both extend $\mathrm{Readable}[\![E]\!]$ and exclude each other, a value of both having one instance of $\mathrm{Readable}$.

The Return Type Rule refuses the following pair, where trait Round extends trait Shape and object Circle extends Round:

$(*$ Not allowed! $*)$
$\mathit{ident}[\![T$ `extends` $\mathrm{Shape}]\!](x\colon T)\colon T = x$
$\mathit{ident}(x\colon \mathrm{Round})\colon \mathrm{Round} = x$

The second declaration is more specific than the first. For an argument of type Circle, the instance of the first at Circle is applicable and returns a Circle, but the second, which runs for such an argument, returns only a Round, and it has no other instance. The pair is allowed when the second declaration is itself generic:

$\mathit{ident}[\![T$ `extends` $\mathrm{Shape}]\!](x\colon T)\colon T = x$
$\mathit{ident}[\![T$ `extends` $\mathrm{Round}]\!](x\colon T)\colon T = x$

Its instance at any type to which it is applicable returns that type.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> This section is new. The Working Draft of February 2011 required overloaded declarations to have identical static parameters and ignored them. The rules stated are those of the type group's paper of 2011 (Allen, Hilburn, Kilpatrick, Luchangco, Ryu, Chase, and Steele, "Type Checking Modular Multiple Dispatch with Parametric Polymorphism and Multiple Inheritance", OOPSLA 2011), which the compiled type checker applies, with its Return Type Rule checked over every instance of the less specific declaration; the pair refused above has the shape of that paper's own example, which the compiled path used to accept and then fail to verify. The interpreter runs the pair allowed above, reading two declarations with static parameters on their declared domains, each over its own instances, where no pair of their parameter types excludes each other; the compiled code generator fails on it (row 537 of the revival's gap ledger). The compiled implementation adds one restriction that is not a rule of the language. When one declaration with static parameters is more specific than another that declares as many static parameters of its own, the two must declare them of the same kinds, position by position, and the return type of the more specific one must be a subtype of the other's when their static parameters are identified position by position; for its code generator hands the static arguments that a call writes, by position, to whichever declaration the call dispatches to (row 398 of the revival's gap ledger). So it refuses this override, whose return type names its static parameters in the other order, where Pr is an object with two type parameters:

```
trait Maker
  make⟦A, B⟧(): Pr⟦A, B⟧
end
object M extends Maker
  make⟦A, B⟧(): Pr⟦B, A⟧ = Pr⟦B, A⟧(1)
end
```

The type group's paper of 2019 (Park, Hong, Steele and Ryu, POPL 2019) instantiates a dispatched declaration from the dynamic types of the arguments and the static type of the call, renaming static parameters freely (Section 15.5); under it the override is valid, and the restriction goes when the compiled dispatcher instantiates a declaration that way.

24.6 Coercion and Overloading Resolution

The overloading rules given in this chapter are sufficient to prove the following two facts:

1. If no declaration is applicable to a static call but there is a declaration that is applicable with coercion then there exists a single most specific declaration that is applicable with coercion to the static call.
2. If any declaration is applicable to a static call then there exists a single most specific declaration that is applicable to the corresponding dynamic call and, unless the static type of the argument stands between a trait with a `comprises` clause and its listed types (Section 24.4), a single most specific declaration that is applicable to the static call.

Moreover, we can prove that the most specific declaration that is applicable to a dynamic call is the same as or more specific than every declaration that is applicable to the corresponding static call, so that, by the Return Type Rule, its return type is a subtype of the return type of each of them. Where the declarations applicable to a static call that no other applicable declaration is more specific than, and every declaration more specific than them that the call may run, have no static parameters, the type of the static call is the intersection of their return types; where there is one such declaration, that is its return type.

Appendix B formally proves that the rules discussed in the previous sections guarantee the static resolution of coercion (described in Section 17.5) is well defined for functions (the case for methods is analogous). Also in Appendix B is a proof that the overloading rules for function declarations are sufficient to guarantee no undefined nor ambiguous calls at run time (again, the case for methods is analogous). That proof covers declarations without static parameters and calls to which declarations are applicable without coercion, and it rests on every value of a trait with a `comprises` clause being a value of one of its listed types (Section 10.1); the appendix states its scope.

REVISED BY THE 2026 REVIVAL SEE Section I.1.34

The Working Draft of February 2011 said that a single most specific declaration is applicable to every static call to which any is applicable, and that the one the dynamic call runs is more specific than it. With the closed-trait case of the Meet Rule, a static call whose argument's static type stands between two traits with `comprises` clauses and their listed types may have several; the declaration that runs is below every one of them, and the call's type is the intersection of their return types, which the Return Type Rule makes sound for declarations without static parameters. For declarations with static parameters one instance would have to meet every one of those return types at once, and the type of such a call is not yet given. The compiled type checker does not yet enforce across components that every value of a trait with a `comprises` clause is a value of one of its listed types (row 487 of the revival's gap ledger), which the proof assumes.

Chapter 25

Operator Declarations

> DRAFT NOTE
> Multifix operators and keyword parameters are not yet supported.

An operator declaration may appear anywhere a top-level function or method declaration may appear. Operator declarations are like other function or method declarations in all respects except that an operator declaration has `opr` and has an operator name (see Section 16.1 for a discussion of valid operator names) instead of an identifier. The precise placement of the operator name within the declaration depends on the fixity of the operator. Like other functionals, operators may have overloaded declarations (see Chapter 15 for a discussion of overloading). These overloadings may be of the same or differing fixities.

Syntax:

FnDecl	::=	*FnMods*? *FnHeaderFront FnHeaderClause* (= *Expr*)?
FnHeaderFront	::=	*OpHeaderFront*
OpHeaderFront	::=	`opr BIG`? ({↦ | *LeftEncloser* | *Encloser*) *StaticParams*? *Params*? (*RightEncloser* | *Encloser*)
	|	`opr` *ValParam* (*Op* | *ExponentOp*) *StaticParams*?
	|	`opr BIG`? (*Op* | ^ | *Encloser* | ∑ | ∏) *StaticParams*? *ValParam*
MdDecl	::=	*MdDef*
	|	`abstract` ? *MdMods*? *MdHeaderFront FnHeaderClause*
MdHeaderFront	::=	*OpMdHeaderFront*
OpMdHeaderFront	::=	`opr BIG`? ({↦ | *LeftEncloser* | *Encloser*) *StaticParams*? *Params*? (*RightEncloser* | *Encloser*) (:= (*SubscriptAssignParam*))?
	|	`opr` *ValParam* (*Op* | *ExponentOp*) *StaticParams*?
	|	`opr BIG`? (*Op* | ^ | *Encloser* | ∑ | ∏) *StaticParams*? *ValParam*
SubscriptAssignParam	::=	*Varargs*
	|	*Param*

An operator declaration has one of eight forms: multifix operator declaration, infix operator declaration, prefix operator declaration, postfix operator declaration, nofix operator declaration, bracketing operator declaration, subscripting operator method declaration, and subscripted assignment operator method declaration. Each is invoked according to specific rules of syntax. An operator method declaration must be a functional method declaration, a subscripting operator method declaration, or a subscripted assignment operator method declaration.

25.1 Multifix Operator Declarations

A multifix operator declaration has `opr` and then an operator name where a functional declaration would have an identifier. The declaration must not have any keyword parameters, and must be capable of accepting at least two arguments. It is permissible to use a varargs parameter; in fact, this is a good way to define a multifix operator. Static parameters (described in Chapter 12) may also be present, between the operator and the parameter list.

An expression consisting of a multifix operator applied to an expression will invoke a multifix operator declaration. The compiler considers all multifix operator declarations for that operator that are both accessible and applicable, and the most specific operator declaration is chosen according to the usual rules for overloaded functionals. The invocation will pass more than two arguments.

A multifix operator declaration may also be invoked by an infix, a prefix, or nofix (but not a postfix) operator application if the declaration is applicable.

Example:

> DRAFT NOTE
> Example is commented out because it is not supported nor run by the interpreter yet.

25.2 Infix Operator Declarations

An infix operator declaration has `opr` and then an operator name where a functional declaration would have an identifier. The declaration must have two value parameter, which must not be a keyword parameter or varargs parameter. Static parameters may also be present, between the operator and the parameter list.

An expression consisting of an infix operator applied to an expression will invoke an infix operator declaration. The compiler considers all infix and multifix operator declarations for that operator that are both accessible and applicable, and the most specific operator declaration is chosen according to the usual rules for overloaded functionals.

Note that superscripting (`^`) may be defined using an infix operator declaration even though it has very high precedence and cannot be used as a multifix operator.

Example:

`opr SMAX`$[\![T$ `extends` $\mathrm{String}]\!](x\colon T, y\colon T)\colon T =$ `if` $x > y$ `then` x `else` y `end`

25.3 Prefix Operator Declarations

A prefix operator declaration has `opr` and then an operator name where a functional declaration would have an identifier. The declaration must have one value parameter, which must not be a keyword parameter or varargs parameter. Static parameters may also be present, between the operator and the parameter list.

An expression consisting of a prefix operator applied to an expression will invoke a prefix operator declaration. The compiler considers all prefix and multifix operator declarations for that operator that are both accessible and applicable, and the most specific operator declaration is chosen according to the usual rules for overloaded functionals.

Example:

`opr INV`$(x\colon \mathrm{Widget})\colon \mathrm{Widget} = x.invert()$

25.4 Postfix Operator Declarations

A postfix operator declaration has `opr` where a functional declaration would have an identifier; the operator name itself *follows* the parameter list. The declaration must have one value parameter, which must not be a keyword parameter or varargs parameter. Static parameters may also be present, between `opr` and the parameter list.

An expression consisting of a postfix operator applied to an expression will invoke a postfix operator declaration. The compiler considers all postfix operator declarations for that operator that are both accessible and applicable, and the most specific operator declaration is chosen according to the usual rules for overloaded functionals.

Example:

$$\texttt{opr}\ (n{:}\,\mathbb{Z}32)!\ =\ \prod_{i \leftarrow 1:n} i \qquad (\star\ \text{factorial}\ \star)$$

25.5 Nofix Operator Declarations

A nofix operator declaration has `opr` and then an operator name where a functional declaration would have an identifier. The declaration must have no parameters.

An expression consisting only of a nofix operator will invoke a nofix operator declaration. The compiler considers all nofix and multifix operator declarations for that operator that are both accessible and applicable, and the most specific operator declaration is chosen according to the usual rules for overloaded functionals.

Uses for nofix operators are rare, but those rare examples are very useful. For example, if the $@$ operator is used to construct subscripting ranges, and it is the nofix declaration of $@$ that allows a lone $@$ to be used as a subscript:

$$\texttt{opr}\ @(){:}\,\mathrm{ImplicitRange} = \mathrm{ImplicitRange}$$

25.6 Bracketing Operator Declarations

A bracketing operator declaration has `opr` where a functional declaration would have an identifier. The value parameter list, rather than being surrounded by parentheses, is surrounded by the brackets being defined. A bracketing operator declaration may have any number of parameters, keyword parameters, and varargs parameters in the value parameter list. Static parameters may also be present, between `opr` and the parameter list. Any paired Unicode brackets may be so defined *except* ordinary parentheses and white square brackets.

An expression consisting of zero or more comma-separated expressions surrounded by a bracket pair will invoke a bracketing operator declaration. The compiler considers all bracketing operator declarations for that type of bracket pair that are both accessible and applicable, and the most specific operator declaration is chosen according to the usual rules for overloaded functionals. For example, the expression $\langle p, q \rangle$ might invoke the following bracketing method declaration:

`(*` angle bracket notation for inner product `*)`

$$\texttt{opr}\ \big\langle [\![T\ \texttt{extends}\ \mathrm{Number}, \texttt{nat}\ k]\!]\ x{:}\,\mathrm{Vector}[\![T,k]\!], y{:}\,\mathrm{Vector}[\![T,k]\!] \big\rangle =$$

$$\sum_{i \leftarrow x.indices()} x_i \cdot y_i$$

25.7 Subscripting Operator Method Declarations

A subscripting operator method declaration has `opr` where a method declaration would have an identifier. The value parameter list, rather than being surrounded by parentheses, is surrounded by a pair of brackets. A subscripting operator method declaration may have any number of value parameters, keyword parameters, and varargs parameters in that value parameter list. Static parameters may also be present, between `opr` and the left bracket. Any paired Unicode brackets may be so defined *except* ordinary parentheses and white square brackets; in particular, the square brackets ordinarily used for indexing may be used.

An expression consisting of a subexpression immediately followed (with no intervening whitespace) by zero or more comma-separated expressions surrounded by brackets will invoke a subscripting operator method declaration. Methods for the expression preceding the bracketed expression list are considered. The compiler considers all subscripting operator method declarations that are both accessible and applicable, and the most specific method declaration is chosen according to the usual overloading rules. For example, the expression foo_p might invoke the following subscripting method declaration because expressions in the square brackets are rendered as subscripts:

```
(* subscripting method *)
opr [x: BizarroIndex] = self.bizarroFetch(x)
```

25.8 Subscripted Assignment Operator Method Declarations

A subscripted assignment operator method declaration has `opr` where a method declaration would have an identifier. The value parameter list, rather than being surrounded by parentheses, is surrounded by a pair of brackets; this is then followed by the operator `:=` and then a second value parameter list in parentheses, which must contain exactly one non-keyword value parameter. A subscripted assignment operator method declaration may have any number of value parameters within the brackets; these value parameters may include keyword parameters and varargs parameters. A result type may appear after the second value parameter list, but it must be $()$. Static parameters may also be present, between `opr` and the left bracket. Any paired Unicode brackets may be so defined *except* ordinary parentheses and white square brackets; in particular, the square brackets ordinarily used for indexing may be used.

An assignment expression consisting of an expression immediately followed (with no intervening whitespace) by zero or more comma-separated expressions surrounded by brackets, followed by the assignment operator `:=`, followed by another expression, will invoke a subscripted assignment operator method declaration. Methods for the expression preceding the bracketed expression list are considered. The compiler considers all subscript operator method declarations that are both accessible and applicable, and the most specific method declaration is chosen according to the usual overloading rules. When a compound assignment operator (described in Section 13.7) is used with a subscripting operator and a subscripted assignment operator, for example $a_3 \mathrel{+}= k$, both a subscripting operator declaration and a subscripted assignment operator declaration are required. For example, the assignment $foo_p := myWidget$ might invoke the following subscripted assignment method declaration:

```
(* subscripted assignment method *)
opr [x: BizarroIndex] := (newValue: Widget) = self.bizarroInstall(x, newValue)
```

25.9 Conditional Operator Declarations

A *conditional operator* is a binary operator (other than '`:`' and '`::`') that is immediately followed by '`:`'; see Section 16.6. A conditional operator expression $x@:y$ is syntactic sugar for $x@(\texttt{fn}\ () \Rightarrow y)$; that is, the right-hand

operand is converted to a "thunk" (zero-parameter function) that then becomes the right-hand operand of the corresponding unconditional operator. Therefore a conditional operator is simply implemented as an overloading of the operator that accepts a thunk.

It is also permitted for a conditional operator to have a preceding as well as a following colon. A conditional operator expression $x : @ : y$ is syntactic sugar for $(\texttt{fn}\ () \Rightarrow x)@(\texttt{fn}\ () \Rightarrow y)$; that is, each operand is converted to a thunk. This mechanism is used, for example, to define the results-comparison operator `:~:`, which takes exceptions into account.

The conditional $\wedge$ and $\vee$ operators for boolean values, for example, are implemented as follows:

$\texttt{opr}\ \vee(a\colon \mathrm{Boolean}, b\colon \mathrm{Boolean})\colon \mathrm{Boolean} = \texttt{if}\ a\ \texttt{then}\ \mathit{true}\ \texttt{else}\ b\ \texttt{end}$

$\texttt{opr}\ \wedge(a\colon \mathrm{Boolean}, b\colon \mathrm{Boolean})\colon \mathrm{Boolean} = \texttt{if}\ a\ \texttt{then}\ b\ \texttt{else}\ \mathit{false}\ \texttt{end}$

$\texttt{opr}\ \vee\big(a\colon \mathrm{Boolean}, b\colon () \to \mathrm{Boolean}\big)\colon \mathrm{Boolean} = \texttt{if}\ a\ \texttt{then}\ \mathit{true}\ \texttt{else}\ b()\ \texttt{end}$

$\texttt{opr}\ \wedge\big(a\colon \mathrm{Boolean}, b\colon () \to \mathrm{Boolean}\big)\colon \mathrm{Boolean} = \texttt{if}\ a\ \texttt{then}\ b()\ \texttt{else}\ \mathit{false}\ \texttt{end}$

25.10 Big Operator Declarations

A *big operator* such as $\sum$ or $\prod$ is declared as a usual operator declaration. See Section 23.8 for an example declaration of a big operator. A big operator application is either a *reduction expression* described in Section 13.17 or a *comprehension* described in Section 13.30.

Chapter 26

Dimension and Unit Declarations

> DRAFT NOTE
> Dimensions and units are not yet supported.

Syntax:

DimUnitDecl ::= `dim` *Id* (= *Type*)? (`unit` | `SI_unit`) *Id*$^+$ (= *Expr*)?
| `dim` *Id* (= *Type*)? (`default` *Id*)?
| (`unit` | `SI_unit`) *Id*$^+$ (: *Type*)? (= *Expr*)?

26.1 Dimension Declarations

Dimensions may be explicitly declared; every declared dimension must be declared at the top level of a program component, not within a block expression or trait. Other dimensions may be constructed by multiplying and dividing other dimensions, as described in Chapter 18. An explicitly declared dimension may be a *base dimension* (with no definition specified) or a *derived dimension* (with a definition specified in the form of an initialization expression).

The set of all dimensions has the algebraic structure of a free abelian group. The identity element of this group is the dimension Unity, which represents dimensionlessness.

For every two dimensions D and E, there is a dimension DE (which may also be written $D \cdot E$), corresponding to the product of the dimensions D and E and a dimension D/E, corresponding to the quotient of the dimensions D and E. The syntactic sugar $1/D$ is equivalent to Unity/D for all dimensions D. A dimension can be raised to a rational power where both the numerator and the denominator of the rational power must be a valid `nat` *parameter* instantiation; D^0 is the same as Unity, D^1 is the same as D, and D^{m+n} is the same as $D^m D^n$. The syntactic sugar D^{-n} is the same as Unity/D^n.

Here are some examples of base dimension declarations:

```
dim Length
dim Mass
dim Time
dim ElectricCurrent
```

Here are some examples of computed dimensions:

$\mathrm{Length}/\mathrm{Time}$
$\mathrm{Velocity}/\mathrm{Time}$
$\mathrm{Length} \cdot \mathrm{Mass}/\mathrm{Time}^2$

$\text{Length Mass Time}^{-2}$
$\text{ElectricCurrent}/\text{Length}^2$

and here some of these computed dimensions are given names through the use of derived dimension declarations:

`dim` $\text{Velocity} = \text{Length}/\text{Time}$
`dim` $\text{Acceleration} = \text{Velocity}/\text{Time}$
`dim` $\text{CurrentDensity} = \text{ElectricCurrent}/\text{Length}^2$

26.2 Unit Declarations

Every unit belongs to exactly one dimension, which is the type of the unit. A dimension may have more than one unit, but one of these units may be singled out as the *default unit* for that dimension by adding a `default` clause:

`dim` Length `default` meter
`dim` Mass `default` kilogram
`dim` Time `default` second

The default unit is used when a numerical type is multiplied by a dimension to produce a new type (see Chapter 18). If no default clause is specified for a base dimension, then it has no default unit. If no default clause is specified for a derived dimension, then it has a default unit if and only if all the dimensions mentioned in its initialization expression have defaults, in which case its default unit is calculated using the initialization expression with each dimension replaced by its default unit.

Some units are explicitly declared; every declared unit must be declared at the top level of a program component, not within a block expression or trait. Other units may be constructed by multiplying and dividing other units. An explicitly declared unit may be a *base unit* (with no definition specified) or a *derived unit* (with a definition specified in the form of an initialization expression).

The set of all units, like the set of all dimensions, has the algebraic structure of a free abelian group. The identity element of this group is the unit dimensionless, of dimension Unity. Note that there may be other units of dimension Unity, such as radian and steradian, but only dimensionless is the identity of the group of all units. (Note that there is a straightforward homomorphism of units onto dimensions, wherein every unit is mapped to its dimension.)

Here are some examples of base unit declarations:

`unit` meter : Length
`unit` kilogram : Mass
`unit` second : Time
`unit` ampere : ElectricCurrent

Here are some examples of computed units:

$\text{meter}/\text{second}$
$(\text{meter}/\text{second})/\text{second}$
$\text{meter} \cdot \text{kilogram}/\text{second}^2$
$\text{meter kilogram second}^{-2}$
$\text{ampere}/\text{meter}^2$

and here some computed units are given names through the use of derived unit declarations:

`unit` $\text{newton}: \text{Force} = \text{meter} \cdot \text{kilogram}/\text{second}^2$
`unit` $\text{joule}: \text{Energy} = \text{newton meter}$
`unit` $\text{pascal}: \text{Pressure} = \text{newton}/\text{meter}^2$

In the preceding examples, the initialization expression for each unit is itself a unit. It is also permitted for the initialization expression to be a dimensioned numerical value, in which case the unit being declared is related to the unit of the dimensioned numerical value by a numerical conversion factor.

As with an ordinary variable declaration, one may omit the dimension for a unit if there is an initialization expression; the dimension of the declared unit is the dimension of the unit of the expression.

> DRAFT NOTE
> COMMENTED TEXT; I like the explanation below better. – Eric
>
> Every unit can be reduced to a canonical unit as follows. A base unit is canonical; a `unit` parameter is canonical; a defined unit is replaced by its initialization expression, but only if the value of the expression is a unit and not a dimensioned value, where every unit in that expression is replaced by its canonical unit; and finally all arithmetic is performed so as to reduce the expression to a single unit. Two units are considered equivalent if their canonical units are identical.
>
> Now here is a slightly different process.

Every unit can be reduced to a canonical value as follows. A base unit is multiplied by the value 1; a `unit` parameter is multiplied by the value 1; a defined unit is replaced by its initialization expression and then every unit in that expression is replaced by its canonical form; and finally all arithmetic is performed so as to reduce the units to a single unit and the numerical values to a single numerical value. A dimensioned value with unit U is convertible by the `in` operator to a value with unit V if the canonical values for U and V have the same unit; the conversion involves multiplying the numerical value by the ratio of the numerical value of the canonical form of V to the numerical value of the canonical form of U.

For example, given the declarations:

`dim` Length
`unit` meter: Length; `unit` meters = meter
`unit` kilometer: Length = 10^3meter; `unit` kilometers = kilometer
`unit` inch: Length = 2.54×10^{-2}meter; `unit` inches = inch
`unit` foot: Length = 12 inch; `unit` feet = foot
`unit` mile: Length = 5280 foot; `unit` miles = mile

then one can say 3 miles `in` kilometers and the `in` operator will multiply the numerical value 3 by the amount of $((2.54 \times 10^{-2})(12)(5280)/10^3)$, or $25146/15625$.

Notice the subtle difference between these two declarations:

`unit` radian = meter/meter
`unit` radian = 1 meter/meter

The first declaration defines radian to be equivalent to dimensionless, and so a value with unit radian can be used anywhere a dimensionless value can be used, and vice versa. The second declaration defines radian to be convertible to dimensionless but not equivalent, and so it is necessary to use the `in` operator (or multiplication and division by radian) to convert between values in radians and truly dimensionless values.

26.3 Abbreviating Dimension and Unit Declarations

For convenience, three forms of syntactic sugar are provided when declaring dimensions and units. First, in a `unit` declaration one may mention more than one name before the colon, and the extra names are defined to be synonyms for the first name; thus

`unit` foot feet ft: Length

means exactly the same thing as

`unit` foot: Length
`unit` feet: Length = foot
`unit` ft: Length = foot

Second, instead of the reserved word `unit` one may use the reserved word `SI_unit`, which has the effect of defining not only the specified names but also names with the various SI prefixes attached. If more than one name is specified, then the last name is assumed to be a symbol and has symbol prefixes (such as M and n) attached; all other names have the full prefixes (such as mega and nano) attached. Thus

`SI_unit` $name_1$ $name_2$ $name_3$: ...

may be regarded as an abbreviation for

`unit` $name_1$ $name_2$ $name_3$: ...
`unit` yotta$name_1$ yotta$name_2$ Y$name_3$ = $10^{24}name_1$
`unit` zetta$name_1$ zetta$name_2$ Z$name_3$ = $10^{21}name_1$
`unit` exa$name_1$ exa$name_2$ E$name_3$ = $10^{18}name_1$
`unit` peta$name_1$ peta$name_2$ P$name_3$ = $10^{15}name_1$
`unit` tera$name_1$ tera$name_2$ T$name_3$ = $10^{12}name_1$
`unit` giga$name_1$ giga$name_2$ G$name_3$ = $10^{9}name_1$
`unit` mega$name_1$ mega$name_2$ M$name_3$ = $10^{6}name_1$
`unit` kilo$name_1$ kilo$name_2$ k$name_3$ = $10^{3}name_1$
`unit` hecto$name_1$ hecto$name_2$ h$name_3$ = $10^{2}name_1$
`unit` deka$name_1$ deka$name_2$ da$name_3$ = $10name_1$
`unit` deci$name_1$ deci$name_2$ d$name_3$ = $10^{-1}name_1$
`unit` centi$name_1$ centi$name_2$ c$name_3$ = $10^{-2}name_1$
`unit` milli$name_1$ milli$name_2$ m$name_3$ = $10^{-3}name_1$
`unit` micro$name_1$ micro$name_2$ $\mu name_3$ = $10^{-6}name_1$
`unit` nano$name_1$ nano$name_2$ n$name_3$ = $10^{-9}name_1$
`unit` pico$name_1$ pico$name_2$ p$name_3$ = $10^{-12}name_1$
`unit` femto$name_1$ femto$name_2$ f$name_3$ = $10^{-15}name_1$
`unit` atto$name_1$ atto$name_2$ a$name_3$ = $10^{-18}name_1$
`unit` zepto$name_1$ zepto$name_2$ z$name_3$ = $10^{-21}name_1$
`unit` yocto$name_1$ yocto$name_2$ y$name_3$ = $10^{-24}name_1$

where μ is the Unicode character U+00B5 MICRO SIGN. Third, a `dim` declaration and a `unit` or `SI_unit` declaration may be collapsed into a single declaration by writing the `unit` or `SI_unit` declaration in place of the `default` clause in the `dim` declaration and omitting the colon and dimension from the `unit` declaration. Thus

`dim` Length `SI_unit` meter meters m
`dim` Power = Energy/Time `SI_unit` watt watts W = joule/second

is understood to abbreviate

`dim` Length `default` meter; `SI_unit` meter meters m: Length
`dim` Power = Energy/Time `default` watt; `SI_unit` watt watts W: Power = joule/second

In this way the names of the seven SI base units, along with all possible plural and prefixed forms, may be concisely defined as follows:

`dim` Length `SI_unit` meter meters m
`dim` Mass `default` kilogram; `SI_unit` gram grams g: Mass
`dim` Time `SI_unit` second seconds s
`dim` ElectricCurrent `SI_unit` ampere amperes A
`dim` Temperature `SI_unit` kelvin kelvins K

`dim` AmountOfSubstance `SI_unit` mole moles mol
`dim` LuminousIntensity `SI_unit` candela candelas cd

Note the subtle difference in the declaration of Mass that allows the default unit to be kilogram rather than gram.

26.4 Absorbing Units

Syntax:

StaticParam ::= *Id Extends*? (`absorbs unit`)?
| `unit` *Id* (: *Type*)? (`absorbs unit`)?

The declaration of a type parameter or a `unit` parameter for a parameterized trait may contain a clause "`absorbs unit`"; at most one static parameter of a trait may have this clause. An instance of a parameterized trait with a static parameter that "`absorbs unit`" may be multiplied or divided by a unit, the result being another instance of that parameterized trait in which the static argument corresponding to the unit-absorbing parameter has been multiplied or divided by the unit.

A few examples should make this clear. Given the declaration

`trait` Vector⟦EltType `extends` Number `absorbs unit`, `nat` *len*⟧ ... `end`

then Vector⟦Float, 3⟧meter means the same as Vector⟦Float meter, 3⟧, and Vector⟦Float, 3⟧/second means the same as Vector⟦Float/second, 3⟧. Therefore, $[(3\,\mathrm{m})(2\,\mathrm{m})(5\,\mathrm{m})]$ means the same as $[3\ 2\ 5]\mathrm{m}$. Similarly, given the declaration

`trait` Float⟦`unit` U `absorbs unit`, `nat` e, `nat` s⟧ ... `end`

then Float⟦meter, 11, 53⟧/second means the same as Float⟦meter/second, 11, 53⟧, and

Float⟦dimensionless, 8, 24⟧meter kilogram/second^2

means the same as

Float⟦dimensionless meter kilogram/second^2, 8, 24⟧,

which is the same as

Float⟦meter kilogram/second^2, 8, 24⟧.

This is the mechanism by which meaning is given to the multiplication and division of library-defined types by units.

Chapter 27

Support for Domain-Specific Languages

> DRAFT NOTE
> This chapter will include the Fortress syntactic abstraction mechanism described in [3].

Part IV

Fortress Interpreter Library APIs and Documentation

> DRAFT NOTE
> The APIs in this part are for the Fortress interpreter; they do not work with the Fortress compiler.

Chapter 28

Structure of the Fortress Libraries

The Fortress libraries are divided into two basic categories. The first, covered in Chapter 29, are the libraries that are automatically imported by every Fortress component and API, chiefly FortressLibrary and FortressBuiltin. These libraries provide the basic numeric types, generators, arrays, booleans, exceptions, and the like. The second, covered in Chapter 30, are libraries that must be explicitly imported by the programmer. These include List, Set, and Map.

The documentation in Part IV is largely automatically generated from the API code for the libraries themselves, and describes the state of the libraries as of the release date of this specification. The libraries are presently in a state of flux, and programmers using a more recent version of the Fortress implementation may find differences between the APIs presented here and those found in the actual implementation.

Chapter 29

Default Libraries

Two sets of libraries are imported into every Fortress program by default. The first, FortressLibrary (Section 29.1), implements the high-level functionality that will be used by most Fortress programmers. It will eventually be possible to selectively exclude portions of this library from a component if desired. By contrast, the second set of libraries are the *builtins* (Section 29.2). These libraries are intended to encompass primitive functionality that must be visible in every Fortress component.

29.1 FortressLibrary

`api` FortressLibrary

`import` Stream.{WriteStream}

`import` FlatString.{FlatString}

`import` String.{StringStats}

Simple Combinators

`Casting`

cast⟦T `extends` Any⟧(x: Any) : T

instanceOf⟦T `extends` Any⟧(x: Any): Boolean

`Useful functions`

ignore(_: Any): ()

identity⟦T `extends` Any⟧($x : T$) : T

`(*` Function composition `*)`

`opr` ∘⟦A, B, C⟧(f: $B \to C$, g: $A \to B$): $A \to C$

fail⟦T⟧(s: String) : T

Control over locality and location

`At the moment, all Fortress objects are immediately shared by default.`

shared⟦T `extends` Any⟧($x : T$): T

isShared(x: Any): Boolean

localize⟦T `extends` Any⟧($x : T$): T
`(*` copy is presently unimplemented. copy[\T extends Any\](x:T): T
`*)`

`trait` Region `extends` Equality⟦Region⟧

    isLocalTo(r: Region): Boolean

`end`

`object` Global `extends` Region `end`

region(a: Any): Region

here(): Region

Equality and ordering

`opr` =(a: Any, b: Any): Boolean

`opr` ≠(a: Any, b: Any): Boolean

`trait` Equality⟦T `extends` Equality⟦T⟧⟧

    `abstract opr` =(`self`, *other* : T): Boolean

`end`

`Total ordering`

`object` LexicographicPartialReduction

        `extends` { MonoidReduction⟦Comparison⟧,

                ReductionWithZeroes⟦Comparison, Comparison⟧ }

    empty(): Comparison

    join(a: Comparison, b: Comparison): Comparison

    isZero(_: Unordered): Boolean

isZero(_: Comparison): Boolean

`end`

`object` LexicographicReduction

`extends` { MonoidReduction⟦TotalComparison⟧,

ReductionWithZeroes⟦TotalComparison, TotalComparison⟧ }

empty(): TotalComparison

join(*a*: TotalComparison, *b*: TotalComparison): TotalComparison

isLeftZero(_: EqualTo): Boolean

isLeftZero(_: TotalComparison): Boolean

`end`

`opr BIG LEXICO`(): BigReduction⟦TotalComparison, TotalComparison⟧

`opr BIG LEXICO`(*g*: Generator⟦TotalComparison⟧): TotalComparison

`trait` Comparison

`extends` { StandardPartialOrder⟦Comparison⟧ }

`comprises` { Unordered, TotalComparison }

`opr` =(`self`, *other*: Comparison): Boolean

`opr LEXICO`(`self`, *other*: Comparison): Comparison

`opr LEXICO`(`self`, *other*: () → Comparison): Comparison

`opr SYMMETRIC_PARTIAL`(`self`, *other*: Comparison): Comparison

`abstract opr INVERSE`(`self`): Comparison

`end`

`Unordered is the outcome of` *a* `CMP` *b* `when` *a* `and` *b* `are partially ordered and no ordering relationship exists between them.`

`object` Unordered `extends` Comparison

`opr` =(`self`, *other*: Unordered): Boolean

`opr` <(`self`, *other*: Comparison): Boolean

`opr INVERSE`(`self`): Comparison

`end`

TotalComparison `is both a partial order (including` Unordered`) and a total order (`TotalComparison `alone). Its method definitions avoid ambiguities between these orderings.`

```
trait TotalComparison
        extends { Comparison, StandardMinMax⟦TotalComparison⟧ }
        comprises { LessThan, EqualTo, GreaterThan }
    opr =(self, other: Comparison): Boolean
    opr CMP(self, other: Unordered): Comparison
    opr ≥(self, other: Unordered): Boolean
    opr ≥(self, other: Comparison): Boolean
    opr ≥(self, other: TotalComparison): Boolean
    opr ≤(self, other: TotalComparison): Boolean
    opr MIN(self, other: TotalComparison): TotalComparison
    opr MAX(self, other: TotalComparison): TotalComparison
    opr MINMAX(self, other: TotalComparison): (TotalComparison, TotalComparison)
    opr LEXICO(self, other: TotalComparison): TotalComparison
    opr LEXICO(self, other: () → TotalComparison): TotalComparison
    opr LEXICO(self, other: () → Comparison): Comparison
    abstract opr INVERSE(self): TotalComparison
end

object LessThan extends TotalComparison
    opr =(self, other: LessThan): Boolean
    opr CMP(self, other: LessThan): TotalComparison
    opr CMP(self, other: TotalComparison): TotalComparison
    opr <(self, other: LessThan): Boolean
    opr <(self, other: TotalComparison): Boolean
    opr SYMMETRIC_PARTIAL(self, other: LessThan): LessThan
    opr SYMMETRIC_PARTIAL(self, other: EqualTo): LessThan
    opr INVERSE(self): TotalComparison
end

object GreaterThan extends TotalComparison
    opr =(self, other: GreaterThan): Boolean
    opr CMP(self, other: GreaterThan): TotalComparison
```

```
    opr CMP(self, other: TotalComparison): TotalComparison
    opr <(self, other: TotalComparison): Boolean
    opr SYMMETRIC_PARTIAL(self, other: GreaterThan): GreaterThan
    opr SYMMETRIC_PARTIAL(self, other: EqualTo): GreaterThan
    opr INVERSE(self): TotalComparison
end

object EqualTo extends TotalComparison
    opr =(self, other: EqualTo): Boolean
    opr CMP(self, other: TotalComparison): TotalComparison
    opr <(self, other: LessThan): Boolean
    opr <(self, other: TotalComparison): Boolean
    opr LEXICO(self, other: TotalComparison): TotalComparison
    opr LEXICO(self, other: () → TotalComparison): TotalComparison
    opr LEXICO(self, other: () → Comparison): Comparison
    opr SYMMETRIC_PARTIAL(self, other: Comparison): Comparison
    opr INVERSE(self): TotalComparison
end

StandardPartialOrder is partial ordering using <,>,≤,≥,=, and CMP. This is
primarily for floating-point values. Minimal complete definition: CMP or {<,=}.

trait StandardPartialOrder⟦T extends StandardPartialOrder⟦T⟧⟧
        extends { Equality⟦T⟧ }
    opr CMP(self, other : T): Comparison
    opr <(self, other : T): Boolean
    opr >(self, other : T): Boolean
    opr =(self, other : T): Boolean
    opr ≤(self, other : T): Boolean
    opr ≥(self, other : T): Boolean
end

StandardMin is the MIN operator; most types that implement MIN will implement a correspondi
total order. It's a separate type to account for the existence of floating point
numbers, for which NaN counts as a bottom that is less than anything else but doesn't
actually participate in the standard total ordering. It is otherwise the case that
a MIN b = a when a ≤ b and that a MIN b = b MIN a.
```

trait StandardMin$[\![T$ **extends** StandardMin$[\![T]\!]]\!]$

excludes { HasRank }

opr MIN(**self**, *other* : T): T

end

StandardMax is the **MAX** operator; most types that implement **MAX** will implement a corresponding total order. It's a separate type to account for the existence of floating point numbers, for which NaN counts as a bottom that is less than anything else but doesn't actually participate in the standard total ordering. It is otherwise the case that a **MAX** $b = a$ when $a \leq b$ and that a **MAX** $b = b$ **MAX** a .

trait StandardMax$[\![T$ **extends** StandardMax$[\![T]\!]]\!]$

excludes { HasRank }

opr MAX(**self**, *other* : T): T

end

StandardMinMax combines MIN and MAX operators, and provides a combined MINMAX operator. This operator returns both its arguments; if equality is possible, self should be the leftmost result. This effectively means that (a **MINMAX** b) stably sorts a and b. In addition, a **MINMAX** $b = (a$ **MIN** b, a **MAX** $b)$ must always hold.

trait StandardMinMax$[\![T$ **extends** StandardMinMax$[\![T]\!]]\!]$

extends { StandardMin$[\![T]\!]$, StandardMax$[\![T]\!]$ }

abstract opr MINMAX(**self**, *other* : T): (T, T)

opr MIN(**self**, *other* : T): T

opr MAX(**self**, *other* : T): T

end

StandardTotalOrder is the usual total order using $<, >, \leq, \geq, =$, and **CMP**. Most values that define a comparison should do so using this. Minimal complete definition: either **CMP** or $<$ (it is advisable to define $=$ in the latter case). As noted above, **MIN** and **MAX** respect the total order and are defined in the obvious way.

trait StandardTotalOrder$[\![T$ **extends** StandardTotalOrder$[\![T]\!]]\!]$

extends { StandardPartialOrder$[\![T]\!]$, StandardMinMax$[\![T]\!]$ }

opr MINMAX(**self**, *other* : T): (T, T)

opr CMP(**self**, *other* : T): TotalComparison

end

Assertions

assert(*flag*: Boolean): ()

assert(*flag*: Boolean, *failMsg*: String): ()

```
This version of assert checks the equality of its first two arguments; if unequal
it includes the remaining arguments in its error indication.
```

assert(x: Any, y: Any, *failMsg*: Any ...): ()

deny(*flag*: Boolean): ()

deny(*flag*: Boolean, *failMsg*: String): ()

```
This version of deny checks the inequality of its first two arguments; if equal
it includes the remaining arguments in its error indication.
```

deny(x: Any, y: Any, *failMsg*: Any ...): ()

shouldRaise⟦Ex `extends` Exception⟧ (*expr*: () → ()): ()

Numeric hierarchy

```
Place holder for exclusions of AdditiveGroup
```

`trait` AnyAdditiveGroup `end`

```
Additive group making use of +.  Must define + and either unary or binary −.
```

`trait` AdditiveGroup⟦T `extends` AdditiveGroup⟦T⟧⟧ `extends` AnyAdditiveGroup

`getter` *zero*(): T

`abstract opr` +(`self`, *other*: T): T

`opr` −(`self`, *other*: T): T

`opr` −(`self`) : T

`end`

```
Place holder for exclusions of MultiplicativeRing
```

`trait` AnyMultiplicativeRing `end`

```
Multiplicative ring using TIMES and juxtaposition.  Define opr TIMES; juxtaposition
is defined in terms of TIMES.
```

`trait` MultiplicativeRing⟦T `extends` MultiplicativeRing⟦T⟧⟧

`extends` { AdditiveGroup⟦T⟧, AnyMultiplicativeRing }

`abstract getter` *one*(): T

```
    abstract opr ×(self, other : T): T
    opr juxtaposition(self, other : T): T
      Exponentiation need only deal with natural exponents.
  abstract opr^{self, other:AnyIntegral}: T
end
trait Number extends { AnyAdditiveGroup, AnyMultiplicativeRing }
       comprises { ℝ64, ℝ32, ℚ, AnyIntegral, IntLiteral }
      The value as a float, the explicit conversion into ℝ64.
  asFloat(self): ℝ64
      Two exact numbers of different types compare as rationals; a float compares
with any number after asFloat, so an exact value equals its nearest float.
  opr =(self, other: Number): Boolean
end
trait ℝ64 extends { Number, StandardPartialOrder⟦ℝ64⟧, StandardMinMax⟦ℝ64⟧,
                        AdditiveGroup⟦ℝ64⟧, MultiplicativeRing⟦ℝ64⟧ }
       excludes { ℚ, AnyIntegral, ℝ32 }
       comprises { Float, FloatLiteral }
    coerce (x: ℤ32)
    coerce (x: ℝ32)
    coerce (x: IntLiteral)
    getter zero(): ℝ64
    getter one(): ℝ64
      returns true if the value is an IEEE NaN
  getter isNaN(): Boolean
      returns true if the value is an IEEE infinity
  getter isInfinite(): Boolean
      returns true if the value is a valid number (not NaN)
  getter isNumber(): Boolean
      returns true if the value is finite
  getter isFinite(): Boolean
      check returns Just(its argument) if it is finite, otherwise Nothing.
  getter check(): Maybe⟦ℝ64⟧
```

check* returns Just(*its argument*) if it is non-NaN, otherwise Nothing.

getter *check**(): Maybe[[$\mathbb{R}64$]]

obtain the raw bits of the IEEE floating-point representation of this value.

getter *rawBits*(): $\mathbb{Z}64$

obtain the sign bit of the IEEE floating-point representation of this value.

getter *signBit*(): $\mathbb{Z}32$

next higher IEEE float

getter *nextUp*(): $\mathbb{R}64$

next lower IEEE float

getter *nextDown*(): $\mathbb{R}64$

opr $=$(self, b: $\mathbb{R}64$): Boolean

opr $<$(self, b: $\mathbb{R}64$): Boolean

opr $\leq$(self, b: $\mathbb{R}64$): Boolean

opr $>$(self, b: $\mathbb{R}64$): Boolean

opr $\geq$(self, b: $\mathbb{R}64$): Boolean

opr CMP(self, b: $\mathbb{R}64$): Comparison

In case of NaN, MIN and MAX return a NaN, otherwise it respects the total order.

opr MIN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr MAX(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr MINMAX(self, b: $\mathbb{R}64$): ($\mathbb{R}64$, $\mathbb{R}64$)

opr $-$(self): $\mathbb{R}64$

opr $+$(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr $-$(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr $\cdot$(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr $\times$(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr *juxtaposition*

(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr $/$(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr $\sqrt{\text{self}}$: $\mathbb{R}64$

opr PLUS_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr MINUS_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr DOT_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr SLASH_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr SQRT_UP(self): $\mathbb{R}64$

opr PLUS_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr MINUS_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr DOT_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr SLASH_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr SQRT_DOWN(self): $\mathbb{R}64$

opr IEEE_PLUS_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_MINUS_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_DOT_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_PLUS_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_MINUS_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_DOT_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_SLASH_DOWN(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr IEEE_SLASH_UP(self, b: $\mathbb{R}64$): $\mathbb{R}64$

opr $|$self$|$: $\mathbb{R}64$

opr$^{\texttt{self},b:\mathbb{R}64}$: $\mathbb{R}64$

opr$^{\texttt{self},b:\text{AnyIntegral}}$: $\mathbb{R}64$

sin(self): $\mathbb{R}64$

cos(self): $\mathbb{R}64$

tan(self): $\mathbb{R}64$

asin(self): $\mathbb{R}64$

acos(self): $\mathbb{R}64$

atan(self): $\mathbb{R}64$

$atan_2$(self, x: Number): $\mathbb{R}64$

log(self): $\mathbb{R}64$

e(self): $\mathbb{R}64$

floor(self): $\mathbb{R}64$

opr $\lfloor$self$\rfloor$: $\mathbb{Z}64$

ceiling(self): $\mathbb{R}64$

`opr` ⌈`self`⌉ : $\mathbb{Z}64$

truncate(`self`): $\mathbb{Z}64$

`MINNUM` and `MAXNUM` return a numeric result where possible (avoiding NaN). Note that `MINNUM` and `MAX` form a lattice with NaN at the top, and that `MAXNUM` and `MIN` form a lattice with NaN at the bottom.

`opr MINNUM`(`self`, *b*: $\mathbb{R}64$): $\mathbb{R}64$

`opr MAXNUM`(`self`, *b*: $\mathbb{R}64$): $\mathbb{R}64$

returns a value of type RR32

narrow(`self`): $\mathbb{R}32$

`end`

Returns the rational p/q such that a <= p/q <= b such that q is minimized, and for this minimal q, p is nearest 0.

simplestRationalBetween(*a*: $\mathbb{Q}$, *b*: $\mathbb{Q}$): $\mathbb{Q}$

`trait` $\mathbb{Q}$ `extends` { Number, StandardPartialOrder⟦$\mathbb{Q}$⟧, StandardMinMax⟦$\mathbb{Q}$⟧,
AdditiveGroup⟦$\mathbb{Q}$⟧, MultiplicativeRing⟦$\mathbb{Q}$⟧ }

`excludes` { $\mathbb{R}64$, AnyIntegral }

`comprises` { ... }

`coerce` (*x*: $\mathbb{Z}32$)

`coerce` (*x*: $\mathbb{Z}64$)

`coerce` (*x*: $\mathbb{N}32$)

`coerce` (*x*: $\mathbb{N}64$)

`coerce` (*x*: $\mathbb{Z}$)

`coerce` (*x*: IntLiteral)

`getter` *zero*(): $\mathbb{Q}$

`getter` *one*(): $\mathbb{Q}$

`getter` *isNaN*(): Boolean

`getter` *isInfinite*(): Boolean

`getter` *isNumber*(): Boolean

`getter` *isFinite*(): Boolean

numerator(`self`): $\mathbb{Z}$

denominator(`self`): $\mathbb{Z}$ (⋆ Ideally would be NN ⋆)

`opr` |`self`| : $\mathbb{Q}$

opr =(self, *other*: ℚ): Boolean

opr ≠(self, *other*: ℚ): Boolean

opr <(self, *other*: ℚ): Boolean

opr ≤(self, *other*: ℚ): Boolean

opr >(self, *other*: ℚ): Boolean

opr ≥(self, *other*: ℚ): Boolean

In case of 0/0, MIN and MAX return 0/0, otherwise it respects the total order.

opr MIN(self, *other*: ℚ): ℚ

opr MAX(self, *other*: ℚ): ℚ

opr MINMAX(self, *other*: ℚ): (ℚ, ℚ)

opr −(self): ℚ

opr +(self, *other*: ℚ): ℚ

opr −(self, *other*: ℚ): ℚ

opr ·(self, *other*: ℚ): ℚ

opr ×(self, *other*: ℚ): ℚ

opr *juxtaposition* (self, *other*: ℚ): ℚ

opr /(self, *other*: ℚ): ℚ

opr$^{\texttt{self}, other:\text{AnyIntegral}}$: ℚ

floor(self): ℤ

opr ⌊self⌋ : ℤ

ceiling(self): ℤ

opr ⌈self⌉ : ℤ

truncate(self): ℤ

round(self): ℤ

opr MINNUM(self, *other*: ℚ): ℚ

opr MAXNUM(self, *other*: ℚ): ℚ

end

trait AnyIntegral extends { Number } comprises { ℤ, ℤ64, ℤ32, ℕ64, ℕ32 }

coerce (x: IntLiteral)

end

not yet: ``comprises Integral⟦I⟧ where ⟦I⟧''

`trait` Integral$[\![I$ `extends` Integral$[\![I]\!]]\!]$ `extends` $\{$ StandardTotalOrder$[\![I]\!]$, MultiplicativeRing$[\![I]\!]$, AnyIntegral $\}$

`getter` $zero()\colon I$

`getter` $one()\colon I$

`opr` $-$(`self`) : I

`opr` $\dot{-}$(`self`) : I

`opr` $=$(`self`, $b\colon I$): Boolean

`opr` $+$(`self`, $b\colon I$) : I

`opr` $\dot{+}$(`self`, $b\colon I$) : I

`opr` $-$(`self`, $b\colon I$) : I

`opr` $\dot{-}$(`self`, $b\colon I$) : I

`opr` $\cdot$(`self`, $b\colon I$) : I

`opr` $\times$(`self`, $b\colon I$) : I

`opr` *juxtaposition*(`self`, $b\colon I$) : I

`opr` $\dot{\times}$(`self`, $b\colon I$) : I

`opr` $\div$(`self`, $b\colon I$) : I

`opr` `REM`(`self`, $b\colon I$) : I

`opr` `MOD`(`self`, $b\colon I$) : I

`opr` `GCD`(`self`, $b\colon I$) : I

`opr` `LCM`(`self`, $b\colon I$) : I

`opr` $|$(`self`, $b\colon I$): Boolean

`opr` `CHOOSE`(`self`, $b\colon I$) : I

`opr` $\mathbb{M}$(`self`, $b\colon I$) : I

`opr` $\mathbb{W}$(`self`, $b\colon I$) : I

`opr` $\underline{\mathbb{W}}$(`self`, $b\colon I$) : I

`opr` `LSHIFT`(`self`, b: AnyIntegral) : I

`opr` `RSHIFT`(`self`, b: AnyIntegral) : I

`opr` $\exists$(`self`) : I

`opr`$^{\texttt{self},b:\text{AnyIntegral}}$: I

`opr` $/$(`self`, *other* : I): $\mathbb{Q}$

numerator(`self`): $\mathbb{Z}$

denominator(`self`): $\mathbb{Z}$

floor(`self`): I

ceiling(`self`): I

truncate(`self`): I

`opr MINNUM`(`self`, *other* : I) : I

`opr MAXNUM`(`self`, *other* : I) : I

odd(`self`): Boolean

even(`self`): Boolean

`end`

`trait` $\mathbb{N}64$ `extends` { AnyIntegral, Integral[[$\mathbb{N}64$]] }

`excludes` { $\mathbb{Z}, \mathbb{N}32$ }

`comprises` { UnsignedLong }

`coerce` (x: $\mathbb{N}32$)

`coerce` (x: IntLiteral)

`opr` |`self`| : $\mathbb{N}64$

`opr` $=$(`self`, b: $\mathbb{N}64$): Boolean

`opr` $<$(`self`, b: $\mathbb{N}64$): Boolean

`opr` $>$(`self`, b: $\mathbb{N}64$): Boolean

`opr` $\geq$(`self`, b: $\mathbb{N}64$): Boolean

`opr` $\leq$(`self`, b: $\mathbb{N}64$): Boolean

`opr CMP`(`self`, b: $\mathbb{N}64$): TotalComparison

`opr MIN`(`self`, *other*: $\mathbb{N}64$): $\mathbb{N}64$

`opr MAX`(`self`, *other*: $\mathbb{N}64$): $\mathbb{N}64$

`opr MINMAX`(`self`, *other*: $\mathbb{N}64$): ($\mathbb{N}64$, $\mathbb{N}64$)

`opr MINNUM`(`self`, *other*: $\mathbb{N}64$): $\mathbb{N}64$

`opr MAXNUM`(`self`, *other*: $\mathbb{N}64$): $\mathbb{N}64$

`opr` $-$(`self`): $\mathbb{N}64$

`opr` $\dot{-}$(`self`): $\mathbb{N}64$

`opr` $+$(`self`, b: $\mathbb{N}64$): $\mathbb{N}64$

`opr` $\dot{+}$(`self`, b: $\mathbb{N}64$): $\mathbb{N}64$

`opr` $-$(`self`, b: $\mathbb{N}64$): $\mathbb{N}64$

`opr` $\dot{-}$(`self`, b: $\mathbb{N}64$): $\mathbb{N}64$

```
    opr ·(self, b: ℕ64): ℕ64
    opr ×(self, b: ℕ64): ℕ64
    opr juxtaposition(self, b: ℕ64): ℕ64
    opr ×̇(self, b: ℕ64): ℕ64
    opr ÷(self, b: ℕ64): ℕ64
    opr REM(self, b: ℕ64): ℕ64
    opr MOD(self, b: ℕ64): ℕ64
    opr GCD(self, b: ℕ64): ℕ64
    opr LCM(self, b: ℕ64): ℕ64
    opr CHOOSE(self, b: ℕ64): ℕ64
    opr ⩓(self, b: ℕ64): ℕ64
    opr ⩔(self, b: ℕ64): ℕ64
    opr ⩒(self, b: ℕ64): ℕ64
    opr LSHIFT(self, b: AnyIntegral): ℕ64
    opr RSHIFT(self, b: AnyIntegral): ℕ64
    opr ¬(self): ℕ64
    opr^self,b:AnyIntegral: ℕ64
    opr^self,b:IntLiteral: ℕ64
    opr /(self, other: ℕ64): ℚ
    numerator(self): ℤ
    narrow(self): ℕ32
    signed(self): ℤ64
end

trait ℤ32 extends { AnyIntegral, Integral⟦ℤ32⟧, Index }
        excludes { ℤ64, ℤ, ℕ64, ℕ32 }
        comprises { Int }
    coerce (x: IntLiteral)
    getter zero(): ℤ32
    getter one(): ℤ32
    getter minimum(): ℤ32
    getter maximum(): ℤ32
```

`opr` $|$`self`$|$: $\mathbb{Z}32$

`opr` $=$(`self`, b: $\mathbb{Z}32$): Boolean

`opr` $<$(`self`, b: $\mathbb{Z}32$): Boolean

`opr` $>$(`self`, b: $\mathbb{Z}32$): Boolean

`opr` $\geq$(`self`, b: $\mathbb{Z}32$): Boolean

`opr` $\leq$(`self`, b: $\mathbb{Z}32$): Boolean

`opr` `CMP`(`self`, b: $\mathbb{Z}32$): TotalComparison

`opr` `MIN`(`self`, *other*: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `MAX`(`self`, *other*: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `MINMAX`(`self`, *other*: $\mathbb{Z}32$): ($\mathbb{Z}32$, $\mathbb{Z}32$)

`opr` `MINNUM`(`self`, *other*: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `MAXNUM`(`self`, *other*: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $-$(`self`): $\mathbb{Z}32$

`opr` $\dot{-}$(`self`): $\mathbb{Z}32$

`opr` $+$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\dot{+}$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $-$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\dot{-}$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\cdot$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\times$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` *juxtaposition*(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\dot{\times}$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\div$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `REM`(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `MOD`(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `GCD`(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `LCM`(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` `CHOOSE`(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\wedge\!\backslash$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\vee$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr` $\underline{\vee}$(`self`, b: $\mathbb{Z}32$): $\mathbb{Z}32$

`opr LSHIFT(self, b`: AnyIntegral): $\mathbb{Z}32$

`opr RSHIFT(self, b`: AnyIntegral): $\mathbb{Z}32$

`opr` $\exists$`(self)`: $\mathbb{Z}32$

`opr`$^{\texttt{self},b:\text{IntLiteral}}$: $\mathbb{Z}32$

`opr` /(`self`, *other*: $\mathbb{Z}32$): $\mathbb{Q}$

numerator(`self`): $\mathbb{Z}$

widen(`self`): $\mathbb{Z}64$

partitionL(`self`): $\mathbb{Z}32$

unsigned(`self`): $\mathbb{N}32$

narrow(`self`): $\mathbb{Z}32$

big(`self`): $\mathbb{Z}$

`end`

`trait` $\mathbb{Z}64$ `extends` { AnyIntegral, Integral$[\![\mathbb{Z}64]\!]$ }

`excludes` { $\mathbb{Z}, \mathbb{N}64, \mathbb{N}32$ }

`comprises` { Long }

`coerce` (x: $\mathbb{Z}32$)

`coerce` (x: $\mathbb{N}32$)

`coerce` (x: IntLiteral)

`getter` *zero*(): $\mathbb{Z}64$

`getter` *one*(): $\mathbb{Z}64$

`getter` *minimum*(): $\mathbb{Z}64$

`getter` *maximum*(): $\mathbb{Z}64$

`opr` |`self`|: $\mathbb{Z}64$

`opr` =(`self`, b: $\mathbb{Z}64$): Boolean

`opr` <(`self`, b: $\mathbb{Z}64$): Boolean

`opr` >(`self`, b: $\mathbb{Z}64$): Boolean

`opr` $\geq$(`self`, b: $\mathbb{Z}64$): Boolean

`opr` $\leq$(`self`, b: $\mathbb{Z}64$): Boolean

`opr CMP(self, b`: $\mathbb{Z}64$): TotalComparison

`opr MIN(self`, *other*: $\mathbb{Z}64$): $\mathbb{Z}64$

`opr MAX(self, other: ℤ64): ℤ64`

`opr MINMAX(self, other: ℤ64): (ℤ64, ℤ64)`

`opr MINNUM(self, other: ℤ64): ℤ64`

`opr MAXNUM(self, other: ℤ64): ℤ64`

`opr −(self): ℤ64`

`opr ∸(self): ℤ64`

`opr +(self, b: ℤ64): ℤ64`

`opr ∔(self, b: ℤ64): ℤ64`

`opr −(self, b: ℤ64): ℤ64`

`opr ∸(self, b: ℤ64): ℤ64`

`opr ·(self, b: ℤ64): ℤ64`

`opr ×(self, b: ℤ64): ℤ64`

`opr` *juxtaposition*`(self, b: ℤ64): ℤ64`

`opr ×̇(self, b: ℤ64): ℤ64`

`opr ÷(self, b: ℤ64): ℤ64`

`opr REM(self, b: ℤ64): ℤ64`

`opr MOD(self, b: ℤ64): ℤ64`

`opr GCD(self, b: ℤ64): ℤ64`

`opr LCM(self, b: ℤ64): ℤ64`

`opr CHOOSE(self, b: ℤ64): ℤ64`

`opr ⩓(self, b: ℤ64): ℤ64`

`opr ⩔(self, b: ℤ64): ℤ64`

`opr ⩒(self, b: ℤ64): ℤ64`

`opr LSHIFT(self, b: AnyIntegral): ℤ64`

`opr RSHIFT(self, b: AnyIntegral): ℤ64`

`opr ⫬(self): ℤ64`

`opr`$^{\texttt{self},b:\text{IntLiteral}}$`: ℤ64`

`opr /(self, other: ℤ64): ℚ`

numerator`(self): ℤ`

narrow`(self): ℤ32`

widen`(self): ℤ64`

unsigned(`self`): ℕ64

big(`self`): ℤ

`end`

`trait` ℤ `extends` { AnyIntegral, Integral⟦ℤ⟧ }

`excludes` { ℕ64, ℕ32 }

`comprises` { BigNum }

`coerce` (*x*: ℤ32)

`coerce` (*x*: ℤ64)

`coerce` (*x*: ℕ32)

`coerce` (*x*: ℕ64)

`coerce` (*x*: IntLiteral)

`opr` =(`self`, *b*: ℤ): Boolean

`opr` <(`self`, *other*: ℤ): Boolean

`opr` ≤(`self`, *other*: ℤ): Boolean

`opr` >(`self`, *other*: ℤ): Boolean

`opr` ≥(`self`, *other*: ℤ): Boolean

`opr CMP`(`self`, *other*: ℤ): TotalComparison

`opr MIN`(`self`, *other*: ℤ): ℤ

`opr MAX`(`self`, *other*: ℤ): ℤ

`opr MINMAX`(`self`, *other*: ℤ): (ℤ, ℤ)

`opr MINNUM`(`self`, *other*: ℤ): ℤ

`opr MAXNUM`(`self`, *other*: ℤ): ℤ

`opr` −(`self`): ℤ

`opr` $\dot{-}$(`self`): ℤ

`opr` +(`self`, *b*: ℤ): ℤ

`opr` $\dot{+}$(`self`, *b*: ℤ): ℤ

`opr` −(`self`, *b*: ℤ): ℤ

`opr` $\dot{-}$(`self`, *b*: ℤ): ℤ

`opr` ·(`self`, *b*: ℤ): ℤ

`opr` *juxtaposition*(`self`, *b*: ℤ): ℤ

`opr` ×(`self`, *b*: ℤ): ℤ

`opr` $\dot{\times}$(`self`, b: $\mathbb{Z}$): $\mathbb{Z}$

`opr` /(`self`, *other*: $\mathbb{Z}$): $\mathbb{Q}$

`opr`$^{\texttt{self},b:\text{IntLiteral}}$: $\mathbb{Z}$

numerator(`self`): $\mathbb{Z}$

narrow(`self`): $\mathbb{Z}32$

widen(`self`): $\mathbb{Z}64$

shift(`self`, k: AnyIntegral): $\mathbb{Z}$

`end`

Generator support

`The type` Contains$[\![T]\!]$ `"contains" objects of type` T`, in the sense that we can check whether a` T `is a member of a` Contains$[\![T]\!]$ `using` `opr` $\in$ `and its ken, or using a` `case` `expression.`

`trait` Contains$[\![T]\!]$

`opr` $\in$ `indicates whether` `self` `contains` *elt*

`abstract opr` $\in$(*elt* : T, `self`): Boolean

`The` `MATCH` `operator is used (as a temporary hack) by case when the match clause is a` Contains$[\![T]\!]$`. We must choose either` $=$`,` `SEQV`, `or` $\in$ `as appropriate, depending on the type of the LHS.`

`opr MATCH`(x: Any, `self`): Boolean

`end`

$x \notin g$ `is simply` $\neg(x \in g)$.

`opr` $\notin[\![E]\!]\big(x\colon E, \mathit{this}\colon \text{Contains}[\![E]\!]\big)\colon \text{Boolean}$

g `NI` x `is equivalent to` $x \in g$

`opr NI`$[\![E]\!]\big(\mathit{this}\colon \text{Contains}[\![E]\!], x : E\big)\colon \text{Boolean}$

$g \not\ni x$ `is equivalent to` $x \notin g$

`opr` $\not\ni[\![E]\!]\big(\mathit{this}\colon \text{Contains}[\![E]\!], x : E\big)\colon \text{Boolean}$

`We say an object which extends` Generator$[\![T]\!]$ `''generates objects of type` T`.''`

`Generators are used to express iteration in Fortress. Every generated expression in Fortress (eg. for loop and comprehension) is desugared into calls to methods of Generator, chiefly the` *generate* `method.`

`Every generator has a notion of a ''natural order'' (which by default is unspecified), which describes the ordering of reduction operations, and also describes the order`

in which elements are produced by the sequential version of the same generator (given by the *seq*(**self**) method). The default implementation of *seq*(**self**) guarantees that these orders will match.

Note in particular that the natural order of a Generator must be consistent; that is, if a **SEQV** b then a and b must yield **SEQV** elements in the same natural order. However, note that unless a type specifically documents otherwise, no particular element ordering is guaranteed, nor is it necessary to guarantee that $a = b$ have the same natural order when equality is defined.

Note that more complex derived generators are specified further down in the definition of Generator. These have the same notions of natural order and by default are defined in terms of the *generate* method.

Minimal complete definition of a Generator is the *generate* method.

trait Generator⟦E⟧ **extends** { Contains⟦E⟧ }

excludes { Number }

reverse returns a generator which generates the same objects in reverse order

getter *reverse*(): Generator⟦E⟧

generate is the core of Generator. It generates elements of type E and passes them to the *body* function. This generation can occur using any mixture of serial and parallel execution desired by the author of the generator; by default uses of a generator must assume every call to *body* occurs in parallel.

The results of generation are combined using the reduction object R, which specifies a monoidal operation (associative and with an identity). Body results must be combined together following the natural order of the generator. The author of the generator is free to use the identity element anywhere desired in this computation, and to group reductions in any way desired; if no elements are generated, the identity must be returned.

abstract *generate*⟦R⟧(r: Reduction⟦R⟧, *body*: $E \to R$): R

Transforming generators into new generators

map applies a function f to each element generated and yields the result. The resulting generator must have the same ordering and cross product properties as the generator from which it is derived.

map⟦G⟧(f: $E \to G$): Generator⟦G⟧

seq produces a sequential version of the same generator, in which elements are produced in the generator's natural order.

seq(**self**): SequentialGenerator⟦E⟧

Nesting of two generators; the innermost is data-dependent upon the outer one. This is specifically designed to be overloaded so that the combined generators have properties appropriate to the pairing. Because of the data dependency, the natural order of the nesting is the natural order of the inner generators, in the natural order the outer nesting produces them. So, for example, if we write:

$\%(0 \# 3).nest⟦\mathbb{Z}32⟧(\texttt{fn}\ (n{:}\ \mathbb{Z}32){:}\ \text{Generator⟦}\mathbb{Z}32\text{⟧} \Rightarrow ((n\ 100) \# 4))$

then the natural order is 0,1,2,3,100,101,102,103,200,201,202,203.

$nest[\![G]\!](f\colon E \to \text{Generator}[\![G]\!])\colon \text{Generator}[\![G]\!]$

Filtering data from a generator. Only elements that satisfy the predicate p are retained. Natural order and cross product properties are otherwise preserved.

$filter(f\colon E \to \text{Condition}[\![()]\!])\colon \text{Generator}[\![E]\!]$

Cross product of two generators. This is specifically designed to be overloaded, such that pairs of independent generators can be combined to produce a generator which possibly interleaves the iteration spaces of the two generators. For example, we might combine

$\%(0 \# 16).cross(0 \# 32)$

such that it first splits the second range in half, then the first range in half, then the second, and so forth.

Consider a grid for which the rows are labeled from top to bottom with the elements of a in natural order, and the columns are labeled from left to right with the elements of g in natural order. Each point in the grid corresponds to a pair (a, b) that must be generated by **self**.$cross(g)$. In the natural order of the cross product, an element must occur after those that lie above and to the left of it in the grid. By default the natural order of the cross product is left-to-right, top to bottom. Programmers must not rely on the default order, except that cross products involving one or more sequential generators are always performed in the default order. Note that this means that the following have the same natural order:

$\%seq(a).cross(b)$

$\%a.cross\big(seq(b)\big)$

$\%seq(a).cross\big(seq(b)\big)$

But $seq\big(a.cross(b)\big)$ may have a different natural order.

$cross[\![G]\!]\big(g\colon \text{Generator}[\![G]\!]\big)\colon \text{Generator}\big[\![(E, G)]\!\big]$

Derived generation operations

mapReduce is equivalent to *generate*, but takes an explicit *join* and *zero* which can have any type. It still assumes *join* is associative and that *zero* is the identity of *join*.

$mapReduce[\![R]\!]\big(body\colon E \to R, join : (R, R) \to R, zero : R\big)\colon R$

reduce works much like *generate* or *mapReduce*, but has no body expression.

$reduce\big(j : (E, E) \to E, z : E\big) : E$

$reduce\big(r\colon \text{Reduction}[\![E]\!]\big) : E$

loop is a version of *generate* which discards the () results of the body computation. It can be used to translate reduction-variable-free **for** loops.

$loop\big(f : E \to ()\big)\colon ()$

$x \in$ **self** holds if x is generated by this generator. By default this is implemented using the naive $O(n)$ algorithm.

`opr` $\in(x : E, \texttt{self}) : \text{Boolean}$

|`self`| is the number of elements generated by this generator. By default this is implemented using the naive $O(n)$ algorithm, which runs the generator; a generator that knows its size overrides it.

`opr` $|\texttt{self}| : \mathbb{Z}32$

`end`

The following stubs exist as a temporary workaround to shortcomings in interpreter type inference, and are intended for use by reduction desugaring.

$__generate[\![E, R]\!]\big(g : \text{Generator}[\![E]\!], r : \text{Reduction}[\![R]\!], b : E \to R\big) : R$

$__filter[\![E]\!]\big(g : \text{Generator}[\![E]\!], p : E \to \text{Condition}[\![()]\!]\big) : \text{Generator}[\![E]\!]$

$__bigOperatorSugar[\![I, O, R, L]\!]\big(o : \text{BigOperator}[\![I, O, R, L]\!], g : \text{Generator}[\![I]\!]\big) : O$

$__bigOperator[\![I, O, R, L]\!]\big(o : \text{BigOperator}[\![I, O, R, L]\!], desugaredClauses : (\text{Reduction}[\![L]\!], I \to L) \to L\big) : O$

Application of two nested BIG operators, possibly with fusion. This covers only comprehensions of the form:

$$\%\ \underset{xs \leftarrow expro}{\mathsf{OUTER}} \left(\underset{x \leftarrow xs}{\mathtt{INNER}}\ expri \right)$$

The desugarer extracts comprehensions of this form from more complex nests of comprehensio using a combination of splitting:

$$\%\ \mathsf{OP}[\, gs_1, gs_2 \,]\ expr = \mathsf{OP}[\, gs_1 \,]\ \left(\mathsf{OP}[\, gs_2 \,]\ expr \right)$$

and filter squeezing:

$$\%\ \underset{\substack{xs \leftarrow g \\ p(xs)}}{\mathsf{OP}}\ expr = \underset{xs \leftarrow g.filter(p)}{\mathsf{OP}}\ expr$$

There are some big caveats to this explanation in practice. Most important is that we don't unlift and lift or do input/output conversion except where neccessary, so splitting skips these operations in between the inner and outer comprehension.

(*

__bigOperator2[\I0,O0,R0,L0,I1,O1 extends I0,R1,L1,E,F extends Generator[\E\]\] (outer:BigOperator[\I0,O0,R0,L0\], inner:BigOperator[\I1,O1,R1,L1\], gg: Generator[\F\], innerBody: E-¿I1):O0

*)

(*

__bigOperator2[\I0,O0,R0,L0,I1,O1 extends I0,R1,L1,E,F extends Generator[\E\]\] (outer:Comprehension[\I0,O0,R0,L0\], inner:Comprehension[\I1,O1,R1,L1\], gg: Generator[\F\], innerBody: E-¿I1):O0

*)

$__bigOperator2\ [\![\text{I0}, \text{O0}, \text{R0}, \text{L0}, \text{I1}, \text{O1}, \text{R1}, \text{L1}, E, F\ \texttt{extends}\ \text{Generator}[\![E]\!]]\!]$

(*outer*: BigOperator⟦I0, O0, R0, L0⟧,

inner: BigOperator⟦I1, O1, R1, L1⟧,

gg: Generator⟦*F*⟧,

innerBody: $E \to$ I1): O0

(* Not currently used for desugaring, but will be used in future. __nest[\E1,E2\](g:Generator[\E1\], f:E1-¿Generator[\E2\]):Generator[\E2\]

__map[\E,R\](g:Generator[\E\], f:E-¿R): Generator[\R\]

*)

Used in desugaring binding **if**

__*cond*⟦*E*, *R*⟧(*c*: Condition⟦*E*⟧, *t* : $E \to R$, *e*: () $\to R$): *R*

__*cond*⟦*E*⟧(*c*: Condition⟦*E*⟧, *t* : $E \to$ ()): ()

Used in desugaring binding **while**

__*whileCond*⟦*E*⟧(*c*: Condition⟦*E*⟧, *b* : $E \to$ ()): Boolean

trait SequentialGenerator⟦*E*⟧ **extends** { Generator⟦*E*⟧ }

excludes { HasRank }

seq(**self**): SequentialGenerator⟦*E*⟧

map⟦*G*⟧(*f*: $E \to G$): SequentialGenerator⟦*G*⟧

nest⟦*G*⟧(*f*: $E \to$ Generator⟦*G*⟧): Generator⟦*G*⟧

cross⟦*F*⟧(*g*: Generator⟦*F*⟧): Generator⟦(*E*, *F*)⟧

end

A Condition is a Generator that generates 0 or 1 element. Conditions can be used as nullary comprehension generators or as predicates in an if expression.

trait Condition⟦*E*⟧ **extends** SequentialGenerator⟦*E*⟧

getter *isEmpty*(): Boolean

getter *nonEmpty*(): Boolean

getter *holds*(): Boolean

getter *size*(): ℤ32

getter *get*(): *E* **throws** NotFound

getter *bounds*(): CompactFullRange⟦ℤ32⟧

getter *indices*(): CompactFullRange⟦ℤ32⟧

getter *indexValuePairs*(): Generator⟦(ℤ32, *E*)⟧

`opr` |`self`|: $\mathbb{Z}32$

`opr` $[i\colon \mathbb{Z}32]\colon E$ `throws` NotFound

cond is the core of a Condition, in terms of which all other methods are written. When *holds*(), *t* is invoked with the value of *get*(); otherwise *e* is called.

`abstract` *cond*$[\![G]\!]\big(t\colon E \to G, e\colon () \to G\big)\colon G$

For a Condition, these methods run eagerly.

generate$[\![G]\!]\big(r\colon \text{Reduction}[\![G]\!], \mathit{body}\colon E \to G\big)\colon G$

map$[\![G]\!](f\colon E \to G)\colon \text{Condition}[\![G]\!]$

ivmap$[\![G]\!]\big(f\colon (\mathbb{Z}32, E) \to G\big)\colon \text{Generator}[\![G]\!]$

nest$[\![G]\!]\big(f\colon E \to \text{Generator}[\![G]\!]\big)\colon \text{Generator}[\![G]\!]$

cross$[\![G]\!]\big(g\colon \text{Generator}[\![G]\!]\big)\colon \text{Generator}\,[\![(E, G)]\!]$

mapReduce$[\![R]\!]\big(\mathit{body}\colon E \to R, _\colon (R, R) \to R, \mathit{zero}\colon R\big)\colon R$

reduce$\big(_\colon (E, E) \to E, z\colon E\big)\colon E$

reduce$\big(r\colon \text{Reduction}[\![E]\!]\big)\colon E$

loop$\big(f\colon E \to ()\big)\colon ()$

`opr` $\in(x\colon E,$ `self`$)\colon$ Boolean

`end`

Conjunction and disjunction of condition functions

`opr ANDCOND`$[\![I]\!]\big(p_1\colon I \to \text{Condition}[\![()]\!], p_2\colon I \to \text{Condition}[\![()]\!]\big)\colon I \to \text{Condition}\,[\![()]\!]$

Conjunction and disjunction of condition functions

`opr ORCOND`$[\![I]\!]\big(p_1\colon I \to \text{Condition}[\![()]\!], p_2\colon I \to \text{Condition}[\![()]\!]\big)\colon I \to \text{Condition}\,[\![()]\!]$

sequential$[\![T]\!]\big(g\colon \text{Generator}[\![T]\!]\big)\colon \text{SequentialGenerator}[\![T]\!]$

29.1.1 The Maybe type

This trait makes excludes work without where clauses, and allows opr = to remain non-parametric.

`value trait` AnyMaybe `extends` { AnyUniqueItem } `excludes` Number

not yet: ``` ``comprises`` Maybe$[\![T]\!]$ `where` $[\![T]\!]$ '' ```

`abstract getter` *holds*()$\colon$ Boolean

`opr` =(`self`, *other*: AnyMaybe)$\colon$ Boolean

`end`

just(*t*: Any): AnyMaybe

Maybe represents either Nothing or a single element of type T (Just⟦T⟧), which may be retrieved by calling *get*. An object of type Maybe⟦T⟧ can be used as a generator; it is either empty (Nothing) or generates the single element yielded by *get*, so there is no issue of canonical order or parallelism.

Thus, Just⟦T⟧ can be used as a single-element generator, and Nothing can be used as an empty generator.

value trait Maybe⟦T⟧

extends { AnyMaybe, Condition⟦T⟧, ZeroIndexed⟦T⟧, UniqueItem⟦T⟧ }

comprises { Nothing⟦T⟧, Just⟦T⟧ }

opr ⊓(**self**, *o*: Maybe⟦T⟧): Maybe⟦T⟧

opr |**self**| : ℤ32

opr [*i*: ℤ32] : T **throws** NotFound

map⟦G⟧(*f*: $T \to G$): Maybe⟦G⟧

ivmap⟦G⟧(*f*: (ℤ32, T) → G): Maybe⟦G⟧

end

value object Just⟦T⟧(*x* : T) **extends** Maybe⟦T⟧

getter *size*(): ℤ32

getter *holds*(): Boolean

getter *get*(): T

opr |**self**| : ℤ32

getDefault(_ : T): T

cond⟦R⟧(*t* : $T \to R$, _: () → R): R

generate⟦R⟧(_: Reduction⟦R⟧, *m* : $T \to R$): R

opr [*i*: ℤ32] : T

opr [*r*: Range⟦ℤ32⟧] : Maybe⟦T⟧

map⟦G⟧(*f*: $T \to G$): Just⟦G⟧

cross⟦G⟧(*g*: Generator⟦G⟧): Generator⟦(T, G)⟧

mapReduce⟦R⟧(*m*: $T \to R$, _ : (R, R) → R, _ : R): R

reduce(_ : (T, T) → T, _ : T) : T

reduce(_: Reduction⟦T⟧) : T

loop(*f* : T → ()): ()

`opr` $=\big($`self`$, o\colon \mathrm{Just}[\![T]\!]\big)\colon \mathrm{Boolean}$

`opr` $\sqcap\big($`self`$, o\colon \mathrm{Maybe}[\![T]\!]\big)\colon \mathrm{Maybe}[\![T]\!]$

`opr` $\sqcap\big($`self`$, o\colon \mathrm{UniqueItem}[\![T]\!]\big)\colon \mathrm{Maybe}[\![T]\!]$

`opr` $\sqcup\big($`self`$, o\colon \mathrm{UniqueItem}[\![T]\!]\big)\colon \mathrm{UniqueItem}[\![T]\!]$

unique(`self`)$\colon \mathrm{Maybe}[\![T]\!]$

`end`

Nothing `will become a non-parametric singleton when we get where clauses working.`

`value object` Nothing$[\![T]\!]$ `extends` Maybe$[\![T]\!]$

`getter` *size*$()\colon \mathbb{Z}32$

`getter` *holds*$()\colon$ Boolean

`getter` *get*$()\colon T$

`opr` $|$`self`$|\colon \mathbb{Z}32$

getDefault$(t : T) : T$

cond$[\![R]\!]\big(t : T \to R, _ \colon () \to R\big)\colon R$

generate$[\![R]\!]\big(r\colon \mathrm{Reduction}[\![R]\!], _ : T \to R\big)\colon R$

`opr` $[_\colon \mathbb{Z}32]\colon T$

`opr` $\big[r\colon \mathrm{Range}[\![\mathbb{Z}32]\!]\big]\colon \mathrm{Nothing}[\![T]\!]$

map$[\![G]\!](f\colon T \to G)\colon \mathrm{Nothing}[\![G]\!]$

cross$[\![G]\!]\big(g\colon \mathrm{Generator}[\![G]\!]\big)\colon \mathrm{Generator}\,[\![(T, G)]\!]$

mapReduce$[\![R]\!]\big(_\colon T \to R, _ : (R, R) \to R, z : R\big)\colon R$

reduce$\big(_ : (T, T) \to T, z : T\big) : T$

reduce$\big(r\colon \mathrm{Reduction}[\![T]\!]\big) : T$

loop$\big(f : T \to ()\big)\colon ()$

`opr` $=\big($`self`$, _\colon \mathrm{Nothing}[\![T]\!]\big)\colon \mathrm{Boolean}$

`opr` $\sqcap\big($`self`$, o\colon \mathrm{Maybe}[\![T]\!]\big)\colon \mathrm{Nothing}[\![T]\!]$

`opr` $\sqcap\big($`self`$, o\colon \mathrm{UniqueItem}[\![T]\!]\big)\colon \mathrm{Nothing}[\![T]\!]$

`opr` $\sqcup\big($`self`$, o\colon \mathrm{UniqueItem}[\![T]\!]\big)\colon \mathrm{UniqueItem}[\![T]\!]$

`end`

`value trait` AnyUniqueItem `extends` Equality$[\![$AnyUniqueItem$]\!]$ `excludes` Number

`getter` *holds*$() : \mathrm{Boolean}$

```
    opr =(self, other: AnyUniqueItem): Boolean
end

The type UniqueItem⟦T⟧ extends Maybe⟦T⟧ from a semilattice to a lattice by adjoining
a top element NotUnique⟦T⟧. An object of type UniqueItem⟦T⟧ can be used as a generator;
it is either empty (Nothing or NotUnique) or generates the single element yielded
by get, so there is no issue of canonical order or parallelism.

value trait UniqueItem⟦T⟧
        extends { AnyUniqueItem, Condition⟦T⟧ }
        comprises { NotUnique⟦T⟧, Maybe⟦T⟧ }
    opr ⊓(self, o: UniqueItem⟦T⟧): UniqueItem⟦T⟧
    opr ⊔(self, o: UniqueItem⟦T⟧): UniqueItem⟦T⟧
    unique(self): Maybe⟦T⟧
end

value object NotUnique⟦T⟧ extends UniqueItem⟦T⟧
    getter size(): ℤ32
    getter holds(): Boolean
    getter get(): T
    getter asString(): String
    opr |self|: ℤ32
    getDefault(t: T): T
    cond⟦R⟧(_: T → R, e: () → R): R
    generate⟦R⟧(r: Reduction⟦R⟧, _: T → R): R
    opr [_: ℤ32]: T
    map⟦G⟧(f: T → G): NotUnique⟦G⟧
    cross⟦G⟧(g: Generator⟦G⟧): Generator⟦(T, G)⟧
    mapReduce⟦R⟧(_: T → R, _: (R, R) → R, z: R): R
    reduce(_: (T, T) → T, z: T): T
    reduce(r: Reduction⟦T⟧): T
    loop(f: T → ()): ()
    opr ∈(x: T, self): Boolean
    opr =(self, _: NotUnique⟦T⟧): Boolean
```

```
    opr ⊓(self, o: UniqueItem⟦T⟧): UniqueItem⟦T⟧
    opr ⊔(self, o: UniqueItem⟦T⟧): NotUnique⟦T⟧
end

object UniqueItemMeetReduction⟦T⟧
        extends { CommutativeMonoidReduction⟦UniqueItem⟦T⟧⟧,
                  ReductionWithZeroes⟦UniqueItem⟦T⟧, UniqueItem⟦T⟧⟧ }
    getter asString(): String
    empty(): UniqueItem⟦T⟧
    join(a: UniqueItem⟦T⟧, b: UniqueItem⟦T⟧): UniqueItem⟦T⟧
    isZero(a: UniqueItem⟦T⟧): Boolean
end

opr BIG ⊓⟦T⟧(): BigReduction⟦UniqueItem⟦T⟧, UniqueItem⟦T⟧⟧

opr BIG ⊓⟦T⟧(g: Generator⟦UniqueItem⟦T⟧⟧): UniqueItem⟦T⟧

object UniqueItemJoinReduction⟦T⟧
        extends { CommutativeMonoidReduction⟦UniqueItem⟦T⟧⟧,
                  ReductionWithZeroes⟦UniqueItem⟦T⟧, UniqueItem⟦T⟧⟧ }
    getter asString(): String
    empty(): UniqueItem⟦T⟧
    join(a: UniqueItem⟦T⟧, b: UniqueItem⟦T⟧): UniqueItem⟦T⟧
    isZero(a: UniqueItem⟦T⟧): Boolean
end

opr BIG ⊔⟦T⟧(): BigReduction⟦UniqueItem⟦T⟧, UniqueItem⟦T⟧⟧

opr BIG ⊔⟦T⟧(g: Generator⟦UniqueItem⟦T⟧⟧): UniqueItem⟦T⟧
```

Exception hierarchy

```
trait Exception comprises { UncheckedException, CheckedException }
end

(* Exceptions which are not checked *)

trait UncheckedException extends Exception excludes CheckedException
```

```
end

object FailCalled(s: String) extends UncheckedException

end

object DivisionByZero extends UncheckedException

end

object UnpastingError extends UncheckedException

end

object CallerViolation extends UncheckedException

end

object CalleeViolation extends UncheckedException

end

object LabelException extends UncheckedException

end

object TestFailure extends UncheckedException

end

object ContractHierarchyViolation extends UncheckedException

end

object NoEqualityOnFunctions extends UncheckedException

end

object InvalidRange extends UncheckedException

end

object ForbiddenException(chain : Exception) extends UncheckedException

end

(* Should this be called "IndexNotFound" instead? *)

object NotFound extends UncheckedException

end

object IndexOutOfBounds[I](range: Range[I], index : I) extends UncheckedException

end
```

```
object EmptyReduction extends UncheckedException
end

object NegativeLength extends UncheckedException
end

object IntegerOverflow extends UncheckedException
end

object RationalComparisonError extends UncheckedException
end

object FloatingComparisonError extends UncheckedException
end

object MatchFailure extends UncheckedException
end

(* Checked Exceptions *)

trait CheckedException extends Exception excludes UncheckedException
end

object CastError extends CheckedException
end

object IOFailure extends CheckedException
end

(* SetsNotDisjoint? *)
object DisjointUnionError extends CheckedException
end

object APIMissing extends CheckedException
end

object APINameCollision extends CheckedException
end

object ExportedAPIMissing extends CheckedException
end
```

```
object HiddenAPIMissing extends CheckedException
end

object TryAtomicFailure extends CheckedException
end

(* Should take a spawned thread as an argument *)
object AtomicSpawnSynchronization extends {UncheckedException}
end
```

Array support

```
trait HasRank extends Equality⟦HasRank⟧ excludes { Number, AnyMaybe }
   not yet:  ``comprises Array⟦T, E, I⟧ where ⟦T, E, I⟧{ T extends Array⟦T, E, I⟧ }''
 abstract rank(): ℤ32
 opr =(self, other: HasRank): Boolean
end

(* Declared Rank-n-ness *)
trait Rank⟦nat n⟧ extends HasRank
  rank(): ℤ32
end

Potemkin exclusion traits.  Really we just want to say that ``Rank⟦n⟧ excludes Rank⟦m⟧ where { m
but we cannot yet.

trait Rank1 extends { Rank⟦1⟧} excludes { Rank2, Rank3, Number, String }
end

trait Rank2 extends { Rank⟦2⟧} excludes { Rank3, Number, String }
end

trait Rank3 extends { Rank⟦3⟧} excludes { Number, String }
end

The trait Indexed_i⟦n⟧ indicates that something has an i^th dimension of size n.  In
general, anything which extends Indexed_i must also extend Indexed_j for j < i.
```

trait Indexed1⟦**nat** n⟧ **end**

trait Indexed2⟦**nat** n⟧ **end**

trait Indexed3⟦**nat** n⟧ **end**

Index is the type of what a subscript takes, apart from the ranges, which exclude it: $\mathbb{Z}32$, and every tuple type whose element types are all Index types. It extends Any, not Object, so a tuple of indices is an Index and no Object.

trait Index **extends** Any **end**

The indexed trait indicates that an object of type T can be indexed using type I to obtain elements with type E.

An object i that is an instance of Indexed defines three basic things:

The indexing operator **opr** [], which must be defined for every instance of the type.

A suite of generators: *i.indices* generates the index space of the array. i itself generates the values contained at those indices. *i.indexValuePairs* yields pairs of $(index, val)$. All of these share the same natural order. It is necessary to define one of *indices*() and *indexValuePairs*(), in addition to *generate*() (but the latter requirement can be dispensed by instead extending DelegatedIndexed).

A set of utility functions, *assign*, *fill*, and *copy*. Only *fill* and *copy* need to be defined.

trait Indexed⟦E, I **extends** Index⟧ **extends** Generator⟦E⟧

isEmpty() indicates whether there are any valid indices. It is defined as $|\mathbf{self}| = 0$.

getter *isEmpty*(): Boolean

getter *nonEmpty*(): Boolean

size() is deprecated; use $|\mathbf{self}|$ instead.

abstract getter *size*(): $\mathbb{Z}32$

bounds() yields a range of indices that are valid for the indexed generator.

abstract getter *bounds*(): CompactFullRange⟦I⟧

indexValuePairs() generates the elements of the indexed object (exactly those elements that are generated by the object itself), but each element is paired with its index. When we obtain (i, v) from *indexValuePairs*() we know that:

- $\mathbf{self}[i] = v$
- the i are distinct and $i \in$ *bounds*()
- stripping away the i yields exactly the results of $v \leftarrow \mathbf{self}$

This generator attempts to follow the structure of the underlying object as closely as possible.

getter *indexValuePairs*(): Indexed⟦$(I, E), I$⟧

reverse gives us an indexed object which is back-to-front: the reversed object from the lower bound to the upper bound is the original object from upper to lower

`getter` *reverse*(): Indexed⟦E, I⟧

indices() yields the indices corresponding to the elements of the indexed object---it corresponds to the index component of *indexValuePairs*(). This may in general be a subset of all the valid indices represented by *bounds*(). This generator attempts to follow the structure of the underlying object as closely as possible.

`getter` *indices*(): Generator⟦I⟧

|`self`| indicates the number of distinct valid indices that may be passed to indexing operations.

`abstract opr` |`self`|: $\mathbb{Z}32$

Indexing. $i \in$ *bounds*() must hold.

`abstract opr` $[i : I] : E$

Indexing by ranges. The results are 0-based when the underlying index type has a notion of 0. This ensures consistency of behavior between types such as vectors that ``only'' support 0-indexing and types such as arrays that permit other choices of lower bounds. The easiest way to write the index by ranges operation for an instance of Indexed is to take advantage of indexing on the ranges themselves by writing $\big($*bounds*()$\big)[r]$ in order to narrow and bounds check the range r and obtain a closed range of indices on the underlying data.

`abstract opr` $\big[r$: Range⟦I⟧$\big]$: Indexed⟦E, I⟧

`opr` [_: TrivialOpenRange] : Indexed⟦E, I⟧

Roughly speaking, *ivmap*(f) is equivalent to *indexValuePairs*.*map*(f). However *ivmap* is not merely a convenient shortcut. It is actually intended to create a copy of the underlying indexed structure when that is appropriate.

The usual *map* function in Generator should do the same (and does for the instances in this library). Copying can be bad for space, but is complexity-preserving if the mapped generator is used more than once.

ivmap⟦R⟧$\big(f : (I, E) \to R\big)$: Indexed⟦$R, I$⟧

map⟦R⟧$(f : E \to R)$: Indexed⟦R, I⟧

indexOf(e) returns an index at which e can be found, or Nothing if no such index exists.

indexOf$(e : E)$: Maybe⟦I⟧

`end`

`trait` ZeroIndexed⟦E⟧ `extends` Indexed⟦$E, \mathbb{Z}32$⟧

bounds(): CompactFullRange⟦$\mathbb{Z}32$⟧

zip⟦F⟧$\big(g$: ZeroIndexed⟦F⟧$\big)$: ZeroIndexed$\big[\!\big[(E, F)\big]\!\big]$

`end`

`object` DefaultZip⟦E, F⟧$\big(e$: ZeroIndexed⟦E⟧, f: ZeroIndexed⟦F⟧$\big)$

extends { ZeroIndexed⟦(E, F)⟧, DelegatedIndexed⟦(E, F), ℤ32⟧ }

getter *size*(): ℤ32

getter *indices*(): Generator⟦ℤ32⟧

opr |self| : ℤ32

opr [*i*: ℤ32] : (E, F)

opr [*r*: Range⟦ℤ32⟧] : ZeroIndexed⟦(E, F)⟧

end

trait LexicographicOrder⟦*T* extends LexicographicOrder⟦*T*, *E*⟧, *E* extends StandardTotalOrder⟦*E*⟧⟧

extends { StandardTotalOrder⟦*T*⟧, ZeroIndexed⟦*E*⟧ }

opr CMP(self, *other* : *T*): TotalComparison

We give a specialized version of = because it can fail faster than CMP by checking sizes early.

opr =(self, *other* : *T*): Boolean

end

toArray⟦*E*⟧(*g*: Indexed⟦*E*, ℤ32⟧): Array⟦*E*, ℤ32⟧

DelegatedIndexed is an indexed generator that has recourse to another indexed generator internally. By default, this in turn is defined in terms of *indexValuePairs*(). Thus, it is only necessary to define either *indexValuePairs*() or *indices*().

This class is designed for convenience; it should not be used as a type in running code, but only as a supertype in lieu of Indexed.

trait DelegatedIndexed⟦*E*, *I* extends Index⟧ extends Indexed⟦*E*, *I*⟧

getter *generator*(): Generator⟦*E*⟧

getter *size*(): ℤ32

opr |self| : ℤ32

generate⟦*R*⟧(*r*: Reduction⟦*R*⟧, *body*: *E* → *R*): *R*

seq(self): SequentialGenerator⟦*E*⟧

cross⟦*G*⟧(*g*: Generator⟦*G*⟧): Generator⟦(*E*, *G*)⟧

mapReduce⟦*R*⟧(*body*: *E* → *R*, *join* : (*R*, *R*) → *R*, *zero* : *R*): *R*

reduce(*j* : (*E*, *E*) → *E*, *z* : *E*) : *E*

reduce(*r*: Reduction⟦*E*⟧) : *E*

loop(*f* : *E* → ()): ()

end

The MutableIndexed trait is an indexed trait whose elements can be mutated using indexed assignment. Right now, we are using this type in a somewhat dangerous way, since, for example, Array1$[\![E, b_0, s_0]\!]$ extends both Indexed$[\![\text{Array1}[\![E, b_0, s_0]\!], E, \mathbb{Z}32]\!]$ and Indexed$[\![\text{Array}[\![E, \mathbb{Z}32]\!], E,$ We will need to find a solution to this at some point.

trait MutableIndexed$[\![E, I$ **extends** Index$]\!]$

extends $\{$ Indexed$[\![E, I]\!]$ $\}$

abstract opr $[i : I] := (v : E) : ()$

For Ranged assignment, the extents of r and $v.bounds()$ must match, but the lower bounds need not.

abstract opr $[r{:}\,\text{Range}[\![I]\!]] := (v{:}\,\text{Indexed}[\![E, I]\!]) : ()$

opr $[_{:}\,\text{TrivialOpenRange}] := (v{:}\,\text{Indexed}[\![E, I]\!]) : ()$

end

Array whose bounds are implicit rather than static, and which may be either mutable or immutable.

trait ReadableArray$[\![E, I$ **extends** Index$]\!]$

extends $\{$ HasRank, Indexed$[\![E, I]\!]$, DelegatedIndexed$[\![E, I]\!]$ $\}$

CONCRETE GETTERS Default implementations of getters based on abstract methods below.

getter *indices*$()$: Generator$[\![I]\!]$

getter *indexValuePairs*$()$: Indexed$[\![(I, E), I]\!]$

getter *generator*$()$: Indexed$[\![E, I]\!]$

CONCRETE METHODS Default implementations of most array stuff based on the above. The things we can't provide are anything involving replica.

opr $[i : I] : E$

Initialize element at index i with value v. This should occur once, before any other access or assignment occurs to element i. An error will be signaled if an uninitialized element is read or an initialized element is re-initialized.

init$(i : I, v : E)$: $()$

generate$[\![R]\!](r{:}\,\text{Reduction}[\![R]\!], body{:}\,E \to R) : R$

seq(**self**): SequentialGenerator$[\![E]\!]$

0-based non-bounds-checked indexing.

abstract *get*$(i : I)$: E

abstract *init*$_0(i : I, e : E)$: $()$

abstract *zeroIndices*$()$: CompactFullRange$[\![I]\!]$

Convert from *base*-based indexing to 0-based indexing, performing bounds checking.

`abstract` *offset*$(i : I)$: I

Convert from 0-based indexing to *base*-based indexing.

`abstract` *toIndex*$(i : I)$: I

Indexed functionality with more specific type information.

`abstract opr` $\big[r$: Range$[\![I]\!]\big]$: ReadableArray$[\![E, I]\!]$

`abstract opr` $[_$: TrivialOpenRange$]$: ReadableArray$[\![E, I]\!]$

`abstract` *ivmap*$[\![R]\!]\big(f : (I, E) \to R\big)$: ReadableArray$[\![R, I]\!]$

`abstract` *map*$[\![R]\!](f : E \to R)$: ReadableArray$[\![R, I]\!]$

Shift the origin of an array. This should yield a new view of the same array; that is, initialization and/or update to either will be reflected in the other.

`abstract` *shift*$(newOrigin : I)$: ReadableArray$[\![E, I]\!]$

Bulk initialization of an array using a given function or value. These are defined with more specific self types in StandardMutableArrayType and ImmutableArray1.

`abstract` *tabulate*$(f : I \to E)$: ReadableArray$[\![E, I]\!]$

`abstract` *fill*$(v : E)$: ReadableArray$[\![E, I]\!]$

`abstract` *copy*$()$: ReadableArray$[\![E, I]\!]$

Create a fresh array structurally identical to the present one, but holding elements of type U.

`abstract` *replica*$[\![U]\!]()$: ReadableArray$[\![U, I]\!]$

`opr` $=$(`self`, *other*: HasRank): Boolean

`end`

`trait` ImmutableArray$[\![E, I$ `extends` Index$]\!]$ `extends` $\{$ ReadableArray$[\![E, I]\!]$ $\}$

`excludes` $\{$ Array$[\![E, I]\!]$ $\}$

`abstract opr` $\big[r$: Range$[\![I]\!]\big]$: ImmutableArray$[\![E, I]\!]$

`abstract opr` $[_$: TrivialOpenRange$]$: ImmutableArray$[\![E, I]\!]$

`abstract` *ivmap*$[\![R]\!]\big(f : (I, E) \to R\big)$: ImmutableArray$[\![R, I]\!]$

`abstract` *map*$[\![R]\!](f : E \to R)$: ImmutableArray$[\![R, I]\!]$

`abstract` *shift*$(newOrigin : I)$: ImmutableArray$[\![E, I]\!]$

`abstract` *tabulate*$(f : I \to E)$: ImmutableArray$[\![E, I]\!]$

`abstract` *fill*$(v : E)$: ImmutableArray$[\![E, I]\!]$

`abstract` *copy*$()$: ImmutableArray$[\![E, I]\!]$

```
    abstract replica⟦U⟧(): ImmutableArray⟦U, I⟧

      Thaw array (return mutable copy).

  abstract thaw(): Array⟦E, I⟧

end

trait Array⟦E, I extends Index⟧ extends { ReadableArray⟦E, I⟧, MutableIndexed⟦E, I⟧ }

    abstract put(i : I, e : E): ()

    opr [i : I] := (v : E): ()

    opr [r: Range⟦I⟧] := (a: Indexed⟦E, I⟧): ()

    abstract opr [r: Range⟦I⟧] : Array⟦E, I⟧

    abstract opr [_: TrivialOpenRange] : Array⟦E, I⟧

    abstract ivmap⟦R⟧(f : (I, E) → R): Array⟦R, I⟧

    abstract map⟦R⟧(f : E → R): Array⟦R, I⟧

    abstract shift(newOrigin : I): Array⟦E, I⟧

    abstract tabulate(f : I → E): Array⟦E, I⟧

    abstract fill(v : E): Array⟦E, I⟧

    abstract assign(f : I → E): Array⟦E, I⟧

    abstract copy(): Array⟦E, I⟧

    abstract replica⟦U⟧(): Array⟦U, I⟧

      Freeze array (return mutable copy).

  abstract freeze(): ImmutableArray⟦E, I⟧

end

Factory for arrays that returns an empty 0-indexed array of a given run-time-determined
size.

array⟦E⟧(x: ℤ32): Array⟦E, ℤ32⟧

array⟦E⟧(x: ℤ32, y: ℤ32): Array⟦E, (ℤ32, ℤ32)⟧

array⟦E⟧(x: ℤ32, y: ℤ32, z: ℤ32): Array⟦E, (ℤ32, ℤ32, ℤ32)⟧

Factory for immutable arrays that returns an empty 0-indexed array of a given run-time-dete
size.

immutableArray⟦E⟧(x: ℤ32): ImmutableArray⟦E, ℤ32⟧
(*
immutableArray[\E\](x:ZZ32,y:ZZ32):ImmutableArray[\E,(ZZ32,ZZ32)\]                    immutableAr-
ray[\E\](x:ZZ32,y:ZZ32,z:ZZ32):ImmutableArray[\E,(ZZ32,ZZ32,ZZ32)\]
```

*)

primitiveArray$[\![E]\!](x\colon \mathbb{Z}32)\colon \text{Array}[\![E, \mathbb{Z}32]\!]$

primitiveImmutableArray$[\![E]\!](x\colon \mathbb{Z}32)\colon \text{ImmutableArray}[\![E, \mathbb{Z}32]\!]$

NOTE: StandardImmutableArrayType is a parent of StandardMutableArrayType. It therefore does not extend ImmutableArray as you might expect. Other types that extend it should also extend ImmutableArray explicitly.

trait StandardImmutableArrayType$[\![T$ **extends** StandardImmutableArrayType$[\![T, E, I]\!], E, I$ **extends** Index$]\!]$

extends $\{$ ReadableArray$[\![E, I]\!]$ $\}$

abstract *tabulate*$(f : I \to E) : T$

abstract *fill*$(v : E) : T$

abstract *copy*$() : T$

end

trait StandardMutableArrayType$[\![T$ **extends** StandardMutableArrayType$[\![T, E, I]\!], E, I$ **extends** Index$]\!]$

extends $\{$ StandardImmutableArrayType$[\![T, E, I]\!]$, Array$[\![E, I]\!]$ $\}$

assign$(v : T) : T$

assign$(f : I \to E) : T$

tabulate$(f : I \to E) : T$

fill$(v : E) : T$

abstract *copy*$() : T$

end

Canonical partitioning of a positive number x into two pieces. If $(a, b) = \mathit{partition}(n)$ and $n > 0$ then $0 < a <= b$, $n = a + b$. As it turns out we choose a to be the largest power of $2 < n$.

partition$(x\colon \mathbb{Z}32)\colon (\mathbb{Z}32, \mathbb{Z}32)$

A ReadableArray1$[\![T, b_0, s_0]\!]$ is an arbitrary 1-dimensional array whose s_0 elements are of type T, and whose lowest index is b_0.

The natural order of all generators is from b_0 to $b_0 + s_0 - 1$.

trait ReadableArray1$[\![T,$ **nat** $b_0,$ **nat** $s_0]\!]$

extends $\{$ Indexed1$[\![s_0]\!]$, Rank1, ReadableArray$[\![T, \mathbb{Z}32]\!]$ $\}$

comprises $\{$ ImmutableArray1$[\![T, b_0, s_0]\!]$, Array1$[\![T, b_0, s_0]\!]$ $\}$

getter *size*$()\colon \mathbb{Z}32$

`getter` *bounds*(): CompactFullRange⟦ℤ32⟧

`abstract getter` *mutability*(): String

`opr` |`self`| : ℤ32

subarray⟦`nat` b, `nat` s, `nat` o⟧(m: ℤ32): ReadableArray1⟦T, b, s⟧

Offset converts from b_0-indexing to 0-indexing, bounds checking en route.

offset(i: ℤ32): ℤ32

toIndex(i: ℤ32): ℤ32

zeroIndices(): CompactFullRange⟦ℤ32⟧

`end`

`trait` ImmutableArray1⟦T, `nat` b_0, `nat` s_0⟧

`extends` { StandardImmutableArrayType⟦ImmutableArray1⟦T, b_0, s_0⟧, T, ℤ32⟧,
ImmutableArray⟦T, ℤ32⟧, ReadableArray1⟦T, b_0, s_0⟧ }

`getter` *mutability*(): String

shift(o: ℤ32): ImmutableArray⟦T, ℤ32⟧

`opr` [r: Range⟦ℤ32⟧] : ImmutableArray⟦T, ℤ32⟧

`opr` [_: OpenRange⟦ℤ32⟧] : ImmutableArray1⟦$T, 0, s_0$⟧

`opr` [_: TrivialOpenRange] : ImmutableArray1⟦$T, 0, s_0$⟧

subarray selects a subarray of this array based on static parameters. $b \# s$ are the new bounds of the array; o is the index of the subarray within the current array.

subarray⟦`nat` b, `nat` s, `nat` o⟧(m: ℤ32): ImmutableArray1⟦T, b, s⟧

The *replica* method returns a replica of the array (similar layout etc.) but with a different element type.

replica⟦U⟧(): ImmutableArray1⟦U, b_0, s_0⟧

copy(): ImmutableArray1⟦T, b_0, s_0⟧

fill($v : T$): ImmutableArray1⟦T, b_0, s_0⟧

tabulate(f: ℤ32 → T): ImmutableArray1⟦T, b_0, s_0⟧

thaw(): Array1⟦T, b_0, s_0⟧

map⟦R⟧($f : T \to R$): ImmutableArray1⟦R, b_0, s_0⟧

ivmap⟦R⟧(f: (ℤ32, T) → R): ImmutableArray1⟦R, b_0, s_0⟧

`end`

Array1$[\![T, b_0, s_0]\!]$ is a 1-dimension array whose s_0 elements are of type T, and whose lowest index is b_0.

trait Array1$[\![T, \texttt{nat}\ b_0, \texttt{nat}\ s_0]\!]$

extends $\{$ ReadableArray1$[\![T, b_0, s_0]\!]$,

StandardMutableArrayType$\big[\![\text{Array1}[\![T, b_0, s_0]\!], T, \mathbb{Z}32]\!\big]$ $\}$

excludes {Number, String}

getter *mutability*(): String

shift(o: $\mathbb{Z}32$): Array$[\![T, \mathbb{Z}32]\!]$

opr $\big[r$: Range$[\![\mathbb{Z}32]\!]\big]$: Array$[\![T, \mathbb{Z}32]\!]$

opr $\big[_$: OpenRange$[\![\mathbb{Z}32]\!]\big]$: Array1$[\![T, 0, s_0]\!]$

opr [_: TrivialOpenRange] : Array1$[\![T, 0, s_0]\!]$

subarray$[\![\texttt{nat}\ b, \texttt{nat}\ s, \texttt{nat}\ o]\!]$($m$: $\mathbb{Z}32$): Array1$[\![T, b, s]\!]$

replica$[\![U]\!]$(): Array1$[\![U, b_0, s_0]\!]$

copy(): Array1$[\![T, b_0, s_0]\!]$

freeze(): ImmutableArray1$[\![T, b_0, s_0]\!]$

map$[\![R]\!]$($f : T \to R$): Array1$[\![R, b_0, s_0]\!]$

ivmap$[\![R]\!]\big(f$: $(\mathbb{Z}32, T) \to R\big)$: Array1$[\![R, b_0, s_0]\!]$

end

trait AnyVector end

trait Vector$\big[\![T$ extends $\{$ Number, MultiplicativeRing$[\![T]\!]$ $\}$, nat $s_0]\!\big]$

extends $\{$ AnyVector, Array1$[\![T, 0, s_0]\!]$, AdditiveGroup$[\![$Vector$[\![T, s_0]\!]]\!]$ $\}$

excludes $\{$ AnyMultiplicativeRing $\}$

opr $+\big($self, v: Vector$[\![T, s_0]\!]\big)$: Vector$[\![T, s_0]\!]$

opr $-\big($self, v: Vector$[\![T, s_0]\!]\big)$: Vector$[\![T, s_0]\!]$

opr $-($self$)$: Vector$[\![T, s_0]\!]$

scale(t: T): Vector$[\![T, s_0]\!]$

pmul$\big(v$: Vector$[\![T, s_0]\!]\big)$: Vector$[\![T, s_0]\!]$

dot$\big(v$: Vector$[\![T, s_0]\!]\big)$: T

fill($v : T$): Vector$[\![T, s_0]\!]$

tabulate(f: $\mathbb{Z}32 \to T$): Vector$[\![T, s_0]\!]$

end

__*builtinFactory1* must be a non-overloaded 0-parameter factory for 1-D arrays. The type parameters are enshrined in LHSEvaluator.java and NonPrimitive.java; the factory name is enshrined in WellKnownNames.java. There must be some factory, named in this file, with this type signature. A similar thing is true for k-dimensional array types.

__*builtinFactory1*$[\![T, \texttt{nat}\ b_0, \texttt{nat}\ s_0]\!]():\text{Array1}[\![T, b_0, s_0]\!]$

__*immutableFactory1* is a non-overloaded 0-parameter factory for 0-indexed 1-D write-once arrays. It is also mentioned in WellKnownNames as it is used to allocate storage for varargs.

__*immutableFactory1*$[\![T, \texttt{nat}\ b_0, \texttt{nat}\ s_0]\!]():\text{ImmutableArray1}[\![T, b_0, s_0]\!]$

$\mathit{array}_1[\![T, \texttt{nat}\ s_0]\!]():\text{Array1}[\![T, 0, s_0]\!]$

$\mathit{array}_1[\![T, \texttt{nat}\ s_0]\!](v:T):\text{Array1}[\![T, 0, s_0]\!]$

$\mathit{tabulatedArray}_1[\![T, \texttt{nat}\ s_0]\!](f:\mathbb{Z}32 \to T):\text{Array1}[\![T, 0, s_0]\!]$

$\mathit{immutableArray}_1[\![T, \texttt{nat}\ s_0]\!]():\text{ImmutableArray1}[\![T, 0, s_0]\!]$

vector is the same as array_1, but specialized to numeric type arguments.

$\mathit{vector}[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ s_0]\!]():\text{Vector}[\![T, s_0]\!]$

$\mathit{vector}[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ s_0]\!](v:T):\text{Vector}[\![T, s_0]\!]$

$\mathit{tabulatedVector}[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ s_0]\!](f:\mathbb{Z}32 \to T):\text{Vector}[\![T, s_0]\!]$

$\mathit{pmul}[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ k]\!]$
$(a:\text{Vector}[\![T, k]\!], b:\text{Vector}[\![T, k]\!]):\text{Vector}[\![T, k]\!]$

$\texttt{opr}\ \cdot[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ n]\!]$
$(\mathit{me}:\text{Vector}[\![T, n]\!], \mathit{other}:\text{Vector}[\![T, n]\!]):T$

$\texttt{opr}\ \mathit{juxtaposition}[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ n]\!]$
$(\mathit{me}:\text{Vector}[\![T, n]\!], \mathit{other}:\text{Vector}[\![T, n]\!]):T$

$\texttt{opr}\ \cdot[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ n]\!]$
$(\mathit{me}:\text{Vector}[\![T, n]\!], \mathit{other}:T):\text{Vector}[\![T, n]\!]$

$\texttt{opr}\ \mathit{juxtaposition}[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ n]\!]$
$(\mathit{me}:\text{Vector}[\![T, n]\!], \mathit{other}:T):\text{Vector}[\![T, n]\!]$

$\texttt{opr}\ \cdot[\![T\ \texttt{extends}\ \{\text{Number}, \text{MultiplicativeRing}[\![T]\!]\}, \texttt{nat}\ n]\!]$
$(\mathit{other}:T, \mathit{me}:\text{Vector}[\![T, n]\!]):\text{Vector}[\![T, n]\!]$

`opr` *juxtaposition*$[\![\, T$ `extends` $\{\,\text{Number}, \text{MultiplicativeRing}[\![T]\!]\,\}, \texttt{nat}\ n\,]\!]$

$\big(other : T, me : \text{Vector}[\![T, n]\!]\big) : \text{Vector}[\![T, n]\!]$

squaredNorm$[\![T$ `extends` $\{\,\text{Number}, \text{MultiplicativeRing}[\![T]\!]\,\}, \texttt{nat}\ s_0]\!]\big(a\colon \text{Vector}[\![T, s_0]\!]\big) : T$

`opr` $\big|\big|[\![\, T$ `extends` $\{\,\text{Number}, \text{MultiplicativeRing}[\![T]\!]\,\}, \texttt{nat}\ k\,]\!] me : \text{Vector}[\![T, k]\!]\big|\big| : \mathbb{R}64$

Array2$[\![T, b_0, s_0, b_1, s_1]\!]$ is the type of 2-dimensional arrays of element type T, with size s_0 in the first dimension and s_1 in the second dimension and lowest index (b_0, b_1). Natural order for all generators in each dimension is from b to $b+s-1$; the overall order of elements need only be consistent with the cross product of these orderings (see Generator.*cross*()).

`trait` Array2$[\![T, \texttt{nat}\ b_0, \texttt{nat}\ s_0, \texttt{nat}\ b_1, \texttt{nat}\ s_1]\!]$

`extends` $\big\{$ Indexed1$[\![s_0]\!]$, Indexed2$[\![s_1]\!]$, Rank2,

StandardMutableArrayType$\big[\![\text{Array2}[\![T, b_0, s_0, b_1, s_1]\!], T, (\mathbb{Z}32, \mathbb{Z}32)]\!]\,\}$

`excludes` $\{\,\text{Number}, \text{String}\,\}$

`getter` *size*(): $\mathbb{Z}32$

`getter` *sizes*(): $(\mathbb{Z}32, \mathbb{Z}32)$

`getter` *bounds*(): CompactFullRange$\big[\![(\mathbb{Z}32, \mathbb{Z}32)]\!]$

`opr` |`self`| : $\mathbb{Z}32$

Translate from b_0, b_1-indexing to 0-indexing, checking bounds.

offset$\big(t_1\colon (\mathbb{Z}32, \mathbb{Z}32)\big)\colon (\mathbb{Z}32, \mathbb{Z}32)$

toIndex$\big(t_1\colon (\mathbb{Z}32, \mathbb{Z}32)\big)\colon (\mathbb{Z}32, \mathbb{Z}32)$

`opr` $[x\colon \mathbb{Z}32, y\colon \mathbb{Z}32] := (v : T)\colon ()$

`opr` $\big[r\colon \text{Range}[\![(\mathbb{Z}32, \mathbb{Z}32)]\!]\big] : \text{Array}\big[\![T, (\mathbb{Z}32, \mathbb{Z}32)]\!]$

`opr` $\big[_\colon \text{OpenRange}[\![\mathbb{Z}32]\!]\big] : \text{Array2}[\![T, 0, s_0, 0, s_1]\!]$

`opr` $[_\colon \text{TrivialOpenRange}] : \text{Array2}[\![T, 0, s_0, 0, s_1]\!]$

shift$\big(t_1\colon (\mathbb{Z}32, \mathbb{Z}32)\big)\colon \text{Array}\big[\![T, (\mathbb{Z}32, \mathbb{Z}32)]\!]$

2-D subarray given static subarray parameters. $(bo_1, bo_2) \# (so_1, so_2)$ are output bounds. The result is the subarray starting at (o_0, o_1) in the original array, striding by (m0,m1).

subarray$[\![\texttt{nat}\ bo_0, \texttt{nat}\ so_0, \texttt{nat}\ bo_1, \texttt{nat}\ so_1, \texttt{nat}\ o_0, \texttt{nat}\ o_1]\!]$

$(m_0\colon \mathbb{Z}32, m_1\colon \mathbb{Z}32)\colon \text{Array2}[\![T, bo_0, so_0, bo_1, so_1]\!]$

zeroIndices(): CompactFullRange$\big[\![(\mathbb{Z}32, \mathbb{Z}32)]\!]$

replica$[\![U]\!]$(): Array2$[\![U, b_0, s_0, b_1, s_1]\!]$

$copy()$: Array2$[\![T, b_0, s_0, b_1, s_1]\!]$

$put\big(t_1\colon(\mathbb{Z}32, \mathbb{Z}32), v : T\big) : ()$

$get\big(t_1\colon(\mathbb{Z}32, \mathbb{Z}32)\big) : T$

$t()$: Array2$[\![T, b_1, s_1, b_0, s_0]\!]$

(⋆ Copied here for better return type information. ⋆)

$map[\![R]\!](f : T \to R)$: Array2$[\![R, b_0, s_0, b_1, s_1]\!]$

$ivmap[\![R]\!]\big(f\colon((\mathbb{Z}32, \mathbb{Z}32), T) \to R\big)$: Array2$[\![R, b_0, s_0, b_1, s_1]\!]$

$freeze()$: ImmutableArray $[\![T, (\mathbb{Z}32, \mathbb{Z}32)]\!]$

`end`

`trait` AnyMatrix `end`

`trait` Matrix$[\![T$ `extends` $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, `nat` s_0, `nat` $s_1]\!]$

`extends` $\{$ AnyMatrix, Array2$[\![T, 0, s_0, 0, s_1]\!]$, AdditiveGroup$[\![$Matrix$[\![T, s_0, s_1]\!]]\!]\}$

`excludes` $\{$ AnyMultiplicativeRing $\}$

`opr` $+\big($`self`$, v$: Matrix$[\![T, s_0, s_1]\!]\big)$: Matrix$[\![T, s_0, s_1]\!]$

`opr` $-\big($`self`$, v$: Matrix$[\![T, s_0, s_1]\!]\big)$: Matrix$[\![T, s_0, s_1]\!]$

`opr` $-($`self`$)$: Matrix$[\![T, s_0, s_1]\!]$

$scale(t_1\colon T)$: Matrix$[\![T, s_0, s_1]\!]$

$mul[\![$ `nat` $s_2\,]\!]\big(other\colon$ Matrix$[\![T, s_1, s_2]\!]\big)$: Matrix$[\![T, s_0, s_2]\!]$

$rmul\big(v\colon$ Vector$[\![T, s_1]\!]\big)$: Vector$[\![T, s_0]\!]$

$lmul\big(v\colon$ Vector$[\![T, s_0]\!]\big)$: Vector$[\![T, s_1]\!]$

$t()$: Matrix$[\![T, s_1, s_0]\!]$

$fill(v : T)$: Matrix$[\![T, s_0, s_1]\!]$

$tabulate\big(f\colon(\mathbb{Z}32, \mathbb{Z}32) \to T\big)$: Matrix$[\![T, s_0, s_1]\!]$

`end`

$__builtinFactory2[\![T,$ `nat` $b_0,$ `nat` $s_0,$ `nat` $b_1,$ `nat` $s_1]\!]()$: Array2$[\![T, b_0, s_0, b_1, s_1]\!]$

$array_2$ `is a factory for 0-based 2-D arrays.`

$array_2[\![T,$ `nat` $s_0,$ `nat` $s_1]\!]()$: Array2$[\![T, 0, s_0, 0, s_1]\!]$

$array_2[\![T,$ `nat` $s_0,$ `nat` $s_1]\!](v : T)$: Array2$[\![T, 0, s_0, 0, s_1]\!]$

$tabulatedArray_2[\![T,$ `nat` $s_0,$ `nat` $s_1]\!]\big(f\colon(\mathbb{Z}32, \mathbb{Z}32) \to T\big)$: Array2$[\![T, 0, s_0, 0, s_1]\!]$

matrix is the same as $array_2$, but specialized to numeric type arguments, except that the default value (if given) is used to construct a multiple of the identity matrix.

matrix$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** s_0, **nat** $s_1]\!]()$: Matrix$[\![T, s_0, s_1]\!]$

matrix$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** s_0, **nat** $s_1]\!](v : T)$: Matrix$[\![T, s_0, s_1]\!]$

Matrix multiplication.

opr $\cdot[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** m, **nat** $p]\!]$

$(me$: Matrix$[\![T, n, m]\!]$, *other*: Matrix$[\![T, m, p]\!])$: Matrix$[\![T, n, p]\!]$

opr *juxtaposition*$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** m, **nat** $p]\!]$

$(me$: Matrix$[\![T, n, m]\!]$, *other*: Matrix$[\![T, m, p]\!])$: Matrix$[\![T, n, p]\!]$

Matrix-vector multiplication.

opr $\cdot[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(me$: Matrix$[\![T, n, m]\!]$, v: Vector$[\![T, m]\!])$: Vector$[\![T, n]\!]$

opr *juxtaposition*$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(me$: Matrix$[\![T, n, m]\!]$, v: Vector$[\![T, m]\!])$: Vector$[\![T, n]\!]$

Vector-matrix multiplication.

opr $\cdot[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(v$: Vector$[\![T, n]\!]$, me: Matrix$[\![T, n, m]\!])$: Vector$[\![T, m]\!]$

opr *juxtaposition*$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(v$: Vector$[\![T, n]\!]$, me: Matrix$[\![T, n, m]\!])$: Vector$[\![T, m]\!]$

opr $\cdot[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(me : \text{Matrix}[\![T, n, m]\!], other : T) : \text{Matrix}[\![T, n, m]\!]$

opr *juxtaposition*$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(me : \text{Matrix}[\![T, n, m]\!], other : T) : \text{Matrix}[\![T, n, m]\!]$

opr $\cdot[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(other : T, me : \text{Matrix}[\![T, n, m]\!]) : \text{Matrix}[\![T, n, m]\!]$

opr *juxtaposition*$[\![T$ **extends** $\{$ Number, MultiplicativeRing$[\![T]\!]\}$, **nat** n, **nat** $m]\!]$

$(other : T, me : \text{Matrix}[\![T, n, m]\!]) : \text{Matrix}[\![T, n, m]\!]$

Array3$[\![T, b_0, s_0, b_1, s_1, b_2, s_2]\!]$ is the type of 3-dimensional arrays of element type T, with size s_i in the ith dimension and lowest index (b_0, b_1, b_2). Natural order for all generators in each dimension is from b to $b + s - 1$; the overall order of elements need only be consistent with the cross product of these orderings (see Generator.*cross*()).

trait Array3⟦T, nat b_0, nat s_0, nat b_1, nat s_1, nat b_2, nat s_2⟧

extends { Indexed1⟦s_0⟧, Indexed2⟦s_1⟧, Indexed3⟦s_2⟧, Rank3,

StandardMutableArrayType⟦Array3⟦$T, b_0, s_0, b_1, s_1, b_2, s_2$⟧, T,

$(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧ }

excludes { Number, String, AnyAdditiveGroup, AnyMultiplicativeRing }

(* Excluding AnyAdditiveGroup picks one of two uses of rank 3. An operator between an Array3 and a lower-rank array, such as a matrix added to every plane, can be declared without colliding with AdditiveGroup's +(self, other: T); in exchange no Array3 can be an additive group, a rank-3 counterpart of Vector and Matrix. Array1 and Array2 cannot exclude it: Vector and Matrix extend them and are additive groups. *)

getter *size*(): $\mathbb{Z}32$

getter *sizes*(): $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$

getter *bounds*(): CompactFullRange⟦$(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧

opr |self|: $\mathbb{Z}32$

Again, *offset* performs bounds checking and shifts to 0-indexing.

offset$\big(t$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$

toIndex$\big(t$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$

And *get* and *put* are 0-indexed without bounds checks.

abstract *put*$\big(t$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32), v : T\big)$: ()

abstract *get*$\big(t$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big) : T$

opr $[i$: $\mathbb{Z}32, j$: $\mathbb{Z}32, k$: $\mathbb{Z}32] := (v : T)$: ()

opr $\big[r$: Range⟦$(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧$\big]$: Array⟦$T, (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧

opr $\big[_$: OpenRange⟦$\mathbb{Z}32$⟧$\big]$: Array3⟦$T, 0, s_0, 0, s_1, 0, s_2$⟧

opr $[_$: TrivialOpenRange$]$: Array3⟦$T, 0, s_0, 0, s_1, 0, s_2$⟧

shift$\big(t$: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)$: Array⟦$T, (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧

3-D subarray given static subarray parameters. $(bo_0, bo_1, bo_2) \# (so_0, so_1, so_2)$ are output bounds. The result is the subarray starting at (o_0, o_1, o_2) in the original array, striding by (m0,m1,m2).

subarray⟦nat bo_0, nat so_0, nat bo_1, nat so_1, nat bo_2, nat so_2,

nat o_0, nat o_1, nat o_2⟧

$(m_0$: $\mathbb{Z}32, m_1$: $\mathbb{Z}32, m_2$: $\mathbb{Z}32)$: Array3⟦$T, bo_0, so_0, bo_1, so_1, bo_2, so_2$⟧

zeroIndices(): CompactFullRange⟦$(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧

replica⟦U⟧(): Array3⟦$U, b_0, s_0, b_1, s_1, b_2, s_2$⟧

copy(): Array3⟦$T, b_0, s_0, b_1, s_1, b_2, s_2$⟧

map⟦R⟧($f : T \to R$): Array3⟦$R, b_0, s_0, b_1, s_1, b_2, s_2$⟧

ivmap⟦R⟧(f: $((\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32), T) \to R$): Array3⟦$R, b_0, s_0, b_1, s_1, b_2, s_2$⟧

freeze(): ImmutableArray ⟦$T, (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)$⟧

`end`

__*builtinFactory3*⟦T, `nat` b_0, `nat` s_0, `nat` b_1, `nat` s_1, `nat` b_2, `nat` s_2⟧() :
Array3⟦$T, b_0, s_0, b_1, s_1, b_2, s_2$⟧

array$_3$⟦T, `nat` s_0, `nat` s_1, `nat` s_2⟧(): Array3⟦$T, 0, s_0, 0, s_1, 0, s_2$⟧

array$_3$⟦T, `nat` s_0, `nat` s_1, `nat` s_2⟧($v : T$): Array3⟦$T, 0, s_0, 0, s_1, 0, s_2$⟧

tabulatedArray$_3$⟦T, `nat` s_0, `nat` s_1, `nat` s_2⟧(f: $(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32) \to T$): Array3⟦$T, 0, s_0, 0, s_1, 0, s_2$⟧

Reductions

`trait` Reduction⟦L⟧

`getter` *reverse*(): Reduction⟦L⟧

empty(): L

join(a: L, b: L): L

`end`

`Invariants: join must be associative with identity empty unlift(lift(x)) = x`

`trait` ActualReduction⟦R, L⟧ `extends` Reduction⟦L⟧

`abstract` *empty*(): L

`abstract` *join*(a: L, b: L): L

`abstract` *lift*($r : R$): L

`abstract` *unlift*($l : L$): R

`If this reduction left-distributes over r, return a pair of reductions with the same lift and unlift`

leftDistribute(r: Reduction⟦R⟧): Maybe⟦(Reduction⟦R⟧, Reduction⟦R⟧)⟧

`If this reduction right-distributes over r, return a pair of reductions with the same lift and unlift`

rightDistribute(r: Reduction⟦R⟧): Maybe⟦(Reduction⟦R⟧, Reduction⟦R⟧)⟧

`If this reduction distributes over r, return a pair of reductions with the same lift and unlift`

```
  distribute(r: Reduction⟦R⟧): Maybe⟦(Reduction⟦R⟧, Reduction⟦R⟧)⟧
end

trait PossibleReductionPair⟦R⟧ extends Condition⟦PossibleReductionPair⟦R⟧⟧
    comprises { NoReductionPair⟦R⟧, SomeReductionPair⟦R⟧ }
end

object NoReductionPair⟦R⟧ extends PossibleReductionPair⟦R⟧
    getter holds(): Boolean
    cond⟦G⟧(t: PossibleReductionPair⟦R⟧ → G, e: () → G): G
end

trait SomeReductionPair⟦R⟧ extends PossibleReductionPair⟦R⟧
    getter holds(): Boolean
    abstract getter outer(): Reduction⟦R⟧
    abstract getter inner(): Reduction⟦R⟧
    cond⟦G⟧(t: PossibleReductionPair⟦R⟧ → G, e: () → G): G
end

trait ReductionPair⟦R, L⟧ extends SomeReductionPair⟦R⟧
    abstract getter outer(): ActualReduction⟦R, L⟧
    abstract getter inner(): ActualReduction⟦R, L⟧
end

The usual lifting to Maybe for identity-less operators
trait AssociativeReduction⟦R⟧ extends ActualReduction⟦R, Maybe⟦R⟧⟧
    empty(): Nothing⟦R⟧
    join(a: Maybe⟦R⟧, b: Maybe⟦R⟧): Maybe⟦R⟧
    abstract simpleJoin(a : R, b : R): R
    lift(r : R): Maybe⟦R⟧
    unlift(r: Maybe⟦R⟧): R
end

trait SomeCommutativeReduction end (* mark for commutative *)

trait DistributesOver⟦E⟧ end (* marking for distributivity *)

trait CommutativeReduction⟦R⟧ extends {AssociativeReduction⟦R⟧, SomeCommutativeReduction} end
```

Monoids don't require a special lift and unlift operation.

trait MonoidReduction$[\![R]\!]$ extends ActualReduction$[\![R, R]\!]$

lift$(r : R)$: R

unlift$(r : R)$: R

end

trait CommutativeMonoidReduction$[\![R]\!]$ extends $\{$MonoidReduction$[\![R]\!]$, SomeCommutativeReduction$\}$ end

trait ReductionWithZeroes$[\![R, L]\!]$ extends ActualReduction$[\![R, L]\!]$

isLeftZero$(l : L)$: Boolean

isRightZero$(l : L)$: Boolean

isZero$(l : L)$: Boolean

end

trait BigOperator$[\![I, O, R, L]\!]$

abstract getter *reduction*(): ActualReduction$[\![R, L]\!]$

abstract getter *body*(): $I \to R$

abstract getter *unwrap*(): $R \to O$

end

object BigReduction$[\![R, L]\!]$(r: ActualReduction$[\![R, L]\!]$) extends BigOperator$[\![R, R, R, L]\!]$

getter *reduction*(): ActualReduction$[\![R, L]\!]$

getter *body*(): $R \to R$

getter *unwrap*(): $R \to R$

end

object Comprehension$[\![I, O, R, L]\!]$(u: $R \to O$, r: ActualReduction$[\![R, L]\!]$, *singleton* : $I \to R$)

extends BigOperator$[\![I, O, R, L]\!]$

getter *reduction*(): ActualReduction$[\![R, L]\!]$

getter *body*(): $I \to R$

getter *unwrap*(): $R \to O$

end

VoidReduction is usually done for effect, so we pretend that the completion performs the effects. This rules out things distributing over void (that would change the number of effects in our program) but not void distributing over other things.

`object` VoidReduction `extends` { CommutativeMonoidReduction⟦()⟧ }

empty(): ()

join(*a*: (), *b*: ()): ()

`end`

`object` SumReduction⟦*T* `extends` AdditiveGroup⟦*T*⟧⟧ `extends` CommutativeMonoidReduction⟦*T*⟧

empty(): *T*

join(*a*: *T*, *b*: *T*): *T*

`end`

`opr` $\sum$⟦*T* `extends` AdditiveGroup⟦*T*⟧⟧(): BigReduction⟦*T*, *T*⟧

`opr` $\sum$⟦*T* `extends` AdditiveGroup⟦*T*⟧⟧(*g*: Generator⟦*T*⟧): *T*

`object` ProdReduction⟦*T* `extends` MultiplicativeRing⟦*T*⟧⟧ `extends` CommutativeMonoidReduction⟦*T*⟧

empty(): *T*

join(*a*: *T*, *b*: *T*): *T*

`end`

`opr` $\prod$⟦*T* `extends` MultiplicativeRing⟦*T*⟧⟧(): BigReduction⟦*T*, *T*⟧

`opr` $\prod$⟦*T* `extends` MultiplicativeRing⟦*T*⟧⟧(*g*: Generator⟦*T*⟧): *T*

`object` MinReduction⟦*T* `extends` StandardMin⟦*T*⟧⟧ `extends` CommutativeReduction⟦*T*⟧

simpleJoin(*a* : *T*, *b* : *T*): *T*

`end`

`opr BIG MIN`⟦*T* `extends` StandardMin⟦*T*⟧⟧(): BigReduction⟦*T*, Maybe⟦*T*⟧⟧

`opr BIG MIN`⟦*T* `extends` StandardMin⟦*T*⟧⟧(*g*: Generator⟦*T*⟧): *T*

`object` MaxReduction⟦*T* `extends` StandardMax⟦*T*⟧⟧ `extends` CommutativeReduction⟦*T*⟧

simpleJoin(*a* : *T*, *b* : *T*): *T*

`end`

`opr BIG MAX`⟦*T* `extends` StandardMax⟦*T*⟧⟧(): BigReduction⟦*T*, Maybe⟦*T*⟧⟧

`opr BIG MAX`⟦*T* `extends` StandardMax⟦*T*⟧⟧(*g*: Generator⟦*T*⟧): *T*

`object` MinMaxReduction⟦*T* `extends` StandardMinMax⟦*T*⟧⟧ `extends` CommutativeReduction⟦(*T*, *T*)⟧

`getter` *asString*(): String

simpleJoin$\big(a:(T,T), b:(T,T)\big):(T,T)$

`end`

`opr BIG MINMAX`$[\![T$ `extends` StandardMinMax$[\![T]\!]]\,():$

Comprehension$[\![T,(T,T),(T,T),$Maybe$[\![(T,T)]\!]]$

`opr BIG MINMAX`$[\![T$ `extends` StandardMinMax$[\![T]\!]]\,\big(g$: Generator$[\![T]\!]\big):(T,T)$

`opr BIG MINNUM`$()$: BigReduction$[\![\mathbb{R}64,\mathbb{R}64]\!]$

`opr BIG MINNUM`$\big(g$: Generator$[\![\mathbb{R}64]\!]\big):\mathbb{R}64$

`opr BIG MAXNUM`$()$: BigReduction$[\![\mathbb{R}64,\mathbb{R}64]\!]$

`opr BIG MAXNUM`$\big(g$: Generator$[\![\mathbb{R}64]\!]\big):\mathbb{R}64$

⊛ MIN/MAX combinations on tuples

`opr BIG MIN_MIN`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,():$

Comprehension$[\![(T,U),(T,U),(T,U),$Maybe$[\![(T,U)]\!]]$

`opr BIG MIN_MIN`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,\big(g$: Generator$[\![(T,U)]\!]\big):(T,U)$

`opr BIG MIN_MAX`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,():$

Comprehension$[\![(T,U),(T,U),(T,U),$Maybe$[\![(T,U)]\!]]$

`opr BIG MIN_MAX`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,\big(g$: Generator$[\![(T,U)]\!]\big):(T,U)$

`opr BIG MAX_MIN`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,():$

Comprehension$[\![(T,U),(T,U),(T,U),$Maybe$[\![(T,U)]\!]]$

`opr BIG MAX_MIN`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,\big(g$: Generator$[\![(T,U)]\!]\big):(T,U)$

`opr BIG MAX_MAX`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,():$

Comprehension$[\![(T,U),(T,U),(T,U),$Maybe$[\![(T,U)]\!]]$

`opr BIG MAX_MAX`$[\![T$ `extends` StandardMinMax$[\![T]\!], U$ `extends` StandardMinMax$[\![U]\!]]\,\big(g$: Generator$[\![(T,U)]\!]\big):(T,U)$

`AndReduction and OrReduction take advantage of natural zeroes for early exit.`

`object` AndReduction

`extends` { CommutativeMonoidReduction$[\![$Boolean$]\!]$,

ReductionWithZeroes$[\![$Boolean, Boolean$]\!]$ }

empty(): Boolean

join(a: Boolean, b: Boolean): Boolean

isZero(*a*: Boolean): Boolean

`end`

`opr BIG` $\wedge[\![T]\!]()$: BigReduction[[Boolean, Boolean]]

`opr BIG` $\wedge[\![T]\!]\big(g$: Generator[[Boolean]]$\big)$: Boolean

`object` OrReduction

`extends` { CommutativeMonoidReduction[[Boolean]],

ReductionWithZeroes[[Boolean, Boolean]] }

empty(): Boolean

join(*a*: Boolean, *b*: Boolean): Boolean

isZero(*a*: Boolean): Boolean

`end`

`opr BIG` $\vee[\![T]\!]()$: BigReduction[[Boolean, Boolean]]

`opr BIG` $\vee[\![T]\!]\big(g$: Generator[[Boolean]]$\big)$: Boolean

```
Operator reductions on integers
(*
```

object BitXorReduction[\T extends Integral[\T\]\] extends { CommutativeMonoidReduction[\T\] } getter asString(): String empty(): T join(a: T, b: T): T end

opr BIG BITXOR[\T extends Integral[\T\]\](): BigReduction[\T, T\]

opr BIG BITXOR[\T extends Integral[\T\]\](g: Generator[\T\]): T

```
 *)
```

`object` BitXorReduction

`extends` { CommutativeMonoidReduction[[$\mathbb{Z}32$]] }

`getter` *asString*(): String

empty(): $\mathbb{Z}32$

join(*a*: $\mathbb{Z}32$, *b*: $\mathbb{Z}32$): $\mathbb{Z}32$

`end`

`opr BIG` $\underline{\mathbb{W}}()$: BigReduction[[$\mathbb{Z}32, \mathbb{Z}32$]]

`opr BIG` $\underline{\mathbb{W}}\big(g$: Generator[[$\mathbb{Z}32$]]$\big)$: $\mathbb{Z}32$

```
A reduction performing String concatenation
```

`object` StringReduction `extends` MonoidReduction⟦String⟧

empty(): String

join(*a*: String, *b*: String): String

`end`

`A reduction performing String concatenation with a space`

`object` SpaceReduction `extends` MonoidReduction⟦String⟧

empty(): String

join(*a*: String, *b*: String): String

`end`

`A reduction performing String concatenation with newline separation`

`object` NewlineReduction `extends` AssociativeReduction⟦String⟧

simpleJoin(*a*: String, *b*: String): String

`end`

`This operator performs string concatenation, first converting its inputs (of type Any) to String if necessary.`

`opr BIG` ‖(): Comprehension⟦Any, String, String, String⟧

`opr BIG` ‖(*g*: Generator⟦Any⟧): String

`This operator performs string concatenation, first converting its inputs (of type Any) to String if necessary, and separating non-empty components by a space.`

`opr BIG` |||(): Comprehension⟦Any, String, String, String⟧

`opr BIG` |||(*g*: Generator⟦Any⟧): String

`This operator performs string concatenation with newline separation, first converting its inputs (of type Any) to String if necessary.`

`opr BIG` // (): Comprehension⟦Any, String, String, Maybe⟦String⟧⟧

`opr BIG` // ⟦*T*⟧(*g*: Generator⟦*T*⟧): String

`A MapReduceReduction takes an associative binary function` *j* `on arguments of type` *R*`, and the identity of that function` *z*`, and returns the corresponding reduction.`

`object` MapReduceReduction⟦*R*⟧(*j* : (*R*, *R*) → *R*, *z* : *R*) `extends` MonoidReduction⟦*R*⟧

empty(): *R*

join(*a* : *R*, *b* : *R*): *R*

`end`

A MIMapReduceReduction takes an associative binary function j on arguments of type R, and the identity of that function z, and returns the corresponding reduction.

object MIMapReduceReduction$[\![R]\!]\big(j\colon(\text{Any},\text{Any})\to R, z\colon R\big)$ **extends** MonoidReduction$[\![\text{Any}]\!]$

empty$(\,)\colon R$

join$(a\colon\text{Any}, b\colon\text{Any})\colon R$

end

embiggen takes a type T and an operation (either an operator OP or binary function f on type T), along with the identity z of the operation, and returns a function suitable as the right-hand side of the definition of the corresponding BIG operator.

```
(*
embiggen[\T,opr OP\](z:T): ((Reduction[\T\],T-¿T) -¿ T) -¿ T
 *)
```

embiggen$[\![T]\!]\big(j:(T,T)\to T, z:T\big):\text{BigReduction}[\![T,T]\!]$

trait FilterGenerator$[\![E]\!]$ **extends** Generator$[\![E]\!]$

excludes { HasRank }

getter $g(\,)\colon\text{Generator}[\![E]\!]$

getter $p(\,)\colon E\to\text{Condition}[\![(\,)]\!]$

generate$[\![R]\!]\big(r\colon\text{Reduction}[\![R]\!], m\colon E\to R\big)\colon R$

reduce$\big(r\colon\text{Reduction}[\![E]\!]\big)\colon E$

filter$\big(p'\colon E\to\text{Condition}[\![(\,)]\!]\big)\colon\text{FilterGenerator}[\![E]\!]$ `(* ' *)`

seq(**self**)$\colon\text{SequentialGenerator}[\![E]\!]$

end

Ranges

Ranges in general represent uses of the # and : operators. It is mostly subtypes of Range that are interesting.

The partial order on ranges describes containment: $a<b$ if and only if all points in a are strictly contained in b.

trait Range$[\![I]\!]$ **extends** { StandardPartialOrder$[\![\text{Range}[\![I]\!]]\!]$, Contains$[\![I]\!]$ }

excludes { Number, String, Index }

abstract getter *stride*$(\,)\colon I$

abstract getter *left*$(\,)\colon\text{Maybe}[\![I]\!]$

abstract getter *right*$(\,)\colon\text{Maybe}[\![I]\!]$

```
    abstract getter extent(): Maybe⟦I⟧
    abstract getter isEmpty(): Boolean
    getter isLeftBounded(): Boolean
    getter isAnyBounded(): Boolean
    abstract truncL(l: I): BoundedRange⟦I⟧
    truncR(r: I): BoundedRange⟦I⟧
    abstract flip(): Range⟦I⟧
    abstract forward(): Range⟦I⟧
    abstract every(s: I): Range⟦I⟧
    abstract imposeStride(s : I): Range⟦I⟧
    abstract atMost(n: I): Range⟦I⟧
    abstract opr ∩(self, other: Range⟦I⟧): Range⟦I⟧
    abstract narrowToRange(other: Range⟦I⟧): Range⟦I⟧
    narrowToRange(other: OpenRange⟦I⟧): Range⟦I⟧
    opr =(self, b: Range⟦I⟧): Boolean
    abstract opr ∈(n: I, self): Boolean
    abstract opr CMP(self, other: Range⟦I⟧): Comparison
    abstract opr FORWARD_CMP(self, other: Range⟦I⟧): Comparison
    asDebugStriing(): String
    check(): Range⟦I⟧
    shiftLeft(amount: I): Range⟦I⟧
    shiftRight(amount: I): Range⟦I⟧
    opr << (self, amount: I): Range⟦I⟧
    opr >> (self, amount: I): Range⟦I⟧
end

trait PartialRange⟦I⟧ extends Range⟦I⟧ end

trait OpenRange⟦I⟧ extends PartialRange⟦I⟧
    getter left(): Nothing⟦I⟧
    getter right(): Nothing⟦I⟧
    getter extent(): Nothing⟦I⟧
```

```
    getter isEmpty(): Boolean
    narrowToRange(other: OpenRange⟦I⟧): Range⟦I⟧
    opr FORWARD_CMP(self, other: OpenRange⟦I⟧): Comparison
    opr FORWARD_CMP(self, other: Range⟦I⟧): Comparison
    opr ∈(n: I, self): Boolean
end

object TrivialOpenRange extends OpenRange⟦Any⟧
    flip(): TrivialOpenRange
    forward(): TrivialOpenRange
    check(): TrivialOpenRange
    opr CMP(self, other: Range⟦Any⟧): Comparison
    narrowToRange(other: Range⟦Any⟧): Range⟦Any⟧
    narrowToRange(other: OpenRange⟦Any⟧): Range⟦Any⟧
end

trait RangeWithExtent⟦I⟧ extends Range⟦I⟧
    getter extent(): Just⟦I⟧
end

trait ExtentRange⟦I⟧ extends { RangeWithExtent⟦I⟧, PartialRange⟦I⟧ }
    getter left(): Nothing⟦I⟧
    getter right(): Nothing⟦I⟧
    getter isEmpty(): Boolean
    opr ∈(n: I, self): Boolean
    abstract opr FORWARD_CMP(self, other: Range⟦I⟧): Comparison
end

trait BoundedRange⟦I⟧ extends Range⟦I⟧
    getter leftOrRight(): I
    abstract flip(): BoundedRange⟦I⟧
    every(s: I): BoundedRange⟦I⟧
    atMost(n: I): FullRange⟦I⟧ where { I extends Index }
    opr ∩(self, other: Range⟦I⟧): BoundedRange⟦I⟧
```

```
    narrowToRange(other: OpenRange⟦I⟧): BoundedRange⟦I⟧
    abstract narrowToRange(other: Range⟦I⟧): BoundedRange⟦I⟧
end

trait RangeWithLeft⟦I⟧ extends BoundedRange⟦I⟧
    getter left(): Just⟦I⟧
    getter leftOrRight(): I
end

trait LeftRange⟦I⟧ extends { RangeWithLeft⟦I⟧, PartialRange⟦I⟧ }
    getter right(): Nothing⟦I⟧
    getter extent(): Nothing⟦I⟧
    getter isEmpty(): Boolean
    abstract opr FORWARD_CMP(self, other: Range⟦I⟧): Comparison
end

trait RangeWithRight⟦I⟧ extends BoundedRange⟦I⟧
    getter right(): Just⟦I⟧
end

trait RightRange⟦I⟧ extends { RangeWithRight⟦I⟧, PartialRange⟦I⟧ }
    getter left(): Nothing⟦I⟧
    getter leftOrRight(): I
    getter extent(): Nothing⟦I⟧
    getter isEmpty(): Boolean
    abstract opr FORWARD_CMP(self, other: Range⟦I⟧): Comparison
end

trait FullRange⟦I extends Index⟧ extends { RangeWithLeft⟦I⟧, RangeWithRight⟦I⟧, RangeWithExtent⟦I⟧, Indexed⟦I, I⟧ }
    getter extent(): Just⟦I⟧
    narrowToRange(other: OpenRange⟦I⟧): FullRange⟦I⟧
    abstract narrowToRange(other: Range⟦I⟧): FullRange⟦I⟧
    abstract opr ∈(n: I, self): Boolean
    opr FORWARD_CMP(self, other: Range⟦I⟧): Comparison
    abstract opr FORWARD_CMP(self, other: FullRange⟦I⟧): Comparison
```

```
end

trait CompactFullRange⟦I extends Index⟧ extends FullRange⟦I⟧
    getter lower(): I
    getter upper(): I
    abstract opr |self| : ℤ32
    forward(): CompactFullRange⟦I⟧
end

trait StridedFullRange⟦I extends Index⟧ extends FullRange⟦I⟧ end

The # and : operators serve as factories for parallel ranges.
opr #(lo: ℤ32, ex: ℤ32): CompactFullRange⟦ℤ32⟧
opr #((lo: (ℤ32, ℤ32), ex: (ℤ32, ℤ32)): CompactFullRange⟦(ℤ32, ℤ32)⟧
opr #((lo: (ℤ32, ℤ32, ℤ32), ex: (ℤ32, ℤ32, ℤ32)): CompactFullRange⟦(ℤ32, ℤ32, ℤ32)⟧
opr :(lo: ℤ32, hi: ℤ32): CompactFullRange⟦ℤ32⟧
opr :((lo: (ℤ32, ℤ32), hi: (ℤ32, ℤ32)): CompactFullRange⟦(ℤ32, ℤ32)⟧
opr :((lo: (ℤ32, ℤ32, ℤ32), hi: (ℤ32, ℤ32, ℤ32)): CompactFullRange⟦(ℤ32, ℤ32, ℤ32)⟧

Factories for incomplete ranges.
opr (x: ℤ32)# : LeftRange⟦ℤ32⟧
opr (p: (ℤ32, ℤ32))# : LeftRange⟦(ℤ32, ℤ32)⟧
opr (t: (ℤ32, ℤ32, ℤ32))# :
        LeftRange⟦(ℤ32, ℤ32, ℤ32)⟧
opr (x: ℤ32): : LeftRange⟦ℤ32⟧
opr (x: (ℤ32, ℤ32)): : LeftRange⟦(ℤ32, ℤ32)⟧
opr (x: (ℤ32, ℤ32, ℤ32)): : LeftRange⟦(ℤ32, ℤ32, ℤ32)⟧
opr #(x: ℤ32) : RangeWithExtent⟦ℤ32⟧
opr #(xy: (ℤ32, ℤ32)) : RangeWithExtent⟦(ℤ32, ℤ32)⟧
opr #(xyz: (ℤ32, ℤ32, ℤ32)) :
        RangeWithExtent⟦(ℤ32, ℤ32, ℤ32)⟧
opr :(x: ℤ32) : RightRange⟦ℤ32⟧
opr :(xy: (ℤ32, ℤ32)) : RightRange⟦(ℤ32, ℤ32)⟧
```

opr : $\big(xyz\colon (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)$:

RightRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

(* Actually want: opr (x:T)#[\T\] : LeftRange[\T\] opr (x:T):[\T\] : LeftRange[\T\] opr #[\T\](x:T) : ExtentRange[\T\] opr :[\T\](x:T) : RightRange[\T\]
*)

opr $\#()$: TrivialOpenRange

opr : $()$: TrivialOpenRange

openRange$[\![I]\!]()$: OpenRange$[\![I]\!]$

Factories for bare strided ranges.

The :: operator is used to construct strided ranges with information missing: $A::$ is equivalent to A: or $A\#$. $::S$ takes every S'th element. $A::S$ is a range open to the right, starting with A and striding by S.

opr $(l\colon \mathbb{Z}32)::$: LeftRange$[\![\mathbb{Z}32]\!]$

opr $\big(l\colon (\mathbb{Z}32, \mathbb{Z}32)\big)::$: LeftRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

opr $\big(l\colon (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)::$: LeftRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

opr $::(s\colon \mathbb{Z}32)$: OpenRange$[\![\mathbb{Z}32]\!]$

opr $::\big(ij\colon (\mathbb{Z}32, \mathbb{Z}32)\big)$: OpenRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

opr $::\big(ijk\colon (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)$: OpenRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

opr $::(l\colon \mathbb{Z}32, s\colon \mathbb{Z}32)$: LeftRange$[\![\mathbb{Z}32]\!]$

opr $::\big(l\colon (\mathbb{Z}32, \mathbb{Z}32), s\colon (\mathbb{Z}32, \mathbb{Z}32)\big)$: LeftRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

opr $::\big(l\colon (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32), s\colon (\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big)$: LeftRange$\big[\!\big[(\mathbb{Z}32, \mathbb{Z}32, \mathbb{Z}32)\big]\!\big]$

Operators on ranges.

Most of these build on an existing range; however, due to operator associativity we can't always handle every case gracefully. Here are fully-parenthesized versions of tight uses of the range construction operators. We indicate which ones are handled:

$\big((A\colon B)\colon C\big)$ $\big((A \# B)\colon C\big)$ $\big((\colon A)\colon B\big)$ $\big((\#A)\colon B\big)$ $\big((::A)\colon B\big)$

All impose striding on the left-hand range. Note that in the case of $A \# B$ this will in general *decrease* the size of the range. You might want to actually write $(A :: B) \# C$, with the parentheses, if that's not what you want.

$\big(A\colon (B \# C)\big)$ For now we reject this. $\big(A\colon (B\colon)\big)$ Don't use these, just write $A\colon B$ instead. $\big(A\colon (B\#)\big)$ $\big(A\colon (B ::)\big)$

$\big((A \# B) \# C\big)$ Reject for now. $\big((\colon A) \# B\big)$ The remaining 3 are sized ranges specifying an upper bound. $\big((\#A) \# B\big)$ $\big((::A) \# B\big)$

$\big(A \# (B\colon)\big)$ Use $A \# B$ instead. $\big(A \# (B\#)\big)$ $\big(A \# (B ::)\big)$

The following are rejected at present:

$\big((A :: B) :: C\big)$ $\big((A \# B) :: C\big)$ $\big((A : B) :: C\big)$ $\big((: A) :: B\big)$ $\big((\# A) :: B\big)$ $\big((:: A) :: B\big)$

$\big(A :: (B \# C)\big)$ Note that you probably wanted $(A :: B) \# C$ (C elements, starting from A and striding by B). For the moment, parenthesize this. In this form we can't distinguish it from the next line, and the two should behave differently when written $A :: B \# C$ and $A :: B : C$. $\big(A :: (B : C)\big)$ $\big(A :: (B :)\big)$ $\big(A :: (B \#)\big)$ $\big(A :: (B ::)\big)$

opr : $[\![I]\!]\big(r\colon \text{Range}[\![I]\!], \mathit{stride} : I\big)\colon \text{Range}[\![I]\!]$
(* This doesn't obey the subtyping rule due to the genericity involved! opr :[\I extends Index\](r: FullRange[\I\], stride:I): FullRange[\I\] = r.imposeStride(stride)

*)

opr $\#[\![I]\!]\big(r\colon \text{PartialRange}[\![I]\!], \mathit{size} : I\big)\colon \text{Range}[\![I]\!]$

object SimpleMappedSeqIndexed$[\![E, F, I$ extends Index$]\!]\big(g_0\colon \text{Indexed}[\![E, I]\!], f_0\colon E \to F\big)$

extends $\{$ SequentialGenerator$[\![F]\!]$, Indexed$[\![F, I]\!]$ $\}$

getter $\mathit{size}()\colon \mathbb{Z}32$

getter $\mathit{bounds}()\colon \text{CompactFullRange}[\![I]\!]$

getter $\mathit{indexValuePairs}()\colon \text{Indexed}\big[\![(I, F), I]\!\big]$

getter $\mathit{indices}()\colon \text{Generator}[\![I]\!]$

opr $|$self$| : \mathbb{Z}32$

opr $[i : I] : F$

opr $\big[r\colon \text{Range}[\![I]\!]\big] : \text{Indexed}[\![F, I]\!]$

$\mathit{generate}[\![R]\!]\big(r\colon \text{Reduction}[\![R]\!], m\colon F \to R\big)\colon R$

seq(self)$\colon \text{SimpleMappedSeqIndexed}[\![E, F, I]\!]$

$\mathit{ivmap}[\![R]\!]\big(\mathit{ff} : (I, F) \to R\big)\colon \text{Indexed}[\![R, I]\!]$

$\mathit{map}[\![G]\!](f'\colon F \to G)\colon \text{SimpleMappedSeqIndexed}[\![E, G, I]\!]$

end

STRINGS

trait String extends $\{$ StandardTotalOrder$[\![$String$]\!]$, ZeroIndexed$[\![$Char$]\!]$, DelegatedIndexed$[\![$Char, $\mathbb{Z}32]\!]$ $\}$

excludes $\{$ AnyMultiplicativeRing $\}$

getter $\mathit{size}() : \mathbb{Z}32$

getter $\mathit{indices}() : \text{CompactFullRange}[\![\mathbb{Z}32]\!]$

getter $\mathit{left}()\colon \text{Maybe}[\![\text{Char}]\!]$

getter $\mathit{right}()\colon \text{Maybe}[\![\text{Char}]\!]$

`getter` *depth*() : $\mathbb{Z}32$

`getter` *asFlatString*(): String

`getter` *isBalanced*(): Boolean

`getter` *generator*(): Generator⟦Char⟧

verify() : () `(*` Verify the data structure invaraints of self `*)`

`abstract` *asDebugStringIndented*(*indent*: $\mathbb{Z}32$): String

`opr` |`self`| : $\mathbb{Z}32$

`opr` `CASE_INSENSITIVE_CMP`(`self`, *other*: String): TotalComparison

`opr` [*i*: $\mathbb{Z}32$]: Char

```
    As a convenience, we permit LeftRange indexing to go 1 past the bounds of the
string, returning the empty string, in order to permit some convenient string-trimming
idioms.
```

`opr` $\left[r_0\text{: Range}[\![\mathbb{Z}32]\!]\right]$: String

`abstract` *get*(*i*: $\mathbb{Z}32$): Char

```
    This version is like [ ] above, but does not do the bounds checking.  Really,
this should be in a "friends" interface.  It is the responsibility of uncheckedSubstring
to do whatever optimizationms for empty ranges and trivial (whole string) ranges
may be appropriate for the representation.
```

uncheckedSubstring$\left(r_0\text{: Range}[\![\mathbb{Z}32]\!]\right)$: String

allButLast(): String

allButFirst(): String

rangeContains$\left(r\text{: Range}[\![\mathbb{Z}32]\!], c\text{: Char}\right)$: Boolean

```
    Answers a subdivision of self into substrings.  This method must take time
proportional to the number of pairs generated.  If there is no such subdivision,
it's fine to answer the empty generator Nothing, but in this case the implementation
of the subtrait must offer linear time indexing.  The pairs (start, str) that are
generated by this method must be such that start[0] = 0, and start[i] = | str[0]
|| ...  || str[i-1] |, and str[0] || str [1] || ...  || str[n] = self
```

`abstract` *splitWithOffsets*(): ZeroIndexed⟦($\mathbb{Z}32$, String)⟧

`abstract` *split*(): Generator⟦String⟧

```
    A balanced version of the receiver
```

balanced(): String `(*` ensures {outcome.isAlmostBalanced AND outcome = self} `*)`

`opr`$^{\texttt{self},n:\mathbb{Z}32}$: String

upto(*c*: Char): String

beyond(*c*: Char): String

```
    The operator ‖ with at least one String argument converts to string and appends
```

`opr` ‖(`self`, *b*: Number): String

`opr` ‖(`self`, *c*: Char): String

`opr` ‖(`self`, *b*: ()): String

`opr` ‖(`self`, *b*: (Any, Any)): String

`opr` ‖(`self`, *b*: (Any, Any, Any)): String

```
    opr // concatenates with a single newline separator.
```

`opr` //(`self`) : String

```
    opr /// concatenates with a double newline separator
```

`opr` ///(`self`) : String

`abstract` *writeOn*(*s*: WriteStream): ()

stats(): StringStats

join[[*E*]](*g*: Generator[[*E*]]): String

`end` String

```
The operator ‖ with at least one String argument converts to string and appends
```

`opr` ‖(*a*: Any, *b*: String): String

`opr` ‖(*a*: String, *b*: String): String

`opr` ‖(*a*: String, *b*: Any): String

```
The operator ||| with at least one String argument converts to string, then concatenates
with a whitespace separator unless one of the two arguments is empty.  If there
is an empty argument, the other argument is returned.
```

`opr` |||(*a*: Any, *b*: String): String

`opr` |||(*a*: String, *b*: String): String

`opr` |||(*a*: String, *b*: Any): String

```
opr // concatenates with a single newline separator.
```

`opr` //(*a*: Any, *b*: String): String

`opr` //(*a*: String, *b*: String): String

`opr` //(*a*: String, *b*: Any): String

`opr /// concatenates with a double newline separator`

`opr` ///(*a*: Any, *b*: String): String

`opr` ///(*a*: String, *b*: String): String

`opr` ///(*a*: String, *b*: Any): String

`Right now for backward compatibility juxtaposition works like` ‖

`opr` *juxtaposition*(*a*: Any, *b*: String): String

`opr` *juxtaposition*(*a*: String, *b*: String): String

`opr` *juxtaposition*(*a*: String, *b*: Any): String

`object` StringJoinReduction(*s*: String) `extends` Reduction⟦Maybe⟦String⟧⟧ `end`

reverse(*sequence*: String): String

29.1.2 Top-level primitives

nanoTime `returns the current time in nanoseconds. Currently, this only supports taking differences of results of` *nanoTime* `to produce an elapsed time interval.`

nanoTime(): ℤ64

printTaskTrace `dumps some internal error state.`

printTaskTrace(): ()

recordTime(*dummy*: Any): ()

printTime(*dummy*: Any): ()

fromRawBits(*a*: ℤ64): ℝ64

random(*a*: Number): ℝ64

$randomZZ_{32}(x: \mathbb{Z}32): \mathbb{Z}32$

match(*regex*: FlatString, *some*: FlatString): Boolean

char `converts an integer unicode code point into the corresponding 16-bit` Char. `Note that we don't presently deal gracefully with` Chars `outside the 16-bit plane.`

char(*a*: ℤ32): Char

print(*a*: String): ()

println(*a*: String): ()

errorPrint(*a*: String): ()

errorPrintln(*a*: String): ()

print(*a*: Number): ()

println(*a*: Number): ()

errorPrintln(*a*: Number): ()

print(*a*: Boolean): ()

println(*a*: Boolean): ()

errorPrintln(*a*: Boolean): ()

print(*a*: Any): ()

println(*a*: Any): ()

errorPrint(*a*: Any): ()

errorPrintln(*a*: Any): ()

0-argument versions handle passing of () to single-argument versions.

print(): ()

println(): ()

errorPrint(): ()

errorPrintln(): ()

forDigit(*x*: ℤ32, *radix*: ℤ32): Maybe⟦Char⟧

forDigit(*x*: ℤ32, *radixString*: String): Maybe⟦Char⟧

Convert string to arbitrary integral type. String may contain leading - or +. Valid radixes are 2,8,10,12, and 16. No spaces may occur in the string, and it must have at least one digit.

Right now we're oblivious to overflow! This ought to be fixed.

This ought to support arbitrary integer types, but right now there's no clean way to convert all the arithmetic to use a provided type without having an instance of that type in hand. Unsigned types in particular are a bit sticky.

strToInt(*s*: String, *radix*: ℤ32): ℤ32

Radix-10 digit conversion. All caveats of the flexible-radix conversion above apply.

strToInt(*s*: String): ℤ32

strToFloat(*s*: String): ℝ64

opr // appends a single newline separator.

opr (*x*: Any)// : String

opr /// appends a double newline separator

opr $(x\colon \text{Any})/\!/\!/\ \colon \text{String}$

The following three functions are useful temporary hacks for debugging multi-threaded programs.

printThreadInfo$(a\colon \text{String})\colon ()$

printThreadInfo$(a\colon \text{Number})\colon ()$

printWithThread$(a\colon \text{String})\colon ()$

*println**WithThread*$(a\colon \text{String})\colon ()$
(* time to get rid of this: throwError(a:String):()
*)

opr SEQV$(a\colon \text{Any}, b\colon \text{Any})\colon \text{Boolean}$

opr $\veebar(a\colon \text{Boolean}, b\colon \text{Boolean})\colon \text{Boolean}$

opr $\vee(a\colon \text{Boolean}, b\colon \text{Boolean})\colon \text{Boolean}$

opr $\wedge(a\colon \text{Boolean}, b\colon \text{Boolean})\colon \text{Boolean}$

opr $\vee\big(a\colon \text{Boolean}, b\colon () \to \text{Boolean}\big)\colon \text{Boolean}$

opr $\wedge\big(a\colon \text{Boolean}, b\colon () \to \text{Boolean}\big)\colon \text{Boolean}$

opr $\neg(a\colon \text{Boolean})\colon \text{Boolean}$

opr $\to(a\colon \text{Boolean}, b\colon \text{Boolean})\colon \text{Boolean}$

opr $\to\big(a\colon \text{Boolean}, b\colon () \to \text{Boolean}\big)\colon \text{Boolean}$

opr $\leftrightarrow(a\colon \text{Boolean}, b\colon \text{Boolean})\colon \text{Boolean}$

⊛true : Boolean
⊛false : Boolean

opr $+[\![T \texttt{ extends } \text{Number}]\!](x : T) : T$

opr $=[\![A, B]\!]\big(t_1 : (A, B), t_2 : (A, B)\big)\colon \text{Boolean}$

opr $<\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!]\big]\!\big]\big(t_1 : (A, B), t_2 : (A, B)\big)\colon \text{Boolean}$

opr $\leq\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!]\big]\!\big]\big(t_1 : (A, B), t_2 : (A, B)\big)\colon \text{Boolean}$

opr $>\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!]\big]\!\big]\big(t_1 : (A, B), t_2 : (A, B)\big)\colon \text{Boolean}$

opr $\geq\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!]\big]\!\big]\big(t_1 : (A, B), t_2 : (A, B)\big)\colon \text{Boolean}$

opr CMP$\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!]\big]\!\big]\big(t_1 : (A, B), t_2 : (A, B)\big)\colon \text{Comparison}$

opr $=[\![A, B, C]\!]\big(t_1 : (A, B, C), t_2 : (A, B, C)\big)\colon \text{Boolean}$

opr $<\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!], C \texttt{ extends } \text{StandardPartialOrder}[\![C]\!]\big]\!\big]\big(t$

opr $\leq\big[\![A \texttt{ extends } \text{StandardPartialOrder}[\![A]\!], B \texttt{ extends } \text{StandardPartialOrder}[\![B]\!], C \texttt{ extends } \text{StandardPartialOrder}[\![C]\!]\big]\!\big]\big(t$

`opr` $>[\![A$ `extends` StandardPartialOrder$[\![A]\!], B$ `extends` StandardPartialOrder$[\![B]\!], C$ `extends` StandardPartialOrder$[\![C]\!]\!]\big(t$

`opr` $\geq[\![A$ `extends` StandardPartialOrder$[\![A]\!], B$ `extends` StandardPartialOrder$[\![B]\!], C$ `extends` StandardPartialOrder$[\![C]\!]\!]\big(t$

`opr CMP`$[\![A$ `extends` StandardPartialOrder$[\![A]\!], B$ `extends` StandardPartialOrder$[\![B]\!], C$ `extends` StandardPartialOrder$[\![C]\!]\!]$

`(*`——————————— for Generators-of-Generators ————————`*)`

switchDispatching$(r:\text{Boolean}):()$

dispatchingEnabled$():\text{Boolean}$

`trait` SomeGenerator2 `end (*` marking for GG `*)`

`(*` non-filtered GG `*)`

`trait` Generator2$[\![\,E\,]\!]$

`extends` $\big\{\text{Generator}[\![\,\text{Generator}[\![\,E\,]\!]\,]\!], \text{SomeGenerator2}\big\}$

`getter` *seed*$():\text{Generator}[\![\,E\,]\!]$

generate$_2[\![\,R, \text{L1}, \text{L2}\,]\!]\big(q:\text{ActualReduction}[\![\,R, \text{L1}\,]\!], r:\text{ActualReduction}[\![\,R, \text{L2}\,]\!], f:E\to R\big):R$

theorems$[\![R, \text{L1}, \text{L2}]\!]():\text{ZeroIndexed}[\![((\text{ActualReduction}[\![\,R, \text{L1}\,]\!], \text{ActualReduction}[\![\,R, \text{L2}\,]\!], E\to R)\to\text{Boolean}, (\text{ActualRed}$

`(*` this is just for internal use `*)`

__generate2filtered$[\![\,R, \text{L1}, \text{L2}\,]\!]\big(q:\text{ActualReduction}[\![\,R, \text{L1}\,]\!], r:\text{ActualReduction}[\![\,R, \text{L2}\,]\!], p:\text{Generator}[\![E]\!]\to\text{Condition}[\![()]\!],$

theoremsFiltered$[\![R, \text{L1}, \text{L2}]\!]():\text{ZeroIndexed}[\![((\text{ActualReduction}[\![\,R, \text{L1}\,]\!], \text{ActualReduction}[\![\,R, \text{L2}\,]\!], \text{Generator}[\![E]\!]\to\text{Condit}$

filter$\big(p\text{: Generator}[\![\,E\,]\!]\to\text{Condition}[\![()]\!]\big):\text{Generator2}[\![E]\!]$

`end`

`(*`——————————Relational Predicates ——————————`*)`

`(*` base of relational predicates condition `*)`

`trait` RelationalPredicateCondition$[\![E]\!]$ `extends` $\big\{\,\text{Condition}[\![()]\!]\,\big\}$

relation$():(E,E)\to\text{Boolean}$

target$():\text{Generator}[\![\,E\,]\!]$

cond$[\![G]\!]\big(t\text{: }()\to G, e\text{: }()\to G\big):G$

`end`

`(*` Shortcut to create a relational predicate `*)`

relationalPredicate$[\![E]\!]\big(\text{relation}:(E,E)\to\text{Boolean}\big):\text{Generator}[\![\,E\,]\!]\to\text{RelationalPredicateCondition}[\![\,E\,]\!]$

⊛ Some random stuff related to arrays and the MCKPE benchmark

opr $\overrightarrow{\sum}(x\colon \text{Array}[\![\mathbb{Z}32, \mathbb{Z}32]\!]) \colon \text{Array}[\![\mathbb{Z}32, \mathbb{Z}32]\!]$

opr $\overleftarrow{\sum}(x\colon \text{Array}[\![\mathbb{Z}32, \mathbb{Z}32]\!]) \colon \text{Array}[\![\mathbb{Z}32, \mathbb{Z}32]\!]$

⊛ Scalar extension: an ar
⊛ All eight return the uns
⊛ operators keep rank an
⊛ time, since map builds
⊛ checker sees only Arra
⊛ to it in the result.

opr $+[\![T \texttt{ extends } \{\text{Number}, \text{AdditiveGroup}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](x\colon \text{Array}[\![T, I]\!], y\colon T) \colon \text{Array}[\![T, I]\!]$
opr $+[\![T \texttt{ extends } \{\text{Number}, \text{AdditiveGroup}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](y\colon T, x\colon \text{Array}[\![T, I]\!]) \colon \text{Array}[\![T, I]\!]$
opr $-[\![T \texttt{ extends } \{\text{Number}, \text{AdditiveGroup}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](x\colon \text{Array}[\![T, I]\!], y\colon T) \colon \text{Array}[\![T, I]\!]$

⊛ Scalar on the left: y - x
⊛ This is the only pair he
⊛ the scalar DOT and jux

opr $-[\![T \texttt{ extends } \{\text{Number}, \text{AdditiveGroup}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](y\colon T, x\colon \text{Array}[\![T, I]\!]) \colon \text{Array}[\![T, I]\!]$

opr MIN$[\![T \texttt{ extends } \{\text{Number}, \text{StandardMin}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](x\colon \text{Array}[\![T, I]\!], y\colon T) \colon \text{Array}[\![T, I]\!]$

opr MIN$[\![T \texttt{ extends } \{\text{Number}, \text{StandardMin}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](y\colon T, x\colon \text{Array}[\![T, I]\!]) \colon \text{Array}[\![T, I]\!]$

opr MAX$[\![T \texttt{ extends } \{\text{Number}, \text{StandardMax}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](x\colon \text{Array}[\![T, I]\!], y\colon T) \colon \text{Array}[\![T, I]\!]$

opr MAX$[\![T \texttt{ extends } \{\text{Number}, \text{StandardMax}[\![T]\!]\}, I \texttt{ extends } \text{Index}]\!](y\colon T, x\colon \text{Array}[\![T, I]\!]) \colon \text{Array}[\![T, I]\!]$

end

29.2 Builtins

There is a single library containing builtins: FortressBuiltin (Section 29.2.1). Every Fortress API and component will unconditionally see the names exported by this builtin API.

29.2.1 FortressBuiltin

api FortressBuiltin

```
The builtinPrimitive function is actually recognized as a special piece of built-in
magic by the Fortress interpreter.  The javaClass argument names a Java Class which
is a subclass of com.sun.fortress.interpreter.glue.NativeApp, which provides code
for the closure which is used in place of the call to builtinPrimitive.  Meanwhile all
the necessary type information, argument names, etc.  must be declared here in Fortress-lan
For examples, see the end of this file.

In practice, if you are extending the interpreter you will probably want to extend
com.sun.fortress.interpreter.glue.NativeFn0,1,2,3,4 or one of their subclasses defined
in com.sun.fortress.interpreter.glue.primitive.  These types are generally easier
to work with, and the boilerplate packing and unpacking of values is done for you.
```

builtinPrimitive⟦T⟧(*javaClass*: String) : T

`trait` Object `extends` Any

`getter` *ilkName*(): String

`getter` *asString*(): String `(*` for normal use `*)`

`getter` *asDebugString*(): String `(*` for debugging; may contain more information `*)`

`getter` *asExprString*(): String `(*` when considered as Fortress expression, will equal self `*)`

`getter` *toString*(): String `(*` deprecated `*)`

`end`

`value object` Float `extends` ℝ64

`end`

`object` FloatLiteral `extends` ℝ64

`end`

`value object` ℝ32 `extends` { Number, StandardPartialOrder⟦ℝ32⟧, StandardMinMax⟦ℝ32⟧, AdditiveGroup⟦ℝ32⟧, MultiplicativeRing⟦ℝ32⟧ }

`returns true if the value is an IEEE NaN`

`getter` *isNaN*(): Boolean

`returns true if the value is an IEEE infinity`

`getter` *isInfinite*(): Boolean

`returns true if the value is a valid number (not NaN)`

`getter` *isNumber*(): Boolean

`returns true if the value is finite`

`getter` *isFinite*(): Boolean

check `returns` Just(*its argument*) `if it is finite, otherwise` Nothing`.`

`getter` *check*(): Maybe⟦ℝ32⟧

check* `returns` Just(*its argument*) `if it is non-NaN, otherwise` Nothing`.`

`getter` *check**(): Maybe⟦ℝ32⟧

`obtain the raw bits of the IEEE floating-point representation of this value.`

`getter` *rawBits*(): ℤ32

`obtain the sign bit of the IEEE floating-point representation of this value.`

`getter` *signBit*(): ℤ32

`next higher IEEE float`

getter *nextUp*(): ℝ32

next lower IEEE float

getter *nextDown*(): ℝ32

opr |self| : ℝ32

opr =(self, *b*: ℝ32): Boolean

opr ≠(self, *b*: ℝ32): Boolean

opr <(self, *b*: ℝ32): Boolean

opr ≤(self, *b*: ℝ32): Boolean

opr >(self, *b*: ℝ32): Boolean

opr ≥(self, *b*: ℝ32): Boolean

opr CMP(self, *b*: ℝ32): Comparison

In case of NaN, MIN and MAX return a NaN, otherwise it respects the total order.

opr MIN(self, *b*: ℝ32): ℝ32

opr MAX(self, *b*: ℝ32): ℝ32

opr MINMAX(self, *b*: ℝ32): (ℝ32, ℝ32)

opr −(self): ℝ32

opr +(self, *b*: ℝ32): ℝ32

opr −(self, *b*: ℝ32): ℝ32

opr ·(self, *b*: ℝ32): ℝ32

opr ×(self, *b*: ℝ32): ℝ32

opr *juxtaposition*

(self, *b*: ℝ32): ℝ32

opr /(self, *b*: ℝ32): ℝ32

opr $\sqrt{\text{self}}$: ℝ32

opr PLUS_UP(self, *b*: ℝ32): ℝ32

opr MINUS_UP(self, *b*: ℝ32): ℝ32

opr DOT_UP(self, *b*: ℝ32): ℝ32

opr SLASH_UP(self, *b*: ℝ32): ℝ32

opr SQRT_UP(self): ℝ32

opr PLUS_DOWN(self, *b*: ℝ32): ℝ32

opr MINUS_DOWN(self, *b*: ℝ32): ℝ32

```
  opr DOT_DOWN(self, b: ℝ32): ℝ32
  opr SLASH_DOWN(self, b: ℝ32): ℝ32
  opr SQRT_DOWN(self): ℝ32
  opr IEEE_PLUS_UP(self, b: ℝ32): ℝ32
  opr IEEE_MINUS_UP(self, b: ℝ32): ℝ32
  opr IEEE_DOT_UP(self, b: ℝ32): ℝ32
  opr IEEE_SLASH_UP(self, b: ℝ32): ℝ32
  opr IEEE_PLUS_DOWN(self, b: ℝ32): ℝ32
  opr IEEE_MINUS_DOWN(self, b: ℝ32): ℝ32
  opr IEEE_DOT_DOWN(self, b: ℝ32): ℝ32
  opr IEEE_SLASH_DOWN(self, b: ℝ32): ℝ32
  opr^self,b:Float: ℝ64
  opr^self,b:AnyIntegral: ℝ32
  floor(self): ℝ32
  opr ⌊self⌋ : ℤ64
  ceiling(self): ℝ32
  opr ⌈self⌉ : ℤ64
  truncate(self): ℤ64
  round(self): ℤ64
  opr^self,b:ℝ32: ℝ32

      MINNUM and MAXNUM return a numeric result where possible (avoiding NaN). Note
that MINNUM and MAX form a lattice with NaN at the top, and that MAXNUM and MIN
form a lattice with NaN at the bottom.

  opr MINNUM(self, b: ℝ32): ℝ32
  opr MAXNUM(self, b: ℝ32): ℝ32
end

value object Int extends ℤ32
end

value object Long extends ℤ64
end

value object ℕ32 extends { AnyIntegral, Integral⟦ℕ32⟧ }
```

`coerce` (x: IntLiteral)

`opr` $|$`self`$|$: $\mathbb{N}32$

`opr` $=$(`self`, b: $\mathbb{N}32$): Boolean

`opr` $<$(`self`, b: $\mathbb{N}32$): Boolean

`opr` $>$(`self`, b: $\mathbb{N}32$): Boolean

`opr` $\geq$(`self`, b: $\mathbb{N}32$): Boolean

`opr` $\leq$(`self`, b: $\mathbb{N}32$): Boolean

`opr` `CMP`(`self`, b: $\mathbb{N}32$): TotalComparison

`opr` `MIN`(`self`, *other*: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `MAX`(`self`, *other*: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `MINMAX`(`self`, *other*: $\mathbb{N}32$): ($\mathbb{N}32$, $\mathbb{N}32$)

`opr` `MINNUM`(`self`, *other*: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `MAXNUM`(`self`, *other*: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $-$(`self`): $\mathbb{N}32$

`opr` $\dot{-}$(`self`): $\mathbb{N}32$

`opr` $+$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\dot{+}$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $-$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\dot{-}$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\cdot$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\times$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` *juxtaposition*(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\dot{\times}$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\div$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `REM`(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `MOD`(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `GCD`(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `LCM`(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` `CHOOSE`(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\curlywedge$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

`opr` $\curlyvee$(`self`, b: $\mathbb{N}32$): $\mathbb{N}32$

```
    opr ⩒(self, b: ℕ32): ℕ32
    opr LSHIFT(self, b: AnyIntegral): ℕ32
    opr RSHIFT(self, b: AnyIntegral): ℕ32
    opr ¬(self): ℕ32
    opr^self,b:AnyIntegral: ℕ32
    opr^self,b:IntLiteral: ℕ32
    opr /(self, other: ℕ32): ℚ
    numerator(self): ℤ
    widen(self): ℕ64
    partitionL(self): ℕ32
    signed(self): ℤ32
end

value object UnsignedLong extends ℕ64
end

object IntLiteral extends { Number }
    getter zero(): IntLiteral
    getter one(): IntLiteral
    getter asZZ_32(): ℤ32
    getter asZZ_64(): ℤ64
    getter asNN_32(): ℕ32
    getter asNN_64(): ℕ64
    getter asZZ(): ℤ
    opr |self| : IntLiteral
    opr =(self, b: IntLiteral): Boolean
    opr <(self, other: IntLiteral): Boolean
    opr ≤(self, other: IntLiteral): Boolean
    opr >(self, other: IntLiteral): Boolean
    opr ≥(self, other: IntLiteral): Boolean
    opr CMP(self, other: IntLiteral): TotalComparison
    opr −(self): IntLiteral
```

```
    opr +(self, b: IntLiteral): IntLiteral
    opr −(self, b: IntLiteral): IntLiteral
    opr ·(self, b: IntLiteral): IntLiteral
    opr juxtaposition(self, b: IntLiteral): IntLiteral
    opr ×(self, b: IntLiteral): IntLiteral
    opr ÷(self, b: IntLiteral): IntLiteral
    opr REM(self, b: IntLiteral): IntLiteral
    opr MOD(self, b: IntLiteral): IntLiteral
    opr GCD(self, b: IntLiteral): IntLiteral
    opr LCM(self, b: IntLiteral): IntLiteral
    opr CHOOSE(self, b: IntLiteral): IntLiteral
    opr ⩓(self, b: IntLiteral): IntLiteral
    opr ⩔(self, b: IntLiteral): IntLiteral
    opr ⩒(self, b: IntLiteral): IntLiteral
    opr LSHIFT(self, b: AnyIntegral): IntLiteral
    opr RSHIFT(self, b: AnyIntegral): IntLiteral
    opr ∋(self): IntLiteral
    opr^self,b:AnyIntegral: IntLiteral
    opr^self,b:IntLiteral: IntLiteral
    opr |(self, b: IntLiteral): Boolean
    floor(self): IntLiteral
    ceiling(self): IntLiteral
    truncate(self): IntLiteral
    even(self): Boolean
    odd(self): Boolean
end

value object BigNum extends ℤ end

value object Boolean
    extends { Condition⟦()⟧, StandardTotalOrder⟦Boolean⟧ }
end

value object Char extends { StandardTotalOrder⟦Char⟧ }
```

char.codePoint converts *char* to the equivalent integer code point. It is always the case that $c = char\big(c.codePoint()\big)$ for c: Char.

getter *codePoint*(): $\mathbb{Z}32$

|c| means the same as *c.ord*(); it's unclear if this is actually a good idea, and we solicit feedback on the subject.

opr |self|: $\mathbb{Z}32$

Ordering respects *codePoint*.

opr $=$(self, *other*: Char): Boolean

opr $<$(self, *other*: Char): Boolean

opr $\simeq$(self, *other*: Char): Boolean

opr $\not\simeq$(self, *other*: Char): Boolean

opr LNSIM(self, *other*: Char): Boolean

opr LESSSIM(self, *other*: Char): Boolean

opr GNSIM(self, *other*: Char): Boolean

opr GTRSIM(self, *other*: Char): Boolean

(* The following methods have the same behavior as the methods in Java Character class, except for methods digit and forDigit. These two particular methods deviate from Java Character class when it gets argument radix = 12. For radix 12, the digits are "0123456789xe" instead of "0123456789ab". *)

(* javaDigit(self, radix:ZZ32): ZZ32 *)

digit(self): Maybe$[\![\mathbb{Z}32]\!]$ (* radix 10 *)

digit(self, *radix*: $\mathbb{Z}32$): Maybe$[\![\mathbb{Z}32]\!]$

getDirectionality(self): $\mathbb{Z}32$

getNumericValue(self): $\mathbb{Z}32$

getType(self): $\mathbb{Z}32$

isDefined(self): Boolean

isDigit(self): Boolean

isFortressIdentifierPart(self): Boolean

isFortressIdentifierStart(self): Boolean

isHighSurrogate(self): Boolean

isIdentifierIgnorable(self): Boolean

isISOControl(self): Boolean

isJavaIdentifierPart(self): Boolean

isJavaIdentifierStart(self): Boolean

isLetter(`self`): Boolean

isLetterOrDigit(`self`): Boolean

isLowerCase(`self`): Boolean

isLowSurrogate(`self`): Boolean

isMirrored(`self`): Boolean

isSpaceChar(`self`): Boolean

isSupplementaryCodePoint(`self`): Boolean

isSurrogatePair(`self`, *low*: Char): Boolean

isTitleCase(`self`): Boolean

isUnicodeIdentifierPart(`self`): Boolean

isUnicodeIdentifierStart(`self`): Boolean

isUpperCase(`self`): Boolean

isValidCodePoint(`self`): Boolean

isWhitespace(`self`): Boolean

toLowerCase(`self`): Char

toTitleCase(`self`): Char

toUpperCase(`self`): Char

`end`

`object` Thread$[\![T]\!]\big(fcn\colon () \to T\big)$

val() : T

ready(): Boolean

wait(): ()

stop(): ()

`end`

abort(): ()

`end`

Chapter 30

Additional Libraries Available to Fortress Programmers

The libraries described in this chapter must be explicitly imported into Fortress programs.

30.1 Constants

`api` Constants

`Useful floating-point constants.`

π : FloatLiteral

e : FloatLiteral

infinity : $\mathbb{R}64$

`end`

30.2 List

`api` List

`Array lists, immutable style (not the mutable Java ArrayList style).`

`A` List `is an immutable segment of an immutable (really write-once) array. The rest of the array may contain elements of lists which overlap this list, or may be free for future use. Every` List `includes two internal flags` *canExtendLeft* `and` *canExtendRight*`; if a flag is true we are permitted to add additional elements to the` List `in place by initializing additional elements of the array. At most one instance sharing the same backing array will obtain permission to extend the array in this way; we atomically check and update the flag to guarantee this. Having obtained permission to extend the list, that permission may be extended to future attempts to extend.`

Eventually the backing array fills and we must allocate a new backing array to accept new elements. At the moment, we are not particularly careful to avoid stealing permission to extend for overflowing ‖ operations.

Note that because of this implementation, a List can be efficiently extended on either side, but only in a non-persistent way; if a single list is extended by two different calls to *addRight* or ‖ then one of them must pay the cost of copying the list elements.

Note also that the implementation has not yet been carefully checked for amortization, so it is quite likely there are a number of asymptotic infelicities.

Finally, note that this is an efficient *amortized* structure. An individual operation may be quite slow due to copying work.

Baking these off vs PureLists (which have good persistent behavior and non-amortized worst case behavior), they look very good in practice. Lists of some item type. Used to collect elements of unknown type into a list whose element type is as specific as possible. This should not be necessary in the presence of true type inference.

trait AnyList **excludes** { Number, HasRank, String }

comprises { List⟦*E*⟧ } **where** ⟦*E*⟧

opr ‖(**self**, *f*: AnyList): AnyList

addLeft(*e*: Any): AnyList

addRight(*e*: Any): AnyList

end

List. We return a Generator for non-list-specific operations for which reuse of the Generator will not increase asymptotic complexity, but return a List in cases (such as *map* and *filter*) where it will. Indexing on lists operates from the left, and ordering is lexicographic reading from the left.

trait List⟦*E*⟧ **extends** { AnyList, ZeroIndexed⟦*E*⟧, Equality⟦List⟦*E*⟧⟧,

LexicographicOrder⟦List⟦*E*⟧, *E*⟧ **where** { *E* **extends** StandardTotalOrder⟦*E*⟧ } }

excludes { Number, HasRank, String }

left and *extractLeft* return the leftmost element in the list (and in the latter case, the remainder of the list without its leftmost element). They return Nothing if the list is empty. *right* and *extractRight* do the same with the rightmost element.

getter *left*(): Maybe⟦*E*⟧

getter *right*(): Maybe⟦*E*⟧

abstract getter *extractLeft*(): Maybe⟦(*E*, List⟦*E*⟧)⟧

abstract getter *extractRight*(): Maybe⟦(List⟦*E*⟧, *E*)⟧

getter *reverse*(): List⟦*E*⟧

the operator ‖ returns a list containing the elements of **self** followed by the elements of *f*

abstract opr $\|\big(\texttt{self}, \mathit{other}\colon \mathrm{List}[\![E]\!]\big)\colon \mathrm{List}[\![E]\!]$

= compares two lists element by element, whether or not their elements are ordered.

opr $=\big(\texttt{self}, \mathit{other}\colon \mathrm{List}[\![E]\!]\big)\colon \mathrm{Boolean}$

addLeft and *addRight* add an element to the left or right of the list, respectively

abstract $\mathit{addLeft}(e : E)\colon \mathrm{List}[\![E]\!]$

abstract $\mathit{addRight}(e : E)\colon \mathrm{List}[\![E]\!]$

take returns the leftmost n elements of the list; if the list is shorter than this, the entire list is returned.

abstract $\mathit{take}(n\colon \mathbb{Z}32)\colon \mathrm{List}[\![E]\!]$

drop drops the leftmost n elements of the list; if the list is shorter than this, an empty list is returned.

abstract $\mathit{drop}(n\colon \mathbb{Z}32)\colon \mathrm{List}[\![E]\!]$

abstract opr $[n\colon \mathbb{Z}32]\colon E$

$l.\mathit{split}(n)$ is equivalent to $\big(l.\mathit{take}(n), l.\mathit{drop}(n)\big)$. Note in particular that appending its results yields the original list.

abstract $\mathit{split}(n\colon \mathbb{Z}32)\colon \big(\mathrm{List}[\![E]\!], \mathrm{List}[\![E]\!]\big)$

split splits the list into two smaller lists. If $|l| > 1$ both lists will be non-empty.

abstract $\mathit{split}()\colon \big(\mathrm{List}[\![E]\!], \mathrm{List}[\![E]\!]\big)$

$\mathit{filter}(p\colon E \to \mathrm{Boolean})\colon \mathrm{List}[\![E]\!]$

concatMap is an in-place version of the *nest* method from Generator; it flattens the result into an actual list, rather than returning an abstract Generator.

$\mathit{concatMap}[\![G]\!]\big(f\colon E \to \mathrm{List}[\![G]\!]\big)\colon \mathrm{List}[\![G]\!]$

end

Vararg factory for lists; provides aggregate list constants:

opr $\langle [\![E]\!]\ \mathit{xs}\colon E \ldots \rangle\colon \mathrm{List}[\![E]\!]$

List comprehensions:

opr BIG $\langle [\![T]\!] \rangle\colon \mathrm{Comprehension}\big[\![T, \mathrm{List}[\![T]\!], \mathrm{List}[\![T]\!], \mathrm{List}[\![T]\!]\big]\!]$

opr BIG $\langle [\![T]\!]\ g\colon \mathrm{Generator}[\![T]\!] \rangle\colon \mathrm{List}[\![T]\!]$

opr BIG CONCAT$[\![T]\!]()\colon \mathrm{BigReduction}\big[\![\mathrm{List}[\![T]\!], \mathrm{List}[\![T]\!]\big]\!]$

opr BIG CONCAT$[\![T]\!]\big(g\colon \mathrm{Generator}[\![\mathrm{List}[\![T]\!]]\!]\big)\colon \mathrm{List}[\![T]\!]$

Convert generator into list (simpler type than comprehension above):

$\mathit{list}[\![E]\!]\big(g\colon \mathrm{Generator}[\![E]\!]\big)\colon \mathrm{List}[\![E]\!]$

Flatten a list of lists

concat⟦*E*⟧(*lists*: Indexed⟦List⟦*E*⟧, ℤ32⟧): List⟦*E*⟧

emptyList⟦*E*⟧(): List⟦*E*⟧

emptyList⟦*E*⟧(*n*) allocates an empty list that can accept *n* *addRight* operations without reallocating the underlying storage.

emptyList⟦*E*⟧(*n*: ℤ32): List⟦*E*⟧

singleton⟦*E*⟧(*e* : *E*): List⟦*E*⟧

A reduction object for concatenating lists.

object Concat⟦*E*⟧ **extends** MonoidReduction⟦ List⟦*E*⟧ ⟧

empty(): List⟦*E*⟧

join(*a*: List⟦*E*⟧, *b*: List⟦*E*⟧): List⟦*E*⟧

end

transpose⟦*E*, *F*⟧(*xs*: List⟦(*E*, *F*)⟧): (List⟦*E*⟧, List⟦*F*⟧)

end

30.3 PureList

api PureList

import CovariantCollection.{...}

Finger trees, based on Ralf Hinze and Ross Paterson's article, Journal of Funtional Programming 16:2 2006 [14].

These are API-compatible with the List library, except that they do not support covariant construction. In most cases, you should be able to replace an import of List by PureList or vice versa and see only performance differences between the two.

Why finger trees? They are balanced and support nearly any operation we care to think of in optimal asymptotic time and space. The code is niggly due to lots of cases, but fast in practice.

It is also a trial for encoding type-based invariants in Fortress. Can we represent ``array of size at most *n*''? Not yet, but we ought to be able to do so. This involves questions about the encoding of existentials, especially constrained existentials If you are curious about the details of type-based invariants, the source code may prove instructive. List. We return a Generator for non-list-specific operations for which reuse of the Generator will not increase asymptotic complexity, but return a List in cases (such as *map* and *filter*) where it will. Indexing on lists operates from the left, and ordering is lexicographic reading from the left.

trait List⟦*E*⟧ **extends** { Equality⟦*E*⟧, ZeroIndexed⟦*E*⟧ }

excludes { Number, HasRank, String }

left and *extractLeft* return the leftmost element in the list (and in the latter case, the remainder of the list without its leftmost element). They return Nothing if the list is empty. *right* and *extractRight* do the same with the rightmost element.

`getter` *left*(): Maybe⟦*E*⟧

`getter` *right*(): Maybe⟦*E*⟧

`getter` *extractLeft*(): Maybe⟦(*E*, List⟦*E*⟧)⟧

`getter` *extractRight*(): Maybe⟦(List⟦*E*⟧, *E*)⟧

`getter` *reverse*(): List⟦*E*⟧

append returns a list containing the elements of `self` followed by the elements of *f*

append(*f*: List⟦*E*⟧): List⟦*E*⟧

the operator ‖ performs the *append* operation

`opr` ‖(`self`, *other*: List⟦*E*⟧): List⟦*E*⟧

addLeft and *addRight* add an element to the left or right of the list, respectively

addLeft(*e* : *E*): List⟦*E*⟧

addRight(*e* : *E*): List⟦*E*⟧

take returns the leftmost *n* elements of the list; if the list is shorter than this, the entire list is returned.

take(*n*: ℤ32): List⟦*E*⟧

drop drops the leftmost *n* elements of the list; if the list is shorter than this, an empty list is returned.

drop(*n*: ℤ32): List⟦*E*⟧

l.*split*(*n*) is equivalent to (*l*.*take*(*n*), *l*.*drop*(*n*)). Note in particular that appending its results yields the original list.

split(*n*: ℤ32): (List⟦*E*⟧, List⟦*E*⟧)

split splits the list into two smaller lists. If $|l| > 1$ both lists will be non-empty.

split(): (List⟦*E*⟧, List⟦*E*⟧)

zip⟦*F*⟧(*other*: List⟦*F*⟧): Generator⟦(*E*, *F*)⟧

filter(*p*: *E* → Boolean): List⟦*E*⟧

concatMap is an in-place version of the *nest* method from Generator; it flattens the result into an actual list, rather than returning an abstract Generator.

concatMap⟦*G*⟧(*f*: *E* → List⟦*G*⟧): List⟦*G*⟧

`end`

Vararg factory for lists; provides aggregate list constants:

`opr` ⟨⟦*E*⟧ *xs*: *E* ...⟩: List⟦*E*⟧

List comprehensions:

opr BIG $\langle [\![T]\!] \rangle$: Comprehension$\big[\![T, \text{List}[\![T]\!], \text{AnyCovColl}, \text{AnyCovColl}\big]\!]$

opr BIG $\langle [\![T]\!]\ g$: Generator$[\![T]\!] \rangle$: List$[\![T]\!]$

opr BIG CONCAT$[\![T]\!]()$: BigReduction$\big[\![\text{List}[\![T]\!], \text{List}[\![T]\!]\big]\!]$

opr BIG CONCAT$[\![T]\!]\big(g$: Generator$[\![\text{List}[\![T]\!]]\!]\big)$: List$[\![T]\!]$

Convert generator into list (simpler type than comprehension above):

list$[\![E]\!]\big(g$: Generator$[\![E]\!]\big)$: List$[\![E]\!]$

Flatten a list of lists

concat$[\![E]\!]\big(x$: List$[\![\text{List}[\![E]\!]]\!]\big)$: List$[\![E]\!]$

emptyList$[\![E]\!]()$: List$[\![E]\!]$

singleton$[\![E]\!](e : E)$: List$[\![E]\!]$

A reduction object for concatenating lists.

object Concat$[\![E]\!]$ extends MonoidReduction$\big[\![\text{List}[\![E]\!]\big]\!]$

empty(): List$[\![E]\!]$

join$\big(a$: List$[\![E]\!], b$: List$[\![E]\!]\big)$: List$[\![E]\!]$

end

end

30.4 File

api File

import FlatString.FlatString

import FileSupport.{...}

FileReadStream(*filename*: String): FileReadStream

object FileReadStream(*filename*: String) extends { FileStream, ReadStream }

getter *fileName*(): String

eof returns true if an end-of-file (EOF) condition has been encountered on the stream.

getter *eof*(): Boolean

ready returns *true* if there is currently input from the stream available to be consumed.

`getter` *ready*(): Boolean

close the stream.

close(): ()

readLine returns the next available line from the stream, discarding line termination characters. Returns "" on EOF.

readLine(): String

Returns the next available character from the stream, or '
0' on EOF.

readChar(): Char

read returns the next k characters from the stream. It will block until at least one character is available, and will then return as many characters as are ready. Will return "" on end of file. If $k <= 0$ or absent a default value is chosen.

read(k: $\mathbb{Z}32$): String

read(): String

All file generators yield file contents in parallel by default, with a natural ordering corresponding to the order data occurs in the underlying file. The file is closed if all its contents have been read.

These generators pull in multiple chunks of data (where a ``chunk'' is a line, a character, or a block) before it is processed. The maximum number of chunks to pull in before processing is given by the last optional argument in every case; if it is $<= 0$ or absent a default value is chosen.

It is possible to read from a ReadStream before invoking any of the following methods. Previously-read data is ignored, and only the remaining data is provided by the generator.

At the moment, it is illegal to read from a ReadStream once any of these methods has been called; we do not check for this condition. However, the sequential versions of these generators may use label/exit or throw in order to escape from a loop, and the ReadStream will remain open and ready for reading.

Once the ReadStream has been completely consumed it is closed.

lines yields the lines found in the file a la *readLine*.

lines(n: $\mathbb{Z}32$): Generator⟦String⟧

lines(): Generator⟦String⟧

characters yields the characters found in the file a la *readChar*.

characters(n: $\mathbb{Z}32$): Generator⟦Char⟧

characters(): Generator⟦Char⟧

chunks returns chunks of characters found in the file, in the sense of *read*. The first argument is equivalent to the argument *k* to read, the second (if present) is the number of chunks at a time.

chunks(n: $\mathbb{Z}32, m$: $\mathbb{Z}32$): Generator⟦String⟧

chunks(n: $\mathbb{Z}32$): Generator⟦String⟧

chunks(): Generator⟦String⟧

end

A FileWriteStream represents a writable stream backed by a file named *fileName*.

FileWriteStream(*fileName*: String): FileWriteStream

object FileWriteStream(*fileName*: FlatString) **extends** { FileStream }

getter *fileName*(): String

write(FlatString) and *write*(Char) are the primitive mechanisms for writing characters to the end of a FileWriteStream.

write(x: FlatString): ()

write(c: Char): ()

write(Any) converts its argument to a String using *toString* and appends the result to the stream.

write(s: String): ()

write(x: Any): ()

writes converts each of the generated elements to a string using *asString* unless the element is already a String in which case it is left alone. The resulting strings are appended together to the end of the stream. To avoid interleaving with concurrent output to the same stream, this append step is done monolithically.

writes(x: Generator⟦Any⟧): ()

print is a varargs function that works much like *writes*

print(x: Any ...): ()

println is a varargs function that works much like *writes*, but in addition appends a final newline character "
n" at the same time.

println(x: Any ...): ()

flush any output to the stream that is still buffered.

flush(): ()

close the stream.

close(): ()

`end`

`end`

30.5 Set

`api` Set

`import` Containment.{...}

`import` CovariantCollection.{...}

```
Thrown when taking big intersection of no sets.
```

`object` EmptyIntersection `extends` UncheckedException `end`

```
Sets represented using a (size-balanced) tree structure.  The underlying type E
must support comparison using < and =.  When generated, these sets produce their
elements in sorted order.
```

`trait` Set⟦E⟧

`extends` { ZeroIndexed⟦E⟧, ContainmentGenerator⟦E, Set⟦E⟧⟧ }

`comprises` {...}

printTree(): ()

minimum(): Maybe⟦E⟧

maximum(): Maybe⟦E⟧

deleteMinimum(): Set⟦E⟧

deleteMaximum(): Set⟦E⟧

extractMinimum(): Maybe⟦(E, Set⟦E⟧)⟧

extractMaximum(): Maybe⟦(E, Set⟦E⟧)⟧

add(x : E): Set⟦E⟧

delete(x : E): Set⟦E⟧

`opr` ∪(`self`, t_2: Set⟦E⟧): Set⟦E⟧

`opr` ∩(`self`, t_2: Set⟦E⟧): Set⟦E⟧

`opr DIFFERENCE`(`self`, t_2: Set⟦E⟧): Set⟦E⟧

```
  Symmetric difference:  all elements in exactly one of the two sets.
```

`opr SYMDIFF`$\big($`self`$, t_2\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Set}[\![E]\!]$

$splitAt(e : E)\colon \big(\mathrm{Set}[\![E]\!], \mathrm{Boolean}, \mathrm{Set}[\![E]\!]\big)$

`opr` $\subset\big($`self`$, other\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Boolean}$

`opr` $\subseteq\big($`self`$, other\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Boolean}$

`opr` $\supset\big($`self`$, other\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Boolean}$

`opr` $\supseteq\big($`self`$, other\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Boolean}$

`opr SETCMP`$\big($`self`$, other\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Comparison}$

```
Ordered concatenation; use only if you know what you are doing.
```

$concat\big(t_2\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Set}[\![E]\!]$

$concat_3\big(v : E, t_2\colon \mathrm{Set}[\![E]\!]\big)\colon \mathrm{Set}[\![E]\!]$

`end`

$singleton[\![E]\!](x : E)\colon \mathrm{Set}[\![E]\!]$

$set[\![E]\!]()\colon \mathrm{Set}[\![E]\!]$

$set[\![E]\!]\big(g\colon \mathrm{Generator}[\![E]\!]\big)\colon \mathrm{Set}[\![E]\!]$

`opr` $\{[\![E]\!]\ es\colon E \ldots\}\colon \mathrm{Set}[\![E]\!]$

`opr BIG` $\{[\![T$ `extends` $\mathrm{StandardTotalOrder}[\![T]\!]]\!]\}\colon \mathrm{Comprehension}[\![T, \mathrm{Set}[\![T]\!], \mathrm{AnyCovColl}, \mathrm{AnyCovColl}]\!]$

`opr BIG` $\{[\![T$ `extends` $\mathrm{StandardTotalOrder}[\![T]\!]]\!]\ g\colon \mathrm{Generator}[\![T]\!]\}\colon \mathrm{Set}[\![T]\!]$

`opr BIG` $\cup[\![R$ `extends` $\mathrm{StandardTotalOrder}[\![R]\!]]\!]()\colon \mathrm{BigReduction}[\![\mathrm{Set}[\![R]\!], \mathrm{Set}[\![R]\!]]\!]$

`opr BIG` $\cup[\![R$ `extends` $\mathrm{StandardTotalOrder}[\![R]\!]]\!]\big(g\colon \mathrm{Generator}[\![\mathrm{Set}[\![R]\!]]\!]\big)\colon \mathrm{Set}[\![R]\!]$

`object` $\mathrm{Union}[\![E]\!]$ `extends` $\mathrm{CommutativeMonoidReduction}[\![\mathrm{Set}[\![E]\!]]\!]$ `end`

`opr BIG` $\cap[\![R$ `extends` $\mathrm{StandardTotalOrder}[\![R]\!]]\!]()\colon$
$\mathrm{BigReduction}[\![\mathrm{Set}[\![R]\!], \mathrm{Maybe}[\![\mathrm{Set}[\![R]\!]]\!]]\!]$

`opr BIG` $\cap[\![R$ `extends` $\mathrm{StandardTotalOrder}[\![R]\!]]\!]\big(g\colon \mathrm{Generator}[\![\mathrm{Set}[\![R]\!]]\!]\big)\colon \mathrm{Set}[\![R]\!]$

`end`

30.6 Map

`api` Map

`import` CovariantCollection.{...}

`import` Set.{Set}

`object` KeyOverlap⟦Key, Val⟧(*key*: Key, val_1: Val, val_2: Val)

`extends` UncheckedException

`end`

Note that the map interface is purely functional; methods return a fresh map rather than updating the receiving map in place. Methods that operate on a particular key leave the rest of the map untouched unless otherwise specified.

`trait` Map⟦Key, Val⟧

`extends` { Generator⟦(Key, Val)⟧, Equality⟦Map⟦Key, Val⟧⟧ }

`comprises` {...}

`getter` *isEmpty*(): Boolean

`getter` *asDebugString*(): String

dom(`self`): Set⟦Key⟧

`opr` | `self` | : ℤ32

`opr` [*k*: Key]: Val `throws` NotFound

member(*x*: Key): Maybe⟦Val⟧

The two-argument version of *member* returns the default value *v* if the key is absent from the map.

member(*x*: Key, *v*: Val): Val

minimum and *maximum* refer to the key.

minimum(): Maybe⟦(Key, Val)⟧

deleteMinimum(): Map⟦Key, Val⟧

extractMinimum(): Maybe⟦(Key, Val, Map⟦Key, Val⟧)⟧

maximum(): Maybe⟦(Key, Val)⟧

deleteMaximum(): Map⟦Key, Val⟧

extractMaximum(): Maybe⟦(Key, Val, Map⟦Key, Val⟧)⟧

If no mapping presently exists, maps *k* to *v*.

add(*k*: Key, *v*: Val): Map⟦Key, Val⟧

Maps *k* to *v*.

update(*k*: Key, *v*: Val): Map⟦Key, Val⟧

Eliminate any mapping for key *k*.

delete(*k*: Key): Map⟦Key, Val⟧

Process mapping for key *k* with function *f*:

- If no mapping exists, *f* is passed Nothing⟦Val⟧

- If k maps to value v, f is passed Just⟦Val⟧(v)

If f returns Nothing, any mapping for k is deleted; otherwise, k is mapped to the value contained in the result.

updateWith$\big(f$: Maybe⟦Val⟧ → Maybe⟦Val⟧, k: Key$\big)$: Map⟦Key, Val⟧

∪ favors the leftmost value when a key occurs in both maps.

opr ∪$\big($**self**, *other*: Map⟦Key, Val⟧$\big)$: Map⟦Key, Val⟧

⊎ (disjoint union) throws the KeyOverlap exception when a key occurs in both maps.

opr ⊎$\big($**self**, *other*: Map⟦Key, Val⟧$\big)$: Map⟦Key, Val⟧

The *union* method takes a function f used to combine the values of keys that overlap.

union$\big(f$: (Key, Val, Val) → Val, *other*: Map⟦Key, Val⟧$\big)$: Map⟦Key, Val⟧

combine is the ''swiss army knife'' combinator on pairs of maps. We call f on keys present in both input maps. We call *doThis* on keys present in **self** but not in *that*. We call *doThat* on keys present in *that* but not in **self**. When any of these functions returns $r =$ Just⟦Result⟧, the key is mapped to $r.get()$ in the result. When any of these functions returns Nothing⟦Result⟧, there is no mapping for the key in the result.

mapThis must be equivalent to *mapFilter*(*doThis*) and *mapThat* must be equivalent to *mapFilter*(*doTh* they are included because often they can do their jobs without traversing their argument (eg. for union and interesection operations we can pass through or discard whole submaps without traversing them).

combine⟦That, Result⟧$(f$: (Key, Val, That) → Maybe⟦Result⟧,

doThis: (Key, Val) → Maybe⟦Result⟧,

doThat: (Key, That) → Maybe⟦Result⟧,

mapThis: Map⟦Key, Val⟧ → Map⟦Key, Result⟧,

mapThat: Map⟦Key, That⟧ → Map⟦Key, Result⟧,

that: Map⟦Key, That⟧$)$: Map⟦Key, Result⟧

self.*mapFilter*(f) is equivalent to:

$\%\{\, k \mapsto v' \mid (k, v) \leftarrow$ **self**$, v' \leftarrow f(k, v) \,\}$

It fuses generation, mapping, and filtering.

mapFilter⟦Result⟧$\big(f$: (Key, Val) → Maybe⟦Result⟧$\big)$: Map⟦Key, Result⟧

end

singleton⟦Key, Val⟧$(k$: Key, v: Val$)$: Map⟦Key, Val⟧

mapping⟦Key, Val⟧$\big(g$: Generator⟦(Key, Val)⟧$\big)$: Map⟦Key, Val⟧

opr $\{\mapsto$ ⟦Key, Val⟧ xs: (Key, Val) ... $\}$: Map⟦Key, Val⟧

`opr` $\{[\![\text{Key}, \text{Val}]\!]\}$: Map$[\![\text{Key}, \text{Val}]\!]$

`opr BIG` $\{\mapsto [\![\text{Key}, \text{Val}]\!]\}$: Comprehension$\big[\![(\text{Key}, \text{Val}), \text{Map}[\![\text{Key}, \text{Val}]\!], \text{AnyCovColl}, \text{AnyCovColl}]\!\big]$

`opr BIG` $\{\mapsto [\![\text{Key}, \text{Val}]\!]\, g$: Generator$[\![(\text{Key}, \text{Val})]\!]\}$: Map$[\![\text{Key}, \text{Val}]\!]$

`opr BIG` $\cup[\![\text{Key}, \text{Val}]\!]()$: BigReduction$\big[\![\text{Map}[\![\text{Key}, \text{Val}]\!], \text{Map}[\![\text{Key}, \text{Val}]\!]]\!\big]$

`opr BIG` $\cup[\![\text{Key}, \text{Val}]\!]\big(g$: Generator$[\![\text{Map}[\![\text{Key}, \text{Val}]\!]]\!]\big)$: Map$[\![\text{Key}, \text{Val}]\!]$

`opr BIG` $\uplus[\![\text{Key}, \text{Val}]\!]()$: BigReduction$\big[\![\text{Map}[\![\text{Key}, \text{Val}]\!], \text{Map}[\![\text{Key}, \text{Val}]\!]]\!\big]$

`opr BIG` $\uplus[\![\text{Key}, \text{Val}]\!]\big(g$: Generator$[\![\text{Map}[\![\text{Key}, \text{Val}]\!]]\!]\big)$: Map$[\![\text{Key}, \text{Val}]\!]$

`end`

30.7 Sparse

`api` Sparse

`Trim array` v `to length` l`.`

trim$[\![T]\!]\big(v$: Array$[\![T, \mathbb{Z}32]\!], l$: $\mathbb{Z}32\big)$: Array$[\![T, \mathbb{Z}32]\!]$

`Sparse vector, represented as ordered sequence of index/value pairs. Absent entries are 0.`

`object` SparseVector$[\![T,$ `nat` $n]\!]\big(mem$: Array$[\![(\mathbb{Z}32, T), \mathbb{Z}32]\!]\big)$

`extends` Vector$[\![T, n]\!]$

`end`

sparse `constructs a sparse vector from dense vector.`

sparse$[\![T$ `extends` Number, `nat` $n]\!]\big(me$: Array1$[\![\mathbb{R}64, 0, n]\!]\big)$: SparseVector$[\![\mathbb{R}64, n]\!]$

`Compressed sparse row representation. A dense array sparse row vectors.`

`object` Csr$[\![N$ `extends` Number, `nat` n, `nat` $m]\!]$

$\big(rows$: Array1$[\![$SparseVector$[\![N, m]\!], 0, n]\!]\big)$

`extends` Matrix$[\![N, n, m]\!]$

`end`

`A compressed sparse column matrix is just the transpose of a Csr matrix.`

`object` Csc$[\![N$ `extends` Number, `nat` n, `nat` $m]\!]$

$\big(cols$: Array1$[\![$SparseVector$[\![N, n]\!], 0, m]\!]\big)$

extends Matrix$[\![N, n, m]\!]$

end

end

30.8 Heap

api Heap

Mergeable, pure priority queues (heaps).

At the moment, we are baking off several potential priority queue implementations, based on part on some advice from ``Purely Functional Data Structures'' by Okasaki [25].

- Pairing heaps, the current default: $O(1)$ merge; $O(\lg n)$ amortized *extractMinimum*, with $O(n)$ worst case. The worst case is proportional to the number of deferred merge operations performed with the min; if you merge n entries tree-fashion rather than one at a time you should obtain $O(\lg n)$ worst case performance as well. The *heap*(*gen*) function will follow the merge structure of the underlying generator; for a well-written generator this will be sufficient to give good performance.
- Lazy-esque pairing heaps (supposedly slower but actually easier in some ways to implement than pairing heaps, and avoiding a potential stack overflow in the latter). These do not seem to be quite as whizzy in this setting as ordinary Pairing heaps: $O(\lg n)$ merge, $O(\lg n)$ worst-case *extractMinimum*.
- Splay heaps (noted as ``fastest in practice'', borne out by other experiments). Problem: $O(n)$ merge operation, vs $O(\lg n)$ for everything else. If we build heaps by performing reductions over a generator, with each iteration generating a few elements, this will be a problem. This is not yet implemented.

Minimum complete implementation of Heap$[\![K, V]\!]$:

%*empty*

%*singleton*

%*isEmpty*

%*extractMinimum*

%*merge*(Heap$[\![K, V]\!]$)

trait Heap$[\![K, V]\!]$ extends Generator$[\![(K, V)]\!]$

getter *isEmpty*(): Boolean

Given an instance of Heap$[\![K, V]\!]$, get the empty Heap$[\![K, V]\!]$.

getter *empty*(): Heap$[\![K, V]\!]$

Get the (key,value) pair with minimal associated key.

`getter` *minimum*(): Maybe$[\![(K, V)]\!]$

Given an instance of Heap$[\![K, V]\!]$, generate a singleton Heap$[\![K, V]\!]$.

singleton($k : K, v : V$): Heap$[\![K, V]\!]$

Return a heap that contains the key-value pairings in both of the heaps passed in.

merge(h: Heap$[\![K, V]\!]$): Heap$[\![K, V]\!]$

Return a heap that contains the additional key-value pairs.

insert($k : K, v : V$): Heap$[\![K, V]\!]$

Extract the (key,value) pair with minimal associated key, along with a heap with that key and value pair removed.

extractMinimum(): Maybe$[\![(K, V,$ Heap$[\![K, V]\!])]\!]$

Delete the minimum (key,value) pair (if any) from the heap, and return the resulting heap.

deleteMinimum(): Heap$[\![K, V]\!]$

`end`

`object` HeapMerge$[\![K, V]\!]$($boiler$: Heap$[\![K, V]\!]$)

`extends` CommutativeMonoidReduction$[\![$Heap$[\![K, V]\!]]\!]$

`end`

`trait` Pairing$[\![K, V]\!]$ `extends` Heap$[\![K, V]\!]$

`comprises` {...}

`end`

emptyPairing$[\![K, V]\!]$(): Pairing$[\![K, V]\!]$

singletonPairing$[\![K, V]\!]$($k : K, v : V$): Pairing$[\![K, V]\!]$

pairing$[\![K, V]\!]$(g: Generator$[\![(K, V)]\!]$): Pairing$[\![K, V]\!]$

Not actually lazy pairing heaps; these are actually more eager in that they merge siblings incrementally on insertion.

`trait` LazyPairing$[\![K, V]\!]$ `extends` Heap$[\![K, V]\!]$

`comprises` {...}

`end`

emptyLazy$[\![K, V]\!]$(): LazyPairing$[\![K, V]\!]$

singletonLazy$[\![K, V]\!]$($k : K, v : V$): LazyPairing$[\![K, V]\!]$

lazy$[\![K, V]\!]$(g: Generator$[\![(K, V)]\!]$): Heap$[\![K, V]\!]$

Use these default factories unless you are experimenting. Right now, they yield non-lazy pairing heaps.

emptyHeap⟦*K*, *V*⟧(): Pairing⟦*K*, *V*⟧

singletonHeap⟦*K*, *V*⟧(*k* : *K*, *v* : *V*): Pairing⟦*K*, *V*⟧

heap⟦*K*, *V*⟧(*g*: Generator⟦(*K*, *V*)⟧): Pairing⟦*K*, *V*⟧

end

30.9 SkipList

api SkipList

import PureList.{...}

A SkipList type consists of a root node and $pInverse = \frac{1}{p}$, where the fraction p is used in the negative binomial distribution to select random levels for insertion.

trait SkipList⟦Key, Val, **nat** *pInverse*⟧

comprises {...}

Depracated. Use |**self**| instead.

getter *size*(): ℤ32

The number of values stored.

opr |**self**|: ℤ32

Given a key, try to return a value that matches that key.

search(*k*: Key): Maybe⟦Val⟧

Add a (key, value) pair.

add(*k*: Key, *v*: Val): SkipList⟦Key, Val, *pInverse*⟧

Remove a (key, value) pair.

remove(*k*: Key): SkipList⟦Key, Val, *pInverse*⟧

Merge two Skip Lists.

merge(*other*: SkipList⟦Key, Val, *pInverse*⟧): SkipList⟦Key, Val, *pInverse*⟧

end

(⋆ Construct an empty Skip List. ⋆)

NewList⟦Key, Val, **nat** *pInverse*⟧(): SkipList⟦Key, Val, *pInverse*⟧

end

30.10 QuickSort

`api` QuickSort

`import` List.{List}

`http://en.wikipedia.org/wiki/Quicksort`

quicksort$[\![T]\!]\big(lt : (T, T) \to \text{Boolean}, arr\colon \text{Array}[\![T, \mathbb{Z}32]\!], left\colon \mathbb{Z}32, right\colon \mathbb{Z}32\big)\colon ()$

quicksort$[\![T]\!]\big(lt : (T, T) \to \text{Boolean}, arr\colon \text{Array}[\![T, \mathbb{Z}32]\!]\big)\colon ()$

quicksort$\big[\![T$ `extends` StandardTotalOrder$[\![T]\!]\big]\big(arr\colon \text{Array}[\![T, \mathbb{Z}32]\!]\big)\colon ()$

quicksort$[\![T]\!]\big(lt : (T, T) \to \text{Boolean}, xs\colon \text{List}[\![T]\!]\big)\colon \text{List}[\![T]\!]$

quicksort$[\![T]\!]\big(xs\colon \text{List}[\![T]\!]\big)\colon \text{List}[\![T]\!]$

`end`

Part V

Fortress APIs and Documentation for Application Programmers

> DRAFT NOTE
> The APIs in this part may not be correct; they are not tested.

Chapter 31

Objects

31.1 The Trait Fortress.Core.Any

The trait Any is the single root of the type hierarchy; every object in Fortress has trait Any and therefore every object implements the methods of this trait. The immediate subtypes of the trait Any are Object, Tuple, (), and Index, the trait of the types that a subscript takes (Section 6.2).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.42
>
> The Working Draft of February 2011 named Object, Tuple and () only; Index, which extends Any and not Object, is new.

```
trait Any
  getter hashCode(): ℕ64
  opr ≡(self, other: Any): Boolean
  opr IDENTITY(self): Any
  hash(maxval: ℕ64): ℕ64
  hash(maxval: ℕ32): ℕ32
  toString(): String
  property ∀(x, y, n: ℕ64) x ≡ y → x.hash(n) ≡ y.hash(n)
  property ∀(x, y, n: ℕ32) x ≡ y → x.hash(n) ≡ y.hash(n)
  property ∀(x) x.hashCode ≡ x.hash(2^64 − 1)
  property ∀(x, y) x ≡ y → x.toString() = y.toString()
  property ∀(a) (IDENTITY a) ≡ a
  property ∀(a) a ≡ a
  property ∀(a, b) a ≡ b ↔ b ≡ a
  property ∀(a, b, c) (a ≡ b∧: b ≡ c) →: a ≡ c
end
```

31.1.1 `getter` $hashCode(): \mathbb{N}64$

Every object has associated with it a 64-bit unsigned integer value called its *hash code*; this is the value returned by the $hash$ method when given the argument $2^{64} - 1$. Hash codes are not necessarily consistent from one Fortress application to another, nor from one execution of a Fortress application to another execution of the same application, but remain fixed during the execution of a single Fortress application. It is permitted for two objects to have the same hashCode, but Fortress programmers and implementors should be aware that assigning distinct hash codes to distinct objects may improve the performance of hash tables.

The trait Any defines its $hash$ methods in terms of $hashCode$; therefore it suffices for a subtrait to override $hashCode$ to get the benefit of the $hash$ methods as well.

31.1.2 `opr` $\equiv$(`self`, $other$: Any): Boolean

The infix operator $\equiv$ (object equivalence) is used to decide whether two objects are "the same object" in the strictest sense possible; this is described in detail in Section 11.4. Note that the properties of trait Any assert that $\equiv$ is an equivalence relation.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.8
>
> The Working Draft of February 2011 added that in Object this property is asserted abstractly using the algebraic constraints of Chapter 40; Object no longer does so (Section I.1.8).

For $\not\equiv$ see Section 34.1.1.

31.1.3 `opr IDENTITY`(`self`): Any

The operator `IDENTITY` simply returns its argument. (This may not be terribly useful for applications programming, but it has technical uses for specifying contracts and algebraic properties in libraries as described in Section 40.3.)

31.1.4 $hash(maxval: \mathbb{N}64): \mathbb{N}64$
31.1.5 $hash(maxval: \mathbb{N}32): \mathbb{N}32$

The $hash$ method returns a *hash value* for the object as an unsigned integer that is less than or equal to the $maxval$ argument. This hash value is not necessarily consistent from one Fortress application to another, nor from one execution of a Fortress application to another execution of the same application, but the hash value produced for a given value of the $maxval$ argument remains fixed during the execution of a single Fortress application. There is no defined relationship between hash values produced for the same object but with different $maxval$ values. Fortress programmers and implementors should be aware that the performance of hash tables is likely to be improved if, for any given collection of objects and given value for the $maxval$ argument, the $hash$ method assigns hash values to those objects with relatively uniform distribution.

31.1.6 $toString(): \mathrm{String}$

The general contract of $toString$ is that it returns a string that "textually represents" this object. The idea is to provide a concise but informative representation that will be useful to a person reading it.

31.2 The Trait Fortress.Core.Object

The trait Object is the root of the type hierarchy for all user-constructed objects and functions. The Object type does not have any additional methods; the properties of the existing operators, such as $\equiv$, are those stated in trait Any.

```
trait Object extends Any
    excludes { Tuple }
end
```

The following declaration, which describes those properties with the algebraic constraints of Chapter 40, is not allowed:

```
(* Not allowed! *)
trait Object extends { Any, EquivalenceRelation⟦Object, ≡⟧, IdentityOperator⟦Object⟧ }
    excludes { Tuple }
end
```

Every trait other than Any, Object and Index extends Object (Section 10.1), and the algebraic constraints make each commutative algebraic type T an EquivalenceRelation$⟦T, =⟧$ (Chapter 40); such a type would extend two instantiations of EquivalenceRelation whose type arguments differ, which instantiation exclusion forbids (Section 6.3).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.8
>
> The Working Draft of February 2011 declared Object with the declaration marked "Not allowed", and said that Object uses algebraic constraints to describe the properties of the existing operators more abstractly. The declaration `trait` Object `extends` Any is the library's.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.42
>
> The Working Draft of February 2011 named only Any and Object as the traits that do not extend Object; Index, which the revival adds to the library, is the third.

31.3 The Trait Fortress.Core.Tuple

The trait Tuple is the root of the type hierarchy for all tuple types (see Section 6.5). No user-defined object can be a subtype of Tuple. As with the type Object, the properties of its existing operators are those stated in trait Any.

```
trait Tuple extends Any
    excludes { Object }
  toString(): String
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.8
>
> The Working Draft of February 2011 gave Tuple the same algebraic supertraits as Object, at Tuple, and said that it uses algebraic constraints to describe its existing methods. No tuple type extends a trait, so the rule does not refuse that declaration; it follows Object's.

31.3.1 $toString(): \mathrm{String}$

The $toString$ method returns a string "" concatenated as follows:

- If the tuple has just one element, concatenate "`tuple`".
- Concatenate "`(`".
- For each element of the tuple, in left-to-right order, taking all plain elements before any varargs or keyword element, taking any varargs element before any keyword element, and taking keyword elements in the order defined by the type of the tuple (which may not be the sorted order):
  - If this is a plain element, concatenate the $toString$ of the element.
  - If this is a varargs element "$e\ldots$", concatenate the $toString$ of e and concatenate "`...`".
  - If this is a keyword element "$id = e$", concatenate the $toString$ of id, concatenate "`=`", and concatenate the $toString$ of e.
  - If this is the last element of the tuple, concatenate "`)`"; otherwise concatenate ", ".

Chapter 32

Booleans and Boolean Intervals

32.1 The Trait Fortress.Core.Boolean

```
trait Boolean
    extends { BooleanAlgebra⟦Boolean, juxtaposition, ∨, ¬, ⊻⟧,
              BooleanAlgebra⟦Boolean, juxtaposition, ∨, ¬, ⊕⟧,
              BooleanAlgebra⟦Boolean, ∧, ∨, ¬, ⊻⟧,
              BooleanAlgebra⟦Boolean, ∧, ∨, ¬, ⊕⟧,
              BooleanAlgebra⟦Boolean, ∨, juxtaposition, ¬, ≡⟧,
              BooleanAlgebra⟦Boolean, ∨, juxtaposition, ¬, ↔⟧,
              BooleanAlgebra⟦Boolean, ∨, ∧, ¬, ≡⟧,
              BooleanAlgebra⟦Boolean, ∨, ∧, ¬, ↔⟧,
              TotalOrder⟦Boolean, →⟧,
              PartialOrderAndBoundedLattice⟦Boolean, →, juxtaposition, ∨⟧,
              PartialOrderAndBoundedLattice⟦Boolean, →, ∧, ∨⟧,
              IdentityEquality⟦Boolean⟧,
              EquivalenceRelation⟦Boolean, ≡⟧,
              EquivalenceRelation⟦Boolean, ↔⟧,
              Symmetric⟦Boolean, ∧⟧, Symmetric⟦Boolean, ∨⟧,
              Symmetric⟦Boolean, ⊻⟧, Symmetric⟦Boolean, ⊕⟧,
              Symmetric⟦Boolean, ⊼⟧, Commutative⟦Boolean, ⊼⟧,
              Symmetric⟦Boolean, ⊽⟧, Commutative⟦Boolean, ⊽⟧ }
    comprises { ... }
  coerce ⟦bool b⟧(x: BooleanLiteral⟦b⟧)
  coerce (x: Identity⟦juxtaposition⟧)
  coerce (x: Identity⟦∧⟧)
  coerce (x: Identity⟦∨⟧)
  coerce (x: Identity⟦⊻⟧)
  coerce (x: Identity⟦⊕⟧)
  coerce (x: Identity⟦≡⟧)
  coerce (x: Identity⟦↔⟧)
  coerce (x: ComplementBound⟦∧⟧)
  coerce (x: ComplementBound⟦∨⟧)
  coerce (x: Zero⟦∧⟧)
```

```
    coerce (x: Zero⟦∨⟧)
    coerce (x: MaximalElement⟦→⟧)
    coerce (x: MinimalElement⟦→⟧)
    getter hashCode(): ℕ64
    opr juxtaposition (self, other: Boolean): Boolean
    opr ∧(self, other: Boolean): Boolean
    opr ∧(self, other: () → Boolean): Boolean
    opr ∨(self, other: Boolean): Boolean
    opr ∨(self, other: () → Boolean): Boolean
    opr ¬(self): Boolean
    opr ⊻(self, other: Boolean): Boolean
    opr ⊕(self, other: Boolean): Boolean
    opr ≡(self, other: Boolean): Boolean
    opr =(self, other: Boolean): Boolean
    opr ↔(self, other: Boolean): Boolean
    opr →(self, other: Boolean): Boolean
    opr →(self, other: () → Boolean): Boolean
    opr ⊼(self, other: Boolean): Boolean
    opr ⊽(self, other: Boolean): Boolean
    opr ≣(self, other: Boolean): Boolean
    majority(self, other₁: Boolean, other₂: Boolean)
    toString(): String
    property ∀(a, b) a b = (a ∧ b)
    property ∀(a, b) (a ⊻ b) = (a ⊕ b)
    property ∀(a, b) (a ≡ b) = (a ↔ b) = (a = b) = (a ≣ b)
    property ∀(a, b) (a → b) = ((¬a) ∨ b)
    property ∀(a, b) (a ⊼ b) = ¬(a ∧ b)
    property ∀(a, b) (a ⊽ b) = ¬(a ∨ b)
end
test testData[ ] = { false, true }
```

32.1.1 coerce ⟦bool b⟧(x: BooleanLiteral⟦b⟧)

A boolean literal can always serve as a Boolean value.

32.1.2 `coerce` (x: Identity⟦`juxtaposition`⟧)
32.1.3 `coerce` (x: Identity⟦$\wedge$⟧)
32.1.4 `coerce` (x: Identity⟦$\vee$⟧)
32.1.5 `coerce` (x: Identity⟦$\veebar$⟧)
32.1.6 `coerce` (x: Identity⟦$\oplus$⟧)
32.1.7 `coerce` (x: Identity⟦$\equiv$⟧)
32.1.8 `coerce` (x: Identity⟦$\leftrightarrow$⟧)
32.1.9 `coerce` (x: ComplementBound⟦$\wedge$⟧)
32.1.10 `coerce` (x: ComplementBound⟦$\vee$⟧)
32.1.11 `coerce` (x: Zero⟦$\wedge$⟧)
32.1.12 `coerce` (x: Zero⟦$\vee$⟧)
32.1.13 `coerce` (x: MaximalElement⟦$\rightarrow$⟧)
32.1.14 `coerce` (x: MinimalElement⟦$\rightarrow$⟧)

The identity for $\wedge$ or `juxtaposition` is *true*; the zero and the complement bound for $\wedge$ or `juxtaposition` are *false*.

The identity for $\vee$ is *false*; the zero and the complement bound for $\vee$ are *false*.

The identity for $\veebar$ or $\oplus$ is *false*; the identity for $\equiv$ or $\leftrightarrow$ is *true*.

The maximal element for $\rightarrow$ (regarded as a partial order operator) is *true*, and the minimal element is *false*.

32.1.15 `getter` *hashCode*(): $\mathbb{N}64$

32.1.16 `opr juxtaposition` (`self`, *other*: Boolean): Boolean

Juxtaposition of boolean expressions is equivalent to using the logical AND operator $\wedge$.

32.1.17 `opr` $\wedge$(`self`, *other*: Boolean): Boolean
32.1.18 `opr` $\wedge$(`self`, *other*: () $\rightarrow$ Boolean): Boolean

The logical AND operator $\wedge$ (`AND`) returns *true* if both arguments are *true*; otherwise it returns *false*.

The conditional logical AND operator $\wedge$: (`AND:`) examines its first argument; if it is *false*, the result is *false*, and the second argument (a thunk) is not evaluated. But if the first argument is *true*, the second argument is evaluated and its result becomes the result of the conditional logical AND operator expression.

32.1.19 `opr` $\vee$(`self`, *other*: Boolean): Boolean
32.1.20 `opr` $\vee$(`self`, *other*: () $\rightarrow$ Boolean): Boolean

The logical OR operator $\vee$ (`OR`) returns *false* if both arguments are *false*; otherwise it returns *true*.

The conditional logical OR operator $\vee$: (`OR:`) examines its first argument; if it is *true*, the result is *true*, and the second argument (a thunk) is not evaluated. But if the first argument is *false*, the second argument is evaluated and its result becomes the result of the conditional logical OR operator expression.

32.1.21 `opr` ¬(`self`): Boolean

The logical NOT operator ¬ (`NOT`) returns *true* if its argument is *false*; it returns *false* if its argument is *true*.

32.1.22 `opr` ⩒(`self`, *other*: Boolean): Boolean
32.1.23 `opr` ⊕(`self`, *other*: Boolean): Boolean

The logical exclusive OR operator ⩒ (`XOR`) returns *true* if the arguments are different, one being *true* and the other *false*; it returns *false* if both arguments are *true* or both arguments are *false*.

The operator ⊕ (`OPLUS`) does the same thing as ⩒.

32.1.24 `opr` ≡(`self`, *other*: Boolean): Boolean
32.1.25 `opr` =(`self`, *other*: Boolean): Boolean
32.1.26 `opr` ↔(`self`, *other*: Boolean): Boolean

The logical equivalence, or exclusive NOR, operator ≡ (`EQV`) returns *true* if both arguments are *true* or both arguments are *false*; it returns *false* if the arguments are different, one being *true* and the other *false*. (Thus its behavior on boolean values happens to be exactly the same as that of the strict equivalence operator ≣.)

The equality operator = and the if-and-only-if operator ↔ (`IFF`) do the same thing as ≡.

For ≢ see Section 34.1.2. For ≠ see Section 34.1.4.

32.1.27 `opr` →(`self`, *other*: Boolean): Boolean
32.1.28 `opr` →(`self`, *other*: () → Boolean): Boolean

The logical implication operator → (`IMPLIES`) returns *false* if the first argument is *true* but the second argument is *false*; otherwise it returns *true*.

The conditional logical implication operator →: (`IMPLIES:`) examines its first argument; if it is *false*, the result is *true*, and the second argument (a thunk) is not evaluated. But if the first argument is *true*, the second argument is evaluated and its result becomes the result of the conditional logical implication operator expression.

32.1.29 `opr` ⊼(`self`, *other*: Boolean): Boolean

The logical NAND (NOT AND) operator ⊼ (`NAND`) returns *false* if both arguments are *true*; otherwise it returns *true*.

32.1.30 `opr` ⊽(`self`, *other*: Boolean): Boolean

The logical NOR (NOT OR) operator ⊽ (`NOR`) returns *false* if both arguments are *false*; otherwise it returns *true*.

32.1.31 `opr` $\equiv$(`self`, *other*: Boolean): Boolean

Two boolean values are strictly equivalent if and only if they are the same boolean value (that is, both *true* or both *false*).

32.1.32 *majority*(`self`, $other_1$: Boolean, $other_2$: Boolean)

If two or three of the argments are *true*, the result is *true*; if two or three of the argments are *false*, the result is *false*.

32.1.33 *toString*(): String

The *toString* method returns either "`true`" or "`false`" as appropriate.

32.2 The Trait Fortress.Standard.BooleanInterval

A boolean interval is a set of boolean values. There are two distinct boolean values, *true* and *false*, and therefore there are four distinct boolean intervals, which for convenience are given names:

$$\begin{aligned} \text{True} &= \{\mathit{true}\} \\ \text{False} &= \{\mathit{false}\} \\ \text{Uncertain} &= \{\mathit{true}, \mathit{false}\} \\ \text{Impossible} &= \{\,\} \end{aligned}$$

Logical operations on intervals obey the interval containment rule: the result interval must contain every boolean result that can be produced by applying the operator to a boolean value taken from each argument interval. For example, if P and Q are boolean intervals, then by definition $P \wedge Q = \{\, x \wedge y \mid x \leftarrow P, y \leftarrow Q \,\}$.

A principal application of boolean intervals is to express the results of numerical comparison of numerical intervals. In this way numerical comparisons can also obey the interval containment rule.

Set operations such as $\cup$ and $\cap$ may also be used on boolean intervals.

```
trait BooleanInterval
      extends { BooleanAlgebra⟦BooleanInterval, ∩, ∪, SET_COMPLEMENT, SYMDIFF⟧,
                Set⟦Boolean⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ∧⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ∨⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ⊻⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ≡⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, =⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ↔⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ⊼⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, ⊽⟧,
                BinaryIntervalContainment⟦BooleanInterval, Boolean, →⟧,
                UnaryIntervalContainment⟦BooleanInterval, Boolean, ¬⟧,
                Generator⟦Boolean⟧ }
      comprises { ... }
   coerce (x: Boolean)
   getter hashCode(): ℤ64
   opr ∧(self, other: BooleanInterval): BooleanInterval
   opr ∨(self, other: BooleanInterval): BooleanInterval
   opr ¬(self): BooleanInterval
   opr ⊻(self, other: BooleanInterval): BooleanInterval
   opr ⊕(self, other: BooleanInterval): BooleanInterval
   opr ≡(self, other: BooleanInterval): BooleanInterval
   opr =(self, other: BooleanInterval): BooleanInterval
   opr ↔(self, other: BooleanInterval): BooleanInterval
   opr →(self, other: BooleanInterval): BooleanInterval
   opr ⊼(self, other: BooleanInterval): BooleanInterval
   opr ⊽(self, other: BooleanInterval): BooleanInterval
   opr ∈(other: Boolean, self): Boolean
   opr ∩(self, other: BooleanInterval): BooleanInterval
   opr ∪(self, other: BooleanInterval): BooleanInterval
   opr SET_COMPLEMENT(self): BooleanInterval
   opr SYMDIFF(self, other: BooleanInterval): BooleanInterval
   opr \(self, other: BooleanInterval): BooleanInterval
   possibly(self): Boolean
   necessarily(self): Boolean
   certainly(self): Boolean
   opr ≡(self, other: BooleanInterval): Boolean
   toString(): String
   property true ∈ True ∧ false ∉ True
   property true ∉ False ∧ false ∈ False
   property true ∈ Uncertain ∧ false ∈ Uncertain
   property true ∉ Impossible ∧ false ∉ Impossible
   property ∀(a) necessarily(a) ≡ ¬possibly(¬a)
   property ∀(a) possibly(a) ↔ true ∈ a
   property ∀(a) certainly(a) ↔ (true ∈ a ∧ false ∉ a)
   property ∀(a, b) (a ⊼ b) ↔ ¬(a ∧ b)
   property ∀(a, b) (a ⊽ b) ↔ ¬(a ∨ b)
end
True: BooleanInterval
False: BooleanInterval
Uncertain: BooleanInterval
```

Impossible: BooleanInterval
`test` *testData*[ ] = { True, False, Uncertain, Impossible }

32.2.1 `coerce` (x: Boolean)

A boolean value can always serve as a BooleanInterval value. The value *true* is coerced to True; the value *false* is coerced to False.

32.2.2 `getter` *hashCode*(): ℤ64

32.2.3 `opr` ∧(`self`, *other*: BooleanInterval): BooleanInterval

The logical AND operator ∧ (`AND`) returns Impossible if either argument is Impossible; otherwise it returns False if either argument is False; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns True. It obeys the interval containment rule. The ∧ operator may be described by this table:

∧	Uncertain	True	False	Impossible
Uncertain	Uncertain	Uncertain	False	Impossible
True	Uncertain	True	False	Impossible
False	False	False	False	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.4 `opr` ∨(`self`, *other*: BooleanInterval): BooleanInterval

The logical OR operator ∨ (`OR`) returns Impossible if either argument is Impossible; otherwise it returns True if either argument is True; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns False. It obeys the interval containment rule. The ∧ operator may be described by this table:

∧	Uncertain	True	False	Impossible
Uncertain	Uncertain	True	Uncertain	Impossible
True	True	True	True	Impossible
False	Uncertain	True	False	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.5 `opr` ¬(`self`): BooleanInterval

The logical NOT operator ¬ (`NOT`) returns Impossible if its argument is Impossible, Uncertain if its argument is Uncertain, False if its argument is True, and True if its argument is False. It obeys the interval containment rule.

32.2.6 `opr` $\underline{\vee}$(`self`, *other*: BooleanInterval): BooleanInterval
32.2.7 `opr` $\oplus$(`self`, *other*: BooleanInterval): BooleanInterval

The logical exclusive OR operator $\underline{\vee}$ (`XOR`) returns Impossible if either argument is Impossible; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns False if the arguments are strictly equivalent; otherwise it returns True. It obeys the interval containment rule.

The operator $\oplus$ (`OPLUS`) does the same thing as $\underline{\vee}$. The $\underline{\vee}$ or $\oplus$ operator may be described by this table:

$\underline{\vee}$ or $\oplus$	Uncertain	True	False	Impossible
Uncertain	Uncertain	Uncertain	Uncertain	Impossible
True	Uncertain	False	True	Impossible
False	Uncertain	True	False	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.8 `opr` $\equiv$(`self`, *other*: BooleanInterval): BooleanInterval
32.2.9 `opr` $=$(`self`, *other*: BooleanInterval): BooleanInterval
32.2.10 `opr` $\leftrightarrow$(`self`, *other*: BooleanInterval): BooleanInterval

The logical equivalence, or exclusive NOR, operator $\equiv$ (`EQV`) returns Impossible if either argument is Impossible; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns True if the arguments are strictly equivalent; otherwise it returns False. It obeys the interval containment rule. (Thus its behavior on boolean interval values is *not* the same as that of the strict equivalence operator $\equiv$.)

The equality operator $=$ and the if-and-only-if operator $\leftrightarrow$ (`IFF`) do the same thing as $\equiv$. The $\equiv$ or $=$ or $\leftrightarrow$ operator may be described by this table:

$\equiv$ or $=$ or $\leftrightarrow$	Uncertain	True	False	Impossible
Uncertain	Uncertain	Uncertain	Uncertain	Impossible
True	Uncertain	True	False	Impossible
False	Uncertain	False	True	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.11 `opr` $\rightarrow$(`self`, *other*: BooleanInterval): BooleanInterval

The logical implication operator $\rightarrow$ (`IMPLIES`) returns Impossible if either argument is Impossible; otherwise it returns True if the first argument is False or the second argument is True; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns False. It obeys the interval containment rule. The $\rightarrow$ operator may be described by this table:

$\rightarrow$	Uncertain	True	False	Impossible
Uncertain	Uncertain	True	Uncertain	Impossible
True	Uncertain	True	False	Impossible
False	True	True	True	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.12 `opr` $\overline{\wedge}$(`self`, *other*: BooleanInterval): BooleanInterval

The logical NAND (NOT AND) operator $\overline{\wedge}$ (`NAND`) returns Impossible if either argument is Impossible; otherwise it returns True if either argument is False; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns False. It obeys the interval containment rule. The $\overline{\wedge}$ operator may be described by this table:

$\overline{\wedge}$	Uncertain	True	False	Impossible
Uncertain	Uncertain	Uncertain	True	Impossible
True	Uncertain	False	True	Impossible
False	True	True	True	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.13 `opr` $\overline{\vee}$(`self`, *other*: BooleanInterval): BooleanInterval

The logical NOR (NOT OR) operator $\overline{\vee}$ (`NOR`) returns Impossible if either argument is Impossible; otherwise it returns False if either argument is True; otherwise it returns Uncertain if either argument is Uncertain; otherwise it returns True. It obeys the interval containment rule. The $\overline{\vee}$ operator may be described by this table:

$\overline{\vee}$	Uncertain	True	False	Impossible
Uncertain	Uncertain	False	Uncertain	Impossible
True	False	False	False	Impossible
False	Uncertain	False	True	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.14 `opr` $\in$(*other*: Boolean, `self`): Boolean

The operator $\in$ (`IN`) returns *true* if its first argument, a boolean value, is contained in its second argument, a boolean interval regarded as a set; otherwise it returns *false*. The $\in$ operator may be described by this table:

$\in$	Uncertain	True	False	Impossible
true	*true*	*true*	*false*	*false*
false	*true*	*false*	*true*	*false*

32.2.15 `opr` $\cap$(`self`, *other*: BooleanInterval): BooleanInterval

The intersection operator $\cap$ (`INTERSECTION` or `CAP`) returns Impossible if either argument is Impossible; otherwise, if either argument is Uncertain, it returns the other argument; otherwise, if the arguments are the same value (strictly equivalent), it returns that value; otherwise it returns Impossible. The $\cap$ operator may be described by this table:

$\cap$	Uncertain	True	False	Impossible
Uncertain	Uncertain	True	False	Impossible
True	True	True	Impossible	Impossible
False	False	Impossible	False	Impossible
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.16 `opr` ∪(`self`, *other*: BooleanInterval): BooleanInterval

The union operator ∪ (`UNION` or `CUP`) returns Uncertain if either argument is Uncertain; otherwise, if either argument is Impossible, it returns the other argument; otherwise, if the arguments are the same value (strictly equivalent), it returns that value; otherwise it returns Uncertain. The ∪ operator may be described by this table:

∪	Uncertain	True	False	Impossible
Uncertain	Uncertain	Uncertain	Uncertain	Uncertain
True	Uncertain	True	Uncertain	True
False	Uncertain	Uncertain	False	False
Impossible	Uncertain	True	False	Impossible

32.2.17 `opr SET_COMPLEMENT`(`self`): BooleanInterval

The set complement operator `SET_COMPLEMENT` returns Uncertain if its argument is Impossible, Impossible if its argument is Uncertain, False if its argument is True, and True if its argument is False.

32.2.18 `opr SYMDIFF`(`self`, *other*: BooleanInterval): BooleanInterval

The symmetric difference operator `SYMDIFF` produces a result that contains a given boolean value if and only if exactly one of the arguments contains that boolean value. The `SYMDIFF` operator may be described by this table:

SYMDIFF	Uncertain	True	False	Impossible
Uncertain	Impossible	False	True	Uncertain
True	False	Impossible	Uncertain	True
False	True	Uncertain	Impossible	False
Impossible	Uncertain	True	False	Impossible

32.2.19 `opr` \(`self`, *other*: BooleanInterval): BooleanInterval

The set difference operator \ (`SETMINUS`) produces a result that contains a given boolean value if and only if the first argument contains that boolean value but the second argument does not. The \ operator may be described by this table:

\	Uncertain	True	False	Impossible
Uncertain	Impossible	False	True	Uncertain
True	Impossible	Impossible	True	True
False	Impossible	False	Impossible	False
Impossible	Impossible	Impossible	Impossible	Impossible

32.2.20 *possibly*(`self`): Boolean
32.2.21 *necessarily*(`self`): Boolean
32.2.22 *certainly*(`self`): Boolean

The predicate *possibly* returns *true* if and only if *true* is a member of this boolean interval.

The predicate *necessarily* returns *true* if and only if *false* is not a member of this boolean interval (thus "necessarily" is a concise way of saying "not possibly not").

The predicate *certainly* returns *true* if and only if this boolean interval is True, that is, it contains *true* but not *false* (thus "certainly" is a concise way of saying "both possibly and necessarily").

The fourteen nontrivial functions from a value x of type BooleanInterval to type Boolean may thus be expressed as follows:

$$\begin{gathered}
necessarily(x) \wedge necessarily(\neg x) \\
certainly(\neg x) \\
necessarily(\neg x) \\
certainly(x) \\
necessarily(x) \\
possibly(x) \equiv necessarily(x) \\
necessarily(x) \vee necessarily(\neg x) \\
possibly(x) \wedge possibly(\neg x) \\
possibly(x) \veebar necessarily(x) \\
possibly(\neg x) \\
\neg certainly(x) \\
possibly(x) \\
\neg certainly(\neg x) \\
possibly(x) \vee possibly(\neg x))
\end{gathered}$$

There are other ways to express some of them; for example, $necessarily(x) \wedge necessarily(\neg x)$ is the same as $x \equiv \text{Impossible}$, and $possibly(x) \wedge possibly(\neg x)$ is the same as $x \equiv \text{Uncertain}$.

32.2.23 `opr` ≡(`self`, *other*: BooleanInterval): Boolean

Two boolean intervals are strictly equivalent if and only if they are the same boolean interval.

32.2.24 *toString*(): String

The *toString* method returns either "`True`" or "`False`" or "`Uncertain`" or "`Impossible`" as appropriate.

32.3 Top-level BooleanInterval Values

32.3.1 True: BooleanInterval
32.3.2 False: BooleanInterval
32.3.3 Uncertain: BooleanInterval
32.3.4 Impossible: BooleanInterval

The immutable variables True, False, Uncertain, and Impossible have as their values the four boolean intervals. They are top-level variables declared in the Fortress standard libraries.

Chapter 33

Numbers

33.1 Rational Numbers

The trait $\mathbb{Q}$ (`QQ`) encompasses all finite rational numbers, the results of dividing any integer by any nonzero integer, and three more elements: $+\infty$ and $-\infty$ (written $1/0$ and $-1/0$), the results of dividing a positive or a negative integer by zero, and the indefinite rational (written $0/0$), which is used as the result of dividing zero by zero or of adding $-\infty$ to $+\infty$.

The number types of this chapter are the integer types $\mathbb{Z}$, $\mathbb{Z}64$, $\mathbb{Z}32$, $\mathbb{N}64$, and $\mathbb{N}32$ (Section 33.2), the rational type $\mathbb{Q}$, and the floating-point types $\mathbb{R}64$ (`RR64`) and $\mathbb{R}32$ (`RR32`). They are siblings under the trait Number: each is a subtype of Number, and none is a subtype of another. Some conversions between them are coercions (Chapter 17), each declared by the type converted to; since coercions do not chain (Section 17.1), each pair is declared. Those among the integer types are given in Section 33.2; the others are these:

$\mathbb{Q}$ (`QQ`) coerces from $\mathbb{Z}$, $\mathbb{Z}64$, $\mathbb{Z}32$, $\mathbb{N}64$, and $\mathbb{N}32$.
$\mathbb{R}64$ (`RR64`) coerces from $\mathbb{R}32$, from $\mathbb{Z}32$, and from integer numerals (Section 13.1).

Every other conversion between number types is written explicitly; in particular, a conversion into $\mathbb{R}64$ from $\mathbb{Z}$, $\mathbb{Z}64$, $\mathbb{N}64$, $\mathbb{N}32$, or $\mathbb{Q}$ is explicit, written $asFloat(x)$ with the method that the Fortress standard libraries declare on Number.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.28
>
> Before this revision the passage named $\mathbb{R}64$ as the one floating-point type, and the Fortress standard libraries declared $\mathbb{R}32$ a subtype of it, so that a value of $\mathbb{R}32$ was also a value of $\mathbb{R}64$, which Section 6.9 excludes. It now names $\mathbb{R}32$ beside $\mathbb{R}64$, a sibling under Number as the Fortress standard libraries now declare it, with the coercion into $\mathbb{R}64$ that the example of Section 17.8 relies on.

It is often desirable to know that a value lies in a subset of its type, such as the positive values, the nonzero values, or the finite values. Such subsets are not types of their own: a program tests a value at run time, with the comparison operators for its sign and, for a floating-point value, with the two checks that the Fortress standard libraries declare on $\mathbb{R}64$:

`getter` $check(): \mathrm{Maybe}[\![\mathbb{R}64]\!]$
`getter` $check^*(): \mathrm{Maybe}[\![\mathbb{R}64]\!]$

The getter $check$ returns $\mathrm{Just}(x)$ for a finite value x and $\mathrm{Nothing}$ otherwise; the getter $check^*$ returns $\mathrm{Just}(x)$ for a value x that is not a NaN and $\mathrm{Nothing}$ otherwise.

Here we present the trait $\mathbb{Q}$ and its methods. Because $\mathbb{Q}$ holds $0/0$, which is unordered with respect to every rational value, $\mathbb{Q}$ is neither totally ordered nor a field. The original design of the rational types, in which $\mathbb{Q}$ and seventeen refined rational types are instantiations of one trait, is kept as a superseded design in Section 41.1.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.11
>
> The Working Draft of February 2011 divided the rationals into eighteen types: $\mathbb{Q}$, the type $\mathbb{Q}^*$ that adds the two infinities, the type $\mathbb{Q}^\#$ that adds $0/0$, and five refinements of each by sign, each a subtype of others and of the corresponding real types, and it said that the type system tracks them through arithmetic. The Fortress standard libraries never declared them, and the trait by which Section 41.1 defines them is refused by instantiation exclusion (Section I.1.1). The text above describes the number types the library declares, siblings under Number, with the coercions it declares; a subset such as the positive values is tested at run time. $\mathbb{Q}$ holds the values that $\mathbb{Q}^*$ and $\mathbb{Q}^\#$ added, as the library's rational type does and as Chapter 16 already says ("For rational results, division by zero produces $1/0$"). Because it holds $0/0$, the listing below no longer gives it the field and order traits of the original design, which that design itself withheld from $\mathbb{Q}^\#$.

```
trait ℚ
    extends { Number, StandardPartialOrder⟦ℚ⟧, StandardMinMax⟦ℚ⟧,
              AdditiveGroup⟦ℚ⟧, MultiplicativeRing⟦ℚ⟧ }
  coerce (x: ℤ)
  coerce (x: ℤ64)
  coerce (x: ℤ32)
  coerce (x: ℕ64)
  coerce (x: ℕ32)
  coerce (x: Identity⟦+⟧)
  coerce (x: Identity⟦·⟧)
  coerce (x: Identity⟦×⟧)
  coerce (x: Identity⟦juxtaposition⟧)
  coerce (x: Zero⟦·⟧)
  coerce (x: Zero⟦×⟧)
  coerce (x: Zero⟦juxtaposition⟧)
  opr juxtaposition (self, other: ℚ): ℚ
  opr +(self): ℚ
  opr +(self, other: ℚ): ℚ
  opr −(self): ℚ
  opr −(self, other: ℚ): ℚ
  opr ·(self, other: ℚ): ℚ
  opr ×(self, other: ℚ): ℚ
  opr /(self): ℚ
  opr /(self, other: ℚ): ℚ
  opr ^(self, power: ℤ): ℚ
  opr <(self, other: ℚ): Boolean
  opr ≤(self, other: ℚ): Boolean
  opr =(self, other: ℚ): Boolean
  opr ≥(self, other: ℚ): Boolean
  opr >(self, other: ℚ): Boolean
  opr CMP(self, other: ℚ): Comparison
  opr MAX(self, other: ℚ): ℚ
  opr MIN(self, other: ℚ): ℚ
  opr MAXNUM(self, other: ℚ): ℚ
  opr MINNUM(self, other: ℚ): ℚ
  opr |self| : ℚ
  signum(self): ℤ
  numerator(self): ℤ
```

```
    denominator(self): ℤ
    floor(self): ℤ
    opr ⌊self⌋: ℤ
    ceiling(self): ℤ
    opr ⌈self⌉: ℤ
    round(self): ℤ
    truncate(self): ℤ
    opr ⟦self⟧: ℤ
    opr ⟦self⟧: ℤ
    opr ⟦⟦self⟧⟧: ℤ
    opr ⟦⟦self⟧⟧: ℤ
    realpart(self): ℚ
    imagpart(self): ℚ
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.12
>
> The Working Draft of February 2011 declared $\mathbb{Q}$ a subtype of the real numbers and of $\mathbb{Q}^*$, a field with the nonzero rationals $\mathbb{Q}_{\neq}$ as its multiplicative group, and totally ordered; it typed the results of the reciprocal, of division and of exponentiation as $\mathbb{Q}^*$ or $\mathbb{Q}^{\#}$, those of the absolute value and of the four hyperfloor and hyperceiling operators as $\mathbb{Q}_{\geq}$ and $\mathbb{N}$, and it declared seventeen check methods, each returning a refined type or throwing $\mathrm{CastError}$. The listing and the method entries below take the type the Fortress standard libraries declare where they declare the method, and $\mathbb{Q}$ or $\mathbb{Z}$ where they do not; the two comparisons by `CMP` become the one the library declares. The check methods leave the listing: the library declares *check* and *check** on $\mathbb{R}64$ (Section 33.1) and no check by sign. What the method entries said "for types $\mathbb{Q}^*$ and $\mathbb{Q}^{\#}$" they now say of $\mathbb{Q}$, which holds those values. The coercions from the five integer types are new, with the method entry that follows.

The four traits that the listing's `extends` clause names after Number are those the Fortress standard libraries declare on $\mathbb{Q}$; their laws hold for the finite rationals and need not hold where $0/0$ or an infinity takes part, as the laws of the same traits on $\mathbb{R}64$ need not hold where a NaN takes part.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.25
>
> Before this revision the listing extended Number alone, the field and order traits of the original design having left it (Section I.1.12); it now names the traits the Fortress standard libraries declare on $\mathbb{Q}$, and the sentence above says where their laws hold.

33.1.1 `coerce` $(x: \mathbb{Z})$
33.1.2 `coerce` $(x: \mathbb{Z}64)$
33.1.3 `coerce` $(x: \mathbb{Z}32)$
33.1.4 `coerce` $(x: \mathbb{N}64)$
33.1.5 `coerce` $(x: \mathbb{N}32)$

Each of these coercions converts an integer to the rational number with the same value.

33.1.6 `coerce` $(x\colon \mathrm{Identity}[\![+]\!])$
33.1.7 `coerce` $(x\colon \mathrm{Identity}[\![\cdot]\!])$
33.1.8 `coerce` $(x\colon \mathrm{Identity}[\![\times]\!])$
33.1.9 `coerce` $(x\colon \mathrm{Identity}[\![\texttt{juxtaposition}]\!])$
33.1.10 `coerce` $(x\colon \mathrm{Zero}[\![\cdot]\!])$
33.1.11 `coerce` $(x\colon \mathrm{Zero}[\![\times]\!])$
33.1.12 `coerce` $(x\colon \mathrm{Zero}[\![\texttt{juxtaposition}]\!])$

The identity for $+$ is 0.

The identity for `juxtaposition` or $\cdot$ or $\times$ is 1.

The zero for `juxtaposition` or $\cdot$ or $\times$ is 0.

33.1.13 `opr juxtaposition` $(\texttt{self}, \mathit{other}\colon \mathbb{Q})\colon \mathbb{Q}$

Juxtaposition of rational expressions is equivalent to using the multiplication operator $\cdot$.

33.1.14 `opr` $+(\texttt{self})\colon \mathbb{Q}$

The unary addition operator $+$ simply returns its argument.

33.1.15 `opr` $+(\texttt{self}, \mathit{other}\colon \mathbb{Q})\colon \mathbb{Q}$

The binary addition operator $+$ returns the sum of its arguments.

The sum of an infinity and either a finite rational or another infinity of the same sign is equal to the given infinity, but the sum of infinities of differing sign is $0/0$, and the sum of $0/0$ and any rational value is $0/0$.

33.1.16 `opr` $-(\texttt{self})\colon \mathbb{Q}$

The unary negation operator $-$ returns the negative of its argument.

The negative of $+\infty$ is $-\infty$, the negative of $-\infty$ is $+\infty$, and the negative of $0/0$ is $0/0$.

33.1.17 `opr` $-(\texttt{self}, \mathit{other}\colon \mathbb{Q})\colon \mathbb{Q}$

The binary subtraction operator $-$ returns the difference of its arguments, which is equal to the sum of (a) the first argument and (b) the negation of the second argument.

33.1.18 `opr` $\cdot$(`self`, *other*: $\mathbb{Q}$): $\mathbb{Q}$
33.1.19 `opr` $\times$(`self`, *other*: $\mathbb{Q}$): $\mathbb{Q}$

The multiplication operator $\cdot$ (`DOT`) returns the product of its arguments. The multiplication operator $\times$ (`TIMES`) does exactly the same thing.

The product of $0/0$ and any rational value is $0/0$, and the product of zero and an infinity (regardless of sign) is $0/0$; the product of an infinity and any rational value other than zero and $0/0$ is an infinity whose sign is positive if and only if the two arguments have the same sign.

33.1.20 `opr` /(`self`): $\mathbb{Q}$

The unary reciprocal operator / returns the reciprocal of its argument. The reciprocal of zero is $+\infty$.

The reciprocal of either $+\infty$ or $-\infty$ is zero, and the reciprocal of $0/0$ is $0/0$.

33.1.21 `opr` /(`self`, *other*: $\mathbb{Q}$): $\mathbb{Q}$

The binary division operator / returns the quotient of its arguments, which is equal to the product of (a) the first argument and (b) the reciprocal of the second argument.

33.1.22 `opr` ^(`self`, *power*: $\mathbb{Z}$): $\mathbb{Q}$

Exponentiation of a rational number to an integer power produces a rational result. If the *power* is 0, then the result is always 1, even if the rational number base is 0 (this definition is somewhat arbitrary but is computationally useful).

`property` $\forall(x, y : \mathbb{Z})\ x^y = 1/(x^{-y})$
`property` $\forall(x, y : \mathbb{Z})\ x^y = x^{(\lfloor y/2 \rfloor)} x^{(\lceil y/2 \rceil)}$

33.1.23 `opr` <(`self`, *other*: $\mathbb{Q}$): Boolean
33.1.24 `opr` $\leq$(`self`, *other*: $\mathbb{Q}$): Boolean
33.1.25 `opr` =(`self`, *other*: $\mathbb{Q}$): Boolean
33.1.26 `opr` $\geq$(`self`, *other*: $\mathbb{Q}$): Boolean
33.1.27 `opr` >(`self`, *other*: $\mathbb{Q}$): Boolean

The comparison operators $<$, $\leq$, $=$, $\geq$, and $>$ allow any rational value to be compared numerically to any other rational value.

The rational values are totally ordered except for $0/0$, which is unordered with respect to all other rational values; moreover, for compatibility with floating-point arithmetic, $0/0$ is unordered with respect to itself, and therefore these five comparison operators always return *false* if either argument is $0/0$. The value $-\infty$ is less than any finite rational value, and $+\infty$ is greater than any finite rational value.

For $\neq$ see Section 34.1.4.

33.1.28 `opr CMP(self, `$other$`: ℚ): Comparison`

The `CMP` operator compares the arguments and returns one of the four values LessThan, EqualTo, GreaterThan, and Unordered.

33.1.29 `opr MAX(self, `$other$`: ℚ): ℚ`
33.1.30 `opr MIN(self, `$other$`: ℚ): ℚ`
33.1.31 `opr MAXNUM(self, `$other$`: ℚ): ℚ`
33.1.32 `opr MINNUM(self, `$other$`: ℚ): ℚ`

The operators `MAX` and `MAXNUM` return whichever argument is larger in the total order defined by $<$, $\leq$, $=$, $\geq$, $>$, and `CMP`, and the operators `MIN` and `MINNUM` return whichever argument is smaller. (For all four, if the arguments are equal, then the result equals that same value.)

The operators `MAXNUM` and `MINNUM` differ from `MAX` and `MIN` in their treatment of $0/0$: if one argument is $0/0$ and the other is not, then `MAX` or `MIN` returns $0/0$ but `MAXNUM` or `MINNUM` returns the argument that is not $0/0$.

33.1.33 `opr |self| : ℚ`

The absolute value operator $|\ldots|$ returns the negative of this rational number if the argument is less than zero, and otherwise returns the argument.

The absolute value of $0/0$ is $0/0$.

33.1.34 $signum$`(self): ℤ`

The method $signum$ returns -1 if this rational number is less than zero, 0 if this rational number is zero, and 1 if this rational number is greater than zero.

The signum of $0/0$ is $0/0$.

33.1.35 $numerator$`(self): ℤ`
33.1.36 $denominator$`(self): ℤ`

The method $numerator$ returns the numerator of this rational number, and the method $denominator$ returns the denominator of this rational number, when this rational number is represented in lowest terms (such that the greatest common divisor of numerator and denominator is 1).

The numerator of $+\infty$ is 1, the numerator of $-\infty$ is -1, and the numerator of $0/0$ is 0; the denominator of $+\infty$, $-\infty$, or $0/0$ is 0.

33.1.37 *floor*(`self`): $\mathbb{Z}$
33.1.38 `opr` $\lfloor$`self`$\rfloor$: $\mathbb{Z}$
33.1.39 *ceiling*(`self`): $\mathbb{Z}$
33.1.40 `opr` $\lceil$`self`$\rceil$: $\mathbb{Z}$
33.1.41 *round*(`self`): $\mathbb{Z}$
33.1.42 *truncate*(`self`): $\mathbb{Z}$

The method *floor*, likewise the enclosing operator $\lfloor \ldots \rfloor$, returns the largest integer that is not greater than this rational number.

The method *ceiling*, likewise the enclosing operator $\lceil \ldots \rceil$, returns the smallest integer that is not less than this rational number.

The method *round* returns the integer that is closest to this rational number, but if this rational number is exactly halfway between two consecutive integers, then *round* returns whichever of the two integers is even.

The method *truncate* returns the ceiling of this rational number if it is negative, and otherwise returns the floor of this rational number. (This has the effect of taking the floor of the magnitude, also called "rounding toward zero.")

All of these methods throw a DivisionByZero if this rational number is $+\infty$, $-\infty$, or $0/0$, none of which has an integer value; so does a division by zero whose result is an integer (Chapter 16).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.39
>
> Before this revision the sentence said that all of these methods simply return the argument if it is $+\infty$, $-\infty$, or $0/0$, a rational number where the entries above declare an integer result. It now says that they throw a DivisionByZero there, as the Fortress standard libraries do.

`opr` $\llbracket$`self`$\rrbracket$: $\mathbb{Z}$
`opr` $\llparenthesis$`self`$\rrparenthesis$: $\mathbb{Z}$
`opr` $\lBrack\!\lBrack$`self`$\rBrack\!\rBrack$: $\mathbb{Z}$
`opr` $\lBrack\!\lBrack\!\lBrack$`self`$\rBrack\!\rBrack\!\rBrack$: $\mathbb{Z}$

The hyperfloor operation $\llbracket x \rrbracket$ computes $2^{(\lfloor log_2 x \rfloor)}$ and returns the result as a natural number. If the argument is equal to 0, the result is 0. If the argument is negative, an InvalidArgument is thrown.

The hyperceiling operation $\llparenthesis x \rrparenthesis$ computes $2^{(\lceil log_2 x \rceil)}$ and returns the result as a natural number. If the argument is equal to 0, the result is 0. If the argument is negative, an InvalidArgument is thrown.

The hyperhyperfloor operation $\lBrack\!\lBrack x \rBrack\!\rBrack$ computes $2^{(\llbracket log_2 x \rrbracket)}$ and returns the result as a natural number. If the argument is equal to 0 or 1, the result is the same as the argument. If the argument is negative, an InvalidArgument is thrown.

The hyperhyperceiling operation $\lBrack\!\lBrack\!\lBrack x \rBrack\!\rBrack\!\rBrack$ computes $2^{(\llparenthesis log_2 x \rrparenthesis)}$ and returns the result as a natural number. If the argument is equal to 0 or 1, the result is the same as the argument. If the argument is negative, an InvalidArgument is thrown.

33.1.43 *realpart*(`self`): $\mathbb{Q}$

The method *realpart* for a rational number simply returns its argument.

33.1.44 *imagpart*(`self`): $\mathbb{Q}$

The method *imagpart* for a rational number simply returns zero.

33.2 Integers

The trait $\mathbb{Z}$ (`ZZ`) encompasses all finite integers, the results of starting from 0 and repeatedly adding or subtracting 1 a finite number of times.

The Fortress standard libraries also declare four integer types of fixed size. The five integer types are these:

$\mathbb{Z}$ (`ZZ`) is the set of integers (it coerces from $\mathbb{Z}64$, $\mathbb{Z}32$, $\mathbb{N}64$, and $\mathbb{N}32$).
$\mathbb{Z}64$ (`ZZ64`) is the set of integers representable in 64-bit two's-complement form (it coerces from $\mathbb{Z}32$ and $\mathbb{N}32$).
$\mathbb{Z}32$ (`ZZ32`) is the set of integers representable in 32-bit two's-complement form.
$\mathbb{N}64$ (`NN64`) is the set of integers representable in 64-bit unsigned binary form (it coerces from $\mathbb{N}32$).
$\mathbb{N}32$ (`NN32`) is the set of integers representable in 32-bit unsigned binary form.

Like every number type, each of them is a subtype of Number and of no other number type (Section 33.1), and each coerces into $\mathbb{Q}$. An integer type coerces from each other integer type all of whose values it holds, as listed above; every other conversion between integer types is written explicitly.

Among the explicit conversions, the method *narrow* converts a value of $\mathbb{Z}64$ or $\mathbb{Z}$ to $\mathbb{Z}32$, and a value of $\mathbb{N}64$ to $\mathbb{N}32$, by keeping its low 32 bits: the result is the value of the narrower type that is congruent to the argument modulo 2^{32}. A value that the narrower type holds is unchanged; any other wraps, and no $\mathrm{IntegerOverflow}$ is thrown.

The operators `LSHIFT` and `RSHIFT` shift the bits of an integer of any of the five types by a count of any integer type, and return a value of the type of the integer shifted. On $\mathbb{Z}$, x `LSHIFT` k is $\mathit{shift}(x, k)$ and x `RSHIFT` k is $\mathit{shift}(x, -k)$. On a fixed-size type of width n, for a nonnegative count k, x `RSHIFT` k is $\lfloor x/2^k \rfloor$, and x `LSHIFT` k is the value of the type that is congruent to $x \cdot 2^k$ modulo 2^n: the bits shifted out are lost, and no $\mathrm{IntegerOverflow}$ is thrown. So a count at or beyond the width gives 0 for a left shift, and for a right shift 0, or -1 if x is negative. A negative count shifts the other way: x `LSHIFT` $(-k)$ is x `RSHIFT` k, and x `RSHIFT` $(-k)$ is x `LSHIFT` k. The count is read by its value, whatever its type: an $\mathbb{N}64$ count of 2^{63} or more is beyond the width, not negative.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.22
>
> The Working Draft of February 2011 named neither *narrow* nor `LSHIFT` and `RSHIFT`, which the Fortress standard libraries declare on the integer types; the two paragraphs above state the rules the revival decided for them.

It is often desirable to know that an integer lies in a subset of its type, such as the positive values or the nonzero values. As for the rationals, such subsets are not types of their own: a program tests an integer's sign at run time with the comparison operators. The integer types hold no infinite value.

Here we present only the trait $\mathbb{Z}$ and its methods. $\mathbb{Z}$ is a commutative ring and is totally ordered. *Future versions of this specification will include the exact details of how all this is implemented.*

> REVISED BY THE 2026 REVIVAL SEE Section I.1.11
>
> The Working Draft of February 2011 gave the integers twelve types, $\mathbb{Z}$, the type $\mathbb{Z}^*$ that adds the two infinities, and five refinements of each by sign, each a subtype of the corresponding rational type, with $\mathbb{N}$ a synonym for $\mathbb{Z}_{\geq}$ and $\mathbb{N}^*$ for $\mathbb{Z}^*_{\geq}$, and it said that the type system tracks them through arithmetic. As for the rationals, the text above describes the integer types the Fortress standard libraries declare instead, siblings under Number, with the coercions they declare: $\mathbb{Z}$ is no longer a subtype of $\mathbb{Q}$, which coerces from it. The library's integer types hold no infinite value. A generic call over two different integer types infers the narrowest type that both coerce into, by the rule of Chapter 20, and need not write its static argument: with a of type $\mathbb{Z}32$ and b of type $\mathbb{Z}64$, $f(a, b)$ infers $\mathbb{Z}64$, as $f[\![\mathbb{Z}64]\!](a, b)$ writes it.

```
trait ℤ
    extends { Number, IntegerLike⟦ℤ⟧,
              CommutativeRing⟦ℤ, +, −, juxtaposition⟧,
              CommutativeRing⟦ℤ, +, −, ·⟧,
```

CommutativeRing⟦$\mathbb{Z}, +, -, \times$⟧,
CommutativeRing⟦$\mathbb{Z}, \boxplus, \boxminus, \boxdot$⟧,
CommutativeRing⟦$\mathbb{Z}, \boxplus, \boxminus, \boxtimes$⟧,
CommutativeRing⟦$\mathbb{Z}, \dot{+}, \dot{-}, \dot{\times}$⟧,
TotalOrderOperators⟦$\mathbb{Z}, <, \leq, \geq, >$, `CMP`⟧,
PartialOrderAndLattice⟦$\mathbb{Z}, \leq$, `MIN`, `MAX`⟧,
BooleanAlgebra⟦$\mathbb{Z}, \curlywedge, \curlyvee, \neg, \underline{\curlyvee}$⟧ }
`coerce` (x: $\mathbb{Z}64$)
`coerce` (x: $\mathbb{Z}32$)
`coerce` (x: $\mathbb{N}64$)
`coerce` (x: $\mathbb{N}32$)
`coerce` (x: Identity⟦$+$⟧)
`coerce` (x: Identity⟦$\boxplus$⟧)
`coerce` (x: Identity⟦$\dot{+}$⟧)
`coerce` (x: Identity⟦`juxtaposition`⟧)
`coerce` (x: Identity⟦$\cdot$⟧)
`coerce` (x: Identity⟦$\times$⟧)
`coerce` (x: Identity⟦$\boxdot$⟧)
`coerce` (x: Identity⟦$\boxtimes$⟧)
`coerce` (x: Identity⟦$\dot{\times}$⟧)
`coerce` (x: Zero⟦`juxtaposition`⟧)
`coerce` (x: Zero⟦$\cdot$⟧)
`coerce` (x: Zero⟦$\times$⟧)
`coerce` (x: Zero⟦$\boxdot$⟧)
`coerce` (x: Zero⟦$\boxtimes$⟧)
`coerce` (x: Zero⟦$\dot{\times}$⟧)
`opr juxtaposition` (`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $+$(`self`): $\mathbb{Z}$
`opr` $\boxplus$(`self`): $\mathbb{Z}$
`opr` $\dot{+}$(`self`): $\mathbb{Z}$
`opr` $+$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\boxplus$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\dot{+}$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $-$(`self`): $\mathbb{Z}$
`opr` $\boxminus$(`self`): $\mathbb{Z}$
`opr` $\dot{-}$(`self`): $\mathbb{Z}$
`opr` $-$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\boxminus$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\dot{-}$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\cdot$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\times$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\boxdot$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\boxtimes$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` $\dot{\times}$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$
`opr` /(`self`, *other*: $\mathbb{Z}$): $\mathbb{Q}$
`opr` $\div$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$ `throws` IntegerDivisionByZero
`opr` `REM`(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$ `throws` IntegerDivisionByZero
`opr` `MOD`(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$ `throws` IntegerDivisionByZero
`opr` `DIVREM`(`self`, *other*: $\mathbb{Z}$): ($\mathbb{Z}, \mathbb{Z}$) `throws` IntegerDivisionByZero
`opr` `DIVMOD`(`self`, *other*: $\mathbb{Z}$): ($\mathbb{Z}, \mathbb{Z}$) `throws` IntegerDivisionByZero
`opr` |(`self`, *other*: $\mathbb{Z}$): Boolean
`opr` _(`self`, *power*: $\mathbb{Z}$): $\mathbb{Z}$

```
    opr GCD(self, other: ℤ): ℤ
    opr LCM(self, other: ℤ): ℤ
    opr (self)! : ℤ
    opr CHOOSE(self, other: ℤ): ℤ
    opr <(self, other: ℤ): Boolean
    opr ≤(self, other: ℤ): Boolean
    opr =(self, other: ℤ): Boolean
    opr ≥(self, other: ℤ): Boolean
    opr >(self, other: ℤ): Boolean
    opr CMP(self, other: ℤ): TotalComparison
    opr MAX(self, other: ℤ): ℤ
    opr MIN(self, other: ℤ): ℤ
    opr MAXNUM(self, other: ℤ): ℤ
    opr MINNUM(self, other: ℤ): ℤ
    opr |self| : ℤ
    signum(self): ℤ
    numerator(self): ℤ
    denominator(self): ℤ
    floor(self): ℤ
    opr ⌊self⌋: ℤ
    ceiling(self): ℤ
    opr ⌈self⌉: ℤ
    round(self): ℤ
    truncate(self): ℤ
    opr ⦇self⦈: ℤ
    opr ⦈self⦇: ℤ
    opr ⦇⦇self⦈⦈: ℤ
    opr ⦈⦇self⦇⦈: ℤ
    shift(self, k: IndexInt): ℤ
    bit(self, k: IndexInt): Bit
    opr ⫬(self): ℤ
    opr ⩓(self, other: ℤ): ℤ
    opr ⩔(self, other: ℤ): ℤ
    opr ⩒(self, other: ℤ): ℤ
    countBits(self): IndexInt
    countFactorsOfTwo(self): IndexInt
    integerLength(self): IndexInt
    lowBits(self, k: IndexInt): ℤ
    even(self): Boolean
    odd(self): Boolean
    prime(self): Boolean
    realpart(self): ℤ
    imagpart(self): ℤ
end
```

REVISED BY THE 2026 REVIVAL SEE Section I.1.13

The Working Draft of February 2011 declared ℤ a subtype of ℚ and of $\mathbb{Z}^*$; it typed the result of division as $\mathbb{Q}^\#$, the power's exponent, the results of the factorial, of CHOOSE and of the four hyperfloor and hyperceiling operators as ℕ, the argument of CMP as $\mathbb{Z}^*$, and the absolute value as $\mathbb{Z}_{\geq}$, and it declared twelve check methods, each returning a refined type or throwing CastError. The listing and the method entries below extend Number in place of ℚ and $\mathbb{Z}^*$, declare

the coercions from the four fixed-size integer types, and take $\mathbb{Z}$ or $\mathbb{Q}$ where the original named a refined type; the check methods leave, since the Fortress standard libraries declare none on the integer types. The sentences about the infinities of $\mathbb{Z}^*$ and $\mathbb{N}^*$ leave the method entries, and so does the sentence that declared the positive integers a type of their own, ordered by divisibility. The coercions from the fixed-size integer types are new, with the method entry that follows. The exponent of the power, $\mathbb{N}$ in the Working Draft, is now $\mathbb{Z}$, so a negative power is admitted. The Working Draft gave the result of a negative power through the exponentiation of $\mathbb{Q}$, since $\mathbb{Z}$ was a subtype of $\mathbb{Q}$: 2^{-1} was $1/2$. The method entry of the power gives it otherwise: an integer raised to a negative power is an integer only at the bases 1 and -1; at a base of 0 the power throws $\mathrm{DivisionByZero}$, at any other base it throws $\mathrm{IntegerOverflow}$, and a fraction is written with a floating-point or rational base. The method entries of the three unary negations gave the result $\mathbb{Q}$ where the listing gave $\mathbb{Z}$; on the flat tower that would make the negative of an integer a rational, and they now give $\mathbb{Z}$, as the listing does.

33.2.1 `coerce` $(x\colon \mathbb{Z}64)$
33.2.2 `coerce` $(x\colon \mathbb{Z}32)$
33.2.3 `coerce` $(x\colon \mathbb{N}64)$
33.2.4 `coerce` $(x\colon \mathbb{N}32)$

Each of these coercions converts a fixed-size integer to the integer with the same value.

33.2.5 `coerce` $(x\colon \mathrm{Identity}[\![+]\!])$
33.2.6 `coerce` $(x\colon \mathrm{Identity}[\![\boxplus]\!])$
33.2.7 `coerce` $(x\colon \mathrm{Identity}[\![\dot{+}]\!])$
33.2.8 `coerce` $(x\colon \mathrm{Identity}[\![\texttt{juxtaposition}]\!])$
33.2.9 `coerce` $(x\colon \mathrm{Identity}[\![\cdot]\!])$
33.2.10 `coerce` $(x\colon \mathrm{Identity}[\![\times]\!])$
33.2.11 `coerce` $(x\colon \mathrm{Identity}[\![\boxdot]\!])$
33.2.12 `coerce` $(x\colon \mathrm{Identity}[\![\boxtimes]\!])$
33.2.13 `coerce` $(x\colon \mathrm{Identity}[\![\dot{\times}]\!])$
33.2.14 `coerce` $(x\colon \mathrm{Zero}[\![\texttt{juxtaposition}]\!])$
33.2.15 `coerce` $(x\colon \mathrm{Zero}[\![\cdot]\!])$
33.2.16 `coerce` $(x\colon \mathrm{Zero}[\![\times]\!])$
33.2.17 `coerce` $(x\colon \mathrm{Zero}[\![\boxdot]\!])$
33.2.18 `coerce` $(x\colon \mathrm{Zero}[\![\boxtimes]\!])$
33.2.19 `coerce` $(x\colon \mathrm{Zero}[\![\dot{\times}]\!])$

The identity for $+$ or $\boxplus$ or $\dot{+}$ is 0.

The identity for `juxtaposition` or $\cdot$ or $\times$ or $\boxdot$ or $\boxtimes$ or $\dot{\times}$ is 1.

The zero for `juxtaposition` or $\cdot$ or $\times$ or $\boxdot$ or $\boxtimes$ or $\dot{\times}$ is 0.

33.2.20 `opr juxtaposition` $(\texttt{self}, \mathit{other}\colon \mathbb{Z})\colon \mathbb{Z}$

Juxtaposition of integer expressions is equivalent to using the multiplication operator $\cdot$.

33.2.21 `opr` $+(\texttt{self}) \colon \mathbb{Z}$
33.2.22 `opr` $\boxplus(\texttt{self}) \colon \mathbb{Z}$
33.2.23 `opr` $\dot{+}(\texttt{self}) \colon \mathbb{Z}$

The unary addition operator $+$ simply returns its argument. The operators $\boxplus$ and $\dot{+}$ do exactly the same thing.

33.2.24 `opr` $+(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.25 `opr` $\boxplus(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.26 `opr` $\dot{+}(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$

The binary addition operator $+$ returns the sum of its arguments. The wrapping and saturating addition operators $\boxplus$ and $\dot{+}$ do exactly the same thing, because operations on type $\mathbb{Z}$ never need to wrap or saturate.

33.2.27 `opr` $-(\texttt{self}) \colon \mathbb{Z}$
33.2.28 `opr` $\boxminus(\texttt{self}) \colon \mathbb{Z}$
33.2.29 `opr` $\dot{-}(\texttt{self}) \colon \mathbb{Z}$

The unary negation operator $-$ returns the negative of its argument. The operators $\boxminus$ and $\dot{-}$ do exactly the same thing.

33.2.30 `opr` $-(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.31 `opr` $\boxminus(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.32 `opr` $\dot{-}(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$

The binary subtraction operator $-$ returns the difference of its arguments, which is equal to the sum of (a) the first argument and (b) the negation of the second argument. The wrapping and saturating subtraction operators $\boxminus$ and $\dot{-}$ do exactly the same thing, because operations on type ZZ never need to wrap or saturate.

33.2.33 `opr` $\cdot(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.34 `opr` $\times(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.35 `opr` $\boxdot(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.36 `opr` $\boxtimes(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$
33.2.37 `opr` $\dot{\times}(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Z}$

The multiplication operator $\cdot$ returns the product of its arguments. The multiplication operators $\times$, $\boxdot$, $\boxtimes$, and $\dot{\times}$ do exactly the same thing.

33.2.38 `opr` $/(\texttt{self}, \mathit{other} \colon \mathbb{Z}) \colon \mathbb{Q}$

The binary division operator $/$ returns the quotient of its arguments as a rational number. Unlike in other programming languages such as Fortran, the operator $/$ does not perform truncating integer division; the operator $\div$ may be used for that purpose.

33.2.39 `opr` $\div$(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$ `throws` IntegerDivisionByZero
33.2.40 `opr REM`(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$ `throws` IntegerDivisionByZero
33.2.41 `opr MOD`(`self`, *other*: $\mathbb{Z}$): $\mathbb{Z}$ `throws` IntegerDivisionByZero
33.2.42 `opr DIVREM`(`self`, *other*: $\mathbb{Z}$): $(\mathbb{Z}, \mathbb{Z})$ `throws` IntegerDivisionByZero
33.2.43 `opr DIVMOD`(`self`, *other*: $\mathbb{Z}$): $(\mathbb{Z}, \mathbb{Z})$ `throws` IntegerDivisionByZero

The operator $\div$ performs truncating integer division; if the quotient is not an exact integer, the result is rounded toward zero.

The operator `REM` returns the remainder that would be left over from a division using the operator $\div$.

The operator `MOD` returns the remainder that would be left over from an integer division that rounds inexact results towards negative infinity ("floor division").

The operator `DIVREM` returns a tuple of two results, the quotient and remainder from a truncating integer division.

The operator `DIVMOD` returns a tuple of two results, the quotient and remainder from an integer floor division.

For all these operators, if *other* is zero then an IntegerDivisionByZero is thrown.

Examples:

$+8 \div +3 = +2$	$+8$ `REM` $+3 = +2$	$+8$ `MOD` $+3 = +2$	$+8$ `DIVREM` $+3 = (+2, +2)$	$+8$ `DIVMOD` $+3 = (+2, +2)$
$-8 \div +3 = -2$	-8 `REM` $+3 = -2$	-8 `MOD` $+3 = +1$	-8 `DIVREM` $+3 = (-2, -2)$	-8 `DIVMOD` $+3 = (-3, +1)$
$+8 \div -3 = -2$	$+8$ `REM` $-3 = +2$	$+8$ `MOD` $-3 = -1$	$+8$ `DIVREM` $-3 = (-2, +2)$	$+8$ `DIVMOD` $-3 = (-3, -1)$
$-8 \div -3 = +2$	-8 `REM` $-3 = -2$	-8 `MOD` $-3 = -2$	-8 `DIVREM` $-3 = (+2, -2)$	-8 `DIVMOD` $-3 = (+2, -2)$

These examples illustrate the fact that truncating division and floor division behave differently when the two operands are of opposite sign and their quotient is not an exact integer.

`property` $\forall(m, n)\ m$ `REM` $n = m - n(m \div n)$
`property` $\forall(m, n)\ m$ `MOD` $n = m - n\lfloor m/n \rfloor$
`property` $\forall(m, n)\ m$ `DIVREM` $n = (m \div n, m$ `REM` $n)$
`property` $\forall(m, n)\ m$ `DIVMOD` $n = (\lfloor m/n \rfloor, m$ `MOD` $n)$
`property` $\forall(m, n, k)\ (m + kn)$ `MOD` $n = m$ `MOD` n
`property` $\forall(m, n)\ m$ `REM` $n = m$ `REM` $(-n)$
`property` $\forall(m, n)\ (-m)$ `REM` $n = -(m$ `REM` $n)$

33.2.44 `opr` |(`self`, *other*: $\mathbb{Z}$): Boolean

The operator | returns *true* if the left-hand operand evenly divides the right-hand operand, and otherwise returns *false*.

`property` $\forall(m, n)\ m \mid n :=: (\lfloor m/n \rfloor = m/n)$

33.2.45 `opr ^(self, power: ℤ): ℤ`

Exponentiation of an integer to a nonnegative integer power produces an integer result. If the *power* is 0, then the result is always 1, even if the base is 0 (this definition is somewhat arbitrary but is computationally useful). An integer raised to a negative power is an integer only when the base is 1 or -1, and the result is then that exact power, of the base's type. If the base is 0, the power throws DivisionByZero; for any other base the result is no integer, and the power throws IntegerOverflow. A fraction is written with a floating-point or rational base: 2.0^{-1} is 0.5, and $(2/3)^{-1}$ is $3/2$.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.13
>
> The Working Draft gave this entry no negative power; its power on $\mathbb{Q}$ gave one, since $\mathbb{Z}$ was a subtype of $\mathbb{Q}$. The three sentences on a negative power are new.

`property` $\forall(x, y : \mathbb{N})\ x^y = x^{(\lfloor y/2 \rfloor)} x^{(\lceil y/2 \rceil)}$

33.2.46 `opr GCD(self, other: ℤ): ℤ`
33.2.47 `opr LCM(self, other: ℤ): ℤ`

The operator `GCD` computes the greatest common divisor of its two arguments. The result is always nonnegative. If either argument is 0, the result is the absolute value of the other argument.

The operator `LCM` computes the least common multiple of its two arguments. The result is always nonnegative. If either argument is 1, the result is the absolute value of the other argument. If either argument is 0, the result is 0.

The strictly positive integers form a partial order and lattice with the "evenly divides" operator $\mid$ as the partial order operator and with `GCD` and `LCM` as the lattice operators.

`property` $\forall(m, n)\ ((m\ \texttt{GCD}\ n) \mid m) \wedge ((m\ \texttt{GCD}\ n) \mid n)$
`property` $\forall(m, n)\ (m \mid (m\ \texttt{LCM}\ n)) \wedge (n \mid (m\ \texttt{LCM}\ n))$
`property` $\forall(m, n)\ (m\ \texttt{GCD}\ n) \cdot (m\ \texttt{LCM}\ n) = |m \cdot n|$

> REVISED BY THE 2026 REVIVAL SEE Section I.1.23
>
> The Working Draft of February 2011 said that the result equals the other argument where the two paragraphs above now say its absolute value, as the first sentence of each requires when the other argument is negative, and its last property equated the product of the two results with $m \cdot n$, which nonnegative results satisfy only up to sign. On a fixed-size integer type, a result that the type cannot hold throws an IntegerOverflow (Chapter 16).

33.2.48 `opr (self)! : ℤ`

The factorial operator is defined only for natural number types; it returns the product of all positive integers that are not less than the integer. If the argument is 0, the result is 1.

`property` $0! = 1$
`property` $\forall(m)\ m \geq 0 \to (m + 1)! = (m + 1)\ m!$

> REVISED BY THE 2026 REVIVAL SEE Section I.1.20
>
> The Working Draft of February 2011 stated this property as $m! = \prod_{k \leftarrow 1:m} k$, a product over a range of $\mathbb{Z}$ values, which

> Section 13.16 no longer admits; the two properties above state it by the value at 0 and a recurrence, for every natural m.

33.2.49 `opr CHOOSE(self, other: ℤ): ℤ`

The `CHOOSE` operator is defined only for natural number types; it computes the binomial coefficient $m \texttt{ CHOOSE } n = m!/(n!(m-n)!)$. If $n < 0$ or $n > m$, the result is 0.

`property` $\forall(m, n{:}\,\mathbb{Z})\ (m \texttt{ CHOOSE } n) + (m \texttt{ CHOOSE } (n+1)) = ((m+1) \texttt{ CHOOSE } (n+1))$

33.2.50 `opr <(self, other: ℤ): Boolean`
33.2.51 `opr ≤(self, other: ℤ): Boolean`
33.2.52 `opr =(self, other: ℤ): Boolean`
33.2.53 `opr ≥(self, other: ℤ): Boolean`
33.2.54 `opr >(self, other: ℤ): Boolean`

The comparison operators $<$, $\leq$, $=$, $\geq$, and $>$ allow any integer value to be compared numerically to any other integer value. Integer values are totally ordered.

For $\neq$ see Section 34.1.4.

33.2.55 `opr CMP(self, other: ℤ): TotalComparison`

The `CMP` operator compares the arguments and returns one of the three values $\mathrm{LessThan}$, $\mathrm{EqualTo}$, and $\mathrm{GreaterThan}$.

33.2.56 `opr MAX(self, other: ℤ): ℤ`
33.2.57 `opr MIN(self, other: ℤ): ℤ`
33.2.58 `opr MAXNUM(self, other: ℤ): ℤ`
33.2.59 `opr MINNUM(self, other: ℤ): ℤ`

The operators `MAX` and `MAXNUM` return whichever argument is larger in the total order defined by $<$, $\leq$, $=$, $\geq$, $>$, and `CMP`, and the operators `MIN` and `MINNUM` return whichever argument is smaller. (For all four, if the arguments are equal, then the result equals that same value.)

33.2.60 `opr |self| : ℤ`

The absolute value operator $|\ldots|$ returns the negative of this integer if the argument is less than zero, and otherwise returns the argument.

33.2.61 *signum*(`self`): $\mathbb{Z}$

The method *signum* returns -1 if this integer is less than zero, 0 if this integer is zero, and 1 if this integer is greater than zero.

33.2.62 *numerator*(`self`): $\mathbb{Z}$
33.2.63 *denominator*(`self`): $\mathbb{Z}$

For finite integers, the method *numerator* returns the argument and the method *denominator* returns 1.

33.2.64 *floor*(`self`): $\mathbb{Z}$
33.2.65 `opr` $\lfloor$`self`$\rfloor$: $\mathbb{Z}$
33.2.66 *ceiling*(`self`): $\mathbb{Z}$
33.2.67 `opr` $\lceil$`self`$\rceil$: $\mathbb{Z}$
33.2.68 *round*(`self`): $\mathbb{Z}$
33.2.69 *truncate*(`self`): $\mathbb{Z}$

These methods and operators simply return the argument when applied to an integer.

33.2.70 `opr` $\llparenthesis$`self`$\rrparenthesis$: $\mathbb{Z}$
33.2.71 `opr` $\llbracket$`self`$\rrbracket$: $\mathbb{Z}$
33.2.72 `opr` $\{\!\{$`self`$\}\!\}$: $\mathbb{Z}$
33.2.73 `opr` $\{\!\{\!\{$`self`$\}\!\}\!\}$: $\mathbb{Z}$

The hyperfloor operation $\llparenthesis x \rrparenthesis$ computes $2^{(\lfloor log_2 x \rfloor)}$ and returns the result as a natural number. If the argument is equal to 0, the result is 0. If the argument is negative, an $\mathrm{InvalidArgument}$ is thrown.

The hyperceiling operation $\llbracket x \rrbracket$ computes $2^{(\lceil log_2 x \rceil)}$ and returns the result as a natural number. If the argument is equal to 0, the result is 0. If the argument is negative, an $\mathrm{InvalidArgument}$ is thrown.

The hyperhyperfloor operation $\{\!\{ x \}\!\}$ computes $2^{(\llparenthesis log_2 x \rrparenthesis)}$ and returns the result as a natural number. If the argument is equal to 0 or 1, the result is the same as the argument. If the argument is negative, an $\mathrm{InvalidArgument}$ is thrown.

The hyperhyperceiling operation $\{\!\{\!\{ x \}\!\}\!\}$ computes $2^{(\llbracket log_2 x \rrbracket)}$ and returns the result as a natural number. If the argument is equal to 0 or 1, the result is the same as the argument. If the argument is negative, an $\mathrm{InvalidArgument}$ is thrown.

33.2.74 *shift*(`self`, k: IndexInt): $\mathbb{Z}$

The result of $\mathit{shift}(x, k)$ is $\lfloor x \cdot 2^k \rfloor$. This corresponds to what is sometimes called an "arithmetic shift" on the two's-complement representation of the integer; positive values of k shifts the bits to the left, and negative values of k shifts the bits to the right. The result is exact for a count of any size, and a result too large for the implementation to represent throws an $\mathrm{IntegerOverflow}$. On $\mathbb{Z}$, the operators `LSHIFT` and `RSHIFT` compute this shift with the count and with its negation.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.22

> The Working Draft of February 2011 did not say how large a count may be or what happens when the result cannot be represented.

33.2.75 $bit(\texttt{self}, k\colon \mathrm{IndexInt})\colon \mathrm{Bit}$

The result of $bit(x, k)$ is $\lfloor x \cdot 2^{-k} \rfloor$ `MOD` 2, as a Bit (0 or 1). If this integer is regarded as represented in binary two's-complement form, with bits numbered in "little-endian order" (least significant bit is bit number 0, least significant bit but one is bit number 1, and so on), then this functional method returns bit k of this representation.

33.2.76 `opr` $\exists(\texttt{self})\colon \mathbb{Z}$

If this integer is regarded as represented in binary two's-complement form, then this functional method returns an integer that is the bitwise complement of this integer—every bit is inverted.

`property` $\forall(x, k\colon \mathrm{IndexInt})\ bit(\exists x, k) = \neg bit(x, k)$

33.2.77 `opr` $\curlywedge(\texttt{self}, other\colon \mathbb{Z})\colon \mathbb{Z}$
33.2.78 `opr` $\curlyvee(\texttt{self}, other\colon \mathbb{Z})\colon \mathbb{Z}$
33.2.79 `opr` $\underline{\curlyvee}(\texttt{self}, other\colon \mathbb{Z})\colon \mathbb{Z}$

If the arguments are regarded as represented in binary two's-complement form, then these functional methods each return an integer that is the result of bitwise operations (*and*, *or*, or *exclusive or*, respectively) on the argument integers.

`property` $\forall(x, y, k\colon \mathrm{IndexInt})\ bit(x \curlywedge y, k) = bit(x, k) \wedge bit(y, k)$
`property` $\forall(x, y, k\colon \mathrm{IndexInt})\ bit(x \curlyvee y, k) = bit(x, k) \vee bit(y, k)$
`property` $\forall(x, y, k\colon \mathrm{IndexInt})\ bit(x \mathbin{\underline{\curlyvee}} y, k) = bit(x, k) \mathbin{\underline{\vee}} bit(y, k)$

33.2.80 $countBits(\texttt{self})\colon \mathrm{IndexInt}$

If this integer is regarded as represented in binary two's-complement form, then this functional method returns the number of 1-bits in the representation if this integer is nonnegative, but returns the number of 0-bits in the representation if this integer is negative.

`property` $\forall(x)\ countBits(0) = 0$
`property` $\forall(x)\ countBits(x) = countBits(\exists x)$
`property` $\forall(x)\ countBits(shift(x, 1)) = countBits(x)$
`property` $\forall(x)\ countBits(shift(x, 1)) + 1 = countBits(x) + 1$

33.2.81 $countFactorsOfTwo(\texttt{self})$: IndexInt

This method returns the number of factors of 2 in this integer, that is, the logarithm of the largest power of 2 that even divides this integer. If this integer is regarded as represented in binary two's-complement form, then this method returns the number of trailing 0-bits. If this integer is zero, this mehtod throws an IntegerOverflow.

property $\forall(x)\ countFactorsOfTwo(1) = 0$
property $\forall(x)\ x \neq 0 \rightarrow$: $countFactorsOfTwo(x) = countfactorsOfTwo(-x)$
property $\forall(x)\ x \neq 0 \rightarrow$: $countFactorsOfTwo(shift(x, 1)) = countfactorsOfTwo(x) + 1$
property $\forall(x)\ countFactorsOfTwo(shift(x, 1)) + 1 = 0$

33.2.82 $integerLength(\texttt{self})$: IndexInt

This method returns $\lceil \log_2(-\texttt{self}) \rceil$ if self is negative, but returns $\lceil \log_2(\texttt{self} + 1) \rceil$ if self is nonnegative. Suppose that this integer is regarded as represented in binary two's-complement form, and this method returns the value k; then k is the smallest integer such that $k + 1$ bits suffice to represent this integer in binary two's-complement form. Moreover, if this integer is nonnegative, then k is the smallest integer such that k bits suffice to represent this integer in unsigned binary form.

property $\forall(x)\ integerLength(0) = 0$
property $\forall(x)\ x \geq 0 \rightarrow integerLength(shift(x, 1)) = integerLength(shift(x, 1) + 1) = integerLength(x) + 1$
property $\forall(x)\ integerLength(x) = integerLength(\neg x)$

33.2.83 $lowBits(\texttt{self}, k$: IndexInt$)$: $\mathbb{Z}$

This method returns the value of this integer modulo 2^k. If this integer is regarded as represented in binary two's-complement form, then this method returns an integer whose k least significant bits are equal to the correpsonding bits of this integer, and whose other bits are all 0.

property $\forall(x, k : \text{IndexInt})\ lowBits(x, k) = x\ \texttt{MOD}\ 2^k$

33.2.84 $even(\texttt{self})$: Boolean
33.2.85 $odd(\texttt{self})$: Boolean

The method *even* returns *true*, and the method *odd* returns *false*, if this integer is evenly divisible by two. The method *even* returns *false*, and the method *odd* returns *true*, if this integer is not evenly divisible by two.

property $\forall(a)\ even\ a \leftrightarrow 2 \mid a$
property $\forall(a)\ odd\ a \leftrightarrow \neg even\ a$

33.2.86 $prime(\texttt{self}) : \text{Boolean}$

The method $prime$ is defined only for natural number types; it returns $true$ if this integer is a prime number, and otherwise returns $false$. It returns $false$ if this integer is 0 or 1.

$\forall(a, b)\ (a > 1) \wedge (a \mid b) \rightarrow : \neg prime\ b$

33.2.87 $realpart(\texttt{self}) : \mathbb{Z}$

The method $realpart$ for an integer simply returns its argument.

33.2.88 $imagpart(\texttt{self}) : \mathbb{Z}$

The method $imagpart$ for an integer simply returns zero.

Future versions of this specification will include trait declarations definitions for the other numeric quantities.

Chapter 34

Negated Relational Operators

34.1 Negated Equivalence Operators

34.1.1 $\texttt{opr} \not\equiv(x\colon \mathrm{Any}, y\colon \mathrm{Any})\colon \mathrm{Boolean}$

The infix operator $\not\equiv$ applies $\neg$ to the result of $\equiv$ on the same operands.

34.1.2 $\texttt{opr} \not\equiv[\![T\ \texttt{extends}\ \mathrm{BinaryPredicate}[\![T, \equiv]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.1.3 $\texttt{opr} \not\equiv[\![T\ \texttt{extends}\ \mathrm{BinaryIntervalPredicate}[\![T, \equiv]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\equiv$ applies $\neg$ to the result of $\equiv$ on the same operands.

34.1.4 $\texttt{opr} \neq[\![T\ \texttt{extends}\ \mathrm{BinaryPredicate}[\![T, =]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.1.5 $\texttt{opr} \neq[\![T\ \texttt{extends}\ \mathrm{BinaryIntervalPredicate}[\![T, =]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\neq$ applies $\neg$ to the result of $=$ on the same operands.

34.1.6 $\texttt{opr} \not\simeq[\![T\ \texttt{extends}\ \mathrm{BinaryPredicate}[\![T, \simeq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.1.7 $\texttt{opr} \not\simeq[\![T\ \texttt{extends}\ \mathrm{BinaryIntervalPredicate}[\![T, \simeq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\simeq$ applies $\neg$ to the result of $\simeq$ on the same operands.

34.1.8 $\texttt{opr} \not\approx[\![T\ \texttt{extends}\ \mathrm{BinaryPredicate}[\![T, \approx]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$

The infix operator $\not\approx$ applies $\neg$ to the result of $\approx$ on the same operands.

34.1.9 `opr` $\not\approx[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \approx]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

34.2 Negated Plain Comparison Operators

34.2.1 `opr` $\not<[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, <]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.2.2 `opr` $\not<[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, <]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not<$ applies $\neg$ to the result of $<$ on the same operands.

34.2.3 `opr` $\not\leq[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \leq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.2.4 `opr` $\not\leq[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \leq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\leq$ applies $\neg$ to the result of $\leq$ on the same operands.

34.2.5 `opr` $\not\geq[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \geq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.2.6 `opr` $\not\geq[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \geq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\geq$ applies $\neg$ to the result of $\geq$ on the same operands.

34.2.7 `opr` $\not>[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, >]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.2.8 `opr` $\not>[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, >]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not>$ applies $\neg$ to the result of $>$ on the same operands.

34.3 Negated Set Comparison Operators

34.3.1 `opr` $\not\subset[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \subset]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.3.2 `opr` $\not\subset[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \subset]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\subset$ applies $\neg$ to the result of $\subset$ on the same operands.

34.3.3 `opr` $\not\subseteq[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \subseteq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.3.4 `opr` $\not\subseteq[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \subseteq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\subseteq$ applies $\neg$ to the result of $\subseteq$ on the same operands.

34.3.5 `opr` $\not\supseteq[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \supseteq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.3.6 `opr` $\not\supseteq[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \supseteq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\supseteq$ applies $\neg$ to the result of $\supseteq$ on the same operands.

34.3.7 `opr` $\not\supset[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \supset]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.3.8 `opr` $\not\supset[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \supset]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\supset$ applies $\neg$ to the result of $\supset$ on the same operands.

34.4 Negated Square Comparison Operators

34.4.1 `opr` $\not\sqsubset[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \sqsubset]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.4.2 `opr` $\not\sqsubset[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \sqsubset]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\sqsubset$ applies $\neg$ to the result of $\sqsubset$ on the same operands.

34.4.3 `opr` $\not\sqsubseteq[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \sqsubseteq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.4.4 `opr` $\not\sqsubseteq[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \sqsubseteq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\sqsubseteq$ applies $\neg$ to the result of $\sqsubseteq$ on the same operands.

34.4.5 `opr` $\not\sqsupseteq[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \sqsupseteq]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.4.6 `opr` $\not\sqsupseteq[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \sqsupseteq]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\sqsupseteq$ applies $\neg$ to the result of $\sqsupseteq$ on the same operands.

34.4.7 `opr` $\not\sqsupset[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \sqsupset]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.4.8 `opr` $\not\sqsupset[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \sqsupset]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\sqsupset$ applies $\neg$ to the result of $\sqsupset$ on the same operands

34.5 Negated Curly Comparison Operators

34.5.1 `opr` $\not\prec[\![T$ `extends` $\mathrm{BinaryPredicate}[\![T, \prec]\!]](x\colon T, y\colon T)\colon \mathrm{Boolean}$
34.5.2 `opr` $\not\prec[\![T$ `extends` $\mathrm{BinaryIntervalPredicate}[\![T, \prec]\!]](x\colon T, y\colon T)\colon \mathrm{BooleanInterval}$

The infix operator $\not\prec$ applies $\neg$ to the result of $\prec$ on the same operands.

34.5.3 `opr` $\not\preceq[\![T$ `extends` $\text{BinaryPredicate}[\![T, \preccurlyeq]\!]](x\colon T, y\colon T)\colon \text{Boolean}$
34.5.4 `opr` $\not\preceq[\![T$ `extends` $\text{BinaryIntervalPredicate}[\![T, \preccurlyeq]\!]](x\colon T, y\colon T)\colon \text{BooleanInterval}$

The infix operator $\not\preceq$ applies $\neg$ to the result of $\preceq$ on the same operands.

34.5.5 `opr` $\not\succeq[\![T$ `extends` $\text{BinaryPredicate}[\![T, \succcurlyeq]\!]](x\colon T, y\colon T)\colon \text{Boolean}$
34.5.6 `opr` $\not\succeq[\![T$ `extends` $\text{BinaryIntervalPredicate}[\![T, \succcurlyeq]\!]](x\colon T, y\colon T)\colon \text{BooleanInterval}$

The infix operator $\not\succeq$ applies $\neg$ to the result of $\succeq$ on the same operands.

34.5.7 `opr` $\not\succ[\![T$ `extends` $\text{BinaryPredicate}[\![T, \succ]\!]](x\colon T, y\colon T)\colon \text{Boolean}$
34.5.8 `opr` $\not\succ[\![T$ `extends` $\text{BinaryIntervalPredicate}[\![T, \succ]\!]](x\colon T, y\colon T)\colon \text{BooleanInterval}$

The infix operator $\not\succ$ applies $\neg$ to the result of $\succ$ on the same operands.

34.6 Negated Miscellaneous Relational Operators

34.6.1 `opr` $\notin[\![T$ `extends` $\text{BinaryPredicate}[\![T, \in]\!]](x\colon T, y\colon T)\colon \text{Boolean}$
34.6.2 `opr` $\notin[\![T$ `extends` $\text{BinaryIntervalPredicate}[\![T, \in]\!]](x\colon T, y\colon T)\colon \text{BooleanInterval}$

The infix operator $\notin$ applies $\neg$ to the result of $\in$ on the same operands.

34.6.3 `opr` `NOTNI`$[\![T$ `extends` $\text{BinaryPredicate}[\![T, \ni]\!]](x\colon T, y\colon T)\colon \text{Boolean}$
34.6.4 `opr` `NOTNI`$[\![T$ `extends` $\text{BinaryIntervalPredicate}[\![T, \ni]\!]](x\colon T, y\colon T)\colon \text{BooleanInterval}$

The infix operator $\not\ni$ applies $\neg$ to the result of $\ni$ on the same operands.

34.7 Other Negated Operators

34.7.1 `opr` $\nparallel[\![T$ `extends` $\text{BinaryPredicate}[\![T, \|]\!]](x\colon T, y\colon T)\colon \text{Boolean}$
34.7.2 `opr` $\nparallel[\![T$ `extends` $\text{BinaryIntervalPredicate}[\![T, \|]\!]](x\colon T, y\colon T)\colon \text{BooleanInterval}$

The infix operator $\nparallel$ applies $\neg$ to the result of $\|$ on the same operands.

Chapter 35

Exceptions

35.1 The Trait Fortress.Standard.Exception

The trait Exception is a single root of the exception hierarchy; every exception in Fortress has trait Exception. An exception is either a CheckedException or an UncheckException. Every exception has optional fields: a message and a chained exception. These fields are default to Nothing⟦String⟧ and Nothing⟦Exception⟧ where an optional value v of type Maybe⟦T⟧ is either Nothing⟦T⟧ or Just(v) as declared in Section 39.2.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.7
>
> The Working Draft of February 2011 wrote both defaults as a bare Nothing.

```
trait Exception comprises { CheckedException, UncheckedException }
  settable message: Maybe⟦String⟧
  settable chain: Maybe⟦Exception⟧
end
```

35.1.1 settable *message*: Maybe⟦String⟧

When an exception is thrown, its *message* may be set.

35.1.2 settable *chain*: Maybe⟦Exception⟧

When an exception is thrown, its *chain* may be set to the exception thrown immediately before this exception.

35.2 The Trait Fortress.Standard.CheckedException

```
trait CheckedException
  extends { Exception }
  excludes { UncheckedException }
end
```

35.3 The Trait Fortress.Standard.UncheckedException

```
trait UncheckedException
  extends { Exception }
  excludes { CheckedException }
end
```

Chapter 36

Threads

36.1 The Trait Fortress.Standard.Thread

Every thread in Fortress has trait Thread.

```
trait Thread⟦T extends Any⟧
   val(): T
   wait(): ()
   ready(): Boolean
   stop(): () throws Stopped
end
```

36.1.1 *val*(): *T*

The *val* method returns the value computed by the expression of the thread. If the thread has not yet completed execution, the invocation of *val* blocks until it has done so.

36.1.2 *wait*(): ()

The *wait* method waits for a thread to complete, but does not return a value.

36.1.3 *ready*(): Boolean

The *ready* method returns *true* if a thread has completed, and returns *false* otherwise.

36.1.4 *stop*(): () throws Stopped

The *stop* method attempts to terminate a thread.

Chapter 37

Dimensions and Units

37.1 Fortress.SIUnits

(∗ Reference: http://physics.nist.gov/cuu/Units/index.html ∗)

(∗ SI base units ∗)

`dim` Length `SI_unit` meter meters m
`dim` Mass `default` kilogram; `SI_unit` gram grams g: Mass
`dim` Time `SI_unit` second seconds s
`dim` ElectricCurrent `SI_unit` ampere amperes A
`dim` Temperature `SI_unit` kelvin kelvins K
`dim` AmountOfSubstance `SI_unit` mole moles mol
`dim` LuminousIntensity `SI_unit` candela candelas cd

(∗ SI derived units with special names and symbols ∗)

`dim` Angle = Unity `SI_unit` radian radians rad
`dim` SolidAngle = Unity `SI_unit` steradian steradians sr
`dim` Frequency = 1/Time `SI_unit` hertz Hz
`dim` Force = Mass Acceleration `SI_unit` newton newtons N
`dim` Pressure = Force/Area `SI_unit` pascal pascals Pa
`dim` Energy = Length Force `SI_unit` joule joules J
`dim` Power = Energy/Time `SI_unit` watt watts W
`dim` ElectricCharge = ElectricCurrent Time `SI_unit` coulomb coulombs C
`dim` ElectricPotential = Power/Current `SI_unit` volt volts V
`dim` Capacitance = ElectricCharge/Voltage `SI_unit` farad farads F
`dim` Resistance = ElectricPotential/Current `SI_unit` ohm ohms Ω
`dim` Conductance = 1/Resistance `SI_unit` siemens S
`dim` MagneticFlux = Voltage Time `SI_unit` weber webers Wb
`dim` MagneticFluxDensity = MagneticFlux/Area `SI_unit` tesla teslas T
`dim` Inductance = MagneticFlux/Current `SI_unit` henry henries H
`dim` LuminousFlux = LuminousIntensity SolidAngle `SI_unit` lumen lumens lm
`dim` Illuminance = LuminousFlux/Area `SI_unit` lux lx
`dim` RadionuclideActivity = 1/Time `SI_unit` becquerel becquerels Bq
`dim` AbsorbedDose = Energy/Mass `SI_unit` gray grays Gy
`dim` CatalyticActivity = AmountOfSubstance/Time `SI_unit` katal katals kat

(∗ Other derived dimensions ∗)

```
dim Area = Length^2
dim Volume = Length^3
dim Velocity = Length/Time
dim Speed = Velocity
dim Acceleration = Velocity/Time
dim Momentum = Mass Velocity
dim AngularVelocity = Angle/Second
dim AngularAcceleration = Angle/Second^2
dim WaveNumber = 1/Length
dim MassDensity = Mass/Volume
dim CurrentDensity = Current/Area
dim MagneticFieldStrength = Current/Length
dim Luminance = LuminousIntensity/Area
dim Work = Energy
dim Action = Energy Time
dim MomentOfForce = Force Length
dim Torque = MomentOfForce
dim MomentOfInertia = Mass Length^2
dim Voltage = ElectricPotential
dim Conductivity = Conductance/Length
dim Resistivity = 1/Conductivity
dim Impedance = Resistance
dim Permittivity = Capacitance/Length
dim Permeability = Inductance/Length
dim Irradiance = Power/Area
dim RadiantIntensity = Power/SolidAngle
dim Radiance = Power/Area SolidAngle
dim AbsorbedDoseRate = AbsorbedDose/Time
dim CatalyticConcentration = CatalyticActivity/Volume
dim HeatCapacity = Energy/Temperature
dim Entropy = Energy/Temperature
dim DynamicViscosity = Pressure Time
dim SpecificHeatCapacity = Energy/Mass Temperature
dim SpecificEntropy = Energy/Mass Temperature
dim SpecificEnergy = Energy/Mass
dim ThermalConductivity = Energy/Length Temperature
dim EnergyDensity = Energy/Volume
dim ElectricFieldStrength = ElectricPotential/Length
dim ElectricChargeDensity = ElectricCharge/Volume
dim ElectricFlux = ElectricCharge
dim ElectricFluxDensity = ElectricCharge/Area
dim MolarEnergy = Energy/AmountOfSubstance
dim MolarHeatCapacity = Energy/AmountOfSubstance Temperature
dim MolarEntropy = Energy/AmountOfSubstance Temperature
dim RadiationExposure = ElectricCharge/Mass

(* Units outside the SI that are accepted for use with the SI *)

unit minute minutes min: Time
unit hour hours h: Time
unit day days d: Time
unit degreeOfAngle degrees: Angle
unit minuteOfAngle minutesOfAngle: Angle
```

```
unit secondOfAngle secondsOfAngle: Angle
SI_unit metricTon metricTons tonne tonnes t: Mass
SI_unit liter liters L: Volume
```

37.2 Fortress.EnglishUnits

```
import { Length, Area, Volume, Time, Mass, millimeters, liters, grams }
       from Fortress.SIUnits

unit inch inches: Length
unit foot feet: Length
unit yard yards: Length
unit mile miles: Length
unit rod rods: Length
unit furlong furlongs: Length

unit surveyFoot surveyFeet: Length
unit surveyMile surveyMiles: Length

unit nauticalMile nauticalMiles: Length
unit knot knots: Speed

unit week weeks: Time
unit fortnight fortnights: Time
unit microfortnight microfortnights: Time

unit gallon gallons: Volume
unit fluidQuart fluidQuarts: Volume
unit fluidPint fluidPints: Volume
unit fluidCup fluidCups: Volume
unit  fluidOunce fluidOunces: Volume
unit fluidDram fluidDrams: Volume
unit minim minims: Volume

unit traditionalTablespoon traditionalTablespoons: Volume
unit traditionalTeaspoon traditionalTeaspoons: Volume
unit federalTablespoon federalTablespoons: Volume
unit federalTeaspoon federalTeaspoons: Volume

unit dryPint dryPints: Volume
unit dryQuart dryQuarts: Volume
unit peck pecks: Volume
unit bushel bushels: Volume

unit acre: Area

unit imperialGallon: Volume
unit imperialQuart: Volume
unit imperialPint: Volume
unit imperialGill: Volume
unit imperialFluidOunce: Volume
unit imperialFluidDrachm: Volume
unit imperialFluidDRam : Volume
unit imperialFluidScruple: Volume
unit imperialMinim: Volume

unit pound pounds lb lbs: Mass
```

```
unit ounce ounces oz: Mass
unit grain grains: Mass
unit troyPound troyPounds: Mass
unit troyOunce troyOunces: Mass
```

37.3 Fortress.InformationUnits

```
dim Information unit bit bits
unit byte bytes: Information
```

Chapter 38

Tests

38.1 The Object Fortress.Standard.TestSuite

An instance of the object $\mathrm{TestSuite}$ contains a set of test functions that can all be called by invoking the method run:

```
test object TestSuite(testFunctions = {})
   add(f: () → ()): ()
   run(): ()
end
```

38.1.1 $add(f\colon () \to ())\colon ()$

The add method adds a given test function to the $testFunctions$ field of this object.

38.1.2 $run()\colon ()$

The run method calls each test function in the $testFunctions$ field of this object. Note that all tests in a $\mathrm{TestSuite}$ are run in parallel.

38.2 Test Functions

38.2.1 `test` $fail(message\colon \mathrm{String})\colon ()$

The helper function $fail$ displays the error message provided and terminates execution of the enclosing test.

Chapter 39

Convenience Functions and Types

39.1 Convenience Functions

39.1.1 $tuple[\![T \texttt{extends} \operatorname{Tuple}]\!](x{:}\,T){:}\,T$

The function $tuple$ returns its argument as a tuple expression.

39.1.2 $\operatorname{coerce}[\![T]\!](x{:}\,T){:}\,T$

The function coerce returns its argument as the given type.

39.2 Convenience Types

An optional value v of type $\mathrm{Maybe}[\![T]\!]$ is either $\mathrm{Nothing}[\![T]\!]$ or $\mathrm{Just}(v)$ declared as follows:

```
(∗ Optional Values ∗)
trait Maybe[[T]] comprises {Nothing[[T]], Just[[T]]}
  isNothing: Boolean
end
value object Nothing[[T]] extends Maybe[[T]]
end
object Just[[T]](just: T) extends Maybe[[T]]
end
```

The empty case is one object for each type T, and each use writes the type, as in $\mathrm{Nothing}[\![\mathrm{String}]\!]$. The following declaration of a single $\mathrm{Nothing}$ for every instantiation of Maybe is not allowed:

```
(∗ Not allowed! ∗)
object Nothing extends Maybe[[T]] excludes Just[[T]] where {T extends Object}
end
```

It would make $\mathrm{Nothing}$ a $\mathrm{Maybe}[\![T]\!]$ for every T at once, which instantiation exclusion forbids (Section 12.6); and an object declaration has no `excludes` clause (Section 11.1), an object trait type already excluding every type that is not its supertype (Section 6.3.1).

> REVISED BY THE 2026 REVIVAL SEE Section I.1.7
>
> The Working Draft of February 2011 declared $\mathrm{Nothing}$ with the declaration marked "Not allowed". The declaration of $\mathrm{Nothing}[\![T]\!]$ is the library's, whose own source says it "will become a non-parametric singleton when we get where clauses working"; instantiation exclusion closes that road.

Part VI

Fortress APIs and Documentation for Library Writers

> DRAFT NOTE
> The APIs in this part may not be correct; they are not tested.

Chapter 40

Algebraic Constraints

The traits in this component are used to describe properties of traits and their associated operators. These traits provide very few concrete methods, but specify abstract methods and `property` declarations.

40.1 Predicates and Equivalence Relations

A *predicate* is an operator that produces a boolean result. A binary predicate may be identified with a mathematical relation, where the predicate returns $true$ in exactly those cases that its two operands satisfy the relation; therefore we use the mathematical terminology usually associated with relations to describe the properties of binary predicates.

```
trait UnaryPredicate⟦T extends UnaryPredicate⟦T, ∼⟧, opr ∼⟧
    extends Any
  abstract opr ∼(self): Boolean
end
```

A unary predicate is a prefix operator that takes one argument and returns a boolean value ($true$ or $false$). Note that the symbol $\sim$ is a static parameter, used here as a "variable" name for an operator.

```
trait BinaryPredicate⟦T extends BinaryPredicate⟦T, ∼⟧, opr ∼⟧
    extends Any
  abstract opr ∼(self, other: T): Boolean
end
```

A binary predicate is an infix operator that takes two arguments and returns a boolean value. Thus, for example, any trait T that extends $\text{BinaryPredicate}[\![T, \sqsubset]\!]$ necessarily has an infix method for the operator $\sqsubset$, and that operator returns a boolean value.

```
trait Reflexive⟦T extends Reflexive⟦T, ∼⟧, opr ∼⟧
    extends { BinaryPredicate⟦T, ∼⟧ }
  property ∀(a: T) (a ∼ a)
end
```

A *reflexive* predicate always returns $true$ when its operands are the same. Because this fact is expressed as a `property` declaration, the behavior of such an operator can be checked for correctness by unit testing.

```
trait Irreflexive⟦T extends Irreflexive⟦T, ∼⟧, opr ∼⟧
```

```
      extends { BinaryPredicate⟦T, ∼⟧ }
   property ∀(a: T) ¬(a ∼ a)
end
```

An *irreflexive* predicate always returns $false$ when its operands are the same. (Note that it is possible for a predicate to be neither reflexive nor irreflexive.)

```
trait Symmetric⟦T extends Symmetric⟦T, ∼⟧, opr ∼⟧
      extends { BinaryPredicate⟦T, ∼⟧ }
   property ∀(a: T, b: T) (a ∼ b) ↔ (b ∼ a)
end
```

A *symmetric* predicate doesn't care in which order its arguments are presented; the result is the same either way.

```
trait Transitive⟦T extends Transitive⟦T, ∼⟧, opr ∼⟧
      extends { BinaryPredicate⟦T, ∼⟧ }
   property ∀(a: T, b: T, c: T) ((a ∼ b) ∧ (b ∼ c)) → (a ∼ c)
end
```

A *transitive* predicate has the property that if a is related to b and b is related to c, then a is related to c.

```
trait EquivalenceRelation⟦T extends EquivalenceRelation⟦T, ∼⟧, opr ∼⟧
       extends { Reflexive⟦T, ∼⟧, Symmetric⟦T, ∼⟧, Transitive⟦T, ∼⟧ }
end
```

An *equivalence relation* is any predicate that is reflexive, symmetric, and transitive. You can think of an equivalence relation as describing a way to separate a set of items into categories, such that each item belongs to exactly one category; the predicate is $true$ of two items if and only if they are in the same category.

```
trait IdentityEquality⟦T extends IdentityEquality⟦T⟧⟧
      extends { EquivalenceRelation⟦T, =⟧ }
   opr =(self, other: T): Boolean
   property ∀(a: T, b: T) (self = other) ↔ (self ≡ other)
end
```

This trait states that $=$ is an equivalence relation over instances of the type T.

```
trait UnaryPredicateSubstitutionLaws⟦T extends UnaryPredicateSubstitutionLaws⟦T, ∼, ≃⟧,
                                     opr ∼, opr ≃⟧
      extends { UnaryPredicate⟦T, ∼⟧, BinaryPredicate⟦T, ≃⟧ }
   property ∀(a: T, a′: T) (a ≃ a′) →: ((∼ a) ↔ (∼ a′))
end
```

This handy trait states that the unary predicate $\sim$ is consistent under substitutions described by the relation $\simeq$ (which is typically, but not always, an equivalence relation); that is, the result produced by $\sim$ is unchanged if its argument is replaced by some other value that is equivalent.

```
trait BinaryPredicateSubstitutionLaws⟦T extends BinaryPredicateSubstitutionLaws⟦T, ∼, ≃⟧,
                                     opr ∼, opr ≃⟧
      extends { BinaryPredicate⟦T, ∼⟧, BinaryPredicate⟦T, ≃⟧ }
   property ∀(a: T, a′: T) (a ≃ a′) →: ∀(b: T) (a ∼ b) ↔ (a′ ∼ b)
   property ∀(b: T, b′: T) (b ≃ b′) →: ∀(a: T) (a ∼ b) ↔ (a ∼ b′)
end
```

This equally handy trait states that the binary predicate $\sim$ is consistent under substitutions described by the relation $\simeq$ (which is typically, but not always, an equivalence relation); that is, the result produced by $\sim$ is unchanged if either argument is replaced by some other value that is equivalent. (It is then easy to prove that the result is unchanged even when *both* arguments are replaced by equivalent values.)

40.2 Partial and Total Orders

```
trait Antisymmetric⟦T extends AntiSymmetric⟦T, ∼⟧, opr ∼⟧
      extends { BinaryPredicate⟦T, ∼⟧, EquivalenceRelation⟦T, =⟧,
                BinaryPredicateSubstitutionLaws⟦T, ∼, =⟧ }
   property ∀(a: T, b: T) ((a ∼ b) ∧ (b ∼ a)) :→: (a = b)
end
```

A binary predicate $\sim$ is *antisymmetric* if and only if two conditions are true: (a) $\sim$ is consistent under substitutions described by the predicate $=$, which must be an equivalence relation; (b) whenever $\sim$ holds true for a pair of arguments and for those same arguments in reverse order, those arguments are equivalent as specified by $=$.

```
trait PartialOrder⟦T extends PartialOrder⟦T, ≼⟧, opr ≼⟧
      extends { Reflexive⟦T, ≼⟧, Antisymmetric⟦T, ≼⟧, Transitive⟦T, ≼⟧ }
end
```

A *partial order* is a binary predicate that is reflexive, antisymmetric, and transitive.

```
trait StrictPartialOrder⟦T extends StrictPartialOrder⟦T, ≺⟧, opr ≺⟧
      extends { Irreflexive⟦T, ≺⟧, Antisymmetric⟦T, ≺⟧, Transitive⟦T, ≺⟧ }
end
```

A *strict partial order* is a binary predicate that is irreflexive, antisymmetric, and transitive. (Thus it differs from an ordinary partial order in being irreflexive rather than reflexive.)

It is easy to prove that, because $\prec$ is irreflexive and antisymmetric, that $a \prec b$ and $a = b$ cannot both be true. (If they were both true, then because antisymmetry requires that $\prec$ obey substitution laws, $a \prec a$ would be true—but that contradicts the fact that $\prec$ is irreflexive.) It is then easy to prove that $a \prec b$ and $b \prec a$ cannot both be true. (If they were both true, then by antisymmetry $a = b$ must be true—but $a \prec b$ and $a = b$ cannot both be true.)

```
trait TotalOrder⟦T extends TotalOrder⟦T, ≼⟧, opr ≼⟧
      extends { PartialOrder⟦T, ≼⟧ }
   property ∀(a: T, b: T) (a ≼ b) ∨ (b ≼ a)
end
```

A *total order* is a partial order in which every pair of operands must be related by the predicate $\preccurlyeq$ in one order or the other; thus no two values are ever unordered with respect to each other.

```
trait StrictTotalOrder⟦T extends StrictTotalOrder⟦T, ≺⟧, opr ≺⟧
      extends { StrictPartialOrder⟦T, ≺⟧ }
   property ∀(a: T, b: T) (a ≺ b) ∨ (b ≺ a) ∨ (a = b)
end
```

A *strict total order* is a strict partial order in which every pair of operands must be related, either by equality $=$ or by the predicate $\prec$ in one order or the other; thus no two values are ever unordered with respect to each other.

For a strict total order *at least* one of $a \prec b$ and $a = b$ and $b \prec a$ is true; but a strict total order is also a strict partial order, for which *at most* one of $a \prec b$ and $a = b$ and $b \prec a$ is true. Therefore a strict total order obeys the *law of trichotomy*: for any a and b, *exactly* one of $a \prec b$ and $a = b$ and $b \prec a$ is true.

```
trait PartialOrderOperators⟦T extends PartialOrderOperators⟦T, ≺, ≼, ≽, ≻, CMP⟧,
                              opr ≺, opr ≼, opr ≽, opr ≻, opr CMP⟧
    extends { StrictPartialOrder⟦T, ≺⟧, PartialOrder⟦T, ≼⟧,
              PartialOrder⟦T, ≽⟧, StrictPartialOrder⟦T, ≻⟧ }
  abstract opr CMP(self, other: T): Comparison
  property ∀(a: T, b: T) ((a CMP b) ≡ LessThan) ↔ ((b CMP a) ≡ GreaterThan)
  property ∀(a: T, b: T) ((a CMP b) ≡ EqualTo) ↔ ((b CMP a) ≡ EqualTo)
  property ∀(a: T, b: T) ((a CMP b) ≡ Unordered) ↔ ((b CMP a) ≡ Unordered)
  property ∀(a: T, b: T) (a ≺ b) ↔ ((a CMP b) ≡ LessThan)
  property ∀(a: T, b: T) (a ≼ b) ↔ ((a CMP b) ≡ LessThan ∨ (a CMP b) ≡ EqualTo)
  property ∀(a: T, b: T) (a = b) ↔ ((a CMP b) ≡ EqualTo)
  property ∀(a: T, b: T) (a ≽ b) ↔ ((a CMP b) ≡ GreaterThan ∨ (a CMP b) ≡ EqualTo)
  property ∀(a: T, b: T) (a ≻ b) ↔ ((a CMP b) ≡ GreaterThan)
end
```

For practical programming, we assume that partial order operators come in groups of five: a "less than" predicate, a "less than or equal to" predicate, a "greater than or equal to" predicate, a "greater than" predicate, and a "comparison" operator that returns one of the four values LessThan, EqualTo, GreaterThan, and Unordered. The trait PartialOrderOperators declares such a set of five operators and describes the necessary algebraic constraints among them and the equality predicate $=$.

```
trait TotalOrderOperators⟦T extends TotalOrderOperators⟦T, ≺, ≼, ≽, ≻, CMP⟧,
                           opr ≺, opr ≼, opr ≽, opr ≻, opr CMP⟧
    extends { PartialOrderOperators⟦T, ≺, ≼, ≽, ≻, CMP⟧,
              StrictTotalOrder⟦T, ≺⟧, TotalOrder⟦T, ≼⟧,
              TotalOrder⟦T, ≽⟧, StrictTotalOrder⟦T, ≻⟧ }
  abstract opr CMP(self, other: T): TotalComparison
end
```

Total order operators likewise come in groups of five: a "less than" predicate, a "less than or equal to" predicate, a "greater than or equal to" predicate, a "greater than" predicate, and a "comparison" operator that returns one of the three values LessThan, EqualTo, and GreaterThan. The trait TotalOrderOperators declares such a set of five operators and describes the necessary algebraic constraints among them and the equality predicate $=$. For a total order, the comparison operator CMP never returns Unordered.

```
trait PartialOrderBasedOnLE⟦T extends PartialOrderBasedOnLE⟦T, ≺, ≼, ≽, ≻, CMP⟧,
                             opr ≺, opr ≼, opr ≽, opr ≻, opr CMP⟧
    extends { PartialOrderOperators⟦T, ≺, ≼, ≽, ≻, CMP⟧ }
  opr CMP(self, other: T): Comparison
  opr ≺(self, other: T): Boolean
  opr =(self, other: T): Boolean
  opr ≽(self, other: T): Boolean
  opr ≻(self, other: T): Boolean
end
```

The trait PartialOrderBasedOnLE implements a partial order by assuming that the "less than or equal to" predicate is already defined and providing definitions for the comparison operator, the "less than" predicate, the equality predicate, the "greater than or equal to" predicate, and the "greater than" predicate in terms of the "less than or equal to" predicate.

```
trait TotalOrderBasedOnLE⟦T extends TotalOrderBasedOnLE⟦T, ≺, ≼, SUCCEQq, ≻, CMP⟧,
                           opr ≺, opr ≼, opr ≽, opr ≻, opr CMP⟧
    extends { TotalOrderOperators⟦T, ≺, ≼, ≽, ≻, CMP⟧ }
  opr CMP(self, other: T): TotalComparison
  opr ≺(self, other: T): Boolean
```

```
  opr =(self, other: T): Boolean
  opr ≽(self, other: T): Boolean
  opr ≻(self, other: T): Boolean
end
```

The trait TotalOrderBasedOnLE implements a total order by assuming that the "less than or equal to" predicate is already defined and providing definitions for the comparison operator, the "less than" predicate, the equality predicate, the "greater than or equal to" predicate, and the "greater than" predicate in terms of the "less than or equal to" predicate.

```
trait TotalOrderBasedOnLT⟦T extends TotalOrderBasedOnLT⟦T, ≺, ≼, ≽, ≻, CMP⟧,
                                    opr ≺, opr ≼, opr ≽, opr ≻, opr CMP⟧
    extends { TotalOrderOperators⟦T, ≺, ≼, ≽, ≻, CMP⟧ }
  opr CMP(self, other: T): TotalComparison
  opr ≼(self, other: T): Boolean
  opr =(self, other: T): Boolean
  opr ≽(self, other: T): Boolean
  opr ≻(self, other: T): Boolean
end
```

The trait TotalOrderBasedOnLT implements a total order by assuming that the "less than" predicate is already defined and providing definitions for the comparison operator, the "less than or equal to" predicate, the equality predicate, the "greater than or equal to" predicate, and the "greater than" predicate in terms of the "less than" predicate.

```
value object MaximalElement⟦opr ≼⟧ end
trait HasMaximalElement⟦T extends HasMaximalElement⟦T, ≼⟧, opr ≼⟧
    extends { PartialOrder⟦T, ≼⟧ }
    where { T coerces MaximalElement⟦≼⟧ }
  property ∀(a: T) a ≼ MaximalElement⟦≼⟧
end
```

The HasMaximalElement trait specifies that a partial order has a maximal element, that is, one to which every other element is related by the ordering predicate $\preccurlyeq$. The maximal element may be identified by coercing the object named MaximalElement⟦≼⟧ to type T.

```
value object MinimalElement⟦opr ≼⟧ end
trait HasMinimalElement⟦T extends HasMinimalElement⟦T, ≼⟧, opr ≼⟧
    extends { PartialOrder⟦T, ≼⟧ }
    where { T coerces MinimalElement⟦≼⟧ }
  property ∀(a: T) MinimalElement⟦≼⟧ ≼ a
end
```

The HasMinimalElement trait specifies that a partial order has a minimal element, that is, one to which every other element relates by the ordering predicate $\preccurlyeq$. The minimal element may be identified by coercing the object named MinimalElement⟦≼⟧ to type T.

```
trait LexicographicPartialOrder⟦T extends LexicographicPartialOrder⟦T, ⊏, ⊑, ≡, ⊒, ⊐, TCMP,
                                                                    X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
                                  opr ⊏, opr ⊑, opr ≡, opr ⊒, opr ⊐, opr TCMP,
                                  X extends TotalOrderOperators⟦X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
                                  opr ≺, opr ≼, opr ≃, opr ≽, opr ≻, opr XCMP⟧
    extends { PartialOrderOperators⟦LexicographicPartialOrder⟦T, ⊏, ⊑, ≡, ⊒, ⊐, TCMP,
                                                              X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
                                      ⊏, ⊑, ⊒, ⊐, TCMP⟧ }
```

```
        where { T extends ZeroBasedIndexing⟦T, X⟧ }
  property ∀(a: T, b: T) (a TCMP b) = ( BIG LEXICO (a_i XCMP b_i)) LEXICO (|a| CMP |b|)
                                      i←a.indices∩b.indices
end
```

The Comparison trait provides an associative operator `LEXICO` whose principal use is in defining lexicographic order on sequences of elements, which may be partial or total depending on whether the ordering of the elements is partial or total.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.3
>
> The Fortress standard libraries declare only the total lexicographic order of lists: a list is ordered only where its element type is totally ordered (Section 12.6), and no partial lexicographic order of lists is declared, so on the compiled type checker a list of floating-point numbers has no order, while its $=$ still compares element by element. The interpreter still orders every list (row 774 of the revival's gap ledger).

A set of lexicographic partial order operators $\sqsubset, \sqsubseteq, \equiv, \sqsupseteq, \sqsupset$, `TCMP` may be defined in terms of a partial order on the elements of the sequence with operators $\prec, \preccurlyeq, \simeq, \succcurlyeq, \succ$, `XCMP`. All that is really necessary is to define the lexicographic sequence comparison operator `TCMP` in terms of the element comparison operator `XCMP`; this is easily expressed by using the associative `LEXICO` operator to reduce the results of elementwise comparisons to a single value. (If the sequences to be compared are of unequal length, then the shorter sequence is compared to a prefix of the longer sequence, and if they are equal, then the longer sequence is considered to be greater than the shorter sequence. This rule is implemented by an additional application of the `LEXICO` operator to the result of comparing the lengths of the sequences.)

```
trait LexicographicTotalOrder⟦T extends LexicographicTotalOrder⟦T, ⊏, ⊑, ≡, ⊒, ⊐, TCMP,
                                                                X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
                                 opr ⊏, opr ⊑, opr ≡, opr ⊒, opr ⊐, opr TCMP,
                                 X extends TotalOrderOperators⟦X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
                                 opr ≺, opr ≼, opr ≃, opr ≽, opr ≻, opr XCMP⟧
    extends { LexicographicPartialOrder⟦T, ⊏, ⊑, ≡, ⊒, ⊐, TCMP, X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
              TotalOrderOperators⟦ LexicographicTotalOrder⟦T, ⊏, ⊑, ≡, ⊒, ⊐, TCMP,
                                                           X, ≺, ≼, ≃, ≽, ≻, XCMP⟧,
                                 ⊏, ⊑, ⊒, ⊐, TCMP⟧ }
end
```

Similarly, a set of lexicographic total order operators $\sqsubset, \sqsubseteq, \equiv, \sqsupseteq, \sqsupset$, `TCMP` may be defined in terms of a total order on the elements of the sequence with operators $\prec, \preccurlyeq, \simeq, \succcurlyeq, \succ$, `XCMP`.

40.3 Operators and Their Properties

For some types, such as the integers $\mathbb{Z}$ or the rationals $\mathbb{Q}$, results are always exact, and algebraic properties can be expected to be obeyed exactly. For other types, such as floating-point numbers, results are not always numerically exact, and algebraic properties can be expected to be obeyed only approximately. For example, given three floating-point values a and b and c, it may well be that $a + (b + c)$ is not equal to $(a + b) + c$; but we would expect their values to be reasonably close—unless, of course, overflow occurred in one expression but not the other.

In order to address the difficulties of such approximate computation, many of the traits described in this section come in two varieties: approximate and exact. The $+$ operator on integers or rationals is correctly described by the trait Associative, and the $+$ operator on floating-point numbers is correctly described by the trait ApproximatelyAssociative. An important distinction is that the predicate used to test acceptability of exact algebraic properties is $=$, which is required to be an equivalence relation and therefore transitive, but a type-dependent binary predicate (typically $\approx$)

may be used to test acceptability of approximate algebraic properties, and this predicate is required only to be reflexive and symmetric.

```
trait UnaryOperator⟦T extends UnaryOperator⟦T, ⊙⟧, opr ⊙⟧
    extends Any
  abstract opr ⊙(self): T
end
```

A *unary operator* is a prefix operator that takes one argument and returns a value of the same type. Note that $\odot$ is a static parameter, used here as a "variable" name for an operator.

```
trait BinaryOperator⟦T extends BinaryOperator⟦T, ⊙⟧, opr ⊙⟧
    extends Any
  abstract opr ⊙(self, other: T): T
end
```

A *binary operator* is an infix operator that takes two arguments of the same type and returns a value of that type. Thus, for example, any trait T that extends BinaryPredicate$⟦T, +⟧$ necessarily has an infix method for the operator $+$, and that operator takes two operands of type T and returns a value of type T.

```
trait IdentityOperator⟦T extends IdentityOperator⟦T⟧⟧
    extends { UnaryOperator⟦T, IDENTITY⟧ }
  property ∀(a: T) (IDENTITY a) ≡ a
end
```

The trait Object extends IdentityOperator⟦Object⟧, so the `IDENTITY` operator is defined for every type whatsoever. (This operator may not be terribly useful for applications programming, but it has technical uses for specifying contracts and algebraic properties in libraries. It is used, for example, when defining the trait BooleanAlgebra in terms of the trait Ring: because every value is its own inverse with respect to the "exclusive OR" operator in a Boolean Algebra, `IDENTITY` is the appropriate additive inverse operator for use with the trait Ring in this connection.)

```
trait ApproximatelyCommutative⟦T extends ApproximatelyCommutative⟦T, ⊙, ≈⟧, opr ⊙, opr ≈⟧
    extends { BinaryOperator⟦T, ⊙⟧, Reflexive⟦T, ≈⟧, Symmetric⟦T, ≈⟧ }
  property ∀(a: T, b: T) (a ⊙ b) :≈: (b ⊙ a)
end
```

The trait ApproximatelyCommutative requires the operator $\odot$ to be *approximately commutative*; that is, reversing the operands produces a result that is considered to be "close enough" as determined by the specified $\approx$ predicate.

```
trait Commutative⟦T extends Commutative⟦T, ⊙⟧, opr ⊙⟧
    extends { ApproximatelyCommutative⟦T, ⊙, =⟧, EquivalenceRelation⟦T, =⟧ }
end
```

The trait Commutative requires the operator $\odot$ to be *commutative*; that is, reversing the operands produces an equal result.

```
trait ApproximatelyAssociative⟦T extends ApproximatelyAssociative⟦T, ⊙, ≈⟧, opr ⊙, opr ≈⟧
    extends { BinaryOperator⟦T, ⊙⟧, Reflexive⟦T, ≈⟧, Symmetric⟦T, ≈⟧ }
  property ∀(a: T, b: T, c: T) ((a ⊙ b) ⊙ c) :≈: (a ⊙ (b ⊙ c))
end
```

The trait ApproximatelyAssociative requires the operator $\odot$ to be *approximately associative*; that is, the expressions $(a \odot b) \odot c$ and $a \odot (b \odot c)$ always produce results that are "close enough" to each other as determined by the specified $\approx$ predicate.

```
trait Associative⟦T extends Associative⟦T, ⊙⟧, opr ⊙⟧
    extends { ApproximatelyAssociative⟦T, ⊙, =⟧, EquivalenceRelation⟦T, =⟧ }
end
```

The trait Associative requires the operator $\odot$ to be *associative*; that is, the expressions $(a \odot b) \odot c$ and $a \odot (b \odot c)$ always produce equal results.

```
trait IdempotentBinaryOperator⟦T extends IdempotentBinaryOperator⟦T, ⊙⟧, opr ⊙⟧
    extends { BinaryOperator⟦T, ⊙⟧, EquivalenceRelation⟦T, =⟧ }
  property ∀(a: T) (a ⊙ a) :=: a
end
```

An idempotent binary operator has the property that if its two arguments are the same then the result is equal to each argument. For example, `MAX` and `MIN` are idempotent, as are $\wedge$ and $\vee$ applied to boolean arguments and $\cap$ and $\cup$ applied to sets; but $+$ applied to integers is not idempotent because $1 + 1$ does not produce 1, and $\oplus$ applied to boolean arguments is not idempotent because $true \oplus true$ produces $false$. The property of idempotency is sometimes of interest when performing reductions such as $\underset{i \leftarrow 1:n}{\texttt{MAX}}\ a_i$.

```
trait HasLeftIdentity⟦T extends HasLeftIdentity⟦T, ⊙⟧, opr ⊙⟧
    extends { BinaryOperator⟦T, ⊙⟧, EquivalenceRelation⟦T, =⟧ }
  abstract isLeftIdentity(): Boolean
  property ∀(a: T, b: T) a.isLeftIdentity() →: ((a ⊙ b) = b)
end
```

A value e is a *left identity* for a binary operator $\odot$ if the result of $\odot$ always equals the right-hand operand whenever e is the left-hand operand. For example, 0 is a left identity for the $+$ operator on integers and the empty set is a left identity for the $\cup$ operator on sets.

```
trait HasRightIdentity⟦T extends HasRightIdentity⟦T, ⊙⟧, opr ⊙⟧
    extends { BinaryOperator⟦T, ⊙⟧, EquivalenceRelation⟦T, =⟧ }
  abstract isRightIdentity(): Boolean
  property ∀(a: T, b: T) b.isRightIdentity() →: ((a ⊙ b) = a)
end
```

A value e is a *right identity* for a binary operator $\odot$ if the result of $\odot$ always equals the right-hand operand whenever e is the left-hand operand. For example, 0 is a right identity (as well as a left identity) for the $+$ operator on integers and the empty set is a right identity (as well as a left identity) for the $\cup$ operator on sets. By way of contrast, 1 is a right identity for division of rationals but not a left identity.

```
value object Identity⟦opr ⊙⟧ end

trait HasIdentity⟦T extends HasIdentity⟦T, ⊙⟧, opr ⊙⟧
    extends { HasLeftIdentity⟦T, ⊙⟧, HasRightIdentity⟦T, ⊙⟧ }
    where { T coerces Identity⟦⊙⟧ }
  property ∀(a: T) (a ⊙ Identity⟦⊙⟧) = a
  property ∀(a: T) (Identity⟦⊙⟧ ⊙ a) = a
  property ∀(a: T) a.isLeftIdentity() ↔ (a = Identity⟦⊙⟧)
  property ∀(a: T) a.isRightIdentity() ↔ (a = Identity⟦⊙⟧)
end
```

If the same value is both a left identity and a right identity for $\odot$, then it may be called simply an *identity*—in fact, *the* identity, for it is unique and may be obtained by coercing the object named Identity⟦⊙⟧ to type T.

```
trait ApproximatelyHasInverses⟦T extends ApproximatelyHasInverses⟦T, ⊙, ⊘, ≈⟧,
                                opr ⊙, opr ⊘, opr ≈⟧
```

```
      extends { HasIdentity⟦T, ⊙⟧, UnaryOperator⟦T, ⊘⟧, BinaryOperator⟦T, ⊘⟧ }
      where { T coerces Identity⟦⊙⟧ }
   property ∀(a: T) (a ⊙ (⊘a)) :≈: Identity⟦⊙⟧
   property ∀(a: T) ((⊘a) ⊙ a) :≈: Identity⟦⊙⟧
   property ∀(a: T, b: T) (a ⊘ b) :≈: (a ⊙ (⊘b))
end
```

A set of values with a binary operator $\odot$ has *approximate inverses* if and only if the operator has an identity and for every value a there is another value a' such that the result of applying $\odot$ to a and a' (in either order) is "close enough" to the identity. The unary operator $\oslash$ returns the approximate inverse of its argument; as a notational convenience, it may also be used as a binary operator.

```
trait HasInverses⟦T extends HasInverses⟦T, ⊙, ⊘⟧, opr ⊙, opr ⊘⟧
      extends { ApproximatelyHasInverses⟦T, ⊙, ⊘, =⟧ }
      where { T coerces Identity⟦⊙⟧ }
end
```

A set of values with a binary operator $\odot$ has *inverses* if and only if the operator has an identity and for every value a there is another value a' such that the result of applying $\odot$ to a and a' (in either order) equals the identity. The unary operator $\oslash$ returns the inverse of its argument; as a notational convenience, it may also be used as a binary operator. A standard example is the operator $+$ on integers; the identity is 0, and the unary operator $-$ returns the additive inverse of its argument, such that $a + (-a) = 0$ and $(-a) + a = 0$. Moreover, $-$ may be used as a binary operator: $a - b$ means $a + (-b)$.

```
value object Operator⟦opr ⊙⟧ end

trait HasLeftZeroes⟦T extends HasLeftZeroes⟦T, ⊙⟧, opr ⊙⟧
      extends { BinaryOperator⟦T, ⊙⟧, EquivalenceRelation⟦T, =⟧ }
   abstract isLeftZero(_: Operator⟦⊙⟧): Boolean
   property ∀(a: T, b: T) a.isLeftZero(Operator⟦⊙⟧) →: ((a ⊙ b) = a)
end
```

A value e is a *left zero* for a binary operator $\odot$ if the result of $\odot$ always equals e whenever e is the left-hand operand. For example, $-\infty$ is a left zero for the `MIN` operator on floating-point values, and $\texttt{7FFFFFFF}_{16}$ is a left zero for the `MAX` operator on values of type $\mathbb{Z}32$. The purpose of this trait is to specify a method that says whether a given element is a left zero for $\odot$.

```
trait HasRightZeroes⟦T extends HasRightZeroes⟦T, ⊙⟧, opr ⊙⟧
      extends { BinaryOperator⟦T, ⊙⟧, EquivalenceRelation⟦T, =⟧ }
   abstract isRightZero(_: Operator⟦⊙⟧): Boolean
   property ∀(a: T, b: T) b.isRightZero(Operator⟦⊙⟧) →: ((a ⊙ b) = b)
end
```

A value e is a *right zero* for a binary operator $\odot$ if the result of $\odot$ always equals e whenever e is the right-hand operand. For example, $-\infty$ is a right zero (as well as a left zero) for the `MIN` operator on floating-point values, and $\texttt{7FFFFFFF}_{16}$ is a right zero (as well as a left zero) for the `MAX` operator on values of type $\mathbb{Z}32$. By way of contrast, 0 is a left zero for the arithmetic shift operator on integers, but is not a right zero. The purpose of this trait is to specify a method that says whether a given element is a right zero for $\odot$.

```
trait ApproximatelyLeftDistributive⟦T extends ApproximatelyLeftDistributive⟦T, ⊗, ⊕, ≈⟧,
                                    opr ⊗, opr ⊕, opr ≈⟧
      extends { BinaryOperator⟦T, ⊗⟧, BinaryOperator⟦T, ⊕⟧, Reflexive⟦T, ≈⟧, Symmetric⟦T, ≈⟧ }
   property ∀(a: T, b: T, c: T) (a ⊗ (b ⊕ c)) :≈: ((a ⊗ b) ⊕ (a ⊗ c))
end
```

The trait ApproximatelyLeftDistributive requires the operator $\otimes$ to be *approximately left distributive* over the operator $\oplus$; that is, the expressions $a \otimes (b \oplus c)$ and $(a \otimes b) \oplus (a \otimes c)$ always produce results that are "close enough" to each other as determined by the specified $\approx$ predicate.

```
trait LeftDistributive[[T extends LeftDistributive[[T, ⊗, ⊕]], opr ⊗, opr ⊕]]
      extends { ApproximatelyLeftDistributive[[T, ⊗, ⊕, =]], EquivalenceRelation[[T, =]] }
end
```

The trait LeftDistributive requires the operator $\otimes$ to be *left distributive* over the operator $\oplus$; that is, the expressions $a \otimes (b \oplus c)$ and $(a \otimes b) \oplus (a \otimes c)$ always produce equal results.

```
trait ApproximatelyRightDistributive[[T extends ApproximatelyRightDistributive[[T, ⊗, ⊕, ≈]],
                                                  opr ⊗, opr ⊕, opr ≈]]
      extends { BinaryOperator[[T, ⊗]], BinaryOperator[[T, ⊕]], Reflexive[[T, ≈]], Symmetric[[T, ≈]] }
   property ∀(a: T, b: T, c: T) ((a ⊕ b) ⊗ c) :≈: ((a ⊗ c) ⊕ (b ⊗ c))
end
```

The trait ApproximatelyRightDistributive requires the operator $\otimes$ to be *approximately right distributive* over the operator $\oplus$; that is, the expressions $(a \oplus b) \otimes c$ and $(a \otimes c) \oplus (b \otimes c)$ always produce results that are "close enough" to each other as determined by the specified $\approx$ predicate.

```
trait RightDistributive[[T extends RightDistributive[[T, ⊗, ⊕]], opr ⊗, opr ⊕]]
      extends { ApproximatelyRightDistributive[[T, ⊗, ⊕, =]], EquivalenceRelation[[T, =]] }
end
```

The trait RightDistributive requires the operator $\otimes$ to be *right distributive* over the operator $\oplus$; that is, the expressions $(a \oplus b) \otimes c$ and $(a \otimes c) \oplus (b \otimes c)$ always produce equal results.

```
trait ApproximatelyDistributive[[T extends ApproximatelyDistributive[[T, ⊗, ⊕, ≈]],
                                                opr ⊗, opr ⊕, opr ≈]]
      extends { ApproximatelyLeftDistributive[[T, ⊗, ⊕, ≈]],
                ApproximatelyRightDistributive[[T, ⊗, ⊕, ≈]] }
end
```

The trait ApproximatelyDistributive requires the operator $\otimes$ to be both approximately left distributive and approximately right distributive over the operator $\oplus$.

```
trait Distributive[[T extends Distributive[[T, ⊗, ⊕]], opr ⊗, opr ⊕]]
      extends { ApproximatelyDistributive[[T, ⊗, ⊕, =]], LeftDistributive[[T, ⊗, ⊕]], RightDistributive[[T, ⊗, ⊕]] }
end
```

The trait Distributive requires the operator $\otimes$ to be both left distributive and right distributive over the operator $\oplus$.

```
trait AbsorptionLaws[[T extends AbsorptionLaws[[T, ⊓, ⊔]], opr ⊓, opr ⊔]]
      extends { BinaryOperator[[T, ⊓]], BinaryOperator[[T, ⊔]], EquivalenceRelation[[T, =]] }
   property ∀(a: T, b: T) a ⊔ (a ⊓ b) = a = a ⊓ (a ⊔ b)
end
```

The *absorption laws* specify certain relationships between two operators $\sqcap$ and $\sqcup$. These laws are one of the defining properties of a lattice.

```
value object Zero[[opr ⊗]] end
trait ApproximateZeroAnnihilation[[ T extends ApproximateZeroAnnihilation[[T, ⊗, ≈]],
                                                  opr ⊗, opr ≈]]
      extends { BinaryOperator[[T, ⊗]], Reflexive[[T, ≈]], Symmetric[[T, ≈]] }
```

```
    where { T coerces Zero⟦⊗⟧ }
  property ∀(a: T) (Zero⟦⊗⟧ ⊗ a) :≈: Zero⟦⊗⟧
  property ∀(a: T) (a ⊗ Zero⟦⊗⟧) :≈: Zero⟦⊗⟧
end
```

An operator $\otimes$ obeys *approximate zero annihilation* if and only if there is an element (call it z) that when used as either operand of $\otimes$ causes the result to be "close enough" to z as determined by the specified $\approx$ predicate. This zero element may be obtained by coercing the object named $\mathrm{Zero}[\![\otimes]\!]$ to type T.

```
trait ZeroAnnihilation⟦T extends ZeroAnnihilation⟦T, ⊗⟧, opr ⊗⟧
    extends { ApproximateZeroAnnihilation⟦T, ⊗, =⟧, EquivalenceRelation⟦T, =⟧,
              HasLeftZeroes⟦T, ⊗⟧, HasRightZeroes⟦T, ⊗⟧ }
    where { T coerces Zero⟦⊗⟧ }
end
```

An operator $\otimes$ obeys *zero annihilation* if and only if there is an element (call it z) that when used as either operand of $\otimes$ causes the result to equal z. This element is both a left zero and a right zero for $\otimes$.

```
trait NoApproximateZeroDivisors⟦T extends NoApproximateZeroDivisors⟦T, ⊗, ≈⟧, opr ⊗, opr ≈⟧
    extends { BinaryOperator⟦T, ⊗⟧, Reflexive⟦T, ≈⟧, Symmetric⟦T, ≈⟧ }
    where { T coerces Zero⟦⊗⟧ }
  property ∀(a: T, b: T) (a ⊗ b :≈: Zero⟦⊗⟧) → (a ≈ Zero⟦⊗⟧ ∨ b ≈ Zero⟦⊗⟧)
end
```

An operator $\otimes$ has *no approximate zero divisors* if and only if, for every pair of elements a and b, if $a \otimes b$ is approximately zero then at least one of a and b is approximately zero. (Great care is needed in the definition of "approximately" for this to work out correctly in practice.)

```
trait NoZeroDivisors⟦T extends NoZeroDivisors⟦T, ⊗⟧, opr ⊗⟧
    extends { NoApproximateZeroDivisors⟦T, ⊗, =⟧, EquivalenceRelation⟦T, =⟧ }
    where { T coerces Zero⟦⊗⟧ }
  property ∀(a: T, b: T) (a ⊗ b = Zero⟦⊗⟧) → (a = Zero⟦⊗⟧ ∨ b = Zero⟦⊗⟧)
end
```

An operator $\otimes$ has *no zero divisors* if and only if, for every pair of elements a and b, if $a \otimes b$ is zero then at least one of a and b is zero (put conversely, if a and b are both nonzero then $a \otimes b$ must be nonzero).

```
trait UnaryOperatorSubstitutionLaws⟦T extends UnaryOperatorSubstitutionLaws⟦T, ⊙, =⟧,
                                    opr ⊙, opr ≃⟧
    extends { UnaryOperator⟦T, ⊙⟧, BinaryPredicate⟦T, ≃⟧ }
  property ∀(a: T, a′: T) (a ≃ a′) →: (⊙a) ≃ (⊙a′)
end
```

This peculiarly spiffy trait states that the unary operator $\odot$ is consistent under substitutions described by the relation $\simeq$ (which is typically, but not always, an equivalence relation); that is, the result produced by $\odot$ is unchanged if its argument is replaced by some other value that is equivalent.

```
trait BinaryOperatorSubstitutionLaws⟦T extends BinaryOperatorSubstitutionLaws⟦T, ⊙, =⟧,
                                     opr ⊙, opr ≃⟧
    extends { BinaryOperator⟦T, ⊙⟧, BinaryPredicate⟦T, ≃⟧ }
  property ∀(a: T, a′: T) (a ≃ a′) →: ∀(b: T) (a ⊙ b) ≃ (a′ ⊙ b)
  property ∀(b: T, b′: T) (b ≃ b′) →: ∀(a: T) (a ⊙ b) ≃ (a ⊙ b′)
end
```

This equally spiffy trait states that the binary operator $\odot$ is consistent under substitutions described by the relation $\simeq$ (which is typically, but not always, an equivalence relation); that is, the result produced by $\odot$ is unchanged if either argument is replaced by some other value that is equivalent. (It is then easy to prove that the result is unchanged even when *both* arguments are replaced by equivalent values.)

40.4 Lattices

```
trait Semilattice[[T extends Semilattice[[T, ⊓]], opr ⊓]]
    extends { Commutative[[T, ⊓]], Associative[[T, ⊓]], IdempotentBinaryOperator[[T, ⊓]] }
end
```

A *semilattice* is a set of values with an operator $\sqcap$ that is commutative, associative, and idempotent. Examples of such operators are $\wedge$, $\vee$, $\cap$, $\cup$, `MAX`, `MIN`, `GCD`, and `LCM`.

```
trait BoundedSemilattice[[T extends BoundedSemilattice[[T, ⊓]], opr ⊓]]
    extends { Semilattice[[T, ⊓]], CommutativeMonoid[[T, ⊓]] }
    where { T coerces Identity[[⊓]] }
end
```

A *bounded semilattice* is a semilattice for which the operator $\sqcap$ has an identity. For example, the boolean value $true$ is the identity for $\wedge$ and the empty set is the identity for $\cup$.

```
trait Lattice[[T extends Lattice[[T, ⊓, ⊔]], opr ⊓, opr ⊔]]
    extends { Semilattice[[T, ⊓]], Semilattice[[T, ⊔]], AbsorptionLaws[[T, ⊓, ⊔]] }
end
```

A *lattice* is a pair of semilattices that share the same set of values and whose operators together obey the absorption laws. One operator (here, by convention, $\sqcap$) is called the *meet* operator and the other (here, by convention, $\sqcup$) is called the *join* operator.

```
trait MeetBoundedLattice[[T extends MeetBoundedLattice[[T, ⊓, ⊔]], opr ⊓, opr ⊔]]
    extends { Lattice[[T, ⊓, ⊔]], BoundedSemilattice[[T, ⊓]] }
    where { T coerces Identity[[⊓]] }
end
```

If the semilattice associated with the meet operator of a lattice is bounded, then the lattice is a *meet-bounded lattice*.

```
trait JoinBoundedLattice[[T extends JoinBoundedLattice[[T, ⊓, ⊔]], opr ⊓, opr ⊔]]
    extends { Lattice[[T, ⊓, ⊔]], BoundedSemilattice[[T, ⊔]]}
    where { T coerces Identity[[⊔]] }
end
```

If the semilattice associated with the join operator of a lattice is bounded, then the lattice is a *join-bounded lattice*.

```
trait BoundedLattice[[T extends BoundedLattice[[T, ⊓, ⊔]], opr ⊓, opr ⊔]]
    extends { MeetBoundedLattice[[T, ⊓, ⊔]], JoinBoundedLattice[[T, ⊓, ⊔]]}
    where { T coerces Identity[[⊓]], T coerces Identity[[⊔]] }
end
```

If a lattice is both meet-bounded and join-bounded, then we simply say that the lattice is *bounded*.

```
trait PartialOrderAndLattice[[ T extends PartialOrderLattice[[T, ≼, ⊓, ⊔]],
                               opr ≼, opr ⊓, opr ⊔]]
```

```
    extends { PartialOrder⟦T, ≼⟧, Lattice⟦T, ⊓, ⊔⟧ }
  property ∀(a: T, b: T) (a ≼ b) ↔ (a ⊓ b = a)
  property ∀(a: T, b: T) (a ≼ b) ↔ (a ⊔ b = b)
end
```

Every lattice defines a partial order, and every partial order defines a lattice. This trait specifies that the type T provides the necessary operators to be regarded as both a partial order and a lattice, states the properties that should relate the partial-order behavior to the lattice behavior. As an example, if the set of values is the integers $\mathbb{Z}$, the partial order operator is `LEQ`, and the lattice operators are `MIN` and `MAX`, then the requirements are that a `LEQ` b if and only if $a = a$ `MIN` b, and that a `LEQ` b if and only if $b = a$ `MAX` b.

```
trait PartialOrderAndMeetBoundedLattice⟦T extends PartialOrderAndMeetBoundedLattice⟦T, ≼, ⊓, ⊔⟧,
                                        opr ≼, opr ⊓, opr ⊔⟧
    extends { PartialOrderAndLattice⟦T, ≼, ⊓, ⊔⟧,
              MeetBoundedLattice⟦T, ⊓, ⊔⟧,
              HasMinimalElement⟦T, ≼⟧ }
    where { T coerces Identity⟦⊓⟧, T coerces MaximalElement⟦≼⟧ }
  property coerce⟦T⟧(Identity⟦⊓⟧) = coerce⟦T⟧(MaximalElement⟦≼⟧)
end
```

The partial order associated with a meet-bounded lattice should have a maximal element, which is the identity of the meet operator.

```
trait PartialOrderAndJoinBoundedLattice⟦T extends PartialOrderAndJoinBoundedLattice⟦T, ≼, ⊓, ⊔⟧,
                                        opr ≼, opr ⊓, opr ⊔⟧
    extends { PartialOrderAndLattice⟦T, ≼, ⊓, ⊔⟧,
              JoinBoundedLattice⟦T, ⊓, ⊔⟧,
              HasMaximalElement⟦T, ≼⟧ }
    where { T coerces Identity⟦⊔⟧, T coerces MinimalElement⟦≼⟧ }
  property coerce⟦T⟧(Identity⟦⊔⟧) = coerce⟦T⟧(MinimalElement⟦≼⟧)
end
```

The partial order associated with a join-bounded lattice should have a minimal element, which is the identity of the join operator.

```
trait PartialOrderAndBoundedLattice⟦T extends PartialOrderAndBoundedLattice⟦T, ≼, ⊓, ⊔⟧,
                                     opr ≼, opr ⊓, opr ⊔⟧
    extends { PartialOrderAndMeetBoundedLattice⟦T, ≼, ⊓, ⊔⟧,
              PartialOrderAndJoinBoundedLattice⟦T, ≼, ⊓, ⊔⟧ }
end
```

The partial order associated with a bounded lattice should have both a minimal element and a maximal element.

40.5 Monoids, Groups, Rings, and Fields

```
trait ApproximateMonoid⟦T extends ApproximateMonoid⟦T, ⊙, ≈⟧, opr ⊙, opr ≈⟧
    extends { ApproximatelyAssociative⟦T, ⊙, ≈⟧, HasIdentity⟦T, ⊙⟧ }
    where { T coerces Identity⟦⊙⟧ }
end
```

An *approximate monoid* is a set of values with an approximately associative binary operator $\odot$ that has an identity. For example, floating-point multiplication has identity 1 and is approximately associative.

```
trait Monoid⟦T extends Monoid⟦T, ⊙⟧, opr ⊙⟧
    extends { ApproximateMonoid⟦T, ⊙, =⟧, Associative⟦T, ⊙⟧ }
    where { T coerces Identity⟦⊙⟧ }
end
```

A *monoid* is a set of values with an associative binary operator $\odot$ that has an identity. For example, trait String extends Monoid⟦String, ||⟧ where || is the string concatenation operator. Note that string concatenation is associative but not commutative and that the empty string is the identity for string concatenation, so coercing Identity⟦ || ⟧ to type String produces the empty string.

```
trait ApproximateCommutativeMonoid⟦ T extends ApproximateCommutativeMonoid⟦T, ⊕, ≈⟧,
                                                opr ⊕, opr ≈⟧
    extends { ApproximateMonoid⟦T, ⊕, ≈⟧, ApproximatelyCommutative⟦T, ⊕, ≈⟧ }
    where { T coerces Identity⟦⊕⟧ }
end
```

An *approximate commutative monoid* is an approximate monoid whose binary operator is also approximately commutative. For example, floating-point multiplication has identity 1 and is approximately associative and also approximately (indeed, exactly) commutative.

```
trait CommutativeMonoid⟦T extends CommutativeMonoid⟦T, ⊕⟧, opr ⊕⟧
    extends { ApproximateCommutativeMonoid⟦T, ⊕, =⟧, Monoid⟦T, ⊕⟧, Commutative⟦T, ⊕⟧ }
    where { T coerces Identity⟦⊕⟧ }
end
```

A *commutative monoid* is a monoid whose binary operator is also commutative. For example, the operator $\wedge$ on boolean values is associative and commutative and has identity $true$; likewise, the operator $\vee$ on boolean values is associative and commutative and has identity $false$.

```
trait ApproximateGroup⟦T extends ApproximateGroup⟦T, ⊙, ⊘, ≈⟧, opr ⊙, opr ⊘, opr ≈⟧
    extends { ApproximateMonoid⟦T, ⊙, ≈⟧, ApproximatelyHasInverses⟦T, ⊙, ⊘, ≈⟧ }
    where { T coerces Identity⟦⊙⟧ }
end
```

An *approximate group* is an approximate monoid that has approximate inverses. For example, a floating-point representation of quaternions with multiplication as the binary operator would form an approximate group.

```
trait Group⟦T extends Group⟦T, ⊙, ⊘⟧, opr ⊙, opr ⊘⟧
    extends { ApproximateGroup⟦T, ⊙, ⊘, =⟧, Monoid⟦T, ⊙⟧, HasInverses⟦T, ⊙, ⊘⟧ }
    where { T coerces Identity⟦⊙⟧ }
end
```

A *group* is monoid that has inverses.

```
trait ApproximateAbelianGroup⟦ T extends ApproximateAbelianGroup⟦T, ⊕, ⊖, ≈⟧,
                                        opr ⊕, opr ⊖, opr ≈⟧
    extends { ApproximateGroup⟦T, ⊕, ⊖, ≈⟧,
              ApproximateCommutativeMonoid⟦T, ⊕, ≈⟧ }
    where { T coerces Identity⟦⊕⟧ }
end
```

An *approximate Abelian group* is an approximate group whose binary operator is also approximately commutative.

```
trait AbelianGroup⟦T extends AbelianGroup⟦T, ⊕, ⊖⟧, opr ⊕, opr ⊖⟧
    extends { ApproximateAbelianGroup⟦T, ⊕, ⊖, =⟧,
```

```
            Group⟦T, ⊕, ⊖⟧, CommutativeMonoid⟦T, ⊕⟧ }
    where { T coerces Identity⟦⊕⟧ }
end
```

An *Abelian group* is group whose binary operator is also commutative. (The term "Abelian" is traditionally used instead of "commutative" when discussing groups, in tribute to mathematician Niels Henrik Abel.) For example, the integers with the binary addition operator $+$, unary negation operator $-$, and identity 0 form an Abelian group. As another example, the boolean values with the binary exclusive OR operator $\oplus$ and unary negation operator IDENTITY form an Abelian group; the value *false* is the identity for $\oplus$.

```
trait ApproximateSemiRing⟦T extends ApproximateSemiRing⟦T, ⊕, ⊗, ≈⟧,
                           opr ⊕, opr ⊗, opr ≈⟧
    extends { ApproximateCommutativeMonoid⟦T, ⊕, ≈⟧,
              ApproximateMonoid⟦T, ⊗, ≈⟧,
              ApproximatelyDistributive⟦T, ⊗, ⊕, ≈⟧,
              ApproximateZeroAnnihilation⟦T, ⊗, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
  property coerce⟦T⟧(Zero⟦⊗⟧) ≈ coerce⟦T⟧(Identity⟦⊕⟧)
end
```

An *approximate semiring* is a set of values that has two binary operators $\oplus$ and $\otimes$, such that (a) the values form an approximate commutative monoid with $\oplus$; (b) the values form an approximate monoid with $\otimes$; (c) $\otimes$ is approximately distributive over $\oplus$; and (d) $\otimes$ obeys approximate zero annihilation, where the zero for $\otimes$ is the same as the identity for $\oplus$.

```
trait SemiRing⟦T extends SemiRing⟦T, ⊕, ⊗⟧, opr ⊕, opr ⊗⟧
    extends { ApproximateSemiRing⟦T, ⊕, ⊗, =⟧,
              CommutativeMonoid⟦T, ⊕⟧,
              Monoid⟦T, ⊗⟧,
              Distributive⟦T, ⊗, ⊕⟧,
              ZeroAnnihilation⟦T, ⊗⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

A *semiring* is a set of values that has two binary operators $\oplus$ and $\otimes$, such that (a) the values form a commutative monoid with $\oplus$; (b) the values form a monoid with $\otimes$; (c) $\otimes$ is distributive over $\oplus$; and (d) $\otimes$ obeys zero annihilation, where the zero for $\otimes$ is the same as the identity for $\oplus$.

```
trait ApproximateRing⟦T extends ApproximateRing⟦T, ⊕, ⊖, ⊗, ≈⟧,
                       opr ⊕, opr ⊖, opr ⊗, opr ≈⟧
    extends { ApproximateAbelianGroup⟦T, ⊕, ⊖, ≈⟧,
              ApproximateSemiRing⟦T, ⊕, ⊗, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

An *approximate ring* is an approximate semiring that also has a unary operator $\ominus$ that returns inverses for the $\oplus$ operator so that the values form an approximate group with $\oplus$ and $\ominus$.

```
trait Ring⟦T extends Ring⟦T, ⊕, ⊖, ⊗⟧, opr ⊕, opr ⊖, opr ⊗⟧
    extends { ApproximateRing⟦T, ⊕, ⊖, ⊗, =⟧,
              AbelianGroup⟦T, ⊕, ⊖⟧,
              SemiRing⟦T, ⊕, ⊗⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

A *ring* is a semiring that also has a unary operator $\ominus$ that returns inverses for the $\oplus$ operator so that the values form a group with $\oplus$ and $\ominus$.

```
trait ApproximateCommutativeRing⟦T extends ApproximateCommutativeRing⟦T, ⊕, ⊖, ⊗, ≈⟧,
                                        opr ⊕, opr ⊖, opr ⊗, opr ≈⟧
    extends { ApproximateRing⟦T, ⊕, ⊖, ⊗, ≈⟧,
              ApproximateCommutativeMonoid⟦T, ⊗, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

An *approximate commutative ring* is an approximate ring for which the binary operator $\otimes$ is also approximately commutative.

```
trait CommutativeRing⟦T extends CommutativeRing⟦T, ⊕, ⊖, ⊗⟧,
                            opr ⊕, opr ⊖, opr ⊗⟧
    extends { ApproximateCommutativeRing⟦T, ⊕, ⊖, ⊗, =⟧,
              Ring⟦T, ⊕, ⊖, ⊗⟧,
              CommutativeMonoid⟦T, ⊗⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

A *commutative ring* is a ring for which the binary operator $\otimes$ is also commutative.

```
trait ApproximateMathematicalDomain⟦T extends ApproximateRing⟦T, ⊕, ⊖, ⊗, ≈⟧,
                                        opr ⊕, opr ⊖, opr ⊗, opr ≈⟧
    extends { ApproximateRing⟦T, ⊕, ⊖, ⊗, ≈⟧,
              NoApproximateZeroDivisors⟦T, ⊗, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

An *approximate mathematical domain* is an approximate ring that has no approximate zero divisors.

```
trait MathematicalDomain⟦T extends Ring⟦T, ⊕, ⊖, ⊗⟧,
                            opr ⊕, opr ⊖, opr ⊗⟧
    extends { ApproximateMathematicalDomain⟦T, ⊕, ⊖, ⊗, =⟧,
              Ring⟦T, ⊕, ⊖, ⊗⟧,
              NoZeroDivisors⟦T, ⊗⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

A *mathematical domain* is a ring that has no zero divisors. (In mathematics, this is called simply a *domain*. In Fortress, the term "mathematical domain" is used instead to avoid consuming this word for a concept that admittedly is used rarely in programming when compared with other common single words such as"ring" and "field.")

```
trait ApproximateIntegralDomain⟦T extends ApproximateRing⟦T, ⊕, ⊖, ⊗, ≈⟧,
                                    opr ⊕, opr ⊖, opr ⊗, opr ≈⟧
    extends { ApproximateMathematicalDomain⟦T, ⊕, ⊖, ⊗, ≈⟧,
              ApproximateCommutativeRing⟦T, ⊕, ⊖, ⊗, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

An *approximate integral domain* is an approximate mathematical domain for which the binary operator $\otimes$ is also approximately commutative.

```
trait IntegralDomain⟦T extends Ring⟦T, ⊕, ⊖, ⊗⟧,
                     opr ⊕, opr ⊖, opr ⊗⟧
    extends { ApproximateIntegralDomain⟦T, ⊕, ⊖, ⊗, =⟧,
              MathematicalDomain⟦T, ⊕, ⊖, ⊗⟧,
              CommutativeRing⟦T, ⊕, ⊖, ⊗⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

An *integral domain* is a mathematical domain for which the binary operator $\otimes$ is also commutative.

```
trait ApproximateDivisionRing⟦T extends ApproximateDivisionRing⟦T, U, ⊕, ⊖, ⊗, ⊘, ≈⟧,
                              U extends T,
                              opr ⊕, opr ⊖, opr ⊗, opr ⊘, opr ≈⟧
    extends { ApproximateMathematicalDomain⟦T, ⊕, ⊖, ⊗, ≈⟧,
              ApproximateGroup⟦U, ⊗, ⊘, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
  property ¬instanceOf⟦U⟧(coerce⟦T⟧(Zero⟦⊕⟧))
  property ∀(a: T) a ≠ Zero⟦⊕⟧ → instanceOf⟦U⟧(a)
end
```

An *approximate division ring* is an approximate ring for which the binary operator $\otimes$ also has approximate inverses, so that the values other than the zero of $\otimes$ form an approximate group with $\otimes$ and $\oslash$. An approximate division ring is in fact also an approximate mathematical domain.

```
trait DivisionRing⟦T extends DivisionRing⟦T, U, ⊕, ⊖, ⊗, ⊘⟧,
                   U extends T,
                   opr ⊕, opr ⊖, opr ⊗, opr ⊘⟧
    extends { ApproximateDivisionRing⟦T, U, ⊕, ⊖, ⊗, ⊘, =⟧,
              MathematicalDomain⟦T, ⊕, ⊖, ⊗, ≈⟧,
              Group⟦U, ⊗, ⊘⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

A *division ring* is a ring for which the binary operator $\otimes$ also has inverses, so that the values other than the zero of $\otimes$ form a group with $\otimes$ and $\oslash$. A division ring is in fact also a mathematical domain.

```
trait ApproximateField⟦T extends ApproximateField⟦T, U, ⊕, ⊖, ⊗, ⊘, ≈⟧,
                       U extends T,
                       opr ⊕, opr ⊖, opr ⊗, opr ⊘, opr ≈⟧
    extends { ApproximateIntegralDomain⟦T, ⊕, ⊖, ⊗, ≈⟧,
              ApproximateDivisionRing⟦T, U, ⊕, ⊖, ⊗, ⊘, ≈⟧,
              ApproximateAbelianGroup⟦T, ⊗, ⊘, ≈⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

An *approximate field* is an approximately commutative ring that is also an approximate division ring: the binary operator $\otimes$ is approximately commutative and also has approximate inverses, so that the values other than the zero of $\otimes$ form an approximate Abelian group with $\otimes$ and $\oslash$. An approximate field is in fact also an approximate integral domain.

```
trait Field⟦T extends Field⟦T, U, ⊕, ⊖, ⊗, ⊘⟧, U extends T, opr ⊕, opr ⊖, opr ⊗, opr ⊘⟧
    extends { ApproximateField⟦T, U, ⊕, ⊖, ⊗, ⊘, =⟧,
              IntegralDomain⟦T, ⊕, ⊖, ⊗⟧,
              DivisionRing⟦T, U, ⊕, ⊖, ⊗, ⊘⟧,
```

```
        AbelianGroup⟦U, ⊗, ⊘⟧ }
    where { T coerces Identity⟦⊕⟧, T coerces Identity⟦⊗⟧, T coerces Zero⟦⊗⟧ }
end
```

A *field* is a commutative ring that is also a division ring: the binary operator $\otimes$ is commutative and also has inverses, so that the values other than the zero of $\otimes$ form an Abelian group with $\otimes$ and $\oslash$. A field is in fact also an integral domain.

40.6 Boolean Algebras

```
value object ComplementBound⟦opr ⊙⟧ end

trait HasComplements⟦T extends HasComplements⟦T, ⊙, ∼⟧, opr ⊙, opr ∼⟧
    extends { BinaryOperator⟦T, ⊙⟧, UnaryOperator⟦T, ∼⟧, EquivalenceRelation⟦T, =⟧ }
    where { T coerces ComplementBound⟦⊙⟧ }
  property ∀(a: T) (a ⊙ (∼ a)) :=: ComplementBound⟦⊙⟧
end
```

A set of values with a binary operator $\odot$ has *complements* if and only if there is a specific value e such that for every value a there is another value a' such that the result of applying $\odot$ to a and a' (in either order) equals the specified value e. This value may be obtained by coercing the object named $\mathrm{ComplementBound}[\![\odot]\!]$ to type T. The unary operator $\sim$ returns the complement of its argument with respect to the operator $\odot$.

Note that the trait HasComplements is similar to the trait HasInverses, but HasComplements does not require that that specified element be an identity of the binary operator.

```
trait DeMorgan⟦T extends DeMorgan⟦T, ⅄, ϒ, ∼⟧, opr ⅄, opr ϒ, opr ∼⟧
    extends { BinaryOperator⟦T, ⅄⟧, BinaryOperator⟦T, ϒ⟧,
              UnaryOperator⟦T, ∼⟧, EquivalenceRelation⟦T, =⟧ }
  property ∀(a: T, b: T) (∼ (a ϒ b)) :=: ((∼ a) ⅄ (∼ b))
end
```

This trait expresses De Morgan's law for two binary operators $⅄$ and $ϒ$ and a unary operator $\sim$: the expressions $\sim (a \ ϒ \ b)$ and $(\sim a) \ ⅄ \ (\sim b)$ produce equal results.

```
trait BooleanAlgebra⟦T extends BooleanAlgebra⟦T, ⅄, ϒ, ∼, ⩒⟧, opr ⅄, opr ϒ, opr ∼, opr ⩒⟧
    extends { Commutative⟦T, ⅄⟧, Associative⟦T, ⅄⟧,
              Commutative⟦T, ϒ⟧, Associative⟦T, ϒ⟧,
              IdempotentBinaryOperator⟦T, ⅄⟧,
              IdempotentBinaryOperator⟦T, ϒ⟧,
              HasIdentity⟦T, ⅄⟧, HasIdentity⟦T, ϒ⟧,
              HasComplements⟦T, ϒ, ∼⟧, HasComplements⟦T, ⅄, ∼⟧,
              Distributive⟦T, ⅄, ϒ⟧, Distributive⟦T, ϒ, ⅄⟧,
              DeMorgan⟦T, ⅄, ϒ, ∼⟧, DeMorgan⟦T, ϒ, ⅄, ∼⟧,
              BoundedLattice⟦T, ⅄, ϒ⟧, Ring⟦T, ⩒, IDENTITY, ⅄⟧ }
    where { T coerces Identity⟦⅄⟧, T coerces ComplementBound⟦⅄⟧,
            T coerces Identity⟦ϒ⟧, T coerces ComplementBound⟦ϒ⟧,
            T coerces Identity⟦⩒⟧, T coerces Zero⟦⅄⟧ }
  property ∀(a: T) (∼ (∼ a)) :=: a
  property ∀(a: T, b: T) a ⩒ b = (a ⅄ (∼ b)) ϒ ((∼ a) ⅄ b))
  property coerce⟦T⟧(Identity⟦⩒⟧) = coerce⟦T⟧(Identity⟦ϒ⟧)
  property coerce⟦T⟧(ComplementBound⟦⅄⟧) = coerce⟦T⟧(Identity⟦ϒ⟧)
  property coerce⟦T⟧(ComplementBound⟦ϒ⟧) = coerce⟦T⟧(Identity⟦⅄⟧)
end
```

A *boolean algebra* is an algebraic structure consisting of a set of values, three binary operators $\curlywedge$, $\curlyvee$, and $\veebar$, and a unary operator $\sim$, such that the operators obey a number of specific properties:

- $\curlywedge$ is commutative, associative, and idempotent, and has a unique identity
- $\curlyvee$ is commutative, associative, and idempotent, and has a unique identity
- $\curlywedge$ has complements with respect to $\sim$
- $\curlyvee$ has complements with respect to $\sim$
- $\curlywedge$ and $\curlyvee$ distribute over each other
- De Morgan's law applies to $\curlywedge$, $\curlyvee$, and $\sim$, and also to $\curlyvee$, $\curlywedge$, and $\sim$
- The values form a bounded lattice with $\curlywedge$ as the meet operator and $\curlyvee$ as the join operator.
- The values form a ring with $\veebar$ as the addition operator, `IDENTITY` as the additive inverse operator, and $\curlywedge$ as the multiplication operator

The type $\mathrm{Boolean}$ is the most familiar example of a boolean algebra. The power set of a set (that is, the set of all subsets of the set) also forms a boolean algebra with operators $\cap$, $\cup$, set complement, and symmetric set difference; the empty set is the identity for $\cup$, and the original set is the identity for $\cap$.

Chapter 41

Numbers

REVISED BY THE 2026 REVIVAL SEE Section I.1.14

This chapter is the original design of the rational types, kept word for word as a superseded design. It defines the eighteen rational types of the Working Draft of February 2011 as instantiations of one trait, $\mathrm{RationalQuantity}$, each of which extends the instantiations with weaker flags. Instantiation exclusion (Section I.1.1) refuses that trait; the compiled type checker refuses it as a cyclic type hierarchy, and the interpreter cannot instantiate it; the Fortress standard libraries never declared these types; and the team's later Types chapter (Luchangco, 2012) comments out the type aliases by which this chapter names them and keeps the statement that the numeric types are mutually exclusive (Section 6.9). Section 33.1 and Section 33.2 describe the number types the library declares, and Appendix I says why. The way back is the later chapter's declaration-site variance: each flag a covariant static parameter, a form the compiled type checker already accepts. It waits for covariance to run on both implementations, which is item 12 of the revival's worklist and row 404 of its gap ledger.

41.1 The Trait Fortress.Standard.RationalQuantity

The standard types for rational numbers such as $\mathbb{Q}$ and $\mathbb{Q}^*$ and $\mathbb{Q}^{\#}_{\leq}$ are defined in terms of a single trait $\mathrm{RationalQuantity}$ that handles dimensions and units as well as performing a case analysis to distinguish rather a particular expression is guaranteed not to produce infinities, $0/0$, or numbers of a particular sign.

The trait $\mathrm{RationalQuantity}$ takes seven static parameters; the first is a dimensional unit, and the others are booleans specifying whether an instance of the trait can possibly be $-\infty$, a finite rational less than zero, zero, a finite rational greater than zero, $+\infty$, or $0/0$. This allows the standard rational types to be represented as follows:

$\texttt{type}\ \mathbb{Q} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}_{<} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{true}, \mathit{false}, \mathit{false}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}_{\leq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}_{\geq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}_{>} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{false}, \mathit{false}, \mathit{true}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}_{\neq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{true}, \mathit{false}, \mathit{true}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^* = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^*_{<} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^*_{\leq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^*_{\geq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^*_{>} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{false}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^*_{\neq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{false}]\!]$
$\texttt{type}\ \mathbb{Q}^{\#} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{true}]\!]$
$\texttt{type}\ \mathbb{Q}^{\#}_{<} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}, \mathit{false}, \mathit{true}]\!]$
$\texttt{type}\ \mathbb{Q}^{\#}_{\leq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{false}, \mathit{true}]\!]$
$\texttt{type}\ \mathbb{Q}^{\#}_{\geq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{true}, \mathit{true}]\!]$
$\texttt{type}\ \mathbb{Q}^{\#}_{>} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{false}, \mathit{false}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{true}]\!]$
$\texttt{type}\ \mathbb{Q}^{\#}_{\neq} = \mathrm{RationalQuantity}[\![\mathrm{dimensionless}, \mathit{true}, \mathit{true}, \mathit{false}, \mathit{true}, \mathit{true}, \mathit{true}]\!]$

Here is the detailed description of $\mathrm{RationalQuantity}$, showing the details of the type calculations:

$$
\begin{array}{l}
\texttt{trait}\ \mathrm{RationalQuantity}[\![\texttt{unit}\ U\ \texttt{absorbs unit},\\
\qquad\qquad\qquad\qquad \texttt{bool}\ \mathit{ninf}, \texttt{bool}\ \mathit{lt}, \texttt{bool}\ \mathit{eq}, \texttt{bool}\ \mathit{gt}, \texttt{bool}\ \mathit{pinf}, \texttt{bool}\ \mathit{nan}]\!]\\
\quad \texttt{extends}\ \{\mathrm{RationalQuantity}[\![U, \mathit{ninf}', \mathit{lt}', \mathit{eq}', \mathit{gt}', \mathit{pinf}', \mathit{nan}']\!]\\
\qquad\qquad \texttt{where}\ \{\texttt{bool}\ \mathit{ninf}', \texttt{bool}\ \mathit{lt}', \texttt{bool}\ \mathit{eq}', \texttt{bool}\ \mathit{gt}', \texttt{bool}\ \mathit{pinf}', \texttt{bool}\ \mathit{nan}',\\
\qquad\qquad\qquad\quad \mathit{ninf} \to \mathit{ninf}', \mathit{lt} \to \mathit{lt}', \mathit{eq} \to \mathit{eq}', \mathit{gt} \to \mathit{gt}', \mathit{pinf} \to \mathit{pinf}', \mathit{nan} \to \mathit{nan}'\ \},\\
\qquad\qquad \mathrm{Field}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!],\\
\qquad\qquad\qquad \mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{false}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], +, -, \cdot, /]\!]\\
\qquad\qquad \texttt{where}\ \{\ \mathit{lt} \wedge \mathit{eq} \wedge \mathit{gt} \wedge \neg \mathit{ninf} \wedge \neg \mathit{pinf} \wedge \neg \mathit{nan}, U = \mathrm{dimensionless}\ \},\\
\qquad\qquad \mathrm{Field}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!],\\
\qquad\qquad\qquad \mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{false}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], +, -, \times, /]\!]\\
\qquad\qquad \texttt{where}\ \{\ \mathit{lt} \wedge \mathit{eq} \wedge \mathit{gt} \wedge \neg \mathit{ninf} \wedge \neg \mathit{pinf} \wedge \neg \mathit{nan}, U = \mathrm{dimensionless}\ \},\\
\qquad\qquad \mathrm{Field}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!],\\
\qquad\qquad\qquad \mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{false}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], +, -, \texttt{juxtaposition}, /]\!]\\
\qquad\qquad \texttt{where}\ \{\ \mathit{lt} \wedge \mathit{eq} \wedge \mathit{gt} \wedge \neg \mathit{ninf} \wedge \neg \mathit{pinf} \wedge \neg \mathit{nan}, U = \mathrm{dimensionless}\ \},\\
\qquad\qquad \mathrm{AbelianGroup}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], +, -]\!],\\
\qquad\qquad \mathrm{TotalOrderOperators}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], <, \leq, \geq, >, \texttt{CMP}]\!]\\
\qquad\qquad \texttt{where}\ \{\ \neg \mathit{nan}\ \},\\
\qquad\qquad \mathrm{PartialOrderAndLattice}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], \leq, \texttt{MIN}, \texttt{MAX}]\!]\\
\qquad\qquad \texttt{where}\ \{\ \neg \mathit{nan}\ \},\\
\qquad\qquad \mathrm{PartialOrderAndMeetBoundedLattice}[\![\mathrm{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!],\\
\qquad\qquad\qquad\qquad\qquad\qquad \leq, \texttt{MIN}, \texttt{MAX}]\!]
\end{array}
$$

$\texttt{where}\ \{\, \mathit{pinf} \wedge \neg \mathit{nan} \,\},$
$\text{PartialOrderAndJoinBoundedLattice}[\![\, \text{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], \leq, \texttt{MIN}, \texttt{MAX}]\!]$
$\texttt{where}\ \{\, \mathit{ninf} \wedge \neg \mathit{nan} \,\},$
$\text{PartialOrderAndBoundedLattice}[\![\, \text{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!], \leq, \texttt{MIN}, \texttt{MAX}]\!]$
$\texttt{where}\ \{\, \mathit{ninf} \wedge \mathit{pinf} \wedge \neg \mathit{nan} \,\}\,\}$
$\texttt{where}\ \{\, \mathit{ninf} \vee \mathit{lt} \vee \mathit{eq} \vee \mathit{gt} \vee \mathit{pinf} \vee \mathit{nan} \,\}$
$\texttt{coerce}\ (x{:}\,\text{Identity}[\![+]\!])$
$\texttt{coerce}\ (x{:}\,\text{Identity}[\![\cdot]\!])$
$\texttt{coerce}\ (x{:}\,\text{Identity}[\![\times]\!])$
$\texttt{coerce}\ (x{:}\,\text{Identity}[\![\texttt{juxtaposition}]\!])$
$\texttt{coerce}\ (x{:}\,\text{Zero}[\![\cdot]\!])$
$\texttt{coerce}\ (x{:}\,\text{Zero}[\![\times]\!])$
$\texttt{coerce}\ (x{:}\,\text{Zero}[\![\texttt{juxtaposition}]\!])$
$\texttt{coerce}\ (x{:}\,\text{IntegerQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!])$
$\texttt{opr juxtaposition}\,[\![\texttt{unit}\ U', \texttt{bool}\ \mathit{ninf}', \texttt{bool}\ \mathit{lt}', \texttt{bool}\ \mathit{eq}', \texttt{bool}\ \mathit{gt}', \texttt{bool}\ \mathit{pinf}', \texttt{bool}\ \mathit{nan}']\!]$
$(\texttt{self}, \mathit{other}{:}\,\text{RationalQuantity}[\![U', \mathit{ninf}', \mathit{lt}', \mathit{eq}', \mathit{gt}', \mathit{pinf}', \mathit{nan}']\!){:}$
$\text{RationalQuantity}[\![\, U\ U',$
$\mathit{ninf} \wedge \mathit{pinf}' \vee \mathit{ninf} \wedge \mathit{gt}' \vee \mathit{lt} \wedge \mathit{pinf}' \vee \mathit{pinf} \wedge \mathit{ninf}' \vee \mathit{pinf} \wedge \mathit{lt}' \vee \mathit{gt} \wedge \mathit{ninf}',$
$\mathit{lt} \wedge \mathit{gt}' \vee \mathit{gt} \wedge \mathit{lt}',$
$\mathit{eq} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{eq}',$
$\mathit{lt} \wedge \mathit{lt}' \vee \mathit{gt} \wedge \mathit{gt}',$
$\mathit{ninf} \wedge \mathit{ninf}' \vee \mathit{ninf} \wedge \mathit{lt}' \vee \mathit{lt} \wedge \mathit{ninf}' \vee \mathit{pinf} \wedge \mathit{pinf}' \vee \mathit{pinf} \wedge \mathit{gt}' \vee \mathit{gt} \wedge \mathit{pinf}',$
$\mathit{nan} \vee \mathit{nan}' \vee \mathit{ninf} \wedge \mathit{eq}' \vee \mathit{pinf} \wedge \mathit{eq}' \vee \mathit{eq} \wedge \mathit{ninf}' \vee \mathit{eq} \wedge \mathit{pinf}'\,]\!]$
$\texttt{opr}\ +(\texttt{self}){:}\,\text{RationalQuantity}[\![U, \mathit{ninf}, \mathit{lt}, \mathit{eq}, \mathit{gt}, \mathit{pinf}, \mathit{nan}]\!]$
$\texttt{opr}\ +[\![\texttt{bool}\ \mathit{ninf}', \texttt{bool}\ \mathit{lt}', \texttt{bool}\ \mathit{eq}', \texttt{bool}\ \mathit{gt}', \texttt{bool}\ \mathit{inf}', \texttt{bool}\ \mathit{nan}']\!]$
$(\texttt{self}, \mathit{other}{:}\,\text{RationalQuantity}[\![U, \mathit{ninf}', \mathit{lt}', \mathit{eq}', \mathit{gt}', \mathit{pinf}', \mathit{nan}']\!]){:}$
$\text{RationalQuantity}[\![\, U,$
$\mathit{ninf} \wedge (\mathit{ninf}' \vee \mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{ninf} \vee \mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{ninf}',$
$\mathit{lt} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{lt}',$
$\mathit{lt} \wedge \mathit{gt}' \vee \mathit{eq} \wedge \mathit{eq}' \vee \mathit{gt} \wedge \mathit{lt}',$
$\mathit{gt} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{gt}',$
$\mathit{pinf} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}' \vee \mathit{pinf}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt} \vee \mathit{pinf}) \wedge \mathit{pinf}',$
$\mathit{nan} \vee \mathit{nan}' \vee \mathit{ninf} \wedge \mathit{pinf}' \vee \mathit{pinf} \wedge \mathit{ninf}'\,]\!]$
$\texttt{opr}\ -(\texttt{self}){:}\,\text{RationalQuantity}[\![U, \mathit{pinf}, \mathit{gt}, \mathit{eq}, \mathit{lt}, \mathit{ninf}, \mathit{nan}]\!]$
$\texttt{opr}\ -[\![\texttt{bool}\ \mathit{ninf}', \texttt{bool}\ \mathit{lt}', \texttt{bool}\ \mathit{eq}', \texttt{bool}\ \mathit{gt}', \texttt{bool}\ \mathit{pinf}', \texttt{bool}\ \mathit{nan}']\!]$
$(\texttt{self}, \mathit{other}{:}\,\text{RationalQuantity}[\![U, \mathit{ninf}', \mathit{lt}', \mathit{eq}', \mathit{gt}', \mathit{pinf}', \mathit{nan}']\!]){:}$
$\text{RationalQuantity}[\![\, U,$
$\mathit{ninf} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}' \vee \mathit{pinf}') \vee (\mathit{ninf} \vee \mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{pinf}',$
$\mathit{lt} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{gt}',$
$\mathit{lt} \wedge \mathit{lt}' \vee \mathit{eq} \wedge \mathit{eq}' \vee \mathit{gt} \wedge \mathit{gt}',$
$\mathit{gt} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{lt}',$
$\mathit{pinf} \wedge (\mathit{ninf}' \vee \mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt} \vee \mathit{pinf}) \wedge \mathit{ninf}',$
$\mathit{nan} \vee \mathit{nan}' \vee \mathit{ninf} \wedge \mathit{ninf}' \vee \mathit{pinf} \wedge \mathit{pinf}'\,]\!]$
$\texttt{opr}\ \cdot[\![\texttt{unit}\ U', \texttt{bool}\ \mathit{ninf}', \texttt{bool}\ \mathit{lt}', \texttt{bool}\ \mathit{eq}', \texttt{bool}\ \mathit{gt}', \texttt{bool}\ \mathit{pinf}', \texttt{bool}\ \mathit{nan}']\!]$
$(\texttt{self}, \mathit{other}{:}\,\text{RationalQuantity}[\![U', \mathit{ninf}', \mathit{lt}', \mathit{eq}', \mathit{gt}', \mathit{pinf}', \mathit{nan}']\!]){:}$
$\text{RationalQuantity}[\![\, U\ U',$
$\mathit{ninf} \wedge \mathit{pinf}' \vee \mathit{ninf} \wedge \mathit{gt}' \vee \mathit{lt} \wedge \mathit{pinf}' \vee \mathit{pinf} \wedge \mathit{ninf}' \vee \mathit{pinf} \wedge \mathit{lt}' \vee \mathit{gt} \wedge \mathit{ninf}',$
$\mathit{lt} \wedge \mathit{gt}' \vee \mathit{gt} \wedge \mathit{lt}',$
$\mathit{eq} \wedge (\mathit{lt}' \vee \mathit{eq}' \vee \mathit{gt}') \vee (\mathit{lt} \vee \mathit{eq} \vee \mathit{gt}) \wedge \mathit{eq}',$
$\mathit{lt} \wedge \mathit{lt}' \vee \mathit{gt} \wedge \mathit{gt}',$

$ninf \wedge ninf' \vee ninf \wedge lt' \vee lt \wedge ninf' \vee pinf \wedge pinf' \vee pinf \wedge gt' \vee gt \wedge pinf',$
$nan \vee nan' \vee ninf \wedge eq' \vee pinf \wedge eq' \vee eq \wedge ninf' \vee eq \wedge pinf' ⟧$

`opr` $\times⟦$`unit` U', `bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U', ninf', lt', eq', gt', pinf', nan'⟧$) :
RationalQuantity$⟦ U\ U',$
$ninf \wedge pinf' \vee ninf \wedge gt' \vee lt \wedge pinf' \vee pinf \wedge ninf' \vee pinf \wedge lt' \vee gt \wedge ninf',$
$lt \wedge gt' \vee gt \wedge lt',$
$eq \wedge (lt' \vee eq' \vee gt') \vee (lt \vee eq \vee gt) \wedge eq',$
$lt \wedge lt' \vee gt \wedge gt',$
$ninf \wedge ninf' \vee ninf \wedge lt' \vee lt \wedge ninf' \vee pinf \wedge pinf' \vee pinf \wedge gt' \vee gt \wedge pinf',$
$nan \vee nan' \vee ninf \wedge eq' \vee pinf \wedge eq' \vee eq \wedge ninf' \vee eq \wedge pinf' ⟧$

`opr` /($\texttt{self}$): RationalQuantity$⟦1/U, eq, lt, ninf \vee pinf, gt, eq, nan⟧$

`opr` /$⟦$`unit` U', `bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U', ninf', lt', eq', gt', pinf', nan'⟧$) :
RationalQuantity$⟦ U/U',$
$ninf \wedge eq' \vee ninf \wedge gt' \vee lt \wedge eq' \vee pinf \wedge lt',$
$lt \wedge gt' \vee gt \wedge lt',$
$eq \wedge (lt' \vee gt') \vee (lt \vee eq \vee gt) \wedge (ninf' \vee pinf')',$
$lt \wedge lt' \vee gt \wedge gt',$
$ninf \wedge lt' \vee pinf \wedge eq' \vee pinf \wedge gt' \vee gt \wedge eq',$
$nan \vee nan' \vee (ninf \vee pinf) \wedge (ninf' \vee pinf') \vee eq \wedge eq' ⟧$

`opr`$⟦$`unit` U', `bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *power*: IntegerQuantity$⟦U', ninf', lt', eq', gt', pinf'⟧$) :
RationalQuantity$⟦ U,$
$ninf \wedge gt' \vee eq \wedge lt',$
$lt \wedge (lt' \vee gt'),$
$eq \wedge gt' \vee (ninf \vee pinf) \wedge lt',$
$(lt \vee gt) \wedge (lt' \vee eq' \vee gt'),$
$pinf \wedge gt' \vee eq \wedge lt',$
needs more work here$⟧$
`where` $\{ U = \text{dimensionless}, U' = \text{dimensionless} \}$

`opr` $<⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', nan'⟧$): Boolean

`opr` $\leq⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', nan'⟧$): Boolean

`opr` $=⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', nan'⟧$): Boolean

`opr` $\geq⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', nan'⟧$): Boolean

`opr` $>⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', nan'⟧$): Boolean

`opr CMP`$⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', false⟧$): TotalComparison

`opr CMP`$⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', true⟧$): Comparison

`opr MAX`$⟦$`bool` $ninf'$, `bool` lt', `bool` eq', `bool` gt', `bool` $pinf'$, `bool` $nan'⟧$
($\texttt{self}$, *other*: RationalQuantity$⟦U, ninf', lt', eq', gt', pinf', nan'⟧$) :
RationalQuantity$⟦ U,$
$ninf \wedge ninf',$
$lt \wedge (ninf' \vee lt') \vee (ninf \vee lt) \wedge lt',$
$eq \wedge (ninf' \vee lt' \vee eq') \vee (ninf \vee lt \vee eq) \wedge eq',$
$gt \wedge (ninf' \vee lt' \vee eq' \vee gt') \vee (ninf \vee lt \vee eq \vee gt) \wedge gt',$

$$\begin{array}{l} \quad\quad\quad\

$check_{>}$(self): $\mathbb{Q}_{>}$ throws CastError
$check_{\neq}$(self): $\mathbb{Q}_{\neq}$ throws CastError
$check^{*}_{<}$(self): $\mathbb{Q}^{*}_{<}$ throws CastError
$check^{*}_{\leq}$(self): $\mathbb{Q}^{*}_{\leq}$ throws CastError
$check^{*}_{\geq}$(self): $\mathbb{Q}^{*}_{\geq}$ throws CastError
$check^{*}_{>}$(self): $\mathbb{Q}^{*}_{>}$ throws CastError
$check^{*}_{\neq}$(self): $\mathbb{Q}^{*}_{\neq}$ throws CastError
$check^{\#}_{<}$(self): $\mathbb{Q}^{\#}_{<}$ throws CastError
$check^{\#}_{\leq}$(self): $\mathbb{Q}^{\#}_{\leq}$ throws CastError
$check^{\#}_{\geq}$(self): $\mathbb{Q}^{\#}_{\geq}$ throws CastError
$check^{\#}_{>}$(self): $\mathbb{Q}^{\#}_{>}$ throws CastError
$check^{\#}_{\neq}$(self): $\mathbb{Q}^{\#}_{\neq}$ throws CastError
end

For descriptions of the methods, see Section 33.1.

41.2 The Trait Fortress.Standard.TotalComparison

The comparison operator CMP, when applied to values belonging to a total order, typically returns a value of type TotalComparison. The three values of type TotalComparison are called LessThan, EqualTo, and GreaterThan.

This trait supports an associative operator LEXICO that is useful for supporting lexicographic comparison of ordered sequences; the trick is to compare the sequences elementwise and then to use the LEXICO operator to reduce the sequence of comparison results. Note that EqualTo is the identity for LEXICO, and all other comparison values are left zeroes for LEXICO.

value trait TotalComparison
    extends { Comparison,
              Associative⟦TotalComparison, LEXICO⟧,
              HasIdentity⟦TotalComparison, LEXICO⟧,
              HasLeftZeroes⟦TotalComparison, LEXICO⟧ }
    comprises { LessThan, EqualTo, GreaterThan }
  opr LEXICO(self, *other*: TotalComparison): TotalComparison
  isLeftZero(_: Operator⟦LEXICO⟧): Boolean
  opr ≡(self, *other*: TotalComparison): Boolean
  getter *hashCode*(): ℤ64
  toString(): String
end

LessThan: TotalComparison
EqualTo: TotalComparison
GreaterThan: TotalComparison

41.2.1 `opr LEXICO`(`self`, *other*: TotalComparison): TotalComparison

The operator `LEXICO` returns its right argument if the left argument is EqualTo; otherwise it returns its left argument. The `LEXICO` operator as applied to total comparison values may be described by this table:

`LEXICO`	LessThan	EqualTo	GreaterThan
LessThan	LessThan	LessThan	LessThan
EqualTo	LessThan	EqualTo	GreaterThan
GreaterThan	GreaterThan	GreaterThan	GreaterThan

41.2.2 *isLeftZero*(_: Operator⟦`LEXICO`⟧): Boolean

This method returns *false* for EqualTo and *true* for all other total comparison values.

41.2.3 `opr` ≡(`self`, *other*: TotalComparison): Boolean

Two total comparison values are strictly equivalent if and only if they are the same.

41.2.4 `getter` *hashCode*(): ℤ64

41.2.5 *toString*(): String

The *toString* method returns either "`LessThan`" or "`EqualTo`" or "`GreaterThan`" as appropriate.

41.3 Top-level Total Comparison Values

41.3.1 LessThan: TotalComparison
41.3.2 EqualTo: TotalComparison
41.3.3 GreaterThan: TotalComparison

The immutable variables LessThan, EqualTo, and GreaterThan have as their values the three total comparison values that respectively signify whether a left-hand comparand is less than, equal to, or greater than a right-hand comparand. They are top-level variables declared in the Fortress standard libraries.

41.4 The Trait Fortress.Standard.Comparison

When the comparison operator `CMP` is applied to values belonging to a partial order, rather than a total order, it typically returns a value of type Comparison, which includes the three values LessThan, EqualTo, and GreaterThan of type TotalComparison and also a fourth value, Unordered.

This trait, like trait TotalComparison, supports an associative operator `LEXICO` that is useful for supporting lexicographic comparison of ordered sequences; the trick is to compare the sequences elementwise and then to use the `LEXICO` operator to reduce the sequence of comparison results. Note that EqualTo is the identity for `LEXICO`, and all other comparison values are left zeroes for `LEXICO`.

```
value trait Comparison
    extends { IdentityEquality⟦Comparison⟧,
              Associative⟦Comparison, LEXICO⟧,
              HasIdentity⟦Comparison, LEXICO⟧,
              HasLeftZeroes⟦Comparison, LEXICO⟧ }
    comprises { TotalComparison, Unordered }
  opr LEXICO(self, other: Comparison): Comparison
  isLeftZero(_: Operator⟦LEXICO⟧): Boolean
  opr ≡(self, other: Comparison): Boolean
  getter hashCode(): ℤ64
  toString(): String
end
```

Unordered: Comparison

41.4.1 `opr LEXICO(self, other: Comparison): Comparison`

The operator `LEXICO` returns its right argument if the left argument is EqualTo; otherwise it returns its left argument. The `LEXICO` operator as applied to comparison values may be described by this table:

`LEXICO`	LessThan	EqualTo	GreaterThan	Unordered
LessThan	LessThan	LessThan	LessThan	LessThan
EqualTo	LessThan	EqualTo	GreaterThan	Unordered
GreaterThan	GreaterThan	GreaterThan	GreaterThan	GreaterThan
Unordered	Unordered	Unordered	Unordered	Unordered

41.4.2 *isLeftZero*(_: Operator⟦`LEXICO`⟧): Boolean

This method returns *false* for EqualTo and *true* for all other comparison values.

41.4.3 `opr ≡(self, other: Comparison): Boolean`

Two comparison values are strictly equivalent if and only if they are the same.

41.4.4 `getter` *hashCode*(): $\mathbb{Z}64$

41.4.5 *toString*(): String

The *toString* method returns either "`LessThan`" or "`EqualTo`" or "`GreaterThan`" or "`Unordered`" as appropriate.

41.5 Top-level Comparison Value

41.5.1 Unordered: Comparison

The immutable variable Unordered has as its value the comparison value that signifies that two comparands are unordered. It is a top-level variable declared in the Fortress standard libraries.

Chapter 42

Components and APIs

We define a special Fortress.Components API that provides handles on components and APIs, and operations on them, for use by components themselves (e.g., development environments), allowing components to build and maintain other components, manipulate projects and components as objects, compile projects into components, link components together, deploy components on specific sites over the internet, etc. This API is also used by the Upgradable and Installable APIs. A component implementing this API is installed along with the Fortress standard libraries on every fortress.

Note that Components and Apis can be constructed only from the factory functions provided in the API. The components and APIs so constructed are also installed and accessible via *getComponent*, *preferences* (which returns a list of components implementing a given API, in order of preference), and *getAPI*. Calling *preferences* on an API in the Fortress standard libraries returns a non-empty list of components. In particular, *preferences*(Fortress.Components) returns a non-empty list whose first element is the very component on which the call to *preferences* was made. Conceptually, this component serves as a handle on the enclosing fortress, which might be necessary for the purposes of certain development tools.

The operations on a fortress provided in this API take components and APIs as arguments directly, rather than names of components and APIs as the corresponding shell operations are described. This decision is done for the sake of convenience. Note, however, that a component name may be rebound on a fortress, or even uninstalled, while some process p keeps a reference to a corresponding Component object. This possibility is not problematic because the component corresponding to this object may be simply kept by the fortress until the object is freed in p. Also, note that `upgrade` operations on a compound component are purely functional: they produce new compound components as a result. Thus, the structure of a component as viewed through a Component object does not became stale in the face of upgrades.

We include a method *getSourceFile* on components that returns the source file the component was compiled from. Source files can be included with simple components during compilation as a compiler option. Doing so allows development tools such as graphical debuggers to easily display the locations of errors without the possibility that source code would not be synchronized with compiled code, as can happen in conventional programming models where compiled code is stored in nonencapsulated object files.

```
api Fortress.Components
import File from Fortress.IO
import { List, Set, Date } from Fortress.Util

trait Fortress
  fortressName: Name
  birthDate: Date
  getComponent(componentName: Name): Component
```

```
    getAPI(apiName: Name): Api
    preferences(ofAPI: Api): ⟨Component⟩
    compile(file: File): SimpleComponent
    install(file: File): Component
    install(file: File, prereqs: Set⟦Api⟧): Component
    upgradeAll(componentName: Name, that: Component): ()
    isValidLink(constituents: ⟨Component⟩, exports = Set⟦Api⟧, hide = Set⟦Api⟧): Boolean
    link(result: Name, constituents: ⟨Component⟩, exports = Set⟦Api⟧, hide = Set⟦Api⟧): Component
      requires { isValidLink(constituents, exports, hide) }
end

object EnclosingFortress extends { Fortress } end

trait FortressElement
    elementName: Name
    vendor: String
    owner: Fortress
    timeStamp: Date
    version: Version
    uninstall(): ()
end

trait Component extends FortressElement
    imports: Set⟦Api⟧
    exports: Set⟦Api⟧
    provides: Set⟦Api⟧
    visibles: Set⟦Api⟧
    constituents: Set⟦Component⟧
    run(args: String ...): ()
    constrain(destination: Name, apis: Set⟦Api⟧): Component
    hide(destination: Name, apis: Set⟦Api⟧): Component
    extract(prereqs: Set⟦Api⟧): File
    isValidUpgrade(that: Component): Boolean
    abstract upgrade(result: Name, that: Component): Component
      requires { self.isValidUpgrade(that) }
    sourceIsAvailable: Boolean
    getSourceFile(): File
      requires { sourceIsAvailable }
    runTests(inclusive = Boolean): ()
end

trait Api extends FortressElement
    uses: Set⟦Api⟧
    extraction: File
end

trait Name end

trait SimpleComponent extends Component end

trait Version
    major: ℕ
    minor: ℕ
end

end
```

Chapter 43

Memory Sequences and Binary Words

These are the lowest-level data structures in Fortress, upon which all others are built. Even such conceptually "primitive" data types as $\mathbb{Z}$ and $\mathbb{Z}32$ and $\mathbb{R}64$ are defined in terms of memory sequences and binary words.

Consider, for example, the types $\mathrm{BinaryWord}[\![6]\!]$, $\mathbb{Z}64$, and $\mathbb{N}64$. All three may be regarded as 64-bit data objects. However, $\mathbb{Z}64$ causes the operator $<$ to compare 64-bit words as two's-complement signed integers, $\mathbb{N}64$ causes the operator $<$ to compare 64-bit words as unsigned integers, and $\mathrm{BinaryWord}[\![6]\!]$ does not support the operator $<$ at all—instead it has two methods named $signedLT$ and $unsignedLT$ (which are, of course, conveniently used to implement the operator $<$ for $\mathbb{Z}64$ and $\mathbb{N}64$). Moreover, $\mathbb{Z}64$ and $\mathbb{N}64$ support units and dimensions, but $\mathrm{BinaryWord}$ values do not. The parameterized type $\mathrm{BinaryWord}$ provides methods that are only a modest abstraction of operations supported by typical hardware instruction sets and serves as the lowest-level substrate that allows types such as $\mathbb{Z}64$ to be defined by libraries coded entirely in Fortress.

Similarly, the parameterized traits $\mathrm{LinearSequence}$ and $\mathrm{HeapSequence}$ describe the lowest-level data structures that are array-like or vector-like, capable of little more than one-dimensional indexing. They serve as the lowest-level substrate that allows the complete distributed and multi-dimensional array types to be defined by libraries coded entirely in Fortress.

For convenience, we use the term *binary linear sequence* to refer to a linear sequence of binary words, and the term *binary heap sequence* to refer to a heap sequence of binary words.

$\texttt{type}\ \mathrm{BinaryLinearSequence}[\![\texttt{nat}\ b, \texttt{nat}\ n]\!] = \mathrm{LinearSequence}[\![\mathrm{BinaryWord}[\![b]\!], n]\!]$
$\texttt{type}\ \mathrm{BinaryHeapSequence}[\![\texttt{nat}\ b]\!] = \mathrm{HeapSequence}[\![\mathrm{BinaryWord}[\![b]\!]]\!]$

Most operations on binary words do not depend on *endianness*, that is, in which order the bits are numbered. For operations that do depend on endianness, the parameterized trait $\mathrm{BinaryEndianWord}$ is provided.

It is also sometimes desirable to perform endianness-dependent operations on a linear sequence of binary words. For this purpose the specialized parameterized traits $\mathrm{BinaryLinearEndianSequence}$ and $\mathrm{BinaryEndianLinearEndianSequence}$ are provided; the former has $\mathrm{BinaryWord}$ values as elements, and the latter has $\mathrm{BinaryEndianWord}$ values as elements.

43.1 The Trait Fortress.Core.LinearSequence

A value of type $\text{LinearSequence}[\![T, n]\!]$ is a sequence of n things of type T, where n may be any natural number. Note that its length is statically fixed and may be described by a static expression. The general intent is that such a sequence will reside in a contiguous region of memory, typically belonging to a single processor or processor node, and that any element (indicated by an integer index) may be fetched or updated quickly by that processor or processor node.

If T is not a value type, then $\text{LinearSequence}[\![T, n]\!]$ describes a sequence of references, and a variable of type $\text{LinearSequence}[\![T, n]\!]$ occupies an amount of storage equal to n times the amount of storage required to hold a reference. If T is a value type, then $\text{LinearSequence}[\![T, n]\!]$ describes a sequence of "unboxed" values, and a variable of type $\text{LinearSequence}[\![T, n]\!]$ occupies an amount of storage equal to n times the amount of storage required to hold one value of type T.

Linear sequences, unlike arrays, are not too fancy. The main things you can do with linear sequences are subscripting and subscripted assignment, as well as assignment of entire sequences. They also support the concatenation operator $\|$. For example:

x: LinearSequence[\![Thread, 3]\!]
y: LinearSequence[\![Thread, 6]\!]
z: LinearSequence[\![Thread, 5]\!] $= x \| y[3 \# 2]$

Note in this example that the lengths are statically checkable. The range $3 \# 2$ is a range of constant size 2, so $y[3 \# 2]$ is known to be of type $\text{LinearSequence}[\![\text{Thread}, 2]\!]$. Indeed, only ranges of static size with element type IndexInt may be used to subscript a linear sequence.

```
value trait LinearSequence[\T extends Any, nat n\] comprises { ... }
  coerce [\nat b, bool bigEndianSequence\]
      (x: BinaryLinearEndianSequence[\b, n, bigEndianSequence\])
    where { T extends BinaryWord[\b\] }
  coerce [\nat b, bool bigEndianBytes, bool bigEndianBits, bool bigEndianSequence\]
      (x: BinaryEndianLinearEndianSequence[\b, bigEndianBytes, bigEndianBits,
                                           n, bigEndianSequence\])
    where { T extends BinaryEndianWord[\b, bigEndianBytes, bigEndianBits\] }
  coerce [\nat b, bool bigEndianBytes, bool bigEndianBits, bool bigEndianSequence\]
      (x: BinaryEndianLinearEndianSequence[\b, bigEndianBytes, bigEndianBits,
                                           n, bigEndianSequence\])
    where { T extends BinaryWord[\b\] }
  opr [j: IndexInt]: T throws { IndexOutOfBounds }
  opr [\nat k\][j: IntegerStatic[\k\]]: T where { k < n }
  opr [\nat m\][r: RangeOfStaticSize[\IndexInt, m\]]: LinearSequence[\T, m\]
    throws { IndexOutOfBounds } where { m ≤ n }
  opr [\int a, nat m, int c\][r: StaticRange[\a, m, c\]]: LinearSequence[\T, m\]
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  opr [j: IndexInt] := (v: T): LinearSequence[\T, n\] throws { IndexOutOfBounds }
  opr [\nat k\][j: IntegerStatic[\k\]] := (v: T): LinearSequence[\T, n\] where { k < n }
  opr [\nat m\][r: RangeOfStaticSize[\IndexInt, m\]] := (v: LinearSequence[\T, m\]):
      LinearSequence[\T, n\]
    throws { IndexOutOfBounds } where { m ≤ n }
  opr [\int a, nat m, int c\][r: StaticRange[\a, m, c\]] := (v: LinearSequence[\T, m\]):
      LinearSequence[\T, n\]
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  update(j: IndexInt, v: T): LinearSequence[\T, n\] throws { IndexOutOfBounds }
```

```
  update⟦nat k⟧(j: IntegerStatic⟦k⟧, v: T): LinearSequence⟦T, n⟧ where { k < n }
  update⟦nat m⟧(r: RangeOfStaticSize⟦IndexInt, m⟧, v: LinearSequence⟦T, m⟧):
      LinearSequence⟦T, n⟧
    throws { IndexOutOfBounds } where { m ≤ n }
  update⟦int a, nat m, int c⟧(r: StaticRange⟦a, m, c⟧, v: LinearSequence⟦T, m⟧):
      LinearSequence⟦T, n⟧
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  opr ‖⟦nat m⟧(self, other: LinearSequence⟦T, m⟧): LinearSequence⟦T, n + m⟧
  getter reverse(): LinearSequence⟦T, n⟧
  getter littleEndian⟦nat b⟧(): BinaryLinearEndianSequence⟦b, n, false⟧
    where { T extends BinaryWord⟦b⟧ }
  getter bigEndian⟦nat b⟧(): BinaryLinearEndianSequence⟦b, n, true⟧
    where { T extends BinaryWord⟦b⟧ }
  getter littleEndian⟦nat b, bool bigEndianBytes, bool bigEndianBits⟧():
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits, n, false⟧
    where { T extends BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧ }
  getter bigEndian⟦nat b, bool bigEndianBytes, bool bigEndianBits⟧():
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits, n, true⟧
    where { T extends BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧ }
end
```

43.1.1
```
coerce ⟦nat b, bool bigEndianSequence⟧
    (x: BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧)
  where { T extends BinaryWord⟦b⟧ }
```

43.1.2
```
coerce ⟦nat b, bool bigEndianBytes, bool bigEndianBits, bool bigEndianSequence⟧
    (x: BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                                  n, bigEndianSequence⟧)
  where { T extends BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧ }
```

Any binary linear endian sequence may be coerced to an ordinary binary linear sequence of corresponding element type. The bit values remain the same; all that is lost is the endianness information of the sequence in the static type.

43.1.3
```
coerce ⟦nat b, bool bigEndianBytes, bool bigEndianBits, bool bigEndianSequence⟧
(x: BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
n, bigEndianSequence⟧)
where { T extends BinaryWord⟦b⟧ }
```

A binary linear endian sequence of binary endian words may be coerced to an ordinary binary linear sequence of ordinary binary words. The bit values remain the same; all that is lost is the endianness information of both the sequence and the elements.

43.1.4 `opr` $[j\colon \mathrm{IndexInt}]\colon T$ `throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$
43.1.5 `opr` $[\![\texttt{nat}\ k]\!][j\colon \mathrm{IntegerStatic}[\![k]\!]]\colon T$ `where` $\{\,k < n\,\}$

Subscripting returns element j of this linear sequence. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $0 \le j < n$, where n is the length of the linear sequence. If the subscript is a static expression, then its validity is checked statically, and no exception will occur at run time.

43.1.6 `opr` $[\![\texttt{nat}\ m]\!][r\colon \mathrm{RangeOfStaticSize}[\![\mathrm{IndexInt}, m]\!]]\colon \mathrm{LinearSequence}[\![T, m]\!]$
`throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$ `where` $\{\,m \le n\,\}$
43.1.7 `opr` $[\![\texttt{int}\ a, \texttt{nat}\ m, \texttt{int}\ c]\!][r\colon \mathrm{StaticRange}[\![a, m, c]\!]]\colon \mathrm{LinearSequence}[\![T, m]\!]$
`where` $\{\,0 \le a < n, 0 \le a + m \cdot c < n\,\}$

Subscripting with a range of static size m returns the indicated subsequence of this linear sequence. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $r \subseteq 0 \,\#\, n$, where n is the length of the linear sequence. If the subscript is a static range, then its validity is checked statically, and no exception will occur at run time. Element k of the result sequence is the same as element $r.\mathit{lowerBound} + k \times r.\mathit{stride}$ of this linear sequence, for all $0 \le k < m$.

43.1.8 `opr` $[j\colon \mathrm{IndexInt}] := (v\colon T)\colon \mathrm{LinearSequence}[\![T, n]\!]$ `throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$
43.1.9 `opr` $[\![\texttt{nat}\ k]\!][j\colon \mathrm{IntegerStatic}[\![k]\!]] := (v\colon T)\colon \mathrm{LinearSequence}[\![T, n]\!]$ `where` $\{\,k < n\,\}$

After subscripted value object assignment, element j of the subscripted variable is the same as the given value v, and all other elements are the same as before. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $0 \le j < n$, where n is the length of the linear sequence. If the subscript is a static expression, then its validity is checked statically, and no exception will occur at run time.

43.1.10 `opr` $[\![\texttt{nat}\ m]\!][r\colon \mathrm{RangeOfStaticSize}[\![\mathrm{IndexInt}, m]\!]] := (v\colon \mathrm{LinearSequence}[\![T, m]\!])\colon$
$\mathrm{LinearSequence}[\![T, n]\!]$
`throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$ `where` $\{\,m \le n\,\}$
43.1.11 `opr` $[\![\texttt{int}\ a, \texttt{nat}\ m, \texttt{int}\ c]\!][r\colon \mathrm{StaticRange}[\![a, m, c]\!]] := (v\colon \mathrm{LinearSequence}[\![T, m]\!])\colon$
$\mathrm{LinearSequence}[\![T, n]\!]$
`where` $\{\,0 \le a < n, 0 \le a + m \cdot c < n\,\}$

After subscripted value object assignment, elements of the subscripted variable selected by r are the same as corresponding elements of v, and all other elements are the same as before; specifically, element $r.\mathit{lowerBound} + k \times r.\mathit{stride}$ of the updated variable is the same as element k of v, for all $0 \le k < m$. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $r \subseteq 0 \,\#\, n$, where n is the length of the linear sequence. If the subscript is a static range, then its validity is checked statically, and no exception will occur at run time.

43.1.12 $\mathit{update}(j\colon \mathrm{IndexInt}, v\colon T)\colon \mathrm{LinearSequence}[\![T, n]\!]$ `throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$
43.1.13 $\mathit{update}[\![\texttt{nat}\ k]\!](j\colon \mathrm{IntegerStatic}[\![k]\!], v\colon T)\colon \mathrm{LinearSequence}[\![T, n]\!]$ `where` $\{\,k < n\,\}$

This is a functional version of subscripted value object assignment: element j of the result is the same as the given value v, and all other elements are the same as before. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $0 \le j < n$, where n is the length of the linear sequence. If the subscript is a static expression, then its validity is checked statically, and no exception will occur at run time.

43.1.14 $update[\![\texttt{nat}\ m]\!](r\colon \text{RangeOfStaticSize}[\![\text{IndexInt}, m]\!], v\colon \text{LinearSequence}[\![T, m]\!])$:
$\text{LinearSequence}[\![T, n]\!]$
`throws` { IndexOutOfBounds } `where` $\{\, m \leq n \,\}$

43.1.15 $update[\![\texttt{int}\ a, \texttt{nat}\ m, \texttt{int}\ c]\!](r\colon \text{StaticRange}[\![a, m, c]\!], v\colon \text{LinearSequence}[\![T, m]\!])$:
$\text{LinearSequence}[\![T, n]\!]$
`where` $\{\, 0 \leq a < n, 0 \leq a + m \cdot c < n \,\}$

This is a functional version of subscripted value object assignment: elements of the result selected by r are the same as corresponding elements of v, and all other elements are the same as before; specifically, element $r.lowerBound + k \times r.stride$ of the result is the same as element k of v, for all $0 \leq k < m$. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $0 \leq j < n$, where n is the length of the linear sequence. If the subscript is a static range, then its validity is checked statically, and no exception will occur at run time.

43.1.16 `opr` $\|\, [\![\texttt{nat}\ m]\!](\texttt{self}, other\colon \text{LinearSequence}[\![T, m]\!])\colon \text{LinearSequence}[\![T, n + m]\!]$

The result is a linear sequence whose length is equal to the sum of the lengths of this linear sequence and the other linear sequence. Element k of the result is the same as element k of this linear sequence if $0 \leq k < n$, and is the same as element $k - n$ of the other linear sequence if $n \leq k < n + m$.

43.1.17 `getter` $reverse()\colon \text{LinearSequence}[\![T, n]\!]$

The result is a linear sequence such that element k of the result is the same as element $n - 1 - k$ of this linear sequence, for all $0 \leq k < n$.

43.1.18 `getter` $littleEndian[\![\texttt{nat}\ b]\!]()\colon \text{BinaryLinearEndianSequence}[\![b, n, false]\!]$
`where` $\{\, T$ `extends` $\text{BinaryWord}[\![b]\!] \,\}$

43.1.19 `getter` $bigEndian[\![\texttt{nat}\ b]\!]()\colon \text{BinaryLinearEndianSequence}[\![b, n, true]\!]$
`where` $\{\, T$ `extends` $\text{BinaryWord}[\![b]\!] \,\}$

43.1.20 `getter` $littleEndian[\![\texttt{nat}\ b, \texttt{bool}\ bigEndianBytes, \texttt{bool}\ bigEndianBits]\!]()$:
$\text{BinaryEndianLinearEndianSequence}[\![b, bigEndianBytes, bigEndianBits, n, false]\!]$
`where` $\{\, T$ `extends` $\text{BinaryEndianWord}[\![b, bigEndianBytes, bigEndianBits]\!] \,\}$

43.1.21 `getter` $bigEndian[\![\texttt{nat}\ b, \texttt{bool}\ bigEndianBytes, \texttt{bool}\ bigEndianBits]\!]()$:
$\text{BinaryEndianLinearEndianSequence}[\![b, bigEndianBytes, bigEndianBits, n, true]\!]$
`where` $\{\, T$ `extends` $\text{BinaryEndianWord}[\![b, bigEndianBytes, bigEndianBits]\!] \,\}$

These conversion getters allow a linear sequence of (possibly endian) binary words to be treated as a specifically little-endian or specifically big-endian linear sequence. This is especially useful just before invoking an endianness-dependent method, for example $s.littleEndian.countLeadingZeroBits()$.

43.2 Constructing Linear Sequences

43.2.1 *makeLinearSequence*⟦T `extends` Any, `nat` n⟧($item$: T): LinearSequence⟦T, n⟧

A new linear sequence of length n is returned. Every element of the linear sequence is initialized to be the same as the given $item$.

43.2.2 *computeLinearSequence*⟦T `extends` Any, `nat` n⟧(f: IndexInt → T): LinearSequence⟦T, n⟧

A new linear sequence of length n is returned. Element j of the new linear sequence is initialized to a value computed by calling the given function f with argument j.

43.3 The Trait Fortress.Core.HeapSequence

A value of type HeapSequence⟦T⟧ is an array-like object that contains items of type T. The length of a heap sequence is in general not known statically, but can be discovered by asking for its length. A variable of type HeapSequence⟦T⟧ occupies the amount of storage required to hold a reference; this reference refers to an object that occupies an amount of storage greater than or equal to the amount that would be occupied by a variable of type LinearSequence⟦T, n⟧ where n is the length of the heap sequence.

Heap sequences, like linear sequences and unlike arrays, are not too fancy. The main things you can do with heap sequences are subscripting and subscripted assignment. Concatenation is *not* supported, because a basic principle of the low-level types is that none of the operations, other than explicit construction of a heap sequence, does any heap allocation. However, a range of static size may be used to index a heap sequence; the result is a linear sequence.

```
trait HeapSequence⟦T extends Any⟧ comprises { ... }
  opr [j: IndexInt]: T throws { IndexOutOfBounds }
  opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧]: LinearSequence⟦T, m⟧
    throws { IndexOutOfBounds }
  opr [j : IndexInt] := (v: T): () throws { IndexOutOfBounds }
  opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] := (v: LinearSequence⟦T, m⟧): ()
    throws { IndexOutOfBounds }
  reverse(selfStart: IndexInt, length: IndexInt): () throws { IndexOutOfBounds }
  opr |self| : IndexInt
end
```

43.3.1 `opr` [j: IndexInt]: T `throws` { IndexOutOfBounds }

Subscripting returns element j of this heap sequence. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $0 \le j < n$, where n is the length of the heap sequence.

43.3.2 `opr` $[\![$`nat` $m]\!][r\colon \mathrm{RangeOfStaticSize}[\![\mathrm{IndexInt}, m]\!]]\colon \mathrm{LinearSequence}[\![T, m]\!]$
`throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$

Subscripting by a range returns the indicated subsequence of this heap sequence. The range must be a range of static size, and the result is returned as a linear sequence (not a heap sequence), so no heap allocation is performed. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $r \subseteq (0 : n-1)$, where n is the length of the heap sequence. Element k of the result linear sequence is the same as element $r.lowerBound + k \times r.stride$ of this heap sequence, for all $0 \le k < m$.

43.3.3 `opr` $[j : \mathrm{IndexInt}] := (v\colon T)\colon ()$ `throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$

After subscripted assignment, element j of this heap sequence is the same as the given value v, and all other elements are the same as before. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $0 \le j < n$, where n is the length of the heap sequence.

43.3.4 `opr` $[\![$`nat` $m]\!][r\colon \mathrm{RangeOfStaticSize}[\![\mathrm{IndexInt}, m]\!]] := (v\colon \mathrm{LinearSequence}[\![T, m]\!])\colon ()$
`throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$

After subscripted assignment using a range as a subscript, elements of the subscripted variable selected by r are the same as corresponding elements of v, and all other elements are the same as before; specifically, element $r.lowerBound + k \times r.stride$ of the updated variable is the same as element k of v, for all $0 \le k < m$. The range must be a range of static size, and the values to be assigned must be passed as linear sequence of the same size. Indexing is zero-origin; an $\mathrm{IndexOutOfBounds}$ is thrown unless $r \subseteq (0 : n-1)$, where n is the length of the heap sequence.

43.3.5 $\mathit{reverse}(\mathit{selfStart}\colon \mathrm{IndexInt}, \mathit{length}\colon \mathrm{IndexInt})\colon ()$ `throws` $\{\,\mathrm{IndexOutOfBounds}\,\}$

Elements $\mathit{selfStart}$ through $\mathit{selfStart} + \mathit{length} - 1$, inclusive, are reversed in order, that is, rearranged so that the value originally stored at element $\mathit{selfStart} + j$ becomes element $\mathit{selfStart} + \mathit{length} - 1 - j$, for all $0 \le j < \mathit{length}$. Other elements of this heap sequence are unaffected.

43.3.6 `opr` $|$`self`$|$: $\mathrm{IndexInt}$

The length of this heap sequence is returned. Note that the size of a heap sequence is specified at run time when the heap sequence is created; once a heap sequence has been created, its length does not change.

43.4 Constructing Heap Sequences

43.4.1 *makeHeapSequence*⟦T `extends` Any⟧(n: IndexInt, *item*: T): HeapSequence⟦T⟧
`throws` { NegativeLength }

A new heap sequence of length n is allocated and returned. A NegativeLength is thrown if $n < 0$. Every element of the heap sequence is initialized to be the same as the given *item*.

43.4.2 *computeHeapSequence*⟦T `extends` Any⟧(n: IndexInt, f: IndexInt → T): HeapSequence⟦T⟧
`throws` { NegativeLength }

A new heap sequence of length n is allocated and returned. A NegativeLength is thrown if $n < 0$. Element j of the new heap sequence is initialized to a value computed by calling the given function f with argument j.

43.5 The Trait Fortress.Core.BinaryWord

A value of type $\mathrm{BinaryWord}[\![b]\!]$ is a binary word of 2^b bits; b may be any natural number, so $\mathrm{BinaryWord}[\![0]\!]$ is a bit, $\mathrm{BinaryWord}[\![3]\!]$ is a byte, $\mathrm{BinaryWord}[\![6]\!]$ is a 64-bit word, and $\mathrm{BinaryWord}[\![10]\!]$ is a 1024-bit word. In fact, for convenience, the type abbreviations Bit and Byte are defined:

`type` $\mathrm{Bit} = \mathrm{BinaryWord}[\![0]\!]$
`type` $\mathrm{Byte} = \mathrm{BinaryWord}[\![3]\!]$

The type $\mathrm{BinaryWord}[\![b]\!]$ has $2^{(2^b)}$ distinct values. When the binary word is regarded as an unsigned integer, these values are identified with the nonnegative integers that are less than $2^{(2^b)}$. A binary word may also be regarded as a signed integer: a value that, when regarded as an unsigned integer, is identified with an integer less than $2^{(2^{b-1})}$, is identified with that same integer when regarded as a signed integer; but a value that, when regarded as an unsigned integer, is identified with an integer not less than $2^{(2^{b-1})}$, is identified with that same integer less $2^{(2^b)}$. (This is the standard "two's complement" representation for signed integers.)

A binary word of one bit can have one of two values, 0 or 1. A binary word of more than one bit has two halves, a high half and a low half, which are binary words of half the size. If v is the unsigned integer value of a binary word of 2^b bits, $b \geq 1$, h is the unsigned integer value of its high half, and l is the unsigned integer value of its low half, then $v = h \cdot 2^{2^{b-1}} + l$.

Operations on binary words include bitwise boolean operations, arithmetic operations, shifts and rotates, population count, and counting of leading and trailing zeros. The type $\mathrm{BinaryWord}[\![b]\!]$ is not "endian" and has no operations that depend on endianness. However, if w is binary word, then $w.\mathit{littleEndian}$ is a little-endian version of w and $w.\mathit{bigEndian}$ is a big-endian version of w; for example, if w is of type $\mathrm{BinaryWord}[\![6]\!]$, then $w.\mathit{littleEndian}_{63}$ is the most significant bit (the sign bit if the word is regarded as a two's-complement integer), and $w.\mathit{bigEndian}_0$ is that same bit.

```
trait BinaryWord⟦nat b⟧ extends { BasicBinaryWordOperations⟦BinaryWord⟦b⟧⟧ }
      comprises { ... }
      where { b ≤ maxBinaryWordBitLog }
   coerce ⟦int r⟧(x: IntegerStatic⟦r⟧) where { −2^(b−1) ≤ r < 2^b }
   coerce ⟦bool bigEndianBytes, bool bigEndianBits⟧
        (x: BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧)
   coerce ⟦nat b′, nat n, bool bigEndianSequence⟧
        (x: BinaryLinearEndianSequence⟦b′, n, bigEndianSequence⟧)
      where { 2^b = n · 2^b′ }
   coerce ⟦nat b′, bool bigEndianBytes, bool bigEndianBits, nat n, bool bigEndianSequence⟧
        (x: BinaryEndianLinearEndianSequence⟦b′, bigEndianBytes, bigEndianBits,
                                              n, bigEndianSequence⟧)
      where { 2^b = n · 2^b′ }
   bit(m: IndexInt): Bit
   getter lowHalf(): BinaryWord⟦b − 1⟧ where { b > 0 }
   getter highHalf(): BinaryWord⟦b − 1⟧ where { b > 0 }
   opr ‖⟦nat m⟧(self, other: BinaryWord⟦b⟧): BinaryWord⟦b + 1⟧
      where { b < maxBinaryWordBitLog }
   bitShuffle(other: BinaryWord⟦b⟧): BinaryWord⟦b + 1⟧
      where { b < maxBinaryWordBitLog }
   bitUnshuffle(): (BinaryWord⟦b − 1⟧, BinaryWord⟦b − 1⟧) where { b > 0 }
end
```

43.5.1 `coerce` $[\![\texttt{int}\ r]\!](x\colon \mathrm{IntegerStatic}[\![r]\!])$ `where` $\{\ -2^{b-1} \leq r < 2^b\ \}$

An static integer may be coerced to a binary word that corresponds to that integer value when interpreted as either a signed integer or an unsigned integer. For example, the type $\mathrm{BinaryWord}[\![3]\!]$ has $2^{(2^3)} = 256$ distinct binary word values; when they are interpreted as signed integers, the integer values range from -128 to 127, and when they are interpreted as unsigned integers, the integer values range from 0 to 255. Therefore any static integer from -128 to 255 may be coerced to type $\mathrm{BinaryWord}[\![3]\!]$.

43.5.2 `coerce` $[\![\texttt{bool}\ \mathit{bigEndianBytes}, \texttt{bool}\ \mathit{bigEndianBits}]\!]$
$(x\colon \mathrm{BinaryEndianWord}[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!])$

Any binary endian word may be coerced to a plain binary word of the same size and value. In effect, this coercion merely discards the endianness information.

43.5.3 `coerce` $[\![\texttt{nat}\ b', \texttt{nat}\ n, \texttt{bool}\ \mathit{bigEndianSequence}]\!]$
$(x\colon \mathrm{BinaryLinearEndianSequence}[\![b', n, \mathit{bigEndianSequence}]\!])$
`where` $\{\ 2^b = n \cdot 2^{b'}\ \}$

A binary linear endian sequence of smaller binary words may be coerced to a single binary word, provided that the length of the linear sequence is an appropriate power of two, so that the total number of bits in the sequence is the same as the total number of bits in the resulting binary word. The manner in which the elements of the sequence are used to form the new binary word value respects the endianness of the sequence, so that element 0 of the sequence supplies the most significant bits of the result if $\mathit{bigEndianSequence}$ is true, but supplies the least significant bits of the result if $\mathit{bigEndianSequence}$ is false.

43.5.4 `coerce` $[\![\texttt{nat}\ b', \texttt{bool}\ \mathit{bigEndianBytes}, \texttt{bool}\ \mathit{bigEndianBits}, \texttt{nat}\ n, \texttt{bool}\ \mathit{bigEndianSequence}]\!]$
$(x\colon \mathrm{BinaryEndianLinearEndianSequence}[\![b', \mathit{bigEndianBytes}, \mathit{bigEndianBits},$
$n, \mathit{bigEndianSequence}]\!])$
`where` $\{\ 2^b = n \cdot 2^{b'}\ \}$

A binary endian linear endian sequence of smaller binary endian words may be coerced to a single binary word, provided that the length of the linear sequence is an appropriate power of two, so that the total number of bits in the sequence is the same as the total number of bits in the resulting binary word. The manner in which the elements of the sequence are used to form the new binary word value respects the endianness of the sequence, so that element 0 of the sequence supplies the most significant bits of the result if $\mathit{bigEndianSequence}$ is true, but supplies the least significant bits of the result if $\mathit{bigEndianSequence}$ is false. In effect, the "bytes and bits" endianness information is simply ignored and discarded.

43.5.5 $bit(m\colon \mathrm{IndexInt})\colon \mathrm{Bit}$

The result is a bit whose value (0 or 1) is equal to $\lfloor v \cdot 2^{-m} \rfloor \bmod 2$ where v is the value of the binary word regarded as an unsigned integer. This formula holds for any value of m; note that if m is negative or greater than $2^b - 1$, the result will always be a 0-bit. Thus the bit method provides a kind of "little-endian" indexing of the bits of a binary word, even a binary word whose type is not intrinsically endian, but it does not require that the bit number identify an actual represented bit of the binary word.

The bit method is particularly useful for describing the behavioral properties of other methods of binary data.

43.5.6 `getter` $lowHalf()\colon \mathrm{BinaryWord}[\![b-1]\!]$ `where` $\{\, b > 0 \,\}$
43.5.7 `getter` $highHalf()\colon \mathrm{BinaryWord}[\![b-1]\!]$ `where` $\{\, b > 0 \,\}$

The getters $lowHalf$ and $highHalf$ each return a binary word of half the size (in bits) of this binary word; $lowHalf$ returns the less significant bits, and $highHalf$ returns the more significant bits.

`property` $\forall(v) \bigwedge_{m \leftarrow 0 \# 2^{b-1}} v.lowHalf.bit(m) = v.bit(m)$
`property` $\forall(v) \bigwedge_{m \leftarrow 0 \# 2^{b-1}} v.highHalf.bit(m) = v.bit(m + 2^{b-1})$

43.5.8 `opr` $\|\, [\![\texttt{nat}\ m]\!](\texttt{self}, other\colon \mathrm{BinaryWord}[\![b]\!])\colon \mathrm{BinaryWord}[\![b+1]\!]$
`where` $\{\, b < maxBinaryWordBitLog \,\}$

The result of concatenating two binary words of size 2^b is a single binary word of size 2^{b+1}. The left-hand operand becomes the high (more significant) half of the result and the right-hand operand becomes the low (less significant) half of the result.

`property` $\forall(v, w)\ (v \,\|\, w).highHalf = v$
`property` $\forall(v, w)\ (v \,\|\, w).lowHalf = w$

43.5.9 $bitShuffle(other\colon \mathrm{BinaryWord}[\![b]\!])\colon \mathrm{BinaryWord}[\![b+1]\!]$
`where` $\{\, b < maxBinaryWordBitLog \,\}$

The bit-shuffle operation interleaves the bits of two words, as if shuffling cards together (using what magicians call a "perfect shuffle"). The result of shuffling the bits two binary words of size 2^b is a single binary word of size 2^{b+1}. This binary word provides the odd-numbered bits of the result and the other binary word provides the even-numbered bits of the result. For example, shuffling `1111` and `0000` produces `10101010`.

`property` $\forall(v, w, m\colon \mathrm{IndexInt})\ v.bitShuffle(w).bit(m) = (\texttt{if}\ odd\ m\ \texttt{then}\ v.bit((m-1)/2)\ \texttt{else}\ w.bit(m/2))$

43.5.10 $bitUnshuffle()\colon (\mathrm{BinaryWord}[\![b-1]\!], \mathrm{BinaryWord}[\![b-1]\!])$ `where` $\{\, b > 0 \,\}$

This is the inverse of the *bitShuffle* method: the odd-numbered bits of this binary word are used to form a binary word of half the size, and likewise the even-numbered bits, and a tuple of the two binary words is returned.

`property` $\forall (v, w)\ v.bitShuffle(w).bitUnshuffle() = (v, w)$

43.6 The Trait Fortress.Core.BinaryEndianWord

The type BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧ is exactly like BinaryWord⟦b⟧ but bears two kinds of endianness information. A BinaryEndianWord may be split into a sequence of smaller words; the result is of type BinaryLinearEndianSequence. The flag *bigEndianBytes* indicates whether subword 0 is the most significant subword (if *bigEndianBytes* is true) or least significant subword (if *bigEndianBytes* is false) of the original binary word. A BinaryEndianWord may also be subscripted to extract a bit or a bit field; the flag *bigEndianBits* indicates whether bit 0 is the most significant bit (if *bigEndianBits* is true) or least significant bit (if *bigEndianBits* is false) of the original binary word. (Yes, it may seem strange for the bit ordering to differ from the subword ordering, but they do differ on a number of architectures, including SPARC.) Extracting a bit produces a Bit, that is, a BinaryWord⟦0⟧. Extracting a bit field of width k produces a BinaryEndianLinearEndianSequence with $n = k$ and $b = 0$; the endianness of the sequence matches the bit-endianness of the original BinaryEndianWord.

```
trait BinaryEndianWord⟦nat b, bool bigEndianBytes, bool bigEndianBits⟧
      extends { BasicBinaryWordOperations⟦BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧, b⟧}
      comprises { ... }
      where { b ≤ maxBinaryWordBitLog }
   coerce ⟦int r⟧(x: IntegerStatic⟦r⟧) where { -2^(b-1) ≤ r < 2^b }
   opr [j: IndexInt] : BinaryEndianWord⟦1, bigEndianBytes, bigEndianBits⟧
      throws { IndexOutOfBounds }
   opr ⟦nat k⟧[j: IntegerStatic⟦k⟧] : BinaryEndianWord⟦1, bigEndianBytes, bigEndianBits⟧
      where { k < 2^b }
   opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :
        BinaryEndianLinearEndianSequence⟦1, bigEndianBytes, bigEndianBits, m, bigEndianBits⟧
      throws { IndexOutOfBounds }
   opr ⟦int a, nat m, int c⟧[r: StaticRange⟦a, m, c⟧] :
        BinaryEndianLinearEndianSequence⟦1, bigEndianBytes, bigEndianBits, m, bigEndianBits⟧
      where { 0 ≤ a < 2^b, 0 ≤ a + m · c < 2^b }
   opr [j: IndexInt] := (v: Bit):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      throws { IndexOutOfBounds }
   opr ⟦nat k⟧[j: IntegerStatic⟦k⟧] := (v: Bit):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      where { k < 2^b }
   opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] := (v: BinaryLinearSequence⟦1, m⟧):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      throws { IndexOutOfBounds }
   opr ⟦int a, nat m, int c⟧[r: StaticRange⟦a, m, c⟧] := (v: BinaryLinearSequence⟦1, k⟧):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      where { 0 ≤ a < 2^b, 0 ≤ a + m · c < 2^b }
   update(j: IndexInt, v: Bit):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      throws { IndexOutOfBounds }
   update⟦nat k⟧(j: IntegerStatic⟦k⟧, v: Bit):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      where { k < 2^b }
   update⟦nat m⟧(r: RangeOfStaticSize⟦IndexInt, m⟧, v: BinaryLinearSequence⟦1, m⟧):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      throws { IndexOutOfBounds }
   update⟦int a, nat m, int c⟧(r: StaticRange⟦a, m, c⟧, v: BinaryLinearSequence⟦1, m⟧):
        BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
      where { 0 ≤ a < 2^b, 0 ≤ a + m · c < 2^b }
```

$\mathit{lowHalf}\,()$: BinaryEndianWord$[\![b-1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$ `where` $\{\, b > 0 \,\}$
$\mathit{highHalf}\,()$: BinaryEndianWord$[\![b-1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$ `where` $\{\, b > 0 \,\}$
`opr` $\|\,[\![\texttt{nat}\ m, \texttt{bool}\ \mathit{bigEndianSequence}]\!]$
$(\texttt{self}, \mathit{other}$: BinaryEndianWord$[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!])$:
BinaryEndianLinearEndianSequence$[\![b+1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}, 2, \mathit{bigEndianSequence}]\!]$
`where` $\{\, \mathit{bigEndianSequence} = \mathit{bigEndianBytes} \,\}$
`opr` $\|\,[\![\texttt{nat}\ m, \texttt{bool}\ \mathit{bigEndianSequence}, \texttt{nat}\ \mathit{radix}, \texttt{nat}\ q, \texttt{nat}\ k, \texttt{nat}\ v]\!]$
$(\texttt{self}, \mathit{other}$: NaturalNumeral$[\![m, \mathit{radix}, v]\!])$:
BinaryEndianLinearEndianSequence$[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}, k+1, \mathit{bigEndianSequence}]\!]$
`where` $\{\, \mathit{bigEndianSequence} = \mathit{bigEndianBytes}, \mathit{radix} = 2^q, q \cdot m = k \cdot 2^b \,\}$
`opr` $\|\,[\![\texttt{nat}\ m, \texttt{bool}\ \mathit{bigEndianSequence}, \texttt{nat}\ \mathit{radix}, \texttt{nat}\ q, \texttt{nat}\ k, \texttt{nat}\ v]\!]$
$(\mathit{other}$: NaturalNumeral$[\![m, \mathit{radix}, v]\!], \texttt{self})$:
BinaryEndianLinearEndianSequence$[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}, k+1, \mathit{bigEndianSequence}]\!]$
`where` $\{\, \mathit{bigEndianSequence} = \mathit{bigEndianBytes}, \mathit{radix} = 2^q, q \cdot m = k \cdot 2^b \,\}$
$\mathit{bitShuffle}(\mathit{other}$: BinaryEndianWord$[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!])$:
BinaryEndianWord$[\![b+1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$
`where` $\{\, b < \mathit{maxBinaryWordBitLog} \,\}$
$\mathit{bitUnshuffle}()$: (BinaryEndianWord$[\![b-1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$,
BinaryEndianWord$[\![b-1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$)
`where` $\{\, b > 0 \,\}$
$\mathit{littleEndian}()$: BinaryEndianWord$[\![b, \mathit{false}, \mathit{false}]\!]$
$\mathit{bigEndian}()$: BinaryEndianWord$[\![b, \mathit{true}, \mathit{true}]\!]$
`end`

43.6.1 `coerce` $[\![\texttt{int}\ r]\!](x$: IntegerStatic$[\![r]\!])$ `where` $\{\, -2^{b-1} \le r < 2^b \,\}$

An static integer may be coerced to a binary endian word exactly as if it were coerced to a plain binary word of the same size; the endian numbering of the bytes and bits does not affect which binary word value is produced from the static integer.

43.6.2 `opr` $[j$: IndexInt$]$: BinaryEndianWord$[\![1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$ `throws` { IndexOutOfBounds }

43.6.3 `opr` $[\![\texttt{nat}\ k]\!][j$: IntegerStatic$[\![k]\!]]$: BinaryEndianWord$[\![1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$ `where` $\{\, k < 2^b \,\}$

Subscripting returns bit j of this binary endian word. The numbering of the bits is dictated by $\mathit{bigEndianBits}$. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $0 \le j < 2^b$. If the subscript is a static expression, then its validity is checked statically, and no exception will occur at run time.

`property` $\forall(v) \bigwedge_{j \leftarrow 0 \# 2^b} v_j = (\texttt{if}\ \mathit{bigEndianBits}\ \texttt{then}\ v.\mathit{bit}(2^b - 1 - j)\ \texttt{else}\ v.\mathit{bit}(j)\ \texttt{end})$

43.6.4 `opr` ⟦`nat` m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :
BinaryEndianLinearEndianSequence⟦1, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianBits*⟧
`throws` { IndexOutOfBounds }

43.6.5 `opr` ⟦`int` a, `nat` m, `int` c⟧[r: StaticRange⟦a, m, c⟧] :
BinaryEndianLinearEndianSequence⟦1, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianBits*⟧
`where` { $0 \le a < 2^b, 0 \le a + m \cdot c < 2^b$ }

Subscripting with a range of static size m returns the indicated subsequence of bits of this binary endian word. The numbering of the bits is dictated by *bigEndianBits*. The result is a binary endian linear endian sequence of bits whose sequence endianness is the same as the bit endianness of this binary endian word. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $r \subseteq 0 \# 2^b$. If the subscript is a static range, then its validity is checked statically, and no exception will occur at run time. Element k of the result sequence is the same as the bit that would be selected from this binary endian word by subscripting it with $r.lowerBound + k \times r.stride$, for all $0 \le k < m$.

43.6.6 `opr` [j: IndexInt] := (v: Bit):
BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧
`throws` { IndexOutOfBounds }

43.6.7 `opr` ⟦`nat` k⟧[j: IntegerStatic⟦k⟧] := (v: Bit):
BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧
`where` { $k < 2^b$ }

After subscripted value object assignment, the bit that would be selected from this binary endian word by subscripting it with j is the same as the given bit v, and all other bits are the same as before. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $0 \le j < 2^b$. If the subscript is a static expression, then its validity is checked statically, and no exception will occur at run time.

43.6.8 `opr` ⟦`nat` m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] := (v: BinaryLinearSequence⟦1, m⟧):
BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧
`throws` { IndexOutOfBounds }

43.6.9 `opr` ⟦`int` a, `nat` m, `int` c⟧[r: StaticRange⟦a, m, c⟧] := (v: BinaryLinearSequence⟦1, m⟧):
BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧
`where` { $0 \le a < 2^b, 0 \le a + m \cdot c < 2^b$ }

After subscripted value object assignment, bits that would be selected from this binary endian word by subscripting it with r are the same as corresponding elements of v, and all other bits are the same as before; specifically, the bit that would be selected from this binary endian word by subscripting it with $r.lowerBound + k \times r.stride$ is the same as element k of v, for all $0 \le k < m$. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $r \subseteq 0 \# 2^b$. If the subscript is a static range, then its validity is checked statically, and no exception will occur at run time.

43.6.10 *update*(*j*: IndexInt, *v*: Bit):
BinaryEndianWord⟦*b*, *bigEndianBytes*, *bigEndianBits*⟧
`throws` { IndexOutOfBounds }

43.6.11 *update*⟦`nat` *k*⟧(*j*: IntegerStatic⟦*k*⟧, *v*: Bit):
BinaryEndianWord⟦*b*, *bigEndianBytes*, *bigEndianBits*⟧
`where` { $k < 2^b$ }

This is a functional version of subscripted value object assignment: the bit that would be selected from the result by subscripting it with j is the same as the given bit v, and all other bits are the same as before. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $0 \le j < 2^b$. If the subscript is a static expression, then its validity is checked statically, and no exception will occur at run time.

43.6.12 *update*⟦`nat` *m*⟧(*r*: RangeOfStaticSize⟦IndexInt, *m*⟧, *v*: BinaryLinearSequence⟦1, *m*⟧):
BinaryEndianWord⟦*b*, *bigEndianBytes*, *bigEndianBits*⟧
`throws` { IndexOutOfBounds }

43.6.13 *update*⟦`int` *a*, `nat` *m*, `int` *c*⟧(*r*: StaticRange⟦*a*, *m*, *c*⟧, *v*: BinaryLinearSequence⟦1, *m*⟧):
BinaryEndianWord⟦*b*, *bigEndianBytes*, *bigEndianBits*⟧
`where` { $0 \le a < 2^b, 0 \le a + m \cdot c < 2^b$ }

This is a functional version of subscripted value object assignment: bits that would be selected from the result by subscripting it with r are the same as corresponding elements of v, and all other bits are the same as before; specifically, the bit that would be selected from the result by subscripting it with $r.lowerBound + k \times r.stride$ is the same as element k of v, for all $0 \le k < m$. Indexing is zero-origin; an IndexOutOfBounds is thrown unless $r \subseteq 0 \# 2^b$. If the subscript is a static range, then its validity is checked statically, and no exception will occur at run time.

43.6.14 *lowHalf*(): BinaryEndianWord⟦$b - 1$, *bigEndianBytes*, *bigEndianBits*⟧ `where` { $b > 0$ }
43.6.15 *highHalf*(): BinaryEndianWord⟦$b - 1$, *bigEndianBytes*, *bigEndianBits*⟧ `where` { $b > 0$ }

The getters *lowHalf* and *highHalf* each return a binary endian word of half the size (in bits) of this binary endian word, and with the same endian characteristics; *lowHalf* returns the less significant bits, and *highHalf* returns the more significant bits.

`property` $\forall(v) \quad \bigwedge_{m \leftarrow 0 \# 2^{b-1}} v.lowHalf.bit(m) = v.bit(m)$
`property` $\forall(v) \quad \bigwedge_{m \leftarrow 0 \# 2^{b-1}} v.highHalf.bit(m) = v.bit(m + 2^{b-1})$
`property` $\forall(v) \quad \bigwedge_{m \leftarrow 0 \# 2^{b-1}} v.lowHalf_m = v[\texttt{if } bigEndianBits \texttt{ then } m + 2^{b-1} \texttt{ else } m \texttt{ end}]$
`property` $\forall(v) \quad \bigwedge_{m \leftarrow 0 \# 2^{b-1}} v.highHalf_m = v[\texttt{if } bigEndianBits \texttt{ then } m \texttt{ else } m + 2^{b-1} \texttt{ end}]$

43.6.16 `opr` ‖ ⟦`nat` *m*, `bool` *bigEndianSequence*⟧
(`self`, *other*: BinaryEndianWord⟦*b*, *bigEndianBytes*, *bigEndianBits*⟧):
BinaryEndianLinearEndianSequence⟦$b + 1$, *bigEndianBytes*, *bigEndianBits*,
2, *bigEndianSequence*⟧
`where` { *bigEndianSequence* = *bigEndianBytes* }

[Description to be supplied.]

43.6.17 `opr` $\|[\![\texttt{nat}\ m, \texttt{bool}\ \mathit{bigEndianSequence}, \texttt{nat}\ \mathit{radix}, \texttt{nat}\ q, \texttt{nat}\ k, \texttt{nat}\ v]\!]$
$(\texttt{self}, \mathit{other}\colon \text{NaturalNumeral}[\![m, \mathit{radix}, v]\!])$:
$\text{BinaryEndianLinearEndianSequence}[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}, k+1, \mathit{bigEndianSequence}]\!]$
`where` $\{\ \mathit{bigEndianSequence} = \mathit{bigEndianBytes}, \mathit{radix} = 2^q, q \cdot m = k \cdot 2^b\ \}$

[Description to be supplied.]

43.6.18 `opr` $\|[\![\texttt{nat}\ m, \texttt{bool}\ \mathit{bigEndianSequence}, \texttt{nat}\ \mathit{radix}, \texttt{nat}\ q, \texttt{nat}\ k, \texttt{nat}\ v]\!]$
$(\mathit{other}\colon \text{NaturalNumeral}[\![m, \mathit{radix}, v]\!], \texttt{self})$:
$\text{BinaryEndianLinearEndianSequence}[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}, k+1, \mathit{bigEndianSequence}]\!]$
`where` $\{\ \mathit{bigEndianSequence} = \mathit{bigEndianBytes}, \mathit{radix} = 2^q, q \cdot m = k \cdot 2^b\ \}$

[Description to be supplied.]

43.6.19 $\mathit{bitShuffle}(\mathit{other}\colon \text{BinaryEndianWord}[\![b, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!])$:
$\text{BinaryEndianWord}[\![b+1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!]$
`where` $\{\ b < \mathit{maxBinaryWordBitLog}\ \}$

[Description to be supplied.]

43.6.20 $\mathit{bitUnshuffle}()\colon (\text{BinaryEndianWord}[\![b-1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!],$
$\text{BinaryEndianWord}[\![b-1, \mathit{bigEndianBytes}, \mathit{bigEndianBits}]\!])$
`where` $\{\ b > 0\ \}$

[Description to be supplied.]

43.6.21 $\mathit{littleEndian}()\colon \text{BinaryEndianWord}[\![b, \mathit{false}, \mathit{false}]\!]$
43.6.22 $\mathit{bigEndian}()\colon \text{BinaryEndianWord}[\![b, \mathit{true}, \mathit{true}]\!]$

[Description to be supplied.]

43.7 The Trait Fortress.Core.BasicBinaryOperations

`trait` BasicBinaryOperations⟦T `extends` BasicBinaryOperations⟦T⟧⟧
- *wrappingAdd*(*other*: T): T
- *add*(*other*: T, *carryIn*: Bit = 0): (T, Bit)
- *signedAdd*(*other*: T, *overflowAction*: () → T): T
- *unsignedAdd*(*other*: T, *overflowAction*: () → T): T
- *saturatingSignedAdd*(*other*: T): T
- *saturatingUnsignedAdd*(*other*: T): T
- *wrappingSubtract*(*other*: T): T
- *subtract*(*other*: T, *carryIn*: Bit = 1): (T, Bit)
- *signedSubtract*(*other*: T, *overflowAction*: () → T): T
- *unsignedSubtract*(*other*: T, *overflowAction*: () → T): T
- *saturatingSignedSubtract*(*other*: T): T
- *saturatingUnsignedSubtract*(*other*: T): T
- *wrappingNegate*(): T
- *negate*(*carryIn*: Bit = 1): (T, Bit)
- *signedNegate*(*overflowAction*: () → T): T
- *unsignedNegate*(*overflowAction*: () → T): T
- *saturatingSignedNegate*(): T
- *bitNot*(): T
- *bitAnd*(*other*: T): T
- *bitOr*(*other*: T): T
- *bitXor*(*other*: T): T
- *bitXorNot*(*other*: T): T
- *bitNand*(*other*: T): T
- *bitNor*(*other*: T): T
- *bitAndNot*(*other*: T): T
- *bitOrNot*(*other*: T): T
- *signedMax*(*other*: T): T
- *signedMin*(*other*: T): T
- *unsignedMax*(*other*: T): T
- *unsignedMin*(*other*: T): T
- `opr` =(`self`, *other*: T): Boolean
- `opr` ≠(`self`, *other*: T): Boolean
- *signedLT*(*other*: T): Boolean
- *signedLE*(*other*: T): Boolean
- *signedGE*(*other*: T): Boolean
- *signedGT*(*other*: T): Boolean
- *unsignedLT*(*other*: T): Boolean
- *unsignedLE*(*other*: T): Boolean
- *unsignedGE*(*other*: T): Boolean
- *unsignedGT*(*other*: T): Boolean
- *signedShift*(*j*: IndexInt): T
- *signedShift*(*j*: IndexInt, *overflowAction*: () → T): T
- *saturatingSignedShift*(*j*: IndexInt): T
- *unsignedShift*(*j*: IndexInt): T
- *unsignedShift*(*j*: IndexInt, *overflowAction*: () → T): T
- *saturatingUnsignedShift*(*j*: IndexInt): T
- *bitRotate*(*j*: IndexInt): T
- *countOneBits*(): IndexInt

countLeadingZeroBits(): IndexInt
countTrailingZeroBits(): IndexInt
leftmostOneBit(): T
rightmostOneBit(): T
bitReverse(): T
signedIndex(): IndexInt `throws` { IntegerOverflow }
unsignedIndex(): IndexInt `throws` { IntegerOverflow }
gatherBits(*mask*: T): T
spreadBits(*mask*: T): T
disentangleBits(*mask*: T): T
intersperseBits(*mask*: T): T
`end`

43.7.1 *wrappingAdd*(*other*: T): T
43.7.2 *add*(*other*: T, *carryIn*: Bit $= 0$): (T, Bit)
43.7.3 *signedAdd*(*other*: T, *overflowAction*: $() \to T$): T
43.7.4 *unsignedAdd*(*other*: T, *overflowAction*: $() \to T$): T
43.7.5 *saturatingSignedAdd*(*other*: T): T
43.7.6 *saturatingUnsignedAdd*(*other*: T): T

[Description to be supplied.]

43.7.7 *wrappingSubtract*(*other*: T): T
43.7.8 *subtract*(*other*: T, *carryIn*: Bit $= 1$): (T, Bit)
43.7.9 *signedSubtract*(*other*: T, *overflowAction*: $() \to T$): T
43.7.10 *unsignedSubtract*(*other*: T, *overflowAction*: $() \to T$): T
43.7.11 *saturatingSignedSubtract*(*other*: T): T
43.7.12 *saturatingUnsignedSubtract*(*other*: T): T

[Description to be supplied.]

43.7.13 *wrappingNegate*(): T
43.7.14 *negate*(*carryIn*: Bit $= 1$): (T, Bit)
43.7.15 *signedNegate*(*overflowAction*: $() \to T$): T
43.7.16 *unsignedNegate*(*overflowAction*: $() \to T$): T
43.7.17 *saturatingSignedNegate*(): T

[Description to be supplied.]

43.7.18 *bitNot*(): T

[Description to be supplied.]

43.7.19 $bitAnd(other\colon T)\colon T$
43.7.20 $bitOr(other\colon T)\colon T$
43.7.21 $bitXor(other\colon T)\colon T$
43.7.22 $bitXorNot(other\colon T)\colon T$
43.7.23 $bitNand(other\colon T)\colon T$
43.7.24 $bitNor(other\colon T)\colon T$
43.7.25 $bitAndNot(other\colon T)\colon T$
43.7.26 $bitOrNot(other\colon T)\colon T$

[Description to be supplied.]

43.7.27 $signedMax(other\colon T)\colon T$
43.7.28 $signedMin(other\colon T)\colon T$

[Description to be supplied.]

43.7.29 $unsignedMax(other\colon T)\colon T$
43.7.30 $unsignedMin(other\colon T)\colon T$

[Description to be supplied.]

43.7.31 `opr` $=$(`self`, $other\colon T)\colon \mathrm{Boolean}$
43.7.32 `opr` $\neq$(`self`, $other\colon T)\colon \mathrm{Boolean}$

[Description to be supplied.]

43.7.33 $signedLT(other\colon T)\colon \mathrm{Boolean}$
43.7.34 $signedLE(other\colon T)\colon \mathrm{Boolean}$
43.7.35 $signedGE(other\colon T)\colon \mathrm{Boolean}$
43.7.36 $signedGT(other\colon T)\colon \mathrm{Boolean}$

[Description to be supplied.]

43.7.37 $unsignedLT(other\colon T)\colon \mathrm{Boolean}$
43.7.38 $unsignedLE(other\colon T)\colon \mathrm{Boolean}$
43.7.39 $unsignedGE(other\colon T)\colon \mathrm{Boolean}$
43.7.40 $unsignedGT(other\colon T)\colon \mathrm{Boolean}$

[Description to be supplied.]

43.7.41 *signedShift*(*j*: IndexInt): *T*
43.7.42 *signedShift*(*j*: IndexInt, *overflowAction*: () → *T*): *T*
43.7.43 *saturatingSignedShift*(*j*: IndexInt): *T*

[Description to be supplied.]

43.7.44 *unsignedShift*(*j*: IndexInt): *T*
43.7.45 *unsignedShift*(*j*: IndexInt, *overflowAction*: () → *T*): *T*
43.7.46 *saturatingUnsignedShift*(*j*: IndexInt): *T*

[Description to be supplied.]

43.7.47 *bitRotate*(*j*: IndexInt): *T*

[Description to be supplied.]

43.7.48 *countOneBits*(): IndexInt

[Description to be supplied.]

43.7.49 *countLeadingZeroBits*(): IndexInt
43.7.50 *countTrailingZeroBits*(): IndexInt

[Description to be supplied.]

43.7.51 *leftmostOneBit*(): *T*
43.7.52 *rightmostOneBit*(): *T*

[Description to be supplied.]

43.7.53 *bitReverse*(): *T*

[Description to be supplied.]

43.7.54 *signedIndex*(): IndexInt `throws` { IntegerOverflow }
43.7.55 *unsignedIndex*(): IndexInt `throws` { IntegerOverflow }

[Description to be supplied.]

43.7.56 *gatherBits*(*mask*: *T*): *T*
43.7.57 *spreadBits*(*mask*: *T*): *T*

[Description to be supplied.]

43.7.58 *disentangleBits*(*mask*: *T*): *T*
43.7.59 *intersperseBits*(*mask*: *T*): *T*

[Description to be supplied.]

43.8 The Trait Fortress.Core.BasicBinaryWordOperations

```
trait BasicBinaryWordOperations⟦T extends BasicBinaryWordOperations⟦T, b⟧, nat b⟧
      extends BasicBinaryOperations⟦T⟧ where { b ≤ maxBinaryWordBitLog }
   multiplyLow(other: T): T where { b ≤ maxMultiplyBitLog }
   multiplyLow(other: T, overflowAction: () → T): T where { b ≤ maxMultiplyBitLog }
   saturatedMultiplyLow(other: T): T where { b ≤ maxMultiplyBitLog }
   multiplyHigh(other: T): T where { b ≤ maxMultiplyBitLog }
   multiplyDouble(other: T): (T, T) where { b ≤ maxMultiplyBitLog }
   signedDivide(other: T, overflowAction: () → T, zeroDivideAction: () → T): T
      where { b ≤ maxDivideBitLog }
   unsignedDivide(other: T, zeroDivideAction: () → T): T where { b ≤ maxDivideBitLog }
   signedDivRem(other: T, overflowAction: () → T, zeroDivideAction: () → T): (T, T)
      where { b ≤ maxDivideBitLog }
   unsignedDivRem(other: T, zeroDivideAction: () → T): (T, T) where { b ≤ maxDivideBitLog }
   signedRemainder(other: T, zeroDivideAction: () → T): T where { b ≤ maxDivideBitLog }
   unsignedModulo(other: T, zeroDivideAction: () → T): T where { b ≤ maxDivideBitLog }
   bitSwap(j: IndexInt): T
   getter littleEndian(): BinaryEndianWord⟦b, false, false⟧
   getter bigEndian(): BinaryEndianWord⟦b, true, true⟧
end
```

43.8.1 *multiplyLow*(*other*: *T*): *T* `where` { *b* ≤ *maxMultiplyBitLog* }
43.8.2 *multiplyLow*(*other*: *T*, *overflowAction*: () → *T*): *T* `where` { *b* ≤ *maxMultiplyBitLog* }
43.8.3 *saturatedMultiplyLow*(*other*: *T*): *T* `where` { *b* ≤ *maxMultiplyBitLog* }
43.8.4 *multiplyHigh*(*other*: *T*): *T* `where` { *b* ≤ *maxMultiplyBitLog* }
43.8.5 *multiplyDouble*(*other*: *T*): (*T*, *T*) `where` { *b* ≤ *maxMultiplyBitLog* }

[Description to be supplied.]

43.8.6 *signedDivide*(*other*: T, *overflowAction*: () $\to T$, *zeroDivideAction*: () $\to T$): T
`where` { $b \leq$ *maxDivideBitLog* }
43.8.7 *unsignedDivide*(*other*: T, *zeroDivideAction*: () $\to T$): T `where` { $b \leq$ *maxDivideBitLog* }

[Description to be supplied.]

43.8.8 *signedDivRem*(*other*: T, *overflowAction*: () $\to T$, *zeroDivideAction*: () $\to T$): (T, T)
`where` { $b \leq$ *maxDivideBitLog* }
43.8.9 *unsignedDivRem*(*other*: T, *zeroDivideAction*: () $\to T$): (T, T) `where` { $b \leq$ *maxDivideBitLog* }

[Description to be supplied.]

43.8.10 *signedRemainder*(*other*: T, *zeroDivideAction*: () $\to T$): T `where` { $b \leq$ *maxDivideBitLog* }
43.8.11 *unsignedModulo*(*other*: T, *zeroDivideAction*: () $\to T$): T `where` { $b \leq$ *maxDivideBitLog* }

[Description to be supplied.]

43.8.12 *bitSwap*(j: IndexInt): T

[Description to be supplied.]

43.8.13 `getter` *littleEndian*(): BinaryEndianWord⟦b, *false*, *false*⟧
43.8.14 `getter` *bigEndian*(): BinaryEndianWord⟦b, *true*, *true*⟧

[Description to be supplied.]

43.9 The Trait Fortress.Core.BinaryLinearEndianSequence

```
trait BinaryLinearEndianSequence⟦nat b, nat n, bool bigEndianSequence⟧
          extends { BasicBinaryOperations⟦BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧⟧ }
    where { b ≤ maxBinaryWordBitLog }
  coerce ⟦nat b′, bool bigEndianBytes, bool bigEndianBits⟧
      (x: BinaryEndianWord⟦b′, bigEndianBytes, bigEndianBits⟧)
    where { bigEndianBytes = bigEndianSequence, 2^b′ = n · 2^b }
  coerce ⟦nat m, nat radix, nat q, nat k, nat v⟧(x: NaturalNumeral⟦m, radix, v⟧)
    where { radix = 2^q, q · m = n · 2^bk }
  opr [j: IndexInt] : BinaryWord⟦b⟧
    throws { IndexOutOfBounds }
  opr ⟦nat k⟧[j: IntegerStatic⟦k⟧] : BinaryWord⟦b⟧ where { k < n }
  opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :
      BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧
    throws { IndexOutOfBounds } where { m ≤ n }
  opr ⟦int a, nat m, int c⟧[r: StaticRange⟦a, m, c⟧] :
      BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  opr [j: IndexInt] := (v: BinaryWord⟦b⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  opr ⟦nat k⟧[j: IntegerStatic⟦k⟧] := (v: BinaryWord⟦b⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧ where { k < n }
  opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :=
      (v: BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  opr ⟦int a, nat m, int c⟧[r: StaticRange⟦a, m, c⟧] :=
      (v: BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  update(j: IndexInt, v: BinaryWord⟦b⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  update⟦nat k⟧(j: IntegerStatic⟦k⟧, v: BinaryWord⟦b⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧ where { k < n }
  update⟦nat m⟧(r: RangeOfStaticSize⟦IndexInt, m⟧,
                v: BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  update⟦int a, nat m, int c⟧(r: StaticRange⟦a, m, c⟧,
                              v: BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧):
      BinaryLinearEndianSequence⟦b, n, bigEndianSequence⟧
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  opr ‖ ⟦nat m⟧(self, other: BinaryLinearEndianSequence⟦b, m, bigEndianSequence⟧):
      BinaryLinearEndianSequence⟦b, n + m, bigEndianSequence⟧
  opr ‖ ⟦nat m, nat radix, nat q, nat k, nat v⟧(self, other: NaturalNumeral⟦m, radix, v⟧):
      LinearSequence⟦BinaryWord⟦b⟧, n + k⟧ where { radix = 2^q, q · m = k · 2^b }
  opr ‖ ⟦nat m, nat radix, nat q, nat k, nat v⟧(other: NaturalNumeral⟦m, radix, v⟧, self):
      LinearSequence⟦BinaryWord⟦b⟧, n + k⟧ where { radix = 2^q, q · m = k · 2^b }
```

littleEndian(): BinaryEndianLinearEndianSequence⟦b, *false*, *false*, n, *bigEndianSequence*⟧
bigEndian(): BinaryEndianLinearEndianSequence⟦b, *true*, *true*, n, *bigEndianSequence*⟧
littleEndianBits(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *false*, n, *bigEndianSequence*⟧
bigEndianBits(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *true*, n, *bigEndianSequence*⟧
littleEndianSequence(): BinaryLinearEndianSequence⟦b, n, *false*⟧
bigEndianSequence(): BinaryLinearEndianSequence⟦b, n, *true*⟧
split⟦`nat` b'⟧(): BinaryLinearEndianSequence⟦b', $n \cdot 2^{b-b'}$, *bigEndianSequence*⟧
`where` { $b' \leq b$ }
`end`

43.9.1 `coerce` ⟦`nat` b', `bool` *bigEndianBytes*, `bool` *bigEndianBits*⟧ (x: BinaryEndianWord⟦b', *bigEndianBytes*, *bigEndianBits*⟧) `where` { *bigEndianBytes* = *bigEndianSequence*, $2^{b'} = n \cdot 2^b$ }

[Description to be supplied.]

43.9.2 `coerce` ⟦`nat` m, `nat` *radix*, `nat` q, `nat` k, `nat` v⟧(x: NaturalNumeral⟦m, *radix*, v⟧) `where` { *radix* = 2^q, $q \cdot m = n \cdot 2^{bk}$ }

[Description to be supplied.]

43.9.3 `opr` [j: IndexInt] : BinaryWord⟦b⟧ `throws` { IndexOutOfBounds }
43.9.4 `opr` ⟦`nat` k⟧[j: IntegerStatic⟦k⟧] : BinaryWord⟦b⟧ `where` { $k < n$ }

[Description to be supplied.]

43.9.5 `opr` ⟦`nat` m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] : BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧ `throws` { IndexOutOfBounds } `where` { $m \leq n$ }
43.9.6 `opr` ⟦`int` a, `nat` m, `int` c⟧[r: StaticRange⟦a, m, c⟧] : BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧ `where` { $0 \leq a < n, 0 \leq a + m \cdot c < n$ }

[Description to be supplied.]

43.9.7 `opr` [j: IndexInt] := (v: BinaryWord⟦b⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.9.8 `opr` ⟦`nat` k⟧[j: IntegerStatic⟦k⟧] := (v: BinaryWord⟦b⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧ `where` { $k < n$ }

[Description to be supplied.]

43.9.9 `opr` ⟦`nat` m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :=
(v: BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.9.10 `opr` ⟦`int` a, `nat` m, `int` c⟧[r: StaticRange⟦a, m, c⟧] :=
(v: BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧
`where` { $0 \le a < n, 0 \le a + m \cdot c < n$ }

[Description to be supplied.]

43.9.11 *update*(j: IndexInt, v: BinaryWord⟦b⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.9.12 *update*⟦`nat` k⟧(j: IntegerStatic⟦k⟧, v: BinaryWord⟦b⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧ `where` { $k < n$ }

[Description to be supplied.]

43.9.13 *update*⟦`nat` m⟧(r: RangeOfStaticSize⟦IndexInt, m⟧,
v: BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.9.14 *update*⟦`int` a, `nat` m, `int` c⟧(r: StaticRange⟦a, m, c⟧,
v: BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧):
BinaryLinearEndianSequence⟦b, n, *bigEndianSequence*⟧
`where` { $0 \le a < n, 0 \le a + m \cdot c < n$ }

[Description to be supplied.]

43.9.15 `opr` || ⟦`nat` m⟧(`self`, *other*: BinaryLinearEndianSequence⟦b, m, *bigEndianSequence*⟧):
BinaryLinearEndianSequence⟦b, $n + m$, *bigEndianSequence*⟧

[Description to be supplied.]

43.9.16 `opr` $\|[\![\texttt{nat}\ m, \texttt{nat}\ \mathit{radix}, \texttt{nat}\ q, \texttt{nat}\ k, \texttt{nat}\ v]\!](\texttt{self}, \mathit{other}\colon \mathrm{NaturalNumeral}[\![m, \mathit{radix}, v]\!])\colon$ $\mathrm{LinearSequence}[\![T, n + k]\!]$ `where` $\{\ \mathit{radix} = 2^q, q \cdot m = k \cdot 2^b\ \}$

[Description to be supplied.]

43.9.17 `opr` $\|[\![\texttt{nat}\ m, \texttt{nat}\ \mathit{radix}, \texttt{nat}\ q, \texttt{nat}\ k, \texttt{nat}\ v]\!](\mathit{other}\colon \mathrm{NaturalNumeral}[\![m, \mathit{radix}, v]\!], \texttt{self})\colon$ $\mathrm{LinearSequence}[\![T, n + k]\!]$ `where` $\{\ \mathit{radix} = 2^q, q \cdot m = k \cdot 2^b\ \}$

[Description to be supplied.]

43.9.18 $\mathit{littleEndian}()\colon \mathrm{BinaryEndianLinearEndianSequence}[\![b, \mathit{false}, \mathit{false}, n, \mathit{bigEndianSequence}]\!]$
43.9.19 $\mathit{bigEndian}()\colon \mathrm{BinaryEndianLinearEndianSequence}[\![b, \mathit{true}, \mathit{true}, n, \mathit{bigEndianSequence}]\!]$

[Description to be supplied.]

43.9.20 $\mathit{littleEndianBits}()\colon \mathrm{BinaryEndianLinearEndianSequence}[\![b, \mathit{bigEndianBytes}, \mathit{false}, n, \mathit{bigEndianSequence}]\!]$
43.9.21 $\mathit{bigEndianBits}()\colon \mathrm{BinaryEndianLinearEndianSequence}[\![b, \mathit{bigEndianBytes}, \mathit{true}, n, \mathit{bigEndianSequence}]\!]$

[Description to be supplied.]

43.9.22 $\mathit{littleEndianSequence}()\colon \mathrm{BinaryLinearEndianSequence}[\![b, n, \mathit{false}]\!]$
43.9.23 $\mathit{bigEndianSequence}()\colon \mathrm{BinaryLinearEndianSequence}[\![b, n, \mathit{true}]\!]$

[Description to be supplied.]

43.9.24 $\mathit{split}[\![\texttt{nat}\ b']\!]()\colon \mathrm{BinaryLinearEndianSequence}[\![b', n \cdot 2^{b-b'}, \mathit{bigEndianSequence}]\!]$ `where` $\{\ b' \leq b\ \}$

[Description to be supplied.]

43.10 The Trait Fortress.Core.BinaryEndianLinearEndianSequence

```
trait BinaryEndianLinearEndianSequence⟦nat b, bool bigEndianBytes, bool bigEndianBits,
                                       nat n, bool bigEndianSequence⟧
    extends { BasicBinaryOperations⟦
                  BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                                   n, bigEndianSequence⟧⟧ }
    where { b ≤ maxBinaryWordBitLog }
  coerce ⟦nat b′, bool bigEndianBytes, bool bigEndianBits⟧
      (x: BinaryEndianWord⟦b′, bigEndianBytes, bigEndianBits⟧)
    where { bigEndianBytes = bigEndianSequence, 2^b′ = n · 2^b }
  coerce ⟦nat m, nat radix, nat q, nat k, nat v⟧(x: NaturalNumeral⟦m, radix, v⟧)
    where { radix = 2^q, q · m = n · 2^bk }
  opr [j: IndexInt] : BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧
    throws { IndexOutOfBounds }
  opr ⟦nat k⟧[j: IntegerStatic⟦k⟧] : BinaryWord⟦b, bigEndianBytes, bigEndianBits⟧
    where { k < n }
  opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       m, bigEndianSequence⟧
    throws { IndexOutOfBounds } where { m ≤ n }
  opr ⟦int a, nat m, int c⟧[r: StaticRange⟦a, m, c⟧] :
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       m, bigEndianSequence⟧
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  opr [j: IndexInt] := (v: BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧):
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  opr ⟦nat k⟧[j: IntegerStatic⟦k⟧] :=
      (v: BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧):
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       n, bigEndianSequence⟧
    where { k < n }
  opr ⟦nat m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :=
      (v: BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                         m, bigEndianSequence⟧):
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  opr ⟦int a, nat m, int c⟧[r: StaticRange⟦a, m, c⟧] :=
      (v: BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                         k, bigEndianSequence⟧):
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       n, bigEndianSequence⟧
    where { 0 ≤ a < n, 0 ≤ a + m · c < n }
  update(j: IndexInt, v: BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧):
      BinaryEndianLinearEndianSequence⟦b, bigEndianBytes, bigEndianBits,
                                       n, bigEndianSequence⟧
    throws { IndexOutOfBounds }
  update⟦nat k⟧(j: IntegerStatic⟦k⟧,
```

v: BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`where` { $k < n$ }
update⟦`nat` m⟧(r: RangeOfStaticSize⟦IndexInt, m⟧, v: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }
update⟦`int` a, `nat` m, `int` c⟧(r: StaticRange⟦a, m, c⟧, v: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`where` { $0 \le a < n, 0 \le a + m \cdot c < n$ }
`opr` ‖ ⟦`nat` m⟧(`self`, *other*: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, $n + m$, *bigEndianSequence*⟧
`opr` ‖ ⟦`nat` m, `nat` *radix*, `nat` q, `nat` k, `nat` v⟧(`self`, *other*: NaturalNumeral⟦m, *radix*, v⟧):
LinearSequence⟦$T, n + k$⟧
`where` { $radix = 2^q, q \cdot m = k \cdot 2^b$ }
`opr` ‖ ⟦`nat` m, `nat` *radix*, `nat` q, `nat` k, `nat` v⟧(*other*: NaturalNumeral⟦m, *radix*, v⟧, `self`):
LinearSequence⟦$T, n + k$⟧
`where` { $radix = 2^q, q \cdot m = k \cdot 2^b$ }
littleEndian(): BinaryEndianLinearEndianSequence⟦b, *false*, *false*, n, *bigEndianSequence*⟧
bigEndian(): BinaryEndianLinearEndianSequence⟦b, *true*, *true*, n, *bigEndianSequence*⟧
littleEndianBits(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *false*, n, *bigEndianSequence*⟧
bigEndianBits(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *true*, n, *bigEndianSequence*⟧
littleEndianSequence(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *false*⟧
bigEndianSequence(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *true*⟧
split⟦`nat` b'⟧() :
BinaryEndianLinearEndianSequence⟦b', *bigEndianBytes*, *bigEndianBits*, $n \cdot 2^{b-b'}$, *bigEndianSequence*⟧
`where` { $b' \le b$ }
`end`

43.10.1 `coerce` ⟦`nat` b', `bool` *bigEndianBytes*, `bool` *bigEndianBits*⟧ (x: BinaryEndianWord⟦b', *bigEndianBytes*, *bigEndianBits*⟧) `where` { $bigEndianBytes = bigEndianSequence, 2^{b'} = n \cdot 2^b$ }

[Description to be supplied.]

43.10.2 `coerce` ⟦`nat` m, `nat` $radix$, `nat` q, `nat` k, `nat` v⟧(x: NaturalNumeral⟦m, $radix$, v⟧)
`where` { $radix = 2^q, q \cdot m = n \cdot 2^{bk}$ }

[Description to be supplied.]

43.10.3 `opr` [j: IndexInt] : BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧
`throws` { IndexOutOfBounds }

43.10.4 `opr` ⟦`nat` k⟧[j: IntegerStatic⟦k⟧] : BinaryWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧
`where` { $k < n$ }

[Description to be supplied.]

43.10.5 `opr` ⟦`nat` m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds } `where` { $m \leq n$ }

43.10.6 `opr` ⟦`int` a, `nat` m, `int` c⟧[r: StaticRange⟦a, m, c⟧] :
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧
`where` { $0 \leq a < n, 0 \leq a + m \cdot c < n$ }

[Description to be supplied.]

43.10.7 `opr` [j: IndexInt] := (v: BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.10.8 `opr` ⟦`nat` k⟧[j: IntegerStatic⟦k⟧] :=
(v: BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`where` { $k < n$ }

[Description to be supplied.]

43.10.9 `opr` ⟦`nat` m⟧[r: RangeOfStaticSize⟦IndexInt, m⟧] :=
(v: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.10.10 `opr` ⟦`int` a, `nat` m, `int` c⟧[r: StaticRange⟦a, m, c⟧] :=
(v: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, k, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`where` { $0 \le a < n, 0 \le a + m \cdot c < n$ }

[Description to be supplied.]

43.10.11 *update*(j: IndexInt, v: BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.10.12 *update*⟦`nat` k⟧(j: IntegerStatic⟦k⟧,
v: BinaryEndianWord⟦b, *bigEndianBytes*, *bigEndianBits*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`where` { $k < n$ }

[Description to be supplied.]

43.10.13 *update*⟦`nat` m⟧(r: RangeOfStaticSize⟦IndexInt, m⟧,
v: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`throws` { IndexOutOfBounds }

43.10.14 *update*⟦`int` a, `nat` m, `int` c⟧
(r: StaticRange⟦a, m, c⟧,
v: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧):
BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *bigEndianSequence*⟧
`where` { $0 \le a < n, 0 \le a + m \cdot c < n$ }

[Description to be supplied.]

43.10.15 `opr` ‖ ⟦`nat` m⟧(`self`, *other*: BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, m, *bigEndianSequence*⟧): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, $n + m$, *bigEndianSequence*⟧

[Description to be supplied.]

43.10.16 `opr` ‖ ⟦`nat` m, `nat` *radix*, `nat` q, `nat` k, `nat` v⟧(`self`, *other*: NaturalNumeral⟦m, *radix*, v⟧): LinearSequence⟦T, $n + k$⟧ `where` { $radix = 2^q, q \cdot m = k \cdot 2^b$ }

[Description to be supplied.]

43.10.17 `opr` ‖ ⟦`nat` m, `nat` *radix*, `nat` q, `nat` k, `nat` v⟧(*other*: NaturalNumeral⟦m, *radix*, v⟧, `self`): LinearSequence⟦T, $n + k$⟧ `where` { $radix = 2^q, q \cdot m = k \cdot 2^b$ }

[Description to be supplied.]

43.10.18 *littleEndian*(): BinaryEndianLinearEndianSequence⟦b, *false*, *false*, n, *bigEndianSequence*⟧
43.10.19 *bigEndian*(): BinaryEndianLinearEndianSequence⟦b, *true*, *true*, n, *bigEndianSequence*⟧

[Description to be supplied.]

43.10.20 *littleEndianBits*(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *false*, n, *bigEndianSequence*⟧
43.10.21 *bigEndianBits*(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *true*, n, *bigEndianSequence*⟧

[Description to be supplied.]

43.10.22 *littleEndianSequence*(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *false*⟧
43.10.23 *bigEndianSequence*(): BinaryEndianLinearEndianSequence⟦b, *bigEndianBytes*, *bigEndianBits*, n, *true*⟧

[Description to be supplied.]

43.10.24 *split*⟦`nat` b'⟧() :
BinaryEndianLinearEndianSequence⟦b', *bigEndianBytes*, *bigEndianBits*, $n \cdot 2^{b-b'}$, *bigEndianSequence*⟧
`where` { $b' \leq b$ }

[Description to be supplied.]

43.11 The Trait Fortress.Core.BinaryHeapEndianSequence

```
trait BinaryHeapEndianSequence⟦nat b, bool bigEndianSequence⟧
    extends { BinaryHeapSequence⟦b⟧,
              BasicBinaryHeapSubsequenceOperations⟦
                BinaryHeapEndianSequence⟦b, bigEndianSequence⟧,
                bigEndianSequence⟧ }
end
```

43.12 The Trait Fortress.Core.BinaryEndianHeapEndianSequence

```
trait BinaryEndianHeapEndianSequence⟦nat b, bool bigEndianBytes, bool bigEndianBits,
                                     bool bigEndianSequence⟧
    extends { HeapSequence⟦BinaryEndianWord⟦b, bigEndianBytes, bigEndianBits⟧⟧,
              BasicBinaryHeapSubsequenceOperations⟦
                BinaryHeapEndianSequence⟦b, bigEndianSequence⟧,
                bigEndianSequence⟧ }
end
```

43.13 The Trait Fortress.Core.BasicBinaryHeapSubsequenceOperations

```
trait BasicBinaryHeapSubsequenceOperations⟦
      T extends BasicBinaryHeapSubsequenceOperations⟦T, bigEndianSequence⟧,
      bool bigEndianSequence⟧
  copy(selfStart: IndexInt, source: T, sourceStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  wrappingAdd(selfStart: IndexInt, source: T, sourceStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  add(selfStart: IndexInt, source: T, sourceStart: IndexInt,
      length: IndexInt, carryIn: Bit = 0): Bit
    throws { IndexOutOfBounds }
  signedAdd(selfStart: IndexInt, source: T, sourceStart: IndexInt,
            length: IndexInt, overflowAction: () → ()): ()
    throws { IndexOutOfBounds }
  unsignedAdd(selfStart: IndexInt, source: T, sourceStart: IndexInt,
              length: IndexInt, overflowAction: () → ()): ()
    throws { IndexOutOfBounds }
  saturatingSignedAdd(selfStart: IndexInt, source: T, sourceStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  saturatingUnsignedAdd(selfStart: IndexInt, source: T, sourceStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  wrappingSubtract(selfStart: IndexInt, source: T, sourceStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  subtract(selfStart: IndexInt, source: T, sourceStart: IndexInt,
           length: IndexInt, carryIn: Bit = 1): Bit
    throws { IndexOutOfBounds }
  signedSubtract(selfStart: IndexInt, source: T, sourceStart: IndexInt,
                 length: IndexInt, overflowAction: () → ()): ()
    throws { IndexOutOfBounds }
  unsignedSubtract(selfStart: IndexInt, source: T, sourceStart: IndexInt,
                   length: IndexInt, overflowAction: () → ()): ()
    throws { IndexOutOfBounds }
  saturatingSignedSubtract(selfStart: IndexInt, source: T, sourceStart: IndexInt,
                           length: IndexInt): ()
    throws { IndexOutOfBounds }
  saturatingUnsignedSubtract(selfStart: IndexInt, source: T, sourceStart: IndexInt,
                             length: IndexInt): ()
    throws { IndexOutOfBounds }
  wrappingNegate(selfStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  negate(selfStart: IndexInt, length: IndexInt, carryIn: Bit = 1): Bit
    throws { IndexOutOfBounds }
  signedNegate(selfStart: IndexInt, length: IndexInt, overflowAction: () → ()): ()
    throws { IndexOutOfBounds }
  unsignedNegate(selfStart: IndexInt, length: IndexInt, overflowAction: () → ()): ()
    throws { IndexOutOfBounds }
  saturatingSignedNegate(selfStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
  bitNot(selfStart: IndexInt, length: IndexInt): ()
    throws { IndexOutOfBounds }
```

bitAnd(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitOr(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitXor(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitXorNot(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitNand(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitNor(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitAndNot(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitOrNot(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
signedMax(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
signedMin(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
unsignedMax(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
unsignedMin(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
equal(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
unequal(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
signedLT(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
signedLE(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
signedGE(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
signedGT(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
unsignedLT(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
unsignedLE(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
unsignedGE(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
unsignedGT(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
signedShift(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt): ()
`throws` { IndexOutOfBounds }
signedShift(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }
saturatingSignedShift(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt): ()
`throws` { IndexOutOfBounds }
unsignedShift(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt): ()
`throws` { IndexOutOfBounds }

unsignedShift(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }
saturatingUnsignedShift(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitRotate(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt): ()
`throws` { IndexOutOfBounds }
countOneBits(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds }
countLeadingZeroBits(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds }
countTrailingZeroBits(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds }
leftmostOneBit(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
rightmostOneBit(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
bitReverse(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
gatherBits(*selfStart*: IndexInt, *mask*: T, *maskStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
spreadBits(*selfStart*: IndexInt, *mask*: T, *maskStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
clearAllBits(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
setAllBits(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
signedIndex(): IndexInt
`throws` { IntegerOverflow }
signedIndex(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds, IntegerOverflow }
unsignedIndex(): IndexInt
`throws` { IntegerOverflow }
unsignedIndex(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds, IntegerOverflow }
`end`

43.13.1 *copy*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.2 *wrappingAdd*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.3 *add*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt, *carryIn*: Bit = 0): Bit
`throws` { IndexOutOfBounds }

43.13.4 *signedAdd*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }

43.13.5 *unsignedAdd*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }

43.13.6 *saturatingSignedAdd*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.7 *saturatingUnsignedAdd*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.8 *wrappingSubtract*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.9 *subtract*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt, *carryIn*: Bit = 1): Bit
`throws` { IndexOutOfBounds }

43.13.10 *signedSubtract*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }

43.13.11 *unsignedSubtract*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }

43.13.12 *saturatingSignedSubtract*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.13 *saturatingUnsignedSubtract*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt,
length: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.14 *wrappingNegate*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.15 *negate*(*selfStart*: IndexInt, *length*: IndexInt, *carryIn*: Bit = 1): Bit
`throws` { IndexOutOfBounds }

43.13.16 *signedNegate*(*selfStart*: IndexInt, *length*: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }

43.13.17 *unsignedNegate*(*selfStart*: IndexInt, *length*: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }

43.13.18 *saturatingSignedNegate*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.19 *bitNot*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.20 *bitAnd*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.21 *bitOr*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.22 *bitXor*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.23 *bitXorNot*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.24 *bitNand*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.25 *bitNor*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.26 *bitAndNot*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.27 *bitOrNot*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.28 *signedMax*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

43.13.29 *signedMin*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.30 *unsignedMax*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
43.13.31 *unsignedMin*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.32 *equal*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.33 *unequal*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.34 *signedLT*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.35 *signedLE*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.36 *signedGE*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.37 *signedGT*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.38 *unsignedLT*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.39 *unsignedLE*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.40 *unsignedGE*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }
43.13.41 *unsignedGT*(*selfStart*: IndexInt, *source*: T, *sourceStart*: IndexInt, *length*: IndexInt): Boolean
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.42 *signedShift*(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt): ()
`throws` { IndexOutOfBounds }
43.13.43 *signedShift*(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }
43.13.44 *saturatingSignedShift*(*selfStart*: IndexInt, *length*: IndexInt, j: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.45 *unsignedShift*(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt): ()
`throws` { IndexOutOfBounds }
43.13.46 *unsignedShift*(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt, *overflowAction*: () → ()): ()
`throws` { IndexOutOfBounds }
43.13.47 *saturatingUnsignedShift*(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.48 *bitRotate*(*selfStart*: IndexInt, *length*: IndexInt, *j*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.49 *countOneBits*(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.50 *countLeadingZeroBits*(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds }
43.13.51 *countTrailingZeroBits*(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.52 *leftmostOneBit*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
43.13.53 *rightmostOneBit*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.54 *bitReverse*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.55 *gatherBits*(*selfStart*: IndexInt, *mask*: T, *maskStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
43.13.56 *spreadBits*(*selfStart*: IndexInt, *mask*: T, *maskStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.57 *clearAllBits*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }
43.13.58 *setAllBits*(*selfStart*: IndexInt, *length*: IndexInt): ()
`throws` { IndexOutOfBounds }

[Description to be supplied.]

43.13.59 *signedIndex*(): IndexInt
`throws` { IntegerOverflow }
43.13.60 *signedIndex*(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds, IntegerOverflow }
43.13.61 *unsignedIndex*(): IndexInt
`throws` { IntegerOverflow }
43.13.62 *unsignedIndex*(*selfStart*: IndexInt, *length*: IndexInt): IndexInt
`throws` { IndexOutOfBounds, IntegerOverflow }

[Description to be supplied.]

Part VII

Appendices

Appendix A

Fortress Calculi

A.1 Basic Core Fortress

In this section, we define a basic core calculus for Fortress. We call this calculus *Basic Core Fortress*. Following the precedent set by prior core calculi such as Featherweight Generic Java [15], we have abided by the restriction that all valid Basic Core Fortress programs are valid Fortress programs.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.10
>
> This calculus predates instantiation exclusion (Section 6.3, Section 10.1). Its only condition on multiple inheritance is that every visible method have one owner, so an object that extends two different instantiations of a parameterized trait declaring no method is a valid Basic Core Fortress program; under the rule it is not a valid Fortress program, and the sentence above no longer holds for every program. The calculus, its rules and its soundness claim are as the team wrote and proved them, and are not changed.

A.1.1 Syntax

A syntax for Basic Core Fortress is provided in Figure A.1. We use the following notational conventions:

- For brevity, we omit separators such as , and ; from Basic Core Fortress.
- $\overrightarrow{\tau}$ is a shorthand for a (possibly empty) sequence $\tau_1, \cdots, \tau_n$.
- Similarly, we abbreviate a sequence of relations α_1 `extends` $N_1, \cdots, \alpha_n$ `extends` N_n to $\overrightarrow{\alpha \texttt{extends}\ N}$
- We use τ_i to denote the ith element of $\overrightarrow{\tau}$.
- For simplicity, we assume that every name (type variables, field names, and parameters) is different and every trait/object declaration declares unique name.
- We prohibit cycles in type hierarchies.

The syntax of Basic Core Fortress allows only a small subset of the Fortress language to be formalized. Basic Core Fortress includes trait and object definitions, method and field invocations, and `self` expressions. The types of Basic Core Fortress include type variables, instantiated traits, instantiated objects, and the distinguished trait Object. Note that we syntactically prohibit extending objects. Among other features, Basic Core Fortress does *not* include top-level variable and function definitions, overloading, `excludes` clauses, `comprises` clauses, `where` clauses, object expressions, and function expressions. Basic Core Fortress will be extended to formalize a larger set of Fortress programs in the future.

α, β			type variables
f			method name
x			field name
T			trait name
O			object name
τ, τ', τ''	::=	α	type
	|	σ	
σ	::=	N	type that is not a type variable
	|	$O[\![\overrightarrow{\tau}]\!]$	
N, M, L	::=	$T[\![\overrightarrow{\tau}]\!]$	type that can be a type bound
	|	Object	
e	::=	x	expression
	|	`self`	
	|	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
	|	$e.x$	
	|	$e.f[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
fd	::=	$f[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x:\tau}):\tau = e$	method definition
td	::=	$\texttt{trait } T[\![\overrightarrow{\alpha \texttt{ extends } N}]\!] \texttt{ extends } \{\overrightarrow{N}\}\ \overrightarrow{fd} \texttt{ end}$	trait definition
od	::=	$\texttt{object } O[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x:\tau}) \texttt{ extends } \{\overrightarrow{N}\}\ \overrightarrow{fd} \texttt{ end}$	object definition
d	::=	td	definition
	|	od	
p	::=	$\overrightarrow{d}\ e$	program

Figure A.1: Syntax of Basic Core Fortress

A.1.2 Dynamic Semantics

A dynamic semantics for Basic Core Fortress is provided in Figure A.2. This semantics has been mechanized via the PLT Redex tool [18]. It therefore follows the style of explicit evaluation contexts and redexes. The Basic Core Fortress dynamic semantics consists of two evaluations rules: one for field access and another for method invocation. For simplicity, we use ' _ ' to denote some parts of the syntax that do not have key roles in a rule. We assume that _ does not expand across definition boundaries unless the entire definition is included in it.

A.1.3 Static Semantics

A static semantics for Basic Core Fortress is provided in Figures A.3, A.4, and A.5. The Basic Core Fortress static semantics is based on the static semantics of Featherweight Generic Java (FGJ) [15]. The major difference is the division of classes into traits and objects. Both trait and object definitions include method definitions but only object definitions include field definitions. With traits, Basic Core Fortress supports multiple inheritance. However, due to the similarity of traits and objects, many of the rules in the Basic Core Fortress dynamic and static semantics combine the two cases. Note that Basic Core Fortress allows parametric polymorphism, subtype polymorphism, and overriding in much the same way that FGJ does.

We proved the type soundness of Basic Core Fortress using the standard technique of proving a progress theorem and a subject reduction theorem.

Values, evaluation contexts, redexes, and trait and object names

$$
\begin{array}{lcll}
v & ::= & O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}) & \text{value} \\
E & ::= & \Box & \text{evaluation context} \\
 & | & O[\![\overrightarrow{\tau}]\!](\overrightarrow{e}\,E\,\overrightarrow{e}) & \\
 & | & E.x & \\
 & | & E.f[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) & \\
 & | & e.f[\![\overrightarrow{\tau}]\!](\overrightarrow{e}\,E\,\overrightarrow{e}) & \\
R & ::= & v.x & \text{redex} \\
 & | & v.f[\![\overrightarrow{\tau}]\!](\overrightarrow{v}) & \\
C & ::= & T & \text{trait name} \\
 & | & O & \text{object name}
\end{array}
$$

Evaluation rules: $\boxed{p \vdash E[R] \longrightarrow E[e]}$

$$\text{[R-FIELD]} \qquad \frac{\texttt{object}\ O\ _(\overrightarrow{x\ _})\ _\ \texttt{end} \in p}{p \vdash E[O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}).x_i] \longrightarrow E[v_i]}$$

$$\text{[R-METHOD]} \qquad \frac{\texttt{object}\ O\ _\ (\overrightarrow{x\ _})\ _\ \texttt{end} \in p \qquad mbody_p(f[\![\overrightarrow{\tau'}]\!], O[\![\overrightarrow{\tau}]\!]) = \{(\overrightarrow{x'}) \to e\}}{p \vdash E[O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}).f[\![\overrightarrow{\tau'}]\!](\overrightarrow{v'})] \longrightarrow E[[\overrightarrow{v}/\overrightarrow{x}][O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})/\texttt{self}][\overrightarrow{v'}/\overrightarrow{x'}]e]}$$

Method body lookup: $\boxed{mbody_p(f[\![\overrightarrow{\tau}]\!], \tau) = \{(\overrightarrow{x}) \to e\}}$

$$\text{[MB-SELF]} \qquad \frac{_\ C[\![\overrightarrow{\alpha\ \texttt{extends}\ _}]\!]\ _\ \overrightarrow{fd}\ _ \in p \qquad f[\![\overrightarrow{\alpha'\ \texttt{extends}\ _}]\!](\overrightarrow{x'\!:\ _})\ _ = e \in \{\overrightarrow{fd}\}}{mbody_p(f[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!]) = \{[\overrightarrow{\tau'}/\overrightarrow{\alpha'}][\overrightarrow{\tau}/\overrightarrow{\alpha}](\overrightarrow{x'}) \to e\}}$$

$$\text{[MB-SUPER]} \qquad \frac{_\ C[\![\overrightarrow{\alpha\ \texttt{extends}\ _}]\!]\ _\ \texttt{extends}\{\overrightarrow{N}\}\ _\ \overrightarrow{fd}\ _ \in p \qquad f \notin \{\overrightarrow{Fname(fd)}\}}{mbody_p(f[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!]) = \bigcup_{N_i \in \{\overrightarrow{N}\}} mbody_p(f[\![\overrightarrow{\tau'}]\!], [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i)}$$

$$\text{[MB-OBJ]} \qquad mbody_p(f[\![\overrightarrow{\tau}]\!], \text{Object}) = \emptyset$$

Function/method name lookup: $\boxed{Fname(fd) = f}$

$Fname(f[\![\overrightarrow{\alpha\ \texttt{extends}\ N}]\!](\overrightarrow{x\!:\tau})\!:\!\tau = e) = f$

Figure A.2: Dynamic Semantics of Basic Core Fortress

Environments

$$\begin{array}{lcll} \Delta & ::= & \overrightarrow{\alpha <: N} & \text{bound environment} \\ \Gamma & ::= & \overrightarrow{x : \tau} & \text{type environment} \end{array}$$

Program typing: $\boxed{\vdash p : \tau}$

$$\text{[T-PROGRAM]} \quad \frac{p = \overrightarrow{d}\ e \qquad p \vdash \overrightarrow{d}\ \mathsf{ok} \qquad p; \emptyset; \emptyset \vdash e : \tau}{\vdash p : \tau}$$

Definition typing: $\boxed{p \vdash d\ \mathsf{ok}}$

$$\text{[T-TRAITDEF]} \quad \frac{\Delta = \overrightarrow{\alpha <: N} \qquad p; \Delta \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{M}\ \mathsf{ok} \qquad p; \Delta; \mathtt{self} : T[\![\overrightarrow{\alpha}]\!]; T \vdash \overrightarrow{fd}\ \mathsf{ok} \qquad p \vdash \mathit{oneOwner}(T)}{p \vdash \mathtt{trait}\ T[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]\ \mathtt{extends}\ \{\overrightarrow{M}\}\ \overrightarrow{fd}\ \mathtt{end}\ \mathsf{ok}}$$

$$\text{[T-OBJECTDEF]} \quad \frac{\Delta = \overrightarrow{\alpha <: N} \qquad p; \Delta \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{M}\ \mathsf{ok} \qquad p; \Delta; \mathtt{self} : O[\![\overrightarrow{\alpha}]\!]\ \overrightarrow{x : \tau}; O \vdash \overrightarrow{fd}\ \mathsf{ok} \qquad p \vdash \mathit{oneOwner}(O)}{p \vdash \mathtt{object}\ O[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!](\overrightarrow{x : \tau})\ \mathtt{extends}\ \{\overrightarrow{M}\}\ \overrightarrow{fd}\ \mathtt{end}\ \mathsf{ok}}$$

Method typing: $\boxed{p; \Delta; \Gamma; C \vdash fd\ \mathsf{ok}}$

$$\text{[T-METHODDEF]} \quad \frac{_\ C\ _\ \mathtt{extends}\{\overrightarrow{M}\}\ _ \in p \qquad p; \Delta \vdash \mathit{override}(f, \{\overrightarrow{M}\}, [\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]\ \overrightarrow{\tau} \to \tau_0) \qquad \Delta' = \Delta\ \overrightarrow{\alpha <: N} \qquad p; \Delta' \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p; \Delta' \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta' \vdash \tau_0\ \mathsf{ok} \qquad p; \Delta'; \Gamma\ \overrightarrow{x : \tau} \vdash e : \tau' \qquad p; \Delta' \vdash \tau' <: \tau_0}{p; \Delta; \Gamma; C \vdash f[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!](\overrightarrow{x : \tau}) : \tau_0 = e\ \mathsf{ok}}$$

Method overriding: $\boxed{p; \Delta \vdash \mathit{override}(f, \{\overrightarrow{N}\}, [\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]\ \overrightarrow{\tau} \to \tau)}$

$$\text{[OVERRIDE]} \quad \frac{\bigcup_{L_i \in \{\overrightarrow{L}\}} \mathit{mtype}_p(f, L_i) = \{[\![\overrightarrow{\beta\ \mathtt{extends}\ M}]\!]\ \overrightarrow{\tau'} \to \tau_0'\} \qquad \overrightarrow{N} = [\overrightarrow{\alpha} / \overrightarrow{\beta}]\overrightarrow{M} \qquad \overrightarrow{\tau} = [\overrightarrow{\alpha} / \overrightarrow{\beta}]\overrightarrow{\tau'} \qquad p; \overrightarrow{\alpha <: N} \vdash \tau_0 <: [\overrightarrow{\alpha} / \overrightarrow{\beta}]\tau_0'}{p; \Delta \vdash \mathit{override}(f, \{\overrightarrow{L}\}, [\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]\ \overrightarrow{\tau} \to \tau_0)}$$

Method type lookup: $\boxed{\mathit{mtype}_p(f, \tau) = \{[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]\ \overrightarrow{\tau} \to \tau\}}$

$$\text{[MT-SELF]} \quad \frac{_\ C[\![\overrightarrow{\alpha\ \mathtt{extends}\ _}]\!]\ _\ \overrightarrow{fd}\ _ \in p \qquad f[\![\overrightarrow{\beta\ \mathtt{extends}\ M}]\!](\overrightarrow{_ : \tau}) : \tau_0' = e \in \{\overrightarrow{fd}\}}{\mathit{mtype}_p(f, C[\![\overrightarrow{\tau}]\!]) = \{[\overrightarrow{\tau} / \overrightarrow{\alpha}][\![\overrightarrow{\beta\ \mathtt{extends}\ M}]\!]\ \overrightarrow{\tau'} \to \tau_0'\}}$$

$$\text{[MT-SUPER]} \quad \frac{_\ C[\![\overrightarrow{\alpha\ \mathtt{extends}\ _}]\!]\ _\ \mathtt{extends}\{\overrightarrow{N}\}\ _\ \overrightarrow{fd}\ _ \in p \qquad f \notin \{\overrightarrow{\mathit{Fname}(fd)}\}}{\mathit{mtype}_p(f, C[\![\overrightarrow{\tau}]\!]) = \bigcup_{N_i \in \{\overrightarrow{N}\}} \mathit{mtype}_p(f, [\overrightarrow{\tau} / \overrightarrow{\alpha}]N_i)}$$

$$\text{[MT-OBJ]} \quad \mathit{mtype}_p(f, \mathrm{Object}) = \emptyset$$

Figure A.3: Static Semantics of Basic Core Fortress (I)

Expression typing: $\boxed{p; \Delta; \Gamma \vdash e : \tau}$

[T-VAR] $$p; \Delta; \Gamma \vdash x : \Gamma(x)$$

[T-SELF] $$p; \Delta; \Gamma \vdash \texttt{self} : \Gamma(\texttt{self})$$

[T-OBJECT] $$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{_ : \tau'})\ _\ \texttt{end} \in p \qquad p; \Delta \vdash O[\![\overrightarrow{\tau}]\!]\ \mathsf{ok} \qquad p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau''} \qquad p; \Delta \vdash \overrightarrow{\tau''} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau'}}{p; \Delta; \Gamma \vdash O[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) : O[\![\overrightarrow{\tau}]\!]}$$

[T-FIELD] $$\frac{p; \Delta; \Gamma \vdash e_0 : \tau_0 \qquad bound_\Delta(\tau_0) = O[\![\overrightarrow{\tau'}]\!] \qquad \texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{x : \tau})\ _\ \texttt{end} \in p}{p; \Delta; \Gamma \vdash e_0.x_i : [\overrightarrow{\tau'}/\overrightarrow{\alpha}]\tau_i}$$

[T-METHOD] $$\frac{p; \Delta; \Gamma \vdash e_0 : \tau_0 \qquad mtype_p(f, bound_\Delta(\tau_0)) = \{[\![\overrightarrow{\alpha \texttt{extends}\ N}]\!]\ \overrightarrow{\tau'} \to \tau_0'\} \qquad p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{N} \qquad p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau''} \qquad p; \Delta \vdash \overrightarrow{\tau''} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau'}}{p; \Delta; \Gamma \vdash e_0.f[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) : [\overrightarrow{\tau}/\overrightarrow{\alpha}]\tau_0'}$$

Subtyping: $\boxed{p; \Delta \vdash \tau <: \tau}$

[S-OBJ] $$p; \Delta \vdash \tau <: \mathrm{Object}$$

[S-REFL] $$p; \Delta \vdash \tau <: \tau$$

[S-TRANS] $$\frac{p; \Delta \vdash \tau_1 <: \tau_2 \qquad p; \Delta \vdash \tau_2 <: \tau_3}{p; \Delta \vdash \tau_1 <: \tau_3}$$

[S-VAR] $$p; \Delta \vdash \alpha <: \Delta(\alpha)$$

[S-TAPP] $$\frac{_\ C[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!]\ _\ \texttt{extends}\{\overrightarrow{N}\}\ _ \in p}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!] <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i}$$

Well-formed types: $\boxed{p; \Delta \vdash \tau\ \mathsf{ok}}$

[W-OBJ] $$p; \Delta \vdash \mathrm{Object}\ \mathsf{ok}$$

[W-VAR] $$\frac{\alpha \in dom(\Delta)}{p; \Delta \vdash \alpha\ \mathsf{ok}}$$

[W-TAPP] $$\frac{_\ C[\![\overrightarrow{\alpha \texttt{extends}\ N}]\!]\ _ \in p \qquad p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{N}}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!]\ \mathsf{ok}}$$

Figure A.4: Static Semantics of Basic Core Fortress (II)

Bound of type: $\boxed{bound_\Delta(\tau) = \sigma}$

$$\begin{array}{rcl} bound_\Delta(\alpha) & = & \Delta(\alpha) \\ bound_\Delta(\sigma) & = & \sigma \end{array}$$

One owner for all the visible methods: $\boxed{p \vdash oneOwner(C)}$

$$[\textsc{OneOwner}] \quad \frac{\forall f \in visible_p(C)\ .\ f \textit{ only occurs once in } visible_p(C)}{p \vdash oneOwner(C)}$$

Auxiliary functions for methods: $\boxed{defined_p \,/\, inherited_p \,/\, visible_p(C) = \{\overrightarrow{f}\}}$

$$defined_p(C) = \{\overrightarrow{Fname(fd)}\} \qquad \text{where } _\ C\ _\ \overrightarrow{fd}\ _\ \in p$$

$$inherited_p(C) = \biguplus_{N_i \in \{\overrightarrow{N}\}} \{f_i \mid f_i \in visible_p(N_i), f_i \notin defined_p(C)\} \qquad \text{where } _\ C\ _\ \texttt{extends}\ \{\overrightarrow{N}\}\ _\ \in p$$

$$visible_p(C) = defined_p(C) \uplus inherited_p(C)$$

Figure A.5: Static Semantics of Basic Core Fortress (III)

A.2 Core Fortress with Where Clauses

> DRAFT NOTE
> Where clauses are not yet supported.

In this section, we define a Fortress core calculus with `where` clauses. We call this calculus *Core Fortress with Where Clauses*. Core Fortress with Where Clauses is an extension of Basic Core Fortress with `where` clauses.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.10
> This calculus predates instantiation exclusion (Section 6.3, Section 10.1) and the rule that a `where`-clause variable may not appear as a static argument in an `extends` clause (Section 12.6). It lets a `where`-clause variable occur in a supertype of a trait or object definition, and its subtyping rule [S-TAPP] then makes the definition a subtype of one instantiation of that supertype for each witness of the variable: the shape of the declarations marked "Not allowed" in Section 12.6. The calculus, its rules and its soundness claim are as the team wrote and proved them, and are not changed.

A.2.1 Syntax

The syntax for Core Fortress with Where Clauses is provided in Figure A.6. For simplicity, we use the following notational conventions:

- We abbreviate a sequence of relations $\alpha_1\,\texttt{extends}\,\{\overrightarrow{K_1}\}\cdots\alpha_n\,\texttt{extends}\,\{\overrightarrow{K_n}\}$ to $\overrightarrow{\alpha\,\texttt{extends}\,\{\overrightarrow{K}\}}$ and $\tau_1 <: H_{11}\ \tau_1 <: H_{12}\cdots\tau_1 <: H_{1l}\ \tau_2 <: H_{21}\cdots\tau_n <: H_{nm}$ to $\overrightarrow{\tau} <: \overrightarrow{\overrightarrow{H}}$.
- Substitutions of x with v and α with τ are denoted as $[v/x]$ and $[\tau/\alpha]$, respectively. A sequence of substitutions represents the composition of those substitutions where the right-most substitution is applied first. For example, $S_n \cdots S_1\tau$ represents $S_n(\cdots S_2(S_1\tau)\cdots)$.

α, β			type variables	
m			method name	
x			field name	
T			trait name	
O			object name	
C			trait or object name	
τ, τ', τ''	$::=$	α	type	
	$\mid$	σ		
σ	$::=$	N	type that is not a type variable	
	$\mid$	$O[\![\overrightarrow{\tau}]\!]$		
N, M, L	$::=$	$T[\![\overrightarrow{\tau}]\!]$	super trait	
	$\mid$	Object		
K, H, J	$::=$	α	bound on a type variable	
	$\mid$	N		
e	$::=$	x	expression	
	$\mid$	`self`		
	$\mid$	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$		
	$\mid$	$e.x$		
	$\mid$	$(e\,\texttt{as}\,C[\![\overrightarrow{\tau}]\!).m[\![\overrightarrow{\tau}]\!](\overrightarrow{e\,\texttt{as}\,\tau})\,\texttt{as}\,\tau$		
	$\mid$	$\texttt{typecase}\,x\ = e\,\texttt{of}\,\overrightarrow{\tau \Rightarrow e}\,\texttt{else}\,e\,\texttt{end}$		
md	$::=$	$m[\![\overrightarrow{\alpha\,\texttt{extends}\,\{\overrightarrow{K}\}}]\!](\overrightarrow{x{:}\tau}){:}\tau = e$	method definition	
td	$::=$	$\texttt{trait}\,T[\![\overrightarrow{\alpha\,\texttt{extends}\,\{\overrightarrow{K}\}}]\!]\,\texttt{extends}\,\{\overrightarrow{N}\}\,\texttt{where}\,\{\overrightarrow{\alpha\,\texttt{extends}\,\{\overrightarrow{K}\}}\}\ \overrightarrow{md}\,\texttt{end}$	trait definition	
od	$::=$	$\texttt{object}\,O[\![\overrightarrow{\alpha\,\texttt{extends}\,\{\overrightarrow{K}\}}]\!](\overrightarrow{x{:}\tau})\,\texttt{extends}\,\{\overrightarrow{N}\}\,\texttt{where}\,\{\overrightarrow{\alpha\,\texttt{extends}\,\{\overrightarrow{K}\}}\}\ \overrightarrow{md}\,\texttt{end}$	object definition	
d	$::=$	td	definition	
	$\mid$	od		
p	$::=$	$\overrightarrow{d}\ e$	program	

Figure A.6: Syntax of Core Fortress with Where Clauses

Most of the syntax is a straightforward extension of Basic Core Fortress. An object or trait definition may include `where` clauses. Every method invocation is annotated with three kinds of static types by type inference: the static types of the receiver, the arguments, and the result. These type annotations appear in Core Fortress with Where Clauses in a form of $\texttt{as}\,\tau$. If the annotated types are not enough (to find "witnesses" for the where-clauses variables), type checking rejects the program and requires more type information from the programmer.

A.2.2 Dynamic Semantics

A dynamic semantics for Core Fortress with Where Clauses is provided in Figures A.7 and A.8.

A.2.3 Static Semantics

A static semantics for Core Fortress with Where Clauses is provided in Figures A.9, A.10, A.11, and A.12.

For simplicity, we use the following conventions:

- $FTV(\tau)$ collects all free type variables in τ.
- Similarly, $FTV(e)$ collects all free type variables in all types in e.

Values, evaluation contexts, and redexes

$$
\begin{array}{rcl}
v & ::= & O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}) \\
E & ::= & \square \\
 & | & O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}\, E\, \overrightarrow{e}) \\
 & | & E.x \\
 & | & E \,\texttt{as}\, \tau.m[\![\overrightarrow{\tau}]\!](\overrightarrow{e \,\texttt{as}\, \tau})\texttt{as}\, \tau \\
 & | & v \,\texttt{as}\, \tau.m[\![\overrightarrow{\tau}]\!](\overrightarrow{v \,\texttt{as}\, \tau}\, E \,\texttt{as}\, \tau\, \overrightarrow{e \,\texttt{as}\, \tau})\texttt{as}\, \tau \\
 & | & \texttt{typecase}\ x\ = E\ \texttt{of}\ \overrightarrow{\tau \Rightarrow e}\ \texttt{else}\ e\ \texttt{end} \\
R & ::= & v.x \\
 & | & v \,\texttt{as}\, \tau.m[\![\overrightarrow{\tau}]\!](\overrightarrow{v \,\texttt{as}\, \tau})\texttt{as}\, \tau \\
 & | & \texttt{typecase}\ x\ = v\ \texttt{of}\ \overrightarrow{\tau \Rightarrow e}\ \texttt{else}\ e\ \texttt{end}
\end{array}
$$

Evaluation rules: $\boxed{p \vdash E[\![R]\!] \longrightarrow E[\![e]\!]}$

[R-FIELD]
$$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{x\colon _})\ _\ \texttt{end} \in p}{p \vdash E[\![O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}).x_i]\!] \longrightarrow E[\![v_i]\!]}$$

[R-METHOD]
$$\frac{\begin{array}{c} mtype_p(m, C[\![\overrightarrow{\tau^c}]\!], \emptyset) = \{(\overrightarrow{[\![\alpha' \texttt{extends}\ _]\!]}\ \overrightarrow{\tau''} \to \tau_0', _)\} \quad S([\overrightarrow{\tau'}/\overrightarrow{\alpha'}]\overrightarrow{\tau''}) = \overrightarrow{\tau^a} \quad S([\overrightarrow{\tau'}/\overrightarrow{\alpha'}]\tau_0') = \tau^r \\ \texttt{object}\ O\ _\ (\overrightarrow{x\colon _})\ _\ \texttt{end} \in p \qquad mbody_p(m[\![\overrightarrow{\tau'}]\!], O[\![\overrightarrow{\tau}]\!], C[\![\overrightarrow{\tau^c}]\!]) = \{(\overrightarrow{x'}) \to e\} \end{array}}{p \vdash E[\![O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})\texttt{as}\, C[\![\overrightarrow{\tau^c}]\!].m[\![\overrightarrow{\tau'}]\!](\overrightarrow{v' \,\texttt{as}\, \tau^a})\texttt{as}\, \tau^r] \longrightarrow E[\![[\overrightarrow{v}/\overrightarrow{x}][O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})/\texttt{self}][\overrightarrow{v'}/\overrightarrow{x'}]Se]\!]}$$

[R-TYPECASE]
$$\frac{type(v) = \tau^v \qquad |\overrightarrow{\tau}| = n \qquad \neg(p; \emptyset \vdash \tau^v\ <:\ \tau_i) \qquad 1 \le i < n \qquad p; \emptyset \vdash \tau^v\ <:\ \tau_n}{p \vdash E[\![\texttt{typecase}\ x\ = v\ \texttt{of}\ \overrightarrow{\tau \Rightarrow e}\ \overrightarrow{\tau' \Rightarrow e'}\ \texttt{else}\ e\ \texttt{end}]\!] \longrightarrow E[\![[v/x]e_n]\!]}$$

[R-TYPECASEELSE]
$$\frac{type(v) = \tau^v \qquad \neg(p; \emptyset \vdash \tau^v\ <:\ \tau_i) \qquad 1 \le i \le |\overrightarrow{\tau}|}{p \vdash E[\![\texttt{typecase}\ x\ = v\ \texttt{of}\ \overrightarrow{\tau \Rightarrow e}\ \texttt{else}\ e'\ \texttt{end}]\!] \longrightarrow E[\![[v/x]e']\!]}$$

Method body lookup: $\boxed{mbody_p(m[\![\overrightarrow{\tau}]\!], \tau, \tau) = \{(\overrightarrow{x}) \to e\}}$

[MB-SELF]
$$\frac{_\ C[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!]\ _\ \overrightarrow{md}\ _ \in p \qquad m[\![\overrightarrow{\alpha' \texttt{extends}\ _}]\!](\overrightarrow{x'\colon _})\colon _\ = e \in \{\overrightarrow{md}\}}{mbody_p(m[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!], C[\![\overrightarrow{\tau^c}]\!]) = \{(\overrightarrow{x'}) \to [\overrightarrow{\tau'}/\overrightarrow{\alpha'}][\overrightarrow{\tau^c}/\overrightarrow{\alpha}]e\}}$$

[MB-WITNESS]
$$\frac{\begin{array}{c} _\ C[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!]\ _\ \overrightarrow{md}\ _ \in p \qquad m[\![\overrightarrow{\alpha' \texttt{extends}\ _}]\!](\overrightarrow{x'\colon _})\colon _\ = e \in \{\overrightarrow{md}\} \\ \tau^o \neq C[\![_]\!] \qquad witness_p(C[\![\overrightarrow{\tau}]\!], \tau^o) = S \end{array}}{mbody_p(m[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!], \tau^o) = \{(\overrightarrow{x'}) \to [\overrightarrow{\tau'}/\overrightarrow{\alpha'}](S([\overrightarrow{\tau}/\overrightarrow{\alpha}]e))\}}$$

[MB-SUPER]
$$\frac{_\ C[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!]\ _\ \texttt{extends}\{\overrightarrow{N}\}\ _\ \overrightarrow{md}\ _ \in p \qquad m \notin \{\overrightarrow{Name(md)}\} \qquad C \neq C'}{mbody_p(m[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!], C'[\![\overrightarrow{\tau^c}]\!]) = \bigcup_{N_i \in \{\overrightarrow{N}\}} mbody_p(m[\![\overrightarrow{\tau'}]\!], [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i, C'[\![\overrightarrow{\tau^c}]\!]))}$$

[MB-STATIC]
$$\frac{_\ C[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!]\ _\ \texttt{extends}\{\overrightarrow{N}\}\ _\ \overrightarrow{md}\ _ \in p \qquad m \notin \{\overrightarrow{Name(md)}\}}{mbody_p(m[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!], C[\![\overrightarrow{\tau^c}]\!]) = \bigcup_{N_i \in \{\overrightarrow{N}\}} mbody_p(m[\![\overrightarrow{\tau'}]\!], [\overrightarrow{\tau^c}/\overrightarrow{\alpha}]N_i, \mathrm{Object})}$$

[MB-OBJ]
$$mbody_p(m[\![\overrightarrow{\tau}]\!], \mathrm{Object}, \tau) = \emptyset$$

Figure A.7: Dynamic Semantics of Core Fortress with Where Clauses (I)

Function/method name lookup: $\boxed{Name(md) = m}$

$Name(m[\![\overrightarrow{\alpha \texttt{ extends } \{K\}}]\!](\overrightarrow{x{:}\tau}){:}\tau = e) = m$

Types of values: $\boxed{type(v) = \tau}$

$type(O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})) = O[\![\overrightarrow{\tau}]\!]$

Finding witnesses from the static type of the receiver: $\boxed{witness_p(\tau, \tau) = \mathrm{S}}$

$$witness_p(\tau, \tau') = \begin{cases} [\,] & \text{if } \tau' = \mathrm{Object} \\ match(C[\![\overrightarrow{\tau}]\!], C[\![\overrightarrow{\tau'}]\!]) & \text{if } \tau = C[\![\overrightarrow{\tau}]\!] \text{ and } \tau' = C[\![\overrightarrow{\tau'}]\!] \\ S_n \cdots\ S_1 & \text{if } \tau = C[\![\overrightarrow{\tau}]\!],\ \tau' = C'[\![\overrightarrow{\tau'}]\!],\ C \neq C', \\ & _\ C[\![\overrightarrow{\alpha \texttt{ extends } _}]\!]\ _\ \texttt{extends}\ \{\overrightarrow{N}\}\ _\ \in p, \\ & \text{and } witness_p([\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i, C'[\![\overrightarrow{\tau'}]\!]) = S_i \text{ for } N_i \in \{\overrightarrow{N}\} \\ [\,] & \text{otherwise} \end{cases}$$

Match two types to get substitutions: $\boxed{match(\tau, \tau) = \mathrm{S}}$

$$match(\tau, \tau') = \begin{cases} [\,] & \text{if } \tau = \tau' \\ [\tau'/\beta] & \text{if } \tau = \beta \\ S_n \cdots\ S_1 & \text{if } \tau = C[\![\overrightarrow{\tau}]\!],\ \tau' = C[\![\overrightarrow{\tau'}]\!], \text{ and } match(S_{i-1} \cdots\ S_1\tau_i, \tau_i') = S_i \text{ for } 1 \leq i \leq n \\ [\,] & \text{otherwise} \end{cases}$$

Figure A.8: Dynamic Semantics of Core Fortress with Where Clauses (II)

Environments and method types

$$
\begin{array}{lcll}
\Delta & ::= & \overrightarrow{\alpha \ <: \ \{\overrightarrow{K}\}} & \text{bound environment} \\
\Gamma & ::= & \overrightarrow{x : \tau} & \text{type environment} \\
\eta & ::= & \overrightarrow{[\![\alpha \texttt{extends} \ \{\overrightarrow{K}\}]\!]} \ \overrightarrow{\tau} \to \tau & \text{method type}
\end{array}
$$

Program typing: $\boxed{\vdash p : \tau}$

$$
\text{[T-PROGRAM]} \quad \frac{p = \overrightarrow{d} \ e \qquad p \vdash \overrightarrow{d} \ \mathsf{ok} \qquad p; \emptyset; \emptyset \vdash e : \tau}{\vdash p : \tau}
$$

Definition typing: $\boxed{p \vdash d \ \mathsf{ok}}$

$$
\text{[T-TRAITDEF]} \quad \frac{\begin{array}{c} p \vdash \mathit{validMultipleInheritance}(T) \qquad \Delta = \overrightarrow{\alpha \ <: \ \{\overrightarrow{K}\}} \ \overrightarrow{\beta \ <: \ \{\overrightarrow{H}\}} \\ p; \Delta \vdash \overrightarrow{\overrightarrow{K}} \ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\overrightarrow{N}} \ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\overrightarrow{H}} \ \mathsf{ok} \qquad p; \Delta; \texttt{self} : T[\![\overrightarrow{\alpha}]\!]; T \vdash \overrightarrow{md} \ \mathsf{ok} \end{array}}{p \vdash \texttt{trait} \ T[\![\overrightarrow{\alpha \texttt{extends} \ \{\overrightarrow{K}\}}]\!] \texttt{extends} \{\overrightarrow{N}\} \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{\overrightarrow{H}\}}\} \ \overrightarrow{md} \texttt{end} \ \mathsf{ok}}
$$

$$
\text{[T-OBJECTDEF]} \quad \frac{\begin{array}{c} p \vdash \mathit{validMultipleInheritance}(O) \qquad \Delta = \overrightarrow{\alpha \ <: \ \{\overrightarrow{K}\}} \qquad \Delta' = \Delta \ \overrightarrow{\beta \ <: \ \{\overrightarrow{H}\}} \\ p; \Delta' \vdash \overrightarrow{\overrightarrow{K}} \ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} \ \mathsf{ok} \qquad p; \Delta' \vdash \overrightarrow{\overrightarrow{N}} \ \mathsf{ok} \qquad p; \Delta' \vdash \overrightarrow{\overrightarrow{H}} \ \mathsf{ok} \\ p; \Delta'; \texttt{self} : O[\![\overrightarrow{\alpha}]\!] \ \overrightarrow{x : \tau}; O \vdash \overrightarrow{md} \ \mathsf{ok} \end{array}}{p \vdash \texttt{object} \ O[\![\overrightarrow{\alpha \texttt{extends} \ \{\overrightarrow{K}\}}]\!] (\overrightarrow{x \tau}) \texttt{extends} \{\overrightarrow{N}\} \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{\overrightarrow{H}\}}\} \ \overrightarrow{md} \texttt{end} \ \mathsf{ok}}
$$

Method typing: $\boxed{p; \Delta; \Gamma; C \vdash md \ \mathsf{ok}}$

$$
\text{[T-METHODDEF]} \quad \frac{\begin{array}{c} p \vdash \mathit{override}(m, C, \overrightarrow{[\![\alpha \texttt{extends} \ \{\overrightarrow{K}\}]\!]} \ \overrightarrow{\tau} \to \tau_0) \qquad \Delta' = \Delta \ \overrightarrow{\alpha \ <: \ \{\overrightarrow{K}\}} \qquad p; \Delta' \vdash \overrightarrow{\overrightarrow{K}} \ \mathsf{ok} \\ p; \Delta' \vdash \overrightarrow{\tau} \ \mathsf{ok} \qquad p; \Delta' \vdash \tau_0 \ \mathsf{ok} \qquad p; \Delta'; \Gamma \ \overrightarrow{x : \tau} \vdash e : \tau' \qquad p; \Delta' \vdash \tau' \ <: \ \tau_0 \end{array}}{p; \Delta; \Gamma; C \vdash m[\![\overrightarrow{\alpha \texttt{extends} \ \{\overrightarrow{K}\}}]\!] (\overrightarrow{x \tau}) : \tau_0 = e \ \mathsf{ok}}
$$

Method overriding: $\boxed{p \vdash \mathit{override}(m, C, \eta)}$

$$
\text{[OVERRIDE]} \quad \frac{\begin{array}{c} _ \ C[\![\overrightarrow{\alpha'' \texttt{extends} \ \{\overrightarrow{K''}\}}]\!] \ _ \ \texttt{extends} \ \{\overrightarrow{N}\} \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{\overrightarrow{H}\}}\} \ _ \ \in p \\ \Delta = \alpha'' \ <: \ \{\overrightarrow{K''}\} \ \overrightarrow{\beta \ <: \ \{\overrightarrow{H}\}} \\ \bigcup_{C'[\![\overrightarrow{\tau''}]\!] \in \{\overrightarrow{N}\}} \mathit{mtype}_p(m, C'[\![\overrightarrow{\tau''}]\!], \Delta) = \{(\overrightarrow{[\![\alpha' \texttt{extends} \ \{\overrightarrow{K'}\}]\!]} \ \overrightarrow{\tau'} \to \tau_0', \Delta')\} \\ \overrightarrow{\overrightarrow{K}} = [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\overrightarrow{\overrightarrow{K'}} \qquad p; \Delta' \vdash [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\overrightarrow{\tau'} \ <: \ \overrightarrow{\tau} \qquad p; \Delta' \vdash \tau_0 \ <: \ [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\tau_0' \end{array}}{p \vdash \mathit{override}(m, C, \overrightarrow{[\![\alpha \texttt{extends} \ \{\overrightarrow{K}\}]\!]} \ \overrightarrow{\tau} \to \tau_0)}
$$

Valid multiple inheritance: $\boxed{p \vdash \mathit{validMultipleInheritance}(C)}$

$$
\text{[VALIDMI]} \quad \frac{p \vdash \mathit{oneOwner}(C) \qquad p \vdash \mathit{validWhere}(C)}{p \vdash \mathit{validMultipleInheritance}(C)}
$$

Figure A.9: Static Semantics of Core Fortress with Where Clauses (I)

One owner for all the visible methods: $\boxed{p \vdash \mathit{oneOwner}(C)}$

[ONEOWNER] $$\frac{\forall m \in \mathit{visible}_p(C)\ .\ f \textit{ only occurs once in } \mathit{visible}_p(C)}{p \vdash \mathit{oneOwner}(C)}$$

Valid where clauses: $\boxed{p \vdash \mathit{validWhere}(C)}$

[VALIDWHERE] $$\frac{\begin{array}{l}\forall m \in \mathit{visible}_p(C)\ . \\ \text{where}\quad _\ C[\![\overrightarrow{\alpha}\ \overrightarrow{_}]\!]\ _\ \texttt{extends}\ \{\overrightarrow{N}\}\ _ \in p \\ \qquad \mathit{mbody}_p(m[\![\overrightarrow{\alpha^f}]\!], C[\![\overrightarrow{\alpha}]\!], C[\![\overrightarrow{\alpha}]\!]) = \{_ \to e_f\}, \qquad \mathit{mtype}_p(m, C[\![\overrightarrow{\alpha}]\!], \emptyset) = \{(\eta_f, \Delta)\} \\ \qquad 1.\quad \forall \beta \in (FTV(e_f) \setminus \{\overrightarrow{\alpha}\ \overrightarrow{\alpha^f}\})\ .\ \beta \in FTV(\eta_f) \\ \qquad 2.\quad \forall \beta \in (FTV(\eta_f) \setminus \{\overrightarrow{\alpha}\ \overrightarrow{\alpha^f}\})\ .\ \forall C'[\![\overrightarrow{\tau^c}]\!] \in \bigcup_{N_i \in \{\overrightarrow{N}\}} \mathit{defining}_p(m, N_i)\ . \\ \qquad\qquad \beta = \tau_i^c \quad \Delta(\beta) = \overrightarrow{K'_i} \quad \text{for } 1 \le i \le |\overrightarrow{\tau^c}| \text{ where } _\ \overrightarrow{C'[\![\alpha' \texttt{extends}\ \{\overrightarrow{K'}\}]\!]}\ _ \in p\end{array}}{p \vdash \mathit{validWhere}(C)}$$

Valid witnesses: $\boxed{p \vdash \mathit{validWitness}(\Delta, \overrightarrow{\alpha\ <:\ \{\overrightarrow{\tau}\}}, \overrightarrow{\tau})}$

[VALIDWITNESS] $$\frac{p; \Delta \vdash \overrightarrow{[\overrightarrow{\tau^\beta}/\overrightarrow{\beta}]\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau^\beta}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau^\beta}\ <:\ \overrightarrow{[\overrightarrow{\tau^\beta}/\overrightarrow{\beta}]\tau} \quad \{\overrightarrow{\beta}\} \cap \mathit{dom}(\Delta) = \emptyset}{p \vdash \mathit{validWitness}(\Delta, \overrightarrow{\beta\ <:\ \{\overrightarrow{\tau}\}}, \overrightarrow{\tau^\beta})}$$

Expression typing: $\boxed{p; \Delta; \Gamma \vdash e : \tau}$

[T-VAR] $$p; \Delta; \Gamma \vdash x : \Gamma(x)$$

[T-SELF] $$p; \Delta; \Gamma \vdash \texttt{self} : \Gamma(\texttt{self})$$

[T-OBJECT] $$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{_ : \tau})\ _\ \texttt{end} \in p \qquad p; \Delta \vdash O[\![\overrightarrow{\tau}]\!]\mathsf{ok} \quad p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau''} \qquad p; \Delta \vdash \overrightarrow{\tau''}\ <:\ \overrightarrow{[\overrightarrow{\tau}/\overrightarrow{\alpha}]\tau'}}{p; \Delta; \Gamma \vdash O[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) : O[\![\overrightarrow{\tau}]\!]}$$

[T-FIELD] $$\frac{p; \Delta; \Gamma \vdash e_0 : \tau_0 \quad \mathit{bound}_\Delta(\tau_0) = \{O[\![\overrightarrow{\tau'}]\!]\} \quad \texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{x : \tau})\ _\ \texttt{end} \in p}{p; \Delta; \Gamma \vdash e_0\,.\,x_i : \overrightarrow{[\overrightarrow{\tau'}/\overrightarrow{\alpha}]\tau_i}}$$

[T-METHOD] $$\frac{\begin{array}{c}p; \Delta; \Gamma \vdash e_0 : \tau_0 \quad p; \Delta \vdash \tau_0\ <:\ C[\![\overrightarrow{\tau^c}]\!] \quad \mathit{mtype}_p(m, C[\![\overrightarrow{\tau^c}]\!], \emptyset) = \{(\overrightarrow{[\![\alpha' \texttt{extends}\ \{\overrightarrow{K'}\}]\!]}\ \overrightarrow{\tau'} \to \tau'_0, \Delta')\} \\ p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash C[\![\overrightarrow{\tau^c}]\!]\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau^a}\ \mathsf{ok} \qquad p; \Delta \vdash \tau^r\ \mathsf{ok} \\ p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau''} \quad p; \Delta \vdash \overrightarrow{\tau''}\ <:\ \overrightarrow{\tau^a} \quad \mathit{dom}(\Delta') = \{\overrightarrow{\beta}\} \quad S = [\overrightarrow{\tau^\beta}/\overrightarrow{\beta}] \\ p \vdash \mathit{validWitness}(\Delta, \Delta', \overrightarrow{\tau^\beta}) \quad p; \Delta \vdash \overrightarrow{\tau}\ <:\ S(\overrightarrow{[\overrightarrow{\tau}/\overrightarrow{\alpha'}]\tau'}) \quad S(\overrightarrow{[\overrightarrow{\tau}/\overrightarrow{\alpha'}]\tau}) = \overrightarrow{\tau^a} \quad S([\overrightarrow{\tau}/\overrightarrow{\alpha'}]\tau'_0) = \tau^r\end{array}}{p; \Delta; \Gamma \vdash e_0\,\texttt{as}\ C[\![\overrightarrow{\tau^c}]\!]\,.\,m[\![\overrightarrow{\tau}]\!](\overrightarrow{e\,\texttt{as}\ \tau^a})\texttt{as}\ \tau^r : \tau^r}$$

[T-TYPECASE] $$\frac{\begin{array}{c}p; \Delta; \Gamma \vdash e : \tau \qquad p; \Delta; \Gamma\ x : \tau_i \vdash e_i : \tau_i^e \qquad p; \Delta \vdash \tau_i^e\ <:\ \tau' \qquad 1 \le i \le |\overrightarrow{\tau}| \\ p; \Delta; \Gamma\ x : \tau \vdash e' : \tau^{e'} \qquad p; \Delta \vdash \tau^{e'}\ <:\ \tau'\end{array}}{p; \Delta; \Gamma \vdash \texttt{typecase}\ x\ = e\,\texttt{of}\ \overrightarrow{\tau \Rightarrow e}\,\texttt{else}\ e'\,\texttt{end} : \tau'}$$

Figure A.10: Static Semantics of Core Fortress with Where Clauses (II)

Subtyping: $\boxed{p; \Delta \vdash \tau \ <: \ \tau}$

[S-OBJ] $$p; \Delta \vdash \tau \ <: \ \text{Object}$$

[S-REFL] $$p; \Delta \vdash \tau \ <: \ \tau$$

[S-TRANS] $$\frac{p; \Delta \vdash \tau_1 \ <: \ \tau_2 \qquad p; \Delta \vdash \tau_2 \ <: \ \tau_3}{p; \Delta \vdash \tau_1 \ <: \ \tau_3}$$

[S-VAR] $$\frac{\tau \in \Delta(\alpha)}{p; \Delta \vdash \alpha \ <: \ \tau}$$

[S-TAPP] $$\frac{\begin{array}{c} _ \ C[\![\overrightarrow{\alpha \texttt{extends} \ \{K\}}]\!] \texttt{extends} \ \{\overrightarrow{N}\} \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{H\}}\} \ _ \ \in p \\ p; \Delta \vdash \overrightarrow{\tau} \ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} \ <: \ [\overrightarrow{\tau^\beta}/\overrightarrow{\beta}][\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{K} \qquad p \vdash \mathit{validWitness}(\Delta, \beta \ <: \ \{[\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{H}\}, \overrightarrow{\tau^\beta}) \end{array}}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!] \ <: \ [\overrightarrow{\tau^\beta}/\overrightarrow{\beta}][\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i}$$

Well-formed types: $\boxed{p; \Delta \vdash \tau \ \mathsf{ok}}$

[W-OBJ] $$p; \Delta \vdash \text{Object} \ \mathsf{ok}$$

[W-VAR] $$\frac{\alpha \in \mathit{dom}(\Delta)}{p; \Delta \vdash \alpha \ \mathsf{ok}}$$

[W-TAPP] $$\frac{\begin{array}{c} _ \ C[\![\overrightarrow{\alpha \texttt{extends} \ \{K\}}]\!] \ _ \ \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{H\}}\} \ _ \ \in p \\ p; \Delta \vdash \overrightarrow{\tau} \ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} \ <: \ [\overrightarrow{\tau^\beta}/\overrightarrow{\beta}][\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{K} \qquad p \vdash \mathit{validWitness}(\Delta, \beta \ <: \ \{[\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{H}\}, \overrightarrow{\tau^\beta}) \end{array}}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!] \ \mathsf{ok}}$$

Method type lookup: $\boxed{\mathit{mtype}_p(m, \tau, \Delta) = \{(\eta, \Delta)\}}$

[MT-SELF] $$\frac{\begin{array}{c} _ \ C[\![\overrightarrow{\alpha \texttt{extends} \ _}]\!] \ _ \ \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{H\}}\} \ _ \ \overrightarrow{md} \ _ \ \in p \qquad m[\![\overrightarrow{\alpha' \texttt{extends} \ \{K'\}}]\!](\overrightarrow{_ : \tau'}) : \tau_0 \ _ \ \in \{\overrightarrow{md}\} \\ \Delta' = \Delta \ \overrightarrow{\beta \ <: \ \{H\}} \end{array}}{\mathit{mtype}_p(m, C[\![\overrightarrow{\tau}]\!], \Delta) = \{([\overrightarrow{\tau}/\overrightarrow{\alpha}][\![\overrightarrow{\alpha' \texttt{extends} \ \{K'\}}]\!] \ \overrightarrow{\tau'} \to \tau_0, [\overrightarrow{\tau}/\overrightarrow{\alpha}]\Delta')\}}$$

[MT-SUPER] $$\frac{\begin{array}{c} _ \ C[\![\overrightarrow{\alpha \texttt{extends} \ _}]\!] \texttt{extends} \ \{\overrightarrow{N}\} \texttt{where} \ \{\overrightarrow{\beta \texttt{extends} \ \{H\}}\} \ _ \ \overrightarrow{md} \ _ \ \in p \qquad m \notin \{\overrightarrow{\mathit{Name}(md)}\} \\ \Delta' = \Delta \ \overrightarrow{\beta \ <: \ \{H\}} \end{array}}{\mathit{mtype}_p(m, C[\![\overrightarrow{\tau}]\!], \Delta) = \bigcup_{N_i \in \{\overrightarrow{N}\}} \mathit{mtype}_p(m, [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i, [\overrightarrow{\tau}/\overrightarrow{\alpha}]\Delta')}$$

[MT-OBJ] $$\mathit{mtype}_p(m, \text{Object}, \Delta) = \emptyset$$

Bound of type: $\boxed{\mathit{bound}_\Delta(\tau) = \{\overrightarrow{\sigma}\}}$

$$\begin{array}{lcl} \mathit{bound}_\Delta(\alpha) & = & \bigcup_{\tau_i \in \Delta(\alpha)} \mathit{bound}_\Delta(\tau_i) \\ \mathit{bound}_\Delta(\sigma) & = & \{\sigma\} \end{array}$$

Figure A.11: Static Semantics of Core Fortress with Where Clauses (III)

Traits defining a method: $\boxed{\mathit{defining}_p(m,\, N) = \{\overrightarrow{N}\}}$

$$\mathit{defining}_p(m, \mathrm{Object}) = \emptyset$$

$$\mathit{defining}_p(m, C[\![\overrightarrow{\tau}]\!]) = \begin{cases} \bigcup_{N_i \in \{\overrightarrow{N}\}} \mathit{defining}_p(m, [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i) & \\ \qquad \text{if } _\, C[\![\overrightarrow{\alpha _}]\!] \,_\, \texttt{extends}\, \{\overrightarrow{N}\} \,_ \in p \text{ and } m \notin \mathit{defined}_p(C) & \\ \bigcup_{N_i \in \{\overrightarrow{N}\}} \mathit{defining}_p(m, [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i) \cup \{C[\![\overrightarrow{\tau}]\!]\} & \\ \qquad \text{if } _\, C[\![\overrightarrow{\alpha _}]\!] \,_\, \texttt{extends}\, \{\overrightarrow{N}\} \,_ \in p \text{ and } m \in \mathit{defined}_p(C) & \end{cases}$$

Auxiliary functions for methods: $\boxed{\mathit{defined}_p \text{ / } \mathit{inherited}_p \text{ / } \mathit{visible}_p(C) = \{\overrightarrow{m}\}}$

$$\mathit{defined}_p(C) = \{\overrightarrow{\mathit{Name}(md)}\} \qquad \text{where } _\ C \,_\, \overrightarrow{md} \,_\ \in p$$

$$\mathit{inherited}_p(C) = \biguplus_{N_i \in \{\overrightarrow{N}\}} \{m_i \mid m_i \in \mathit{visible}_p(N_i), m_i \notin \mathit{defined}_p(C)\} \qquad \text{where } _\ C \,_\, \texttt{extends}\, \{\overrightarrow{N}\} \,_ \in p$$

$$\mathit{visible}_p(C) = \mathit{defined}_p(C) \uplus \mathit{inherited}_p(C)$$

Figure A.12: Static Semantics of Core Fortress with Where Clauses (IV)

We proved the type soundness of Core Fortress with Where Clauses using the standard technique of proving a progress theorem and a subject reduction theorem.

A.3 Core Fortress with Overloading

In this section, we define a Fortress core calculus with overloading for dotted methods and first-order functions. We call this calculus *Core Fortress with Overloading*. Core Fortress with Overloading is an extension of Basic Core Fortress with overloading.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.10
>
> This calculus predates instantiation exclusion (Section 6.3, Section 10.1). Its only condition on multiple inheritance is its rule [VALIDMETH], which constrains pairs of visible methods of one name, so, as in Basic Core Fortress, an object that extends two different instantiations of a parameterized trait declaring no method is well typed, and its subtyping rule [S-TAPP] makes the object a subtype of both instantiations; under the rule it is not a valid Fortress program. The calculus, its rules and its soundness claim are as the team wrote and proved them, and are not changed.

A.3.1 Syntax

The syntax for Core Fortress with Overloading is provided in Figure A.13.

α, β			type variables
m			function or method name
x			field name
T			trait name
O			object name
τ, τ', τ''	$::=$	α	type
	$\mid$	σ	
σ	$::=$	N	type that is not a type variable
	$\mid$	$O[\![\overrightarrow{\tau}]\!]$	
N, M, L	$::=$	$T[\![\overrightarrow{\tau}]\!]$	type that can be a type bound
	$\mid$	Object	
e	$::=$	x	expression
	$\mid$	`self`	
	$\mid$	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
	$\mid$	$e.x$	
	$\mid$	$e.m[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
	$\mid$	$m[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
md	$::=$	$m[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x{:}\tau}){:}\tau = e$	function or method definition
td	$::=$	$\texttt{trait } T[\![\overrightarrow{\alpha \texttt{ extends } N}]\!] \texttt{ extends } \{\overrightarrow{N}\} \ \overrightarrow{md} \texttt{ end}$	trait definition
od	$::=$	$\texttt{object } O[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x{:}\tau}) \texttt{ extends } \{\overrightarrow{N}\} \ \overrightarrow{md} \texttt{ end}$	object definition
d	$::=$	md	definition
	$\mid$	td	
	$\mid$	od	
p	$::=$	$\overrightarrow{d}\ e$	program

Figure A.13: Syntax of Core Fortress with Overloading

A.3.2 Dynamic Semantics

A dynamic semantics for Core Fortress with Overloading is provided in Figure A.14.

Values, evaluation contexts, and redexes

$$
\begin{array}{lcll}
v & ::= & O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}) & \text{value} \\
E & ::= & \Box & \text{evaluation context} \\
 & | & O[\![\overrightarrow{\tau}]\!](\overrightarrow{e}\,E\,\overrightarrow{e}) & \\
 & | & E.x & \\
 & | & E.m[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) & \\
 & | & e.m[\![\overrightarrow{\tau}]\!](\overrightarrow{e}\,E\,\overrightarrow{e}) & \\
 & | & m[\![\overrightarrow{\tau}]\!](\overrightarrow{e}\,E\,\overrightarrow{e}) & \\
R & ::= & v.x & \text{redex} \\
 & | & v.m[\![\overrightarrow{\tau}]\!](\overrightarrow{v}) & \\
 & | & m[\![\overrightarrow{\tau}]\!](\overrightarrow{v}) &
\end{array}
$$

Evaluation rules: $\boxed{p \vdash E[R] \longrightarrow E[e]}$

[R-FIELD]
$$\frac{\texttt{object}\ O\ _(\overrightarrow{x\colon _})\ _\ \texttt{end} \in p}{p \vdash E[O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}).x_i] \longrightarrow E[v_i]}$$

[R-METHOD]
$$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha\ \texttt{extends}\ N}]\!](\overrightarrow{x\colon _})\ _\ \texttt{end} \in p \qquad \overrightarrow{type(v')} = \overrightarrow{\tau''} \qquad mostSpecificDef_{p;\emptyset}(applicable_{p;\emptyset}(m[\![\overrightarrow{\tau'}]\!](\overrightarrow{\tau''}), visible_p(O[\![\overrightarrow{\tau}]\!]))) = m[\![\overrightarrow{\alpha'\ \texttt{extends}\ N'}]\!](\overrightarrow{x'\colon _})\colon _ = e}{p \vdash E[O[\![\overrightarrow{\tau}]\!](\overrightarrow{v}).m[\![\overrightarrow{\tau'}]\!](\overrightarrow{v'})] \longrightarrow E[[\overrightarrow{v}/\overrightarrow{x}][O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})/\texttt{self}][\overrightarrow{v'}/\overrightarrow{x'}]e]}$$

[R-FUNCTION]
$$\frac{\overrightarrow{type(v)} = \overrightarrow{\tau'} \qquad mostSpecificDef_{p;\emptyset}(applicable_{p;\emptyset}(m[\![\overrightarrow{\tau}]\!](\overrightarrow{\tau'}), \{(md, \text{Object}) \mid md \in p\})) = m[\![\overrightarrow{\alpha\ \texttt{extends}\ N}]\!](\overrightarrow{x\colon _})\colon _ = e}{p \vdash E[m[\![\overrightarrow{\tau}]\!](\overrightarrow{v})] \longrightarrow E[[\overrightarrow{v}/\overrightarrow{x}]e]}$$

Types of values: $\boxed{type(v) = \tau}$

$type(O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})) = O[\![\overrightarrow{\tau}]\!]$

Figure A.14: Dynamic Semantics of Core Fortress with Overloading

A.3.3 Static Semantics

A static semantics for Core Fortress with Overloading is provided in Figures A.15, A.16, A.17, and A.18.

Environments and trait or object names

Δ	::=	$\overrightarrow{\alpha <: N}$	bound environment
Γ	::=	$\overrightarrow{x : \tau}$	type environment
C	::=	T	trait name
	$\mid$	O	object name

Program typing: $\boxed{\vdash p : \tau}$

$$\text{[T-PROGRAM]} \quad \frac{p = \overrightarrow{d}\ e \qquad p; \emptyset; \emptyset \vdash \overrightarrow{d}\ \mathsf{ok} \qquad p \vdash \mathit{validFun}(\overrightarrow{d}) \qquad p; \emptyset; \emptyset \vdash e : \tau}{\vdash p : \tau}$$

Trait typing: $\boxed{p; \emptyset; \emptyset \vdash td\ \mathsf{ok}}$

$$\text{[T-TRAITDEF]} \quad \frac{\begin{array}{c}\Delta = \overrightarrow{\alpha <: N} \qquad p; \Delta \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{M}\ \mathsf{ok} \\ p; \Delta; \mathtt{self} : T[\![\overrightarrow{\alpha}]\!] \vdash \overrightarrow{md}\ \mathsf{ok} \qquad p \vdash \mathit{validMeth}(T)\end{array}}{p; \emptyset; \emptyset \vdash \mathtt{trait}\ T[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]\ \mathtt{extends}\ \{\overrightarrow{M}\}\ \overrightarrow{md}\ \mathtt{end}\ \mathsf{ok}}$$

Object typing: $\boxed{p; \emptyset; \emptyset \vdash od\ \mathsf{ok}}$

$$\text{[T-OBJECTDEF]} \quad \frac{\begin{array}{c}\Delta = \overrightarrow{\alpha <: N} \qquad p; \Delta \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{M}\ \mathsf{ok} \\ p; \Delta; \mathtt{self} : O[\![\overrightarrow{\alpha}]\!]\ \overrightarrow{x : \tau} \vdash \overrightarrow{md}\ \mathsf{ok} \qquad p \vdash \mathit{validMeth}(O)\end{array}}{p; \emptyset; \emptyset \vdash \mathtt{object}\ O[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!](\overrightarrow{x{:}\tau})\ \mathtt{extends}\ \{\overrightarrow{M}\}\ \overrightarrow{md}\ \mathtt{end}\ \mathsf{ok}}$$

Function and method typing: $\boxed{p; \Delta; \Gamma \vdash md\ \mathsf{ok}}$

$$\text{[T-FUNMETHDEF]} \quad \frac{\begin{array}{c}\Delta' = \Delta\ \overrightarrow{\alpha <: \{\overrightarrow{N}\}} \qquad p; \Delta' \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p; \Delta' \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta' \vdash \tau_0\ \mathsf{ok} \\ p; \Delta'; \Gamma\ \overrightarrow{x : \tau} \vdash e : \tau' \qquad p; \Delta' \vdash \tau' <: \tau_0\end{array}}{p; \Delta; \Gamma \vdash m[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!](\overrightarrow{x{:}\tau}){:}\tau_0 = e\ \mathsf{ok}}$$

Valid method declarations: $\boxed{p \vdash \mathit{validMeth}(C)}$

$$\text{[VALIDMETH]} \quad \frac{\begin{array}{l}\forall\ (md, C[\![\overrightarrow{\tau^c}]\!]), (md', C'[\![\overrightarrow{\tau^{c'}}]\!]) \in \mathit{visible}_p(C^o[\![\overrightarrow{\alpha^b}]\!]). \\ \quad \text{where} \quad _\ C^o[\![\overrightarrow{\alpha^o\ \mathtt{extends}\ _}]\!]\ _ \in p, \\ \qquad md \neq md' \quad \text{(not same declaration)}, \\ \qquad md = m[\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!](\overrightarrow{_{:}\tau}){:}\tau^r = _\,, \quad md' = m[\![\overrightarrow{\alpha'\ \mathtt{extends}\ N'}]\!](\overrightarrow{_{:}\tau'}){:}\tau^{r'} = _\,, \\ p \vdash \mathit{valid}([\![\overrightarrow{\alpha\ \mathtt{extends}\ N}]\!]C[\![\overrightarrow{\tau^c}]\!]\overrightarrow{\tau} \to \tau^r, [\![\overrightarrow{\alpha'\ \mathtt{extends}\ N'}]\!]C'[\![\overrightarrow{\tau^{c'}}]\!]\overrightarrow{\tau'} \to \tau^{r'}, \mathit{visible}_p(C^o[\![\overrightarrow{\alpha^b}]\!]))\end{array}}{p \vdash \mathit{validMeth}(C^o)}$$

Figure A.15: Static Semantics of Core Fortress with Overloading (I)

Valid function declarations: $\boxed{p \vdash \mathit{validFun}(\overrightarrow{d})}$

$$\text{[VALIDFUN]}\quad \frac{\begin{array}{l}\forall\, md, md' \in \overrightarrow{d}\,. \\ \text{where}\quad md \neq md' \quad \text{(not same declaration)}, \\ \qquad md = m[\![\overrightarrow{\alpha\,\texttt{extends}\,N}]\!](\overrightarrow{_:\tau}):\tau^r = _\,, \quad md' = m[\![\overrightarrow{\alpha'\,\texttt{extends}\,N'}]\!](\overrightarrow{_:\tau'}):\tau^{r'} = _\,, \\ p \vdash \mathit{valid}([\![\overrightarrow{\alpha\,\texttt{extends}\,N}]\!]\mathrm{Object}\overrightarrow{\tau} \to \tau^r, [\![\overrightarrow{\alpha'\,\texttt{extends}\,N'}]\!]\mathrm{Object}\overrightarrow{\tau'} \to \tau^{r'}, \{(md, \mathrm{Object}) \mid md \in \overrightarrow{d}\})\end{array}}{p \vdash \mathit{validFun}(\overrightarrow{d})}$$

Valid declarations: $\boxed{p \vdash \mathit{valid}(\overrightarrow{[\![\alpha\,\texttt{extends}\,N]\!]}\overrightarrow{\tau} \to \tau, \overrightarrow{[\![\alpha\,\texttt{extends}\,N]\!]}\overrightarrow{\tau} \to \tau, \{\overrightarrow{(md, \tau)}\})}$

$$\text{[VALID]}\quad \frac{\begin{array}{l}\Delta = \overrightarrow{\alpha\ <:\ \{N\}}, \quad |\overrightarrow{\tau}| = n \\ \quad 1. \quad |\overrightarrow{\tau}| \neq |\overrightarrow{\tau'}| \\ \vee\ \ 2. \quad 1)\ \overrightarrow{N} = [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\overrightarrow{N'} \\ \qquad \wedge\ \ 2)\ \forall\, 1 \leq i \leq n.\ p; \Delta \vdash \tau_i\ <:\ [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\tau_i' \quad \vee \quad p; \Delta \vdash [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\tau_i'\ <:\ \tau_i \\ \qquad \wedge\ \ 3)\ \exists\, 1 \leq i \leq n.\ \tau_i \neq [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\tau_i' \\ \qquad \wedge\ \ 4)\ \exists\, (m[\![\overrightarrow{\alpha''\,\texttt{extends}\,N''}]\!](\overrightarrow{_:\tau''}):\tau^{r''} = _\,, \tau_0'') \in S. \\ \qquad\quad \text{where} \\ \qquad\qquad \forall\, 0 \leq i \leq n. \quad p; \Delta \vdash \mathit{mostSpecificType}(\{\tau_i, [\overrightarrow{\alpha}/\overrightarrow{\alpha'}]\tau_i'\}, [\overrightarrow{\alpha}/\overrightarrow{\alpha''}]\tau_i'') \\ \qquad\quad\ \wedge\ \ \overrightarrow{N} = [\overrightarrow{\alpha}/\overrightarrow{\alpha''}]\overrightarrow{N''} \\ \qquad\quad\ \wedge\ \ p; \Delta \vdash [\overrightarrow{\alpha}/\overrightarrow{\alpha''}]\tau^{r''}\ <:\ \tau^r \\ \qquad\quad\ \wedge\ \ p; \Delta \vdash [\overrightarrow{\alpha'}/\overrightarrow{\alpha''}]\tau^{r''}\ <:\ \tau^{r'}\end{array}}{p \vdash \mathit{valid}(\overrightarrow{[\![\alpha\,\texttt{extends}\,N]\!]}\overrightarrow{\tau} \to \tau^r, \overrightarrow{[\![\alpha'\,\texttt{extends}\,N']\!]}\overrightarrow{\tau'} \to \tau^{r'}, S)}$$

Expression typing: $\boxed{p; \Delta; \Gamma \vdash e : \tau}$

$$\text{[T-VAR]}\quad p; \Delta; \Gamma \vdash x : \Gamma(x)$$

$$\text{[T-SELF]}\quad p; \Delta; \Gamma \vdash \texttt{self} : \Gamma(\texttt{self})$$

$$\text{[T-OBJECT]}\quad \frac{\texttt{object}\ O[\![\overrightarrow{\alpha\texttt{extends}\ _}]\!](\overrightarrow{_:\tau'})\ _\ \texttt{end} \in p \qquad p; \Delta \vdash O[\![\overrightarrow{\tau}]\!]\,\texttt{ok} \qquad p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau''} \qquad p; \Delta \vdash \overrightarrow{\tau''}\ <:\ [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau'}}{p; \Delta; \Gamma \vdash O[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) : O[\![\overrightarrow{\tau}]\!]}$$

$$\text{[T-FIELD]}\quad \frac{p; \Delta; \Gamma \vdash e_0 : \tau_0 \qquad \mathit{bound}_\Delta(\tau_0) = O[\![\overrightarrow{\tau'}]\!] \qquad \texttt{object}\ O[\![\overrightarrow{\alpha\texttt{extends}\ _}]\!](\overrightarrow{x:\tau})\ _\ \texttt{end} \in p}{p; \Delta; \Gamma \vdash e_0\,.\,x_i : [\overrightarrow{\tau'}/\overrightarrow{\alpha}]\tau_i}$$

$$\text{[T-METHOD]}\quad \frac{p; \Delta; \Gamma \vdash e_0 : \tau_0 \qquad p; \Delta \vdash \overrightarrow{\tau}\ \texttt{ok} \qquad p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau'} \qquad \mathit{mostSpecificDef}_{p;\Delta}(\mathit{applicable}_{p;\Delta}(m[\![\overrightarrow{\tau}]\!][\overrightarrow{\tau'}), \mathit{visible}_p(\mathit{bound}_\Delta(\tau_0)))) = m[\![\overrightarrow{\alpha\texttt{extends}\ N}]\!](_):\tau^r\ _}{p; \Delta; \Gamma \vdash e_0.m[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) : \tau^r}$$

$$\text{[T-FUNCTION]}\quad \frac{p; \Delta \vdash \overrightarrow{\tau}\ \texttt{ok} \qquad p; \Delta; \Gamma \vdash \overrightarrow{e} : \overrightarrow{\tau'} \qquad \mathit{mostSpecificDef}_{p;\Delta}(\mathit{applicable}_{p;\Delta}(m[\![\overrightarrow{\tau}]\!][\overrightarrow{\tau'}), \{(md, \mathrm{Object}) \mid md \in p\}))) = m[\![\overrightarrow{\alpha\texttt{extends}\ N}]\!](_):\tau^r\ _}{p; \Delta; \Gamma \vdash m[\![\overrightarrow{\tau}]\!](\overrightarrow{e}) : \tau^r}$$

Figure A.16: Static Semantics of Core Fortress with Overloading (II)

Subtyping: $\boxed{p; \Delta \vdash \tau \ <: \ \tau}$

[S-OBJ] $$p; \Delta \vdash \tau \ <: \ \text{Object}$$

[S-REFL] $$p; \Delta \vdash \tau \ <: \ \tau$$

[S-TRANS] $$\frac{p; \Delta \vdash \tau_1 \ <: \ \tau_2 \qquad p; \Delta \vdash \tau_2 \ <: \ \tau_3}{p; \Delta \vdash \tau_1 \ <: \ \tau_3}$$

[S-VAR] $$p; \Delta \vdash \alpha \ <: \ \Delta(\alpha)$$

[S-TAPP] $$\frac{_ \ C[\![\overrightarrow{\alpha \texttt{ extends } _}]\!] \ _ \ \texttt{extends} \{\overrightarrow{N}\} \ _ \in p}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!] \ <: \ [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i}$$

Well-formed types: $\boxed{p; \Delta \vdash \tau \ \mathsf{ok}}$

[W-OBJ] $$p; \Delta \vdash \text{Object} \ \mathsf{ok}$$

[W-VAR] $$\frac{\alpha \in \mathit{dom}(\Delta)}{p; \Delta \vdash \alpha \ \mathsf{ok}}$$

[W-TAPP] $$\frac{_ \ C[\![\overrightarrow{\alpha \texttt{ extends } N}]\!] \ _ \in p \qquad p; \Delta \vdash \overrightarrow{\tau} \ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} \ <: \ [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{N}}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!] \ \mathsf{ok}}$$

Most specific definitions: $\boxed{\mathit{mostSpecificDef}_{p;\Delta}(\{\overrightarrow{(md, \tau)}\}) = md}$

$$\frac{\begin{array}{c}\overrightarrow{md} = m[\![(\overrightarrow{\alpha \texttt{ extends } N})_1]\!]((\overrightarrow{_ : \tau^a})_1) : \tau_1^r \ _ \cdots m[\![(\overrightarrow{\alpha \texttt{ extends } N})_n]\!]((\overrightarrow{_ : \tau^a})_n) : \tau_n^r \ _ \\ 1 \le i \le n \qquad (\overrightarrow{\tau^a})_i = \tau_{i1}^a \cdots \tau_{im}^a \qquad \tau_{i0}^a = \tau_i \qquad \forall \, 0 \le j \le m \, . \, p; \Delta \vdash \mathit{mostSpecificType}(\{\tau_{1j}^a, \cdots, \tau_{nj}^a\}, \tau_{ij}^a)\end{array}}{\mathit{mostSpecificDef}_{p;\Delta}(\{\overrightarrow{(md, \tau)}\}) = md_i}$$

Applicable definitions: $\boxed{\mathit{applicable}_{p;\Delta}(m[\![\overrightarrow{\tau}]\!](\overrightarrow{\tau}), \{\overrightarrow{(md, \tau)}\}) = \{\overrightarrow{(md, \tau)}\}}$

$$\mathit{applicable}_{p;\Delta}(m[\![\overrightarrow{\tau'}]\!](\overrightarrow{\tau''}), S) = \left\{ \begin{array}{ll} \{([\overrightarrow{\tau'}/\overrightarrow{\alpha}]md, \tau) \mid & \text{where } (md, \tau) \in S, \\ & md = m[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x : \tau'''}) : _ \, , \\ & p; \Delta \vdash \overrightarrow{\tau''} \ <: \ [\overrightarrow{\tau'}/\overrightarrow{\alpha}]\overrightarrow{\tau'''} \\ & p; \Delta \vdash \overrightarrow{\tau'} \ <: \ [\overrightarrow{\tau'}/\overrightarrow{\alpha}]\overrightarrow{N}\} \end{array} \right.$$

Figure A.17: Static Semantics of Core Fortress with Overloading (III)

Method definition lookup: $\boxed{visible_p \,/\, defined_p(C[\![\overrightarrow{\tau}]\!]) = \{\overrightarrow{(md, C[\![\overrightarrow{\tau}]\!])}\}}$

$$visible_p(C[\![\overrightarrow{\tau}]\!]) = defined_p(C[\![\overrightarrow{\tau}]\!]) \cup \bigcup_{C'[\![\overrightarrow{\tau'}]\!] \in \{\overrightarrow{N}\}} visible_p([\overrightarrow{\tau}/\overrightarrow{\alpha}]C'[\![\overrightarrow{\tau'}]\!]) \quad \text{where } _\ C[\![\overrightarrow{\alpha}\ \overrightarrow{_}]\!]\ _\ \texttt{extends}\ \{\overrightarrow{N}\}\ _ \in p$$

$$defined_p(C[\![\overrightarrow{\tau}]\!]) = \{\overrightarrow{([\overrightarrow{\tau}/\overrightarrow{\alpha}]md, C[\![\overrightarrow{\tau}]\!])}\} \quad \text{where } _\ C[\![\overrightarrow{\alpha}\ \overrightarrow{_}]\!]\ _\ \overrightarrow{md}\ _ \in p$$

Most specific type: $\boxed{p; \Delta \vdash mostSpecificType(\{\overrightarrow{\tau}\}, \tau)}$

$$[\textsc{Meet}] \quad \frac{\tau' \in \{\overrightarrow{\tau}\} \qquad \forall\, 1 \le i \le |\overrightarrow{\tau}|\ .\ p; \Delta \vdash \tau' <: \tau_i}{p; \Delta \vdash mostSpecificType(\{\overrightarrow{\tau}\}, \tau')}$$

Bound of type: $\boxed{bound_\Delta(\tau) = \sigma}$

$$\begin{array}{lcl} bound_\Delta(\alpha) & = & \Delta(\alpha) \\ bound_\Delta(\sigma) & = & \sigma \end{array}$$

Figure A.18: Static Semantics of Core Fortress with Overloading (IV)

We proved the type soundness of Core Fortress with Overloading using the standard technique of proving a progress theorem and a subject reduction theorem.

A.4 Acyclic Core Fortress with Field Definitions

In this section, we define a Fortress core calculus with acyclic type hierarchy and field definitions inside object definitions. We call this calculus *Acyclic Core Fortress with Field Definitions*. Acyclic Core Fortress with Field Definitions is an extension of Basic Core Fortress with acyclic type hierarchy and field definitions inside object definitions.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.10
>
> This calculus predates instantiation exclusion (Section 6.3, Section 10.1). Conditions (2) and (3) of its rule [ACYCLIC] admit one self-extension, whose static arguments may be the parameters' own bounds, so that a trait that extends an instantiation of itself at its parameter's bound, such as `trait` $\mathrm{T}[\![X$ `extends` $\mathrm{Object}]\!]$ `extends` $\mathrm{T}[\![\mathrm{Object}]\!]$ `end`, is well formed, and $\mathrm{T}[\![A]\!]$ is then a subtype of $\mathrm{T}[\![\mathrm{Object}]\!]$ for every A; Fortress no longer allows such a trait, for the reason the covariance example of Section 12.6 is not allowed. The calculus, its rules and its soundness claim are as the team wrote and proved them, and are not changed.

A.4.1 Syntax

The syntax for Acyclic Core Fortress with Field Definitions is provided in Figure A.19.

A.4.2 Dynamic Semantics

A dynamic semantics for Acyclic Core Fortress with Field Definitions is provided in Figure A.20.

α, β			type variables
m			method name
x, y, z			field names
T			trait name
O			object name
τ, τ', τ''	$::=$	α	type
	$\mid$	σ	
σ	$::=$	N	type that is not a type variable
	$\mid$	$O[\![\overrightarrow{\tau}]\!]$	
N, M, L	$::=$	$T[\![\overrightarrow{\tau}]\!]$	type that can be a type bound
	$\mid$	Object	
e	$::=$	x	expression
	$\mid$	`self`	
	$\mid$	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
	$\mid$	$e.x$	
	$\mid$	$e.m[\![\overrightarrow{\tau}]\!](\overrightarrow{e})$	
md	$::=$	$m[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x:\tau}):\tau = e$	method definition
vd	$::=$	$x:\tau = e$	field definition
td	$::=$	`trait` $T[\![\overrightarrow{\alpha \texttt{ extends } N}]\!]$ `extends` $\{\overrightarrow{N}\}$ $\overrightarrow{md}$ `end`	trait definition
od	$::=$	`object` $O[\![\overrightarrow{\alpha \texttt{ extends } N}]\!](\overrightarrow{x:\tau})$ `extends` $\{\overrightarrow{N}\}$ $\overrightarrow{md}$ `end`	object definition
d	$::=$	td	definition
	$\mid$	od	
p	$::=$	$\overrightarrow{d}\ e$	program

Figure A.19: Syntax of Acyclic Core Fortress with Field Definitions

A.4.3 Static Semantics

A static semantics for Acyclic Core Fortress with Field Definitions is provided in Figures A.21, A.22, and A.23.

We proved the type soundness of Acyclic Core Fortress with Field Definitions and the acyclic type hierarchy of a well-type program in Acyclic Core Fortress with Field Definitions using the standard technique of proving a progress theorem and a subject reduction theorem.

Values, intermediate expressions, evaluation contexts, redexes, and trait and object names

v	::=	$O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\}$	value
ϵ	::=	x	intermediate expression
	|	`self`	
	|	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon})$	
	|	$\epsilon.x$	
	|	$\epsilon.m[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon})$	
	|	$O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto \epsilon}; \overrightarrow{x \mapsto \epsilon}\}$	
E	::=	$\square$	evaluation context
	|	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon}\, E\, \overrightarrow{\epsilon})$	
	|	$E.x$	
	|	$E.m[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon})$	
	|	$\epsilon.m[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon}\, E\, \overrightarrow{\epsilon})$	
	|	$O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\ x \mapsto E\ \overrightarrow{x \mapsto e}\}$	
R	::=	$O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})$	redex
	|	$O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\ x \mapsto v\ \overrightarrow{x \mapsto e}\}$	
	|	$v.x$	
	|	$v.m[\![\overrightarrow{\tau}]\!](\overrightarrow{v})$	
C	::=	T	trait name
	|	O	object name

Evaluation rules: $\boxed{p \vdash E[R] \longrightarrow E[e]}$

[R-OBJECT]
$$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha\ \texttt{extends}\ _}]\!](\overrightarrow{x:_})\ _\ \overrightarrow{x':_ = e'}\ _\ \texttt{end} \in p}{p \vdash E[O[\![\overrightarrow{\tau}]\!](\overrightarrow{v})] \longrightarrow E[O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v}\ ; \overrightarrow{x' \mapsto [\overrightarrow{v}/\overrightarrow{x}][\overrightarrow{\tau}/\overrightarrow{\alpha}]e'}\}]}$$

[R-SUB]
$$p \vdash E[O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\ y \mapsto v\ \overrightarrow{z \mapsto e}\}] \longrightarrow E[O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v}\ y \mapsto v;\ \overrightarrow{z \mapsto [v/y]e}\}]$$

[R-FIELD]
$$p \vdash E[O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\}.x_i] \longrightarrow E[v_i]$$

[R-METHOD]
$$\frac{mbody_p(m[\![\overrightarrow{\tau'}]\!], O[\![\overrightarrow{\tau}]\!]) = \{(\overrightarrow{x'}) \to e\}}{p \vdash E[O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\}.m[\![\overrightarrow{\tau'}]\!](\overrightarrow{v'})] \longrightarrow E[[\overrightarrow{v}/\overrightarrow{x}][O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x \mapsto v};\}/\texttt{self}][\overrightarrow{v'}/\overrightarrow{x'}]e]}$$

Method body lookup: $\boxed{mbody_p(m[\![\overrightarrow{\tau}]\!], \tau) = \{(\overrightarrow{x}) \to e\}}$

[MB-SELF]
$$\frac{_\ C[\![\overrightarrow{\alpha\ \texttt{extends}\ _}]\!]\ _\ \overrightarrow{md}\ _ \in p \qquad m[\![\overrightarrow{\alpha'\ \texttt{extends}\ _}]\!](\overrightarrow{x':_}):_ = e \in \{\overrightarrow{md}\}}{mbody_p(m[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!]) = \{[\overrightarrow{\tau'}/\overrightarrow{\alpha'}][\overrightarrow{\tau}/\overrightarrow{\alpha}](\overrightarrow{x'}) \to e\}}$$

[MB-SUPER]
$$\frac{_\ C[\![\overrightarrow{\alpha\ \texttt{extends}\ _}]\!]\ _\ \texttt{extends}\{\overrightarrow{K}\}\ _\ \overrightarrow{md}\ _ \in p \qquad m \notin \{\overrightarrow{Name(md)}\}}{mbody_p(m[\![\overrightarrow{\tau'}]\!], C[\![\overrightarrow{\tau}]\!]) = \bigcup_{N_i \in \{\overrightarrow{N}\}} mbody_p(m[\![\overrightarrow{\tau'}]\!], [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i)}$$

[MB-OBJ]
$$mbody_p(m[\![\overrightarrow{\tau}]\!], \mathrm{Object}) = \emptyset$$

Function/method name lookup: $\boxed{Name(md) = m}$

$Name(m[\![\overrightarrow{\alpha\ \texttt{extends}\ N}]\!](\overrightarrow{x:\tau}):\tau = e) = m$

Figure A.20: Dynamic Semantics of Acyclic Core Fortress with Field Definitions

Environments

$\Delta \quad ::= \quad \overrightarrow{\alpha <: N}$ bound environment
$\Gamma \quad ::= \quad \overrightarrow{x : \tau}$ type environment

Program typing: $\boxed{\vdash p : \tau}$

[T-PROGRAM] $$\frac{p = \overrightarrow{d}\ e \qquad p;\emptyset;\emptyset \vdash \overrightarrow{d}\ \mathsf{ok} \qquad p;\emptyset;\emptyset \vdash e : \tau \qquad acyclic(\overrightarrow{d})}{\vdash p : \tau}$$

Acyclic type hierarchy: $\boxed{acyclic(\overrightarrow{d})}$

[ACYCLIC] $$\frac{\begin{array}{c}\mathtt{trait}\ T[\![\overrightarrow{\alpha \mathtt{extends}\ N}]\!] \mathtt{extends}\ \{\overrightarrow{M}\}\ _\ \mathtt{end} \in \overrightarrow{d} \qquad 1 \le i \le |\overrightarrow{\alpha}| \\ (1)\quad M_i \neq T[\![_]\!] \quad \text{implies} \quad p;\ _ \vdash [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{M}_i \not<: T[\![\overrightarrow{\tau}]\!] \\ \textit{(The type names excluding self-extensions form an acyclic hierarchy.)} \\ (2)\quad M_i = T[\![_]\!] \quad \text{implies} \quad M_j \neq T[\![_]\!] \quad 1 \le j \le |\overrightarrow{M}| \quad i \neq j \\ (3)\quad M_i = T[\![\overrightarrow{\tau}]\!] \quad \text{implies} \quad (\tau_i = \alpha_i) \vee (\tau_i = N_i) \vee (\tau_i = \alpha_j \wedge \alpha_i = N_j) \quad 1 \le j \le |\overrightarrow{\alpha}| \quad i \neq j\end{array}}{acyclic(\overrightarrow{d})}$$

Definition typing: $\boxed{p;\emptyset;\emptyset \vdash d\ \mathsf{ok}}$

[T-TRAITDEF] $$\frac{\Delta = \overrightarrow{\alpha <: N} \qquad p;\Delta \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p;\Delta \vdash \overrightarrow{M}\ \mathsf{ok} \qquad p;\Delta;\mathtt{self} : T[\![\overrightarrow{\alpha}]\!] \vdash T\ \mathsf{ok}\overrightarrow{md} \qquad p \vdash oneOwner(T)}{p;\emptyset;\emptyset \vdash \mathtt{trait}\ T[\![\overrightarrow{\alpha \mathtt{extends}\ N}]\!] \mathtt{extends}\ \{\overrightarrow{M}\}\ \overrightarrow{md}\ \mathtt{end}\ \mathsf{ok}}$$

[T-OBJECTDEF] $$\frac{\begin{array}{c}\Delta = \overrightarrow{\alpha <: N} \qquad p;\Delta \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p;\Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p;\Delta \vdash \overrightarrow{M}\ \mathsf{ok} \\ \overrightarrow{vd} = \overrightarrow{x' : \tau' = e} \qquad p;\Delta;\overrightarrow{x : \tau}\ x'_1 : \tau'_1\ \ldots\ x'_{i-1} : \tau'_{i-1} \vdash x'_i : \tau'_i = e_i\ \mathsf{ok} \quad 1 \le i \le n \\ p;\Delta;\mathtt{self} : O[\![\overrightarrow{\alpha}]\!]\ \overrightarrow{x : \tau}\ \overrightarrow{x' : \tau'} \vdash O\ \mathsf{ok}\overrightarrow{md} \qquad p \vdash oneOwner(O)\end{array}}{p;\emptyset;\emptyset \vdash \mathtt{object}\ O[\![\overrightarrow{\alpha \mathtt{extends}\ N}]\!](\overrightarrow{x : \tau}) \mathtt{extends}\ \{\overrightarrow{M}\}\ \overrightarrow{md}\ \mathtt{end}\ \mathsf{ok}}$$

One owner for all the visible methods: $\boxed{p \vdash oneOwner(C)}$

[ONEOWNER] $$\frac{\forall m \in visible_p(C)\ .\ f\ \textit{only occurs once in}\ visible_p(C)}{p \vdash oneOwner(C)}$$

Method typing: $\boxed{p;\Delta;\Gamma \vdash C\ \mathsf{ok}md}$

[T-METHODDEF] $$\frac{\begin{array}{c}_\ C[\![\overrightarrow{\alpha' \mathtt{extends}\ _}]\!]\ _\ \mathtt{extends}\{\overrightarrow{K}\}\ _ \in p \qquad p \vdash override(m, \{\overrightarrow{M}\}, [\![\overrightarrow{\alpha \mathtt{extends}\ N}]\!]\ \overrightarrow{\tau} \to \tau_0) \\ \Delta' = \Delta\ \overrightarrow{\alpha <: N} \qquad p;\Delta' \vdash \overrightarrow{N}\ \mathsf{ok} \qquad p;\Delta' \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p;\Delta' \vdash \tau_0\ \mathsf{ok} \\ p;\Delta';\Gamma\ \overrightarrow{x : \tau} \vdash e : \tau' \qquad p;\Delta' \vdash \tau' <: \tau_0\end{array}}{p;\Delta;\Gamma \vdash C\ \mathsf{ok}m[\![\overrightarrow{\alpha \mathtt{extends}\ N}]\!](\overrightarrow{x : \tau}) : \tau_0 = e}$$

Figure A.21: Static Semantics of Acyclic Core Fortress with Field Definitions (I)

Field typing: $\boxed{p; \Delta; \Gamma \vdash vd\ \mathsf{ok}}$

[T-FIELDDEF] $$\frac{p; \Delta \vdash \tau\ \mathsf{ok} \qquad p; \Delta; \Gamma \vdash e : \tau' \qquad p; \Delta \vdash \tau' <: \tau}{p; \Delta; \Gamma \vdash x\texttt{:}\ \tau = e\ \mathsf{ok}}$$

Expression typing: $\boxed{p; \Delta; \Gamma \vdash \epsilon : \tau}$

[T-VAR] $$p; \Delta; \Gamma \vdash x : \Gamma(x)$$

[T-SELF] $$p; \Delta; \Gamma \vdash \texttt{self} : \Gamma(\texttt{self})$$

[T-OBJECT] $$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{_ : \tau}) _\ \texttt{end} \in p \qquad p; \Delta \vdash O[\![\overrightarrow{\tau}]\!]\ \mathsf{ok} \qquad p; \Delta; \Gamma \vdash \overrightarrow{\epsilon} : \overrightarrow{\tau''} \qquad p; \Delta \vdash \overrightarrow{\tau''} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau'}}{p; \Delta; \Gamma \vdash O[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon}) : O[\![\overrightarrow{\tau}]\!]}$$

[T-INT-OBJECT] $$\frac{\texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{x' : \tau'})\ _\ \overrightarrow{x'' : \tau'' = _}\ _\ \texttt{end} \in p \qquad \overrightarrow{x^1}\overrightarrow{x^2} = \overrightarrow{x'}\overrightarrow{x''} \qquad \overrightarrow{\epsilon^1}\overrightarrow{\epsilon^2} = \overrightarrow{\epsilon'}\overrightarrow{\epsilon''} \qquad p; \Delta; \Gamma \vdash O[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon}) : O[\![\overrightarrow{\tau}]\!] \qquad p; \Delta; \Gamma\ \overrightarrow{x' : \tau'}\ \overrightarrow{x'' : \tau''} \vdash \overrightarrow{\epsilon''} : \overrightarrow{\tau'''} \qquad p; \Delta \vdash [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau'''} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau''}}{p; \Delta; \Gamma \vdash O[\![\overrightarrow{\tau}]\!]\{\overrightarrow{x^1 \mapsto \epsilon^1};\ \overrightarrow{x^2 \mapsto \epsilon^2}\} : O[\![\overrightarrow{\tau}]\!]}$$

[T-FIELD] $$\frac{p; \Delta; \Gamma \vdash \epsilon : \tau_0 \qquad bound_\Delta(\tau_0) = O[\![\overrightarrow{\tau^o}]\!] \qquad \texttt{object}\ O[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!](\overrightarrow{x' : \tau'})\ _\ \overrightarrow{x'' : \tau'' = _}\ _\ \texttt{end} \in p \qquad \overrightarrow{x} = \overrightarrow{x'}\overrightarrow{x''} \qquad \overrightarrow{\tau} = \overrightarrow{\tau'}\overrightarrow{\tau''}}{p; \Delta; \Gamma \vdash \epsilon\texttt{.}\,x_i : [\overrightarrow{\tau^o}/\overrightarrow{\alpha}]\tau_i}$$

[T-METHOD] $$\frac{p; \Delta; \Gamma \vdash \epsilon : \tau_0 \qquad mtype_{p;\Delta}(m, bound_\Delta(\tau_0)) = \{[\![\overrightarrow{\alpha \texttt{extends}\ N}]\!]\ \overrightarrow{\tau'} \to \tau_0'\} \qquad p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{N} \qquad p; \Delta; \Gamma \vdash \overrightarrow{\epsilon} : \overrightarrow{\tau''} \qquad p; \Delta \vdash \overrightarrow{\tau''} <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{\tau'}}{p; \Delta; \Gamma \vdash \epsilon m[\![\overrightarrow{\tau}]\!](\overrightarrow{\epsilon}) : [\overrightarrow{\tau}/\overrightarrow{\alpha}]\tau_0'}$$

Subtyping: $\boxed{p; \Delta \vdash \tau <: \tau}$

[S-OBJ] $$p; \Delta \vdash \tau <: \mathrm{Object}$$

[S-REFL] $$p; \Delta \vdash \tau <: \tau$$

[S-TRANS] $$\frac{p; \Delta \vdash \tau_1 <: \tau_2 \qquad p; \Delta \vdash \tau_2 <: \tau_3}{p; \Delta \vdash \tau_1 <: \tau_3}$$

[S-VAR] $$p; \Delta \vdash \alpha <: \Delta(\alpha)$$

[S-TAPP] $$\frac{_\ C[\![\overrightarrow{\alpha \texttt{extends}\ _}]\!]\ _\ \texttt{extends}\{\overrightarrow{N}\}\ _ \in p}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!] <: [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i}$$

Figure A.22: Static Semantics of Acyclic Core Fortress with Field Definitions (II)

Well-formed types: $\boxed{p; \Delta \vdash \tau \ \mathsf{ok}}$

[W-OBJ] $$p; \Delta \vdash \mathrm{Object}\ \mathsf{ok}$$

[W-VAR] $$\frac{\alpha \in \mathit{dom}(\Delta)}{p; \Delta \vdash \alpha\ \mathsf{ok}}$$

[W-TAPP] $$\frac{_\ C[\![\overrightarrow{\alpha \texttt{ extends } N}]\!]\ _ \in p \qquad p; \Delta \vdash \overrightarrow{\tau}\ \mathsf{ok} \qquad p; \Delta \vdash \overrightarrow{\tau}\ <:\ [\overrightarrow{\tau}/\overrightarrow{\alpha}]\overrightarrow{N}}{p; \Delta \vdash C[\![\overrightarrow{\tau}]\!]\ \mathsf{ok}}$$

Method overriding: $\boxed{p \vdash \mathit{override}(m, \{\overrightarrow{N}\}, [\![\overrightarrow{\alpha \texttt{ extends } N}]\!]\ \overrightarrow{\tau} \to \tau)}$

[OVERRIDE] $$\frac{\bigcup_{L_i \in \{\overrightarrow{L}\}} \mathit{mtype}_{p;\Delta}(m, L_i) = \{[\![\overrightarrow{\beta \texttt{ extends } M}]\!]\ \overrightarrow{\tau'} \to \tau_0'\} \quad \overrightarrow{N} = [\overrightarrow{\alpha}/\overrightarrow{\beta}]\overrightarrow{M} \quad \overrightarrow{\tau} = [\overrightarrow{\alpha}/\overrightarrow{\beta}]\overrightarrow{\tau'} \quad p; \Delta\ \overrightarrow{\alpha\ <:\ \{N\}} \vdash \tau_0\ <:\ [\overrightarrow{\alpha}/\overrightarrow{\beta}]\tau_0'}{p \vdash \mathit{override}(m, \{\overrightarrow{L}\}, [\![\overrightarrow{\alpha \texttt{ extends } N}]\!]\ \overrightarrow{\tau} \to \tau_0)}$$

Method type lookup: $\boxed{\mathit{mtype}_{p;\Delta}(m, \tau) = \{[\![\overrightarrow{\alpha \texttt{ extends } N}]\!]\ \overrightarrow{\tau} \to \tau\}}$

[MT-SELF] $$\frac{_\ C[\![\overrightarrow{\alpha \texttt{ extends } _}]\!]\ _\ \overrightarrow{md}\ _ \in p \qquad m[\![\overrightarrow{\beta \texttt{ extends } M}]\!](\overrightarrow{_ : \tau}) : \tau_0' = _ \in \{\overrightarrow{md}\}}{\mathit{mtype}_{p;\Delta}(m, C[\![\overrightarrow{\tau}]\!]) = \{[\overrightarrow{\tau}/\overrightarrow{\alpha}][\![\overrightarrow{\beta \texttt{ extends } M}]\!]\ \overrightarrow{\tau'} \to \tau_0'\}}$$

[MT-SUPER] $$\frac{_\ C[\![\overrightarrow{\alpha \texttt{ extends } _}]\!]\ _\ \texttt{extends}\{\overrightarrow{K}\}\ _\ \overrightarrow{md}\ _ \in p \qquad m \notin \{\overrightarrow{\mathit{Name}(md)}\}}{\mathit{mtype}_{p;\Delta}(m, C[\![\overrightarrow{\tau}]\!]) = \bigcup_{N_i \in \{\overrightarrow{N}\}} \mathit{mtype}_{p;\Delta}(m, [\overrightarrow{\tau}/\overrightarrow{\alpha}]N_i)}$$

[MT-OBJ] $$\mathit{mtype}_{p;\Delta}(m, \mathrm{Object}) = \emptyset$$

Auxiliary functions for methods: $\boxed{\mathit{defined}_p \,/\, \mathit{inherited}_p \,/\, \mathit{visible}_p(C) = \{\overrightarrow{m}\}}$

$$\mathit{defined}_p(C) = \{\overrightarrow{\mathit{Name}(md)}\} \quad \text{where } _\ C\ _\ \overrightarrow{md}\ _ \in p$$

$$\mathit{inherited}_p(C) = \biguplus_{N_i \in \{\overrightarrow{N}\}} \{m_i \mid m_i \in \mathit{visible}_p(N_i), m_i \notin \mathit{defined}_p(C)\} \quad \text{where } _\ C[\![\overrightarrow{\alpha \texttt{ extends } _}]\!]\ _\ \texttt{extends}\ \{\overrightarrow{N}\}\ _ \in p$$

$$\mathit{visible}_p(C) = \mathit{defined}_p(C) \uplus \mathit{inherited}_p(C)$$

Bound of type: $\boxed{\mathit{bound}_\Delta(\tau) = \tau}$

$$\begin{array}{lcl} \mathit{bound}_\Delta(\alpha) & = & \Delta(\alpha) \\ \mathit{bound}_\Delta(N) & = & N \\ \mathit{bound}_\Delta(O[\![\overrightarrow{\tau}]\!]) & = & O[\![\overrightarrow{\tau}]\!] \end{array}$$

Figure A.23: Static Semantics of Acyclic Core Fortress with Field Definitions (III)

Appendix B

Overloaded Functional Declarations

As mentioned in Chapter 24, this appendix proves that the overloading rules discussed in Chapter 24 guarantee no undefined nor ambiguous call at run time.

B.1 Proof of Coercion Resolution for Functions

This section proves that the overloading rules discussed in the previous sections guarantee the static resolution of coercion (described in Section 17.5) is well defined for functions (the case for methods is analogous).

Consider a static function call $f(A)$ at some program point Z and its corresponding dynamic function call $f(X)$. Let Σ be the set of parameter types of function declarations of f that are visible at Z and applicable to the static call $f(A)$. Let Σ' be the set of parameter types of function declarations of f that are visible at Z and applicable with coercion to the static call $f(A)$. Moreover, let σ' be the subset of Σ' for which no type in Σ' is more specific:

$$\sigma' \quad = \quad \{S \in \Sigma' \quad | \quad \neg\exists S' \in \Sigma' : S' \triangleleft S \quad \}.$$

We prove the following:

$$|\Sigma| = 0 \text{ and } |\Sigma'| \neq 0 \text{ imply } |\sigma'| = 1.$$

Informally, if no declaration is applicable to a static call but there is a declaration that is applicable with coercion then there exists a single most specific declaration that is applicable with coercion to the static call.

Lemma 1. *Given an acyclic, irreflexive binary relation R on a set S, and a finite nonempty subset A of S, the set $\{a \in A | \neg\exists a' \in A : (a', a) \in R\}$ is nonempty.*

Proof. Consider the relation R on S as a directed acyclic graph. Let A represent a subgraph. Then the Lemma amounts to proving that there exists a node in the graph represented by A with no edges pointing to it. This follows from the fact that A is finite and the graph is acyclic. □

Lemma 2. *If $|\Sigma'| \geq 1$ then $|\sigma'| \geq 1$.*

Proof. Follows from Lemma 1 where S is the set of all types, A is Σ', and the relation $\triangleleft$ is acyclic and irreflexive. □

Lemma 3. *If $|\Sigma| = 0$ then $|\sigma'| \leq 1$.*

Proof. For the purpose of contradiction suppose there are two declarations $f(P)$ and $f(Q)$ in σ'. Since both $f(P)$ and $f(Q)$ are applicable with coercion to $f(A)$ and $|\Sigma| = 0$ there must exist a coercion from some type P' to P and a coercion from some type Q' to Q such that $A \preceq P' \cap Q'$. Therefore it is not the case that $P \blacklozenge Q$. By the overloading rules, $P \neq Q$ and either $P \lhd Q$ or $Q \lhd P$ or for all $P' \in \mathcal{S}$ and $Q' \in \mathcal{T}$ either $P' \diamondsuit Q'$, or there is a declaration $f(P' \cap Q')$ visible at Z. If $P \lhd Q$ or $Q \lhd P$ then we contradict our assumption. Otherwise, if there exists a declaration $f(P' \cap Q')$ visible at Z then this declaration is applicable to $f(A)$ without coercion. This contradicts $|\Sigma| = 0$. If such a declaration does not exist then it must be the case that $P' \diamondsuit Q'$. Then both $f(P)$ and $f(Q)$ can not be applicable to the call $f(A)$ which is a contradiction. □

Theorem 1. *If* $|\Sigma| = 0$ *and* $|\Sigma'| \neq 0$ *then* $|\sigma'| = 1$.

Proof. Follows from Lemmas 2 and 3. □

B.2 Proof of Overloading Resolution for Functions

This section proves that the overloading rules for function declarations are sufficient to guarantee no undefined nor ambiguous call at run time (the case for methods is analogous).

The proof covers declarations whose parameter types and return types have no static parameters, and calls to which declarations are applicable without coercion. It assumes that the declarations visible at the call satisfy the overloading rules, so that no two of them have the same parameter type; that each of them has a body and that the bodies are well typed; that the subtype relation is sound, so that the dynamic type of an argument is a subtype of its static type, and transitive; and that every value of a trait with a `comprises` clause is a value of one of its listed types (Section 10.1). The compiled type checker does not yet enforce the last assumption across components: a type declared in a later component below a trait with static parameters that stands between a trait with a `comprises` clause and its listed types escapes its check (row 487 of the revival's gap ledger), and where the assumption fails the uniqueness proved below is not guaranteed. The proof does not extend to declarations with static parameters, whose instantiation keeps obligations of its own (Section 15.5, Chapter 20), nor to coercion (Section B.1), and it is not a proof of the soundness of the whole type system.

Consider a static function call $f(A)$ at some program point Z and its corresponding dynamic function call $f(X)$. Let Δ be the set of parameter types of function declarations of f that are visible at Z and applicable to the dynamic call $f(X)$. Let Σ be the set of parameter types of function declarations of f that are visible at Z and applicable to the static call $f(A)$. Moreover, let σ be the subset of Σ for which no type in Σ is more specific and let δ be the subset of Δ for which no type in Δ is more specific:

$$\begin{array}{rcl} \sigma & = & \{S \in \Sigma \quad | \quad \neg\exists S' \in \Sigma : S' \prec S \quad \} \\ \delta & = & \{D \in \Delta \quad | \quad \neg\exists D' \in \Delta : D' \prec D \quad \}. \end{array}$$

For a declaration whose parameter type is P, let R_P be its return type.

From the Meet Rule (Section 24.4) the proof uses the following. For any two declarations $f(P)$ and $f(Q)$ visible at Z such that neither P nor Q is a subtype of the other and P does not exclude Q, there are declarations $f(C_1), \ldots, f(C_n)$ visible at Z such that $C_i \preceq P$ and $C_i \preceq Q$ for each i, and every value of both P and Q is a value of some C_i: every value that a program can make, not only the values of a particular call. A declaration $f(P \cap Q)$ is the case of one such declaration. This holds of every such pair of the declarations, the pairs among the declarations $f(C_i)$ included.

Below we prove:

$|\Sigma| \neq 0$ implies $|\delta| = 1$,
$\delta = \{D\}$ implies $D \preceq S$ for every $S \in \Sigma$, and
$\delta = \{D\}$ implies $R_D \preceq R_S$ for every $S \in \sigma$.

Informally, if any declaration is applicable to a static call then there exists a single most specific declaration that is applicable to the corresponding dynamic call; it is the same as or more specific than every declaration that is applicable to the static call; and its return type is a subtype of the return type of every declaration applicable to the static call that no other such declaration is more specific than, so that the value the call returns belongs to the intersection of those return types. The static call may have more than one most specific declaration: covering gives a declaration that holds the value of the argument, not one whose parameter type is a supertype of the static type of the argument.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.35
>
> The Working Draft of February 2011 proved that $|\sigma| = 1$ and $|\delta| = 1$ whenever $|\Sigma| \neq 0$, and that $D \preceq S$ where $\sigma = \{S\}$ and $\delta = \{D\}$, from a Meet Rule that asks for a declaration on the intersection itself. With the case of the Meet Rule in which declarations together cover the intersection, the proof is revised: the lemmas of existence are kept, the uniqueness of the most specific declaration for the static call is given up, the uniqueness for the dynamic call is proved by a covering declaration, and two steps replace the last theorem, the refinement of every declaration applicable to the static call and the result type of the call. The scope and the assumptions stated above are new.

Lemma 4. $\Sigma \subseteq \Delta$.

Proof. Notice that $X \preceq A$ by type soundness. If $f(P)$ is applicable to the call $f(A)$ then $A \preceq P$. Notice that $X \preceq A$ implies $X \preceq P$. Therefore $f(P)$ is applicable to the call $f(X)$. □

Lemma 5. *If* $|\Delta| \geq 1$ *then* $|\delta| \geq 1$. *Also, if* $|\Sigma| \geq 1$ *then* $|\sigma| \geq 1$.

Proof. Follows from Lemma 1 where S is the set of all types, A is Δ and Σ respectively, and the relation $\prec$ is acyclic and irreflexive. □

Lemma 6. *If* $|\Sigma| \geq 1$ *then* $|\delta| \geq 1$.

Proof. Follows from Lemmas 4 and 5. □

Lemma 7. $|\delta| \leq 1$.

Proof. For the purpose of contradiction suppose there are two declarations $f(P)$ and $f(Q)$ in δ. Since both are in δ and $P \neq Q$, neither P nor Q is a subtype of the other. Since both $f(P)$ and $f(Q)$ are applicable without coercion to the call $f(X)$, the value of the argument is a value of both P and Q, so P does not exclude Q. By the Meet Rule as the proof uses it, there is a declaration $f(C)$ visible at Z with $C \preceq P$ and $C \preceq Q$ of which the value is a value, so that $X \preceq C$ and $C \in \Delta$. Since P is not a subtype of Q, $C \neq P$, so $C \prec P$, which contradicts $P \in \delta$. □

The same argument does not bound $|\sigma|$: the declaration $f(C)$ holds the value of the argument, but C need not be a supertype of A.

Theorem 2. *If* $|\Sigma| \neq 0$ *then* $|\delta| = 1$.

Proof. Follows from Lemmas 6 and 7. □

Theorem 3. *If* $\delta = \{D\}$ *then* $D \preceq S$ *for every* $S \in \Sigma$.

Proof. Let $S \in \Sigma$. By Lemma 4, $S \in \Delta$. If $S \prec D$ then $D \notin \delta$, a contradiction. Suppose that neither S nor D is a subtype of the other. The value of the argument is a value of both, so S does not exclude D, and by the Meet Rule as the proof uses it there is a declaration $f(C)$ visible at Z with $C \preceq S$, $C \preceq D$ and $X \preceq C$. Then $C \in \Delta$, and $C \neq D$ since D is not a subtype of S, so $C \prec D$, a contradiction. Therefore $D \preceq S$. □

Theorem 4. *If $\delta = \{D\}$ then $R_D \preceq R_S$ for every $S \in \sigma$, and a value that the dynamic call returns belongs to the intersection of the types R_S for $S \in \sigma$.*

Proof. Let $S \in \sigma$. By Theorem 3, $D \preceq S$. If $D = S$ the claim is immediate. Otherwise $D \prec S$, and the only overloading rule that accepts such a pair is the Subtype Rule (Section 24.2), which requires $R_D \preceq R_S$. The declaration with parameter type D is the one that runs, and its body is well typed, so a value it returns belongs to R_D, and so to R_S for every $S \in \sigma$. This proves neither that the call terminates nor that nothing else fails at run time. □

Appendix C

Components and APIs

> DRAFT NOTE
> This chapter might not be up to date.

As mentioned in Chapter 22, we formally specify key functionality of the Fortress component system, and illustrate how we can reason about the correctness of the system.

Components One important restriction on components is that no API may be both imported and exported by the same component. Formally, we introduce two functions on components, *imp* and *exp*, that return the imported and exported APIs of the component, respectively. For any component c, $\mathit{imp}(c) \cap \mathit{exp}(c) = \emptyset$. This restriction is required throughout to ground the semantics of operations on components, as discussed in Section 22.8.

APIs Other than its identity, the only relevant characteristic of an API a is the set of APIs that it uses, denoted by $\mathit{uses}(a)$. Because an API a might expose types defined in $\mathit{uses}(a)$, we require that a component that exports a also exports all APIs in $\mathit{uses}(a)$ that it does not import. Formally, the following condition holds on the exported APIs of a component c:

$$a \in \mathit{exp}(c) \wedge a' \in \mathit{uses}(a) \implies a' \in \mathit{imp}(c) \cup \mathit{exp}(c)$$

Link Given a set $C = \{c_1, \ldots, c_k\}$ of components, we define a partial function $\mathit{link}(C)$ that returns the component resulting from c_1 through c_k. If $c = \mathit{link}(C)$, then $\mathit{exp}(c) = \bigcup_{c' \in C} \mathit{exp}(c')$ and $\mathit{imp}(c) = \bigcup_{c' \in C} \mathit{imp}(c') - \mathit{exp}(c)$.

The function *link* is partial because we do not allow arbitrary sets of components to be linked. In particular, Two components cannot be linked if they export the same API.[1] This restriction is made for the sake of simplicity; it allows programmers to link a set of components without having to specify explicitly which constituent exporting an API A provides the implementation exported by the linked component, and which constituent connects to the constituents that import A: only one component exports A, so there is only one choice. Although we lose expressiveness with this design, the user interface to link is vastly simplified, and it is rare that including multiple components that export a given API in a set of linked components is even desirable. We discuss how even such rare cases can be supported in Section 22.9.

For a compound component, in addition to the exported and imported APIs, we want to know what its constituents are. So we introduce another function *cns*, which takes a component and returns the set of its constituents. That is, $\mathit{cns}(\mathit{link}(C)) = C$. It is an invariant of the system that for any compound component C (i.e., $\mathit{cns}(c) \neq \emptyset$),

[1] There is one exception to this rule: the special API $\mathrm{Upgradable}$, which is used during upgrades discussed below.

any API imported by any of its constituents is either imported by C or exported by one of its constituents (i.e., $\bigcup_{c' \in \mathit{cns}(c)} \mathit{imp}(c') \subseteq \mathit{imp}(c) \cup \bigcup_{c' \in \mathit{cns}(c)} \mathit{exp}(c')$). This property is crucial for executing components, as we discuss below. A simple component (i.e., one produced directly by compilation) has no constituents (i.e., $\mathit{cns}(c) = \emptyset$).

Upgrade A predicate *upg?* takes two components and indicates whether the first can be upgraded with the second; that is, $\mathit{upg?}(c_\mathrm{t}, c_\mathrm{r})$ returns true if and only if c_t can be upgraded with c_r. This predicate captures both the constraints imposed by a component's $isValidUpgrade$ function and the conditions that guarantee the well-formedness of the result. That is,

$$\begin{aligned}\mathit{upg?}(c_\mathrm{t}, c_\mathrm{r}) \implies\ & c_\mathrm{t}.isValidUpgrade(c_\mathrm{r}) \\ & \wedge\ \mathit{imp}(c_\mathrm{r}) \subseteq \mathit{exp}(c_\mathrm{t}) \cup \mathit{imp}(c_\mathrm{t}) \\ & \wedge\ \mathit{exp}(c_\mathrm{r}) \subset \mathit{exp}(c_\mathrm{t}) \\ & \wedge\ \forall c \in \mathit{cns}(c_\mathrm{t}).(\mathit{exp}(c) \subseteq \mathit{exp}(c_\mathrm{r}) \vee \mathit{exp}(c) \cap \mathit{exp}(c_\mathrm{r}) = \emptyset \vee \mathit{upg?}(c, c_\mathrm{r}))\end{aligned}$$

Visible and Provided We introduce two new functions on components: *vis*, which returns the APIs of a component that have not been hidden; and *prov*, which returns those visible APIs that are exported by some top-level constituent of the component (or all the exported APIs of a simple component); we say these APIs are *provided* by the component. We need to distinguish provided APIs because they can be imported by the top-level constituents of a component, and thus by a replacement component in an upgrade, while other visible APIs cannot be. Thus, for a compound component c, $\mathit{prov}(c) = \mathit{vis}(c) \cap \bigcup_{c' \in \mathit{cns}(c)} \mathit{exp}(c')$. For a simple component c, $\mathit{prov}(c) = \mathit{vis}(c) = \mathit{exp}(c)$.

Constrain If c is a compound component and $A \subset \mathit{exp}(c)$ is a set of APIs such that $a \in \mathit{exp}(c) \wedge a' \in \mathit{uses}(a) \cap A \implies a \in A$, we define $c' = \mathit{constrain}(c, A)$ such that $\mathit{exp}(c') = \mathit{exp}(c) - A$ and for any component c'', $\mathit{upg?}(c', c'') \iff \mathit{upg?}(c, c'') \wedge \mathit{exp}(c') \not\subseteq \mathit{exp}(c'')$. The *imp*, *vis*, *prov* and *cns* functions all have the same values for c and c'. The extra condition on the upgrade compatibility simply captures the restriction we mentioned above, that a replacement component should not export every API exported by the target.

Hide If c is a compound component and $A \subset \mathit{vis}(c)$ is a set of APIs such that $\mathit{exp}(c) \not\subseteq A$ and $a \in \mathit{vis}(c) \wedge a' \in \mathit{uses}(a) \cap A \implies a \in A$, we define $c' = \mathit{hide}(c, A)$ such that $\mathit{vis}(c') = \mathit{vis}(c) - A$, $\mathit{prov}(c') = \mathit{prov}(c) - A$, $\mathit{exp}(c') = \mathit{exp}(c) - A$, and for any component c'', $\mathit{upg?}(c', c'') \iff \mathit{upg?}(c, c'') \wedge \mathit{exp}(c') \not\subseteq \mathit{exp}(c'') \wedge \mathit{vis}(c'') \subseteq \mathit{vis}(c')$. The additional clause in $\mathit{upg?}(c', c'')$ (compared with that of *constrain*) reflects the hiding of the APIs: we can no longer upgrade APIs that are hidden.

Upgrade The interplay between imported, exported, visible and provided APIs introduces subtleties. In particular, the last of the three conditions imposed for well-formedness of upgrades is modified to state that for any constituent that is not subsumed by a replacement component, either it can be upgraded with the replacement, or its *visible* APIs are disjoint from the APIs exported by the replacement (i.e., it is unaffected by the upgrade). To maintain the invariant that no two constituents export the same API, we need another condition, which was implied by the previous condition when no APIs were constrained or hidden: if the replacement subsumes any constituents of the target, then its exported APIs must exactly match the exported APIs of some subset of the constituents of the target. That is, if $\mathit{upg?}(c_\mathrm{t}, c_\mathrm{r}) \wedge \exists c \in \mathit{cns}(c_\mathrm{t}).\ \mathit{exp}(c) \subseteq \mathit{exp}(c_\mathrm{r})$ then $\mathit{exp}(c_\mathrm{r}) = \bigcup_{c \in C} \mathit{exp}(c)$ for some $C \subset \mathit{cns}(c_\mathrm{t})$. In practice, this restriction is rarely a problem; in most cases, a user wishes to upgrade a target with a new version of a single constituent component, where the APIs exported by the old and new versions are either an exact match, or there are new APIs introduced by the new component that have no implementation in the target.

Appendix D

Rendering of Fortress Code

> DRAFT NOTE
> Identifiers including connecting punctuations are not yet supported.

In order to more closely approximate mathematical notation, Fortress mandates standard rendering for various input elements, particularly for numerals and identifiers, as specified in Section 4.17.

In this appendix, we describe the detailed rules for rendering an identifier, as well as rules used for rendering other constructions.

D.1 Rendering of Fortress Identifiers

If an identifier consists of letters and possibly digits, but no underscores, prime marks, or apostrophes, then the rules are fairly simple:

(a) If the identifier consists of two ASCII capital letters that are the same, possibly followed by digits, then a single capital letter is rendered double-struck, followed by full-sized (not subscripted) digits in roman font.

`QQ`	*is rendered as*	$\mathbb{Q}$	`RR64`	*is rendered as*	$\mathbb{R}64$
`ZZ`	*is rendered as*	$\mathbb{Z}$	`ZZ512`	*is rendered as*	$\mathbb{Z}512$

(b) Otherwise, if the identifier has more than two characters and begins with a capital letter, then it is rendered in roman font. (Such names are typically used as names of types in Fortress. Note that an identifier cannot consist entirely of capital letters, because such a token is considered to be an operator.)

`Integer`	*is rendered as*	Integer	`Matrix`	*is rendered as*	Matrix
`TotalOrder`	*is rendered as*	TotalOrder	`BooleanAlgebra`	*is rendered as*	BooleanAlgebra
`Fred17`	*is rendered as*	Fred17	`R2D2`	*is rendered as*	R2D2

(c) Otherwise, if the identifier consists of one or more letters followed by one or more digits, then the letters are rendered in italic and the digits are rendered as roman subscripts.

`a3`	*is rendered as*	a_3	`foo7`	*is rendered as*	foo_7
`M1`	*is rendered as*	M_1	`z128`	*is rendered as*	z_{128}
`OMEGA13`	*is rendered as*	Ω_{13}	`myFavoriteThings1625`	*is rendered as*	$myFavoriteThings_{1625}$

(d) The following names are always rendered in roman type out of respect for tradition:

sin	cos	tan	cot	sec	csc
sinh	cosh	tanh	coth	sech	csch
arcsin	arccos	arctan	arccot	arcsec	arccsc
arsinh	arcosh	artanh	arcoth	arsech	arcsch
arg	deg	det	exp	inf	sup
lg	ln	log	gcd	max	min

(e) Otherwise the identifier is simply rendered entirely in italic type.

`a`	*is rendered as*	a	`foobar`	*is rendered as*	$foobar$
`length`	*is rendered as*	$length$	`isInstanceOf`	*is rendered as*	$isInstanceOf$
`foo7a`	*is rendered as*	$foo7a$	`l33tsp33k`	*is rendered as*	$l33tsp33k$

If the identifier begins or ends with an underscore, or both, but has no other underscores, or prime marks, or apostrophes:

(f) If the identifier, ignoring its underscores, consists of two ASCII capital letters that are the same, possibly followed by one or more digits, then a single capital letter is rendered in sans-serif (for a leading underscore), script (for a trailing underscore), or italic san-serif (for both a leading and a trailing underscore), and any digits are rendered as roman subscripts.

(g) Otherwise, the identifier without its underscores is rendered in boldface (for a leading underscore), roman (for a trailing underscore), or bold italic (for both a leading and a trailing underscore); except that if the identifier, ignoring its underscores, consists of one or more letters followed by one or more digits, then the digits are rendered as roman subscripts regardless of the underscores.

`m_`	*is rendered as*	m	`s_`	*is rendered as*	s
`km_`	*is rendered as*	km	`kg_`	*is rendered as*	kg
`V_`	*is rendered as*	V	`kW_`	*is rendered as*	kW
`_v`	*is rendered as*	$\mathbf{v}$	`_foo13`	*is rendered as*	$\mathbf{foo}_{13}$

(Roman identifiers are typically used for names of SI dimensional units. See sections 6.1.1 and 6.2.1 of [28] for style questions with respect to dimensions and units.)

These last two rules are actually special cases of the following general rules that apply whenever an identifier contains at least one underscore, prime mark, or apostrophe:

An identifier containing underscores is divided into portions by its underscores; in addition, any apostrophe, prime, or double prime character separates portions and is also itself a portion.

(h) If any portion is empty other than the first or last, then the entire identifier is rendered in italics, underscores and all.

Otherwise, the portions are rendered as follows. The idea is that there is a *principal portion* that may be preceded and/or followed by modifiers, and there may also be a *face portion*:

- If the first portion is not empty, `script`, `fraktur`, `sansserif`, or `monospace`, then the principal portion is the first portion and there is no face portion.
- If the first portion is `script`, `fraktur`, `sansserif`, or `monospace`, then the principal portion is the second portion and the face portion is the first portion.
- If the first portion is empty and the second portion is not `script`, `fraktur`, `sansserif`, or `monospace`, then the principal portion is the second portion and there is no face portion.
- Otherwise the principal portion is the third portion and the face portion is the second portion.

If there is no face portion, the principal portion will be rendered in ordinary italics. However, if the first portion is empty (that is, the identifier begins with a leading underscore), then the principal portion is to be rendered in roman

boldface. If the last portion is empty (that is, the identifier ends with a trailing underscore), then the principal portion will be roman rather than italic, or bold italic rather than bold.

If there is a face portion, then that describes an alternate typeface to be used in rendering the principal portion. If there is no face portion, but the principal portion consists of two copies of the same letter, then it is rendered as a single letter in a double-struck face (also known as "blackboard bold"), sans-serif, script, or italic sans-serif font according to whether the first and last portions are (not empty, not empty), (empty, not empty), (not empty, empty), or (empty, empty), respectively. Otherwise, if the first portion is empty (that is, the identifier begins with a leading underscore), then the principal portion is to be rendered in a bold version of the selected face, and if the last portion is empty (that is, the identifier ends with a trailing underscore), then the principal portion to be rendered in an italic (or bold italic) version of the selected face. The bold and italic modifiers may be used only in combination with certain faces; the following are the allowed combinations:

```
script
bold script
fraktur
bold fraktur
double-struck
sans-serif
bold sans-serif
italic sans-serif
bold italic sans-serif
monospace
```

If a combination can be properly rendered, then the principal portion is rendered but not any preceding portions or underscores. If a combination cannot be properly rendered, then the principal portion and all portions and underscores preceding it are rendered all in italics if possible, and otherwise all in some other default face.

If the principal portion consists of a sequence of letters followed by a sequence of digits, then the letters are rendered in the chosen face and the digits are rendered as roman subscripts. Otherwise the entire principal portion is rendered in the chosen face. The remaining portions (excepting the last, if it is empty) are then processed according to the following rules:

- If a portion is `bar`, then a bar is rendered above what has already been rendered, excluding superscripts and subscripts. For example, `x_bar` is rendered as $\bar{x}$, `x17_bar` is rendered as $\bar{x}_{17}$, `x_bar_bar` is rendered as $\bar{\bar{x}}$, and `foo_bar` is rendered as $\overline{foo}$. (Contrast this last with `foo_baz`, which is rendered as foo_baz.)
- If a portion is `vec`, then a right-pointing arrow is rendered above what has already been rendered, excluding superscripts and subscripts. For example, `v_vec` is rendered as $\vec{v}$, `v17_vec` is rendered as $\vec{v}_{17}$, and `zoom_vec` is rendered as $\overrightarrow{zoom}$.
- If a portion is `hat`, then a hat is rendered above what has already been rendered, excluding superscripts and subscripts. For example, `x17_hat` is rendered as $\hat{x}_{17}$.
- If a portion is `dot`, then a dot is rendered above what has already been rendered, excluding superscripts and subscripts; but if the preceding portion was also `dot`, then the new dot is rendered appropriately relative to the previous dot(s). Up to four dots will be rendered side-by-side rather than vertically. For example, `a_dot` is rendered as $\dot{a}$, `a_dot_dot` is rendered as $\ddot{a}$, `a_dot_dot_dot` is rendered as $\dddot{a}$, `a_dot_dot_dot_dot` is rendered as $\ddddot{a}$. Also, `a_vec_dot` is rendered as $\dot{\vec{v}}$.
- If a portion is `star`, then an asterisk `*` is rendered as a superscript. For example, `a_star` is rendered as a^*, `a_star_star` is rendered asa^{**}, `ZZ_star` is rendered as $\mathbb{Z}^*$.
- If a portion is `splat`, then a number sign `#` is rendered as a superscript. For example, `QQ_splat` is rendered as $\mathbb{Q}^\#$.
- If a portion is `prime`, then a prime mark is rendered as a superscript.

- A prime character is treated the same as `prime`, and a double prime character is treated the same as two consecutive `prime` portions. An apostrophe is treated the same as a prime character, but only if all characters following it in the identifier, if any, are also apostrophes. For example, `a'` is rendered as a', `a13'` is rendered as a'_{13}, and `a''` is rendered as a'', but `don't` is rendered as *don't*.

- If a portion is `super` and another portion follows, then that other portion is rendered as a superscript in roman type, and enclosed in parentheses if it is all digits.

- If a portion is `sub` and another portion follows, then that other portion is rendered as a subscript in roman type, and enclosed in parentheses if it is all digits, and preceded by a subscript-separating comma if this portion was immediately preceded by another portion that was rendered as a subscript.

- If a portion consists entirely of capital letters and would, if considered by itself as an identifier, be the name of a non-letter Unicode character that would be subject to replacement by preprocessing, then that Unicode character is rendered as a subscript. For example, `id_OPLUS` is rendered as $id_{\oplus}$, `ZZ_GT` is rendered as $\mathbb{Z}_{>}$, and `QQ_star_LE` is rendered as $\mathbb{Q}^*_{\leq}$.

- If the portion is the last portion, and the principal portion was a single letter (or two letters indicating a double-struck letter), and none of the preceding rules in this list applies, it is rendered as a subscript in roman type. For example, `T_min` is rendered as T_{min}. Note that `T_MAX` is rendered simply as `T_MAX`—because all its letters are capital letters, it is considered to be an operator—but `T_sub_MAX` is rendered as T_{MAX}.

- Otherwise, this portion and all succeeding portions are rendered in italics, along with any underscores that appear adjacent to any of them.

Examples:

`M`	*is rendered as*	M	`_M`	*is rendered as*	$\mathbf{M}$
`v_vec`	*is rendered as*	$\vec{v}$	`_v_vec`	*is rendered as*	$\vec{\mathbf{v}}$
`v1`	*is rendered as*	v_1	`v_x`	*is rendered as*	v_{x}
`_v1`	*is rendered as*	$\mathbf{v}_1$	`_v_x`	*is rendered as*	$\mathbf{v}_{\mathrm{x}}$
`a_dot`	*is rendered as*	$\dot{a}$	`a_dot_dot`	*is rendered as*	$\ddot{a}$
`a_dot_dot_dot`	*is rendered as*	$\dddot{a}$	`a_dot_dot_dot_dot`	*is rendered as*	$\ddddot{a}$
`a_dot_dot_dot_dot_dot`	*is rendered as*	$\dddot{\dddot{a}}$	`p13'`	*is rendered as*	p'_{13}
`p'`	*is rendered as*	p'	`p_prime`	*is rendered as*	p'
`T_min`	*is rendered as*	T_{min}	`T_max`	*is rendered as*	T_{max}
`foo_bar`	*is rendered as*	$\overline{foo}$	`foo_baz`	*is rendered as*	foo_baz

In this way, through the use of underscore characters and annotation portions delimited by underscores, the programmer can exercise considerable typographical control over the rendering of variable names; but if no underscores are used, the rendering rules are quite simple.

D.2 Rendering of Other Fortress Constructs

A parameterized type of the form `T[\S\]` is rendered by replacing the brackets `[\` and `\]` with ⟦ (U+27E6, `MATHEMATICAL LEFT WHITE SQUARE BRACKET`) and ⟧ (U+27E7, `MATHEMATICAL RIGHT WHITE SQUARE BRACKET`), respectively.

An expression of the form `a^b` or `a^(b)` is rendered by making the expression b a superscript, and possibly removing an outer set of parentheses, if it is reasonably simple; otherwise it is rendered as if `^` were an ordinary binary operator.

`x^y`	*is rendered as*	x^y
`x^43`	*is rendered as*	x^{43}
`x^(n+1)`	*is rendered as*	x^{n+1}
`x^(|s.substring(a,b)|)`	*is rendered as*	$x\hat{}(\|s.substring(a,b)\|)$

An expression of the form `a[b]` is rendered by making the expression b a subscript, if it is reasonably simple.

`a[43]`	*is rendered as*	a_{43}
`a[k]`	*is rendered as*	a_k
`a[n_max]`	*is rendered as*	$a_{n_{\max}}$
`a[k+k']`	*is rendered as*	$a_{k+k'}$
`a[b-c]`	*is rendered as*	a_{b-c}
`a[3 k + 1]`	*is rendered as*	a_{3k+1}
`a[b[c[d]]]`	*is rendered as*	$a[b[c_d]]$
`a[s.substring(p,q)]`	*is rendered as*	$a[s.substring(p,q)]$

If there is more than one subscript expression, then they are rendered as subscripts only if each subscript is a single letter or digit, or a single letter followed by digits and/or prime marks:

`a[1,1]`	*is rendered as*	a_{11}
`a[1,n]`	*is rendered as*	a_{1n}
`a[j,m,n]`	*is rendered as*	a_{jmn}
`a[n1,n2,n3]`	*is rendered as*	$a_{n_1n_2n_3}$
`a[k,k',k"]`	*is rendered as*	$a_{kk'k''}$
`a[1,n-1]`	*is rendered as*	$a[1,n-1]$

(There is, of course, some opportunity here to make the rules for subscripted rendering more elaborate.)

If there is whitespace adjacent to the subscripting brackets or any separating comma, then actual subscripts are not used and the brackets remain (so putting spaces after the commas of a multidimensional subscript is a simple way to guarantee that bracket syntax will be used in the formatted version):

`a[i,j]`	*is rendered as*	a_{ij}
`a[i, j]`	*is rendered as*	$a[i,j]$
`a[b[c[ d ]]]`	*is rendered as*	$a[b[c[\,d\,]]]$

The presence or absence of whitespace inside and adjacent to enclosing operators is reflected in the rendered form by the presence or absence of a thin space:

`a[s.substring(p,q)]`	*is rendered as*	$a[s.substring(p,q)]$
`a[ s.substring(p,q) ]`	*is rendered as*	$a[\,s.substring(p,q)\,]$
`{1,2}`	*is rendered as*	$\{1,2\}$
`{ 1, 2 }`	*is rendered as*	$\{\,1,2\,\}$
`<|1,2,3|>`	*is rendered as*	$\langle 1,2,3\rangle$
`<| x+y | x<-a, y<-b |>`	*is rendered as*	$\langle\, x+y \mid x \leftarrow a, y \leftarrow b \,\rangle$

Rendered comprehensions look best if there is whitespace inside the enclosers and on both sides of the separating vertical bar.

The presence or absence of whitespace on either side of a colon is also handled especially carefully. In general, one should put a space on both sides, or neither side, of a colon that is used as an operator to construct a range; on the other hand, one should put a space after, but *not* before, a colon that separates a variable or function header from a type. This will produce the best rendered spacing. (The assignment operator := is recognized separately and whitespace doesn't matter.)

`for i <- 1:10 do print i end`	*is rendered as*	`for` $i \leftarrow 1:10$ `do` $print\, i$ `end`
`for i <- 0 : n+1 do print i end`	*is rendered as*	`for` $i \leftarrow 0 : n+1$ `do` $print\, i$ `end`
`k: ZZ32 := 5`	*is rendered as*	$k\colon \mathbb{Z}32 := 5$
`factorial(n: NN): NN`	*is rendered as*	$factorial(n\colon \mathbb{N})\colon \mathbb{N}$
`a:=b`	*is rendered as*	$a := b$

Generators and filters in brackets after a reduction operator are rendered by stacking them beneath the operator:

`PROD[k<-1#n] n`	*is rendered as*	$\prod_{k \leftarrow 1\#n} n$
`SUM[i<-1:n, j<-1:p, prime j] a[i] x^j`	*is rendered as*	$\sum_{\substack{i \leftarrow 1:n \\ j \leftarrow 1:p \\ prime\, j}} a_i\, x^j$
`MAX[j<-S, k<-j:j+m] a[k]`	*is rendered as*	$\underset{\substack{j \leftarrow S \\ k \leftarrow j:j+m}}{\mathtt{MAX}} a_k$

Appendix E

Support for Unicode Input in ASCII

> DRAFT NOTE
> Only Section E.2 is supported.

> DRAFT NOTE
> QUESTIONS/NOTES from Victor:
>
> - does micro convert to MICRO SIGN by itself, or only as part of a restricted word? (The latter is currently specified.)
> - Is ” ASCII shorthand for double prime (and the analogous qns for ”’ “ and “‘)? If so, then we need to fix the example in the character literal section, which says ”’ is a character literal that evaluates to the APOSTROPHE character.
> - The section on conversion of ASCII symbol sequences still needs to be fixed up. I think it should list all the ASCII shorthand (certainly it needs to list all the shorthand for non-operators), and then it needs to deal with not converting sequences that are subsequences of multicharacter enclosing operators.

To facilitate the writing of Fortress programs using ASCII-based tools, Fortress programs are subject to *ASCII conversion*, an idempotent process consisting of three steps described in this appendix. We expect that IDEs that support Unicode may apply ASCII conversion immediately, saving programs as converted Unicode character sequences.

E.1 Word Pasting across Line Terminators

Consider any line terminator in the program for which the last non-whitespace character before the line terminator is an ampersand (`&`) that is immediately preceded by a word character, and the first non-whitespace character after the line terminator is an ampersand that is immediately followed by a word character. All the characters between the ampersands inclusive are removed from the program. Note that the removed characters other than the two ampersands must be whitespace characters (including one or more line terminators). This step permits very long names and numerals to be split across line boundaries. For example:

```
supercalifragilisticexpiali&
   &docious = 0.142857142857142857&
   &142857 TIMES &
       GREEK_SMALL_LETTER_&
```

```
   &UPSILON_WITH_DIALYTICA_AND_TONOS
```

becomes

```
supercalifragilisticexpialidocious = 0.142857142857142857142857 TIMES &
       GREEK_SMALL_LETTER_UPSILON_WITH_DIALYTICA_AND_TONOS
```

To ensure the idempotence of ASCII conversion, it is a static error if an ampersand is immediately followed by another ampersand, or if it is immediately followed by one or more whitespace characters, not including a line terminator, followed by an ASCII letter or ampersand. In other words, if an ampersand is not the last non-whitespace character on a line then the next non-whitespace character must not be an ampersand, nor can it be an ASCII letter unless it is adjacent to the ampersand.

E.2 Converting Names of Unicode Characters and ASCII Shorthand

After word pasting across line terminators, certain ASCII character sequences are converted into single Unicode characters. These character sequences fall into two categories: restricted words (or parts of restricted words) and ASCII shorthand for operators and special characters. In some cases, ampersands adjacent to converted sequences are removed.

E.2.1 Determining Potentially Convertible Character Sequences

We first identify nonoverlapping contiguous subsequences of the program that may be subject to conversion in this step. In particular, names of characters and ASCII shorthand consist of only ASCII characters, and they are *not* converted within string literals.

The boundaries of string literals (and comments) are determined follows: There are three modes of processing: outside any comment or string literal, inside a string literal and inside a comment. Within a comment, we also keep track of "nesting depth" (this is 0 when not within a comment). Processing proceeds from left to right. Outside any comment or string literal, encountering an unescaped string literal delimiter (i.e., a string literal delimiter not immediately preceded by an unescaped backslash) changes processing to the mode for within a string literal (however, it is a static error if the string literal delimiter is the right double quotation mark), and encountering the opening comment delimiter "`(*`" changes processing to the mode for within a comment, incrementing the nesting depth (to 1). All other characters are ignored, except to note they are not within a string literal. In particular, because character literal delimiters are ignored, string literal delimiters must be escaped within character literals.

Within a string literal, all characters other than an unescaped string literal delimiter are ignored (except to note that they are within a string literal). An unescaped string literal delimiter switches processing to the mode for outside any comment or string literal. However, it is a static error if a line terminator appears within a string literal (Fortress provides *escape sequences* to include line terminators in strings; see Section 4.10), or if the opening and closing string literal delimiters do not "match" (again see Section 4.10).

Within a comment, all characters, including unescaped string literal delimiters, are ignored (except to note that they are not within a string literal) other than the two-character opening and closing comment delimiters "`(*`" and "`*)`". Whenever the opening comment delimiter is encountered, the nesting depth is incremented, and each time the closing comment delimiter is encountered, the nesting depth is decremented, until it becomes 0. At that point, processing is changed again to the mode for outside any comment or string literal.

A contiguous subsequence of ASCII characters not within any string literal is a candidate for conversion in this step if it is either a restricted word, or it is a maximal such subsequence that does not contain any restricted-word characters,

control characters, whitespace characters or ampersands (i.e., it must be delimited by whitespace, a control or non-ASCII characters, string literals, restricted words, ampersands, or the beginning or end of the file). We call the latter kind of subsequence an *ASCII symbol sequence*. The conversion of restricted words and ASCII symbol sequences are described in Section E.2.2 and Section E.2.3 respectively. Whitespace characters and ampersands are not converted, but some ampersands may be eliminated, as described in Section E.2.4.

E.2.2 Converting Restricted Words

A restricted word that "names" a Unicode character that is not protected[1] (i.e., not an ASCII or control character nor a string literal delimiter) is replaced by that character. A restricted word may name a character in various ways, described below. In a few cases, a restricted word may name more than one character (via different ways), so the order in which we consider these ways is important. Also, in some cases, only part of the restricted word is replaced by a Unicode character.

Fortress explicitly provides short ASCII names for many characters, especially ones that programmers might most commonly want. Some characters have more than one short name. For operators, these names are given in Appendix F, including the following common ones:

LE	*becomes*	$\leq$	GE	*becomes*	$\geq$	NE	*becomes*	$\neq$
BY	*becomes*	$\times$	TIMES	*becomes*	$\times$	CROSS	*becomes*	$\times$
DOT	*becomes*	$\cdot$	PROD	*becomes*	$\prod$	SUM	*becomes*	$\sum$
CUP	*becomes*	$\cup$	CAP	*becomes*	$\cap$	SUBSET	*becomes*	$\subset$
EMPTYSET	*becomes*	$\emptyset$	AND	*becomes*	$\wedge$	OR	*becomes*	$\vee$

To provide a level of compatibility with Fortran, Appendix F also gives short names for the following ASCII (and thus protected) characters:

LT	*becomes*	$<$	GT	*becomes*	$>$	EQ	*becomes*	$=$

These names are the only ones converted to a protected character. Furthermore, they are *not* replaced by the corresponding character unless they are delimited by whitespace characters (*not* ampersands). Thus, they cannot participate in further conversion, maintaining the idempotence of ASCII conversion.

The following non-operator characters also have short names:

[1] Protecting the backslash (an ASCII character) and string literal delimiters preserves the boundaries of string literals determined as described above; protecting the other ASCII characters ensures that the ASCII conversion process is idempotent.

ALPHA	*becomes*	A	alpha	*becomes*	α	BOTTOM	*becomes*	$\bot$
BETA	*becomes*	B	beta	*becomes*	β	TOP	*becomes*	$\top$
GAMMA	*becomes*	Γ	gamma	*becomes*	γ	INF	*becomes*	∞
DELTA	*becomes*	Δ	delta	*becomes*	δ	FORALL	*becomes*	$\forall$
EPSILON	*becomes*	E	epsilon	*becomes*	ϵ	EXISTS	*becomes*	$\exists$
ZETA	*becomes*	Z	zeta	*becomes*	ζ			
ETA	*becomes*	H	eta	*becomes*	η			
THETA	*becomes*	Θ	theta	*becomes*	θ			
IOTA	*becomes*	I	iota	*becomes*	ι			
KAPPA	*becomes*	K	kappa	*becomes*	κ			
LAMBDA	*becomes*	Λ	lambda	*becomes*	λ			
MU	*becomes*	M	mu	*becomes*	μ			
NU	*becomes*	N	nu	*becomes*	ν			
XI	*becomes*	Ξ	xi	*becomes*	ξ			
OMICRON	*becomes*	O	omicron	*becomes*	o			
PI	*becomes*	Π	pi	*becomes*	π			
RHO	*becomes*	P	rho	*becomes*	ρ			
SIGMA	*becomes*	Σ	sigma	*becomes*	σ			
TAU	*becomes*	T	tau	*becomes*	τ			
UPSILON	*becomes*	Υ	upsilon	*becomes*	υ			
PHI	*becomes*	Φ	phi	*becomes*	ϕ			
CHI	*becomes*	X	chi	*becomes*	χ			
PSI	*becomes*	Ψ	psi	*becomes*	ψ			
OMEGA	*becomes*	Ω	omega	*becomes*	ω			

A restricted word that is not a short name specified by Fortress is compared to names of characters specified by the Unicode Standard [29]. Because restricted words consist only of letters, digits and underscores, while Unicode names may include hyphens and spaces (but not underscores), hyphens and spaces in the Unicode names are replaced by underscores for this comparison. Specifically, if a restricted word is the official Unicode 5.0 name of a character with hyphens and spaces replaced by underscores, it is replaced by that character. For any Unicode character other than the control characters, there is a unique official Unicode 5.0 name not shared by any other Unicode character. Since control characters are protected, they do not present a problem in this regard.

If there is no matching official Unicode 5.0 name, the restricted word is compared to the alternative names for characters specified by the Unicode Standard, again with hyphens and spaces replaced by underscores, and converted to a character with such a matching name. If more than one character has a matching alternative name, then the restricted word is converted to the first such non-protected character (i.e., the one with the smallest code point). If there is no matching alternative name, then the restricted word is compared to official Unicode 5.0 names and any alternative names specified in the Unicode Standard, with underscores in place of hyphens and spaces, where any of the following substrings may be omitted:

```
"LETTER_"
"DIGIT_"
"RADICAL_"
"NUMERAL_"
```

If there are multiple such substrings in a given name, any combination of them may be omitted. Again, if there are multiple characters with matching abridged names, the restricted word is converted to the first such non-protected character.

If the restricted word is not converted to a single Unicode character by any of the above rules, parts of it may be converted as follows: If the restricted word begins with the short name (i.e., the name in the table above) of a Greek letter followed by an underscore or a digit, or ends with the short name of a Greek letter that is preceded by an underscore, or contains the short name of a Greek letter with an underscore on each side of it, then the short name of

the Greek letter is replaced by the Greek letter itself. In the same manner, the character sequence "micro" is replaced with the character `MICRO SIGN` μ (U+00B5, which looks just like the Greek lowercase mu μ but is different). If a word-part being thus replaced has an underscore to each side, and the underscore on the right is the last character of the restricted word, then the underscore on the left is removed as the name is replaced; this ad hoc rule is for the sake of the abbreviations of certain dimensional units, so that, for example, `micro_OMEGA_` is converted to $\mu\Omega_$, signifying micro-ohms, and `G_OMEGA_` is converted to $\mathrm{G}\Omega_$, signifying gigaohms. Here are some other examples:

`alpha`	*becomes*	α	`OMEGA13`	*becomes*	$\Omega 13$
`alpha_hat`	*becomes*	α_hat	`theta_elephant`	*becomes*	$\theta_elephant$
`OMEGA_`	*becomes*	$\Omega_$	`_XI`	*becomes*	$_\Xi$

E.2.3 Converting ASCII Symbol Sequences

For the sequences of characters other than restricted words, each is converted from left to right, with the longest possible substring being converted at once, with one exception: The sequence "`(<`" is not converted if it is immediately followed by any of the following characters: '`<`', '`|`', '`/`', '`\`', '`*`', or '`.`'. That is, with this one exception, the longest shorthand beginning from the first character, if any, is converted first. Then, the longest shorthand beginning from the next character of the string after replacement is converted, and so on. Here are the ASCII shorthands for some of the characters we expect to be most frequently used:

`[\`	*becomes*	$[\![$	`\]`	*becomes*	$]\!]$
`->`	*becomes*	$\rightarrow$	`=>`	*becomes*	$\Rightarrow$
`~>`	*becomes*	$\rightsquigarrow$	`|->`	*becomes*	$\mapsto$
`>=`	*becomes*	$\geq$	`<=`	*becomes*	$\leq$
`=/=`	*becomes*	$\neq$	`...`	*becomes*	$\ldots$
`:=`	*becomes*	$:=$			

Although the characters with string literals are generally not subject to this step of ASCII conversion, They are if they are part of a restricted-word escape sequence or a quoted-character escape sequence. See Section 4.10 for details.

E.2.4 Eliminating Ampersands

Finally, if an ampersand is adjacent to a sequence of characters that is changed by this step of ASCII conversion (even if the sequence was only partly changed, as long as the character adjacent to the ampersand is changed), the ampersand is removed after the transformation, *unless* the ampersand is the first or last non-whitespace character on the line. Thus, the expression

Why leave the first?

```
(GREEK_SMALL_LETTER_PHI&GREEK_SMALL_LETTER_PSI +
GREEK_SMALL_LETTER_OMEGA&GREEK_SMALL_LETTER_LAMBDA)
```

is converted to:

$(\phi\psi\ +\ \omega\lambda)$

in which there are two identifiers, each consisting of two Greek letters. On the other hand,

```
(GREEK_SMALL_LETTER_PHI GREEK_SMALL_LETTER_PSI +
GREEK_SMALL_LETTER_OMEGA GREEK_SMALL_LETTER_LAMBDA)
```

is converted to:

$(\phi\ \psi\ +\ \omega\ \lambda)$

in which there are four identifiers.

Because we never convert to protected characters, and converted sequences consist only of protected characters, this step is idempotent: repeated application has no effect beyond the first.

> DRAFT NOTE
>
> Commented out text: However, to ensure that the entire ASCII conversion process is idempotent, it is a static error if an ampersand is immediately followed by a word character after this step. This restriction prevents any word pasting in the converted program.

E.3 Apostrophe Replacement within Numerals

The last step of ASCII conversion simply replaces apostrophes with digit-group separators when they appear in words that would otherwise be numerals. Specifically, consider any word that is not within a string literal (determined as described in Section E.2.1), has only digits, letters, underscores and apostrophes, and neither begins nor ends with an apostrophe or an underscore. Call such a word a *proto-numeral*. If a proto-numeral begins with a digit or ends with a radix specifier (see Section 4.13), or if it is immediately followed by a '.' and then a proto-numeral that ends with a radix specifier, or it is immediately preceded by a '.', which is immediately preceded by a proto-numeral that begins with a digit, then replace each apostrophe in that proto-numeral with a digit-group separator.

E.4 Equivalence under ASCII Conversion

Two (valid) Fortress programs are equivalent under ASCII conversion if they are identical after ASCII conversion except for string literals (determined as described in Section E.2.1), the converted programs have string literals in exactly corresponding places, and corresponding string literals evaluate to the same string. This exception and additional rule are necessary because string literals are not subject to ASCII conversion.

Appendix F

Operator Precedence, Associativity, Chaining, and Enclosure

This appendix contains the detailed rules for which Unicode 5.0 characters may be used as operators, which operators form enclosing pairs, which operators may be chained, which operators are associative, and what precedence relationships exist among the various operators. An infix operator is left associative, nonassociative, or chainable. If no precedence relationship is stated explicitly for any given pair of operators, then there is no precedence relationship between those two operators. Remember that precedence is not transitive in Fortress.

In each of the character lists below, each line gives the Unicode codepoint, the full Unicode 5.0 name, a rendering of the character in TEX (if possible), and any ASCII shorthand or short names for the character.

F.1 Bracket Pairs for Static Type Information

These brackets are the exception: they are *not* operators, but are used to enclose static parameters and static arguments.

U+27E6	MATHEMATICAL LEFT WHITE SQUARE BRACKET	⟦	[\
U+27E7	MATHEMATICAL RIGHT WHITE SQUARE BRACKET	⟧	\]

F.2 Bracket Pairs for Enclosing Operators

Here are the bracket pairs that may be used as enclosing operators. Note that there is one group of four brackets; within that group, either left bracket may be paired with either right bracket to form an enclosing operator.

U+005B	LEFT SQUARE BRACKET	[	[
U+005D	RIGHT SQUARE BRACKET	]	]
U+007B	LEFT CURLY BRACKET	{	{
U+007D	RIGHT CURLY BRACKET	}	}
U+2045	LEFT SQUARE BRACKET WITH QUILL		[./
U+2046	RIGHT SQUARE BRACKET WITH QUILL		/.]
U+2308	LEFT CEILING	⌈	| /
U+2309	RIGHT CEILING	⌉	| \

U+230A	LEFT FLOOR	⌊	| \
U+230B	RIGHT FLOOR	⌋	/ |
U+27C5	LEFT S-SHAPED BAG DELIMITER		| . \
U+27C6	RIGHT S-SHAPED BAG DELIMITER		/ . |
U+27E8	MATHEMATICAL LEFT ANGLE BRACKET	⟨	< |
U+27E9	MATHEMATICAL RIGHT ANGLE BRACKET	⟩	| >
U+27EA	MATHEMATICAL LEFT DOUBLE ANGLE BRACKET	⟪	<< |
U+27EB	MATHEMATICAL RIGHT DOUBLE ANGLE BRACKET	⟫	| >>
U+2983	LEFT WHITE CURLY BRACKET	⦃	{ \
U+2984	RIGHT WHITE CURLY BRACKET	⦄	\ }
U+2985	LEFT WHITE PARENTHESIS		(. \
U+2986	RIGHT WHITE PARENTHESIS		\ .)
U+2987	Z NOTATION LEFT IMAGE BRACKET		(. /
U+2988	Z NOTATION RIGHT IMAGE BRACKET		/ .)
U+2989	Z NOTATION LEFT BINDING BRACKET		<| |
U+298A	Z NOTATION RIGHT BINDING BRACKET		| | >
U+298B	LEFT SQUARE BRACKET WITH UNDERBAR		[. \
U+298C	RIGHT SQUARE BRACKET WITH UNDERBAR		\ .]
U+298D	LEFT SQUARE BRACKET WITH TICK IN TOP CORNER		[. / /
U+298E	RIGHT SQUARE BRACKET WITH TICK IN BOTTOM CORNER		/ / .]
U+298F	LEFT SQUARE BRACKET WITH TICK IN BOTTOM CORNER		[. \ \
U+2990	RIGHT SQUARE BRACKET WITH TICK IN TOP CORNER		\ \ .]
U+2991	LEFT ANGLE BRACKET WITH DOT		< . |
U+2992	RIGHT ANGLE BRACKET WITH DOT		| . >
U+2993	LEFT ARC LESS-THAN BRACKET		(. <
U+2994	RIGHT ARC GREATER-THAN BRACKET		> .)
U+2995	DOUBLE LEFT ARC GREATER-THAN BRACKET		((. >
U+2996	DOUBLE RIGHT ARC LESS-THAN BRACKET		< .))
U+2997	LEFT BLACK TORTOISE SHELL BRACKET		[* /
U+2998	RIGHT BLACK TORTOISE SHELL BRACKET		/ *]
U+29D8	LEFT WIGGLY FENCE		[/ \ /
U+29D9	RIGHT WIGGLY FENCE		/ \ /]
U+29DA	LEFT DOUBLE WIGGLY FENCE		[/ \ / \ /
U+29DB	RIGHT DOUBLE WIGGLY FENCE		/ \ / \ /]
U+29FC	LEFT-POINTING CURVED ANGLE BRACKET		<| | |
U+29FD	RIGHT-POINTING CURVED ANGLE BRACKET		| | | >
U+300C	LEFT CORNER BRACKET	⌜	< /
U+300D	RIGHT CORNER BRACKET	⌝	\ >
U+300E	LEFT WHITE CORNER BRACKET		<< /
U+300F	RIGHT WHITE CORNER BRACKET		\ >>
U+3010	LEFT BLACK LENTICULAR BRACKET		{ * /
U+3011	RIGHT BLACK LENTICULAR BRACKET		/ * }
U+3018	LEFT WHITE TORTOISE SHELL BRACKET		[/ /
U+3014	LEFT TORTOISE SHELL BRACKET		[/
U+3015	RIGHT TORTOISE SHELL BRACKET		/]

U+3019 RIGHT WHITE TORTOISE SHELL BRACKET		//]
U+3016 LEFT WHITE LENTICULAR BRACKET		{/
U+3017 RIGHT WHITE LENTICULAR BRACKET		/}

F.3 Vertical-Line Operators

The following are vertical-line operators. They are left associative when they are used as infix operators.

U+007C VERTICAL LINE	|	|
U+2016 DOUBLE VERTICAL LINE	‖	||
U+2AF4 TRIPLE VERTICAL BAR BINARY RELATION		|||

F.4 Arithmetic Operators

F.4.1 Multiplication and Division

The following are multiplication operators. They are left associative.

U+002A ASTERISK	*	*
U+00B7 MIDDLE DOT	·	DOT
U+00D7 MULTIPLICATION SIGN	×	TIMES BY
U+2217 ASTERISK OPERATOR	∗	*
U+228D MULTISET MULTIPLICATION		
U+2297 CIRCLED TIMES	⊗	OTIMES
U+2299 CIRCLED DOT OPERATOR	⊙	ODOT
U+229B CIRCLED ASTERISK OPERATOR	⊛	CIRCLEDAST
U+22A0 SQUARED TIMES	⊠	BOXTIMES
U+22A1 SQUARED DOT OPERATOR	⊡	BOXDOT
U+22C5 DOT OPERATOR	⋅	
U+29C6 SQUARED ASTERISK		BOXAST
U+29D4 TIMES WITH LEFT HALF BLACK		
U+29D5 TIMES WITH RIGHT HALF BLACK		
U+2A2F VECTOR OR CROSS PRODUCT	×	CROSS
U+2A30 MULTIPLICATION SIGN WITH DOT ABOVE		DOTTIMES
U+2A31 MULTIPLICATION SIGN WITH UNDERBAR		
U+2A34 MULTIPLICATION SIGN IN LEFT HALF CIRCLE		
U+2A35 MULTIPLICATION SIGN IN RIGHT HALF CIRCLE		
U+2A36 CIRCLED MULTIPLICATION SIGN WITH CIRCUMFLEX ACCENT		
U+2A37 MULTIPLICATION SIGN IN DOUBLE CIRCLE		
U+2A3B MULTIPLICATION SIGN IN TRIANGLE		TRITIMES

The following are division operators. They are nonassociative.

U+002F SOLIDUS	/	/
U+00F7 DIVISION SIGN	÷	DIV
U+2215 DIVISION SLASH	∕	/
U+2298 CIRCLED DIVISION SLASH	⊘	OSLASH
U+29B8 CIRCLED REVERSE SOLIDUS		

U+29BC CIRCLED ANTICLOCKWISE-ROTATED DIVISION SIGN
U+29C4 SQUARED RISING DIAGONAL SLASH BOXSLASH
U+29F5 REVERSE SOLIDUS OPERATOR \ \
U+29F8 BIG SOLIDUS ⧸
U+29F9 BIG REVERSE SOLIDUS ⧹
U+2A38 CIRCLED DIVISION SIGN ODIV
U+2AFD DOUBLE SOLIDUS OPERATOR // //
U+2AFB TRIPLE SOLIDUS BINARY RELATION ///

Note also that `per` is treated as a division operator.

F.4.2 Addition and Subtraction

Addition and subtraction operators are left associative.

The following three operators have the same precedence and may be mixed.

U+002B PLUS SIGN + +
U+002D HYPHEN-MINUS − -
U+2212 MINUS SIGN − -

They each have lower precedence than any of the following multiplication and division operators:

U+002A ASTERISK * *
U+002F SOLIDUS / /
U+00B7 MIDDLE DOT · DOT
U+00D7 MULTIPLICATION SIGN × TIMES BY
U+00F7 DIVISION SIGN ÷ DIV
U+2215 DIVISION SLASH ∕ /
U+2217 ASTERISK OPERATOR ∗ *
U+22C5 DOT OPERATOR ⋅
U+2A2F VECTOR OR CROSS PRODUCT × CROSS

The following two operators have the same precedence and may be mixed.

U+2214 DOT PLUS ∔ DOTPLUS
U+2238 DOT MINUS ∸ DOTMINUS

They each have lower precedence than this multiplication operator:

U+2A30 MULTIPLICATION SIGN WITH DOT ABOVE DOTTIMES

The following two operators have the same precedence and may be mixed.

U+2A25 PLUS SIGN WITH DOT BELOW
U+2A2A MINUS SIGN WITH DOT BELOW

The following two operators have the same precedence and may be mixed.

U+2A39 PLUS SIGN IN TRIANGLE TRIPLUS
U+2A3A MINUS SIGN IN TRIANGLE TRIMINUS

They each have lower precedence than this multiplication operator:

U+2A3B MULTIPLICATION SIGN IN TRIANGLE TRITIMES

The following two operators have the same precedence and may be mixed.

U+2295	CIRCLED PLUS	⊕	OPLUS
U+2296	CIRCLED MINUS	⊖	OMINUS

They each have lower precedence than any of the following multiplication and division operators:

U+2297	CIRCLED TIMES	⊗	OTIMES
U+2298	CIRCLED DIVISION SLASH	⊘	OSLASH
U+2299	CIRCLED DOT OPERATOR	⊙	ODOT
U+229B	CIRCLED ASTERISK OPERATOR	⊛	CIRCLEDAST
U+2A38	CIRCLED DIVISION SIGN		ODIV

The following two operators have the same precedence and may be mixed.

U+229E	SQUARED PLUS	⊞	BOXPLUS
U+229F	SQUARED MINUS	⊟	BOXMINUS

They each have lower precedence than any of these multiplication or division operators:

U+22A0	SQUARED TIMES	⊠	BOXTIMES
U+22A1	SQUARED DOT OPERATOR	⊡	BOXDOT
U+29C4	SQUARED RISING DIAGONAL SLASH		BOXSLASH
U+29C6	SQUARED ASTERISK		BOXAST

These are other miscellaneous addition and subtraction operators:

U+00B1	PLUS-MINUS SIGN	±
U+2213	MINUS-OR-PLUS SIGN	∓
U+2242	MINUS TILDE	
U+2A22	PLUS SIGN WITH SMALL CIRCLE ABOVE	$\overset{\circ}{+}$
U+2A23	PLUS SIGN WITH CIRCUMFLEX ACCENT ABOVE	$\overset{\wedge}{+}$
U+2A24	PLUS SIGN WITH TILDE ABOVE	$\overset{\sim}{+}$
U+2A26	PLUS SIGN WITH TILDE BELOW	$+\!\!\underset{\sim}{}$
U+2A27	PLUS SIGN WITH SUBSCRIPT TWO	$+_2$
U+2A28	PLUS SIGN WITH BLACK TRIANGLE	
U+2A29	MINUS SIGN WITH COMMA ABOVE	$\overset{,}{-}$
U+2A2B	MINUS SIGN WITH FALLING DOTS	
U+2A2C	MINUS SIGN WITH RISING DOTS	
U+2A2D	PLUS SIGN IN LEFT HALF CIRCLE	
U+2A2E	PLUS SIGN IN RIGHT HALF CIRCLE	

F.4.3 Miscellaneous Arithmetic Operators

The operators MAX, MIN, REM, MOD, GCD, LCM, CHOOSE, and per, none of which corresponds to a single Unicode character, are considered to be arithmetic operators, having higher precedence than certain relational operators, as described in a later section. Among these operators, MAX, MIN, GCD, and LCM are left associative and REM, MOD, CHOOSE, and per are nonassociative.

F.4.4 Set Intersection, Union, and Difference

The following are the set intersection operators. They are left associative.

U+2229 INTERSECTION ∩ CAP INTERSECT
U+22D2 DOUBLE INTERSECTION ⋒ CAPCAP
U+2A40 INTERSECTION WITH DOT
U+2A43 INTERSECTION WITH OVERBAR ⩃
U+2A44 INTERSECTION WITH LOGICAL AND
U+2A4B INTERSECTION BESIDE AND JOINED WITH INTERSECTION
U+2A4D CLOSED INTERSECTION WITH SERIFS
U+2ADB TRANSVERSAL INTERSECTION

The following are the set union operators. They are left associative.

U+222A UNION ∪ CUP UNION
U+228E MULTISET UNION ⊎ UPLUS
U+22D3 DOUBLE UNION ⋓ CUPCUP
U+2A41 UNION WITH MINUS SIGN
U+2A42 UNION WITH OVERBAR ⩂
U+2A45 UNION WITH LOGICAL OR
U+2A4A UNION BESIDE AND JOINED WITH UNION
U+2A4C CLOSED UNION WITH SERIFS
U+2A50 CLOSED UNION WITH SERIFS AND SMASH PRODUCT

They each have lower precedence than any of the set intersection operators.

This is a miscellaneous set operator. It is nonassociative.

U+2216 SET MINUS \ SETMINUS

F.4.5 Square Arithmetic Operators

Square arithmetic operators are left associative.

The following are the square intersection operators.

U+2293 SQUARE CAP ⊓ SQCAP
U+2A4E DOUBLE SQUARE INTERSECTION SQCAPCAP

The following are the square union operators:

U+2294 SQUARE CUP ⊔ SQCUP
U+2A4F DOUBLE SQUARE UNION SQCUPCUP

They each have lower precedence than either of the square intersection operators.

F.4.6 Curly Arithmetic Operators

Curly arithmetic operators are left associative.

The following is the curly intersection operator:

U+22CF CURLY LOGICAL AND ⋏ CURLYAND

The following is the curly union operator:

U+22CE CURLY LOGICAL OR ⋎ CURLYOR

It has lower precedence than the curly intersection operator.

F.5 Relational Operators

F.5.1 Equivalence and Inequivalence Operators

Every operator listed in this section has lower precedence than any operator listed in Section F.4.

The following are equivalence operators. They may be chained. Moreover, they may be chained with any other single group of chainable relational operators, as described in later sections.

U+003D EQUALS SIGN	=	EQ
U+2243 ASYMPTOTICALLY EQUAL TO	≃	SIMEQ
U+2245 APPROXIMATELY EQUAL TO	≅	
U+2246 APPROXIMATELY BUT NOT ACTUALLY EQUAL TO		
U+2248 ALMOST EQUAL TO	≈	APPROX
U+224A ALMOST EQUAL OR EQUAL TO	≋	APPROXEQ
U+224C ALL EQUAL TO		
U+224D EQUIVALENT TO	≍	
U+224E GEOMETRICALLY EQUIVALENT TO	≎	BUMPEQV
U+2251 GEOMETRICALLY EQUAL TO	≑	DOTEQDOT
U+2252 APPROXIMATELY EQUAL TO OR THE IMAGE OF	≒	
U+2253 IMAGE OF OR APPROXIMATELY EQUAL TO	≓	
U+2256 RING IN EQUAL TO	≖	EQRING
U+2257 RING EQUAL TO	≗	RINGEQ
U+225B STAR EQUALS		
U+225C DELTA EQUAL TO	≜	EQDEL
U+225D EQUAL TO BY DEFINITION		EQDEF
U+225F QUESTIONED EQUAL TO		
U+2261 IDENTICAL TO	≡	EQV EQUIV
U+2263 STRICTLY EQUIVALENT TO		=== SEQV
U+229C CIRCLED EQUALS		
U+22CD REVERSED TILDE EQUALS	⋍	
U+22D5 EQUAL AND PARALLEL TO		
U+29E3 EQUALS SIGN AND SLANTED PARALLEL		
U+29E4 EQUALS SIGN AND SLANTED PARALLEL WITH TILDE ABOVE		
U+29E5 IDENTICAL TO AND SLANTED PARALLEL		
U+2A66 EQUALS SIGN WITH DOT BELOW		
U+2A67 IDENTICAL WITH DOT ABOVE		
U+2A6C SIMILAR MINUS SIMILAR		
U+2A6E EQUALS WITH ASTERISK		
U+2A6F ALMOST EQUAL TO WITH CIRCUMFLEX ACCENT		
U+2A70 APPROXIMATELY EQUAL OR EQUAL TO		
U+2A71 EQUALS SIGN ABOVE PLUS SIGN		
U+2A72 PLUS SIGN ABOVE EQUALS SIGN		
U+2A73 EQUALS SIGN ABOVE TILDE OPERATOR		
U+2A75 TWO CONSECUTIVE EQUALS SIGNS		
U+2A76 THREE CONSECUTIVE EQUALS SIGNS		
U+2A77 EQUALS SIGN WITH TWO DOTS ABOVE AND TWO DOTS BELOW		
U+2A78 EQUIVALENT WITH FOUR DOTS ABOVE		
U+2AAE EQUALS SIGN WITH BUMPY ABOVE		
U+FE66 SMALL EQUALS SIGN		
U+FF1D FULLWIDTH EQUALS SIGN		

The following are inequivalence operators. They might not be chained and they are nonassociative.

U+2244	NOT ASYMPTOTICALLY EQUAL TO	≄	NSIMEQ
U+2247	NEITHER APPROXIMATELY NOR ACTUALLY EQUAL TO	≇	
U+2249	NOT ALMOST EQUAL TO	≉	NAPPROX
U+2260	NOT EQUAL TO	≠	=/= NE
U+2262	NOT IDENTICAL TO	≢	NEQV
U+226D	NOT EQUIVALENT TO	≭	

F.5.2 Plain Comparison Operators

Every operator listed in this section has lower precedence than any operator listed in Sections F.4.1, F.4.2, and F.4.3.

The following are less-than operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+003C	LESS-THAN SIGN	<	< LT
U+2264	LESS-THAN OR EQUAL TO	≤	<= LE
U+2266	LESS-THAN OVER EQUAL TO	≦	
U+2268	LESS-THAN BUT NOT EQUAL TO	≨	
U+226A	MUCH LESS-THAN	≪	<<
U+2272	LESS-THAN OR EQUIVALENT TO	≲	
U+22D6	LESS-THAN WITH DOT	⋖	DOTLT
U+22D8	VERY MUCH LESS-THAN	⋘	<<<
U+22DC	EQUAL TO OR LESS-THAN		
U+22E6	LESS-THAN BUT NOT EQUIVALENT TO	⋦	
U+29C0	CIRCLED LESS-THAN		
U+2A79	LESS-THAN WITH CIRCLE INSIDE		
U+2A7B	LESS-THAN WITH QUESTION MARK ABOVE		
U+2A7D	LESS-THAN OR SLANTED EQUAL TO		
U+2A7F	LESS-THAN OR SLANTED EQUAL TO WITH DOT INSIDE		
U+2A81	LESS-THAN OR SLANTED EQUAL TO WITH DOT ABOVE		
U+2A83	LESS-THAN OR SLANTED EQUAL TO WITH DOT ABOVE RIGHT		
U+2A85	LESS-THAN OR APPROXIMATE		
U+2A87	LESS-THAN AND SINGLE-LINE NOT EQUAL TO		
U+2A89	LESS-THAN AND NOT APPROXIMATE		
U+2A8D	LESS-THAN ABOVE SIMILAR OR EQUAL		
U+2A95	SLANTED EQUAL TO OR LESS-THAN		
U+2A97	SLANTED EQUAL TO OR LESS-THAN WITH DOT INSIDE		
U+2A99	DOUBLE-LINE EQUAL TO OR LESS-THAN		
U+2A9B	DOUBLE-LINE SLANTED EQUAL TO OR LESS-THAN		
U+2A9D	SIMILAR OR LESS-THAN		
U+2A9F	SIMILAR ABOVE LESS-THAN ABOVE EQUALS SIGN		
U+2AA1	DOUBLE NESTED LESS-THAN		
U+2AA3	DOUBLE NESTED LESS-THAN WITH UNDERBAR		
U+2AA6	LESS-THAN CLOSED BY CURVE		
U+2AA8	LESS-THAN CLOSED BY CURVE ABOVE SLANTED EQUAL		
U+2AF7	TRIPLE NESTED LESS-THAN		
U+2AF9	DOUBLE-LINE SLANTED LESS-THAN OR EQUAL TO		
U+FE64	SMALL LESS-THAN SIGN		
U+FF1C	FULLWIDTH LESS-THAN SIGN		

The following are greater-than operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+003E GREATER-THAN SIGN > > GT
U+2265 GREATER-THAN OR EQUAL TO ≥ >= GE
U+2267 GREATER-THAN OVER EQUAL TO ≧
U+2269 GREATER-THAN BUT NOT EQUAL TO ≩
U+226B MUCH GREATER-THAN ≫ >>
U+2273 GREATER-THAN OR EQUIVALENT TO ≳
U+22D7 GREATER-THAN WITH DOT ⋗ DOTGT
U+22D9 VERY MUCH GREATER-THAN ⋙ >>>
U+22DD EQUAL TO OR GREATER-THAN
U+22E7 GREATER-THAN BUT NOT EQUIVALENT TO ⋧
U+29C1 CIRCLED GREATER-THAN
U+2A7A GREATER-THAN WITH CIRCLE INSIDE
U+2A7C GREATER-THAN WITH QUESTION MARK ABOVE
U+2A7E GREATER-THAN OR SLANTED EQUAL TO
U+2A80 GREATER-THAN OR SLANTED EQUAL TO WITH DOT INSIDE
U+2A82 GREATER-THAN OR SLANTED EQUAL TO WITH DOT ABOVE
U+2A84 GREATER-THAN OR SLANTED EQUAL TO WITH DOT ABOVE LEFT
U+2A86 GREATER-THAN OR APPROXIMATE
U+2A88 GREATER-THAN AND SINGLE-LINE NOT EQUAL TO
U+2A8A GREATER-THAN AND NOT APPROXIMATE
U+2A8E GREATER-THAN ABOVE SIMILAR OR EQUAL
U+2A96 SLANTED EQUAL TO OR GREATER-THAN
U+2A98 SLANTED EQUAL TO OR GREATER-THAN WITH DOT INSIDE
U+2A9A DOUBLE-LINE EQUAL TO OR GREATER-THAN
U+2A9C DOUBLE-LINE SLANTED EQUAL TO OR GREATER-THAN
U+2A9E SIMILAR OR GREATER-THAN
U+2AA0 SIMILAR ABOVE GREATER-THAN ABOVE EQUALS SIGN
U+2AA2 DOUBLE NESTED GREATER-THAN
U+2AA7 GREATER-THAN CLOSED BY CURVE
U+2AA9 GREATER-THAN CLOSED BY CURVE ABOVE SLANTED EQUAL
U+2AF8 TRIPLE NESTED GREATER-THAN
U+2AFA DOUBLE-LINE SLANTED GREATER-THAN OR EQUAL TO
U+FE65 SMALL GREATER-THAN SIGN
U+FF1E FULLWIDTH GREATER-THAN SIGN

The following are miscellaneous plain comparison operators. They might not be mixed or chained and they are nonassociative.

U+226E NOT LESS-THAN ≮ NLT
U+226F NOT GREATER-THAN ≯ NGT
U+2270 NEITHER LESS-THAN NOR EQUAL TO ≰ NLE
U+2271 NEITHER GREATER-THAN NOR EQUAL TO ≱ NGE
U+2274 NEITHER LESS-THAN NOR EQUIVALENT TO ≴
U+2275 NEITHER GREATER-THAN NOR EQUIVALENT TO ≵
U+2276 LESS-THAN OR GREATER-THAN ≶
U+2277 GREATER-THAN OR LESS-THAN ≷
U+2278 NEITHER LESS-THAN NOR GREATER-THAN
U+2279 NEITHER GREATER-THAN NOR LESS-THAN
U+22DA LESS-THAN EQUAL TO OR GREATER-THAN ⋚
U+22DB GREATER-THAN EQUAL TO OR LESS-THAN ⋛
U+2A8B LESS-THAN ABOVE DOUBLE-LINE EQUAL ABOVE GREATER-THAN
U+2A8C GREATER-THAN ABOVE DOUBLE-LINE EQUAL ABOVE LESS-THAN
U+2A8F LESS-THAN ABOVE SIMILAR ABOVE GREATER-THAN

U+2A90 GREATER-THAN ABOVE SIMILAR ABOVE LESS-THAN
U+2A91 LESS-THAN ABOVE GREATER-THAN ABOVE DOUBLE-LINE EQUAL
U+2A92 GREATER-THAN ABOVE LESS-THAN ABOVE DOUBLE-LINE EQUAL
U+2A93 LESS-THAN ABOVE SLANTED EQUAL ABOVE GREATER-THAN ABOVE SLANTED EQUAL
U+2A94 GREATER-THAN ABOVE SLANTED EQUAL ABOVE LESS-THAN ABOVE SLANTED EQUAL
U+2AA4 GREATER-THAN OVERLAPPING LESS-THAN
U+2AA5 GREATER-THAN BESIDE LESS-THAN

The following are not really comparison operators, but it is convenient to list them here because they also have lower precedence than any operator listed in Sections F.4.1, F.4.2, and F.4.3. They are left associative and they have the same precedence.

U+0023 NUMBER SIGN	#	#
U+003A COLON	:	:
U+2237 PROPORTION		::

F.5.3 Set Comparison Operators

Every operator listed in this section has lower precedence than any operator listed in Section F.4.4.

The following are subset comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+2282 SUBSET OF	⊂	SUBSET
U+2286 SUBSET OF OR EQUAL TO	⊆	SUBSETEQ
U+228A SUBSET OF WITH NOT EQUAL TO	⊊	SUBSETNEQ
U+22D0 DOUBLE SUBSET	⋐	SUBSUB
U+27C3 OPEN SUBSET		
U+2ABD SUBSET WITH DOT		
U+2ABF SUBSET WITH PLUS SIGN BELOW		
U+2AC1 SUBSET WITH MULTIPLICATION SIGN BELOW		
U+2AC3 SUBSET OF OR EQUAL TO WITH DOT ABOVE		
U+2AC5 SUBSET OF ABOVE EQUALS SIGN		
U+2AC7 SUBSET OF ABOVE TILDE OPERATOR		
U+2AC9 SUBSET OF ABOVE ALMOST EQUAL TO		
U+2ACB SUBSET OF ABOVE NOT EQUAL TO		
U+2ACF CLOSED SUBSET		
U+2AD1 CLOSED SUBSET OR EQUAL TO		
U+2AD5 SUBSET ABOVE SUBSET		

The following are superset comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+2283 SUPERSET OF	⊃	SUPSET
U+2287 SUPERSET OF OR EQUAL TO	⊇	SUPSETEQ
U+228B SUPERSET OF WITH NOT EQUAL TO	⊋	SUPSETNEQ
U+22D1 DOUBLE SUPERSET	⋑	SUPSUP
U+27C4 OPEN SUPERSET		
U+2ABE SUPERSET WITH DOT		
U+2AC0 SUPERSET WITH PLUS SIGN BELOW		
U+2AC2 SUPERSET WITH MULTIPLICATION SIGN BELOW		
U+2AC4 SUPERSET OF OR EQUAL TO WITH DOT ABOVE		
U+2AC6 SUPERSET OF ABOVE EQUALS SIGN		

U+2AC8 SUPERSET OF ABOVE TILDE OPERATOR
U+2ACA SUPERSET OF ABOVE ALMOST EQUAL TO
U+2ACC SUPERSET OF ABOVE NOT EQUAL TO
U+2AD0 CLOSED SUPERSET
U+2AD2 CLOSED SUPERSET OR EQUAL TO
U+2AD6 SUPERSET ABOVE SUPERSET

The following are miscellaneous set comparison operators. They might not be mixed or chained and they are nonassociative.

U+2284 NOT A SUBSET OF ⊄ NSUBSET
U+2285 NOT A SUPERSET OF ⊅ NSUPSET
U+2288 NEITHER A SUBSET OF NOR EQUAL TO ⊈ NSUBSETEQ
U+2289 NEITHER A SUPERSET OF NOR EQUAL TO ⊉ NSUPSETEQ
U+2AD3 SUBSET ABOVE SUPERSET
U+2AD4 SUPERSET ABOVE SUBSET
U+2AD7 SUPERSET BESIDE SUBSET
U+2AD8 SUPERSET BESIDE AND JOINED BY DASH WITH SUBSET

F.5.4 Square Comparison Operators

Every operator listed in this section has lower precedence than any operator listed in Section F.4.5.

The following are square "image of" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+228F SQUARE IMAGE OF ⊏ SQSUBSET
U+2291 SQUARE IMAGE OF OR EQUAL TO ⊑ SQSUBSETEQ
U+22E4 SQUARE IMAGE OF OR NOT EQUAL TO

The following are square "original of" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+2290 SQUARE ORIGINAL OF ⊐ SQSUPSET
U+2292 SQUARE ORIGINAL OF OR EQUAL TO ⊒ SQSUPSETEQ
U+22E5 SQUARE ORIGINAL OF OR NOT EQUAL TO

The following are miscellaneous square comparison operators. They might not be mixed or chained and they are nonassociative.

U+22E2 NOT SQUARE IMAGE OF OR EQUAL TO ⋢
U+22E3 NOT SQUARE ORIGINAL OF OR EQUAL TO ⋣

F.5.5 Curly Comparison Operators

Every operator listed in this section has lower precedence than any operator listed in Section F.4.6.

The following are curly "precedes" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+227A PRECEDES ≺ PREC
U+227C PRECEDES OR EQUAL TO ≼ PRECEQ
U+227E PRECEDES OR EQUIVALENT TO ≾ PRECSIM
U+22B0 PRECEDES UNDER RELATION

U+22DE EQUAL TO OR PRECEDES ⋞ EQPREC
U+22E8 PRECEDES BUT NOT EQUIVALENT TO ⋨ PRECNSIM
U+2AAF PRECEDES ABOVE SINGLE-LINE EQUALS SIGN
U+2AB1 PRECEDES ABOVE SINGLE-LINE NOT EQUAL TO
U+2AB3 PRECEDES ABOVE EQUALS SIGN
U+2AB5 PRECEDES ABOVE NOT EQUAL TO
U+2AB7 PRECEDES ABOVE ALMOST EQUAL TO
U+2AB9 PRECEDES ABOVE NOT ALMOST EQUAL TO
U+2ABB DOUBLE PRECEDES

The following are curly "succeeds" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+227B SUCCEEDS ≻ SUCC
U+227D SUCCEEDS OR EQUAL TO ≽ SUCCEQ
U+227F SUCCEEDS OR EQUIVALENT TO ≿ SUCCSIM
U+22B1 SUCCEEDS UNDER RELATION
U+22DF EQUAL TO OR SUCCEEDS ⋟ EQSUCC
U+22E9 SUCCEEDS BUT NOT EQUIVALENT TO ⋩ SUCCNSIM
U+2AB0 SUCCEEDS ABOVE SINGLE-LINE EQUALS SIGN
U+2AB2 SUCCEEDS ABOVE SINGLE-LINE NOT EQUAL TO
U+2AB4 SUCCEEDS ABOVE EQUALS SIGN
U+2AB6 SUCCEEDS ABOVE NOT EQUAL TO
U+2AB8 SUCCEEDS ABOVE ALMOST EQUAL TO
U+2ABA SUCCEEDS ABOVE NOT ALMOST EQUAL TO
U+2ABC DOUBLE SUCCEEDS

The following are miscellaneous curly comparison operators. They might not be mixed or chained and they are nonassociative.

U+2280 DOES NOT PRECEDE ⊀ NPREC
U+2281 DOES NOT SUCCEED ⊁ NSUCC
U+22E0 DOES NOT PRECEDE OR EQUAL ⋠
U+22E1 DOES NOT SUCCEED OR EQUAL ⋡

F.5.6 Triangular Comparison Operators

The following are triangular "subgroup" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+22B2 NORMAL SUBGROUP OF ◁
U+22B4 NORMAL SUBGROUP OF OR EQUAL TO ⊴

The following are triangular "contains as subgroup" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+22B3 CONTAINS AS NORMAL SUBGROUP ▷
U+22B5 CONTAINS AS NORMAL SUBGROUP OR EQUAL TO ⊵

The following are miscellaneous triangular comparison operators. They might not be mixed or chained and they are nonassociative.

U+22EA NOT NORMAL SUBGROUP OF ⋪
U+22EB DOES NOT CONTAIN AS NORMAL SUBGROUP ⋫
U+22EC NOT NORMAL SUBGROUP OF OR EQUAL TO ⋬

U+22ED DOES NOT CONTAIN AS NORMAL SUBGROUP OR EQUAL ⋭

F.5.7 Chickenfoot Comparison Operators

The following are chickenfoot "smaller than" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+2AAA SMALLER THAN ⪷ SMALLER
U+2AAC SMALLER THAN OR EQUAL TO ⪻ SMALLEREQ

The following are chickenfoot "larger than" comparison operators. They may be mixed and chained with each other and with equivalence operators (see Section F.5.1).

U+2AAB LARGER THAN ⪸ LARGER
U+2AAD LARGER THAN OR EQUAL TO ⪼ LARGEREQ

F.5.8 Miscellaneous Relational Operators

The following operators are considered to be relational operators, having higher precedence than certain boolean operators, as described in a later section. They are nonassociative.

U+2208 ELEMENT OF ∈ IN
U+2209 NOT AN ELEMENT OF ∉ NOTIN
U+220A SMALL ELEMENT OF ∊
U+220B CONTAINS AS MEMBER ∋ CONTAINS
U+220C DOES NOT CONTAIN AS MEMBER ∌
U+220D SMALL CONTAINS AS MEMBER ∍
U+22F2 ELEMENT OF WITH LONG HORIZONTAL STROKE
U+22F3 ELEMENT OF WITH VERTICAL BAR AT END OF HORIZONTAL STROKE
U+22F4 SMALL ELEMENT OF WITH VERTICAL BAR AT END OF HORIZONTAL STROKE
U+22F5 ELEMENT OF WITH DOT ABOVE ⋵
U+22F6 ELEMENT OF WITH OVERBAR ⋶
U+22F7 SMALL ELEMENT OF WITH OVERBAR ⋷
U+22F8 ELEMENT OF WITH UNDERBAR ⋸
U+22F9 ELEMENT OF WITH TWO HORIZONTAL STROKES
U+22FA CONTAINS WITH LONG HORIZONTAL STROKE
U+22FB CONTAINS WITH VERTICAL BAR AT END OF HORIZONTAL STROKE
U+22FC SMALL CONTAINS WITH VERTICAL BAR AT END OF HORIZONTAL STROKE
U+22FD CONTAINS WITH OVERBAR ⋽
U+22FE SMALL CONTAINS WITH OVERBAR ⋾
U+22FF Z NOTATION BAG MEMBERSHIP

F.6 Boolean Operators

Every operator listed in this section has lower precedence than any operator listed in Section F.5.

The following are the boolean conjunction operators. They are left associative.

U+2227 LOGICAL AND ∧ AND
U+27D1 AND WITH DOT

U+2A51 LOGICAL AND WITH DOT ABOVE ⩑
U+2A53 DOUBLE LOGICAL AND
U+2A55 TWO INTERSECTING LOGICAL AND ⩕
U+2A5A LOGICAL AND WITH MIDDLE STEM
U+2A5C LOGICAL AND WITH HORIZONTAL DASH
U+2A5E LOGICAL AND WITH DOUBLE OVERBAR
U+2A60 LOGICAL AND WITH DOUBLE UNDERBAR

The following are the boolean disjunction operators. They are left associative.

U+2228 LOGICAL OR ∨ OR
U+2A52 LOGICAL OR WITH DOT ABOVE ⩒
U+2A54 DOUBLE LOGICAL OR
U+2A56 TWO INTERSECTING LOGICAL OR ⩖
U+2A5B LOGICAL OR WITH MIDDLE STEM
U+2A5D LOGICAL OR WITH HORIZONTAL DASH
U+2A62 LOGICAL OR WITH DOUBLE OVERBAR
U+2A63 LOGICAL OR WITH DOUBLE UNDERBAR

They each have lower precedence than any of the boolean conjunction operators.

The following are miscellaneous boolean operators that are nonassociative.

U+2192 RIGHTWARDS ARROW → -> IMPLIES
U+2194 LEFT RIGHT ARROW ↔ <-> IFF
U+22BC NAND ⊼
U+22BD NOR ⊽

The following is a miscellaneous boolean operator that is left associative.

U+22BB XOR ⊻

F.7 Other Operators

All the operators listed in this section are nonassociative.

As specified in Section 16.2, the following operator has higher precedence than any other operator.

U+005E CIRCUMFLEX ACCENT ^ ^

Each of the following operators has no defined precedence relationships to any of the other operators listed in this appendix.

U+0021 EXCLAMATION MARK ! !
U+0024 DOLLAR SIGN $ $
U+0025 PERCENT SIGN % %
U+003F QUESTION MARK ? ?
U+0040 COMMERCIAL AT @ @
U+007E TILDE ~ ~
U+00A1 INVERTED EXCLAMATION MARK ¡
U+00A2 CENT SIGN CENTS
U+00A3 POUND SIGN

U+00A4 CURRENCY SIGN
U+00A5 YEN SIGN
U+00A6 BROKEN BAR
U+00AC NOT SIGN ¬ NOT
U+00B0 DEGREE SIGN ° DEGREES
U+00BF INVERTED QUESTION MARK ¿
U+2020 DAGGER †
U+2021 DOUBLE DAGGER ‡
U+203C DOUBLE EXCLAMATION MARK ‼ !!
U+2190 LEFTWARDS ARROW ← <-
U+2191 UPWARDS ARROW ↑ UPARROW
U+2193 DOWNWARDS ARROW ↓ DOWNARROW
U+2195 UP DOWN ARROW ↕ UPDOWNARROW
U+2196 NORTH WEST ARROW ↖ NWARROW
U+2197 NORTH EAST ARROW ↗ NEARROW
U+2198 SOUTH EAST ARROW ↘ SEARROW
U+2199 SOUTH WEST ARROW ↙ SWARROW
U+219A LEFTWARDS ARROW WITH STROKE ↚ <-/-
U+219B RIGHTWARDS ARROW WITH STROKE ↛ -/->
U+219C LEFTWARDS WAVE ARROW
U+219D RIGHTWARDS WAVE ARROW ⇝ LEADSTO
U+219E LEFTWARDS TWO HEADED ARROW
U+219F UPWARDS TWO HEADED ARROW
U+21A0 RIGHTWARDS TWO HEADED ARROW
U+21A1 DOWNWARDS TWO HEADED ARROW
U+21A2 LEFTWARDS ARROW WITH TAIL
U+21A3 RIGHTWARDS ARROW WITH TAIL
U+21A4 LEFTWARDS ARROW FROM BAR
U+21A5 UPWARDS ARROW FROM BAR
U+21A6 RIGHTWARDS ARROW FROM BAR ↦ MAPSTO |->
U+21A7 DOWNWARDS ARROW FROM BAR
U+21A8 UP DOWN ARROW WITH BASE
U+21A9 LEFTWARDS ARROW WITH HOOK
U+21AA RIGHTWARDS ARROW WITH HOOK
U+21AB LEFTWARDS ARROW WITH LOOP
U+21AC RIGHTWARDS ARROW WITH LOOP
U+21AD LEFT RIGHT WAVE ARROW
U+21AE LEFT RIGHT ARROW WITH STROKE
U+21AF DOWNWARDS ZIGZAG ARROW
U+21B0 UPWARDS ARROW WITH TIP LEFTWARDS
U+21B1 UPWARDS ARROW WITH TIP RIGHTWARDS
U+21B2 DOWNWARDS ARROW WITH TIP LEFTWARDS
U+21B3 DOWNWARDS ARROW WITH TIP RIGHTWARDS
U+21B4 RIGHTWARDS ARROW WITH CORNER DOWNWARDS
U+21B5 DOWNWARDS ARROW WITH CORNER LEFTWARDS
U+21B6 ANTICLOCKWISE TOP SEMICIRCLE ARROW
U+21B7 CLOCKWISE TOP SEMICIRCLE ARROW
U+21B8 NORTH WEST ARROW TO LONG BAR
U+21B9 LEFTWARDS ARROW TO BAR OVER RIGHTWARDS ARROW TO BAR
U+21BA ANTICLOCKWISE OPEN CIRCLE ARROW
U+21BB CLOCKWISE OPEN CIRCLE ARROW
U+21BC LEFTWARDS HARPOON WITH BARB UPWARDS ↼ LEFTHARPOONUP

U+21BD LEFTWARDS HARPOON WITH BARB DOWNWARDS ↽ LEFTHARPOONDOWN
U+21BE UPWARDS HARPOON WITH BARB RIGHTWARDS ↾ UPHARPOONRIGHT
U+21BF UPWARDS HARPOON WITH BARB LEFTWARDS ↿ UPHARPOONLEFT
U+21C0 RIGHTWARDS HARPOON WITH BARB UPWARDS ⇀ RIGHTHARPOONUP
U+21C1 RIGHTWARDS HARPOON WITH BARB DOWNWARDS ⇁ RIGHTHARPOONDOWN
U+21C2 DOWNWARDS HARPOON WITH BARB RIGHTWARDS ⇂ DOWNHARPOONRIGHT
U+21C3 DOWNWARDS HARPOON WITH BARB LEFTWARDS ⇃ DOWNHARPOONLEFT
U+21C4 RIGHTWARDS ARROW OVER LEFTWARDS ARROW ⇄ RIGHTLEFTARROWS
U+21C5 UPWARDS ARROW LEFTWARDS OF DOWNWARDS ARROW
U+21C6 LEFTWARDS ARROW OVER RIGHTWARDS ARROW ⇆ LEFTRIGHTARROWS
U+21C7 LEFTWARDS PAIRED ARROWS ⇇ LEFTLEFTARROWS
U+21C8 UPWARDS PAIRED ARROWS ⇈ UPUPARROWS
U+21C9 RIGHTWARDS PAIRED ARROWS ⇉ RIGHTRIGHTARROWS
U+21CA DOWNWARDS PAIRED ARROWS ⇊ DOWNDOWNARROWS
U+21CB LEFTWARDS HARPOON OVER RIGHTWARDS HARPOON
U+21CC RIGHTWARDS HARPOON OVER LEFTWARDS HARPOON ⇌ RIGHTLEFTHARPOONS
U+21CD LEFTWARDS DOUBLE ARROW WITH STROKE ⇍
U+21CE LEFT RIGHT DOUBLE ARROW WITH STROKE ⇎
U+21CF RIGHTWARDS DOUBLE ARROW WITH STROKE ⇏
U+21D0 LEFTWARDS DOUBLE ARROW ⇐
U+21D1 UPWARDS DOUBLE ARROW ⇑
U+21D2 RIGHTWARDS DOUBLE ARROW ⇒ =>
U+21D3 DOWNWARDS DOUBLE ARROW ⇓
U+21D4 LEFT RIGHT DOUBLE ARROW ⇔ <=>
U+21D5 UP DOWN DOUBLE ARROW ⇕
U+21D6 NORTH WEST DOUBLE ARROW
U+21D7 NORTH EAST DOUBLE ARROW
U+21D8 SOUTH EAST DOUBLE ARROW
U+21D9 SOUTH WEST DOUBLE ARROW
U+21DA LEFTWARDS TRIPLE ARROW ⇐
U+21DB RIGHTWARDS TRIPLE ARROW ⇛
U+21DC LEFTWARDS SQUIGGLE ARROW
U+21DD RIGHTWARDS SQUIGGLE ARROW ⇝
U+21DE UPWARDS ARROW WITH DOUBLE STROKE
U+21DF DOWNWARDS ARROW WITH DOUBLE STROKE
U+21E0 LEFTWARDS DASHED ARROW ⇠--
U+21E1 UPWARDS DASHED ARROW
U+21E2 RIGHTWARDS DASHED ARROW --⇢
U+21E3 DOWNWARDS DASHED ARROW
U+21E4 LEFTWARDS ARROW TO BAR
U+21E5 RIGHTWARDS ARROW TO BAR
U+21E6 LEFTWARDS WHITE ARROW
U+21E7 UPWARDS WHITE ARROW
U+21E8 RIGHTWARDS WHITE ARROW
U+21E9 DOWNWARDS WHITE ARROW
U+21EA UPWARDS WHITE ARROW FROM BAR
U+21EB UPWARDS WHITE ARROW ON PEDESTAL
U+21EC UPWARDS WHITE ARROW ON PEDESTAL WITH HORIZONTAL BAR
U+21ED UPWARDS WHITE ARROW ON PEDESTAL WITH VERTICAL BAR
U+21EE UPWARDS WHITE DOUBLE ARROW
U+21EF UPWARDS WHITE DOUBLE ARROW ON PEDESTAL
U+21F0 RIGHTWARDS WHITE ARROW FROM WALL

U+21F1 NORTH WEST ARROW TO CORNER
U+21F2 SOUTH EAST ARROW TO CORNER
U+21F3 UP DOWN WHITE ARROW
U+21F4 RIGHT ARROW WITH SMALL CIRCLE
U+21F5 DOWNWARDS ARROW LEFTWARDS OF UPWARDS ARROW
U+21F6 THREE RIGHTWARDS ARROWS
U+21F7 LEFTWARDS ARROW WITH VERTICAL STROKE
U+21F8 RIGHTWARDS ARROW WITH VERTICAL STROKE
U+21F9 LEFT RIGHT ARROW WITH VERTICAL STROKE
U+21FA LEFTWARDS ARROW WITH DOUBLE VERTICAL STROKE
U+21FB RIGHTWARDS ARROW WITH DOUBLE VERTICAL STROKE
U+21FC LEFT RIGHT ARROW WITH DOUBLE VERTICAL STROKE
U+21FD LEFTWARDS OPEN-HEADED ARROW
U+21FE RIGHTWARDS OPEN-HEADED ARROW
U+21FF LEFT RIGHT OPEN-HEADED ARROW
U+2201 COMPLEMENT ∁
U+2202 PARTIAL DIFFERENTIAL ∂ DEL
U+2204 THERE DOES NOT EXIST ∄
U+2206 INCREMENT ∆
U+220F N-ARY PRODUCT ∏ PROD
U+2210 N-ARY COPRODUCT ∐ COPROD
U+2211 N-ARY SUMMATION ∑ SUM
U+2218 RING OPERATOR ∘ CIRC RING COMPOSE
U+2219 BULLET OPERATOR ∙
U+221A SQUARE ROOT √ SQRT
U+221B CUBE ROOT CBRT
U+221C FOURTH ROOT FOURTHROOT
U+221D PROPORTIONAL TO ∝ PROPTO
U+2223 DIVIDES ∣ DIVIDES
U+2224 DOES NOT DIVIDE ∤
U+2225 PARALLEL TO ∥ PARALLEL
U+2226 NOT PARALLEL TO ∦ NPARALLEL
U+222B INTEGRAL ∫
U+222C DOUBLE INTEGRAL
U+222D TRIPLE INTEGRAL
U+222E CONTOUR INTEGRAL ∮
U+222F SURFACE INTEGRAL
U+2230 VOLUME INTEGRAL
U+2231 CLOCKWISE INTEGRAL
U+2232 CLOCKWISE CONTOUR INTEGRAL
U+2233 ANTICLOCKWISE CONTOUR INTEGRAL
U+2234 THEREFORE ∴
U+2235 BECAUSE ∵
U+2236 RATIO
U+2239 EXCESS
U+223A GEOMETRIC PROPORTION
U+223B HOMOTHETIC
U+223C TILDE OPERATOR ∼
U+223D REVERSED TILDE ∽
U+223E INVERTED LAZY S
U+223F SINE WAVE
U+2240 WREATH PRODUCT ≀ WREATH

U+2241 NOT TILDE ≁
U+224B TRIPLE TILDE
U+224F DIFFERENCE BETWEEN ≏ BUMPEQ
U+2250 APPROACHES THE LIMIT ≐ DOTEQ
U+2258 CORRESPONDS TO
U+2259 ESTIMATES
U+225A EQUIANGULAR TO
U+225E MEASURED BY
U+226C BETWEEN ≬
U+228C MULTISET
U+229A CIRCLED RING OPERATOR ⊚ CIRCLEDRING
U+229D CIRCLED DASH ⊝
U+22A2 RIGHT TACK ⊢ VDASH TURNSTILE
U+22A3 LEFT TACK ⊣ DASHV
U+22A6 ASSERTION ⊦
U+22A7 MODELS ⊧
U+22A8 TRUE ⊨
U+22A9 FORCES ⊩
U+22AA TRIPLE VERTICAL BAR RIGHT TURNSTILE ⊪
U+22AB DOUBLE VERTICAL BAR DOUBLE RIGHT TURNSTILE
U+22AC DOES NOT PROVE ⊬
U+22AD NOT TRUE
U+22AE DOES NOT FORCE ⊮
U+22AF NEGATED DOUBLE VERTICAL BAR DOUBLE RIGHT TURNSTILE ⊯
U+22B6 ORIGINAL OF
U+22B7 IMAGE OF
U+22B8 MULTIMAP ⊸
U+22B9 HERMITIAN CONJUGATE MATRIX
U+22BA INTERCALATE ⊺
U+22BE RIGHT ANGLE WITH ARC
U+22BF RIGHT TRIANGLE
U+22C0 N-ARY LOGICAL AND ⋀ BIGAND ALL
U+22C1 N-ARY LOGICAL OR ⋁ BIGOR ANY
U+22C2 N-ARY INTERSECTION ⋂ BIGCAP BIGINTERSECT
U+22C3 N-ARY UNION ⋃ BIGCUP BIGUNION
U+22C4 DIAMOND OPERATOR ⋄ DIAMOND
U+22C6 STAR OPERATOR ⋆ STAR
U+22C7 DIVISION TIMES ⋇
U+22C8 BOWTIE ⋈
U+22C9 LEFT NORMAL FACTOR SEMIDIRECT PRODUCT ⋉
U+22CA RIGHT NORMAL FACTOR SEMIDIRECT PRODUCT ⋊
U+22CB LEFT SEMIDIRECT PRODUCT ⋋
U+22CC RIGHT SEMIDIRECT PRODUCT ⋌
U+22D4 PITCHFORK ⋔
U+22EE VERTICAL ELLIPSIS
U+22EF MIDLINE HORIZONTAL ELLIPSIS
U+22F0 UP RIGHT DIAGONAL ELLIPSIS
U+22F1 DOWN RIGHT DIAGONAL ELLIPSIS
U+27C0 THREE DIMENSIONAL ANGLE
U+27C1 WHITE TRIANGLE CONTAINING SMALL WHITE TRIANGLE
U+27C2 PERPENDICULAR PERP
U+27D0 WHITE DIAMOND WITH CENTRED DOT

U+27D2 ELEMENT OF OPENING UPWARDS
U+27D3 LOWER RIGHT CORNER WITH DOT
U+27D4 UPPER LEFT CORNER WITH DOT
U+27D5 LEFT OUTER JOIN
U+27D6 RIGHT OUTER JOIN
U+27D7 FULL OUTER JOIN
U+27D8 LARGE UP TACK
U+27D9 LARGE DOWN TACK
U+27DA LEFT AND RIGHT DOUBLE TURNSTILE
U+27DB LEFT AND RIGHT TACK
U+27DC LEFT MULTIMAP
U+27DD LONG RIGHT TACK
U+27DE LONG LEFT TACK
U+27DF UP TACK WITH CIRCLE ABOVE
U+27E0 LOZENGE DIVIDED BY HORIZONTAL RULE
U+27E1 WHITE CONCAVE-SIDED DIAMOND
U+27E2 WHITE CONCAVE-SIDED DIAMOND WITH LEFTWARDS TICK
U+27E3 WHITE CONCAVE-SIDED DIAMOND WITH RIGHTWARDS TICK
U+27E4 WHITE SQUARE WITH LEFTWARDS TICK
U+27E5 WHITE SQUARE WITH RIGHTWARDS TICK
U+27F0 UPWARDS QUADRUPLE ARROW
U+27F1 DOWNWARDS QUADRUPLE ARROW
U+27F2 ANTICLOCKWISE GAPPED CIRCLE ARROW
U+27F3 CLOCKWISE GAPPED CIRCLE ARROW
U+27F4 RIGHT ARROW WITH CIRCLED PLUS
U+27F5 LONG LEFTWARDS ARROW
U+27F6 LONG RIGHTWARDS ARROW
U+27F7 LONG LEFT RIGHT ARROW
U+27F8 LONG LEFTWARDS DOUBLE ARROW
U+27F9 LONG RIGHTWARDS DOUBLE ARROW
U+27FA LONG LEFT RIGHT DOUBLE ARROW
U+27FB LONG LEFTWARDS ARROW FROM BAR
U+27FC LONG RIGHTWARDS ARROW FROM BAR
U+27FD LONG LEFTWARDS DOUBLE ARROW FROM BAR
U+27FE LONG RIGHTWARDS DOUBLE ARROW FROM BAR
U+27FF LONG RIGHTWARDS SQUIGGLE ARROW
U+2900 RIGHTWARDS TWO-HEADED ARROW WITH VERTICAL STROKE
U+2901 RIGHTWARDS TWO-HEADED ARROW WITH DOUBLE VERTICAL STROKE
U+2902 LEFTWARDS DOUBLE ARROW WITH VERTICAL STROKE
U+2903 RIGHTWARDS DOUBLE ARROW WITH VERTICAL STROKE
U+2904 LEFT RIGHT DOUBLE ARROW WITH VERTICAL STROKE
U+2905 RIGHTWARDS TWO-HEADED ARROW FROM BAR
U+2906 LEFTWARDS DOUBLE ARROW FROM BAR
U+2907 RIGHTWARDS DOUBLE ARROW FROM BAR
U+2908 DOWNWARDS ARROW WITH HORIZONTAL STROKE
U+2909 UPWARDS ARROW WITH HORIZONTAL STROKE
U+290A UPWARDS TRIPLE ARROW
U+290B DOWNWARDS TRIPLE ARROW
U+290C LEFTWARDS DOUBLE DASH ARROW
U+290D RIGHTWARDS DOUBLE DASH ARROW
U+290E LEFTWARDS TRIPLE DASH ARROW
U+290F RIGHTWARDS TRIPLE DASH ARROW

U+2910 RIGHTWARDS TWO-HEADED TRIPLE DASH ARROW
U+2911 RIGHTWARDS ARROW WITH DOTTED STEM
U+2912 UPWARDS ARROW TO BAR
U+2913 DOWNWARDS ARROW TO BAR
U+2914 RIGHTWARDS ARROW WITH TAIL WITH VERTICAL STROKE
U+2915 RIGHTWARDS ARROW WITH TAIL WITH DOUBLE VERTICAL STROKE
U+2916 RIGHTWARDS TWO-HEADED ARROW WITH TAIL
U+2917 RIGHTWARDS TWO-HEADED ARROW WITH TAIL WITH VERTICAL STROKE
U+2918 RIGHTWARDS TWO-HEADED ARROW WITH TAIL WITH DOUBLE VERTICAL STROKE
U+2919 LEFTWARDS ARROW-TAIL
U+291A RIGHTWARDS ARROW-TAIL
U+291B LEFTWARDS DOUBLE ARROW-TAIL
U+291C RIGHTWARDS DOUBLE ARROW-TAIL
U+291D LEFTWARDS ARROW TO BLACK DIAMOND
U+291E RIGHTWARDS ARROW TO BLACK DIAMOND
U+291F LEFTWARDS ARROW FROM BAR TO BLACK DIAMOND
U+2920 RIGHTWARDS ARROW FROM BAR TO BLACK DIAMOND
U+2921 NORTH WEST AND SOUTH EAST ARROW
U+2922 NORTH EAST AND SOUTH WEST ARROW
U+2923 NORTH WEST ARROW WITH HOOK
U+2924 NORTH EAST ARROW WITH HOOK
U+2925 SOUTH EAST ARROW WITH HOOK
U+2926 SOUTH WEST ARROW WITH HOOK
U+2927 NORTH WEST ARROW AND NORTH EAST ARROW
U+2928 NORTH EAST ARROW AND SOUTH EAST ARROW
U+2929 SOUTH EAST ARROW AND SOUTH WEST ARROW
U+292A SOUTH WEST ARROW AND NORTH WEST ARROW
U+292B RISING DIAGONAL CROSSING FALLING DIAGONAL
U+292C FALLING DIAGONAL CROSSING RISING DIAGONAL
U+292D SOUTH EAST ARROW CROSSING NORTH EAST ARROW
U+292E NORTH EAST ARROW CROSSING SOUTH EAST ARROW
U+292F FALLING DIAGONAL CROSSING NORTH EAST ARROW
U+2930 RISING DIAGONAL CROSSING SOUTH EAST ARROW
U+2931 NORTH EAST ARROW CROSSING NORTH WEST ARROW
U+2932 NORTH WEST ARROW CROSSING NORTH EAST ARROW
U+2933 WAVE ARROW POINTING DIRECTLY RIGHT
U+2934 ARROW POINTING RIGHTWARDS THEN CURVING UPWARDS
U+2935 ARROW POINTING RIGHTWARDS THEN CURVING DOWNWARDS
U+2936 ARROW POINTING DOWNWARDS THEN CURVING LEFTWARDS
U+2937 ARROW POINTING DOWNWARDS THEN CURVING RIGHTWARDS
U+2938 RIGHT-SIDE ARC CLOCKWISE ARROW
U+2939 LEFT-SIDE ARC ANTICLOCKWISE ARROW
U+293A TOP ARC ANTICLOCKWISE ARROW
U+293B BOTTOM ARC ANTICLOCKWISE ARROW
U+293C TOP ARC CLOCKWISE ARROW WITH MINUS
U+293D TOP ARC ANTICLOCKWISE ARROW WITH PLUS
U+293E LOWER RIGHT SEMICIRCULAR CLOCKWISE ARROW
U+293F LOWER LEFT SEMICIRCULAR ANTICLOCKWISE ARROW
U+2940 ANTICLOCKWISE CLOSED CIRCLE ARROW
U+2941 CLOCKWISE CLOSED CIRCLE ARROW
U+2942 RIGHTWARDS ARROW ABOVE SHORT LEFTWARDS ARROW
U+2943 LEFTWARDS ARROW ABOVE SHORT RIGHTWARDS ARROW

U+2944 SHORT RIGHTWARDS ARROW ABOVE LEFTWARDS ARROW
U+2945 RIGHTWARDS ARROW WITH PLUS BELOW
U+2946 LEFTWARDS ARROW WITH PLUS BELOW
U+2947 RIGHTWARDS ARROW THROUGH X
U+2948 LEFT RIGHT ARROW THROUGH SMALL CIRCLE
U+2949 UPWARDS TWO-HEADED ARROW FROM SMALL CIRCLE
U+294A LEFT BARB UP RIGHT BARB DOWN HARPOON
U+294B LEFT BARB DOWN RIGHT BARB UP HARPOON
U+294C UP BARB RIGHT DOWN BARB LEFT HARPOON
U+294D UP BARB LEFT DOWN BARB RIGHT HARPOON
U+294E LEFT BARB UP RIGHT BARB UP HARPOON
U+294F UP BARB RIGHT DOWN BARB RIGHT HARPOON
U+2950 LEFT BARB DOWN RIGHT BARB DOWN HARPOON
U+2951 UP BARB LEFT DOWN BARB LEFT HARPOON
U+2952 LEFTWARDS HARPOON WITH BARB UP TO BAR
U+2953 RIGHTWARDS HARPOON WITH BARB UP TO BAR
U+2954 UPWARDS HARPOON WITH BARB RIGHT TO BAR
U+2955 DOWNWARDS HARPOON WITH BARB RIGHT TO BAR
U+2956 LEFTWARDS HARPOON WITH BARB DOWN TO BAR
U+2957 RIGHTWARDS HARPOON WITH BARB DOWN TO BAR
U+2958 UPWARDS HARPOON WITH BARB LEFT TO BAR
U+2959 DOWNWARDS HARPOON WITH BARB LEFT TO BAR
U+295A LEFTWARDS HARPOON WITH BARB UP FROM BAR
U+295B RIGHTWARDS HARPOON WITH BARB UP FROM BAR
U+295C UPWARDS HARPOON WITH BARB RIGHT FROM BAR
U+295D DOWNWARDS HARPOON WITH BARB RIGHT FROM BAR
U+295E LEFTWARDS HARPOON WITH BARB DOWN FROM BAR
U+295F RIGHTWARDS HARPOON WITH BARB DOWN FROM BAR
U+2960 UPWARDS HARPOON WITH BARB LEFT FROM BAR
U+2961 DOWNWARDS HARPOON WITH BARB LEFT FROM BAR
U+2962 LEFTWARDS HARPOON WITH BARB UP ABOVE LEFTWARDS HARPOON WITH BARB DOWN
U+2963 UPWARDS HARPOON WITH BARB LEFT BESIDE UPWARDS HARPOON WITH BARB RIGHT
U+2964 RIGHTWARDS HARPOON WITH BARB UP ABOVE RIGHTWARDS HARPOON WITH BARB DOWN
U+2965 DOWNWARDS HARPOON WITH BARB LEFT BESIDE DOWNWARDS HARPOON WITH BARB RIGHT
U+2966 LEFTWARDS HARPOON WITH BARB UP ABOVE RIGHTWARDS HARPOON WITH BARB UP
U+2967 LEFTWARDS HARPOON WITH BARB DOWN ABOVE RIGHTWARDS HARPOON WITH BARB DOWN
U+2968 RIGHTWARDS HARPOON WITH BARB UP ABOVE LEFTWARDS HARPOON WITH BARB UP
U+2969 RIGHTWARDS HARPOON WITH BARB DOWN ABOVE LEFTWARDS HARPOON WITH BARB DOWN
U+296A LEFTWARDS HARPOON WITH BARB UP ABOVE LONG DASH
U+296B LEFTWARDS HARPOON WITH BARB DOWN BELOW LONG DASH
U+296C RIGHTWARDS HARPOON WITH BARB UP ABOVE LONG DASH
U+296D RIGHTWARDS HARPOON WITH BARB DOWN BELOW LONG DASH
U+296E UPWARDS HARPOON WITH BARB LEFT BESIDE DOWNWARDS HARPOON WITH BARB RIGHT
U+296F DOWNWARDS HARPOON WITH BARB LEFT BESIDE UPWARDS HARPOON WITH BARB RIGHT
U+2970 RIGHT DOUBLE ARROW WITH ROUNDED HEAD
U+2971 EQUALS SIGN ABOVE RIGHTWARDS ARROW
U+2972 TILDE OPERATOR ABOVE RIGHTWARDS ARROW
U+2973 LEFTWARDS ARROW ABOVE TILDE OPERATOR
U+2974 RIGHTWARDS ARROW ABOVE TILDE OPERATOR
U+2975 RIGHTWARDS ARROW ABOVE ALMOST EQUAL TO
U+2976 LESS-THAN ABOVE LEFTWARDS ARROW
U+2977 LEFTWARDS ARROW THROUGH LESS-THAN

U+2978 GREATER-THAN ABOVE RIGHTWARDS ARROW
U+2979 SUBSET ABOVE RIGHTWARDS ARROW
U+297A LEFTWARDS ARROW THROUGH SUBSET
U+297B SUPERSET ABOVE LEFTWARDS ARROW
U+297C LEFT FISH TAIL
U+297D RIGHT FISH TAIL
U+297E UP FISH TAIL
U+297F DOWN FISH TAIL
U+2980 TRIPLE VERTICAL BAR DELIMITER
U+2981 Z NOTATION SPOT
U+2982 Z NOTATION TYPE COLON
U+2999 DOTTED FENCE
U+299A VERTICAL ZIGZAG LINE
U+299B MEASURED ANGLE OPENING LEFT
U+299C RIGHT ANGLE VARIANT WITH SQUARE
U+299D MEASURED RIGHT ANGLE WITH DOT
U+299E ANGLE WITH S INSIDE
U+299F ACUTE ANGLE
U+29A0 SPHERICAL ANGLE OPENING LEFT
U+29A1 SPHERICAL ANGLE OPENING UP
U+29A2 TURNED ANGLE
U+29A3 REVERSED ANGLE
U+29A4 ANGLE WITH UNDERBAR
U+29A5 REVERSED ANGLE WITH UNDERBAR
U+29A6 OBLIQUE ANGLE OPENING UP
U+29A7 OBLIQUE ANGLE OPENING DOWN
U+29A8 MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING UP AND RIGHT
U+29A9 MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING UP AND LEFT
U+29AA MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING DOWN AND RIGHT
U+29AB MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING DOWN AND LEFT
U+29AC MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING RIGHT AND UP
U+29AD MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING LEFT AND UP
U+29AE MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING RIGHT AND DOWN
U+29AF MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING LEFT AND DOWN
U+29B0 REVERSED EMPTY SET
U+29B1 EMPTY SET WITH OVERBAR
U+29B2 EMPTY SET WITH SMALL CIRCLE ABOVE
U+29B3 EMPTY SET WITH RIGHT ARROW ABOVE
U+29B4 EMPTY SET WITH LEFT ARROW ABOVE
U+29B5 CIRCLE WITH HORIZONTAL BAR
U+29B6 CIRCLED VERTICAL BAR
U+29B7 CIRCLED PARALLEL
U+29B9 CIRCLED PERPENDICULAR
U+29BA CIRCLE DIVIDED BY HORIZONTAL BAR AND TOP HALF DIVIDED BY VERTICAL BAR
U+29BB CIRCLE WITH SUPERIMPOSED X
U+29BD UP ARROW THROUGH CIRCLE
U+29BE CIRCLED WHITE BULLET
U+29BF CIRCLED BULLET
U+29C2 CIRCLE WITH SMALL CIRCLE TO THE RIGHT
U+29C3 CIRCLE WITH TWO HORIZONTAL STROKES TO THE RIGHT
U+29C5 SQUARED FALLING DIAGONAL SLASH
U+29C7 SQUARED SMALL CIRCLE

U+29C8 SQUARED SQUARE
U+29C9 TWO JOINED SQUARES
U+29CA TRIANGLE WITH DOT ABOVE
U+29CB TRIANGLE WITH UNDERBAR
U+29CC S IN TRIANGLE
U+29CD TRIANGLE WITH SERIFS AT BOTTOM
U+29CE RIGHT TRIANGLE ABOVE LEFT TRIANGLE
U+29CF LEFT TRIANGLE BESIDE VERTICAL BAR
U+29D0 VERTICAL BAR BESIDE RIGHT TRIANGLE
U+29D1 BOWTIE WITH LEFT HALF BLACK
U+29D2 BOWTIE WITH RIGHT HALF BLACK
U+29D3 BLACK BOWTIE
U+29D6 WHITE HOURGLASS
U+29D7 BLACK HOURGLASS
U+29DC INCOMPLETE INFINITY
U+29DD TIE OVER INFINITY
U+29DE INFINITY NEGATED WITH VERTICAL BAR
U+29DF DOUBLE-ENDED MULTIMAP
U+29E0 SQUARE WITH CONTOURED OUTLINE
U+29E1 INCREASES AS
U+29E2 SHUFFLE PRODUCT
U+29E6 GLEICH STARK
U+29E7 THERMODYNAMIC
U+29E8 DOWN-POINTING TRIANGLE WITH LEFT HALF BLACK
U+29E9 DOWN-POINTING TRIANGLE WITH RIGHT HALF BLACK
U+29EA BLACK DIAMOND WITH DOWN ARROW
U+29EB BLACK LOZENGE
U+29EC WHITE CIRCLE WITH DOWN ARROW
U+29ED BLACK CIRCLE WITH DOWN ARROW
U+29EE ERROR-BARRED WHITE SQUARE
U+29EF ERROR-BARRED BLACK SQUARE
U+29F0 ERROR-BARRED WHITE DIAMOND
U+29F1 ERROR-BARRED BLACK DIAMOND
U+29F2 ERROR-BARRED WHITE CIRCLE
U+29F3 ERROR-BARRED BLACK CIRCLE
U+29F4 RULE-DELAYED
U+29F6 SOLIDUS WITH OVERBAR
U+29F7 REVERSE SOLIDUS WITH HORIZONTAL STROKE
U+29FA DOUBLE PLUS ++
U+29FB TRIPLE PLUS +++
U+29FE TINY
U+29FF MINY
U+2A00 N-ARY CIRCLED DOT OPERATOR ⨀ BIGODOT
U+2A01 N-ARY CIRCLED PLUS OPERATOR ⨁ BIGOPLUS
U+2A02 N-ARY CIRCLED TIMES OPERATOR ⨂ BIGOTIMES
U+2A03 N-ARY UNION OPERATOR WITH DOT BIGUDOT
U+2A04 N-ARY UNION OPERATOR WITH PLUS BIGUPLUS
U+2A05 N-ARY SQUARE INTERSECTION OPERATOR BIGSQCAP
U+2A06 N-ARY SQUARE UNION OPERATOR BIGSQCUP
U+2A07 TWO LOGICAL AND OPERATOR
U+2A08 TWO LOGICAL OR OPERATOR
U+2A09 N-ARY TIMES OPERATOR BIGTIMES

U+2A0A MODULO TWO SUM
U+2A10 CIRCULATION FUNCTION
U+2A11 ANTICLOCKWISE INTEGRATION
U+2A12 LINE INTEGRATION WITH RECTANGULAR PATH AROUND POLE
U+2A13 LINE INTEGRATION WITH SEMICIRCULAR PATH AROUND POLE
U+2A14 LINE INTEGRATION NOT INCLUDING THE POLE
U+2A1D JOIN ⋈ JOIN
U+2A1E LARGE LEFT TRIANGLE OPERATOR
U+2A1F Z NOTATION SCHEMA COMPOSITION
U+2A20 Z NOTATION SCHEMA PIPING
U+2A21 Z NOTATION SCHEMA PROJECTION
U+2A32 SEMIDIRECT PRODUCT WITH BOTTOM CLOSED
U+2A33 SMASH PRODUCT
U+2A3C INTERIOR PRODUCT
U+2A3D RIGHTHAND INTERIOR PRODUCT
U+2A3E Z NOTATION RELATIONAL COMPOSITION
U+2A3F AMALGAMATION OR COPRODUCT
U+2A57 SLOPING LARGE OR
U+2A58 SLOPING LARGE AND
U+2A61 SMALL VEE WITH UNDERBAR
U+2A64 Z NOTATION DOMAIN ANTIRESTRICTION
U+2A65 Z NOTATION RANGE ANTIRESTRICTION
U+2A68 TRIPLE HORIZONTAL BAR WITH DOUBLE VERTICAL STROKE
U+2A69 TRIPLE HORIZONTAL BAR WITH TRIPLE VERTICAL STROKE
U+2A6A TILDE OPERATOR WITH DOT ABOVE
U+2A6B TILDE OPERATOR WITH RISING DOTS
U+2A6D CONGRUENT WITH DOT ABOVE
U+2ACD SQUARE LEFT OPEN BOX OPERATOR
U+2ACE SQUARE RIGHT OPEN BOX OPERATOR
U+2AD9 ELEMENT OF OPENING DOWNWARDS
U+2ADA PITCHFORK WITH TEE TOP
U+2ADC FORKING
U+2ADD NONFORKING
U+2ADE SHORT LEFT TACK
U+2ADF SHORT DOWN TACK
U+2AE0 SHORT UP TACK
U+2AE1 PERPENDICULAR WITH S
U+2AE2 VERTICAL BAR TRIPLE RIGHT TURNSTILE
U+2AE3 DOUBLE VERTICAL BAR LEFT TURNSTILE
U+2AE4 VERTICAL BAR DOUBLE LEFT TURNSTILE
U+2AE5 DOUBLE VERTICAL BAR DOUBLE LEFT TURNSTILE
U+2AE6 LONG DASH FROM LEFT MEMBER OF DOUBLE VERTICAL
U+2AE7 SHORT DOWN TACK WITH OVERBAR
U+2AE8 SHORT UP TACK WITH UNDERBAR
U+2AE9 SHORT UP TACK ABOVE SHORT DOWN TACK
U+2AEA DOUBLE DOWN TACK
U+2AEB DOUBLE UP TACK
U+2AEC DOUBLE STROKE NOT SIGN
U+2AED REVERSED DOUBLE STROKE NOT SIGN
U+2AEE DOES NOT DIVIDE WITH REVERSED NEGATION SLASH
U+2AEF VERTICAL LINE WITH CIRCLE ABOVE
U+2AF0 VERTICAL LINE WITH CIRCLE BELOW

U+2AF1 DOWN TACK WITH CIRCLE BELOW
U+2AF2 PARALLEL WITH HORIZONTAL STROKE
U+2AF3 PARALLEL WITH TILDE OPERATOR
U+2AF5 TRIPLE VERTICAL BAR WITH HORIZONTAL STROKE
U+2AF6 TRIPLE COLON OPERATOR
U+2AFC LARGE TRIPLE VERTICAL BAR OPERATOR
U+2AFE WHITE VERTICAL BAR
U+2AFF N-ARY WHITE VERTICAL BAR

Appendix G

Simplified Grammar for Application Programmers and Library Writers

> DRAFT NOTE
> The chapter is not up to date.

In this chapter, we provide a simplified grammar of the Fortress concrete syntax as documentation, to enable Fortress programmers to understand it more easily. It includes unimplemented Fortress language features such as dimensions and units, tests and properties, coercions, and where clauses. The full Fortress grammar is described in Appendix H.

The simplified grammar does not describe the details of the uses of operators, whitespaces, and semicolons. Instead, they are described as follows according to three different contexts influencing the whitespace-sensitivity of expressions:

statement Expressions immediately enclosed by an expression block are in a statement-like context. Multiple expressions can appear on a line if they are separated (or terminated) by semicolons. If an expression can legally end at the end of a line, it does; if it cannot, it does not. A prefix or infix operator that lacks its last operand prevents an expression from ending. For example,

$$\begin{array}{l} an = expression+ \\ \quad\quad spanning+ \\ \quad\quad four+ \\ \quad\quad lines \\ a = oneLiner \\ four(); on(); one(); line(); \end{array}$$

nested An expression or list of expressions immediately enclosed by parentheses or braces is nested. Multiple expressions are separated by commas, and the end of a line does not end an expression. Because of this effect, the meaning of a several lines of code can change if they are wrapped in parentheses. Parentheses can also be used to ensure that a multiline expression is not terminated prematurely without paying special attention to line endings.

$$\begin{array}{l} lhs = rhs \\ -aSeparateExpression \\ postProfit(revenue \\ \quad\quad\quad -expenses) \end{array}$$

pasted Fortress has special syntax for matrix pasting. Within square brackets, whitespace-separated expressions are treated (depending on their type) as either matrix elements or submatrices within a row. Because whitespace is the separator, it also ends expressions where possible. In addition, newline-or-semicolon-separated rows are pasted vertically along their columns. Higher-dimensional pasting is expressed with repeated semicolons, but repeated newlines do not have the same effect.

$$
\begin{aligned}
&id2a = [1\ 0; 0\ 1\ ] \\
&id2b = [1\ 0; \\
&\qquad 0\ 1\ ] \\
&id2c = [1\ 0 \\
&\qquad 0\ 1\ ] \\
&cube_2 = [1\ 0; 0\ 1; ; 1 - 1; 1\ 1\ ]
\end{aligned}
$$

The simplified grammar is written in the extended BNF form with the following three postfix operators:

- ?: which means that its argument (the symbol or group of symbols in parentheses to the left of the operator) can appear zero or one time

- $*$: which means that its argument can appear zero or more times

- $+$: which means that its argument can appear one or more times

G.1 Components and APIs

File	::=	*CompilationUnit*
	|	*Imports*? *Exports Decls*?
	|	*Imports*? *AbsDecls*
	|	*Imports AbsDecls*?
CompilationUnit	::=	*Component*
	|	*Api*
Component	::=	`native` ? `component` *APIName Imports*? *Exports Decls*? `end` (`component` ? *APIName*)?
Api	::=	`api` *APIName Imports*? *AbsDecls*? `end` (`api` ? *APIName*)?
Imports	::=	*Import*$^+$
Import	::=	`import api` *AliasedAPINames*
	|	`import` *ImportedNames*
ImportedNames	::=	*APIName* . { ... } (`except` *SimpleNames*)?
	|	*APIName* . { *AliasedSimpleNameList* (, ...)? }
	|	*QualifiedName* (`as` *Id*)?
SimpleNames	::=	*SimpleName*
	|	{ *SimpleNameList* }
SimpleNameList	::=	*SimpleName*(, *SimpleName*)*
AliasedSimpleName	::=	*Id* (`as` *Id*)?
	|	`opr BIG` ? (*Encloser* | *Op*) (`as` (*Encloser* | *Op*))?
	|	`opr BIG` ? *EncloserPair* (`as` *EncloserPair*)?
AliasedSimpleNameList	::=	*AliasedSimpleName*(, *AliasedSimpleName*)*
AliasedAPINames	::=	*AliasedAPIName*
	|	{ *AliasedAPINameList* }
AliasedAPIName	::=	*APIName* (`as` *Id*)?
AliasedAPINameList	::=	*AliasedAPIName*(, *AliasedAPIName*)*
Exports	::=	*Export*$^+$
Export	::=	`export` *APINames*
APINames	::=	*APIName*
	|	{ *APINameList* }
APINameList	::=	*APIName*(, *APIName*)*

G.2 Top-level Declarations

Decls	::=	*Decl*$^+$
Decl	::=	*TraitDecl*
	|	*ObjectDecl*
	|	*VarDecl*
	|	*FnDecl*
	|	*DimUnitDecl*
	|	*TypeAlias*
	|	*TestDecl*
	|	*PropertyDecl*
	|	*ExternalSyntax*
AbsDecls	::=	*AbsDecl*$^+$
AbsDecl	::=	*AbsTraitDecl*
	|	*AbsObjectDecl*
	|	*AbsVarDecl*
	|	*AbsFnDecl*
	|	*DimUnitDecl*
	|	*TypeAlias*
	|	*TestDecl*
	|	*PropertyDecl*
	|	*AbsExternalSyntax*

G.3 Trait and Object Declarations

A *TraitHeaderFront* may have 0 or 1 of each *TraitClause*.

TraitDecl	::=	*TraitMods*? *TraitHeaderFront TraitClauses GoInATrait*? `end` (`trait` ? *Id*)?
TraitHeaderFront	::=	`trait` *Id StaticParams*? *ExtendsWhere*?
TraitClauses	::=	*TraitClause**
TraitClause	::=	*Excludes*
	|	*Comprises*
	|	*Where*
GoInATrait	::=	*Coercions*? *GoFrontInATrait GoBackInATrait*?
	|	*Coercions*? *GoBackInATrait*
	|	*Coercions*
Coercions	::=	*Coercion*$^+$
GoFrontInATrait	::=	*GoesFrontInATrait*$^+$
GoesFrontInATrait	::=	*AbsFldDecl*
	|	*GetterSetterDecl*
	|	*PropertyDecl*
GoBackInATrait	::=	*GoesBackInATrait*$^+$
GoesBackInATrait	::=	*MdDecl*
	|	*PropertyDecl*
ObjectDecl	::=	*ObjectMods*? *ObjectHeader GoInAnObject*? `end` (`object` ? *Id*)?
ObjectHeader	::=	`object` *Id StaticParams*? *ObjectValParam*? *ExtendsWhere*? *FnClauses*

ObjectValParam ::= (*ObjectParams*?)
ObjectParams ::= (*ObjectParam* ,)* (*ObjectVarargs* ,)? *ObjectKeyword*(, *ObjectKeyword*)*
| (*ObjectParam* ,)* *ObjectVarargs*
| *ObjectParam* (, *ObjectParam*)*
ObjectVarargs ::= `transient` *Varargs*
ObjectKeyword ::= *ObjectParam*= *Expr*
ObjectParam ::= *ParamFldMods*? *Param*
| `var transient` *ParamWType*
| `transient` ? `var` *ParamWType*
| `transient` *Param*
ParamWType ::= *BindId IsType*?
GoInAnObject ::= *Coercions*? *GoFrontInAnObject GoBackInAnObject*?
| *Coercions*? *GoBackInAnObject*
| *Coercions*
GoFrontInAnObject ::= *GoesFrontInAnObject*+
GoesFrontInAnObject ::= *FldDecl*
| *GetterSetterDef*
| *PropertyDecl*
GoBackInAnObject ::= *GoesBackInAnObject*+
GoesBackInAnObject ::= *MdDef*
| *PropertyDecl*
AbsTraitDecl ::= *AbsTraitMods*? *TraitHeaderFront AbsTraitClauses AbsGoInATrait*? `end` (`trait` ? *Id*)?
AbsTraitClauses ::= *AbsTraitClause**
AbsTraitClause ::= *Excludes*
| *AbsComprises*
| *Where*
AbsGoInATrait ::= *AbsCoercions*? *AbsGoFrontInATrait AbsGoBackInATrait*?
| *AbsCoercions*? *AbsGoBackInATrait*
| *AbsCoercions*
AbsCoercions ::= *AbsCoercion*+
AbsGoFrontInATrait ::= *AbsGoesFrontInATrait*+
AbsGoesFrontInATrait ::= *ApiFldDecl*
| *AbsGetterSetterDecl*
| *PropertyDecl*
AbsGoBackInATrait ::= *AbsGoesBackInATrait*+
AbsGoesBackInATrait ::= *AbsMdDecl*
| *PropertyDecl*
AbsObjectDecl ::= *AbsObjectMods*? *ObjectHeader AbsGoInAnObject*? `end` (`object` ? *Id*)?
AbsGoInAnObject ::= *AbsCoercions*? *AbsGoFrontInAnObject AbsGoBackInAnObject*?
| *AbsCoercions*? *AbsGoBackInAnObject*
| *AbsCoercions*
AbsGoFrontInAnObject ::= *AbsGoesFrontInAnObject*+
AbsGoesFrontInAnObject ::= *ApiFldDecl*
| *AbsGetterSetterDecl*
| *PropertyDecl*
AbsGoBackInAnObject ::= *AbsGoesBackInAnObject*+
AbsGoesBackInAnObject ::= *AbsMdDecl*
| *PropertyDecl*

G.4 Variable Declarations

VarDecl	::=	*VarMods*? *VarWTypes InitVal*
	|	*VarImmutableMods*? *BindIdOrBindIdTuple* = *Expr*
	|	*VarMods*? *BindIdOrBindIdTuple* : *Type*... *InitVal*
	|	*VarMods*? *BindIdOrBindIdTuple* : *TupleType InitVal*
VarMods	::=	*VarMod*$^+$
VarImmutableMods	::=	*VarImmutableMod*$^+$
VarMod	::=	*AbsVarMod* | `private`
VarImmutableMod	::=	*AbsVarImmutableMod* | `private`
VarWTypes	::=	*VarWType*
	|	(*VarWType*(, *VarWType*)$^+$)
VarWType	::=	*BindId IsType*
InitVal	::=	(= |:=) *Expr*
AbsVarDecl	::=	*AbsVarMods*? *VarWTypes*
	|	*AbsVarMods*? *BindIdOrBindIdTuple* : *Type*...
	|	*AbsVarMods*? *BindIdOrBindIdTuple* : *TupleType*
AbsVarMods	::=	*AbsVarMod*$^+$
AbsVarMod	::=	`var` | `test`
AbsVarImmutableMod	::=	`test`

G.5 Function Declarations

FnDecl	::=	*FnMods*? *FnHeaderFront FnHeaderClause* = *Expr*
	|	*FnMods*? *AbsFnHeaderFront FnHeaderClause*
	|	*FnSig*
FnSig	::=	*SimpleName* : *Type*
AbsFnDecl	::=	*AbsFnMods*? *AbsFnHeaderFront FnHeaderClause*
	|	*FnSig*
FnHeaderFront	::=	*NamedFnHeaderFront*
	|	*OpHeaderFront*
AbsFnHeaderFront	::=	*AbsNamedFnHeaderFront*
	|	*AbsOpHeaderFront*
NamedFnHeaderFront	::=	*Id StaticParams*? *ValParam*
AbsNamedFnHeaderFront	::=	*Id StaticParams*? *AbsValParam*

G.6 Dimension, Unit, Type Alias, Test, Property, and External Syntax Declarations

DimUnitDecl	::=	`dim` *Id* (= *Type*)? (`unit` | `SI_unit`) *Id*$^+$ (= *Expr*)?
	|	`dim` *Id* (= *Type*)? (`default` *Id*)?
	|	(`unit` | `SI_unit`) *Id*$^+$ (: *Type*)? (= *Expr*)?
TypeAlias	::=	`type` *Id StaticParams*? = *Type*
TestDecl	::=	`test` *Id* [*GeneratorClauseList*] = *Expr*
PropertyDecl	::=	`property` (*Id* =)? (∀ *ValParam*)? *Expr*
ExternalSyntax	::=	`syntax` *OpenExpander Id CloseExpander* = *Expr*
OpenExpander	::=	*Id*
	|	*LeftEncloser*
	|	*Encloser*
CloseExpander	::=	*Id*
	|	*RightEncloser*
	|	*Encloser*
	|	`end`
AbsExternalSyntax	::=	`syntax` *OpenExpander Id CloseExpander*

G.7 Headers

Each modifier should not appear multiple times in a declaration.

IsType	::=	: *Type*
ExtendsWhere	::=	`extends` *TraitTypeWheres*
TraitTypeWheres	::=	*TraitTypeWhere*
	|	{ *TraitTypeWhereList* }
TraitTypeWhereList	::=	*TraitTypeWhere*(, *TraitTypeWhere*)*
TraitTypeWhere	::=	*TraitType Where*?
Extends	::=	`extends` *TraitTypes*
Excludes	::=	`excludes` *TraitTypes*
Comprises	::=	`comprises` *ComprisingTypes*
AbsComprises	::=	`comprises` *AbsComprisingTypes*
TraitTypes	::=	*TraitType*
	|	{ *TraitTypeList* }
TraitTypeList	::=	*TraitType*(, *TraitType*)*
ComprisingTypes	::=	*ComprisingType*
	|	{ *ComprisingTypeList* }
ComprisingTypeList	::=	*ComprisingType*(, *ComprisingType*)*
ComprisingType	::=	*TraitType*
	|	*Id*
AbsComprisingTypes	::=	*ComprisingType*
	|	{ *AbsComprisingTypeList*? }
AbsComprisingTypeList	::=	...
	|	*ComprisingTypeList* (, ...)?
Where	::=	`where` ⟦ *WhereBindingList* ⟧ ({ *WhereConstraintList* })?
	|	`where` { *WhereConstraintList* }
WhereBindingList	::=	*WhereBinding*(, *WhereBinding*)*
WhereBinding	::=	*Id Extends*?
	|	`nat` *Id*
	|	`int` *Id*
	|	`bool` *Id*
	|	`unit` *Id*
WhereConstraintList	::=	*WhereConstraint*(, *WhereConstraint*)*
WhereConstraint	::=	*Id Extends*
	|	*TypeAlias*
	|	*Type* `coerces` *Type*
	|	*Type* `widens` *Type*
	|	*UnitConstraint*
	|	*QualifiedName* = *QualifiedName*
	|	*IntConstraint*
	|	*BoolExpr*
UnitConstraint	::=	dimensionless = *Id*
	|	*Id* = dimensionless
IntConstraint	::=	*IntExpr* ≤ *IntExpr*
	|	*IntExpr* < *IntExpr*
	|	*IntExpr* ≥ *IntExpr*
	|	*IntExpr* > *IntExpr*
	|	*IntExpr* = *IntExpr*
IntVal	::=	*IntLiteralExpr*
	|	*QualifiedName*

IntExpr ::= *IntVal*
| *IntExpr* + *IntExpr*
| *IntExpr* − *IntExpr*
| *IntExpr* · *IntExpr*
| *IntExpr IntExpr*
| *IntExpr* ^ *IntVal*
| (*IntExpr*)

BoolConstraint ::= `NOT` *BoolExpr*
| *BoolExpr* `OR` *BoolExpr*
| *BoolExpr* `AND` *BoolExpr*
| *BoolExpr* `IMPLIES` *BoolExpr*
| *BoolExpr* = *BoolExpr*

BoolVal ::= *true*
| *false*
| *QualifiedName*

BoolExpr ::= *BoolVal*
| *BoolConstraint*
| (*BoolExpr*)

UnitExpr ::= *dimensionless*
| *QualifiedName*
| *UnitExpr* · *UnitExpr*
| *UnitExpr UnitExpr*
| *UnitExpr* / *UnitExpr*
| *UnitExpr* per *UnitExpr*
| *UnitExpr* ^ *UnitExpr*
| (*UnitExpr*)

FnHeaderClause ::= *IsType*? *FnClauses*

FnClauses ::= *Throws*? *Where*? *Contract*

Throws ::= `throws` *MayTraitTypes*

MayTraitTypes ::= {}
| *TraitTypes*

Contract ::= *Requires*? *Ensures*? *Invariant*?

Requires ::= `requires` { *ExprList*? }

Ensures ::= `ensures` { *EnsuresClauseList*? }

EnsuresClauseList ::= *EnsuresClause*(, *EnsuresClause*)*

EnsuresClause ::= *Expr* (`provided` *Expr*)?

Invariant ::= `invariant` { *ExprList*? }

TraitMods ::= *TraitMod*+

TraitMod ::= *AbsTraitMod* | `private`

AbsTraitMods ::= *AbsTraitMod*+

AbsTraitMod ::= `value` | `test`

ObjectMods ::= *TraitMods*

AbsObjectMods ::= *AbsTraitMods*

MdMods ::= *MdMod*+

MdMod ::= *FnMod* | `override`

AbsMdMods ::= *AbsMdMod*+

AbsMdMod ::= *AbsFnMod* | `override`

FnMods ::= *FnMod*+

FnMod ::= *AbsFnMod* | `private`

AbsFnMods ::= *AbsFnMod*+

AbsFnMod ::= *LocalFnMod* | `test`

LocalFnMods ::= *LocalFnMod*+

LocalFnMod ::= `atomic` | `io`

ParamFldMods	::=	*ParamFldMod*$^+$
ParamFldMod	::=	`hidden` | `settable` | `wrapped`
FldMods	::=	*FldMod*$^+$
FldImmutableMods	::=	*FldImmutableMod*$^+$
FldMod	::=	`var` | *AbsFldMod*
FldImmutableMod	::=	*AbsFldMod*
AbsFldMods	::=	*AbsFldMod*$^+$
AbsFldMod	::=	*ApiFldMod* | `wrapped` | `private`
ApiFldMods	::=	*ApiFldMod*$^+$
ApiFldMod	::=	`hidden` | `settable` | `test`
StaticParams	::=	⟦*StaticParamList*⟧
StaticParamList	::=	*StaticParam*(, *StaticParam*)*
StaticParam	::=	*Id Extends*? (`absorbs unit`)?
	|	`nat` *Id*
	|	`int` *Id*
	|	`bool` *Id*
	|	`dim` *Id*
	|	`unit` *Id* (: *Type*)? (`absorbs unit`)?
	|	`opr` *Op*
StaticArgs	::=	⟦*StaticArgList* ⟧
StaticArgList	::=	*StaticArg*(, *StaticArg*)*
StaticArg	::=	*Op*
	|	Unity
	|	dimensionless
	|	1 / *Type*
	|	*DimPrefixOp Type*
	|	*IntExpr*
	|	*BoolExpr*
	|	*Type*
	|	*UnitExpr*

G.8 Parameters

ValParam	::=	*BindId*
	|	(*Params*?)
AbsValParam	::=	(*AbsParams*?)
	|	*Type*
Params	::=	(*Param*,)* (*Varargs*,)? *Keyword*(, *Keyword*)*
	|	(*Param*,)* *Varargs*
	|	*Param*(, *Param*)*
AbsParams	::=	(*AbsParam*,)* (*Varargs*,)? *Keyword*(, *Keyword*)*
	|	(*AbsParam*,)* *Varargs*
	|	*AbsParam*(, *AbsParam*)*
VarargsParam	::=	*BindId*: *Type*...
Varargs	::=	*VarargsParam*
Keyword	::=	*Param*= *Expr*
PlainParam	::=	*BindId IsType*?
Param	::=	*PlainParam*
AbsPlainParam	::=	*BindId IsType*
	|	*Type*
AbsParam	::=	*AbsPlainParam*
OpHeaderFront	::=	`opr BIG`? (*LeftEncloser* $\mapsto$ | *LeftEncloser* | *Encloser*) *StaticParams*? *Params*? (*RightEncloser* | *Encloser*)
	|	`opr` *ValParam* (*Op* | *ExponentOp* | `^`) *StaticParams*?
	|	`opr BIG`? (*Op* | `^` | *Encloser* | $\sum$ | $\prod$) *StaticParams*? *ValParam*
AbsOpHeaderFront	::=	`opr BIG`? (*LeftEncloser* $\mapsto$ | *LeftEncloser* | *Encloser*) *StaticParams*? *AbsParams*? (*RightEncloser* | *Encloser*)
	|	`opr` *AbsValParam* (*Op* | *ExponentOp* | `^`) *StaticParams*?
	|	`opr BIG`? (*Op* | `^` | *Encloser* | $\sum$ | $\prod$) *StaticParams*? *AbsValParam*

G.9 Method Declarations

MdDecl	::=	*MdDef*
	|	`abstract` ? *MdMods*? *AbsMdHeaderFront FnHeaderClause*
MdDef	::=	*MdMods*? *MdHeaderFront FnHeaderClause* = *Expr*
AbsMdDecl	::=	`abstract` ? *AbsMdMods*? *AbsMdHeaderFront FnHeaderClause*
MdHeaderFront	::=	*NamedMdHeaderFront*
	|	*OpMdHeaderFront*
AbsMdHeaderFront	::=	*AbsNamedMdHeaderFront*
	|	*AbsOpMdHeaderFront*
NamedMdHeaderFront	::=	*Id StaticParams*? *MdValParam*
AbsNamedMdHeaderFront	::=	*Id StaticParams*? *AbsMdValParam*
GetterSetterDecl	::=	*GetterSetterDef*
	|	`abstract` ? *FnMods*? *GetterSetterMod AbsMdHeaderFront FnHeaderClause*
GetterSetterDef	::=	*FnMods*? *GetterSetterMod MdHeaderFront FnHeaderClause* = *Expr*
GetterSetterMod	::=	`getter` | `setter`
AbsGetterSetterDecl	::=	`abstract` ? *AbsFnMods*? *GetterSetterMod AbsMdHeaderFront FnHeaderClause*
Coercion	::=	`coerce` *StaticParams*? (*BindId IsType*) *CoercionClauses* `widens` ? = *Expr*
AbsCoercion	::=	`coerce` *StaticParams*? (*BindId IsType*) *CoercionClauses* `widens` ?
CoercionClauses	::=	*CoercionWhere*? *Ensures*? *Invariant*?
CoercionWhere	::=	`where` ⟦ *WhereBindingList* ⟧ ({ *CoercionWhereConstraintList* })?
	|	`where` { *CoercionWhereConstraintList* }
CoercionWhereConstraintList	::=	*CoercionWhereConstraint*(, *CoercionWhereConstraint*)*
CoercionWhereConstraint	::=	*WhereConstraint*
	|	*Type* `widens or coerces` *Type*

G.10 Method Parameters

MdValParam ::= (*MdParams*?)
AbsMdValParam ::= (*AbsMdParams*?)
MdParams ::= (*MdParam*,)* (*Varargs*,)? *MdKeyword*(, *MdKeyword*)*
| (*MdParam*,)* *Varargs*
| *MdParam*(, *MdParam*)*
AbsMdParams ::= (*AbsMdParam*,)* (*Varargs*,)? *MdKeyword*(, *MdKeyword*)*
| (*AbsMdParam*,)* *Varargs*
| *AbsMdParam*(, *AbsMdParam*)*
MdKeyword ::= *MdParam*= *Expr*
MdParam ::= *Param*
| `self`
AbsMdParam ::= `self`
| *AbsParam*
OpMdHeaderFront ::= `opr BIG`? (*LeftEncloser* $\mapsto$ | *LeftEncloser* | *Encloser*) *StaticParams*? *Params*?
(*RightEncloser* | *Encloser*) (:= (*SubscriptAssignParam*))?
| `opr` *ValParam* (*Op* | *ExponentOp*) *StaticParams*?
| `opr BIG`? (*Op* | ^ | *Encloser* | $\sum$ | $\prod$) *StaticParams*? *ValParam*
AbsOpMdHeaderFront ::= `opr BIG`? (*LeftEncloser* $\mapsto$ | *LeftEncloser* | *Encloser*) *StaticParams*? *AbsParams*?
(*RightEncloser* | *Encloser*) (:= (*AbsSubscriptAssignParam*))?
| `opr` *AbsValParam* (*Op* | *ExponentOp*) *StaticParams*?
| `opr BIG`? (*Op* | ^ | *Encloser* | $\sum$ | $\prod$) *StaticParams*? *AbsValParam*
SubscriptAssignParam ::= *Varargs*
| *Param*
AbsSubscriptAssignParam ::= *Varargs*
| *AbsParam*

G.11 Field Declarations

FldDecl ::= *FldMods*? *FldWTypes InitVal*
| *FldImmutableMods*? *BindIdOrBindIdTuple* = *Expr*
| *FldMods*? *BindIdOrBindIdTuple* : *Type*... *InitVal*
| *FldMods*? *BindIdOrBindIdTuple* : *TupleType InitVal*
FldWTypes ::= *FldWType*
| (*FldWType*(, *FldWType*)$^+$)
FldWType ::= *BindId IsType*

G.12 Abstract Field Declarations

AbsFldDecl ::= *AbsFldMods*? *AbsFldWTypes*
| *AbsFldMods*? *BindIdOrBindIdTuple* : *Type*...
| *AbsFldMods*? *BindIdOrBindIdTuple* : *TupleType*
AbsFldWTypes ::= *AbsFldWType*
| (*AbsFldWType*(, *AbsFldWType*)$^+$)
AbsFldWType ::= *BindId IsType*
ApiFldDecl ::= *ApiFldMods*? *BindId IsType*

G.13 Expressions

Expr	::=	*AssignExpr*
	|	*OpExpr*
	|	*DelimitedExpr*
	|	*FlowExpr*
	|	`fn` *BindId Throws*? ⇒ *Expr*
	|	`fn` (*Params*) *IsType*? *Throws*? ⇒ *Expr*
	|	*Expr* `typed` *Type*
	|	*Expr* `asif` *Type*
AssignExpr	::=	*AssignLefts AssignOp Expr*
AssignLefts	::=	(*AssignLeft*(, *AssignLeft*)*)
	|	*AssignLeft*
AssignLeft	::=	*SubscriptExpr*
	|	*FieldSelection*
	|	*QualifiedName*
SubscriptExpr	::=	*Primary LeftEncloser StaticArgs*? *ExprList*? *RightEncloser*
FieldSelection	::=	*Primary*. *Id*
OpExpr	::=	*EncloserOp OpExpr*? *EncloserOp*?
	|	*OpExpr EncloserOp OpExpr*?
	|	*Primary*
EncloserOp	::=	*Encloser*
	|	*Op*
Primary	::=	*PrimaryItem* (, *PrimaryItem*)*
PrimaryItem	::=	*ArrayExpr*
	|	*MapExpr*
	|	*Comprehension*
	|	*LeftEncloser StaticArgs*? *ExprList*? *RightEncloser*
	|	*ParenthesisDelimited*
	|	*LiteralExpr*
	|	*VarOrFnRef*
	|	`self`
	|	*Primary LeftEncloser StaticArgs*? *ExprList*? *RightEncloser*
	|	*Primary*. *Id StaticArgs*? *ParenthesisDelimited*
	|	*Primary*. *Id*
	|	*Primary* ^ *Exponent*
	|	*Primary ExponentOp*
	|	*Primary ArgExpr*
	|	*Primary Primary*
VarOrFnRef	::=	*Id StaticArgs*?

ParenthesisDelimited ::= *Parenthesized*
| *ArgExpr*
| `()`

Exponent ::= *ParenthesisDelimited*
| *LiteralExpr*
| *Id*
| `self`

FlowExpr ::= `exit` *Id*? (`with` *Expr*)?
| *Accumulator StaticArgs*? (`[` *GeneratorClauseList* `]`)? *Expr*
| `BIG` *LeftEncloser* ↦ *StaticArgs*? *RightEncloser Expr*
| `BIG` *LeftEncloser StaticArgs*? *RightEncloser Expr*
| `atomic` *AtomicBack*
| `tryatomic` *AtomicBack*
| `spawn` *Expr*
| `throw` *Expr*

AtomicBack ::= *AssignExpr*
| *OpExpr*
| *DelimitedExpr*

GeneratorClauseList ::= *GeneratorBinding*(, *GeneratorClause*)*

GeneratorBinding ::= *BindIdOrBindIdTuple*← *Expr*

GeneratorClause ::= *GeneratorBinding*
| *Expr*

G.14 Expressions Enclosed by Keywords or Symbols

DelimitedExpr	::=	*ArgExpr*
	|	*Parenthesized*
	|	`object` *ExtendsWhere*? *GoInAnObject*? `end`
	|	*Do*
	|	`label` *Id BlockElems* `end` *Id*
	|	`while` *GeneratorClause Do*
	|	`for` *GeneratorClauseList DoFront* `end`
	|	`if` *GeneratorClause* `then` *BlockElems Elifs*? *Else*? `end`
	|	(`if` *GeneratorClause* `then` *BlockElems Elifs*? *Else* `end` ?)
	|	`case` *Expr* (*Encloser* | *Op*)? `of` *CaseClauses CaseElse*? `end`
	|	`case most` (*Encloser* | *Op*) `of` *CaseClauses* `end`
	|	`typecase` *TypecaseBindings* `of` *TypecaseClauses CaseElse*? `end`
	|	`try` *BlockElems Catch*? (`forbid` *TraitTypes*)? (`finally` *BlockElems*)? `end`
Do	::=	(*DoFront* `also`)* *DoFront* `end`
DoFront	::=	(`at` *Expr*)? `atomic` ? `do` *BlockElems*?
ArgExpr	::=	`(` (*Expr*,)* (*Expr*... ,)? *KeywordExpr* (, *KeywordExpr*)* `)`
	|	`(` (*Expr*,)* *Expr*... `)`
	|	*TupleExpr*
TupleExpr	::=	`(` (*Expr*,)$^+$ *Expr* `)`
KeywordExpr	::=	*BindId* = *Expr*
Parenthesized	::=	`(` *Expr* `)`
Elifs	::=	*Elif*$^+$
Elif	::=	`elif` *GeneratorClause* `then` *BlockElems*
Else	::=	`else` *BlockElems*
CaseClauses	::=	*CaseClause*$^+$
CaseClause	::=	*Expr* ⇒ *BlockElems*
CaseElse	::=	`else` ⇒ *BlockElems*

TypecaseBindings	::=	*TypecaseVars* (= *Expr*)?
TypecaseVars	::=	*BindId*
	|	(*BindId*(, *BindId*)$^+$)
TypecaseClauses	::=	*TypecaseClause*$^+$
TypecaseClause	::=	*TypecaseTypes* ⇒ *BlockElems*
TypecaseTypes	::=	(*TypeList*)
	|	*Type*
Catch	::=	`catch` *BindId CatchClauses*
CatchClauses	::=	*CatchClause*$^+$
CatchClause	::=	*TraitType* ⇒ *BlockElems*
MapExpr	::=	*LeftEncloser StaticArgs*? *EntryList RightEncloser*
Comprehension	::=	`BIG` ? `[` *StaticArgs*? *ArrayComprehensionClause*$^+$ `]`
	|	`BIG` ? *LeftEncloser StaticArgs*? *Entry* | *GeneratorClauseList RightEncloser*
	|	`BIG` ? *LeftEncloser StaticArgs*? *Expr* | *GeneratorClauseList RightEncloser*
Entry	::=	*Expr* ↦ *Expr*
ArrayComprehensionClause	::=	*ArrayComprehensionLeft* | *GeneratorClauseList*
ArrayComprehensionLeft	::=	*IdOrInt* ↦ *Expr*
	|	(*IdOrInt*, *IdOrIntList*) ↦ *Expr*
IdOrInt	::=	*Id*
	|	*IntLiteralExpr*
IdOrIntList	::=	*IdOrInt*(, *IdOrInt*)*
ExprList	::=	*Expr*(, *Expr*)*
EntryList	::=	*Entry*(, *Entry*)*

G.15 Local Declarations

BlockElems	::=	*BlockElem*$^+$
BlockElem	::=	*LocalVarFnDecl*
	|	*Expr*(, *GeneratorClauseList*)?
LocalVarFnDecl	::=	*LocalFnDecl*$^+$
	|	*LocalVarDecl*
LocalFnDecl	::=	*LocalFnMods*? *Id StaticParams*? *ValParam FnHeaderClause* = *Expr*
LocalVarDecl	::=	`var` ? *VarMayTypes* = *Expr*
	|	`var` ? *VarWTypes* := *Expr*
	|	`var` ? *VarWTypes*
	|	*VarWoTypes* = *Expr*
	|	`var` ? *VarWoTypes* : *Type*... *InitVal*?
	|	`var` ? *VarWoTypes* : *TupleType InitVal*?
VarMayTypes	::=	*VarMayType*
	|	(*VarMayType*(, *VarMayType*)$^+$)
VarMayType	::=	*BindId* (*IsType*)?
VarWoTypes	::=	*VarWoType*
	|	(*VarWoType*(, *VarWoType*)$^+$)
VarWoType	::=	*BindId*
	|	*Unpasting*
Unpasting	::=	`[` *UnpastingElems* `]`
UnpastingElems	::=	*UnpastingElem RectSeparator UnpastingElems*
	|	*UnpastingElem*
UnpastingElem	::=	*BindId* (`[` *UnpastingDim* `]`)?
	|	*Unpasting*
UnpastingDim	::=	*ExtentRange* (× *ExtentRange*)$^+$

G.16 Literals

LiteralExpr	::=	( )
	|	*NumericLiteralExpr*
	|	*CharLiteralExpr*
	|	*StringLiteralExpr*
ArrayExpr	::=	[*StaticArgs*? *RectElements*]
RectElements	::=	*Expr MultiDimCons**
MultiDimCons	::=	*RectSeparator Expr*

G.17 Types

Type	::=	*TypeRef*
	|	*TraitType*
	|	*TupleType*
	|	(*Type*)
	|	()
	|	*ArgType* $\rightarrow$ *Type Throws*?
	|	*Type DimType*
TypeRef	::=	*DottedId StaticArgs*?
TraitType	::=	*TypeRef*
	|	*Type* [*ArraySize*?]
	|	*Type* ^ *IntExpr*
	|	*Type* ^ (*ExtentRange* ($\times$ *ExtentRange*)*)
ArgType	::=	((*Type*,)* (*Type*... ,)? *KeywordType*(, *KeywordType*)*)
	|	((*Type*,)* *Type*...)
	|	*TupleType*
KeywordType	::-	*BindId*= *Type*
TupleType	::=	(*Type*, *TypeList*)
TypeList	::=	*Type*(, *Type*)*
ArraySize	::=	*ExtentRange*(, *ExtentRange*)*
ExtentRange	::=	*StaticArg*?# *StaticArg*?
	|	*StaticArg*?: *StaticArg*?
	|	*StaticArg*
DimType	::=	*DottedId*
	|	(*DimType*)
	|	1
	|	*DimPrefixOp DimType*
	|	*DimType DimInfixOp DimType*
	|	*DimType DimPostfixOp*
	|	*DimType* `in` *Expr*
DimPrefixOp	::=	`square` | `cubic` | `inverse`
DimInfixOp	::=	$\cdot$ | / | `per`
DimPostfixOp	::=	`squared` | `cubed`

G.18 Symbols and Operators

EncloserPair ::= (*LeftEncloser* | *Encloser*) · ? (*RightEncloser* | *Encloser*)
ExponentOp ::= `^T` | `^`(*Encloser* | *Op*)
CompoundOp ::= (*Encloser* | *Op*)=
AssignOp ::= := | *CompoundOp*
Accumulator ::= ∑ | ∏ | `BIG` (*Encloser* | *Op*)

G.19 Identifiers

IdOrOpName ::= *Id*
| *OpName*
BindId ::= *Id*
| _
BindIdList ::= *BindId*(, *BindId*)*
BindIdOrBindIdTuple ::= *BindId*
| (*BindId*, *BindIdList*)
SimpleName ::= *Id*
| `opr BIG` ? (*Encloser* | *Op*)
| `opr BIG` ? *EncloserPair*
APIName ::= *Id*(. *Id*)*
QualifiedName ::= *Id*(. *Id*)*

G.20 Spaces and Comments

RectSeparator ::= ;+
| *Whitespace*

Appendix H

Full Grammar for Fortress Implementors

> DRAFT NOTE
> The chapter is not up to date.

In this chapter, we provide a complete definition of the Fortress concrete syntax. It includes unimplemented Fortress language features such as dimensions and units, tests and properties, and where clauses. This syntax has been extracted from the parser-generator source files of an open source partial Fortress reference implementation [13], available at:

`http://projectfortress.sun.com`

The parser of this reference implementation is generated by Rats! [12], which generates "packrat parsers" that memoize intermediate results to ensure linear time performance in the presence of unlimited lookahead and backtracking. Rats! includes production naming, module modifications, and semantic predicates. Actions that generate semantic values are omitted for simplicity. See [12] for more information about the Rats! parser generator.

H.1 Components and APIs

```
File =
     w CompilationUnit w EndOfFile
   / w Exports w ";"? w Imports w ";"? (w Decls w ";"?)? w EndOfFile // Error production
   / (w Imports w ";"?)? (w Decls w ";"?)? w EndOfFile // Error production
   / (w Imports w ";"?)? w Exports w ";"? (w Decls w ";"?)? w EndOfFile
   / (w Imports w ";"?)? w AbsDecls w ";"? w EndOfFile
   / w Imports w ";"? (w AbsDecls w ";"?)? w EndOfFile

CompilationUnit =
     Component
   / Api

Component =
     ("native" w)? "component" w APIName w Exports w ";"? w Imports w ";"? (w Decls w ";"?)?
     w "end" // Error production
   / ("native" w)? "component" w APIName (w Imports w ";"?)? w Decls w ";"? w "end"
     // Error production
   / ("native" w)? "component" w APIName (w Imports w ";"?)? w Exports w ";"? (w Decls w ";"?)?
     w "end" ((s "component")? s APIName)?

Api =
     "native" w "api" w APIName (w Imports w ";"?)? (w AbsDecls w ";"?)? w "end"
```

```
     // Error production
   / "api" w APIName (w Imports w ";"?)? (w AbsDecls w ";"?)? w "end" ((s "api")? s APIName)?

Imports = Import (br Import)*

Import =
     "import" w "api" w AliasedAPINames
   / "import" w ImportedNames

ImportedNames =
     APIName "." w "{" w "..." w "}" (w "except" w SimpleNames)?
   / APIName "." w "{" w AliasedSimpleNameList (w "," w "...")? w "}"
   / Id "." QualifiedName (w "as" w Id)?
   / Id (w "as" w Id)?          // Error production

SimpleNames =
     SimpleName
   / "{" w SimpleNameList w "}"

SimpleNameList = SimpleName (w "," w SimpleName)*

AliasedSimpleName =
     Id (w "as" w Id)?
   / "opr" (w "BIG")? w (Encloser / Op) (w "as" w (Encloser / Op))?
   / "opr" (w "BIG")? w EncloserPair (w "as" w EncloserPair)?

AliasedSimpleNameList = AliasedSimpleName (w "," w AliasedSimpleName)*

AliasedAPINames =
     AliasedAPIName
   / "{" w AliasedAPINameList w "}"

AliasedAPIName = APIName (w "as" w Id)?

AliasedAPINameList = AliasedAPIName (w "," w AliasedAPIName)*

Exports = Export (br Export)*

Export = "export" w APINames

APINames =
     APIName
   / "{" w APINameList w "}"

APINameList = APIName (w "," w APIName)*
```

H.2 Top-level Declarations

```
Decls = Decl (br Decl)*

Decl =
     TraitDecl
   / ObjectDecl
   / VarDecl
   / FnDecl
   / DimUnitDecl
   / TypeAlias
   / TestDecl
   / PropertyDecl
   / ExternalSyntax

AbsDecls = AbsDecl (br AbsDecl)*
```

```
AbsDecl =
     AbsTraitDecl
   / AbsObjectDecl
   / AbsVarDecl
   / AbsFnDecl
   / DimUnitDecl
   / TypeAlias
   / TestDecl
   / PropertyDecl
   / AbsExternalSyntax
```

H.3 Trait and Object Declarations

```
TraitDecl = TraitMods? TraitHeaderFront TraitClauses (w GoInATrait)? w "end" ((s "trait")? s Id)?

TraitHeaderFront =
     "trait" w OpName (w StaticParams)? (w ExtendsWhere)? // Error production
   / "trait" w Id (w StaticParams)? (w ExtendsWhere)?

/* Each trait clause cannot appear more than once. */
TraitClauses = (w TraitClause)*

TraitClause =
     Excludes
   / Comprises
   / Where
   / ExtendsWhere    // Error production

GoInATrait =
     Coercions
   / GoFrontInATrait br Coercions (br GoBackInATrait)?    // Error production
   / GoFrontInATrait br GoBackInATrait br Coercions    // Error production
   / GoBackInATrait br Coercions    // Error production
   / GoBackInATrait br GoFrontInATrait    // Error production
   / (Coercions br)? GoFrontInATrait (br GoBackInATrait)?
   / (Coercions br)? GoBackInATrait

Coercions = Coercion (br Coercion)*

GoFrontInATrait = GoesFrontInATrait (br GoesFrontInATrait)*

GoesFrontInATrait =
     AbsFldDecl
   / GetterSetterDecl
   / PropertyDecl

GoBackInATrait = GoesBackInATrait (br GoesBackInATrait)*

GoesBackInATrait =
      MdDecl
    / PropertyDecl

ObjectDecl = ObjectMods? ObjectHeader (w GoInAnObject)? w "end" ((s "object")? s Id)?

ObjectHeader =
      "object" w OpName (w StaticParams)? (w ObjectValParam)? (w ExtendsWhere)? FnClauses
      // Error production
   /  "object" w Id (w StaticParams)? (w ObjectValParam)? (w ExtendsWhere)? FnClauses

ObjectValParam = "(" (w Params)? w ")"

Varargs :=
     "transient" w VarargsParam
```

```
   / VarargsParam    // Error production

Param :=
     ParamFldMods? PlainParam
   / "var" w "transient" w PlainParamWType
   / ("transient" w)? "var" w PlainParamWType
   / "transient" w PlainParam
   / "var" w "transient" w PlainParam // Error production
   / ("transient" w)? "var" w PlainParam // Error production

PlainParamWType = BindId w IsType

GoInAnObject =
     Coercions
   / GoFrontInAnObject br Coercions (br GoBackInAnObject)?    // Error production
   / GoFrontInAnObject br GoBackInAnObject br Coercions    // Error production
   / GoBackInAnObject br Coercions    // Error production
   / GoBackInAnObject br GoFrontInAnObject    // Error production
   / (Coercions br)? GoFrontInAnObject (br GoBackInAnObject)?
   / (Coercions br)? GoBackInAnObject

GoFrontInAnObject = GoesFrontInAnObject (br GoesFrontInAnObject)*

GoesFrontInAnObject =
     FldDecl
   / GetterSetterDef
   / PropertyDecl

GoBackInAnObject = GoesBackInAnObject (br GoesBackInAnObject)*

GoesBackInAnObject =
      MdDef
    / PropertyDecl

AbsTraitDecl =
      AbsTraitMods? TraitHeaderFront AbsTraitClauses (w AbsGoInATrait)? w "end" ((s "trait")? s Id)?

/* Each trait clause cannot appear more than once. */
AbsTraitClauses = (w AbsTraitClause)*

AbsTraitClause =
     Excludes
   / AbsComprises
   / Where
   / ExtendsWhere    // Error production

AbsGoInATrait =
     AbsCoercions
   / AbsGoFrontInATrait br AbsCoercions (br AbsGoBackInATrait)?    // Error production
   / AbsGoFrontInATrait br AbsGoBackInATrait br AbsCoercions    // Error production
   / AbsGoBackInATrait br AbsCoercions    // Error production
   / AbsGoBackInATrait br AbsGoFrontInATrait    // Error production
   / (AbsCoercions br)? AbsGoFrontInATrait (br AbsGoBackInATrait)?
   / (AbsCoercions br)? AbsGoBackInATrait

AbsCoercions = AbsCoercion (br AbsCoercion)*

AbsGoFrontInATrait = AbsGoesFrontInATrait (br AbsGoesFrontInATrait)*

AbsGoesFrontInATrait =
     ApiFldDecl
   / AbsGetterSetterDecl
   / PropertyDecl

AbsGoBackInATrait = AbsGoesBackInATrait (br AbsGoesBackInATrait)*
```

```
AbsGoesBackInATrait =
      AbsMdDecl
    / PropertyDecl

AbsObjectDecl = AbsObjectMods? ObjectHeader (w AbsGoInAnObject)? w "end" ((s "object")? s Id)?

AbsGoInAnObject =
     AbsCoercions
   / AbsGoFrontInAnObject br AbsCoercions (br AbsGoBackInAnObject)?    // Error production
   / AbsGoFrontInAnObject br AbsGoBackInAnObject br AbsCoercions    // Error production
   / AbsGoBackInAnObject br AbsCoercions    // Error production
   / AbsGoBackInAnObject br AbsGoFrontInAnObject    // Error production
   / (AbsCoercions br)? AbsGoFrontInAnObject (br AbsGoBackInAnObject)?
   / (AbsCoercions br)? AbsGoBackInAnObject

AbsGoFrontInAnObject = AbsGoesFrontInAnObject (br AbsGoesFrontInAnObject)*

AbsGoesFrontInAnObject =
     ApiFldDecl
   / AbsGetterSetterDecl
   / PropertyDecl

AbsGoBackInAnObject = AbsGoesBackInAnObject (br AbsGoesBackInAnObject)*

AbsGoesBackInAnObject =
      AbsMdDecl
    / PropertyDecl
```

H.4 Variable Declarations

```
VarDecl =
     VarMods? NoNewlineVarWTypes w InitVal
   / VarImmutableMods? BindIdOrBindIdTuple w "=" w NoNewlineExpr
   / VarMods? BindIdOrBindIdTuple w ":" w Type w "..." w InitVal
   / VarMods? BindIdOrBindIdTuple w ":" w TupleType w InitVal
   / "var" w BindIdOrBindIdTuple w "=" w NoNewlineExpr // Error production

/* Each modifier cannot appear more than once. */
VarMods = (VarMod w)+

/* Each modifier cannot appear more than once. */
VarImmutableMods = (VarImmutableMod w)+

VarMod =
     AbsVarMod
   / "private"

VarImmutableMod =
     AbsVarImmutableMod
   / "private"

VarWTypes =
     VarWType
   / "(" w VarWType (w "," w VarWType)+ w ")"

VarWType = BindId w IsType

InitVal = ("=" / ":=") w NoNewlineExpr

AbsVarDecl =
     AbsVarMods? VarWTypes
   / AbsVarMods? BindIdOrBindIdTuple w ":" w Type w "..."
```

```
   / AbsVarMods? BindIdOrBindIdTuple w ":" w TupleType

/* Each modifier cannot appear more than once. */
AbsVarMods = (AbsVarMod w)+

AbsVarMod =
     "var"
   / AbsVarImmutableMod

AbsVarImmutableMod = "test"
```

H.5 Function Declarations

```
FnDecl =
     FnMods? FnHeaderFront FnHeaderClause w "=" w NoNewlineExpr
   / FnMods? AbsFnHeaderFront FnHeaderClause
   / FnSig

FnSig = SimpleName w ":" w NoNewlineType

AbsFnDecl =
     AbsFnMods? AbsFnHeaderFront FnHeaderClause
   / FnSig

FnHeaderFront =
     NamedFnHeaderFront
   / OpHeaderFront

AbsFnHeaderFront =
     AbsNamedFnHeaderFront
   / AbsOpHeaderFront

NamedFnHeaderFront = Id (w StaticParams)? w ValParam

AbsNamedFnHeaderFront = Id (w StaticParams)? w AbsValParam
```

H.6 Dimension, Unit, Type Alias, Test, Property, and External Syntax Declarations

```
DimUnitDecl =
     "dim" w OpName (w "=" w NoNewlineType)? s ("unit" / "SI_unit") w Id (wr Id)*
       (w "=" w NoNewlineExpr)? // Error production
   / "dim" w Id (w "=" w NoNewlineType)? s ("unit" / "SI_unit") w OpName (wr Id)*
       (w "=" w NoNewlineExpr)? // Error production
   / "dim" w Id (w "=" w NoNewlineType)? s ("unit" / "SI_unit") w Id (wr Id)*
       (w "=" w NoNewlineExpr)?
   / "dim" w OpName (w "=" w NoNewlineType)? (w "default" w Id)? // Error production
   / "dim" w Id (w "=" w NoNewlineType)? w "default" w OpName // Error production
   / "dim" w Id (w "=" w NoNewlineType)? (w "default" w Id)?
   / ("unit" / "SI_unit") w OpName (wr Id)* (w ":" w NoNewlineType)? (w "=" w NoNewlineExpr)?
     // Error production
   / ("unit" / "SI_unit") w Id (wr Id)* (w ":" w NoNewlineType)? (w "=" w NoNewlineExpr)?

TypeAlias =
     "type" w Id (w StaticParams)? w "=" w NoNewlineType
   / "type" w OpName (w StaticParams)? w "=" w NoNewlineType // Error production

TestDecl =
     "test" w Id w "[" w GeneratorClauseList w "]" w "=" w NoNewlineExpr
```

```
   / "test" w OpName w "[" w GeneratorClauseList w "]" w "=" w NoNewlineExpr // Error production

PropertyDecl =
     "property" (w Id w "=")? (w "FORALL" w ValParam)? w NoNewlineExpr
   / "property" (w OpName w "=")? (w "FORALL" w ValParam)? w NoNewlineExpr // Error production

ExternalSyntax = "syntax" w OpenExpander w Id w CloseExpander w "=" w NoNewlineExpr

OpenExpander =
     Id
   / LeftEncloser
   / Encloser

CloseExpander =
     Id
   / RightEncloser
   / Encloser
   / "end"

AbsExternalSyntax = "syntax" w OpenExpander w Id w CloseExpander
```

H.7 Headers without Newlines

```
ExtendsWhere =
     extends w TraitTypeWhere
   / extends w opencurly w TraitTypeWhere (w comma w TraitTypeWhere)*
     w closecurly

TraitTypeWhere =
     TraitTypeWithError (w Where)?

Extends = extends w TraitTypes

Excludes = excludes w TraitTypes

TraitTypes =
     TraitTypeWithError
   / opencurly w TraitTypeWithError (w comma w TraitTypeWithError)* w closecurly

Comprises =
     comprises w ComprisingTypes
   / comprises w opencurly (w AbsComprisingTypeList)? w closecurly

AbsComprisingTypeList =
     ellipses
   / ComprisingTypeList (w comma w ellipses)?

ComprisingTypes =
     ComprisingTypeWithError
   / opencurly w ComprisingTypeList w closecurly

ComprisingTypeList =
     ComprisingTypeWithError (w comma w ComprisingTypeWithError)*

ComprisingTypeWithError =
     TraitTypeWithError
   / Id

Where =
     where w opendoublesquare w WhereBinding (w comma w WhereBinding)* w
     closedoublesquare (w opencurly w WhereConstraintList w closecurly)?
   / where w opencurly w WhereConstraintList w closecurly
```

```
WhereBinding =
     nat w IdOrOpName
   / int w IdOrOpName
   / bool w IdOrOpName
   / unit w IdOrOpName
   / IdOrOpName (w Extends)?
   / <ErrorProduction> opendoublesquare

FnHeaderClause = (w NoNewlineIsType)? FnClauses

FnClauses = FnClause*
FnClause =
     w Throws
   / w Where
   / w Requires
   / w Ensures
   / w Invariant
   / w ExtendsWhere

Throws =
     throws w opencurly w closecurly
   / throws w TraitTypes

Mods = (Mod w)+

Mod =
     abstract
   / atomic
   / getter
   / hidden
   / io
   / override
   / private
   / settable
   / setter
   / test
   / value
   / var
   / wrapped

StaticParams =
     opendoublesquare w StaticParam (w comma w StaticParam)* w
     closedoublesquare

StaticParam =
     nat w IdOrOpName
   / int w IdOrOpName
   / bool w IdOrOpName
   / dim w IdOrOpName
   / unit w IdOrOpName (w colon w NoNewlineType)? (w absorbs w unit)?
   / opr w Op
   / IdOrOpName w Extends (w absorbs w unit)?
   / Id (w Extends)? (w absorbs w unit)?
   / <ErrorProduction>
     opendoublesquare
```

H.8 Headers Maybe with Newlines

```
IsType = colon w Type

WhereConstraintList =
     WhereConstraint (w comma w WhereConstraint)*
```

```
WhereConstraint =
     IdOrOpName w Extends
   / TypeAlias
   / Type w coerces w Type
   / Type w widens w Type
   / UnitConstraint
   / QualifiedName w equals w QualifiedName
   / IntConstraint
   / BoolExpr
   / Type w widens w or w coerces w Type

UnitConstraint =
     "dimensionless" w equals w IdOrOpName
   / IdOrOpName w equals w "dimensionless"

IntConstraint =
     IntExpr w lessthanequal w IntExpr
   / IntExpr w lessthan w IntExpr
   / IntExpr w greaterthanequal w IntExpr
   / IntExpr w greaterthan w IntExpr
   / IntExpr w equals w IntExpr

IntVal =
     IntLiteralExpr
   / QualifiedName

IntExpr =
     SumExpr IntExprTail*
IntExprTail =
     SumIntExpr
   / MinusIntExpr
SumIntExpr =
     w plus w SumExpr
MinusIntExpr =
     w minus w SumExpr

SumExpr =
     MulExpr SumExprTail*
SumExprTail =
     ProductIntExpr
ProductIntExpr =
     (w DOT w / sr) MulExpr

MulExpr =
     IntBase caret IntVal
   / IntBase

IntBase =
     IntVal
   / openparen w IntExpr w closeparen

BoolExpr = OpBool

OpBool =
     BoolPrimary
   / BoolPrefix

BoolPrimary =
     BoolPrimaryFront BoolPrefix
   / BoolPrimaryFront wr BoolPrimary
   / BoolPrimaryFront wr Op wr OpBool
   / BoolPrimaryFront

BoolPrefix =
     Op OpBool
```

```
   / Op wr OpBool

BoolPrimaryFront =
     "true"
   / "false"
   / QualifiedName
   / openparen w OpBool w closeparen

UnitVal =
     "dimensionless"
   / QualifiedName

UnitExpr =
     MulDivUnit UnitExprTail*
UnitExprTail =
     ProductUnitExpr
   / QuotientUnitExpr
ProductUnitExpr =
     (w DOT w / sr) MulDivUnit
QuotientUnitExpr =
     (slash / per) MulDivUnit

MulDivUnit =
     UnitBase caret UnitVal
   / UnitBase

UnitBase =
     UnitVal
   / openparen w UnitExpr w closeparen

Requires = requires w opencurly (w ExprList)? w closecurly

Ensures =
     ensures w opencurly (w EnsuresClauseList)? w closecurly

EnsuresClauseList =
     EnsuresClause (w comma w EnsuresClause)*

EnsuresClause = Expr (w provided w Expr)?

Invariant =
     invariant w opencurly (w ExprList)? w closecurly

StaticArgs =
    opendoublesquare w StaticArgList w closedoublesquare

StaticArgList = StaticArg (w comma w StaticArg)*

StaticArg =
     Op &(w comma / w closedoublesquare)
   / !(QualifiedName (w closedoublesquare / w closesquare / w comma /
                      w opendoublesquare / w opensquare / w rightarrow /
                      w OR / w AND / w IMPLIES / w equals) /
       "Unity" / "dimensionless" / "true" / "false")
     IntExpr
   / !(QualifiedName (w closedoublesquare / w closesquare / w comma /
                      w opendoublesquare / w opensquare / w rightarrow) /
       "Unity" / "dimensionless")
     BoolExpr
   / "true"
   / "false"
   / !(QualifiedName (w DOT / w slash / w per / w DimPostfixOp) / "dimensionless")
     Type
   / UnitExpr
```

H.9 Parameters

```
ValParam =
     BindId
   / openparen (w Params)? w closeparen

AbsValParam =
     openparen (w AbsParams)? w closeparen
   / Type

Parameter = Keyword / Varargs / Param
Params =
     Parameter (w comma w Parameter)+
   / Parameter (w commaOrNot w Parameter)+
   / Parameter

AbsParameter = Keyword / Varargs / AbsParam
AbsParams =
     AbsParameter (w comma w AbsParameter)+
   / AbsParameter (w commaOrNot w AbsParameter)+
   / AbsParameter

VarargsParam = BindId w colon w Type w ellipses
Varargs = VarargsParam

Keyword = Param w equals w Expr

PlainParam =
     BindId (w IsType)?
Param = PlainParam

AbsPlainParam =
     BindId w IsType
   / Type
AbsParam = AbsPlainParam

LeftOp =
     LeftEncloser w mapsto
   / LeftEncloser
   / Encloser
SingleOp =
     Op
   / caret
   / Encloser
   / SUM
   / PROD
OpHeaderFront =
     <Enclosing> opr (w BIG)? w opLeftOp
     (w StaticParams)? (w Params)? w (RightEncloser / Encloser)
   / opr w ValParam w (Op / ExponentOp / caret) (w StaticParams)?
   / opr (w BIG)? w SingleOp (w StaticParams)? w ValParam

AbsOpHeaderFront =
     <Enclosing> opr (w BIG)? w opLeftOp
     (w StaticParams)? (w AbsParams)? w (RightEncloser / Encloser)
   / opr w AbsValParam w (Op / ExponentOp / caret) (w StaticParams)?
   / opr (w BIG)? w SingleOp (w StaticParams)? w AbsValParam
```

H.10 Method Declarations

```
MdDecl =
     MdDef
```

```
   / Mods? AbsMdHeaderFront FnHeaderClause

MdDef =
     Mods? MdHeaderFront FnHeaderClause w equals w NoNewlineExpr

MdHeaderFront =
     NamedMdHeaderFront
   / OpHeaderFront

AbsMdHeaderFront =
     AbsNamedMdHeaderFront
   / AbsOpHeaderFront

NamedMdHeaderFront =
     Id (w StaticParams)? w ValParam

AbsNamedMdHeaderFront =
     Id (w StaticParams)? w AbsValParam

Coercion =
     coerce (w StaticParams)? w openparen w BindId (w IsType)? w closeparen
     FnClauses (w widens)? (w equals w NoNewlineExpr)?
   / coerce (w StaticParams)? w ValParam (w IsType)?
     FnClauses (w widens)? (w equals w NoNewlineExpr)?
```

H.11 Method Parameters

```
ValParam := openparen (w Params)? w closeparen

AbsValParam := openparen (w AbsParams)? w closeparen

Param :=
     PlainParam
   / self

AbsParam :=
     self
   / AbsPlainParam

OpHeaderFront :=
     <Enclosing> opr (w BIG)? w opLeftOp
     (w StaticParams)? (w Params)? w (RightEncloser / Encloser)
     (w colonequals w openparen w SubscriptAssignParam w closeparen)?
   / ...

AbsOpHeaderFront :=
     <Enclosing> opr (w BIG)? w opLeftOp
     (w StaticParams)? (w AbsParams)? w (RightEncloser / Encloser)
     (w colonequals w openparen w AbsSubscriptAssignParam w closeparen)?
   / ...

SubscriptAssignParam =
     Varargs
   / Param

AbsSubscriptAssignParam =
     Varargs
   / AbsParam
```

H.12 Expressions

```
Expr =
     ExprFront ExprTail*

ExprFront =
     <AssignExpr> AssignExpr
   / OpExpr
   / DelimitedExpr
   / <Flow> FlowExpr
   / <Fn1> fn w BindId (w Throws)? w match w Expr
   / <Fn2> fn w openparen (w Params)? w closeparen (w IsType)? (w Throws)? w match w Expr
   / <ErrorProduction1> fn w ValParam (w IsType)? (w Throws)? w match w Expr
   / <ErrorProduction2> fn w ValParam (w IsType)? (w Throws)? w Expr

ExprTail =
     <As> As
   / <Asif> AsIf

As =
     w typed w Type

AsIf =
     w asif w Type

AssignExpr = AssignLefts w AssignOp w Expr

AssignLefts =
     openparen w AssignLeft (w comma w AssignLeft)* w closeparen
   / AssignLeft

AssignLeft =
     PrimaryFront AssignLeftTail+
   / QualifiedName

AssignLeftTail =
     SubscriptAssign
   / FieldSelectionAssign

SubscriptAssign =
     LeftEncloser (w StaticArgs)? (w ExprList)? w RightEncloser

FieldSelectionAssign =
    dot (Reserved / Id)

OpExpr =
     <FIRST> OpExprNoEnc
   / OpExprLeftEncloser
   / Encloser

OpExprNoEnc =
     <FIRST> OpExprPrimary
   / OpExprPrefix
   / caret
   / Op

TightInfixRight =
     Encloser OpExprPrimary
   / Encloser OpExprPrefix
   / <Primary>   Encloser wr OpExprPrimary
   / <Loose>     Encloser wr LooseInfix
   / <LeftLoose> Encloser wr LeftLooseInfix
   / Encloser

LeftLooseInfix =
```

```
     OpExprLeftEncloser
   / <Primary> Encloser wr OpExprPrimary
   / <Prefix>  Encloser wr OpExprPrefix
   / <Left>    Encloser wr OpExprLeftEncloser

OpExprLeftEncloser =
     Encloser OpExprNoEnc

OpExprPrimary =
     Primary TightInfixPostfix
   / Primary TightInfixRight
   / <Primary>   Primary wr OpExprPrimary
   / <Loose>     Primary wr LooseInfix
   / <LeftLoose> Primary wr LeftLooseInfix
   / Primary

OpExprPrefix =
     Op OpExprPrimary
   / Op OpExprPrefix
   / Op OpExprLeftEncloser
   / <Primary> Op wr OpExprPrimary
   / <Prefix>  Op wr OpExprPrefix
   / <Left>    Op wr OpExprLeftEncloser
   / caret w OpExprPrimary
   / caret w OpExprPrefix
   / caret w OpExprLeftEncloser

TightInfixPostfix =
     Op OpExprPrimary
   / Op OpExprPrefix
   / Op OpExprLeftEncloser
   / <Primary> Op wr OpExprPrimary
   / <Prefix>  Op wr OpExprPrefix
   / <Left>    Op wr OpExprLeftEncloser
   / caret wr OpExprPrimary
   / caret wr OpExprPrefix
   / caret wr OpExprLeftEncloser
   / caret
   / Op

LooseInfix =
     Op wr OpExprPrimary
   / Op wr OpExprPrefix
   / <Left> Op wr OpExprLeftEncloser

Primary =
    PrimaryItem (w comma w PrimaryItem)+
    &(Encloser (wr / br / w comma / w RightEncloser))
   / PrimaryItem

PrimaryItem =
     (LeftAssociatedPrimary / MathPrimary)

LeftAssociatedPrimary =
     DottedIdChain w StaticArgs ParenthesisDelimited
     ParenthesisDelimitedLeft* Selector*
   / DottedIdChain SubscriptingLeft+ ParenthesisDelimitedLeft*
     Selector*
   / DottedIdChain ParenthesisDelimited ParenthesisDelimitedLeft*
     Selector*
   / DottedIdChain Selector*
   / PrimaryFront SubscriptingLeft* ParenthesisDelimitedLeft*
     Selector+

DottedIdChain =
```

```
    (Reserved / Id) (dot w (Reserved / Id))+

MathPrimary =
    PrimaryFront MathItem*

PrimaryFront =
     ArrayExpr
   / MapExpr
   / Comprehension
   / LeftEncloser (w StaticArgs)? (w ExprList)? w RightEncloser
   / ParenthesisDelimited
   / LiteralExpr
   / VarOrFnRef
   / self

VarOrFnRef =
     Id w StaticArgs
   / Id

SubscriptingLeft =
     (opensquare / opencurly) (w StaticArgs)? (w ExprList)? w
     (closesquare / closecurly)
   / LeftEncloser (w StaticArgs)? (w ExprList)? w RightEncloser

ParenthesisDelimitedLeft =
    ParenthesisDelimited

ParenthesisDelimited =
     Parenthesized
   / ArgExpr
   / openparen w closeparen

Selector =
     MethodInvocationSelector
   / FieldSelectionSelector

MethodInvocationSelector = /* REVERSE ORDER */
     dot w (Reserved / Id) (w StaticArgs)? ParenthesisDelimited
     ParenthesisDelimitedLeft*

FieldSelectionSelector =
     dot w (Reserved / Id) SubscriptingLeft* ParenthesisDelimitedLeft*

MathItem =
     Subscripting
   / Exponentiation
   / ParenthesisDelimited
   / LiteralExpr
   / VarOrFnRef
   / self

Subscripting =
     (opensquare / opencurly) (w StaticArgs)? (w ExprList)? w
     (closesquare / closecurly)
   / LeftEncloser (w StaticArgs)? (w ExprList)? w RightEncloser

Exponentiation =
     caret hasW Exponent
   / ExponentOp

Exponent =
     ParenthesisDelimited
   / LiteralExpr
   / Id
   / self
```

```
FlowExpr =
     exit (w Id)? (w with w Expr)?
   / Accumulator (w StaticArgs)?
     (w opensquare w GeneratorClauseList w closesquare)? w Expr
   / BIG w LeftEncloser w mapsto (w StaticArgs)? w RightEncloser w Expr
   / BIG w LeftEncloser (w StaticArgs)? w RightEncloser w Expr
   / atomic w AtomicBack
   / tryatomic w AtomicBack
   / spawn w Expr
   / throw w Expr

AtomicBack =
     AssignExpr
   / OpExpr
   / DelimitedExpr

GeneratorClauseList =
     GeneratorBinding (w comma w GeneratorClause)*

GeneratorBinding =
     BindIdOrBindIdTuple w leftarrow w Expr

GeneratorClause =
     GeneratorBinding
   / Expr
```

H.13 Expressions Enclosed by Keywords or Symbols

```
DelimitedExpr =
     ArgExpr
   / Parenthesized
   / object (w ExtendsWhere)? (w GoInAnObject)? w end
   / Do
   / label w IdOrOpName w BlockElems w end w IdOrOpName
   / while w GeneratorClause w Do
   / for w GeneratorClauseList w DoFront w end
   / if w GeneratorClause w then w BlockElems (w Elifs)? (w Else)? w end
   / if w GeneratorClause w then w BlockElems (w Elifs)? w Else
     (w end)? &(w closeparen)
   / openparen w if w GeneratorClause w then w BlockElems (w Elifs)?
     w Else (w end)? w closeparen
   / case w Expr (w (Encloser / Op))? w (of / do / in) w CaseClauses
     (br CaseElse)? w end
   / case w most w (Encloser / Op) w (of / do / in) w CaseClauses w end
   / typecase w TypecaseBindings w (of / do / in) w TypecaseClauses
     (br CaseElse)? w end
   / <ErrorProduction>
     typecase w self w (of / do / in) w TypecaseClauses (br CaseElse)? w end
   / <TRY> try w BlockElems (w Catch)? (w forbid w TraitTypes)?
     (w finally w BlockElems)? w end

Do = (DoFront w also w)* DoFront w end

DoFront =
     (at w Expr w)? (atomic w)? do (w BlockElems)?

ArgExpr =
     openparen w (Expr w comma w)* (Expr w ellipses w comma w)?
     KeywordExpr (w comma w KeywordExpr)* w closeparen
   / openparen w (Expr w comma w)* Expr w ellipses w closeparen
   / TupleExpr
```

```
TupleExpr =
     openparen w (Expr w comma w)+ Expr w closeparen

KeywordExpr = BindId w equals w Expr

Parenthesized =
     openparen w Expr w closeparen

Elifs = Elif (w Elif)*

Elif = elif w GeneratorClause w then w BlockElems

Else = else w BlockElems

CaseClauses = CaseClause (br CaseClause)*

CaseElse = else w match w BlockElems

TypecaseBindings =
     BindIdOrBindIdTuple w equals w Expr &(w of)
   / BindIdOrBindIdTuple &(w of)
   / <ErrorProduction> Expr

TypecaseClauses =
     TypecaseClause (br TypecaseClause)*

TypecaseClause =
     TypecaseTypes w match w BlockElems

TypecaseTypes =
     openparen w TypeList w closeparen
   / Type

Catch = catch w BindId w CatchClauses

CatchClauses = CatchClause (br CatchClause)*

CatchClause = TraitType w match w BlockElems

MapExpr =
     LeftEncloser (w StaticArgs)? w EntryList w RightEncloser

Comprehension =
     (BIG w)? opensquare (w StaticArgs)? w ArrayComprehensionClause
     (br ArrayComprehensionClause)* w closesquare
   / (BIG w)? LeftEncloser (w StaticArgs)? w Entry wr bar wr
     GeneratorClauseList w RightEncloser
   / (BIG w)? LeftEncloser (w StaticArgs)? w Expr wr bar wr
     GeneratorClauseList w RightEncloser

mapstoOp =
     !("|->" w Expr (w mapsto / wr bar / w closecurly / w comma)) "|->"

leftarrowOp = !("<-" w Expr (w leftarrow / w comma)) "<-"

Entry = Expr w mapsto w Expr

ArrayComprehensionLeft =
     IdOrInt w mapsto w Expr
   / openparen w IdOrInt w comma w IdOrIntList w closeparen w mapsto w
     Expr
   / openparen w IdOrInt w IdOrIntList w closeparen w mapsto w
     Expr

IdOrInt =
```

```
     Id
   / IntLiteralExpr

IdOrIntList =
     IdOrInt (w comma w IdOrInt)+
   / IdOrInt (w commaOrNot w IdOrInt)+
   / IdOrInt

ExprList = Expr (w comma w Expr)*

EntryList = Entry (w comma w Entry)*
```

H.14 Local Declarations

```
BlockElems =
     BlockElemCollection

BlockElemCollection =
     BlockElem br BlockElemCollection
   / BlockElem w semicolon
     &(w elif / w also / w end / w catch / w forbid / w finally / w closeparen)
   / BlockElem
     &(w elif / w also / w end / w catch / w forbid / w finally / w closeparen)
   / BlockElem &(w Else / br CaseClause / br TypecaseTypes / br CaseElse)
   / BlockElem w semicolon  &(w Else)

BlockElem =
     LocalVarFnDecl
   / NoNewlineExpr (s comma w NoNewlineGeneratorClauseList)?

LocalVarFnDecl =
     LocalFnDecl (br LocalFnDecl)*
   / LocalVarDecl

LocalFnDecl =
     Mods? Id (w StaticParams)? w ValParam FnHeaderClause w equals
     w NoNewlineExpr

ValParam =
     openparen (w Params)? w closeparen
   / BindId

Params =
     <ErrorProduction1>
     (Param w comma w)+ (Keyword w comma w)+ Varargs
   / <ErrorProduction2>
     Varargs w comma w (Keyword w comma w)+ Param (w comma w Param)*
   / <ErrorProduction3>
     Varargs (w comma w Param)+ (w comma w Keyword)*
   / (Param w comma w)* (Varargs w comma w)? Keyword
       (w comma w Keyword)*
   / (Param w comma w)* Varargs
   / Param (w comma w Param)*

Param =
     BindId (s colon s NoNewlineType)?

Varargs = BindId s colon s NoNewlineType s ellipses

Keyword = Param s equals w NoNewlineExpr

LocalVarDecl =
     Mods? VarMayTypes s equals w NoNewlineExpr
```

```
   / Mods? NoNewlineVarWTypes s colonequals w NoNewlineExpr
   / Mods? NoNewlineVarWTypes
   / VarWoTypes s equals w NoNewlineExpr
   / Mods? VarWoTypes s colon s NoNewlineType s ellipses (s InitVal)?
   / Mods? VarWoTypes s colon s NoNewlineTupleType (s InitVal)?

VarMayTypes =
     VarMayType
   / openparen w VarMayType (w comma w VarMayType)+ w closeparen

VarMayType = BindId (s colon s NoNewlineType)?

VarWoTypes =
     VarWoType
   / openparen w VarWoType (w comma w VarWoType)+ w closeparen

VarWoType =
     BindId
   / Unpasting

Unpasting =
     opensquare w UnpastingElems w closesquare

UnpastingElems =
     UnpastingElem RectSeparator UnpastingElems
   / UnpastingElem

UnpastingElem =
     BindId (opensquare w UnpastingDim w closesquare)?
   / Unpasting

UnpastingDim =
     NoNewlineExtentRange (w cross w NoNewlineExtentRange)+

CaseClause = NoNewlineExpr w match w BlockElems
```

H.15 Literals

```
LiteralExpr =
     <VOID> VoidLiteralExpr
   / <NUMERICAL> NumericLiteralExpr
   / <CHAR> CharLiteralExpr
   / <STRING> StringLiteralExpr

VoidLiteralExpr =
     <FIRST> openparen w closeparen

IntLiteralExpr =
     <FIRST> a1:NumericLiteralExpr &{ (a1 instanceof IntLiteralExpr) }

NumericLiteralExpr =
     NumericLiteralWithRadix
   / a1:NumericWord RestNumericWord* &{ Character.isDigit(a1.charAt(0)) }

NumericLiteralWithRadix =
     NumericWord RestNumericWord* "_" RadixSpecifier

NumericWord =
     NumericCharacter+
NumericCharacter = [0-9a-zA-Z]

RestNumericWord =
    NumericSeparator NumericWord
```

```
NumericSeparator = NumericSpace / "."
NumericSpace = "’" / "\u202f"

RadixSpecifier =
     a1:DigitString !([a-zA-Z]) &{ NodeUtil.validRadix(writer, createSpan(yyStart,yyCount), a1) }
   / RadixNames

DigitString = [0-9]+

RadixNames =
     "SIXTEEN"
   / "FIFTEEN"
   / "FOURTEEN"
   / "THIRTEEN"
   / "TWELVE"
   / "ELEVEN"
   / "TEN"
   / "NINE"
   / "EIGHT"
   / "SEVEN"
   / "SIX"
   / "FIVE"
   / "FOUR"
   / "THREE"
   / "TWO"

CharLiteralExpr =
     <FIRST> "’" CharLiteralContent "’"
   / "`" CharLiteralContent "’"
   / "\u2018" CharLiteralContent "\u2019"
   / <ErrorProduction1> "’" CharLiteralContent "\u2019"
   / <ErrorProduction2> "`" CharLiteralContent "\u2019"
   / <ErrorProduction3> "\u2018" CharLiteralContent "’"

StringLiteralExpr =
     <FIRST> ["] StringLiteralContent* ["]
   / "\u201c" StringLiteralContent* "\u201d"
   / <ErrorProduction1> ["] StringLiteralContent* "\u201d"
   / <ErrorProduction2> "\u201c" StringLiteralContent* ["]
StringLiteralContent =
     EscapeSequence
   / (!InvalidStringLiteralContent _)

InvalidStringLiteralContent =
     ["\u201c\u201d]
   / [\\]
   / [\n\f\r\u0009\u000b\u001c\u001d\u001e\u001f\u2028\u2029]
   / c:_ &{ Character.getType(c) == Character.CONTROL }

EscapeSequence =
     ’\\’ [btnfr"\\]
   / ’\\’ [\u201c]
   / ’\\’ [\u201d]
```

H.16 Literals within Array Expressions

```
ArrayExpr =
     opensquare (w StaticArgs)? w NoSpaceExpr MultiDimCons* w
     closesquare

MultiDimCons =
     RectSeparator NoSpaceExpr
```

H.17 Expressions without Newlines

```
NoNewlineExpr = Expr
Expr := ...
ExprFront :=
    <AssignExpr> NoNewlineAssignExpr
  / ...
NoNewlineAssignExpr =
    AssignLefts s AssignOp w NoNewlineExpr

ExprTail :=
     <As> NoNewlineAs
   / <AsIf> NoNewlineAsIf

NoNewlineAs =
     s typed w NoNewlineType

NoNewlineAsIf =
     s asif w NoNewlineType

TightInfixRight := ...
   / <Primary>   Encloser sr OpExprPrimary
   / <Loose>     Encloser sr LooseInfix
   / <LeftLoose> Encloser sr LeftLooseInfix

LeftLooseInfix := ...
   / <Primary> Encloser sr OpExprPrimary
   / <Prefix>  Encloser sr OpExprPrefix
   / <Left>    Encloser sr OpExprLeftEncloser

OpExprPrimary := ...
   / <Primary>   Primary sr OpExprPrimary
   / <Loose>     Primary sr LooseInfix
   / <LeftLoose> Primary sr LeftLooseInfix

OpExprPrefix := ...
   / <Primary> Op sr OpExprPrimary
   / <Prefix>  Op sr OpExprPrefix
   / <Left>    Op sr OpExprLeftEncloser

TightInfixPostfix := ...
   / <Primary> Op sr OpExprPrimary
   / <Prefix>  Op sr OpExprPrefix
   / <Left>    Op sr OpExprLeftEncloser

LooseInfix := ...
   / <Left> Op sr OpExprLeftEncloser

GeneratorClauseList :=
     GeneratorBinding (s comma w GeneratorClause)*
NoNewlineGeneratorClauseList = GeneratorClauseList

NoNewlineVarWTypes =
     NoNewlineVarWType
   / openparen w NoNewlineVarWType (w comma w NoNewlineVarWType)+ w
     closeparen

 NoNewlineVarWType = BindId s NoNewlineIsType

NoNewlineIsType = colon w NoNewlineType
```

H.18 Expressions within Array Expressions

```
ExprFront -= <Flow> , <Fn1> , <Fn2>
NoSpaceExpr = ExprFront

OpExpr :=
     OpExprNoEnc
   / OpExprLeftEncloser
   / Encloser

TightInfixRight -= <Primary>, <Loose>, <LeftLoose>

OpExprPrimary -= <Primary>, <Loose>, <LeftLoose>

OpExprPrefix -= <Primary>, <Prefix>, <Left>

TightInfixPostfix -= <Primary>, <Prefix>, <Left>
```

H.19 Types

```
Type =
     !(one) (io w)? TypePrimary (w in w Expr)?

OpType =
     TypePrimary
   / TypePrefix

TypePrimary =
     TypePrimaryFront TightInfixPostfix
   / <LooseJuxt> TypePrimaryFront wr TypePrimary
   / <LooseInfix> TypePrimaryFront wr LooseInfix
   / TypePrimaryFront

TypePrefix =
     DimPrefixOp TypePrimary
   / DimPrefixOp TypePrefix
   / <Prefix> DimPrefixOp wr TypePrimary
   / <PrePrefix> DimPrefixOp wr TypePrefix

TightInfixPostfix =
     <Arrow> TypeInfixOp TypePrimary (w Throws)?
   / <ArrowPrefix> TypeInfixOp TypePrefix (w Throws)?
   / DimInfixOp TypePrimary
   / DimInfixOp TypePrefix
   / <Postfix> DimPostfixOp wr TypePrimary
   / <PostPrefix> DimPostfixOp wr TypePrefix
   / DimPostfixOp

LooseInfix =
     <Arrow> TypeInfixOp wr TypePrimary (w Throws)?
   / <ArrowPrefix> TypeInfixOp wr TypePrefix (w Throws)?
   / <Infix> DimInfixOp wr TypePrimary
   / <InPrefix> DimInfixOp wr TypePrefix

TypePrimaryFront =
    TypeFront TypeTail+
  / TypeFront

TypeFront =
     ParenthesizedType
   / Domain
   / TupleType
```

```
   / TypeRef
   / VoidType
   / one

ParenthesizedType =
     openparen w Type w closeparen

Domain =
     openparen w (Type w comma w)* (Type w ellipses w comma w)?
     KeywordType (w comma w KeywordType)* w closeparen
   / openparen w (Type w comma w)* Type w ellipses w closeparen

KeywordType = BindId w equals w Type

TupleType =
     openparen w Type w comma w TypeList w closeparen

TypeList = Type (w comma w Type)*

TypeRef =
     "Unity"
   / Id w StaticArgs
   / Id

VoidType =
     openparen w closeparen

TypeTail =
     ArrayTypeSize
   / Exponentiation
   / ParenthesizedTypeLeft
   / IdLeft

ArrayTypeSize =
     opensquare (w ArraySize)? w closesquare

ArraySize = ExtentRange (w comma w ExtentRange)*

ExtentRange =
     (StaticArg w)? pound (w StaticArg)?
   / (StaticArg w)? colon (w StaticArg)?
   / StaticArg

Exponentiation =
     caret hasW IntExpr
   / caret hasW openparen w ExtentRange (w cross w ExtentRange)* w
     closeparen

MayParenthesizedOpType =
     openparen w OpType w closeparen
   / OpType

ParenthesizedTypeLeft =
     openparen w MayParenthesizedOpType w closeparen

IdLeft =
     Id

TypeInfixOp =
     rightarrow

DimInfixOp =
     (DOT / slash / per)

DimPrefixOp =
```

```
     (square / cubic / inverse)

DimPostfixOp =
     (squared / cubed)

TraitType =
     TraitTypeFront TraitTypeTail+
   / TypeRef

TraitTypeFront =
     ParenthesizedType
   / TupleType
   / TypeRef
   / VoidType

TraitTypeTail =
     ArrayTypeSizeTrait
   / ExponentiationTrait

ArrayTypeSizeTrait =
     opensquare (w ArraySize)? w closesquare

ExponentiationTrait =
     caret hasW IntExpr
   / caret hasW openparen w ExtentRange (w cross w ExtentRange)* w
     closeparen

TraitTypeWithError =
     TraitTypeFrontWithError TraitTypeTail+
   / TypeRefWithError

TraitTypeFrontWithError =
     ParenthesizedType
   / TupleType
   / TypeRefWithError
   / VoidType

TypeRefWithError =
     "Unity"
   / Id w StaticArgs
   / Id
   / Op (w StaticArgs)?
```

H.20 Types without Newlines

```
Type := ...
TupleType := ...
NoNewlineTupleType = TupleType
ExtentRange := ...
NoNewlineExtentRange = ExtentRange

NoNewlineType =
     !(one) (io w)? TypePrimary (w in w NoNewlineExpr)?

TypePrimary := ...
   / <LooseJuxt> TypePrimaryFront sr TypePrimary
   / <LooseInfix> TypePrimaryFront sr LooseInfix

TypePrefix := ...
   / <Prefix> DimPrefixOp sr TypePrimary
   / <PrePrefix> DimPrefixOp sr TypePrefix

TightInfixPostfix := ...
```

```
   / <Arrow> TypeInfixOp TypePrimary (s Throws)?
   / <ArrowPrefix> TypeInfixOp TypePrefix (s Throws)?
   / <Postfix> DimPostfixOp sr TypePrimary
   / <PostPrefix> DimPostfixOp sr TypePrefix

LooseInfix := ...
   / <Arrow> TypeInfixOp sr TypePrimary (s Throws)?
   / <ArrowPrefix> TypeInfixOp sr TypePrefix (s Throws)?
   / <Infix> DimInfixOp sr TypePrimary
   / <InPrefix> DimInfixOp sr TypePrefix
```

H.21 Symbols and Operators

```
Encloser = encloser

LeftEncloser = !(opendoublesquare) leftEncloser

RightEncloser = rightEncloser

ExponentOp = exponentOp

EncloserPair =
     (LeftEncloser / Encloser) (w DOT)? w (RightEncloser / Encloser)

bar = &("|" wr GeneratorClauseList closingComprehension) "|"
closingComprehension =
     w rightEncloser
   / w closecurly
   / w closeangle
   / br ArrayComprehensionClause
   / w closesquare
sd = [*.]?
bars = "|" (sd "|")*
slashes = "/" (sd "/")*
                                  / "\\" (sd "\\")*
lesses = "<" (sd "<")*
greaters = ">" (sd ">")*

encloser =
     !(bar) bars !([*.>/\\] / "->")
   / "\u2016"
   / "\u2af4"

leftEncloser =
     leftEncloserMulti &{PrecedenceMap.ONLY.isLeft(yyValue)}
   / c:_ &{c != '|' && PrecedenceMap.ONLY.isLeft(""+c)}

leftEncloserMulti =
     "(." ("/"+ / "\\"+)
   / "[/\\/\\/" / "[/\\/"
   / "[" (sd slashes)
   / "{" (sd slashes)
   / lesses sd (slashes / bars)
   / bars sd slashes
   / "{*" /  "[*"
   / "((.>" / "(.<"

rightEncloser =
     rightEncloserMulti &{ PrecedenceMap.ONLY.isRight(yyValue) }
   / c:_ &{c != '|' && PrecedenceMap.ONLY.isRight(""+c)}

rightEncloserMulti =
     "/"+ ".)"
```

```
   / "\\"+ ".)"
   / slashes sd (greaters / bars / [\]}])
   / bars sd greaters
   / "*]" / "*}"
   / "]" / "}"
   / ">.)" / "<.))"
   / "/\\/\\/]" / "/\\/]"

exponentOp =
     "^T"
   / "^" (encloser / op)

OpName =
     opn:id &{NodeUtil.validOp(opn) }

Op =
    (condOp / op !(equalsOp) / compOp) !(Symbol)
   / Symbols

Symbols =
     Symbol+

Symbol = [+]

compOp =
     "==="
   / "=/="
   / "<="
   / ">="

condOp =
     <ErrorProduction> ":::"
   / ":" (encloser / op) colon
   / (encloser / op) colon

multiOp =
     "-/->"
   / "<-/-"
   / "-->"
   / "==>"
   / ">>>"
   / mapstoOp
   / "<<<"
   / "<->"
   / leftarrowOp
   / "<=>"
   / "->"
   / doublerightarrow
   / ">>"
   / "<<"
   / "!!"
   / "::"
   / !(rightEncloserMulti) "///"
   / !(rightEncloserMulti) "//"

singleOp =
     !(encloser / leftEncloser / rightEncloser / multiOp / compOp / match)
     a1:_ !("*") &{ PrecedenceMap.ONLY.isOperator("" + a1) }

op =
     OpName
   / multiOp
   / singleOp

CompoundOp =
```

```
     (encloser / op) equalsOp

doublerightarrow = "=>" &(w BlockElems w match)

crossOp = "BY":OpName &(w ExtentRange) / "\u2a2f":singleOp

leftarrow = "<-" / "\u2190"

caret = "^" !("T")
colonequals = ":=" / "\u2254"
equals = "=" (!op)
equalsOp = "=":singleOp / "\u003d":singleOp
semicolon = ";"
one = "1"

colon = ":" (!op)
colonOp = ":":singleOp / "\u003a":singleOp
closecurly = "}"
opencurly = !(leftEncloserMulti) "{":leftEncloser
closesquare = "]"
opensquare = !(opendoublesquare) "[":leftEncloser
ellipses = "..."

lessthanequal    = "<=":op / "LE":op / "\u2264":op
lessthan         = "<":op  / "LT":op / "\u003c":op
greaterthanequal = ">=":op / "GE":op / "\u2265":op
greaterthan      = ">":op  / "GT":op / "\u003e":op

NOT     = "NOT":op / "\u00ac":op
OR      = "OR":op / "\u2228":op
AND     = "AND":op / "\u2227":op
IMPLIES = "IMPLIES":op / "\u2192":op

DOT = "DOT":OpName / "\u00b7":op
slash = "/":singleOp / "\u002f":singleOp
rightarrow = "->" / "\u2192"
underscore = "_"

closeangle = "|>" / "\u27e9"
closedoublesquare = "\\]" / "\u27e7"
closeparen = ")"
cross = "BY" / "\u2a2f"
mapsto = "|->" / "\u21a6"
match = "=>" / "\u21d2"
minus = "-":singleOp / "\u2212":singleOp
openangle = "<|":leftEncloser / "\u27e8":leftEncloser
opendoublesquare = "[\\" / "\u27e6"
openparen = !(leftEncloser) "("
plus = "+":singleOp / "\u002b":singleOp
pound = "#":singleOp / "\u0023":singleOp
star = !("**") "*":singleOp

comma = ","
commaOrNot = "," / w
dot   = "."

AssignOp =
     colonequals
   / CompoundOp

SUM = "SUM"
PROD = "PROD"

Accumulator =
     SUM
```

```
   / PROD
   / BIG w (Encloser / Op)

ArrayComprehensionClause =
     ArrayComprehensionLeft wr bar wr GeneratorClauseList
```

H.22 Identifiers

```
PrimeCharacter = [\u2032-\u2037]
id      = s:(idstart idrest*) &{ !FORTRESS_KEYWORDS.contains(s) }
idstart = UnicodeIdStart / [_]
idrest  = UnicodeIdStart / ['] / PrimeCharacter / UnicodeIdRest

IdText  = a1:id &{ NodeUtil.validId(a1) }

reserved = s:(idstart idrest*) &{ FORTRESS_KEYWORDS.contains(s) }
Reserved = reserved

Id =
     <FIRST> IdText

IdOrOpName =
     Id
   / OpName

BindId =
     Id
   / "_"

BindIdList =
     BindId (w comma w BindId)+
   / BindId (w commaOrNot w BindId)+
   / BindId

BindIdOrBindIdTuple =
     <FIRST> BindId
   / openparen w BindId w comma w BindIdList w closeparen

commaOrNot = "," / w

SimpleName =
     Id
   / opr (w BIG)? w (Encloser / Op)
   / opr (w BIG)? w EncloserPair

idOrKeyword = (idstart idrest*) &{ true }

APIName =
     idOrKeyword &(w ellipses)
   / idOrKeyword (dot idOrKeyword)* &(w ellipses)
   / idOrKeyword (dot idOrKeyword)*

QualifiedName =
     Id &(w ellipses)
   / Id (dot Id)* &(w ellipses)
   / Id (dot Id)*
```

H.23 Spaces and Comments

```
EndOfFile  = "\u001a"? w !_
```

```
Whitespace =
     Space
   / Newline
Space      =
     " "
   / "\f"
   / "&" s Whitespace
   / InvalidSpace
   / NoNewlineComment
InvalidSpace =
     <ErrorProduction1> ("\t" / "\u000b")
   / <ErrorProduction2> ("\u001c" / "\u001d" / "\u001e" / "\u001f")
Newline =
     "\r\n" / "\r" !("\n") / "\n" / [\u2028\u2029]
   / !("&") NewlineComment
Comment =
     "(*" CommentContents "*)"
   / <ErrorProduction> "(*" CommentContents w EndOfFile

NoNewlineComment =
     "(*)" NNCommentContents
   / "(*" NNCommentContents "*)"
   / "&" s NewlineComment
   / <ErrorProduction> "(*" CommentContents w EndOfFile

CommentContents =
     CommentContent*

NNCommentContents =
     NNCommentContent*

WhitespaceCharacters =
     [\u0009\u000a\u000b\u000c\u000d\u001c\u001d\u001e\u001f\u2028\u2029]

NoNewlineWhitespaceCharacters =
     [\u0009\u000b\u000c\u001c\u001d\u001e\u001f]

CommentContent =
     "(*)"
   / Comment
   / '*' !')'
   / WhitespaceCharacters
   / c:_ &{ c != '*' && Character.getType(c) != Character.CONTROL }

NNCommentContent =
     "(*)"
   / NoNewlineComment
   / '*' !')'
   / NoNewlineWhitespaceCharacters
   / c:_ &{ c != '*' && c != '\n' && c != '\r' && Character.getType(c) != Character.CONTROL }

NewlineComment = Comment

w  = Whitespace*
wr = Whitespace+

s  = Space*
sr = Space+

nl = s Newline w
br = nl / s semicolon w

WhitespaceString =
     Space
```

```
   / Newline
hasW =
     WhitespaceString*

RectSeparator =
     (w semicolon)+ w
   / nl
   / sr
```

Appendix I

Changes Since Fortress 1.0 Specifications

> DRAFT NOTE
> Changes Since http://projectfortress.sun.com/Projects/Community/changeset/1444

I.1 Changes Made by the 2026 Revival to the Working Draft

This section lists the changes made to this working draft by the 2026 revival of the Fortress project (`https://github.com/palimondo/fortress`). The revival works to finish what the designers intended, judged by this working draft, and changes the text only where a decision it has taken for the implementation would otherwise leave the text and the implementation in open disagreement. Each changed passage carries a box headed "Revised by the 2026 revival" that points to its entry here.

Each entry gives the affected sections, the change, its rationale, its effect, the original text, and what reversing to the alternative not taken would require. The original text is that of the *Working Draft of February 2011*, the unrevised copy of this document kept in the revival's repository under the directory `Specification-1.0-frozen/`, cited by path and line there. The directory's name is misleading: its sources are the working draft of 2 February 2011, and only the file `fortress.1.0.pdf` in it is the Fortress 1.0 release. The full reasoning behind every entry, with each original passage and each new one, is in the revival's decision records, `explorations/compile-ladder/rung-spec-route-a/decision-record.md` and, for the number chapters, `explorations/compile-ladder/rung-spec-numbers/decision-record.md`, in the same repository.

Each change below follows a decision of the revival. The first of those decisions, taken on 24 September 2026 and called *route A*, is that the compiled type checker keeps the type group's rule that no type is a subtype of two different instantiations of one parameterized trait, a rule this working draft never stated, and that the library's numeric types are reorganized so that they satisfy it. An entry whose change does not follow from route A says so, and names the decision it follows or gives its reason. The revision of the number chapters (Section I.1.11 to Section I.1.17) also carries out three decisions of 26 September 2026 that follow from route A: on the refined number types, on conversions between number types, and on sums and products. The alternative not taken, *route C*, is described once, in Section I.1.44.

I.1.1 Instantiation exclusion

Affected sections Section 6.3; Section 10.1.

Change A property of the exclusion relation is added: two instantiations of the same parameterized trait exclude each other unless every static argument other than an operator argument is the same. The rule it imposes on declarations is added beside the example of InclusiveMolecule: a trait or object may not extend two such instantiations, directly or through its supertraits.

Rationale Without the rule a value may belong to two instantiations of one parameterized trait, and code specialised for one of them can be chosen for a caller that was promised the other; with the rule relaxed, the compiled implementation accepts such a program and fails at run time. The type group adopted the rule as *multiple instantiation exclusion* (Allen, Hilburn, Kilpatrick, Luchangco, Ryu, Chase, and Steele, "Type Checking Modular Multiple Dispatch with Parametric Polymorphism and Multiple Inheritance", OOPSLA 2011), and the compiled type checker has enforced it since 2010. The team's later, unfinished Types chapter (Luchangco, 2012) states it in the two parts used here, under the name *instantiation exclusion*. The type group's paper of 2019 states it as its rule [Anc-Same-Trait], which it credits to Steele and Chase in 2012: a class that extends two instances of a trait extends an instance below both, which for an invariant static parameter is the same instance (Park, Hong, Steele and Ryu, "Polymorphic Symmetric Multiple Dispatch with Variance", POPL 2019, Figure 4). Operator arguments do not count: an operator argument only names the operator a subtrait inherits (Section 12.5), neither implementation distinguishes instantiations by it at run time, and the algebra of the library chapters depends on types that are several instantiations of one trait differing only in operator arguments. `nat`, `int` and `bool` arguments count: they are values at run time (Section 12.2, Section 12.3).

Effect Programs in which a type is a subtype of two instantiations of one parameterized trait that differ in any other static argument are no longer valid. The compiled type checker refuses them where the arguments that differ are types, or numerals given as `nat` or `int` arguments (row 402 of the revival's gap ledger). It does not yet refuse them where those arguments are `bool` arguments, though it accepts boolean parameters written out (row 406), or where one of them is a static parameter of the trait or object that extends both (row 414). The interpreter does not check the rule. The number chapters are revised to the reorganized numeric types (Section I.1.11).

Original text The Working Draft of February 2011 states no such property; its relations are "the smallest ones that satisfy all the properties given in those sections (and this one)" (`Specification-1.0-frozen/basic/types-vals-vars.tex`, lines 185–189), and the properties it gives for trait types are these (lines 212–216):

> A trait declaration may also include an `excludes` clause, in which case the defined trait type excludes every type that appears in that clause. Every trait type also excludes every arrow type and every tuple type, and the special types () and BottomType.

Its rule on declarations is only that of exclusive traits, "no trait can extend them both" (`Specification-1.0-frozen/basic/traits.tex`, lines 218–222).

Route C Replaces this rule by the rule of Section I.1.44.

I.1.2 Where-clause variables in extends clauses

Affected sections Section 12.6; Section 10.1.

Change One sentence is added: a `where`-clause variable may not appear as a static argument in an `extends` clause. The three declarations that followed it are kept, marked "Not allowed", each with the way to write it (Section I.1.4, Section I.1.5, Section I.1.6). A box after Victor Luchangco's two draft notes in Section 10.1, which say that a trait can explicitly extend itself through hidden type variables and that "we could just rule this a special case", answers them.

Rationale Such a variable makes the declared type a subtype of one instantiation for each type the variable may stand for, and instantiation exclusion forbids any two of them.

Effect None of the three declarations ran before the revision: both implementations stop at name resolution, because a `where`-clause variable is not in scope in an `extends` clause. The revision makes that the rule. The calculus of Appendix A.2 still admits such declarations (Section I.1.10).

Original text No such sentence; the draft notes are unchanged.

Route C The same under route C, which places no type below infinitely many instantiations of one trait.

I.1.3 Conditional extension

Affected sections Section 12.6; Section 40.2; the overview of operators in Chapter 16.

Change A paragraph and an example are added, with a box: where the `where` clause of an entry of an `extends` clause introduces no `where`-clause variable and all its constraints are trait constraints, the declared trait or object extends the entry's type exactly where the constraints hold, and the constraints hold while the entry itself, the declaration's value parameters and its members are checked, and not while the rest of the declaration is, which holds at every instance: its static parameters, its own `where` clause, its `throws` clause and contract, its `excludes` and `comprises` clauses and the other entries of its `extends` clause; the example is the Fortress standard libraries's list, totally ordered exactly where its elements are. Callouts at the lexicographic order of Section 40.2 and at the comparison of lists in the overview of operators say that a list is ordered only where its element type is totally ordered, and that the interpreter still orders every list.

Rationale This change does not follow from route A; the revival decided on 16 October 2026 to build the forms of `where` clause the team wrote down, among them that a trait that extends another under a `where` condition is that type only where the condition holds and that its members are checked under the condition, as Haskell checks an instance's context, Rust a conditional implementation and Swift a conditional conformance. The grammar gives each entry of an `extends` clause its own optional `where` clause, and the text orders lists "whose elements support these same comparison operators" (`Specification-1.0-frozen/basic/operators/opr-overview.tex`, lines 284–285), which no unconditional declaration of the list type can state: the Fortress standard libraries declared their list type as lexicographically ordered at every element type, so the compiled type checker refused the comparison of two elements of an unbounded type in the order's own body. The team's internal notes foresaw conditional subtyping and its cost, that "we may also not be able to tell whether a trait has a particular method" (`Specification-1.0-frozen/appendices/internal-document.tex`, lines 538–543, in its subsection "No Wrapped Fields Whose Types Are Naked Type Variables"). A clause that introduces a `where`-clause variable, or whose constraints are not all trait constraints, keeps the reading it had: the entry is extended at every instance. the Fortress standard libraries's list keeps an element-wise $=$ of its own outside the condition, so that a list whose elements are not ordered, a list of pairs among them, is still compared element by element. The alternative not taken is to read the clause as limiting nothing, the extension holding at every instance, under which the clause has no effect and the list's order needs another construct. The full reasoning is in `explorations/compile-ladder/rung-checker-conditional-extension/REPORT.md` in the revival's repository, and for the two callouts in `explorations/compile-ladder/rung-slices-list-equality/REPORT.md`.

Effect The compiled type checker reads such a clause as this section says: a value of an instance whose static arguments satisfy the constraints is of the entry's type, one whose static arguments do not is refused where that type is expected, and the entry, the declaration's value parameters and its members are checked with the constraints holding, the rest of the declaration without them. Of that rest, the compiled type checker never sees a trait's or an object's `throws` clause or contract: the disambiguator drops both, so neither is checked, and neither path evaluates an object's contract (row 781). the Fortress standard libraries's $\mathrm{LexicographicOrder}$ bounds its element type by $\mathrm{StandardTotalOrder}$, and its list extends it under the constraint that its element type does, so the compiled type checker orders a list exactly where its elements are totally ordered. It gives no order to a list of partially ordered elements, of $\mathrm{RR64}$, QQ or ranges among them, where Section 40.2 says that a lexicographic order "may be partial or total depending on whether the ordering of the elements is partial

or total" (`Specification-1.0-frozen/advanced-lib/algebraic-constraints.tex`, lines 520–522), and where the operators chapter's subsection "Comparisons Operators" compares lists "whose elements support these same comparison operators", as floating-point numbers do: no partial lexicographic order of lists is declared. The compiled type checker still lets a member that it checked with the constraints holding be called where they fail: it accepts a call of such a member on an instance whose static arguments do not satisfy them (row 773 of the revival's gap ledger). Its other readers of an `extends` clause, the members a type inherits and its checks of the type hierarchy, read the entry as extended at every instance. An entry under a clause that introduces a `where`-clause variable or holds a constraint that is not a trait constraint, as the entries of the team's sketch of conditional subtyping do (`Specification-1.0-frozen/appendices/internal-document.tex`, lines 483–512, its subsection "Conditional Subtyping"), is read as extended at every instance on both paths. The interpreter reads the extension as holding at every instance, and so does the compiled run time: a type test of an instance whose static arguments fail the constraints answers *true* on both (row 774, gated for the interpreter by `ProjectFortress/tests/XXXConditionalExtensionTypeTestWalk.fss`), so the interpreter keeps the order of a list of tuples or of floating-point numbers. The code generator compiles a declaration whose `extends` clause has such a clause; it refuses one that carries a `where` clause of its own (row 701).

Original text No such paragraph. The grammar is unchanged: `Specification-1.0-frozen/basic/traits.tex`, line 83,

> *TraitTypeWhere* ::= *TraitType Where*?

and the front matter lists "where clauses and conditional extension" among the features not yet supported (`Specification-1.0-frozen/fortress/preamble.tex`, line 82). The two passages that order lists are unchanged (`Specification-1.0-frozen/advanced-lib/algebraic-constraints.tex`, lines 519–522, and `Specification-1.0-frozen/basic/operators/opr-overview.tex`, lines 284–285); each gains its callout.

Route C The same under route C.

I.1.4 The covariance example

Affected section Section 12.6.

Change The declaration is kept, marked "Not allowed"; the claims that trait declarations may extend other instantiations of themselves and that the declaration expresses covariance are withdrawn; static parameters are stated to be invariant, and a widening function with bounded static parameters is given as the way to convert one instantiation to another.

Rationale $C[\![S]\!]$ would be a subtype of $C[\![T]\!]$ for every supertype T of S. The widening function is how the library writes covariance, in the conversions and the `APPCOV` operator of its covariant collections, and it runs on both implementations.

Effect Covariance is written as an explicit conversion, with the static arguments given at the call. The box at the example adds that the type group's later design declares variance with a `covariant` modifier, implemented on neither implementation; the decision record and row 404 of the revival's gap ledger say what is known of it.

Original text `Specification-1.0-frozen/basic/trait-parameters.tex`, lines 339–351; the declaration is the one kept above, marked "Not allowed", and the text around it read:

> Trait declarations are allowed to extend other instantiations of themselves. For example, we can write: [the declaration] In this declaration, for every subtype S of T, $C[\![S]\!]$ is a subtype of $C[\![T]\!]$. Effectively, we have expressed the fact that the static parameter S of C is covariant.

Route C The same under route C.

I.1.5 A subtrait of every instantiation

Affected section Section 12.6.

Change "the following trait declaration is legal" becomes "is not allowed"; the paragraph after the declaration is kept, in the conditional, and a closing sentence says why the declaration is not allowed.

Rationale The paragraph is the team's own account of why such a trait is dangerous, written before the rule; it becomes the reason for the rule.

Effect None beyond Section I.1.2.

Original text `Specification-1.0-frozen/basic/trait-parameters.tex`, lines 354–380; the declaration is the one kept above, and the text around it read:

> Trait declarations need not have any static parameters in order to have a `where` clause. For example, the following trait declaration is legal: [the declaration] In this declaration, trait C is a subtrait of *every* instantiation of parametric trait D. Thus, trait C has all of the methods of every instantiation of D. By thinking of the declaration this way, we can see what restrictions we need to impose on the trait C in order for it to be sensible. [The rest of the paragraph is unchanged.]

Route C The same under route C.

I.1.6 The empty list

Affected section Section 12.6.

Change The declaration of Empty is kept, marked "Not allowed"; $\mathrm{Empty}[\![T]\!]$, one empty list for each element type, is given beside it.

Rationale The original makes Empty a $\mathrm{List}[\![T]\!]$ for every T at once. The replacement is the library's own shape for its empty collections and the same choice as for $\mathrm{Nothing}$ (Section I.1.7); a program of this shape runs on both implementations.

Effect The element type is written at each use, as in $\mathrm{Empty}[\![\mathbb{Z}32]\!]$, until the static arguments of a generic object can be inferred, which neither implementation does today. The section no longer has a running example of a `where` clause on an object.

Original text `Specification-1.0-frozen/basic/trait-parameters.tex`, lines 383–399; the declaration is the one kept above, introduced by:

> Object or functional declarations may include `where` clauses. Here is an example declaration of an Empty list:

Route C The same under route C.

I.1.7 The empty optional value

Affected sections Section 39.2; Section 14.3; Chapter 35.

Change $\mathrm{Nothing}$ becomes `value object` $\mathrm{Nothing}[\![T]\!]$ extending $\mathrm{Maybe}[\![T]\!]$, with no `excludes` clause, and Maybe comprises $\mathrm{Nothing}[\![T]\!]$ and $\mathrm{Just}[\![T]\!]$; the original declaration is kept, marked "Not allowed". The defaults of an exception's message and chain become $\mathrm{Nothing}[\![\mathrm{String}]\!]$ and $\mathrm{Nothing}[\![\mathrm{Exception}]\!]$.

Rationale The original makes Nothing a Maybe⟦T⟧ for every T at once. Its `excludes` clause is also one the grammar gives no object declaration (Section 11.1), and is redundant, an object trait type already excluding every type that is not its supertype (Section 6.3.1). The replacement is the library's declaration.

Effect The empty case is written with its type, as in Nothing⟦String⟧, until the static arguments of a generic object can be inferred from the type expected. The compiled path's own library, used until the one library replaces it, still declares a single Nothing with a coercion to every Maybe⟦T⟧, and refuses Nothing⟦String⟧ (row 331 of the revival's gap ledger).

Original text `Specification-1.0-frozen/basic-lib/convenience.tex`, lines 34–53: the declaration kept above, marked "Not allowed", together with:

> An optional value v is either Nothing or Just(v) declared as follows: [...]
>
> `trait` Maybe⟦T⟧ `comprises` { Nothing, Just⟦T⟧ }
>
> [...]

`Specification-1.0-frozen/basic/exceptions.tex`, lines 111–130:

> These fields are default to Nothing as follows: [...]
>
> `getter` *message*(): Maybe⟦String⟧ = Nothing
> ...
> `getter` *chain*(): Maybe⟦Exception⟧ = Nothing
>
> [...] where an optional value v is either Nothing or Just(v).

`Specification-1.0-frozen/basic-lib/exception.tex`, lines 22–24:

> These fields are default to Nothing where an optional value v is either Nothing or Just(v) as declared in Section 39.2.

Route C The same under route C.

I.1.8 The root's algebra

Affected sections Chapter 31.

Change Object extends Any alone and Tuple likewise, each still excluding the other; the sentences saying that they describe their operators with algebraic constraints are removed. The original declaration of Object is kept, marked "Not allowed".

Rationale Every trait but Any, Object and Index extends Object, and every commutative algebraic type T is an EquivalenceRelation⟦T, =⟧; with the original declaration such a type is also an EquivalenceRelation⟦Object, ≡⟧, and the two differ in a type argument. Tuple's declaration is not refused by the rule, since no tuple type extends a generic trait (a tuple of indices is a subtype of the trait Index, which takes no static parameter, Section I.1.42), but its text follows Object's. The replacement is the library's declaration.

Effect The properties of ≡ are those stated in trait Any; they are no longer restated at Object through the algebra chapter.

Original text `Specification-1.0-frozen/basic-lib/objects.tex` (whose lines are two fewer than this document's source), line 82:

> In Object this property is asserted abstractly using the algebraic constraints of Chapter 40.

lines 125–139, the declaration kept above, marked "Not allowed", after:

The Object type does not have any additional methods, but uses algebraic constraints (see Chapter 40) to describe the properties of the existing operators more abstractly.

and lines 145–156:

As with the type Object, the type Tuple uses algebraic constraints to describe its existing methods.

```
trait Tuple extends { Any, EquivalenceRelation⟦Tuple, ≡⟧, IdentityOperator⟦Tuple⟧ }
    excludes { Object }
  toString(): String
end
```

Route C By the revival's reading, not measured, the original declaration of Object is one of the two passages that would read differently under route C, the numeric chapters being the other.

I.1.9 The internal document

Affected sections In the appendix "Internal Document", which follows this one in a draft build: "Disjunctive Constraints", "Weirdness with Multiple Inheritance and Polymorphism" and "Conditional Subtyping".

Change One box after each: the first program's premise is not allowed; the second is the case the rule answers; the third declaration is not allowed for the reason the covariance example is not.

Rationale Instantiation exclusion.

Effect None; the internal document is printed only in a draft build.

Original text Unchanged.

Route C The same under route C.

I.1.10 The calculi

Affected sections In Appendix A: Basic Core Fortress (Section A.1), Core Fortress with Where Clauses (Appendix A.2), Core Fortress with Overloading (Appendix A.3) and Acyclic Core Fortress with Field Definitions (Section A.4).

Change One box at the opening of each calculus, saying that the calculus predates instantiation exclusion and naming the shape it admits that the rule refuses: in Basic Core Fortress, an object that extends two different instantiations of a parameterized trait declaring no method, so that its statement that all of its valid programs are valid Fortress programs no longer holds for every program; in Core Fortress with Where Clauses, a `where`-clause variable in a supertype, which its subtyping rule turns into one supertype for each witness of the variable; in Core Fortress with Overloading, the same object as in Basic Core Fortress, which its rule for valid method declarations admits because the trait declares no method; in Acyclic Core Fortress with Field Definitions, a trait that extends an instantiation of itself at its parameter's bound, which conditions (2) and (3) of its rule for an acyclic type hierarchy admit. The rules of the calculi and their soundness claims are not changed.

Rationale The calculi are the team's formal record of this working draft as it stood, and the rule changes the language, not that record. New premises in their rules would be new formal work, and would reopen soundness claims that were proved for the rules as they are written. The revival decided on this treatment on 26 September 2026.

Effect None on the language or on either implementation. The four calculi admit programs, in the shapes named, that are not valid Fortress programs.

Original text The calculi are unchanged. The statement that the box in Basic Core Fortress qualifies is in `Specification-1.0-frozen/appendices/calculi/basic/calculus.tex`, lines 16–19:

> Following the precedent set by prior core calculi such as Featherweight Generic Java [15], we have abided by the restriction that all valid Basic Core Fortress programs are valid Fortress programs.

The shapes the other three boxes name are admitted by `Specification-1.0-frozen/appendices/calculi/where/syntax.tex`, lines 82–91, and `Specification-1.0-frozen/appendices/calculi/where/static.tex`, lines 53–81 and 329–342, by `Specification-1.0-frozen/appendices/calculi/overloading/static.tex`, lines 52–82, 107–136 and 270–275, and by `Specification-1.0-frozen/appendices/calculi/acffd/static.tex`, lines 46–88. Before this entry, Section I.1.43 said of the calculi:

> The calculi of Appendix A predate the rule and are not yet revised: the calculus with `where` clauses (Appendix A.2) lets a `where`-clause variable occur in a supertype, the calculus with an acyclic type hierarchy admits a trait that extends an instantiation of itself at its parameter's bound, and the basic calculus states that all of its valid programs are valid Fortress programs, while it admits an object that extends two instantiations of a parameterized trait declaring no method. How they are to be revised awaits a decision of the revival.

That paragraph did not name Core Fortress with Overloading, whose box was added after the other three.

Route C By the revival's reading, not measured, the same under route C: none of the shapes named is a chain of instantiations of a self-typed parameterized trait, the one case route C relaxes.

I.1.11 The number types

Affected sections Section 33.1; Section 33.2.

Change The eighteen rational types and the twelve integer types of the original, with $\mathbb{N}$ and $\mathbb{N}^*$, each a subtype of others, are replaced by seven number types the Fortress standard libraries declare: $\mathbb{Z}$, $\mathbb{Z}64$, $\mathbb{Z}32$, $\mathbb{N}64$, $\mathbb{N}32$, $\mathbb{Q}$ and $\mathbb{R}64$, siblings under Number, with the coercions the library declares: $\mathbb{Z}64$ from $\mathbb{Z}32$ and $\mathbb{N}32$; $\mathbb{N}64$ from $\mathbb{N}32$; $\mathbb{Z}$ from the four fixed-size types; $\mathbb{Q}$ from the five integer types; $\mathbb{R}64$ from $\mathbb{Z}32$ and from integer numerals. Every other conversion between them is explicit, the one into $\mathbb{R}64$ written $asFloat(x)$ with the method the library declares on Number. $\mathbb{Q}$ includes $+\infty$, $-\infty$ and $0/0$; the integer types hold no infinite value. A subset such as the positive values is tested at run time, and the two checks the library declares, $check$ and $check^*$ on $\mathbb{R}64$, returning $\mathrm{Maybe}[\![\mathbb{R}64]\!]$, are stated. The sentences that the type system tracks the refined types through arithmetic are removed, and so are the sentences that $\mathbb{Q}$ is a field and totally ordered and that $\mathbb{Z}^*$ is not a ring; $\mathbb{Q}$, which holds $0/0$, is stated to be neither totally ordered nor a field.

Rationale Route A makes the library's numeric types siblings under Number, each with its own algebra, a wider type converting a narrower one by coercion, and $\mathbb{Z}$ inside $\mathbb{Q}$ by coercion. The revival decided on 26 September 2026 that the refined types become the library's own run-time checks, which return a Maybe, and that their original design is kept as a superseded design (Section I.1.14); and that a conversion between number types is a coercion where it is exact and stays explicit where it can lose precision, the coercion into $\mathbb{R}64$ being declared from $\mathbb{Z}32$ and the numerals only. The Fortress standard libraries never declared the refined types, and the trait that defines them is refused by instantiation exclusion (Section I.1.1). $\mathbb{Q}$ holds the values that $\mathbb{Q}^*$ and $\mathbb{Q}^\#$ added, as the library's rational type does, and as Chapter 16 already says: "For rational results, division by zero produces $1/0$." The type that holds $0/0$ is "neither totally ordered nor a field" in the original's own words. The team's later Types chapter (Luchangco, 2012) keeps the statement that the numeric types are mutually exclusive (Section 6.9).

Effect A program names $\mathbb{Q}$ or $\mathbb{Z}$ where it named a refined type, and tests a sign or finiteness at run time. A generic call over two different integer types writes its static argument until the rule that infers the narrowest type both coerce into is written into Chapter 20 together with its implementation. The conversion from $\mathbb{N}32$ into $\mathbb{R}64$, though exact, stays explicit, since the decision names only $\mathbb{Z}32$ and the numerals. The library's $\mathbb{R}32$, which

this change left below $\mathbb{R}64$, is made a sibling of the other number types by Section I.1.28, and $\mathbb{R}64$ coerces from it.

Original text `Specification-1.0-frozen/basic-lib/numbers.tex`, lines 19–105:

The trait $\mathbb{Q}$ (`QQ`) encompasses all finite rational numbers, the results of dividing any integer by any nonzero integer. The trait $\mathbb{Q}^*$ (`QQ_star`) is $\mathbb{Q}$ with two extra elements, $+\infty$ and $-\infty$. The trait $\mathbb{Q}^\#$ (`QQ_splat`) is $\mathbb{Q}^*$ with one additional element, the indefinite rational (written $0/0$), which is used as the result of dividing zero by zero or of adding $-\infty$ to $+\infty$.

Often it is desirable to indicate that a variable ranges over only a subset of the rationals, such as only positive values or only nonnegative values or only nonzero values. Unfortunately, traditional notations such as $\mathbb{Q}^+$ are not used consistently in the literature; one author may use $\mathbb{Q}^+$ to mean the set of strictly positive rationals and another may use it to mean the set of nonnegative rationals. Fortress therefore uses a notation that is novel but unambiguous:

$\mathbb{Q}$ (`QQ`) is the set of rationals (it is a subtype of $\mathbb{R}$ and $\mathbb{Q}^*$).
$\mathbb{Q}_<$ (`QQ_LT`) is the set of strictly negative rationals (it is a subtype of $\mathbb{R}_<$, $\mathbb{Q}^*_<$, $\mathbb{Q}$, $\mathbb{Q}_\leq$, and $\mathbb{Q}_\neq$).
$\mathbb{Q}_\leq$ (`QQ_LE`) is the set of nonpositive rationals, that is, $\mathbb{Q}_< \cup \{0\}$ (it is a subtype of $\mathbb{R}_\leq$, $\mathbb{Q}^*_\leq$, and $\mathbb{Q}$).
$\mathbb{Q}_\geq$ (`QQ_GE`) is the set of nonnegative rationals, that is, $\mathbb{Q}_> \cup \{0\}$ (it is a subtype of $\mathbb{R}_\geq$, $\mathbb{Q}^*_\geq$, and $\mathbb{Q}$).
$\mathbb{Q}_>$ (`QQ_GT`) is the set of strictly positive rationals (it is a subtype of $\mathbb{R}_>$, $\mathbb{Q}^*_>$, $\mathbb{Q}$, $\mathbb{Q}_\geq$, and $\mathbb{Q}_\neq$).
$\mathbb{Q}_\neq$ (`QQ_NE`) is the set of strictly nonzero rationals (that is, $\mathbb{Q}_< \cup \mathbb{Q}_>$) (it is a subtype of $\mathbb{R}_\neq$, $\mathbb{Q}^*_\neq$, and $\mathbb{Q}$).
$\mathbb{Q}^*$ (`QQ_star`) is $\mathbb{Q}$ with extra elements $+\infty$ and $-\infty$ (it is a subtype of $\mathbb{R}^*$ and $\mathbb{Q}^\#$).
$\mathbb{Q}^*_<$ (`QQ_star_LT`) is $\mathbb{Q}_<$ with extra element $-\infty$ (it is a subtype of $\mathbb{R}^*_<$, $\mathbb{Q}^\#_<$, $\mathbb{Q}^*$, $\mathbb{Q}^*_\leq$, and $\mathbb{Q}^*_\neq$).
$\mathbb{Q}^*_\leq$ (`QQ_star_LE`) is $\mathbb{Q}_\leq$ with extra element $-\infty$ (it is a subtype of $\mathbb{R}^*_\leq$, $\mathbb{Q}^\#_<$, and $\mathbb{Q}^*$).
$\mathbb{Q}^*_\geq$ (`QQ_star_GE`) is $\mathbb{Q}_\geq$ with extra element $+\infty$ (it is a subtype of $\mathbb{R}^*_\geq$, $\mathbb{Q}^\#_>$, and $\mathbb{Q}^*$).
$\mathbb{Q}^*_>$ (`QQ_star_GT`) is $\mathbb{Q}_>$ with extra element $+\infty$ (it is a subtype of $\mathbb{R}^*_>$, $\mathbb{Q}^\#_>$, $\mathbb{Q}^*$, $\mathbb{Q}^*_\geq$, and $\mathbb{Q}^*_\neq$).
$\mathbb{Q}^*_\neq$ (`QQ_star_NE`) is $\mathbb{Q}_\neq$ with extra elements $+\infty$ and $-\infty$ (it is a subtype of $\mathbb{R}^*_\neq$, $\mathbb{Q}^\#_\neq$, and $\mathbb{Q}^*$).
$\mathbb{Q}^\#$ (`QQ_splat`) is $\mathbb{Q}^*$ with extra element $0/0$ (it is a subtype of $\mathbb{R}^\#$).
$\mathbb{Q}^\#_<$ (`QQ_splat_LT`) is $\mathbb{Q}^*_<$ with extra element $0/0$ (it is a subtype of $\mathbb{R}^\#_<$, $\mathbb{Q}^\#$, $\mathbb{Q}^\#_\leq$, and $\mathbb{Q}^\#_\neq$).
$\mathbb{Q}^\#_\leq$ (`QQ_splat_LE`) is $\mathbb{Q}^*_\leq$ with extra element $0/0$ (it is a subtype of $\mathbb{R}^\#_\leq$ and $\mathbb{Q}^\#$).
$\mathbb{Q}^\#_\geq$ (`QQ_splat_GE`) is $\mathbb{Q}^*_\geq$ with extra element $0/0$ (it is a subtype of $\mathbb{R}^\#_\geq$ and $\mathbb{Q}^\#$).
$\mathbb{Q}^\#_>$ (`QQ_splat_GT`) is $\mathbb{Q}^*_>$ with extra element $0/0$ (it is a subtype of $\mathbb{R}^\#_>$, $\mathbb{Q}^\#$, $\mathbb{Q}^\#_\geq$, and $\mathbb{Q}^\#_\neq$).
$\mathbb{Q}^\#_\neq$ (`QQ_splat_NE`) is $\mathbb{Q}^*_\neq$ with extra element $0/0$ (it is a subtype of $\mathbb{R}^\#_\neq$ and $\mathbb{Q}^\#$).

The Fortress type system tracks these types closely through various arithmetic operations; for example, adding two values of type $\mathbb{Q}_>$ produces a result of type $\mathbb{Q}_>$, and adding a value of type $\mathbb{Q}^*_>$ and a value of type $\mathbb{Q}_\geq$ produces a value of type $\mathbb{Q}^*_\geq$.

Here we present only the trait $\mathbb{Q}$ and its methods. The other rational types have exactly the same methods and differ only in the details of the types of method arguments and results and exactly what traits are extended by each rational type. For example, $\mathbb{Q}$ is a field and is totally ordered, $\mathbb{Q}^*$ is totally ordered but is not a field, and $\mathbb{Q}^\#$ is neither totally ordered nor a field. For the exact details of how all this is implemented, see Section 41.1.

`Specification-1.0-frozen/basic-lib/basic-integers.tex`, lines 16–84:

The trait $\mathbb{Z}$ (`ZZ`) encompasses all finite integers, the results of starting from 0 and repeatedly adding or subtracting 1 a finite number of times. The trait $\mathbb{Z}^*$ (`ZZ_star`) is $\mathbb{Z}$ with two extra elements, $+\infty$ and $-\infty$.

Often it is desirable to indicate that a variable ranges over only a subset of the integers, such as only positive values or only nonnegative values or only nonzero values. Fortress uses a system similar

to that for rational numbers, but also accommodates the traditional use of $\mathbb{N}$ to denote the natural numbers:

$\mathbb{Z}$ (`ZZ`) is the set of integers (it is a subtype of $\mathbb{Q}$ and $\mathbb{Z}^*$).
$\mathbb{Z}_<$ (`ZZ_LT`) is the set of strictly negative integers (it is a subtype of $\mathbb{Q}_<$, $\mathbb{Z}^*_<$, $\mathbb{Z}$, $\mathbb{Z}_\leq$, and $\mathbb{Z}_\neq$).
$\mathbb{Z}_\leq$ (`ZZ_LE`) is the set of nonpositive integers, that is, $\mathbb{Z}_< \cup \{0\}$ (it is a subtype of $\mathbb{Q}_\leq$, $\mathbb{Z}^*_\leq$, and $\mathbb{Z}$).
$\mathbb{Z}_\geq$ (`ZZ_GE`) is the set of nonnegative integers, that is, $\mathbb{Z}_> \cup \{0\}$ (it is a subtype of $\mathbb{Q}_\geq$, $\mathbb{Z}^*_\geq$, and $\mathbb{Z}$).
$\mathbb{Z}_>$ (`ZZ_GT`) is the set of strictly positive integers (it is a subtype of $\mathbb{Q}_>$, $\mathbb{Z}^*_>$, $\mathbb{Z}$, $\mathbb{Z}_\geq$, and $\mathbb{Z}_\neq$).
$\mathbb{Z}_\neq$ (`ZZ_NE`) is the set of strictly nonzero integers (that is, $\mathbb{Z}_< \cup \mathbb{Z}_>$) (it is a subtype of $\mathbb{Q}_\neq$, $\mathbb{Z}^*_\neq$, and $\mathbb{Z}$).
$\mathbb{Z}^*$ (`ZZ_star`) is $\mathbb{Z}$ with extra elements $+\infty$ and $-\infty$ (it is a subtype of $\mathbb{Q}^*$).
$\mathbb{Z}^*_<$ (`ZZ_star_LT`) is $\mathbb{Z}_<$ with extra element $-\infty$ (it is a subtype of $\mathbb{Q}^*_<$, $\mathbb{Z}^*$, $\mathbb{Z}^*_\leq$, and $\mathbb{Z}^*_\neq$).
$\mathbb{Z}^*_\leq$ (`ZZ_star_LE`) is $\mathbb{Z}_\leq$ with extra element $-\infty$ (it is a subtype of $\mathbb{Q}^*_\leq$ and $\mathbb{Z}^*$).
$\mathbb{Z}^*_\geq$ (`ZZ_star_GE`) is $\mathbb{Z}_\geq$ with extra element $+\infty$ (it is a subtype of $\mathbb{Q}^*_\geq$ and $\mathbb{Z}^*$).
$\mathbb{Z}^*_>$ (`ZZ_star_GT`) is $\mathbb{Z}_>$ with extra element $+\infty$ (it is a subtype of $\mathbb{Q}^*_>$, $\mathbb{Z}^*$, $\mathbb{Z}^*_\geq$, and $\mathbb{Z}^*_\neq$).
$\mathbb{Z}^*_\neq$ (`ZZ_star_NE`) is $\mathbb{Z}_\neq$ with extra elements $+\infty$ and $-\infty$ (it is a subtype of $\mathbb{Q}^*_\neq$ and $\mathbb{Z}^*$).
$\mathbb{N}$ (`NN`) is a synonym for $\mathbb{Z}_\geq$.
$\mathbb{N}^*$ (`NN_star`) is a synonym for $\mathbb{Z}^*_\geq$.

The Fortress type system tracks these types closely through various arithmetic operations; for example, adding two values of type $\mathbb{Z}_>$ produces a result of type $\mathbb{Z}_>$, and adding a value of type $\mathbb{Z}^*_>$ and a value of type $\mathbb{Z}_\geq$ produces a value of type $\mathbb{Z}^*_>$.

Here we present only the trait $\mathbb{Z}$ and its methods. The other integer types have exactly the same methods and differ only in the details of the types of method arguments and results and exactly what traits are extended by each integer type. For example, $\mathbb{Z}$ is a commutative ring and is totally ordered, and $\mathbb{Z}^*$ is totally ordered but is not a ring. *Future versions of this specification will include the exact details of how all this is implemented.*

Route C By the revival's reading, not measured, route C could keep the nesting of the widths, each level a link in a chain of instantiations of a self-typed trait, but not the refined types: $\mathrm{RationalQuantity}$ places each type below a lattice of its own instantiations, not a chain, and names them through `where`-clause variables in its `extends` clause (Section I.1.2).

I.1.12 The rational trait

Affected section Section 33.1.

Change The listing of $\mathbb{Q}$ extends Number in place of the real numbers, $\mathbb{Q}^*$, three instantiations of Field with $\mathbb{Q}_\neq$ as the multiplicative group, $\mathrm{TotalOrderOperators}$ and $\mathrm{PartialOrderAndLattice}$, and declares the coercions from the five integer types, with a method entry for them. The reciprocal, division and exponentiation return $\mathbb{Q}$, and so does the absolute value; the four hyperfloor and hyperceiling operators return $\mathbb{Z}$; the two comparisons by `CMP` become one, returning $\mathrm{Comparison}$. The seventeen check methods leave the listing and the method entries. Ten sentences of the method entries lose the words "For types $\mathbb{Q}^*$ and $\mathbb{Q}^\#$," or "For type $\mathbb{Q}^\#$,", the next word taking the capital (the sentence on `MAXNUM` and `MINNUM` now begins "The operators"); the parenthesis on the reciprocal's result type and the sentence on $\mathrm{TotalComparison}$ are removed.

Rationale Section I.1.11. A signature that named a refined type takes the type the Fortress standard libraries declare for the method where they declare it (division and the absolute value return the rational type, exponentiation by an integer returns it, and the comparison `CMP` returns $\mathrm{Comparison}$), and $\mathbb{Q}$ or $\mathbb{Z}$ where they do not. The library declares *check* and *check** on $\mathbb{R}64$, where Section 33.1 states them, and no check by sign. What the entries said of $\mathbb{Q}^*$ and $\mathbb{Q}^\#$ holds of $\mathbb{Q}$, which holds their values.

Effect As Section I.1.11.

Original text `Specification-1.0-frozen/basic-lib/numbers.tex`: the listing's header, lines 177–183, and its lines 198–200, 206–207, 212, 222–225 and 228–244:

$\texttt{trait}\ \mathbb{Q}$
$\quad\texttt{extends}\ \{\mathbb{R}, \mathbb{Q}^*,$
$\qquad\qquad\text{Field}[\![\mathbb{Q}, \mathbb{Q}_{\neq}, +, -, \cdot, /]\!],$
$\qquad\qquad\text{Field}[\![\mathbb{Q}, \mathbb{Q}_{\neq}, +, -, \times, /]\!],$
$\qquad\qquad\text{Field}[\![\mathbb{Q}, \mathbb{Q}_{\neq}, +, -, \texttt{juxtaposition}, /]\!],$
$\qquad\qquad\text{TotalOrderOperators}[\![\mathbb{Q}, <, \leq, \geq, >, \texttt{CMP}]\!],$
$\qquad\qquad\text{PartialOrderAndLattice}[\![\mathbb{Q}, \leq, \texttt{MIN}, \texttt{MAX}]\!]\ \}$
$\quad\texttt{opr}\ /(\texttt{self}) \colon \mathbb{Q}^*$
$\quad\texttt{opr}\ /(\texttt{self}, \mathit{other} \colon \mathbb{Q}) \colon \mathbb{Q}^{\#}$
$\quad\texttt{opr}\ \hat{}\,(\texttt{self}, \mathit{power} \colon \mathbb{Z}) \colon \mathbb{Q}^{\#}$
$\quad\texttt{opr}\ \texttt{CMP}(\texttt{self}, \mathit{other} \colon \mathbb{Q}^*) \colon \text{TotalComparison}$
$\quad\texttt{opr}\ \texttt{CMP}(\texttt{self}, \mathit{other} \colon \mathbb{Q}^{\#}) \colon \text{Comparison}$
$\quad\texttt{opr}\ |\texttt{self}| : \mathbb{Q}_{\geq}$
$\quad\texttt{opr}\ \lfloor\!\lfloor\texttt{self}\rfloor\!\rfloor \colon \mathbb{N}$
$\quad\texttt{opr}\ \lceil\!\lceil\texttt{self}\rceil\!\rceil \colon \mathbb{N}$
$\quad\texttt{opr}\ \lfloor\!\lfloor\!\lfloor\texttt{self}\rfloor\!\rfloor\!\rfloor \colon \mathbb{N}$
$\quad\texttt{opr}\ \lceil\!\lceil\!\lceil\texttt{self}\rceil\!\rceil\!\rceil \colon \mathbb{N}$
$\quad\mathit{check}(\texttt{self}) \colon \mathbb{Q}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^*(\texttt{self}) \colon \mathbb{Q}^*\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}_{<}(\texttt{self}) \colon \mathbb{Q}_{<}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}_{\leq}(\texttt{self}) \colon \mathbb{Q}_{\leq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}_{\geq}(\texttt{self}) \colon \mathbb{Q}_{\geq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}_{>}(\texttt{self}) \colon \mathbb{Q}_{>}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}_{\neq}(\texttt{self}) \colon \mathbb{Q}_{\neq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^*_{<}(\texttt{self}) \colon \mathbb{Q}^*_{<}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^*_{\leq}(\texttt{self}) \colon \mathbb{Q}^*_{\leq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^*_{\geq}(\texttt{self}) \colon \mathbb{Q}^*_{\geq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^*_{>}(\texttt{self}) \colon \mathbb{Q}^*_{>}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^*_{\neq}(\texttt{self}) \colon \mathbb{Q}^*_{\neq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^{\#}_{<}(\texttt{self}) \colon \mathbb{Q}^{\#}_{<}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^{\#}_{\leq}(\texttt{self}) \colon \mathbb{Q}^{\#}_{\leq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^{\#}_{\geq}(\texttt{self}) \colon \mathbb{Q}^{\#}_{\geq}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^{\#}_{>}(\texttt{self}) \colon \mathbb{Q}^{\#}_{>}\ \texttt{throws}\ \text{CastError}$
$\quad\mathit{check}^{\#}_{\neq}(\texttt{self}) \colon \mathbb{Q}^{\#}_{\neq}\ \texttt{throws}\ \text{CastError}$

The method entries at lines 327–328, 345–346, 388, 416–417, 488–493 and 546–562 repeated these signatures. Two did not: line 339 gave division the result $\mathbb{Q}^*$, and line 387 gave `CMP` an argument of type $\mathbb{Q}$ and the result TotalComparison:

$\texttt{opr}\ /(\texttt{self}, \mathit{other} \colon \mathbb{Q}) \colon \mathbb{Q}^*$

$\texttt{opr}\ \texttt{CMP}(\texttt{self}, \mathit{other} \colon \mathbb{Q}) \colon \text{TotalComparison}$

In the chapter each of these two lines was the heading of its entry; here each is set as a line. The sentences of lines 289, 300, 320, 334, 374, 444 and 479 began "For types $\mathbb{Q}^*$ and $\mathbb{Q}^{\#}$," and those of lines 410, 422 and 431 "For type $\mathbb{Q}^{\#}$,"; lines 331–332:

> The reciprocal of zero is $+\infty$ (and therefore the result type of / when given an arguments of type $\mathbb{Q}$ is necessarily $\mathbb{Q}^*$).

lines 390–393:

> The `CMP` operator compares the arguments and returns one of the four values $\mathrm{LessThan}$, $\mathrm{EqualTo}$, $\mathrm{GreaterThan}$, and $\mathrm{Unordered}$. If the argument types are such that the result cannot be $\mathrm{Unordered}$, then the result has type $\mathrm{TotalComparison}$ rather than simply $\mathrm{Comparison}$.

and lines 564–566:

> Each of these methods checks this rational number to see whether it belongs to the result type of the method. If, the number is returned; if not, a $\mathrm{CastError}$ is thrown.

Route C The same under route C, which rescues none of the refined types (Section I.1.11).

I.1.13 The integer trait

Affected section Section 33.2.

Change The listing of $\mathbb{Z}$ extends Number in place of $\mathbb{Q}$ and $\mathbb{Z}^*$, keeping its algebra, and declares the coercions from the four fixed-size integer types, with a method entry for them. Division returns $\mathbb{Q}$; the exponent of the power, the results of the factorial, of `CHOOSE` and of the four hyperfloor and hyperceiling operators, the argument of `CMP`, the absolute value, and the result of $lowBits$ in its method entry are $\mathbb{Z}$. The twelve check methods leave the listing and the method entries. The method entries of the three unary negations give the result $\mathbb{Z}$, as the listing does. The sentences on the infinities of $\mathbb{Z}^*$ and $\mathbb{N}^*$ leave the method entries, and so do the words that made the strictly positive integers a type extending a lattice trait; the lattice they describe stays. Another revision gives the result of an integer raised to a negative power in the method entry of the power: the bases 1 and -1 give their exact power, a base of 0 throws $\mathrm{DivisionByZero}$, and any other base throws $\mathrm{IntegerOverflow}$, since the result is no integer; a fraction is written with a floating-point or rational base. The box above the listing and a new box after the method entry say so.

Rationale Section I.1.11. The Fortress standard libraries declare no infinite integer and no check on the integer types. Division takes the type the library declares for it; the other signatures take $\mathbb{Z}$, since the library declares none of those methods with a natural-number type (its power on integers takes an exponent of any integer type and returns the integer's own type). Route A gives each number type its own algebra, and the listing's commutative ring over $\mathbb{Z}$ negates within $\mathbb{Z}$; the method entries' $\mathbb{Q}$ held of an integer only while $\mathbb{Z}$ was a subtype of $\mathbb{Q}$. The revision of the negative power follows the revival's decision of 20 October 2026 that each integer type's power answers its own type at every exponent, raising an error where the result is no integer: the type group's later code types an integer power by its integer at a signed exponent; no type carries the sign of an exponent or of a numeral, so that no overloading can give 10^{16} and 10^{-16} different types, and the split is on the base, with the sign tested at run time as the Fortress standard libraries test a sign elsewhere. Its reasoning is in `explorations/reviews/integer-power-fable-judgement.md`, and its measurement in `explorations/perf-probes/integer-power/REPORT.md`, in the revival's repository. A reversal to the zero that truncation gives would change only the interpreter's natives at a negative exponent and the sentences of the method entry; one to a rational result, typed by the exponent as the Working Draft typed it, would need the sign of a numeral exponent in its type, which neither implementation gives.

Effect As Section I.1.11. The integer types no longer hold infinities, so their total order is that of the finite integers. Negating an integer gives an integer. The exponent of the power, $\mathbb{N}$ in the Working Draft, is now $\mathbb{Z}$, so a negative power is admitted. The Working Draft gave the result of a negative power through the exponentiation of $\mathbb{Q}$, since $\mathbb{Z}$ was a subtype of $\mathbb{Q}$: 2^{-1} was $1/2$ (`Specification-1.0-frozen/basic-lib/numbers.tex`, lines 344–355). The revised text of Section 33.2 gives it otherwise: 2^{-1} throws $\mathrm{IntegerOverflow}$, and a program that wants $1/2$ writes 2.0^{-1} or a rational base. The interpreter does so at every integer type, at an exponent of any integer type, and its integer powers answer the base's own type, as the Fortress standard libraries declare them; the compiled type checker types them so. The compiled path's own library does not: its $\mathbb{Z}32$ power answers 0 at a negative exponent, and a Java exception that no `catch` sees at a base of 0 (rows 441 and 523 of the revival's gap ledger).

Original text `Specification-1.0-frozen/basic-lib/basic-integers.tex`: the listing's lines 196, 239, 246, 249–250, 256, 261, 271–274 and 290–301:

```
   extends { ℚ, ℤ*, IntegerLike⟦ℤ⟧,
opr /(self, other: ℤ): ℚ#
opr _(self, power: ℕ): ℤ
opr (self)! : ℕ
opr CHOOSE(self, other: ℤ): ℕ
opr CMP(self, other: ℤ*): TotalComparison
opr |self| : ℤ≥
opr ⌊self⌋: ℕ
opr ⌈self⌉: ℕ
opr ⫡self⫢: ℕ
opr ⫣self⫤: ℕ
check(self): ℤ throws CastError
check*(self): ℤ* throws CastError
check<(self): ℤ< throws CastError
check≤(self): ℤ≤ throws CastError
check≥(self): ℤ≥ throws CastError
check>(self): ℤ> throws CastError
check≠(self): ℤ≠ throws CastError
check*<(self): ℤ*< throws CastError
check*≤(self): ℤ*≤ throws CastError
check*≥(self): ℤ*≥ throws CastError
check*>(self): ℤ*> throws CastError
check*≠(self): ℤ*≠ throws CastError
```

The method entries at lines 425, 507, 551, 600, 669–672 and 888–899 repeated these signatures, line 507 setting the power operator as a caret where the listing's line 246 set an underscore. Two did not: line 622 gave the absolute value the result $\mathbb{N}$, where the listing had $\mathbb{Z}_{\geq}$, of which $\mathbb{N}$ was a synonym; and line 816 gave *lowBits* the result $\mathbb{N}$, where the listing's line 284, which the revision leaves as it was, already said $\mathbb{Z}$:

```
opr |self| : ℕ

lowBits(self, k: IndexInt): ℕ
```

In the chapter each of these two lines was the heading of its entry; here each is set as a line. Lines 380–382 gave the three unary negations the result $\mathbb{Q}$, where the listing's lines 228–230 gave $\mathbb{Z}$:

```
opr −(self): ℚ

opr ⊟(self): ℚ

opr ∸(self): ℚ
```

In the chapter the first line was the heading of its entry and the other two continued it; here each is set as a line. Lines 372–374, 387–388 and 418–421:

> For type $\mathbb{Z}^*$, the sum of an infinity and either a finite integer or another infinity of the same sign is equal to the given infinity, but attempting to sum infinities of differing sign throws an IndeterminateInteger.
>
> For types $\mathbb{Z}^*$ and $\mathbb{Z}^\#$, the negative of $+\infty$ is $-\infty$, and the negative of $-\infty$ is $+\infty$.
>
> For types $\mathbb{Z}^*$ and $\mathbb{Z}^\#$, attempting to multiply zero and an infinity (regardless of sign) throws an IndeterminateInteger. The product of an infinity and any nonzero integer (including an infinity) is an infinity whose sign is positive if and only if the two arguments have the same sign.

lines 531–533:

> The type $\mathbb{Z}_{>}$ extends the trait $\mathrm{PartialOrderAndMeetBoundedLattice}[\![\mathbb{Z}_{>}, |, \texttt{GCD}, \texttt{LCM}]\!]$, which is to say that the strictly positive integers form a partial order and lattice with

lines 555–556, 592–594, 643–646 and 838–839:

> If the argument is 0, the result is 1. For type $\mathbb{N}^*$, if the argument is $+\infty$, the result is $+\infty$.
>
> Integer values, including $+\infty$ and $-\infty$, are totally ordered. The value $-\infty$ is less than any finite integer value, and $+\infty$ is greater than any finite integer value.
>
> For type $\mathbb{Z}^*$, the numerator of $+\infty$ is 1, and the numerator of $-\infty$ is -1; the denominator of either $+\infty$ or $-\infty$ is 0.
>
> For type $\mathbb{Z}^*$, if this integer is $+\infty$ or $-\infty$, an $\mathrm{InvalidArgument}$ is thrown.

and lines 901–903:

> Each of these methods checks this integer to see whether it belongs to the result type of the method. If, the number is returned; if not, a $\mathrm{CastError}$ is thrown.

Until the revision of the negative power, the revival's text said of it, in this entry's Rationale and Effect, in the box above the listing, and in Section I.1.43:

> its power on integers is another method, returning a floating-point value
>
> The revised text of Section 33.2 does not give it.
>
> This chapter does not give it.
>
> The result of an integer raised to a negative power is not yet given

Route C By the revival's reading, not measured, route C could keep $\mathbb{Z}$ below $\mathbb{Q}$, but not $\mathbb{Z}^*$ or the refined integer types (Section I.1.11).

I.1.14 The superseded chapter on rational quantities

Affected sections Section 41.1; the box after "Conditional Subtyping" in the appendix "Internal Document".

Change One box at the head of the chapter says that it is a superseded design, why, and where the way back is. No word of the chapter changes. The box in the internal document (Section I.1.9), which said that the number chapters were not yet revised, now points here.

Rationale The revival decided on 26 September 2026 to keep the chapter word for word as the record of the original design. Instantiation exclusion refuses $\mathrm{RationalQuantity}$, which extends its own instantiations. The compiled type checker refuses the team's own test of the declaration with "Cyclic type hierarchy: Type RationalQuantity transitively extends itself"; the interpreter runs that test only because nothing instantiates the trait, and an instantiation fails there, on the trait's `where` clause or, without it, with "Missing type" for a flag of the conditional `extends` clause. The team's later Types chapter comments out the section on type aliases by which the chapter names the eighteen types. The way back is declaration-site variance, the later chapter's `covariant` static parameters, with each flag a covariant type argument: the compiled type checker accepts that form, and neither implementation carries covariance to run time yet (row 404 of the revival's gap ledger; item 12 of its worklist).

Effect None on the language or on either implementation.

Original text Unchanged.

Route C By the revival's reading, not measured, the same under route C (Section I.1.11).

I.1.15 Rational literals

Affected section Section 13.1.

Change $\mathbb{Q}$, $\mathbb{Q}^*$ and $\mathbb{Q}^\#$ become one type that includes $1/0$, $-1/0$ and $0/0$; the run-time error on assigning a $\mathbb{Q}^*$ to a $\mathbb{Q}$, the two ASCII names and the reference to Section 41.1 are removed, and the team's margin note is kept.

Rationale Section I.1.11.

Effect As Section I.1.11.

Original text `Specification-1.0-frozen/basic/expressions/literals.tex`, lines 188–209:

> The type $\mathbb{Q}$ includes all such rational numbers.
>
> The type $\mathbb{Q}^*$ relaxes the requirement $q > 0$ to $q \geq 0$ and includes two extra values, $1/0$ and $-1/0$ (sometimes called "the infinite rational" and "the indefinite rational"). [The margin note of lines 193–197, kept unchanged in the revised text.] If a value of type $\mathbb{Q}^*$ is assigned to a variable of type $\mathbb{Q}$, a $\mathrm{DivisionByZero}$ is thrown at run time if the value is $1/0$ or $-1/0$. The type $\mathbb{Q}^\#$ includes all of $1/0$, $-1/0$, and $0/0$. The advantage of $\mathbb{Q}^\#$ is that it is closed under the rational operations $+$, $-$, $\times$, and $/$. In ASCII, $\mathbb{Q}$, $\mathbb{Q}^*$, and $\mathbb{Q}^\#$ are written as `QQ`, `QQ_star`, and `QQ_splat`, respectively. See Section 41.1 for definitions of $\mathbb{Q}$, $\mathbb{Q}^*$, and $\mathbb{Q}^\#$.

Route C The same under route C.

I.1.16 Integers in floating-point expressions

Affected section Section 17.1.

Change The automatic conversion of integer values to floating-point values becomes that of $\mathbb{Z}32$ values and of integer numerals to $\mathbb{R}64$; any other integer type converts to $\mathbb{R}64$ explicitly.

Rationale Section I.1.11: an exact conversion is a coercion and one that can lose precision stays explicit, and the decision names $\mathbb{Z}32$ and the numerals.

Effect The paragraph's examples convert numerals and are unchanged, and Example 1 of Section 17.1, that a decimal integer literal argument may be used for any floating-point parameter, stands as written. A program converts a $\mathbb{Z}64$, $\mathbb{Z}$, $\mathbb{N}64$ or $\mathbb{N}32$ value to $\mathbb{R}64$ explicitly.

Original text `Specification-1.0-frozen/basic/conversions-coercions.tex`, lines 64–66:

> type is not a subtype of any floating-point type. Fortress supports the automatic conversion of integer values to floating-point values (the technical term for this kind of automatic conversion is coercion). In

Route C The same under route C, which keeps the coercions.

I.1.17 Reductions whose element type nothing fixes

Affected section Section 13.17.

Change One box after the first sentence says that neither implementation directs the desugaring of a reduction by type yet, so a reduction with a generator clause list whose element type no argument fixes writes its static argument; and that until the compiled path is built on the libraries this specification describes, it compiles against a smaller library of its own, whose big operators take no static argument and range over $\mathbb{Z}32$ only, so that there the clause form runs without its static argument and a written one is refused. The normative text is

unchanged. The box said at first that "the interpreter, and the compiled type checker when it checks the libraries this specification describes, each take $\mathrm{BottomType}$ for it, and the reduction then accepts no element"; it now says that the interpreter does, and that the compiled type checker takes the bound of the big operator's static parameter, which gives the reduction the type of a bound such as Number and no instance that applies where the bound mentions the static parameter itself. A revision of the interpreter of 9 October 2026 changes the box once more. It said that "When no argument of the big operator fixes the element type, the interpreter takes $\mathrm{BottomType}$ for it, and the reduction then accepts no element"; it now says that where the bound of the big operator's static parameter mentions the static parameter itself the interpreter leaves that parameter open and the reduction runs on the types of its elements, an empty one taking the identity of $\mathbb{Z}32$, and that until the compiled type checker directs the desugaring by type a program that both implementations are to run writes the static argument. A revision of the interpreter of 14 October 2026 changes the box once more. It said that "Neither implementation directs it by type yet", and that where the bound of the big operator's static parameter mentions the static parameter itself "the interpreter leaves that parameter open", the reduction running on the types of its elements "except that an empty one takes the identity of $\mathbb{Z}32$ whatever the type of the expression"; it now says that neither implementation directs it by the type N of the expression yet, that the interpreter, which has no static types, instantiates the big operator by the declared type of the local variable, or the declared return type of the top-level or local function, whose value the reduction is, otherwise by the join of the run-time types of its elements, and where there is none at the bound of its static parameter, or open where that bound mentions the parameter itself, and that an empty reduction that no declared type directs takes the identity of $\mathbb{Z}32$.

Rationale The revival decided on 26 September 2026 that the library's `SUM` and `PROD` become generic reductions over the library's algebra, each joined with its element type's own operator and taking that type's identity, and that such a clause form writes its static argument, as the team's own tests do. Until 9 October 2026 the interpreter took $\mathrm{BottomType}$ for the static parameter of a big operator that no argument fixes (row 424 of the revival's gap ledger). The revival decided on 8 October 2026 that the interpreter leaves open a type parameter whose bound mentions the type parameter itself and that nothing at a call fixes (Section I.1.26): having no static types, the interpreter cannot take the type N of the expression, and an open parameter refuses no element where $\mathrm{BottomType}$ refused every one, so that the reduction runs on the types of its elements, as the desugaring by type would run it. Reversing that revision takes one block out of the interpreter's instantiation of a call and returns such a parameter to $\mathrm{BottomType}$; the specification's examples that write no static argument for such a reduction would stop again at their first element, and refusing the call, as the compiled type checker does, would stop them too. The revival decided on 14 October 2026 that the interpreter instantiates the big operator of a reduction or a comprehension that writes no static argument as the desugaring by the type N would, with run-time types in place of N: by the declared type of the local variable, or the declared return type of the top-level or local function, whose value the reduction is, which constrains the instance as a call's static return type does (Section 20.1), and otherwise by the join of the run-time types of its elements, which the interpreter collects before it reduces them, as it already took the types of the elements for N where it left the parameter open; with neither, the parameter takes its bound, or is left open where the bound mentions it. Until then the interpreter left a big operator's static parameter whose bound does not mention it at $\mathrm{BottomType}$ (Section I.1.26), and an empty reduction, having no element, took the identity of $\mathbb{Z}32$. Reversing the revision of 14 October 2026 takes the declared type and the collection of the elements out of the interpreter's evaluation of such a reduction and returns those static parameters to $\mathrm{BottomType}$: an empty reduction of $\mathbb{R}64$ elements then takes the identity of $\mathbb{Z}32$ again, the library's set reductions $\bigcup$ and $\bigcap$ stop again at their first element, a comprehension is built again at the run-time types of its elements wherever its type is declared, and the library's *body* in the application of a big operator can no longer declare the type of its parameter. The reasoning is in `explorations/perf-probes/walk-plain-bound/REPORT.md` and `explorations/compile-ladder/rung-walk-elements/REPORT.md` in the revival's repository. The compiled type checker, when it checks the libraries this specification describes, took $\mathrm{BottomType}$ too until it was revised to the rule of Section 20.1 for a type parameter that nothing at a call fixes; it now takes the parameter's bound, which is not the element type, or, since 14 October 2026, where the reduction's type at that bound does not fit the reduction's expected type, that expected type, the big operator being a call written as an argument of another call (Section I.1.27) (row 425). The compiled path's own library declares

$\sum$ and `BIG MAX` for $\mathbb{Z}32$ alone, with no static parameter (`Library/CompilerLibrary.fsi`, lines 180–184), and refuses $\sum[\![\mathbb{Z}32]\!][j \leftarrow 0\#4]\, j$ (`explorations/compile-ladder/rung-spec-numbers/probes/skeptic/SkSumClauseTyped.t1.txt`).

Effect None beyond the box: the forms without a generator clause list need no static argument. Since 9 October 2026, under the interpreter, a reduction with a generator clause list whose element type nothing fixes, over a big operator whose bound mentions its static parameter, runs where it stopped at its first element, the specification's examples of that form among them. Since 14 October 2026, under the interpreter, such a reduction or comprehension is instantiated by the declared type of its value, by the join of its elements' run-time types or at its bound: an empty sum that is the value of a function declared $\mathbb{R}64$ gives the $\mathbb{R}64$ zero, where it gave the identity of $\mathbb{Z}32$ (row 645 of the revival's gap ledger), the library's set reductions $\bigcup$ and $\bigcap$ run where they stopped at their first element (row 662), and a comprehension that is the value of a local variable or of a top-level or local function declared with its type is built at that type. An empty reduction that no declared type directs still gives the identity of $\mathbb{Z}32$ whatever the type of the expression, and an empty comprehension is built at its big operator's bound (row 716); a comprehension that no declared type reaches, such as the value of a top-level variable, a field or a method, or the body of another reduction, is built at the run-time types of its elements (rows 20 and 717). Since 16 October 2026 a reduction over a big operator whose declared result is a trait other than the library's $\mathrm{BigReduction}$, $\mathrm{Comprehension}$ and $\mathrm{BigOperator}$ is instantiated through the one of those three that the trait extends, its static arguments put for the trait's, as one declared with that supertype is (row 718). The compiled type checker still refuses that form (row 425).

Original text Unchanged.

Route C The same under route C.

I.1.18 Initializing an array from a function

Affected section Section 23.4.

Change In Figure 23.1, the method that calls a function at each index and initializes the corresponding element with the result is named $tabulate$; *fill* keeps the method that initializes every element with one value.

Rationale This change does not follow from route A; the revival decided it on 26 September 2026. The library declares the element type E of an array without a bound and instantiates it at tuple types, which the implicit bound Object of Section 12.1 excludes; the interpreter reads such a parameter as bounded by Any, and so does the compiled type checker when it checks the library. E may then be Any or a function type, and a function passed to *fill* is a value of type E as well as a function from I to E: both declarations apply to the call and neither is more specific, which the overloading rules forbid (Section 24.3). The compiled type checker refuses the pair; the interpreter chose between the two by the argument's run-time type, so that on an array of element type Any a function meant as the value was called at each index instead of being stored. Where the library derived a second name, it gave it to a form that takes a function ($ivmap$ beside map); most languages that have both operations name them differently, keeping *fill* for the value; and $tabulate$ is Scala's name for the other.

Effect A program initializes an array from a function with $tabulate$. A call of *fill* with a function stores the function, on an array whose element type admits it. The library's factories that build an array from a function are renamed with the method; this specification does not list them.

Original text `Specification-1.0-frozen/advanced/parallelism-locality/arrays-distributed.tex`, line 55, a row of the lower table of the figure:

$a.fill(f : I \to E)$	Calls f at each index and initializes the corresponding element with the result of the call.

Route C The same under route C.

I.1.19 The integer type of a range

Affected sections Section 13.16; Section 33.2; Section I.1.11; Section I.1.43.

Change The components of an explicit range are values of type $\mathbb{Z}32$, into which an integer numeral converts, and it is a static error if a component has another integer type. A paragraph shows how a loop whose counter must have a wider integer type is written: over a range of $\mathbb{Z}32$ values, widening each element. The box in Section 33.2, the effect of Section I.1.11 and Section I.1.43 no longer say that a range over two different integer types infers the narrowest type that both coerce into; they keep that rule for a generic call.

Rationale This change does not follow from route A; the revival decided it on 27 September 2026. The Working Draft named no integer type for a range, and the Fortress standard libraries declared their ranges generic over an integer type named by a type parameter. The compiled type checker refused that code wherever it converted a numeral to the range's integer type, since a numeral does not convert to a type named by a type parameter (Section 17.3). The compiled path's own smaller library declares its ranges over $\mathbb{Z}32$ alone (`Library/CompilerLibrary.fsi`, lines 173–174), and on the Java Virtual Machine, which runs Fortress, an array holds at most $2^{31} - 1$ elements, so a range of $\mathbb{Z}32$ values indexes any of them. With every range over $\mathbb{Z}32$, no range is over two different integer types, and the rule that infers the narrowest type that both coerce into concerns a generic call alone. The full reasoning is in `explorations/compile-ladder/rung-spec-ranges/decision-record.md` in the revival's repository.

Effect A program writes a range over $\mathbb{Z}32$ where it wrote one over $\mathbb{Z}64$, $\mathbb{N}32$, $\mathbb{N}64$ or $\mathbb{Z}$, and widens its elements where it needs a wider counter; a range over another integer type is refused. Whether a `nat` parameter may be a component of a range, which this change left open, is settled by Section I.1.26: it converts to $\mathbb{Z}32$ as an integer numeral does.

Original text `Specification-1.0-frozen/basic/expressions/ranges.tex`, lines 42–44:

> An *explicit range* is self-contained and completely describes a set of integers. Assume that a, b, and c are expressions that produce integer values.

The box in Section 33.2 and two sentences of this appendix are the revival's own text of 27 September 2026, not the Working Draft's. The box said, in `Specification/basic-lib/basic-integers.tex`, lines 61–65, before this revision:

> A generic call or a range over two different integer types is to infer the narrowest type that both coerce into, by a rule to be written into Chapter 20 together with its implementation; until then such a call writes its static argument, as in $f[\![\mathbb{Z}64]\!](a, b)$.

The effect of Section I.1.11 said:

> A generic call or a range over two different integer types writes its static argument until the rule that infers the narrowest type both coerce into is written into Chapter 20 together with its implementation.

and Section I.1.43 said:

> The rule by which a generic call or a range over two different integer types infers the narrowest type that both coerce into is to be written into Chapter 20 together with its implementation (Section I.1.11).

Route C The same under route C.

I.1.20 Examples and a property with ranges over wider integer types

Affected sections Section 13.11; Section 23.8, Figure 23.2 and the generator adapted to a commutative monoid; Section 33.2.

Change The function $mySum$ of Section 13.11 takes a parameter of type $\mathbb{Z}32$ and keeps its sum of type $\mathbb{Z}64$. The generator of Figure 23.2, and the generator of Section 23.8 that adapts it to a commutative monoid, are declared over $\mathbb{Z}32$, and the figure's midpoint is $lo + (hi - lo) \div 2$. The factorial property of $\mathbb{Z}$ is stated by the value at 0 and a recurrence, without a range.

Rationale Each built a range over an integer type other than $\mathbb{Z}32$ (Section I.1.19): the loop of $mySum$ over $0 : i$ with i of type $\mathbb{Z}64$, the method seq of the figure's generator over $lo : hi$ with bounds of type $\mathbb{Z}64$, and the property's product over $1 : m$ with m of type $\mathbb{Z}$. The adapted generator builds no range in its own text, but its ellipsis stands for the figure's members. The figure's midpoint $\lfloor (lo + hi)/2 \rfloor$ is a value of type $\mathbb{Z}$ (Section 33.1), which converts to neither $\mathbb{Z}64$ nor $\mathbb{Z}32$ (Section 33.2), so the recursive construction of the generator did not accept it; the new midpoint is a $\mathbb{Z}32$ value equal to it whenever $lo \le hi$ and the generator's size $hi - lo + 1$ is a $\mathbb{Z}32$ value, and it does not form the sum $lo + hi$, which exceeds $\mathbb{Z}32$ when both bounds are large. The property of $\mathbb{Z}$ holds for every natural m; over a range of $\mathbb{Z}32$ values it would hold only below 2^{31} and need two conversions, so it is stated by a recurrence, as Section 33.2 states the binomial coefficient by Pascal's rule.

Effect None beyond the examples and the property.

Original text The two examples are typeset from the files `SpecData/examples/basic/Expr.Do.mySum.fss` and `SpecData/examples/advanced/Generators.GeneratorDefn.fss` of the revival's repository, identical in the commit of 2 February 2011 of which `Specification-1.0-frozen/` is a copy and unchanged until this revision; `Specification-1.0-frozen/basic/expressions/blocks.tex`, line 72, and `Specification-1.0-frozen/advanced/parallelism-locality/defining-generators.tex`, line 50, input them. As typeset from those files:

$mySum(i\colon \mathbb{Z}64)\colon \mathbb{Z}64 =$ `do`

$acc\colon \mathbb{Z}64 := 0$

`for` $j \leftarrow 0 : i$ `do`

$acc := acc + j$

`end`

acc

`end`

and

`object` BlockedRange($lo\colon \mathbb{Z}64, hi\colon \mathbb{Z}64, b\colon \mathbb{Z}64$) `extends` Generator$[\![\mathbb{Z}64]\!]$

$size : \mathbb{Z}64 = hi - lo + 1$

$seq($`self`$)\colon$ SequentialGenerator$[\![\mathbb{Z}64]\!] = seq(lo : hi)$

$generate[\![R]\!]\big(reduction\colon$ Reduction$[\![R]\!], body\colon \mathbb{Z}64 \to R\big)\colon R =$

`if` $size \le (b$ `MAX` $1)$ `then`

(⋆ Blocks smaller than b run sequentially ⋆)

$r : R := reduction.empty()$

$i : \mathbb{Z}64 := lo$

`while` $i \le hi$ `do`

$v : R = body(i)$

$r := reduction.join(r, v)$

$i \mathrel{+}= 1$

`end`

r

`else`

`(*` Blocks larger than b are split in half and generated in parallel. `*)`

$mid = \left\lfloor \frac{lo + hi}{2} \right\rfloor$

$reduction.join(\text{BlockedRange}(lo, mid, b).generate[\![R]\!](reduction, body),$

$\text{BlockedRange}(mid + 1, hi, b).generate[\![R]\!](reduction, body))$

`end`

`end`

$f() = \langle\, 2\,x \mid x \leftarrow \text{BlockedRange}(1, 10, 3) \,\rangle$

`Specification-1.0-frozen/advanced/parallelism-locality/defining-generators.tex`, lines 405, 407 and 411, the three lines of the adapted generator that named $\mathbb{Z}64$, quoted without their indentation and alignment commands:

`value object` $\text{BlockedRange}(lo\colon \mathbb{Z}64, hi\colon \mathbb{Z}64, b\colon \mathbb{Z}64)$ `extends` $\text{Generator}[\![\mathbb{Z}64]\!]$
$generate[\![R$ `extends` $\text{CommutativeMonoid}[\![R, \oplus]\!], \texttt{opr}\ \oplus]\!](body : \mathbb{Z}64 \to R) : R =$ `do`
$i : \mathbb{Z}64 = l$

`Specification-1.0-frozen/basic-lib/basic-integers.tex`, lines 559–561:

`property` $\forall(m)\ m! = \prod_{k \leftarrow 1:m} k$

Route C The same under route C.

I.1.21 The traits that extend a closed trait

Affected sections Section 10.1; Section 12.2, by the amendment of 9 October 2026.

Change The first sentence of a draft note in Section 10.1, which said that the traits listed in a `comprises` clause are exactly the traits that immediately extend the declared trait and that they must explicitly extend it, is replaced by normative text: every listed type extends the declared trait T, and every value of type T is a value of one of the listed types; a listed type may be an instantiation of a parameterized trait whose static arguments contain no static variable other than the static parameters of T; and a trait or object that explicitly extends T is a subtype of a listed type, or a trait with a `comprises` clause of its own each of whose listed types meets the same requirement, or a trait with static parameters that at least one trait or object extends and every trait or object extending it is a subtype of a listed type. The note's second sentence, on a clause that includes "...", is kept in its note, and the example of Molecule is unchanged. The new text answers the third question of Victor Luchangco's draft note on trait identifiers earlier in the section, whether an instantiation of a parameterized trait may be listed: it may. An amendment of 9 October 2026 lets the static arguments of a listed type contain a `where`-clause variable of the declaration of T as well, the listed type then holding at every value of the variable, and adds one sentence to Section 12.2: an argument whose type is a trait whose clause lists a type at a `nat` or

`int` `where`-clause variable, passed where that listed type is expected with a `nat` or `int` parameter in the variable's place, instantiates that parameter, for that call, with the size of the value itself, a size known to equal only itself.

Rationale This change does not follow from route A; the revival decided it on 28 September 2026. The team's later Types chapter (Luchangco, 2012) reads a `comprises` clause as coverage: the clause "specifies a set of types and generic types determined by the generic type of the declaration", and an instantiation of the declared generic type "is covered by the union of the types in this set and the corresponding instantiations of the generic type in this set", one generic type G determining another G' "if every parameter of G' is a parameter of G" (`Documentation/Specification/Prose/Language/types.tick`, lines 277–283 and 384–390, in the revival's repository). Welterweight Fortress, the calculus of the type group's paper of 2012 (Chase, Hilburn, Luchangco, Naden, Ryu, Steele, and Tristan, "Dynamic Dispatch and Type Inference Semipredicates"), reads it at the level of values: "no value can belong to the trait unless it also belongs to one of the comprised types"; its rule for a trait declaration asks nothing of the traits that extend a trait with such a clause (`Papers/Welterweight/grammar.tick`, lines 21–22; `Papers/Welterweight/fig-wellformeddecls.tick`, lines 38–62). The draft note restated the rule of the compiled type checker of 2009, under which the Fortress standard libraries could not close their marker of the integer types as the team meant to: $\mathrm{AnyIntegral}$ lists the five integer types, and the parameterized trait $\mathrm{Integral}[\![I]\!]$, which each of them extends at itself, stands between them unlisted, as the team's comment on the declaration records ("not yet: comprises $\mathrm{Integral}[\![I]\!]$ where $[\![I]\!]$", `Library/FortressLibrary.fsi`, line 434). $\mathrm{Integral}$ has a static parameter that $\mathrm{AnyIntegral}$ does not have, so it can be listed only at particular static arguments, and every value of it is a value of one of the five. The full reasoning is in `explorations/compile-ladder/rung-spec-comprises/decision-record.md` in the revival's repository. The amendment follows the revival's default for the third question of its array design, a size known only at run time, built on 9 October 2026 and listed for the curator's review. the Fortress standard libraries have carried on their trait $\mathrm{NatParam}$, since 2007, the clause "`comprises` $\{N[\![n]\!]\}$ `where` $[\![$`nat` $n]\!]$" as a comment, with the reading that a function taking an $N[\![n]\!]$ may be passed a $\mathrm{NatParam}$, and "within that function n becomes a static nat parameter" (`ProjectFortress/LibraryBuiltin/NatReflect.fss`, lines 15–26, in the revival's repository). The sentence of 28 September followed the Types chapter's rule by which one generic type determines another, which does not consider `where`-clause variables; Section 12.6 allows a declaration to introduce them. A size so opened is bound for the call, as inference binds a type parameter to an argument's type, and since no program can write its name it cannot be equal to a size written elsewhere. The team wrote the same form at a type `where`-clause variable as a comment on $\mathrm{AnyList}$ ("Not yet: comprises $\mathrm{List}[\![E]\!]$ where $[\![E]\!]$", `Library/List.fsi`, line 56, in the revival's repository); the revival decided on 16 October 2026 to build the forms of `where` clause the team wrote down, that comment among them. The full reasoning of the amendment is in `explorations/compile-ladder/rung-size-expressions/REPORT.md` and `explorations/reviews/array-fork3-judgement.md` in the revival's repository.

Effect The compiled type checker accepts the declaration of $\mathrm{AnyIntegral}$ in the Fortress standard libraries as written (`Library/FortressLibrary.fsi`, lines 433–436). No example of this specification changes: $\mathrm{ExclusiveMolecule}$, which has no static parameters, is still not allowed. The compiled type checker enforces the requirement on a trait with static parameters over the declarations and object expressions it sees: a trait or object declared in another component, below such a trait and a subtype of no listed type, is not refused (row 487 of the revival's gap ledger). It compares the declarations it sees by their names: for a parameterized T its verdict can depend on how an extender spells its static parameters, and a trait or object that extends a trait of the same simple name in another API counts as an extender (rows 489 and 490). It refuses an object expression that extends a trait with a `comprises` clause, explicitly or through the traits it extends, and is a subtype of no listed type, as the requirement on the trait's extenders together demands of it, an object expression having neither static parameters nor a clause of its own; before 3 October 2026 it read no object expression against a clause (row 597). The interpreter checks at load the traits and objects that explicitly extend a trait with a `comprises` clause declared in the component it executes, and refuses one that is neither a subtype of a listed type nor another of the extenders that Section 10.1 allows. It does not yet check a clause declared in another component, of the program or of the Fortress standard libraries (rows 22, 487, 551, 595 and 596), an object expression that

extends such a trait (row 597), or that each listed type extends T (row 598). By the amendment, the compiled type checker reads a `nat` or `int where`-clause variable as a size, reads a listed type at every value of such a variable when it checks the traits and objects that extend T, and opens an argument as the new sentence of Section 12.2 says, so that the value of *reflect* in the Fortress standard libraries is accepted where an $N[\![n]\!]$ is expected; it refuses an opened size that reaches a type naming another size, and gives two values opened in one call two sizes, so it refuses a call that needs them equal where the interpreter runs it when they are. It reads a declaration whose parameter type is such a trait and a declaration with a `nat` parameter whose parameter type is the listed type at it as applicable to the same calls, and refuses the pair as duplicates, where the interpreter runs it and calls the second; the text does not say whether the opening counts for the rules of Section 24.5 (row 682 of the revival's gap ledger). Since 17 October 2026 it opens a type `where`-clause variable in the same way, as in the clause of AnyList in the Fortress standard libraries (row 636): an argument whose type is such a trait, passed where the listed type is expected, takes the listed type at a type of the value's own for that call, bounded as the variable's binding bounds it and by each trait constraint of the clause on the variable (Section 12.6), and known to equal only itself; before 18 October 2026 it did not bound that type by such a constraint, and refused a call that needs the constraint's bound, a call the interpreter runs (row 771); it refuses an opened type that reaches a type naming another type, gives two values opened in one call two types, and reads a listed type at the types below the variable's bounds, its binding's and its constraints', when it checks the traits and objects that extend T, so that it refuses an extender that the clause lists only below a bound its static argument does not meet; before 18 October 2026 it read a listed type at every type of the variable, whatever its bound, and took such an extender (row 753). It does not open a trait whose clause lists a type at a type `where`-clause variable where it reaches that trait as a supertype of another type, so that a value of a trait that extends it, and is not itself listed, is not taken where a listed type is expected (row 752); it opens a trait whose clause lists a type at a size there, as before. It opens such a trait, of either kind, also where it is the result type of a function type, once for all the function's results, so it takes a function whose results are values of the trait where a function whose results share one listed type is expected, a call the interpreter refuses (row 751). The code generator does not yet compile a trait with a `where` clause or read a size off an argument at a call (rows 307 and 701). The interpreter binds a `nat` or `int where`-clause variable as a size, and a type `where`-clause variable as a type that stands for every type, when it loads the trait, reads a listed type at every value of either in its check at load, a type whatever its bound (row 754), and binds the size of an argument when it dispatches. It binds the type `where`-clause variable of a trait without static parameters in its component's environment, so that a static parameter of another declaration that has the variable's name and whose bound names it is read as the variable, and a call of that declaration fails (row 778). It refuses at load such a trait one of whose members has a declared type that names the variable, of either kind (row 755); the compiled type checker accepts that declaration, and stops with an internal error at a call, outside the trait, of such a member whose type names a type `where`-clause variable (row 756).

Original text `Specification-1.0-frozen/basic/traits.tex`, lines 230–241, a draft note:

> If a trait declaration of T includes a `comprises` clause then the traits listed in its `comprises` clause are exactly the traits that immediately extend T and they must explicitly extend T (i.e., list T in their `extends` clause). If the `comprises` clause of a trait T includes "..." (and must therefore be in an API), then in a component exporting the enclosing API, other traits may explicitly extend T, but these traits may not be declared or imported by the API (see Section 22.3).

Its second sentence is kept, in its note. The passage of Section 12.2 that the amendment adds to is unchanged: `Specification-1.0-frozen/basic/trait-parameters.tex`, lines 76–90.

Route C The same under route C, under which the self-typed traits carry `comprises` clauses of their own (Section I.1.44): a parameterized trait between a closed trait and its listed types then meets the requirement through its own clause, and the last case of the requirement is not needed. The amendment is the same under route C. A return to the text before it would put NatParam's clause back into a comment, as the Fortress standard libraries had it, and leave the compiled type checker refusing each call that passes the value of *reflect* where an $N[\![n]\!]$ is expected (row 664). Not opening a type `where`-clause variable would put AnyList's clause back into a comment, as the Fortress standard libraries had it, and leave the compiled type checker refusing the call in AnyList's concatenation operator that reads its two arguments as lists (row 636).

I.1.22 Shifts and narrowing on the integer types

Affected sections Section 33.2; the overview of operators in Chapter 16.

Change Two paragraphs after the coercions of Section 33.2 state the method *narrow* and the operators `LSHIFT` and `RSHIFT`, which this working draft did not name: *narrow* keeps the low 32 bits of a value of $\mathbb{Z}64$, $\mathbb{Z}$ or $\mathbb{N}64$; on a fixed-size type the two operators lose the bits shifted out, give 0, or -1 for a negative value shifted right, at a count at or beyond the width, and shift the other way for a negative count; on $\mathbb{Z}$ they are *shift*; the count may be of any integer type and is read by its value. The entry of *shift* adds that the result is exact for a count of any size and that a result too large for the implementation throws $\mathrm{IntegerOverflow}$, and that `LSHIFT` and `RSHIFT` compute the same shift on $\mathbb{Z}$. The overview of operators names `LSHIFT` and `RSHIFT` beside the bitwise operators.

Rationale This change does not follow from route A; the revival decided it on 22 September 2026, with two further decisions on 24 September: that the signed *narrow* truncates too, and that $\mathbb{Z}32$'s shifts take a count of any integer type. The Fortress standard libraries declare `LSHIFT` and `RSHIFT` on every integer type with a count of any integer type (`Library/FortressLibrary.fsi`, lines 459–460), and *narrow* on $\mathbb{Z}64$, $\mathbb{N}64$ and $\mathbb{Z}$ (lines 506, 594 and 608), and this working draft said nothing of them. In the languages the revival compared a shift is an operation on bits whose bits shifted out are lost, and the revival kept it one on the fixed-size types, with an exact arithmetic shift on $\mathbb{Z}$ under the name *shift*, which this working draft already defines. For a count at or beyond the width the rule is the one the team's interpreter has followed since 2007 and three of its gated tests assert (`ProjectFortress/tests/BitTwiddle.fss`, lines 33, 36 and 39; `ProjectFortress/tests/UnsignedTest.fss`, lines 30, 32, 135 and 137; `ProjectFortress/tests/QuickCheckTest.fss`, lines 97–103). Nothing pinned a negative count; it shifts the other way, and the count is read by its value, so that a large unsigned count is not read as a negative one. *narrow* keeps the low 32 bits as the team's own test asserts at the values $2^{64}-1$ and 2^{63} (`ProjectFortress/tests/UnsignedTest.fss`, lines 210–211), on the signed types as on the unsigned. The full reasoning is in `explorations/compile-ladder/rung-spec-integer-rules/decision-record.md` in the revival's repository.

Effect A count at or beyond the width, or a negative count, has a specified result, and so has *narrow* of a value that does not fit. Both implementations of the revival compute these results on $\mathbb{Z}32$ and $\mathbb{Z}64$ for a count of the shifted value's own type, and the interpreter on all five types for a count of any integer type; the compiled path, which until it is built on the Fortress standard libraries compiles against a smaller library of its own, declares `LSHIFT` and `RSHIFT` on $\mathbb{Z}32$ and $\mathbb{Z}64$ only, with a count of the shifted value's own type, and neither *narrow* on $\mathbb{Z}$ nor *shift* (rows 381, 383 and 442 of the revival's gap ledger).

Original text None for the two paragraphs and for the sentence of the overview: no chapter of this working draft names `LSHIFT`, `RSHIFT` or *narrow*. The entry of *shift*, `Specification-1.0-frozen/basic-lib/basic-integers.tex`, lines 698–702, which the revision keeps and extends:

> The result of $\mathit{shift}(x,k)$ is $\lfloor x \cdot 2^k \rfloor$. This corresponds to what is sometimes called an "arithmetic shift" on the two's-complement representation of the integer; positive values of k shifts the bits to the left, and negative values of k shifts the bits to the right.

Route C The same under route C.

I.1.23 The greatest common divisor and the least common multiple

Affected sections Section 33.2; the overview of operators in Chapter 16.

Change In the entry of `GCD` and `LCM`, the result for an argument 0 (of `GCD`) and for an argument 1 (of `LCM`) is the absolute value of the other argument, where it was the other argument, and the misspelling "argment" is corrected in those two sentences; the last property equates the product of the two results with $|m \cdot n|$. The

overview of operators adds that both results are nonnegative and that, on a fixed-size integer type, a result that the type cannot hold throws an $\mathrm{IntegerOverflow}$, with two examples on $\mathbb{Z}32$.

Rationale This change does not follow from route A; the revival decided it on 22 September 2026. Each paragraph of the entry begins by saying that the result is always nonnegative, and the source of its declarations gives both results the type $\mathbb{N}$ (`Specification-1.0-frozen/basic-lib/basic-integers.tex`, lines 518–519); the sentences that followed contradicted that when the other argument is negative, and the revival reads them as the absolute value, as both of its implementations compute. The overview of operators says that for integer results overflow throws an $\mathrm{IntegerOverflow}$; the revival applies that to `GCD` and `LCM`, as both of its implementations do on $\mathbb{Z}32$ and $\mathbb{Z}64$.

Effect 0 `GCD` -4 is 4, 1 `LCM` -5 is 5, and on $\mathbb{Z}32$ and $\mathbb{Z}64$ a greatest common divisor or a least common multiple that the type does not hold throws an $\mathrm{IntegerOverflow}$, on both implementations of the revival.

Original text `Specification-1.0-frozen/basic-lib/basic-integers.tex`, lines 523–529:

> The operator `GCD` computes the greatest common divisor of its two arguments. The result is always nonnegative. If either argment is 0, the result equals the other argument.
>
> The operator `LCM` computes the least common multiple of its two arguments. The result is always nonnegative. If either argment is 1, the result equals the other argument. If either argument is 0, the result is 0.

and line 544, the last property:

> `property` $\forall(m, n)\ (m$ `GCD` $n) \cdot (m$ `LCM` $n) = m \cdot n$

The overview of operators said nothing of either operator's sign or overflow.

Route C The same under route C.

I.1.24 The example of coercion resolution

Affected section Section 17.5.

Change In the example over $\mathbb{Z}32$, $\mathbb{Z}64$ and $\mathbb{Z}128$, $\mathbb{Z}32$ excludes the two others and $\mathbb{Z}64$ excludes $\mathbb{Z}128$; the sentence after it names the exclusion and the rejection from which $\mathbb{Z}64 \triangleleft \mathbb{Z}128$ follows, and says that the same facts make the example's two overloadings valid.

Rationale This change does not follow from route A. The example contradicted the definition before it: T is no less specific than U only if T is a subtype of U or excludes, coerces to and rejects U, and without the clauses $\mathbb{Z}64$ neither excludes $\mathbb{Z}128$ nor rejects it. The clauses added are those the definition and the Meet Rule of Section 24.4 need, spelled as the Fortress standard libraries spell the exclusions of their integer types, each type excluding the wider ones (`Library/FortressLibrary.fsi`, lines 510–511 and 552–553). The team's later Types chapter says the same of an `excludes` clause: an instantiation of the declared trait "excludes every type in this set" (`Documentation/Specification/Prose/Language/types.tick`, lines 344–351, in the revival's repository).

Effect The example is a valid program, and both implementations of the revival run it as printed, the call resolving to $f(\mathbb{Z}64)$; as the working draft printed it, both refuse it.

Original text `Specification-1.0-frozen/basic/conversions-coercions.tex`, lines 551–552, the first two lines of the example:

> `trait` $\mathbb{Z}32$ `end`
> `trait` $\mathbb{Z}64$

and lines 562–565:

> Then the call $f(\mathbb{Z}32)$ resolves to the declaration $f(\mathbb{Z}64)$ because $\mathbb{Z}64 \lhd \mathbb{Z}128$, which follows from the fact that $\mathbb{Z}64$ coerces to $\mathbb{Z}128$.

Route C The same under route C.

I.1.25 The algebra of the rational trait

Affected section Section 33.1.

Change The listing of $\mathbb{Q}$ extends $\mathrm{StandardPartialOrder}[\![\mathbb{Q}]\!]$, $\mathrm{StandardMinMax}[\![\mathbb{Q}]\!]$, $\mathrm{AdditiveGroup}[\![\mathbb{Q}]\!]$ and $\mathrm{MultiplicativeRing}[\![\mathbb{Q}]\!]$ besides Number, and one sentence after it says that their laws hold for the finite rationals and need not hold where $0/0$ or an infinity takes part.

Rationale The number chapters describe the number types that the Fortress standard libraries declare (Section I.1.11). The Fortress standard libraries declare these four traits on $\mathbb{Q}$ (`Library/FortressLibrary.fsi`, lines 387–388), and their sums and products over $\mathbb{Q}$ rely on the group and the ring (the bounds of its `SUM` and `PROD`, lines 1898–1909); the revision of the number chapters had left the listing extending Number alone, because $0/0$ breaks the laws of a group and a ring (Section I.1.12). The Fortress standard libraries declare the same four traits on $\mathbb{R}64$, whose NaN breaks them likewise (lines 291–292), and Section 40.3 already says that the algebraic properties of the rationals are obeyed exactly and those of floating-point numbers only approximately.

Effect None on programs, since the Fortress standard libraries already declare the traits; the listing agrees with the library's declaration.

Original text `Specification-1.0-frozen/basic-lib/numbers.tex`, lines 177–183, the header of the listing, also quoted in Section I.1.12:

> `trait` $\mathbb{Q}$
> `extends` $\{\mathbb{R}, \mathbb{Q}^*,$
> $\mathrm{Field}[\![\mathbb{Q}, \mathbb{Q}_{\neq}, +, -, \cdot, /]\!]$,
> $\mathrm{Field}[\![\mathbb{Q}, \mathbb{Q}_{\neq}, +, -, \times, /]\!]$,
> $\mathrm{Field}[\![\mathbb{Q}, \mathbb{Q}_{\neq}, +, -,$ `juxtaposition`$, /]\!]$,
> $\mathrm{TotalOrderOperators}[\![\mathbb{Q}, <, \leq, \geq, >,$ `CMP`$]\!]$,
> $\mathrm{PartialOrderAndLattice}[\![\mathbb{Q}, \leq,$ `MIN`, `MAX`$]\!]\}$

Before this revision the listing's header was `trait` $\mathbb{Q}$ `extends` Number.

Route C By the revival's reading, not measured, the same under route C.

I.1.26 The inference of a call's static arguments

Affected sections Chapter 20; Section 13.16; Section 33.2; Section I.1.19; Section I.1.43.

Change Chapter 20, which held a heading and two draft notes, states how the static arguments of a call that does not write them are inferred, for type parameters and for `nat` and `int` parameters. The declaration is chosen first, as Section 17.5 chooses it, a declaration with static parameters being applicable to a call if some instance of it within its bounds is, and compared on the parameter types it declares. Its static arguments are then inferred: a type, `nat` or `int` parameter that a parameter type mentions inside another type is fixed by the argument, and a type parameter that the return type mentions inside another type by the expected type; a type parameter that is the whole type of parameters takes the narrowest type that their arguments are substitutable for, which for number arguments is the narrowest type into which every one of them converts, and otherwise, since 29 September 2026, its bound (below); every argument is admitted by substitutability, the coercions the instance needs are inserted

statically, and the converted call dispatches by subtyping. A call written by tight juxtaposition, as a method invocation or as an operator application, has an expected type in the contexts the chapter lists, a variable or field declaration and an assignment to a variable among them, and is inferred again without it when its result then converts. A numeral whose conversions tie is read as a $\mathbb{Z}32$, or as a $\mathbb{Z}64$ or a $\mathbb{Z}$ by its value; any other tie among declarations applicable only with coercion is a static error. The chapter lists what it does not yet describe, the static arguments of a `bool`, `dim`, `unit` or operator parameter among them, keeps the second of the two draft notes, and has three boxes: on the Working Draft, the bound it assumes and the passages it leaves to the revision of the overloading chapters; on the interpreter; and on a type parameter that only the return type mentions and on the `bool` parameters that neither implementation infers. The ranges section says that a `nat` or `int` parameter used as a component converts to $\mathbb{Z}32$ as an integer numeral does, with a box. The box in Section 33.2 says that a generic call over two different integer types infers the narrowest type that both coerce into by the rule of Chapter 20. Section I.1.43 says that the rule is written and names the passages the chapter leaves unrevised, and it and the effect of Section I.1.19 say that the question of a `nat` parameter in a range is settled. On 29 September 2026 one sentence of the rule changed. Of a type parameter that is the whole type of parameters, where no narrowest candidate exists and the rule for numerals does not settle the tie, the chapter said that it "takes the union of the types of those arguments (Section 6.10), if its bounds permit that, and none of those arguments is converted"; it now takes the intersection of its upper bounds, its declared bound, Any if it has none, and what the expected type requires of the declared return type, none of those arguments converted, with a box. With the revision of the overloading chapters (Section I.1.29), the first box points to this entry's list of the cases in which the compiled type checker departs from the chapter and no longer names the passages that revision revised, and the box on the interpreter says to which type the interpreter binds such a type parameter. A later revision adds an item to the rule: a type parameter that neither an argument nor the expected type fixes, such as one that only the return type mentions, as its whole type, likewise takes the intersection of its upper bounds, and inference never instantiates a static parameter at $\mathrm{BottomType}$, with an example and a box. The list of what the chapter does not yet describe named instead "a static parameter that neither an argument nor the expected type fixes, such as a type parameter that only the return type mentions, as its whole type, or the element type of a reduction that only its generator clauses determine", and left open "whether inference may give such a parameter $\mathrm{BottomType}$"; it now names a `nat` or `int` parameter that nothing fixes, and the element type of a reduction that only its generator clauses determine, to which the rule gives the bound of the big operator's static parameter. The box after the list said that the compiled type checker binds such a type parameter to $\mathrm{BottomType}$, except that it refuses a call with no expected type when the parameter is declared `extends` Any, and that "whether inference may do so is the team's question, kept in the draft note below"; it now says that the checker follows the rule, what the compiled path's bound Object makes of it, what reaches a reduction's big operator and what the interpreter does, and that the rule answers the third item of the team's draft note, which the chapter keeps. A later revision of the interpreter changes both boxes on it. The box in Section 20.1 said that, where no argument's type is the narrowest, the interpreter "binds a type parameter that is the whole type of parameters to the narrowest named common supertype of the arguments' run-time types that the bounds permit", the bound only where the bound is that supertype; it now says that the interpreter gives such a type parameter its bound, Any if it has none, and does the same where the arguments that fix a type parameter inside other types have several minimal common supertypes, except a type parameter whose bound mentions a static parameter and one declared with several bounds that the interpreter cannot meet, which it binds as before to the narrowest named common supertype of the arguments' run-time types, a call that fixes one of the second kind failing where there is none; that when it chooses among declarations it passes over a declaration with static parameters whose arguments have several minimal common supertypes in favour of another applicable without coercion; and, until 3 October 2026, that it instantiates at $\mathrm{BottomType}$ a type parameter that an argument bounds only from above. The box after the list said that "The interpreter erases a type parameter that nothing at a call fixes, of a function, to $\mathrm{BottomType}$"; it now says that the interpreter instantiates such a type parameter at its bound, and one whose bound mentions other static parameters at that bound with their instances, except a type parameter that only the bound of a type parameter that the arguments fix mentions, which it instantiates at the arguments' type, and names the type parameters it still erases: one declared with several bounds that it cannot meet, one whose bound mentions the type parameter itself, a big operator's that a reduction or a comprehension does not write, and one that only the result of a function expression without a declared return type fixes. A revision of the

interpreter of 3 October 2026 changes the box in Section 20.1 once more. It said that the interpreter "instantiates at $\mathrm{BottomType}$ a type parameter that an argument bounds only from above, as a function argument bounds the type parameter of its parameter type"; it now says that the interpreter instantiates such a type parameter at the intersection of the upper bounds that its declared bound and the arguments place on it, with two examples, and still at $\mathrm{BottomType}$ one whose bound mentions a static parameter or that is declared with several bounds that it cannot meet, and that a call whose arguments place on such a type parameter two upper bounds that it cannot meet fails. A revision of the interpreter of 9 October 2026 changes the box after the list once more. It said that the interpreter "still erases to $\mathrm{BottomType}$ a type parameter whose bound mentions the type parameter itself, as the big operators $\sum$ and $\prod$ declare theirs, and the static parameters of a big operator that a reduction or a comprehension invokes without its static arguments"; it now says that the interpreter leaves open a type parameter whose bound mentions the type parameter itself and that nothing at a call fixes, admitting every value where it checks a value against that type, what that gives a reduction whose element type nothing fixes, and that it still erases the other static parameters of such a big operator. A revision of the interpreter of 14 October 2026 changes both boxes on it once more. The box in Section 20.1 said that the interpreter "does not yet fix a type parameter by the expected type"; it now says that it does so only at a reduction or a comprehension that writes no static argument and whose value is that of a local variable declared with a type or of a top-level or local function declared with a return type. The box after the list said that the interpreter "still erases to $\mathrm{BottomType}$ the other static parameters of a big operator that a reduction or a comprehension invokes without its static arguments, a comprehension's collection being then built at the type of its elements", and that an empty reduction whose element type nothing fixes takes the identity of $\mathbb{Z}32$ and the library's set reductions stop at their first element; it now says that the interpreter instantiates such a big operator by the declared type of the local variable, or the declared return type of the top-level or local function, whose value the reduction or the comprehension is, otherwise by the join of the run-time types of its elements, and where there is none at its bound, or open where the bound mentions the parameter, and names what that leaves: an empty reduction or comprehension that no declared type directs, a comprehension that no declared type reaches, and a big operator whose declared result is not one of the library's three. A revision of the interpreter of 17 October 2026 changes both boxes on it once more. The box in Section 20.1 said that the interpreter compares declarations with static parameters on the parameter types they declare, "except generic dotted methods"; it now says that the interpreter infers the static arguments of a generic dotted method as it infers a generic function's, alone under its name or among the overloads of its receiver, and compares such a method on the parameter types it declares only where exactly one of two declarations' declared parameter types lies inside the other's, says that it compares a functional method of a generic trait or object on the instance that a call makes, names the call of an inherited generic functional method that finds no overload, names the two forms of generic dotted method on which it still stops, and says that it instantiates one that a trait declares without reading its bounds where they are declared. The box after the list said that the interpreter "leaves a generic method's static parameters uninstantiated"; it now says that it instantiates one that only the expected type would fix at its bound, as it does one of a generic function, except a bounded one of a method that a trait declares, and that it stops at load on an object or trait whose method has a parameter type with a `bool` static argument.

Rationale This change does not follow from route A. The revival decided on 26 September 2026 that a generic call over two different integer types infers the narrowest type that both coerce into, by a rule written into this chapter; on 27 September 2026 that the compiled type checker infers static arguments with coercion and keeps the expected type at a call written as $f(x)$, with a retry without it so that a coercion of the result still applies, and that the chapter is written with it; and on 28 September 2026 that a coercion makes a call possible and never changes which declaration runs when one is applicable without it, the declaration chosen being then instantiated by that rule, that a numeral whose conversions tie is read as a $\mathbb{Z}32$, and that a `nat` parameter used as a value converts to $\mathbb{Z}32$ as a numeral does. Chapter 15, Section 13.2 and Section 13.4 said that static arguments are inferred as Chapter 20 describes, and it described nothing; once the compiled type checker infers them by a rule the text does not state, the text and the implementation disagree, so the chapter states the rule the checker was built to, and the effect below lists where the checker departs from it. The bound in place of the union is the rule by which the dispatcher of the type group's paper of 2019 instantiates a type parameter that the arguments do not fix, the intersection of its upper bounds, never $\mathrm{BottomType}$; the team had assumed the union of the

lower bounds, of which the authors write "we were unable to complete the proofs of correctness" (Park, Hong, Steele and Ryu, "Polymorphic Symmetric Multiple Dispatch with Variance", POPL 2019, Section 5.4). The paper gives no algorithm for static inference and asks of it only that it never instantiate at $\mathrm{BottomType}$; its one example of a static call is typed at the union of its arguments' types. The revival decided on 29 September 2026 to take the paper's rule for static inference as well, which the paper allows and does not require. The same rule gives a type parameter that neither an argument nor the expected type fixes the intersection of its upper bounds: the paper's solving step "never instantiates method type parameters with Bottom" (Section 5.2), which answers the question the Working Draft's own note asks, whether type inference may infer $\mathrm{BottomType}$ for a static parameter. The reasoning for that item, and for the order in which the compiled type checker tries a call, is in `explorations/compile-ladder/rung-instance-bound/decision-record.md` in the revival's repository. The revival's decision of 29 September 2026 provided that the interpreter follow the same rule where it instantiates a declaration, from the run-time types of the arguments; it has no static return type to add to the bound, except, since 14 October 2026, the declared type of a local variable or of a top-level or local function whose value is a reduction or a comprehension that writes no static argument (below). It departs from the rule for the type parameters that the two boxes name, three kinds by decision and the others as defects. Until 9 October 2026, by decision, a type parameter whose bound mentions the type parameter itself kept $\mathrm{BottomType}$: it has no bound to take before its own instance is known, the paper's calculus admits no type parameter of a method in an upper bound (p. 11:11), and the revival had not decided that case. The revival decided on 8 October 2026 that the interpreter leaves such a type parameter open where nothing at a call fixes it: it does not instantiate it, and it admits every value where it checks a value against that type. An open type parameter is no instance: the rule that inference never instantiates a static parameter at $\mathrm{BottomType}$ keeps a call from an instance that admits no value, and an open parameter refuses none. The reasoning is in `explorations/reviews/p1-judgement.md` in the revival's repository. Until 14 October 2026, by decision, a big operator's static parameter whose bound does not mention it, that a reduction or a comprehension does not write, kept $\mathrm{BottomType}$: it is the element type of the reduction, which this chapter does not yet describe, and from $\mathrm{BottomType}$ the interpreter's library builds a comprehension's collection at the type of its elements, where from the bound it would build it at the bound. The revival decided on 14 October 2026 that the interpreter instantiates such a big operator as the desugaring by the type N of the expression would (Section I.1.17), with run-time types in place of N: by the declared type of the local variable, or the declared return type of the top-level or local function, whose value the reduction or the comprehension is, which is the static return type that the rule above lets constrain the instance, and otherwise by the join of the run-time types of the elements that its generator clauses give, which fix the parameter as N would and so are not the value's type of a parameter that nothing fixes; where there is neither, the parameter takes its bound, or is left open where the bound mentions it. The reasoning is in `explorations/perf-probes/walk-plain-bound/REPORT.md` and `explorations/compile-ladder/rung-walk-elements/REPORT.md` in the revival's repository. By decision, a type parameter that only the bound of a type parameter that the arguments fix mentions takes the arguments' type: the interpreter has no expected type to fix it, and the library's operator that appends two covariant collections builds its result at such a type parameter, which at its bound would be Any. By decision, a lone type parameter whose bound mentions a static parameter, itself or another, keeps the narrowest named common supertype of the arguments rather than its bound: the interpreter cannot evaluate such a bound before the instances are known, and keeping the candidates it had leaves unchanged the case of a type parameter whose bound mentions itself and that the arguments fix, which the revival has not yet decided. As defects, each gated by an expected failure of the revival's tests, the interpreter erases to $\mathrm{BottomType}$ a type parameter declared with several bounds that it cannot meet, having no intersection type, and a type parameter that only the result of a function expression without a declared return type fixes, since it gives that result the type $\mathrm{BottomType}$. Until 3 October 2026 it also erased a type parameter that an argument bounds only from above, taking the lower end of the interval that the arguments give the type parameter; it now takes the upper end, the intersection of the upper bounds that the rule gives, except, as defects gated by expected failures, for a type parameter whose bound mentions a static parameter, which it cannot evaluate before the instances are known, and one declared with several bounds that it cannot meet. The reasoning is in `explorations/compile-ladder/rung-walk-instance/decision-record.md` in the revival's repository. The order, a declaration chosen and then instantiated, is that of Section 17.5, read on the

parameter types that declarations declare, as the type group's paper of 2011 reads a declaration with static parameters: it "is applicable to a type if and only if at least one of its instances is" (`Papers/Types/setup.tick`, lines 396–397, in the revival's repository). The numeral's default is what the interpreter did before this change, giving a numeral the narrowest of $\mathbb{Z}32$, $\mathbb{Z}64$ and $\mathbb{Z}$ that holds its value, the Fortress standard libraries declaring their numeral type below $\mathbb{Z}32$; the compiled path, whose library declares its numeral type beside $\mathbb{Z}32$ and excluding it, took one of the declarations that a numeral left tied, and not always the same one (row 391 of the revival's gap ledger). It is also the Java programming language's type for an integer literal. The compiled type checker gives a `nat` or `int` parameter used as a value the type of a numeral, and the interpreter makes it a $\mathbb{Z}32$, so an `int` parameter is read as a `nat` parameter is; on the Java Virtual Machine the length and the indices of an array are 32-bit integers. The team's later Types chapter (Luchangco, 2012) gives the coercion relation, the coercion of a generic type and the valid instantiations of a quantified type (`Documentation/Specification/Prose/Language/types.tick`, lines 938–947, 393–400 and 771–789, in the revival's repository) and never uses the word "infer"; the type group's paper leaves the run-time inference of the static arguments of a dispatched declaration "beyond the scope of this paper" (`Papers/Types/rules.tick`, lines 118–129). The full reasoning is in `explorations/compile-ladder/rung-spec-inference/decision-record.md` in the revival's repository.

Effect A generic call over two different integer types, or over a number type and a numeral, need not write its static argument, and the interim rule of the effect of Section I.1.11 ends. A declared type fixes a type parameter that the return type of a call written as $f(x)$ mentions inside another type, and a call whose result the declared type converts is accepted as before. Where the conversions of a numeral tie, the $\mathbb{Z}32$ declaration or instance is taken; where another argument's conversions tie, the call is a static error. A `nat` or `int` parameter may be a component of a range, as in the specification's own examples. The overloading chapters compare declarations on the parameter types they declare and infer static arguments after the selection, and Section 12.1 gives a type parameter without an `extends` clause the bound Any (Section I.1.29, Section I.1.30). A type parameter that nothing at a call fixes is instantiated at its bound under the expected type and never at $\mathrm{BottomType}$: a call whose compiled run failed loading an instance over $\mathrm{BottomType}$, or failed verification where the call's result was carried as one, now runs; a call with no expected type whose type parameter is declared `extends` Any is accepted; and a call where the bound and the expected type have no common subtype, such as a type parameter bounded by Object where $()$ is expected, is a static error unless the result of the instance at the bound converts to the expected type. The compiled type checker departed from the chapter in five cases, each gated by an expected failure of its tests, until it was revised with the item above: the expected type could let a declaration applicable only with coercion be chosen over a declaration with static parameters that is applicable without it (row 508 of the revival's gap ledger); a call that the rule instantiates at the bound, with another argument converted, was refused (row 515); a type parameter declared without an `extends` clause, which has no narrowest candidate, was instantiated at the union of the arguments' types where the chapter gives Any (row 516); an argument that is an instance at a union, which only the checker's own union made, was refused where it fixes a type parameter (row 518); and an argument whose static type is an intersection, which the typing of a call between two traits with `comprises` clauses gives (Section I.1.34), fixed a type parameter at one of its conjuncts (row 541). It now follows the chapter in each, trying a call by subtyping without the expected type first and keeping that attempt when its result converts to the expected type, and the five tests are plain tests of the rule. The compiled path's run time does not yet load an instance at an intersection type (row 559), which the rule gives by two routes: an argument whose static type is an intersection, as in the last of them, and a type parameter that nothing at a call fixes whose declared bound and expected type are two traits that meet at their intersection. A program that the compiled path ran before this revision, at one conjunct of such an intersection or at $\mathrm{BottomType}$, now dies loading that instance. A written bound, such as Number, is taken by both implementations where the chapter gives it. Under the interpreter, a type parameter that nothing at a call fixes runs at its bound where it ran at $\mathrm{BottomType}$, and a type parameter with no narrowest candidate at its bound where it ran at the arguments' narrowest named common supertype, except the type parameters that the boxes on the interpreter name; a call whose arguments have several minimal common supertypes runs where it stopped the interpreter, unless they fix a type parameter declared with several bounds that the interpreter cannot meet or one whose bound mentions the

type parameter itself. Since 3 October 2026, under the interpreter, a type parameter that an argument bounds only from above runs at the intersection of its upper bounds where it ran at $\mathrm{BottomType}$, except the type parameters that the box in Section 20.1 names; a call whose arguments place on such a type parameter two upper bounds that the interpreter cannot meet, as functions of two traits neither of which extends the other do, still fails with an error of unification, as it failed before (row 616 of the revival's gap ledger). Since 9 October 2026, under the interpreter, a type parameter whose bound mentions the type parameter itself and that nothing at a call fixes is left open where it was instantiated at $\mathrm{BottomType}$, so that a reduction whose element type nothing fixes runs on the types of its elements where it stopped at its first element (Section I.1.17); until 14 October 2026 an empty such reduction took the identity of $\mathbb{Z}32$ whatever the type of the expression (row 645), and the library's set reductions $\bigcup$ and $\bigcap$, whose reductions take sets of the open type, stopped (row 662). Since 14 October 2026, under the interpreter, the big operator of a reduction or a comprehension that writes no static argument is instantiated by the declared type of its value, by the join of its elements' run-time types or at its bound, where its static parameters with a bound that does not mention them were instantiated at $\mathrm{BottomType}$: an empty sum that is the value of a function declared $\mathbb{R}64$ gives the $\mathbb{R}64$ zero, the set reductions run, and a comprehension bound to a local variable declared with its type is admitted; an empty reduction that no declared type directs still takes the identity of $\mathbb{Z}32$, and an empty comprehension is built at its big operator's bound (row 716); a comprehension that no declared type reaches is built at the run-time types of its elements (rows 20 and 717). Since 16 October 2026 a big operator whose declared result is a trait other than the library's three is instantiated through the one of them that the trait extends, as one declared with it is (row 718). Since 17 October 2026, under the interpreter, a dotted call that writes no static argument to a generic method infers the method's own static arguments from the run-time types of the arguments, as a call of a generic function does, where it stopped with an error of unification, or found no matching overload where the receiver overloads the method's name (row 21): a generic method that a trait declares and an object inherits beside a plain overload of its own is chosen where the plain one does not apply. A generic functional method that a trait declares and an object inherits beside a plain functional method of its own was not chosen where only it applies: the call found no overload (row 765), until 18 October 2026. It still stops at load on an object or trait that declares a generic dotted method beside a plain one of the same name (row 760) and on an object or trait whose method has a parameter type with a `bool` static argument (row 761); it stops on an object that extends a generic trait whose dotted method has a static parameter of its own bounded by the trait's, at load or at a generic object's first instance (row 762); and a dotted call that writes static arguments to a method its receiver overloads still stops (row 744). Until 18 October 2026 it instantiated a generic dotted method that a trait declares without reading the bounds where they are declared: a type parameter that is the whole type of parameters took the narrowest named common supertype of the arguments' run-time types, none of them converted, where the same method declared in an object takes the narrowest type into which they convert, and a bounded type parameter that nothing fixes took $\mathrm{BottomType}$ (row 766). Since then it reads the bounds of a generic method that a trait declares, and the types into which the arguments convert, where the trait declares the method, as for a method that an object declares, and so it gives such a method the instance that it gives the same method declared in an object, and chooses the inherited generic functional method above where only it applies.

Original text `Specification-1.0-frozen/basic/inference.tex`, lines 12–27, which held after the chapter's heading two draft notes:

> This chapter will include the Fortress static type inference mechanism.

and the note that the chapter keeps:

- There seems to be a circular dependency between inference and juxtaposition disambiguation – you have to know if the subexpressions have arrow types to disambiguate, but you can't do that without using inference, which requires knowing how the variable of unknown type is used.
- Type Inference (Dan's email titled "Comments on a first reading of the spec" on 06/05/07)
- Do we want to forbid such cases where type inference infers $\mathrm{BottomTypes}$ for static parameters?

The other passages this change revises are the revival's own text of 27 and 28 September 2026, not the Working Draft's. The ranges section said, in `Specification/basic/expressions/ranges.tex`, lines 46–48, before this revision:

> An integer numeral converts to $\mathbb{Z}32$ (Section 13.1), and it is a static error if a component of an explicit range has an integer type other than $\mathbb{Z}32$.

The box in Section 33.2 ended, in `Specification/basic-lib/basic-integers.tex`, lines 90–94:

> A generic call over two different integer types is to infer the narrowest type that both coerce into, by a rule to be written into Chapter 20 together with its implementation; until then such a call writes its static argument, as in $f[\![\mathbb{Z}64]\!](a, b)$.

The effect of Section I.1.19 ended:

> Whether a `nat` parameter may be a component of a range is not settled (Section I.1.43).

and Section I.1.43 said:

> The rule by which a generic call over two different integer types infers the narrowest type that both coerce into is to be written into Chapter 20 together with its implementation (Section I.1.11).

and

> Whether a `nat` parameter, which may appear wherever a variable of type $\mathbb{N}32$ can (Section 12.2), may be a component of a range is not settled (Section I.1.19); the static ranges of Section 13.16 and the examples whose ranges a `nat` parameter bounds are not revised.

Route C The same under route C.

I.1.27 The contexts that give a call an expected type

Affected sections Section 20.1; Section 20.2.

Change The list of the contexts in which a call has an expected type, in Section 20.1, gives the last expression of a block the block's expected type whether or not local declarations precede it, and gains two contexts: a clause of an `if` expression without an `else` clause, whose expected type is $()$, and a clause of a `typecase` expression that has an expected type, its `else` clause among them, whose expected type is that of the `typecase` expression. The sentence after the list says that the expected type is used also for a call written by loose juxtaposition, as $f\ x$. A second revision adds a third context to the list: the body of a `label` expression that has an expected type, and the `with` clause expression of each `exit` expression whose target is that `label` expression, whose expected type is that of the `label` expression. A third revision adds a fourth: an argument of a call that the section does not accept with its arguments typed without an expected type, where the argument is itself a call with inferred static arguments, whose expected type is the parameter type at its position of each declaration of the enclosing call in turn, instantiated, where that declaration has static parameters, by the arguments not so typed and the expected type of the enclosing call; and a paragraph after the list says that the enclosing call then invokes, of the declarations that are applicable to it with its arguments so typed and whose return types fit its expected type, the one more specific than every other on the parameter types they declare, and is otherwise typed as before, so that a declaration that the section accepts for the call as its arguments are typed without an expected type is never passed over. The item of the list of what the chapter does not yet describe, in Section 20.2, names the body of a `for` loop, and no longer names the contexts that the list of contexts now gives, the body of a `label` expression and an argument of another call among them. Each changed passage has a box. How a call is instantiated under its expected type is unchanged.

Rationale This change does not follow from route A; it follows the revival's decision of 8 October 2026 that these four contexts give a call the type that the text already requires of them, a clause of a `typecase` expression by

the rule for the type of the `typecase` expression. The text gives each context its type. "If there is no `else` clause in an `if` expression, then every clause must have type $()$" (Section 13.19). "If the last element of the expression block is an expression, the value and type of this expression are the value and type of the expression block as a whole" (Section 13.11), whatever local declarations precede it. The static arguments of an identifier reference "are statically inferred from the context of the function call" (Section 13.2), however the call is written. "The type of a `typecase` expression is the union of types of all right-hand sides of the typecase clauses" (Section 13.22), which is a subtype of the expected type when the type of each clause is. Since the revival's decision of 29 September 2026 a type parameter that nothing at a call fixes takes the intersection of its upper bounds, never $\mathrm{BottomType}$ (Section I.1.26), so a call of $fail$ or $builtinPrimitive$ of the Fortress standard libraries, whose type parameter only the result mentions, took the bound Any in these contexts, and the compiled type checker refused thirteen such calls of the Fortress standard libraries that the interpreter runs (row 560 of the revival's gap ledger). The alternative not taken was that the library write the static argument at each such call, as $fail[\![()]\!](s)$: the revival decided that the library's own practice is the standard, and the library does not route round a gap of the type checker. A reversal would take the four contexts out of the list of contexts and back into the item, and the compiled type checker would refuse those calls again. The second revision follows the revival's decision of 9 October 2026 that the body of a `label` expression and the `with` clause values of its exits take the expected type of the whole `label` expression, as the clauses of a `typecase` expression take that of the `typecase` expression. "The type of a `label` expression is the union of the type of the last expression of its expression block and the types of the `with` clause values of any `exit` expressions within the `label` expression whose target is the `label` expression's name" (Section 13.12), which is a subtype of the expected type when each of those types is. The thirteenth of the calls of the Fortress standard libraries above ends a clause of a `typecase` expression that ends the body of a `label` expression, and took the bound Any there (row 642 of the revival's gap ledger). A reversal of the second revision would take the body of a `label` expression and its exits' `with` clause values out of the list of contexts and back into the item, and the compiled type checker would refuse that call again. The third revision follows the revival's decision of 9 October 2026 that a call written as an argument of another call is typed again with the parameter type of each declaration of the enclosing call where the enclosing call is not accepted otherwise, so that an expected type fixes static arguments by one rule in every context. The static arguments of an identifier reference "are statically inferred from the context of the function call" (Section 13.2), and the context of a call written as an argument is the parameter to which it is passed; since a declaration is chosen on the parameter types it declares (Section 20.1), each declaration of the enclosing call gives the argument its own expected type. The compiled type checker typed every argument without an expected type before it chose a declaration, so that a type parameter of the argument that only its return type mentions took its bound, and one that only a numeral fixes took the numeral's own type: for the declarations $mk[\![T]\!]():\mathrm{Box}[\![T]\!]$, $wrap[\![T]\!](x:T):\mathrm{Box}[\![T]\!]$ and $takes(b:\mathrm{Box}[\![\mathbb{Z}64]\!])$, it refused $takes(mk())$ and $takes(wrap(3))$ (row 455 of the revival's gap ledger). The argument is typed again only where the enclosing call is not accepted otherwise, as a coercion is applied only to make a call possible (Section 20.1), so that a declaration that the section accepts for a call as its arguments are typed without an expected type is never passed over. The alternative not taken was to type again only an argument whose own declaration has a type parameter that its parameter types do not mention, which accepts $takes(mk())$ but not $takes(wrap(3))$, whose type parameter the numeral fixes. A reversal of the third revision would take the argument of a call out of the list of contexts and back into the item, and take out the paragraph after the list, and the compiled type checker would refuse those calls again. The full reasoning is in `explorations/compile-ladder/rung-checker-expected-type/REPORT.md` and, for the second revision, in `explorations/compile-ladder/rung-checker-contexts/REPORT.md`, and, for the third, in `explorations/compile-ladder/rung-argument-context/REPORT.md` and `explorations/reviews/argument-context-probe.md` in the revival's repository.

Effect The compiled type checker gives these contexts their expected types: it checks a clause of an `if` expression without an `else` clause against $()$, and it gives the expected type of the enclosing expression to the last expression of a block after local declarations, to a clause of a `typecase` expression, to the body of a `label` expression and the `with` clause expressions of its exits, and to the application that a loose juxtaposition makes. The thirteen calls of the Fortress standard libraries that it refused now check. It gives an argument of another call the parameter type of each declaration of the enclosing call in turn, as the list of contexts and the paragraph after it say,

where the argument is a call with static arguments and no attempt at the enclosing call with its arguments typed without an expected type is accepted; an argument that writes its static arguments it types again too, to the same type, and such an argument fixes the static arguments of a generic declaration of the enclosing call as any other argument does. Until 14 October 2026 it gave none there (row 455 of the revival's gap ledger); it now accepts $takes(mk())$ and $takes(wrap(3))$ above, and such an argument of a generic declaration, of a method, of an operator or of a constructor, at any depth of nesting. Where two declarations, neither more specific than the other, accept the enclosing call with the argument so typed, it reports the call as not applicable to the argument typed without an expected type, as the paragraph after the list has it, and not as ambiguous. A reduction whose big operator's static argument is not written, which the reduction's desugaring makes a call written as an argument of another call (Section 13.17), so takes the reduction's expected type where the big operator's instance at its bound gives the reduction a type that does not fit it; it is refused as before where the big operator has no instance at its bound, as one whose bound mentions its own static parameter has none (row 425). It gives the expected type also to an operator repeated between three or more operands, as $p \oplus q \oplus r$: to the multifix application where a multifix declaration accepts the operands, and otherwise to the outer of the binary applications $(p \oplus q) \oplus r$ that it stands for, each an operator application, which the sentence after the list gives the expected type; whether the multifix application applies is decided without the expected type, as for a loose juxtaposition. Until 9 October 2026 it gave none there (row 644). It gives the expected type also to the `juxtaposition` operator application that a tight juxtaposition of items none of which is a function stands for (Section 16.8), as $a(b)$ where a is not a function: to the multifix application where a multifix declaration accepts the items, and otherwise to the outermost of the left-associated binary applications that it stands for, as to $(a(b))(c)$ for $a(b)(c)$, each an operator application, which the sentence after the list gives the expected type; whether the multifix application applies is decided without the expected type, as for a loose juxtaposition. Until 9 October 2026 it gave none there (row 660); the reasoning is in `explorations/compile-ladder/rung-array-bound/REPORT.md` in the revival's repository. The interpreter has no static types and is unchanged; it fixes no type parameter by an expected type (Section 20.1), so that $takes(mk())$ and $takes(wrap(3))$ fail there when they run, mk instantiated at the bound of its type parameter and $wrap$ at the numeral's type, and a call that another declaration of the enclosing call accepts with the argument so instantiated runs that declaration (row 705 of the revival's gap ledger).

Original text The two passages are the revival's own text of 29 September 2026, not the Working Draft's, whose chapter held a heading and two draft notes (Section I.1.26); the paragraph after the list of contexts is new in the third revision. The list of contexts said, in `Specification/basic/inference.tex`, lines 129–139, before this revision:

> A call has an expected type in these contexts: as the right-hand side of a variable or field declaration that declares its type, that type; as the right-hand side of an assignment to a variable whose type is declared, that type; as the body of a functional that declares its return type, that type; as the last expression of a block that has an expected type, where no local declaration precedes it in the block, the block's expected type; and as a branch of an `if` expression with an `else` clause that has an expected type, the expected type of the `if` expression. The expected type is used whether the call is written by tight juxtaposition, as $f(x)$ (Section 16.8), or as a method invocation or an operator application.

The item of the list of what the chapter does not yet describe said, in lines 254–258:

> an expected type in any context other than those listed above, such as an argument of another call, an expression that a local declaration precedes in a block, an `if` expression without an `else` clause, a `typecase` expression, the body of a `for` loop, or a call written by loose juxtaposition, as $f\ x$;

Before the second revision, the list of contexts ended, in lines 138–142:

> . . . ; and as a clause of a `typecase` expression that has an expected type, its `else` clause among them, the expected type of the `typecase` expression, of which the union of the types of its clauses, the type of the `typecase` expression (Section 13.22), is then a subtype where the type of each clause is.

and the item said, in lines 266–268:

> an expected type in any context other than those listed above, such as an argument of another call, the body of a `for` loop or the body of a `label` expression;

Before the third revision, the list of contexts ended, in lines 143–148:

> . . . ; and as the body of a `label` expression that has an expected type, or as the `with` clause expression of an `exit` expression whose target is that `label` expression, the expected type of the `label` expression, of which the union of the type of its body and the types of those `with` clause values, the type of the `label` expression (Section 13.12), is then a subtype where each of those types is.

and the item said, in lines 277–278:

> an expected type in any context other than those listed above, such as an argument of another call or the body of a `for` loop;

Route C The same under route C.

I.1.28 The single-precision floating-point type

Affected sections Section 33.1; Section I.1.11.

Change The passage that names the number types names $\mathbb{R}32$ (`RR32`) beside $\mathbb{R}64$, a sibling of the others under Number, and says that $\mathbb{R}64$ coerces from $\mathbb{R}32$. The effect of Section I.1.11 no longer says that the library's $\mathbb{R}32$ stays below $\mathbb{R}64$.

Rationale The Fortress standard libraries declared $\mathbb{R}32$ a value object extending $\mathbb{R}64$, so that a value of $\mathbb{R}32$ was also a value of $\mathbb{R}64$, which Section 6.9 excludes: "These types are mutually exclusive; no value has more than one of them." The team's later Types chapter (Luchangco, 2012) keeps that sentence (`Documentation/Specification/Prose/Language/types.tick`, lines 977–978, in the revival's repository). Route A makes the number types siblings under Number, each with its own algebra, and the revival decided on 26 September 2026 that a conversion between number types is a coercion where it is exact; every $\mathbb{R}32$ value is exact in $\mathbb{R}64$. The Fortress standard libraries now declare $\mathbb{R}32$ extending Number and the four algebraic traits at its own type, as they declare $\mathbb{R}64$, and $\mathbb{R}64$'s coercion from it (`ProjectFortress/LibraryBuiltin/FortressBuiltin.fsi`, lines 47–48; `Library/FortressLibrary.fsi`, lines 291–296). The team's own compiler library has declared the two types siblings since its change of 31 August 2009, whose message says "RR32 is NOT a subtype of RR64" and "this would be a good time to get coercion working"; it declares the same coercion today (`ProjectFortress/LibraryBuiltin/CompilerBuiltin.fsi`, lines 433–435 and 475). The example of Section 17.8 computes an $\mathbb{R}32$ with an $\mathbb{R}64$ by the declaration on $\mathbb{R}64$ after a coercion from $\mathbb{R}32$, which is what the library now does. The full reasoning is in `explorations/compile-ladder/rung-rr32-sibling/decision-record.md` in the revival's repository.

Effect A value of $\mathbb{R}32$ is not a value of $\mathbb{R}64$. Two $\mathbb{R}32$ values combine by the operators declared on $\mathbb{R}32$, into an $\mathbb{R}32$; an $\mathbb{R}32$ combined with an $\mathbb{R}64$, a floating-point numeral or a $\mathbb{Z}32$ is converted to $\mathbb{R}64$, and the result is an $\mathbb{R}64$; an $\mathbb{R}32$ is converted where an $\mathbb{R}64$ is expected. A conversion from $\mathbb{R}64$ into $\mathbb{R}32$, which can lose precision, stays explicit.

Original text The Working Draft of February 2011 names $\mathbb{R}32$ in no passage of the number chapters; it says that "32-bit precision floats are written $\mathbb{R}32$" (`Specification-1.0-frozen/preliminaries/overview.tex`, line 579), that the numeric types are mutually exclusive (`Specification-1.0-frozen/basic/types-vals-vars.tex`, line 502), and gives the example of Section 17.8 (`Specification-1.0-frozen/basic/conversions-coercions.tex`, lines 844–896). The passages this change revises are the revival's own text of 27 September 2026, not the Working Draft's. The passage said, in `Specification/basic-lib/numbers.tex`, lines 27–31, before this revision:

The number types of this chapter are the integer types $\mathbb{Z}$, $\mathbb{Z}64$, $\mathbb{Z}32$, $\mathbb{N}64$, and $\mathbb{N}32$ (Section 33.2), the rational type $\mathbb{Q}$, and the floating-point type $\mathbb{R}64$ (`RR64`).

and, in line 43:

$\mathbb{R}64$ (`RR64`) coerces from $\mathbb{Z}32$, and from integer numerals (Section 13.1).

The effect of Section I.1.11 said:

The library's $\mathbb{R}32$ stays below $\mathbb{R}64$, as before, a value object that adds no instantiation of its own.

Route C The same under route C: the numeric types are mutually exclusive whichever rule on instantiations the compiled type checker keeps.

I.1.29 Overloading with static parameters

Affected sections Section 15.1; Section 15.2; Section 15.3; Section 15.5; Section 24.1; Section 24.5; Section 17.5; and, in a draft build, the proposed features of the appendix "Internal Document".

Change The sentence of Section 15.1 that declarations of one name may not differ in their static parameters, and its echo in Section 24.1, are replaced: declarations of one name may have different static parameters, or some of them static parameters and others none, and a declaration with static parameters is read as one declaration whose parameter type ranges over its instances. Such a declaration is applicable to a call when some instance of it within its bounds is, and it is compared with others on the parameter types they declare, taken over all their instances; the static arguments that a call does not write are inferred for the declaration it selects, after the selection (Chapter 20). The three sentences of Chapter 15 that inferred static parameters before applicability was checked and before declarations were compared, and the chapter's assumption that the static variables of a call have been instantiated or inferred, are rewritten to match. A new section, Section 24.5, states for declarations with static parameters the three rules of the type group's paper of 2011, the No Duplicates Rule, the Meet Rule and the Return Type Rule, the last checked over every instance of the less specific declaration; how two parameter types with static parameters exclude each other, by existential reduction with instantiation exclusion; and a pair that the Return Type Rule refuses, in the shape of that paper's own example, beside the pair that repairs it. A box in that section states the restriction that the compiled implementation adds, that of two declarations with as many static parameters of their own, one of which is more specific than the other, the two agree on their static parameters position by position, with an override that reorders them as its example. A later revision states the section's No Duplicates Rule on the relation without "and not the other way round", that d_1 and d_2 are not each applicable to every call to which the other is applicable, with a box, and the box on Jan's example in the internal document gives that rule as its reason. A further revision has that box say that the factory of the Fortress standard libraries gives a Vector where its type argument is one of the eight number types and a plain array where it is any other type, Number among them: the factory called the vector factory at its own type argument, which has no bound, an instance that the bound of a Vector's element type refuses (Section 12.1), and it now builds the vector at each number type, so that a Vector of Number, which is not well-formed, is no longer its result. Another revision has the section's existential reduction read instantiation exclusion where the two parameter types, or their elements at one position, are or extend instances of one parameterized trait, where it read it only where they mention one, with a box, and the internal document's box on the questions on exclusion says the same. Section 17.5 says in three places that a declaration with static parameters is applicable when some instance of it is, that it is compared on the parameter types it declares, and that the declaration selected is then instantiated, the coercions of its instance inserted statically. In the internal document, the proposed features on relaxing the restrictions on the static parameters of overloaded functionals and on overloading with static parameters are marked done, Jan's example answered by the Fortress standard libraries's factory and the questions on exclusion by the existential reduction.

Rationale This change does not follow from route A; the revival decided on 26 September 2026 that the sentence goes and that this working draft states the model of the type group's paper of 2011 (Allen, Hilburn, Kil-

patrick, Luchangco, Ryu, Chase, and Steele, "Type Checking Modular Multiple Dispatch with Parametric Polymorphism and Multiple Inheritance", OOPSLA 2011; `Papers/Types/rules.tick`, lines 132–180, and `Papers/Types/setup.tick`, lines 393–397, in the revival's repository), which the compiled type checker applies, with two changes to the checker: the Return Type Rule checked over every instance, and the positional restriction. On 28 September 2026 it decided that a coercion makes a call possible and never changes which declaration runs when one is applicable without it, declarations being chosen on the parameter types they declare and the declaration chosen being then instantiated. Neither implementation enforced the sentence: the interpreter has overloaded declarations with different static parameters since May 2007, the compiled type checker applies the rules of that paper to them, and the Fortress standard libraries overload across different static parameters throughout. The sentence is text of 2009. The Fortress 1.0 release of 2008 compared generic declarations instead by using "their bounds as their supertypes", in a section whose type relations "may be replaced when the static type checker and the type inference engine are implemented" (`Specification-1.0-frozen/fortress.1.0.pdf`, Section 15.6.1), and the team's unfinished restart of this specification in 2012 dropped the sentence (`Documentation/Specification/Prose/Language/overloading.tick`, lines 100–114, in the revival's repository). The restart's Types chapter (Luchangco, 2012) gives function types that hold generic and plain declarations together, as sets of arrow types and universal arrow types, and drops from a quantified type a static parameter that appears in none of its types (`Documentation/Specification/Prose/Language/types.tick`, lines 553–558, 764–767 and 794–801). The positional restriction is the compiled implementation's and not the language's: its code generator hands the static arguments that a call writes, by position, to whichever declaration dispatch reaches (row 398 of the revival's gap ledger), while the type group's paper of 2019 instantiates a dispatched declaration by substitution and renames static parameters freely (Park, Hong, Steele and Ryu, "Polymorphic Symmetric Multiple Dispatch with Variance", POPL 2019, Sections 4 and 5.2). That paper's soundness proof excludes an upper bound that names a type parameter, its own included, and closed types, so that the Fortress standard libraries's families over a type parameter bounded by an instance of a trait at itself, such as `SUM`, `BIG MAX` and those over `Integral`, rest on the rules of 2011 as the compiled type checker implements them, and not on the proof of 2019. The full reasoning is in `explorations/compile-ladder/rung-spec-overloading/decision-record.md` in the revival's repository. The No Duplicates Rule was first stated on the strict sense of more specific that the section defines, under which no two declarations are each more specific than the other, so that it refused no pair, and two declarations each applicable to every call to which the other is fell to the Meet Rule (row 696 of the revival's gap ledger). The paper of 2011 defines its relation of specificity without strictness, one declaration being more specific than another when every type to which the first is applicable is one to which the second is, and its No Duplicates Rule "forbids distinct declarations from being equally specific" (`Papers/Types/rules.tick`, lines 97–108, 158–162 and 184–185); the paper of 2019 states it as "Distinct method declarations must have different applicable sets" (Section 2.2), and its rules [No-Dup-Not-Less] and [No-Dup-Not-Gtr] refuse exactly a pair whose domain types are each a subtype of the other (Section 4.1 and Figure 5). The restatement follows the revival's decision of 23 September 2026 that the type group's later word, that paper among it, outweighs the early text; its reasoning is in `explorations/compile-ladder/rung-popl-examples/REPORT.md` in the revival's repository. The revision of the existential reduction follows the paper of 2011, whose exclusion of two instances of type constructors reads every pair of their ancestors (`Papers/Types/fig-exclusion.tick`, lines 136–147, in the revival's repository), and the revival's decision of 17 October 2026 that two declarations with static parameters exclude each other as the existential reduction reads their whole parameter types, in both implementations; its reasoning is in `explorations/compile-ladder/rung-existential-reduction/REPORT.md` in the revival's repository. The compiled type checker's test of exclusion reads the static parameters' bounds by the revival's decision of 23 October 2026: the existential reduction of the paper of 2011 solves the condition that the meet is not empty together with the bounds of its static parameters (`Papers/Types/exc-reduction.tick`, lines 40–46 and 68, in the revival's repository), which the compiled type checker carried but from which its solver took no equality; its reasoning is in `explorations/compile-ladder/rung-self-type-exclusion/REPORT.md` in the revival's repository.

Effect Declarations of one name may differ in their static parameters, as the Fortress standard libraries's do. The

compiled type checker refuses the pair of the Return Type Rule's example, which it accepted before and whose compiled class then failed verification, and an override whose return type reorders its static parameters, which it accepted before and which then failed at run time (row 398). Neither implementation runs every set these rules allow. The compiled type checker refuses more than the Return Type Rule where the more specific declaration's parameter type is an object type, as in $ident[\![T]\!](x\colon T)\colon T$ beside $ident(x\colon \mathrm{Circle})\colon \mathrm{Circle}$ where Circle is an object (row 542); the compiled code generator fails on two declarations with static parameters whose bounds differ, the pair the Return Type Rule allows in Section 24.5 among them, and on the example of the type group's paper of 2019 that overloads a method over an array list and a list (row 537); and the interpreter reads two functions with static parameters, no pair of whose parameter types excludes each other as it compares them parameter by parameter, on the two declared domains, each over its own instances, by these rules, but finds a third declaration that holds the overlap of two neither of which lies inside the other only where their parameter types are, position by position, instances of one generic type or static parameters, so that it refuses that example (row 539). Of two generic functional methods one of whose declared domains lies inside the other's, it refuses at load the pair that a trait or object without static parameters declares, as failing the Meet Rule, where it loads the same two functions (row 779), and it loads the pair that a trait with static parameters declares, where a call to which both apply then fails, finding no definition of the method (row 780). Jan's example is still not a valid overloading, its two declarations having the same parameter type. The compiled type checker applies the No Duplicates Rule as it is now stated: it refuses two declarations whose parameter types, read over their instances, are each a subtype of the other, "There are multiple declarations of dup with the same parameter type", two functional methods among them wherever their `self` parameters stand. It accepts one such pair, a declaration whose parameter types are its type parameters, written with no bound and so bounded by Any, beside one without static parameters whose parameter types are Any, as $g[\![T]\!](a\colon T)$ beside $g(a\colon \mathrm{Any})$, and runs the first for a call to which both apply; it refuses the same pair where the bound is written, Any or one below it (row 759 of the revival's gap ledger). It reads the abstract functional methods that a trait or object provides as it reads the abstract dotted ones, so that an abstract functional method beside a concrete one with the same parameter type is among the pairs it refuses, which only its code generator refused before, as a malformed overload set. It checks no overloading of an operator whose name is not a word of capitals (row 584 of the revival's gap ledger): two declarations of such an operator with the same parameter type pass it, and only its code generator refuses them, as a malformed overload set (row 726). The interpreter refuses at load two declarations with static parameters whose parameter types, each read over its own instances, are each a subtype of the other, as dup's are, and so a declaration whose parameter types are its static parameters beside one whose parameter types are their bounds, a plain one or one whose parameter types mention none of its static parameters. The compiled type checker and the interpreter both find that two declarations, one or both with static parameters, whose parameter types exclude only together, through a static argument that another parameter's type fixes, exclude each other, and the compiled type checker finds so too of two whose parameter types exclude through the one instance of a supertype they share, which the interpreter loaded already. The compiled type checker's test of whether two such parameter types exclude each other also reads each static parameter that stands at the top of their meet, the meet itself, an element of a tuple or a member of an intersection or union, as within its declared bounds. So it accepts, beside the $+$ of $\mathrm{AdditiveGroup}$, the $+$ of the Fortress standard libraries between a number of type T and an $\mathrm{Array}[\![T, I]\!]$, with T bounded by Number and $\mathrm{AdditiveGroup}[\![T]\!]$, since no array is a number; and it accepts $f[\![T\ \texttt{extends}\ \{K, R[\![T]\!]\}]\!](x\colon T)$ beside $f(x\colon Z)$, where K comprises A, B and C, Z excludes A and B, and C extends Z but not R, which the interpreter refuses at load as failing the Meet Rule (row 713 of the revival's gap ledger). The interpreter refuses at load the first of these shapes in a program's own component, generic operators between an array of a number type and a number, on either side, beside the operator of a trait with a self-type parameter, as having no pair of parameters whose types exclude each other, which the compiled type checker accepts and runs (row 804 of the revival's gap ledger). The interpreter does not so read a declaration with a `where` clause of its own, and refuses such a pair, which the compiled type checker accepts and its code generator does not compile (row 787 of the revival's gap ledger). The interpreter also reads an object type over a static parameter as excluding a type that the object type is at some instance of the static parameters, and so loads pairs that the Meet Rule refuses, as $g[\![T]\!](x\colon \mathrm{Box}[\![T]\!], y\colon \mathbb{Z}32)$ beside $g(x\colon \mathrm{Box}[\![\mathbb{Z}32]\!], y\colon \mathrm{Any})$, where Box is an object type: the two overlap at a $\mathrm{Box}[\![\mathbb{Z}32]\!]$ and a $\mathbb{Z}32$, and neither is more specific than the other; the interpreter runs the first for a call to which both apply, and the compiled type checker refuses the pair (row 788

of the revival's gap ledger). They also differ on a pair whose static parameters' bounds exclude each other, as $tint[\![T$ `extends` $\mathrm{Red}]\!](x\colon \mathrm{Box}[\![T]\!])$ beside $tint[\![T$ `extends` $\mathrm{Blue}]\!](x\colon \mathrm{Box}[\![T]\!])$ where Red, Blue and Box are object types: the interpreter loads it, reading two static parameters whose bounds exclude each other as never one type, and the compiled type checker refuses it, its reduction making of the two static parameters one bounded by both, which it does not find empty: though its test of exclusion reads the bounds, $\mathrm{Box}[\![\mathrm{BottomType}]\!]$ is an instance of both domains (row 786 of the revival's gap ledger). The calculus of Appendix A.3 keeps its rule that two declarations with the same number of parameters have static parameters with the same bounds; it admits fewer overloadings than these chapters, so that its valid programs are still valid, and its rules and its soundness claim are unchanged (Section I.1.10).

Original text `Specification-1.0-frozen/basic/overloading.tex`, lines 100–108:

> Although there may be multiple declarations with the same functional name, it is an error for their static parameters to differ (up to α-equivalence), or for one declaration to have static parameters and another to not have them. Hence, static parameters do not enter into the determination of which declarations are applicable, so we ignore them for most of this chapter.

lines 137–138:

> We assume throughout this chapter that all static variables in functional calls have been instantiated or inferred.

lines 173–175, and in the same words for the types P_0 and P of a dotted method declaration, lines 205–207:

> If the parameter type P includes static parameters, they are inferred as described in Chapter 20 before checking the applicability of the declaration to the call.

lines 292–295:

> If the declarations include static parameters, they are inferred as described in Chapter 20 before comparing their parameter types to determine which declaration is more specific.

and `Specification-1.0-frozen/advanced/overloading.tex`, lines 95–103:

> Overloaded declarations must have static parameters that are identical (up to α-equivalence). Also, valid overloadings for declarations that contain keyword or varargs parameters are determined by analyzing the expansion of these declarations as described in Section 15.4. Therefore, static parameters, keyword parameters, and varargs parameters are ignored in the remainder of this chapter.

The sentences added to Section 17.5, after the Working Draft's lines 455–459, 522–524 and 567–570 of `Specification-1.0-frozen/basic/conversions-coercions.tex`, and the new section have no counterpart there. The new section, the revival's text of 30 September 2026, stated:

> The *No Duplicates Rule*: d_1 and d_2 are not each more specific than the other.

and, of the existential reduction:

> where the two parameter types mention one parameterized trait, instantiation exclusion (Section 6.3) lets them meet only where the static arguments of those mentions are the same

and the box on Jan's example in the internal document gave as its reason that the two declarations have the same parameter type "and each is more specific than the other"; until the further revision, the box ended: "which chooses its result by a `typecase` on its type argument, a Vector where the argument is a Number".

Route C The same under route C, except that the existential reduction uses the rule of Section I.1.44 in place of instantiation exclusion; the No Duplicates Rule does not depend on the rule on instantiations.

I.1.30 The implicit bound of a type parameter

Affected sections Section 12.1; Section 24.1.

Change A type parameter declared without an `extends` clause has the implicit clause "`extends` Any" in place of "`extends` Object", with a box. The restriction of Chapter 24 that a functional whose single parameter's type is a naked type parameter of bound Any cannot be overloaded is stated for a bound written Any, and a sentence says that a type parameter declared without an `extends` clause does not fall under it, with the Fortress standard libraries's value factory of one-dimensional arrays as its example, and a box.

Rationale This change does not follow from route A; the revival took Any as the implicit bound on 27 September 2026 and decided it on 29 September 2026. Any holds every value, tuples, functions and $()$ among them; the team's later Types chapter makes Object a supertype of every trait type, arrow type and function type that excludes $()$ and every tuple type (Luchangco, 2012, `Documentation/Specification/Prose/Language/types.tick`, lines 232–236, in the revival's repository). The Fortress standard libraries instantiate type parameters declared without a bound at tuple types, an array indexed by pairs among them, which the bound Object would refuse; the interpreter, and the compiled type checker when it checks the Fortress standard libraries, read such a parameter as bounded by Any. The type group's paper of 2019 gives the same default: "By default, each type parameter has an upper bound Any and a lower bound Bottom" (Park, Hong, Steele and Ryu, POPL 2019, Section 3.2). Under the implicit bound Any, the restriction on a naked type parameter of bound Any would forbid overloading every functional whose single parameter's type is a type parameter declared without a bound, the Fortress standard libraries's factory $array_1[\![T, \texttt{nat}\ s_0]\!](v{:}\,T)$ beside $array_1[\![T, \texttt{nat}\ s_0]\!]()$ among them; the compiled type checker applies the restriction only to a bound written Any, and draws no such refusal on the Fortress standard libraries. The full reasoning is in `explorations/compile-ladder/rung-spec-overloading/decision-record.md` in the revival's repository.

Effect A type parameter declared without a bound may be instantiated with a tuple type or with $()$. The compiled path still gives such a type parameter of a program's own declarations the bound Object (row 412 of the revival's gap ledger), a divergence that lasts until the compiled path is switched over to the interpreter's library. The rationale of Section I.1.18, written before this change, speaks of the implicit bound Object.

Original text `Specification-1.0-frozen/basic/trait-parameters.tex`, lines 49–50:

> If a type parameter does not have an `extends` clause, it has an implicit "`extends` Object" clause.

and `Specification-1.0-frozen/advanced/overloading.tex`, lines 114–116:

> In addition, a functional which takes a single parameter of type a naked type parameter of bound Any cannot be overloaded.

Route C The same under route C.

I.1.31 The instance at which a dispatched declaration runs

Affected section Section 15.5.

Change The paragraph on dispatch gains sentences: a declaration with static parameters is applicable to a call at run time when some instance of it within its bounds is, so the declaration to which a call dispatches may be one that the static call did not select; a declaration that the static call selected runs at the instance inferred for the call, and any other at the instance in which each type parameter is the intersection of the upper bounds placed on it, its declared bound and those that the dynamic types of the arguments and the static type of the call place on it, fixed by the value where the type parameter is a static argument of a parameterized trait in a parameter type, and never $\mathrm{BottomType}$ nor the union of the types of the arguments. A box says that both implementations take the dynamic type of the argument where the value does not fix the type parameter. Two examples follow

the definition of "more specific": a generic declaration beside a plain one that is more specific, and a generic declaration that a call reaches only by dispatch, at the instance the value fixes, with a box.

Rationale This change does not follow from route A. Which declaration runs follows from the revival's decision of 28 September 2026 that declarations are compared on the parameter types they declare, a declaration with static parameters being applicable when some instance of it is; the revival confirmed on 29 September 2026 that a generic declaration that a call reaches only by dispatch runs. The instance is the rule of the type group's paper of 2019, which the revival took on 29 September 2026: its dispatcher solves the bounds that the dynamic types of the arguments and the static type of the call place on each type parameter and "computes a substitution ... by taking the intersection of the upper bounds", never instantiating a type parameter with $\mathrm{BottomType}$; the team had assumed "that once the tightest possible upper and lower bounds for a method type parameter have been found, then the union (or pseudo-union) of the lower bounds should be chosen", and for it "we were unable to complete the proofs of correctness" (Park, Hong, Steele and Ryu, POPL 2019, Sections 5.2 and 5.4). Naden's notes of 2012 on instantiation at run time restricted by the return type (`Papers/RuntimeInstantiation/`, in the revival's repository) are the type group's draft of that rule; the type group's paper of 2011 left the run-time inference of the static arguments of a dispatched declaration "beyond the scope of this paper" (`Papers/Types/rules.tick`, lines 118–129). The paper's dispatcher solves every call so; this text keeps the instance inferred for the declaration that the static call selected, which differs from the paper's only for a type parameter that neither the static type of the call nor an invariant position in a parameter type constrains. The full reasoning is in `explorations/compile-ladder/rung-spec-overloading/decision-record.md` in the revival's repository.

Effect No value that a valid program computes changes. The interpreter, which has no static types, and the compiled path, whose dispatcher is not given the static type of the call, take the dynamic type of the argument where the value does not fix the type parameter; the compiled path's departure is gated as an expected failure (row 538 of the revival's gap ledger). The compiled path does not yet run a generic declaration that a call reaches only by dispatch beside a plain one (row 496). The instance of a `nat` or `int` parameter that nothing fixes at run time is not described (Section I.1.43).

Original text The paragraph is unchanged: `Specification-1.0-frozen/basic/overloading.tex`, lines 263–276:

> To evaluate a given functional call, it is necessary to determine which functional declaration to dispatch to. To do so, we consider the declarations that are applicable to that call at run time. If there is exactly one such declaration, then the call dispatches to that declaration. If there is no such declaration, then the call is *undefined*, which is a static error. (However, see Section 17.4 for how coercion may add to the set of applicable declarations.) If multiple declarations are applicable to the call at run time, then we choose an arbitrary declaration among the declarations such that no other applicable declaration is more specific than them.

The Working Draft forbade both pairs of the examples, by its lines 100–105, quoted in Section I.1.29.

Route C The same under route C.

I.1.32 The interpreter's choice of a coercion

Affected section Section 17.5.

Change One box after the example of the paragraph that says that coercion is resolved statically; the paragraph and its example are unchanged. The box says that the compiled path follows the text; that the interpreter, which has no static types, chooses a coercion on the value and so runs $f(B)$ for the example's $f(c)$, and answers the same way wherever an argument's value is of a type that another declaration already fits; that both cases are gated as expected failures; and that the interpreter is to follow the text once it takes the coercions that the compiled type checker inserts.

Rationale This change does not follow from route A; the revival decided on 29 September 2026 that the interpreter's choice of a coercion on the value is its limit, since it has no static types from which to resolve a coercion statically, and that the interpreter is made to follow the text once the compiled type checker checks the programs it runs and it takes the coercions the checker inserts. The type group's paper of 2019 also carries the static types of calls to run time for its dispatcher, which the interpreter does not have.

Effect None on the language. The interpreter's answer, 4 where the text gives 3, and its answer in the narrower case are gated by `ProjectFortress/tests/XXXCoercionStaticRungC.fss` and `ProjectFortress/tests/XXXCoercionStaticNarrowRungC.fss` (row 19 of the revival's gap ledger).

Original text The passage is unchanged: `Specification-1.0-frozen/basic/conversions-coercions.tex`, lines 567–604, which begin:

> Notice that coercion is resolved statically. Once a coercion is statically chosen for a call, this coercion is conceptually inserted into the call. This means that the statically chosen coercion is applied at run time.

Route C The same under route C.

I.1.33 Traits with comprises clauses read at the level of values

Affected sections Section 6.10; Section 9.3; Section 10.2; a draft note in the subsection "Export Statements" of Section 22.2; the effect of Section I.1.21.

Change Section 6.10 defines covering: a type covers another when every value of the second is a value of the first; a type covers each of its subtypes, and a trait with a `comprises` clause is covered by the union of its listed types; covering is not the subtype relation. Its example concludes that V covers $S \cap T$, where it concluded that $V = S \cap T$, and a sentence says that a trait may stand between S or T and their listed types, so that $S \cap T$ is not in general equal to a named type. Section 9.3 asks the argument types of the concrete declarations to cover the argument type of the abstract declaration, where it asked their union to equal it, and, in its example, for a definition for every type listed in the `comprises` clause of $\mathrm{Molecule}$ or definitions that together cover those types, where it asked for one for every immediate subtype. Each of these four passages has a box. The draft note says "all its listed types" where it said "all its immediate subtraits"; a draft note prints only in a draft build, so it has no box. The effect of Section I.1.21 no longer says that these passages are not revised. A correction of 9 October 2026 changes the last sentence of the box after the example in Section 9.3 and the sentence of this entry's effect that repeats it. They said that the compiled type checker checks that an object's concrete methods cover the abstract methods it inherits, "reading no `comprises` clause"; they now say that it reads such clauses to do so, so that concrete methods over the types that a trait lists together cover an abstract method over that trait. The normative text is unchanged. An amendment of 14 October 2026 revises the sentence of Section 10.2 on abstract methods: an object inheriting an abstract method defines it by a method definition whose parameter type, not counting the type of the self parameter, is that of the abstract method declaration, or by definitions whose parameter types are subtypes of it and together cover it, or it is a static error, where the sentence said only that the object must define a body expression for the method. The sentence has a box. The requirement of Section 11.1 that an object define all abstract methods inherited from its supertraits is unchanged; the box says that it is read in this sense.

Rationale This change does not follow from route A; the revival decided on 29 September 2026 that the passages that read a `comprises` clause at the level of types are restated at the level of values, which is how Section 10.1 reads the clause since its revision of 28 September 2026 (Section I.1.21). The team's later Types chapter names covering beside subtyping and exclusion as a relation between types, and says that an instantiation of a trait with a `comprises` clause "is covered by the union" of the listed types (`Documentation/Specification/Prose/Language/types.tick`, lines 36–38 and 384–390, in the revival's repository). Welterweight Fortress, the type group's calculus of 2012, reads the clause at the level

of values and counts an abstract declaration as covered when it is implemented for all the types comprised (`Papers/Welterweight/grammar.tick`, lines 21–22; `Papers/Welterweight/fig-covered.tick`). Under that reading a trait with static parameters may stand unlisted between a trait with a `comprises` clause and its listed types; an instantiation of it is then a subtype of the closed trait and of no listed type, though every value of it is a value of a listed type, so the passages' conclusions hold of values and not of types. The full reasoning is in `explorations/compile-ladder/rung-spec-overloading/decision-record.md` and `explorations/reviews/comprises-type-level-judgement.md` in the revival's repository. The sentence that the correction of 9 October 2026 changes was false when it was written: since 6 June 2012 the team's compiled type checker, testing whether an abstract method's parameter type is a subtype of the union of the parameter types of the concrete methods below it, tests whether the union of the types that a trait with a `comprises` clause lists is a subtype of that union (`ProjectFortress/src/com/sun/fortress/scala_src/typechecker/AbstractMethodChecker.scala` and `ProjectFortress/src/com/sun/fortress/scala_src/types/TypeAnalyzer.scala`, in the revival's repository). It accepts an object whose methods over the two types that a trait lists cover an abstract method over that trait, and refuses the same object when the trait has no `comprises` clause. The full reasoning is in `explorations/compile-ladder/rung-walk-override/REPORT.md` in the revival's repository. The amendment of 14 October 2026 follows the revival's decision of 13 October 2026 that an object defines an inherited abstract method when its definitions of that name at narrower parameter types together cover the abstract method's parameter types, `comprises` clauses read as the compiled type checker reads them; a definition at the abstract method's own parameter type is the plain case. It is the type group's rule of 2012. The compiled type checker checks, at each object declaration, whether each inherited abstract method "is covered by one or more concrete definitions", its parameter type a subtype of the union of theirs (Steele, May 2012, in `ProjectFortress/src/com/sun/fortress/scala_src/typechecker/AbstractMethodChecker.scala`, in the revival's repository), and reads `comprises` clauses to do so since 6 June 2012. Welterweight Fortress uses the same rules to check methods as functions, the type of the trait or object within which the method is declared not considered (`Papers/Welterweight/static.tick`, lines 27–47). The sentence of Section 10.2 is older than that reading; the chapter's own sentence on overloaded functional methods points to Section 9.3, which asks for coverage, as the Meet Rule does. The team's library and tests define abstract methods by covering definitions: the trait $\mathrm{RunRanges}$ of the component Pairs (`Library/Pairs.fss`) and the trait dna of the test `ProjectFortress/tests/FileConversion.fss`, which the sentence read to the letter would refuse. The full reasoning is in `explorations/reviews/walk-load-readings-check.md`, section 2, in the revival's repository.

Effect No example of this specification becomes invalid. Covering is not subtyping: where an instantiation of a trait with static parameters stands between S and T and their listed types, its value is a value of V but its static type is not a subtype of V, and the compiled type checker refuses to assign it to a variable of type V; the interpreter, which has no static types, runs such an assignment. The compiled type checker checks, at each object declaration, that the object's concrete methods cover the abstract methods it inherits, and reads `comprises` clauses to do so: concrete methods over the types that a trait lists together cover an abstract method over that trait; top-level abstract function declarations run on neither implementation. Since the amendment of 14 October 2026 the interpreter checks the same at load, at each object without static parameters, reading `comprises` clauses as its check of the Meet Rule reads them: it accepts an object whose definitions at narrower parameter types together cover an inherited abstract method, as before, and refuses one whose definitions do not cover it, where it accepted any one definition at narrower parameter types before. It does not check objects with static parameters and object expressions in generic functions (row 665 of the revival's gap ledger), and it refuses an object where its search through the `comprises` clauses does not settle: a cycle, its bound of 10,000 steps or of 4,096 combinations of parts, or a type it cannot read. It refuses an object whose one definition of an inherited abstract method has a static parameter of its own and covers the method, as $g[\![T\ \texttt{extends}\ \mathrm{Side}]\!](\texttt{self}, x{:}\,T)$ covers $g(\texttt{self}, x{:}\,\mathrm{Side})$, which the compiled type checker accepts (row 728 of the revival's gap ledger). The compiled type checker splits a tuple of parameter types through `comprises` clauses as the interpreter does, and refuses where its search meets a cycle or passes the same bounds: it accepts an object whose concrete methods

cover an inherited abstract method of two parameters only through combinations of the types that a trait lists, as the interpreter does. It splits no tuple where the abstract method or a concrete method below it has static parameters of its own, and so refuses an object whose concrete methods, one of them with a static parameter of its own, cover such a method only through those combinations (row 725 of the revival's gap ledger).

Original text `Specification-1.0-frozen/basic/types-vals-vars.tex`, lines 568–570:

> because of the `comprises` clauses of S and T and the `excludes` clause of U, any subtype of both S and T must be a subtype of V. Thus, $V = S \cap T$.

`Specification-1.0-frozen/basic/functions.tex`, lines 377–378:

> Furthermore, the union of the argument types of the concrete declarations for f must be equal to T.

and lines 413–415:

> For the latter, the programmer must provide a definition for every immediate subtype of $\mathrm{Molecule}$, or it is a static error.

`Specification-1.0-frozen/basic/components/source-code.tex`, line 429, the words of the draft note's second item that changed:

> provided that all its immediate subtraits are considered to have concrete method declaration with that signature.

`Specification-1.0-frozen/basic/traits.tex`, lines 510–512:

> An abstract method declaration declares an *abstract method*; any object inheriting an abstract method must define a body expression for the method.

The effect of Section I.1.21, the revival's text of 28 September 2026, ended:

> The passages that read a `comprises` clause at the level of types, the intersection of two traits with such clauses (Section 6.10, Section 24.4) and the definitions an abstract function declaration requires (Section 9.3), are not revised; whether they are to be restated at the level of values is not settled.

The box after the example in Section 9.3, the revival's text of 30 September 2026, ended:

> The compiled type checker checks, at each object declaration, that the object's concrete methods cover the abstract methods it inherits, reading no `comprises` clause; top-level abstract function declarations run on neither implementation of the revival.

and the effect of this entry said the same in its words.

Route C The same under route C: the reading of a `comprises` clause at the level of values does not depend on the rule on instantiations.

I.1.34 The Meet Rule for traits with comprises clauses

Affected sections Section 24.1; Section 24.4; Section 24.5; Section 24.6; and, in a draft build, the proposed features of the appendix "Internal Document".

Change Where the Meet Rule for functions, or for functional methods, asks for a declaration on the intersection of two parameter types, declarations whose parameter types are each a subtype of both and together cover the intersection also satisfy it, every pair of those declarations satisfying the rules in turn; the case does not apply to the condition on the types from which the two parameter types define coercions. A box at the rule answers the draft note that asks what the intersection of two declarations is; the Meet Rule of Section 24.5 gains the same case, with a box. The Meet Rule's example concludes that V covers $S \cap T$, and that $f(V)$ is applicable to and

more specific for any call at run time to which $f(S)$ and $f(T)$ are applicable. Section 24.1 says that a static call whose argument's static type stands between a trait with a `comprises` clause and its listed types may have several declarations applicable to it that no other applicable declaration is more specific than, and is not a static error. The second of the two facts of Section 24.6 keeps a single most specific declaration for the dynamic call and, but for such a call, for the static call; the declaration that the dynamic call runs is the same as or more specific than every declaration applicable to the static call, its return type a subtype of theirs; the type of a static call whose most specific declarations, and every declaration more specific than them that the call may run, have no static parameters is the intersection of their return types; and a sentence gives the scope of the proof. In the internal document, the proposed feature on the intersection of two types with `comprises` clauses is marked done.

Rationale This change does not follow from route A; the revival decided on 29 September 2026 that the Meet Rule gains the case of declarations that together cover the intersection, and that a call whose argument's static type stands between two traits with `comprises` clauses is typed by the intersection of its candidates' return types, with the proof of Appendix B revised to the uniqueness of the most specific declaration at run time (Section I.1.35). The team's list of proposed features records the same plan for the example: "We agreed to revise the Meet rule to address this with the coverage check". Under the reading of Section 10.1, the intersection of two traits with `comprises` clauses is not in general equal to a named type, and without the case the Meet Rule would refuse the example given to illustrate it. The case is an addition to the Meet Rule of the type group's paper of 2011, which asks for a declaration on the meet, as the type group's calculus Welterweight Fortress does (`Papers/Welterweight/fig-meetrule.tick`, in the revival's repository). The typing of such a call follows from the Return Type Rule: the declaration that runs is below every most specific declaration of the static call, and its return type below each of theirs. For a declaration with static parameters one instance would have to meet every one of those return types at once, which the proof does not show, so the typing is stated for declarations without static parameters only. The covering is a judgement of the overloading rules alone: it makes no type a subtype of another. The full reasoning is in `explorations/compile-ladder/rung-spec-overloading/decision-record.md` and `explorations/reviews/comprises-type-level-proof-addendum.md` in the revival's repository.

Effect The Meet Rule's example is valid; both implementations refused it before this change (row 492 of the revival's gap ledger). A static call whose argument's static type stands between two traits with `comprises` clauses, as an instantiation of a trait with static parameters that extends both unlisted does, is not a static error: it runs the declaration that covers its value, and, where those declarations and every declaration more specific than them that the call may run have no static parameters, its type is the intersection of the return types of its most specific declarations. The compiled type checker departs from this in four ways: it types such a call by the intersection only in a family none of whose declarations has static parameters, and otherwise by one of its most specific declarations (row 543 of the revival's gap ledger); its code generator stops at a method call on a value of such a type (row 540); it refuses the case for declarations with static parameters (row 546); and its verdict on a trait between the two that provides both declarations of a functional method can depend on the components it compiled before (row 547). The Meet Rule for dotted methods keeps its form, asking for a declaration on the intersection (Section I.1.43).

Original text The Working Draft's rules are unchanged, the cases being added after them. `Specification-1.0-frozen/advanced/overloading.tex`, lines 303–307, the example's conclusion:

> Because of the `comprises` clauses of S and T and the `excludes` clause of U, any subtype of both S and T must be a subtype of V. Thus, $V = S \cap T$, and the declaration $f(V)$ "disambiguates" $f(S)$ and $f(T)$, i.e., it is applicable to and more specific for any call to which both $f(S)$ and $f(T)$ are applicable.

and lines 460–468, the second fact and the sentence after the two:

> If any declaration is applicable to a static call then there exists a single most specific declaration that is applicable to the static call and a single most specific declaration that is applicable to the

corresponding dynamic call.

Moreover, we can prove that the most specific declaration that is applicable to a dynamic call is more specific than the most specific declaration that is applicable to the corresponding static call.

Route C The same under route C.

I.1.35 The proof of overloading resolution

Affected section Section B.2.

Change The proof states its scope and its assumptions: declarations whose parameter and return types have no static parameters, calls to which declarations are applicable without coercion, the overloading rules, bodies that exist and are well typed, a sound and transitive subtype relation, and every value of a trait with a `comprises` clause a value of one of its listed types, which the compiled type checker does not yet enforce across components. It states the Meet Rule as it uses it: declarations below both parameter types that together hold every value of both, of which a declaration on the intersection is the case of one. Lemma 7 proves only that the dynamic call has at most one most specific declaration, taking a covering declaration where it took the declaration on the intersection; Theorem 2 proves only that the dynamic call has exactly one when the static call has any; Theorem 3 proves that the declaration that runs is the same as or more specific than every declaration applicable to the static call; and a new Theorem 4 proves that its return type is a subtype of the return type of every most specific declaration of the static call. The lemmas of existence are unchanged.

Rationale This change does not follow from route A; the revival decided on 29 September 2026, with the case of the Meet Rule of Section I.1.34, that the proof keeps its lemmas of existence and gives up the uniqueness of the most specific declaration for the static call for its uniqueness for the dynamic call, with its scope and its assumption on `comprises` clauses marked. Covering gives a declaration that holds the value of the argument, not one whose parameter type is a supertype of the static type of the argument, so the static call may have more than one most specific declaration. The type group's paper of 2019 keeps a unique most specific declaration for every call as a theorem (Park, Hong, Steele and Ryu, POPL 2019, Theorem 4.1); a call whose argument's static type stands between two traits with `comprises` clauses is outside that paper, which has no closed types, and the revival gives up the unique most specific declaration for the static call there on the assumptions this proof states. The full reasoning is in `explorations/reviews/comprises-type-level-proof-addendum.md` and `explorations/compile-ladder/rung-spec-overloading/decision-record.md` in the revival's repository.

Effect The appendix proves that no call is undefined or ambiguous at run time within its scope and on its assumptions; it does not prove the soundness of declarations with static parameters, of inference or of coercion. Where a type declared in a later component escapes the check of a `comprises` clause (row 487 of the revival's gap ledger), the uniqueness of the most specific declaration for the dynamic call is not guaranteed.

Original text `Specification-1.0-frozen/appendices/overloading-function.tex`, lines 37–48:

> Below we prove: $|\Sigma| \neq 0$ implies $|\sigma| = 1$, and $|\Sigma| \neq 0$ implies $|\delta| = 1$. Informally, if any declaration is applicable to a static call then there exists a single most specific declaration that is applicable to the static call and a single most specific declaration that is applicable to the corresponding dynamic call.

lines 84–135, the lemma and the two theorems this change revises:

> *Lemma.* $|\sigma| \leq 1$. Also, $|\delta| \leq 1$.
>
> *Proof.* We prove this for δ, but the case for σ is identical. For the purpose of contradiction suppose there are two declarations $f(P)$ and $f(Q)$ in δ. Since both $f(P)$ and $f(Q)$ are applicable without

coercion to the call $f(X)$ we have $X \preceq P \cap Q$. Therefore it is not the case that $P \blacklozenge Q$. By the overloading rules, $P \neq Q$ and either $P \lozenge Q$ or there is a declaration $f(P \cap Q)$ visible at Z. Since it cannot be the case that $P \lozenge Q$ there must exist a declaration $f(P \cap Q)$ visible at Z. Since $P \cap Q \preceq P$ and $P \cap Q \preceq Q$ we know $f(P \cap Q)$ is applicable without coercion to the call $f(X)$. Since $P \neq Q$ either $P \cap Q \prec P$ or $P \cap Q \prec Q$. Either case contradicts our assumption.

Theorem. If $|\Sigma| \neq 0$ then $|\sigma| = 1$. Also, if $|\Sigma| \neq 0$ then $|\delta| = 1$.

Proof. Follows from the three lemmas before it.

Theorem. If $\sigma = \{S\}$ and $\delta = \{D\}$ then $D \preceq S$.

Proof. If the declaration with parameter type S and the declaration with parameter type D satisfy the Subtype Rule then the theorem is proved. Otherwise, by the definition of σ we have $\sigma \subseteq \Sigma$. Therefore $S \in \Sigma$. By the first lemma, $S \in \Delta$. Notice that $S, D \in \Delta$ implies $X \preceq S$ and $X \preceq D$. Therefore, $S \not\lozenge D$. By the More Specific Rule for Functions, there must exist a declaration with parameter type $S \cap D$. Because $X \preceq (S \cap D)$, $(S \cap D) \in \Delta$. Notice $(S \cap D) \preceq S$ and $(S \cap D) \preceq D$. By the definition of δ, we have $\neg\exists D' \in \Delta : D' \prec D$. In particular, $(S \cap D) \not\prec D$. Therefore $(S \cap D) = D$.

Route C The same under route C.

I.1.36 The type of an integer numeral

Affected sections Section 13.1; Section 17.1.

Change Section 13.1 says that the Fortress standard libraries give every numeral without a radix point the type $\mathrm{IntLiteral}$, which extends Number and none of the number types, and that each of the five integer types, $\mathbb{Q}$ and $\mathbb{R}64$ declares a coercion from it. The box of Section 17.1 says the same of the coercion into $\mathbb{R}64$, and that the interpreter does not yet give a numeral that type.

Rationale This change does not follow from route A; the revival decided on 29 September 2026 that a numeral has a type of its own, $\mathrm{IntLiteral}$, from which each number type converts, the compiled path's own library's working version taken in place of the numeral types of Section 13.1, which no library declares, and that the Fortress standard libraries take it before the interpreter does, since the interpreter can give a numeral that type without changing the declarations its calls reach only once it runs with the static types of the compiled type checker. The Fortress standard libraries declared their numeral type an object extending $\mathbb{Z}32$, its arithmetic withheld by the comment "Do not enable these until coercion is implemented; doing so will cause all our arithmetic to occur on IntLiterals." (`ProjectFortress/LibraryBuiltin/FortressBuiltin.fss`, lines 495–496 before this change, in the revival's repository). Coercion is implemented on both paths now, and the Fortress standard libraries declare the type as the compiled path's own library does (`ProjectFortress/LibraryBuiltin/CompilerBuiltin.fsi`, lines 390–431): beside the number types under Number, with the arithmetic that comment withheld, each integer type declaring a coercion from it. $\mathbb{Q}$ and $\mathbb{R}64$, which that library does not declare from a numeral, coerce from it too, since $\mathbb{Q}$ coerces from every integer type (Section 33.1) and $\mathbb{R}64$ from integer numerals (Section I.1.16).

Effect The compiled type checker converts an integer numeral to the integer type, $\mathbb{Q}$ or $\mathbb{R}64$ that its context expects, $\mathbb{N}32$ and $\mathbb{N}64$ among them, which refused a numeral before. A numeral beside a value of an integer type is converted to that type where no declaration of the operator is applicable to the two without coercion, and that type's own declaration of the operator is then the most specific (Section 17.5); the Fortress standard libraries state on each integer type the comparisons $<$, $\leq$, $>$ and $\geq$, `CMP`, `MAXNUM`, `MINNUM` and a power with a numeral exponent, which the types inherited before from a declaration that a converted numeral leaves tied with those of $\mathbb{Z}64$, $\mathbb{Q}$ and $\mathbb{R}64$. Equality and inequality are not converted so: the declaration of $=$ on Number is applicable to an integer value and a numeral without coercion, and so is the top-level declaration of $\neq$, whose parameters are of type Any; the compiled type checker selects these and converts neither operand, and

$=$ then compares the two as rationals, through the case for a numeral that this change adds to $exactValue$, the function by which Number's $=$ takes an exact value. $\mathbb{Z}$ states in its api, for the same reason, the additive and multiplicative operators its body declares. $\mathrm{IntLiteral}$ declares its own $zero$, one, $even$, odd, $floor$, $ceiling$, $truncate$, `DIVIDES`, `TIMES` and a power with a numeral exponent, which a numeral reached before as a $\mathbb{Z}32$, since no member of $\mathrm{Integral}$ is applicable to a numeral of that type, even with coercion (Section 17.4). A numeral given where any integer type is accepted, as the count of a shift, is converted to the narrowest of $\mathbb{Z}32$, $\mathbb{Z}64$ and $\mathbb{Z}$ that holds its value, as Section 20.2 reads a numeral whose conversions tie. The interpreter still gives an integer numeral the type $\mathbb{Z}32$, $\mathbb{Z}64$ or $\mathbb{Z}$ by its value, so a numeral reaches none of the declarations of $\mathrm{IntLiteral}$ there. Its calls of the comparisons, `CMP`, `MAXNUM` and `MINNUM` on an integer type reach that type's own declarations, which answer as the declarations inherited before did, except that `MAXNUM` and `MINNUM` with a converted argument, which stopped at a tie with those of $\mathbb{Q}$ or answered at $\mathbb{Q}$, now answer at the integer type both arguments convert to.

Original text `Specification-1.0-frozen/basic/expressions/literals.tex`, lines 146–148, which stand unchanged and are now followed by the new sentence:

> The Fortress standard libraries define coercions from numerals to integers (for simple numerals) and rational numbers (for compound numerals).

The box of Section 17.1 is the revival's own text of 26 September 2026, not the Working Draft's; it said, in `Specification/basic/conversions-coercions.tex`, lines 84–85, before this revision:

> The interpreter gives an integer numeral outside the range of $\mathbb{Z}32$ a wider integer type, and so does not yet convert it.

Route C The same under route C.

I.1.37 The type of a varargs parameter

Affected sections Section 9.1.

Change A box after the sentence of Section 9.1 that gives a varargs parameter the type $\mathrm{HeapSequence}[\![T]\!]$ says that no library declares that type, nor the immutable sequence that the team's draft note at the start of Section 9.2 agreed on in its place; that both implementations give the parameter the array type $\mathrm{ImmutableArray}[\![T, \mathbb{Z}32]\!]$ of the Fortress standard libraries; and that the compiled path's own library declares no such type. The normative text is unchanged.

Rationale This change does not follow from route A; it follows the revival's decision of 23 September 2026 that the implementers' later word outweighs text that the designers left unfinished. The text and its draft notes name $\mathrm{HeapSequence}$ and $\mathrm{ReadOnlyHeapSequence}$ ("We agreed to implement varargs tuples using $\mathrm{ReadOnlyHeapSequence}$ which is immutable and invariant. We'll add $\mathrm{MakeReadOnlyHeapSequence}$ into the core library."), and no library declares either. The authors of the compiled type checker named a type for the parameter, $\mathrm{ImmutableArray}[\![T, \mathbb{Z}32]\!]$, with the comment "Replace ImmutableArray with ImmutableHeapSequence when ImmutableHeapSequence is put into the libraries" (`ProjectFortress/src/com/sun/fortress/compiler/Types.java`, lines 47–49, in the revival's repository), and the interpreter binds the parameter to an immutable one-dimensional array of the arguments that the factory *__immutableFactory1* of the Fortress standard libraries builds (`ProjectFortress/src/com/sun/fortress/interpreter/evaluator/values/NonPrimitive.java`, lines 224–251), a value of that type. The type is immutable, as the draft note asks. Before this change the compiled type checker typed the parameter in the body as T, the type of one argument, and read a parameter list with plain bindings before a varargs binding as if the varargs binding were one more plain binding, so that a call that passed no argument, or more than one, for the varargs binding after plain bindings was refused (row 604 of the revival's gap ledger). The full reasoning is in `explorations/compile-ladder/rung-checker-defects/REPORT.md` in the revival's repository.

Effect The compiled type checker applies a declaration with a varargs parameter to a call as Section 9.2 and Section 15.4 say, whatever the number of arguments for the varargs binding, none among them, each a subtype of the type of the binding or convertible to it; and it types the parameter in the body at $\mathrm{ImmutableArray}[\![T, \mathbb{Z}32]\!]$ of the Fortress standard libraries, where it typed it as T. Under the compiled path's own library, which declares no $\mathrm{ImmutableArray}$, it refuses every use of a varargs parameter, in a body, a contract or a function expression, naming the type. The code generator does not yet compile a function with a varargs parameter, so no program with one runs on the compiled path. The interpreter is unchanged.

Original text `Specification-1.0-frozen/basic/functions.tex`, lines 178–180, which stand unchanged and are now followed, after the draft note that refers to the library section for $\mathrm{HeapSequence}$, by the box:

> The type of a varargs parameter is $\mathrm{HeapSequence}[\![T]\!]$ where T is the type mentioned in (or inferred for) that binding.

Route C The same under route C.

I.1.38 The type of a parameter that a declaration leaves out

Affected section Section 22.6.

Change One box after the paragraph of Section 22.6 that says that type inference is performed over all program constructs that still include elided types; the paragraph is unchanged. The box says that the compiled type checker does not yet infer the type of a parameter that a function declaration leaves out, at top level, in a trait or object, or local to an expression block, and refuses the component with the message "Missing parameter type for" followed by the parameter's name; and that the inference the paragraph asks for is not yet described.

Rationale This change does not follow from route A; it follows the revival's decision of 8 October 2026 that a local function whose parameter type is left out is refused with a message, as a top-level function is, instead of stopping the compiled type checker. The inference the paragraph asks for is not described: Chapter 20 names the inference over a whole program of the types that declarations elide among what it does not yet describe. Before this change the compiled type checker refused such a parameter of a top-level function or a method with that message, which no passage stated (row 405 of the revival's gap ledger), and stopped on such a parameter of a local function, with "Type is not inferred", "TryChecker returned an untyped expr" or "Result of typechecking still contains intermediate nodes" (rows 620 to 622). A stop of the checker is no answer that the text gives; the refusal names the parameter, and departs from the text as the refusal at top level already did. Inferring the types, from the uses of the parameter or from an arrow type that the context expects, is the repair, which waits until the text describes the inference. The full reasoning is in `explorations/compile-ladder/rung-checker-overloading/decision-record.md` in the revival's repository.

Effect The compiled path refuses a component that leaves out the type of a parameter of a function declaration other than the self parameter of a method. Among them is a keyword parameter, whose type Section 9.1 infers from its default expression; the compiled path does not yet read a keyword parameter as one, and the interpreter stops on a call that leaves out its argument (row 652 of the revival's gap ledger). The parameter of a function expression is not affected: where the context expects an arrow type, the compiled type checker takes the parameter's type from it. The interpreter, which has no static types, runs a top-level or local function whose parameter type is left out, and stops on a method that leaves it out and implements an abstract declaration (row 405).

Original text The passage is unchanged: `Specification-1.0-frozen/basic/components/type-inference.tex`, lines 44–46:

> Once C is expanded, type inference is performed over all program constructs that still include elided types. Empty bodies are ignored.

Route C The same under route C.

I.1.39 The rounding of an infinite or indefinite rational

Affected section Section 33.1.

Change The sentence after the method entries of *floor*, *ceiling*, *round* and *truncate* and of the enclosing operators $\lfloor \ldots \rfloor$ and $\lceil \ldots \rceil$ said that all of these methods simply return the argument if it is $+\infty$, $-\infty$, or $0/0$; it now says that they throw a DivisionByZero there, none of these values having an integer value, as a division by zero whose result is an integer does (Chapter 16). The entries' result type $\mathbb{Z}$ is unchanged. The sentence's opening words, "For types $\mathbb{Q}^*$ and $\mathbb{Q}^\#$,", left it before (Section I.1.12).

Rationale This change does not follow from route A; it follows the revival's decision of 9 October 2026 that the declared integer result and the peers win over the sentence. The method entries declare the result $\mathbb{Z}$, and the Fortress standard libraries declare it too; a method that returned its argument at $+\infty$, $-\infty$ or $0/0$ returned a rational number where it declares an integer, and the compiled type checker refused the two bodies of the Fortress standard libraries that did so (row 635 of the revival's gap ledger). A rational number with the denominator zero is the quotient of a division by zero, and a division by zero whose result is an integer throws a DivisionByZero (Chapter 16); the Working Draft of February 2011 threw the same exception when a value $1/0$ or $-1/0$ was assigned to a variable of the rational type without the infinities (Section I.1.15). Among the peers, Python's exact rational type refuses a zero denominator with a `ZeroDivisionError`, and its floor, ceiling and truncation of a floating-point infinity or NaN raise an error instead of returning a value. The full reasoning is in `explorations/compile-ladder/rung-library-slips/REPORT.md` in the revival's repository.

Effect The Fortress standard libraries throw DivisionByZero from *ceiling* and *truncate* at these three values, and so from *floor*, *round* and the two enclosing operators, which reach them; the interpreter runs them so. Before this change it returned the argument from all of them but *round*, which stopped the program there. The compiled path's own library, used until the one library replaces it, declares no rational type. The floating-point types' rounding methods are not affected; which of them keep the floating-point type, and what they return at an infinity or a NaN, is row 330 of the revival's gap ledger.

Original text `Specification-1.0-frozen/basic-lib/numbers.tex`, lines 479–481:

> For types $\mathbb{Q}^*$ and $\mathbb{Q}^\#$, all of these methods simply return the argument if it is $+\infty$, $-\infty$, or $0/0$.

Route C The same under route C. A return to the original sentence would need the six methods and operators to declare a rational result, against their entries' $\mathbb{Z}$, or would leave them returning a value outside the type they declare.

I.1.40 When two sizes are the same

Affected section Chapter 6.

Change Two sentences and a box after the sentence of Chapter 6 on identical types, which is unchanged. They say that two `nat` or `int` static arguments are identical if and only if they are written alike once each operation whose operands are numerals is replaced by its value, so that $2\ 3$ is 6 and $s_0\ s_1$ is not $s_1\ s_0$; that two such arguments are known to differ when they are two different numerals; and that a static parameter, or an expression in which one occurs, is not known to differ from a numeral.

Rationale This change does not follow from route A; it follows the revival's decision of 9 October 2026 on arithmetic in a size. The text allows a sum, a difference, a product or a power as a static argument (Appendix G) and says that its value is determined once the static parameters in it are instantiated (Section 13.27), but it never said when two such arguments are identical: the sentence on identical types says only that their static arguments are, beside the team's note that it "isn't complete in any case". the Fortress standard libraries write a product in the size of the storage of a matrix (mem: PrimitiveArray$[\![T, (s_0\ s_1)]\!]$) and sums in result types; written forms compared once numerals are folded accept what they write, each product written once, with no solver of arithmetic,

which nothing in them needs. The team's own questions and answers on overloaded operators, in the appendix "Internal Document", refuse $f(x{:}\,T[\![n+1]\!])$ beside $f(x{:}\,T[\![0]\!])$, "now that $n+1$ is necessarily nonzero", and allow $T[\![1]\!]$ beside $T[\![0]\!]$: a numeral differs from another numeral, and an expression with a static parameter in it is not known to differ from a numeral. The full reasoning is in `explorations/compile-ladder/rung-size-expressions/REPORT.md` in the revival's repository.

Effect The compiled type checker accepts a sum, a difference, a product or a power in a size, computes each operation whose operands are numerals, checks the result against the parameter's kind as it checks a numeral, and compares sizes as the new sentences say; a power of a base other than 0, 1 and -1 at an exponent above 4096 it does not compute, and refuses as outside every kind; a clause $N[\![0]\!]$ of a `typecase` on $N[\![b_0]\!]$ is reachable. It still counts two different static parameters as different sizes, which the text does not say (row 683 of the revival's gap ledger). It infers a static argument written with a static parameter in it: a call of $get[\![\texttt{int}\ i]\!](b{:}\,\mathrm{IBox}[\![i]\!])$ with an argument of type $\mathrm{IBox}[\![i-1]\!]$ instantiates get at $i-1$. Where a parameter type writes a static parameter of the callee inside an operation, neither it nor the interpreter infers that parameter: a call of $getp[\![\texttt{int}\ i]\!](b{:}\,\mathrm{IBox}[\![i+1]\!])$ with an argument of type $\mathrm{IBox}[\![3]\!]$ is refused, and under the interpreter fails, unless it writes $getp[\![2]\!]$, though the instance at 2 has the argument's type (row 767). The compiled path computes a size when it names a class, an operation on numerals when the program is compiled and the rest when the class is first loaded, so that $\mathrm{Box}[\![2+1]\!]$ and $\mathrm{Box}[\![3]\!]$ are one class; it does not check a size computed at that time against its parameter's kind, and makes a size beyond $\mathbb{N}32$ that the interpreter refuses (row 675 of the revival's gap ledger). The interpreter computes a sum, a difference, a product and a power exactly when it instantiates a type, refuses a power at a negative exponent, and refuses a size outside its kind.

Original text The passage is unchanged: `Specification-1.0-frozen/basic/types-vals-vars.tex`, lines 51–52:

> Two types are identical if and only if they are the same kind and their names and static arguments (if any) are identical.

Route C The same under route C. A return to the text before this change would leave the identity of two size expressions unsaid, as the compiled type checker read it before 9 October 2026: a size with arithmetic in it refused, and a static parameter counted as different from every numeral.

I.1.41 The type of `self` in a trait with a self-type parameter

Affected sections Section 13.2; Section 10.1; Section 10.2.

Change In Section 13.2, a sentence after the one on the self-type idiom: in a declaration of a trait T with no `comprises` clause, or one that lists only "...", and a self-type parameter X, `self` has the type $T \cap X$ in the bodies of the trait's methods, and the `self` parameter keeps the type T in the signature of each method declaration. In Section 10.1, a paragraph after the one on `comprises` clauses defines a self-type parameter: a static parameter of a trait with no `comprises` clause, or one that lists only "...", that is bounded by the trait at its own static parameters, or by an instantiation of a trait with a self-type parameter that the trait also lists in its `extends` clause, with that static parameter as the static argument for the self-type parameter. The paragraph requires every trait or object declaration and every object expression that extends an instantiation of such a trait to be a subtype of the static argument of that instantiation for the self-type parameter, a trait standing as the type of `self` in its bodies, the intersection of its type with the types its `comprises` clause lists or with its own self-type parameter, and gives an example marked "Not allowed". In Section 10.2, a box after the sentence that a functional method's `self` parameter implicitly has the type of the enclosing trait or object says that the sentence still holds of the declaration's signature. Each of the three passages has a box.

Rationale This change does not follow from route A; it follows the revival's decision of 13 October 2026 on the bodies of the self-typed traits, and its second form of a self-type parameter, through a trait that the trait extends, follows a default taken under that decision on 14 October 2026. the Fortress standard libraries

declare their orders and their algebra as traits whose static parameter is bounded by the trait itself, as in `trait` Integral⟦I `extends` Integral⟦I⟧⟧, and give them default bodies that return `self` where that parameter is declared, or pass it where the parameter is expected: the team's minimal complete definitions, such as `MIN` on StandardTotalOrder and *floor* on Integral. The interpreter runs them; the compiled type checker refused 29 of them, since it typed `self` as the trait. A bound constrains the static argument, not `self`: an object may extend Integral⟦$\mathbb{Z}32$⟧ without being a $\mathbb{Z}32$, and a body that treats `self` as the parameter is sound only if every declaration that extends the trait is a subtype of its static argument for that parameter. Scala states that rule for its declared self types ("The self type of a class or object must conform to the self types of all classes which are inherited by the template", *Scala Language Specification 2.13*, section 5.1); the revival reads the self type from the bound, which the Fortress standard libraries already write, as the team's compiled type checker reads it from a `comprises` clause (Section 13.2), a reading that the team's gated test `ProjectFortress/compiler_tests/XXX10k.test` pins. Naden's account of the Return Type Rule (2012) asked that the self-type idiom become "a separate construct which cannot be used as a first class type, but instead only in bounds" (`Papers/Types/journal/justificationOfRTR.tex`, lines 626–630). The signatures keep the type of the trait because the type group's account of overloading of 2019 (Park, Hong, Steele and Ryu, POPL 2019) assumes, for the completeness of its matching of types, that no declared type holds a type parameter inside an intersection (pp. 11:21–22), and its Return Type Rule compares declared domains (p. 11:13): with the intersection in the signatures, the domains of the two declarations of `CMP` on StandardPartialOrder and StandardTotalOrder became equivalent, and the compiled type checker reported 18 errors of the Return Type Rule that the declarations do not make. The test by which the compiled type checker's overloading check decides whether the parameter types of two declarations exclude each other reads the `self` parameter of a functional method of such a trait as $T \cap X$ all the same, following the revival's decision of 23 October 2026: every declaration that extends an instantiation of T is a subtype of its static argument for X, so every value that the method takes as `self` is an X, and the two reasons above bear on which declaration is more specific and on the Return Type Rule, not on whether a value lies in both domains. With the overloads of symbolic operators checked, the plain trait type in that test, beside an existential reduction that read no bounds (Section I.1.29), made the compiled type checker refuse 8 pairs of the Fortress standard libraries between the $+$ and $-$ of AdditiveGroup, which Vector and Matrix extend, and the operators between an array and a number, though no call fits both: a vector's own $+$ takes two vectors, the other an array and a number, and arrays exclude numbers. The second form of a self-type parameter keeps valid as written the mixins of the Fortress standard libraries that the team wrote in 2009, ContainmentBySubset and ContainmentByComparison, whose static parameter is bounded by the trait Containment, which they extend at it. The full reasoning is in `explorations/reviews/self-typed-bodies-judgement.md` and `explorations/compile-ladder/rung-self-type/decision-record.md`, and for the test of exclusion in `explorations/compile-ladder/rung-self-type-exclusion/REPORT.md`, in the revival's repository.

Effect The compiled type checker types `self` as $T \cap X$ in the bodies of the dotted and functional methods of a trait with a self parameter X, keeps T in the signatures that overloading, the Return Type Rule and the choice of a declaration at a call read, except that the test by which its overloading check decides whether two declarations' parameter types exclude each other reads the `self` parameter of a functional method as $T \cap X$ where the self type comes from the bound, and refuses a trait or object declaration or an object expression that the new paragraph of Section 10.1 does not allow, with the message "D extends $C⟦A⟧$, whose self parameter is A, but D is not a subtype of A.". The interpreter has no static types: it runs the bodies as before, and loads a declaration that the paragraph refuses. The code generator compiles a default body that returns `self` as the self parameter with no conversion to the type of the instance, so a program that runs such a body, which the compiled type checker now accepts, with the parameter at an object type fails the JVM's verification when the body's class is loaded (row 695 of the revival's gap ledger).

Original text `Specification-1.0-frozen/basic/expressions/var-ref.tex`, lines 53–54 and 64–69, which stand unchanged and are now followed by the new sentence:

> The type of `self` is the type of the trait or object being declared by the innermost enclosing trait or

object declaration or object expression.

and

If the innermost enclosing such construct is a trait declaration (for a trait called, say, 'T'), and that trait declaration has a `comprises` clause, and 'U_1', 'U_2', ..., 'U_n' are the traits mentioned in the `comprises` clause, then the type of `self` is instead "$T \cap (U_1 \cup U_2 \cup \ldots \cup U_\mathrm{n})$".

The paragraph of Section 10.1 is new. `Specification-1.0-frozen/basic/traits.tex`, line 486, the end of the sentence of Section 10.2 that the new box follows, stands unchanged:

and implicitly has the type of the enclosing trait or object.

Route C Under route C a self-typed trait is read by a `comprises` clause that lists its static parameter, as the compiled path's own library declares its orders (`Library/CompilerAlgebra.fsi`, lines 16 and 24, in the revival's repository); under such a clause the self-type idiom of Section 13.2 types `self`, and this rule is not reached. A return to that form would need the clause on the self-typed traits of the Fortress standard libraries, which would also close each of them to its static parameter; a way for the interpreter past its stack overflow on a clause that lists a static parameter (row 407 of the revival's gap ledger); and leave in Section 10.1 for a static parameter as a listed type, which the type group's calculus of 2012 does not admit.

I.1.42 The index type of a subscript

Affected sections Section 6.2; Section 6.3; Section 6.5; Section 10.1; Section 2.15; Chapter 31; and, in a draft build, the item on array indexing among the proposed features of the appendix "Internal Document".

Change the Fortress standard libraries declare the trait Index, `trait` Index `extends` Any, which $\mathbb{Z}32$ extends and Range excludes, and bound every index parameter of their generic indexed types by it, I `extends` Index: $\mathrm{Indexed}$, its delegated and mutable forms, the array traits, the full ranges, and the mapped, reversed and index-value views over an indexed generator. $\mathrm{BoundedRange}$, whose index type stays unbounded since the open range $(:)$ is a range over Any, keeps its narrowed $atMost$, whose result is a full range, under the method condition `where` $\{I$ `extends` $\mathrm{Index}\}$. In Section 6.5, a sentence says that a tuple type whose element types are all subtypes of Index is a subtype of Index, and no subtype of Object; the sentence on the exclusion of non-tuple types names Index beside Any, and says that a tuple type excludes Index if and only if one of its element types excludes Index. In Section 6.3, the sentence that every trait type excludes every tuple type excepts Index. In Section 6.2, the immediate subtypes of Any include Index. In Section 10.1, the sentence on the implicit extension of Object names Index beside Any and Object, a sentence says that a trait that extends Index and no other trait but Any implicitly extends Object too, and the hierarchy is rooted at Object and Index. In Chapter 31, the immediate subtypes of the trait Any include Index. In the same chapter, the sentence that every trait other than Any and Object extends Object, which is the revival's (Section I.1.8), names Index beside them. In Section 2.15, the hierarchy of traits is rooted at the traits Object and Index. The item on array indexing among the proposed features gains a box, and so does each changed passage.

Rationale This change does not follow from route A; it follows the revival's decision of 23 October 2026 on the index type of the Fortress standard libraries's subscripts, and its place of Index outside Object and its $atMost$ follow the revival's answers of the same day. The bound finishes the team's wish. Their item on array indexing calls $\mathrm{Indexed}$'s two subscripts, one over an index and one over a range of indices, "not really a valid overloading", and goes on: "We want to say that the type variable I extends a type, say, Index which either excludes $\mathrm{Range}[\![I]\!]$ or is a supertype of $\mathrm{Range}[\![I]\!]$" (`Specification-1.0-frozen/appendices/future.tex`, lines 200–203). With I bounded by an Index that Range excludes, the two subscripts never apply to one value, and the pair is a valid overloading at every instance (Section 24.5); unbounded, at the instance Any the range subscript is the more specific, and its result is no element, which the Return Type Rule refuses. The compiled type checker, made to check the overloading of symbolic operators over the Fortress standard libraries, reported 124 such errors in the generic indexed types, and none at the bound with the rule below. The tuple rule is the

revival's departure from the designers' text on tuple types. The team named the obstacle themselves: "However, we cannot say that because when we invoke that subscripting method, we write $a[1, 2, 3]$ where the arguments are passed as a tuple, $(1, 2, 3)$. If we want to use Index, we need a magic to convert that tuple argument expression to something of type Index." (lines 204–207). They asked for a conversion, and a conversion would change every index value that a subscript passes; the rule makes the tuple type a subtype of Index instead, a typing rule that changes no value. The Working Draft's text on tuple types forbids it (quoted below), and so does the team's later Types chapter, twice, its macros for the void and bottom types written out: "Every trait type excludes $()$, $\mathrm{BottomType}$ and every tuple type." and "A tuple type excludes $\mathrm{BottomType}$, $()$, and every trait type, arrow type or function type." (Luchangco, 2012, `Documentation/Specification/Prose/Language/types.tick`, lines 341 and 424, in the revival's repository). Index stands outside Object, as Any and Object do, so that a tuple of indices is no Object (Section I.1.30); the specification's library declares its trait of all tuples so too, `trait` Tuple `extends` Any `excludes` $\{\,\mathrm{Object}\,\}$ (Section 31.3). Two ways stay open, each with its own cost: a constraint `where` $\{I$ `excludes` $\mathrm{Range}[\![I]\!]\}$, which the team left as an open question (the item "Type variables in `excludes` clauses" of the same appendix) and which is not among the Working Draft's forms of constraint; and subscripts by a range only at the index types that the Fortress standard libraries fix, as the arrays of rank one to three, the strings, Just and $\mathrm{Nothing}$ already have them, at the price of slicing in the generic views. A reversal would remove the rule, the bound and Index, and leave the 124 errors standing, or take one of the two ways. The full reasoning is in `explorations/compile-ladder/rung-index-bound/decision-record.md` in the revival's repository.

Effect The interpreter and the compiled type checker, when it checks the Fortress standard libraries, read the rule and the exclusion by the qualified name of the Fortress standard libraries's trait, so a program's own trait named Index is not read as it, and both keep Index outside Object; a program with an index type of its own declares it an Index. A program's own generic declaration that instantiates an indexed type of the Fortress standard libraries at its own type parameter bounds that parameter by Index; the compiled type checker refuses it unbounded, "The static argument I does not satisfy the corresponding bound Index", and the interpreter runs it. The compiled type checker refuses a tuple expression of indices passed to a function whose one parameter's type is Index, as $f(i, j)$ for $f(x\colon \mathrm{Index})$, though it takes a variable of the tuple type (row 805 of the revival's gap ledger); the interpreter runs it. A numeral is no Index to the compiled type checker, which types it $\mathrm{IntLiteral}$ (Section I.1.36); the interpreter, whose numerals are values of the integer types (row 364), passes one to a parameter of type Index. The interpreter loads a function over Index beside a function over a tuple type whose element types are neither subtypes of Index nor exclude it, as $h(x\colon \mathrm{Index})$ beside $h(x\colon (\mathrm{Foo}, \mathrm{Foo}))$ for a trait Foo, which the Meet Rule refuses (Section 24.4) and the compiled type checker refuses (row 806 of the revival's gap ledger). The interpreter refuses a program's own type named Index as ambiguous with the Fortress standard libraries's, as it refuses one named $\mathrm{Indexed}$. The compiled path still compiles programs against its own library, which declares no Index, until it is switched over to the interpreter's. The code generator refuses a method with a `where` clause of its own (row 701 of the revival's gap ledger), so after the switch-over $\mathrm{BoundedRange}$'s $atMost$ waits on that row, beside the two $distribute$ methods of the Fortress standard libraries's sum reduction.

Original text `Specification-1.0-frozen/basic/types-vals-vars.tex`, lines 134–136, 215–216 and 272–273, which the change revises:

> We say that a value is *an instance of* its runtime type and of every supertype of its runtime type; immediate subtypes of Any comprises of tuple types, arrow types, $()$, and Object.
>
> Every trait type also excludes every arrow type and every tuple type, and the special types $()$ and $\mathrm{BottomType}$.
>
> A tuple type excludes any non-tuple type other than Any.

lines 263–265, which stand unchanged and are now followed by the rule:

> Every tuple type is a subtype of Any. No other type encompasses all tuple types. Tuple types cannot be extended by trait types.

`Specification-1.0-frozen/basic/traits.tex`, lines 181–183 and 193–194, which the change revises:

> In addition, every trait except Any and Object implicitly extends the trait Object if it does not do so explicitly.
>
> This relation must form an acyclic hierarchy rooted at trait Object.

`Specification-1.0-frozen/basic-lib/objects.tex`, lines 19–20, which the change revises:

> The immediate subtypes of the trait Any are Object, Tuple, and $()$.

and `Specification-1.0-frozen/preliminaries/overview.tex`, lines 1061–1062, which the change revises:

> Traits in Fortress exist in a multiple inheritance hierarchy rooted at trait Object.

The sentence of Chapter 31 on the traits that do not extend Object has no counterpart in the Working Draft's chapter; it restates the sentence of `Specification-1.0-frozen/basic/traits.tex`, lines 181–183, quoted above. The box in the appendix of proposed features is new.

Route C The same under route C.

I.1.43 Passages not yet revised

No passage that instantiation exclusion or the reorganized numeric types contradict is known to be unrevised. The rule by which a generic call over two different integer types infers the narrowest type that both coerce into is written into Chapter 20 together with its implementation (Section I.1.26); the passages that chapter left unrevised, the sentences of Chapter 15 by which static parameters were inferred before the applicability of a declaration was checked and before declarations were compared, the implicit bound of Section 12.1, and the rule of Chapter 24 on a naked type parameter of bound Any, are revised with the overloading chapters (Section I.1.29, Section I.1.30). The instance of a `nat` or `int` parameter of a declaration that a call reaches only by dispatch, where nothing fixes it at run time, is not described; the rule of Section 15.5 is for type parameters (row 446 of the revival's gap ledger). The sentence of Section 12.6 that a `where`-clause variable may not appear as a static argument in an `extends` clause also refuses a variable that a constraint fixes to one type; whether it is to be narrowed is not settled. The Meet Rule for dotted methods (Section 24.4) asks for a declaration on the intersection of two parameter types; whether declarations that together cover the intersection satisfy it too is not settled. The type of a static call whose argument's static type stands between two traits with `comprises` clauses, where its most specific declarations, or a declaration more specific than them that the call may run, have static parameters, is not given (Section I.1.34). Whether two parameter types exclude each other when the only static arguments at which they could meet are ones that a bound rules out is not settled: the compiled type checker takes them to exclude where the static argument that the two parameter types force is a named type that fails the bound, and where a static parameter that stands at the top of their meet, read as within its declared bounds, leaves the meet empty, as for row 713's pair, which the interpreter refuses at load; it reads no bound of a static parameter that stands inside a static argument (Section 24.5). Whether a `nat` parameter, which may appear wherever a variable of type $\mathbb{N}32$ can (Section 12.2), may be a component of a range is settled (Section I.1.26): it converts to $\mathbb{Z}32$ as an integer numeral does, so the static ranges of Section 13.16 and the examples whose ranges a `nat` parameter bounds stand as written. Before the number chapters were revised, this section said:

> The number chapters, Chapter 33 (whose integers are Section 33.2) and Chapter 41, are not yet revised. Their nested tower, in which each integer type is a subtype of the rational one above it and each carries its algebra at its own type, and the conditional self-extension of $\mathrm{RationalQuantity}$, are refused by instantiation exclusion, as the compiled type checker refuses the library's nested tower today. They will be revised together with the reorganization of the library's numeric types that route A decides, and not before, because a specification ahead of its library is as much an open discrepancy as one behind it. Until then the rule refuses the tower these chapters describe.

I.1.44 Route C, the alternative not taken

The team's 2012 patent applications (US 8,843,887 and US 8,898,632; Chase, Steele, Naden, Hilburn, and Luchangco) relax the rule for a self-typed parameterized trait: a type may be a subtype of several of its instantiations when they form a chain. The revival built that rule whole as an experiment and measured it. It closes the unsoundness that instantiation exclusion guards against only together with a rule on return types over every level of a chain and a change in how a call chooses among generic methods specialised to different instantiations: 188 lines of code in five files of the compiled type checker and code generator. Reversing route A for route C would also need a method per level of a chain in the code generator, work in the interpreter, a way past a stack overflow of the type checker on the self-typing clauses, and seven library declarations, besides undoing the reorganization of the library's numeric types. Route C rescues none of the `where`-clause examples of Section 12.6 nor the Nothing of Section 39.2, each of which places one type below infinitely many instantiations, which no chain contains. Under route C the numeric chapters could have kept the nesting of their widths, and their original text, quoted in Section I.1.11 and the entries after it and kept in the Working Draft of February 2011, is route C's for that nesting; route C rescues neither the refined number types nor RationalQuantity (Section I.1.11). The record of the experiment is `explorations/reviews/mie-probes/route-c-experiment.md` in the revival's repository.

Appendix J

Internal Document

J.1 Frequently Asked Questions

- Are spaces by tabulation allowed?

  No. It is a static error for a Fortress program to contain CHARACTER TABULATION (U+0009) or LINE TABULATION (U+000B) outside a comment. See Section 4.1.

- Cyclic definitions of top-level variables

- `par` and `seq` and `for`

- "immediate supertype" vs "explicit supertype" and "immediately extend" vs "explicitly extend"

- The term "structural type" historically means a type that corresponds to the structure of its values. Tuple types are structural, as are record types and arrow types (depending on your view of the structure of a function). Union, intersection, and fixed point types can be either structural or nominal depending on the types they're composed of.

  I suggest we use the term "structured type" to refer to types composed of other types, like those you mention above, as well as generic types. A purely nominal type system can include structured types, but not structural types. If it includes structured types, it's possible for two syntactically distinct types to have the same extension (e.g., A U B and B U A). In fact, if there are type aliases, even two unstructured types can have the same extension.

  Note, BTW, that all structural types are (necessarily) structured. A type is structural by virtue of the fact that its structure corresponds to the structure of its values.

  Also, perhaps arrow types are better described as nominal. It's really not the structure of the function that they represent; it's the behavior of the function. We can think of the "name" of the type as "-¿"; it's a generic type parameterized by its input and output type. They're often characterized as structural, but this may be more of a historical accident; they first appeared in languages with mostly structural types.

- Spawn is considered an IO operation. Why is that?

  We did this because we don't want you to use spawn within an atomic expression. A lot of tricky issues arise when you do this, and we just want to avoid them for now. Eventually, we will probably relax this restriction (as we may also want to do for IO operations more generally).

- How can a programmer create variable-sized arrays in Fortress, and how to take advantage of the static bounds checking offered by the language at the same time?

  `http://projectfortress.sun.com/Projects/Community/wiki/AnySizeArrays`

- Why does Fortress have a multiple dispatch instead of a single dispatch?

  We want to disallow ambiguous calls such as the following:

  ```
  trait T
      f(ℤ32) = 1
  end
  object S extends T
      f(Number) = 2
  end
  z: ℤ32 = 3
  S.(z)
  ```

- A function type may be an intersection of several arrow types instead of an arrow type.
- Rendering issues vs ASCII conversion issues
- Functional methods rewriting rules are for scoping not for overloading checking.
- Operator: `juxtaposition`
  - It is a reserved word because it cannot be used as an identifier.
  - It is the only operator name that is not composed of ALLCAPS.
- Hacks: arrow types in mathematical notation
  - Functions can be declared with arrow types unless they have generic types.
  - Arrow types can use $\times$ instead of tuple types unless they are generic types.
  - Method declarations cannot use these hacks.
  - Field declarations can use arrow types unless they have generic types.
- Terminology
  - A declaration is either an abstract declaration or a concrete declarations which is a definition.
  - Special reserved words are "keywords". Special words that are not reserved include context-sensitive keywords such as `name`. We want to use $name$ as an identifier in value contexts and use `name` as a keyword in type contexts. However, we don't support this yet. Currently, `name` is a special reserved word. Finally, reserved words that are not special include not-yet special reserved words such as `private` and forbidden identifiers such as $alpha$.
- The Fortress Core Library and the Fortress Standard Library
  - While the Fortress Standard Library can be written in Fortress, the Fortress Core Library cannot be written in real Fortress. We provide the "fictitious" Fortress Core Library in Fortress to help programmers understand the Fortress Core Library.
  - While the Fortress Standard Library can be redefined in Fortress, the Fortress Core Library cannot be redefined in real Fortress. For example, the top-level variables $true$ and $false$ declared in the Fortress Core Library cannot be redefined.
  - While unit operators such as per, cubic, and $\mathrm{squared}$ are defined in the Fortress Standard Library and can be redefined, they are operators only in the unit contexts.

- Why not single inheritance between objects?
  - We can do that but...
  - we can mimic the functionality by using wrapped fields;
  - we can enjoy nice property of objects being leaves of type hierarchy such as excluding each other.
- Meaning of extension: how is it different from subtyping?

  Extension means there exists an `extends` clause.
- BottomType
  - Can a programmer write BottomType? No.
  - (BottomType...) is not BottomType.
- Spaces

  We allow spaces between enclosing operators and static arguments: `<| [\ZZ32\] 3 |>`. Also for the functional names and static arguments.
- Abstract methods in a trait and implementing concrete methods in an object

  An abstract method can be implemented by concrete methods whose return types are subtypes of the abstract method.
- `label` / `exit` across object / function expressions (07/28/08)
  - one-shot continuation semantics
  - How to do the closure conversion?

    ```
    label foo
      object extends Bar
        m() = exit foo with 5
      end
    end
    ```

    is desugared to

    ```
    object Bar′(xit: ℤ32 → BottomType) extends Bar
      m() = xit(5)
    end
    label foo
      Bar′(fn (e : ℤ32): BottomType ⇒ exit foo with e)
    end
    ```

    Even though Fortress programmers are not allowed to write BottomType, the desugarer can use it internally.
- How can I re-export stuff I imported without having to maintain cut-and-pasted declarations in my api?

```
api A
  trait T
  f(T)
end
component C
  import A.{...}
```

```
  object O extends T
  f(O)
end
component D
  import A.{T}
  object O extends T
  f(O)
end
```

J.2 Proposed Features

This section includes the technical issues that are proposed but not yet fully discussed.

J.2.1 Syntax and Evaluation

- Extension of Bracketmania for multiset notation

  Guy's email from 11/05/09 titled "Technical proposal: small extension of Bracketmania"

- Mathematical notation for alternatives

  Victor's email from 03/27/07 15:34 titled "More mathematical syntax"

- Jan's short note #82: "Unboxed Types: the design space" (07/24/07)
  - Only heap objects have a notion of object identity, and for heap objects SEQUIV is object identity. All types have a notion of strict equivalence, which can be based upon content.
  - What do we mean by the `value` modifier? Do we mean an "unboxed type" with the `value` modifier? Do we mean an "immutable object" with the `value` modifier? We mean an "unboxed type" with the `value` modifier.
  - Flattened representation
  - Copying semantics
  - A fixed-size representation
  - Header-freedom: If we have a field or array of unboxed object type, the representation should not need to carry around header or type information as part of the representation.
  - Immutability: It is not necessary for unboxed objects to be immutable.
  - Object identity: No object identity. Strict equivalence is defined recursively for value objects, based upon content.
  - Unboxed traits: What happens when we have a field or an array element whose type is a trait type, one of whose instances is an unboxed type? Box the object or similar techniques.
  - Size of an unboxed object which is recursively defined with unboxed traits. Forbid recursive definitions in value objects.
  - Do we want to permit arrays of fixed bounds to be unboxed? No. Unboxed arrays are a different other thing. Among other things, an array of fixed bounds may be a segment of another array.

  - What happens if we mutate a LinearSequence? Only initializing writes are permitted and the sequence is then frozen.
  - Guy's email titled "Re: value vs unboxed" on September 3, 2008
- Mutual recursion in value objects (08/02/07)
  - Eric: There's a technical problem with our decision to forbid recursive definitions in value objects. What if there is mutual recursion spanning component boundaries, and the recursion isn't evident via the APIs? If there's an error for recursive value objects in components but no error for recursive value objects spanning component boundaries, it doesn't look like consistent.
  - Jan: Module recursion is a bad idea.
  - David: Would it make sense, after all, to mandate some ordering of APIs?
  - Dan: How about just forcing a value object to declare its contents explicitly in the API?
  - Eric: Value objects should be layed out in APIs.
  - David: Link-time error.
  - If there is no cycle in value objects spanning component boundaries, the programmers can get precise size information. If there is a cycle, linker will have a way to break the cycle but, in this case, there is no size guarantee. Contingent upon Eric's agreement.
  - Guy's email titled "More on value objects" on September 4, 2008

J.2.2 Types

- Guy's proposal for making tensors the principal user-level indexed aggregate type in Fortress (06/27/07)

  Guy's email titled "Arrays, matrices, and tensors (short)" on 12/15/06

  We discussed the "story #1: Arrays". Among others, we discussed how we would require that "it is forbidden for a subscript on the left-hand-side of an assignment to generate the same integer twice." There were three options: 1) require extending a $\mathrm{NoDuplicate}$ trait; 2) give an undefined semantics; and 3) give some operational semantics. Jan and Guy were for the option 3). We decided to continue the discussion of the "story #1: Arrays" after Guy's detailed write up. Meanwhile Jan may work on a semantics for the option 3) and all may discuss syntax issues via emails. Other stories will be discussed after story #1.

- More powerful type aliases

  For floating-point numbers, Fortress supports types $\mathbb{R}32$ and $\mathbb{R}64$ to be 32 and 64-bit IEEE 754 floating-point numbers respectively, and defines two functions on types: $\mathrm{Double}[\![F]\!]$ is a floating-point type twice the size of the floating-point type F, and $\mathrm{Extended}[\![F]\!]$ is a floating-point type sufficiently larger than the floating-point type F to perform summations of "reasonable" size.[1]

  $$\texttt{type}\ \mathrm{Double}[\![F\ \texttt{extends}\ \mathrm{FloatNumber}[\![e, s, \ldots]\!]]\!]\ \texttt{where}\ \{\texttt{nat}\ e, \texttt{nat}\ s, \ldots\} = \mathrm{FloatNumber}[\![e+3, 2s-6, \ldots]\!]$$

- _

  We're considering supporting _ (pronounced "whatever") not only for variables but also for type variables.

  We have several use cases for this:

[1] This formulation of floating-point types follows a proposal under consideration by the IEEE 754 committee.

1. Eric doesn't need to define extra trait List in addition to $\mathrm{List}[\![T]\!]$.
2. Jan can use $\mathrm{List}[\![_]\!]$ in a `typecase` expression.
3. Jan can use $_$ for redundant static parameters (required by overloading restrictions) for overloaded methods.
4. We have several proposals for supporting some forms of pattern matching including David and Sam's and Eric's.

This requires extending the left-hand-sides of `typecase` clauses to be binding places of type variables.

There are several problems:

1. It's unclear whether $\mathrm{List}[\![\mathrm{String}]\!]$ is a binding place or a reference of String in a `typecase` expression.
2. Using $_$ in any place where a type variable can occur would have the same power with "wildcards" in Java.
3. Restricting occurences of $_$ seems to be ad hoc.

Jan's new example:

$g(x : A[\![T]\!])$ `where` $\{\, T$ `extends` $\mathrm{Object}\,\} =$
  `typecase` x `of`
    $A[\![U]\!]$ `where` $\{\, U$ `extends` $\mathrm{Object}\,\} \Rightarrow \ldots$
    $\ldots$
  `end`

And the interesting question is to what extent we're allowed to use U (or not) given the existence of stuff like Empty. My hope had been that we could do stuff like type-equality tests this way, but that hope may have been a vain one. And this syntax is just a pipe dream which I'm faking using overloading (and the dreaded $__\mathrm{Proxy}$ type) for the moment.

J.2.3 Traits and Objects

- overriding (05/23/07)

  Victor's email titled "Re: Method dispatch and overriding" sent on 05/22/07

- overriding (03/10/08)

```
trait A
  m(x : ℤ) = 3
end
trait B extends A
  override m(x: Object) = 4
end
trait C extends B
  m(x: Number) = 6
end
trait D extends {A, B} end
```

The method declaration in A is not inherited in C because it is overriden in B.

Do we want to allow to have `override` when the parameter types of the two declarations are the same?

- Array indexing

  ```
  trait Indexed⟦E, I⟧ extends Generator⟦E⟧
    opr [i: I]: E
    opr [r : Range⟦I⟧]: Indexed⟦E, I⟧
    ...
  end
  ```

  The above overloaded declarations are not really a valid overloading. We want to say that the type variable I extends a type, say, Index which either excludes Range⟦I⟧ or is a supertype of Range⟦I⟧. However, we cannot say that because when we invoke that subscripting method, we write $a[1, 2, 3]$ where the arguments are passed as a tuple, $(1, 2, 3)$. If we want to use Index, we need a magic to convert that tuple argument expression to something of type Index.

  > REVISED BY THE 2026 REVIVAL SEE Section I.1.42
  >
  > Done, by subtyping in place of a conversion: the library declares `trait` Index `extends` Any, $\mathbb{Z}32$ extends Index, Range excludes it, and every index parameter of the library's generic indexed types is bounded I `extends` Index; a tuple type whose element types are all subtypes of Index is a subtype of Index (Section 6.5), so $a[1, 2, 3]$ passes its tuple as before.

- Do we want to allow object trait types to appear in `extends` clauses? (Jan's email titled "Re: Question about not_passing_yet/extendsParam.fss" sent on 14 Nov 2007)

- Type variables in `excludes` clauses

  ```
  trait Indexed⟦I extends Equality⟧ excludes Indexed⟦U⟧ where ⟦U⟧{I excludes U}
    ...
  end
  ```

J.2.4 Functions and Overloading

- Applicability to Named Functional Calls

  whether a "named functional call" is rewritten to a "dotted method call" only if the function name is the name of a method provided by the enclosing trait/object declaration or object expression (this requires a slight rewriting of the relevant paragraph).

- Relaxing restrictions on static parameters of overloaded functionals or replace static parameters with where clauses as much as possible

  - Jan's email titled "Re: Issues related to "spec" vs "nice" overloading of generics" on 05/21/07
  - Jan's email titled "Another overloading conundrum" on 04/06/07
  - Jan's example

    $array_1⟦T$ `extends` Object, `nat` $s_0⟧() : \text{Array1}⟦T, 0, s_0⟧ = __DefaultArray1⟦T, 0, s_0⟧()$
    $array_1⟦T$ `extends` Number, `nat` $s_0⟧() : \text{Vector}⟦T, s_0⟧ = vector⟦T, s_0⟧()$

  - Marc's example

    $conjugate⟦T$ `extends` Number$⟧(x : \text{Complex}⟦T⟧) : \text{Complex}⟦T⟧$
    $conjugate⟦T$ `extends` RealNumber$⟧(x : T) : T$

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> Done: declarations of one name may have different static parameters, under the rules of Section 24.5. Jan's pair is still not a valid overloading, since the two declarations have the same parameter type, each applicable to every call to which the other is, which the No Duplicates Rule refuses; the Fortress standard libraries write it as one factory, $array_1[\![T, \texttt{nat}\ s_0]\!]()$, which chooses its result by a `typecase` on its type argument, a Vector where the argument is one of the eight number types ZZ32, ZZ64, NN32, NN64, ZZ, QQ, RR32 and RR64, and a plain array where it is any other type, Number among them, since a Vector of Number is not well-formed under the bound that the Fortress standard libraries give the element type of a Vector (Section 12.1).

- Overloaded functionals with different `throws` clauses
- Overloading with static parameters
  - David's "nice" overloading proposal (His email titled "Comparison of generic overloading rules" sent on 05/18/07)
  - Victor's email titled "Re: Overloading and the open world assumption" sent on 11/08/07
  - What is the exclusion rule for a pair of overloaded declarations with different static parameters?
  - Do we want to consider only the exclusion between ground types or also the exclusion between the bounds of static parameters?

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> Answered by the existential reduction of Section 24.5: each static parameter stands for any type within its bound, and where the two parameter types, or their elements at one position, are or extend instances of one parameterized trait, instantiation exclusion lets them meet only where the static arguments of those instances are the same. Whether a bound that rules out the only static arguments at which two parameter types could meet makes them exclude each other is not settled (Section I.1.43).

- Identifying the intersection of any two types with `comprises` clauses with the union of their common subtypes

```
trait S comprises {U, V} end
trait T comprises {V, W} end
trait U extends S excludes W end
trait V extends {S, T} end
trait W extends T end
```

S&T ~ ~ V We agreed to revise the Meet rule to address this with the coverage check.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.34
>
> Done: V covers the intersection of S and T (Section 6.10), and the Meet Rule is also satisfied by declarations that together cover the intersection of two parameter types (Section 24.4).

- overloading in multiple scopes
- overloading with different static parameters

> REVISED BY THE 2026 REVIVAL SEE Section I.1.29
>
> Done (Section 24.5).

- exporting partial overloading

I'm hoping we export all of an overloading, or none of it. – Jan

This a reasonable restriction that I'm happy to back, unless someone has both a use case and a soundness proof for a less restrictive rule. :-) – Eric

The use case I have in mind is that within a component, you might manipulate internal data types, and use overloading on those types with methods that are also exported. I don't have a specific example in mind, but this seems like a useful and natural feature to have. So I'd like to revisit this issue sometime, but not before 1.0. :) – Victor

J.2.5 Expressions

- Pattern matching (06/18/09)
- Generator clauses

  $inferenceRules = constructiveRules \cup \{\, lawOfExcludedMiddle \mid classicalLogic = true \,\}$

  - Status quo: must begin with a generator binding
  - Proposal: allow a list of filter expressions (Guy's email "Re: comprehension confusion" on 09/10/08)
  - Question: simple syntax for generator clause lists

- Comprehension syntax (Steve)

  - "What I really want is a different comprehension syntax, analogous to `BIG` syntax, more aligned with causality, and similar to the way programmers write loops:

    $$\begin{array}{l} genParaffins(n : \mathbb{Z}32) = \\ \quad \langle\, [radList \leftarrow genRadicals(n-1)] \\ \quad\quad quotient \langle\, [r \leftarrow radList] \,\mathrm{Paraffin}(H,H,H,r) \,\rangle \\ \quad \rangle \end{array}$$

    In case I haven't already gotten myself into hot water, we can have both I suppose, unifying comprehension syntax and `BIG` syntax:

    $$\begin{array}{l} \texttt{BIG}[generators\ and\ filters]\ expression \mid more\ generators\ and\ filters\ \texttt{end} \\ \langle\, [generators\ and\ filters]\ expression \mid more\ generators\ and\ filters \,\rangle \end{array}$$

    I think I prefer just having generator and filters in [] to the left."

  - Guy comments: Because expressions and generators have mutually exclusive syntax, it should in principle be trivial to allow either $\{\, expression \mid generators \,\}$ or $\{\, generators \mid expression \,\}$ to be recognized as a comprehension. However, in practice, people might find it confusing. Taking a cue from Python, I wonder if we could support both $\{\, expression \mid generators \,\}$ and $\{\, \texttt{for}\ generators \mid expression \,\}$?

  - Proposals

    * $\{\, generators \mid expression \,\}$
    * $\{\, \texttt{for}\ generators \mid expression \,\}$
    * $\{\, \texttt{for}\ generators\ collect\ expression \,\}$
    * $\texttt{for}\ generators\ collect \{\, expression \,\}$

- Exiting from a function expression

  Jan's email from 06/05/08 12:01 titled "Re: cleaning up exceptions hierarchy"

  Eric proposed to statically disallow it.

- Types of branch expressions: We discussed propagation of the type information around the expression to the subexpressions of a branch expression. We may want to include this propagation later.

  $z\colon \mathbb{Z}32$
  `if` p `then` z `else` 0 `end`
  `if` p `then` z `else` $\mathrm{Identity}[\![+]\!]$ `end`
  `if` p `then` 0 `else` $\mathrm{Identity}[\![+]\!]$ `end`
  $a\colon \mathbb{Z}32 =$ `if` p `then` z `else` $\texttt{FFFF0000FFFF}_{16}$ `end`

J.2.6 Operators and Coercions

- `BIG AND:` (Jan's email titled "Re: BIG AND:" on August 25, 2008)
- Conditional chaining operators (Victor's email titled "Re: Conditional chaining operators?" on August 19, 2009)
- Surprising behavior with coercion (Victor's email titled "Overloading rules and a proposed "change"")

J.2.7 Exceptions

- Exception behavior of `atomic` expressions in Fortress
  - Checked exceptions must be caught before they escape from the `atomic` expression, or it is a static error. If necessary, programmers can be wrap a checked exception in an unchecked exception to get it out of the transaction.
  - There is some syntax which says which checked exceptions are allowed to escape from an `atomic` expression.
  - They propagate to the context of the `atomic`, as above. This is the de facto behavior in the absence of a story about checked exceptions, since it's the same as the behavior for unchecked exceptions. (the status quo)
- Naked type variables in a throws clause or in a catch clause.

  Since a `throws` clause is a set, for full generality, we would probably need expressible union types at the call site, or a variable representing a "set" of types.
- When we have our $\mathrm{StackTraceElement}$ trait, add the following methods to the $\mathrm{Exception}$ trait:
  - $\mathit{fillInStackTrace}()\colon \mathrm{Exception}$: Called to initialize the stack trace data in a newly created exception. This method can be called more than once.
  - $\mathit{getStackTrace}()\colon \mathrm{StackTraceElement}[]$
  - $\mathit{setStackTrace}(\mathit{stackTrace}\colon \mathrm{StackTraceElement}[])\colon ()$:
- light-weight exceptions
- interaction between exceptions and transactions

- interaction between exceptions and multithreads
- syntactic sugar for expand/wrap/catch/unwrap idiom

  It occurs to me that we could have a very concise form of syntactic sugar for this expand/wrap/catch/unwrap idiom via a syntax expander (or maybe just primitive syntactic sugar if necessary), which I'll call "handle". This expander eta expands every function argument (promoting every checked exception of the function that is not anticipated by the callee), and wraps the whole function call in a `try`/`catch` block that unwraps the exceptions. For example, with the above definition of f, and the following function g:

  $g(\mathbb{Z}) : \mathbb{Z}$ `throws` $\{\text{IOFailure}, \text{SQLError}\} = \ldots$

  the expression:

  $handle[\![, f[\![\mathbb{Z}, \mathbb{Z}, 3]\!](g)\,]$

  expands to:

  ```
  try
     f⟦nullℤ, ℤ, 3⟧
           (fn n ⇒
              try gn
              catch e
                 SQLError ⇒throw WrappedException(e)
              end)
  catch e
     WrappedException⟦SQLError⟧ ⇒throw e.inner
  end
  ```

  Now, the context of the call site must deal with a thrown SQLError, or a static error is signaled. Moreover, without "handle" the call

  $f[\![\mathbb{Z}, \mathbb{Z}, 3]\!](g)$

  will result in a static error.

  – Eric

- `throws` clauses with naked type variables
- Inference for `throws` clauses

  No, and we shouldn't. `throws` clauses are documentation that a programmer should provide in a function's header. – Eric

  Okay, but I thought Jan wanted to do this. I thought Sukyoung did too, actually. Only for "internal" functions though. Probably I misunderstood what they were saying–it was related to that talk at OOPSLA last year. Anyway, I don't think we should worry about this too much until we get type inference for function/method calls worked out. – Victor

J.2.8 Components and APIs

- Top-level expressions

- APIs (Victor)

  Jan's email "Re: Correct API syntax for object type without constructor" July 9, 2007 8:14:09 AM

- Namespace of APIs (Victor)

  Because API names consist of one or more identifiers separated by dots, and "field access" (i.e., getter and setter invocations) may produce similar sequences, we need some rule here. One possibility is to forbid the declaration of any (simple) name in a component from being the same as the first identifier of any name of an imported API.

```
api A.B
  f() : ()
end

component C
  export Executable
  import A.B
  object O
    f() = ()
  end
  object A
    B: T = O
  end
  run(args) = A.B.f()
end
```

  We agreed on the high-level idea but the details should be worked out more.

- Automatically imported APIs (Victor's email titled "Automatically imported APIs (was: Dan's unresolved issues)" on 08/31/07)

- Section 22.8 **Extract and Install**

  "The set of APIs must be closed under imports." What if standard lib APIs are imported?

- Section 22.9 **Constrain**

  "If we constrain an API that is used by any other API exported by the component, then we must also constrain that other API." This seems problematic. Suppose that two components both export a standard lib. Is there any way to link them?

  (Possible solution: Perhaps we should provide a facility for "flipping" an API export to an import. A flipped API, when linked, could upgrade its constituent.)

- Exact matching between components and APIs

  - ```
    api A
      trait T end
      trait V extends T end
    end

    component C
    export A
      trait T end
      trait U extends T end
      trait V extends U end
    end
    ```
    ```

  - `trait` A `end`
    `private trait` B `end`
    `trait` C `extends` $\{A, B\}$ `end`

- Exporting multiple APIs with overlapping names

```
api A
  e() : ℤ32
  trait T
    f() : T
  end
end
api B
  e() : ℤ32
  trait T
    g() : ℤ32
  end
end
component G
  export {A, B}
  e() = 42
  trait A.T
    f() = self
  end
  trait B.T
    g() = 13
  end
end
```

- Why not allow components to import and export the same API, but prevent cyclic component linking (except for a special `linkCyclic` command)?
- Subtyping of APIs (Is this just imports? No.)

  `component` C `extends` $A(*$ an API $*)$
- Coercion of APIs? Widening?
- Deprecation of API members in new versions?
- Component/API names with the URL of the development team and the time stamp

J.2.9 Others

- Properties (Eric's email titled "Properties, concurrency" on 07/30/07)
- The reflection story for Fortress (Guy's email titled "Re: do you like object expression?" on August 26, 2008)
- Native stuff (Victor's email titled "Re: Thinking about native stuff some more" on 08/21/07)
  - foreign function interface
  - reflection mechanism
- Purity and "sandbox"

  - How about a "sandbox" function that executes a function for its value, throwing away all effects? It is a call site annotation to call a non-pure function in a pure context.
  - What do we want we purity? What semantics can programmers exploit?
  - Is purity a part of the type?
  - purity for enabling CSE?
  - purity for David's example?

- `idiom`

  Guy's email titled "Notes on today's meeting" on 12/12/05

  Idioms are like properties but they are hints to compilers. Like properties, idioms describe that the left-hand-side and right-hand-side of $\Rightarrow$ are same. Unlike properties, idioms instruct compilers to replace any occurrence of the left-hand-side of $\Rightarrow$ to the right-hand-side of $\Rightarrow$. For example,

  `idiom` $\forall(x{:}\,\mathbb{Z}, y : \mathbb{Z})\ \mathit{floor}(x/y) \Rightarrow \mathit{floordiv}(x, y)$
  `idiom` $x/N(log_2 N = k) \Rightarrow x \gg k$
  `idiom` $\mathit{floor}(x/(\texttt{MAXINT} - 1)) \Rightarrow 0$
  `test` $data[] = \{ -\texttt{MAXINT} - 1, -\texttt{MAXINT}, -\texttt{MAXINT} + 1, -2, -1, 0, 1, 2, \texttt{MAXINT} - 1, \texttt{MAXINT}, 43 \}$

  [The last two idioms were by David, intended as illustrations of screw cases.] a unit testing framework should check that $floor(x/y)$ and $floordiv(x, y)$ are same and the compiler should replace any occurrence of $floor(x/y)$ to $floordiv(x, y)$.

- Compliant Implementation Chapter

  I suppose it might help clarify things if we include a new chapter that states the properties that must hold for a "compliant implementation". For example, we could say things like "A compliant implementation provides a "compile" procedure that takes a sequence of Unicode characters as input. If this input is not a valid Fortress program, it is a static error. Otherwise..." – Eric

- Rewriting inheritance in terms of implicit declarations so that they are not parts of the "inner theory core".

- Equivalence and equality: Equivalence $\equiv$ ought to be built in (ie automagically overridden by every trait / object declaration). Equality $=$ is overridable and should be for apples-to-apples comparisons. By default equality is equivalence. It may be its own trait, with equivalence as the default definition. Right now equivalence on reference objects is pointer equality, and equivalence on value objects is equality of content. Equality on functions (and thus on value objects with function-typed fields) throws an exception. The first bone of contention is equivalence on immutable reference objects, with the following options:
  - Always return False. This makes the default definition of equality junk, and is strictly less expressive than equivalence on mutable reference objects. It seems bad, but was considered.
  - As any mutable reference object, using pointer equality (current semantics). This explicitly rules out combining indistinguishable instances of immutable reference objects—they can still be distinguished by $\equiv$ and thus must be kept distinct. Thus, for example, a string constant mentioned in the body of a loop must be copied at each loop iteration. This permits us to give a deterministic semantics.
  - If mutable reference objects would be considered equivalent, immutable reference objects would be considered equivalent. If immutable reference objects are equivalent, they must be otherwise semantically indistinguishable. This is *ad hoc* and lies between the previous solution and the next one.
  - By content, except that if any field has function type (or perhaps if equivalence checking of fields throws that exception, since subtyping may make this ambiguous) we treat it like a mutable reference object. This would rule out pointer equality unless we did hash consing, so immutable reference objects would become

more expensive than their mutable counterparts (either equivalence checks would get more expensive, or allocation would become more expensive).

- By content. This causes equivalence on immutable reference objects to fail if they contain functions, even when an equivalent mutable reference object would not. This may encourage programmers to throw in gratuitous unused mutable fields, and is thus probably a bad idea.

The second bone of contention is function equality. Note that changing the definition of function equality may make some of the choices above more or less usable.

- Always return false. Less bad than for objects, but still deceptive. Would mean that equality on value objects with function-typed fields always returns false, too.
- Always throw an exception (current semantics). Equality on value objects with function-typed fields always throws the same exception.
- When $f \equiv g$ the functions must be semantically interchangeable. Otherwise the functions may or may not be semantically interchangeable. Note that always returning false is a valid implementation of this semantics, and that there is non-determinism here.

A final issue to settle is what happens when we compare functions to other objects. Do we throw an exception, or just return "false"?

J.2.10 Tools

- The `api` tool

  Problem: The extracted APIs do not include comments.

  Two nodes: maintenance vs development

J.3 Examples

J.3.1 Scope of inherited method declarations

$f(x{:}\,\mathbb{Z}) = x + 1$
`trait` T
  $f(x{:}\,\mathbb{Z}){:}\,\mathbb{Z} = x + 2$
`end`
`object` O `extends` T
  $g(x{:}\,\mathbb{Z}) = f(x) + 3$
`end`

Allow this and $O.g(4) = 9$.

J.3.2 Scope of `self`

`trait` T
  `getter` $y(){:}\,\mathbb{Z}$
`end`
`object` $O(x{:}\,\mathbb{Z})$
  $n() =$ `object extends` T

```
            y: ℤ = self.x
        end
end
```

Note that the reference to `self` in the initialization expression for y cannot refer to the object being constructed, so if this is allowed, then that `self` must refer to the object O.

J.3.3 Object name shadowed by methods

```
object O(x: ℤ)
  O(y: ℤ) = y + 3
  m(z: ℤ) = O(z)
end
```

Allow this and $O(1).m(2) = 5$.

J.3.4 Function name shadowed by keyword parameters

$f(f\colon \mathbb{Z} = 0) = f + 1$
$g(g\colon \mathbb{Z} \to \mathbb{Z} = \texttt{fn}\ x \Rightarrow x) = g(2)$(* this is not a recursive function *)

Allowed because the names of keyword parameters, unlike the names of other parameters, are part of the "interface" of the functional.

J.3.5 Type names referred by imported entities in a component implementing the types

A fully qualified type, $\mathrm{APIName}.\mathit{typeName}$, "available" in a component, $\mathrm{ComponentName}$, that implements the API, $\mathrm{APIName}$, denotes the same type $\mathit{typeName}$, which should be defined in the component, $\mathrm{ComponentName}$. For example:

```
api A
  trait T
    m(): T
  end
end
api B
  import A.T
  f(): T
end
component C
  import B.f
  export A
  trait T
    m(): T
    private n(): ()
  end
  g() = f().n()
end
```

the method call $f().n()$ is valid.

However, a fully qualified type, $\mathrm{APIName}.typeName$, "occurring" in a component, $\mathrm{ComponentName}$, that implements the API, $\mathrm{APIName}$ is not allowed because $\mathrm{APIName}$ is not imported to the component.

J.3.6 Overloaded operators

- Are we allowed to say:

  ```
  trait Foo⟦opr OP⟧
    f(x: T⟦OP⟧, y: ℤ32) = ...
    f(x: T⟦⊗⟧, y: ℤ32) = ...
    opr OP(self, other: FOO⟦OP⟧) = ...
    opr ⊗(self, other: FOO⟦OP⟧) = ...
  end
  ```

  now that `OP` is forbidden to be $\otimes$, so maybe it's okay???

  If T is a trait, it is not allowed because $T⟦\texttt{OP}⟧$ may not exclude $T⟦\otimes⟧$.

  No. We don't want to allow this. The static checker does not take the implicit constraint from the body of the trait.

- Are we allowed to say:

  ```
  trait Foo⟦nat n⟧
    f(x: T⟦n + 1⟧, y: ℤ32) = ...
    f(x: T⟦0⟧, y: ℤ32) = ...
  end
  object T⟦nat n⟧(...) end
  ```

  now that $n + 1$ is necessarily nonzero, so maybe it's okay???

  Not allowed.

- Are we allowed to say:

  ```
  trait Foo⟦nat n⟧
    f(x: T⟦1⟧, y: ℤ32) = ...
    f(x: T⟦0⟧, y: ℤ32) = ...
  end
  object T⟦nat n⟧(...) end
  ```

  now that `nat` parameters are all instantiated, so maybe it's okay???

  Allowed.

- Are we even allowed to say this simpler case:

  ```
  f(x: ℤ32, y: ℤ32) = ...
  f(x: T⟦0⟧, y: ℤ32) = ...
  ```

  if our implementation strategy for `nat` parameters is "don't instantiate"?

  Allowed.

- Do we allow this:

```
trait F⟦nat n⟧ extends G⟦n⟧ end
trait G⟦nat n⟧ extends F⟦n + 1⟧ end
```

Not allowed by the type acyclicity restriction.

J.3.7 Implicitly Imported Types

When a component imports a functional f (either a top-level function or a functional method) by an import statement, the imported f may be overloaded with a functional f declared by the component. When a component imports a top-level declaration f from an API A, all the relevant types to type check the uses of f are implicitly imported from A. However, these implicitly imported types for type checking are not expressible by programmers; programmers must import the types by import statements to use them. For example, the two functional calls in the following component C are valid:

```
api A
  trait T
    m() : ()
  end
end

api B
  import A.{...}
  f(): T
  g(T): ()
end

component C
  import B.{f, g}
  export Executable
  run(args) = do f().m(); g(f()) end
end
```

because T is implicitly imported from B to type check the functional calls. However, the programmers cannot write T in C because T is not imported by an import statement.

J.3.8 Component and Value Namespace

```
component Surprise
import api Foo
export Executable

object Foo
  foo() : String = "foo"
  bar() : String = "foo"
  baz() : String = "foo"
end

run(args : String ...) = do
  Foo.foo()
  Foo.bar()
  Foo.baz()
end

end
```

Static error for the declaration of the object Foo in the above example.

J.3.9 Disjunctive Constraints

Here is a program that will only typecheck correctly if the fortress typechecker supports these disjunctions:

```
component NeedsDisjunction
  export Executable
  object Right end
  object Wrong end
  bar⟦T⟧(p1 : Plussable⟦T⟧, p2 : Plussable⟦T⟧) : Plussable⟦T⟧ = p1
  trait Plussable⟦T⟧
  end
  object Parent extends {Plussable⟦Right⟧, Plussable⟦Wrong⟧}
  end
  foo(p : Plussable⟦Wrong⟧) : () = ()
  run(args : String ...) : () = do
    a : Parent = Parent
    b : Parent = Parent
    c = bar(a, b)
    foo(c)
  end
end
```

When $bar()$ is called without explicit type parameters, we make its signature into:

$bar⟦\$i⟧(p_1 : \text{Plussable}⟦\$i⟧, p_2 : \text{Plussable}⟦\$i⟧) : \text{Plussable}⟦\$i⟧$.

$bar()$ then becomes a legal overloading if, "$\$i$ = Right OR $\$i$ = Wrong". Later, the call to $foo()$ forces the issue: $\$i$ must be "Right", but unless we kept both possibilities alive, our program might not typecheck correctly.

> REVISED BY THE 2026 REVIVAL SEE Section I.1.9
>
> Under instantiation exclusion the program's premise is not allowed: Parent may not extend both Plussable⟦Right⟧ and Plussable⟦Wrong⟧ (Section 6.3, Section 10.1), so the disjunction does not arise for this program.

J.3.10 Weirdness with Multiple Inheritance and Polymorphism

We noticed that you can get some fairly odd behavior if you allow inheriting from the same trait multiple times with different static arguments. For example in the following code there is no good way to decide what the value of $foo()$ will be.

```
component Bad
export Executable
trait Parent⟦T⟧
end
object goodChild⟦T⟧ extends Parent⟦T⟧
end
```

```
object badChild extends { Parent⟦ℤ32⟧, Parent⟦String⟧ }
end
spank⟦T⟧(x: Parent⟦T⟧): Parent⟦T⟧ =
  typecase a = "UnOh" of
    T ⇒ x
    else⇒ goodCHild⟦T⟧
  end
foo(): ℤ32 =
  typecase x = spank(badChild) of
    badChild ⇒ 1
    else⇒ 0
  end
run(args: String) : () = ignore(foo())
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.9
>
> This is the case instantiation exclusion answers: *badChild* may not extend both Parent⟦ℤ32⟧ and Parent⟦String⟧ (Section 6.3, Section 10.1), so the question of what *foo*() returns does not arise.

J.3.11 `where` Clauses on Methods

```
value trait LinearSequence⟦T extends Object, nat n⟧
    coercion ⟦nat b⟧(x : BinaryLinearSequence⟦b, n⟧) where {T extends BinaryWord⟦b⟧}
    opr [n: IndexInt] : T throws { IndexOutOfBounds }
    opr ⟦nat k⟧[r: StaticWidthRange⟦k⟧] : LinearSequence⟦T, k⟧
        throws { IndexOutOfBounds } where { k ≤ n }
    opr [n: IndexInt] := (v : T) : LinearSequence⟦T, n⟧ throws { IndexOutOfBounds }
    opr ⟦nat k⟧[r: StaticWidthRange⟦k⟧] := (v : LinearSequence⟦T, k⟧) : LinearSequence⟦T, n⟧
        throws { IndexOutOfBounds } where { k ≤ n }
    opr ||⟦nat m⟧(self, LinearSequence⟦T, m⟧) : LinearSequence⟦T, n + m⟧
end

value trait IntegerValue⟦bool negative, nat magnitude⟧
      extends { IntegerConstant, ExtendedIntegerValue⟦negative, magnitude, false⟧,
                RationalValue⟦negative, magnitude, 1⟧ }
    opr /(self, other: IntegerValue⟦neg, mag⟧ : RationalValue⟦negative ⩒ neg, magnitude, mag⟧ where { mag ≠ 0 }
    opr /(self, other: IntegerValue⟦neg, mag⟧ : ExtendedRationalValue⟦negative ⩒ neg, magnitude, mag⟧
    opr √(self) : RationalValueTimesSqrt⟦magnitude⟧ where { ¬negative }
    ...
end
value trait ExtendedIntegerValue⟦bool negative, nat magnitude, bool infinite⟧
      extends { ExtendedIntegerConstant,
                ExtendedRationalValue⟦negative,
                                      if ¬infinite then magnitude elif negative then −1 else 1 end,
                                      if infinite then 0 else 1 end⟧ }
    opr /(self, other: ExtendedIntegerValue⟦neg, mag, inf⟧) : RationalValue⟦...⟧ where {...}
    opr /(self, other: ExtendedIntegerValue⟦neg, mag, inf⟧) : ExtendedRationalValue⟦...⟧
    opr √(self) : RationalValueTimesSqrt⟦false, 1, 1, magnitude⟧ where { ¬negative ∧ ¬infinite }
    ...
end
```

J.3.12 Conditional Subtyping

```
trait RationalQuantity⟦unit U absorbs unit, bool ninf, bool lt, bool eq, bool gt, bool pinf, bool nan⟧
    extends { RationalQuantity⟦U, ninf′, lt′, eq′, gt′, pinf′, nan′⟧
              where { bool ninf′, bool lt′, bool eq′, bool gt′, bool pinf′, bool nan′,
                      ninf → ninf′, lt → lt′, eq → eq′, gt → gt′, pinf → pinf′, nan → nan′ },
              Field⟦RationalQuantity⟦U, ninf, lt, eq, gt, pinf, nan⟧,
                    RationalQuantity⟦U, ninf, lt, false, gt, pinf, nan⟧, +, −, ·, /, zero, one⟧
              where { lt ∧ eq ∧ gt ∧ ¬ninf ∧ ¬pinf ∧ ¬nan, U = dimensionless },
              CommutativeMonoid⟦RationalQuantity⟦U, ninf, lt, eq, gt, pinf, nan⟧, +, −, zero⟧
              TotalOrderOperators⟦RationalQuantity⟦U, ninf, lt, eq, gt, inf, nan⟧, <, ≤, ≥, >, CMP⟧
              where { ¬nan } }
    where { ninf ∨ lt ∨ eq ∨ gt ∨ pinf ∨ nan }
end
```

> REVISED BY THE 2026 REVIVAL SEE Section I.1.9
>
> Under instantiation exclusion this declaration is not allowed, for the reason the covariance example of Section 12.6 is not: its `extends` clause names RationalQuantity with `where`-clause variables as static arguments. The chapter that describes the same design is kept as a superseded design (Section I.1.14).

J.3.13 No Wrapped Fields Whose Types Are Naked Type Variables

We don't want to allow wrapped fields to have types that are naked type variables because then we cannot tell what methods an object has, and therefore, what names may be shadowed within the object definition:

```
object F⟦T⟧(x : ℤ, wrapped y : T)
  f() = object extends Object
          getX() = x
        end
end

trait T1(∗ has getter for x ∗)
  x : ℤ = 2
end
```

With conditional subtyping (or conditional methods), we may also not be able to tell whether a trait has a particular method, but we can at least bound the names of the methods it may have. Still, we should watch out for this issue in the formulation of conditional subtyping.

J.3.14 Tricky Example for Explaining Shadowing

```
trait S
  f() = 1(∗ Declaration 1 ∗)
  g() = 5
end

trait T
  f() = 2(∗ Declaration 2 ∗)
  m() = 6
  n(): S
end

object O extends S
```

```
  h() : T = object extends T
                        m() = f()
                        n() = object extends S
                                        g() = f()
                              end
          end
end
```

The reach of declaration 1 includes the entire declaration of O, but it is shadowed by declaration 2, within the object expression that extends T, which in turn is shadowed by declaration 1 in the object expression that extends S.

J.3.15 A Pattern for Shadowing

```
var count: ℤ = 0
getGlobalCount() = count
setGlobalCount(i : ℤ) = count := i
object LocalCounter(var count: ℤ)
   localIncrement() = count += 1
   getter totalCount = count + getGlobalCount()
   atomic transfer() = setGlobalCount(totalCount)
end
```

Note the need for *getGlobalCount* and *setGlobalCount* just to get access to the top-level variable *count* within the body of LocalCounter (in which the top-level *count* variable declaration is shadowed). Presumably the exported API would not include them.

I think we were toying aroud with a direct means to acess a shadowed name using a new reserved word "`outer`" (or Guy proposed "periscope"). I don't like this, but it would eliminate the need for the pattern.

J.3.16 Implicitly Parameterized Object Expressions

What are the values of the last two expressions?

```
value trait A
   x : ℤ
end
f⟦T⟧(y : ℤ) = object extends A x = y end
f⟦ℝ⟧(1) = f⟦ℤ⟧(1)
f⟦ℝ⟧(1) = f⟦ℝ⟧(1)
```

J.4 Things to Check for Consistency

- For each section in the Expressions chapter, describe the following:
  - syntax
  - how it is reduced / how we evaluate it
  - value
  - type

- Avoid the following words: must ¿ should ¿ may / might avoid ”may not” use ”might not” or ”need not”
- AVOID global, top-level (except top-level within components/APIs)
- AVOID statement (except import/export statements)
- AVOID compliant compilers (We may have a meta circular API for compliant compilers.)
- Three levels of Fortress libraries
  - the Fortress core language
  - the Fortress core APIs (implicitly imported libraries)
  - the Fortress standard libraries
- execute: programs, components, Fortress applications, functions, threads, tests
- evaluate: expressions,
- should ⇒ shall, must
- may not ⇒ might not, need not
- . ⇒ ‘.’
- “...,” ⇒ “...”,
- api ⇒ API
- iff ⇒ if and only if
- reduction ⇒ evaluation
- compile-time ⇒ static
- object type, object trait ⇒ object trait type
- “scope over” ⇒ “in scope”
- “a static error is signaled” ⇒ “it is a static error”
- “a runtime error is signaled” ⇒ “an exception is thrown”
- Boolean (as an adjective) ⇒ boolean
- greatest lower bound ⇒ intersection types
- least upper bound ⇒ union types
- built-in types ⇒ simple standard types
- topmost, uppermost ⇒ leftmost (define leftmost early on)
- atomic block ⇒ atomic expression
- function and/or method ⇒ functional
- visible ⇒ in scope (cf. scope defined in the Names chapter)
- `in` ⇒ `in`
- $indices()$ ⇒ $indices$
- `self` vs $self$

- What errors do we want to catch?
- What codes would be rejected that we'd like to reject?

Bibliography

[1] Ole Agesen, Lars Bak, Craig Chambers, Bay-Wei Chang, Urs Hölzle, John Maloney, Randall B. Smith, David Ungar, and Mario Wolczko. *The Self Programmer's Reference Manual*. `http://research.sun.com/self/release_4.0/Self-4.0/manuals/Self-4.1-Pgmers-Ref.pdf`, 2000.

[2] Eric Allen, David Chase, Joe Hallett, Victor Luchangco, Jan-Willem Maessen, Sukyoung Ryu, Guy L. Steele Jr., and Sam Tobin-Hochstadt. The Fortress Language Specification Version 1.0. `http://research.sun.com/projects/plrg/Publications/fortress.1.0.pdf`, March 2008.

[3] Eric Allen, Ryan Culpepper, Janus Dam Nielsen, Jon Rafkind, and Sukyoung Ryu. Growing a syntax. In *Foundations of Object-Oriented Languages*, 2009.

[4] Eric Allen, Victor Luchangco, and Sam Tobin-Hochstadt. Encapsulated Upgradable Components, March 2005.

[5] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. Cilk: An efficient multithreaded run-time system. *Journal of Parallel and Distributed Computing*, 37(1):55–69, August 1996.

[6] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, September 1999.

[7] Gilad Bracha, Guy Steele, Bill Joy, and James Gosling. *Java™ Language Specification, The (3rd Edition) (Java Series)*. Addison-Wesley Professional, July 2005.

[8] R. Cartwright and G. Steele. Compatible genericity with run-time types for the Java Programming Language. In *OOPSLA*, 1998.

[9] Stéphane Ducasse, Oscar Nierstrasz, Nathanael Schärli, Roel Wuyts, and Andrew P. Black. Traits: A mechanism for fine-grained reuse. *ACM Trans. Program. Lang. Syst.*, 28(2):331–388, 2006.

[10] Robert B. Findler, Mario Latendresse, and Matthias Felleisen. Behavioral contracts and behavioral subtyping. In *ESEC/FSE-9: Proceedings of the 8th European software engineering conference held jointly with 9th ACM SIGSOFT international symposium on Foundations of software engineering*, pages 229–236. ACM Press, September 2001.

[11] Seth C. Goldstein, Klaus E. Schauser, and Dave E. Culler. Lazy Threads: Implementing a Fast Parallel Call. *Journal of Parallel and Distributed Computing*, 37(1), August 1996.

[12] Robert Grimm. Rats! – an easily extensible parser generator. `http://cs.nyu.edu/~rgrimm/xtc/rats.html`.

[13] Sun's Programming Language Research Group. Project fortress. `http://projectfortress.sun.com`.

[14] Ralf Hinze and Ross Paterson. Finger trees: a simple general-purpose data structure. *Journal of Functional Programming*, 16(2):197–217, March 2006.

[15] Atshushi Igarashi, Benjamin Pierce, and Philip Wadler. Featherweight Java: A minimal core calculus for Java and GJ. In Loren Meissner, editor, *Proceedings of the 1999 ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages & Applications (OOPSLA'99)*, volume 34(10), pages 132–146, N. Y., 1999.

[16] Richard Kelsey, William Clinger, and Jonathan Rees. Revised^5 report on the algorithmic language Scheme. *ACM SIGPLAN Notices*, 33(9):26–76, 1998.

[17] Xavier Leroy, Damien Doligez, Jacques Garrigue, Didier Rémy, and Jérôme Vouillon. *The Objective Caml System, release 3.08*. `http://caml.inria.fr/distrib/ocaml-3.08/ocaml-3.08-refman.pdf`, 2004.

[18] J. Matthews, R. B. Findler, M. Flatt, and M. Felleisen. A Visual Environment for Developing Context-Sensitive Term Rewriting Systems (system description). In V. van Oostrom, editor, *Rewriting Techniques and Applications, 15th International Conference, RTA-04*, LNCS 3091, pages 301–311, Valencia, Spain, June 3-5, 2004. Springer.

[19] B. Meyer. *Object-oriented Software Construction*. Prentice Hall, 1988.

[20] Todd Millstein and Craig Chambers. Modular statically typed multimethods. *Information and Computation*, 175(1):76–118, May 2002.

[21] Robin Milner, Mads Tofte, Robert Harper, and David MacQueen. *The Definition of Standard ML (Revised)*. The MIT Press, 1997.

[22] Eric Mohr, David A. Kranz, and Robert H. Halstead, Jr. Lazy task creation: A technique for increasing the granularity of parallel programs. Technical Report TM-449, MIT/LCS, 1991.

[23] G. M. Morton. A computer oriented geodetic data base and a new technique in file sequencing. Technical report, IBM Ltd., March 1966.

[24] Martin Odersky, Philippe Altherr, Vincent Cremet, Burak Emir, Stéphane Micheloud, Nikolay Mihaylov, Michel Schinz, Erik Stenman, and Matthias Zenger. *The Scala Language Specification*. `http://scala.epfl.ch/docu/files/ScalaReference.pdf`, 2004.

[25] Chris Okasaki. *Purely Functional Data Structures*. Cambridge University Press, Cambridge, UK, 1998.

[26] OpenMP Architecture Review Board. *OpenMP Fortran Application Program Interface Version 2.0*. `http://www.openmp.org/specs/mp-documents/fspec20_bars.pdf`, November 2000.

[27] Simon Peyton-Jones. *Haskell 98 Language and Libraries*. Cambridge University Press, 2003.

[28] Barry N. Taylor. Guide for the use of the international system of units (si). Technical report, United States Department of Commerce, National Institute of Standards and Technology, April 1995.

[29] The Unicode Consortium. *The Unicode Standard, Version 5.0*. Addison-Wesley, 2006.